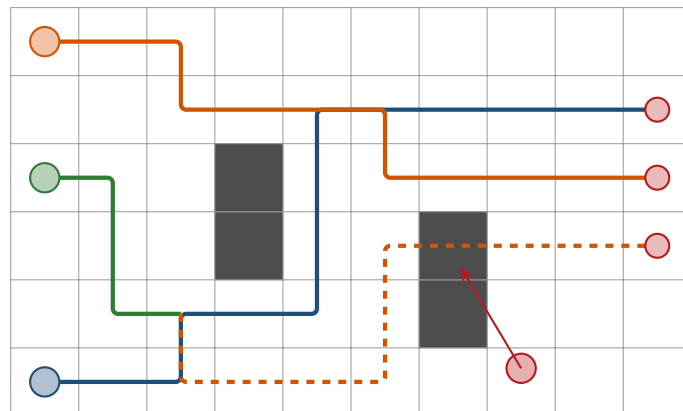


Multi-Agent Path Planning and Drone Collision Avoidance

A Textbook of Algorithms:
From Graph Search to Hybrid Swarm Systems



The Searchbook Project

September 10, 2026

Multi-Agent Path Planning and Drone Collision Avoidance

A Textbook of Algorithms: From Graph Search to Hybrid Swarm Systems

Typeset with L^AT_EX (pdfT_EX), TikZ and pgfplots. Figures were drawn in TikZ or generated from the Python reference implementations that accompany the book.

This book was written to accompany a twelve-week research training plan on multi-agent path planning and collision avoidance for drone swarms. It is intended for self-study. All algorithms described here are established published methods; the original sources are cited in each chapter and collected in the bibliography.

Source code, figures and L^AT_EX sources are distributed together with this book so that every example can be reproduced and every figure regenerated.

Compiled on September 10, 2026.

Contents

Preface	xiii
Notation	xvii
I Foundations	1
1 Planning for Drone Swarms	2
1.1 The problem: many drones, one airspace, and a stranger	2
1.2 Vocabulary: the words you need before anything else	6
1.3 A preview of the hybrid architecture	9
1.4 A map of the book	12
1.5 What is established and where research remains	15
1.6 How to read this book	16
1.7 Summary	19
1.8 Exercises	19
2 The Toolbox: Graphs, Grids, Time, Motion and Uncertainty	22
2.1 Why this matters for a drone swarm	22
2.2 Graphs, grids and lattices	23
2.3 Workspace and configuration space	25
2.4 Time: space-time states, paths, plans and trajectories	27
2.5 Cost measures	29
2.6 Motion models for a drone	31
2.7 A geometry toolkit	33
2.8 Uncertainty: random vectors, Gaussians and Bayes' rule	36
2.9 Computation: complexity, priority queues and hashing	38
2.10 The Python stack and the code of the book	39
2.11 Where this fits in the drone system	40
2.12 Summary	40
2.13 Exercises	41
II Single-Agent Graph Search	43
3 Dijkstra's Algorithm	44
3.1 Why this matters for a drone swarm	44
3.2 The idea in plain words	45
3.3 Problem statement and notation	46
3.4 The algorithm	47
3.5 A worked example	48
3.6 Properties: correctness, optimality, complexity	50
3.7 Variants and extensions	54
3.8 Implementation notes	58

3.9	Where this fits in the drone system	61
3.10	Summary	61
3.11	Exercises	62
4	A* Search	64
4.1	Why this matters for a drone swarm	64
4.2	The idea in plain words	65
4.3	Problem statement and notation	66
4.4	The algorithm	68
4.5	A worked example	69
4.6	Properties: correctness, optimality, complexity	70
4.7	Variants and extensions	74
4.8	Space-time A*	76
4.9	Implementation notes	82
4.10	Where this fits in the drone system	85
4.11	Summary	85
4.12	Exercises	86
5	Incremental Search: LPA* and D* Lite	88
5.1	Why this matters for a drone swarm	88
5.2	The idea in plain words	89
5.3	Problem statement and notation	90
5.4	The algorithm: LPA*	92
5.5	A worked example: LPA* repairs a path	94
5.6	The algorithm: D* Lite	96
5.7	A worked example: D* Lite repairs a plan	100
5.8	Properties: correctness, optimality, complexity	101
5.9	Variants and extensions	104
5.10	Implementation notes	105
5.11	Where this fits in the drone system	108
5.12	Summary	109
5.13	Exercises	110
6	Anytime Search: ARA*	112
6.1	Why this matters for a drone swarm	112
6.2	The idea in plain words	113
6.3	Problem statement and notation	114
6.4	The algorithm	115
6.5	A worked example	117
6.6	Properties: bounds, completeness, complexity	121
6.7	The ε schedule	123
6.8	Variants and extensions	125
6.9	Implementation notes	126
6.10	Where this fits in the drone system	128
6.11	Summary	129
6.12	Exercises	130

III	Multi-Agent Path Finding	133
7	The Multi-Agent Path Finding Problem	134
7.1	Why this matters for a drone swarm	134
7.2	The idea in plain words	135
7.3	Problem statement and notation	136
7.4	The algorithm: conflict detection	139
7.5	A worked example	141
7.6	Properties: feasibility, optimality and complexity	143
7.7	Solver families	144
7.8	Benchmarks and instance files	146
7.9	Implementation notes	147
7.10	MAPF for drones and its place in the system	149
7.11	Summary	150
7.12	Exercises	151
8	Prioritized Planning and Space-Time A*	153
8.1	Why this matters for a drone swarm	153
8.2	The idea in plain words	154
8.3	Problem statement and notation	155
8.4	The algorithm	156
8.5	A worked example	157
8.6	Properties	160
8.7	Variants and extensions	163
8.8	Implementation notes	167
8.9	Where this fits in the drone system	170
8.10	Summary	171
8.11	Exercises	171
9	Conflict-Based Search	173
9.1	Why this matters for a drone swarm	173
9.2	The idea in plain words	174
9.3	Problem statement and notation	175
9.4	The algorithm	176
9.5	A worked example	178
9.6	Properties: correctness, optimality, complexity	180
9.7	Variants and extensions	182
9.8	Implementation notes	187
9.9	Where this fits in the drone system	190
9.10	Summary	191
9.11	Exercises	192
10	Bounded-Suboptimal Search: ECBS	194
10.1	Why this matters for a drone swarm	194
10.2	The idea in plain words	195
10.3	Problem statement and notation	196
10.4	The algorithm	197
10.5	A worked example	201
10.6	Properties: bound, completeness, complexity	204
10.7	Variants and extensions	207
10.8	Implementation notes	210

10.9	Where this fits in the drone system	214
10.10	Summary	214
10.11	Exercises	215
11	M*, Push-and-Swap, Push-and-Rotate	217
11.1	Why this matters for a drone swarm	217
11.2	The idea in plain words	218
11.3	Problem statement and notation	219
11.4	The algorithms	222
11.5	A worked example	223
11.6	Properties: termination, completeness, optimality	227
11.7	Constructive methods: Push-and-Swap and Push-and-Rotate	229
11.8	Variants and extensions	232
11.9	Implementation notes	233
11.10	Where this fits in the drone system	235
11.11	Summary	236
11.12	Exercises	236
IV	Local and Reactive Collision Avoidance	239
12	Velocity Obstacles	240
12.1	Why this matters for a drone swarm	240
12.2	The idea in plain words	241
12.3	Problem statement and notation	242
12.4	The algorithm	247
12.5	A worked example	249
12.6	Properties: safety, feasibility, complexity	251
12.7	The oscillation problem	253
12.8	Variants and extensions	255
12.9	The cone in three dimensions	256
12.10	Implementation notes	256
12.11	Where this fits in the drone system	259
12.12	Summary	260
12.13	Exercises	261
13	Reciprocal Avoidance: RVO and ORCA	263
13.1	Why this matters for a drone swarm	263
13.2	The idea in plain words	264
13.3	Problem statement and notation	265
13.4	The ORCA construction	269
13.5	The per-agent program	273
13.6	A worked example	276
13.7	Eight agents on a circle	277
13.8	Properties: what ORCA guarantees and what it does not	278
13.9	Variants and extensions	280
13.10	Implementation notes	281
13.11	Where this fits in the drone system	284
13.12	Summary	284
13.13	Exercises	285

14 The Dynamic Window Approach	288
14.1 Why this matters for a drone swarm	288
14.2 The idea in plain words	289
14.3 Problem statement and notation	290
14.4 The algorithm	294
14.5 A worked example	295
14.6 Properties: safety, feasibility, locality	297
14.7 Tuning the weights	300
14.8 Variants and extensions	302
14.9 Implementation notes	303
14.10 Where this fits in the drone system	306
14.11 Summary	306
14.12 Exercises	307
15 Artificial Potential Fields	309
15.1 Why this matters for a drone swarm	309
15.2 The idea in plain words	310
15.3 Problem statement and notation	311
15.4 The algorithm	312
15.5 A worked example	314
15.6 Properties: continuity, descent, safety, and where the minimum is	315
15.7 The failure modes	317
15.8 Remedies	322
15.9 Swarms: inter-agent repulsion, flocking and consensus	324
15.10 Implementation notes	326
15.11 Where this fits in the drone system	327
15.12 Summary	328
15.13 Exercises	328
V Sampling-Based Motion Planning	331
16 Rapidly-Exploring Random Trees	332
16.1 Why this matters for a drone swarm	332
16.2 The idea in plain words	333
16.3 Problem statement and notation	334
16.4 The algorithm	336
16.5 A worked example	337
16.6 Properties: completeness, optimality, complexity	338
16.7 Variants and extensions	340
16.8 Implementation notes	345
16.9 Grid search or sampling?	349
16.10 Where this fits in the drone system	350
16.11 Summary	350
16.12 Exercises	351
17 RRT* and Informed RRT*	353
17.1 Why this matters for a drone swarm	353
17.2 The idea in plain words	354
17.3 Problem statement and notation	355
17.4 The algorithm	356

17.5 A worked example	360
17.6 Properties: correctness, optimality, complexity	361
17.7 Informed RRT*: sampling where a better path can pass	364
17.8 Variants and extensions	366
17.9 Implementation notes	368
17.10 Where this fits in the drone system	370
17.11 Summary	371
17.12 Exercises	372
VI Tracking and Prediction	374
18 The Kalman Filter	375
18.1 Why this matters for a drone swarm	375
18.2 The idea in plain words	376
18.3 Problem statement and notation	377
18.4 The algorithm	380
18.5 A worked example	383
18.6 Properties: optimality, consistency, observability	385
18.7 Tuning, diagnostics, gating, and imperfect sensors	387
18.8 Predicting ahead: uncertainty propagation	391
18.9 An experiment on simulated data	392
18.10 Variants and extensions	393
18.11 Implementation notes	394
18.12 Where this fits in the drone system	396
18.13 Summary	396
18.14 Exercises	397
19 Nonlinear Filtering	399
19.1 Why this matters	399
19.2 The idea in plain words	400
19.3 Problem and notation	401
19.4 The extended Kalman filter	403
19.5 The unscented Kalman filter	406
19.6 The particle filter	410
19.7 Properties and cost	413
19.8 Variants and extensions	415
19.9 Implementation notes	416
19.10 Where this fits	417
19.11 Summary	418
19.12 Exercises	418
20 Trajectory Prediction: From Constant Velocity to LSTMs and Transformers	420
20.1 Why this matters for a drone swarm	420
20.2 The idea in plain words	421
20.3 Problem statement, metrics and evaluation protocol	422
20.4 The physics baselines	423
20.5 LSTM and sequence-to-sequence prediction	424
20.6 Transformers and attention over agents	429
20.7 Multimodality and uncertainty	433
20.8 A worked example	434

20.9 Properties	437
20.10 Variants and extensions	440
20.11 Implementation notes	440
20.12 Where this fits in the drone system	442
20.13 Summary	443
20.14 Exercises	444
VII Optimization and Control	446
21 Model Predictive Control	447
21.1 Why this matters for a drone swarm	447
21.2 The idea in plain words	448
21.3 Problem statement and notation	449
21.4 The algorithm	453
21.5 A worked example	456
21.6 Properties: stability, feasibility and cost	458
21.7 Variants and extensions	461
21.8 Implementation notes	464
21.9 Where this fits in the drone system	467
21.10 Summary	467
21.11 Exercises	468
22 Mixed-Integer Linear Programming for Planning	470
22.1 Why this matters for a drone swarm	470
22.2 The idea in plain words	471
22.3 Problem statement and notation	472
22.4 Modeling obstacles and separation	473
22.5 The algorithm: branch-and-bound	476
22.6 A worked example	479
22.7 Properties: correctness, optimality, complexity	480
22.8 Variants and extensions	482
22.9 The cost of exactness	484
22.10 Implementation notes	485
22.11 Where this fits in the drone system	488
22.12 Summary	488
22.13 Exercises	489
23 Consensus and Formation Control	491
23.1 Why this matters for a drone swarm	491
23.2 The idea in plain words	492
23.3 Problem statement and notation	493
23.4 The algorithm: consensus and its extensions	495
23.5 A worked example	498
23.6 Properties: convergence, rate and step size	499
23.7 Variants and extensions	504
23.8 Implementation notes	508
23.9 Where this fits in the drone system	510
23.10 Summary	511
23.11 Exercises	512

VIII	Putting It Together	514
24	The Hybrid Architecture	515
24.1	Why this matters for a drone swarm	515
24.2	The idea in plain words	516
24.3	The world model shared by the layers	517
24.4	The decision logic	519
24.5	The safety horizon	520
24.6	Reconnecting to the nominal path	523
24.7	Replanning and the conflict re-check	525
24.8	Formation and communication constraints	526
24.9	Prediction-aware constraints	527
24.10A	worked scenario	529
24.11	What guarantees survive	530
24.12	Implementation architecture	533
24.13	Research directions	535
24.14	Where this fits in the drone system	537
24.15	Summary	537
24.16	Exercises	538
25	Evaluating a Hybrid Planner	540
25.1	Why this matters for a drone swarm	540
25.2	The idea in plain words	541
25.3	Metrics: what to measure and how	541
25.4	The experiment matrix and the baselines	544
25.5	Scenario generation	546
25.6	Statistics: from runs to claims	549
25.7	A worked mini-study	552
25.8	Reproducibility	555
25.9	Reporting the results	556
25.10	The benchmark landscape and physics simulators	557
25.11	Implementation notes	558
25.12	Where this fits in the drone system	562
25.13	Summary	563
25.14	Exercises	564
IX	Appendices	567
A	The Twelve-Week Study Plan	568
A.1	How to use this plan	568
A.2	The schedule at a glance	570
A.3	The twelve weeks on one timeline	571
A.4	Study notes, week by week	571
A.5	The capstone: a hybrid collision-avoidance prototype	578
A.6	Completion checklist	581
A.7	Traceability: from the plan to the chapters	582
A.8	Reading order and primary sources	583

B Mathematical Refresher	586
B.1 Vectors and matrices	586
B.2 Calculus for motion	590
B.3 Probability and Gaussians	592
B.4 Optimization	596
B.5 Graphs and the Laplacian	602
B.6 Complexity and data structures	603
B.7 Summary	605
C Hints and Solutions to Selected Exercises	607
Glossary	667
Bibliography	693
Index	704

List of Algorithms

2.1	Priority queue with lazy deletion	38
3.1	Dijkstra's algorithm with a binary heap and lazy deletion. Without a target it computes $\text{dist}(s, v)$ for every vertex; with a target γ it stops as soon as γ is settled (uniform-cost search).	47
3.2	Backward Dijkstra: the true-distance heuristic table for a goal γ	55
4.1	A^* graph search with a closed set and a lazy priority queue	69
4.2	Space-time A^* for one agent under vertex and edge constraints	80
5.1	Lifelong Planning A^* (LPA *) [90]	93
5.2	D * Lite, final version [88]	99
6.1	ARA * , inner loop: weighted A^* without re-expansions	116
6.2	ARA * , outer loop: the schedule, the bookkeeping and the certificates	117
7.1	First conflict of a plan by pairwise comparison	139
7.2	First conflict of a plan with hash maps	140
8.1	Prioritized planning	157
8.2	Cooperative A^* as a delta on space-time A^* : the six lines by which Algorithm 4.2 becomes a search against a reservation table	158
8.3	Windowed Cooperative A^* (WHCA *)	164
9.1	Conflict-based search: the high level	177
9.2	The low level as a delta on space-time A^* : the four lines by which Algorithm 4.2 becomes a search for one agent under its constraints	178
10.1	The ECBS low level as a delta on space-time A^* : the ten lines by which Algorithm 4.2 becomes a focal search that also returns a lower bound	199
10.2	ECBS: the high level	201
11.1	Simple independence detection	222
11.2	M * with collision sets and backpropagation	224
11.3	Push-and-Swap (conceptual)	230
11.4	Push-and-Rotate (conceptual)	231
12.1	Time to collision and membership in the truncated velocity obstacle	248
12.2	Sampled velocity selection for one agent	248
12.3	Synchronous velocity-obstacle simulation	249
13.1	The ORCA half-plane of agent A induced by neighbor B	271
13.2	Incremental two-dimensional linear program over half-planes and the speed disk	274
13.3	Dense fallback: minimize the largest violation when the half-planes do not intersect	275
14.1	One step of the dynamic window approach (drone form)	295

14.2 The DWA control loop with a global path and hysteresis	295
15.1 The potential-field force at a point: hybrid attraction, Khatib repulsion, optional GNRON factor	313
15.2 Gradient-descent controller with a velocity limit and local-minimum detection .	314
16.1 RRT with goal bias	336
16.2 RRT-Connect [94]	341
16.3 Kinodynamic RRT for the double integrator [100]	343
16.4 Path post-processing: random shortcutting and greedy pruning	344
17.1 RRT* (Karaman and Frazzoli)	357
17.2 The helpers of RRT*	358
17.3 Informed RRT* (Gammell, Srinivasa and Barfoot)	367
18.1 The Kalman filter: one predict step and one update step	383
18.2 Tracking an intruder from a stream of position measurements	384
19.1 One step of the extended Kalman filter	403
19.2 One step of the unscented Kalman filter (additive noise)	408
19.3 Systematic resampling	411
19.4 One step of the bootstrap particle filter with systematic resampling	412
20.1 Physics baselines for a horizon of H steps	423
20.2 Sequence-to-sequence prediction with an encoder and a decoder LSTM	428
21.1 ADMM for the quadratic program Equation (21.7) (the OSQP iteration)	455
21.2 Model predictive control loop with linearized avoidance and keep-in constraints .	456
22.1 Branch-and-bound for a MILP with binary variables	477
23.1 One control period of the displacement-based formation controller on drone i . .	497
23.2 One control period of the second-order formation controller on drone i	498
24.1 The per-drone control cycle of the executive	521
24.2 The safety-horizon trigger	523
24.3 Reconnection search along the nominal path	524
24.4 Replanning from the current state and the conflict re-check	526
25.1 Paired evaluation of a set of strategies on one cell	548

Preface

Why this book exists

Imagine a small fleet of drones that must fly through the same airspace at the same time. Each drone has a job to do and a route to fly. Before take-off, a planner must make sure that no two routes collide. During the flight, something unplanned happens: another drone, not part of the fleet and not talking to it, crosses the formation. The fleet must notice it, guess where it is going, get out of its way, and then return to the plan—or, if that is impossible, make a new plan and check it against the rest of the fleet.

Every part of that story is a well-studied algorithmic problem. Planning a route is *graph search*. Planning routes for many drones at once without collisions is *multi-agent path finding*. Reacting to a surprise in a fraction of a second is *local collision avoidance*. Guessing where the intruder is going is *tracking and trajectory prediction*. Making a new plan without starting from scratch is *incremental replanning*. Keeping the fleet in formation and within radio range while doing all of this is *control*. The individual algorithms are established and published. What is not settled is how to combine them into one system that keeps the guarantees of the global plan, reacts fast enough to be safe, and can be evaluated honestly.

This book teaches all of those algorithms, one at a time, from first principles, and then shows how they fit together into a hybrid swarm planner. It was written to accompany a twelve-week research training plan; the plan is reproduced, with week-by-week reading and coding assignments, in [Appendix A](#).

Who should read it

The book is for a graduate student or engineer who is starting research on drone swarms, or on multi-robot motion planning more generally, and who wants to *implement* the algorithms rather than merely recognize their names. You should be comfortable with Python and with the mathematics of a first course in linear algebra, calculus and probability. Everything else is introduced when it is needed, and [Appendix B](#) collects the mathematical facts that the book relies on. No prior knowledge of robotics or planning is assumed.

What you will be able to do

When you have worked through the book and its exercises you will be able to

- implement and explain A^* and space-time A^* , and say exactly why they return optimal paths under the required conditions;
- repair a path with D^* Lite instead of replanning from scratch, and measure when that is worth it;
- represent and detect the conflicts of a multi-agent plan, and solve the problem optimally with conflict-based search or approximately, with a guaranteed bound, with ECBS;
- explain velocity obstacles, reciprocal velocity obstacles and ORCA using relative position and velocity, and use them to keep drones apart;

- track an unknown drone from noisy observations with a Kalman filter, and compare a constant-velocity predictor with a learned one;
- trigger local avoidance from a time-to-collision or safety-horizon rule, return to the global route, or replan;
- enforce formation and communication-range constraints, and
- evaluate a hybrid planner with reproducible metrics and controlled scenarios.

How the book is organized

Part I, Foundations.

[Chapter 1](#) states the problem and previews the hybrid architecture that the whole book builds toward. [Chapter 2](#) is a toolbox: graphs, grids, configuration space, time-indexed paths, simple motion models, the geometry of moving disks, Gaussians and priority queues.

Part II, Single-Agent Graph Search.

Dijkstra’s algorithm ([Chapter 3](#)), A^* with admissible and consistent heuristics and its space-time version ([Chapter 4](#)), incremental replanning with LPA^* and D^* Lite ([Chapter 5](#)) and anytime search with ARA^* ([Chapter 6](#)).

Part III, Multi-Agent Path Finding.

The problem and its conflicts ([Chapter 7](#)), prioritized planning ([Chapter 8](#)), conflict-based search ([Chapter 9](#)), bounded-suboptimal search with ECBS ([Chapter 10](#)), and the coupled and constructive alternatives M^* , Push-and-Swap and Push-and-Rotate ([Chapter 11](#)).

Part IV, Local and Reactive Collision Avoidance.

Velocity obstacles ([Chapter 12](#)), reciprocal avoidance with RVO and ORCA ([Chapter 13](#)), the dynamic window approach ([Chapter 14](#)) and artificial potential fields ([Chapter 15](#)).

Part V, Sampling-Based Motion Planning.

RRT ([Chapter 16](#)) and the asymptotically optimal RRT^* with informed sampling ([Chapter 17](#)).

Part VI, Tracking and Prediction.

The Kalman filter ([Chapter 18](#)), its nonlinear relatives and the particle filter ([Chapter 19](#)), and trajectory prediction from constant-velocity baselines to LSTMs and Transformers ([Chapter 20](#)).

Part VII, Optimization and Control.

Model predictive control ([Chapter 21](#)), mixed-integer linear programming ([Chapter 22](#)) and consensus and formation control ([Chapter 23](#)).

Part VIII, Putting It Together.

The hybrid collision-avoidance architecture ([Chapter 24](#)) and how to evaluate it ([Chapter 25](#)).

How to read it

Read the chapters in order if you can: each part builds on the previous ones, and the notation is introduced once. If you are short of time, the training plan’s reading order is a good fast path; [Section 1.6](#) draws it and this is its list: [Chapter 4](#) (A^*), [Chapter 5](#) (D^* Lite), [Chapter 9](#) (CBS), [Chapter 7](#) (the MAPF problem), [Chapters 12 and 13](#) (velocity obstacles and ORCA), [Chapter 14](#) (DWA), [Chapters 16 and 17](#) (RRT and RRT^*), [Chapter 2](#) (the toolbox, which stands in for the plan’s “planning reference”), [Chapter 21](#) (MPC) and [Chapter 20](#) (trajectory prediction), then [Chapter 24](#).

Three remarks come with that order. First, the plan lists CBS before the MAPF overview; if you have never seen a multi-agent path finding instance, read [Chapter 7](#) first—it is short,

and [Chapter 9](#) assumes it. Second, the order reaches trajectory prediction without the Kalman filter, although [Chapter 20](#) extrapolates with one and takes its input from the tracker of [Chapter 18](#); read [Chapter 18](#) first, or at least its section on predicting ahead. Third, the toolbox of [Chapter 2](#) and the mathematical refresher of [Appendix B](#) are reference material, to be opened whenever a chapter assumes something you do not have, rather than a stop to be postponed until the tenth item of the list.

Every chapter ends with a box that says where its algorithm sits in the hybrid system, so you can also read by *layer*: the global planner (Parts II and III), the local safety layer (Part IV), the prediction layer (Part VI) and the constraints (Part VII).

Conventions

Every chapter opens with a box of *learning objectives* and closes with a *summary* box and exercises. Three kinds of colored box appear inside every chapter: *key idea* (green) states the one thing to remember; *in the drone system* (purple) says how the algorithm is used in the hybrid architecture; and *pitfall* (orange) warns about a mistake that is easy to make. Two more appear occasionally: a *historical note* (gray) gives context, and a plain note carries its own title. Definitions, examples, theorems and remarks share one counter per chapter, so “Example 4.3” comes after “Definition 4.2”. Exercises are marked with one to three stars (★ routine, ★★ needs thought, ★★★ a small project), and hints or solutions for a selection of them are in [Appendix C](#). The priority the training plan assigns to an algorithm is shown as a tag such as [Essential](#); the tags are collected in [Table 1.4](#) and [Table A.5](#), and appear in running text only in [Chapter 1](#) and [Appendix A](#), where the plan itself is being reproduced. Pseudocode is written in a uniform style with numbered lines; the text refers to line numbers. Vectors are bold ($\mathbf{p}$, $\mathbf{v}$), matrices are bold capitals ($\mathbf{A}$), and the full notation is listed in the table that follows this preface. The book uses American spelling throughout (*optimize*, *neighbor*, *center*); titles in the bibliography keep the spelling of the original publication.

The code

Every algorithm in the book comes with a Python implementation in the `code/` folder of the L^AT_EX project. The implementations are deliberately simple—plain Python and NumPy, a few hundred lines each, no framework—because the goal is that you can read them in one sitting and then write your own. Each file runs a self-test when executed (`python3 code/ch04_astar.py`), and the numbers quoted in the worked examples were produced by that code. The figures that show algorithm output were generated by the scripts in `code/figures/` from fixed random seeds, so you can regenerate and modify them. The listings in the text are excerpts; the files contain the complete programs.

Sources

None of the algorithms in this book is new. Each chapter cites the paper in which its algorithm was introduced and a textbook that treats it at greater length, and the *further reading* paragraph at the end of every chapter points to the developments that the chapter leaves out. The bibliography lists all sources. Where the book simplifies an algorithm for teaching, it says so and names the full version.

Errata

The manuscript was reviewed in several rounds for technical accuracy, completeness against the training plan, and clarity. Mistakes will nevertheless remain; if you find one, the L^AT_EX sources and code are distributed with the book precisely so that it can be corrected.

Notation

This book draws on eight fields that grew up apart, and their alphabets collide. The tables below list every symbol used in more than one place, say what it means, and name the chapter that introduces it. Where a letter really does carry two meanings—**Q** and **R** are the clearest case—the table says so rather than pretend otherwise.

Conventions

Fonts. Scalars are italic (v, t, c, λ), vectors are bold lower case (**p**, **v**, **x**, **u**, **z**) and matrices are bold upper case (**A**, **P**, **Σ**). The zero vector is **0**, the identity matrix **I** and the vector of ones **1**; their sizes follow from the context. Sets and spaces are calligraphic ($\mathcal{C}$, $\mathcal{C}_{\text{free}}$, $\mathcal{C}_i$, $\mathcal{W}$, $\mathcal{N}$), with the one classical exception that the vertex and edge sets of a graph keep the plain letters V and E ; data structures are small capitals (OPEN, CLOSED, FOCAL) and names taken from code are typeset as `this`. $\mathbf{A}^\top$ is the transpose and $\mathbf{A}^{-1}$ the inverse; $\|\cdot\|$ is the Euclidean norm unless a subscript says otherwise, and $|\cdot|$ is an absolute value or, for a finite set, its number of elements.

Indices. A subscript k or t is a time index: $\mathbf{x}_k$ is the state at step k and $\pi[t]$ the vertex occupied at step t . A subscript i or j is an agent index: π_i is the path of agent a_i , $\mathbf{p}_i$ its position, r_i its radius. When both are needed the agent moves up and the time stays down, as in $\mathbf{p}_k^i$; such a superscript is a label, never an exponent, while $\mathbf{A}^k$ is a power. Parenthesized superscripts count samples: $\mathbf{x}^{(i)}$ is the i -th particle or sigma point. Search chapters count time in unit steps, $t = 0, 1, 2, \dots$; control and estimation chapters count steps of the sampling interval, so step k is the instant $k \Delta t$.

Decorations. A star marks an optimal quantity (g^* , h^* , C^* , J^*), a hat an estimate from a filter or a predictor ($\hat{\mathbf{x}}_k$, $\hat{\mathbf{p}}_{T+h}$), a bar an average ($\bar{x}$), and a superscript minus a *prediction* made before the current measurement has been used ($\hat{\mathbf{x}}_k^-$, $\mathbf{P}_k^-$).

General mathematics

Symbol	Meaning	Introduced in
$\mathbb{R}, \mathbb{N}, \mathbb{Z}$	reals, naturals including 0, integers; $\mathbb{R}^n$ holds the n -vectors	Chapter 2
x , A	vector (bold lower case), matrix (bold upper case)	Chapter 2
0 , 1 , I	zero vector, vector of ones, identity matrix	Chapter 2
$\mathbf{A}^\top$, $\mathbf{A}^{-1}$	transpose and inverse of a matrix	Chapter 2
$\ \mathbf{x}\ $, $ a $	Euclidean norm; absolute value, or size of a finite set	Chapter 2
$\mathbf{a} \cdot \mathbf{b}$, $\mathbf{a} \times \mathbf{b}$	dot and cross product; $\mathbf{a}^\perp = (-a_y, a_x)$ is a quarter turn	Chapter 2
$A \oplus B$	Minkowski sum; inflating obstacles by the drone radius	Chapter 2
$\arg \min$, $\arg \max$	the argument that minimizes or maximizes a function	Chapter 2
$\text{diag}(\cdot)$, $\text{tr}(\cdot)$	diagonal matrix built from a list; trace of a matrix	Chapter 2
$\mathcal{O}(f(n))$	asymptotic upper bound in the size of the input	Chapter 2
$\hat{x}$, $\bar{x}$, x^* , x^-	estimate, average, optimal value, prediction before the update	Chapter 18

Graphs, grids and search

Symbol	Meaning	Introduced in
$G = (V, E)$	graph with vertex set V and edge set E ; G_4 , G_8 are grids	Chapter 2
$c(u, v)$	cost of the edge (u, v) , also written w ; costs are non-negative	Chapter 2
$\text{Succ}(u)$, $\text{Pred}(u)$	successors and predecessors; b is the branching factor	Chapter 2
s , γ	start and goal vertex, everywhere in the book; Chapter 5 keeps the s_{start} , s_{goal} of the D* Lite paper	Chapter 2
π , $\text{cost}(\pi)$, $L(\pi)$	a path, its cost, and its Euclidean length	Chapter 2
$\text{dist}(u, v)$	shortest-path distance from u to v ; CLRS write it $\delta(u, v)$	Chapter 2
(v, t) , $T_{\max}$	space-time state (vertex v at step t); planning horizon	Chapter 2
$g(n)$, $g^*(n)$	best cost from s to node n found so far; its optimum	Chapter 4
$h(n)$, $h^*(n)$	heuristic estimate of the cost to go; the true cost to go	Chapter 4
$f(n)$, C^*	priority $g + h$; optimal solution cost $C^* = h^*(s)$	Chapter 4
OPEN, CLOSED	the frontier (a priority queue) and the expanded nodes	Chapter 3
FOCAL, $f_{\min}$	focal band $\{n \in \text{OPEN} : f(n) \leq w f_{\min}\}$; $f_{\min}$ bounds C^*	Chapter 10
$w \geq 1$	suboptimality factor of weighted A*, focal search and ECBS	Chapter 4
ε	inflation factor of ARA*; $w = 1 + \varepsilon$ in A^*_ε	Chapter 6
$\text{rhs}(s)$	one-step look-ahead value; $g(s) = \text{rhs}(s)$ is local consistency	Chapter 5
$\text{key}(s)$, k_m	two-component priority and key modifier of D* Lite	Chapter 5

Multi-agent path finding

Symbol	Meaning	Introduced in
k , a_i	number of agents; agent i , with start s_i and goal γ_i	Chapter 7
π_i , $\pi_i[t]$	path of a_i and the vertex it occupies at step t	Chapter 7
$\text{cost}(\pi_i)$, T_i	time of final arrival at the goal; last stored index of π_i	Chapter 7
$\Pi = (\pi_1, \dots, \pi_k)$	a plan: one time-indexed path per agent	Chapter 7
$\text{SoC}(\Pi)$	sum of costs: the total of $\text{cost}(\pi_i)$ over all agents	Chapter 7
$\text{MS}(\Pi)$	makespan: the largest $\text{cost}(\pi_i)$, $\text{MS} \leq \text{SoC} \leq k \text{MS}$	Chapter 7
$\langle a_i, a_j, v, t \rangle$	vertex conflict: both agents occupy v at time t	Chapter 7
$\langle a_i, a_j, u, v, t \rangle$	edge or swapping conflict on $\{u, v\}$ between t and $t + 1$	Chapter 7
$\langle a_i, v, t \rangle$	vertex constraint: a_i may not occupy v at time t	Chapter 9
$\mathcal{C}_i$, N	constraints on a_i ; constraint-tree node N . $\mathcal{C}$, N . π , N .cost	Chapter 9
σ	priority order: a permutation of the agents, $\sigma(1)$ first	Chapter 8
h_c	number of remaining conflicts, the FOCAL heuristic of ECBS	Chapter 10
$C(v)$, $\Psi(v, v')$	collision set of a joint vertex and of a joint move; policy ϕ_i	Chapter 11

Geometry and motion

Symbol	Meaning	Introduced in
$\mathcal{W}, \mathcal{O}$	workspace and the obstacle region inside it	Chapter 2
$\mathcal{C}, \mathcal{C}_{\text{free}}, \mathcal{C}_{\text{obs}}$	configuration space, its free part, its obstacle region	Chapter 2
$D(\mathbf{c}, r)$	closed disk (in 3D, ball) of radius r around $\mathbf{c}$	Chapter 2
$r_i, R = r_A + r_B$	radius of an agent; collision distance of a pair	Chapter 2
$\mathbf{p}, \mathbf{v}, \mathbf{a}$	position, velocity, acceleration; limits v_{max} and a_{max}	Chapter 2
Δt	time step (sampling interval), in seconds	Chapter 2
$\mathbf{x}, \mathbf{u}$	state and control input of a motion model	Chapter 2
$\mathbf{A}, \mathbf{B}$	state-transition and input matrix of the double integrator	Chapter 2
$d_{ij}(t), d_{\min}$	separation of two agents; its minimum over the execution	Chapter 2
$d_{\text{stop}}(v)$	stopping distance $v^2/(2a_{\text{max}})$ of a braking drone	Chapter 2
$\mathbf{p}_{\text{rel}}, \mathbf{v}_{\text{rel}}$	relative position $\mathbf{p}_B - \mathbf{p}_A$ and relative velocity $\mathbf{v}_A - \mathbf{v}_B$ of a pair; these signs hold in Chapters 2, 12 and 13 alike	Chapter 2
t^*, t_c	time of closest approach and time to collision of $\mathbf{p}_{\text{rel}}, \mathbf{v}_{\text{rel}}$	Chapter 2
τ	horizon of the local safety layer, never a time to collision; Chapters 1 and 24 write this horizon τ_h and the time to collision t_c	Chapter 12
τ_h	safety horizon of the hybrid architecture: a predicted conflict counts as a near-term risk when $t_c < \tau_h$, and Chapter 24 takes τ_h equal to the ORCA horizon τ	Chapter 24
R_{safe}	safety radius of a pair in the hybrid architecture, $R_{\text{safe}} = r_A + r_B + \Delta r$ with the safety margin $\Delta r \geq 0$; the separation the trigger and the replanner keep	Chapter 24
$VO_{A B}, VO_{A B}^\tau$	velocity obstacle and its truncation at τ ; $CC_{A B}$ is its cone	Chapter 12
$RVO_{A B}$	reciprocal velocity obstacle, apex at $\frac{1}{2}(\mathbf{v}_A + \mathbf{v}_B)$	Chapter 13
$ORCA_{A B}^\tau$	the ORCA half-plane of velocities A may choose	Chapter 13
$\mathbf{u}, \mathbf{n}$	smallest change of the relative velocity and the outward normal	Chapter 13
$\mathbf{v}^{\text{pref}}, \mathbf{v}^{\text{new}}$	preferred velocity and the velocity finally commanded	Chapter 13
V_s, V_a, V_d, V_r	possible, admissible, dynamic-window and resulting velocities	Chapter 14
$U(\mathbf{p}), \rho, \rho_0$	potential field; clearance and influence distance	Chapter 15
$\mathcal{X}_{\text{free}}, \mathcal{X}_{\text{obs}}$	free and blocked parts of the state space $\mathcal{X}$ of Part V, the $\mathcal{C}_{\text{free}}$, $\mathcal{C}_{\text{obs}}$ above; its points are states $\mathbf{x}$	Chapter 16
$\mathcal{T} = (V, E), \eta$	tree grown by RRT and its step size; Nearest, Steer extend it	Chapter 16

Probability and filtering

Symbol	Meaning	Introduced in
$\mathbb{P}(\cdot), p(\mathbf{x})$	probability of an event; density of a random vector	Chapter 2
$\mathbb{E}[\cdot], \text{Var}, \text{Cov}$	expectation, variance, covariance	Chapter 2
$\mathcal{N}(\mu, \Sigma)$	Gaussian of mean μ , covariance Σ ; σ_i, ρ_{ij} are its spreads	Chapter 2
$\mathcal{E}_k$	$k\sigma$ covariance ellipse: Mahalanobis distance k from the mean	Chapter 2
$\mathbf{x}_k, \mathbf{z}_k$	state and measurement at step k ; $\mathbf{z}_{1:k}$ all up to k	Chapter 2
$\hat{\mathbf{x}}_k^-, \mathbf{P}_k^-$	predicted mean and covariance, before $\mathbf{z}_k$ is used	Chapter 18
$\hat{\mathbf{x}}_k, \mathbf{P}_k$	updated (posterior) mean and covariance	Chapter 18
$\mathbf{F}_k$	state-transition matrix, or the Jacobian $\partial f / \partial \mathbf{x}$	Chapter 18
$\mathbf{H}_k$	measurement matrix, or the Jacobian $\partial h / \partial \mathbf{x}$	Chapter 18
$\mathbf{Q}_k, \mathbf{R}_k$	process-noise and measurement-noise covariance	Chapter 18
$\mathbf{w}_k, \mathbf{v}_k$	process noise and measurement noise, of covariance $\mathbf{Q}_k, \mathbf{R}_k$	Chapter 19
$\mathbf{y}_k, \mathbf{S}_k$	innovation $\mathbf{z}_k - h(\hat{\mathbf{x}}_k^-)$ and its covariance	Chapter 19
$\mathbf{K}_k$	Kalman gain $\mathbf{P}_k^- \mathbf{H}_k^\top \mathbf{S}_k^{-1}$	Chapter 18
$\mathcal{X}_i, \mathbf{x}^{(i)}, \omega^{(i)}$	sigma point; particle i and its weight, ω to keep $\mathbf{w}_k$ for the process noise	Chapter 19
$T_{\text{obs}}, T_{\text{pred}}$	observation window and prediction horizon; h is a horizon step, as in $\hat{\mathbf{p}}_{T+h}$, and is not the consensus step of Chapter 23	Chapter 20
ADE, FDE	average and final displacement error; minADE_K over K samples	Chapter 20
$\mathbf{Q}, \mathbf{K}, \mathbf{V}$	query, key and value matrices of attention; this meaning only here	Chapter 20
κ_p	uncertainty inflation factor: the predicted mean is padded by $\kappa_p \sigma(h)$ with $\sigma(h) = \sqrt{\lambda_{\max}(\Sigma_h)}$, so that the disk holds the prediction with probability p ($\kappa_p = 2.45$ for 95 %)	Chapter 24

Warnings about letters. First, $\mathbf{Q}$ and $\mathbf{R}$ are *noise covariances* in Part VI, where they say how far the motion model and the sensor may be trusted, and *cost weights* in Part VII, where they say how much a tracking error and a control effort cost. The two roles are unrelated, they carry different units, and no equation in this book uses both at once; Chapter 21 repeats the warning where the weights appear. Second, this book follows the estimation literature and writes $\mathbf{F}$ for the state-transition matrix and $\mathbf{H}$ for the measurement matrix of a filter, where control texts usually write $\mathbf{A}$ and $\mathbf{C}$. The plant models of Chapter 2 and the controller of Chapter 21 do use $\mathbf{A}$ and $\mathbf{B}$, because there they describe a vehicle and not a filter. For the same reason Chapter 23 writes its adjacency matrix $\mathbf{W}$: only the pinning matrix $\mathbf{B}$ of its leader keeps a plant letter, and it is graph data, not a plant. Third, h carries two meanings that no equation combines: it is the step of a prediction horizon in Chapters 20 and 24, where $\hat{\mathbf{p}}_{T+h}$, Σ_h and $\mathcal{B}_h$ are indexed by it, and the step size of the discrete consensus iteration $(\mathbf{I} - h\mathbf{L})$ in Chapter 23, where it is a positive real bounded by $2/\lambda_n$ and not an index at all. Finally, ε is the inflation factor of ARA^* in Chapter 6 (and of A_ε^* in Chapter 10) and a numerical tolerance in Chapters 13, 21 and 22; the two are listed in different tables above for that reason.

Optimization and control

Symbol	Meaning	Introduced in
J, J^*	objective of an optimization problem and its optimal value	Chapter 21
subject to	introduces the constraints of an optimization problem	Chapter 21
N	prediction horizon in steps of Δt ; $k = 0$ is the present	Chapter 21
$\mathbf{x}_k^{\text{ref}}$	reference state at step k of the horizon	Chapter 21
$\mathbf{Q} \succeq \mathbf{0}, \mathbf{R} \succ \mathbf{0}$	weights on the state error and on the control effort	Chapter 21
$\mathbf{P}_f \succeq \mathbf{0}$	terminal cost weight on the last predicted state	Chapter 21
$\mathbf{S}_x, \mathbf{S}_u$	prediction matrices that stack the horizon into one map	Chapter 21
$\mathbf{z}$	stacked decision variable of the resulting QP or LP	Chapter 21
$\mathbf{H}, \mathbf{f}, \mathbf{G}$	Hessian, linear term and constraint matrix, $\mathbf{1} \leq \mathbf{G}\mathbf{z} \leq \mathbf{u}_{\max}$	Chapter 21
$\mathbf{y}, \rho$	Lagrange multipliers and penalty parameter of the ADMM solver	Chapter 21
s_k	slack variable that softens a constraint at step k	Chapter 21
ε	tolerance of a numerical solver (residuals, integrality)	Chapter 21
$\mathbf{c}, \mathbf{A}, \mathbf{b}$	LP data: $\min \mathbf{c}^T \mathbf{z}$ over $\mathbf{A}\mathbf{z} \leq \mathbf{b}$; P is the feasible polyhedron	Chapter 22
$z_j \in \mathbb{Z}, b$	integer variable and binary variable of a mixed-integer program	Chapter 22
M	big- M constant: $b = 1$ switches an inequality off	Chapter 22

Networks and consensus

Symbol	Meaning	Introduced in
$G = (V, E)$	communication graph: the drones and the pairs that can talk	Chapter 23
N_i	the neighbors of drone i in that graph	Chapter 23
$\mathbf{W} = (w_{ij}), \mathbf{D}$	adjacency matrix, $\mathbf{W}$ and not $\mathbf{A}$; degree matrix $\text{diag}(d_1, \dots, d_n)$	Chapter 23
$\mathbf{L} = \mathbf{D} - \mathbf{W}$	graph Laplacian; symmetric, $\mathbf{L}\mathbf{1} = \mathbf{0}$	Chapter 23
λ_2, λ_n	algebraic connectivity (Fiedler value); largest eigenvalue of $\mathbf{L}$	Chapter 23
$\bar{x}, \delta$	average of the states; disagreement vector $\mathbf{x} - \bar{x}\mathbf{1}$	Chapter 23
h	step size of discrete consensus, $\mathbf{x}_{k+1} = (\mathbf{I} - h\mathbf{L})\mathbf{x}_k$; unrelated to the prediction-horizon step h of Chapters 20 and 24	Chapter 23
R_{comm}	communication range: $\{i, j\} \in E$ iff $\ \mathbf{p}_i - \mathbf{p}_j\ \leq R_{\text{comm}}$	Chapter 23
$\mathbf{o}_i, \mathbf{d}_{ij}$	formation offset of drone i ; desired displacement $\mathbf{o}_j - \mathbf{o}_i$	Chapter 23
$e_F, e_{\max}$	formation error (RMS displacement violation) and its tolerance	Chapter 23
k_c, k_l	consensus gain of the first-order protocol; pinning gain of the leader	Chapter 23
k_p, k_v, k_d	position, velocity and damping gains of second-order consensus	Chapter 23

Evaluation

Symbol	Meaning	Introduced in
$N, T_{\max}$	number of runs in one cell of the experiment matrix; cap on the length of a run. This N is not the horizon N of Chapter 21	Chapter 25
$X_{\text{coll}}, n_{\text{coll}}$	collision indicator of a run and its number of colliding pairs	Chapter 25
$\hat{p}_{\text{coll}}$	collision rate of a cell: the fraction of its runs with $X_{\text{coll}} = 1$	Chapter 25
$d_{\min}^{\text{dd}}, d_{\min}^{\text{di}}$	minimum separation of a run, drone–drone and drone–intruder; a near miss is a run whose $d_{\min}$ falls below the safety radius	Chapter 25
L_i, ρ_i, ρ_L	flown path length of drone i , its ratio L_i/L_i^{nom} to the nominal plan, and the mean ratio of a run	Chapter 25
T_i, ε_g	travel time of drone i and the goal tolerance that ends it; MS = $\max_i T_i$ and SoC = $\sum_i T_i$ are then read in seconds	Chapter 25
$n_{\text{rp}}, n_{\text{av}}$	replanning count of a run and its number of steps spent in Avoiding or Reconnecting	Chapter 25
$c_i(t), \bar{c}, c_{99}$	computation time of one decision, its mean and its 99th percentile; C_{tot} is the total per run	Chapter 25
$\bar{e}_F, e_F^{\max}$	time average and peak of the formation error $e_F(t)$ of Chapter 23	Chapter 25
n_{cv}	communication violations: steps at which a kept edge is longer than R_{comm} or the radius graph is disconnected	Chapter 25

Part I

Foundations

Planning for Drone Swarms

Learning objectives

- Describe the problem that this book solves: a swarm of controlled drones flying planned, time-indexed routes through a shared space in which unknown, non-cooperative moving objects appear.
- Distinguish a *planned conflict* between two drones of the swarm from an *unexpected obstacle*, and say which of the two is resolved before take-off and which one during the flight.
- Use the vocabulary of the field correctly: workspace and configuration space, discrete and continuous planning, global and local planning, deliberative and reactive, centralized and decentralized, cooperative and non-cooperative, tracking and prediction, and the properties completeness, optimality, bounded suboptimality, anytime, incremental and real-time.
- Name the four layers of the hybrid architecture, say what each layer receives and delivers, and give the time scale on which each one works.
- Locate every algorithm of the training plan in this book, state its priority, and say which parts of the field are established and where research questions remain.
- Choose a reading path through the book and set up the Python stack that its code uses.

1.1 The problem: many drones, one airspace, and a stranger

Imagine that you are responsible for a fleet of eight quadrotors that inspect the roof of a large factory every morning. Each drone has its own list of places to photograph, and all of them share the same airspace. Before take-off you need a route for every drone, and you need to be sure that no two routes bring two drones to the same place at the same time. During the flight something unplanned happens: a drone that is not yours crosses the site. It does not talk to your fleet, it does not follow your plan, and you cannot know exactly where it is going. Your drones must notice it, guess where it is heading, get out of its way, and then return to the plan—or, if that is impossible, make a new plan and check it against the rest of the fleet.

This book is about exactly that problem, and this chapter states it carefully. Every later chapter teaches one algorithm that solves one piece of it. The setting has four ingredients.

- **A swarm of controlled drones.** There are k drones that we command. We know their positions, we can give each of them a route, and they do what we tell them. In this book they are called the *agents* of the swarm, or simply drones $A, B, C, \dots$
- **A shared space, represented on a grid or a graph.** The drones fly through one

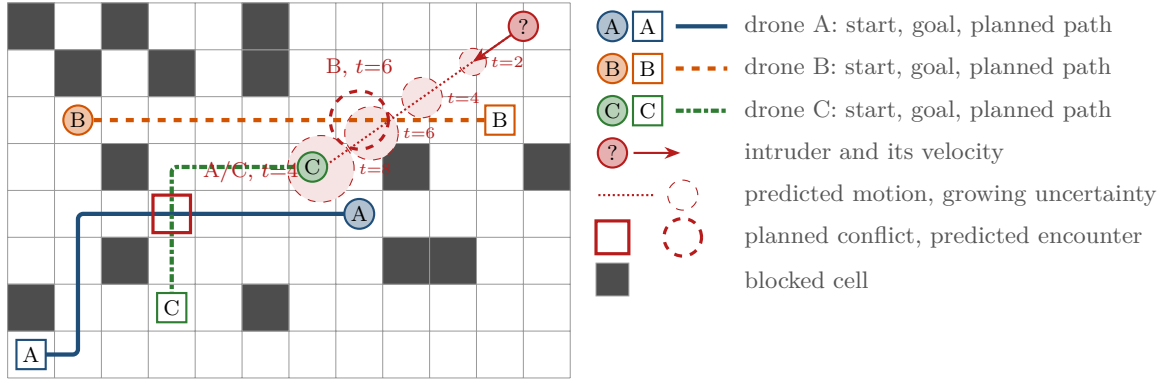


Figure 1.1. The scenario of Example 1.3: a 12×8 grid with 14 blocked cells, three controlled drones with their independently planned shortest paths, and one intruder entering from the top right with a velocity of $(-0.54, -0.38)$ cells per step. Two things go wrong. The paths of A and C both pass through cell (3,3) at $t = 4$ (red square): a *planned conflict*, visible before take-off. The intruder's predicted motion (dotted, with growing uncertainty disks) passes within 0.38 cells of drone B at $t = 6$: an *unexpected obstacle*, visible only during the flight. The red dashed circle marks the cell that B occupies at $t = 6$, where that encounter happens; its radius is not the 0.38-cell gap.

three-dimensional volume. For planning we discretize that volume into cells of a grid, or more generally into the vertices of a graph $G = (V, E)$ whose edges are the moves that a drone can make in one time step. Chapter 2 explains the representations in detail; most figures of the book show a horizontal slice of the space, that is, a two-dimensional grid at the altitude of the swarm.

- **Planned, time-indexed routes.** A route is not just a curve. It says *where* a drone is at *each* time step. Only with time attached can we say whether two routes collide, and only then can a planner remove the collision before anybody takes off.
- **Unknown, non-cooperative moving objects.** During the flight, objects that are not part of the swarm may appear: other drones, most often. We do not know their plans. We observe them with noisy sensors, so even their current state is uncertain, and their future motion is more uncertain still. The book calls such an object an **intruder**.

1.1.1 Two kinds of trouble

Figure 1.1 shows a small instance of the problem, and it shows the two kinds of trouble that this book is about. The first kind is visible before anybody takes off. Drone A wants to fly from (7,3) to (0,0) and drone C from (6,4) to (3,1). Each of them, planning alone, chooses a shortest path, and both shortest paths use cell (3,3) at time step 4. Nothing is unknown here: we know both routes, we know both clocks, and we can see the collision coming while the drones are still on the ground. The second kind of trouble is invisible before take-off. The intruder in the top right corner was not in anybody's plan. Once it is observed, we can estimate its velocity and extrapolate: if it keeps flying straight, it will pass within 0.38 cells of drone B at $t = 6$. But it may also turn, slow down or stop, and the further we look ahead, the less we know. The two kinds are defined next.

Definition 1.1 (Planned conflict). A **planned conflict** is a collision between two controlled drones that follows from their planned routes alone: at some time step both routes occupy the same vertex, or they traverse the same edge in opposite directions. A planned conflict can be detected, and resolved, before the flight, by a *multi-agent planner* that knows all routes.

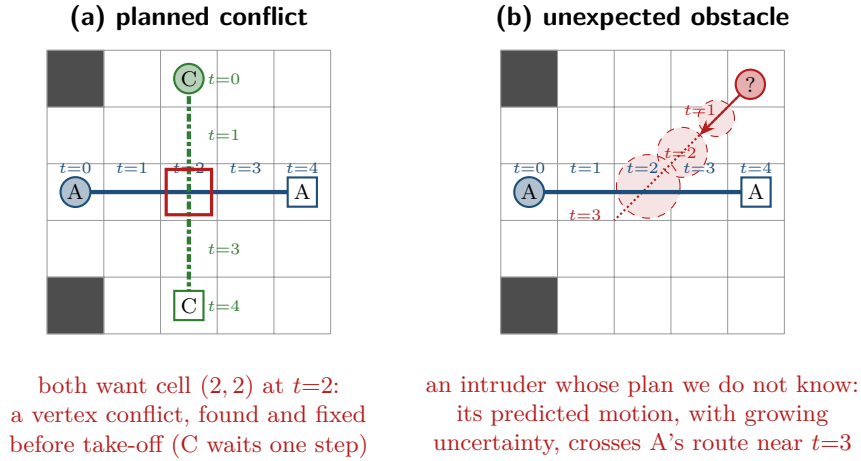


Figure 1.2. The two kinds of trouble on a 5×5 grid. (a) Drones A and C both want cell (2,2) at $t = 2$: a planned conflict. Everything about it is known before take-off, and letting C wait one step removes it. (b) An intruder whose plan we do not know: from its observed velocity we predict where it will be at $t = 1, 2, 3$, with a disk of uncertainty that grows with the horizon, and the prediction crosses the route of A near $t = 3$. Nothing about it could have been planned for.

Definition 1.2 (Unexpected obstacle). An **unexpected obstacle** is an object that was not part of the plan and that is observed only during the flight. Its current state is known only through noisy measurements and its future motion is uncertain. It must be handled *during* the flight, by whatever the drone can decide in the time it has.

The difference matters because the two kinds of trouble call for different tools. A planned conflict is a problem of *coordination* among agents that we fully control; it is solved once, before the flight, by a planner that may take a few seconds and that can guarantee a collision-free plan. An unexpected obstacle is a problem of *reaction* under uncertainty; it must be solved many times per second, by each drone on its own, and no algorithm can guarantee safety against an object whose motion is completely unknown; one can only do so by bounding its speed and planning against everywhere it could reach, which at a horizon of a few seconds blocks so much of the airspace that the swarm cannot fly (Chapter 12 truncates the velocity obstacle for exactly this reason). Figure 1.2 contrasts the two side by side.

The core idea of the book

Use a *global planner* for the planned routes, a *local collision-avoidance method* for the surprises, and a *replanner* only when the local response is insufficient.

This one sentence is the plan of the whole book. The global planner is built by Chapters 3 and 4 and Chapters 6 to 11, from single-agent graph search to multi-agent path finding. Chapters 12 to 15 build the local layer. Chapters 18 to 20 build the tracking and prediction that the local layer needs in order to see the surprise coming. Chapters 5, 16 and 17 and Chapters 21 to 23 provide the replanning, the continuous-space planning and the control that hold everything together, and Chapters 24 and 25 assemble and evaluate the result.

1.1.2 A scenario in numbers

Numbers make the distinction concrete. The scenario of Figure 1.1 was produced by the scenario generator `code/ch01_scenario.py` of this chapter with the fixed seed 1462; every number quoted below is printed by that program.

Table 1.1. Time line of [Example 1.3](#): the cell of each drone, the predicted position of the intruder, and its distance to the center of each drone’s cell (in cells). The planned conflict is the row in which A and C share a cell; the encounter is the row in which d_B drops to 0.38. Drones stay at their goals after arriving.

t	A	B	C	Intruder (x, y)	d_A	d_B	d_C
0	(7,3)	(1,5)	(6,4)	(11.00, 7.50)	5.32	9.71	5.41
1	(6,3)	(2,5)	(5,4)	(10.46, 7.12)	5.37	8.12	5.61
2	(5,3)	(3,5)	(4,4)	(9.92, 6.74)	5.48	6.54	5.86
3	(4,3)	(4,5)	(3,4)	(9.38, 6.36)	5.66	4.96	6.17
4	(3,3)	(5,5)	(3,3)	(8.84, 5.98)	5.89	3.37	5.89
5	(2,3)	(6,5)	(3,2)	(8.30, 5.60)	6.17	1.80	5.71
6	(1,3)	(7,5)	(3,1)	(7.76, 5.22)	6.49	0.38	5.66
7	(1,2)	(8,5)	(3,1)	(7.22, 4.84)	6.18	1.44	5.00
8	(1,1)	(9,5)	(3,1)	(6.68, 4.46)	5.97	3.01	4.34
9	(1,0)	(10,5)	(3,1)	(6.14, 4.08)	5.86	4.59	3.69
10	(0,0)	(10,5)	(3,1)	(5.60, 3.70)	6.02	5.22	3.04

Example 1.3 (Three drones and one intruder). The grid has 12×8 cells, 14 of which are blocked. Cell (x, y) covers the unit square with lower-left corner (x, y) , its center is $(x+0.5, y+0.5)$, and the grid is 4-connected: in one time step a drone moves to a horizontally or vertically adjacent free cell, or it waits. A drone that has reached its goal stays there.

- Drone A flies from $(7, 3)$ to $(0, 0)$. Its shortest path has 10 steps: $(7, 3), (6, 3), \dots, (1, 3), (1, 2), (1, 1), (1, 0), (0, 0)$.
- Drone B flies from $(1, 5)$ to $(10, 5)$ along row 5 in 9 steps.
- Drone C flies from $(6, 4)$ to $(3, 1)$ in 6 steps: $(6, 4), (5, 4), (4, 4), (3, 4), (3, 3), (3, 2), (3, 1)$.

Planned independently, the three paths have a sum of costs of $10 + 9 + 6 = 25$ steps and a makespan of 10 steps ([Chapter 2](#) defines both measures). They contain exactly one planned conflict: a vertex conflict of A and C in cell $(3, 3)$ at $t = 4$. If either drone waits one step before entering $(3, 3)$, the conflict disappears and the sum of costs rises to 26.

The intruder is first observed at $(11.0, 7.5)$ with velocity $(-0.54, -0.38)$ cells per step, that is, a speed of 0.66 cells per step. If it keeps that velocity, then at $t = 6$ it is at $(7.76, 5.22)$ while drone B occupies cell $(7, 5)$ with center $(7.5, 5.5)$: a distance of 0.38 cells, far less than the safety distance that any real swarm would demand. If a cell is 2 m wide, the intruder passes 0.76 m from the center of B ’s cell—well inside the 2 m separation used in [Section 1.3.3](#).

[Table 1.1](#) lists the whole time line.

Look at what the table does and does not tell you. The conflict in row $t = 4$ is a fact: given the two routes, it will happen. The encounter in row $t = 6$ is a *prediction*: it assumes that the intruder flies straight at constant velocity, which is the simplest model we could adopt and, as [Chapter 20](#) shows, not a bad one over a short horizon. The dashed disks in [Figure 1.1](#) grow with t to remind you that the prediction gets worse the further ahead it looks. A good swarm planner acts on the conflict now and on the encounter only when it is close enough in time to be believed—which is the subject of the safety horizon in [Section 1.3](#).

An intruder is not a wall

A tempting shortcut is to treat an intruder as a static obstacle: block the cells it currently occupies and replan around them. This fails twice. The intruder will have moved by the time the new plan is executed, so the plan avoids a place where nothing is, and it may

Table 1.2. The contrasts that organize the book. Most algorithms sit at one end of each pair; the hybrid architecture of [Chapter 24](#) deliberately combines both ends.

Pair	One end	The other end
Workspace / configuration space	the physical volume the drone flies in	the set of all configurations of the drone; obstacles are inflated by its size
Discrete / continuous	finite vertices and time steps; graph search	real-valued positions and time; sampling, optimization
Global / local	whole route from start to goal, all obstacles known	next few seconds, only what the sensors see
Planning / control	decides <i>where</i> to go and <i>when</i>	makes the vehicle actually follow the decision
Deliberative / reactive	thinks ahead, uses a model, takes time	maps the current situation directly to an action
Centralized / decentralized	one computer knows and decides everything	every drone decides from what it knows and hears
Cooperative / non-cooperative	the other agent shares the avoidance rules	the other agent does what it likes
Tracking / prediction	estimating the state <i>now</i> from noisy data	extrapolating the state into the future

steer the drone straight into where the intruder is heading. A moving object must be avoided in *space and time*, using a prediction of where it will be, not a snapshot of where it was.

1.1.3 Why not one method for everything?

Why not pick the most powerful planner and run it whenever anything changes? Two arguments answer this, and they recur throughout the book. First, *time*: a multi-agent planner that guarantees a collision-free plan for the whole swarm needs seconds, sometimes far longer, and a drone flying at a few meters per second cannot wait that long when another drone is two meters away. Second, *guarantees*: the global plan is collision-free among the swarm, optimal or provably close to it, and respects formation and communication constraints; throwing it away at every surprise means paying for those properties again, and possibly losing them in the meantime. The opposite extreme is equally poor: a swarm that only reacts never plans, and [Exercise 1.4](#) asks you to show that it can lock up in a corridor with nobody willing to go first. The hybrid architecture of [Section 1.3](#) keeps the global plan as the default, reacts locally when it must, and replans only when the reaction cannot bring the drone back.

1.2 Vocabulary: the words you need before anything else

Planning has a vocabulary of contrasts. Most of them are pairs of words that describe two ends of a spectrum, and every algorithm in this book sits somewhere between them. [Table 1.2](#) collects the pairs; the paragraphs below explain each one once, and the rest of the book uses them without further comment.

Workspace and configuration space. The **workspace** is the physical space in which the drone moves. The **configuration space** $\mathcal{C}$ is the set of all configurations of the drone; for a drone modeled as a point with a safety radius, a configuration is a position, and the

obstacle region $\mathcal{C}_{\text{obs}}$ is the workspace obstacle inflated by that radius. Planning takes place in $\mathcal{C}$, so that a path of the point in $\mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obs}}$ is a collision-free motion of the real drone. [Chapter 2](#) gives the definitions and the inflation operation [99].

Discrete and continuous planning. **Discrete planning** works on a graph: finitely many vertices and moves, and integer time steps. Dijkstra’s algorithm, A^* , D^* Lite and all multi-agent planners of the book are discrete. **Continuous planning** works with real-valued positions, velocities and times: RRT (rapidly-exploring random tree), RRT^* , model predictive control (MPC), mixed-integer linear programming (MILP) and the velocity-space methods of local avoidance. A grid is a sampling of the continuous workspace, and a discrete plan becomes a continuous trajectory when the drone flies between the cell centers.

Time-indexed paths. The single most important object in this book is a path with time attached.

Definition 1.4 (Time-indexed path). A **time-indexed path** of an agent on a graph $G = (V, E)$ is a sequence $\pi = (v_0, v_1, \dots, v_T)$ of vertices such that v_0 is the agent’s start, v_T is its goal, and for every $t < T$ either $v_{t+1} = v_t$ (a *wait* action) or $\{v_t, v_{t+1}\} \in E$ (a *move* action). The agent occupies v_t during time step t and stays at v_T for all $t \geq T$. A **plan** for a swarm is one time-indexed path per agent.

Without the index t two paths can cross without a collision, because the drones pass the crossing at different times; with the index, a collision is a precise statement, $v_t^A = v_t^C$ for some t . [Chapter 2](#) turns this into the time-expanded graph, also called the space-time graph, and [Chapter 7](#) lists every kind of conflict that time-indexed paths can have, in the standard vocabulary of **multi-agent path finding** (MAPF) [161].

Global and local planning. A **global planner** knows the goal but not the surprise; a **local planner** knows the surprise but not the goal. The local methods of the book are those of [Chapters 12 to 15](#); the search and sampling methods of Parts II, III and V are global.

Planning and control. The book **plans** and treats the flight **controller**, which turns tracking error into motor commands, as a black box. Model predictive control ([Chapter 21](#)) is the exception that does both at once, which is why it is such a natural place to enforce constraints.

Deliberative and reactive. Being **reactive** instead of **deliberative** makes a method fast and robust to modeling errors, but short-sighted [142]. Global planners are deliberative, local avoidance is reactive, and the hybrid architecture layers the one on the other.

Centralized and decentralized. The global planner of the book is **centralized**, because conflict-based search (CBS) needs all routes in one place; local avoidance is **decentralized**, because each drone runs ORCA (optimal reciprocal collision avoidance) on what it sees and hears from its neighbors. Consensus and formation control ([Chapter 23](#)) are decentralized by design, and the communication radius that limits them is one of the constraints of [Section 1.5](#).

Cooperative and non-cooperative agents. Two agents are **cooperative** if both follow the same avoidance rules, so that each may count on the other to do its share; reciprocal velocity obstacles and ORCA rest on exactly this assumption [14]. A **non-cooperative** agent does whatever it does, and a drone that meets it must take the full responsibility for avoiding it. The drones of the swarm are cooperative with one another and non-cooperative with the intruder; [Chapter 13](#) handles both cases with a change of a single factor.

Table 1.3. Properties of planning algorithms, informally. The formal versions appear as theorems in the chapters of the last column. No algorithm in the book has all six.

Property	Informal meaning	Examples in this book
Complete	if a solution exists, the algorithm finds one; in the <i>strong</i> form it also reports failure when none exists, which needs a finite search space or a separate feasibility test. Sampling-based planners are <i>probabilistically</i> complete: the probability of finding a solution tends to one as the number of samples grows	Dijkstra and A* in the strong form (Chapters 3 and 4); CBS on solvable instances (Chapter 9; see the remark there on unsolvable instances); RRT, probabilistically (Chapter 16); prioritized planning is <i>not</i> complete (Chapter 8)
Optimal	the solution it returns has the least possible cost; RRT* is <i>asymptotically</i> optimal, that is, optimal in the limit of infinitely many samples	A* with an admissible heuristic (Chapter 4); CBS for the sum of costs (Chapter 9); RRT* (Chapter 17)
Bounded suboptimal	the cost is at most w times the optimal cost, for a factor $w \geq 1$ chosen by the user; $w = 1$ recovers optimality, and a larger w buys speed, a trade-off measured for ECBS in Chapter 10	weighted A* (Chapter 4); ECBS, Enhanced CBS (Chapter 10)
Anytime	returns a first, possibly poor, solution quickly and improves it while time remains; interrupt it whenever you like and take the best solution so far	ARA*, Anytime Repairing A*, which lowers w step by step (Chapter 6)
Incremental	reuses the previous search when the graph changes a little, an obstacle appears or an edge cost changes, instead of starting over	LPA*, Lifelong Planning A*, and D* Lite (Chapter 5): the replanning layer
Real-time	the work per call is bounded, so an answer is guaranteed within a fixed, short time budget; such methods search over a bounded set of velocities rather than over routes, which is why they can run at tens of hertz	ORCA, one small linear program per cycle (Chapter 13); the dynamic window approach (DWA), a fixed number of candidate velocities (Chapter 14)

Tracking and prediction. **Tracking** estimates the *current* state of a moving object, position and velocity, from a sequence of noisy measurements; the Kalman filter of Chapter 18 is the classical tool [168]. **Prediction** extrapolates that estimate into the *future* over a horizon of a few seconds and reports the growing uncertainty along the way (Chapter 20). Tracking ends at the present moment; prediction begins there.

1.2.1 Properties of algorithms

When the book says that an algorithm is complete, or optimal, or anytime, it means something precise. The precise statements come with the algorithms; here are the informal meanings, so that you can read the map of the book in Section 1.4 with them in mind. Table 1.3 summarizes them.

These properties conflict with one another, and choosing among them is the designer's job. An optimal multi-agent planner is not real-time; a real-time local method is not complete; an incremental planner can only repair what a complete planner has built. The four layers of the next section are chosen so that each property is provided by the layer that can afford it.

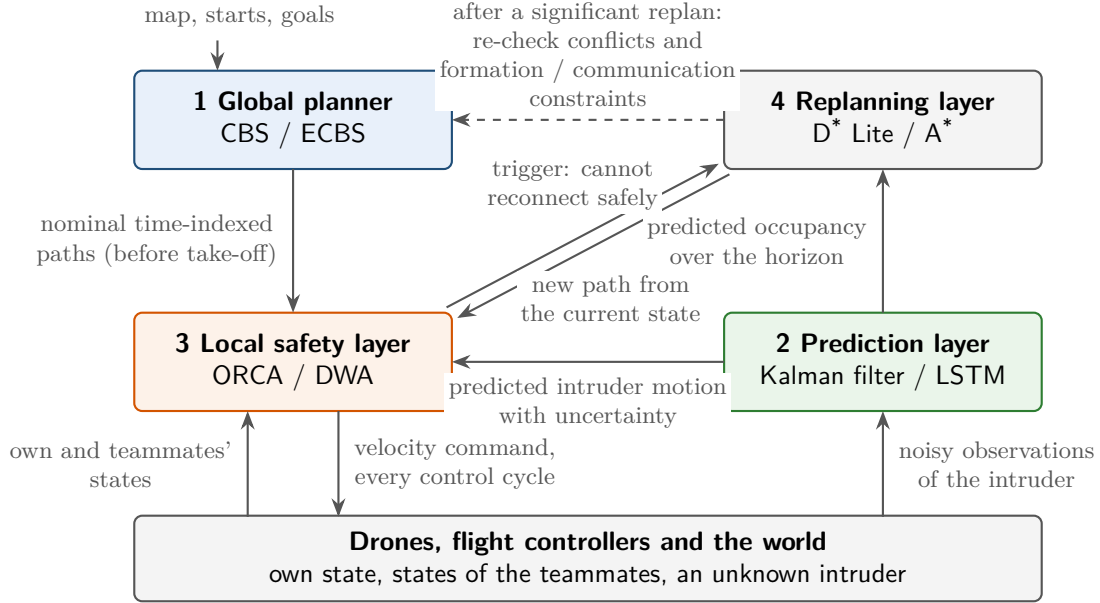


Figure 1.3. The four layers of the hybrid architecture and what flows between them. The global planner runs before take-off and hands nominal time-indexed paths to the local safety layer, which alone talks to the flight controllers. The prediction layer turns noisy observations of an intruder into a predicted motion with uncertainty, used by both the local layer and the replanner. The replanner is called only when the local layer cannot return to the nominal path; after a significant replan the swarm-level conflicts are checked again (dashed arrow).

1.3 A preview of the hybrid architecture

The book builds toward one system, and it helps to see the whole of it before studying its parts. The training plan that this book follows (reproduced in [Appendix A](#)) proposes a **hybrid collision-avoidance architecture** with four layers, shown in [Figure 1.3](#). This section is a preview in plain words; [Chapter 24](#) gives the full treatment, with the state machine, the triggers, the constraints and a discussion of which guarantees survive the combination.

1.3.1 The four layers

1. Global planner (CBS, ECBS). Before the flight, the global planner receives the map, the start and goal of every controlled drone, and any constraints on formation or communication range. It returns a nominal, time-indexed path for each drone such that no two drones are ever in conflict. The book’s global planners are conflict-based search, which is optimal for the sum of costs [150], and its bounded-suboptimal variant ECBS, which is much faster for a chosen factor w ([Chapters 9](#) and [10](#)). Both build on single-agent search in space and time ([Chapter 4](#)). This layer resolves the planned conflicts of [Definition 1.1](#); it never sees the intruder.

The four names just used—*global planner*, *prediction layer*, *local safety layer*, *replanning layer*—are the names this book uses for the four layers everywhere, in the text, in [Figure 1.3](#) and in [Table 24.1](#). Two short forms are allowed once the layer has been named: *local layer* for the local safety layer, and *replanner* for the replanning layer. *Reactive* and *deliberative* keep the meanings they have in [Section 1.2](#): they describe a *style* of method, not one of the four boxes.

2. Prediction layer (Kalman filter, LSTM). During the flight, the prediction layer observes every object that is not part of the swarm. From noisy position measurements it estimates the object’s current position and velocity with a Kalman filter [168] ([Chapters 18](#)

and 19) and then predicts a short future trajectory, either by extrapolating at constant velocity or with a learned model such as a long short-term memory (LSTM) network (Chapter 20). The output is not a single line but a line *with uncertainty*: a sequence of predicted positions, each with a covariance that grows along the horizon, exactly as the disks in Figure 1.1 do.

3. Local safety layer (ORCA, DWA). Every control cycle, each drone runs a local method that takes its own state, the states of its teammates and the predicted motion of any intruder, and produces a safe velocity for the next fraction of a second. The book’s local methods are ORCA [14], which chooses the velocity closest to the preferred one inside a set of half-planes (Chapter 13), and the dynamic window approach, which scores a small set of dynamically feasible velocities (Chapter 14). When nothing threatens, the preferred velocity is the one that follows the nominal path, and the local layer simply passes it on. When a predicted collision enters the safety horizon, the local layer deviates temporarily *without discarding the global route*.

4. Replanning layer (D* Lite, A*). Sometimes the local maneuver cannot bring the drone safely back to its nominal path: the intruder parks in the corridor, the detour has become too long, or the deviation has broken the formation. Then the drone replans from its current state to its goal. D* Lite [88] does this incrementally, reusing the previous search (Chapter 5); plain A* in space and time is the fallback (Chapter 4). A new path for one drone may conflict with the others, so a significant replan is followed by a conflict re-check at the swarm level—the dashed arrow back to the global planner in Figure 1.3.

1.3.2 The decision logic in five steps

The layers are only useful with a rule that says which one acts when. The training plan states that rule in five steps, and Figure 1.4 draws it. Two quantities appear in it. The **time to collision** t_c of a drone with a predicted object is the earliest future time at which the prediction brings the two closer than the safety distance (infinite if it never does). The **safety horizon** τ_h is a look-ahead time chosen by the designer, a few seconds at most; a predicted conflict counts as a *near-term risk* when $t_c < \tau_h$. Chapter 12 computes t_c from relative position and velocity (Definition 12.5) and uses τ for the horizon over which a velocity obstacle is truncated. Chapter 24 writes this horizon τ_h as well, and takes it equal to the ORCA horizon τ of Chapter 13.

1. **Follow the nominal path.** As long as no predicted conflict lies inside the safety horizon, every drone follows the time-indexed path that the global planner computed. This is the normal state, and most of the time nothing else happens.
2. **Avoid locally.** If an unknown moving object creates a near-term risk, the drone invokes ORCA or DWA to produce a local avoidance action: a velocity that keeps it clear of the predicted motion of the object and of its teammates, and that deviates from the nominal path as little as possible.
3. **Reconnect.** When the conflict has cleared, the drone tries to rejoin its nominal path at a *safe future waypoint*: a point on the path that it can reach in time without a new conflict. If it succeeds, the global plan is intact and nothing else needs to change.
4. **Replan.** If reconnection is impossible, too costly, or would violate a formation or communication constraint, the drone runs D* Lite or A* from its current state and follows the new path.
5. **Re-check.** After any significant replan, the swarm checks the new path against the paths of the other drones, and against the formation and communication constraints, and repairs what the replan has broken.

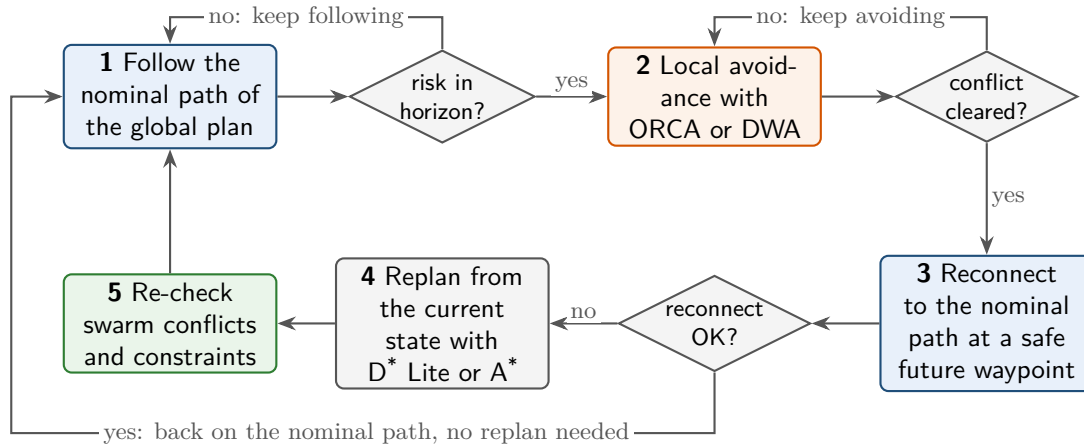


Figure 1.4. The five-step decision logic as a loop. The normal path is the top row: follow, and keep following while no risk lies inside the safety horizon. Only when a local avoidance action cannot be reconnected to the nominal path does the bottom row run: replan, re-check the swarm, and continue.

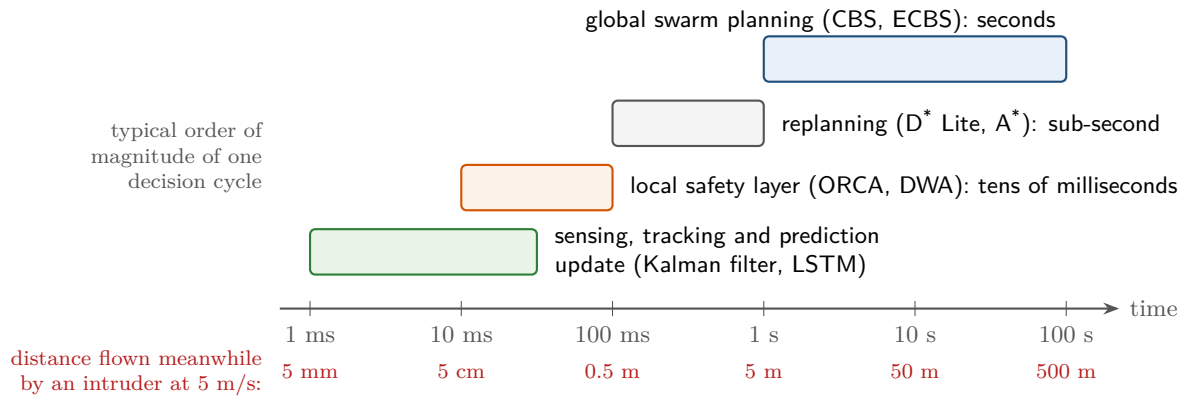


Figure 1.5. Typical orders of magnitude of one decision cycle of each layer, on a logarithmic time axis. The local safety layer acts in tens of milliseconds, a replan takes a fraction of a second, and a global multi-agent plan takes seconds. The lower row shows how far an intruder flying at 5 m/s moves in the meantime: a layer that needs one second cannot be the one that reacts to an object two meters away.

Notice the order of preference built into the rule. Doing nothing is best; a local maneuver is second best because it keeps the global plan; a replan is a last resort because it costs computation and may invalidate the plans of the others. Deciding *automatically* where the line between step 3 and step 4 lies is one of the open questions of [Section 1.5](#).

1.3.3 Time scales

Why four layers rather than one? Because they work on different time scales, and no single algorithm can serve all of them. [Figure 1.5](#) places the layers on a logarithmic time axis. The numbers are typical orders of magnitude for the kind of implementation this book describes, not measurements; your own numbers will depend on the hardware, the size of the swarm and the map, and [Chapter 25](#) shows how to measure them.

- The **local safety layer** must deliver a velocity every control cycle, typically every few tens of milliseconds; ORCA and DWA can, because their work per cycle is small and

bounded.

- **Replanning** takes a fraction of a second: D^* Lite repairs only the part of the search that the change affected, and the local layer keeps the drone safe while the replanner works.
- **Global planning** for the whole swarm takes seconds, on hard instances much longer; it runs before take-off and after a significant replan, never inside the control loop.
- **Sensing, tracking and prediction** run at the rate of the sensors; a Kalman update is a handful of small matrix operations and a trained LSTM evaluates in milliseconds.

The lower row of [Figure 1.5](#) explains why the order matters. An intruder flying at 5 m/s covers 10 cm during a 20 ms control cycle, 2.5 m during a half-second replan, and 15 m during a three-second global plan. A safety horizon of a few seconds, a local safety layer that acts within one control cycle, and a replanner that is allowed to take longer because the local layer covers for it: that is the division of labor of [Figure 1.3](#).

Numbers on a logarithmic axis are not measurements

It is easy to read [Figure 1.5](#) as a promise. It is not. A Python implementation of ORCA for fifty drones may take longer than ten milliseconds; a CBS instance with eight agents in a corridor may not finish in a minute. Treat the figure as a statement of *requirements*: whatever runs in the local safety layer must fit into a control cycle, and whatever cannot must be moved to a slower layer that the local layer can cover for. [Chapter 25](#) explains how to measure computation time honestly, and that is where real numbers belong.

Interfaces between the layers

Read the architecture as a set of interfaces. The global planner ([Chapters 9 and 10](#)) receives the map, the starts, the goals and the swarm constraints and delivers one time-indexed path per drone. The prediction layer ([Chapters 18 to 20](#)) receives noisy observations of the intruder and delivers predicted positions with covariances over the safety horizon. The local layer ([Chapters 13 and 14](#)) receives the nominal path, the teammates' states and the predictions and delivers one velocity command per control cycle. The replanner ([Chapters 4 and 5](#)) receives the current state and the changed map and delivers a new path, which the global planner re-checks ([Chapter 7](#)). The system is assembled in [Chapter 24](#), evaluated in [Chapter 25](#), and is the capstone of week 12 of the study plan in [Appendix A](#).

1.4 A map of the book

The training plan groups the algorithms of a complete autonomous-drone system into six families, labeled A to F, and gives each algorithm a priority: **Essential** for what a working prototype cannot do without, **High** for what you should be able to implement, **Medium** for what you should understand, and **Awareness** for what you should recognize. [Table 1.4](#) lists every algorithm of the plan with its role in one line, its priority and the chapter that teaches it. The priority tags of the plan are collected here, in [Table 1.4](#), and the twelve-week schedule of [Section 1.6](#) tells you when to read each chapter.

Table 1.4. Map of the book: every algorithm of the training plan, its role, its priority in the plan, and the chapter that teaches it. Groups A–C are to be learned deeply, D and E to be known well, and F well enough to integrate.

Algorithm	Role in one line	Priority	Chapter
A. Single-agent graph search (learn deeply)			
Dijkstra	Baseline shortest-path search on graphs with non-negative edge weights; the template that A* refines	High	Chapter 3
A*	The main deterministic shortest-path planner, guided by a heuristic; with time as a state variable it plans one drone around known moving obstacles	Essential	Chapter 4
D* Lite	Efficient replanning when the environment changes after planning; the replanning layer of the architecture	Essential	Chapter 5
LPA*	Conceptual foundation of D* Lite and of incremental search	Medium	Chapter 5
ARA*	Anytime search: a quick suboptimal path, improved while time remains	Medium	Chapter 6
B. Multi-agent path finding (learn deeply)			
Prioritized planning	Plans the agents one after another in a fixed order; fast, but incomplete and suboptimal in general	High	Chapter 8
CBS	High-level conflict resolution over a constraint tree plus low-level per-agent search; optimal	Essential	Chapter 9
ECBS	Bounded-suboptimal variant of CBS based on focal search; much faster for a chosen factor w	Essential	Chapter 10
M* (read <i>M-star</i>)	Couples only the agents that are actually in conflict (subdimensional expansion)	Medium	Chapter 11
Push-and-Swap, Push-and-Rotate	Constructive, rule-based methods; complete on a large class of instances but not optimal; a conceptual contrast to CBS	Awareness	Chapter 11
C. Local and reactive collision avoidance (learn deeply)			
Velocity obstacles (VO)	The set of velocities that lead to a future collision with a moving obstacle	Essential	Chapter 12
Reciprocal VO (RVO)	Extends VO to the case in which both agents share the avoidance effort; removes oscillation	High	Chapter 13
ORCA	Efficient reciprocal avoidance: one half-plane constraint per neighbor and a small linear program	Essential	Chapter 13
Dynamic window approach (DWA)	Chooses a safe short-horizon velocity command under acceleration limits	High	Chapter 14
Artificial potential fields (APF)	Attractive and repulsive forces; simple, but prone to local minima and oscillation	High	Chapter 15
D. Sampling-based motion planning (know well)			

continued on the next page

Table 1.4 (continued)

Algorithm	Role in one line	Priority	Chapter
RRT	Explores continuous configuration spaces quickly by random sampling; probabilistically complete	High	Chapter 16
RRT*	Asymptotically optimal version of RRT, obtained by rewiring the tree	High	Chapter 17
Informed RRT*	Focuses the sampling on an ellipsoid once a first solution is known	Medium	Chapter 17
E. Tracking and prediction (know well)			
Kalman filter	Tracks linear motion under Gaussian noise; the tracker of the prediction layer	Essential	Chapter 18
Extended Kalman filter (EKF)	Handles nonlinear motion and measurement models by linearization	High	Chapter 19
Unscented Kalman filter (UKF)	Nonlinear estimator based on sigma points; needs no Jacobians	Medium	Chapter 19
Particle filter	Nonlinear, non-Gaussian tracking with weighted samples	Medium	Chapter 19
LSTM prediction	Learns temporal motion patterns from recorded trajectories; the plan marks it essential for prediction-based research	Essential	Chapter 20
Transformer prediction	Attention-based alternative for longer horizons and interaction-aware prediction	Awareness-High	Chapter 20
F. Optimization and control (learn enough to integrate)			
Model predictive control (MPC)	Optimizes the controls over a receding horizon; handles constraints naturally	Essential	Chapter 21
Mixed-integer linear programming (MILP)	Exact constrained formulations of scheduling and path problems; can be expensive	Medium	Chapter 22
Consensus, formation control	Maintains relative positions and shared swarm objectives; the plan marks it essential for formation-preserving work	Essential	Chapter 23

Two chapters do not appear in the table because they teach no algorithm of the plan's list. [Chapter 2](#) is the toolbox: graphs, grids, configuration space, space-time states, path costs, drone motion models, the geometry of moving disks, Gaussians and priority queues. [Chapter 7](#) defines the multi-agent path finding problem, its conflicts and its benchmarks, following the standard terminology of Stern et al. [161]; every planner of group B is stated in that language. It does teach two algorithms of its own, the pairwise and the hashed conflict detectors ([Algorithms 7.1](#) and [7.2](#)), which [Chapter 9](#) and the conflict re-check of [Chapter 24](#) both reuse—but the plan counts conflict detection as part of the MAPF problem rather than as one of its twenty-seven algorithms.

How old is all this?

Older than you may think. Dijkstra published his algorithm in 1959 [34] and Kalman his filter in 1960 [76]; A* dates from 1968 [62]. The dynamic window approach appeared in 1997 [46], velocity obstacles and RRT in 1998 [45, 98], D* Lite in 2002 [88], ORCA in 2011 [14] and conflict-based search in 2012, with the journal version in 2015 [150]. The learned predictors are the youngest members: the LSTM cell is from 1997 [64], its use for trajectory prediction from the mid-2010s, and the Transformer from 2017 [169].

1.5 What is established and where research remains

Let us be honest about what is new here and what is not. Every algorithm in Table 1.4 is established: published, proved, implemented many times, and taught. Nobody starting research on drone swarms needs to invent a new shortest-path algorithm or a new Kalman filter, and a thesis that did would have to compete with sixty years of prior work. The training plan says it plainly, and this book repeats it: *the contribution does not need to be a completely new planner. Strong research opportunities remain in how the components are combined, constrained, predicted and evaluated.*

The plan names seven such directions. Each of them is a place where the established algorithms, put together, leave a question open, and each is discussed at length in Chapters 24 and 25 once the algorithms are in place.

Unknown, non-cooperative drones whose future motion is uncertain rather than shared.

Multi-agent planners assume that all routes are known, and reciprocal avoidance assumes that the other agent cooperates; an intruder offers neither. Tracking, prediction and full-responsibility avoidance are established piece by piece (Chapters 13, 18 and 20); how well they work together against real, unpredictable intruders is not.

Combining CBS-style coordination with real-time local avoidance without destroying the global guarantees unnecessarily.

The moment a drone leaves its nominal path, the conflict-freeness that CBS proved no longer holds for it. Which guarantees survive a deviation, for how long, and when a deviation can be reconnected without replanning the swarm, is a question about the combination, not about either algorithm (Chapter 24).

Formation preservation and communication-range constraints during avoidance and replanning.

A swarm that must keep a formation, or stay within radio range of its neighbors, cannot let each drone dodge as it likes. The tools exist (Chapters 21 and 23); trading a constraint violation against a collision risk during a maneuver is open.

Prediction-aware constraints that include uncertainty rather than treating the predicted path as exact.

A prediction is a distribution, not a line. Inflating obstacles by a multiple of the predicted standard deviation is the simplest use of it (Chapters 20 and 24); doing so without making the planner hopelessly conservative at long horizons is not solved.

Deciding automatically when a local avoidance action is sufficient and when a global replan is necessary.

Step 3 versus step 4 of the decision logic. A threshold on the detour length or on the time to collision is a start; a rule that adapts to the situation, and that can be shown to be safe, is a research topic.

Reducing computation for larger swarms while maintaining safety in dynamic 3D environments.

CBS is exponential in the number of conflicts in the worst case, and even ECBS struggles with dozens of agents in tight spaces (Chapters 10 and 11); larger swarms, three dimensions and moving obstacles all make the problem harder.

Benchmarks that compare local-only, replan-only and hybrid approaches under the same conditions.

Most published systems are evaluated on their own scenarios. A shared benchmark with controlled numbers of drones and intruders, controlled prediction quality and a fixed set of metrics would allow honest comparison; Chapter 25 shows how to build one.

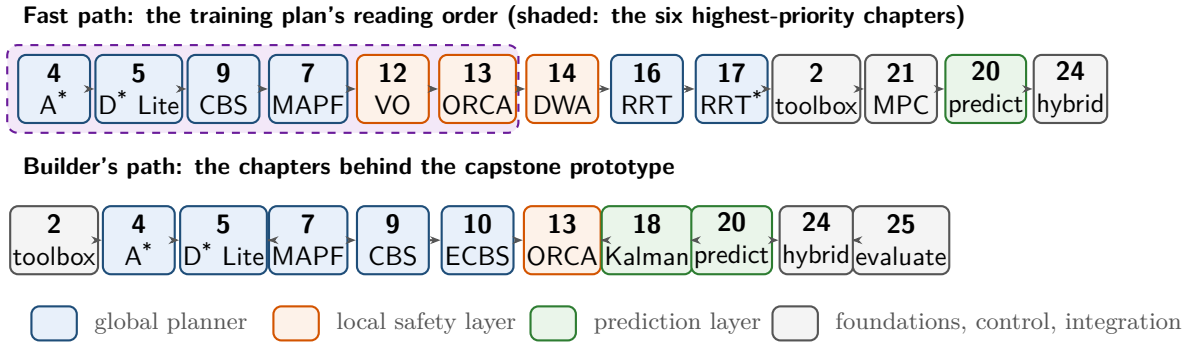


Figure 1.6. Two reading paths. The top row is the reading order of the training plan: its twelve items, with the plan's “planning reference” shown as the book's own toolbox chapter, followed by the capstone; the first six items are the highest priority. The bottom row is the shortest route to a working capstone prototype. Colors mark the layer of the architecture that a chapter belongs to.

You will meet these directions again as *dronebox* notes at the end of many chapters, where each algorithm's role in the combination is spelled out. Keep a list of them; by the time you reach [Chapter 24](#) you will have opinions about every one. The plan also fixes the metrics—collision rate, minimum separation, path length, travel time, makespan, sum of costs, replanning count, computation time, formation error and communication violations—which [Chapter 25](#) defines and measures.

1.6 How to read this book

The chapters are ordered so that each one uses only what came before it, and reading them in order is always safe. But the book was written to accompany a training plan with a schedule, and the plan suggests two shorter routes, drawn in [Figure 1.6](#).

The fast path. The top row of [Figure 1.6](#) is the plan's own order, of which the first six items are the highest priority. Three remarks. The plan lists CBS before the MAPF overview; if you have never seen a multi-agent path finding instance, read [Chapter 7](#) first, it is short and [Chapter 9](#) assumes it. The order also reaches trajectory prediction without the Kalman filter, although [Chapter 20](#) extrapolates with one and takes its input from the tracker of [Chapter 18](#); read [Chapter 18](#) first, or at least its section on predicting ahead. And the “planning reference” of the plan is not a chapter but a book to keep at hand: we recommend LaValle [99] for motion planning and Russell and Norvig [142] for search, and the book's own reference material is the toolbox of [Chapter 2](#) and the mathematical refresher of [Appendix B](#).

The builder's path. If your aim is the capstone prototype as soon as possible, follow the bottom row of [Figure 1.6](#) and read everything else when the prototype demands it.

The twelve-week path. The plan's schedule is reproduced in full, with study notes and a traceability table, in [Appendix A](#); [Table 1.5](#) is the one-page version. Each week has a coding exercise, which reappears as the coding exercise of the corresponding chapter, and a milestone sentence that you should be able to say truthfully at the end of the week.

Table 1.5. The twelve-week schedule of the training plan in one page. Chapters in parentheses are optional in that week. The full plan, with the coding exercises and milestones, is in [Appendix A](#).

Wk	Topic	What you build	Chapters
1	Foundations: Dijkstra and A*	grid A* with time as a state variable	Chapters 3 and 4 (Chapters 1 and 2)
2	Dynamic replanning: LPA*, D* Lite	a path that is repaired, not recomputed	Chapter 5 (Chapter 6)
3	MAPF basics and conflict types	prioritized planning for 2–5 agents	Chapters 7 and 8
4	Conflict-based search	CBS from scratch on a grid	Chapter 9
5	ECBS and practical MAPF	ECBS benchmarked against CBS	Chapter 10 (Chapter 11)
6	VO, RVO and ORCA	2D crossing simulation: none / VO / ORCA	Chapters 12 and 13
7	DWA and potential fields	a local planner and its failure cases	Chapters 14 and 15
8	RRT, RRT* and continuous 3D planning	RRT* in a 3D obstacle field	Chapters 16 and 17
9	Tracking unknown moving obstacles	a Kalman filter tracking a noisy intruder	Chapters 18 and 19
10	Trajectory prediction	constant velocity versus LSTM, ADE/FDE	Chapter 20
11	MPC and formation constraints	a constrained controller keeping a formation	Chapters 21 and 23 (Chapter 22)
12	Hybrid system integration	the capstone and its experiment matrix	Chapters 24 and 25

1.6.1 Conventions, code and tools

Every algorithm chapter has the same fixed sequence of sections: why the algorithm matters for a drone swarm, the idea in plain words, the problem statement, pseudocode with a walkthrough, a worked example with a figure and a trace table, the properties with proofs, variants, implementation notes with a listing, the algorithm’s place in the drone system, a summary, further reading and exercises. Once you know the sequence you can find the proof of optimality or the pitfalls of any algorithm without searching. The conventions of the book, including the colored boxes and the difficulty stars of the exercises, are described in the preface.

Code files. Each chapter has one Python file, `code/chNN_slug.py`, that implements the chapter’s algorithm in full and runs a self-test when executed. The listings in the text are excerpts of these files, never separate code, so what you read is what runs. Every number in a worked example was produced by the corresponding file, and the figures that show algorithm output were generated by scripts in `code/figures/`. The file of this chapter is `code/ch01_scenario.py`, a small scenario generator that needs only the standard library. [Listing 1.1](#) shows the function that found the planned conflict of [Table 1.1](#).

Listing 1.1. Finding the planned conflicts of a set of time-indexed paths (from `code/ch01_scenario.py`). A drone that has reached its goal stays there, which `position_on_path` implements by clamping the time index. The two functions are not adjacent in the file: the `Conflict` record type and the helper `cell_center` sit between them and are omitted here, and the type alias `Cell` is defined earlier, so copy the excerpt from the file rather than from the page.

```
1 def position_on_path(path: list, t: int) -> Cell:
2     """Cell occupied at step ``t``; the drone stays at its goal afterward."""
```



```

3     return path[min(t, len(path) - 1)]
4
5
6 def find_conflicts(paths: list) -> list:
7     """All vertex and edge (swap) conflicts between the given paths."""
8     conflicts = []
9     horizon = max(len(p) for p in paths)
10    for i in range(len(paths)):
11        for j in range(i + 1, len(paths)):
12            for t in range(horizon):
13                a0 = position_on_path(paths[i], t)
14                b0 = position_on_path(paths[j], t)
15                if a0 == b0:
16                    conflicts.append(Conflict(t, "vertex", (i, j), a0))
17                a1 = position_on_path(paths[i], t + 1)
18                b1 = position_on_path(paths[j], t + 1)
19                if a0 == b1 and b0 == a1 and a0 != a1:
20                    conflicts.append(Conflict(t, "edge", (i, j), (a0, a1)))
21    conflicts.sort(key=lambda c: (c.time, c.agents))
22    return conflicts

```

`find_conflicts` compares exact plans and reports facts: a vertex conflict when two drones occupy the same cell at the same step, an edge conflict when they swap cells. Its companion `intruder_encounters` in the same file compares a plan with a *prediction*, the intruder extrapolated at constant velocity, within a radius, and its answer is only as good as the prediction: the asymmetry of [Section 1.1.1](#) in two functions. Running the file executes a self-test (reproducibility of the generator, validity of the scenarios, shortest paths, the conflict rules including stay-at-goal, the intruder motion, a JSON round trip) and prints the scenario of [Example 1.3](#). [Exercise 1.8](#) asks you to write a generator of your own.

The Python stack. The training plan suggests Python, NumPy, NetworkX, Matplotlib and PyBullet or gym-pybullet-drones. The book’s reference implementations use only Python 3.10 or newer and NumPy (SciPy for the quadratic program (QP) and MILP solvers of [Chapters 21](#) and [22](#), plus an optional PyTorch version of the predictor in [Chapter 20](#), which the self-tests never run), so that you can read every line. One line installs everything the exercises need: `python3 -m pip install numpy networkx matplotlib`. NetworkX [\[59\]](#) is useful for quick experiments with graphs and for checking your shortest paths against an independent implementation. Matplotlib draws what the exercises ask you to look at: open and closed sets, trajectories, velocity obstacles, benchmark curves. PyBullet, through the gym-pybullet-drones environment [\[125\]](#), provides a physics simulation of quadrotors for the day when a point-mass model is no longer enough; nothing in the book requires it before [Chapter 24](#). [Chapter 2](#) describes the small conventions that the book’s code shares: how a grid is stored and how a path is represented. The JSON layout written by `to_json` and read by `from_json` in `code/ch01_scenario.py` is the one that the book’s later experiment code reuses.

1.7 Summary

Chapter summary

- The problem: a swarm of controlled drones flies planned, time-indexed routes through a shared space represented on a grid or a graph, while unknown, non-cooperative moving objects appear whose future motion is uncertain.
- Two kinds of trouble: *planned conflicts* between drones of the swarm are facts, visible before take-off and resolved by a multi-agent planner; *unexpected obstacles* are predictions, visible only during the flight and handled by whatever the drone can decide in a control cycle.
- The core idea: a global planner for the routes, a local collision-avoidance method for the surprises, a replanner only when the local response is insufficient.
- The vocabulary and the properties: Table 1.2 holds the eight contrasts that organize the book, Table 1.3 the six properties an algorithm may have, and no algorithm in the book has all six.
- The architecture: global planner (CBS/ECBS), prediction layer (Kalman filter/LSTM), local safety layer (ORCA/DWA), replanning layer (D^* Lite/ A^*), with a five-step decision logic—follow, avoid, reconnect, replan, re-check—and time scales of tens of milliseconds, sub-second and seconds.
- The map: twenty-seven entries in six groups, each with a priority from the training plan and a chapter in this book.
- The honesty: the algorithms are established; the research lies in their combination, their constraints, the use of uncertain predictions, and honest evaluation.

Further reading. For the general background of search and agents, Russell and Norvig [142] is the standard text, and LaValle [99] is the reference for motion planning in configuration space; both are freely available online from their authors. The multi-agent path finding problem, its conflict types and its benchmarks are defined by Stern et al. [161], and conflict-based search by Sharon et al. [150]. Reciprocal local avoidance is introduced by Berg et al. [14], incremental replanning by Koenig and Likhachev [88], and state estimation with the Kalman filter and its relatives by Thrun, Burgard, and Fox [168]. The simulation environment gym-pybullet-drones is described by Panerati et al. [125]. The full training plan, with its schedule, capstone description and completion checklist, is Appendix A.

1.8 Exercises

Exercise 1.1 (★). For each of the following events, say which layer of the architecture of Figure 1.3 handles it (global planner, prediction layer, local safety layer, replanning layer), or whether it is handled by none of them (for example by the flight controller, or by nobody at all). Give a one-line reason each.

- Before take-off, the routes of drones B and D both use cell $(4, 7)$ at $t = 12$.
- A bird appears, predicted to cross the route of drone A in 1.5 seconds.
- The intruder stops and hovers in the only corridor that drone C must use.
- A new no-fly zone is uploaded to the swarm before take-off.
- Drone A has replanned, and its new route now conflicts with the route of drone C .
- The sensor delivers a noisy position of the intruder every 50 ms.
- A gust of wind pushes drone B twenty centimeters off its path.

- (h) Drone D must stay within 30 m of drone B while it dodges the intruder.

Exercise 1.2 (★). Copy the following list of algorithms and, using [Table 1.4](#) and the informal definitions of [Section 1.2.1](#), mark for each one which of the six properties it has: Dijkstra, weighted A^* with $w = 1.5$, prioritized planning, CBS, ECBS, ARA^* , D^* Lite, ORCA, RRT, RRT*. Then name a pair of properties that never appear together in your table and explain why that conflict is structural rather than accidental.

Exercise 1.3 (★★). Sketch, on a grid of at most 8×8 cells, a scenario in which local avoidance alone cannot bring a drone to its goal and the replanning layer must act. Draw the nominal path, the intruder and its motion, and explain in a few sentences why every velocity that the local layer can choose either collides or makes no progress. Then say what the replanner does instead and why step 5 of the decision logic is needed afterward.

Exercise 1.4 (★★). Explain why a swarm that uses *only* a reactive local method, with no global plan, can deadlock. Use two settings: (a) two drones that want to pass each other in a corridor one cell wide, and (b) four drones at the corners of a square, each wanting to reach the opposite corner, all running the same symmetric avoidance rule. What information does a global plan add that a reactive rule lacks? Would giving the reactive rule a random component fix the problem, and at what cost?

Exercise 1.5 (★★). Work with [Table 1.1](#) and [Example 1.3](#).

- Verify from the two paths that A and C have a vertex conflict at $t = 4$ and that they have no edge (swap) conflict.
- Let C wait one step in cell $(3, 4)$ before entering $(3, 3)$. Write down the new path of C , check that the conflict is gone, and compute the new sum of costs and makespan. Would letting A wait instead be cheaper?
- Treat the motion of B as continuous: at time $t \in [0, 9]$ its center is at $(1.5 + t, 5.5)$. Write the relative position of the intruder with respect to B as a function of t , find the time of closest approach and the minimum distance, and compare with the entry 0.38 at $t = 6$ in the table.
- How many steps must B wait at its start for the intruder, if it keeps its velocity, to stay at least one cell away from the center of B 's cell at every integer time step? Which layer of the architecture would make such a decision, and what is wrong with making it before take-off?

Exercise 1.6 (★★). The function `intruder_encounters` of `code/ch01_scenario.py` checks, by default, only the time steps up to the makespan plus one, which is $t = 11$ in [Example 1.3](#). Using $\mathbf{p}(t) = \mathbf{p}_0 + t\mathbf{v}$ with $\mathbf{p}_0 = (11.0, 7.5)$ and $\mathbf{v} = (-0.54, -0.38)$, compute the intruder's predicted position at $t = 14, 15$ and 19 , and its distance to the goal cells of C and A , where those drones are parked. What does the result say about the choice of the horizon, and about the rule that a drone stays at its goal? Which layer should protect a parked drone?

Exercise 1.7 (★). An intruder flies at 5 m/s. (a) How far does it move during one 20 ms control cycle, during a 0.5 s replan, and during a 3 s global plan? (b) Drone A flies toward it at 3 m/s, the required separation is 2 m, and the local layer needs at least ten of the 20 ms control cycles of part (a) to complete an avoidance maneuver. What is the smallest safety horizon τ_h that gives the local layer that time, and at what distance is the risk then detected? (c) Why would a safety horizon of 30 s be useless with a constant-velocity prediction, even though it gives the local layer plenty of time?

Exercise 1.8 (★★★ Coding). Write a scenario generator of your own, without reading `code/ch01_scenario.py` first. Given a grid size, an obstacle density, a number of drones k and a seed, it must return blocked cells, pairwise distinct free start and goal cells, each goal reachable from its start, and one intruder that enters from a border and flies straight across the grid at constant velocity without touching a blocked cell. Then:

- (a) Check your generator with the tests of the self-test in the book's file: the same seed gives the same scenario, different seeds give different ones, the obstacle count is right, every goal is reachable, and the intruder's line is free.
- (b) Add a function that plans a shortest path for every drone independently (breadth-first search is enough on a 4-connected grid) and a function that finds the vertex and edge conflicts among the paths.
- (c) On a 12×8 grid with 15% blocked cells, generate 200 scenarios each for $k = 2, 4$ and 8 drones and report the fraction of scenarios whose independent shortest paths contain at least one planned conflict. What does the trend tell you about the need for a multi-agent planner as swarms grow?
- (d) Extend the generator to several intruders and write the scenario to a JSON file, keeping the layout of `to_json`, so that the same files can drive a later experiment matrix.

The Toolbox: Graphs, Grids, Time, Motion and Uncertainty

Learning objectives

- Define graphs, grid graphs and 3D lattices precisely, including the cost of a diagonal move and the corner-cutting rule.
- Explain why a disk-shaped drone can be planned as a point once the obstacles have been inflated by a Minkowski sum.
- Tell a path, a plan and a trajectory apart; build the time-expanded graph with wait actions; compute path length, travel time, makespan, sum of costs and minimum separation.
- Write down the single- and double-integrator models of a drone with velocity and acceleration limits, and compute its stopping distance.
- Use the geometric primitives of later chapters: distance to a segment, ray–circle intersection, tangents to a circle and the time of closest approach.
- Read a covariance matrix as an ellipse, state Bayes’ rule, and use the lazy-deletion priority queue and the hashed states on which every search in this book relies.

2.1 Why this matters for a drone swarm

Picture eight quadrotors inspecting the racks of a warehouse. A global planner has given each of them a route. Halfway through the mission a forklift-mounted drone that nobody planned for crosses the main aisle. To handle this scene the four layers of [Chapter 24](#) need a shared vocabulary. The global planner needs a *map* it can search: a graph, usually a grid. It also needs to know how *big* a drone is, so that a route through a narrow gap is not a route into a shelf. It needs *time*, because two drones may use the same aisle if they do so at different moments, and *cost measures* to compare routes and whole multi-agent plans.

The local safety layer needs the drone’s *motion limits* (how fast, how hard it can brake) and a small kit of *geometry*: when do two moving disks touch, and which velocities steer clear of an obstacle? The prediction layer describes the intruder with a mean and a covariance, so the planner must understand *uncertainty*. Finally, everything has to run in a few milliseconds, which is a question of *computation*: complexity, priority queues and hashing.

This chapter collects these tools in one place. It is the only chapter with no planning algorithm of its own—its single piece of pseudocode is the priority queue that all the others run on—and it fixes instead the definitions and conventions that every later chapter cites by number. Read it once, then use it as a reference; the drone box of [Section 2.11](#) says which layer uses what.

Every formal object gets a numbered definition, and every number quoted in the text comes from one of the companion files `code/ch02_toolbox.py`, `code/figures/gen_ch02_gaussian.py` and `code/figures/gen_ch02_double_integrator.py`.

2.2 Graphs, grids and lattices

Key idea

Every planner in this book searches a graph. The graph may be explicit (a road map stored in memory), implicit (a grid whose neighbors are computed on demand) or time-expanded (one copy of the map per time step). The search algorithms of Parts II and III do not care which; they only ask a vertex for its successors and their costs.

2.2.1 Graphs, paths and shortest paths

Definition 2.1 (Graph, weighted graph). A **directed graph** $G = (V, E)$ consists of a finite set V of **vertices** (also called *nodes*) and a set $E \subseteq V \times V$ of **edges**; the edge (u, v) leads from u to v . The graph is **undirected** if $(u, v) \in E$ implies $(v, u) \in E$; we then write $\{u, v\}$ for the pair of opposite edges and count it once. A **weighted graph** carries a cost function $c: E \rightarrow [0, \infty)$; $c(u, v)$ is the **edge cost**. In an undirected weighted graph we also require $c(u, v) = c(v, u)$, so that the pair $\{u, v\}$ carries a single cost. The **successors** of u are $\text{Succ}(u) = \{v : (u, v) \in E\}$ and its **predecessors** are $\text{Pred}(u) = \{v : (v, u) \in E\}$; $|\text{Succ}(u)|$ is the out-degree of u , and the largest out-degree in the graph is the **branching factor** b .

Edge costs in this book are non-negative, and almost always positive: a move on a grid costs its length, a wait costs one time step. Dijkstra's algorithm (Chapter 3) needs exactly this non-negativity.

A graph can be stored in three ways. An *adjacency matrix* keeps a $|V| \times |V|$ table of costs; it answers “is there an edge from u to v ?” in constant time but needs $\mathcal{O}(|V|^2)$ memory. An **adjacency list** keeps, for every vertex, the list of its successors with their costs; in Python this is a dictionary such as `{‘a’: [(‘b’, 1.0), (‘c’, 2.5)], ‘b’: [(‘c’, 1.0)], ‘c’: []}`, and it needs $\mathcal{O}(|V| + |E|)$ memory. An *implicit graph* stores nothing at all: a successor function computes $\text{Succ}(u)$ on demand. Grids are implicit graphs, and so are the time-expanded graphs of Section 2.4. All the search code of this book is written against the implicit interface, a function `neighbors(v)` that returns pairs (v', cost) , because an adjacency list can always be wrapped in such a function.

Definition 2.2 (Path, path cost). A **path** from s to γ in G is a sequence $\pi = (v_0, v_1, \dots, v_k)$ of vertices with $v_0 = s$, $v_k = \gamma$ and $(v_{i-1}, v_i) \in E$ for $i = 1, \dots, k$. Its **cost** is the sum of its edge costs,

$$\text{cost}(\pi) = \sum_{i=1}^k c(v_{i-1}, v_i), \quad (2.1)$$

and k is its number of edges. A path is *simple* if no vertex appears twice. The one-vertex sequence (s) is a path from s to s of cost 0.

Definition 2.3 (Shortest path, distance). A **shortest path** from s to γ is a path of minimum cost among all paths from s to γ . Its cost is the **distance** $\text{dist}(s, \gamma) = \min_{\pi} \text{cost}(\pi)$; we set $\text{dist}(s, \gamma) = \infty$ when no path exists, and $\text{dist}(s, s) = 0$. The distance from s to γ is also written $h^*(s)$ when γ is fixed (Chapter 4).

Proposition 2.4 (Triangle inequality). For all vertices u, v, w of a graph with non-negative costs, $\text{dist}(u, w) \leq \text{dist}(u, v) + \text{dist}(v, w)$.

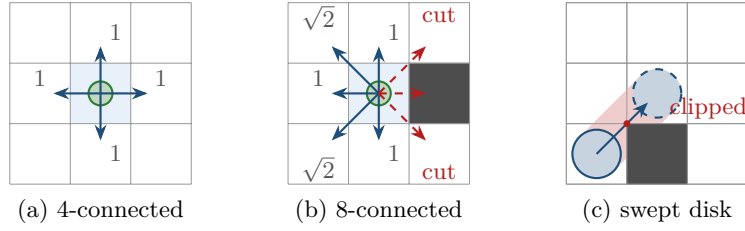


Figure 2.1. Grid connectivity. (a) The four straight moves of G_4 cost 1 each. (b) In G_8 the diagonal moves cost $\sqrt{2}$; the move into the obstacle is forbidden because the cell is blocked, and the two diagonals past its corners are forbidden by the corner-cutting rule. (c) Why: a disk that slides diagonally past an obstacle corner clips it, even though the start and end cells are free.

Proof. If either distance on the right is infinite there is nothing to show. Otherwise take a shortest path from u to v and append a shortest path from v to w . The result is a path from u to w of cost $\text{dist}(u, v) + \text{dist}(v, w)$, and the shortest path from u to w cannot cost more. $\square$

This small fact is why consistent heuristics work (Chapter 4) and why the backward search of Chapter 3 yields an exact heuristic.

2.2.2 Grid graphs

Most maps in this book are **occupancy grids**: the workspace is cut into unit squares, and each square is either free or blocked. A grid is a graph in disguise, as Definition 2.5 makes precise. Our coordinate convention is the mathematical one: x grows to the right, y grows upward, the cell $(0, 0)$ sits in the lower-left corner, and we identify a cell with its center when we measure distances. The ASCII maps used by the code (`.` for free, `#` for blocked) list the *top* row first, so the code flips the row index when it reads them.

Definition 2.5 (Grid graph, 4- and 8-connectivity, corner-cutting rule). A **grid** of width W and height H is the set of **cells** $C = \{(x, y) : x \in \{0, \dots, W - 1\}, y \in \{0, \dots, H - 1\}\}$, together with a set $O \subseteq C$ of **obstacle cells**; the cells in $C \setminus O$ are *free*. The **4-connected grid graph** G_4 has the free cells as vertices and an undirected edge of cost 1 between two free cells that share a side (a *straight move*). The **8-connected grid graph** G_8 has, in addition, an edge of cost $\sqrt{2}$ between two free cells that share only a corner (a *diagonal move*), subject to the **corner-cutting rule**: the diagonal move from (x, y) to $(x + d_x, y + d_y)$ is allowed only if the two cells $(x + d_x, y)$ and $(x, y + d_y)$ it passes between are both free.

Three choices in this definition deserve a comment. First, the diagonal cost is $\sqrt{2}$ and not 1 because $\sqrt{2}$ is the distance between the centers of two diagonal neighbors. With costs 1 and $\sqrt{2}$ the cost of a grid path is exactly the Euclidean length of the polyline through the cell centers, and the straight-line and octile heuristics of Chapter 4 never overestimate it. Second, the corner-cutting rule exists because a drone is not a point: Figure 2.1(c) shows a disk sliding diagonally past an obstacle corner and clipping it. Some code bases forbid the diagonal only when *both* side cells are blocked; that rule is unsafe for a robot of positive size and this book never uses it. Third, Definition 2.5 treats the grid as undirected: every move can be reversed. Later chapters add directed edges only when time enters (Section 2.4).

Which connectivity should you use? Table 2.1 lists the **stretch** of each lattice, the worst-case ratio between the cost of the cheapest lattice path and the straight-line distance in an obstacle-free space. On G_4 the worst direction is the diagonal, where the grid path is $\sqrt{2} \approx 1.41$ times longer than the straight line; G_8 brings the worst case down to 1.08, at 22.5° from an axis, which is why single-agent planners prefer it. The multi-agent benchmarks of Chapter 7, following Stern et al. [161], use G_4 , because a unit cost per move makes “one move per time step” and “one unit of cost per time step” the same thing, which keeps conflicts simple.

Table 2.1. Neighborhoods on grids and lattices. The stretch is the worst-case ratio between the cost of the cheapest lattice path and the straight-line distance in an empty space, attained in the direction that the available moves imitate worst. The four values are printed by `worked_example` in `ch02_toolbox.py` as `stretch2`, `stretch3`, `stretch2_straight` and `stretch3_straight`.

Lattice	Moves	Move costs	Corner-cutting check	Stretch
2D, 4-connected	4	1	none	$\sqrt{2} \approx 1.414$
2D, 8-connected	8	1, $\sqrt{2}$	2 side cells free	1.082
3D, 6-connected	6	1	none	$\sqrt{3} \approx 1.732$
3D, 26-connected	26	1, $\sqrt{2}$, $\sqrt{3}$	2 or 6 box cells free	1.128

2.2.3 Lattices in three dimensions

A drone flies in three dimensions, and the same construction works there.

Definition 2.6 (Lattice in three dimensions, 6- and 26-connectivity). A **lattice** of size $W \times H \times D$ has the cells (x, y, z) with $0 \leq x < W$, $0 \leq y < H$, $0 \leq z < D$, some of which are obstacle cells. A *move* changes each coordinate by -1 , 0 or $+1$ and is not the zero move. The **6-connected lattice** allows the six moves with one non-zero component, at cost 1. The **26-connected lattice** allows all 26 moves; a move with k non-zero components costs $\sqrt{k}$, and the corner-cutting rule requires every cell of the axis-aligned box spanned by the two endpoints to be free. The intermediate *18-connected* lattice allows the moves with at most two non-zero components.

The box rule generalizes the two side cells of the plane: a move with two non-zero components spans a 2×2 box and checks the two other cells in it, one with three spans a $2 \times 2 \times 2$ box and checks six. In an empty 26-connected lattice the cheapest path to a cell at displacement (a, b, c) with $a \geq b \geq c \geq 0$ uses c triple-diagonal, $b - c$ double-diagonal and $a - b$ straight moves, so it costs $(a - b) + \sqrt{2}(b - c) + \sqrt{3}c$; maximizing the ratio of this cost to $\sqrt{a^2 + b^2 + c^2}$ over all directions gives the stretch 1.128 of Table 2.1. The price of 26-connectivity is a branching factor of 26 instead of 6, which multiplies the work of every search by more than four. Chapters 16 and 17 show the alternative for 3D problems: do not discretize at all and sample the free space instead.

2.3 Workspace and configuration space

The map lives in the **workspace** $\mathcal{W} \subseteq \mathbb{R}^2$ or $\mathbb{R}^3$, the physical space in which obstacles occupy a region $\mathcal{O} \subset \mathcal{W}$. A drone is not a point of this space but a body that occupies a region, so planning a route for its center and then flying it would scrape every wall. The classical remedy, due to Lozano-Pérez [109] and presented in every planning textbook [28, 99, 109], is to move the problem into the configuration space.

Definition 2.7 (Configuration space, free space, obstacle region). A **configuration** q of a robot is a tuple of parameters that fixes the position of every point of the robot; the set of all configurations is the **configuration space** $\mathcal{C}$. Write $\mathcal{A}(q) \subset \mathcal{W}$ for the region of the workspace occupied by the robot in configuration q . The **obstacle region** and the **free space** are

$$\mathcal{C}_{\text{obs}} = \{q \in \mathcal{C} : \mathcal{A}(q) \cap \mathcal{O} \neq \emptyset\}, \quad \mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obs}}. \quad (2.2)$$

A path of configurations is *collision-free* if it lies entirely in $\mathcal{C}_{\text{free}}$.

For a disk-shaped robot of radius r whose orientation does not matter, the configuration is the center $\mathbf{p} \in \mathbb{R}^2$, $\mathcal{C} = \mathbb{R}^2$ and $\mathcal{A}(\mathbf{p}) = D(\mathbf{p}, r)$, the closed disk of radius r around $\mathbf{p}$. The

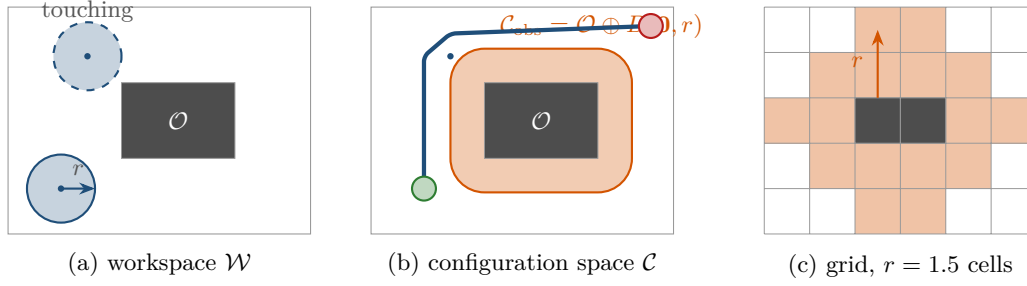


Figure 2.2. Obstacle inflation. (a) In the workspace the disk robot of radius r can at best touch the obstacle. (b) In the configuration space the obstacle grows by r (note the rounded corners) and the robot shrinks to a point; the path that touches the inflated boundary corresponds to the touching disk in (a). (c) On a grid, every cell whose center lies within $r = 1.5$ cells of an obstacle square is blocked: the two-cell obstacle drawn here blocks 16 further cells (the text counts 5, 9 and 21 for one cell at $r = 0.6, 1.0, 1.6$).

obstacle region then has a simple description, which needs one more definition.

Definition 2.8 (Minkowski sum, inflation). The **Minkowski sum** of two sets $A, B \subseteq \mathbb{R}^n$ is $A \oplus B = \{\mathbf{a} + \mathbf{b} : \mathbf{a} \in A, \mathbf{b} \in B\}$. **Inflating** a closed set A by the radius r means forming $A \oplus D(\mathbf{0}, r) = \{\mathbf{x} : \text{dist}(\mathbf{x}, A) \leq r\}$, the set of points within distance r of A . Here $\text{dist}(\mathbf{x}, A) = \inf_{\mathbf{a} \in A} \|\mathbf{x} - \mathbf{a}\|$ is the Euclidean distance from a point to a set; the same symbol dist denotes graph distance in Definition 2.3 and point-to-segment distance in Definition 2.26, and which one is meant is always clear from its arguments.

Proposition 2.9 (Point-robot planning on the inflated map is exact). *For a disk (in 3D: a ball) robot of radius r whose configuration is its center $\mathbf{p}$, and a closed obstacle region $\mathcal{O}$,*

$$\mathcal{C}_{\text{obs}} = \mathcal{O} \oplus D(\mathbf{0}, r). \quad (2.3)$$

Consequently $\mathbf{p} \in \mathcal{C}_{\text{free}}$ if and only if $\text{dist}(\mathbf{p}, \mathcal{O}) > r$, and a path of the center is collision-free for the disk robot if and only if it is collision-free for a point robot on the map whose obstacles are inflated by r .

Proof. The disk $D(\mathbf{p}, r)$ meets $\mathcal{O}$ if and only if there is an obstacle point $\mathbf{o} \in \mathcal{O}$ with $\|\mathbf{p} - \mathbf{o}\| \leq r$, that is, if and only if $\mathbf{p} = \mathbf{o} + \mathbf{b}$ for some $\mathbf{o} \in \mathcal{O}$ and some $\mathbf{b}$ with $\|\mathbf{b}\| \leq r$. The last statement says precisely that $\mathbf{p} \in \mathcal{O} \oplus D(\mathbf{0}, r)$. The second claim applies the first to every point of the path. $\square$

Figure 2.2 shows the construction. Inflating a polygon by a disk rounds its corners, and inflating a union of obstacles inflates each of them. The same idea serves twice more: Chapter 12 turns an encounter between two disks into a point and a disk of radius $r_A + r_B$ (Proposition 2.28), and in Chapter 24 the inflation radius grows with the uncertainty of a predicted position.

The drone as a bounding sphere. A quadrotor with its rotor guards is not a disk, and it rotates about its vertical axis while flying. Modeling it exactly would add the yaw angle to the configuration, giving $\mathcal{C} = \mathbb{R}^3 \times S^1$ and a heading-dependent obstacle region [99]. For a drone that is unnecessary: the body fits into a **bounding sphere** of radius r that is the same in every heading, so the configuration is the position alone, $\mathcal{C} = \mathbb{R}^3$ (or $\mathbb{R}^2$ at constant altitude), and Proposition 2.9 applies with the sphere radius. In this book every drone is such a sphere. Its radius includes a safety margin, as the following pitfall explains.

Inflation on a grid. On a grid the inflation is approximated cell by cell: a cell is added to O if its center lies within r of some obstacle square, which is what `Grid.inflate` does. Blocking a single cell and inflating by $r = 0.6, 1.0$ and 1.6 cell widths blocks 5, 9 and 21 cells respectively: a plus shape, a 3×3 block, and the block with a ring of 12 further cells. The grid version is only a sampled Minkowski sum; a free cell may still contain points that are closer than r to an obstacle. This is harmless for straight moves between cell centers, because obstacle centers and move endpoints both lie on the integer lattice, so for grid-aligned squares the distance to the obstacle along such a move is smallest at one of its endpoints, and the corner-cutting rule guards the diagonal moves.

Inflating by the body radius alone

The radius you inflate by is a promise that the drone's center will never leave the inflated free space. Three things break that promise: the drone is larger than its nominal radius when propellers and guards are counted, the position estimate is off by the localization error, and the controller does not track the plan exactly. Use $r = r_{\text{body}} + \text{margin}$, where the margin covers the localization and tracking errors you expect, and on a grid round the result *up* to the next cell width. A margin that is too small shows up as scraped walls; one that is too large closes narrow passages and makes the planner report that no path exists. [Chapter 25](#) measures the resulting minimum separation.

2.4 Time: space-time states, paths, plans and trajectories

A path says *where* to go; a multi-agent plan must also say *when*. The book handles time in two ways. Search-based planners (Parts II and III) use discrete time: they fix a time step Δt and let every action take exactly one step. Controllers and reactive methods (Parts IV, VI and VII) use continuous time and describe motion by trajectories. This section defines both and the words that connect them.

2.4.1 Discrete time and the time-expanded graph

Definition 2.10 (Space-time state, move and wait actions). Fix a graph $G = (V, E)$ and a time step $\Delta t > 0$. A **space-time state** is a pair (v, t) with $v \in V$ and $t \in \mathbb{N} = \{0, 1, 2, \dots\}$; it means “the agent is at vertex v at time $t \Delta t$ ”. Two kinds of **actions** lead from (v, t) to time $t + 1$: a **move** along an edge $(v, v') \in E$ leads to $(v', t + 1)$, and the **wait action** leads to $(v, t + 1)$. The move costs $c(v, v')$ and the wait costs $c_{\text{wait}} > 0$ for every wait made before the agent's final arrival at its goal; by the stay-at-goal convention of [Definition 2.12](#) the waits at the goal after arrival are free. In the unit-cost problems of [Chapter 7](#), every action costs 1. The wait cost must be positive: a free wait lets a search postpone progress forever on an unbounded horizon.

Space-time states are the reason why a planner can let two agents share a corridor: the states $(v, 3)$ and $(v, 7)$ are different vertices of the graph below, so occupying one does not block the other.

Definition 2.11 (Time-expanded graph). The **time-expanded graph** of G with horizon T_{max} is the directed graph with vertex set $V \times \{0, \dots, T_{\text{max}}\}$ and, for every $t < T_{\text{max}}$, the **wait edges** $((v, t), (v, t + 1))$ for all $v \in V$ and the **move edges** $((u, t), (v, t + 1))$ for all $(u, v) \in E$, weighted as in [Definition 2.10](#).

[Figure 2.3](#) shows a base graph with three vertices and its time-expanded graph with three layers. Note what the construction costs: $|V| (T_{\text{max}} + 1)$ vertices and $T_{\text{max}} (|V| + 2|E|)$ edges for an undirected G ([Exercise 2.4](#)). Nobody stores this graph. The planner keeps G and

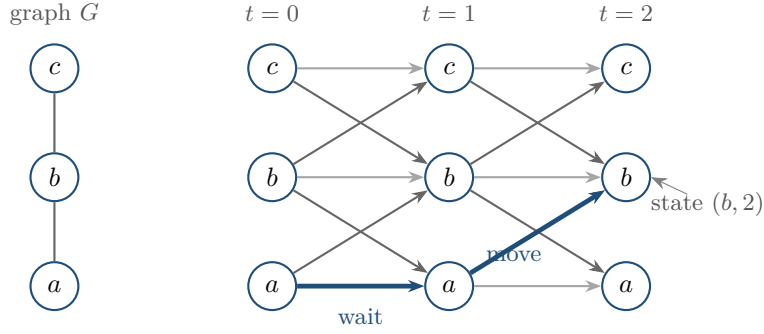


Figure 2.3. A three-vertex graph G (left) and its time-expanded graph with three time layers (right). Gray arrows are wait edges, black arrows are move edges; every edge goes from layer t to layer $t+1$, so the graph is acyclic. The highlighted time-indexed path waits at a for one step and then moves to b , ending in the state $(b, 2)$.

generates the successors of (v, t) on demand with a function like `space_time_successors` in the toolbox, which returns the wait successor and one move successor per free neighbor, all at time $t+1$. Since all edges point forward in time, the time-expanded graph is acyclic; a search in it terminates even without a closed set, though a closed set still saves work.

Definition 2.12 (Time-indexed path, arrival time, stay-at-goal convention). A **time-indexed path** for an agent with start s and goal γ is a sequence $\pi = (v_0, v_1, \dots, v_{T_{\max}})$ with $v_0 = s$, $v_{T_{\max}} = \gamma$ and, for every $t < T_{\max}$, either $v_{t+1} = v_t$ (a wait) or $(v_t, v_{t+1}) \in E$ (a move); the last index $T_{\max}$ is the **horizon** of the stored list. The agent occupies v_t at time t , and by the **stay-at-goal convention** it occupies γ at every time $t > T_{\max}$. The **arrival time cost** $\text{cost}(\pi)$ is the smallest t such that $v_{t'} = \gamma$ for all $t' \geq t$; an agent that passes through γ and leaves again has not arrived. It carries the cost operator because with unit action costs the two agree (Definition 2.16), and because the letter T is reserved for horizons throughout the book. The two indices differ: $\text{cost}(\pi) \leq T_{\max}$, with equality exactly when the path ends without a trailing wait at the goal. Chapter 7 writes the horizon of agent i 's stored path as T_i and the plan horizon as $T = \max_i T_i$.

The stay-at-goal convention matters for multi-agent planning: an agent that has landed still occupies its cell, and a later agent must route around it (Chapters 7 and 8). The arrival time is the individual cost that the multi-agent objectives of Section 2.5 add up. It is computed by `path_time` in the toolbox, which walks backward from the end of the sequence while the vertex equals the goal.

2.4.2 Plans and trajectories

Definition 2.13 (Plan). A **plan** for k agents is a tuple $\Pi = (\pi_1, \dots, \pi_k)$ of time-indexed paths, one per agent, all with the same time step. The position of agent i at time t is v_t^i (Chapter 7 writes the same position as $\pi_i[t]$), extended by the stay-at-goal convention when t exceeds the horizon of π_i . Which pairs of positions count as a *conflict* is a modeling choice fixed in Chapter 7; this chapter only measures how close the agents come.

Definition 2.14 (Trajectory). A **trajectory** is a function $\mathbf{p}: [0, t_f] \rightarrow \mathcal{C}_{\text{free}}$ that gives the position at every continuous time. In this book a trajectory is represented by a sequence of **waypoints** $(t_j, \mathbf{p}_j, \mathbf{v}_j)$ with strictly increasing times and a rule for the motion between them: constant velocity, or the double integrator of Section 2.6. A time-indexed path becomes a trajectory by straight-line motion at constant speed between consecutive vertices,

$$\mathbf{p}((t+s)\Delta t) = (1-s)\mathbf{p}(v_t) + s\mathbf{p}(v_{t+1}), \quad s \in [0, 1], \quad (2.4)$$

Table 2.2. Path, time-indexed path, plan and trajectory. The columns say what each object fixes, which chapters produce it and which chapters consume it.

Object	Fixes	Produced by	Consumed by
Path (Definition 2.2)	where, not when	Chapters 3 to 5 and 16	timing, a tracking controller
Time-indexed path (Definition 2.12)	where and when, in steps of Δt	Chapters 4, 8 and 9	conflict checks, execution
Plan (Definition 2.13)	one time-indexed path per agent	Chapters 8 to 11	execution, the metrics of Chapter 25
Trajectory (Definition 2.14)	position at every time	Chapters 13, 14 and 21	the flight controller

where $\mathbf{p}(v)$ is the position of vertex v in the workspace.

Table 2.2 summarizes the vocabulary; keep the three words apart.

2.5 Cost measures

Definition 2.15 (Path length). The **length** of a path $\pi = (v_0, \dots, v_k)$ whose vertices have workspace positions $\mathbf{p}(v_i)$ is

$$L(\pi) = \sum_{i=1}^k \|\mathbf{p}(v_i) - \mathbf{p}(v_{i-1})\|. \quad (2.5)$$

Waiting adds nothing to the length. On a grid with the costs of Definition 2.5 a path that contains no wait has $L(\pi) = \text{cost}(\pi)$; a time-indexed path with w waits before its arrival has $\text{cost}(\pi) = L(\pi) + w c_{\text{wait}}$, so for unit costs the path cost is exactly the arrival time while $L(\pi)$ counts only the distance flown (in Example 2.20, $L(\pi_B) = 2$ but $\text{cost}(\pi_B) = 3$).

Definition 2.16 (Travel time). The **travel time** of a time-indexed path π is its arrival time $\text{cost}(\pi)$ of Definition 2.12, measured in steps; in seconds it is $\text{cost}(\pi) \Delta t$. It counts moves and waits alike.

With unit move costs, and with the waits at the goal free as in Definition 2.10, the accumulated cost of Equation (2.1) is exactly this travel time: a search that adds up action costs in g returns the arrival time, which is why the single symbol $\text{cost}(\pi)$ serves for both. That is the convention of Stern et al. [161], what Chapter 7 writes as $\text{cost}(\pi_i)$, and what the notation table lists under SoC.

Definition 2.17 (Makespan). The **makespan** of a plan $\Pi = (\pi_1, \dots, \pi_k)$ is the arrival time of the last agent,

$$\text{MS}(\Pi) = \max_{1 \leq i \leq k} \text{cost}(\pi_i). \quad (2.6)$$

Definition 2.18 (Sum of costs). The **sum of costs** of a plan $\Pi = (\pi_1, \dots, \pi_k)$ is the total travel time of all agents,

$$\text{SoC}(\Pi) = \sum_{i=1}^k \text{cost}(\pi_i). \quad (2.7)$$

Later chapters write these two objectives with the macros MS and SoC, exactly as above. They are the two standard objectives of multi-agent path finding [161]; the sum of costs is the

Table 2.3. Trace of the plan of [Example 2.20](#). The separation is evaluated at integer times and, in the last column, at the closest approach during the following step.

t	A	B	$d_{AB}(t)$	closest approach in $[t, t + 1]$	Comment
0	(0, 1)	(1, 2)	$\sqrt{2} \approx 1.414$	1.000 at $t = 1$	B waits
1	(1, 1)	(1, 2)	1.000	0.707 at $t = 1.5$	perpendicular crossing
2	(2, 1)	(1, 1)	1.000	1.000 at $t = 2$	
3	(3, 1)	(1, 0)	$\sqrt{5} \approx 2.236$	—	both have arrived

default in [Chapters 9](#) and [10](#). They do not agree in general: a plan that minimizes one may not minimize the other ([Exercise 2.5](#)).

Definition 2.19 (Separation, minimum separation, collision distance). For two agents with positions $\mathbf{p}_i(t)$ and $\mathbf{p}_j(t)$, the **separation** at time t is $d_{ij}(t) = \|\mathbf{p}_i(t) - \mathbf{p}_j(t)\|$. The **minimum separation** of a plan or of a set of trajectories is $d_{\min} = \min_t \min_{i < j} d_{ij}(t)$ over the whole execution. Two disk (or sphere) agents of radii r_i and r_j **collide** at time t if $d_{ij}(t) < r_i + r_j$; the sum $R_{ij} = r_i + r_j$ is their **collision distance**. The touching case $d_{ij}(t) = R_{ij}$ counts as safe here and in [Chapters 12](#) and [13](#), which test the strict inequality; [Proposition 2.28](#) calls it an overlap only because two closed disks that touch share a boundary point.

For a plan on a graph the separation can be evaluated at the integer times only (the toolbox function `min_separation`) or along the straight-line motion of [Equation \(2.4\)](#) between consecutive steps (`min_separation_continuous`, which uses the closest-approach formula of [Section 2.7](#) on every interval). The two answers differ, as the worked example shows.

Example 2.20 (Two agents on a small grid). Take an empty 4-connected grid with $W = 4$ and $H = 3$. Agent A goes from (0, 1) to (3, 1) along the middle row, agent B from (1, 2) down to (1, 0). If both move without waiting, $\pi_A = ((0, 1), (1, 1), (2, 1), (3, 1))$ and $\pi'_B = ((1, 2), (1, 1), (1, 0))$ put both agents in cell (1, 1) at $t = 1$: the separation is 0. Let B wait one step first, $\pi_B = ((1, 2), (1, 2), (1, 1), (1, 0))$. [Table 2.3](#) traces the plan $\Pi = (\pi_A, \pi_B)$. The lengths are $L(\pi_A) = 3$ and $L(\pi_B) = 2$, the travel times are $\text{cost}(\pi_A) = 3$ and $\text{cost}(\pi_B) = 3$, hence $\text{MS}(\Pi) = 3$ and $\text{SoC}(\Pi) = 6$. At integer times the minimum separation is 1. Between $t = 1$ and $t = 2$, however, A moves from (1, 1) to (2, 1) while B moves from (1, 2) to (1, 1); in the middle of that step, at $t = 1.5$, the agents are at (1.5, 1) and (1, 1.5), only $\sqrt{0.5} \approx 0.707$ apart. With a body radius of 0.3 cells (collision distance 0.6) the plan is safe; with a radius of 0.4 (collision distance 0.8) the two agents collide, since their separation 0.707 drops below the collision distance, although they never share a cell.

A conflict-free plan is not automatically collision-free

The conflict definitions of [Chapter 7](#) look at integer times and at swaps along an edge. [Example 2.20](#) shows a plan without any such conflict in which two agents of radius 0.4 cells still collide halfway through a step, their separation dropping below the collision distance. Grid planning is physically safe only if the agents are small compared with the cell (a collision distance below $\sqrt{0.5} \approx 0.71$ cells for perpendicular crossings), or if you inflate the cells, add a check along the straight-line motion, or forbid such crossings as an extra conflict type. [Chapter 25](#) reports the minimum separation of an execution precisely so that this gap between the model and the physics becomes visible.

2.6 Motion models for a drone

Graph search ignores dynamics: a move is a move, whatever the speed. The local safety layer and the controller cannot afford that, because a drone at 5 m/s needs meters to stop. The two motion models below are both *linear*, which is what makes the Kalman filter of Chapter 18 and the model predictive controller of Chapter 21 tractable.

Definition 2.21 (Single integrator). The **single integrator** (point-mass model) has the state $\mathbf{p} \in \mathbb{R}^n$, $n \in \{2, 3\}$, and the velocity $\mathbf{v} \in \mathbb{R}^n$ as its control input, limited by $\|\mathbf{v}\| \leq v_{\max}$. In continuous time $\dot{\mathbf{p}} = \mathbf{v}$; in discrete time with step Δt ,

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \Delta t \mathbf{v}_k. \quad (2.8)$$

The single integrator assumes that the drone can change its velocity instantly. It is the model behind velocity obstacles and optimal reciprocal collision avoidance (ORCA; Chapters 12 and 13), and implicitly the model of a grid planner, where one cell per step is a velocity of one cell per Δt .

Definition 2.22 (Double integrator). The **double integrator** has the state $\mathbf{x} = (\mathbf{p}, \mathbf{v}) \in \mathbb{R}^{2n}$ and the acceleration $\mathbf{a} \in \mathbb{R}^n$ as its control input, limited by $\|\mathbf{a}\| \leq a_{\max}$; the speed is limited by $\|\mathbf{v}\| \leq v_{\max}$. In continuous time $\dot{\mathbf{p}} = \mathbf{v}$ and $\dot{\mathbf{v}} = \mathbf{a}$. If the acceleration is held constant during each step of length Δt (a *zero-order hold*), the discrete-time model is exactly

$$\mathbf{x}_{k+1} = \mathbf{A} \mathbf{x}_k + \mathbf{B} \mathbf{a}_k, \quad \mathbf{A} = \begin{pmatrix} \mathbf{I} & \Delta t \mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \frac{1}{2} \Delta t^2 \mathbf{I} \\ \Delta t \mathbf{I} \end{pmatrix}, \quad (2.9)$$

where $\mathbf{I}$ is the $n \times n$ identity.

To see where $\mathbf{A}$ and $\mathbf{B}$ come from, integrate a constant acceleration $\mathbf{a}$ over one step: the velocity becomes $\mathbf{v} + \Delta t \mathbf{a}$ and the position, which is the integral of the velocity, becomes $\mathbf{p} + \Delta t \mathbf{v} + \frac{1}{2} \Delta t^2 \mathbf{a}$. The two rows of Equation (2.9) say exactly this. The model is exact, not an approximation; the forward-Euler update $\mathbf{p}_{k+1} = \mathbf{p}_k + \Delta t \mathbf{v}_k$ that many programs use drops the term $\frac{1}{2} \Delta t^2 \mathbf{a}$ and is exact only when the acceleration is zero. The Kalman filter of Chapter 18 uses the same $\mathbf{A}$ with the acceleration replaced by noise (the *constant-velocity model*), and the controller of Chapter 21 uses $\mathbf{A}$ and $\mathbf{B}$ as its prediction model.

Holonomic and non-holonomic. Not every robot can move in every direction; the distinction has a name.

Definition 2.23 (Holonomic and non-holonomic systems). A robot is **holonomic** if every constraint on its motion restricts only the configurations it may occupy, so that from any configuration it can move in every direction of the configuration space. It is **non-holonomic** if some constraint restricts its velocity without restricting the reachable configurations.

A car is the textbook non-holonomic system: it cannot slide sideways, yet it can reach any parking spot by maneuvering. Planning for such systems needs the tools of LaValle [99], which this book does not cover. A quadrotor, at the level of position, is holonomic: it can accelerate in any direction by tilting, and it can hover. That is why a drone can be abstracted as a point mass at all, while a fixed-wing aircraft, which must keep flying forward, cannot.

Why a quadrotor is a double integrator, and when it is not. A quadrotor produces one thrust force along its body axis and controls its direction by tilting. Its center of mass therefore obeys $m\ddot{\mathbf{p}} = \mathbf{f}_{\text{thrust}} - mg\mathbf{e}_z$: given a desired acceleration $\mathbf{a}$, the flight controller solves for the thrust magnitude and the attitude that produce it. Because the attitude loop of a small quadrotor settles in a fraction of a second, the position dynamics seen by a planner that

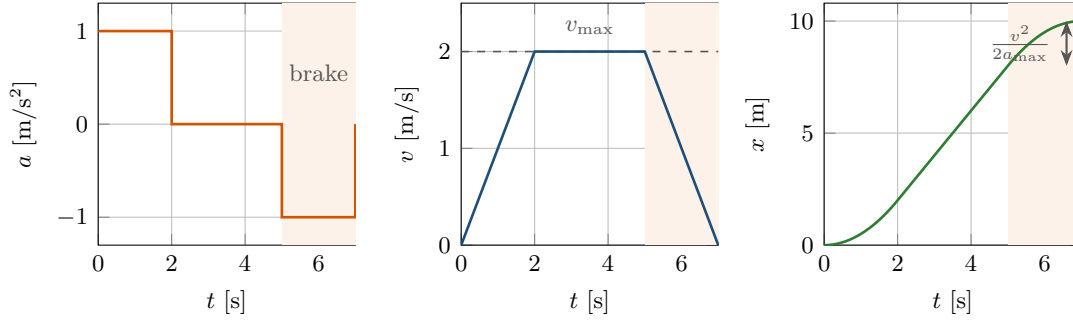


Figure 2.4. A one-dimensional double integrator with $a_{\max} = 1 \text{ m/s}^2$, $v_{\max} = 2 \text{ m/s}$ and $\Delta t = 0.1 \text{ s}$ travels 10 m. It accelerates until it reaches $v_{\max}$ at $t = 2 \text{ s}$, cruises, and starts braking at $t = 5 \text{ s}$ when the remaining distance equals the stopping distance $v_{\max}^2/(2a_{\max}) = 2 \text{ m}$ (shaded). It stops exactly at 10 m at $t = 7 \text{ s}$, because Equation (2.9) is exact for piecewise-constant acceleration.

Table 2.4. Motion models used in the book. The graph model has no dynamics at all; the two integrator models are linear; the full quadrotor model appears only inside a physics simulator.

Model	State	Input	Limits	Used in
Graph move	vertex v	move or wait	one edge per step	Parts II and III
Single integrator	$\mathbf{p}$	$\mathbf{v}$	$\ \mathbf{v}\ \leq v_{\max}$	Chapters 12, 13 and 15 to 17
Double integrator	$(\mathbf{p}, \mathbf{v})$	$\mathbf{a}$	$\ \mathbf{a}\ \leq a_{\max}$, $\ \mathbf{v}\ \leq v_{\max}$	Chapters 14, 18, 21 and 22
Unicycle (ground robot)	$(\mathbf{p}, \theta)$	speed, turn rate	non-holonomic	contrast only
Full quadrotor	$\mathbf{p}, \mathbf{v}$, attitude, rates	thrust, torques	motor limits	simulation only

replans every $\Delta t = 0.1 \text{ s}$ or slower are very nearly the double integrator, with $a_{\max}$ set by the thrust-to-weight ratio and the maximum tilt. The theory of differential flatness makes this precise: any sufficiently smooth position trajectory can be tracked by a quadrotor [117]. The caveats are that the acceleration cannot jump (the attitude must first rotate), and that the limits are not symmetric (gravity helps descending and hinders climbing). Choose $a_{\max}$ and $v_{\max}$ with a margin below the physical limits, and the abstraction holds.

Proposition 2.24 (Stopping distance). *A double integrator moving at speed v that brakes with the maximal deceleration $a_{\max}$ comes to rest after the time $v/a_{\max}$ and the **stopping distance***

$$d_{\text{stop}}(v) = \frac{v^2}{2a_{\max}}. \quad (2.10)$$

Consequently a drone that must be able to stop before an obstacle at distance d straight ahead may not move faster than $\sqrt{2a_{\max}d}$.

Proof. Along the direction of motion the speed is $v(t) = v - a_{\max}t$, which reaches zero at $t_s = v/a_{\max}$. The distance covered is $\int_0^{t_s} v(t) dt = vt_s - \frac{1}{2}a_{\max}t_s^2 = v^2/a_{\max} - v^2/(2a_{\max}) = v^2/(2a_{\max})$, which is Equation (2.10). Solving $d_{\text{stop}}(v) \leq d$ for v gives the bound. $\square$

Figure 2.4 shows the formula at work. The stopping distance is the single most important number of the local safety layer: it sets the safety horizon after which an intruder must trigger avoidance (Chapter 24), defines the admissible velocities of the dynamic window approach (Chapter 14), and tells you how far ahead a prediction must look (Chapter 20). Table 2.4 lists the models side by side.

Clipping breaks linearity

The limits $\|\mathbf{a}\| \leq a_{\max}$ and $\|\mathbf{v}\| \leq v_{\max}$ are not part of the linear model Equation (2.9); they are constraints on top of it. Code that first applies $\mathbf{A}$ and $\mathbf{B}$ and then clips the velocity has silently changed the position update too, because the position moved with the unclipped velocity for half a step. The toolbox function `double_integrator_step` avoids the inconsistency by clipping the input, computing the new velocity, and then advancing the position with the average of the old and the new velocity, which coincides with $\mathbf{Ax} + \mathbf{Bv}$ whenever no limit is active. Model predictive control (Chapter 21) handles the limits properly, as constraints of an optimization problem.

2.7 A geometry toolkit

The local safety layer of the book is built from a handful of geometric primitives. They are collected here with their formulas and proofs; the toolbox implements each of them on plain tuples of floats. The vector algebra they rest on—norms, dot and cross products, projections—is in Appendix B.1.

Definition 2.25 (Vectors, dot product, 2D cross product). For $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ the **dot product** is $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i = \|\mathbf{a}\| \|\mathbf{b}\| \cos \phi$, where ϕ is the angle between the vectors and $\|\mathbf{a}\| = \sqrt{\mathbf{a} \cdot \mathbf{a}}$. The **projection** of $\mathbf{p}$ onto a unit vector $\mathbf{u}$ is $(\mathbf{p} \cdot \mathbf{u}) \mathbf{u}$, and $\mathbf{p} - (\mathbf{p} \cdot \mathbf{u}) \mathbf{u}$ is the part of $\mathbf{p}$ perpendicular to $\mathbf{u}$. In the plane, the **2D cross product** $\mathbf{a} \times \mathbf{b} = a_x b_y - a_y b_x = \|\mathbf{a}\| \|\mathbf{b}\| \sin \phi$ is the signed area of the parallelogram spanned by the two vectors; it is positive when $\mathbf{b}$ lies counterclockwise from $\mathbf{a}$ (a left turn). The vector $\mathbf{a}^\perp = (-a_y, a_x)$ is $\mathbf{a}$ rotated by 90° counterclockwise. In three dimensions $\mathbf{a} \times \mathbf{b}$ is the vector perpendicular to both with length $\|\mathbf{a}\| \|\mathbf{b}\| |\sin \phi|$.

Definition 2.26 (Distance from a point to a segment). Let $\mathbf{a}$ and $\mathbf{b}$ be the endpoints of a segment and $\mathbf{p}$ a point. With

$$s = \min\left(1, \max\left(0, \frac{(\mathbf{p} - \mathbf{a}) \cdot (\mathbf{b} - \mathbf{a})}{\|\mathbf{b} - \mathbf{a}\|^2}\right)\right), \quad \mathbf{q} = \mathbf{a} + s(\mathbf{b} - \mathbf{a}), \quad (2.11)$$

the point $\mathbf{q}$ is the **closest point** of the segment to $\mathbf{p}$ and $\text{dist}(\mathbf{p}, \overline{\mathbf{ab}}) = \|\mathbf{p} - \mathbf{q}\|$. When $\mathbf{a} = \mathbf{b}$ we set $s = 0$.

The unclamped fraction in Equation (2.11) is the projection of $\mathbf{p}$ onto the line through $\mathbf{a}$ and $\mathbf{b}$, expressed as a multiple of $\mathbf{b} - \mathbf{a}$. Clamping it to $[0, 1]$ handles the three cases: the foot of the perpendicular lies inside the segment ($0 < s < 1$), or before $\mathbf{a}$ ($s = 0$), or beyond $\mathbf{b}$ ($s = 1$). This one function is the collision checker of the book: a disk obstacle of radius r blocks a straight motion from $\mathbf{a}$ to $\mathbf{b}$ if and only if the distance from its center to the segment is at most r , and the clearance of a candidate motion in Chapter 14 is the smallest such distance over all obstacles.

Proposition 2.27 (Ray–circle and segment–circle intersection). Let $\mathbf{o} + t\mathbf{d}$, $t \geq 0$, be a ray and $\|\mathbf{x} - \mathbf{c}\| = r$ a circle, and put $\mathbf{f} = \mathbf{o} - \mathbf{c}$. The ray meets the circle where

$$(\mathbf{d} \cdot \mathbf{d}) t^2 + 2(\mathbf{f} \cdot \mathbf{d}) t + (\mathbf{f} \cdot \mathbf{f} - r^2) = 0, \quad (2.12)$$

$$t_{1,2} = \frac{-(\mathbf{f} \cdot \mathbf{d}) \mp \sqrt{\Delta}}{\mathbf{d} \cdot \mathbf{d}}, \quad \Delta = (\mathbf{f} \cdot \mathbf{d})^2 - (\mathbf{d} \cdot \mathbf{d})(\mathbf{f} \cdot \mathbf{f} - r^2).$$

If $\Delta < 0$ the ray misses the circle; otherwise the first contact is at the smaller root t_1 , provided $t_1 \geq 0$. If $\mathbf{f} \cdot \mathbf{f} \leq r^2$ the ray starts inside the disk and the contact time is 0. The segment from $\mathbf{a}$ to $\mathbf{b}$ meets the disk $D(\mathbf{c}, r)$ if and only if $\text{dist}(\mathbf{c}, \overline{\mathbf{ab}}) \leq r$; it crosses the circle at the roots of Equation (2.12) with $\mathbf{o} = \mathbf{a}$, $\mathbf{d} = \mathbf{b} - \mathbf{a}$ that lie in $[0, 1]$.

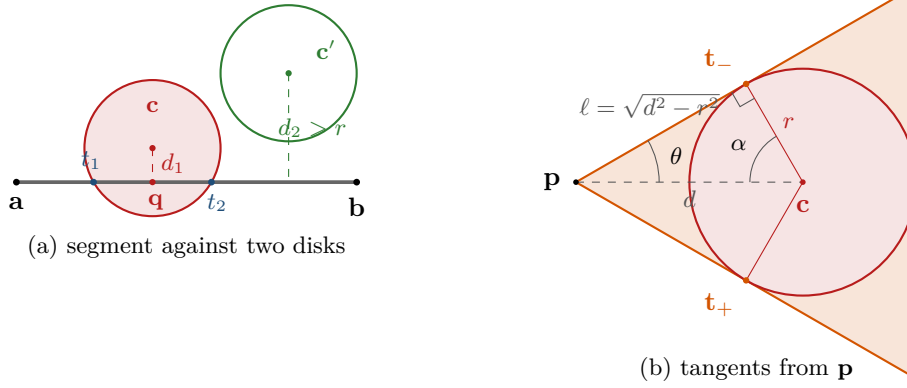


Figure 2.5. (a) The segment $\overline{ab}$ meets the disk around $\mathbf{c}$ because the closest point $\mathbf{q}$ is at the point-to-segment distance $d_1 = \text{dist}(\mathbf{c}, \overline{ab}) \leq r$; the ray $\mathbf{a} + t(\mathbf{b} - \mathbf{a})$ crosses the circle at t_1 and t_2 . The disk around $\mathbf{c}'$ is missed because its distance d_2 to the segment exceeds r . (b) Here $d = \|\mathbf{p} - \mathbf{c}\|$ is the distance to the center, the quantity of Proposition 2.29. The two tangents from $\mathbf{p}$ to a circle touch it at $\mathbf{t}_\pm$; the radius is perpendicular to the tangent, so $\cos \alpha = r/d$ and $\sin \theta = r/d$. Every ray from $\mathbf{p}$ inside the shaded cone hits the disk; this cone is the velocity obstacle of Chapter 12.

Proof. Substitute $\mathbf{x} = \mathbf{o} + t\mathbf{d}$ into $\|\mathbf{x} - \mathbf{c}\|^2 = r^2$ and expand $\|\mathbf{f} + t\mathbf{d}\|^2$; the quadratic formula gives the roots. The segment statement restates the definition of the distance to a segment. $\square$

Proposition 2.28 (Minkowski sum of two disks). $D(\mathbf{c}_1, r_1) \oplus D(\mathbf{c}_2, r_2) = D(\mathbf{c}_1 + \mathbf{c}_2, r_1 + r_2)$. In particular, two disks $D(\mathbf{p}_A, r_A)$ and $D(\mathbf{p}_B, r_B)$ overlap if and only if $\|\mathbf{p}_B - \mathbf{p}_A\| \leq r_A + r_B$, i.e., if and only if the point $\mathbf{p}_B - \mathbf{p}_A$ lies in $D(\mathbf{0}, r_A + r_B)$.

Proof. If $\mathbf{x} = \mathbf{a} + \mathbf{b}$ with $\|\mathbf{a} - \mathbf{c}_1\| \leq r_1$ and $\|\mathbf{b} - \mathbf{c}_2\| \leq r_2$, the triangle inequality gives $\|\mathbf{x} - \mathbf{c}_1 - \mathbf{c}_2\| \leq r_1 + r_2$. Conversely, assume $r_1 + r_2 > 0$ (if both radii vanish the two sides are the single point $\mathbf{c}_1 + \mathbf{c}_2$ and the claim is trivial); if $\mathbf{w} = \mathbf{x} - \mathbf{c}_1 - \mathbf{c}_2$ has $\|\mathbf{w}\| \leq r_1 + r_2$, take $\mathbf{a} = \mathbf{c}_1 + \frac{r_1}{r_1 + r_2}\mathbf{w}$ and $\mathbf{b} = \mathbf{c}_2 + \frac{r_2}{r_1 + r_2}\mathbf{w}$; then $\mathbf{a} + \mathbf{b} = \mathbf{x}$ and both points lie in their disks. The overlap statement is Proposition 2.9 applied with $\mathcal{O} = D(\mathbf{p}_B, r_B)$ and the robot $D(\mathbf{p}_A, r_A)$. $\square$

This is the *relative frame* that Chapters 12 and 13 use throughout: put drone A at the origin as a point, and replace drone B by a disk of radius $r_A + r_B$ at the relative position $\mathbf{p}_B - \mathbf{p}_A$. The next primitive describes the directions in which that disk is visible from the origin.

Proposition 2.29 (Tangent lines from a point to a circle). Let $d = \|\mathbf{p} - \mathbf{c}\| > r$ and $\mathbf{u} = (\mathbf{p} - \mathbf{c})/d$. The two tangents from $\mathbf{p}$ to the circle $\|\mathbf{x} - \mathbf{c}\| = r$ touch it at

$$\mathbf{t}_\pm = \mathbf{c} + r (\cos \alpha \mathbf{u} \pm \sin \alpha \mathbf{u}^\perp), \quad \cos \alpha = \frac{r}{d}, \quad \sin \alpha = \sqrt{1 - \frac{r^2}{d^2}}. \quad (2.13)$$

Both tangents have length $\ell = \sqrt{d^2 - r^2}$ and make the half-angle $\theta = \arcsin(r/d)$ with the line from $\mathbf{p}$ to $\mathbf{c}$; a ray from $\mathbf{p}$ meets the disk if and only if its direction is within θ of $\mathbf{c} - \mathbf{p}$.

Proof. A radius is perpendicular to the tangent at its endpoint, so $\mathbf{p}$, $\mathbf{c}$ and $\mathbf{t}_\pm$ form a right triangle with the right angle at $\mathbf{t}_\pm$, hypotenuse d and legs r and ℓ . Hence $\cos \alpha = r/d$ for the angle α at $\mathbf{c}$, $\ell = \sqrt{d^2 - r^2}$, and $\sin \theta = r/d$ for the angle θ at $\mathbf{p}$. Rotating $\mathbf{u}$ by $\pm \alpha$ and scaling by r gives the two tangent points relative to $\mathbf{c}$. The rays that hit the disk are exactly those between the two tangents. $\square$

Figure 2.5(b) shows the geometry for $\mathbf{p} = (0, 0)$, $\mathbf{c} = (4, 0)$ and $r = 2$: the tangent points are $(3, \pm\sqrt{3})$, the tangent length is $\sqrt{12}$ and the half-angle is 30° (Exercise 2.8). When $\mathbf{p}$ approaches the circle, θ grows toward 90° and the cone opens into a half-plane; when $\mathbf{p}$ is inside, there are no tangents, and the toolbox returns `None`.

Definition 2.30 (Closest approach and time to collision). Let two objects move with constant velocities, $\mathbf{p}_A(t) = \mathbf{p}_A + t\mathbf{v}_A$ and $\mathbf{p}_B(t) = \mathbf{p}_B + t\mathbf{v}_B$, and write

$$\mathbf{p}_{\text{rel}} = \mathbf{p}_B - \mathbf{p}_A, \quad \mathbf{v}_{\text{rel}} = \mathbf{v}_A - \mathbf{v}_B \quad (2.14)$$

for the **relative position** of B with respect to A and the **relative velocity** of A with respect to B . These are the signs that Chapters 12 and 13 use to build the velocity obstacle, and this book uses no others. Their distance is $D(t) = \|\mathbf{p}_{\text{rel}} - t\mathbf{v}_{\text{rel}}\|$. The **time of closest approach** is $t^* = \arg \min_{t \geq 0} D(t)$, the **minimum distance** is $d_{\min} = D(t^*)$, and, for a collision distance R , the **time to collision** is $t_c = \min \{t \geq 0 : D(t) \leq R\}$, or ∞ if no such time exists (the toolbox function `time_to_collision` returns `None` in that case). With a horizon τ the minimization is restricted to $t \in [0, \tau]$.

Proposition 2.31 (Closest approach of two constant-velocity objects). *If $\mathbf{v}_{\text{rel}} \neq \mathbf{0}$ then*

$$t^* = \max\left(0, \frac{\mathbf{p}_{\text{rel}} \cdot \mathbf{v}_{\text{rel}}}{\mathbf{v}_{\text{rel}} \cdot \mathbf{v}_{\text{rel}}}\right), \quad d_{\min} = \|\mathbf{p}_{\text{rel}} - t^*\mathbf{v}_{\text{rel}}\|, \quad (2.15)$$

and when the maximum is not clamped, $d_{\min} = |\mathbf{p}_{\text{rel}} \times \mathbf{v}_{\text{rel}}| / \|\mathbf{v}_{\text{rel}}\|$ in the plane (in 3D, $\|\mathbf{p}_{\text{rel}} \times \mathbf{v}_{\text{rel}}\| / \|\mathbf{v}_{\text{rel}}\|$). The objects come within R of each other if and only if $d_{\min} \leq R$, and then

$$t_c = \max\left(0, t^* - \frac{\sqrt{R^2 - d_{\min}^2}}{\|\mathbf{v}_{\text{rel}}\|}\right). \quad (2.16)$$

If $\mathbf{v}_{\text{rel}} = \mathbf{0}$ the distance is constant, $t^ = 0$ and t_c is 0 if $\|\mathbf{p}_{\text{rel}}\| \leq R$ and ∞ otherwise.*

Proof. $D(t)^2 = \|\mathbf{p}_{\text{rel}}\|^2 - 2t(\mathbf{p}_{\text{rel}} \cdot \mathbf{v}_{\text{rel}}) + t^2\|\mathbf{v}_{\text{rel}}\|^2$ is a parabola in t with positive leading coefficient; its derivative $-2(\mathbf{p}_{\text{rel}} \cdot \mathbf{v}_{\text{rel}}) + 2t\|\mathbf{v}_{\text{rel}}\|^2$ vanishes at $t = (\mathbf{p}_{\text{rel}} \cdot \mathbf{v}_{\text{rel}}) / \|\mathbf{v}_{\text{rel}}\|^2$. If this value is negative, the objects are already separating and the closest approach on $t \geq 0$ is at $t = 0$, which the maximum expresses. At the unclamped minimum, $D^2 = \|\mathbf{p}_{\text{rel}}\|^2 - (\mathbf{p}_{\text{rel}} \cdot \mathbf{v}_{\text{rel}})^2 / \|\mathbf{v}_{\text{rel}}\|^2 = (\|\mathbf{p}_{\text{rel}}\|^2 \|\mathbf{v}_{\text{rel}}\|^2 - (\mathbf{p}_{\text{rel}} \cdot \mathbf{v}_{\text{rel}})^2) / \|\mathbf{v}_{\text{rel}}\|^2 = (\mathbf{p}_{\text{rel}} \times \mathbf{v}_{\text{rel}})^2 / \|\mathbf{v}_{\text{rel}}\|^2$ by Lagrange's identity. For the time to collision, $D(t)^2 = R^2$ is the quadratic of Proposition 2.27 with $\mathbf{o} = \mathbf{p}_{\text{rel}}$, $\mathbf{d} = -\mathbf{v}_{\text{rel}}$, $\mathbf{c} = \mathbf{0}$; it has real roots exactly when $d_{\min} \leq R$, symmetric about t^* at distance $\sqrt{R^2 - d_{\min}^2} / \|\mathbf{v}_{\text{rel}}\|$, and the smaller root is the first contact, clamped at 0 when the disks already overlap. $\square$

Example 2.32 (Two drones on crossing courses). Drone A is at the origin and flies along the x -axis at 1 m/s; drone B is at $(4, 2)$ and flies straight down at 1 m/s. Then $\mathbf{p}_{\text{rel}} = (4, 2)$ and $\mathbf{v}_{\text{rel}} = \mathbf{v}_A - \mathbf{v}_B = (1, 0) - (0, -1) = (1, 1)$. From Equation (2.15), $t^* = (\mathbf{p}_{\text{rel}} \cdot \mathbf{v}_{\text{rel}}) / (\mathbf{v}_{\text{rel}} \cdot \mathbf{v}_{\text{rel}}) = 6/2 = 3$ s and $d_{\min} = |\mathbf{p}_{\text{rel}} \times \mathbf{v}_{\text{rel}}| / \|\mathbf{v}_{\text{rel}}\| = |4 \cdot 1 - 2 \cdot 1| / \sqrt{2} = 2/\sqrt{2} = \sqrt{2} \approx 1.414$ m. If both drones have radius 0.75 m, the collision distance is $R = 1.5 > \sqrt{2}$, and Equation (2.16) gives the first contact at $t_c = 3 - \sqrt{2.25 - 2}/\sqrt{2} \approx 2.646$ s. If both have radius 0.5 m then $R = 1 < \sqrt{2}$ and they pass each other safely. The toolbox functions `time_of_closest_approach` and `time_to_collision` return exactly these numbers; Figure 2.6 draws the situation.

Two remarks close the toolkit. The horizon τ of Definition 2.30 is the form in which the local safety layer of Chapter 24 asks the question. And everything here carries over to three dimensions, except that the cross product becomes a vector (use its norm in Equation (2.15)) and the two tangent lines become a cone with the same half-angle $\theta = \arcsin(r/d)$.

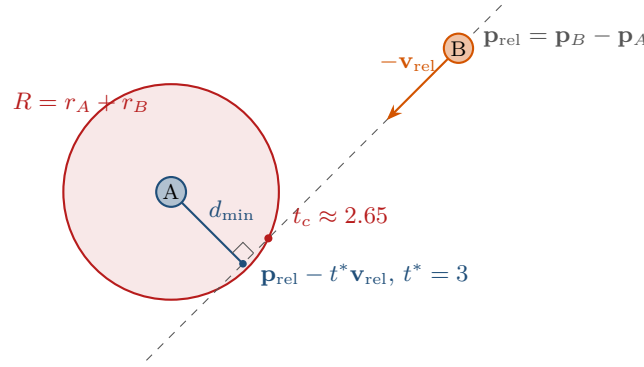


Figure 2.6. Closest approach in the relative frame of Example 2.32. Drone A sits at the origin as a point; drone B starts at $\mathbf{p}_{\text{rel}} = (4, 2)$ and closes on A along $-\mathbf{v}_{\text{rel}} = (-1, -1)$, the relative velocity being $\mathbf{v}_{\text{rel}} = \mathbf{v}_A - \mathbf{v}_B = (1, 1)$. The perpendicular from the origin to the line of motion gives $d_{\min} = \sqrt{2}$ at $t^* = 3$. With collision distance $R = 1.5$ the relative motion enters the disk at $t_c \approx 2.65$.

2.8 Uncertainty: random vectors, Gaussians and Bayes' rule

The tracker of Chapter 18 does not hand the planner a position; it hands over an estimate *and* a covariance that says how much to trust it. Appendix B owns the mathematics of that object for the whole book: Appendix B.3 defines the mean and covariance of a random vector (Definition B.16), proves the affine rule $\text{Cov}(\mathbf{A}\mathbf{x} + \mathbf{b}) = \mathbf{A}\Sigma\mathbf{A}^\top$, which needs no Gaussian assumption (Proposition B.17), gives the multivariate Gaussian density and its Mahalanobis exponent (Definition B.18), and shows that Gaussians survive affine maps and the addition of independent Gaussian noise (Theorem B.19). Those last two closure properties are the whole reason the Kalman filter has a closed form: pushing a Gaussian through the linear motion model of Definition 2.22 and adding noise gives another Gaussian, so the filter has only a mean and a covariance to update (Chapter 18). This section assumes all of that and spends its space on the use Appendix B does not make of it: reading a covariance as a picture of an intruder's uncertainty, the form in which Chapter 20 hands a prediction to Chapter 24. A covariance matrix is hard to read as numbers; it is easy to read as an ellipse.

Definition 2.33 (Covariance ellipse). The $k\sigma$ **covariance ellipse** (in 3D: ellipsoid) of $\mathcal{N}(\mu, \Sigma)$ is the set of points at Mahalanobis distance k from the mean,

$$\mathcal{E}_k = \left\{ \mathbf{x} : (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu) = k^2 \right\}. \quad (2.17)$$

Proposition 2.34 (Axes of the covariance ellipse). Let $\Sigma = \mathbf{E}\mathbf{\Lambda}\mathbf{E}^\top$ be the eigen-decomposition of a 2×2 covariance, with orthonormal eigenvectors $\mathbf{e}_1, \mathbf{e}_2$ as the columns of $\mathbf{E}$ and eigenvalues $\lambda_1 \geq \lambda_2 > 0$. Then $\mathcal{E}_k$ is the ellipse centered at μ with semi-axis $k\sqrt{\lambda_1}$ along $\mathbf{e}_1$ and $k\sqrt{\lambda_2}$ along $\mathbf{e}_2$,

$$\mathbf{x}(\varphi) = \mu + k(\sqrt{\lambda_1} \cos \varphi \mathbf{e}_1 + \sqrt{\lambda_2} \sin \varphi \mathbf{e}_2), \quad \varphi \in [0, 2\pi), \quad (2.18)$$

and the major axis makes the angle $\text{atan2}(e_{1,y}, e_{1,x})$ with the x -axis, with $\mathbf{e}_1$ normalized so that $e_{1,x} \geq 0$ (an eigenvector has no canonical sign, and this is the choice **covariance_ellipse** makes). In two dimensions the probability mass inside $\mathcal{E}_k$ is $1 - e^{-k^2/2}$: 39.3% for $k = 1$, 86.5% for $k = 2$ and 98.9% for $k = 3$.

Proof. Substitute $\mathbf{y} = \mathbf{E}^\top(\mathbf{x} - \mu)$. Since $\mathbf{E}$ is orthogonal, $\Sigma^{-1} = \mathbf{E}\mathbf{\Lambda}^{-1}\mathbf{E}^\top$ and the quadratic form becomes $y_1^2/\lambda_1 + y_2^2/\lambda_2 = k^2$, an axis-aligned ellipse with semi-axes $k\sqrt{\lambda_i}$, which Equation (2.18) parametrizes; $\mathbf{x} = \mu + \mathbf{E}\mathbf{y}$ rotates it back onto the eigenvectors. For the mass, note that $\mathbf{y}$ scaled by $\mathbf{\Lambda}^{-1/2}$ is a standard Gaussian in $\mathbb{R}^2$, whose squared norm is

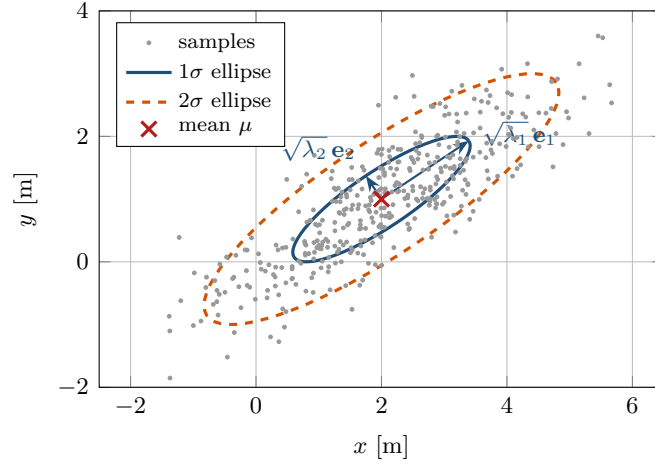


Figure 2.7. Samples from the Gaussian of [Example 2.35](#) with its 1σ and 2σ covariance ellipses. The arrows are the semi-axes $\sqrt{\lambda_i} \mathbf{e}_i$ of the 1σ ellipse; the strong positive correlation shows as the tilt. Only about 39% of the samples lie inside the 1σ ellipse.

χ^2 -distributed with two degrees of freedom ([Theorem B.23](#)), i.e., exponential with mean 2; hence $\mathbb{P}(\text{inside } \mathcal{E}_k) = 1 - e^{-k^2/2}$. $\square$

To draw the ellipse, call `numpy.linalg.eigh` on Σ (its eigenvalues come in ascending order, so reverse them), take the square roots and evaluate [Equation \(2.18\)](#) at, say, 72 angles; the toolbox does this in `covariance_ellipse` and `ellipse_points`.

Example 2.35 (Reading a covariance matrix). Let $\mu = (2, 1)$ and $\Sigma = \begin{pmatrix} 2 & 1.2 \\ 1.2 & 1 \end{pmatrix}$, so $\sigma_x = \sqrt{2}$, $\sigma_y = 1$ and $\rho = 1.2/\sqrt{2} \approx 0.85$. The eigenvalues solve $\lambda^2 - 3\lambda + 0.56 = 0$, giving $\lambda_1 = 2.8$ and $\lambda_2 = 0.2$, with $\mathbf{e}_1 \propto (3, 2)$. The 1σ ellipse therefore has semi-axes $\sqrt{2.8} \approx 1.673$ and $\sqrt{0.2} \approx 0.447$, and its major axis is tilted by $\arctan(2/3) \approx 33.7^\circ$. [Figure 2.7](#) shows 400 samples drawn with a fixed seed: 39.8% of them fall inside the 1σ ellipse and 85.8% inside the 2σ ellipse, close to the theoretical 39.3% and 86.5%.

The 68–95 rule is one-dimensional

In one dimension 68.3% of the mass of a Gaussian lies within 1σ and 95.4% within 2σ . In two dimensions the 1σ ellipse holds only 39.3% and the 2σ ellipse 86.5%; in three dimensions the figures drop to 19.9% and 73.9%. If you inflate a predicted obstacle by “one sigma” to be safe, you are covering less than half of the cases. Choose k from the χ^2 distribution with the right number of degrees of freedom for the confidence you need; [Chapter 24](#) does this when it turns predictions into constraints.

Bayes' rule. Everything in Part VI is an application of one formula, posterior $\propto$ likelihood $\times$ prior, stated as [Equation \(B.21\)](#) and turned into the predict–update recursion of a filter in [Equations \(B.22\)](#) and [\(B.23\)](#). Tracking applies it again and again: today's posterior, pushed through the motion model, becomes tomorrow's prior. With a Gaussian prior, a linear motion model and a linear measurement with Gaussian noise the posterior is again Gaussian and the rule reduces to a few matrix operations, which is the Kalman filter of [Chapter 18](#); [Chapter 19](#) covers what to do when the models are not linear. The standard reference for all of this is Thrun, Burgard, and Fox [168].

2.9 Computation: complexity, priority queues and hashing

Asymptotic notation, the mechanics of a binary heap and the cost model of a hash table are stated once, with their proofs, in [Appendix B.6](#): $f(n) = \mathcal{O}(g(n))$ means $f(n) \leq c g(n)$ beyond some n_0 ([Definition B.37](#)), a binary heap inserts and extracts the minimum in $\mathcal{O}(\log n)$ and peeks at it in $\mathcal{O}(1)$, a hash table inserts, looks up and deletes in constant expected time, and [Table B.3](#) collects the cost of every container the book’s code uses. What a *search* adds is the reading. Its running time is expressed in the sizes of *its* input—the numbers of vertices $|V|$ and edges $|E|$, the branching factor b , the horizon $T_{\max}$, the number of agents k —and which of them a bound is read in matters more than the bound itself: the exponential worst case of [Chapters 9](#) and [10](#) is in k , not in $|V|$. Big-O also hides constants, and in Python they are large: a loop iteration costs about a hundred nanoseconds, while NumPy does the same work per element roughly a hundred times faster, because the loop runs in C. Two of the operations in [Table B.3](#) deserve a closer look, because getting them wrong turns an $\mathcal{O}(n \log n)$ search into an $\mathcal{O}(n^2)$ one: the missing decrease-key, and the hashing of a state.

The lazy-deletion idiom. Dijkstra’s algorithm and A^* ([Chapters 3](#) and [4](#)) must lower the key of a vertex whenever they find a cheaper path to it, and Lifelong Planning A^* (LPA*) and D* Lite ([Chapter 5](#)) must also remove vertices from the queue. Python’s `heapq`, the binary heap of [Appendix B.6](#) over a plain list, offers neither operation, and searching the list for the entry would cost $\mathcal{O}(n)$. The idiom used throughout this book, called **lazy deletion**, avoids both: never touch an entry that is already in the heap; when a key improves, push a *second* entry with the better key; when popping, skip any entry whose key is no longer the item’s best known key. [Algorithm 2.1](#) states the two operations; the class `LazyPQ` of `code/ch02_toolbox.py` implements them.

Algorithm 2.1. Priority queue with lazy deletion

Data: binary heap H of triples $(key, counter, item)$; dictionary *best* mapping items to their best pushed key; counter c

```

1 Function LazyPush(key, item):
2   if item  $\in$  best and best[item]  $\leq$  key then
3     return                                     // no improvement: ignore
4     best[item]  $\leftarrow$  key;  $c \leftarrow c + 1$ 
5     push (key,  $c$ , item) onto  $H$                 // duplicates are allowed
6 Function LazyPop():
7   while  $H$  is not empty do
8     (key,  $\cdot$ , item)  $\leftarrow$  pop the minimum of  $H$ 
9     if item  $\in$  best and best[item] = key then
10      delete best[item]; return (key, item)
11      // otherwise the entry is stale: a better key was pushed later
12 return empty

```

Line 4 of [Algorithm 2.1](#) records the best key of each item, which is what makes a stale entry recognizable at line 8: an entry is stale exactly when its key differs from the recorded best key. Each push costs $\mathcal{O}(\log n)$, each pop costs $\mathcal{O}(\log n)$ plus one extra pop per stale entry it skips; since every entry is popped at most once, the total cost of a search with P pushes is $\mathcal{O}(P \log P)$. A search that relaxes every edge once pushes at most $|E|$ entries, so the heap can hold $|E|$ rather than $|V|$ entries; because $\log |E| \leq 2 \log |V|$, the asymptotic bound of the search is unchanged. Two variants appear in the literature: comparing the popped key with the

Table 2.5. The software stack. Only the first two rows are required to run the code of the book.

Package	Status	Used for
Python ≥ 3.10	required	all code; tuples, dictionaries, <code>heapq</code> , <code>math</code>
NumPy	required	arrays, linear algebra (<code>eigh</code> , <code>solve</code>), seeded random numbers
SciPy (<code>scipy.optimize</code>)	Part VII only	quadratic programs and <code>milp</code> in Chapters 21 and 22
Matplotlib	optional	your own plots and animations; the book’s figures are TikZ
NetworkX [59]	optional	explicit graphs, Laplacians, quick prototypes
PyBullet, gym-pybullet-drones [125]	optional	physics simulation of quadrotors for the capstone

vertex’s current g instead of keeping a separate dictionary, which is equivalent to the version above, and skipping a popped vertex that is already in a closed set, which never re-opens a node while [Algorithm 2.1](#) does. The two coincide whenever the heuristic is consistent, and [Chapter 4](#) is where reopening is analyzed. Where [Algorithm 2.1](#) returns “empty”, `LazyPQ.pop` raises `IndexError`, so a search loop tests `len(pq)` before popping. The toolbox version also counts pushes and stale pops so that you can watch the overhead in the exercises.

Encoding a state. The dictionaries of a search (g -values, parents, the closed set, the best keys of the queue) need hashable keys, and [Appendix B.6](#) explains why floating-point keys are legal but dangerous. The book’s convention follows from that warning: a state is always a tuple of integers, (x, y) for a grid cell, $((x, y), t)$ for a space-time state, (x, y, z) for a lattice cell. Such tuples hash in constant time, compare quickly and cost about a hundred bytes each, and—unlike a rounded float—two physically identical states always produce the same key, so the closed set of [Chapters 3](#) and [4](#) cannot silently stop working. Continuous states, as in [Chapters 16](#) and [17](#), are *not* hashed at all; they live in arrays and are searched with a KD-tree.

Why dictionaries are enough. The largest instances in this book are grids of about 100×100 cells with horizons of about 100 steps, i.e., 10^6 space-time states, and a Python dictionary holding 10^6 tuple keys occupies about 150 MB and is filled in a few seconds. A search touches far fewer states than that, so no chapter uses a specialized data structure for its closed set; [Chapter 25](#) is where faster encodings are discussed.

2.10 The Python stack and the code of the book

The training plan behind this book ([Appendix A](#)) prescribes a lean stack, listed in [Table 2.5](#). The code is one file per chapter, `Overleaf/code/chNN_slug.py` (`ch02_toolbox.py` here, `ch04_astar.py` for A^*), self-contained apart from NumPy and the occasional import from an earlier chapter, and ending in an `if __name__ == "__main__":` block that runs a self-test of `assert` statements in a few seconds; every number quoted in a worked example comes from running it.

[Listing 2.1](#) shows the successor function of the toolbox grid, the one function that the searches of [Chapters 3](#) to [6](#) call most often; note the corner-cutting test and the two edge costs of [Definition 2.5](#). In the same file `tangent_points` transcribes [Equation \(2.13\)](#) of [Proposition 2.29](#) line for line, `time_of_closest_approach` transcribes [Equation \(2.15\)](#) and

`time_to_collision` is a one-line call to `ray_circle_intersection` in the relative frame. The class `LazyPQ` in `code/ch02_toolbox.py` implements [Algorithm 2.1](#) line for line, and counts its pushes and stale pops.

Listing 2.1. The successor function of the grid (from `code/ch02_toolbox.py`).

```

1  def neighbors(self, cell):
2      """Return [(neighbor, cost)] for the free neighbors of ``cell``."""
3      x, y = cell
4      moves = MOVES4 if self.connectivity == 4 else MOVES8
5      result = []
6      for dx, dy in moves:
7          nxt = (x + dx, y + dy)
8          if not self.is_free(nxt):
9              continue
10         if dx != 0 and dy != 0:                # diagonal move
11             if not (self.is_free((x + dx, y))
12                     and self.is_free((x, y + dy))):
13                 continue                # corner cutting
14             result.append((nxt, SQRT2))
15         else:
16             result.append((nxt, 1.0))
17     return result

```

2.11 Where this fits in the drone system

In the drone system

Each of the four layers of the hybrid architecture in [Chapter 24](#) is built from tools of this chapter. The *global planner* searches a grid or lattice ([Definitions 2.5](#) and [2.6](#)) of inflated obstacles ([Proposition 2.9](#)) in space-time ([Definition 2.10](#)), compares plans by makespan and sum of costs ([Definitions 2.17](#) and [2.18](#)), and runs on the lazy priority queue ([Algorithm 2.1](#)). The *prediction layer* describes an intruder by a Gaussian ([Definition B.18](#)), read as the covariance ellipse of [Definition 2.33](#) and propagated through the double-integrator model ([Definition 2.22](#)) with Bayes' rule ([Equation \(B.21\)](#)). The *local safety layer* works in the relative frame of two disks ([Proposition 2.28](#)), decides with the time to collision ([Proposition 2.31](#)) whether an intruder is inside the safety horizon, avoids it with the tangent cone ([Proposition 2.29](#)), and respects the stopping distance ([Proposition 2.24](#)). The *replanning layer* re-searches the grid from the current space-time state with hashed states. In the twelve-week study plan of [Appendix A](#) this chapter accompanies Week 1, the foundations week, and is the reference to return to in every later week.

2.12 Summary

Chapter summary

- A planner searches a weighted graph through a successor function; grids and lattices are implicit graphs with move costs 1, $\sqrt{2}$, $\sqrt{3}$ and a corner-cutting rule ([Definitions 2.1](#), [2.5](#) and [2.6](#)).
- A disk or ball robot becomes a point when the obstacles are inflated by its radius: $\mathcal{C}_{\text{obs}} = \mathcal{O} \oplus D(\mathbf{0}, r)$ ([Proposition 2.9](#)); a drone is such a ball, radius plus margin.

- Discrete time adds the wait action and turns the map into the time-expanded graph; a time-indexed path fixes where and when, a plan is one such path per agent, a trajectory is continuous motion (Definitions 2.12 to 2.14).
- Path length, travel time, makespan $MS = \max_i T_i$, sum of costs $SoC = \sum_i T_i$ and minimum separation are the cost measures of the book; a plan without grid conflicts can still bring physical agents too close.
- The single integrator chooses velocities, the double integrator chooses accelerations with exact discrete matrices $\mathbf{A}, \mathbf{B}$; a quadrotor is nearly the latter; the stopping distance is $v^2/(2a_{\max})$.
- Distance to a segment, ray-circle intersection, the Minkowski disk, tangent lines and the time of closest approach $t^* = (\mathbf{p}_{\text{rel}} \cdot \mathbf{v}_{\text{rel}}) / \|\mathbf{v}_{\text{rel}}\|^2$ with $d_{\min} = |\mathbf{p}_{\text{rel}} \times \mathbf{v}_{\text{rel}}| / \|\mathbf{v}_{\text{rel}}\|$ are the geometry of collision avoidance.
- A covariance is an ellipse with axes $\sqrt{\lambda_i} \mathbf{e}_i$ (Proposition 2.34); in 2D the 1σ ellipse holds only 39% of the mass; Bayes' rule turns a prior and a measurement into a posterior.
- Searches use a binary heap with lazy deletion (push duplicates, skip stale pops) and dictionaries keyed by integer tuples; Python and NumPy are enough for every instance in this book.

Further reading. Graphs, shortest paths, heaps and big-O notation are covered in Cormen et al. [31], and the search view of planning in Russell and Norvig [142]. The configuration-space construction and the inflation of Proposition 2.9 go back to Lozano-Pérez [109]. Configuration space, Minkowski sums, holonomic and non-holonomic systems and motion models are the subject of LaValle [99] and Choset et al. [28]; the latter also contains the geometry of distance computations. The definitions of time-indexed paths, makespan and sum of costs follow Stern et al. [161]. The velocity-obstacle construction that motivates the tangent-line and closest-approach primitives is due to Fiorini and Shiller [45]. Gaussians, covariance ellipses and Bayes' rule in the form used by tracking are treated in Thrun, Burgard, and Fox [168]. For the quadrotor as a double integrator and differential flatness see Mellinger and Kumar [117]; for the simulation stack see Panerati et al. [125].

2.13 Exercises

Exercise 2.1 (★). Write the adjacency list of the 8-connected grid graph of the 4×3 map `["....", ".#..", "...."]` for the cells $(0,0)$, $(2,1)$ and $(3,0)$, including edge costs. Then show that an empty $W \times H$ grid has $W(H-1) + H(W-1)$ undirected edges when 4-connected and $2(W-1)(H-1)$ more when 8-connected; check the totals 17 and 29 for $W = 4$, $H = 3$ against the self-test of the toolbox.

Exercise 2.2 (★). Suppose a diagonal move cost 1 instead of $\sqrt{2}$. Give a start and goal on an empty 8-connected grid for which the cheapest grid path is then not the shortest polyline through cell centers, and explain what this would do to a heuristic that estimates the straight-line distance (Chapter 4).

Exercise 2.3 (★★). A single cell of a large grid is blocked. Using the cell-center rule of `Grid.inflate`, determine by hand which cells are blocked after inflating by $r = 0.6$, $r = 1.0$ and $r = 1.6$, and count them in each case. At which values of r does the count change next? Compare the blocked-cell count with the area $1 + 4r + \pi r^2$ of the exact Minkowski sum of the unit square with the disk of radius r , and check that the ratio tends to 1.

Exercise 2.4 (★★★). Let $G = (V, E)$ be undirected. Show that its time-expanded graph with horizon $T_{\max}$ has $|V|(T_{\max} + 1)$ vertices and $T_{\max}(|V| + 2|E|)$ edges, that it is acyclic, and that its paths from $(s, 0)$ to $(\gamma, T_{\max})$ correspond one-to-one to the time-indexed paths of [Definition 2.12](#) with horizon $T_{\max}$.

Exercise 2.5 (★★★). Compute the makespan and the sum of costs of the plan in [Example 2.20](#). Then construct an instance with three agents on a corridor in which the plan with the smallest makespan does not have the smallest sum of costs, and vice versa, and prove that no plan is optimal for both objectives.

Exercise 2.6 (★★). A drone flies at $v = 2$ m/s with $a_{\max} = 1$ m/s² and $\Delta t = 0.1$ s. How many steps of full braking does it need, and how far does it travel according to the exact model [Equation \(2.9\)](#)? Repeat with the forward-Euler update $\mathbf{p}_{k+1} = \mathbf{p}_k + \Delta t \mathbf{v}_k$, $\mathbf{v}_{k+1} = \mathbf{v}_k + \Delta t \mathbf{a}_k$ and explain the difference. Finally, write the rule “a velocity v is admissible if the drone can still stop before the nearest obstacle at distance d ” as an inequality; this rule returns in [Chapter 14](#).

Exercise 2.7 (★★). Derive [Equation \(2.15\)](#) by minimizing $D(t)^2$, show that $d_{\min} = |\mathbf{p}_{\text{rel}} \times \mathbf{v}_{\text{rel}}| / \|\mathbf{v}_{\text{rel}}\|$ when the minimum is not clamped, and derive the time to collision [Equation \(2.16\)](#). Evaluate all three for $\mathbf{p}_{\text{rel}} = (4, 2)$, $\mathbf{v}_{\text{rel}} = (1, 1)$ and collision distances $R = 1.5$ and $R = 1$.

Exercise 2.8 (★★). For $\mathbf{p} = (0, 0)$, $\mathbf{c} = (4, 0)$ and $r = 2$ compute the tangent points, the tangent length and the half-angle θ of the cone from [Proposition 2.29](#). Show in general that $\theta = \arcsin(r/d)$, and describe what happens to the cone as $d \rightarrow r^+$ and as $d \rightarrow \infty$. Why does this matter for a drone that is close to another drone?

Exercise 2.9 (★★). For $\Sigma = \begin{pmatrix} 2 & 1.2 \\ 1.2 & 1 \end{pmatrix}$ compute the eigenvalues, the eigenvectors and the orientation of the major axis by hand, and give the semi-axes of the 1σ and 2σ ellipses. Then compute the probability mass inside each ellipse and compare it with the one-dimensional 68% and 95% figures.

Exercise 2.10 (★★★ Coding). Extend the toolbox to three dimensions and measure the priority queue. (a) Write a class `Grid3D` with 6- and 26-connectivity, the move costs of [Definition 2.6](#), the box version of the corner-cutting rule and an `inflate` method that samples the Minkowski sum with a ball. (b) Implement `segment_sphere_intersects` and the 3D versions of `time_of_closest_approach` and `time_to_collision`, and test them with `asserts` against hand-computed values, as the self-test of `ch02_toolbox.py` does. (c) Using `space_time_successors` and `LazyPQ`, write a uniform-cost search over space-time states on a 2D grid ([Chapter 3](#) explains why it is correct), run it on a 50×50 map with random obstacles, and report the pushes and stale pops; explain why the stale pops never exceed the pushes.

Part II

Single-Agent Graph Search

Dijkstra's Algorithm

Learning objectives

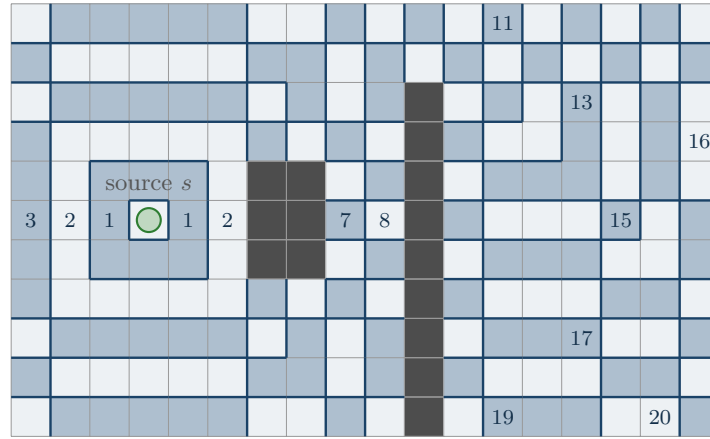
- Explain what Dijkstra's algorithm computes (the cheapest cost from one source to every vertex of a graph with non-negative edge costs) and when it is the right tool.
- Trace it by hand on a small weighted graph and on a grid, keeping track of the priority queue, the settled set and the parent pointers.
- Prove the settled-node invariant, explain why it needs non-negative edge costs, and derive the running time $\mathcal{O}((|V| + |E|) \log |V|)$.
- Implement it in Python with `heapq` and lazy deletion, on explicit graphs and on occupancy grids, with the early exit (uniform-cost search) and path reconstruction.
- Use one backward run from the goal as an exact heuristic, the table that feeds A^* (Chapter 4) and the single-agent searches inside the multi-agent solvers of Chapters 8 and 9.

3.1 Why this matters for a drone swarm

Picture one delivery drone at a depot on the edge of a city. Its map is the occupancy grid of Chapter 2: cells are free or blocked, and moving into a cell costs one unit of time, or more when the cell lies in a wind-exposed corridor or above a schoolyard where the operator has attached a risk penalty. No-fly zones are simply removed from the grid. The customer is twenty blocks away. Before the drone lifts off, the planner has to answer one question: which route is the cheapest, and how do we know that no cheaper route exists?

This is the **single-source shortest-path problem** on a graph with non-negative edge costs, and Dijkstra's algorithm [34] solves it exactly. It is the oldest algorithm this book studies in detail, and the one you will use most often, for three reasons.

1. *It is the baseline planner.* Every other single-agent planner in Chapters 4 to 6 returns a path whose cost is judged against the cost that Dijkstra's algorithm would have found. When the experiments of Chapter 25 report a path length "relative to the optimum", the optimum comes from this chapter.
2. *It is the heuristic factory.* A^* (Chapter 4) needs, for every vertex, a lower bound on the cost that remains to the goal. One run of Dijkstra's algorithm *backward* from the goal produces the exact cost-to-go for every vertex of the map. That table is the perfect heuristic for A^* , and it is the heuristic that the single-agent searches inside prioritized planning (Chapter 8) and conflict-based search (Chapter 9) use when they route one drone at a time through space and time.
3. *It is the ancestor.* A^* is Dijkstra's algorithm with a heuristic added to the priority;



numbers: $\lfloor g \rfloor$, the wave settles band k before band $k + 1$

Figure 3.1. Dijkstra's algorithm as a spreading wave on an 8-connected grid (diagonal moves cost $\sqrt{2}$). Cells are shaded by the band $\lfloor g \rfloor$ of their cost from the source; the wave settles every cell of band k before any cell of band $k + 1$, bends around the block and passes through the gap at the top of the wall. Behind the wall the bands are centered on the gap, not on the source: cost is measured along paths, not as the crow flies.

uniform-cost search is Dijkstra's algorithm with an early exit; D^* Lite ([Chapter 5](#)) is a backward search from the goal that repairs its distance table when the map changes. Understand this chapter thoroughly and the next three chapters become variations on one theme.

In the four-layer hybrid architecture of [Chapter 24](#), Dijkstra's algorithm therefore lives in the global planner: it plans a single drone on a static map, and it supplies the distance tables that the multi-agent solver and the replanning layer consume. It knows nothing about other drones or about moving intruders; those are the subjects of Parts III and IV.

Roadmap. [Section 3.2](#) gives the picture of a wave spreading over the map. [Section 3.3](#) fixes the notation, [Section 3.4](#) states the algorithm with a binary heap and lazy deletion, and [Section 3.5](#) traces it on a seven-vertex graph and on a 5×5 grid. [Section 3.6](#) proves that the algorithm is correct, shows what goes wrong with a negative edge, and derives the running time. [Section 3.7](#) collects the variants you will meet later in the book, above all the backward run that produces exact heuristics. [Section 3.8](#) discusses the Python implementation and its pitfalls, and [Section 3.9](#) places the algorithm in the drone system.

3.2 The idea in plain words

Drop a stone into a pond. The ripple spreads at constant speed, reaches nearby points before distant ones, bends around a rock and squeezes through a gap in a wall. If you write down the moment at which the ripple first reaches each point, you have written down the distance from the stone to every point of the pond, because the ripple always arrives along a shortest route. [Figure 3.1](#) shows this picture on a grid map. The bands are the cells whose cost from the source lies between k and $k + 1$, and band k is completely settled before the wave enters band $k + 1$.

Dijkstra's algorithm simulates this wave on a graph, one vertex at a time. Every vertex is in one of three states.

- *Unseen*: no path to it has been found; its tentative cost is $g(v) = \infty$.

- *Open*: at least one path to it is known, with tentative cost $g(v)$, but a cheaper path may still turn up. The open vertices form the frontier of the wave and are stored in a priority queue keyed by g .
- *Settled* (or *closed*): the wave has reached it, and $g(v)$ is final.

The loop is the same at every step: take the open vertex with the smallest tentative cost, declare it settled, and offer each of its neighbors the path that goes through it. If that path is cheaper than the neighbor's current tentative cost, update the cost and the parent pointer of the neighbor and put it on the frontier. This update is called **relaxation**.

Key idea

Always settle the open vertex with the smallest tentative cost. Because no edge has a negative cost, every path that is still to be discovered starts from a vertex that is at least as expensive, so nothing found later can undercut it: the cost of a vertex is final the moment it is settled.

Two consequences follow, and both are worth fixing in your mind now. First, vertices are settled in non-decreasing order of cost; that is the wave. Second, if you only want the cost of one target γ , you may stop the moment γ is settled: everything settled before γ was at least as cheap, everything still open is at least as expensive, and the answer cannot change any more. With this early exit the algorithm is what the artificial-intelligence literature calls **uniform-cost search** [142].

3.3 Problem statement and notation

We use the graphs of Chapter 2: an explicit graph is a set of vertices with adjacency lists, and a grid is an implicit graph whose neighbors are generated on demand by a successor function.

Definition 3.1 (Weighted graph, path, path cost). A **weighted directed graph** is a triple $G = (V, E, c)$ with a finite vertex set V , an edge set $E \subseteq V \times V$ and a cost function $c: E \rightarrow \mathbb{R}_{\geq 0}$. We write $c(u, v)$ for the cost of the edge (u, v) and $\text{Succ}(u) = \{v : (u, v) \in E\}$ for the successors of u . An undirected edge $\{u, v\}$ of cost w stands for the two directed edges (u, v) and (v, u) , both of cost w . A **path** from u to v is a sequence $\pi = (v_0, v_1, \dots, v_k)$ with $v_0 = u$, $v_k = v$ and $(v_{i-1}, v_i) \in E$ for $i = 1, \dots, k$; its cost is $c(\pi) = \sum_{i=1}^k c(v_{i-1}, v_i)$.

Definition 3.2 (Shortest-path distance). The **shortest-path distance** $\text{dist}(u, v)$ is the smallest cost of any path from u to v , and $\text{dist}(u, v) = \infty$ if no such path exists. A path with $c(\pi) = \text{dist}(u, v)$ is a **shortest path**; there may be several. We use the book-wide dist of Chapter 2 throughout; CLRS [31] writes $\delta(u, v)$ for the same quantity, so keep that in mind when you read the standard proofs alongside this chapter.

Three facts about dist are used throughout the chapter. For all $u, v, w \in V$,

$$\begin{aligned} \text{dist}(u, u) &= 0, & \text{dist}(u, w) &\leq \text{dist}(u, v) + \text{dist}(v, w), \\ \text{dist}(u, v) &\leq \text{dist}(u, w) + c(w, v) & \text{if } (w, v) \in E. \end{aligned} \tag{3.1}$$

The second is the triangle inequality: going via v can only be a detour. The third is the triangle inequality with a single edge as the second leg. A fourth fact is that *subpaths of shortest paths are shortest paths*: if π is a shortest path from u to v and w lies on π , then the prefix of π up to w is a shortest path from u to w , because a cheaper prefix would give a cheaper π .

Definition 3.3 (Single-source shortest-path problem). Given $G = (V, E, c)$ with $c \geq 0$ and a source $s \in V$, compute $\text{dist}(s, v)$ for every $v \in V$ together with a **shortest-path tree**: a

parent pointer $\text{parent}(v)$ for every reachable $v \neq s$ such that following the pointers from v back to s traces a shortest path. In the **single-pair** variant a target γ is also given, and only $\text{dist}(s, \gamma)$ and one shortest path from s to γ are required.

During the search every vertex carries a **tentative cost** $g(v)$, the cost of the best path to v found so far, so that $g(v) \geq \text{dist}(s, v)$ at all times. The open vertices form the priority queue OPEN and the settled vertices the set CLOSED. Both are new here; from Chapter 2 we reuse the implicit successor interface Succ and the lazy-deletion idiom of its priority queue LazyPQ (Algorithm 2.1), in the closed-set form: since $h = 0$ here, the two forms settle exactly the same vertices. In later chapters g is joined by a heuristic h ; in this chapter $h = 0$ and the queue is ordered by g alone.

3.4 The algorithm

Algorithm 3.1 is Dijkstra's algorithm in the form that every search algorithm of this book reuses. A binary min-heap holds pairs (g, v) , and instead of decreasing the key of a vertex that is already in the heap, we push a second entry and ignore the older one when it surfaces. This is the **lazy deletion** idiom of Chapter 2 (Algorithm 2.1). An optional target γ turns the algorithm into uniform-cost search.

Algorithm 3.1. Dijkstra's algorithm with a binary heap and lazy deletion. Without a target it computes $\text{dist}(s, v)$ for every vertex; with a target γ it stops as soon as γ is settled (uniform-cost search).

Input: graph $G = (V, E, c)$ with $c \geq 0$, given by a successor function Succ; source s ; optional target γ

Output: tentative costs g , exact for every settled vertex; parent pointers

```

1 Function Dijkstra( $G, s, \gamma$ ):
2    $g(v) \leftarrow \infty$  for all  $v \in V$ ;  $g(s) \leftarrow 0$ ;  $\text{parent}(s) \leftarrow \text{nil}$ 
3   OPEN  $\leftarrow$  min-heap holding the single entry  $(0, s)$ ; CLOSED  $\leftarrow \emptyset$ 
4   while OPEN  $\neq \emptyset$  do
5      $(g, u) \leftarrow$  pop the entry with the smallest  $g$  from OPEN
6     if  $u \in \text{CLOSED}$  then
7       continue // stale entry:  $u$  was settled earlier with a smaller key
8     add  $u$  to CLOSED // settle  $u$ : from now on  $g(u) = \text{dist}(s, u)$ 
9     if  $u = \gamma$  then
10      break // early exit: uniform-cost search
11    foreach  $v \in \text{Succ}(u)$  with  $v \notin \text{CLOSED}$  do
12      if  $g + c(u, v) < g(v)$  then
13         $g(v) \leftarrow g + c(u, v)$ ;  $\text{parent}(v) \leftarrow u$  // relaxation
14        push  $(g(v), v)$  onto OPEN // lazy insertion, no decrease-key
15  return  $g, \text{parent}$ 

```

Walkthrough. Lines 2–3 initialize: only the source has a finite cost and only the source is on the frontier. In an implementation the assignment $g(v) \leftarrow \infty$ for all v is never executed; a dictionary that answers ∞ for a missing key does the same job without touching V , which matters when V is a large grid or the space-time graph of Chapter 8.

Line 5 removes the entry with the smallest key; with a binary heap this costs $\mathcal{O}(\log |\text{OPEN}|)$. Line 6 is the lazy-deletion test. A vertex that was pushed several times sits in the heap several

times, and every entry of it after the first one to be popped is **stale**: its key is at least the key with which the vertex was settled, and it is simply discarded. Line 8 settles u ; this is the moment at which [Theorem 3.7](#) guarantees $g(u) = \text{dist}(s, u)$.

Line 9 is the early exit. Note where it sits: the target is tested when it is *popped*, not when it is first discovered on line 13. A target that has just been discovered may still be reached more cheaply through a vertex that is waiting in the queue; a target that has been popped cannot. [Section 3.5](#) shows both cases on a concrete graph, and [Corollary 3.8](#) states exactly which vertices the early exit settles.

Lines 11–14 *expand* u : they generate its successors and relax every edge. The strict inequality on line 12 keeps the first-found path when two paths cost the same; this is a tie-breaking rule, and [Section 3.8](#) discusses its consequences. Skipping settled successors on line 11 is an optimization, not a necessity: with $c \geq 0$ the test on line 12 would fail for them anyway. The word **expansion** names the unit in which we count the work of a search: one expansion is one non-stale pop together with the generation of its successors. The number of expansions is what [Figure 3.5](#) and the comparisons of [Chapter 4](#) measure.

The result. After the loop, $g(v) = \text{dist}(s, v)$ for every settled vertex, and $g(v) = \infty$ for every vertex the wave never reached because there is no path from s . Without a target every reachable vertex is settled; [Corollary 3.8](#) says exactly which vertices a run with a target settles. The shortest path to a settled vertex v is read off the parent pointers: start at v , move to $\text{parent}(v)$, and repeat until the source is reached; reversing the list gives the path from s to v (**path reconstruction**). If γ is unreachable, the queue runs empty before γ is popped and the search reports failure, which for a drone means that the goal is sealed off by obstacles or no-fly zones and the map, not the planner, has to change.

3.5 A worked example

Two examples follow: an explicit graph small enough to trace on paper with every state of the heap written out, and a grid on which the wave of [Section 3.2](#) becomes visible. Both live in `code/ch03_dijkstra.py`, whose self-test checks every number quoted here.

Example 3.4 (Seven waypoints). [Figure 3.2](#) shows a waypoint graph with seven vertices $A, \dots, G$ and eleven undirected edges whose costs are flight times in minutes. The source is A and the target is G . We first run [Algorithm 3.1](#) from A *without* a target, so that the whole shortest-path tree appears, and then look at what the early exit would have changed. The graph is `EXAMPLE_GRAPH` in the chapter code; `python3 ch03_dijkstra.py` prints the trace of [Table 3.1](#).

Reading the trace. Step 1 settles the source and discovers its two neighbors. Step 2 is the first improvement: C was discovered from A at cost 5, but through B it costs $2 + 1 = 3$, so $(3, C)$ is pushed while $(5, C)$ stays in the heap; step 4 pops the old entry and discards it, lazy deletion at work. The same pattern repeats for D (steps 3–6) and for E (steps 5–8). Vertex F is the most instructive. It is discovered at cost 13 through D , improved to 12 through E and to 11 through G , and settled only at step 10 with $\text{dist}(A, F) = 11$ along the six-edge path A, B, C, D, E, G, F , which beats both the direct edge $D-F$ ($5 + 8 = 13$) and the edge $E-F$ ($8 + 4 = 12$). The settling order A, B, C, D, E, G, F has costs 0, 2, 3, 5, 8, 10, 11: non-decreasing, as the wave picture promised.

Counting. There are twelve pops, seven of which settle a vertex and five of which are stale, and twelve pushes: one per vertex and one per improvement. The heap never holds more than

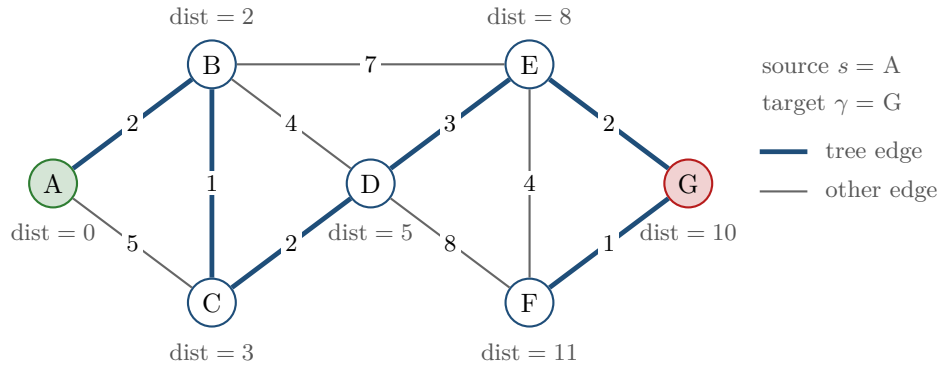


Figure 3.2. The waypoint graph of [Example 3.4](#). Edge labels are costs; after the search the thick blue edges form the shortest-path tree from A , and the gray numbers are the final distances $\text{dist}(A, v)$. Notice that the direct edge $A-C$ of cost 5 is not in the tree, because the detour through B costs only 3, and that F is reached through the target G .

four entries. Every stale entry is the trace of one improvement, so without an early exit the number of stale pops equals the number of improvements; [Theorem 3.10](#) turns this observation into a bound.

The early exit. With the target G the search stops at step 9, after nine pops. The parent pointers give $\text{parent}(G) = E$, $\text{parent}(E) = D$, $\text{parent}(D) = C$, $\text{parent}(C) = B$ and $\text{parent}(B) = A$; reversed, that is the path A, B, C, D, E, G of cost 10. Every vertex settled before G carries its final cost. But F is still in the heap with the tentative cost 12, not the true 11: the improvement through G is made only when G is expanded, and the early exit skips that expansion. `dijkstra(EXAMPLE_GRAPH, 'A', target='G')` indeed returns `dist['F'] == 12`. The tentative cost of an unsettled vertex is an upper bound and nothing more.

Example 3.5 (A 5×5 grid with slow cells). [Figure 3.3](#) shows a 4-connected 5×5 grid. Cells are named (r, c) by row and column, counted from the top-left corner; the start S is $(0, 0)$ and the target T is $(4, 4)$. Two cells are blocked, the center $(2, 2)$ and $(2, 4)$ on the right edge. The eight hatched cells around the center are *slow*: entering one of them costs 3 instead of 1, the kind of penalty an operator attaches to a wind-exposed corridor. Leaving a slow cell costs 1, so the grid graph is directed. The instance is `EXAMPLE_GRID` in the chapter code, with 23 free cells. We run [Algorithm 3.1](#) from S with the target T .

Following the wave. The first pop settles S and discovers $(0, 1)$ and $(1, 0)$ at cost 1. Ties are broken by comparing the cells themselves, so $(0, 1)$ is settled before $(1, 0)$; it discovers $(0, 2)$ at cost 2 and the slow cell $(1, 1)$ at cost $1 + 3 = 4$. After five expansions (first panel) every cell with $g \leq 2$ is settled and the heap holds $(0, 3)$ and $(3, 0)$ at cost 3, $(1, 1)$ at 4, and $(1, 2)$ and $(2, 1)$ at 5. The wave keeps running along the free top row and left column, where every step costs 1, and enters the slow ring only when nothing cheaper is left: $(1, 1)$ is settled ninth, at cost 4, after $(0, 4)$ and before $(4, 0)$, which have the same cost. After ten expansions all cells with $g \leq 4$ are settled (second panel), after seventeen all cells with $g \leq 6$ (third panel), and the nineteenth pop settles T with $g(T) = 8$ (fourth panel). The path read off the parent pointers runs down the left column and along the bottom row, eight moves of cost 1; the route along the top row is blocked by $(2, 4)$, and every route through the slow ring enters at least two slow cells and costs 12 or more. The full settling order, with costs, is

$(0, 0): 0$, $(0, 1): 1$, $(1, 0): 1$, $(0, 2): 2$, $(2, 0): 2$, $(0, 3): 3$, $(3, 0): 3$, $(0, 4): 4$, $(1, 1): 4$,
 $(4, 0): 4$, $(1, 2): 5$, $(1, 4): 5$, $(2, 1): 5$, $(4, 1): 5$, $(1, 3): 6$, $(3, 1): 6$, $(4, 2): 6$, $(4, 3): 7$, $(4, 4): 8$.

Table 3.1. Trace of Algorithm 3.1 on Example 3.4 from A without a target. Each row is one pop from the heap. “Push $(3, C)$ (improves 5)” means that the tentative cost of C drops from 5 to 3 and the entry $(3, C)$ is pushed; the old entry $(5, C)$ stays in the heap and is discarded as stale when it surfaces. The last column is the heap after the row, sorted by key. Produced by `dijkstra_trace`.

Step	Popped	Action	OPEN after the step
1	$(0, A)$	settle A ; push $(2, B)$, $(5, C)$	$(2, B)$ $(5, C)$
2	$(2, B)$	settle B ; push $(3, C)$ (improves 5), $(6, D)$, $(9, E)$	$(3, C)$ $(5, C)$ $(6, D)$ $(9, E)$
3	$(3, C)$	settle C ; push $(5, D)$ (improves 6)	$(5, C)$ $(5, D)$ $(6, D)$ $(9, E)$
4	$(5, C)$	stale, C is already settled: discarded	$(5, D)$ $(6, D)$ $(9, E)$
5	$(5, D)$	settle D ; push $(8, E)$ (improves 9), $(13, F)$	$(6, D)$ $(8, E)$ $(9, E)$ $(13, F)$
6	$(6, D)$	stale: discarded	$(8, E)$ $(9, E)$ $(13, F)$
7	$(8, E)$	settle E ; push $(12, F)$ (improves 13), $(10, G)$	$(9, E)$ $(10, G)$ $(12, F)$ $(13, F)$
8	$(9, E)$	stale: discarded	$(10, G)$ $(12, F)$ $(13, F)$
9	$(10, G)$	settle G , the target (an early exit stops here); push $(11, F)$ (improves 12)	$(11, F)$ $(12, F)$ $(13, F)$
10	$(11, F)$	settle F ; nothing improves	$(12, F)$ $(13, F)$
11	$(12, F)$	stale: discarded	$(13, F)$
12	$(13, F)$	stale: discarded	empty

Nineteen of the 23 free cells are settled when the search stops. Three cells, $(2, 3)$, $(3, 2)$ and $(3, 3)$, are in the heap with the tentative costs 9, 9 and 10; a full run without a target settles them at exactly these values, because no cheaper route to them exists, but the search cannot know that when it stops. One cell, $(3, 4)$, has not been discovered at all: its only free neighbors are $(3, 3)$, which is not expanded, and T , which is popped but, because of the early exit, never expanded. This instance produces no stale entries, since every cell keeps the cost of its first discovery; Exercise 3.5 shows that 8-connectivity changes that.

3.6 Properties: correctness, optimality, complexity

Throughout this section $G = (V, E, c)$ is a finite graph with $c \geq 0$, s is the source, and g , OPEN and CLOSED refer to Algorithm 3.1. Two facts hold at every moment of a run and are used without further comment. First, $g(v) \geq \text{dist}(s, v)$ for every v , because $g(v)$ is always the cost of an actual path from s to v , the one recorded by the parent pointers: line 13 only ever stores the cost of a path to u extended by one edge. Second, $g(v)$ never increases, because line 12 allows only strict improvements.

Lemma 3.6 (Monotone settling order). *The keys of the entries popped on line 5 form a non-decreasing sequence. Moreover, when an entry (g, u) is popped and u is not yet settled, g equals the current tentative cost $g(u)$. Hence vertices are settled in non-decreasing order of the cost they carry at the moment of settling.*

Proof. Let (g, u) be popped. Every entry still in the heap has a key of at least g , because the heap returned the minimum. Every entry pushed while u is expanded has the key $g + c(u, v) \geq g$, because $c \geq 0$. So the next pop has a key of at least g , and by induction so do all later pops. For the second claim, suppose (g, u) is popped, u is not settled, and $g(u) < g$. Then $g(u)$ was lowered after (g, u) was pushed, and at that moment an entry $(g(u), u)$ with a

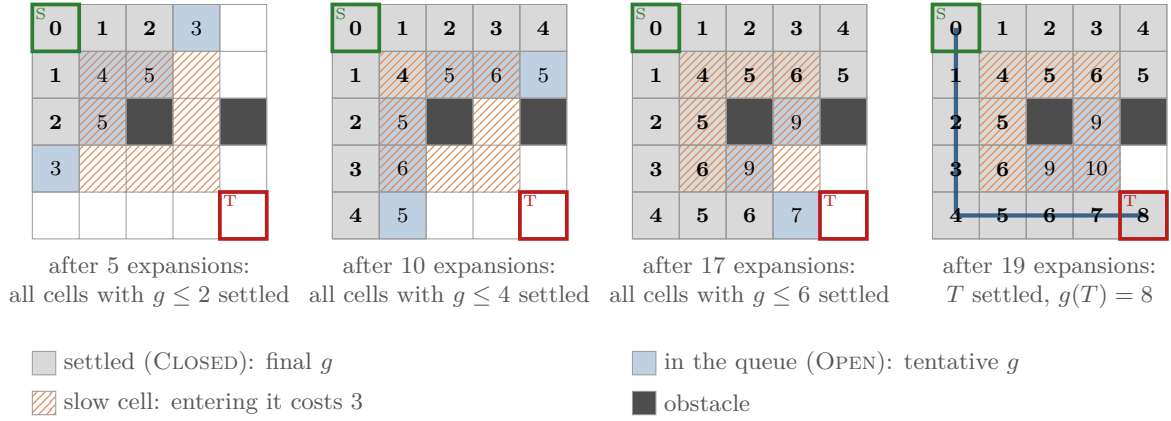


Figure 3.3. The closed set growing on the grid of [Example 3.5](#): snapshots after 5, 10, 17 and 19 expansions of a run with target T . Gray cells are settled and carry their final cost, blue cells are in the heap with their tentative cost. At every moment the settled cells are exactly the cells whose cost is at most the last popped key: this is the wave. It runs along the free border before it seeps into the slow ring, and it stops when T is popped with $g(T) = 8$; the blue line is the path read off the parent pointers.

smaller key was pushed. That entry is still in the heap, since an entry of u leaves the heap only by being popped, which either settles u or requires u to be settled already. A heap never returns (g, u) while it holds a smaller key, a contradiction. $\square$

The lemma is the wave: the radius of the settled region, measured by the popped key, never shrinks. Note where $c \geq 0$ entered. With a negative edge, an entry pushed during an expansion could have a smaller key than the entry just popped, and the popped keys would no longer be sorted.

Theorem 3.7 (Settled-node invariant). *When [Algorithm 3.1](#) settles a vertex u on line 8, $g(u) = \text{dist}(s, u)$, and from then on $g(u)$ does not change. Consequently:*

1. *Without a target, the algorithm settles every vertex reachable from s , returns $g(v) = \text{dist}(s, v)$ for all $v \in V$ (with $g(v) = \infty$ exactly for the unreachable vertices), and its parent pointers form a shortest-path tree.*
2. *With a target γ , the algorithm returns $g(\gamma) = \text{dist}(s, \gamma)$ and a shortest path from s to γ if γ is reachable, and ends with an empty heap otherwise.*

Proof. Suppose the claim fails, and let u be the *first* vertex settled with $g(u) > \text{dist}(s, u)$. Then $u \neq s$, because s is settled first with $g(s) = 0 = \text{dist}(s, s)$, and $\text{dist}(s, u) < \infty$, so a shortest path π from s to u exists. Look at the moment just before u is settled. The first vertex of π , namely s , is settled and the last, u , is not, so π has a first unsettled vertex y ; let x be its predecessor on π (x may be s). We collect three facts.

- (i) $g(x) = \text{dist}(s, x)$, because x was settled before u and u is the first violation.
- (ii) $g(y) = \text{dist}(s, y)$, and the heap holds the entry $(g(y), y)$. When x was settled it was expanded, and the edge (x, y) was relaxed on line 12: afterward $g(y) \leq g(x) + c(x, y) = \text{dist}(s, x) + c(x, y) = \text{dist}(s, y)$, the last step because the prefix of π up to y is a shortest path to y (subpaths of shortest paths are shortest paths, [Section 3.3](#)). Together with $g(y) \geq \text{dist}(s, y)$ this gives equality. The push that established this value put $(\text{dist}(s, y), y)$ into the heap, and since y is not settled, the entry is still there.
- (iii) $\text{dist}(s, y) \leq \text{dist}(s, u)$, because y precedes u on π and the edges between them cost at least 0. *This is the only place where $c \geq 0$ is used.*

Now u is being settled because the heap returned $(g(u), u)$ as its minimum, so every key in the

heap is at least $g(u)$. But by (ii) and (iii) the heap holds the key $\text{dist}(s, y) \leq \text{dist}(s, u) < g(u)$. This contradiction shows that no violation exists. That $g(u)$ does not change after settling follows: it cannot increase, and it cannot decrease below $\text{dist}(s, u)$.

For consequence 1, let v be reachable and let π be a shortest path to v . By induction along π , every vertex of π is settled: the source is, and when a vertex of π is expanded, its successor on π is pushed, or already has a finite cost and an entry in the heap, and is therefore popped before the heap runs empty. Unreachable vertices are never pushed. For the tree: when u is settled, $\text{parent}(u)$ is the vertex x whose expansion produced $g(u) = g(x) + c(x, u)$; x was settled, so $g(x) = \text{dist}(s, x)$ and $\text{dist}(s, u) = \text{dist}(s, x) + c(x, u)$: the edge (x, u) lies on a shortest path, and following the pointers back to s traces one. Consequence 2 is the invariant applied to γ : if γ is reachable it is pushed and settled by the argument above, and if it is not, it is never pushed, so the loop ends with an empty heap and never breaks on line 9. $\square$

Corollary 3.8 (Uniform-cost search). *With a target γ , Algorithm 3.1 settles only vertices v with $\text{dist}(s, v) \leq \text{dist}(s, \gamma)$, and it settles every vertex with $\text{dist}(s, v) < \text{dist}(s, \gamma)$. The tentative costs of the vertices left in the heap are upper bounds on their distances and may be strictly larger.*

Proof. By Lemma 3.6 every vertex settled before γ has a key of at most $g(\gamma)$, and by Theorem 3.7 these keys are the true distances. For the second claim, suppose $\text{dist}(s, v) < \text{dist}(s, \gamma)$ and v is not settled at the moment γ is popped. Take a shortest path from s to v and let y be its first unsettled vertex; it exists, since s is settled and v is not. Note first that γ does not lie on this path: the prefix of the path ending at γ would cost at least $\text{dist}(s, \gamma)$ and the rest of the path at least 0, so $\text{dist}(s, \gamma) \leq \text{dist}(s, v)$, contradicting $\text{dist}(s, v) < \text{dist}(s, \gamma)$ (here $c \geq 0$ is used again). This step is needed because γ is the one vertex that the early exit on line 9 settles without expanding. Hence $y \neq \gamma$, and the predecessor of y on the path is a vertex settled and expanded strictly before γ was popped, so, as in step (ii) of the proof of Theorem 3.7, it has relaxed the edge into y and the heap holds the entry $(\text{dist}(s, y), y)$ with $\text{dist}(s, y) \leq \text{dist}(s, v) < \text{dist}(s, \gamma) = g(\gamma)$, contradicting that $(g(\gamma), \gamma)$ was the minimum of the heap. Which of the vertices with $\text{dist}(s, v) = \text{dist}(s, \gamma)$ are settled depends on tie-breaking, and termination in the presence of zero-cost edges is Exercise 3.3; Example 3.4 shows the upper bound $g(F) = 12 > 11$ at the exit. $\square$

Why the costs must be non-negative. The proof used $c \geq 0$ exactly once, in step (iii), and a single negative edge is enough to break the algorithm.

Example 3.9 (A negative edge). For this example only we drop the non-negativity requirement of Definition 3.1. Figure 3.4 shows a directed graph with four vertices and one negative edge, $B \rightarrow A$ of cost -2 . Run Algorithm 3.1 from S . The first pop settles S and pushes $(1, A)$ and $(2, B)$. The second pop settles A with $g(A) = 1$ and pushes $(2, T)$. The third pop settles B with cost 2 and looks at the edge $B \rightarrow A$: the new cost $2 + (-2) = 0$ would improve A , but A is settled, so line 11 skips it (and had it not, the successor T of A would still keep the cost 2 it inherited from the wrong value). The fourth pop settles T with $g(T) = 2$ and the path S, A, T . The true distances are $\text{dist}(S, A) = 0$ and $\text{dist}(S, T) = 1$, along S, B, A, T . In the proof, the first violation is $u = A$: the shortest path to A is S, B, A , its first unsettled vertex is $y = B$ with $g(B) = \text{dist}(S, B) = 2$, and step (iii) would need $\text{dist}(S, B) \leq \text{dist}(S, A)$, that is $2 \leq 0$, which is false precisely because $c(B, A) < 0$. The chapter code reproduces this run as `NEGATIVE_EXAMPLE: dijkstra` returns 2 for T , and the Bellman–Ford reference returns 1.

The failure is silent, which is what makes it dangerous. On graphs with negative edges but no negative cycles the algorithm of choice is Bellman–Ford, which relaxes every edge $|V| - 1$ times and runs in $\mathcal{O}(|V| |E|)$ [31]; the chapter code uses it only as a reference in the self-test, because in planning negative costs are best avoided altogether (see the pitfalls in Section 3.8).

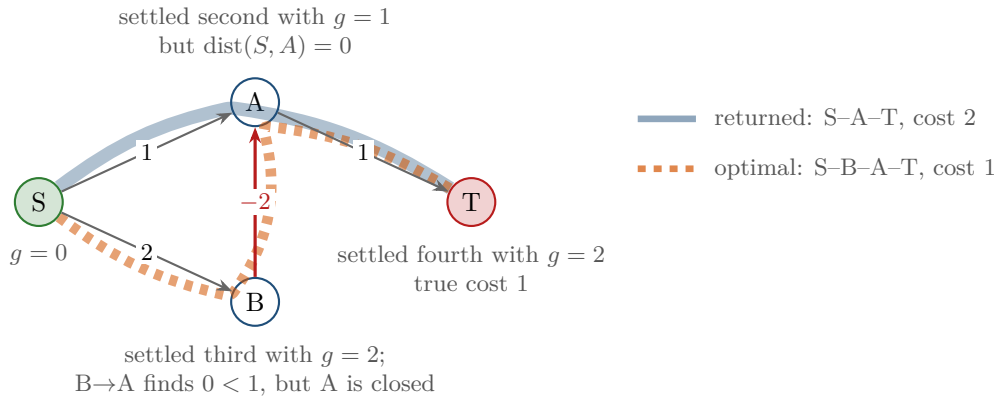


Figure 3.4. The graph of Example 3.9. Vertex A is settled with $g = 1$ before B reveals the route S, B, A of cost 0; a settled value is never revised, so the search returns S, A, T (cost 2) instead of the optimal S, B, A, T (cost 1). Nothing warns the caller.

Theorem 3.10 (Running time and memory). *On a graph with $|V|$ vertices and $|E|$ directed edges, Algorithm 3.1 with a binary heap and lazy deletion performs at most $|V|$ expansions and at most $|E| + 1$ pushes and pops, runs in $\mathcal{O}((|V| + |E|) \log |V|)$ time, and uses $\mathcal{O}(|V| + |E|)$ memory beyond the graph.*

Proof. A vertex is settled at most once, because line 6 discards every later entry, so the loop of line 11 runs at most once per vertex and relaxes each directed edge at most once. Each relaxation pushes at most one entry, so the heap receives at most $|E| + 1$ entries including the initial one, and every pop removes one, so there are at most $|E| + 1$ pops. The heap never holds more than $|E| + 1$ entries, and $|E| \leq |V|^2$, so each push or pop costs $\mathcal{O}(\log(|E| + 1)) = \mathcal{O}(\log |V|)$. Generating successors costs $\mathcal{O}(|V| + |E|)$ in total, and each dictionary lookup or update costs $\mathcal{O}(1)$ expected time. Adding up gives $\mathcal{O}(|V| + |E| + (|E| + 1) \log |V|) \subseteq \mathcal{O}((|V| + |E|) \log |V|)$. The memory is the heap, at most $|E| + 1$ entries, plus the dictionaries g , parent and CLOSED of at most $|V|$ entries each. $\square$

Reading the bound. On a grid every cell has at most 8 neighbors, so $|E| \leq 8|V|$ and the time is $\mathcal{O}(|V| \log |V|)$, almost linear in the number of cells. The classic version with a decrease-key operation keeps at most $|V|$ entries in the heap but performs the same $|E|$ key updates at $\mathcal{O}(\log |V|)$ each, so its bound is the same and its heap operations are more complex. Two refinements are worth knowing.

- *Fibonacci heaps* [47] make decrease-key cost $\mathcal{O}(1)$ amortized and bring the time to $\mathcal{O}(|E| + |V| \log |V|)$. The gain is real only when $|E| \gg |V|$; on sparse graphs such as grids the binary heap with lazy deletion is faster in practice, and it is the version used throughout this book.
- *Dense graphs.* When $|E| = \Theta(|V|^2)$, for instance on a complete visibility graph between waypoints, the heap bound becomes $\mathcal{O}(|V|^2 \log |V|)$, and the original formulation of Dijkstra [34], which keeps the tentative costs in an array and finds the minimum by scanning it in $\mathcal{O}(|V|)$ time per expansion, is better: $\mathcal{O}(|V|^2)$ in total.

Expansions in practice. The bound counts every vertex, but the work of a run is really the area of the wave. Figure 3.5 measures it on random $n \times n$ grids with 25 % obstacles, 8-connected, with the source in the top-left corner, averaged over ten seeded instances per size (code/figures/gen_ch03_expansions.py). Without a target the wave settles every reachable cell, about 99 % of the free cells (7 438 of 7 499 at $n = 100$; the rest are pockets

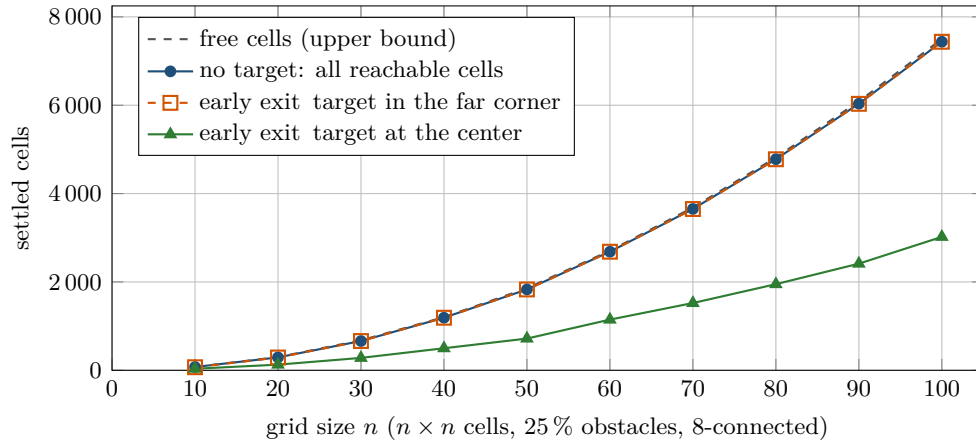


Figure 3.5. Settled cells against grid size on random 8-connected grids with 25 % obstacles, mean of ten seeded instances per size; the source is the top-left cell. Without a target almost every free cell is settled. The early exit saves nothing when the target is the far corner, the last region the wave reaches, and about 60 % of the expansions when the target is the center of the map, whose distance from the source is about half that of the far corner.

Table 3.2. Variants of Dijkstra's algorithm and what they deliver. G^R is the graph with every edge reversed, and μ is the cost of the best connecting path found so far by a bidirectional search.

Variant	Starts from	Runs on	Stops when	Delivers
Dijkstra	s	G	heap empty	$\text{dist}(s, v)$ for all v , tree
Uniform-cost search	s	G	γ settled	$\text{dist}(s, \gamma)$, one path
Breadth-first search	s	G , unit costs	γ discovered	hop counts, one path
Multi-source	every $s \in S$	G	heap empty	distance to the nearest source
Backward	γ	G^R	heap empty	$h^*(v) = \text{dist}(v, \gamma)$ for all v
Bidirectional	s and γ	G and G^R	smallest keys sum to $\geq \mu$	$\text{dist}(s, \gamma)$, one path

sealed off by obstacles), so the count grows with the area of the map: four times as many cells from $n = 50$ to $n = 100$. With the early exit and the target in the far corner the saving is nil, 7 437 settled cells instead of 7 438, because the far corner is among the last cells the wave reaches, at a cost of 165.5 on average. With the target at the center of the map, at a cost of 85.3, the wave stops at about half the radius and settles 3 020 cells, 41 % of the full run. The early exit therefore helps exactly when the target is close, and the only way to do better when it is not is to steer the wave toward the target, which is what the heuristic of A^* does in Chapter 4.

3.7 Variants and extensions

The invariant of Theorem 3.7 survives a number of changes to Algorithm 3.1, as long as they alter only where the wave starts, on which graph it runs, or when it stops. Table 3.2 lists the variants used in this book; the rest of the section explains them.

3.7.1 Several sources

Push every source of a set $S \subseteq V$ with key 0 instead of the single entry $(0, s)$ on line 3, and leave the rest of the algorithm unchanged. The run is identical to a single-source run from a virtual vertex joined to every $s \in S$ by an edge of cost 0, so [Theorem 3.7](#) applies and the result is

$$g(v) = \text{dist}(S, v) = \min_{s \in S} \text{dist}(s, v),$$

the cost from the source *nearest* to v , with parent pointers that lead back to it. Started with key 0 from every free cell that touches an obstacle, on a grid with unit costs, the run labels each free cell with the number of steps to the nearest obstacle boundary, one less than its distance to the nearest blocked cell (the grid graph of [Chapter 2](#) has only free cells as vertices, so the blocked cells cannot themselves be sources). That table is the distance map behind the inflated obstacles of [Chapter 2](#) and behind clearance penalties; started from all depots, it tells every drone which depot is closest in a single run. The function `multi_source_dijkstra` in the chapter code does this, and its self-test checks the table against the element-wise minimum over single-source runs.

3.7.2 Backward search: the true-distance heuristic

A^* ([Chapter 4](#)) orders its queue by $g(v) + h(v)$, where $h(v)$ estimates the cost that remains from v to the target γ . The best conceivable estimate is the true cost-to-go $\text{dist}(v, \gamma)$, and Dijkstra's algorithm can compute it for *every* vertex at once: run it from γ on the graph with all edges reversed. [Algorithm 3.2](#) states the recipe. The resulting table is written h^* and called the **true-distance heuristic** or *perfect heuristic*.

Algorithm 3.2. Backward Dijkstra: the true-distance heuristic table for a goal γ .

Input: graph $G = (V, E, c)$ with $c \geq 0$; goal γ

Output: table h^* with $h^*(v) = \text{dist}(v, \gamma)$ for every v from which γ can be reached, and $h^*(v) = \infty$ otherwise

```

1 Function BackwardDijkstra( $G, \gamma$ ):
2    $G^R \leftarrow (V, E^R, c^R)$  with  $E^R = \{(v, u) : (u, v) \in E\}$  and  $c^R(v, u) = c(u, v)$  // reverse
   every edge
3    $(g^R, \cdot) \leftarrow \text{Dijkstra}(G^R, \gamma, \text{nil})$  // one full run from  $\gamma$ , no target
4   return  $h^* \leftarrow g^R$ 

```

Reversing matters whenever the graph is directed: a path $v = v_0, v_1, \dots, v_k = \gamma$ in G is a path $\gamma = v_k, \dots, v_0 = v$ in G^R with the same cost, so $\text{dist}_G(v, \gamma) = \text{dist}_{G^R}(\gamma, v)$, which is what the run from γ on G^R computes. On an undirected graph $G^R = G$, and a plain forward run from γ suffices, as in [Exercise 3.6\(a\)](#). The grid of [Example 3.5](#) is directed, and [Figure 3.6](#) shows the difference: a forward run from T would say that $(3, 3)$ costs 4 from T (leave T for 1, enter the slow cell for 3), whereas $h^*(3, 3) = 2$, because from $(3, 3)$ the drone leaves the slow cell for 1 and enters T for 1. An implementation does not need to build G^R : `GridGraph.reversed()` in the chapter code simply charges the cost of a slow cell when the cell is *left* instead of when it is entered, and `reverse_graph` flips an adjacency list in $\mathcal{O}(|V| + |E|)$ time.

Theorem 3.11 (The true-distance heuristic). *Let h^* be the table returned by `BackwardDijkstra`(G, γ) for a graph with $c \geq 0$. Then:*

1. *Exactness. $h^*(v) = \text{dist}(v, \gamma)$ for every $v \in V$, with $h^*(v) = \infty$ if γ cannot be reached from v . Assume in addition $\text{dist}(s, \gamma) < \infty$. Then a vertex v lies on some shortest path from s to γ if and only if $\text{dist}(s, v) + h^*(v) = \text{dist}(s, \gamma)$; both terms are then finite.*

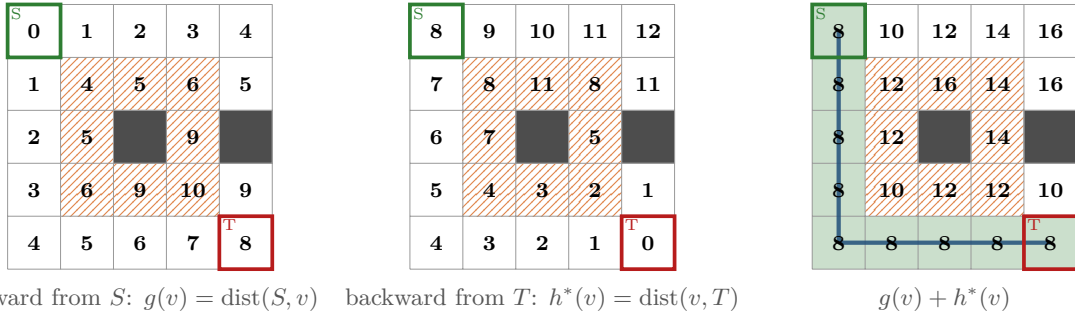


Figure 3.6. Forward and backward tables on the grid of [Example 3.5](#), both from full runs. Left: $g(v) = \text{dist}(S, v)$ from S . Middle: $h^*(v) = \text{dist}(v, T)$ from [Algorithm 3.2](#); because entering a slow cell costs 3 but leaving it costs 1, this is not the distance from T (compare $h^*(3, 3) = 2$ with the 4 that a forward run from T gives). Right: $g(v) + h^*(v)$, the cost of the best path from S to T through v . The green cells, where the sum equals $\text{dist}(S, T) = 8$, are exactly the cells of the optimal path; every other cell has a larger sum, which is how A^* will know to avoid them.

2. Consistency. $h^*(\gamma) = 0$ and $h^*(u) \leq c(u, v) + h^*(v)$ for every edge $(u, v) \in E$. Consequently $h^*(v) \leq c(\pi)$ for every path π from v to γ .
3. Admissibility in space-time. Consider the space-time graph of [Chapter 2](#) built on G , in which a state is a pair (vertex, time), a move along (u, v) costs $c(u, v)$, waiting costs a non-negative amount, and any set of states or moves may be forbidden. Every path in it from a state (v, t) to a state (γ, t') costs at least $h^*(v)$.

Proof. (1) By the correspondence between paths in G and in G^R , $\text{dist}_{G^R}(\gamma, v) = \text{dist}_G(v, \gamma)$, and [Theorem 3.7](#) applied to G^R gives $h^*(v) = \text{dist}_{G^R}(\gamma, v)$. For the second statement, if v lies on a shortest s - γ path, its two halves are shortest paths, so $\text{dist}(s, \gamma) = \text{dist}(s, v) + \text{dist}(v, \gamma)$. Conversely, if $\text{dist}(s, v) + \text{dist}(v, \gamma) = \text{dist}(s, \gamma) < \infty$, then both summands are finite, since their sum is, so a shortest s - v path and a shortest v - γ path both exist; concatenated they cost $\text{dist}(s, \gamma)$. The concatenation may repeat vertices, but removing the cycles costs nothing extra ($c \geq 0$) and leaves a shortest path through v . The hypothesis $\text{dist}(s, \gamma) < \infty$ cannot be dropped: if γ is unreachable from s , every v that is itself unreachable satisfies $\infty + h^*(v) = \infty$ without lying on any s - γ path. (2) $h^*(\gamma) = \text{dist}(\gamma, \gamma) = 0$. For an edge (u, v) , going from u to v and then along a shortest path from v to γ is a path from u to γ of cost $c(u, v) + \text{dist}(v, \gamma)$, which is at least $\text{dist}(u, \gamma)$; this is the third fact of [Equation \(3.1\)](#). Applying the inequality to the edges $(v_0, v_1), \dots, (v_{k-1}, v_k)$ of a path π from $v_0 = v$ to $v_k = \gamma$ and adding up, the h^* terms telescope to $h^*(v) \leq c(\pi) + h^*(\gamma) = c(\pi)$. (3) Drop the time index from the space-time path and delete every wait step. What remains is a walk in G from v to γ whose cost is at most that of the space-time path, because waiting costs at least 0 and every move keeps its cost. A walk from v to γ costs at least $\text{dist}(v, \gamma) = h^*(v)$, since removing its cycles cannot increase the cost. Forbidding states or moves removes paths from the space-time graph and adds none, so the bound is unaffected. $\square$

[Chapter 4](#) calls property 2 of [Theorem 3.11](#) *consistency* and its consequence *admissibility*, and proves that they make A^* optimal. Property 1 is visible in the right-hand panel of [Figure 3.6](#): the cells with $g(v) + h^*(v) = \text{dist}(S, T) = 8$, shaded green, are exactly the cells of the optimal path, and every other cell has a larger sum. An A^* search guided by h^* therefore expands only cells that lie on a shortest path; no heuristic can do better on a static map, which is why h^* is the reference against which the grid heuristics of [Chapter 4](#) are judged. Property 3 is what makes the table useful beyond this chapter. The multi-agent solvers of [Chapters 8](#) and [9](#) route one drone at a time through *space-time*, where waiting is an action and the reservations or constraints of other drones forbid certain cells at certain times, and

they do so thousands of times on the same map with the same goals. One backward run per goal, costing $\mathcal{O}(|V| \log |V|)$ on a grid, produces a table that every one of those searches reads in $\mathcal{O}(1)$ per expansion and that never overestimates, whatever the constraints. It is no longer exact there, since a forced wait or detour is not in the table ([Exercise 3.6\(c\)](#)), but admissibility is all that A^* needs for optimality. D^* Lite ([Chapter 5](#)) searches backward from the goal in the same way and repairs the table when the map changes instead of recomputing it.

3.7.3 Bidirectional search

For a single pair (s, γ) on a large graph, one may grow two waves at once: a forward wave from s on G with costs g_f , and a backward wave from γ on G^R with costs g_b , each with its own heap and closed set, alternating expansions between the two sides. Whenever one side scans an edge (u, v) during an expansion, including edges to vertices that either side has settled, and both $g_f(u)$ and $g_b(v)$ are finite, the concatenation is an s - γ path of cost $g_f(u) + c(u, v) + g_b(v)$, and the best such cost seen so far is kept in μ . The search stops when

$$\min_{\text{OPEN}_f} g_f + \min_{\text{OPEN}_b} g_b \geq \mu, \quad (3.2)$$

and returns the path that achieved μ . The tempting rule “stop as soon as one vertex has been settled by both sides” is wrong: the first common vertex need not lie on a shortest path, because the two waves may meet at a vertex that is close to both but not on the cheapest route. In [Equation \(3.2\)](#) an empty OPEN counts as $\min = \infty$, so the rule also fires correctly when one side exhausts its component. The rule is correct, and the argument has two cases. Let π be an s - γ path with $c(\pi) < \mu$ at the moment the rule fires. If π holds a vertex still open on the forward side and, *later* on π , a vertex still open on the backward side, let u be the first vertex of π the forward side has not settled and w the last one the backward side has not settled; by the case hypothesis u comes no later than w on π . The predecessor of u on π is settled forward, so step (ii) of the proof of [Theorem 3.7](#), applied to the forward run, puts u in OPEN_f with $g_f(u) = \text{dist}(s, u)$, and symmetrically w is in OPEN_b with $g_b(w) = \text{dist}(w, \gamma)$; hence $\min_{\text{OPEN}_f} g_f \leq \text{dist}(s, u)$ and $\min_{\text{OPEN}_b} g_b \leq \text{dist}(w, \gamma)$. The prefix of π up to u costs at least $\text{dist}(s, u)$, its suffix from w at least $\text{dist}(w, \gamma)$, and the piece between them at least 0, so $c(\pi)$ is at least the sum of the two smallest keys, hence at least μ : a contradiction. Otherwise the forward-settled prefix of π and its backward-settled suffix touch, so π has an edge (x, y) with x settled forward and y settled backward. Whichever of x and y was settled later scanned that edge during its own expansion — x scans (x, y) forward, y scans it backward on G^R , and this is what the clause “including edges to vertices that either side has settled” is for — and by then both $g_f(x)$ and $g_b(y)$ were already final, so $\mu \leq g_f(x) + c(x, y) + g_b(y) \leq c(\pi)$, again a contradiction. This second case is the crux; [Exercise 3.8\(b\)](#) asks for it in full, and part (a) for the counterexample to the naive rule.

The saving comes from geometry: on a map the number of settled cells grows with the area of the wave, one wave of radius d covers an area proportional to d^2 , and two waves of radius $d/2$ that meet halfway cover $2(d/2)^2 = d^2/2$, half as much (a quarter in three dimensions). The gain is real but modest, and it shrinks when a good heuristic is available, so the book uses bidirectional search only as a benchmark; Russell and Norvig [142] give a careful treatment, including the heuristic version.

3.7.4 Unit costs: breadth-first search

When every edge has the same cost $c_0 > 0$, the keys in the heap are multiples of c_0 and the heap can be replaced by a first-in-first-out queue. The reason is [Lemma 3.6](#): at any moment the queue holds vertices of at most two consecutive levels k and $k+1$ (in units of c_0), all of level k ahead of all of level $k+1$, so the front of the queue always has the smallest key. Two further

simplifications follow. A vertex is pushed at most once, because its first discovery is already at the smallest possible level, so there are no stale entries and no relaxation test. And the target may be recognized when it is *discovered*, since no later discovery can be cheaper. The result is **breadth-first search**, which runs in $\mathcal{O}(|V| + |E|)$ time, without the logarithm. The function `bfs` in the chapter code implements it, and the self-test checks it against `dijkstra` on random 4-connected grids with unit costs. On an 8-connected grid the diagonal costs $\sqrt{2}$, the levels interleave, and breadth-first search is wrong; every search from [Chapter 4](#) onward is built on the heap version for this reason.

3.8 Implementation notes

[Listing 3.1](#) is the function `dijkstra` of `code/ch03_dijkstra.py`, a direct transcription of [Algorithm 3.1](#). The remarks below explain the choices in it; [Listing 3.2](#) adds path reconstruction and the backward run.

The heap. `heapq` is a binary min-heap stored in a list; it offers `heappush` and `heappop` but no decrease-key, which is why the listing pushes duplicates. Do not “fix” this by searching the list for the old entry: that costs $\mathcal{O}(|\text{OPEN}|)$ per relaxation and destroys the bound of [Theorem 3.10](#). Entries are tuples `(g, node)`; Python compares tuples lexicographically, so ties in `g` are broken by comparing the nodes themselves. This works for strings and for `(row, col)` tuples and makes the search deterministic, which is what you want when you compare runs or reproduce a trace. If your nodes are not comparable, insert a running counter as the second element of the tuple, as `LazyPQ` in [Chapter 2](#) does. ([Appendix B.6](#) reviews binary heaps, the cost of their operations and the asymptotic notation used here.)

Costs and parents. `dist` is a dictionary: a missing key means ∞ , so `dist.get(v, INF)` is the test of line 12 and no array of size $|V|$ is ever allocated. `parent` maps every discovered node to its predecessor and the source to `None`; `reconstruct_path` follows the pointers back and reverses the list. Because the parent is overwritten at every relaxation, the pointers always describe the path that produced the current `dist`.

The closed set. `closed` is a set, and the membership test on pop is the lazy-deletion check. After the loop, `dist[v]` is exact if and only if `v in closed`; with an early exit, the tentative costs of open nodes are upper bounds only (recall F in [Example 3.4](#)). If your caller needs the guarantee, return `closed` as well, or return only the settled entries.

Implicit graphs. The function only ever evaluates `graph[u]`, which must yield `(neighbor, cost)` pairs. A dictionary of lists does this for explicit graphs. The class `GridGraph` in the same file does it for occupancy grids: it generates the 4 or 8 neighbors of a cell on the fly, with cost 1 or $\sqrt{2}$, forbids corner cutting as in [Chapter 2](#), and multiplies the cost of a move by an optional factor attached to the cell being entered. Its method `reversed()` flips the direction in which the factors are applied, so that [Algorithm 3.2](#) can run on a grid without ever building the reversed graph explicitly.

Tie-breaking. Among several shortest paths, the one returned depends on two rules: the strict `<` on line 12, which keeps the first path found at equal cost, and the order of the heap among equal keys. Both are deterministic here, but neither is canonical: a different neighbor order yields a different, equally optimal path. Planners that must be reproducible fix the neighbor order, as `MOVES` in the code does. A^* in [Chapter 4](#) adds a tie-breaking rule on `g` that changes the number of expansions substantially.

Listing 3.1. The core of the implementation (from `code/ch03_dijkstra.py`): the main loop with `heapq`, a dictionary of tentative costs, a closed set tested on every pop, and lazy insertion instead of decrease-key.

```

1 def dijkstra(graph, source, target=None):
2     """Shortest paths from source; every edge cost must be >= 0.
3
4     graph[u] yields the (v, cost) pairs of the out-edges of u; a dict of
5     lists or a GridGraph both work. Returns (dist, parent): dist[v] is the
6     cost of the best path found to v and parent[v] its predecessor on that
7     path (parent[source] is None). Unreached nodes are absent (infinity).
8     With a target the loop stops as soon as the target is settled
9     (uniform-cost search); dist is then exact for settled nodes only.
10    """
11    dist = {source: 0.0}
12    parent = {source: None}
13    closed = set()
14    queue = [(0.0, source)]           # min-heap of (g, node) entries
15    while queue:
16        g, u = heapq.heappop(queue)   # smallest g first, ties by node
17        if u in closed:               # stale entry: u was settled earlier
18            continue
19        closed.add(u)                 # settle u: dist[u] == g is final
20        if u == target:
21            break                     # early exit
22        for v, cost in graph[u]:
23            if v in closed:
24                continue
25            g_new = g + cost
26            if g_new < dist.get(v, INF):
27                dist[v] = g_new        # relaxation
28                parent[v] = u
29                heapq.heappush(queue, (g_new, v))    # lazy insertion
30    return dist, parent

```

Listing 3.2. Path reconstruction from the parent pointers and the backward run that produces the true-distance heuristic table (from `code/ch03_dijkstra.py`).

```

1 def reconstruct_path(parent, target):
2     """Follow parent pointers back from target; None if unreachable."""
3     if target not in parent:
4         return None
5     path = []
6     node = target
7     while node is not None:
8         path.append(node)
9         node = parent[node]
10    path.reverse()
11    return path
12
13
14 def backward_dijkstra_heuristic(graph, goal):
15     """True-distance heuristic h*(v) = dist(v, goal) for every node v.
16
17     One run of dijkstra() from the goal over the reversed graph. The
18     table is a perfect heuristic: admissible and consistent on the static
19     map, and still admissible when wait actions or constraints are added
20     (Chapters 4, 8 and 9). Unreached nodes are absent: read as infinity.

```

```

21     """
22     if hasattr(graph, "reversed"):
23         rev = graph.reversed()          # GridGraph knows how to flip itself
24     else:
25         rev = reverse_graph(graph)
26     h_star, _ = dijkstra(rev, goal)
27     return h_star

```

Testing for the goal at the wrong time

The most common bug in a first implementation is to return as soon as the target is *discovered*, on line 13, instead of when it is *popped*. In [Example 3.4](#) the vertex *F* is discovered with cost 13, later improved to 12 and then to 11. A search that stops at discovery would report 13, almost twenty percent too much, and the “optimal” path it returns would be *A, B, C, D, F*. Breadth-first search may test at discovery because every edge costs the same; Dijkstra’s algorithm and A* may not. The same rule applies to the early exit of every search in [Chapters 4, 8 and 9](#).

Negative costs and the constant-shift trap

Dijkstra’s algorithm is silently wrong when an edge has a negative cost ([Example 3.9](#)): it does not crash, it returns a path that is not optimal. Negative costs sneak into planners as rewards, for instance a bonus for flying through a cell that must be surveyed. Adding a constant to all edge costs does not help, because paths with more edges are penalized more and the optimal path changes ([Exercise 3.4](#)). Model rewards as reduced non-negative costs, or use the Bellman–Ford algorithm. Zero costs are fine.

Forgetting the stale check

Without the test on line 6 every duplicate entry is expanded again. The costs stay correct, because a relaxation from a stale entry can never improve anything, but a vertex is then “settled” several times: the count of expansions is wrong, a visualization of the closed set shows cells being settled twice, and in the incremental algorithms of [Chapter 5](#), where an expansion has side effects, the result is wrong. Test membership in the closed set, or compare the popped key with the current `dist[u]`; either works, but one of them must be there.

Comparing floating-point costs for equality

On 8-connected grids the costs 1 and $\sqrt{2}$ are added in different orders along different paths, so two paths of the same true cost can end with values that differ in the last binary digit. Never test costs for equality: `g_new == dist[v]` then silently decides which of two optimal paths survives, and a test such as `g + h == best` can fail on the very cell it was meant to catch. Compare with a tolerance, or scale the costs to integers (many grid planners use 10 for a straight move and 14 for a diagonal one). The self-test of the chapter code compares against a brute-force enumeration with a tolerance of 10^{-9} and checks that h^* from the backward run agrees with a forward search from every cell.

3.9 Where this fits in the drone system

In the drone system

Dijkstra’s algorithm sits in the global planner of the hybrid architecture of [Chapter 24](#), in the two roles of [Section 3.1](#): the baseline planner and the heuristic factory. Both run offline, on the static map, before take-off, and both feed layers above them — the multi-agent solver of [Chapters 8 and 9](#) consumes the h^* tables, the experiments of [Chapter 25](#) consume the baseline costs. What the algorithm does *not* do is react. It does not see the other drones of the swarm, and it does not see the intruder that appears after take-off; those belong to the multi-agent solver, the prediction layer and the local safety layer. It also assumes the map it was given is still true, which it is not once a drone reports a new obstacle: the replanning layer therefore runs the same backward wave in incremental form as D^* Lite ([Chapter 5](#)), repairing the distance table instead of recomputing it. In the study plan ([Appendix A](#)) this chapter is the first half of Week 1; the second half is A^* .

3.10 Summary

Chapter summary

- Dijkstra’s algorithm computes $\text{dist}(s, v)$ for every vertex of a graph with non-negative edge costs, together with a shortest-path tree, by settling vertices in non-decreasing order of tentative cost: a wave spreading from the source.
- The settled-node invariant ([Theorem 3.7](#)) says that a vertex’s cost is final when it is popped. Its proof uses $c \geq 0$ exactly once, and a single negative edge breaks it ([Example 3.9](#)).
- With a binary heap and lazy deletion the running time is $\mathcal{O}((|V| + |E|) \log |V|)$, the number of expansions grows with the area of the wave, and the early exit (uniform-cost search) stops the wave at the radius of the target.
- The implementation needs a heap of (g, v) pairs, a dictionary of tentative costs, a closed set that is tested on every pop, parent pointers for the path, and tolerance-based comparisons of floating-point costs.
- One backward run from the goal on the reversed graph gives the true-distance heuristic $h^*(v) = \text{dist}(v, \gamma)$, which is consistent, exact on the static map, and still admissible in space-time and under constraints: the heuristic used throughout [Chapters 4, 8 and 9](#).

Further reading. The original three-page paper is [\[34\]](#); the array-scanning version in it is still the right choice for dense graphs. Cormen et al. [\[31\]](#) give the standard textbook treatment with the proof of the invariant, the Bellman–Ford algorithm for negative costs and the priority-queue trade-offs, and Fredman and Tarjan [\[47\]](#) introduce the Fibonacci heap. Russell and Norvig [\[142\]](#) present the algorithm as uniform-cost search alongside breadth-first and bidirectional search, and Felner [\[39\]](#) explains why that presentation is the better one for implicit graphs. The step from Dijkstra’s algorithm to A^* was taken by Hart, Nilsson, and Raphael [\[62\]](#); Pearl [\[126\]](#) is the classic reference on heuristics, including the exact heuristic of [Section 3.7.2](#).

3.11 Exercises

Exercise 3.1 (★). In [Example 3.4](#) reduce the cost of the edge $B-E$ from 7 to 4 and trace [Algorithm 3.1](#) from A by hand in the format of [Table 3.1](#). Give the final distances, the shortest path to G , the number of pops and the number of stale entries. Check your trace with `dijkstra_trace` in the chapter code.

Exercise 3.2 (★). Suppose [Algorithm 3.1](#) returned as soon as the target is pushed on line 14 rather than when it is popped. Using [Table 3.1](#), give the cost and path it would return for the target F , and for the target G . Which step of the proof of [Theorem 3.7](#) no longer applies to a vertex that has been discovered but not settled?

Exercise 3.3 (★★). [Corollary 3.8](#) settles the two strict cases; this exercise closes the gap at equality and at cost 0. (a) Show that which of the vertices v with $\text{dist}(s, v) = \text{dist}(s, \gamma)$ uniform-cost search settles depends only on the order in which the heap returns entries of equal key, and give a three-vertex graph on which two tie-breaking rules settle different sets. (b) Show that the tentative cost of every vertex left in the queue at the exit is at least $\text{dist}(s, \gamma)$, and explain why it may nevertheless differ from $\text{dist}(s, v)$. (c) Show that edges of cost 0, including zero-cost cycles, do not endanger termination. *Hint for (c):* count pushes, not pops — a vertex is pushed only when its tentative cost strictly decreases, and by [Theorem 3.7](#) it is settled at most once.

Exercise 3.4 (★★). Let G have some negative edge costs but no negative cycle, and let G' be obtained by adding the same constant $K > 0$ to every edge cost so that all costs become non-negative. Show with a graph of four vertices that a shortest path in G' need not be a shortest path in G . Then explain, in one sentence, what property of paths makes the shift harmless when all shortest paths of interest have the same number of edges.

Exercise 3.5 (★★). Repeat [Example 3.5](#) on the same grid with 8-connectivity: a diagonal move costs $\sqrt{2}$, entering a hatched cell multiplies the cost of the move by 3, and corner cutting is forbidden. Determine $\text{dist}(S, T)$ and an optimal path by hand, then run `dijkstra_trace` on `GridGraph(cells, 8, cost)` and count the expansions and the stale entries. Why does this instance produce stale entries when the 4-connected one did not?

Exercise 3.6 (★★). (a) Compute the true-distance heuristic $h^*(v) = \text{dist}(v, G)$ for every vertex of [Example 3.4](#) by running [Algorithm 3.2](#) by hand, and verify the consistency inequality $h^*(u) \leq c(u, v) + h^*(v)$ on every edge. (b) On the grid of [Example 3.5](#), explain why the forward table from T and the backward table to T differ, and name every cell where they differ. (c) A drone must reach T on that grid but is forbidden to enter the cell $(4, 2)$ at time step 6 (a vertex constraint of the kind used in [Chapter 9](#)). Explain why the table of [Figure 3.6](#) is still an admissible heuristic for the constrained space-time search, and why it is no longer exact.

Exercise 3.7 (★★★ Coding). This is the first half of the Week 1 exercise of the study plan ([Appendix A](#)); [Chapter 4](#) completes it. Implement Dijkstra's algorithm on an occupancy grid from scratch, with 4- and 8-connectivity, a successor function that forbids corner cutting, lazy deletion, the early exit and path reconstruction. (a) Visualize the closed set at several moments of a search on a 30×30 grid with a few walls, as in [Figure 3.3](#), and check that the settled cells at any moment are exactly the cells whose cost is at most the last popped key. (b) Count expansions against grid size for random grids with and without the early exit, and reproduce the shape of [Figure 3.5](#). (c) Add a function that returns the true-distance table for a goal and check, on random grids, that $h^*(v)$ equals the cost of a forward search from v for every free cell v . Keep the code: the next chapter turns it into A^* by changing the key of the heap.

Exercise 3.8 (★★★). Implement bidirectional Dijkstra as described in [Section 3.7.3](#). (a) Construct a small graph on which stopping at the first vertex reached by both waves returns a suboptimal path, and show that the stopping rule with μ returns the optimal one. (b) Prove that the rule is correct: when the two smallest keys sum to at least μ , no path cheaper than μ exists. (c) On the random grids of [Figure 3.5](#), measure the number of settled cells of bidirectional search relative to uniform-cost search for the far-corner target, and compare with the factor of one half predicted by the disk argument.

A* Search

Learning objectives

- Explain best-first search with $f = g + h$ and trace A* by hand on a small grid while keeping track of OPEN and CLOSED.
- State the exact definitions of admissible and consistent heuristics, prove that consistency implies admissibility, and prove that A* returns an optimal path.
- Choose a heuristic for a 4- or 8-connected grid (Manhattan, octile, Euclidean, Chebyshev), break ties well, and predict what a weight w does to cost and effort.
- Add time to the state: run A* in space-time with wait actions, vertex and edge constraints, a reservation table and a correct goal test.
- Implement grid A* and space-time A* in Python, visualize the open and closed sets, and recognize the failure modes that matter in a drone planner.

4.1 Why this matters for a drone swarm

Picture a delivery drone that has to cross a city district represented by a 100×100 grid of cells, a quarter of them blocked by buildings. Dijkstra's algorithm from [Chapter 3](#) finds the shortest route, but it does so by settling almost every free cell: in the experiment of [Section 4.7.3](#) it expands about 7 450 of the 7 500 free cells on such maps. The drone knows something Dijkstra ignores: the goal lies to the north-east, and a cell far to the south-west is unlikely to be on the way. A* puts this knowledge into the search. With the Manhattan distance as a guide it solves the same instances with about 750 expansions, ten times fewer, and it still guarantees the shortest route.

For a single query this is a convenience. For a swarm it is a necessity. The global planner of the hybrid architecture in [Chapter 24](#) is conflict-based search (CBS) or its bounded-suboptimal variant, enhanced CBS (ECBS; [Chapters 9](#) and [10](#)), and every node of their constraint tree calls a single-agent search for one drone under a set of constraints. A run on a few dozen drones can easily make thousands of such calls, and each of them is an A* search in *space-time*: the state of the drone is a cell together with the time at which it is there, the drone may wait, and it must avoid the cells and edges that other drones have reserved. The replanning layer of [Chapter 24](#) calls A* again whenever a local avoidance maneuver has pushed a drone off its nominal route, and the incremental planners of [Chapter 5](#) (lifelong planning A*, LPA*, and D* Lite) and the anytime planner of [Chapter 6](#) (anytime repairing A*, ARA*) are best understood as A* with extra bookkeeping. This chapter is therefore the foundation of everything that follows in [Chapters 5](#), [6](#) and [8](#) to [10](#).

The chapter proceeds as follows. [Section 4.2](#) gives the idea, [Section 4.3](#) the definitions of

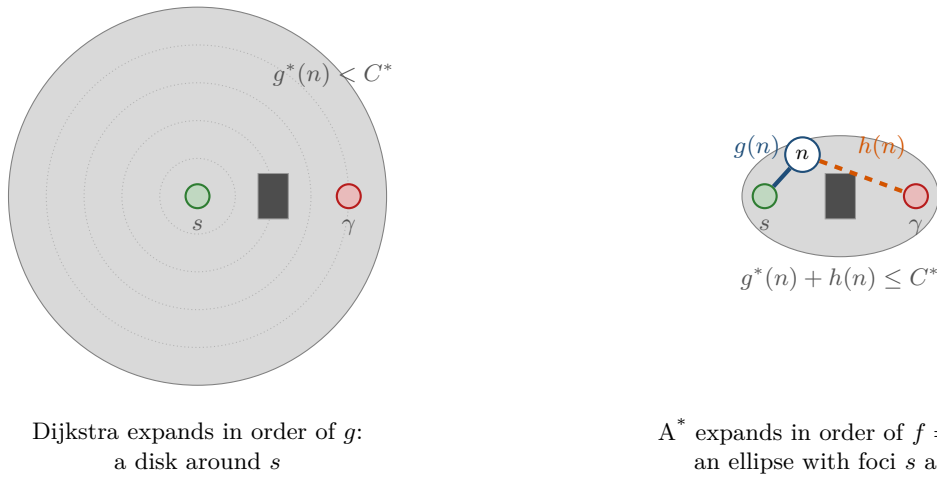


Figure 4.1. Left: Dijkstra expands nodes in order of g , so its closed set is a disk around the start s that reaches the goal γ only when it has covered every cheaper node. Right: A* expands nodes in order of $f = g + h$. With the straight-line distance as h , the nodes with $f \leq C^*$ lie inside an ellipse with foci s and γ when the space is obstacle-free, so that g^* is the straight-line distance; the small obstacle drawn here deforms the region but does not change the picture. The search never leaves it. For the node n , the solid segment is the known cost $g(n)$ and the dashed segment the estimate $h(n)$.

admissible and consistent heuristics and the standard grid heuristics. [Section 4.4](#) presents the graph-search version of A* with a closed set, [Section 4.5](#) runs it by hand on a 6×6 grid, and [Section 4.6](#) proves the optimality theorems that the Week 1 milestone of the study plan ([Appendix A](#)) asks you to be able to explain exactly. [Section 4.7](#) covers tie-breaking, weighted A* and an experiment on random grids. [Section 4.8](#) is a full treatment of space-time A*, the exact form of the search used by the multi-agent planners of [Chapters 8](#) and [9](#). [Section 4.9](#) discusses the implementation and [Section 4.10](#) its place in the drone system.

4.2 The idea in plain words

Dijkstra's algorithm always expands the node that is cheapest to reach from the start. Its frontier grows like a ripple in a pond: a disk around the start that spreads evenly in all directions until it touches the goal ([Figure 4.1](#), left). A* adds a second ingredient. For every node n it keeps the cost $g(n)$ of the best path found so far from the start *and* an estimate $h(n)$ of the cost that remains from n to the goal. Their sum

$$f(n) = g(n) + h(n) \quad (4.1)$$

estimates the cost of the cheapest solution that passes through n . A* is a **best-first search** on f : it always expands the node with the smallest f . The function h is called a **heuristic** because it is a rule of thumb, computed without looking at the obstacles, for example the straight-line distance to the goal. A node far from the goal has a large h and is expanded late or not at all. The frontier stops being a disk and becomes an elongated region pointing at the goal ([Figure 4.1](#), right).

Two questions decide whether this idea is safe. First, can a heuristic mislead the search into returning a path that is not the shortest? Yes, if it *overestimates*: a node on the true shortest path may then look worse than it is and never be expanded. Second, can a node be expanded twice? Yes, if the heuristic is not *consistent*, because a cheaper path to an already expanded node may show up later. The two conditions that rule out these problems, admissibility and consistency, are the heart of this chapter.

Key idea

Expand the node with the smallest $f = g + h$. If h never overestimates the remaining cost, the first goal that A* *pops* is reached by an optimal path, provided a closed node may be re-opened when a cheaper path to it turns up (Algorithm 4.1, line 15). If in addition h obeys the triangle inequality along every edge, no node is ever expanded twice, and A* is Dijkstra's algorithm with its effort focused toward the goal.

4.3 Problem statement and notation

4.3.1 Search problems and heuristics

Definition 4.1 (Heuristic search problem). A **heuristic search problem** consists of a finite directed graph $G = (V, E)$ with edge costs $c(u, v) \geq 0$, a start node $s \in V$, a non-empty set of goal nodes $V_\gamma \subseteq V$ (usually a single node γ), and a **heuristic function** $h: V \rightarrow \mathbb{R}_{\geq 0}$. The cost of a path is the sum of its edge costs. We write $g^*(n)$ for the cost of a cheapest path from s to n , $h^*: V \rightarrow \mathbb{R}_{\geq 0} \cup \{\infty\}$ for the cost of a cheapest path from n to a goal ($h^*(n) = \infty$ if no goal is reachable; the heuristic h itself is assumed finite everywhere), and $C^* = h^*(s)$ for the optimal solution cost.

During the search, $g(n)$ denotes the cost of the best path from s to n found *so far*; it satisfies $g(n) \geq g^*(n)$, with equality once the best path has been found. The successor function $\text{Succ}(n)$ returns the pairs $(n', c(n, n'))$ for all edges $(n, n') \in E$, exactly as in Chapter 3. For grids the nodes are cells (x, y) , x the column and y the row, with $(0, 0)$ in the bottom-left corner as in Chapter 2.

Definition 4.2 (Admissible heuristic). A heuristic h is **admissible** if it never overestimates the true cost to go:

$$0 \leq h(n) \leq h^*(n) \quad \text{for every } n \in V. \quad (4.2)$$

In particular $h(\gamma) = 0$ for every goal node γ .

Definition 4.3 (Consistent heuristic). A heuristic h is **consistent** (also called **monotone**) if $h(\gamma) = 0$ for every goal node γ and, for every edge $(n, n') \in E$,

$$h(n) \leq c(n, n') + h(n'). \quad (4.3)$$

Equation (4.3) is a triangle inequality: the estimate from n may not exceed the cost of first stepping to n' and estimating from there. An equivalent formulation is that f never decreases along an edge when g is computed along the path: $f(n') = g(n) + c(n, n') + h(n') \geq g(n) + h(n) = f(n)$. Consistency is the stronger property.

Lemma 4.4 (Consistency implies admissibility). *Every consistent heuristic is admissible. The converse is false.*

Proof. Let h be consistent and let $n \in V$. If no goal is reachable from n , then $h^*(n) = \infty$ and Equation (4.2) holds. Otherwise let $n = n_0, n_1, \dots, n_k = \gamma$ be a cheapest path from n to a goal, so that $h^*(n) = \sum_{i=0}^{k-1} c(n_i, n_{i+1})$. Applying Equation (4.3) to each edge in turn,

$$h(n_0) \leq c(n_0, n_1) + h(n_1) \leq c(n_0, n_1) + c(n_1, n_2) + h(n_2) \leq \dots \leq \sum_{i=0}^{k-1} c(n_i, n_{i+1}) + h(\gamma) = h^*(n),$$

because $h(\gamma) = 0$. For the converse, take the graph with nodes s, a, b, γ and edges $s \rightarrow a$ (cost 1), $s \rightarrow b$ (cost 3), $a \rightarrow b$ (cost 1), $b \rightarrow \gamma$ (cost 2). The heuristic $h(a) = 3$, $h(s) = h(b) = h(\gamma) = 0$ is admissible, since $h^*(a) = 3$, but $h(a) = 3 > c(a, b) + h(b) = 1$ violates Equation (4.3). We return to this graph in Section 4.6. $\square$

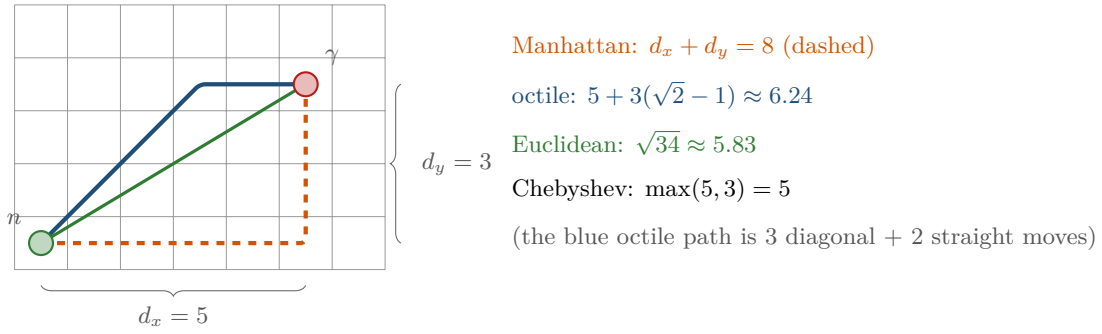


Figure 4.2. The four grid heuristics from the cell n to the goal γ with $d_x = 5$ and $d_y = 3$. The Manhattan path (orange, dashed) has length 8; the octile path (blue) uses three diagonal moves and two straight ones, length $5 + 3(\sqrt{2} - 1) \approx 6.24$; the Euclidean distance (green) is $\sqrt{34} \approx 5.83$; the Chebyshev distance counts only the longer side, 5. On a 4-connected grid all four are admissible; on an 8-connected grid the Manhattan distance overestimates every diagonal move.

4.3.2 Heuristics for grids

Let $n = (x_n, y_n)$ be a cell and $\gamma = (x_\gamma, y_\gamma)$ the goal, and write $d_x = |x_n - x_\gamma|$ and $d_y = |y_n - y_\gamma|$. Four distances are in everyday use (Figure 4.2):

$$h_M(n) = d_x + d_y \quad \text{Manhattan distance,} \quad (4.4)$$

$$h_O(n) = \max(d_x, d_y) + (\sqrt{2} - 1) \min(d_x, d_y) \quad \text{octile distance,} \quad (4.5)$$

$$h_E(n) = \sqrt{d_x^2 + d_y^2} \quad \text{Euclidean distance,} \quad (4.6)$$

$$h_C(n) = \max(d_x, d_y) \quad \text{Chebyshev distance.} \quad (4.7)$$

The Manhattan distance is the length of a cheapest path on an empty 4-connected grid, where every move costs 1. The octile distance is the length of a cheapest path on an empty 8-connected grid, where a straight move costs 1 and a diagonal move $\sqrt{2}$: use $\min(d_x, d_y)$ diagonal moves and then $\max(d_x, d_y) - \min(d_x, d_y)$ straight moves. The Chebyshev distance is the same count when diagonal moves also cost 1. On an 8-connected grid we use the successor rule of Definition 2.5 throughout: a diagonal move is allowed only when both orthogonal neighbors it passes are free, so a drone of positive size never clips an obstacle corner. Forbidding those moves only lengthens paths, so it does not endanger the admissibility of any heuristic below, and `grid_successors()` in `ch04_astar.py` enforces it in every search of this chapter, including the space-time search of Section 4.8. Each of the four is the norm of the displacement vector (ℓ_1 , octagonal, ℓ_2 and ℓ_∞ norm), so each satisfies the triangle inequality.

Proposition 4.5 (Metric heuristics are consistent). *Let d be a metric on V such that $d(n, n') \leq c(n, n')$ for every edge $(n, n') \in E$, and let $h(n) = d(n, \gamma)$. Then h is consistent, hence admissible.*

Proof. $h(\gamma) = d(\gamma, \gamma) = 0$, and by the triangle inequality $h(n) = d(n, \gamma) \leq d(n, n') + d(n', \gamma) \leq c(n, n') + h(n')$. $\square$

To apply Proposition 4.5 we only have to compare each distance with the cost of a single move. A straight move costs 1, and all four distances assign 1 to it. A diagonal move on an 8-connected grid costs $\sqrt{2}$: the octile and Euclidean distances assign $\sqrt{2}$, the Chebyshev distance 1, but the Manhattan distance assigns $2 > \sqrt{2}$. Table 4.1 summarizes the result. The Manhattan distance is the classic example of a heuristic that is perfect for one grid and wrong for another.

Table 4.1. Grid heuristics and their properties. “Exact” means equal to h^* on an obstacle-free grid. A heuristic that is exact on empty grids stays admissible with obstacles, because obstacles can only lengthen paths.

Heuristic	4-connected (cost 1)	8-connected (costs 1, $\sqrt{2}$)
Manhattan $d_x + d_y$	exact on empty grids; consistent	not admissible (diagonal: $2 > \sqrt{2}$)
Octile $\max + (\sqrt{2} - 1) \min$	not exact; consistent (below Manhattan)	exact on empty grids; consistent
Euclidean $\sqrt{d_x^2 + d_y^2}$	consistent	consistent; dominated by octile
Chebyshev $\max(d_x, d_y)$	consistent; weakest of the four	consistent; exact if diagonals cost 1

The Manhattan distance on an 8-connected grid

This is the most common inadmissible heuristic in robotics code, and it is invisible: the search runs, returns a path, and the path is simply too long. The reason is one line of Table 4.1: a diagonal move costs $\sqrt{2} \approx 1.41$ but changes $d_x + d_y$ by 2, so $h_M(n) > c(n, n') + h_M(n')$ and, worse, h_M can exceed h^* . Theorem 4.9 then does not apply. In the self-test of `ch04_astar.py`, A* with the Manhattan heuristic on 8-connected random grids returns a strictly longer path on 6 of 40 instances — one in seven, silently. The bug is easy to introduce when a 4-connected planner is extended to diagonal moves and nobody revisits the heuristic. Use the octile distance Equation (4.5) on 8-connected maps, and test optimality against a Dijkstra run on random instances, as the self-test does.

None of the four sees obstacles, so with walls in the way they all underestimate, sometimes badly: the worked example of Section 4.5 shows the search walking into a dead end because the Manhattan distance does not know about a wall. The exact heuristic h^* itself can be computed by a backward Dijkstra search from the goal, as in Chapter 3. That costs a full search and is pointless for a single query, but it becomes the heuristic of choice in space-time (Section 4.8), where the same static distances serve every time layer and every constrained re-run of the search.

4.4 The algorithm

Algorithm 4.1 is A* as a *graph search*: it remembers the nodes it has expanded in the set CLOSED so that a node is not expanded again unless a strictly cheaper path to it is found. The nodes that have been generated but not yet expanded form the frontier OPEN, a priority queue keyed on f . The queue is *lazy*: when the g of a node improves, the algorithm pushes a fresh entry instead of searching the queue for the old one, and the old entry is discarded when it is popped and found to be stale. The weight w is 1 for plain A*; Section 4.7 discusses $w > 1$.

Walkthrough. Line 3 seeds the frontier with the start, whose f is $h(s)$. The loop repeatedly takes the most promising entry (line 5). The key is the pair $(f, -g)$, compared lexicographically: smaller f first and, among equal f , larger g first. This tie-breaking rule is not needed for correctness, but Section 4.7 shows that it can make a tenfold difference in effort. Line 7 implements lazy deletion: a node may have several entries in OPEN, one per improvement of its g ; only the entry carrying the current value is valid. The goal test happens when the goal is *popped* (line 9), not when it is first generated. This detail is what makes the optimality proof of Section 4.6 work; testing at generation time returns the first path found to the goal, which need not be the cheapest. Line 10 records the expansion. The inner loop relaxes every outgoing edge (line 13) exactly as Dijkstra’s algorithm does; the only differences from Chapter 3 are

Algorithm 4.1. A* graph search with a closed set and a lazy priority queue

Input: graph $G = (V, E)$ with costs c , start s , goal test, heuristic h , weight $w \geq 1$
 ($w = 1$ for A*)

Output: a cheapest path from s to a goal, or *failure*

```

1 Function AStar( $G, s, isGoal, h, w$ ):
2    $g(s) \leftarrow 0$ ;  $parent(s) \leftarrow \mathbf{nil}$ ;  $g(n) \leftarrow \infty$  for all other nodes (implicitly)
3   OPEN  $\leftarrow$  priority queue holding  $s$  with key  $(wh(s), 0)$ ; CLOSED  $\leftarrow \emptyset$ 
4   while OPEN  $\neq \emptyset$  do
5      $(n, g_n) \leftarrow$  pop the entry with the smallest key  $(f, -g)$  from OPEN
6     if  $g_n > g(n)$  then
7       continue                                // stale entry: a cheaper path to  $n$  was found later
8     if  $n$  is a goal then
9       return ReconstructPath( $parent, n$ )
10    add  $n$  to CLOSED
11    foreach  $(n', c(n, n')) \in Succ(n)$  do
12       $g' \leftarrow g(n) + c(n, n')$ 
13      if  $g' < g(n')$  then
14        if  $n' \in CLOSED$  then
15          remove  $n'$  from CLOSED                // re-open  $n'$ ; never happens if  $h$  is
16          consistent
17           $g(n') \leftarrow g'$ ;  $parent(n') \leftarrow n$ 
18          push  $(n', g')$  into OPEN with key  $(g' + wh(n'), -g')$ 
19  return failure

```

the key of the priority queue and line 15, which re-opens a closed node when a cheaper path to it appears. With a consistent heuristic this line is never executed (Theorem 4.10), and many implementations drop it together with the ability to correct a closed node; with an admissible but inconsistent heuristic it is required for optimality (Theorem 4.9). The function `ReconstructPath` follows the parent pointers back from the goal to s and reverses the list.

Setting $h \equiv 0$ turns the key into $(g, -g)$ and Algorithm 4.1 into Dijkstra's algorithm with early exit; every property of Chapter 3 is the special case $h = 0$ of a property in this chapter.

4.5 A worked example

Example 4.6 (A* on a 6×6 grid). Figure 4.3 shows a 6×6 4-connected grid with two walls: the cells $(2, 0)$ to $(2, 3)$ and $(4, 1)$ to $(4, 4)$ are blocked. The start is $s = (0, 0)$ in the bottom-left corner and the goal $\gamma = (5, 2)$. Every move costs 1 and the heuristic is the Manhattan distance Equation (4.4), so $h(s) = 5 + 2 = 7$. Ties in f are broken toward the larger g , and among equal keys the entry pushed first is popped first. The instance is `worked_example_grid()` in `ch04_astar.py`, and every number below is produced by its self-test.

Table 4.2 lists the first ten expansions. Step 1 expands the start $(0, 0)$; steps 2–4 then walk up the column $x = 1$: the three cells expanded there have $f = 7$, because moving toward the goal decreases h by exactly the move cost. At $(1, 2)$ the search would like to continue east, but $(2, 2)$ is blocked; going north to $(1, 3)$ moves away from the goal row, and f rises to 9. A* therefore turns to the remaining cells with $f = 7$, the column $x = 0$ (steps 5 and 6), before

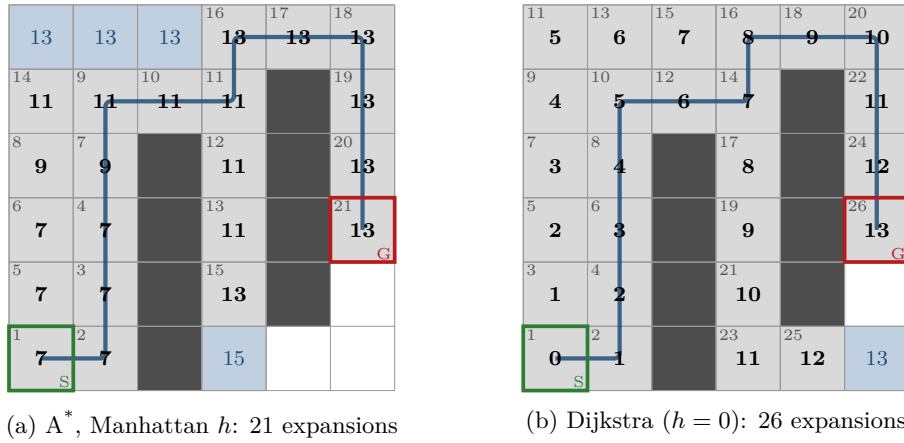


Figure 4.3. Example 4.6 after termination. Gray cells are CLOSED (expanded), blue cells are still in OPEN; the bold number is f at the time of expansion (for Dijkstra $f = g$) and the small gray number in the corner is the expansion order. The blue line is the returned path of cost 13. (a) A* with the Manhattan heuristic expands 21 cells; note the three wasted expansions (3,3), (3,2), (3,1) below the junction at (3,4), where the heuristic cannot see the wall at $x = 4$. (b) Dijkstra's algorithm ($h = 0$) expands 26 cells, five more than A*: the top-row cells (0,5), (1,5), (2,5) and the bottom-row cells (3,0), (4,0), which A* either leaves in OPEN or never generates.

it accepts any cell with $f = 9$ (steps 7 and 8). From step 9 on the frontier has $f = 11$. The search passes the first wall at (2,4) and, led by the decreasing h , dives down the column $x = 3$ as far as (3,1), where the second wall stops it: the three cells (3,3), (3,2), (3,1) below the junction at (3,4) are a dead end that the Manhattan distance cannot foresee. The cell (0,4) carries $f = 11$ and (3,1) carries $f = 13$, so (0,4) is popped first and (3,1) one pop later; the expansion order is $\dots, (3,4), (3,3), (3,2), (0,4), (3,1), (3,5), \dots$. From (3,1) on the frontier is at $f = 13$: the search crosses the top row (3,5), (4,5), (5,5) and descends to the goal, which is popped at step 21 with $g = 13$. At that moment three cells with $f = 13$ but smaller g are still in OPEN together with (3,0) at $f = 15$; none of them is ever expanded. Dijkstra's algorithm needs 26 expansions for the same instance, and A* with ties broken toward the *smaller* g needs 24. On the 8-connected version of the grid, A* with the octile heuristic finds a path of cost $9 + 2\sqrt{2} \approx 11.83$ with 24 expansions.

Two things in the table deserve a second look. In step 4 the cell (0,2) enters OPEN with $g = 3$ (through (1,2)) and $f = 9$; in step 5 the expansion of (0,1) finds the cheaper route with $g = 2$, the entry is replaced and f drops to 7. This is the relaxation of line 13, and it happens while (0,2) is still open, so no re-opening is involved. Second, the f of the expanded nodes reads 7, 7, 7, 7, 7, 7, 9, 9, 11, 11, $\dots$, 13: it never decreases. This is not luck; Theorem 4.10 shows that it holds for every consistent heuristic.

4.6 Properties: correctness, optimality, complexity

Throughout this section G is finite, costs are non-negative and Algorithm 4.1 runs with $w = 1$. The proofs follow Hart, Nilsson and Raphael [62, 63] in the form given by Russell and Norvig [142]; Pearl [126] treats the general case.

Table 4.2. The first ten expansions of Algorithm 4.1 on Example 4.6. The last column lists the cells in OPEN after the step with their current f ; a cell whose g improves, such as $(0, 2)$ in step 5, keeps a single valid entry.

Step	Expanded n	g	h	f	OPEN after the step (cell: f)
1	(0, 0)	0	7	7	(0, 1):7, (1, 0):7
2	(1, 0)	1	6	7	(0, 1):7, (1, 1):7
3	(1, 1)	2	5	7	(0, 1):7, (1, 2):7
4	(1, 2)	3	4	7	(0, 1):7, (0, 2):9, (1, 3):9
5	(0, 1)	1	6	7	(0, 2):7, (1, 3):9
6	(0, 2)	2	5	7	(0, 3):9, (1, 3):9
7	(1, 3)	4	5	9	(0, 3):9, (1, 4):11
8	(0, 3)	3	6	9	(0, 4):11, (1, 4):11
9	(1, 4)	5	6	11	(0, 4):11, (1, 5):13, (2, 4):11
10	(2, 4)	6	5	11	(0, 4):11, (1, 5):13, (2, 5):13, (3, 4):11

4.6.1 Termination and completeness

Proposition 4.7 (Termination and completeness). *Algorithm 4.1 terminates. It returns a path if and only if some goal is reachable from s .*

Proof. Every push into OPEN strictly decreases $g(n')$ for some node n' (line 13). The path recorded by the parent pointers of n' is simple: if it returned to n' , the non-negative costs along the cycle would give $g(n) + c(n, n') \geq g(n')$, contradicting the strict decrease. A finite graph has finitely many simple paths, so each node's g can decrease only finitely often, there are finitely many pushes, and the loop ends. If no goal is reachable, no goal is ever popped and the algorithm returns *failure* when OPEN runs empty. If a goal is reachable, Lemma 4.8 below shows that OPEN is non-empty until a goal is expanded, so the algorithm returns a path. $\square$

4.6.2 Optimality with an admissible heuristic

The proofs rest on one invariant. It concerns a cheapest path to an arbitrary node m ; we apply it with m a goal and later with m an ordinary node.

Lemma 4.8 (An optimal-path node is always open). *Let $P = (s = n_0, n_1, \dots, n_k = m)$ be a cheapest path from s to a node m . At any time before Algorithm 4.1 expands m with $g(m) = g^*(m)$, OPEN contains a node n_i of P with $g(n_i) = g^*(n_i)$.*

Proof. Every prefix of a cheapest path is a cheapest path, so $g^*(n_{i+1}) = g^*(n_i) + c(n_i, n_{i+1})$. Let i be the smallest index such that n_i has not yet been expanded while holding $g(n_i) = g^*(n_i)$; the index exists because $m = n_k$ qualifies. If $i = 0$, then s has not been expanded (it always holds $g(s) = 0 = g^*(s)$), so its initial entry is still in OPEN. If $i > 0$, then n_{i-1} was expanded with the optimal value, and at that moment line 13 set $g(n_i) \leq g^*(n_{i-1}) + c(n_{i-1}, n_i) = g^*(n_i)$. Since every g value is the cost of an actual path, $g(n_i) \geq g^*(n_i)$, hence $g(n_i) = g^*(n_i)$ from then on. The push that established this value (either at that relaxation, which also re-opened n_i if it was closed, or earlier) put an entry with $g^*(n_i)$ into OPEN. That entry is never stale, and by the choice of i it has not been popped for expansion; so it is still in OPEN. $\square$

Theorem 4.9 (Optimality of A*; Hart, Nilsson and Raphael 1968). *If h is admissible and a goal is reachable from s , then Algorithm 4.1 (with re-opening, line 15) returns a path of cost C^* .*

Proof. By [Proposition 4.7](#) the algorithm returns a path to some goal γ' ; its cost is $g(\gamma')$ at the moment γ' is popped. Suppose $g(\gamma') > C^*$. Let $P = (s, \dots, \gamma)$ be an optimal solution, of cost C^* . The goal γ has not been expanded with $g(\gamma) = C^*$, since the algorithm would have stopped there. By [Lemma 4.8](#), at the moment γ' is popped OPEN contains a node n_i of P with $g(n_i) = g^*(n_i)$. The rest of P from n_i to γ costs $C^* - g^*(n_i)$ and is one particular path to a goal, so $h^*(n_i) \leq C^* - g^*(n_i)$, and admissibility gives

$$f(n_i) = g^*(n_i) + h(n_i) \leq g^*(n_i) + h^*(n_i) \leq C^*.$$

But $f(\gamma') = g(\gamma') + h(\gamma') = g(\gamma') > C^* \geq f(n_i)$, so line 5 would have popped n_i , or some other entry with a smaller key, before γ' . This contradiction shows $g(\gamma') \leq C^*$, and $g(\gamma') \geq C^*$ because it is the cost of a real path. $\square$

The proof used admissibility only in one place: to bound $f(n_i)$ by C^* . It used re-opening, through [Lemma 4.8](#), to make sure that the optimal-path node is in OPEN even if it had been expanded earlier with a worse g . With a consistent heuristic the second precaution is unnecessary.

4.6.3 Consistent heuristics: no node is expanded twice

Theorem 4.10 (Consistent heuristics). *Let h be consistent. Then in [Algorithm 4.1](#):*

- (i) *whenever a node n is expanded, $g(n) = g^*(n)$; hence line 15 is never executed and every node is expanded at most once;*
- (ii) *the f values of the expanded nodes, in order of expansion, are non-decreasing.*

Proof. (i) Suppose n is popped for expansion with $g(n) > g^*(n)$. Let $P = (s, \dots, n)$ be a cheapest path to n . By [Lemma 4.8](#) with $m = n$, OPEN contains a node n_i of P with $g(n_i) = g^*(n_i)$, and $n_i \neq n$ because $g(n)$ is not optimal. Applying [Equation \(4.3\)](#) along the edges of P from n_i to n , exactly as in the proof of [Lemma 4.4](#), gives $h(n_i) \leq (g^*(n) - g^*(n_i)) + h(n)$. Therefore

$$f(n_i) = g^*(n_i) + h(n_i) \leq g^*(n) + h(n) < g(n) + h(n) = f(n),$$

and line 5 would have chosen n_i instead of n : a contradiction. So every expanded node carries its optimal g , no later path can be cheaper, and a closed node is never re-opened.

(ii) Let n be expanded and let n' be the node expanded next. When n was popped, every other entry in OPEN had $f \geq f(n)$. The expansion of n changes OPEN in two ways: it pushes successors n'' with $f(n'') = g(n) + c(n, n'') + h(n'') \geq g(n) + h(n) = f(n)$ by consistency, and it leaves the other entries unchanged. Hence every entry in OPEN has $f \geq f(n)$ and in particular $f(n') \geq f(n)$. $\square$

Corollary 4.11 (Which nodes A* expands). *With a consistent heuristic, A* expands every node n with $g^*(n) + h(n) < C^*$, expands no node with $g^*(n) + h(n) > C^*$, and may or may not expand nodes with $g^*(n) + h(n) = C^*$ depending on tie-breaking.*

Proof. By [Theorem 4.10\(ii\)](#) and $f(\gamma) = C^*$ at termination, no expanded node has $f > C^*$. Conversely let $g^*(n) + h(n) < C^*$ and let P be a cheapest path to n . Along P the value $g^* + h$ is non-decreasing (consistency), so it is below C^* at every node of P . By induction along P , each node of P is generated with its optimal g by the expansion of its predecessor and, having $f < C^* = f(\gamma)$, is popped before the goal. $\square$

[Theorem 4.10\(i\)](#) is the reason why the closed set can be a plain “expanded” flag when h is consistent, and part (ii) why the trace in [Table 4.2](#) shows non-decreasing f ; [Corollary 4.11](#) is why the f values in [Figure 4.3](#) never exceed 13.

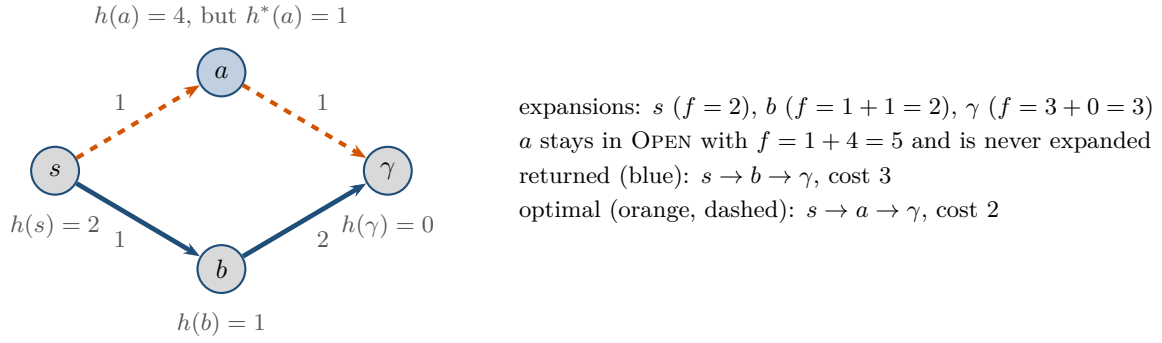


Figure 4.4. An inadmissible heuristic misleads A*. The value $h(a) = 4$ overestimates $h^*(a) = 1$, so a looks worse than the detour through b and is never expanded (blue fill: still in OPEN). A* returns the path of cost 3 although a path of cost 2 exists.

4.6.4 What goes wrong without the conditions

Inadmissible heuristics. Figure 4.4 shows the smallest kind of failure. The heuristic is wrong at a single node, a , where it says 4 while the true cost to go is 1. The search expands s , then b (with $f = 2$), then the goal with $g = 3$, and it stops. The node a sits in OPEN with $f = 5$ and the optimal path $s \rightarrow a \rightarrow \gamma$ of cost 2 is never found. Nothing in the run signals the error: an overestimate makes A* *confidently* wrong, which is why the Manhattan heuristic on an 8-connected grid (Table 4.1) is a real bug and not a matter of taste.

Admissible but inconsistent heuristics. Take the graph from the proof of Lemma 4.4: $s \rightarrow a$ (1), $s \rightarrow b$ (3), $a \rightarrow b$ (1), $b \rightarrow \gamma$ (2), with $h(a) = 3$ and $h = 0$ elsewhere. A* expands s ($f = 0$) and pushes a with $f = 1 + 3 = 4$ and b with $f = 3 + 0 = 3$. It expands b first and pushes γ with $g = 5$. Next it expands a and finds b with $g = 2 < 3$: b is closed, and line 15 re-opens it. The expansion of b (now $f = 2$) improves γ to $g = 4$, and the goal is returned with the optimal cost 4. The sequence of f values, 0, 3, 4, 2, 4, is not monotone, and b is expanded twice. If re-opening is disabled, the same run returns the path $s \rightarrow b \rightarrow \gamma$ of cost 5. The self-test of `ch04_astar.py` reproduces both runs.

4.6.5 Complexity

With a consistent heuristic every node is expanded at most once and every edge relaxed at most once, so Algorithm 4.1 performs at most $|E|$ pushes and $|V|$ expansions; with a binary heap the running time is $\mathcal{O}((|V| + |E|) \log |V|)$ and the memory $\mathcal{O}(|V| + |E|)$, the same bounds as for Dijkstra’s algorithm in Chapter 3. The bounds are worst cases. What decides the actual effort is the number of expanded nodes, that is, by Corollary 4.11, the number of nodes with $g^*(n) + h(n) < C^*$. With $h = 0$ this is every node cheaper than the goal; with the exact heuristic $h = h^*$ it is empty, and with larger- g -first tie-breaking A* then expands only the nodes of one optimal path. Between these extremes the effort grows with the error $h^* - h$: on a grid with obstacles every detour that the heuristic does not foresee adds a band of cells with $f < C^*$. On an *implicit* graph, one whose nodes are generated on demand and whose size is exponential in the solution depth, the $|V|$ in the bound is that exponential number, and the heuristic is what decides whether the search is practical. Pearl [126] makes this precise: A* still expands exponentially many nodes unless the error of the heuristic grows more slowly than the true cost, roughly $h^*(n) - h(n) = \mathcal{O}(\log h^*(n))$ — which is why Section 4.7 buys speed by giving up optimality rather than by sharpening h . All of this counts *distinct* nodes: if h is admissible but not consistent, line 15 may fire again and again, and Martelli [113] builds graphs on which A* re-expands nodes exponentially often in the number of nodes, so

the $\mathcal{O}(|V|)$ expansion count above needs [Theorem 4.10\(i\)](#). The grids of this book are explicit and small enough that this does not bite, but the space-time graphs of [Section 4.8](#) and the constraint trees of [Chapter 9](#) are where it starts to.

4.6.6 Heuristic dominance

Definition 4.12 (Dominance). A heuristic h_2 **dominates** h_1 if $h_2(n) \geq h_1(n)$ for every node n . If both are admissible, h_2 is the *more informed* of the two.

Theorem 4.13 (More informed, fewer expansions). *Let h_1 and h_2 be consistent and let h_2 dominate h_1 . Every node that A^* with h_2 expands with $f < C^*$ is also expanded by A^* with h_1 . Consequently the number of expansions with h_2 is at most the number with h_1 , up to nodes with $f = C^*$.*

Proof sketch. By [Corollary 4.11](#), A^* with h_2 expands with $f < C^*$ exactly the nodes with $g^*(n) + h_2(n) < C^*$. For such a node, $g^*(n) + h_1(n) \leq g^*(n) + h_2(n) < C^*$, so [Corollary 4.11](#) applied to h_1 says that it is expanded there too. Nodes with $f = C^*$ escape the argument: whether they are expanded depends on tie-breaking, and h_2 may expand a few of them that h_1 skips. Pearl [126] and Dechter and Pearl [33] extend the statement to admissible heuristics. Their optimality result is a statement about a specific class: among the admissible algorithms that search the same graph and are *equally informed*, that is, given the same heuristic and no other information about the instance, no algorithm expands a strict subset of the nodes that A^* with a consistent heuristic surely expands. It does not say that A^* is the fastest possible planner for grids, which it is not ([Section 4.7.4](#)). $\square$

On grids the theorem orders the heuristics of [Table 4.1](#): on an 8-connected grid the octile distance dominates the Euclidean distance, which dominates the Chebyshev distance, and the self-test of `ch04_astar.py` checks the theorem on random grids. The theorem also shows why the maximum of several admissible heuristics ([Exercise 4.5](#)) is never worse than any of them, and it gives the rule of thumb for designing heuristics: relax the problem (drop the obstacles, drop a constraint), solve the relaxed problem exactly, and use its cost. That is precisely how the Manhattan and octile distances arise.

4.7 Variants and extensions

4.7.1 Tie-breaking

[Corollary 4.11](#) leaves the nodes with $f = C^*$ to the tie-breaking rule, and near the goal there are many of them: every node on an optimal path has $g^*(n) + h^*(n) = C^*$, and with an exact heuristic all of them tie. On an obstacle-free grid the Manhattan distance is exact, and every cell of the 20×20 grid in [Figure 4.5](#) lies on some optimal path from the bottom-left to the top-right corner: all 400 cells have $f = 38$. Which of them are expanded is entirely up to the tie-breaking rule. Preferring the larger g (equivalently, the smaller h) means: among nodes that look equally good, take the one that is furthest along. The search then behaves like a depth-first dive along one optimal path and expands $2 \cdot 20 - 1 = 39$ cells. Preferring the smaller g , or breaking ties first-in-first-out without looking at g , expands all 400 cells before the goal is popped. This is why [Algorithm 4.1](#) uses the key $(f, -g)$ and why the implementation in [Section 4.9](#) adds a counter as a third component: the counter keeps the order deterministic and stops Python from comparing nodes when the first two components tie. In the worked example the rule saves three of 24 expansions; in the last stretch of any search with a good heuristic it saves far more.

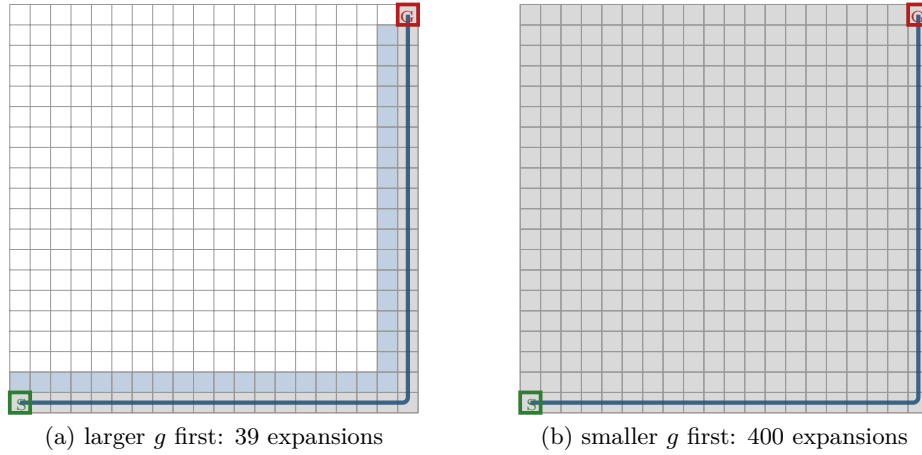


Figure 4.5. Tie-breaking on an empty 20×20 grid with the Manhattan heuristic. Every cell has $f = 38 = C^*$. (a) Breaking ties toward the larger g , A^* expands only the 39 cells of one optimal path. (b) Breaking ties toward the smaller g , it expands all 400 cells. Both runs return a path of cost 38.

4.7.2 Weighted A^*

Pohl [132] proposed to inflate the heuristic: **weighted A^*** runs Algorithm 4.1 with the key

$$f_w(n) = g(n) + w h(n), \quad w \geq 1. \quad (4.8)$$

For $w = 1$ this is A^* ; as $w \rightarrow \infty$ the g term becomes irrelevant and the search turns into *greedy best-first search*, which always expands the node that seems closest to the goal. In between, a larger w makes the search more greedy: it trusts the heuristic more, expands fewer nodes and accepts a longer path. The price is bounded.

Theorem 4.14 (Bounded suboptimality of weighted A^*). *If h is admissible and a goal is reachable, weighted A^* with re-opening returns a path of cost at most $w \cdot C^*$.*

Proof. Lemma 4.8 did not use the heuristic at all, so it holds for the key Equation (4.8): when the goal γ' is popped, OPEN contains a node n_i of an optimal path P with $g(n_i) = g^*(n_i)$. As in the proof of Theorem 4.9, $h(n_i) \leq h^*(n_i) \leq C^* - g^*(n_i)$, hence

$$f_w(n_i) = g^*(n_i) + w h(n_i) \leq g^*(n_i) + w (C^* - g^*(n_i)) \leq w C^*,$$

where the last step uses $w \geq 1$ and $g^*(n_i) \geq 0$. The goal was popped before n_i , so $g(\gamma') = f_w(\gamma') \leq f_w(n_i) \leq w C^*$. $\square$

Note that wh is in general not admissible and not consistent, even when h is, so weighted A^* may re-open nodes and Theorem 4.10 does not apply. Likhachev, Gordon and Thrun [108] showed that, *when h is consistent*, the bound $w C^*$ survives even when re-opening is switched off and each node is expanded at most once per iteration; the assumption matters, because with a merely admissible h the argument breaks. That observation is the basis of ARA^* in Chapter 6, which decreases w step by step while reusing the search. The focal search of ECBS in Chapter 10 obtains the same guarantee by a different mechanism.

4.7.3 An experiment on random grids

Figure 4.6 compares the effort of the searches in this chapter on random grids with 25% obstacles (script `gen_ch04_expansions.py`, ten instances per size, start and goal in opposite corners). Three observations. First, Dijkstra's algorithm expands nearly every free cell (7452

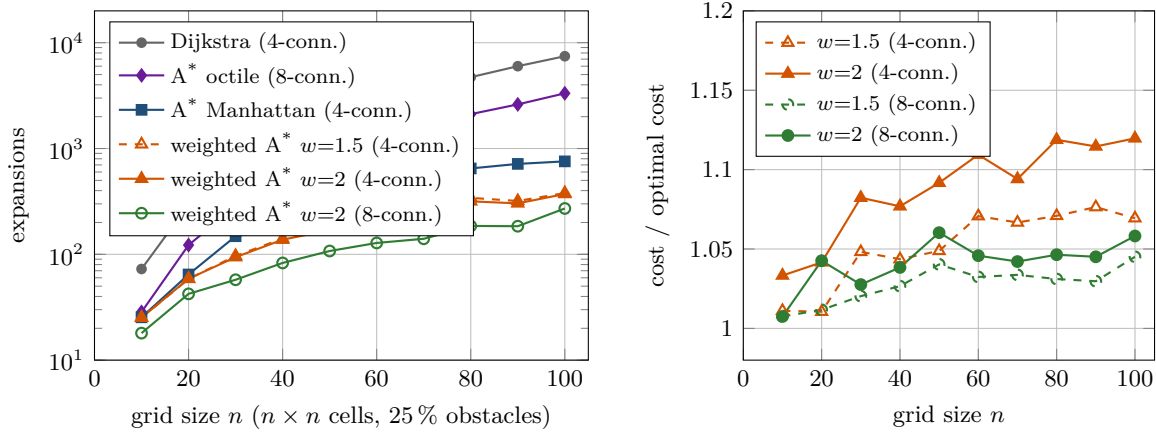


Figure 4.6. Left: mean number of expansions (log scale) on random $n \times n$ grids; Dijkstra’s algorithm grows with the number of free cells, A* grows much more slowly, and the weight w cuts the effort again. Right: the mean ratio between the cost of the path returned by weighted A* and the optimal cost; the guaranteed bounds are 1.5 and 2, the observed ratios stay below 1.13.

of 7513 at $n = 100$), while A* with the Manhattan heuristic on the 4-connected grid expands 756 on average: the heuristic removes about 90% of the work. Second, on the 8-connected grid A* with the octile heuristic still expands 3324 cells. Diagonal moves make detours around obstacles cheap, so many cells have $g^* + h < C^*$; the error $h^* - h$, not the connectivity, is what the search pays for. Third, weighted A* with $w = 2$ needs 372 expansions on the 4-connected and 271 on the 8-connected grid, while its paths are on average only 12% and 6% longer than optimal, far inside the guaranteed factor of 2; with $w = 1.5$ the average excess is 7% and 5%. Bounded suboptimality is cheap on these maps, which is why ECBS (Chapter 10) is so much faster than CBS.

4.7.4 Other relatives

A* has a large family. LPA* and D* Lite (Chapter 5) reuse the search when edge costs change; ARA* (Chapter 6) is weighted A* with a decreasing weight; focal search (Chapter 10) keeps a second queue of nodes whose f is within a factor w of the best and picks among them by a different criterion. Jump point search [61] exploits the symmetry of uniform grids to skip most cells, and safe interval path planning [131] compresses the time dimension of the search described next. All of them keep the structure of Algorithm 4.1; what changes is the key, the successor function or what is remembered between searches.

4.8 Space-time A*

Every search so far assumed a static world: an obstacle blocks a cell, and it blocks it for ever. A drone that shares its airspace with other drones faces a different problem. The cell in front of it may be free now and occupied in three seconds, when a neighbor flies through it; a corridor may be usable in one direction only, because a drone is coming the other way. The planner that resolves such interactions is a **multi-agent path finding** (MAPF) algorithm, and the standard formulation is the one of Stern et al. [161], stated in full in Chapter 7: agents move on a shared graph in discrete time steps, each agent may move to a neighbor or wait, and no two agents may occupy the same vertex at the same time step or swap places along an edge. Chapters 8 to 10 build three planners on top of it. All three share one component, and it is an A* search: plan for *one* agent, in time, subject to a list of forbidden (cell, time) and

(edge, time) pairs. That search is **space-time A***, the subject of this section.

4.8.1 The state is a cell and a time

Definition 4.15 (Time-expanded graph and space-time state). Recall the space-time state and the time-expanded graph of [Definitions 2.10](#) and [2.11](#); here every action, move or wait, costs one time step, so time and cost coincide. Let $G = (V, E)$ be the grid graph of [Definition 2.5](#). A **space-time state** is a pair (v, t) with $v \in V$ and $t \in \{0, 1, 2, \dots\}$: the agent is in cell v at time step t . From (v, t) the agent may either **wait**, reaching $(v, t + 1)$, or move along an edge $(v, v') \in E$, reaching $(v', t + 1)$. Every action costs one time step. The graph on these states, with these edges, is the **time-expanded graph** of G .

Three consequences of [Definition 4.15](#) shape the search.

The graph is a directed acyclic graph. Every edge increases t by exactly one, so no cycle exists, and the cost of a path from $(s, 0)$ to (v, t) is exactly t : $g(v, t) = t$ for every state, no matter which route reached it. A state therefore has a single possible g , and the first time the search generates it, it is already the cheapest. Relaxation (line 13 of [Algorithm 4.1](#)) is replaced by plain duplicate detection: a state that has been generated is never generated again, and no node is ever re-opened. The two theorems of [Section 4.6](#) apply as they stand, with the trivial consistency check below.

Time is the cost. Cost and time are the same quantity here, which is why a diagonal move on an 8-connected grid also costs one step rather than $\sqrt{2}$: a drone that flies diagonally takes one time slot, like every other maneuver. If you need geometric length as well, carry it as a second number and optimize time first; [Chapter 9](#) discusses the trade-off, and safe interval path planning [\[131\]](#) avoids the discretization altogether.

The state space is infinite unless you bound it. Waiting is always allowed, so from every state there is an infinite chain of wait actions. The time horizon of [Section 4.8.4](#) cuts the chain at a point where nothing is lost.

4.8.2 Vertex and edge constraints

Definition 4.16 (Vertex and edge constraint). In the notation of Stern et al. [\[161\]](#), a **vertex constraint** $\langle a, v, t \rangle$ forbids agent a to occupy cell v at time step t ; an **edge constraint** $\langle a, u, v, t \rangle$ forbids agent a to move from u to v between time steps t and $t + 1$. A path $\pi = (\pi_0, \pi_1, \dots, \pi_T)$ of agent a , with π_0 its start and π_T its goal, **respects** a set of constraints if $\langle a, \pi_t, t \rangle$ is in none of them for $0 \leq t \leq T$ and $\langle a, \pi_t, \pi_{t+1}, t \rangle$ is in none of them for $0 \leq t < T$. By the stay-at-goal convention of [Definition 2.12](#) the agent still occupies $\gamma = \pi_T$ at every $t > T$, so a path that respects the constraints must also avoid $\langle a, \gamma, t \rangle$ for every $t > T$.

A single-agent search only ever sees the constraints of its own agent, so the agent index is stripped off before the search starts (`constraints_for_agent()` in the chapter code) and the search works with pairs (v, t) and triples (u, v, t) .

Where do the constraints come from? In [Chapter 9](#) the high level of CBS invents them, one per node of its constraint tree, to resolve a collision it has detected. In prioritized planning ([Chapter 8](#)) they come from the paths of the agents that were planned earlier, collected in a reservation table — the data structure Silver introduced for cooperative pathfinding [\[153\]](#) and defined formally in [Definition 8.1](#). Either way the search sees nothing but a set of forbidden pairs (v, t) and triples (u, v, t) , and the class `ReservationTable` in `ch04_astar.py` answers the only two questions it asks: is v blocked at time t , and is the move $u \rightarrow v$ blocked at time t .

4.8.3 The goal test: the agent has to stay

Reaching the goal is not enough. The plan of an agent ends when it lands, and the drone then occupies its landing pad for the rest of the episode. If some other agent is scheduled to fly over that pad at time step 9, arriving there at step 7 and stopping is not a solution.

Definition 4.17 (Goal-stay time). Let t_γ be the largest time step at which the goal γ carries a vertex constraint ($t_\gamma = -1$ if it carries none, $t_\gamma = \infty$ if another agent is parked on it). A state (γ, t) is a **goal state** if and only if $t > t_\gamma$.

With this test the returned path π can be extended by waiting at γ for ever without violating a constraint, which is what Definition 4.16 demands of a MAPF solution by the stay-at-goal convention of Definition 2.12. The value t_γ is computed once, before the search, by `last_blocked_time()`. If $t_\gamma = \infty$ the instance is unsolvable and the search reports failure without expanding anything.

Forgetting that the goal must stay free after arrival

Using the plain test “ $v = \gamma$ ” turns a correct plan into a collision: the drone lands and the next agent flies into it. Using a test that is too strong is just as bad. A frequent variant is “ $v = \gamma$ and $t \geq H$ ”, with H the last constrained time step of the *whole* table; on a map where some far-away agent is still moving at step 50, every plan is then padded to 50 steps, and a sub-problem that has a 3-step solution is reported as needing 50 or, with a tight horizon, as unsolvable. Test the constraints of *this* goal cell only, as in Definition 4.17. In the chapter code the vertex constraint $\langle a, (2, 1), 5 \rangle$ on the goal of the mini example makes the search hop off the goal and come back: the returned path is $(0, 1), (1, 1), (2, 1), (2, 1), (2, 1), (2, 2), (2, 1)$ of cost 6, and it is optimal.

4.8.4 Time horizon, completeness and termination

Because waiting is always possible, the time-expanded graph is infinite, and an unsolvable instance would make the search run for ever. Cutting the search at an arbitrary horizon is not safe either: it can report failure on an instance that has a solution one step later. The following bound is both safe and easy to compute. Write H for the last time step at which the table blocks anything: the largest t of a vertex constraint, the largest $t + 1$ of an edge constraint, and the time from which each parked agent occupies its cell ($H = -1$ if the table is empty). This is what `ReservationTable.horizon()` computes. (The letter H is the horizon of a stored time-indexed path in Definition 2.12; here it is a property of the constraint table, and the horizon of the *search* is $T_{\max}$.) Write D for the largest number of moves from any cell to γ — among the cells from which γ is reachable at all — in the grid in which the parked cells of the table are treated as walls.

Proposition 4.18 (A sufficient time horizon). *If a path from s that respects the constraints and satisfies Definition 4.17 exists, then one exists that reaches γ no later than time*

$$T_{\max} = H + 1 + D. \quad (4.9)$$

Space-time A restricted to states with $t \leq T_{\max}$ is therefore complete, and it terminates on every instance.*

Proof. Let π be a solution and suppose it arrives after $T_{\max}$. No vertex or edge constraint is active at a time step larger than H , so from time $H + 1$ on the only forbidden cells are the parked ones; every parked agent has arrived by time H , so from $H + 1$ on those cells are blocked for ever. Let $v = \pi_{H+1}$ be the cell of the agent at time $H + 1$. The suffix of π from $H + 1$ is a walk from v to γ that avoids the parked cells, so γ is reachable from v in the

reduced grid and a shortest such route has at most D moves. Following it from time $H + 1$ gives a path that respects every constraint, arrives no later than $H + 1 + D$, and satisfies [Definition 4.17](#) because $H \geq t_\gamma$. Its prefix up to $H + 1$ is the prefix of π , which respects the constraints by assumption. For termination, the set of states with $t \leq T_{\max}$ is finite and each of them is generated at most once, so the loop ends after finitely many iterations; if it ends without reaching a goal state, no solution exists. $\square$

[Equation \(4.9\)](#) is what `default_horizon()` returns. On the mini example below, $H = 1$ and $D = 3$, so $T_{\max} = 5$, while the solution needs 3 steps; the bound is loose on purpose, since a loose bound costs nothing when the search finds a solution early and only matters when it has to prove that none exists. Supplying a smaller horizon by hand is allowed and useful in a real-time planner, but it makes the search incomplete: with `max_time = 2` the mini example is reported unsolvable, with `max_time = 3` it is solved.

4.8.5 The heuristic: static distances, computed once

Which heuristic should the search use? The Manhattan distance is admissible on a 4-connected grid, but it ignores the obstacles, and in a constrained re-run of the same agent that error is paid over and over. The remark at the end of [Section 4.3.2](#) said that the exact heuristic h^* is too expensive for a single query because computing it costs a full search. In space-time the arithmetic changes completely:

- the goal of an agent does not change between the calls, so one backward search per agent serves all of them — in a CBS run with thousands of constraint-tree nodes, that is one search instead of thousands;
- the same table serves every time layer, because the distance from v to γ does not depend on t ;
- on a grid the backward search is a breadth-first search, not a Dijkstra search, because every action costs one time step.

So let $h(v)$ be the number of moves of a shortest *unconstrained* path from v to γ , computed once by a backward breadth-first search (`static_distances()`), and let $h(v, t) = h(v)$.

Proposition 4.19 (The static heuristic is consistent). *h is admissible and consistent on the time-expanded graph, for any set of constraints.*

Proof. Consistency: for the wait action, $h(v) \leq 1 + h(v)$; for a move (v, v') , $h(v) \leq 1 + h(v')$, because appending the move to a shortest route from v' gives a route from v . At a goal state $h(\gamma) = 0$. Consistency implies admissibility by [Lemma 4.4](#); alternatively, note that removing states and edges from a graph cannot shorten a path, so the unconstrained distance is a lower bound on the constrained one. Note that h is a lower bound but not exact: it neither knows the constraints nor the waits they force, which is exactly the room in which the search does its work. $\square$

A cell with $h(v) = \infty$ cannot reach the goal at all and is never generated. By [Proposition 4.19](#) the search inherits [Theorem 4.10](#): expansions come out in non-decreasing order of $f = t + h(v)$, and no state is expanded twice.

4.8.6 The algorithm

[Algorithm 4.2](#) puts the pieces together. It differs from [Algorithm 4.1](#) in four places only: the successor function adds the wait action and filters on the constraints, the goal test is the one of [Definition 4.17](#), the horizon truncates the search, and relaxation becomes duplicate detection.

Line 2 is the backward breadth-first search of [Section 4.8.5](#); in a planner that calls the search many times for the same agent it is hoisted out and computed once. Line 4 catches the

Algorithm 4.2. Space-time A* for one agent under vertex and edge constraints

Input: grid graph G , start s , goal γ , constraint set (or reservation table) R of this agent, horizon $T_{\max}$

Output: a time-indexed path $\pi_0, \dots, \pi_T$ of minimum arrival time T that respects R , or *failure*

```

1 Function SpaceTimeAStar( $G, s, \gamma, R, T_{\max}$ ):
2    $h \leftarrow$  static distances to  $\gamma$  by a backward breadth-first search in  $G$ 
3    $t_\gamma \leftarrow$  largest  $t$  with  $(\gamma, t) \in R$  ( $-1$  if none,  $\infty$  if an agent is parked on  $\gamma$ )
4   if  $h(s) = \infty$  or  $t_\gamma = \infty$  then return failure
5   OPEN  $\leftarrow$  priority queue holding  $(s, 0)$  with key  $(h(s), 0)$ ;  $Gen \leftarrow \{(s, 0)\}$ ;
   parent( $s, 0$ )  $\leftarrow$  nil
6   while OPEN  $\neq \emptyset$  do
7      $(v, t) \leftarrow$  pop the entry with the smallest key  $(f, -t)$  from OPEN
8     if  $v = \gamma$  and  $t > t_\gamma$  then
9       return ReconstructPath( $parent, (v, t)$ )
10    if  $t = T_{\max}$  then
11      continue // horizon reached: no successors
12    foreach  $v' \in \{v\} \cup \{v'' : (v'', c) \in \text{Succ}(v)\}$  do
13      // wait first, then the moves
14      if  $h(v') = \infty$  then continue
15      if  $(v', t + 1) \in R$  or  $(v, v', t) \in R$  then continue
16      if  $(v', t + 1) \in Gen$  then continue //  $g = t + 1$  always: the first is the best
17       $Gen \leftarrow Gen \cup \{(v', t + 1)\}$ ; parent( $v', t + 1$ )  $\leftarrow (v, t)$ 
18      push  $(v', t + 1)$  into OPEN with key  $(t + 1 + h(v'), -(t + 1))$ 
19  return failure

```

two hopeless cases before any work is done. It does not test the start state itself: both the algorithm and `space_time_astar()` assume that the caller hands over a start that is free at time 0, as the planners of [Chapters 8 and 9](#) do; if an earlier agent may park on this agent's start from $t = 0$, add *or* $(s, 0) \in R$ to the same line. Line 12 generates the wait action first, so that among states with equal keys the wait is popped first; this is a choice of tie-breaking, not of correctness. Line 13 drops cells from which the goal cannot be reached at all, and line 14 is the whole of the constraint handling: one lookup in a hash set per candidate. Line 15 is duplicate detection; because $g(v', t + 1) = t + 1$ for every route, a second parent could never be better. The set Gen of generated states plays the role of both OPEN and CLOSED of [Algorithm 4.1](#). Note that the search minimizes the *arrival time* T ; if an agent must also stand still at its goal until some later step, that is a property of the plan, not of the search.

4.8.7 A worked mini-example

Example 4.20 (One vertex constraint, one forced wait). Take the empty 3×3 grid, 4-connected, with $s = (0, 1)$ and $\gamma = (2, 1)$, and the single vertex constraint $\langle a, (1, 1), 1 \rangle$: another drone crosses the middle cell at time step 1. The instance is `spacetime_example()` in `ch04_astar.py`. Without the constraint the search expands the three states $((0, 1), 0)$, $((1, 1), 1)$, $((2, 1), 2)$ and returns the path $(0, 1), (1, 1), (2, 1)$ of cost 2: with an exact heuristic and $f = 2$ everywhere, the tie-breaking rule walks straight down the optimal path. With the

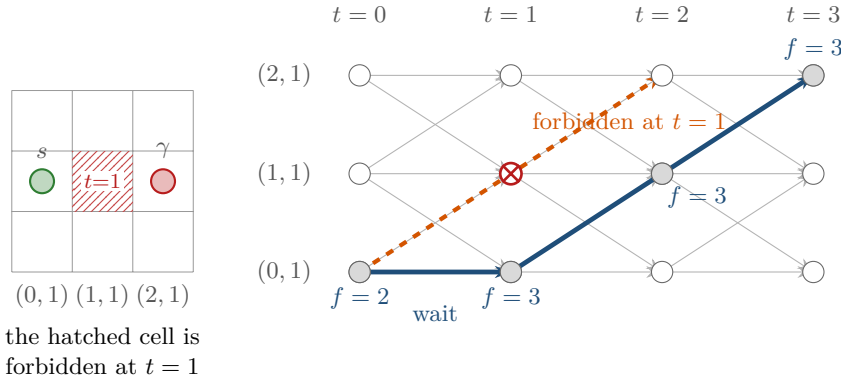


Figure 4.7. Example 4.20. Left: the 3×3 grid; the hatched cell $(1, 1)$ is forbidden at $t = 1$. Right: the states (v, t) of the middle row for $t = 0, \dots, 3$. Horizontal edges are wait actions, diagonal edges are moves, and every edge costs one time step. The crossed-out state $((1, 1), 1)$ is the one removed by the vertex constraint; it lies on the dashed path of cost 2, which is therefore no longer available. Gray states are the four the search expands, with their $f = t + h(v)$; the cheapest remaining path (solid) waits once and costs 3. The states of the top and bottom rows, such as $((0, 0), 1)$ and $((0, 2), 1)$ in Table 4.3, exist too but are never expanded, so the drawing omits them.

Table 4.3. Trace of Algorithm 4.2 on Example 4.20. Note that g is always the time t , that f never decreases (Theorem 4.10(ii) applies, since h is consistent), and that the state $((1, 1), 1)$ never enters OPEN: the constraint filter of line 14 removes it at generation time.

Step	Expanded (v, t)	g	h	f	OPEN after the step (state: f)
1	$((0, 1), 0)$	0	2	2	$((0, 0), 1):4, ((0, 1), 1):3, ((0, 2), 1):4$
2	$((0, 1), 1)$	1	2	3	$((1, 1), 2):3, ((0, 1), 2):4, ((0, 0), 1):4, ((0, 2), 1):4, ((0, 0), 2):5, ((0, 2), 2):5$
3	$((1, 1), 2)$	2	1	3	$((2, 1), 3):3, ((1, 1), 3):4, ((0, 1), 3):5, ((1, 0), 3):5, ((1, 2), 3):5$, and the five entries of step 2 that are still open
4	$((2, 1), 3)$	3	0	3	goal state, $t = 3 > t_\gamma = -1$: return the path

constraint, the state $((1, 1), 1)$ no longer exists, the search expands four states and returns

$$(0, 1), (0, 1), (1, 1), (2, 1),$$

a path of cost 3 with one wait at the start. Figure 4.7 shows the two paths in the time-expanded graph and Table 4.3 the trace.

Two variations, both in the self-test, show the other two mechanisms at work. Adding the edge constraint $\langle a, (0, 1), (1, 1), 1 \rangle$ on top of the vertex constraint forbids the move that the path above makes at $t = 1$; the search then waits twice and returns $(0, 1), (0, 1), (0, 1), (1, 1), (2, 1)$ of cost 4 after five expansions. Replacing the constraints by the single vertex constraint $\langle a, (2, 1), 5 \rangle$ on the goal makes $t_\gamma = 5$: the search reaches $(2, 1)$ at $t = 2$ but may not stop there, so it waits, steps aside to $(2, 2)$ at $t = 5$ and returns at $t = 6$, for a path of cost 6.

Example 4.21 (Two drones in a corridor with a passing bay). A 5×2 map has a corridor along $y = 0$ and a single free cell $(2, 1)$ above it, a passing bay. Agent 1 flies from $(0, 0)$ to $(4, 0)$ and is planned first; its path $(0, 0), (1, 0), (2, 0), (3, 0), (4, 0)$ goes into the reservation table, together with the reversed edges and the permanent block of $(4, 0)$ from $t = 4$. Agent 2 has to fly from $(3, 0)$ to $(0, 0)$, which on the empty corridor takes 3 steps. Under the table,

space-time A* returns

(3, 0), (2, 0), (2, 1), (2, 0), (1, 0), (0, 0),

of cost 5: agent 2 hides in the bay at $t = 2$, lets agent 1 pass, and drops back into the corridor behind it. No sequence of waits in the corridor itself would work, because the reversed-edge constraints forbid the head-on swap. If agent 2 instead starts at (4, 0), no path exists at all — agent 1 parks on that cell — and the search proves it in finite time, six expansions, thanks to [Proposition 4.18](#): here $H = 4$ and $D = 3$, so $T_{\max} = 8$. Every number in this example is asserted by the self-test of `ch04_astar.py`. That failure is not a bug in the search but the known incompleteness of prioritized planning, and it is the reason [Chapter 9](#) searches over constraints instead of fixing an order.

4.9 Implementation notes

[Listing 4.1](#) is the main loop of `astar()` in `code/ch04_astar.py`, the function that produced every number in this chapter. It is [Algorithm 4.1](#) line by line, and the rest of this section explains the six decisions hidden in it.

Listing 4.1. The main loop of A* (from `code/ch04_astar.py`): a lazy binary heap, a dictionary of g values, a closed set and parent pointers. `key()` builds the priority tuple $(f, -g, counter)$; EPS is 10^{-9} .

```

1  g = {start: 0.0}
2  parent = {start: None}
3  closed = set()
4  expansions = []
5  reopenings = 0
6  open_heap = [(key(weight * heuristic(start), 0.0), 0.0, start)]
7  while open_heap:
8      _, g_entry, n = heapq.heappop(open_heap)
9      if g_entry > g[n]:                # stale: a cheaper path was found
10         continue
11     h_n = heuristic(n)
12     expansions.append(Expansion(n, g[n], h_n, g[n] + weight * h_n))
13     if is_goal(n):
14         snapshot = live_open()
15         if record_open:
16             expansions[-1].open_after = snapshot
17         return SearchResult(_reconstruct(parent, n), g[n], expansions,
18                             closed, {m for (m, _) in snapshot}, g,
19                             reopenings)
20     closed.add(n)
21     for n2, c in successors(n):
22         g2 = g[n] + c
23         if g2 < g.get(n2, INF) - EPS: # relaxation (EPS: fp noise)
24             if n2 in closed:
25                 if not reopen:
26                     continue
27                 closed.discard(n2) # re-open (never for consistent h)
28                 reopenings += 1
29                 g[n2] = g2
30                 parent[n2] = n
31                 f2 = g2 + weight * heuristic(n2)
32                 heapq.heappush(open_heap, (key(f2, g2), g2, n2))
33     if record_open:
```



```

34         expansions[-1].open_after = live_open()
35     return SearchResult(None, INF, expansions, closed, set(), g, reopenings)

```

Hashing the state. The state is the dictionary key, so it must be hashable and cheap to hash. A grid cell is the tuple (x, y) of two ints; a space-time state is the nested tuple $((x, y), t)$. Both hash in constant time and compare by value, which is what the sets `closed` and `parent` need. Do not use a NumPy array or a list as a state: they are unhashable, and a class without `__hash__` and `__eq__` would compare by identity, so two states describing the same cell would look different and the search would degenerate into a tree search.

The priority tuple and why it needs a counter. The heap entries are $(key, g, node)$ with $key = (f, -g, counter)$. Python compares tuples lexicographically, so the first component orders by f , the second breaks ties toward the larger g (Section 4.7.1), and the third, a value from `itertools.count()`, breaks the remaining ties first-in-first-out. The counter is not cosmetic. Without it, two entries with identical f and g would make `heapq` compare the next component, the node itself; for grid cells that silently sorts by coordinate, which is a bias nobody intended, and for a state that is not comparable at all, such as a `dataclass` without an ordering, it raises `TypeError` in the middle of a long run. The counter guarantees that no two keys are ever equal, so the node is never reached by the comparison.

Parents and path reconstruction. A dictionary `parent[node] = predecessor` costs one pointer per generated node and turns path extraction into a walk from the goal back to the start (`_reconstruct()`), which is then reversed. Storing whole paths in the queue instead is the classic beginner’s implementation; it multiplies the memory by the path length and makes every push copy a list.

Lazy deletion instead of decrease-key. When the g of a node improves, the tidy operation is *decrease-key*, which `heapq` does not offer. The implementation pushes a second entry instead and lets the stale one age in the heap; the test `g_entry > g[n]` on the pop discards it. The heap therefore holds at most one entry per relaxation, so its size is bounded by the number of edges, and the cost of a pop stays $\mathcal{O}(\log |E|) = \mathcal{O}(\log |V|)$ on a grid. The space-time search does not need the mechanism at all, because g is the time and can never improve.

Memory. Memory, not time, is what stops A* on large problems. Every generated node occupies an entry in `g`, in `parent` and, until it is popped, in the heap. On a 1000×1000 grid with 4-connectivity, a search that touches every cell holds a few million small Python objects, hundreds of megabytes. This is why Chapter 6 discusses anytime and memory-bounded variants, and why the space-time search bounds its state space with Equation (4.9) rather than letting the wait actions run free.

Floating point: `round(f, 9)` and `EPS`. On an 8-connected grid the costs are 1 and $\sqrt{2}$, so g is a sum of irrational numbers and two routes of equal true cost can differ in the last bit. Two places react badly to that. The relaxation test `g2 < g.get(n2, INF) - EPS` would otherwise accept an “improvement” of 10^{-16} , re-open the node, push it again and, with an unlucky cycle of rounding, loop; subtracting `EPS` makes the test insist on a real improvement. The key uses `round(f, 9)` so that two f values that are equal in exact arithmetic are also equal in the tuple, which lets the $-g$ component do the tie-breaking it was put there for — without the rounding, the tie-breaking experiment of Section 4.7.1 is decided by floating-point

noise. The self-test relies on this: it checks that the f sequence is non-decreasing and that no node is re-opened with a consistent heuristic, and both assertions fail without the guards.

Listing 4.2 shows the corresponding heart of `space_time_astar()`: the goal test of Definition 4.17, the wait action, the constraint filter and the duplicate test, in the same order as lines 9–18 of Algorithm 4.2.

Listing 4.2. The expansion step of space-time A* (from `code/ch04_astar.py`): the goal-stay test, the wait action, the constraint filter and duplicate detection. `h` is the static distance table and `table` the reservation table. The `closed` set is recorded only for the traces and the figures; correctness needs only `parent`.

```

1  while heap:
2      _, (v, t) = heapq.heappop(heap) # every state is pushed exactly once
3      closed.add((v, t))
4      expansions.append(Expansion((v, t), t, h[v], t + h[v]))
5      if v == goal and t > t_goal: # stay-at-goal is safe
6          path = [s[0] for s in _reconstruct(parent, (v, t))]
7          if record_open:
8              expansions[-1].open_after = snapshot()
9          return SearchResult(path, float(t), expansions, closed,
10                             {s for (_, s) in heap})
11     if t < max_time:
12         for v2 in [v] + [n for n, _ in succ(v)]: # wait first, then moves
13             if v2 not in h: # cannot reach the goal
14                 continue
15             if (table.vertex_blocked(v2, t + 1)
16                 or table.edge_blocked(v, v2, t)):
17                 continue
18             s2 = (v2, t + 1)
19             if s2 in parent: # g = t + 1: first is best
20                 continue
21             parent[s2] = (v, t)
22             heapq.heappush(heap, (key(*s2), s2))

```

Testing for the goal when the node is generated

It is tempting to check “is this successor the goal?” inside the successor loop and to return at once. The path that comes back is then the first one found, not the cheapest. In Example 4.6 the goal (5, 2) is generated in step 20 with $g = 13$, which happens to be optimal, but that is luck. On the four-node graph of Section 4.6 with the inconsistent heuristic, the goal is generated by the first expansion of b with $g = 5$ and only later improved to 4: a search that returns at generation time reports a path 25 % too expensive. The proof of Theorem 4.9 breaks at exactly this point — it compares $f(\gamma')$ with the keys of the entries in OPEN when γ' is *popped*, and a generated node has not been popped. Test the goal after the pop, on line 9. The one search in this book that may test at generation time is breadth-first search on unit-cost graphs, where all paths to a node found at the same depth cost the same.

4.10 Where this fits in the drone system

In the drone system

A* is the single most reused component of the hybrid planner of Chapter 24. It appears in three roles.

1. The low-level search of the multi-agent planner. CBS (Chapter 9) and ECBS (Chapter 10) split a collision into constraints and call, at every node of their constraint tree, a single-agent search for *one* drone under *that node's* constraints. That call is Algorithm 4.2, with the constraints of Definition 4.16 and the goal test of Definition 4.17. Prioritized planning (Chapter 8) makes the same call with a reservation table instead. A run on thirty drones makes thousands of these calls, so the static distance table of Section 4.8.5 is computed once per drone and reused, and every constant factor in Listing 4.1 is multiplied by a four-digit number.

2. The fallback of the replanning layer. The local safety layer of Chapter 24 — optimal reciprocal collision avoidance (ORCA, Chapter 13) and the dynamic window approach (DWA, Chapter 14) — handles the seconds-scale avoidance of an intruder locally. When a maneuver has pushed a drone so far off its nominal route that the local controller cannot rejoin it, the replanning layer asks for a new global path from the current cell, and the question goes back to A* — in space-time again, with the other drones' remaining plans reserved, so that the repair does not create a new collision.

3. The baseline for the incremental planners. LPA* and D* Lite (Chapter 5) repair the previous search instead of restarting it, which is the right answer when the map changes a little and often; they are A* plus bookkeeping, and their correctness arguments are the ones proved here. Use plain A* when the change is large or rare, and Chapter 5 when a drone discovers obstacles as it flies.

This chapter is the Week 1 milestone of the study plan (Appendix A), whose coding exercise — grid A*, time as a state variable, a picture of the open and closed sets — is Exercise 4.8 below.

4.11 Summary

Chapter summary

- A* is best-first search on $f = g + h$: the cost so far plus an estimate of the cost to go. With $h \equiv 0$ it is Dijkstra's algorithm.
- h is *admissible* if $h \leq h^*$ everywhere, and *consistent* if $h(\gamma) = 0$ and $h(n) \leq c(n, n') + h(n')$ on every edge. Consistency implies admissibility; the converse fails.
- With an admissible h and re-opening enabled, the first goal popped is optimal (Theorem 4.9). With a consistent h , no node is expanded twice and the f of the expansions never decreases (Theorem 4.10); the nodes with $g^* + h < C^*$ are exactly the ones that must be expanded (Corollary 4.11). Both theorems assume a finite graph, non-negative costs and the goal test on the pop.
- On grids: Manhattan for 4-connected maps, octile for 8-connected maps, Euclidean and Chebyshev when nothing better is available. The Manhattan distance is *not* admissible on an 8-connected grid and silently returns longer paths (Table 4.1).
- Ties in f decide most of the work near the goal: prefer the larger g , and add a counter to keep the comparison total.
- Weighted A* with $f_w = g + wh$ returns a path of cost at most $w C^*$ and is usually

far better than the bound.

- Space-time A* searches over states (v, t) with a wait action, one time step per action, vertex constraints $\langle a, v, t \rangle$ and edge constraints $\langle a, u, v, t \rangle$, a goal test that requires the goal to stay free ($t > t_\gamma$), a horizon $T_{\max} = H + 1 + D$ and a static-distance heuristic computed once per goal. It is the low-level search of every multi-agent planner in this book.

Further reading. A* was introduced by Hart, Nilsson and Raphael [62], whose correction note [63] fixes the definition of the “more informed” relation used in Theorem 4.13. Pearl [126] is the reference book on heuristic search and contains the sharpest form of the optimality results, including the complexity discussion of Section 4.6; Dechter and Pearl [33] settle exactly in which class A* is optimal. Russell and Norvig [142] give the textbook treatment that the proofs above follow, and Pohl [132] the original weighted search. For grids, jump point search [61] is the standard way to exploit the symmetry of uniform maps, and safe interval path planning [131] replaces the time layers of Section 4.8 by intervals, which pays off when the horizon is long and the constraints are sparse. Silver [153] introduced the reservation table and cooperative pathfinding in the setting that Chapter 8 formalizes, and Stern et al. [161] fix the MAPF terminology, including the vertex and edge constraints of Definition 4.16, used throughout Chapters 8 to 10.

4.12 Exercises

Exercise 4.1 (★). Continue Table 4.2 by hand for two more steps of Example 4.6. Give, for steps 11 and 12, the expanded cell with its g , h and f , and the contents of OPEN after the step with the current f of each cell. Then answer: why is $(0, 4)$, which has been in OPEN since step 8, still not expanded?

Exercise 4.2 (★). On the grid of Example 4.6 with $\gamma = (5, 2)$, compute all four heuristics of Equation (4.4)–Equation (4.7) at the cells $(1, 4)$ and $(3, 1)$. (a) Order them by dominance and say which is the most informed. (b) Which of them are admissible for the 4-connected instance, and which for the 8-connected one? (c) The true costs to go on the 4-connected instance are $h^*(1, 4) = 8$ and $h^*(3, 1) = 5$. By how much does the best of the four underestimate, and which obstacle causes the error?

Exercise 4.3 (★★). Prove that the octile distance Equation (4.5) is consistent on an 8-connected grid whose moves obey the corner-cutting rule of Definition 2.5. Show first that h_O is the exact cost on an obstacle-free grid, then verify Equation (4.3) on a straight and on a diagonal move directly from the formula (four cases, depending on whether the move changes $\max(d_x, d_y)$ or $\min(d_x, d_y)$), and finally explain in one sentence why forbidding some moves cannot break consistency.

Exercise 4.4 (★★). Theorem 4.14 assumes that closed nodes may be re-opened. Prove the same bound wC^* for the variant that never re-opens a closed node, assuming that h is consistent. Hint: show first that every expanded node n satisfies $g(n) \leq w g^*(n)$ by induction along a cheapest path to n , then repeat the argument of Theorem 4.14 with that inequality in place of Lemma 4.8. Where exactly does your proof use consistency, and what goes wrong when h is only admissible?

Exercise 4.5 (★★). Let h_1 and h_2 be admissible and let $h(n) = \max(h_1(n), h_2(n))$. (a) Show that h is admissible, and that it is consistent when both h_1 and h_2 are. (b) Show that h dominates both, and state what Theorem 4.13 then predicts about the number of expansions. (c) Give two admissible heuristics on the 4-connected grid of Example 4.6 whose maximum

is strictly better than both, and explain why one does not simply always use the maximum of every heuristic one can think of.

Exercise 4.6 (★★). Chapter 2 defines 6-, 18- and 26-connected lattices in three dimensions, with move costs 1, $\sqrt{2}$ and $\sqrt{3}$ respectively for moves that change one, two or three coordinates. Give an admissible and consistent heuristic for each connectivity, exact on an obstacle-free lattice, in the style of Equation (4.5). Hint: sort d_x, d_y, d_z so that $d_{(1)} \geq d_{(2)} \geq d_{(3)}$ and count how many moves of each kind an optimal obstacle-free route uses. Then say which of them is admissible on which of the other two lattices, in the manner of Table 4.1.

Exercise 4.7 (★★). Construct a grid of at most 4×4 cells, an 8-connected instance and a start/goal pair for which A* with the Manhattan heuristic returns a path that is strictly longer than the optimum, and give both costs. Then explain, using Theorem 4.9, which step of the optimality proof the Manhattan heuristic invalidates, and check your instance with `astar_grid(grid, s, z, "manhattan", 8)` against `dijkstra_grid(grid, s, 8)`.

Exercise 4.8 (★★★ Coding). This is the Week 1 coding exercise of the study plan (Appendix A); it continues Exercise 3.7. Implement A* yourself, in the API of `code/ch04_astar.py`, and do not read that file until you are stuck. (a) Write `astar(start, goal, successors, heuristic)` with a lazy heap, a closed set, the priority tuple $(f, -g, counter)$ and parent pointers, and `astar_grid()` on top of it for 4- and 8-connected grids. Reproduce the numbers of Example 4.6: cost 13, 21 expansions, and the first ten rows of Table 4.2. (b) Add `record_open=True` and draw the open and closed sets of your search, in the style of Figure 4.3, after every expansion; watch the frontier of the Manhattan search walk into the dead end of the column $x = 3$. (c) Add time as a state variable: implement `space_time_astar(grid, start, goal, constraints, max_time)` with the wait action, the constraint filter of Definition 4.16, the goal test of Definition 4.17 and the horizon of Equation (4.9). Check it against Example 4.20 and Example 4.21. (d) Verify on 50 random 12×12 grids that your A* returns the same cost as your Dijkstra of Chapter 3, that no node is re-opened when the heuristic is consistent, and that the space-time cost without constraints equals the static distance.

Exercise 4.9 (★★★). Take Example 4.20 and add, by hand, the vertex constraint $\langle a, (1, 1), 2 \rangle$ to the existing $\langle a, (1, 1), 1 \rangle$. (a) Predict the cost of the new optimal path, the number of waits and the route. (b) Now replace both by the single edge constraint $\langle a, (1, 1), (2, 1), 1 \rangle$ and predict again; then say what would change if its time index were 2 instead of 1. (c) Check both with `space_time_astar()` and compare the expansion counts with the four expansions of Table 4.3. (d) Which of the two instances would become unsolvable if the horizon were set to `max_time = 3`, and why does that not contradict Proposition 4.18?

Exercise 4.10 (★★★). Proposition 4.18 bounds the arrival time by $T_{\max} = H + 1 + D$. (a) Show by an example that the term D cannot be dropped, that is, that $H + 1$ alone is not a sufficient horizon. (b) Show that the bound is not tight in general by giving an instance whose optimal arrival time is far below $T_{\max}$. (c) In Example 4.21, compute H , D and $T_{\max}$ for agent 2 and compare with the arrival time 5 that the search returns. (d) A colleague proposes to drop the parked cells from the computation of D , using the eccentricity of the goal in the raw grid instead. Show that the resulting horizon can be too small, and describe the failure the planner would report.

Incremental Search: LPA* and D* Lite

Learning objectives

- Explain why re-running A* after a small map change wastes work, and what an *incremental* search reuses instead.
- Define the *g*- and *rhs*-values of LPA*, local consistency, over- and underconsistency, and the two-component priority key, and trace LPA* by hand on a small grid.
- Explain why D* Lite searches backward from the goal, what the key modifier k_m repairs when the robot moves, and reproduce its pseudocode.
- State the conditions under which D* Lite returns the same path costs as re-running A*, and bound its number of vertex expansions.
- Implement D* Lite in Python, change obstacles while the agent executes its plan, and measure the saving relative to re-running A*.

5.1 Why this matters for a drone swarm

Picture one drone of a swarm on its nominal route through a warehouse. The route was computed by the global planner (Chapters 9 and 10) before take-off, on a grid of 200×200 cells at ten flight levels. Halfway through the mission the prediction layer (Chapters 18 and 20) announces that an unknown drone will cross the corridor ahead within the next few seconds. The local layer (Chapters 13 and 14) can dodge for a moment, but the intruder is going to hover in the corridor, and the local maneuver cannot return the drone to its nominal route. Now the replanning layer must deliver a new route from the drone's *current* position, and it has one control cycle, say 50 ms, to do so.

Re-running A* (Chapter 4) from scratch is the obvious answer. It is also wasteful: of the 400 000 cells of the map, only a few dozen have changed. Every distance that A* computed last time is still valid, except in a small region around the intruder. An *incremental* search keeps the old search results and repairs only what the change invalidated. D* Lite, the algorithm of this chapter, does exactly that, and on top of it handles the fact that the drone keeps moving while it replans. It is the replanning algorithm of the hybrid architecture in Chapter 24, and the training plan rates it [Essential](#); its conceptual parent Lifelong Planning A* (LPA*) is rated [Medium](#).

The chapter proceeds in two halves. The first half develops LPA*, which repairs a shortest path from a fixed start to a fixed goal when edge costs change. It introduces the two numbers stored per vertex (*g* and *rhs*), the notion of local consistency, and the two-component key that orders the work. The second half turns LPA* into D* Lite by reversing the search direction and adding the key modifier k_m , so that the start vertex may move. Both halves have a fully

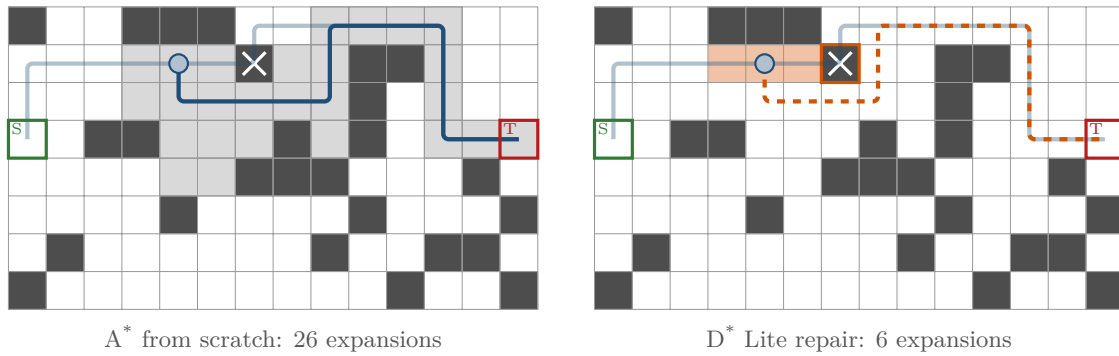


Figure 5.1. An obstacle (black, with a cross) appears two cells ahead of the drone (blue disk) on its planned route (faded blue). Left: A* re-run from scratch from the drone’s cell expands all gray cells. Right: D* Lite expands only the orange cells, those whose goal distance was invalidated or is needed for the detour (dashed). The number of expansions is printed under each panel. Note that D* Lite is repairing a search it had already run from the drone’s original start (52 expansions); [Section 5.8.1](#) accounts for that first search. The newly blocked cell (orange frame) is itself one of the expanded cells: its stale distance is retracted first.

worked grid example whose numbers come from the chapter’s Python code, and the chapter ends with an experiment that compares D* Lite with repeated A* on a 50×50 grid as obstacles appear one by one.

5.2 The idea in plain words

[Figure 5.1](#) shows the situation of the motivation on a small grid. The drone has a shortest path to its goal and has already flown a third of it when the cell two steps ahead of it turns out to be blocked. In the left panel, A* is re-run from scratch from the drone’s cell: it expands every cell shaded gray, 26 of them, most of which lie far from the obstacle and whose distance to the goal did not change at all. In the right panel, an incremental search touches only the orange cells, four cells in six expansions: the ones whose stored distance became stale because the obstacle removed the edge they relied on, plus the few cells needed to find the detour. Those six expansions repair a search that the drone had already run from its original start, and that search cost 52 expansions on this map; [Section 5.8.1](#) weighs the first search against the cheap repairs it buys.

How does the incremental search know which cells are stale? It stores *two* numbers per vertex instead of one. The first, $g(s)$, is the distance value from the last search, exactly as in A*. The second, $rhs(s)$, is what the neighbors of s currently say the value *should* be: one step of lookahead, “the best neighbor’s g plus the edge cost”. As long as the two numbers agree, nothing at s needs attention. When an edge cost changes, the rhs -value of the affected vertex changes but its g -value does not, and the disagreement flags the vertex as *locally inconsistent*. Inconsistent vertices go into a priority queue, ordered by the same kind of key that A* uses ($g + h$), and are processed in that order. Processing a vertex either makes it consistent or replaces its stale value by a larger, honest one, and may make its neighbors inconsistent, so the repair spreads outward from the change, but only as far as distance values actually differ, and only until the goal is consistent and no queued key is smaller than the goal’s key. Cells far from the change, or cells whose distance happens to stay the same, are never touched.

Key idea

Store, for every vertex, the value from the last search (g) next to the value its neighbors currently imply (rhs). Only vertices where the two disagree need work. Process them in A* order, from the change outward, and stop as soon as the query vertex agrees with its neighbors and nothing with a smaller key is left. The result equals what A* would compute from scratch, at a fraction of the expansions when the change is small.

D* Lite adds one more idea. A moving robot changes the *start* of the search at every step. If distance values were measured from the start, every one of them would change with every step. Distances *to the goal* do not change when the robot moves. So D* Lite searches backward from the goal, and the robot's motion only affects the heuristic part of the keys, which a single running offset k_m can absorb.

5.3 Problem statement and notation

We use the graph notation of Chapters 2 and 3: a finite directed graph $G = (V, E)$ with edge costs $c(u, v) \in (0, \infty]$ for $(u, v) \in E$, a start vertex s_{start} and a goal vertex s_{goal} . $\text{Pred}(s)$ and $\text{Succ}(s)$ are the predecessors and successors of s . A cost of ∞ means that the edge is currently unusable; we write $c(u, v) = \infty$ also for pairs that are not adjacent, which keeps formulas uniform. The true distance from u to v under the current costs is $\text{dist}(u, v)$, and $\text{dist}(u, v) = \infty$ if no path exists.

This is the notation of Chapter 4 in new clothes: s_{start} is the start node s of Definition 4.1, s_{goal} is the goal γ , and $\text{dist}(s_{\text{start}}, n)$ is the true distance written $g^*(n)$ there. One symbol changes meaning, and it is the important one: in Chapter 4, $g(n)$ is the cost of the best path found *so far in the current search*, whereas here $g(s)$ is a value stored from an *earlier* search that may be stale until the repair reaches it.

Definition 5.1 (Incremental shortest-path problem). An **incremental shortest-path problem** is a sequence of shortest-path queries between the same two vertices s_{start} and s_{goal} on a graph whose edge costs change between queries. An **incremental search algorithm** answers each query using information saved from the previous queries, and it must return a path of cost $\text{dist}(s_{\text{start}}, s_{\text{goal}})$ under the costs current at the time of the query. In the **moving-start** variant, which D* Lite solves, the goal is fixed but before each query the start vertex is replaced by the vertex the agent has moved to, which is required to lie on the path returned by the previous query.

Sections 5.4 and 5.5 treat the fixed-start case (LPA*); Sections 5.6 and 5.7 add the moving start (D* Lite).

The grids of this chapter are 4-connected with unit costs (Chapter 2). A cell is either *free* or *blocked*. Blocking a cell b is modeled as an edge-cost change: *every* edge that enters b and every edge that leaves b changes from cost 1 to cost ∞ . Unblocking reverses the change. We keep blocked cells as vertices of the graph, with all their edges at ∞ , rather than deleting them; that way the graph structure is fixed and only the costs vary, which is exactly what the algorithms assume. The heuristic on these grids is the Manhattan distance, which is consistent (Chapter 4).

LPA* stores two values per vertex.

Definition 5.2 (g -value and rhs -value). For every vertex $s \in V$, $g(s) \in [0, \infty]$ is the **g -value**, an estimate of $\text{dist}(s_{\text{start}}, s)$ that is only changed when s is expanded. The **rhs -value**

(“right-hand side”, after the Bellman equation it comes from) is the one-step lookahead

$$rhs(s) = \begin{cases} 0 & \text{if } s = s_{\text{start}}, \\ \min_{s' \in \text{Pred}(s)} (g(s') + c(s', s)) & \text{otherwise.} \end{cases} \quad (5.1)$$

The *rhs*-value is always kept up to date with respect to the current *g*-values of the predecessors and the current edge costs; the *g*-value is allowed to lag behind. The gap between them is the central concept.

Definition 5.3 (Local consistency). A vertex s is **locally consistent** if $g(s) = rhs(s)$, and **locally inconsistent** otherwise. An inconsistent vertex is **overconsistent** if $g(s) > rhs(s)$ (its neighbors now offer a cheaper way to reach it than it knows) and **underconsistent** if $g(s) < rhs(s)$ (it believes in a cost that its neighbors can no longer support).

A freshly discovered vertex has $g = \infty$ and a finite *rhs*, so it is overconsistent; this is the only kind of inconsistency A* ever meets. Underconsistent vertices appear when a cost *increases*, for example when an obstacle appears on the path: the vertex behind the obstacle still holds its old, now too optimistic *g*-value. The following proposition says why consistency is the right target.

Proposition 5.4 (Consistency everywhere means correct distances). *Suppose all edge costs are positive. If every vertex is locally consistent, then $g(s) = \text{dist}(s_{\text{start}}, s)$ for all $s \in V$, and a shortest path from s_{start} to any reachable s is obtained by moving backward from s to a predecessor s' that attains the minimum in Equation (5.1), until s_{start} is reached.*

Proof. If all vertices are consistent, the *g*-values satisfy $g(s_{\text{start}}) = 0$ and $g(s) = \min_{s' \in \text{Pred}(s)} (g(s') + c(s', s))$ for all other s : the Bellman equations of the shortest-path problem. With positive edge costs, the true distances are the unique solution of these equations (Chapter 3), so $g \equiv \text{dist}(s_{\text{start}}, \cdot)$. Following a minimizing predecessor decreases *g* by exactly the edge cost at every step, so the walk terminates at s_{start} and has cost $g(s)$. $\square$

LPA* does not make *all* vertices consistent; like A* it stops early. What it guarantees is that the goal is consistent and that no queued vertex could still improve the goal, and that suffices for the goal’s *g*-value and the path traced from it to be correct (Theorem 5.8). To decide the order of work it uses a key with two components.

Definition 5.5 (Priority key). The **key** of a vertex s (written $key(s)$ in the notation table) is the pair

$$k(s) = [k_1(s); k_2(s)] = [\min(g(s), rhs(s)) + h(s); \min(g(s), rhs(s))], \quad (5.2)$$

where $h(s)$ estimates $\text{dist}(s, s_{\text{goal}})$ and is nonnegative and consistent: $h(s_{\text{goal}}) = 0$ and $h(s) \leq c(s, s') + h(s')$ for all $(s, s') \in E$. Keys are compared **lexicographically**: $k \leq k'$ iff $k_1 < k'_1$, or $k_1 = k'_1$ and $k_2 \leq k'_2$; and $k < k'$ iff $k_1 < k'_1$, or $k_1 = k'_1$ and $k_2 < k'_2$.

Read k_1 as the *f*-value of A* and k_2 as its *g*-value used for tie-breaking, both computed from the *smaller* of the two stored values. For an overconsistent vertex the smaller value is *rhs*, the tentative new distance, exactly as in A*. For an underconsistent vertex it is the old *g*, which is too small. Using the too-small value on purpose puts the vertex *early* in the queue, before every vertex whose value depends on it, so that the stale value is retracted before it can be propagated further. Figure 5.2 illustrates the three states.

The priority queue U holds exactly the locally inconsistent vertices, each with the key it had when it was last inserted. $U.\text{TopKey}()$ returns the smallest key in U (or $[\infty; \infty]$ if U is empty), $U.\text{Pop}()$ removes and returns the vertex with that key, $U.\text{Insert}(s, k)$ inserts s with key k , and $U.\text{Remove}(s)$ removes s .

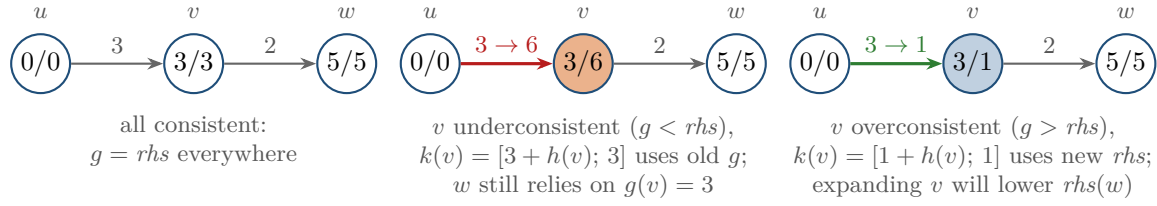


Figure 5.2. Local consistency on a three-vertex fragment (edge costs on the arrows, g/rhs inside the vertices). Left: all consistent. Middle: the edge into v became expensive, so $rhs(v)$ rose and v is underconsistent; its key uses the old $g(v) = 3$ and puts v before w , whose rhs still relies on $g(v)$. Right: a new cheap edge into v makes v overconsistent; its key uses $rhs(v) = 1$.

5.4 The algorithm: LPA*

Algorithm 5.1 is Lifelong Planning A* (LPA*) of Koenig, Likhachev and Furcy [90], in the form of the original paper. It searches *forward* from s_{start} ; g -values are start distances, and rhs -values look at predecessors.

Walkthrough. Initialize sets every g and rhs to ∞ except $rhs(s_{\text{start}}) = 0$ (line 6); the start is thus the only inconsistent vertex and the only entry of U , just as it is the only entry of OPEN at the start of A*.

UpdateVertex(u) is the maintenance routine. Line 9 recomputes $rhs(u)$ from Equation (5.1); the start keeps $rhs = 0$ forever. Lines 10–11 then restore the invariant “ U contains exactly the inconsistent vertices, with current keys”: u is taken out, and put back with a fresh key only if it is inconsistent. Every path of the algorithm that touches a g -value or an edge cost calls **UpdateVertex** on the vertices whose rhs could have changed, and nothing else ever modifies rhs .

ComputeShortestPath is the search proper. Its loop condition (line 13) mirrors the termination of A*: continue while some queued vertex has a key smaller than the goal’s, or while the goal itself is inconsistent. Each iteration pops the vertex u with the smallest key (line 14) and treats the two kinds of inconsistency differently:

- *Overconsistent* ($g(u) > rhs(u)$, line 15): accept the better value, $g(u) \leftarrow rhs(u)$. This is the expansion of A*. The successors of u may now have a better predecessor, so they are updated (line 17). After this step u is consistent.
- *Underconsistent* ($g(u) < rhs(u)$, line 19): the old value cannot be trusted, so it is thrown away, $g(u) \leftarrow \infty$. Now every successor that computed its rhs through u must recompute it, and u itself must recompute its own rhs and re-enter the queue if it is not ∞ (line 20, note the $\cup \{u\}$). After this step u is either consistent (both ∞) or overconsistent with a fresh, larger key, and may be expanded a second time later, as an ordinary A* expansion, if the loop ever reaches its new, larger key; Theorem 5.9 bounds this at one further expansion. Step 10 of Table 5.6 shows a vertex that is retracted and never re-expanded, because the search stops first.

Main glues the pieces together: after each search, wait for the environment to report changed edges, update their costs, and call **UpdateVertex** on the *head* v of every changed edge (line 28), because it is v whose rhs depends on $c(u, v)$. Then search again. The first call of **ComputeShortestPath** is exactly an A* search (Theorem 5.9); each later call is a repair.

Why the two-component key works. Consider what the loop condition must guarantee: when it stops, the goal’s g -value must equal the true distance. The first key component $k_1 = \min(g, rhs) + h$ is a lower bound on the cost of any path to the goal through the queued

Algorithm 5.1. Lifelong Planning A* (LPA*) [90]**Input:** graph G , costs c , start s_{start} , goal s_{goal} , consistent heuristic h **Output:** after every call of **ComputeShortestPath**: $g(s_{\text{goal}}) = \text{dist}(s_{\text{start}}, s_{\text{goal}})$ and a shortest path (Theorem 5.8)

```

1 Function CalculateKey( $s$ ):
2   return  $[\min(g(s), rhs(s)) + h(s); \min(g(s), rhs(s))]$ 
3 Function Initialize():
4    $U \leftarrow \emptyset$ 
5   foreach  $s \in V$  do  $rhs(s) \leftarrow g(s) \leftarrow \infty$ 
6    $rhs(s_{\text{start}}) \leftarrow 0$ 
7    $U.\text{Insert}(s_{\text{start}}, [h(s_{\text{start}}); 0])$ 
8 Function UpdateVertex( $u$ ):
9   if  $u \neq s_{\text{start}}$  then  $rhs(u) \leftarrow \min_{s' \in \text{Pred}(u)} (g(s') + c(s', u))$ 
10  if  $u \in U$  then  $U.\text{Remove}(u)$ 
11  if  $g(u) \neq rhs(u)$  then  $U.\text{Insert}(u, \text{CalculateKey}(u))$ 
12 Function ComputeShortestPath():
13  while  $U.\text{TopKey}() < \text{CalculateKey}(s_{\text{goal}})$  or  $rhs(s_{\text{goal}}) \neq g(s_{\text{goal}})$  do
14     $u \leftarrow U.\text{Pop}()$ 
15    if  $g(u) > rhs(u)$  then // overconsistent
16       $g(u) \leftarrow rhs(u)$ 
17      foreach  $s \in \text{Succ}(u)$  do UpdateVertex( $s$ )
18    else // underconsistent
19       $g(u) \leftarrow \infty$ 
20      foreach  $s \in \text{Succ}(u) \cup \{u\}$  do UpdateVertex( $s$ )
21 Function Main():
22   Initialize()
23   while true do
24     ComputeShortestPath()
25     wait for changes in edge costs
26     foreach directed edge  $(u, v)$  whose cost changed do
27       update the stored cost  $c(u, v)$ 
28       UpdateVertex( $v$ )

```

vertex, so as long as $\text{TopKey}() \geq k(s_{\text{goal}})$ no queued vertex can lead to a cheaper path to the goal than the goal's own value, exactly the argument that proves A* optimal with a consistent heuristic (Chapter 4). The second component breaks ties in favor of *smaller* $\min(g, rhs)$, which matters more here than in A*: among vertices with equal k_1 , an underconsistent vertex u with a small old $g(u)$ must be processed before a successor w whose $rhs(w) = g(u) + c(u, w)$ was derived from it. If w is overconsistent, then $k_2(w) = rhs(w) = g(u) + c(u, w) > g(u) = k_2(u)$ because $c(u, w) > 0$, so u comes first among the ties and the stale value is withdrawn before w is recomputed from correct information; if w is itself underconsistent, it is retracted in the same way before it can be expanded with a stale value. Without the second component, w could be expanded with a value that is about to be retracted, and would have to be expanded a third time.

Comparing keys as single numbers

A key is a pair, and both components matter. Comparing only k_1 (or, worse, only k_2) breaks the ordering argument above and can make LPA* expand vertices with stale values, or terminate with a wrong $g(s_{\text{goal}})$. In Python, store keys as tuples (k_1 , k_2); tuple comparison is lexicographic, which is what the algorithm needs. Also never compare keys for equality with floating-point tolerance tricks: the algorithm only needs $<$ and $\leq$.

5.5 A worked example: LPA* repairs a path

Example 5.6 (An obstacle appears on the path). Figure 5.3 shows a 5×3 grid of free cells. Cells are named (x, y) with x the column from the left and y the row from the bottom, both starting at 0. The start is $S = (0, 1)$, the goal is $T = (4, 1)$, moves are 4-connected with unit cost, and h is the Manhattan distance to T . Ties between equal keys are broken toward the smaller x , then the smaller y . First LPA* computes a shortest path. Then cell $(3, 1)$, which lies on the path, becomes blocked, and LPA* repairs the path.

The first search. Table 5.1 traces the first call of `ComputeShortestPath`. Every popped vertex is overconsistent, so this is an A* search: the cells of the middle row are expanded one after the other, each with key $[4; x]$, because for a cell on the straight line $g + h = 4$. The side cells receive finite rhs -values and enter the queue with $k_1 = 6$, but they are never expanded: after T is popped and made consistent, the smallest queued key is $[6; 1]$, which is not smaller than $k(T) = [4; 4]$, and the loop stops. Five expansions, ten cells left in the queue with correct rhs -values but $g = \infty$.

The change. Blocking $(3, 1)$ changes eight directed edges: the four that enter $(3, 1)$ and the four that leave it. `Main` calls `UpdateVertex` on the head of each. For the entering edges the head is $(3, 1)$ itself: all its predecessors are now at cost ∞ , so $rhs(3, 1) = \infty$ while $g(3, 1) = 3$; the cell is underconsistent and enters the queue with key $[3 + 1; 3] = [4; 3]$. For the leaving edges the heads are the neighbors. $T = (4, 1)$ loses its only finite predecessor, $rhs(T) = \infty$ against $g(T) = 4$: underconsistent, key $[4; 4]$. $(2, 1)$ still has S 's side as predecessor and keeps $rhs = 2$. $(3, 0)$ and $(3, 2)$ had $rhs = 4$ through $(3, 1)$; now their only finite predecessor is gone, $rhs = \infty = g$, and they leave the queue. The middle panel of Figure 5.3 shows this state: two underconsistent cells, eight further queued cells, and every g -value untouched.

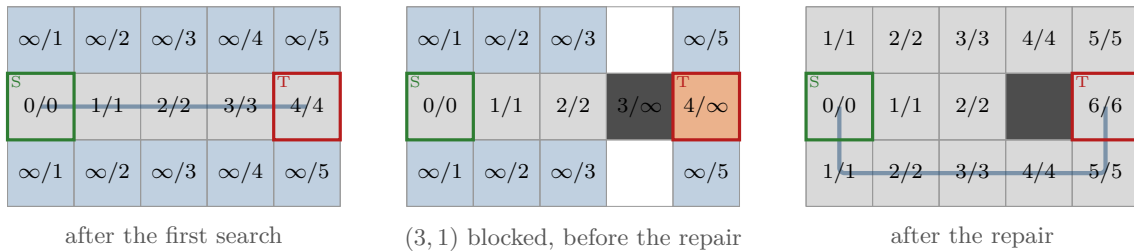


Figure 5.3. LPA* on Example 5.6. Each cell shows g/rhs ; gray cells are locally consistent with finite values (expanded), blue cells are overconsistent and orange cells underconsistent (both kinds are in the queue U); cells without numbers have $g = rhs = \infty$. Left: after the first search, path of cost 4. Middle: right after cell $(3, 1)$ is blocked and `UpdateVertex` has run on the affected vertices, before `ComputeShortestPath`. Right: after the repair, the new path of cost 6.

Table 5.1. First call of `ComputeShortestPath` in [Example 5.6](#). “Updated” lists the vertices on which `UpdateVertex` changed something, with their new *rhs*-value and key.

Step	Popped u (key)	Case	$g(u)$ after	Updated: <i>rhs</i> , key
1	(0, 1) [4; 0]	over	0	(1, 1): 1, [4; 1]; (0, 2): 1, [6; 1]; (0, 0): 1, [6; 1]
2	(1, 1) [4; 1]	over	1	(2, 1): 2, [4; 2]; (1, 2): 2, [6; 2]; (1, 0): 2, [6; 2]
3	(2, 1) [4; 2]	over	2	(3, 1): 3, [4; 3]; (2, 2): 3, [6; 3]; (2, 0): 3, [6; 3]
4	(3, 1) [4; 3]	over	3	(4, 1): 4, [4; 4]; (3, 2): 4, [6; 4]; (3, 0): 4, [6; 4]
5	(4, 1) [4; 4]	over	4	(4, 2): 5, [6; 5]; (4, 0): 5, [6; 5]

Table 5.2. Second call of `ComputeShortestPath` in [Example 5.6](#), after (3, 1) was blocked. Steps 1–2 retract stale values (underconsistent); the remaining steps are ordinary expansions. “–” means that `UpdateVertex` changed nothing, “out” that the vertex left the queue.

Step	Popped u (key)	Case	$g(u)$ after	Updated: <i>rhs</i> , key
1	(3, 1) [4; 3]	under	∞	–
2	(4, 1) [4; 4]	under	∞	(4, 2): ∞ , out; (4, 0): ∞ , out
3	(0, 0) [6; 1]	over	1	–
4	(0, 2) [6; 1]	over	1	–
5	(1, 0) [6; 2]	over	2	–
6	(1, 2) [6; 2]	over	2	–
7	(2, 0) [6; 3]	over	3	(3, 0): 4, [6; 4]
8	(2, 2) [6; 3]	over	3	(3, 2): 4, [6; 4]
9	(3, 0) [6; 4]	over	4	(4, 0): 5, [6; 5]
10	(3, 2) [6; 4]	over	4	(4, 2): 5, [6; 5]
11	(4, 0) [6; 5]	over	5	(4, 1): 6, [6; 6]
12	(4, 2) [6; 5]	over	5	–
13	(4, 1) [6; 6]	over	6	–

The repair. [Table 5.2](#) traces the second call. The two underconsistent cells come first, because their keys use the old, small g -values. Popping (3, 1) sets $g(3, 1) = \infty$; its successors are updated but nothing changes, since their *rhs*-values already ignored (3, 1). Popping T sets $g(T) = \infty$, and now the cells (4, 0) and (4, 2), whose *rhs* = 5 came through T , also drop out of the queue. From here on the search is a plain A* over the side rows: the queued cells are expanded in order of their keys, all with $k_1 = 6$ and increasing k_2 , the detour $(2, 0) \rightarrow (3, 0) \rightarrow (4, 0)$ is discovered, and T is finally re-expanded with $g(T) = 6$. The repair costs 13 expansions, of which the first two are underconsistent. Compare this with A* from scratch on the changed grid, which expands only 7 cells: on this small instance the repair costs almost twice as much as starting over. Two effects are at work. The stale values must be retracted first, which costs two expansions that A* does not have. And the key order expands every queued cell with $k_1 = 6$ in order of increasing g , both side rows in full, whereas A* breaks ties among equal f -values toward the larger g , the usual rule on grids ([Chapter 4](#)), and runs down one side row straight to the goal. What LPA* reuses are the g -values of S , (1, 1) and (2, 1), which are never expanded again; on a 5×3 grid that is not much. [Section 5.8.1](#) shows that on larger maps, when the change is local, the reused part dominates, and [Section 5.8](#) returns to the tie-breaking.

Reading the path off as in [Proposition 5.4](#), with ties broken toward the smaller coordinates, gives $T \rightarrow (4, 0) \rightarrow (3, 0) \rightarrow (2, 0) \rightarrow (1, 0) \rightarrow (0, 0) \rightarrow S$, the blue path of cost 6 in the right panel of [Figure 5.3](#). Other paths of the same cost exist; the tie-break selects this one.

Table 5.3. LPA* and D* Lite are mirror images. The only genuinely new ingredient of D* Lite is the key modifier k_m .

	LPA* (Algorithm 5.1)	D* Lite (Algorithm 5.2)
Search direction	from s_{start}	from s_{goal}
$g(s)$ estimates	$\text{dist}(s_{\text{start}}, s)$	$\text{dist}(s, s_{\text{goal}})$
$rhs(s)$	$\min_{s' \in \text{Pred}(s)} g(s') + c(s', s)$	$\min_{s' \in \text{Succ}(s)} c(s, s') + g(s')$
fixed $rhs = 0$ at	s_{start}	s_{goal}
expansion updates	$\text{Succ}(u)$	$\text{Pred}(u)$
edge (u, v) changed	$\text{UpdateVertex}(v)$	$\text{UpdateVertex}(u)$
heuristic	$h(s) \approx \text{dist}(s, s_{\text{goal}})$	$h(s_{\text{start}}, s) \approx \text{dist}(s_{\text{start}}, s)$
loop tests key of	s_{goal}	s_{start}
key modifier	none	k_m added to k_1
path extraction	backward from s_{goal} over Pred	forward from s_{start} over Succ

5.6 The algorithm: D* Lite

LPA* assumes that start and goal stay fixed. A drone that executes its plan moves, and every step moves the start. D* Lite [88, 89] is LPA* adapted to a moving start with two changes: the search runs backward from the goal, and a key modifier k_m keeps the priority queue valid across moves.

5.6.1 Reversing the search

Suppose we kept the forward search of LPA* and simply moved s_{start} one cell after each step. The g -values are distances *from the start*; after the move, every one of them is off by the length of the step (or more), so every vertex would become inconsistent and the “repair” would redo the whole search. Distances *to the goal*, on the other hand, are unaffected by where the robot is. D* Lite therefore lets $g(s)$ estimate $\text{dist}(s, s_{\text{goal}})$ and searches from the goal toward the robot. Everything in Section 5.4 is mirrored, as Table 5.3 summarizes:

- the rhs -value looks at *successors*: $rhs(s) = \min_{s' \in \text{Succ}(s)} (c(s, s') + g(s'))$ for $s \neq s_{\text{goal}}$, and $rhs(s_{\text{goal}}) = 0$;
- the queue is initialized with s_{goal} , and expanding a vertex updates its *predecessors*;
- when the cost of edge (u, v) changes, it is the *tail* u whose rhs depends on it, so $\text{UpdateVertex}(u)$ is called;
- the heuristic $h(s_{\text{start}}, s)$ estimates $\text{dist}(s_{\text{start}}, s)$, the distance from the robot to s , and the termination test compares against $k(s_{\text{start}})$;
- the path is read off *forward* from the robot: from s , move to the successor s' that minimizes $c(s, s') + g(s')$.

The heuristic must now be consistent with respect to the *start*: nonnegative, $h(s_{\text{start}}, s_{\text{start}}) = 0$ and $h(s_{\text{start}}, s) \leq h(s_{\text{start}}, s') + c(s', s)$ for all s and $s' \in \text{Pred}(s)$. In addition, because the start will move, we require the triangle inequality between any three vertices, $h(a, c) \leq h(a, b) + h(b, c)$; this is what the key modifier relies on. The Manhattan distance on a 4-connected unit-cost grid satisfies all of these (it is a metric that never exceeds the true distance), and so does the octile distance on an 8-connected grid.

Table 5.4. Why the key modifier is needed. A queued vertex u and a vertex v inserted after the robot has moved two cells, so $h(s_{\text{old}}, s_{\text{new}}) = k_m = 2$; $m = \min(g, rhs)$, and the last two columns are the keys that **CalculateKey** returns now, without and with the modifier. The row of v lists both keys it could receive; it is inserted once.

Vertex	m	$h(s_{\text{old}}, \cdot)$	$h(s_{\text{new}}, \cdot)$	stored key	now, no k_m	now, $k_m = 2$
u (old entry)	4	6	4	[10; 4]	[8; 4]	[10; 4]
v (new entry)	6	–	3	[9; 6] / [11; 6]	[9; 6]	[11; 6]
popped first				v , from the stored keys	u	u

Wrong heuristic direction

In D* Lite the heuristic argument is the *robot's* position, not the goal: $h(s_{\text{start}}, s)$. A common slip is to keep the A* habit and use the distance from s to the goal. The search then still terminates and usually still returns a path, but the path may be *longer than the shortest one*. The loop test compares $U.\text{TopKey}()$ with $k(s_{\text{start}})$, and that comparison only proves that nothing cheaper is left while k_1 is a lower bound on the cost of a path through the queued vertex; with the heuristic measured from the wrong end the bound is gone, so the loop can stop while a queued vertex still hides a cheaper route. The search also explores the wrong region (around the goal instead of around the robot) and breaks the property that the key modifier relies on. Whenever you swap the search direction, swap the heuristic with it.

5.6.2 The key modifier k_m

Reversing the search removes the robot's position from the g -values, but not from the keys: $k_1(s) = \min(g(s), rhs(s)) + h(s_{\text{start}}, s)$ contains the distance from the robot to s . After the robot moves, every key stored in the queue is computed with the old start. The obvious fix, recomputing all keys and rebuilding the heap after every step, costs time proportional to the queue size at every step, which is exactly the kind of work we set out to avoid.

D* Lite avoids the rebuild with an observation. Let the robot move from s_{old} to s_{new} . By the triangle inequality,

$$h(s_{\text{old}}, s) \leq h(s_{\text{old}}, s_{\text{new}}) + h(s_{\text{new}}, s) \quad \text{for every vertex } s, \quad (5.3)$$

so every old key is at most the new key plus the constant $h(s_{\text{old}}, s_{\text{new}})$. Instead of lowering the old keys, D* Lite *raises* all keys computed from now on by that constant: it keeps a running sum k_m , adds $h(s_{\text{old}}, s_{\text{new}})$ to it whenever edge costs are processed after a move, and defines

$$k(s) = [\min(g(s), rhs(s)) + h(s_{\text{start}}, s) + k_m; \min(g(s), rhs(s))]. \quad (5.4)$$

With this definition every key that sits in the queue is a *lower bound* on the key that **CalculateKey** would return for the same vertex now. Lower bounds are harmless as long as the algorithm checks them before acting: when a vertex is popped whose stored key is smaller than its current key, D* Lite simply reinserts it with the current key and pops again (lines 15–18 of [Algorithm 5.2](#)). Because keys are only ever compared with one another and with $k(s_{\text{start}})$, which also contains k_m , adding the same constant to every key computed from now on leaves every comparison among current keys unchanged.

[Table 5.4](#) gives numbers. Vertex u was inserted before the move with key $[4 + 6; 4] = [10; 4]$. The robot then moves two cells toward u , so its heuristic drops to 4 and the correct key becomes $[8; 4]$. A new vertex v is inserted after the move. Without the modifier it gets $[6 + 3; 6] = [9; 6]$

and sits *above* u in the queue, although u should be processed first ($[8; 4] < [9; 6]$). The search would expand v using a stale rhs -value that may still depend on u , and the loop could even terminate with u buried under an inflated key. With $k_m = 2$ the new vertex gets $[11; 6]$, the stored key $[10; 4]$ of u is exactly its current key $[4 + 4 + 2; 4]$, and the order is correct. In general the stored key is only a lower bound, and the reinsert check repairs the rest.

5.6.3 Pseudocode

Algorithm 5.2 is the final version of D* Lite (the one with the key modifier) as published by Koenig and Likhachev [88]. Compared with LPA*, read Succ for Pred and vice versa, note the k_m term in the key, the reinsert check, and the main loop that moves the robot. The same paper also gives an *optimized* variant, which computes the same values with the same expansions but maintains rhs incrementally instead of recomputing the minimum over all successors on every **UpdateVertex**: after an overconsistent expansion of u it only relaxes the single edge, $rhs(s) \leftarrow \min(rhs(s), c(s, u) + g(u))$ for each predecessor s , it recomputes the full minimum in the underconsistent branch only for those predecessors whose rhs came from the retracted g -value, and it updates a queue entry in place instead of removing and reinserting it. That variant removes the factor Δ of Section 5.8 from the overconsistent case and is what a production implementation should use; we print the plain version because its two branches are easier to follow and to prove correct.

Walkthrough. **Initialize** seeds the queue with the goal (line 8), the mirror image of LPA*'s start. **UpdateVertex** recomputes $rhs(u)$ over the successors of u (line 10); the goal keeps $rhs = 0$.

ComputeShortestPath runs until the robot's vertex is consistent and no queued key is smaller than the robot's key (line 14). It records the top key before popping (line 15) and compares it with the freshly computed key. If the stored key was a strict lower bound, the vertex is reinserted with its current key and is *not* expanded (line 18); this costs one heap operation, never an expansion. Otherwise the vertex is expanded as in LPA*, with predecessors in place of successors (lines 20–24).

Main is the execution loop. After the initial search (line 28), the robot repeatedly takes the greedy step toward the successor with the smallest $c + g$ (line 31); by Theorem 5.8 this follows a shortest path. Then it looks for cost changes (line 33). If there are none, it keeps walking without touching the queue, and s_{last} stays where it was. If there are changes, the key modifier is increased by the heuristic distance from s_{last} , the position at which the queue keys were last valid, to the current position (line 35); note that this is done once per batch of changes, not once per step, which is why s_{last} is needed. Then every changed edge is processed at its tail (line 39) and the search is repaired (line 40). Line 30 is the exit when no path exists: the drone must then hover, land or ask the swarm planner for a new goal.

Forgetting the vertex itself on underconsistency

Line 24 updates $\text{Pred}(u) \cup \{u\}$, not just $\text{Pred}(u)$ (and line 20 of LPA* updates $\text{Succ}(u) \cup \{u\}$). After $g(u)$ is set to ∞ , u itself is usually overconsistent again, with a finite rhs through another neighbor, and must go back into the queue to be expanded properly later. Dropping the $\cup \{u\}$ leaves u inconsistent but absent from the queue, and the vertices behind it never receive the corrected value: the robot follows a path through a cost that no longer exists. The symmetric mistake, updating only u and forgetting its predecessors (successors in LPA*), leaves the vertices that depended on u with stale rhs -values.

Algorithm 5.2. D* Lite, final version [88]

Input: graph G , costs c , robot position s_{start} , goal s_{goal} , heuristic $h(\cdot, \cdot)$ consistent w.r.t. the start

Output: the robot moves along shortest paths under the current costs until it reaches s_{goal} or no path exists

```

1 Function CalculateKey( $s$ ):
2    $m \leftarrow \min(g(s), rhs(s))$ 
3   return  $[m + h(s_{\text{start}}, s) + k_m; m]$ 
4 Function Initialize():
5    $U \leftarrow \emptyset; k_m \leftarrow 0$ 
6   foreach  $s \in V$  do  $rhs(s) \leftarrow g(s) \leftarrow \infty$ 
7    $rhs(s_{\text{goal}}) \leftarrow 0$ 
8    $U.\text{Insert}(s_{\text{goal}}, [h(s_{\text{start}}, s_{\text{goal}}); 0])$ 
9 Function UpdateVertex( $u$ ):
10  if  $u \neq s_{\text{goal}}$  then  $rhs(u) \leftarrow \min_{s' \in \text{Succ}(u)} (c(u, s') + g(s'))$ 
11  if  $u \in U$  then  $U.\text{Remove}(u)$ 
12  if  $g(u) \neq rhs(u)$  then  $U.\text{Insert}(u, \text{CalculateKey}(u))$ 
13 Function ComputeShortestPath():
14  while  $U.\text{TopKey}() < \text{CalculateKey}(s_{\text{start}})$  or  $rhs(s_{\text{start}}) \neq g(s_{\text{start}})$  do
15     $k_{\text{old}} \leftarrow U.\text{TopKey}()$ 
16     $u \leftarrow U.\text{Pop}()$ 
17    if  $k_{\text{old}} < \text{CalculateKey}(u)$  then // stale lower bound
18       $U.\text{Insert}(u, \text{CalculateKey}(u))$ 
19    else if  $g(u) > rhs(u)$  then // overconsistent
20       $g(u) \leftarrow rhs(u)$ 
21      foreach  $s \in \text{Pred}(u)$  do UpdateVertex( $s$ )
22    else // underconsistent
23       $g(u) \leftarrow \infty$ 
24      foreach  $s \in \text{Pred}(u) \cup \{u\}$  do UpdateVertex( $s$ )
25 Function Main():
26    $s_{\text{last}} \leftarrow s_{\text{start}}$ 
27   Initialize()
28   ComputeShortestPath()
29   while  $s_{\text{start}} \neq s_{\text{goal}}$  do
30     if  $g(s_{\text{start}}) = \infty$  then return no path
31      $s_{\text{start}} \leftarrow \arg \min_{s' \in \text{Succ}(s_{\text{last}})} (c(s_{\text{last}}, s') + g(s'))$ 
32     move the robot to  $s_{\text{start}}$ 
33     scan the graph for changed edge costs
34     if any edge cost changed then
35        $k_m \leftarrow k_m + h(s_{\text{last}}, s_{\text{start}})$ 
36        $s_{\text{last}} \leftarrow s_{\text{start}}$ 
37       foreach directed edge  $(u, v)$  whose cost changed do
38         update the stored cost  $c(u, v)$ 
39         UpdateVertex( $u$ )
40       ComputeShortestPath()

```

5.7 A worked example: the robot moves and an obstacle appears

Example 5.7 (D* Lite with a moving robot). Figure 5.4 shows a 7×5 grid. A wall occupies $(3, 0)$, $(3, 1)$ and $(3, 2)$, leaving a gap of two cells at the top of column 3. The robot starts at $S = (0, 3)$ and the goal is $T = (6, 2)$; the heuristic is the Manhattan distance from the robot's current cell. D* Lite plans, the robot takes two steps, and then cell $(3, 3)$, the lower cell of the gap and the next cell on the robot's path, becomes blocked. D* Lite repairs the plan and the robot continues through the remaining gap cell $(3, 4)$.

Table 5.5. Events of Example 5.7. “Updated” counts the vertices whose *rhs*-value or queue membership changed in `UpdateVertex`, “Exp.” the vertex expansions of `ComputeShortestPath` in that row; $g(s_{\text{start}})$ is the remaining path cost afterward.

t	Robot at	Event	k_m	Updated	Exp.	$g(s_{\text{start}})$
0	(0, 3)	initial search	0	–	10	7
1	(1, 3)	move, no change	0	–	0	6
2	(2, 3)	move, no change	0	–	0	5
2	(2, 3)	(3, 3) blocked, repair	2	3	12	7
9	(6, 2)	goal reached	2	–	0	0

Table 5.5 lists the events. The initial search (Algorithm 5.2, line 28) expands 10 cells, working outward from T until S is consistent and no queued key is smaller than $k(S) = [7; 7]$, and finds the path of cost 7 through the gap. The g -values in the left panel are goal distances; cells off the corridor between T and S were never expanded and keep $g = \infty$, although several of them sit in the queue with finite *rhs*-values and keys $[9; \cdot]$, which will matter in a moment.

The robot moves to $(1, 3)$ and then to $(2, 3)$ without any queue work: no edge cost changed, so lines 35–40 are skipped, and the remaining cost read off at the robot's cell falls from 7 to 6 to 5. Then $(3, 3)$ becomes blocked. Line 35 first sets $k_m = h((0, 3), (2, 3)) = 2$: every key computed from now on is inflated by 2, and the keys already in the queue, computed when the robot stood at S , become lower bounds. Then the eight directed edges of $(3, 3)$ change cost and `UpdateVertex` runs on their tails. For $(3, 3)$ itself all outgoing edges are now ∞ , so $rhs(3, 3) = \infty$ while $g(3, 3) = 4$: underconsistent, key $[4 + 1 + 2; 4] = [7; 4]$. The robot's own cell $(2, 3)$ loses its best successor; its *rhs* rises from 5 to 7 through $(1, 3)$, so it is underconsistent too, with key $[5 + 0 + 2; 5] = [7; 5]$. The gap cell $(3, 4)$ was queued with *rhs* = 5 through $(3, 3)$

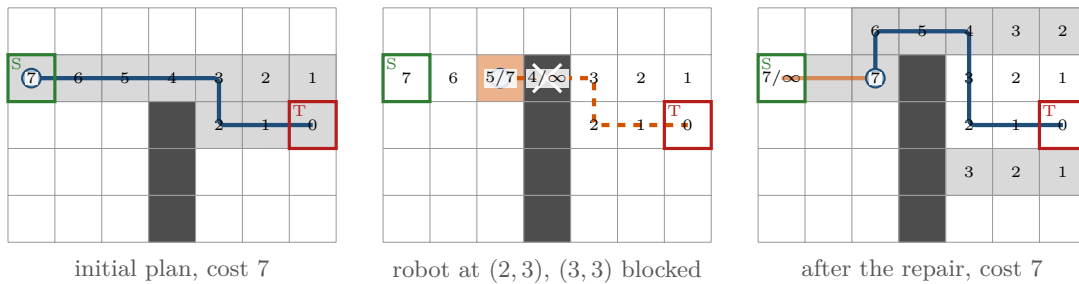


Figure 5.4. D* Lite on Example 5.7. Numbers are g -values (goal distances), written g/rhs where the two disagree, as in Figure 5.3; cells without a number still have $g = \infty$. Left: the initial search from T and the planned path. Middle: the robot (blue disk) has moved two steps and $(3, 3)$ becomes blocked; orange cells are the ones `UpdateVertex` made inconsistent (the third changed vertex, the gap cell $(3, 4)$, simply leaves the queue), the old path is dashed. Right: after the repair, with the cells expanded by the repair shaded and the new path; the robot's traversed route is drawn in orange.

Table 5.6. The repair in [Example 5.7: ComputeShortestPath](#) after (3, 3) was blocked, with the robot at (2, 3) and $k_m = 2$. Keys are $[\min(g, rhs) + h((2, 3), \cdot) + k_m; \min(g, rhs)]$; the loop stops once the robot’s cell is consistent and no queued key is smaller than $k(2, 3) = [9; 7]$. “–” means that `UpdateVertex` changed nothing, “out” that the vertex left the queue.

Step	Popped u (key)	Case	$g(u)$ after	Updated: rhs , key
1	(3, 3) [7; 4]	under	∞	–
2	(2, 3) [7; 5]	under	∞	(1, 3): 8, [9; 6]; (2, 4): ∞ , out; (2, 2): ∞ , out; (2, 3): 7, [9; 7]
3	(6, 1) [9; 1]	over	1	(6, 0): 2, [11; 2]
4	(5, 1) [9; 2]	over	2	(5, 0): 3, [11; 3]
5	(6, 4) [9; 2]	over	2	–
6	(4, 1) [9; 3]	over	3	(4, 0): 4, [11; 4]
7	(5, 4) [9; 3]	over	3	–
8	(4, 4) [9; 4]	over	4	(3, 4): 5, [9; 5]
9	(3, 4) [9; 5]	over	5	(2, 4): 6, [9; 6]
10	(1, 3) [9; 6]	under	∞	(2, 3): ∞ , out; (0, 3): ∞ , [11; 7]; (1, 4): ∞ , out; (1, 2): ∞ , out; (1, 3): 8, [11; 8]
11	(2, 4) [9; 6]	over	6	(1, 4): 7, [11; 7]; (2, 3): 7, [9; 7]
12	(2, 3) [9; 7]	over	7	(2, 2): 8, [11; 8]

and now has $rhs = \infty = g$, so it leaves the queue. (4, 3) keeps $rhs = 3$ through (4, 2), and the wall cell (3, 2) stays at ∞ . Three vertices changed, as the middle panel of [Figure 5.4](#) shows.

[Table 5.6](#) traces the repair, which expands 12 vertices. The two underconsistent cells come first. Retracting $g(3, 3)$ changes nothing else, since no rhs -value still relies on it. Retracting $g(2, 3)$ does: (1, 3), whose rhs came through the robot’s cell, now has $rhs = 8$ through S and is underconsistent with key $[6 + 1 + 2; 6] = [9; 6]$; (2, 4) and (2, 2), queued with values derived from $g(2, 3) = 5$, drop out; and (2, 3) itself re-enters as overconsistent with key $[7 + 0 + 2; 7] = [9; 7]$, which is now the key the loop compares against. Next come the cells that the initial search had left in the queue with keys $[9; \cdot]$, among them (6, 1), (5, 1) and (4, 1) in the bottom right, which have nothing to do with the detour: their k_2 is smaller than 7, so the key order expands them before the robot’s cell (the band effect discussed in [Section 5.8](#)). For every one of them the stored key equals the current key, because the robot moved two cells straight toward them and $k_m = 2$ compensates exactly; the reinsert check of line 18 therefore never fires in this example. Expanding (4, 4) gives the gap cell (3, 4) its value 5, expanding (3, 4) gives (2, 4) the value 6, the stale (1, 3) is retracted on the way (which briefly removes the robot’s cell from the queue, until (2, 4) is expanded and hands it $rhs = 7$ again), and finally (2, 3) is expanded with $g(2, 3) = 7$: the detour through (3, 4) costs 2 more than the old plan. A* from scratch from (2, 3) on the changed grid also expands 12 cells. From here the robot walks $(2, 4) \rightarrow (3, 4) \rightarrow (4, 4) \rightarrow (4, 3) \rightarrow (4, 2) \rightarrow (5, 2) \rightarrow T$ with no further changes and reaches the goal at $t = 9$.

5.8 Properties: correctness, optimality, complexity

Throughout this section the assumptions are: all edge costs are positive or ∞ , the graph is finite, and the heuristic is nonnegative and consistent with respect to the search direction ([Definition 5.5](#) for LPA*, [Section 5.6.1](#) for D* Lite, including the triangle inequality). The results are those of Koenig, Likhachev, and Furcy [90] and Koenig and Likhachev [88]; we state them for D* Lite and give the proof ideas, which transfer to LPA* by mirroring.

Theorem 5.8 (Termination and correctness). *Every call of `ComputeShortestPath` in Algorithm 5.2 terminates. When it returns, $g(s_{\text{start}}) = \text{dist}(s_{\text{start}}, s_{\text{goal}})$ under the current edge costs, and if this value is finite, moving from the current vertex s to any successor s' that minimizes $c(s, s') + g(s')$, starting at s_{start} , reaches s_{goal} along a shortest path. Consequently the robot in `Main` follows, at every moment, a path whose cost equals the cost that A^* would compute from scratch from its current position.*

Proof sketch. Three invariants hold throughout. (i) rhs is always the one-step lookahead of the current g -values and costs, because every change of a g -value or a cost is followed by `UpdateVertex` on all vertices whose rhs could be affected. (ii) U contains exactly the locally inconsistent vertices, and every stored key is a lower bound on the vertex's current key: at insertion the two are equal, and afterward the current key can only grow, either because k_m grew by $h(s_{\text{last}}, s_{\text{start}})$ while $h(s_{\text{start}}, s)$ fell by at most that much (Equation (5.3)), or because a vertex's key is left untouched between insertions. (iii) Vertices are expanded in nondecreasing order of their current keys, because the reinsert check never expands a vertex whose stored key is stale.

From (iii) and the consistency of h one shows, as for A^* , that when an overconsistent vertex u is expanded its new $g(u) = rhs(u)$ equals $\text{dist}(u, s_{\text{goal}})$ under the current costs, provided all vertices with smaller keys have already been made consistent; the underconsistent expansions ensure that no stale finite value survives among those. When the loop stops, s_{start} is consistent and every remaining queued vertex has a key at least $k(s_{\text{start}})$, so no queued vertex can offer a shorter path to the goal than $g(s_{\text{start}})$ itself. Every vertex on a minimizing successor chain from s_{start} has a key no larger than $k(s_{\text{start}})$ and is therefore consistent, so the chain is a shortest path by the argument of Proposition 5.4. Termination follows from Theorem 5.9: each vertex is expanded at most twice, and a reinsert without expansion happens at most once per stale entry. $\square$

Theorem 5.9 (Expansion bound). *Within one call of `ComputeShortestPath`, every vertex is expanded at most twice: at most once when it is underconsistent and at most once when it is overconsistent. Vertices that are consistent before the call and whose g -value does not need to change are never expanded. The first call, with all g -values at ∞ , expands the same vertices in the same order as A^* with the same heuristic and the same tie-breaking (ties among equal f -values broken toward the smaller g) would, on the reversed graph.*

Proof sketch. An overconsistent expansion of u sets $g(u) = rhs(u)$ with key $[g(u) + h(s_{\text{start}}, u) + k_m; g(u)]$. Any vertex s' expanded afterward has a key at least as large, so $g(s') + h(s_{\text{start}}, s') \geq g(u) + h(s_{\text{start}}, u)$. The consistency condition of Section 5.6.1, applied to the edge (u, s') with $u \in \text{Pred}(s')$, reads $h(s_{\text{start}}, s') \leq h(s_{\text{start}}, u) + c(u, s')$, hence $c(u, s') + g(s') \geq g(u)$: no later expansion can lower $rhs(u)$ below $g(u)$, so u is never popped as overconsistent again in this call. An underconsistent expansion sets $g(u) = \infty$; a vertex with $g = \infty$ cannot be underconsistent, and $g(u)$ only becomes finite again through an overconsistent expansion, after which the first argument applies. The statement about the first call follows because with all $g = \infty$ there are no underconsistent vertices, every expansion is an A^* expansion with rhs playing the role of the tentative g , and the keys order vertices exactly as f with g tie-breaking does. The full proofs are in Koenig, Likhachev, and Furcy [90]. $\square$

Three remarks put the bound in perspective. First, “at most twice” is a worst case; the typical repair after a local change expands a small fraction of the vertices, as the worked examples and Figure 5.5 show. Second, the bound does not say that D* Lite is always cheaper than A^* : if a change invalidates most of the stored values, D* Lite may expand vertices twice and pay for queue maintenance that A^* does not need (see the pitfall in Section 5.10). The key modifier adds no expansions at all; within one call of `ComputeShortestPath` its only cost is at most one extra pop-and-reinsert per queue entry that was inserted before a move (across

several calls an entry can be reinserted once per increase of k_m). Third, the theorem compares with A* *with the same tie-breaking*. The second key component orders vertices with equal k_1 by increasing g ; on a grid, where many cells share the same f -value, this expands the whole band of cells with $f \leq \text{dist}(s_{\text{start}}, s_{\text{goal}})$ in order of their distance, whereas the usual A* rule of breaking ties toward the larger g (Chapter 4) runs to the goal along one path of that band. The first search of D* Lite is therefore often several times more expensive than a well-tuned A* (Section 5.8.1 measures a factor of 4.6 on a 50×50 grid), and a repair that raises the path cost re-expands the part of the band that the new cost uncovers, as the worked examples show. This is the price of the second component, and it cannot be dropped: it is what guarantees that stale values are retracted before the vertices that depend on them are expanded (Exercise 5.4).

The bound also fixes the running time of a call. At most $2|V|$ expansions take place; each expansion calls `UpdateVertex` on $O(\deg u)$ vertices, and each such call recomputes an rhs -value in time proportional to the degree of that vertex and inserts or removes one queue entry in $O(\log |U|)$ time. A call therefore costs $O(|E|(\Delta + \log |V|))$ in the worst case, where Δ is the maximum degree of the graph; on a 4-connected grid $\Delta = 4$ is a constant and this is $O((|V| + |E|)\log |V|)$, the same bound as a search from scratch with A*. Recomputing an rhs -value as a minimum over all successors, rather than relaxing one edge as A* does, is what costs the extra factor Δ ; the optimized variant mentioned in Section 5.6.3 relaxes single edges in the overconsistent case and pays Δ only where a value is retracted. The saving is not asymptotic: it is that the constant is paid only where stored values actually changed.

5.8.1 An experiment: D* Lite versus repeated A*

Figure 5.5 was produced by `code/figures/gen_ch05_replanning.py`. On a 50×50 grid with 20 % random obstacles, the robot starts in the top-left corner and heads for the bottom-right corner. At each of 40 events it first takes one step along its current path; then a new obstacle appears, with probability one half on the robot's current path (which forces a repair) and otherwise at a random free cell, never disconnecting the goal. D* Lite processes the change with Algorithm 5.2; the baseline runs A* from scratch from the robot's current cell on the changed grid, once with the usual grid tie-breaking (ties among equal f -values toward the smaller h) and once with the tie-breaking of the D* Lite key (toward the smaller g). All three count vertex expansions and wall-clock time; the curves average over 10 random grids, and event 0 is the initial search. The expansion counts below are seed-deterministic and reproduce exactly; the wall-clock times were measured on a 2024 laptop and will differ on your machine.

Over the 40 replanning events, A* expanded 14 166 vertices in total and D* Lite 1 031, a ratio of about 14; the same repairs took 60 ms and 33 ms of wall-clock time, a much smaller gap, because one D* Lite expansion costs about $28 \mu\text{s}$ in this implementation against $4 \mu\text{s}$ for an A* expansion: `UpdateVertex` recomputes the rhs -value of every neighbor from all of its neighbors, and the lazy queue does extra bookkeeping. Three features of the curves are worth noticing. First, the initial search (event 0) costs D* Lite 1 285 expansions and 32 ms, against 279 expansions and 1.3 ms for A*. The dotted curve shows that A* with the tie-breaking of the D* Lite key needs 1 247 expansions for the same search, so the difference is the band effect of Section 5.8, not the incremental machinery. In cumulative expansions D* Lite overtakes A* after four events; in cumulative time A* is still marginally ahead at the end of the run (61 ms against 65 ms), so on this instance and in this implementation the incremental search has not quite recovered the cost of its first search after 40 events, although its repairs are on average nearly twice as fast as a fresh A* (0.8 ms against 1.5 ms); at 7 of the 40 events, those in which the obstacle forced a long detour, the repair was nevertheless the slower of the two (up to 5.2 ms against 1.6 ms). Second, events whose obstacle lands off the path cost D* Lite next to nothing: over all grids and events, 197 such events needed 1.5 expansions on average and none at all in 68 % of the cases, because `UpdateVertex` finds every affected vertex consistent

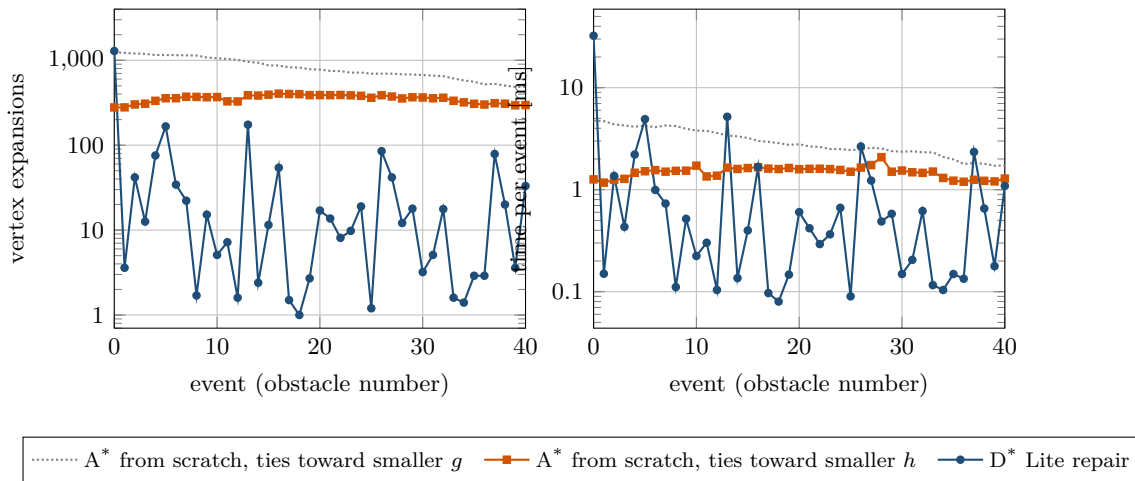


Figure 5.5. Replanning cost as obstacles appear one by one on a 50×50 grid (means over 10 grids, logarithmic scales; event 0 is the initial search). Left: vertex expansions per event. A* re-expands a few hundred cells every time; D* Lite expands almost nothing when the obstacle is off the path and a small region when it is on the path, but its initial search costs as much as A* with the same tie-breaking (dotted). Right: wall-clock time per event of the Python implementation; the gap is smaller than for expansions because D* Lite does more work per expansion.

or irrelevant and `ComputeShortestPath` returns immediately. Third, the 203 events on the path needed 49 expansions on average but only 4 in the median: most repairs are tiny, and the mean is dominated by a few events in which the obstacle closes a corridor and the detour is long. The largest single repair expanded 1704 vertices, several times the cost of an A* search, which is the “almost everything changed” case of [Theorem 5.9](#) and the situation of the last pitfall in [Section 5.10](#).

5.9 Variants and extensions

The original D*. Stentz’s D* (“Dynamic A*”) [159] was the first algorithm to repair a robot’s path incrementally as its sensors revealed unknown terrain, and its Focused variant [160] added a heuristic. It works backward from the goal, like D* Lite, but propagates cost changes with RAISE and LOWER states and a set of case distinctions that are hard to get right. D* Lite computes the same paths, expands no more vertices than D* in the experiments of Koenig and Likhachev [89], and fits on half a page; this is why it replaced D* in practice.

Anytime D*. [Chapter 6](#) presents ARA*, which finds a path quickly with an inflated heuristic εh and improves it while time remains. Anytime D* [106] combines that schedule with the repair machinery of this chapter: it keeps g and rhs across both ε changes and edge-cost changes, lowering ε when the world is quiet and raising it again after a large change. It is the natural choice when the replanning layer has a hard deadline and a suboptimal path now is worth more than an optimal one later.

Field D*. Paths on a grid are restricted to a few headings: up to about 41% longer than the true shortest path on a 4-connected grid and up to about 8% longer on an 8-connected one. Field D* [42] keeps the D* Lite update scheme but lets a path leave a cell through any point of an edge, computing the cost of a cell by linear interpolation between the g -values of the two corner vertices of that edge. The result is smooth, nearly any-angle paths at little extra cost; it was used on the Mars rovers and on several autonomous cars.

Moving Target D* Lite. D* Lite handles a moving start. When the *goal* moves too, for example when a drone must intercept or follow another vehicle, the goal distances change with every goal step and the backward search loses its advantage. Moving Target D* Lite [163] extends D* Lite to targets that move: it reuses the part of the previous search tree that remains valid after both the agent and the target have moved, deletes the rest, and handles edge-cost changes with the same g/rhs machinery.

5.10 Implementation notes

The priority queue. The algorithms need `Insert`, `Pop`, `TopKey`, membership tests and, in `UpdateVertex`, `Remove` of an arbitrary element. Python’s `heapq` offers no removal, so Listing 5.1 uses *lazy deletion*: a dictionary maps each queued vertex to its current key, and heap entries are only trusted if their key equals the dictionary entry. `Remove` deletes the dictionary entry; the stale heap entry is skipped when it surfaces. The heap holds at most one *live* entry per vertex, but a stale entry is only discarded when it reaches the top, so the heap grows with the number of insertions rather than with $|U|$: every operation costs $\mathcal{O}(\log m)$ with m the current heap *size*, and on a long mission with many changes m can exceed $|U|$ by a wide margin. Rebuild the heap from `key_of` whenever m grows past a few times $|U|$; the rebuild is $\mathcal{O}(|U|)$ and amortizes away. An alternative is an indexed binary heap with an explicit `decrease_key` and `remove`; it is faster in compiled languages but longer to write. Store keys as tuples so that comparison is lexicographic, and break ties among equal keys deterministically (here by the vertex coordinates), which makes traces reproducible.

Memory. Two floats per vertex, g and rhs , are kept for the whole lifetime of the mission — $\mathcal{O}(|V|)$ storage that a search from scratch releases when it returns, and 400 000 vertices, a few megabytes, for the $200 \times 200 \times 10$ voxel grid of Section 5.1. This is what an incremental search buys its savings with. The queue is the exception: unlike the two arrays it is not $\mathcal{O}(|V|)$, because the lazy heap keeps stale entries until they surface, which is the reason for the occasional rebuild above.

Infinity. Use `float("inf")`: it compares, adds and takes minima correctly, and $\infty = \infty$ is true, which the consistency test $g(u) = rhs(u)$ relies on. Never encode blocked cells as a large finite cost such as 10^9 in the incremental algorithms; a “blocked” path would then have a finite cost, the algorithms would happily compute it, and the robot would walk into the wall when nothing else is available.

Scanning changed edges. The pseudocode “scans the graph”. In a grid world the environment reports the cells whose status flipped; the planner converts each such cell b into its eight directed edges and calls `UpdateVertex` on each tail, that is on b and on each of its four neighbors. Doing this once per changed cell is enough; calling `UpdateVertex` twice on the same vertex is harmless. Edge-cost *decreases* (an obstacle disappears or a region is no longer predicted to be occupied) are handled by the same code and produce only overconsistent vertices.

When does a key change? Only inside `UpdateVertex`, which removes and reinserts the vertex, and implicitly for all queued vertices when k_m grows. Nothing else touches keys, and there is no `decrease_key` call anywhere in the algorithm. This is what makes lazy deletion sufficient.

Termination and the no-path case. ComputeShortestPath stops with $g(s_{\text{start}}) = \infty$ if the goal is unreachable; the queue is then empty or holds only keys $\geq [\infty; \infty]$. The main loop must check this before taking a step (Algorithm 5.2, line 30). When the obstacle disappears later, the same queue continues to work: no re-initialization is needed, ever.

Listing 5.1. The priority queue with lazy deletion (from code/ch05_dstar_lite.py).

```

1  class PriorityQueue:
2
3      def __init__(self):
4          self.heap = []
5          self.key_of = {}
6
7      def __contains__(self, s):      # the test "u in U"
8          return s in self.key_of
9
10     def insert(self, s, key):
11         self.key_of[s] = key
12         heapq.heappush(self.heap, (key, s))
13
14     def remove(self, s):
15         del self.key_of[s]          # the heap entry becomes stale
16
17     def _purge(self):
18         while self.heap and self.key_of.get(self.heap[0][1]) != self.heap[0][0]:
19             heapq.heappop(self.heap)
20
21     def top_key(self):
22         self._purge()
23         return self.heap[0][0] if self.heap else (INF, INF)
24
25     def pop(self):
26         self._purge()
27         key, s = heapq.heappop(self.heap)
28         del self.key_of[s]
29         return s

```

Listing 5.2. Algorithm 5.2 line by line: key, UpdateVertex and ComputeShortestPath of class DStarLite (from code/ch05_dstar_lite.py). Missing dictionary entries stand for ∞ ; `_trace` records the events for the worked examples.

```

1      def calculate_key(self, s):
2          m = min(self.g_of(s), self.rhs_of(s))
3          return (m + self.h(self.start, s) + self.k_m, m)
4
5      def update_vertex(self, u):
6          rhs_old, queued_old = self.rhs_of(u), u in self.U
7          if u != self.goal:
8              self.rhs[u] = min(self.grid.cost(u, s) + self.g_of(s)
9                              for s in self.grid.neighbors(u))
10
11          if u in self.U:
12              self.U.remove(u)
13          if self.g_of(u) != self.rhs_of(u):
14              self.U.insert(u, self.calculate_key(u))
15          self._trace("update", u, rhs_old, self.rhs_of(u), queued_old,
16                     u in self.U, self.U.key_of.get(u))

```

```

17 def compute_shortest_path(self):
18     """Repair the search; returns the number of vertex expansions."""
19     n = 0
20     while (self.U.top_key() < self.calculate_key(self.start)
21            or self.rhs_of(self.start) != self.g_of(self.start)):
22         k_old = self.U.top_key()
23         u = self.U.pop()
24         k_new = self.calculate_key(u)
25         if k_old < k_new:                                # stale lower bound
26             self.U.insert(u, k_new)
27             self._trace("reinsert", u, k_old, k_new)
28         elif self.g_of(u) > self.rhs_of(u):              # overconsistent
29             self.g[u] = self.rhs_of(u)
30             n += 1
31             self._trace("pop", u, k_old, "over", self.g[u])
32             for s in self.grid.neighbors(u):
33                 self.update_vertex(s)
34         else:                                           # underconsistent
35             self.g[u] = INF
36             n += 1
37             self._trace("pop", u, k_old, "under", INF)
38             for s in self.grid.neighbors(u) + [u]:
39                 self.update_vertex(s)
40     self.expansions += n
41     return n

```

Listing 5.3. Processing a batch of changed cells, lines 35–40 of Main (from code/ch05_dstar_lite.py).

```

1 def notify_changed_cells(self, cells, replan=True):
2     """Process cells whose blocked status flipped (D* Lite main loop).
3
4     Adds h(s_last, s_start) to k_m, updates the tail of every changed
5     edge (the cell and each neighbor) and, if replan, repairs the
6     search. Returns the number of expansions.
7     """
8     if not cells:
9         return 0
10    self.k_m += self.h(self.s_last, self.start)
11    self.s_last = self.start
12    for b in cells:
13        self.update_vertex(b)
14        for u in self.grid.neighbors(b):
15            self.update_vertex(u)
16    return self.compute_shortest_path() if replan else 0

```

Listings 5.1 to 5.3 are the core of the chapter’s implementation; Listing 5.2 mirrors Algorithm 5.2 line by line, and Listing 5.3 is the change-processing part of Main, called with the cells whose status flipped after the robot has moved. The full file also contains LPA*, an A* baseline with both tie-breaking rules, the worked examples as functions and a self-test. The self-test draws random 20×20 grids for many seeds, applies random sequences of obstacle insertions and removals while the robot walks, and checks after every change that $g(s_{\text{start}})$ of D* Lite (and $g(s_{\text{goal}})$ of LPA*) equals the cost that A* computes from scratch, that the extracted path is valid and has that cost, that the queue holds exactly the inconsistent vertices with keys that are lower bounds, and that the first search expands exactly as many vertices as A* with the same tie-breaking (Theorem 5.9).

Blocking a cell changes all of its edges, in both directions

An obstacle that appears at cell b changes the edges *into* b and the edges *out of* b . Updating only the incoming edges (the natural thought: “nobody may enter b ”) leaves $rhs(b)$ and $g(b)$ finite in D* Lite, so b looks like a consistent cell with a valid goal distance; the neighbors’ rhs -values still ignore it because their edges into b cost ∞ , so the robot does not enter b today, but when the obstacle later disappears the neighbors are updated from a stale $g(b)$. Updating only the outgoing edges is wrong in the other direction: in LPA* the successors of b are corrected but b keeps a finite rhs through its unchanged incoming edges, and the backward path extraction of Proposition 5.4 happily routes *through* the obstacle. Treat a blocked cell as a vertex all of whose edges cost ∞ , and update the tail of every one of them.

Replanning can be slower than A*

Incremental search pays off when the change is small relative to the search. If the environment changes almost everywhere between two queries, for example when a fresh occupancy map arrives from a different sensor, or when the start and the goal both jump, D* Lite may expand many vertices twice (once to retract, once to recompute) and spends time in `UpdateVertex` on every affected neighbor. In such cases a fresh A* is faster and simpler. A robust replanning layer measures the fraction of changed cells and falls back to A* above a threshold; Exercise 5.8 asks you to find that threshold for your implementation. Remember also that the *first* search of D* Lite is more expensive than a well-tuned A* because of its tie-breaking (Section 5.8); an incremental planner pays this once and must recover it over the repairs.

5.11 Where this fits in the drone system

In the drone system

D* Lite is the *replanning layer* of the hybrid architecture in Chapter 24. The global planner (conflict-based search, CBS, or its bounded-suboptimal variant ECBS, Chapters 9 and 10) delivers a nominal time-indexed route; the prediction layer (Chapters 18 and 20) delivers, for each unknown vehicle, a predicted trajectory with uncertainty; the local layer (optimal reciprocal collision avoidance, ORCA, or the dynamic window approach, DWA, Chapters 13 and 14) executes short avoidance maneuvers. The replanning layer is invoked when the local maneuver cannot safely return the drone to its nominal route: reconnection is impossible, too costly, or would violate formation or communication constraints. It then runs `ComputeShortestPath` from the drone’s *current* cell and hands the new route back to the executive, which re-checks it for conflicts with the other swarm members.

“Edge costs change” has a precise meaning here. The prediction layer inflates the predicted intruder trajectory by the intruder’s radius, the drone’s radius and a margin proportional to the prediction uncertainty (Chapters 2 and 20), and marks every grid cell that this tube touches within the safety horizon as blocked, or as expensive if soft avoidance is preferred. Each such cell is one changed vertex whose edges are updated as in Section 5.10; when the prediction is revised or the intruder leaves, the cells are released again, which is a cost decrease that the same code handles. Because the drone keeps flying while this happens, its cell changes between calls, which is exactly what the key modifier was designed for. This is Week 2 of the study plan (Appendix A): implement or adapt D* Lite, change obstacles during execution, and compare with re-running A* (Exercise 5.7). Figure 5.6

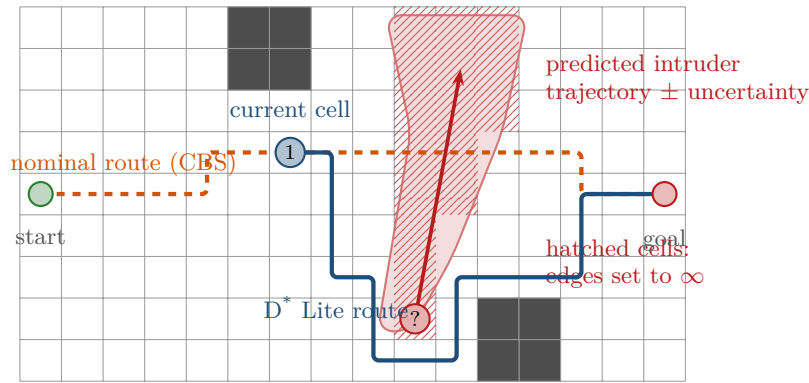


Figure 5.6. The replanning layer. A predicted intruder trajectory (red, with its uncertainty tube) is rasterized into blocked cells (hatched). The drone (blue) cannot rejoin its nominal route (dashed) with a local maneuver, so D* Lite is run from the drone’s current cell; only the cells around the tube are updated and the new route (solid blue) is returned to the executive for a conflict re-check: the drone slips below the predicted region through the one row the tube does not reach. The “?” marks the non-cooperative intruder, the “1” the drone under our control.

sketches the data flow.

5.12 Summary

Chapter summary

- Re-running A^* after a small change wastes work; an incremental search keeps the previous g -values and repairs only what the change invalidated.
- LPA* stores $g(s)$ and the one-step lookahead $rhs(s)$; a vertex is locally consistent if they agree, overconsistent if $g > rhs$ (a better value is available), underconsistent if $g < rhs$ (the old value is stale).
- The key $[\min(g, rhs) + h; \min(g, rhs)]$ orders the queue like A^* ’s f with g tie-breaking; using the smaller stored value processes stale vertices before the vertices that depend on them.
- `ComputeShortestPath` pops until the query vertex is consistent and no key is smaller; overconsistent pops accept rhs , underconsistent pops reset g to ∞ and update the vertex and its neighbors.
- D* Lite searches backward from the goal so that the robot may move; rhs looks at successors, expansions update predecessors, the heuristic is measured from the robot, and paths are read forward.
- The key modifier k_m accumulates $h(s_{last}, s_{start})$; it turns stale queue keys into lower bounds, which a reinsert check repairs, so the heap is never rebuilt.
- D* Lite returns the same path costs as A^* from scratch, expands each vertex at most twice per repair, and behaves exactly like A^* with ties broken toward the smaller g on the first search, which on grids is several times more expensive than A^* with the usual tie-breaking.
- Blocking a cell changes all of its edges in both directions; the tail of every changed edge must be updated. When almost everything changes, fall back to A^* .

Further reading. The original sources are the D* Lite paper [88], the journal version with the full proofs and the extension to unknown terrain [89], and the LPA* paper [90], which also contains an excellent discussion of why incremental heuristic search works. Stentz's D* [159] is the historical root. Anytime D* [106] and Field D* [42] are the two extensions most used on real robots. Textbook treatments of the underlying A* theory are in Russell and Norvig [142] and LaValle [99].

5.13 Exercises

Exercise 5.1 (★). For each of the following situations in LPA*, say whether the vertex s becomes overconsistent, underconsistent or stays consistent, and what its key is: (a) the cost of the edge from s 's best predecessor into s doubles; (b) a new predecessor with a cheaper total cost appears; (c) an edge *out of* s becomes ∞ ; (d) the best predecessor of s is popped as underconsistent and its g becomes ∞ , but s has a second predecessor with the same total cost.

Exercise 5.2 (★). In [Example 5.7](#) the robot walks from $(0, 3)$ to $(2, 3)$ and k_m stays 0 ([Table 5.5](#)). Explain why no key in U has to be touched during those two steps, and why [line 35](#) then adds 2 in one lump instead of $1 + 1$.

Exercise 5.3 (★★ Hand trace). Repeat [Example 5.6](#), starting from the state in the left panel of [Figure 5.3](#), but let cell $(1, 0)$ become blocked instead of $(3, 1)$. Give the vertices on which `UpdateVertex` is called, their new *rhs*-values and keys, and the full trace of `ComputeShortestPath`. How many expansions does the repair take, and why is the answer so different from [Table 5.2](#)? Then unblock $(3, 1)$ in the state of the right panel and trace the repair; which kind of inconsistency appears now?

Exercise 5.4 (★★). Construct a graph with edge costs, a start and a goal, and a single edge-cost increase such that LPA* with keys compared by k_1 only (ties broken arbitrarily) expands some vertex three times in one call of `ComputeShortestPath`, whereas the two-component key of [Definition 5.5](#) expands every vertex at most twice.

Exercise 5.5 (★★★ Proof). Let the robot have moved from s_{last} to s_{start} and let [Algorithm 5.2](#) execute [line 35](#). Prove that afterward every key stored in U is a lower bound on the key that `CalculateKey` would return for that vertex, assuming only that h satisfies the triangle inequality $h(a, c) \leq h(a, b) + h(b, c)$. Then show by an example with concrete numbers that without [line 35](#) some stored key can be a strict *upper* bound, and explain, using the proof sketch of [Theorem 5.8](#), which invariant breaks. Finally, explain why k_m must be accumulated from s_{last} rather than recomputed as $h(s_{\text{orig}}, s_{\text{start}})$ from the original start.

Exercise 5.6 (★★). Write down LPA* for the *reversed* graph (all edges flipped, goal and start swapped) and simplify the result. Show that you obtain [Algorithm 5.2](#) with $k_m \equiv 0$ and without the reinsert check, and identify precisely which lines of `Main` are needed only because the start moves.

Exercise 5.7 (★★★ Coding). This is the coding exercise of Week 2 of the study plan ([Appendix A](#)). Implement D* Lite on a 4-connected grid (or adapt `code/ch05_dstar_lite.py`), together with an A* baseline. Let an agent execute its plan one cell per tick. At random ticks, block a random free cell (with some probability, a cell on the agent's current path) or unblock a blocked one; never block the agent's cell or the goal. After every change, replan with D* Lite and, separately, with A* from the agent's cell, and record expansions, wall-clock time and path cost for both. (a) Verify on at least 100 random runs that the path costs agree. (b) Plot expansions and time per change for both methods, as in [Figure 5.5](#), for grid sizes 30, 50 and 100. (c) Extend the grid to 8-connectivity with octile costs and check that the Manhattan heuristic must be replaced; explain what goes wrong if it is not. The milestone

of the week is reached when your agent repairs its path rather than restarting every search.

Exercise 5.8 (★★★ Coding). Using your implementation from [Exercise 5.7](#), change a fraction ϕ of all free cells at once (block half of them, unblock half of the currently blocked cells) and measure the replanning time of D* Lite and of A* from scratch as a function of $\phi \in \{0.001, 0.01, 0.05, 0.1, 0.2, 0.5\}$ on a 100×100 grid. At which ϕ does D* Lite stop paying off? Explain the crossover with [Theorem 5.9](#), and propose a rule for the replanning layer of [Chapter 24](#) that chooses between the two.

Exercise 5.9 (★★). The prediction layer reports an intruder with predicted positions $\mathbf{p}_1, \dots, \mathbf{p}_H$ over the next H ticks and a growing uncertainty radius $r_t = r_0 + \sigma t$. (a) Describe how you would convert this into a set of changed cells for D* Lite, and how the set changes when the next prediction arrives one tick later. (b) D* Lite plans in space only; the intruder occupies each cell only during some ticks. Discuss what is lost by blocking the whole tube for the entire horizon, and sketch how a space-time variant ([Chapter 4](#)) could reduce the loss. (c) What must change in [Algorithm 5.2](#) if the *goal* of the drone is itself moving, and why does the backward search then lose its advantage?

Anytime Search: ARA*

Learning objectives

- Explain what an *anytime* planner delivers, and why a drone that must replan within a deadline needs one rather than an optimal planner that may finish late.
- Trace ARA* by hand: the inflated key $g + \varepsilon h$, the lists OPEN, CLOSED and INCONS, the termination test of `ImprovePath` and the bookkeeping between two iterations.
- State the ε -suboptimality theorem, follow its proof sketch, and compute the sharper certificate ε' from the search lists.
- Choose an ε schedule and predict its effect on the time to the first path, the number of published improvements and the total work, using the experiment of this chapter.
- Implement ARA* in Python as a generator that publishes every improved path together with its bound, and plot cost against computation time.
- Recognize the three classic mistakes: treating the inflated key as consistent, forgetting INCONS, and decreasing ε too fast.

6.1 Why this matters for a drone swarm

A drone of the swarm in [Chapter 24](#) is flying the path that the global planner (conflict-based search, CBS, or its bounded-suboptimal variant ECBS, [Chapters 9 and 10](#)) assigned to it when an unknown quadrotor crosses its route. The local layer (optimal reciprocal collision avoidance, ORCA, or the dynamic window approach, DWA, [Chapters 13 and 14](#)) swerves, and after two seconds the drone is three cells off its nominal path with the intruder still between it and the next waypoint. The replanning layer must now produce a new global path from the current cell, and it has one control cycle to do it, say 50 ms on a flight computer that is also running the tracker of [Chapter 18](#) and the attitude controller. D* Lite ([Chapter 5](#)) repairs the previous search quickly when the map changed a little and the drone moved a little; after an evasive maneuver both changes are large, and a repair can cost as much as a fresh search. Plain A* ([Chapter 4](#)) returns an optimal path, but it returns nothing at all until it has finished, and on a 60×60 grid with a third of the cells blocked it expands about 1 180 cells before it does ([Section 6.7.1](#)). If the cycle ends first, the drone has no path.

What the drone needs is a *safe path now and a better one when time permits*. Weighted A* with $w = 3$ delivers a path after about 215 expansions on the same grids, on average 8% longer than optimal, which is a perfectly good path to start flying. The question this chapter answers is how to keep improving that path in the remaining time *without throwing the first search away*, and how to know, at every moment, how far from optimal the current path can be. ARA* (Anytime Repairing A*, Likhachev, Gordon and Thrun [\[108\]](#)) does both: it runs

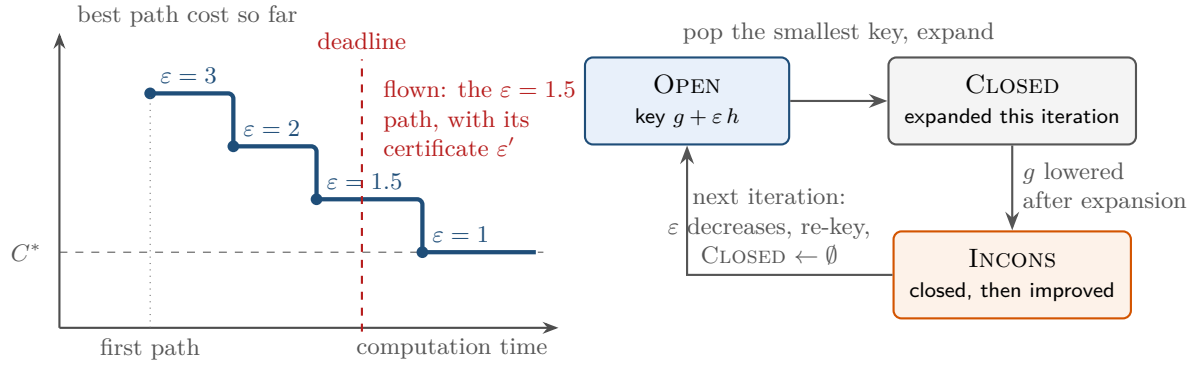


Figure 6.1. Left: an anytime planner publishes a first path early and better paths later; a deadline that arrives between two publications finds the last published path ready, together with its certificate ε' . Right: the three lists of ARA*. States wait in OPEN, are expanded once per iteration into CLOSED, and go to INCONS if a cheaper way to them turns up after they were expanded; when ε decreases, INCONS and OPEN are merged, their keys are recomputed and CLOSED is emptied.

weighted A* once with a large weight, then lowers the weight step by step and repairs the search instead of restarting it, and after every step it publishes the current path together with a certificate ε' such that the path costs at most ε' times the optimum. In the study plan (Appendix A) it is the optional second half of Week 2, next to D* Lite: one planner repairs the search when the *map* changes, the other when the *time budget* changes.

The chapter proceeds as follows. Section 6.2 explains the anytime idea and why restarting weighted A* wastes work, Section 6.3 fixes the notation, Section 6.4 gives the pseudocode, Section 6.5 runs three iterations by hand on an 8×6 grid, Section 6.6 proves the guarantees, Section 6.7 studies the ε schedule with an experiment, Section 6.8 introduces Anytime D*, and Sections 6.9 and 6.10 cover the implementation and the place of the algorithm in the drone system.

6.2 The idea in plain words

6.2.1 A path now, a better path later

An **anytime algorithm** is one that can be interrupted at any moment and still returns a useful answer: a first, possibly poor, solution appears quickly, the solution improves while the algorithm keeps running, and when it is stopped the best solution found so far is handed back. For a planner two more things are wanted. The improvements should not be random luck but should come with a *quality certificate*, a number ε' with the meaning “this path costs at most ε' times the optimal cost”; and the improvements should be cheap, so that the sequence of paths costs little more in total than a single optimal search would have. The left panel of Figure 6.1 shows the resulting picture: the cost of the best available path decreases in steps, one step per published solution, and a deadline that falls anywhere on the time axis finds a path ready.

6.2.2 Weighted A*, and why restarting it wastes work

Recall from Chapter 4 that weighted A* is A* with the key $g(n) + w h(n)$ for a weight $w \geq 1$ [132]. The weight makes the search greedy: cells that look close to the goal are expanded first, the frontier stops sweeping the whole band $f \leq C^*$ and dives toward the goal instead, and

the search ends after far fewer expansions. The price is bounded: when h is consistent the returned path costs at most $w \cdot C^*$ even though closed states are never re-opened, which is the case [Theorem 6.8](#) proves below; with a merely admissible h the bound needs re-opening ([Theorem 4.14](#)), and [Exercise 6.3](#) shows what breaks. On the random grids of [Chapter 4](#), $w = 2$ cut the effort from 3324 to 271 expansions on the 8-connected grid and from 756 to 372 on the 4-connected one, while lengthening the paths by 6 and 12 % respectively.

The obvious anytime planner is therefore: run weighted A^* with $w = 3$, publish the path, run it again with $w = 2$, publish, run it again with $w = 1$, publish the optimal path. [Section 6.7.1](#) measures this **restarted weighted A^*** on random 60×60 grids with the schedule $w = 3, 2.5, 2, 1.75, 1.5, 1.25, 1.1, 1$: it expands 3963 cells in total, 3.4 times as many as a single optimal A^* , because every run starts from nothing and expands again most of the cells that the previous run had already settled. Two kinds of work are repeated. First, the g -values are thrown away, although every one of them is the cost of a real path and remains a valid upper bound. Second, most of the states that the previous run expanded were expanded with a g that later runs never improve, so expanding them again produces nothing new.

6.2.3 What can be reused

ARA* keeps three things between runs: the g -values with their parent pointers, the frontier OPEN, and a third list INCONS ([Figure 6.1](#), right). The key observation is that after a search only the *inconsistent* states matter: a state whose g -value has been lowered since it was last expanded, or that has never been expanded at all. Expanding a consistent state again would relax the same edges with the same value and change nothing. The inconsistent states are exactly the states in OPEN plus those that were improved *after* they were expanded; weighted A^* with re-opening would put the latter back into OPEN at once, ARA* records them in INCONS and postpones their re-expansion to the next iteration. So the next iteration starts from $OPEN \cup INCONS$ with keys recomputed for the smaller ε , and everything else, every consistent state, keeps its value and costs nothing.

Key idea

ARA* is weighted A^* with a decreasing weight ε that never restarts. Each state is expanded at most once per iteration; a state improved after its expansion goes to INCONS instead of being re-expanded. When ε decreases, OPEN and INCONS are merged and re-keyed, CLOSED is emptied, and the search continues from where it stopped. Every published path costs at most $\varepsilon' C^*$ with $\varepsilon' = \min(\varepsilon, g(\gamma) / \min_{n \in OPEN \cup INCONS} (g(n) + h(n)))$.

6.3 Problem statement and notation

We use the notation of [Chapter 4](#): a finite directed graph $G = (V, E)$ with edge costs $c(n, n') \geq 0$, a start s , a goal γ , the successor function $\text{Succ}(n)$, the cost $g^*(n)$ of a cheapest path from s to n , the cost $h^*(n)$ of a cheapest path from n to γ , the optimal solution cost $C^* = g^*(\gamma) = h^*(s)$, and a heuristic $h: V \rightarrow \mathbb{R}_{\geq 0}$ that is **consistent**: $h(\gamma) = 0$ and $h(n) \leq c(n, n') + h(n')$ on every edge. Consistency is assumed throughout the chapter; [Section 6.6](#) shows where it is used and [Exercise 6.3](#) what breaks without it. During the search, $g(n)$ is the cost of the best path from s to n found so far (∞ if none), with a parent pointer $\text{parent}(n)$ that records the predecessor on that path. On grids the nodes are cells (x, y) , and the worked example uses an 8-connected grid with move costs 1 and $\sqrt{2}$, the corner-cutting rule of [Chapter 2](#) and the octile distance of [Chapter 4](#) as h .

Definition 6.1 (Anytime search with certificates). Given a heuristic search problem and a **schedule** of inflation factors $\varepsilon_1 > \varepsilon_2 > \dots > \varepsilon_K = 1$, an anytime planner with certificates outputs a sequence of solution paths $P_1, P_2, \dots, P_K$ together with numbers $\varepsilon'_1, \varepsilon'_2, \dots, \varepsilon'_K$ such that $\text{cost}(P_k) \leq \varepsilon'_k \cdot C^*$ for every k and $\varepsilon'_K = 1$. The planner may be interrupted after any P_k ; the last published pair (P_k, ε'_k) is its answer.

Definition 6.2 (Inflated key). For an inflation factor $\varepsilon \geq 1$ the **inflated key** of a state is

$$f_\varepsilon(n) = g(n) + \varepsilon h(n). \quad (6.1)$$

For $\varepsilon = 1$ it is the f of A^* ; for $\varepsilon > 1$ it is the key of weighted A^* with $w = \varepsilon$. In ARA^* the value of ε changes between iterations, so f_ε of a state in OPEN must be recomputed whenever ε does. Pearl and Kim's A^*_ε [127] writes the slack as ε , so its bound reads $1 + \varepsilon$; here ε is the multiplier itself and the bound is εC^* .

Definition 6.3 (Consistent and inconsistent states). Let $v(n)$ denote the value that $g(n)$ had when n was expanded for the last time, and $v(n) = \infty$ if n has never been expanded. A state n with $g(n) < \infty$ is **consistent** if $v(n) = g(n)$ and **inconsistent** if $v(n) > g(n)$, that is, if a cheaper path to n has been found since its last expansion, or it has never been expanded. (Since g only decreases, $v(n) < g(n)$ never happens.) ARA^* keeps the inconsistent states in two lists: OPEN holds those that may still be expanded in the current iteration, and INCONS holds those that were already expanded in the current iteration. Together, $\text{OPEN} \cup \text{INCONS}$ is exactly the set of inconsistent states.

The word *inconsistent* is used here in the sense of Lifelong Planning A^* (LPA*) and D* Lite (Chapter 5), where a vertex is inconsistent when its stored g disagrees with the value its predecessors would give it. It is a different notion from a consistent *heuristic*; the two meet in Section 6.9, where inflating a consistent heuristic is what makes states inconsistent.

Definition 6.4 (Lower bound and certificate). When an iteration of ARA^* ends, let

$$L = \min_{n \in \text{OPEN} \cup \text{INCONS}} (g(n) + h(n)) \quad (6.2)$$

($L = \infty$ if both lists are empty) and define the **certificate**

$$\varepsilon' = \min\left(\varepsilon, \frac{g(\gamma)}{L}\right). \quad (6.3)$$

ARA^* never expands γ , because the loop stops as soon as the goal holds the smallest key, so a finite $g(\gamma)$ means $\gamma \in \text{OPEN}$ and therefore $L \leq g(\gamma) < \infty$ and $\varepsilon' \geq 1$. Both lists can be empty, $L = \infty$, only when γ was never generated, which is the failure exit on line 13.

Section 6.6 shows that $L \leq C^*$ unless the current path is already optimal, so ε' is a valid bound: the published path costs at most $\varepsilon' C^*$. In the worked example ε' is 1.65 after the first iteration although $\varepsilon = 3$, and on the random grids of Section 6.7.1 it is 1.41 for $\varepsilon = 3$. The certificate is what the drone's executive reads; the inflation factor is only what the search was asked for.

6.4 The algorithm

ARA^* consists of an inner loop **ImprovePath**, which is weighted A^* without re-expansions (Algorithm 6.1), and an outer loop **ARAStar**, which runs through the schedule, does the bookkeeping between two values of ε and publishes the paths (Algorithm 6.2). The pseudocode follows Likhachev, Gordon and Thrun [108], with parent pointers added so that the path can

be read off.

Algorithm 6.1. ARA*, inner loop: weighted A* without re-expansions

Input: graph G , goal γ , consistent heuristic h , current ε ; the global tables g , parent and the lists OPEN, CLOSED, INCONS

Output: on return, $g(\gamma) \leq \varepsilon C^*$ and the parent pointers from γ give an ε -suboptimal path

```

1 Function fvalue( $n$ ):
2   return  $g(n) + \varepsilon h(n)$ 
3 Function ImprovePath():
4   while fvalue( $\gamma$ ) >  $\min_{n \in \text{OPEN}} \text{fvalue}(n)$  do
5      $n \leftarrow$  remove the state with the smallest fvalue from OPEN
6     add  $n$  to CLOSED
7     foreach  $(n', c(n, n')) \in \text{Succ}(n)$  do
8       if  $g(n) + c(n, n') < g(n')$  then
9          $g(n') \leftarrow g(n) + c(n, n')$ ; parent( $n'$ )  $\leftarrow n$ 
10        if  $n' \notin \text{CLOSED}$  then
11           $\hookrightarrow$  insert  $n'$  into OPEN with key fvalue( $n'$ ), or update its key
12        else
13           $\hookrightarrow$  insert  $n'$  into INCONS    // improved after its expansion: postponed

```

Walkthrough of ImprovePath. The loop condition on line 4 replaces the goal test of A*. A* stops when it *pops* the goal; ImprovePath stops as soon as the goal's key is no larger than the smallest key in OPEN, which is the moment at which A* would pop it. The difference is that the goal is never removed from OPEN: it stays there with its key, so that the next iteration, with a smaller ε , can test it again. A consequence worth remembering is that γ is never expanded, and once it has a finite g it is always in OPEN. Line 5 pops the best state and line 6 closes it; a closed state is not popped again in this iteration. The relaxation on line 8 is that of A*: a strictly cheaper path to n' updates $g(n')$ and the parent pointer *whatever list n' is in*; only what happens next depends on the list. If n' is not closed it goes into OPEN (line 11) with a fresh key, or its key is updated if it is there already. If n' is closed (line 13), A* with re-opening would put it back into OPEN and expand it a second time; ARA* puts it into INCONS instead. Its successors keep their old, larger g -values for the rest of this iteration, which is exactly what makes the guarantee of this iteration ε -suboptimal rather than optimal, and what makes each iteration cheap. Figure 6.2 shows the situation in which line 13 fires in the worked example.

Walkthrough of ARAStar. Lines 2–11 of the first pass are weighted A* with $w = \varepsilon_1$ without re-opening, so the first published path is exactly the path that weighted A* would return, after the same number of expansions. Between iterations four things happen (lines 7–10): ε is decreased; the states of INCONS, whose improvements have not yet been propagated, are merged into OPEN; the keys of all states in OPEN are recomputed for the new ε , since a smaller ε changes the order of the queue; and CLOSED is emptied, so that every state may be expanded once more. Nothing else is touched: the g -values and parent pointers of all states, including the goal, survive. The certificate on line 14 is computed from the lists at the end of the iteration and published with the path on line 16. Line 18 is an early exit: if the certificate already says 1, the rest of the schedule cannot improve anything, which happens regularly in

Algorithm 6.2. ARA*, outer loop: the schedule, the bookkeeping and the certificates

Input: graph G , start s , goal γ , consistent heuristic h , schedule $\varepsilon_1 > \dots > \varepsilon_K = 1$
Output: one path with certificate per iteration; the last one is optimal

```

1 Function ARAStar( $G, s, \gamma, h, (\varepsilon_1, \dots, \varepsilon_K)$ ):
2    $g(n) \leftarrow \infty$  for all  $n$ ;  $g(s) \leftarrow 0$ ;  $\text{parent}(s) \leftarrow \text{nil}$ 
3   OPEN  $\leftarrow \emptyset$ ; CLOSED  $\leftarrow \emptyset$ ; INCONS  $\leftarrow \emptyset$ ;  $\varepsilon \leftarrow \varepsilon_1$ 
4   insert  $s$  into OPEN with key  $\text{fvalue}(s)$ 
5   for  $k \leftarrow 1$  to  $K$  do
6     if  $k > 1$  then
7        $\varepsilon \leftarrow \varepsilon_k$ 
8       move all states of INCONS into OPEN
9       recompute the key  $\text{fvalue}(n)$  of every  $n \in \text{OPEN}$ 
10      CLOSED  $\leftarrow \emptyset$ 
11      ImprovePath()
12      if  $g(\gamma) = \infty$  then
13        return failure                                // no path exists; OPEN ran empty
14       $L \leftarrow \min_{n \in \text{OPEN} \cup \text{INCONS}} (g(n) + h(n))$ 
15       $\varepsilon' \leftarrow \min(\varepsilon, g(\gamma)/L)$            // the certificate of Equation (6.3)
16      Publish(PointerPath( $\text{parent}, \gamma$ ),  $\varepsilon'$ )
17      if  $\varepsilon' \leq 1$  then
18        return                                       // optimal: stop early

```

practice (Section 6.7). If no path exists, the first call of `ImprovePath` empties OPEN, the loop condition $\infty > \infty$ is false, and line 13 reports the failure.

The published path. `PointerPath` follows the parent pointers back from γ to s . This path may be *cheaper* than $g(\gamma)$ says: when a state on it was improved and sent to INCONS, its parent pointer changed but the g -values of its descendants, including $g(\gamma)$, were not updated. The pointer path is always a valid path of cost at most $g(\gamma)$ (Proposition 6.7), so publishing it can only help; the certificate is still computed from $g(\gamma)$, which is the quantity the theorem bounds. Table 6.1 summarizes how one call of `ImprovePath` differs from the A* of Chapter 4.

6.5 A worked example

Example 6.5 (ARA* on an 8×6 grid). Figure 6.3 shows an 8-connected 8×6 grid with ten blocked cells, $(0, 2)$, $(0, 4)$, $(2, 0)$, $(2, 5)$, $(3, 4)$, $(4, 1)$, $(5, 2)$, $(6, 0)$, $(6, 3)$ and $(6, 4)$. The start is $s = (0, 3)$, the goal $\gamma = (7, 2)$; straight moves cost 1, diagonal moves $\sqrt{2} \approx 1.41$ and are allowed only when neither cell they cut across is blocked. The heuristic is the octile distance, so $h(s) = 7 + (\sqrt{2} - 1) \cdot 1 = 7.41$, and the schedule is $\varepsilon = 3, 2, 1$. Ties in the key are broken toward the smaller h , and among equal keys the state pushed first is popped first. The instance is `example_grid()` in `ch06_arastar.py`, and every number below is produced by its self-test. The optimal cost is $C^* = 6 + 3\sqrt{2} = 10.24$.

First iteration, $\varepsilon = 3$. Table 6.2 lists every expansion; three moments matter. The greedy dive: from the start, key $0 + 3 \cdot 7.41 = 22.24$, through its only free neighbor $(1, 3)$, whose diagonal successors $(2, 2)$ and $(2, 4)$ have the larger g but the smaller keys because h counts three times, on to $(3, 2)$ and $(4, 2)$, the keys falling to 13.41 in five steps. The wall: the obstacle

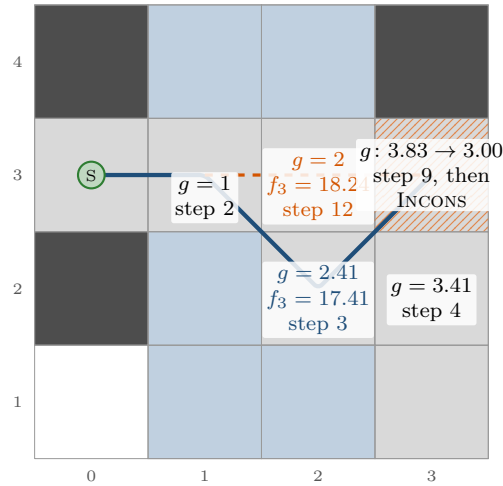


Figure 6.2. How inflation creates an inconsistent state (cells of Example 6.5, first iteration, $\varepsilon = 3$). The cell (3,3) can be reached from (1,3) by two diagonal moves through (2,2) (blue, reaching it with $g = 1 + 2\sqrt{2} = 3.83$) or by two straight moves through (2,3) (orange, $g = 3$). With $\varepsilon = 3$ the diagonal neighbor has the smaller key ($17.41 < 18.24$), because it is closer to the goal in h , so (2,2) is expanded in step 3 and (3,3) receives $g = 3.83$ and is expanded in step 9. Only in step 12 is (2,3) expanded and the cheaper way found: (3,3) is closed, so its value is corrected to 3.00 and it is moved to INCONS (hatched) rather than expanded again. With $\varepsilon = 1$ both neighbors would have the key 7.41 and the cheap way would be found in time.

(5,2) ends the dive, the next key jumps to 15.07, the climb to (5,4) in step 8 meets the segment (6,3)–(6,4), and the search falls back on cells with keys around 17. And step 12, which expands (2,3), in OPEN since step 2 with key 18.24: it finds the cheaper way to (3,3), $g = 3.00$ instead of 3.83 (Figure 6.2), so (3,3), being closed, goes to INCONS while its successors, (4,3) with $g = 4.83$ among them, keep their values. Five more expansions cross the wall and generate the goal with $g(\gamma) = 12.24 = 8 + 3\sqrt{2}$, the smallest key in OPEN, and the loop stops after 17 expansions with the path of the left panel of Figure 6.3. The certificate: over the eight cells in OPEN and (3,3) in INCONS the smallest $g + h$ is $L = 3.00 + 4.41 = 7.41$, at (3,3), so $\varepsilon' = \min(3, 12.24/7.41) = 1.65$, against a true ratio of $12.24/10.24 = 1.20$.

Second iteration, $\varepsilon = 2$. The nine inconsistent states are re-keyed with $\varepsilon = 2$. The key of the goal is still 12.24; the smallest key belongs to (3,3) from INCONS, $3.00 + 2 \cdot 4.41 = 11.83$,

Table 6.1. One call of ImprovePath compared with the graph-search A* of Chapter 4. The middle row is the defining difference; the others follow from it.

	A* (Chapter 4)	ImprovePath (Algorithm 6.1)
Key of OPEN	$g + h$	$g + \varepsilon h$, recomputed when ε changes
Cheaper path to a closed state	re-open it and expand it again	record it, put the state into INCONS, expand it in the next iteration
Expansions per state	at most once (consistent h)	at most once per iteration
Termination	the goal is popped	$f_\varepsilon(\gamma) \leq \min_{\text{OPEN}} f_\varepsilon$; the goal stays in OPEN
Guarantee on return	optimal	$g(\gamma) \leq \varepsilon C^*$, and $\leq \varepsilon' C^*$ with $\varepsilon' \leq \varepsilon$
State kept for the next call	nothing	g , parents, OPEN, INCONS

Table 6.2. Complete trace of Algorithm 6.1 on Example 6.5, one row per expansion. Columns: the expanded cell n with its g , h and key $f_\varepsilon = g + \varepsilon h$; the successors whose g is lowered by the expansion (with the new value; a cell marked I was closed and is moved to INCONS); and the smallest key in OPEN after the step. An iteration stops as soon as that key is at least the key of the goal.

Step	n	g	h	f_ε	Successors improved (cell: g)	$\min_{\text{OPEN}} f_\varepsilon$
<i>Iteration 1, $\varepsilon = 3$; OPEN holds the start only</i>						
1	(0, 3)	0.00	7.41	22.24	(1, 3):1.00	20.24
2	(1, 3)	1.00	6.41	20.24	(2, 3):2.00, (1, 4):2.00, (1, 2):2.00, (2, 4):2.41, (2, 2):2.41	17.41
3	(2, 2)	2.41	5.00	17.41	(3, 2):3.41, (2, 1):3.41, (3, 3):3.83, (3, 1):3.83, (1, 1):3.83	15.41
4	(3, 2)	3.41	4.00	15.41	(4, 2):4.41, (4, 3):4.83	13.41
5	(4, 2)	4.41	3.00	13.41	—	15.07
6	(4, 3)	4.83	3.41	15.07	(5, 3):5.83, (4, 4):5.83, (5, 4):6.24	13.07
7	(5, 3)	5.83	2.41	13.07	—	14.73
8	(5, 4)	6.24	2.83	14.73	(5, 5):7.24, (4, 5):7.66	17.07
9	(3, 3)	3.83	4.41	17.07	—	17.07
10	(3, 1)	3.83	4.41	17.07	(3, 0):4.83	17.31
11	(4, 4)	5.83	3.83	17.31	(4, 5):6.83	18.24
12	(2, 3)	2.00	5.41	18.24	(3, 3):3.00 I	18.73
13	(5, 5)	7.24	3.83	18.73	(6, 5):8.24	18.49
14	(6, 5)	8.24	3.41	18.49	(7, 5):9.24	18.24
15	(7, 5)	9.24	3.00	18.24	(7, 4):10.24	16.24
16	(7, 4)	10.24	2.00	16.24	(7, 3):11.24	14.24
17	(7, 3)	11.24	1.00	14.24	(7, 2):12.24	12.24
stop: $f_3(\gamma) = 12.24 \leq 12.24$; $L = 7.41$ from (3, 3), $\varepsilon' = 1.65$						
<i>Iteration 2, $\varepsilon = 2$; OPEN holds the eight open cells and (3, 3) from INCONS, re-keyed</i>						
1	(3, 3)	3.00	4.41	11.83	(4, 3):4.00	10.83
2	(4, 3)	4.00	3.41	10.83	(5, 3):5.00, (4, 4):5.00, (5, 4):5.41	9.83
3	(5, 3)	5.00	2.41	9.83	—	11.07
4	(5, 4)	5.41	2.83	11.07	(5, 5):6.41	12.24
stop: $f_2(\gamma) = 12.24 \leq 12.24$; $L = 8.00$ from (1, 2), $\varepsilon' = 1.53$; pointer path costs 11.41						
<i>Iteration 3, $\varepsilon = 1$; OPEN holds ten cells, re-keyed</i>						
1	(1, 2)	2.00	6.00	8.00	(1, 1):3.00	8.24
2	(2, 4)	2.41	5.83	8.24	—	8.83
3	(4, 4)	5.00	3.83	8.83	(4, 5):6.00	8.83
4	(2, 1)	3.41	5.41	8.83	—	8.83
5	(1, 4)	2.00	6.83	8.83	(1, 5):3.00	9.41
6	(1, 1)	3.00	6.41	9.41	(0, 1):4.00, (1, 0):4.00, (0, 0):4.41	9.66
7	(3, 0)	4.83	4.83	9.66	(4, 0):5.83	9.66
8	(4, 0)	5.83	3.83	9.66	(5, 0):6.83	9.66
9	(5, 0)	6.83	2.83	9.66	(5, 1):7.83	10.24
10	(5, 1)	7.83	2.41	10.24	(6, 1):8.83	10.24
11	(6, 1)	8.83	1.41	10.24	(7, 1):9.83, (6, 2):9.83, (7, 2):10.24	10.24
stop: $f_1(\gamma) = 10.24 \leq 10.24$; $L = 10.24$, $\varepsilon' = 1$: optimal						

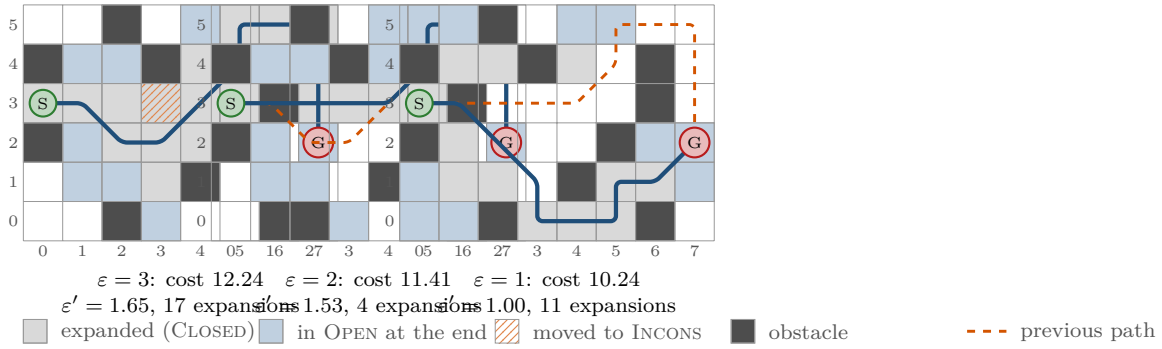


Figure 6.3. The three iterations of ARA* on Example 6.5. Each panel shows the cells expanded in that iteration (gray), the cells in OPEN when the iteration stopped (blue), the cell moved to INCONS (hatched), the published path (solid) and the previously published path (dashed). With $\epsilon = 3$ the search dives greedily along the row of the goal, is stopped by the wall at $x = 6$, climbs over it and publishes a path of cost 12.24 after 17 expansions. With $\epsilon = 2$ four expansions propagate the correction stored in INCONS and shorten the path to 11.41. With $\epsilon = 1$ eleven expansions of the cheap cells that the greedy search had skipped find the optimal route below the wall, cost 10.24.

and it is expanded first. This is the postponed re-expansion: it lowers $(4, 3)$ from 4.83 to 4.00, and the next three expansions carry the correction on to $(5, 3)$, $(4, 4)$, $(5, 4)$ and $(5, 5)$, each of which is in OPEN again because CLOSED was emptied. After step 4 the smallest key in OPEN is 12.24, the goal's own key: the cells at the top of the wall, $(5, 5)$ and $(6, 5)$ onward, are not re-expanded because their keys $6.41 + 2 \cdot 3.83 = 14.07$ and higher exceed it. So $g(\gamma)$ stays at 12.24, but the parent pointers now run $\gamma \rightarrow (7, 3) \rightarrow \dots \rightarrow (5, 5) \rightarrow (5, 4) \rightarrow (4, 3) \rightarrow (3, 3) \rightarrow (2, 3) \rightarrow (1, 3) \rightarrow s$, and the pointer path costs $10 + \sqrt{2} = 11.41$: the published path is cheaper than its own g -value, the effect described in Section 6.4. Four expansions, against 20 for weighted A* with $w = 2$ started from scratch, and the certificate improves to $\epsilon' = \min(2, 12.24/8.00) = 1.53$, with $L = 8.00$ now attained by $(1, 2)$.

Third iteration, $\epsilon = 1$. With $\epsilon = 1$ the keys are the plain f -values of A*, and the cells that the greedy first iteration skipped, $(1, 2)$, $(2, 4)$, $(2, 1)$, $(1, 4)$ and $(1, 1)$ with f between 8 and 9.41, are five of the first six expansions, most of them without improving anything: this is the price of the guarantee, A* must rule them out. From step 7 on the search follows the bottom corridor $(3, 0)$, $(4, 0)$, $(5, 0)$, $(5, 1)$, $(6, 1)$, whose cells had been generated in the first iteration with their optimal g -values but never expanded, and step 11 generates the goal with $g(\gamma) = 10.24$. The smallest key in OPEN is now 10.24, the loop stops, and $L = 10.24$ gives $\epsilon' = 1$: the path is optimal. Eleven expansions, 32 in total for the three iterations. Plain A* with the same tie-breaking, which is what `weighted_astar(grid, start, goal, 1.0)` runs, needs 22 expansions for this instance — the reference A* of the self-test breaks ties differently and needs 25 — and restarting weighted A* for each ϵ would need $17 + 20 + 22 = 59$. Table 6.3 collects the numbers.

The pattern carries over to every instance: the first iteration is greedy and cheap and its certificate comes from the lists, not from ϵ ; the middle iteration only propagates corrections; and the last one pays for optimality by expanding the cheap cells near the start that the greedy search skipped, exactly the cells that A* expands with $f < C^*$ (Chapter 4).

Table 6.3. The three iterations of [Example 6.5](#): expansions, the sizes of the lists when the iteration stops, the value $g(\gamma)$ and the cost of the pointer path, the lower bound L , the certificate ε' and the true ratio to $C^* = 10.24$. In iteration 2 the pointer path is cheaper than $g(\gamma)$.

k	ε_k	Exp.	Total	OPEN	INCONS	$g(\gamma)$	Path cost	L	ε'_k	Cost/ C^*
1	3	17	17	8	$\{(3, 3)\}$	12.24	12.24	7.41	1.65	1.20
2	2	4	21	10	$\emptyset$	12.24	11.41	8.00	1.53	1.11
3	1	11	32	9	$\emptyset$	10.24	10.24	10.24	1.00	1.00

6.6 Properties: bounds, completeness, complexity

Throughout this section G is finite, the edge costs are non-negative and h is consistent. The results and the proofs are those of Likhachev, Gordon and Thrun [108], in the journal form of [107], adapted to the notation of [Chapter 4](#). We first collect the invariants that the two algorithms maintain.

Lemma 6.6 (Invariants of ARA*). *At every moment during [Algorithm 6.2](#):*

- (i) $g(n)$ never increases, and whenever it is finite it is the cost of the path recorded by the parent pointers, so $g(n) \geq g^*(n)$;
- (ii) if n has been expanded, with the value $v(n)$ of [Definition 6.3](#), then $g(n') \leq v(n) + c(n, n')$ for every successor n' of n ;
- (iii) $\text{OPEN} \cup \text{INCONS}$ is the set of inconsistent states, and OPEN and INCONS are disjoint; a state in INCONS was expanded in the current iteration.

Proof sketch. (i) is line 8 of [Algorithm 6.1](#): the only assignment to $g(n')$ is a strict decrease to the cost of a real path. (ii) holds at the moment n is expanded, because the relaxation sets $g(n') \leq g(n) + c(n, n') = v(n) + c(n, n')$, and it keeps holding because $g(n')$ never increases. (iii) A state enters OPEN when it is generated ($v = \infty > g$) or when its g decreases while it is not closed; it enters INCONS when its g decreases while it is closed, that is, after its expansion in this iteration; in both cases it is inconsistent. A state leaves OPEN only by being expanded, which sets $v(n) = g(n)$ and makes it consistent, and leaves INCONS only on line 8 of [Algorithm 6.2](#), into OPEN . Conversely an inconsistent state has been improved after its last expansion, or generated and never expanded, and both events put it into one of the two lists; nothing removes it from them except an expansion. Emptying CLOSED on line 10 does not change any g and so does not change which states are inconsistent. $\square$

Proposition 6.7 (The pointer path). *If $g(\gamma) < \infty$, the parent pointers from γ lead back to s along a simple path whose cost is at most $g(\gamma)$.*

Proof. When $\text{parent}(n') \leftarrow n$ is set, $g(n') = g(n) + c(n, n')$; afterward $g(n)$ can only decrease, so $g(n) + c(n, n') \leq g(n')$ holds from then on for every pointer, and summing along the pointers gives a path cost of at most $g(\gamma)$. The chain cannot cycle. Suppose it closed one and look at the moment its last pointer was set, say $\text{parent}(u) \leftarrow p$: that relaxation was strict, $g(p) + c(p, u) < g(u)$ for the value $g(u)$ had a moment earlier. The other pointers of the cycle were already in place, so telescoping the inequality above from p around the cycle back to u gives $g(u) \leq g(p)$ for non-negative costs, and with the strict relaxation that forces $c(p, u) < 0$, which is impossible. So the chain ends at the only state without a parent, s . $\square$

Theorem 6.8 (ε -suboptimality; Likhachev, Gordon and Thrun 2003). *Whenever [Algorithm 6.1](#) selects a state n for expansion, $g(n) \leq \varepsilon g^*(n)$. When it returns with $g(\gamma) < \infty$, then $g(\gamma) \leq \varepsilon C^*$.*

Proof sketch. Fix one call of **ImprovePath** and consider its expansions in order. We show the following claim by induction over that order: *at any moment at which some state n satisfies $f_\varepsilon(n) \leq \min_{m \in \text{OPEN}} f_\varepsilon(m)$, it holds that $g(n) \leq \varepsilon g^*(n)$.* A state selected for expansion satisfies the premise, since it holds the minimum key; so does γ at the moment the loop stops, which gives the second statement because $g^*(\gamma) = C^*$.

Suppose the claim fails for n and let $P = (s = n_0, n_1, \dots, n_k = n)$ be a cheapest path to n , so that $g^*(n_{i+1}) = g^*(n_i) + c(n_i, n_{i+1})$. Let j be the smallest index with $g(n_j) > \varepsilon g^*(n_j)$; since $g(s) = 0$ we have $j \geq 1$, and $m = n_{j-1}$ satisfies $g(m) \leq \varepsilon g^*(m)$. By Lemma 6.6(iii) exactly one of three cases applies to m .

Case 1: m is consistent, $v(m) = g(m)$. By Lemma 6.6(ii), $g(n_j) \leq g(m) + c(m, n_j) \leq \varepsilon g^*(m) + c(m, n_j) \leq \varepsilon (g^*(m) + c(m, n_j)) = \varepsilon g^*(n_j)$, contradicting the choice of j .

Case 2: $m \in \text{OPEN}$. Consistency of h applied along the segment of P from m to n , as in the proof that consistency implies admissibility in Chapter 4, gives $h(m) \leq (g^*(n) - g^*(m)) + h(n)$. Therefore

$$\begin{aligned} f_\varepsilon(m) &= g(m) + \varepsilon h(m) \leq \varepsilon g^*(m) + \varepsilon (g^*(n) - g^*(m)) + \varepsilon h(n) \\ &= \varepsilon g^*(n) + \varepsilon h(n) < g(n) + \varepsilon h(n) = f_\varepsilon(n), \end{aligned}$$

using $g(n) > \varepsilon g^*(n)$ in the last step. But $m \in \text{OPEN}$ and $f_\varepsilon(n)$ is at most the smallest key in OPEN : a contradiction.

Case 3: $m \in \text{INCONS}$. Then m was expanded earlier in this call with a value $v(m) > g(m)$, and by Lemma 6.6(ii) $g(n_j) \leq v(m) + c(m, n_j)$. Combined with $g(n_j) > \varepsilon g^*(n_j) = \varepsilon g^*(m) + \varepsilon c(m, n_j) \geq \varepsilon g^*(m) + c(m, n_j)$ this gives $v(m) > \varepsilon g^*(m)$: when m was selected for expansion, it violated the claim. That contradicts the induction hypothesis for that earlier expansion. (For the first expansion of the call INCONS is empty, so the case cannot occur and the induction has a base.)

All three cases are contradictory, so the claim holds. $\square$

The proof uses the consistency of h in Case 2 only, and for the *uninflated* heuristic; with a merely admissible heuristic the theorem is false (Exercise 6.3). It holds for every call, whatever g -values were inherited from earlier iterations with larger ε : Cases 1 and 2 do not care when a value was computed.

Proposition 6.9 (The certificate is valid). *When Algorithm 6.1 returns with $g(\gamma) < \infty$, either $g(\gamma) = C^*$ or $L \leq C^*$, with L as in Equation (6.2). In both cases $g(\gamma) \leq \varepsilon' C^*$ with ε' as in Equation (6.3), and $L \leq g(\gamma)$, so $\varepsilon' \geq 1$.*

Proof. Let $P = (s = n_0, \dots, n_k = \gamma)$ be an optimal path and u the first inconsistent state on P ; it exists, because γ is never expanded (Definition 6.4) and is therefore inconsistent as soon as $g(\gamma) < \infty$. Telescoping Lemma 6.6(ii) with $v = g$ over the consistent states before u gives $g(u) \leq g^*(u)$, hence $g(u) = g^*(u)$ by Lemma 6.6(i), and Lemma 6.6(iii) puts $u \in \text{OPEN} \cup \text{INCONS}$. Admissibility of h (implied by consistency) gives $g(u) + h(u) \leq g^*(u) + h^*(u) = C^*$, hence $L \leq C^*$. If $u = \gamma$ that reads $g(\gamma) = C^* \leq \varepsilon' C^*$; otherwise $g(\gamma) = (g(\gamma)/L) L \leq (g(\gamma)/L) C^*$, and $g(\gamma) \leq \varepsilon C^*$ by Theorem 6.8, so the minimum of the two factors works. Finally γ is never expanded, so it is in OPEN with $g(\gamma) + h(\gamma) = g(\gamma)$, and $L \leq g(\gamma)$. $\square$

Proposition 6.10 (Expansions, completeness, termination). *Let $\varepsilon \geq 1$. Then:*

- (i) *In one call of **ImprovePath** every state is expanded at most once; the call performs at most $|V|$ expansions and $|E|$ relaxations and terminates.*
- (ii) *For every $\varepsilon \geq 1$, **ImprovePath** returns with $g(\gamma) < \infty$ if and only if γ is reachable from s ; ARA* is complete.*
- (iii) *Over the iterations $g(\gamma)$ never increases, and the iteration with $\varepsilon = 1$ returns $g(\gamma) = C^*$:*

the last published path is optimal.

- (iv) *If, after the keys have been recomputed for a new ε , the key of γ is at most the smallest key in OPEN, the iteration performs no expansion at all.*

Proof. (i) A state is expanded only when it is popped from OPEN and it is then put into CLOSED; while it is closed, an improvement sends it to INCONS, never to OPEN (line 13), and CLOSED is emptied only between calls. Each expansion relaxes the out-edges of a distinct state. (ii) If γ is unreachable, no path to it is ever recorded and $g(\gamma)$ stays ∞ . If it is reachable, suppose the call returns with $g(\gamma) = \infty$; the loop can only have stopped with OPEN empty. Let u be the first state on a cheapest path to γ with $g(u) = \infty$ and p its predecessor on that path, which has a finite g ($u \neq s$ because $g(s) = 0$). If p is consistent it has been expanded, and Lemma 6.6(ii) gives $g(u) \leq v(p) + c(p, u) < \infty$. If p is inconsistent it lies in $\text{OPEN} \cup \text{INCONS}$ by Lemma 6.6(iii); OPEN is empty, so p is in INCONS and was expanded in this call, and again $g(u) < \infty$. Both contradict the choice of u . (iii) is Lemma 6.6(i) together with Theorem 6.8 for $\varepsilon = 1$ and $g(\gamma) \geq C^*$. (iv) is the loop condition on line 4, which is false at once. $\square$

Part (iv) is why consecutive iterations of the experiment in Section 6.7.1 can cost nothing, and part (iii) says what the last iteration is: plain A^* on the inconsistent states.

Complexity. With a binary heap one iteration takes $\mathcal{O}((|V| + |E|) \log |V|)$ time, since it performs at most $|V|$ expansions and $|E|$ relaxations, like A^* (Chapter 4); re-keying OPEN between iterations is linear in the size of OPEN when the heap is rebuilt from scratch, and the certificate is a linear scan of the lists. With K inflation factors the worst case is K times the cost of A^* , and the memory is that of a single A^* : one g -value, one parent pointer and one list membership per touched state. The worst case is not what happens: Section 6.7.1 finds a total of 1.3 times the work of A^* for eight iterations. What the analysis cannot promise is that the total stays *below* A^* : the first, greedy iteration expands states with $g^* + h > C^*$ that A^* never touches, and the last iteration must still expand every state with $g^* + h < C^*$ that the earlier ones skipped (Chapter 4). The anytime property has a price; the schedule of the next section decides how small it is.

6.7 The ε schedule

6.7.1 An experiment on random grids

The script `gen_ch06_anytime.py` runs ARA* and restarted weighted A^* on 25 random 60×60 grids, 8-connected with the octile heuristic, 32 % of the cells blocked, start and goal in opposite corners, through the schedule $\varepsilon = 3, 2.5, 2, 1.75, 1.5, 1.25, 1.1, 1$, and records the cost ratio to C^* , the certificate and the number of expansions of every iteration; Table 6.4 and Figure 6.4 show the means. A single optimal A^* on the same instances expands 1 177 cells.

Four observations.

1. The cheap path comes first, the work is spent at the end: iteration 1 is weighted A^* and buys a path 8.3 % above optimal, certified 1.41, for 18 % of the work of A^* , while the three iterations with $\varepsilon \leq 1.25$ cost 1 243 of the 1 541 expansions, because a small ε makes the search behave like A^* and rule out every cell with $g^* + h < C^*$.
2. Iterations 2 and 3 expand nothing: the $\varepsilon = 3$ path already passes the termination test for $\varepsilon = 2.5$ and 2 (Proposition 6.10(iv)), so they cost only the re-keying of OPEN and the certificate stays where it was. Restarting weighted A^* at those ε costs 224 and 244 expansions and does improve the path, to 7 % and 5 % above optimal: ARA* delivers the guarantee, not the best path the effort could buy.

Table 6.4. Means over 25 random 60×60 grids for the schedule of Section 6.7.1: the cost of the published path relative to C^* , the certificate ε' of ARA*, and the expansions of the iteration and in total, for ARA* and for weighted A* restarted at every ε . One optimal A* expands 1177 cells on average.

k	ε_k	ARA*				restarted weighted A*		
		cost/ C^*	ε'_k	exp.	total	cost/ C^*	exp.	total
1	3	1.0827	1.412	215	215	1.0827	215	215
2	2.5	1.0827	1.412	0	215	1.0702	224	439
3	2	1.0827	1.412	0	215	1.0518	244	683
4	1.75	1.0824	1.412	1	216	1.0312	298	980
5	1.5	1.0698	1.378	83	298	1.0216	346	1326
6	1.25	1.0105	1.226	435	733	1.0084	590	1916
7	1.1	1.0003	1.086	480	1213	1.0003	875	2791
8	1	1.0000	1.000	328	1541	1.0000	1173	3963

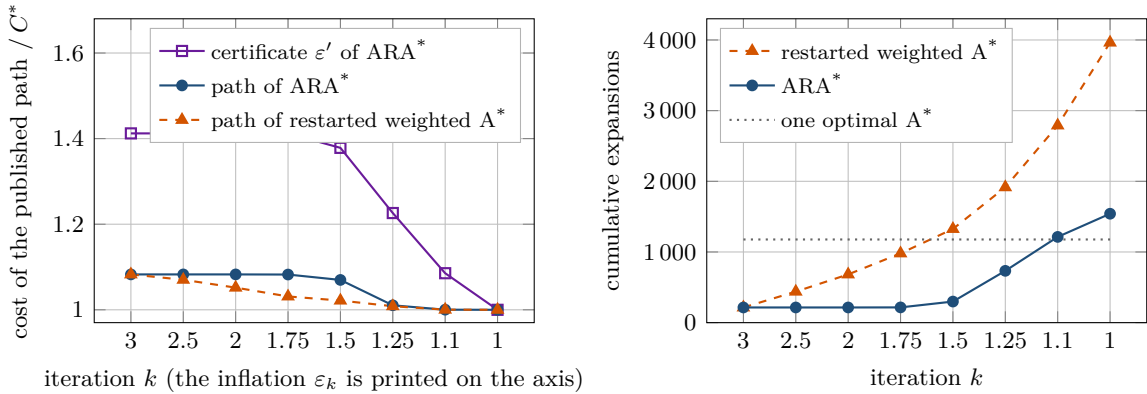


Figure 6.4. The experiment of Table 6.4. Left: the cost of the published path relative to C^* and the certificate ε' of ARA*. The certificate is always above the true ratio, and both reach 1 at the end; the restarts publish slightly better paths in the middle iterations because they search afresh, while ARA* stops as soon as its guarantee holds. Right: cumulative expansions. ARA* spends nothing on the iterations with $\varepsilon = 2.5$ and 2, and reaches optimality with 1.3 times the work of one A* (dotted); the restarts need 3.4 times.

3. The total is 1.3 times one A* for ARA* and 3.4 times for the restarts. ARA* to optimality is more expensive than A* alone, by 31 % here; this is the price of having had a path after 215 expansions and a near-optimal one (1.01 C^* , certificate 1.23) after 733.
4. The certificate lags behind the truth: at $\varepsilon = 1.25$ the path is within 1.1 % of optimal but the certificate says 22.6 %; the gap closes only when the last iterations expand the cheap cells near the start and L rises.

6.7.2 Choosing the schedule

The schedule is the only parameter of ARA*, and Table 6.5 summarizes what its two knobs, the initial factor ε_1 and the decrement, do. Three things the table cannot say are worth adding. Values of ε_1 between 2 and 3 are the usual choice on grids and lattices [107, 108]: a greedier first search must back out of every dead end cell by cell and is rarely faster. Small decrements cost almost nothing, since an iteration that expands nothing costs only the re-keying of OPEN and one that expands something publishes a path; 0.2 is a common value. And the

Table 6.5. The two knobs of the schedule and their effect. “First path” is the time to the first published solution, “steps” the number of intermediate paths, “total” the work until optimality.

Choice	Effect	Comment
ε_1 large (≥ 5)	first path very fast, poor, erratic	greedy dead ends can cost more than a moderate ε_1 ; certificate weak
$\varepsilon_1 = 2-3$	first path after 10–20 % of the work of A^*	the usual choice on grids; certificate ≈ 1.4 in Table 6.4
one jump to $\varepsilon = 1$	no intermediate paths; total $\approx$ weighted $A^* + A^*$	no anytime behavior between the two paths
decrement 0.1–0.25	many steps; zero-work iterations are free	smooth improvement; the default
adaptive $\varepsilon_{k+1} = \max(1, \varepsilon'_k - \Delta)$	skips the factors whose promise the incumbent already keeps	needs the certificate, which is available anyway

certificate can drive the schedule. The inflation ε_{k+1} is only the worst case the next iteration promises, so a value at or above the certified ε'_k promises nothing the published path does not already deliver, and an iteration whose re-keyed OPEN leaves the goal on top expands nothing (Proposition 6.10(iv)). Starting at $\varepsilon_{k+1} = \max(1, \varepsilon'_k - \Delta)$ therefore skips the factors whose promise the incumbent already keeps and makes every iteration aim at a strictly better bound (Exercise 6.5 runs the rule on Example 6.5). A larger ε can still improve the certificate in practice, as $g(\gamma)$ falls and L rises — the $\varepsilon = 2$ iteration of Example 6.5 goes from 1.65 to 1.53 — so the rule trades such gains for a faster approach to optimality. A schedule that stops before $\varepsilon = 1$ loses nothing the certificate does not report: the last path still carries its own ε' , which in Table 6.4 is already 1.086 at $\varepsilon = 1.1$ for a path that is in fact within 0.03 % of optimal.

6.8 Variants and extensions

6.8.1 Anytime D*

ARA* repairs the search when the *time budget* changes; D* Lite (Chapter 5) repairs it when the *edge costs* change. Anytime D* (AD*, Likhachev, Ferguson, Gordon, Stentz and Thrun [106]) does both, and it is the natural planner for a drone that must replan on a changing map within a deadline. We describe the ideas only; the pseudocode is in the paper and in the journal version [107].

- *Search direction and bookkeeping.* Like D* Lite, AD* searches backward from the goal toward the drone’s current state, so that the moving start is the query vertex and the heuristic $h(s_{\text{current}}, \cdot)$ is measured from it. Every state carries the two values g and rhs of Chapter 5; a state is overconsistent if $g > rhs$ (a cheaper way has appeared) and underconsistent if $g < rhs$ (its way became more expensive).
- *Inflated keys for improvements only.* The key of an overconsistent state is $[rhs + \varepsilon h; rhs]$, inflated as in ARA*; the key of an underconsistent state is $[g + h; g]$, *not* inflated. Cost increases are thus propagated exactly and early, while improvements are propagated greedily, and the ε -suboptimality guarantee of Theorem 6.8 survives.
- *The same lists.* Overconsistent states are expanded at most once per iteration and go to CLOSED; an overconsistent state that is improved again after its expansion goes to INCONS. Underconsistent states are handled as in D* Lite: their g is reset to ∞ and they and their neighbors are re-evaluated.

- *The outer loop.* After each search the current ε -suboptimal path is published. When edge costs change, the affected states are updated; if the change is large the planner *increases* ε (or restarts with ε_1) to get a new path quickly, if it is small it keeps ε ; otherwise ε is decreased as in ARA*. In every case INCONS is merged into OPEN, the keys are recomputed and CLOSED is emptied before the next search.

The price is two mechanisms in one algorithm, with the traps of [Chapter 5](#) included. Static map: ARA*; changing map and a generous deadline: D* Lite; both: AD*.

6.8.2 Other anytime searches

Anytime weighted A* (Hansen and Zhou [60]) takes the opposite decision on inconsistent states: it re-opens them immediately and keeps running weighted A* *past* its first solution, pruning every state whose uninflated f is at least the cost of the best solution so far, and reporting the ratio of that cost to the smallest uninflated f in OPEN as its certificate. It publishes a path whenever a better one is found rather than once per ε , at the price of re-expanding states many times, which ARA*'s postponement avoids. The certificate of both algorithms is the ratio of an incumbent solution to a lower bound, the same reasoning by which ECBS ([Chapter 10](#)) bounds the suboptimality of a multi-agent plan. RRT* ([Chapter 17](#)) is anytime by construction, improving per sample rather than per ε , without a certificate.

6.9 Implementation notes

The reference implementation `ch06_arastar.py` follows [Algorithms 6.1](#) and [6.2](#) on the grids of [Chapter 2](#). The tables g and parent are dictionaries, so untouched states cost nothing; CLOSED and INCONS are sets; and OPEN is the lazy heap of [Chapter 4](#), whose entries carry $(f_\varepsilon, h, \text{counter}, n, g_{\text{at push}})$. An entry is stale when its state has left OPEN or its stored g differs from the current one, and stale entries are skipped when the minimum is read, so “update its key” on line 11 is simply a second push. The tie-breaking toward the smaller h keeps the greedy dive of the first iteration going and reproduces the expansion order of [Table 6.2](#). The heart of the code is `improve_path()` in [Listing 6.1](#), a direct transcription of [Algorithm 6.1](#): the while condition compares the goal's key with the top of the heap, the assertion documents [Proposition 6.10\(i\)](#), and the branch on `t` in `self.closed` is line 13.

Listing 6.1. One `ImprovePath` call (from `code/ch06_arastar.py`). `fvalue` returns $g + \varepsilon h$; `_pop_open` and `_min_open_f` skip stale heap entries; the elided `trace` block records the rows of [Table 6.2](#).

```

1  def improve_path(self):
2      """One ImprovePath call at the current eps; returns #expansions."""
3      expansions = 0
4      while self.fvalue(self.goal) > self._min_open_f():
5          s = self._pop_open()
6          assert s not in self.closed # expanded at most once per call
7          self.closed.add(s)
8          expansions += 1
9          lowered = []
10         for t, c in self.grid.successors(s):
11             g_new = self.g[s] + c
12             if g_new < self.g.get(t, INF):
13                 self.g[t] = g_new
14                 self.parent[t] = s
15                 if t in self.closed:
16                     self.incons.add(t) # improved but already closed

```

```

17         lowered.append((t, g_new, "INCONS"))
18     else:
19         self._push(t)          # insert or update key
20         lowered.append((t, g_new, "OPEN"))
21         # (trace bookkeeping for the trace table omitted)
22     return expansions

```

Listing 6.2 shows the bookkeeping between iterations (lines 7–10 of Algorithm 6.2) and the certificate. Re-keying is done by rebuilding the heap from the merged set; the sort only makes the order of equal keys deterministic. The certificate takes L over both lists and guards the degenerate case $g(\gamma) \leq L$, which means that the path is optimal.

Listing 6.2. Start of an iteration and the certificate (from `code/ch06_arastar.py`).

```

1  def _start_iteration(self, eps):
2      """OPEN := OPEN u INCONS with keys recomputed for eps; CLOSED := {}."""
3      self.eps = eps
4      self.open_set |= self.incons
5      self.incons = set()
6      self.closed = set()
7      self._heap = []
8      for s in sorted(self.open_set):
9          self._push(s)
10
11  def _bound(self):
12      """Return (L, eps') for the current solution.
13
14      L = min over OPEN u INCONS of g + h is a lower bound on C*. The
15      published bound is eps' = min(eps, g(goal)/L); if g(goal) <= L the
16      solution is already optimal and eps' = 1.
17      """
18      candidates = self.open_set | self.incons
19      lower = min((self.g[s] + self.h(s) for s in candidates), default=INF)
20      g_goal = self.g.get(self.goal, INF)
21      ratio = 1.0 if g_goal <= lower else g_goal / lower
22      return lower, max(1.0, min(self.eps, ratio))

```

The anytime interface. `ARAStar.run()` is a Python generator: it yields a `Solution` object (path, cost, $g(\gamma)$, L , ϵ' , expansion counts and snapshots of the lists) after every iteration and stops when the certificate reaches 1. A consumer that needs a path by a deadline simply stops iterating; `anytime_plan()` wraps this in a callback that a drone controller would replace by “send the path to the trajectory follower”. Two details matter for such a consumer. The path in the `Solution` is the pointer path, whose cost can be below $g(\gamma)$ (Proposition 6.7); the self-test checks that it never exceeds it. And a deadline that interrupts `ImprovePath` in the middle of an iteration is harmless: the parent pointers form a valid path at every moment, its cost is at most the current $g(\gamma)$, which is at most the value published last, so the last certificate still holds for it. If the interrupt must be fine-grained, check the clock every few hundred expansions inside the loop and return; the lists are consistent between two expansions.

Numerics and testing. On an 8-connected grid g is a sum of 1s and $\sqrt{2}$ s, and two routes of equal true cost can differ in the last bit; a spurious improvement by 10^{-16} would then send a closed cell to INCONS for nothing. The self-test passes without a tolerance, but a production version should insist on an improvement of at least a small ϵ in the relaxation, as the A* code of Chapter 4 does. The self-test runs ARA* with the schedule 3, 2, 1.5, 1 on 80 random 24×18

grids with a quarter of the cells blocked, half of them 4-connected. On the 55 solvable ones it checks against an independent A^* that every published path is valid, that $L \leq C^*$, $1 \leq \varepsilon' \leq \varepsilon$ and $\text{cost} \leq \varepsilon' C^*$, that $g(\gamma)$ never increases and that the last path is optimal with $\varepsilon' = 1$; on the other 25 it checks that nothing is published; and on [Example 6.5](#) it reproduces [Table 6.3](#). Weighted A^* is ARA* with a one-element schedule — that is exactly how `weighted_astar()` is implemented, so the first iteration and the restarts of [Section 6.7.1](#) use the same code path and tie-breaking as ARA* itself. The whole test runs in well under a second.

Treating the inflated key as if it were consistent

With $\varepsilon > 1$ the key $g + \varepsilon h$ is not consistent even when h is, and [Figure 6.2](#) shows the consequence: a closed state can receive a cheaper path later. Two wrong reactions are common. Ignoring the improvement (“the state is closed, skip it”) leaves a wrong g -value and a wrong parent pointer, and the next iteration cannot repair what it does not know; the certificate then rests on a false L . Re-opening the state at once, as A^* does, keeps the bound but gives up the “once per iteration” property, and on inflated keys a state can be re-expanded many times before the iteration ends. The right move is the one on [line 13](#): record the improvement, put the state into INCONS, and expand it in the next iteration.

Forgetting Incons

INCONS looks like a bookkeeping detail, and an implementation that drops it, or that computes L over OPEN only, runs and returns paths. It is wrong in two ways. The lower bound L of [Equation \(6.2\)](#) is a lower bound on C^* only because the first inconsistent state on an optimal path is in one of the *two* lists ([Proposition 6.9](#)); computed over OPEN alone it can exceed C^* , and the certificate becomes a false promise, up to claiming $\varepsilon' = 1$ for a suboptimal path. And the next iteration starts from a search in which the improved states are not in OPEN: their successors keep stale values, the search may commit to a worse route, and even the final iteration with $\varepsilon = 1$ can return a suboptimal path. [Exercise 6.4](#) builds a seven-node graph on which both failures occur at once. Merging INCONS on [line 8](#) and scanning both lists on [line 14](#) are not optional.

Decreasing ε too fast

The schedule 3,1 is tempting: one quick path, then the optimal one. But the second iteration is then nearly a full A^* (15 of the 22 A^* expansions in [Example 6.5](#)), nothing is published while it runs, and a deadline inside it leaves the drone with the greedy path, although in [Table 6.4](#) a path within 7% of optimal was available for 83 more expansions and one within 1.1% for 518 more. The opposite mistake is cheap: an iteration whose ε is still too large expands nothing and costs one re-keying of OPEN. Decrease in small steps, or use the adaptive rule of [Section 6.7](#), and always keep the last published path until the next one is complete.

6.10 Where this fits in the drone system

In the drone system

ARA* belongs to the **replanning layer** of the hybrid architecture of [Chapter 24](#), next to D^* Lite and A^* , and it is the planner to call when that layer has a deadline. It receives the drone’s current cell after a local avoidance maneuver, a map in which the predicted

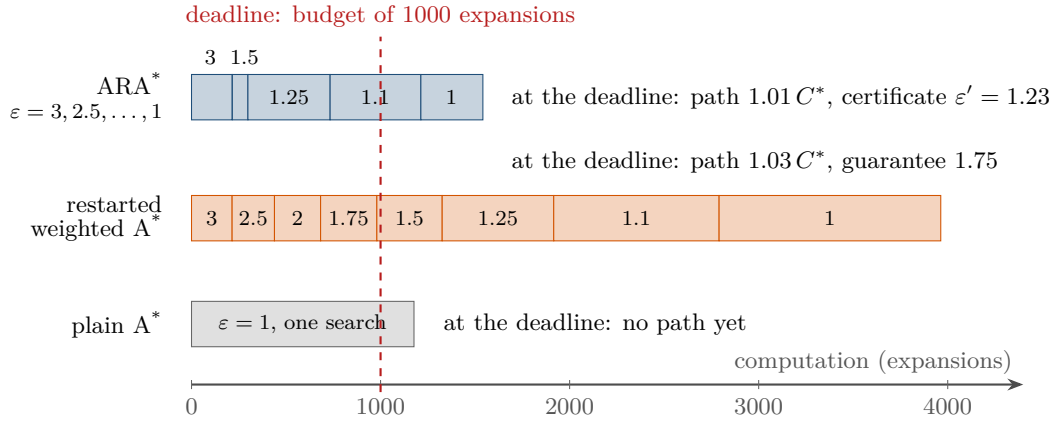


Figure 6.5. What each planner has delivered when a budget of 1000 expansions is exhausted, drawn to scale from the means of Table 6.4 (one centimeter is 400 expansions; the labels are the inflation factors, ARA*'s iterations with $\varepsilon = 2.5$ and 2 have zero length, and $\varepsilon = 1.75$ is a single expansion wide). Plain A* has nothing to show; the restarts have the $\varepsilon = 1.75$ path; ARA* has the $\varepsilon = 1.25$ path with its certificate, and is already working on $\varepsilon = 1.1$.

positions of the intruder (Chapters 18 and 20), inflated by their uncertainty, and the reservations of the other drones are obstacles, and a time budget; it delivers a path and a certificate ε' . Figure 6.5 shows what the three planners of this part of the book have to offer when the budget runs out: on the grids of Section 6.7.1 a budget of 1000 expansions leaves plain A* without any path, restarted weighted A* with a path 3% above optimal, and ARA* with a path 1% above optimal and a certificate of 1.23.

Three practical rules follow from the chapter. *Fly the first path:* with $\varepsilon_1 = 3$ it arrives after a fifth of the work of A*, and the later iterations can run in following control cycles while the drone moves. *Read the certificate, not ε :* the executive of Chapter 24 can commit to a path with $\varepsilon' \leq 1.2$ and spend one more cycle on one with $\varepsilon' = 1.6$. *Search backward when the start moves.* A drone that flies while the planner improves changes its start; the forward search of this chapter keeps its guarantee only for the part of the path not yet flown, which is why the anytime replanner of choice on a changing map is Anytime D* (Section 6.8.1), whose backward search and key modifier absorb both the motion of the drone and the moving obstacle.

The same certificate reasoning, an incumbent divided by a lower bound, returns at the swarm level in ECBS (Chapter 10). This chapter is the optional second half of Week 2 of the study plan (Appendix A); its coding exercise is Exercise 6.6.

6.11 Summary

Chapter summary

- An anytime planner publishes a first path quickly and better paths later, each with a certificate ε' such that the path costs at most $\varepsilon' C^*$; a deadline finds the last published path ready.
- ARA* runs weighted A* with the key $g + \varepsilon h$ through a schedule $\varepsilon_1 > \dots > \varepsilon_K = 1$ and never restarts: the g -values, parent pointers and OPEN survive, and the states improved after their expansion wait in INCONS until the next iteration. Between

iterations ε decreases, INCONS is merged into OPEN, the keys are recomputed and CLOSED is emptied.

- Each state is expanded at most once per iteration, the goal is never expanded, and every iteration ends with $g(\gamma) \leq \varepsilon C^*$ if h is consistent (Theorem 6.8); the search is complete for every $\varepsilon \geq 1$ and optimal at $\varepsilon = 1$.
- The certificate $\varepsilon' = \min(\varepsilon, g(\gamma)/L)$ with L the smallest $g + h$ over $\text{OPEN} \cup \text{INCONS}$ is valid (Proposition 6.9) and usually far below ε : 1.41 for $\varepsilon = 3$ on random grids. The published pointer path may be cheaper than $g(\gamma)$.
- On random 60×60 grids the first path costs 18% of the work of A^* and is 8% too long; optimality costs 1.3 times one A^* in total, against 3.4 times for restarted weighted A^* . The work is spent in the last iterations.
- Start with $\varepsilon_1 = 2-3$ and decrease in small steps, or by the adaptive rule $\varepsilon_{k+1} = \max(1, \varepsilon'_k - \Delta)$; one big jump destroys the anytime behavior. Never ignore an improvement of a closed state, never forget INCONS, never compute L over OPEN alone.
- Anytime D^* combines ARA* with D^* Lite: inflated keys for overconsistent states, exact keys for underconsistent ones, a backward search from the goal, and an outer loop that raises ε after large map changes and lowers it otherwise.

Further reading. ARA* is due to Likhachev, Gordon and Thrun [108]; the journal article by Likhachev, Ferguson, Gordon, Stentz and Thrun [107] contains the complete proofs of Section 6.6, experiments on robot platforms and Anytime D^* , which was introduced in [106] on top of D^* Lite [88]. Weighted A^* goes back to Pohl [132]; Pearl [126] is the textbook treatment of search with inflated and otherwise “ ε -admissible” heuristics, with the semi-admissible algorithms of Pearl and Kim [127] as the closest ancestor of the certificate idea, and Russell and Norvig [142] give the modern textbook account of weighted A^* . Hansen and Zhou [60] analyze anytime weighted A^* and compare it with ARA* on a range of problems.

6.12 Exercises

Exercise 6.1 (★). Work with Table 6.2. (a) In step 2 of iteration 1 the expansion of (1, 3) generates five cells. Compute their keys for $\varepsilon = 3$ and for $\varepsilon = 1$, and explain why the diagonal neighbors are expanded first with $\varepsilon = 3$ although they have the larger g . (b) At the start of iteration 2 recompute the keys of the nine states in OPEN for $\varepsilon = 2$ and check that (3, 3) has the smallest one and the goal the second smallest. (c) Iteration 2 stops after four expansions with $g(\gamma) = 12.24$ while the pointer path costs 11.41. Derive both numbers from the parent pointers, and list the expansions that would have been needed to make $g(\gamma)$ equal to 11.41 within that iteration.

Exercise 6.2 (★). At the end of iteration 1 of Example 6.5 the lists are $\text{OPEN} = \{(1, 1), (1, 2), (1, 4), (2, 1), (2, 4), (3, 0), (4, 5), (7, 2)\}$ and $\text{INCONS} = \{(3, 3)\}$, with the g -values of Table 6.2. (a) Compute $g + h$ for all nine states and verify $L = 7.41$ and $\varepsilon' = 1.65$. (b) Which L and ε' would an implementation publish that ignores INCONS in Equation (6.2)? (c) Show that $h(s) \leq g(n) + h(n)$ for every state n with finite g when h is consistent, conclude that $\min(\varepsilon, g(\gamma)/h(s))$ is also a valid certificate, and explain why it is never better than Equation (6.3). Compute it for iteration 1 and compare.

Exercise 6.3 (★★ Proof). (a) Prove Lemma 6.6(iii) in full, by induction over the operations of Algorithms 6.1 and 6.2, and say why the statement would fail if CLOSED were *not* emptied between iterations. (b) Write out the proof of Proposition 6.9 for the case in which the first

inconsistent state u of the optimal path is in INCONS, and check every use of an invariant. (c) Take the graph of [Chapter 4](#) with an admissible but inconsistent heuristic: nodes s, a, b, γ , edges $s \rightarrow a$ (cost 1), $s \rightarrow b$ (3), $a \rightarrow b$ (1), $b \rightarrow \gamma$ (2), and $h(a) = 3$, $h = 0$ elsewhere. Run `ImprovePath` with $\varepsilon = 1$ by hand and show that it returns $g(\gamma) = 5 > C^* = 4$ and publishes the certificate $\varepsilon' = 1$; note whether that certificate is about $g(\gamma)$ or about the cost of the path the pointers record. Which case of the proof of [Theorem 6.8](#) breaks, and which inequality does that case need?

Exercise 6.4 (★★). Consider the graph with nodes s, a, b, c, d, e, γ and the edges $s \rightarrow a$ (cost 1), $s \rightarrow b$ (1), $s \rightarrow d$ (2), $a \rightarrow c$ (1), $b \rightarrow c$ (2), $c \rightarrow e$ (1), $c \rightarrow \gamma$ (5), $d \rightarrow \gamma$ (3.5), $e \rightarrow \gamma$ (2), and the heuristic $h(s) = 2$, $h(a) = 2$, $h(b) = 1$, $h(c) = 1$, $h(d) = 2$, $h(e) = 2$, $h(\gamma) = 0$. (a) Check that h is consistent and determine C^* . (b) Run ARA* with the schedule 3, 1 by hand: for each iteration give the expansions in order, the contents of INCONS, the published pointer path with its cost, $g(\gamma)$, L and ε' . (c) Repeat the second iteration with INCONS discarded at the end of the first: which path and which certificate are published, and what is wrong with each? (d) Explain why an implementation that re-opens c immediately would also be correct on this graph, and what it costs in general.

Exercise 6.5 (★★). Use [Table 6.4](#). (a) For every iteration compute the decrease of the mean cost ratio per 100 expansions, for ARA* and for the restarts; in which iterations is improvement cheapest? (b) A replanning cycle allows 800 expansions. Which iteration of the schedule is complete when the cycle ends, what are the published cost and certificate, and what do restarted weighted A* and plain A* deliver at that moment? (c) Explain with [Proposition 6.10\(iv\)](#) why iterations 2 and 3 expand nothing, and why the certificate nevertheless stays at 1.41 instead of falling to 2.5 or 2 — or rising to them. (d) Apply the adaptive rule $\varepsilon_{k+1} = \max(1, \varepsilon'_k - 0.2)$ to [Example 6.5](#), starting from $\varepsilon_1 = 3$: which schedule results, and, running ARAStar with it, how many expansions does each iteration cost and when does the optimal path appear?

Exercise 6.6 (★★★ Coding). This is the optional coding exercise of Week 2 of the study plan ([Appendix A](#)). Implement ARA* yourself, in the API of `code/ch06_arastar.py`, and do not read that file until you are stuck. (a) Write `ARAStar(grid, start, goal, schedule).run()` as a generator that yields, after every iteration, the pointer path, its cost, $g(\gamma)$, L , ε' and the expansion count, with a lazy heap, the sets CLOSED and INCONS, and the re-keying of OPEN between iterations. Reproduce [Table 6.3](#) and the first ten rows of [Table 6.2](#). (b) On 50 random 40×40 grids, check every published path against an independent A*: valid path, cost $\leq \varepsilon' C^*$, $L \leq C^*$, last path optimal. (c) Plot the cost ratio and the certificate against *wall-clock time* and against cumulative expansions for the schedules (3, 2, 1), (3, 2.5, ..., 1) and (5, 4, ..., 1), and add restarted weighted A* and plain A*; mark the first-solution times. (d) Add a deadline: `run(budget_s=0.02)` must return the best path available after 20 ms, interrupting an iteration if necessary, and never a path worse than the last published one.

Exercise 6.7 (★★★). Anytime replanning under a deadline. The replanning layer of [Chapter 24](#) runs on a flight computer that manages about 20 000 expansions per second, and a control cycle lasts 50 ms. (a) How many expansions fit into a cycle? On maps like those of [Table 6.4](#), which iteration finishes within the cycle, and what do ARA*, the restarts and plain A* deliver? Repeat for a 20 ms cycle. (b) If the deadline falls inside an iteration, argue from [Proposition 6.7](#) and [Theorem 6.8](#) that the pointer path is then a valid path within the previous iteration's certificate. (c) After iteration 2 of [Example 6.5](#) the intruder blocks the cell (4, 3). Which cells now have an invalid parent pointer or g -value, that is, are underconsistent in the sense of [Chapter 5](#)? What would Anytime D* do with them and with ε , and what is the best ARA* alone can do? (d) Propose a rule for the replanning layer that chooses between “keep improving”, “raise ε and repair” and “restart” from the number of

■ changed edges, the certificate and the time left in the cycle.

Part III

Multi-Agent Path Finding

The Multi-Agent Path Finding Problem

Learning objectives

- State the classical multi-agent path finding problem exactly: graph, agents, discrete synchronous time, move and wait actions, time-indexed paths and plans, and the stay-at-target convention.
- Recognize the five conflict types of Stern et al. (vertex, edge, swapping, following, cycle), know which two this book forbids and why, and find them by hand in a table of paths.
- Compute the sum of costs and the makespan of a plan, explain why the two can disagree, and state what is known about the complexity of feasibility and of optimality.
- Read the `.map` and `.scen` formats of the standard benchmark and name the solver families that the following chapters develop.
- Implement a plan validator and conflict detectors in $\mathcal{O}(k^2T)$ and $\mathcal{O}(kT)$ time in Python, and use them to build deliberate conflict cases.

7.1 Why this matters for a drone swarm

Twelve inspection drones share the hall of a warehouse. Each has its own start pad and its own target shelf, and each, on its own, can find a shortest route with the A^* of [Chapter 4](#) in a few milliseconds. Fly those twelve routes at the same time and two of them meet head-on in an aisle, a third crosses the first at an intersection at the wrong moment, and a fourth arrives at a shelf in front of which a drone that has already finished is still hovering. None of the single-agent planners of [Chapters 3 to 6](#) can see these events, because each of them plans one agent in a world that contains no others.

[Figure 7.1](#) measures how bad the problem is. On random 16×16 grids with 20 % obstacles we planned every agent independently and counted the conflicts of the resulting **naive plan**. With two agents, 95 % of the plans happen to be conflict-free. With eight, only 5 % are, and a plan contains 3.5 conflicts on average; with twenty agents no naive plan out of sixty was free of conflicts, and with forty agents a plan has about 97 conflicts among 72 conflicting pairs. The number of conflicts grows roughly in proportion to the number of pairs, $k(k-1)/2$. Coordination is not an optional refinement: beyond a handful of agents, independent planning never produces a flyable plan.

The problem of finding routes for many agents that never collide is **multi-agent path finding (MAPF)**. This chapter defines it; the four chapters that follow solve it. In the hybrid architecture of [Chapter 24](#), MAPF is the job of the *global* layer: it produces the nominal,

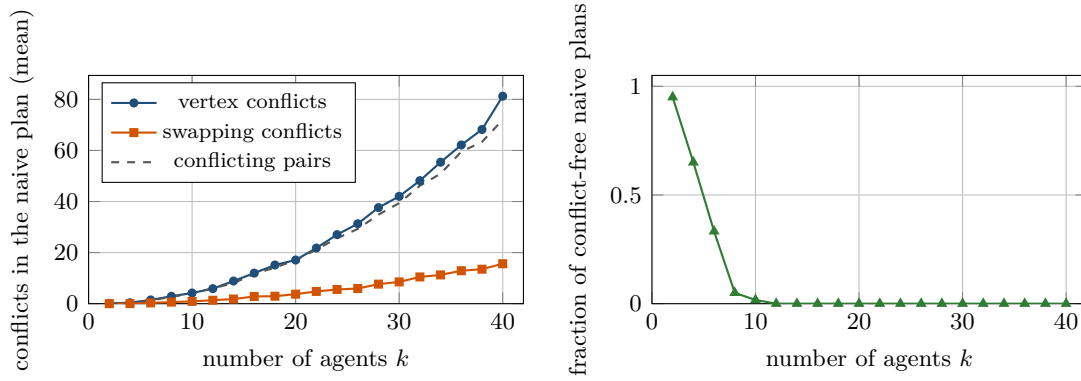


Figure 7.1. Naive plans (independent shortest paths) on random 16×16 grids with 20 % obstacles, 60 instances per value of k (`gen_ch07_conflict_growth.py`). Left: the mean number of vertex and swapping conflicts and of conflicting agent pairs grows roughly quadratically with k . Right: the fraction of instances whose naive plan is conflict-free drops to zero at about twelve agents.

time-indexed routes of all drones before take-off, and it is consulted again whenever the local safety layer or the replanning layer has changed a route and the swarm must re-check it against the others. That re-check is the conflict detector of this chapter, the one piece of MAPF that every layer uses.

A problem chapter has a different shape from an algorithm chapter. [Section 7.2](#) gives the idea and [Section 7.3](#) the exact definitions of Stern et al. [161]: instances, paths, plans, the five kinds of conflict and the two objectives. The algorithm of the chapter, in [Section 7.4](#), is conflict detection, and [Section 7.5](#) runs it by hand on three agents. The rest collects the complexity ([Section 7.6](#)), the solver families with the chapters that treat them ([Section 7.7](#)), the benchmark files you will use for the rest of Part III ([Section 7.8](#)), the Python ([Section 7.9](#)) and what changes for drones ([Section 7.10](#)).

7.2 The idea in plain words

Freeze time into steps of length Δt and write down where every agent is at every step. A route becomes a row of a table, one column per time step, and a plan for k agents is a table with k rows. Two agents *conflict* when the table shows them in the same place in the same column, or shows them exchanging places along one edge between two consecutive columns. Finding a plan means filling in the table so that every row is a legal route for its agent and no such coincidence occurs anywhere; finding a good plan means doing so with rows that are as short as possible.

This is the space-time view of [Chapter 4](#): the state of an agent is a pair (v, t) , a step either moves along an edge or waits, and time advances by one for every agent at once; what is new is that the rows are no longer independent. The table also shows what a conflict is *not*. If agent 2 leaves a cell in the step in which agent 1 enters it, the two never share a column entry; they follow each other one cell apart, like the carriages of a train, which the physics of the agents may forbid ([Section 7.10](#)) but the discrete model does not. [Figure 7.2](#) shows the five patterns that the literature distinguishes.

Key idea

A MAPF plan is a table of positions, one row per agent and one column per time step, in which an agent that has reached its goal keeps that cell in every later column. The plan is valid when every row is a legal path and no two rows share a cell in a column or swap

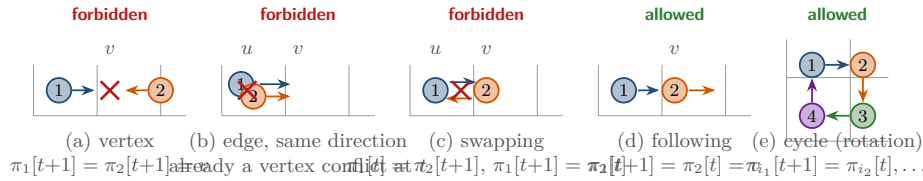


Figure 7.2. The conflict types of Stern et al. [161] between time t and $t + 1$. (a) Two agents enter the same vertex. (b) Two agents traverse the same edge in the same direction, which means they already shared u at time t . (c) Two agents exchange vertices along an edge. (d) One agent enters the vertex that the other is leaving. (e) A rotation along a fully occupied cycle of $m \geq 3$ distinct vertices. Classical MAPF forbids (a)–(c) and allows (d), and (e) for $m \geq 3$.

two cells between consecutive columns. Everything else is built on this table.

7.3 Problem statement and notation

7.3.1 Instances, actions and paths

Definition 7.1 (Classical MAPF instance). A **classical MAPF instance** is written $\langle G, s, \gamma \rangle$ and consists of an undirected graph $G = (V, E)$, a number k of **agents** $a_1, \dots, a_k$, and two maps $s, \gamma: \{1, \dots, k\} \rightarrow V$ that assign to every agent a **start vertex** s_i and a **goal vertex** γ_i . The starts are pairwise distinct and so are the goals. Time is discrete, $t \in \{0, 1, 2, \dots\}$, and all agents act synchronously: in every time step each agent either **moves** along one edge to an adjacent vertex or **waits** at its vertex. Both actions take exactly one time step and cost 1.

The graph is usually the 4-connected grid G_4 of [Chapter 2](#), whose vertices are the free cells. We write a cell as (r, c) with the row r counted from the top and the column c from the left, because that is the order in which a benchmark map file is read ([Section 7.8](#)), while the single-agent chapters used (x, y) with the origin at the bottom left; both are only names for vertices. A start vertex may coincide with a goal vertex, of the same agent or of another one; only the starts among themselves and the goals among themselves must be distinct.

Definition 7.2 (Time-indexed path, stay-at-target). A **time-indexed path** for agent a_i is a sequence $\pi_i = (\pi_i[0], \pi_i[1], \dots, \pi_i[T_i])$ of vertices with $\pi_i[0] = s_i$, $\pi_i[T_i] = \gamma_i$ and, for every $t < T_i$, either $\pi_i[t+1] = \pi_i[t]$ (a wait) or $\{\pi_i[t], \pi_i[t+1]\} \in E$ (a move). Under the **stay-at-target** convention the agent remains at its goal after the end of its path: we set $\pi_i[t] = \gamma_i$ for all $t > T_i$, so that $\pi_i[t]$ is defined for every $t \geq 0$. The **cost** of the path is the time of its last arrival at the goal,

$$\text{cost}(\pi_i) = \min \{t' \geq 0 : \pi_i[t] = \gamma_i \text{ for all } t \geq t'\}. \quad (7.1)$$

Two remarks. First, a path may visit its goal, leave it and come back; Equation (7.1) charges every step until the *final* arrival, and trailing waits at the goal are free. The sequence $((0, 0), (0, 1), (0, 1), (0, 2), (0, 2))$ has $T_i = 4$ but cost 3. Second, the alternative **disappear-at-target** convention removes an agent from the graph when it first reaches its goal; it models robots that leave the warehouse and makes some instances easier, because a finished agent blocks nobody. A hovering drone blocks its cell as much as a flying one, so this book uses stay-at-target throughout, as do the benchmark and almost all solvers [161].

Definition 7.3 (Plan and solution). A **plan** is a tuple $\Pi = (\pi_1, \dots, \pi_k)$ of one time-indexed path per agent. Its **horizon** is $T = \max_i T_i$. A plan is a **solution** (or is *valid*) if it contains no vertex conflict and no swapping conflict in the sense of Definition 7.4 at any time $t > 0$.

One warning about T , because the neighboring chapters use the letter differently. [Chapter 2](#) writes the final-arrival time of a path as $T(\pi_i)$; here that quantity is $\text{cost}(\pi_i)$, while T_i is only the last index the stored list mentions. Hence $T_i \geq \text{cost}(\pi_i)$ and $T \geq \text{MS}(\Pi)$: the horizon equals the makespan exactly when no path ends with waits at its goal, which is how [Chapters 8](#) and [9](#) store their paths. Read T as “how far the tables go”, never as a cost.

7.3.2 Conflicts

Definition 7.4 (Conflicts [\[161\]](#)). Let Π be a plan, $i \neq j$ two agents and $t \geq 0$ a time step.

- (a) A **vertex conflict** $\langle a_i, a_j, v, t \rangle$ occurs if $\pi_i[t] = \pi_j[t] = v$: both agents occupy v at time t .
- (b) An **edge conflict** $\langle a_i, a_j, u, v, t \rangle$ occurs if $\pi_i[t] = \pi_j[t] = u$ and $\pi_i[t+1] = \pi_j[t+1] = v \neq u$: both traverse the edge $\{u, v\}$ in the same direction in the same step.
- (c) A **swapping conflict** $\langle a_i, a_j, u, v, t \rangle$ occurs if $\pi_i[t] = \pi_j[t+1] = u$ and $\pi_i[t+1] = \pi_j[t] = v \neq u$: the agents exchange vertices along the edge $\{u, v\}$ between t and $t+1$.
- (d) A **following conflict** occurs if $\pi_i[t+1] = \pi_j[t]$: agent a_i enters at $t+1$ the vertex that a_j occupied at t .
- (e) A **cycle conflict** occurs if agents $a_{i_1}, \dots, a_{i_m}$ ($m \geq 3$) all move at step t , stand on m pairwise distinct vertices $\pi_{i_1}[t], \dots, \pi_{i_m}[t]$, and satisfy $\pi_{i_1}[t+1] = \pi_{i_2}[t]$, $\pi_{i_2}[t+1] = \pi_{i_3}[t]$, $\dots$, $\pi_{i_m}[t+1] = \pi_{i_1}[t]$: they rotate along a fully occupied cycle of m vertices. Distinctness costs nothing: two of these vertices that coincide are already a vertex conflict at time t .

The five types are not independent, and it pays to see exactly how they overlap before deciding which of them to forbid.

Proposition 7.5 (Relations between conflict types). (a) An edge conflict $\langle a_i, a_j, u, v, t \rangle$ implies the vertex conflicts $\langle a_i, a_j, u, t \rangle$ and $\langle a_i, a_j, v, t+1 \rangle$.

- (b) The degenerate case $m = 2$ of the rotation pattern of [Definition 7.4\(e\)](#) is exactly the swapping conflict, which is why the cycle conflict is defined for $m \geq 3$; a swap also consists of two following conflicts, one in each direction.
- (c) A cycle conflict consists of m following conflicts and contains no vertex conflict and no swapping conflict.
- (d) A following conflict in which the leading agent waits, $\pi_j[t+1] = \pi_j[t] = \pi_i[t+1]$, is the vertex conflict $\langle a_i, a_j, \pi_j[t], t+1 \rangle$.

Proof. Parts (a), (b) and (d) are the definitions read twice. For (c), the m vertices of the cycle are distinct by definition and at time t each holds exactly one of the agents; at $t+1$ each agent has moved to the vertex of its predecessor, so the m agents again occupy the m distinct vertices and no vertex conflict arises. No pair swaps either, and this has to be checked for *every* pair, not only for neighbors in the rotation. Suppose a_{i_a} and a_{i_b} swap, so $\pi_{i_a}[t+1] = \pi_{i_b}[t]$ and $\pi_{i_b}[t+1] = \pi_{i_a}[t]$. The rotation gives $\pi_{i_a}[t+1] = \pi_{i_{a+1}}[t]$, and because the m vertices are distinct, $\pi_{i_b}[t] = \pi_{i_{a+1}}[t]$ forces $i_b = i_{a+1}$; symmetrically the second equation forces $i_a = i_{b+1}$ (indices modulo m). Together $a \equiv a+2 \pmod{m}$, so $m = 2$, and no pair of a cycle with $m \geq 3$ swaps. $\square$

What this book forbids. Following Stern et al. [\[161\]](#), *classical* MAPF forbids vertex and swapping conflicts and allows following and cycle conflicts. Vertex conflicts are collisions by definition, and so are swaps: two agents that exchange the endpoints of an edge in one step pass through each other in its middle. Edge conflicts need no rule of their own, because by [Proposition 7.5\(a\)](#) they are already vertex conflicts. Following is allowed because synchronous unit-speed motion keeps the follower one full edge behind the leader at every integer time;

between integer times two agents in distinct cells stay at least $\ell/\sqrt{2}$ apart on a lattice of cell side ℓ , a bound attained exactly when the follower turns a corner (Exercise 7.7). Forbidding following would rule out convoys through a corridor and the plan of Section 7.5, and a rotation on a fully occupied cycle has the same geometry and the same bound, so cycle conflicts are allowed too. Some hardware disagrees, for example robots that need a free cell ahead before they start moving, or drones so large that following around a corner brings them too close (Exercise 7.7); such systems add following conflicts to the forbidden list and pay with longer plans. Table 7.1 sums up the rules.

Table 7.1. The conflict types of Definition 7.4 and their status in classical MAPF.

Conflict	Pattern between t and $t + 1$	Classical MAPF	Why
vertex	$\pi_i[t] = \pi_j[t]$	forbidden	two agents in one cell
edge	same edge, same direction	forbidden	implies a vertex conflict
swapping	same edge, opposite directions	forbidden	agents pass through each other
following	$\pi_i[t + 1] = \pi_j[t]$, a_j moves on	allowed	one edge apart at integer times
cycle ($m \geq 3$)	rotation on an occupied cycle	allowed	m following conflicts, no collision

7.3.3 Objectives

Definition 7.6 (Sum of costs and makespan). The **sum of costs** and the **makespan** of a plan $\Pi = (\pi_1, \dots, \pi_k)$ are

$$\text{SoC}(\Pi) = \sum_{i=1}^k \text{cost}(\pi_i), \quad \text{MS}(\Pi) = \max_{i=1, \dots, k} \text{cost}(\pi_i). \quad (7.2)$$

A solution is **optimal** for an objective if no solution of the instance has a smaller value of it.

The sum of costs is the total number of time steps that agents spend before they are finished, so it measures total flight time and energy; the makespan is the time at which the last agent arrives, so it measures the duration of the mission. Every wait before the final arrival counts in both. The two objectives agree on what a good plan looks like but not on which plan is best, and it takes only three agents to separate them.

Example 7.7 (Where the objectives disagree). Figure 7.3 shows three agents on an empty 4×5 grid: a_1 flies along row 1 from $(1, 0)$ to $(1, 4)$, a_2 down column 1 from $(0, 1)$ to $(2, 1)$, and a_3 up column 2 from $(3, 2)$ to $(0, 2)$. Their shortest paths have costs 4, 2 and 3 and collide twice: a_1 and a_2 both reach $(1, 1)$ at $t = 1$, and a_1 and a_3 both reach $(1, 2)$ at $t = 2$. Plan Y lets a_1 wait one step at its start; it then reaches each crossing one step after the other agent has passed, the costs are 5, 2, 3, and $\text{SoC}(Y) = 10$, $\text{MS}(Y) = 5$. Plan X instead lets a_2 and a_3 each wait once, so that a_1 passes first: costs 4, 3, 4, $\text{SoC}(X) = 11$, $\text{MS}(X) = 4$. A search of the joint state space (`optimal_soc_bruteforce` and `optimal_makespan_bruteforce` in `ch07_mapf.py`) confirms that 10 and 4 are the optimal values and that no solution attains both: every solution with $\text{SoC} = 10$ has makespan 5.

Which objective should a drone swarm use? The sum of costs is the standard choice of the MAPF literature and of conflict-based search (CBS, Chapter 9), because it spreads delays over the agents and decomposes over them, which the solvers exploit; the makespan is the natural choice when the mission ends only when the last drone has landed. Chapter 25 reports both.

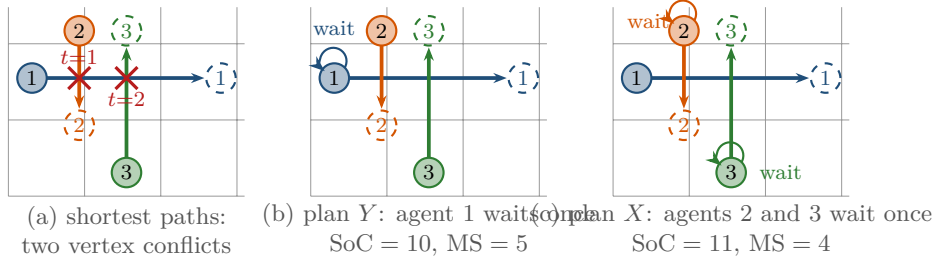


Figure 7.3. Three agents whose shortest paths cross at (1,1) and (1,2). Plan Y delays the long path once and minimizes the sum of costs; plan X delays the two short paths and minimizes the makespan. No plan achieves $\text{SoC} = 10$ and $\text{MS} = 4$ at the same time.

7.3.4 Feasibility versus optimality

An instance is **feasible** (or solvable) if it has at least one solution. Feasibility can fail for simple reasons, such as a goal unreachable from its start, and for combinatorial ones: two agents at the ends of a corridor that must exchange places have no solution unless the corridor has a pocket to step into. Optimality asks for the best solution with respect to SoC or MS; Section 7.6 shows that the two questions are of very different difficulty.

7.4 The algorithm: conflict detection

The solvers of Chapters 8 to 11 all contain the same subroutine: given a plan, find its first conflict or report that there is none. It is also the validator that you run on any plan before you trust it, and the re-check of the drone architecture. Algorithm 7.1 states the direct version of that subroutine.

Algorithm 7.1. First conflict of a plan by pairwise comparison

Input: plan $\Pi = (\pi_1, \dots, \pi_k)$, each π_i a list of $T_i + 1$ vertices

Output: the earliest vertex or swapping conflict, or *nil*

```

1 Function Position( $\pi_i, t$ ):
2   if  $t \leq T_i$  then return  $\pi_i[t]$ 
3   return  $\pi_i[T_i]$                                      // stay-at-target padding
4 Function FirstConflictPairwise( $\Pi$ ):
5    $T \leftarrow \max_i T_i$ 
6   for  $t \leftarrow 0$  to  $T$  do
7     foreach pair  $i < j$  do
8       if Position( $\pi_i, t$ ) = Position( $\pi_j, t$ ) then
9         return vertex conflict  $\langle a_i, a_j, \pi_i[t], t \rangle$ 
10    if  $t < T$  then
11      foreach pair  $i < j$  do
12         $u \leftarrow \text{Position}(\pi_i, t); v \leftarrow \text{Position}(\pi_i, t+1)$ 
13        if  $u \neq v$  and Position( $\pi_j, t$ ) =  $v$  and Position( $\pi_j, t+1$ ) =  $u$  then
14          return swapping conflict  $\langle a_i, a_j, u, v, t \rangle$ 
15  return nil

```

Walkthrough. The helper `Position` implements the padding of [Definition 7.2](#): a question about a time beyond the end of a path is answered with the goal. This is what makes an arrived agent block its cell for everybody who comes later, and it is the first thing that home-made detectors get wrong ([Section 7.9](#)). Line 5 fixes the horizon, and scanning further is pointless: after T no agent moves, so no swap can occur, and two agents that share a cell at some $t > T$ already shared it at T ([Exercise 7.5](#)). For each time step the algorithm first looks for vertex conflicts (line 9) and then, unless $t = T$, for swaps between t and $t + 1$ (line 14); the condition $u \neq v$ excludes a pair of waiting agents, already reported as a vertex conflict. Returning at the first hit gives the earliest conflict, ties broken toward vertex conflicts and then toward the smallest pair (i, j) ; collecting the hits gives all conflicts in the same order.

Cost. Each time step compares all $k(k - 1)/2$ pairs twice, so the running time is $\mathcal{O}(k^2T)$. For a solver that calls the detector at every node of a search tree this quadratic factor matters, and it is unnecessary: the agents at a vertex are found by hashing positions, not by comparing pairs. [Algorithm 7.2](#) keeps, per time step, a dictionary from vertices to the agent seen there, and a second dictionary from directed edges (u, v) to the agent traversing them; a swap is a traversal of (v, u) by an earlier agent when (u, v) is inserted.

Algorithm 7.2. First conflict of a plan with hash maps

Input: plan Π as in [Algorithm 7.1](#)

Output: the earliest vertex or swapping conflict, or *nil*

```

1 Function FirstConflictHashed( $\Pi$ ):
2    $T \leftarrow \max_i T_i$ 
3   for  $t \leftarrow 0$  to  $T$  do
4      $occupied \leftarrow$  empty map from vertices to agents
5     for  $i \leftarrow 1$  to  $k$  do
6        $v \leftarrow \text{Position}(\pi_i, t)$ 
7       if  $v \in occupied$  then return vertex conflict  $\langle a_{occupied[v]}, a_i, v, t \rangle$ 
8        $occupied[v] \leftarrow i$ 
9     if  $t = T$  then return nil
10     $moving \leftarrow$  empty map from directed edges to agents
11    for  $i \leftarrow 1$  to  $k$  do
12       $u \leftarrow \text{Position}(\pi_i, t); v \leftarrow \text{Position}(\pi_i, t + 1)$ 
13      if  $u \neq v$  then
14        if  $(v, u) \in moving$  then return swapping conflict
15         $\langle a_{moving[(v,u)]}, a_i, v, u, t \rangle$ 
16         $moving[(u, v)] \leftarrow i$ 
17  return nil

```

Proposition 7.8 (Correctness and cost of conflict detection). *Algorithms 7.1 and 7.2 return nil if and only if the plan is a solution in the sense of [Definition 7.3](#), and otherwise a conflict with the smallest possible time t . Algorithm 7.1 runs in $\mathcal{O}(k^2T)$ time; Algorithm 7.2 runs in $\mathcal{O}(kT)$ expected time and uses $\mathcal{O}(k)$ extra space.*

Proof. Both algorithms test, for each $t \leq T$, exactly the conditions of [Definition 7.4\(a\)](#) and (c) on the padded paths, and the walkthrough showed that no conflict of either kind can first occur after T . In [Algorithm 7.2](#) a vertex conflict at t between a_j and a_i with $j < i$ is detected when a_i is inserted, because a_j was stored at line 8 earlier in the same step; a swap between

a_j and a_i is detected at the insertion of the later of the two directed edges. Each time step performs $\mathcal{O}(k)$ dictionary operations of expected constant time, and the maps are cleared every step. $\square$

7.5 A worked example

Example 7.9 (Three agents on a 4×4 grid). Figure 7.4 shows a 4×4 grid whose cells $(1,0)$ and $(1,2)$ are blocked. Agent a_1 must fly along the top row from $(0,0)$ to $(0,3)$, agent a_2 along the same row in the opposite direction from $(0,3)$ to $(0,0)$, and agent a_3 starts in the pocket $(1,1)$ below the row and wants the cell $(0,1)$ right above it. The instance is `worked_example()` in `ch07_mapf.py`; every number below is checked by its self-test, which numbers the agents from 0 where the text numbers them from 1.

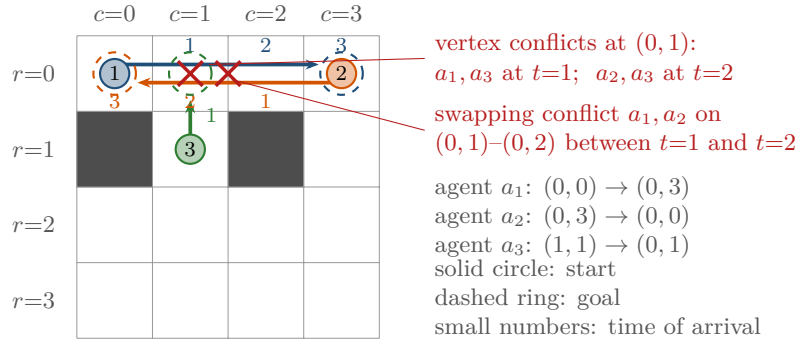


Figure 7.4. Example 7.9 with the naive plan of independent shortest paths. Solid circles are starts, dashed rings goals, and the small numbers give the time at which each agent reaches a cell. The cross on $(0,1)$ carries both vertex conflicts found by hand in Table 7.2, and the cross on the edge $(0,1)-(0,2)$ marks the swap.

The naive plan. Planning each agent alone with breadth-first search (or with A^* , Chapter 4) gives the three paths in the upper half of Table 7.2: π_1 and π_2 run along the top row in three moves each and π_3 needs a single move up. Their costs are 3, 3 and 1, so $\text{SoC} = 7$ and $\text{MS} = 3$; by Proposition 7.10 below no solution can beat either number. Agent a_3 arrives at $t = 1$, and the gray entries are the padding of Definition 7.2: from then on $\pi_3[t] = (0,1)$.

Table 7.2. The naive plan (top) and the conflict-free plan (bottom) of Example 7.9 as tables of positions. Gray entries are positions after the final arrival, added by the stay-at-target padding; they cost nothing, but they do block the cell.

Agent	$t = 0$	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$	$\text{cost}(\pi_i)$
<i>Naive plan (independent shortest paths): SoC = 7, MS = 3, three conflicts</i>							
a_1	(0,0)	(0,1)	(0,2)	(0,3)	(0,3)	(0,3)	3
a_2	(0,3)	(0,2)	(0,1)	(0,0)	(0,0)	(0,0)	3
a_3	(1,1)	(0,1)	(0,1)	(0,1)	(0,1)	(0,1)	1
<i>Conflict-free plan, optimal for both objectives: SoC = 12, MS = 5</i>							
a_1	(0,0)	(0,1)	(1,1)	(0,1)	(0,2)	(0,3)	5
a_2	(0,3)	(0,2)	(0,1)	(0,0)	(0,0)	(0,0)	3
a_3	(1,1)	(2,1)	(2,1)	(1,1)	(0,1)	(0,1)	4

Detection by hand. Run [Algorithm 7.1](#) on the upper table. The horizon is $T = 3$. At $t = 0$ the three positions $(0, 0)$, $(0, 3)$, $(1, 1)$ are distinct, and no pair reverses an edge between $t = 0$ and $t = 1$. At $t = 1$ the column reads $(0, 1)$, $(0, 2)$, $(0, 1)$: agents a_1 and a_3 share $(0, 1)$, and the algorithm returns the vertex conflict $\langle a_1, a_3, (0, 1), 1 \rangle$. If we collect every conflict instead of stopping, the swap check between $t = 1$ and $t = 2$ finds a_1 moving $(0, 1) \rightarrow (0, 2)$ while a_2 moves $(0, 2) \rightarrow (0, 1)$, the swapping conflict $\langle a_1, a_2, (0, 1), (0, 2), 1 \rangle$. At $t = 2$ the column reads $(0, 2)$, $(0, 1)$, $(0, 1)$, a second vertex conflict $\langle a_2, a_3, (0, 1), 2 \rangle$, caused by nothing but the padding: a_3 has been parked at its goal since $t = 1$ and a_2 runs into it. The moves between $t = 2$ and $t = 3$ reverse no edge, the column at $t = 3$ holds three different cells, and the scan ends. The plan has exactly these three conflicts, in this order, and `all_conflicts(naive_plan(inst))` prints the same three.

The space-time view. [Figure 7.5](#) draws the same three paths in the time-expanded graph of [Chapter 4](#): one copy of every vertex per time step, a vertical edge for a wait and a diagonal edge for a move. A vertex conflict is a space-time node used by two paths; a swapping conflict is a pair of crossing move edges. This picture is behind every solver of [Chapters 8 to 10](#). A single agent is planned by space-time A^* ; a vertex constraint $\langle a_i, v, t \rangle$ deletes the node (v, t) for agent a_i , an edge constraint $\langle a_i, u, v, t \rangle$ deletes its move edge from (u, t) to $(v, t + 1)$, and the goal-stay test makes sure that the agent can park at γ_i for ever once it arrives. What a MAPF solver adds is the choice of the constraints; what this chapter adds is the detector that tells it which conflict to resolve next.

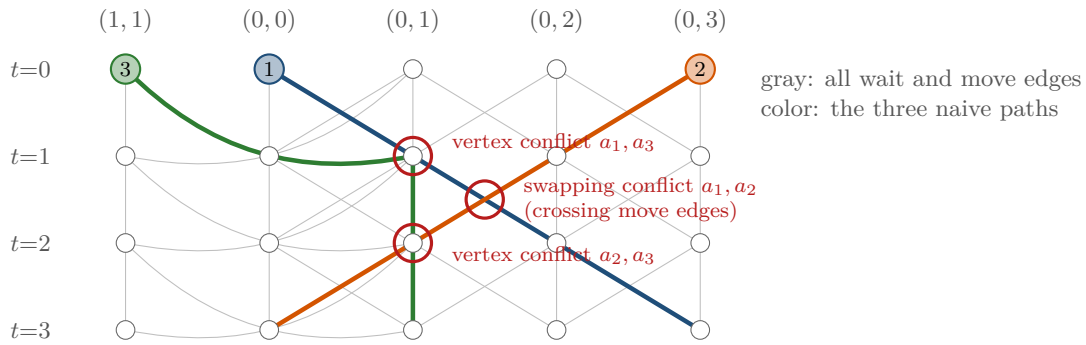


Figure 7.5. The naive plan of [Example 7.9](#) in space-time. Columns are the cells of the top row and the pocket $(1, 1)$, rows are time steps, and the gray lines are the wait and move edges of the time-expanded graph. The two red rings on nodes are the vertex conflicts at $(0, 1)$; the ring between the rows marks the crossing move edges of the swap.

A conflict-free plan. The lower half of [Table 7.2](#) and [Figure 7.6](#) show a solution. Agent a_3 backs down into $(2, 1)$ and waits there for one step; a_1 steps into the vacated pocket $(1, 1)$ at $t = 2$ and lets a_2 pass along the row; then a_1 climbs back and continues, and a_3 follows one step behind it to its goal. Check the columns: at $t = 2$ the positions are $(1, 1)$, $(0, 1)$, $(2, 1)$, at $t = 3$ they are $(0, 1)$, $(0, 0)$, $(1, 1)$, no column repeats a cell, and no pair of moves reverses an edge. The plan does contain following conflicts. Between $t = 1$ and $t = 2$, a_2 enters $(0, 1)$ in the step in which a_1 leaves it; between $t = 2$ and $t = 3$, a_1 enters $(0, 1)$ as a_2 leaves it while a_3 enters $(1, 1)$ as a_1 leaves it, a convoy of three. Classical MAPF allows all of them, and `validate_plan` accepts the plan. Its costs are 5, 3 and 4, so $\text{SoC} = 12$ and $\text{MS} = 5$: coordination costs five extra time steps in total and two extra steps of mission time compared with the lower bounds 7 and 3. The price is unavoidable: a_1 and a_2 must pass each other, the only side cells of the top row are $(1, 1)$ and $(1, 3)$, and an agent that ducks into $(1, 3)$ can return to the top row only through $(0, 3)$, which a_1 occupies from $t = 3$ on, or by

the detour through row 2 and the pocket (1, 1); so the pocket (1, 1) must be used, and a_3 has to clear it first. The brute-force searches of the joint state space in `ch07_mapf.py` confirm that no solution has $\text{SoC} < 12$ or $\text{MS} < 5$, so this plan is optimal for both objectives at once; Example 7.7 showed that this is a property of the instance, not a rule.

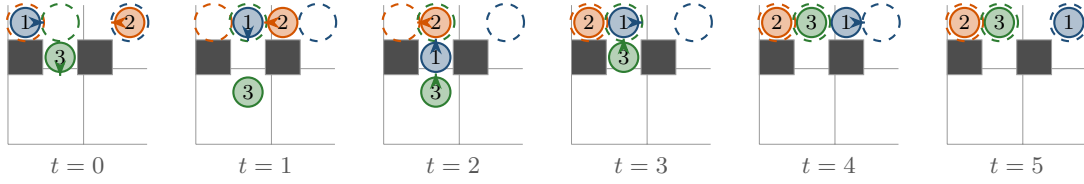


Figure 7.6. The conflict-free plan of Example 7.9 as six snapshots; an arrow shows the move that an agent executes in the step that follows. At $t = 2$, a_1 hides in the pocket while a_2 passes; from $t = 3$ on, a_3 follows a_1 one cell behind, which classical MAPF allows.

7.6 Properties: feasibility, optimality and complexity

7.6.1 Lower bounds

Proposition 7.10 (Elementary bounds). *Let $\text{dist}(u, v)$ be the distance of Chapter 2 in G and let Π be any solution of $\langle G, s, \gamma \rangle$. Then*

$$\text{SoC}(\Pi) \geq \sum_{i=1}^k \text{dist}(s_i, \gamma_i), \quad \text{MS}(\Pi) \geq \max_i \text{dist}(s_i, \gamma_i), \quad \text{MS}(\Pi) \leq \text{SoC}(\Pi) \leq k \cdot \text{MS}(\Pi). \quad (7.3)$$

Proof. The path π_i is at γ_i from time $\text{cost}(\pi_i)$ on and makes at most one move per step, so $\text{cost}(\pi_i) \geq \text{dist}(s_i, \gamma_i)$; summing or maximizing over i gives the first two inequalities. The third holds because a sum of k non-negative numbers lies between their maximum and k times their maximum. $\square$

The first two bounds are the values of the naive plan, the root of the constraint tree of CBS (Chapter 9) and the yardstick of enhanced CBS (ECBS, Chapter 10); the gap between them and the optimal cost is the price of coordination, 5 and 2 in Example 7.9.

7.6.2 Feasibility is easy

Theorem 7.11 (Feasibility can be decided in polynomial time). *Let G be connected and let at least two vertices be empty, $k \leq |V| - 2$. Whether $\langle G, s, \gamma \rangle$ has a solution can be decided in polynomial time, and if it has one, a solution with $\mathcal{O}(|V|^3)$ moves can be constructed in polynomial time [92, 178].*

Proof sketch. Kornhauser, Miller and Spirakis [92] studied the **pebble motion** problem, the sequential version of MAPF in which one agent at a time moves to an adjacent empty vertex. They showed that its solvability can be decided in polynomial time and that a solvable instance has a solution of $\mathcal{O}(|V|^3)$ moves. A sequence of single moves is a classical MAPF plan in which all other agents wait, so a solvable pebble instance is a solvable MAPF instance. Conversely, as long as one vertex is empty, every parallel step of a MAPF solution can be serialized: the moves of a step form chains and cycles, a chain ends at an empty vertex and is executed from its head backward, and a rotation on a fully occupied cycle is done by walking the empty vertex to a neighbor of the cycle, rotating the agents through it one at a time, and walking the empty vertex back along the same route, which undoes the displacement it caused.

The two problems therefore have the same solvable instances, and the decision procedure for pebble motion decides MAPF. Push-and-Rotate (Chapter 11) turns this argument into an algorithm: it is complete for instances with at least two empty vertices, reports the unsolvable ones, and builds a solution in polynomial time [178]. $\square$

Two remarks. First, decidable does not mean solvable: two agents at the ends of a corridor that must exchange places have no solution however many empty cells the corridor contains, because on a path graph their left-to-right order can only change through a swap (Exercise 7.6); a single pocket cell makes the instance solvable, and with exactly one empty vertex the classic case is the 15-puzzle, half of whose arrangements are unreachable. Second, the solutions of the theorem are long, $\mathcal{O}(|V|^3)$ moves executed largely one at a time, and nobody would fly them. What feasibility buys is a certificate: a planner that fails on a solvable instance, as prioritized planning does in Chapter 8, is failing for reasons of its own.

7.6.3 Optimality is hard

Theorem 7.12 (Optimal MAPF is NP-hard). *Given a classical MAPF instance and an integer B , deciding whether a solution with $MS \leq B$ exists is NP-hard, and so is deciding whether a solution with $SoC \leq B$ exists [165, 184]. Unless $P = NP$, no algorithm finds optimal solutions for either objective in time polynomial in the size of the instance.*

Proof idea. Both results are reductions from Boolean satisfiability: Surynek [165] for the optimization variant of multi-robot path planning on graphs, Yu and LaValle [184] from 3-SAT for the makespan, for the total arrival time (our sum of costs) and for the total distance. They share one idea. Every variable becomes an agent with exactly two shortest routes, one standing for *true* and one for *false*, and every clause becomes a gadget that the routes of its literals pass through, timed so that the plan can stay at the naive lower bound of Proposition 7.10 only if at least one literal is true; when all three are false, some agent has to wait or detour and pays an extra step. A solution at the lower bound therefore exists if and only if the formula is satisfiable. The gadgets are the substance of the two papers. $\square$

NP-hardness does not say that every instance is hard; it says that a solver which guarantees optimality must, on some instances, take exponential time. The solvers of Chapters 9 to 11 exploit three properties of the instances that occur in practice: most pairs of agents never interact, so a solver should couple agents only where their paths conflict (independence detection [156] and M^* , Chapter 11); the optimal cost is usually close to the naive lower bound, so a search that starts from the naive plan and adds one constraint at a time has little ground to cover (that is CBS); and a few per cent above the optimum is often good enough (that is ECBS). Searching the joint state space directly is optimal but hopeless beyond a handful of agents: it has $|V|^k$ configurations per time step, $14^3 = 2744$ for the three agents on the fourteen free cells of Example 7.9 but 14^{10} for ten.

7.7 Solver families

Theorems 7.11 and 7.12 split the design space. A solver that promises optimality must in the worst case take exponential time, a solver that promises polynomial time must give up on optimality, and the families of Table 7.3 differ in what they promise about solution quality and in what they pay for it [41, 161].

What the families search. The table says what each family guarantees; what distinguishes the members is what they search. CBS [150] searches a tree of constraints, branching on the first conflict that the detector of Section 7.4 reports; M^* searches the joint space, coupling only the agents that actually conflict; ICTS searches the vectors of path costs in increasing order.

Table 7.3. Solver families for classical MAPF. “Complete” means that the solver returns a solution whenever one exists, given enough time; the guarantee refers to the objective the solver is run with.

Family	Guarantee	Representatives	Where to use it
optimal, search-based	optimal and complete	CBS (Chapter 9); M* (Chapter 11); increasing cost tree search (ICTS) [151]; A* in the joint space with operator decomposition and independence detection [156]	tens of agents; when the plan is flown many times or optimality is the point
bounded-suboptimal	cost $\leq w \cdot$ optimum, complete	ECBS (Chapter 10); EECBS [105]	hundreds of agents; the default choice for a swarm planner
unbounded-suboptimal	none on cost; prioritized planning is incomplete, Push-and-Rotate is complete	prioritized planning, cooperative A* [153] (Chapter 8); Push-and-Swap, Push-and-Rotate (Chapter 11)	real-time replanning, very many agents, sparse maps
reduction-based	optimal for the encoded objective, usually the makespan; complete	SAT encodings (surveyed in [41]); integer programming [185] (Chapter 22)	small dense instances; when a strong external solver is at hand
learned	none	PRIMAL [145]	decentralized execution with partial observation; needs a validator and a fallback

All three pay for optimality with exponential worst-case time, as Theorem 7.12 demands. ECBS [8] searches the tree of CBS with focal search instead of best-first search and returns a solution of cost at most w times the optimum for a chosen $w \geq 1$, typically 1.1 to 1.5; for a drone swarm this is the family to start from, because a few per cent of extra flight time buys an order of magnitude in the number of agents. Prioritized planning [153] searches one agent at a time in a fixed order, with space-time A* around the paths of the earlier ones, which is fast but can fail on solvable instances (Chapter 8 gives counterexamples), while Push-and-Swap and Push-and-Rotate [110, 178] search nothing: they move the agents to their goals one by one and resolve blockages with fixed local maneuvers, and Push-and-Rotate is the complete algorithm behind Theorem 7.11. Reduction-based solvers search over a bound: they fix a makespan bound B , encode “is there a solution with $MS \leq B$?” as a Boolean formula or an integer program with one variable per agent, vertex and time step, hand it to a SAT or integer-linear-programming (ILP) solver, and increase B until the answer is yes, so the first yes is makespan-optimal by construction (Chapter 22 shows the encoding [185]). Learned solvers such as PRIMAL [145] search nothing at run time: a decentralized policy trained by imitation of an optimal solver and by reinforcement learning maps each agent’s local observation to its next move. Such policies scale and react, but they guarantee neither completeness nor collision freedom, so their plans must pass the validator of this chapter and a classical solver must stand by as a fallback.

7.8 Benchmarks and instance files

Solvers are compared on the MAPF benchmark of Stern et al. [161], which is hosted together with the older single-agent grid benchmarks at movingai.com. It consists of grid maps in the `.map` format and, for every map, scenario files in the `.scen` format that list start–goal pairs. Listing 7.1 shows a complete map with two agents; the same text is parsed in the self-test of `ch07_mapf.py`, and you will use the real files from Chapter 8 to Chapter 11 and in the experiments of Chapter 25.

Listing 7.1. A complete `.map` file (top: four header lines, then one text row per grid row) and a `.scen` file with two agents (bottom). The fields of a scenario line are separated by tabs; spaces are shown here.

```

type octile
height 3
width 4
map
....
.@T.
....

version 1
0  small.map  4  3  0  0  3  2  5
1  small.map  4  3  3  0  0  2  5

```

The map format. The header names the movement model of the single-agent benchmark (octile; MAPF work uses 4-connected moves and ignores it), the height and the width, and the word `map`. Then follow `height` lines of `width` characters, the first line being the top row of the map. A dot is a free cell; `@` and `T` are obstacles (`T` stands for trees on the game maps). The older single-agent maps also use `G` and `S` (passable) and `O` and `W` (blocked); our parser accepts `.`, `G` and `S` as free and treats every other character as blocked. The cell in row y and column x of the file is our vertex $(r, c) = (y, x)$.

The scenario format. After the line `version 1`, every line describes one agent by nine fields: a bucket number (a distance group of the single-agent benchmark, ignored by MAPF solvers), the name of the map file, its width and height, the start s_x, s_y , the goal γ_x, γ_y , and the length of the agent’s shortest path as computed by the benchmark’s own tools, which you should treat as informative only: in the real files it is an *octile* distance written as a floating-point number, 4.82843 rather than the 4-connected integer of Listing 7.1, so parse it as a float or skip it. The coordinates are x (column) first and y (row) second, the opposite of our (r, c) : the first agent of Listing 7.1 starts in cell $(0, 0)$ and its goal $(3, 2)$ is the cell $(r, c) = (2, 3)$. Transposing x and y is the most common bug in home-made parsers, and it goes unnoticed on square maps until an agent starts inside a wall. An *instance* is made from a scenario by taking its first k agents. The standard protocol runs a solver with a time limit (one minute is common) for $k = 10, 20, 30, \dots$ and reports the **success rate**, the fraction of the scenarios of a map solved within the limit, as a function of k . Every map comes with two families of scenarios, `random-1` to `random-25`, whose starts and goals are drawn uniformly, and `even-1` to `even-25`, whose start–goal distances are spread evenly over the distance buckets of the single-agent benchmark.

Map families. Table 7.4 lists the families of maps in the benchmark with examples and with what makes each of them hard. The names encode the size and, for random and maze

maps, the obstacle density or corridor width; the set of maps has grown since 2019, so take the current list from the website and name the exact maps and scenarios in any result you report.

Table 7.4. Families of maps in the MAPF benchmark [161]. The “watch out” column names the feature of the family that decides which solvers succeed on it.

Family	Examples	Structure	Watch out for
empty	empty-8-8, empty-32-32, empty-48-48	no obstacles	only the agent density matters; many equally short paths per agent
random	random-32-32-10, random-32-32-20, random-64-64-20	10 % or 20 % random obstacles	narrow passages appear by chance; the most used family in papers
room	room-32-32-4, room-64-64-8, room-64-64-16	square rooms joined by single-cell doors	doors are bottlenecks that only one agent can use per step
maze	maze-32-32-2, maze-32-32-4, maze-128-128-10	corridors of width 2, 4 or 10	long corridors with few alternatives; incomplete solvers fail at small k
warehouse	warehouse-10-20-10-2-1, warehouse-20-40-10-2-2	rows of shelves with aisles between them	the layout closest to the hall of Section 7.1 ; aisles force convoys
city	Berlin_1_256, Boston_0_256, Paris_1_256	street networks of 256×256 cells	long paths, large horizon T ; memory of space-time searches
game	den312d, den520d, brc202d, w_woundedcoast	maps from the games <i>Dragon Age: Origins</i> and <i>Dragon Age 2</i>	irregular open areas and chokepoints; the classic single-agent test maps

A caveat. The benchmark is two-dimensional and 4-connected, its agents have unit size and unit speed and its time steps are synchronous, so it is the right tool for comparing MAPF solvers with each other, which is how [Chapter 8](#) to [Chapter 11](#) use it, and the wrong tool for predicting how a swarm of drones will behave. [Chapter 25](#) builds the drone-specific scenarios on top of these files rather than instead of them, and quotes no success rate without naming the map, k and the time limit.

7.9 Implementation notes

ch07_mapf.py is the foundation that the solvers of [Chapters 8](#) to [11](#) build on. A cell is a tuple (row, col), a path a Python list of cells with `path[t]` the cell at time t , and a plan a list of paths; `MAPFInstance` holds the grid, the starts and the goals and checks [Definition 7.1](#) in `check()`. [Listing 7.2](#) shows the three functions that encode the stay-at-target convention (`sum_of_costs` and `makespan` are then one-line reductions of `path_cost` over the plan). `position` is `Position` of [Algorithm 7.1](#); `path_cost` walks back from the end of the list over the trailing waits at the goal, so that a list which ends with several copies of the goal cell gets the cost of [Equation \(7.1\)](#), and a list that leaves the goal and comes back is charged until the last arrival.

Listing 7.2. Positions with stay-at-target padding, the horizon and the path cost (from code/ch07_mapf.py).

```

1 def position(path: Path, t: int) -> Cell:
2     """Cell of an agent at time t; after its last step it stays at the goal."""

```

```

3     return path[t] if t < len(path) else path[-1]
4
5
6 def horizon(paths: Sequence[Path]) -> int:
7     """Largest time index that any path mentions explicitly."""
8     return max(len(p) for p in paths) - 1
9
10
11 def path_cost(path: Path) -> int:
12     """Time of the final arrival at the last cell (trailing waits cost
13     nothing)."""
14     goal = path[-1]
15     t = len(path) - 1
16     while t > 0 and path[t - 1] == goal:
17         t -= 1
18     return t

```

`first_conflict_hashed` in `ch07_mapf.py` is [Algorithm 7.2](#), with two differences from the pseudocode. First, when several conflicts occur in the same time step the pseudocode returns whichever it meets first, but the answer of a detector should not depend on the order of the agents in memory, so the implementation finishes the step and keeps the conflict with the smallest agent pair (i, j) , which is exactly what the pairwise `first_conflict` returns. Second, the swap test stores the directed edge (u, v) and looks up (v, u) ; two agents on the same directed edge are not reported here because they already produced a vertex conflict at u . The self-test checks on sixty random plans, half of them with random waits inserted so that the paths have different lengths, that the pairwise and the hashed detectors return identical results, both for the first conflict and for the list of all conflicts.

The validator `validate_plan` first checks that every path is legal for its agent, that is, starts at s_i , ends at γ_i , stays on free cells and moves at most one cell per step, and only then runs the detector; an illegal path would otherwise produce misleading conflicts. Use it as an oracle for every solver you write in the next four chapters: a solver whose plan the validator rejects has a bug, however plausible the plan looks. The brute-force searches `optimal_soc_bruteforce` and `optimal_makespan_bruteforce` explore the joint state space and verify tiny instances such as [Example 7.9](#), nothing larger.

Agents that have arrived still block their cell

The most common detector bug is to compare paths only up to the shorter length. Then the third conflict of [Example 7.9](#), a_2 running into the parked a_3 at $t = 2$, is missed, the plan is declared valid, and two drones meet at $(0, 1)$. Every question about a time beyond the end of a path must be answered with the goal cell, as `Position` does, and every solver must treat a parked agent's goal as blocked for all later times: this is the goal-stay handling of space-time A^* in [Chapter 4](#) and the reason why the reservation tables of [Chapter 8](#) reserve a goal cell “until infinity”.

Off-by-one in time

A path with $T_i + 1$ entries has cost at most T_i , not $T_i + 1$: using `len(path)` as the cost overcounts every agent by one and, worse, makes the sum of costs depend on how many trailing waits a solver happens to store. A swap between t and $t + 1$ is indexed by t , the time before the move; the same conflict written with $t + 1$ points a solver's constraint at the wrong time step. Vertex conflicts are checked for $t = 0, \dots, T$ but swaps only for $t = 0, \dots, T - 1$, since no move starts at T . A conflict at $t = 0$ is impossible in a legal

plan because starts are distinct, yet a validator must check it anyway: a path that starts at the wrong cell is exactly the kind of bug it exists to catch.

Edge conflicts are more than swaps

On a 4-connected grid the only way for two moves to collide without a shared cell is the swap, and forbidding it in a single-agent search means forbidding the *reverse* of every move that another agent makes: an agent that moves $u \rightarrow v$ between t and $t + 1$ produces the edge constraint $\langle a, v, u, t \rangle$ for every other agent a , not $\langle a, u, v, t \rangle$ (Chapter 4). On an 8-connected grid the rule is no longer enough: two agents moving along the two diagonals of the same 2×2 block cross in its center without sharing a vertex or an edge, so a detector for diagonal moves needs a third test for crossing diagonals. On a lattice flown by drones of finite size, even following around a corner can bring two agents closer than the cell side (Exercise 7.7). Which pairs of moves are physically safe is a property of the vehicle and of the graph, and it must be decided before the detector is written.

7.10 MAPF for drones and its place in the system

Classical MAPF was formulated for robots on a warehouse floor and for characters in computer games. Three things change when the agents are drones, and none of them changes the definitions of this chapter.

Three dimensions. The graph becomes a 3D lattice (Figure 7.7): the vertices are the centers of cubic cells of side ℓ , an agent has six move actions ($\pm x, \pm y, \pm z$) and the wait action, which for a drone means hovering and costs energy like any other step. Nothing in Definitions 7.1 and 7.4 refers to the dimension, and neither does the detector: the implementation of Section 7.9 only needs a hashable cell and a neighbor function that returns six neighbors instead of four. The cell side is chosen, as in Chapter 2, at least as large as the drone diameter plus a safety margin; Exercise 7.7 shows that following moves around a corner cut the guaranteed separation to $\ell/\sqrt{2}$ between integer times, so either ℓ is enlarged by that factor or following conflicts are forbidden as well. Vertical moves usually cost more energy than horizontal ones; weighted edges are a small change for the solvers, and the objectives of Definition 7.6 then sum edge costs.

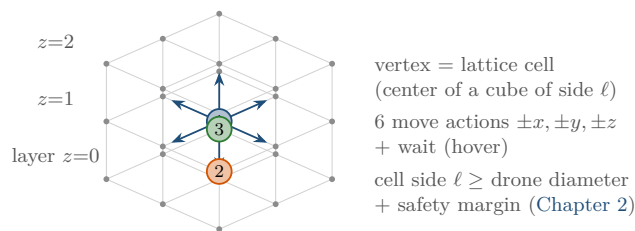


Figure 7.7. A $3 \times 3 \times 3$ block of a 3D flight lattice. A vertex is the center of a cubic cell, a drone has six move actions and a wait (hover) action, and the cell side ℓ is chosen from the drone size and the safety margin; only the neighbor function differs from the 2D case.

Kinematics and continuous time. A drone cannot jump from one cell center to the next in exactly one time step, stop, and turn; it accelerates, decelerates and rounds corners. Hönig et al. [66] showed that this does not force a new problem definition. Their MAPF-POST takes a classical MAPF plan, keeps the *order* in which the agents visit every cell, and turns

the time steps into a simple temporal network: an agent may enter a cell only after its predecessor has left it, and every move respects the velocity limits. Deciding that network is a shortest-path computation on its distance graph (a special linear program), so real arrival times and trajectories with a guaranteed safety distance follow in polynomial time; later work plans quadrotor swarms with the same discrete-then-continuous split [67]. The other route is to make time continuous inside the solver: continuous-time CBS [4] allows actions of arbitrary duration, defines a conflict as two actions whose time intervals overlap and whose geometry collides, and uses safe interval path planning [131] as its low-level search. The price is a far more expensive conflict check, geometric instead of a dictionary lookup.

Lifelong missions. Delivery and inspection missions are lifelong: a drone that reaches its target receives the next one. Lifelong MAPF [112], introduced for warehouse pick-up and delivery, replaces the single goal by a stream of goals and re-plans in a rolling window (Chapter 8 touches windowed planning), so stay-at-target becomes “stay until the next task”.

Why classical MAPF remains the right abstraction. Classical MAPF isolates the combinatorial part of the problem, who passes first at each shared cell, from the continuous part, how a drone flies from one cell to the next, and it is the combinatorial part that is NP-hard (Theorem 7.12). The discrete plan fixes the order of visits at every cell, and this order is what MAPF-POST and the local safety layer of Chapter 24 preserve while they adapt speeds and trajectories. The guarantees, completeness and optimality within a factor w , are proved at the discrete level and survive as long as the cell side, the time step and the safety margins are chosen so that the drone model of Chapter 2 can execute every move and every hover (Exercise 7.7). Continuous-time solvers push the boundary, but they keep the structure of CBS and the conflict types of this chapter: they change what a conflict is geometrically, not what a solver does with it.

In the drone system

In the four-layer architecture of Chapter 24, the global layer solves a classical MAPF instance on the flight lattice with CBS or ECBS (Chapters 9 and 10) before take-off. Its output is exactly the table of Section 7.2: one time-indexed route per drone, with stay-at-target describing a drone that hovers or lands at its target. The prediction layer (Chapters 18 and 20) and the local safety layer (Chapters 13 and 14) know nothing about MAPF; each sees the nominal route of its own drone and the intruders around it. Whenever the local safety layer has deviated from a route or the replanning layer (Chapter 5) has computed a new one, the changed route is re-checked against the others with the detector of Section 7.4, and a conflict triggers a new MAPF solve for the affected agents. The sum of costs and the makespan are two of the metrics of Chapter 25. This chapter is Week 3 of the study plan (Appendix A); its milestone is that you can represent and detect MAPF conflicts correctly, which Exercises 7.4, 7.8 and 7.9 test.

7.11 Summary

Chapter summary

- A classical MAPF instance is $\langle G, s, \gamma \rangle$: an undirected graph, k agents with distinct starts and distinct goals, discrete synchronous time, unit-cost move and wait actions. A plan is one time-indexed path per agent, a table with agents as rows and time steps as columns.

- Stay-at-target: an arrived agent keeps its goal cell for ever and blocks it; a path costs the time of its final arrival, and trailing waits at the goal are free.
- Classical MAPF forbids vertex and swapping conflicts (edge conflicts are vertex conflicts in disguise) and allows following conflicts and cycle conflicts, the rotation of $m \geq 3$ agents on a fully occupied cycle; the degenerate case $m = 2$ of that rotation is the forbidden swap.
- $\text{SoC} = \sum_i \text{cost}(\pi_i)$ and $\text{MS} = \max_i \text{cost}(\pi_i)$; the naive plan bounds both from below, $\text{MS} \leq \text{SoC} \leq k \text{MS}$, and three agents suffice for the two objectives to disagree.
- Conflict detection over the padded paths up to the horizon T costs $\mathcal{O}(k^2 T)$ pairwise and $\mathcal{O}(kT)$ expected with hash maps.
- Feasibility is polynomial on a connected graph with two empty vertices; optimality is NP-hard for both objectives, and the five solver families trade that hardness for guarantees in different ways.
- Instances come as `.map` files (`.` free, `@` and `T` blocked) and `.scen` files (x = column, y = row); the first k agents of a scenario form an instance. Drones add 3D lattices, kinematics and continuous time, but the classical problem keeps the combinatorial core.

Further reading. Stern et al. [161] is the source of the definitions and of the benchmark and surveys the variants; read it in full. Felner et al. [41] survey the search-based optimal solvers, Sharon et al. [150] is the reference for CBS (Chapter 9), and Silver [153] the origin of cooperative A* (Chapter 8). Complexity: Surynek [165] and Yu and LaValle [184] for hardness, Kornhauser et al. [92] and de Wilde et al. [178] for feasibility. For drones: Hönig et al. [66, 67] on kinematic constraints, Andreychuk et al. [4] on continuous time and Ma et al. [112] on lifelong MAPF.

7.12 Exercises

Exercise 7.1 (★). Two agents move on the path graph with vertices A, B, C and edges $A-B, B-C$. For each of the following pairs of actions between $t = 0$ and $t = 1$, list every conflict type of Definition 7.4 that occurs and say whether classical MAPF forbids the pair: (a) $a_1: A \rightarrow B, a_2: B \rightarrow C$; (b) $a_1: A \rightarrow B, a_2: B \rightarrow A$; (c) $a_1: A \rightarrow B, a_2: C \rightarrow B$; (d) a plan handed to a validator in which both agents are at A at $t = 0$ and at B at $t = 1$; (e) $a_1: A \rightarrow B, a_2$ waits at B .

Exercise 7.2 (★). Three agents on an empty 3×3 grid have the goals $(0, 2), (2, 2)$ and $(1, 1)$ and the paths $\pi_1 = ((0, 0), (0, 1), (0, 1), (0, 2), (0, 2), (0, 2))$, $\pi_2 = ((2, 2))$ and $\pi_3 = ((2, 1), (1, 1), (1, 2), (1, 1))$. Compute $\text{cost}(\pi_i)$ for each agent with Equation (7.1), then SoC and MS. Explain why π_3 does not cost 1 although it reaches its goal at $t = 1$. Check by hand that the plan has no vertex or swapping conflict.

Exercise 7.3 (★★). (a) Show that both inequalities $\text{MS} \leq \text{SoC} \leq k \text{MS}$ of Proposition 7.10 can hold with equality, by giving a plan for each case. (b) Suppose all k shortest paths have the same length L and some solution has makespan L . Show that a solution is optimal for the sum of costs if and only if it is optimal for the makespan. (c) Example 7.7 has costs 4, 2, 3 in the naive plan and optimal values $\text{SoC} = 10$ and $\text{MS} = 4$. Use Proposition 7.10 and the fact that the shortest paths of all three agents are unique to explain, without any search, why no solution can have $\text{SoC} = 9$ and why $\text{MS} = 4$ forces at least two agents to be delayed.

Exercise 7.4 (★★). Create deliberate conflict cases, each as a small grid with explicit paths, and predict what Algorithm 7.1 returns before you run `all_conflicts` on them: (a) exactly

one vertex conflict at a single time step and nothing else; (b) a swapping conflict between two agents that never share a cell; (c) two agents following each other through a corridor without any forbidden conflict; (d) a conflict caused only by the stay-at-target padding, an agent running into one that arrived earlier; (e) four agents rotating on a 2×2 block, and the verdict of the validator. Then explain why case (b) needs a corridor of even length.

Exercise 7.5 (★★). (a) Prove the claim of the walkthrough of [Algorithm 7.1](#) in two steps. First show that in any padded plan, whose paths need not end at distinct goals, a vertex or swapping conflict at some $t > T$ implies a vertex conflict at T ; then show that in a legal plan, whose paths do end at the distinct goals, no conflict of either kind occurs at any $t \geq T$. (b) Under disappear-at-target an agent is removed from the graph when it first reaches its goal. Which of the three conflicts of the naive plan in [Example 7.9](#) remain, and how must [Position](#) and [Equation \(7.1\)](#) change to implement this convention? (c) Show that every stay-at-target solution whose paths visit their goals only at the end is also a disappear-at-target solution, and give a plan that is a solution under disappear-at-target but not under stay-at-target.

Exercise 7.6 (★★). Two agents sit at the two ends of a corridor of $n \geq 2$ cells (a path graph) and must exchange ends. (a) Prove that the instance has no solution: define the left-to-right order of the two agents and show that no allowed step changes it. (b) Add one pocket cell adjacent to an interior cell of the corridor; give a solution for $n = 4$ and compute its SoC and MS. (c) The corridor with $n = 4$ has two empty cells. Explain why this does not contradict [Theorem 7.11](#), and what Push-and-Rotate ([Chapter 11](#)) is guaranteed to report on it.

Exercise 7.7 (★★★). Two drones fly on a 2D lattice with cell side ℓ ; within a time step each either hovers or moves along one edge at constant speed. Show that any plan without vertex and swapping conflicts keeps them at least $\ell/\sqrt{2}$ apart at every instant, that the bound is attained by a following move around a corner, and derive the cell side needed to keep two drones of diameter d at least m apart at all times. How does the answer change if following conflicts are forbidden as well?

Exercise 7.8 (★★★ Coding). (a) Implement a plan validator and both conflict detectors of [Section 7.4](#) without looking at `ch07_mapf.py`; test them on [Exercise 7.4](#) and [Example 7.9](#) and check that they agree on random plans. (b) Write the `.map` and `.scen` parsers, download `random-32-32-20` with its scenario `random-1`, and for the first $k = 10, 20, \dots, 100$ agents count the conflicts of the naive plan and time both detectors. (c) Extend the detector to 8-connected grids with a test for crossing diagonals, and exhibit a plan that the 4-connected detector accepts although two agents collide.

Exercise 7.9 (★★★ Coding). This is the first half of the Week 3 coding exercise ([Appendix A](#)) — represent MAPF instances and plans, and detect their conflicts; the second half, prioritized planning for two to five agents, is the coding exercise of [Chapter 8](#). (a) Put the pieces of [Exercises 7.4](#) and [7.8](#) together into one small library: an instance $\langle G, s, \gamma \rangle$ read from a `.map` and a `.scen` file or written by hand, a plan as one time-indexed path per agent padded by the stay-at-target rule of [Definition 7.3](#), and the detector on top of them. (b) You need plans to check, so plan the agents one after another in index order with the space-time A^* of [Chapter 4](#), each agent treating the cells, the reversed moves and the parked goals of the earlier agents as forbidden. This is prioritized planning in its crudest form, and here it is only a source of plans: run the detector on every plan it returns and confirm that it reports no conflict whenever the planner succeeds, on [Example 7.9](#) and on the corridor with one pocket of [Exercise 7.6](#). (c) Report $\text{cost}(\pi_i)$, SoC and MS for the plan of [Example 7.9](#) and compare the sum of costs with the optimum 12; explain the gap in terms of the conflicts your detector found in the naive plan. [Chapter 8](#) turns the sketch of part (b) into a planner — reservation table, priority orders, the revised rule — and that is where the second half does its work.

Prioritized Planning and Space-Time A^*

Learning objectives

- Explain prioritized planning in one paragraph: agents are planned one after another, and each treats the paths of the agents before it as moving obstacles.
- Build a reservation table by hand, including the stay-at-target reservation, and run Cooperative A^* against it on a small grid.
- Show with small instances that the priority order matters, that no order may work even when a solution exists, and that the best order can still be suboptimal.
- State and prove the completeness result for well-formed instances.
- Implement prioritized planning for two to five agents, create deliberate conflict cases, and validate the resulting plans.
- Explain the hierarchical and windowed variants of Cooperative A^* , choose a priority order with a simple heuristic, and describe priority-based search.

8.1 Why this matters for a drone swarm

Picture a warehouse roof with forty inspection drones. Each drone has a start pad and an inspection point, and the airspace between them is cut into a grid of cells by the support pillars. [Chapter 7](#) taught you how to describe this situation as a multi-agent path finding (MAPF) instance and how to detect the conflicts of a plan. It also warned you that finding an optimal plan is NP-hard. Forty drones is far beyond what a joint search over all agents can handle, and even the optimal solvers of [Chapters 9](#) and [10](#) need time that grows quickly with the number of interacting agents. The operator, however, wants the drones in the air within a second.

The oldest and simplest answer is **prioritized planning**: line the drones up in some order, plan the first one as if it were alone, then plan the second one while treating the first drone's path as a moving obstacle, then the third one against the first two, and so on. Each planning step is a single-agent space-time A^* search of the kind you implemented in [Chapter 4](#). Forty drones cost forty searches. That is why prioritized planning, introduced by Erdmann and Lozano-Pérez [38] and made popular for grids by Silver's *Cooperative A^** [153], is the workhorse of video-game crowds, warehouse robots and large drone swarms. It is also the study plan's Week 3 exercise ([Appendix A](#)): implement it for two to five agents and create deliberate conflict cases. In the hybrid architecture of [Chapter 24](#) the global layer is conflict-based search (CBS, [Chapter 9](#)) or its bounded-suboptimal variant ECBS ([Chapter 10](#)); prioritized planning is the natural fallback when they run out of time, and its reservation table is the bookkeeping that a replanning layer needs when one drone has to re-route around the frozen plans of all

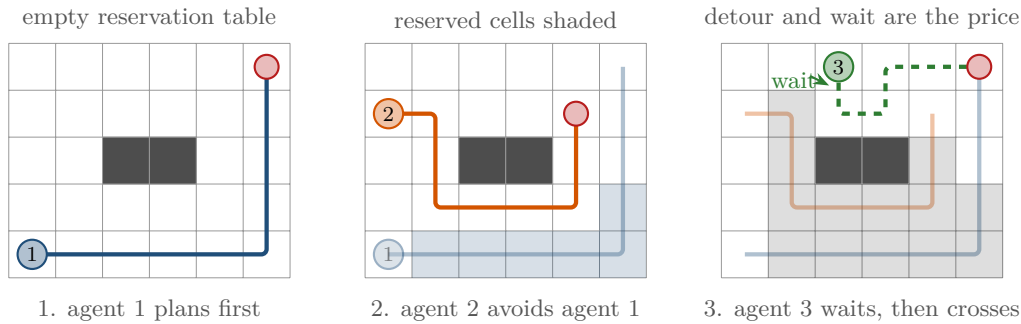


Figure 8.1. Prioritized planning on a 6×5 grid with two pillars. Left: agent 1 is planned first against an empty reservation table and takes a shortest path. Middle: the cells of that path are now reserved at the times agent 1 occupies them (shaded), and agent 2 plans around them. Right: agent 3 plans against both earlier paths; it waits one step before crossing the corridor between the pillars. Notice that the earlier agents never move for the later ones: waits and detours are always paid by whoever comes later in the order.

the others.

The price of the speed is that the method is greedy. A drone that is planned early never makes room for a drone that is planned late. This chapter shows exactly when that matters: the order can change the cost, the method can fail although a solution exists, and even its best order can be more expensive than the optimum. It also gives the positive result that makes prioritized planning safe in practice: on *well-formed* instances, where every agent can reach its goal without touching another agent's start or goal, prioritized planning succeeds in every order [27].

Roadmap. Section 8.2 explains the idea and the reservation table. Section 8.3 fixes the notation and Section 8.4 gives the pseudocode of prioritized planning and of Cooperative A*. Section 8.5 works through a three-agent instance in every one of its six orders. Section 8.6 collects the properties: soundness, incompleteness, suboptimality, complexity and the completeness theorem for well-formed instances. Section 8.7 covers the hierarchical and windowed variants, priority ordering heuristics with a small experiment, and priority-based search. Sections 8.8 and 8.9 give the implementation notes and the place of the method in the drone system.

8.2 The idea in plain words

Suppose you have already planned drone 1. Its path tells you where it will be at every time step: cell v_0 at time 0, cell v_1 at time 1, and so on until it reaches its goal, where it stays. Now you plan drone 2. Instead of searching the joint space of both drones, you search only for drone 2, in space-time, and you forbid every (cell, time) pair that drone 1 occupies. Drone 1 is a moving obstacle: a wall that appears in one cell at one time step and moves on. If drone 2 wants to enter a cell that drone 1 occupies at that moment, drone 2 waits a step or goes around. When drone 2 is done, both paths are frozen, and drone 3 plans against both. The bookkeeping that stores the frozen paths is the **reservation table**: a set of (cell, time) entries that are taken, plus a set of (edge, time) entries so that two drones cannot swap cells by moving through each other.

Figure 8.1 shows the three steps on a small map, and Figure 8.3 in the worked example draws the table itself as a space-time diagram. Three details matter and are easy to get wrong.

- **Stay at target.** Under the stay-at-target convention of Chapter 7, an agent that arrives

at its goal at time T remains there for ever. Its goal cell must therefore be reserved for *all* times $t \geq T$, not only for $t = T$. Forgetting this lets a later agent drive through a parked drone.

- **Edge reservations.** Two agents that exchange cells between t and $t + 1$ never occupy the same cell at the same time, so vertex reservations alone do not catch this swapping conflict. When agent 1 moves from u to v between t and $t + 1$, the table records that move at time t , and no later agent may make the reverse move $v \rightarrow u$ in that step.
- **The goal of a later agent.** A later agent may only finish its path at a time T after which its goal is never reserved. If a higher-priority agent passes through that goal cell at some time $t' > T$, the later agent must wait somewhere and arrive after t' .

Key idea

Prioritized planning turns a k -agent problem into k single-agent space-time searches. Agent i sees agents $1, \dots, i - 1$ as moving obstacles stored in a reservation table, and never sees agents $i + 1, \dots, k$ at all. This is what makes it fast, and it is also the only reason it can fail.

Cooperative A* [153] is the name Silver gave to the combination of space-time A* with a reservation table. The word *cooperative* is slightly optimistic: only the later agents cooperate, by yielding to the earlier ones. Nevertheless, in open maps with few agents this one-sided cooperation is all that is needed, and the plans it produces are usually close to optimal. The trouble starts in corridors and near goals, as the counterexamples of Section 8.6 show.

8.3 Problem statement and notation

We use the classical MAPF setting of Chapter 7 and of Stern et al. [161]: an undirected graph $G = (V, E)$, usually a 4-connected grid, and k agents $a_1, \dots, a_k$ with distinct start vertices s_i and distinct goal vertices γ_i . Time is discrete. In one time step an agent either moves along an edge or waits in its cell; both actions cost one. A **path** for agent i is a sequence $\pi_i = (\pi_i[0], \pi_i[1], \dots, \pi_i[T_i])$ with $\pi_i[0] = s_i$, $\pi_i[T_i] = \gamma_i$ and consecutive vertices equal or adjacent. The agent stays at γ_i afterward, so we define $\pi_i[t] = \gamma_i$ for all $t > T_i$. The cost of the path is its arrival time T_i and the cost of a plan is the sum of costs $\text{SoC} = \sum_i T_i$; its makespan is $\text{MS} = \max_i T_i$. Two paths π_i and π_j have a **vertex conflict** at time t if $\pi_i[t] = \pi_j[t]$, and a **swapping conflict** at time t if $\pi_i[t] = \pi_j[t + 1]$ and $\pi_i[t + 1] = \pi_j[t]$. A plan is valid if no pair of paths has a conflict of either kind. Following conflicts, where an agent enters the cell that another agent has just left, are allowed, as in the classical setting.

Definition 8.1 (Reservation table). A **reservation table** is a pair $R = (R_V, R_E)$ of sets

$$R_V \subseteq V \times \mathbb{N}, \quad R_E \subseteq E' \times \mathbb{N},$$

where E' contains both orientations (u, v) and (v, u) of every edge. An entry $(v, t) \in R_V$ means that some higher-priority agent occupies vertex v at time t ; an entry $((u, v), t) \in R_E$ means that some higher-priority agent moves from u to v between t and $t + 1$, so that moving from v to u in that step would be a swap. Reserving a path π with arrival time T adds $(\pi[t], t)$ for $0 \leq t \leq T$, adds $((\pi[t], \pi[t + 1]), t)$ for every move $\pi[t] \neq \pi[t + 1]$, and marks the goal $\pi[T]$ as **parked** from time T on, which stands for the infinite set of entries $\{(\pi[T], t) : t \geq T\}$.

A space-time state (v, t) is **blocked** by R if $(v, t) \in R_V$ or v is parked on at a time $\leq t$. A move from u to v at time t is blocked if $(v, t + 1)$ is blocked or $((v, u), t) \in R_E$. Waiting at u is blocked if $(u, t + 1)$ is blocked.

Definition 8.2 (Priority order and prioritized plan). A **priority order** is a permutation σ of $\{1, \dots, k\}$; agent $\sigma(1)$ has the highest priority. The **prioritized plan** for σ is obtained by planning the agents in the order $\sigma(1), \sigma(2), \dots$ where agent $\sigma(j)$ receives a shortest path that is not blocked by the reservation table containing the paths of $\sigma(1), \dots, \sigma(j-1)$. If some agent has no such path, the plan for σ *fails*. Under the **revised rule** of Čáp et al. [27], agent $\sigma(j)$ additionally treats the start vertices $s_{\sigma(j+1)}, \dots, s_{\sigma(k)}$ of the lower-priority agents as static obstacles.

The definition makes precise what *greedy* means here: each agent takes a path that is optimal for itself given the agents before it, and that path is never revised. The revised rule costs nothing in open maps and is the ingredient that makes the completeness theorem of Section 8.6 work: a lower-priority agent that has not moved yet can never be run over. The rule does need the goal γ_i of the planned agent not to be the start of a lower-priority agent. That is automatic on the well-formed instances of Definition 8.3, whose endpoints are all distinct; on an arbitrary instance in which one agent's goal is another agent's start, the rule turns that goal into a static obstacle and the search fails at once.

Definition 8.3 (Well-formed instance). Let $P = \{s_1, \dots, s_k, \gamma_1, \dots, \gamma_k\}$ be the set of all **endpoints**, and assume that they are pairwise distinct, $|P| = 2k$. An instance is **well-formed** if for every agent i there is a path in G from s_i to γ_i that visits no endpoint of P other than s_i and γ_i . In words: every agent can reach its goal without passing through the start or goal of any other agent [27].

Distinctness is not a technicality. If some γ_j were the start s_i of another agent, the revised rule would turn γ_j into a static obstacle for agent j and its own search would fail at once, as noted after Definition 8.2; the completeness theorem of Section 8.6 would then be false for the very instances the definition is meant to describe. The function `is_well_formed` of `code/ch08_prioritized.py` therefore tests distinctness first and only then runs the k breadth-first searches.

Well-formedness is a property of the map and the endpoints alone; it does not depend on time or on the priority order, and it is cheap to test with k breadth-first searches (Chapter 3) on the graph with the other endpoints removed. Most warehouse layouts, where starts and goals are parking bays off the main aisles, are well-formed by design.

8.4 The algorithm

Algorithm 8.1 is the outer loop. It takes an order, keeps one reservation table, and calls Cooperative A* (Algorithm 8.2) once per agent. Cooperative A* is not a new search. It is the space-time A* of Chapter 4, printed in full as Algorithm 4.2, reading its filter as a reservation table; Algorithm 8.2 therefore lists only the lines in which the two differ — the two blocking tests, on vertices and on reverse edges, the goal-stay test computed from the table, and the static obstacles of the revised rule. Everything else, from the wait-first successor order to the duplicate detection, is Algorithm 4.2 unchanged.

Walkthrough of Algorithm 8.1. Line 4 picks the next agent in priority order. Line 5 builds the set B of static obstacles: empty in Silver's original method, the starts of all lower-priority agents under the revised rule. Line 6 runs a full single-agent search; the reservation table R is the only channel through which earlier agents influence it. If the search fails (line 8) the whole procedure fails: there is no backtracking, no re-ordering and no replanning of earlier agents. The callers of Section 8.7 add restarts with new orders around this loop. Line 9 reserves the new path exactly as Definition 8.1 says.

Algorithm 8.1. Prioritized planning

Input: graph G , agents with starts s_i and goals γ_i , priority order σ , horizon $T_{\max}$, flag *revised*

Output: a conflict-free plan $(\pi_1, \dots, \pi_k)$ or *failure*

```

1 Function PrioritizedPlanning( $G, s, \gamma, \sigma, T_{\max}$ , revised):
2    $R \leftarrow (\emptyset, \emptyset)$  // empty reservation table
3   for  $j \leftarrow 1$  to  $k$  do
4      $i \leftarrow \sigma(j)$ 
5      $B \leftarrow \{s_{\sigma(m)} : m > j\}$  if revised, otherwise  $B \leftarrow \emptyset$ 
6      $\pi_i \leftarrow \text{CooperativeAStar}(G, s_i, \gamma_i, R, B, T_{\max}, h)$ 
7     if  $\pi_i = \text{failure}$  then
8       return failure
9      $\text{Reserve}(R, \pi_i)$  // vertices, reverse edges, parked goal
10  return  $(\pi_1, \dots, \pi_k)$ 

```

Walkthrough of Algorithm 8.2. The machinery that is *not* listed matters as much as the six lines that are. Because every action costs one time step, the cost so far of a state (v, t) is simply t , whichever route reached it; the key is therefore $t + h(v)$, the first time a space-time state is generated is also the cheapest, and no entry ever needs its cost updated. That is the argument of Chapter 4, and it carries over unchanged, because a reservation table only removes states.

Line 3 computes the goal-stay time t_γ of Definition 4.17: here it is the last time step at which some higher-priority agent visits γ according to the table. If a higher-priority agent is parked on γ , then γ is reserved for ever and the search cannot succeed, so the function returns failure immediately. The goal test in line 4 accepts (γ, t) only if $t > t_\gamma$: from then on nobody else will use γ , so the agent can stay there. Line 5 generates the wait action first and then the moves, skipping the static obstacles in B ; line 6 discards successors whose target cell is reserved, and line 7 discards moves that would swap with a reserved move. These two tests are the whole of the difference between planning around walls and planning around drones, and Definition 8.1 is what makes them single lookups. The horizon test in line 8 stops the search from waiting for ever: without it, a blocked agent would generate $(s, 1), (s, 2), \dots$ without end. A safe choice of $T_{\max}$, and a trick that removes the need for it, are discussed in Section 8.8.

The heuristic h must be admissible for the static single-agent problem; the Manhattan distance is the usual first choice on 4-connected grids (Chapter 4). Reservations only remove states, so a heuristic that is admissible without reservations is admissible with them, and Algorithm 8.2 returns a path that is shortest among all unblocked paths whose arrival time is at most $T_{\max}$. Ties between states with equal f are broken toward the larger t , the tie-breaking rule of Chapter 4, and then in order of generation, so that the search prefers to keep moving rather than to wait.

8.5 A worked example

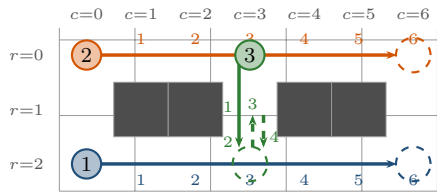
Example 8.4 (Three agents and a short corridor). Figure 8.2 shows a 3×7 grid. Cells are written (r, c) with the row $r \in \{0, 1, 2\}$ counted from the top and the column $c \in \{0, \dots, 6\}$ from the left, as in Chapter 7. The middle row is blocked except for its three open cells $(1, 0)$, $(1, 3)$ and $(1, 6)$, so the map consists of two corridors joined at both ends and in the middle. Agent 1 flies along the bottom corridor from $s_1 = (2, 0)$ to $\gamma_1 = (2, 6)$, agent 2 along

Algorithm 8.2. Cooperative A* as a delta on space-time A*: the six lines by which Algorithm 4.2 becomes a search against a reservation table

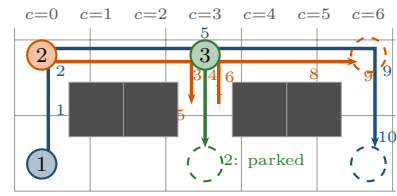
Input: graph G , start s , goal γ , reservation table $R = (R_V, R_E)$, static obstacles B , horizon $T_{\max}$, heuristic h

Output: a shortest unblocked path from s to γ or *failure*

- 1 **Function** CooperativeAStar($G, s, \gamma, R, B, T_{\max}, h$):
- 2 run SpaceTimeAStar($G, s, \gamma, R, T_{\max}$) of Algorithm 4.2 — states (v, t) , $g(v, t) = t$, key $f(v, t) = t + h(v)$, wait action first, one entry per generated state — with the following changes
- 3 $t_\gamma \leftarrow$ last time at which γ is reserved in R (-1 if never, ∞ if parked on); **return** *failure* if $t_\gamma = \infty$ **or** $(s, 0)$ is blocked in R
- 4 *goal test:* return the path as soon as a popped state has $v = \gamma$ **and** $t > t_\gamma$
- 5 *successors:* take $v' \in \{v\} \cup \text{Succ}(v)$ with $v' \notin B$
- 6 *vertex test:* skip v' if $(v', t + 1)$ is blocked in R , that is, if $(v', t + 1) \in R_V$ or v' is parked on from some time $\leq t + 1$
- 7 *edge test:* skip the move if $v' \neq v$ **and** $((v', v), t) \in R_E$, the reverse of a reserved move
- 8 *horizon:* generate no successors of a state with $t \geq T_{\max}$



(a) order (1, 2, 3): costs $6 + 6 + 4 = 16$



(b) order (3, 1, 2): costs $10 + 9 + 2 = 21$

Figure 8.2. Example 8.4 in two priority orders. Small numbers are arrival times. (a) In the order (1, 2, 3) the two corridor agents take their straight paths; agent 3 reaches its goal at $t = 2$, but must step back up at $t = 3$ to let agent 1 pass underneath and returns at $t = 4$ (dashed). (b) In the order (3, 1, 2) agent 3 parks on the bottom corridor at $t = 2$, agent 1 must take the top corridor (10 steps), and agent 2 has to dodge into (1, 3) at $t = 5$ to let agent 1 through before it can follow it to its goal (9 steps).

the top corridor from $s_2 = (0, 0)$ to $\gamma_2 = (0, 6)$, and agent 3 crosses from $s_3 = (0, 3)$ down through the middle opening to $\gamma_3 = (2, 3)$. Alone, the agents need 6, 6 and 2 steps; the sum 14 is a lower bound on every valid plan. The instance is not well-formed: every path of agent 3 ends on the bottom corridor, which agent 1 cannot avoid.

Building the table. Take the order (1, 2, 3). Agent 1 plans against an empty table and gets the straight path $\pi_1[t] = (2, t)$ for $t = 0, \dots, 6$. Its reservation adds the seven vertex entries $((2, t), t)$, the six move entries $((2, t), (2, t + 1), t)$ for $t = 0, \dots, 5$, and parks $(2, 6)$ from $t = 6$ on. Agent 2 then plans against this table. Nothing in it touches the top corridor, so it gets $\pi_2[t] = (0, t)$ and reserves the corresponding entries. Table 8.1 lists the table that agent 3 now faces. Only four of its entries matter for agent 3: the vertices $(2, 3)$ and $(0, 3)$ are taken at $t = 3$, the move $(2, 2) \rightarrow (2, 3)$ at $t = 2$ forbids the reverse move at that time, and $(0, 2)$ is taken at $t = 2$. The goal-stay time of agent 3 is $t_\gamma = 3$, the last time at which $(2, 3)$ is reserved.

Running Cooperative A* for agent 3. Table 8.2 traces Algorithm 8.2 for agent 3 with the Manhattan heuristic, and Figure 8.3 draws the same search as a space-time diagram of

Table 8.1. The reservation table of Example 8.4 after agents 1 and 2 have been planned in the order (1, 2, 3). The third row lists the entries that affect agent 3.

Agent	Vertex entries R_V	Move entries R_E	Parked
1	$((2, t), t)$ for $t = 0, \dots, 6$	$((2, t), (2, t+1)), t)$ for $t = 0, \dots, 5$	$(2, 6)$ from $t = 6$
2	$((0, t), t)$ for $t = 0, \dots, 6$	$((0, t), (0, t+1)), t)$ for $t = 0, \dots, 5$	$(0, 6)$ from $t = 6$
seen by 3	$((2, 3), 3), ((0, 3), 3), ((0, 2), 2)$	$((2, 2), (2, 3)), 2)$: no move $(2, 3) \rightarrow (2, 2)$ at $t = 2$	$t_\gamma = 3$

Table 8.2. Trace of Algorithm 8.2 for agent 3 of Example 8.4 in the order (1, 2, 3), produced by code/ch08_prioritized.py. Generated successors are space-time states at time $t + 1$; V marks a vertex reservation, E a swap with a reserved move. Ties in f are broken toward the larger t .

Step	Expanded (v, t)	t	h	f	Generated	Blocked
1	$((0, 3), 0)$	0	2	2	$(0, 3)$ wait, $(1, 3)$, $(0, 2)$, $(0, 4)$	–
2	$((1, 3), 1)$	1	1	2	$(1, 3)$ wait, $(0, 3)$, $(2, 3)$	–
3	$((2, 3), 2)$	2	0	2	$(1, 3)$, $(2, 4)$	$(2, 3)$ wait: V, $(2, 2)$: E
4	$((1, 3), 2)$	2	1	3	$(1, 3)$ wait	$(0, 3)$: V, $(2, 3)$: V
5	$((0, 3), 1)$	1	2	3	$(0, 3)$ wait, $(0, 4)$	$(0, 2)$: V
6	$((1, 3), 3)$	3	1	4	$(1, 3)$ wait, $(0, 3)$, $(2, 3)$	–
7	$((2, 3), 4)$	4	0	4	goal: $t = 4 > t_\gamma = 3$	

column $c = 3$. The first two expansions go straight down. At step 3 the search pops $((2, 3), 2)$: the goal, but too early, because $t = 2 \leq t_\gamma = 3$. Its three successors show all three kinds of blocking at once: waiting at $(2, 3)$ until $t = 3$ is a vertex conflict with agent 1, moving left to $(2, 2)$ would swap with agent 1's move $(2, 2) \rightarrow (2, 3)$, and only moving up to $(1, 3)$ and right to $(2, 4)$ survive. Steps 4 and 5 pop the two remaining states with $f = 3$ and find their moves toward the goal blocked as well (moving up from $(1, 3)$ at $t = 2$ runs into agent 2 at $(0, 3)$). Step 6 expands $((1, 3), 3)$, and step 7 pops $((2, 3), 4)$ with $t = 4 > t_\gamma$: the goal test succeeds. The path is $(0, 3), (1, 3), (2, 3), (1, 3), (2, 3)$ with cost 4, two more than the undisturbed 2, and the sum of costs of the order (1, 2, 3) is $6 + 6 + 4 = 16$.

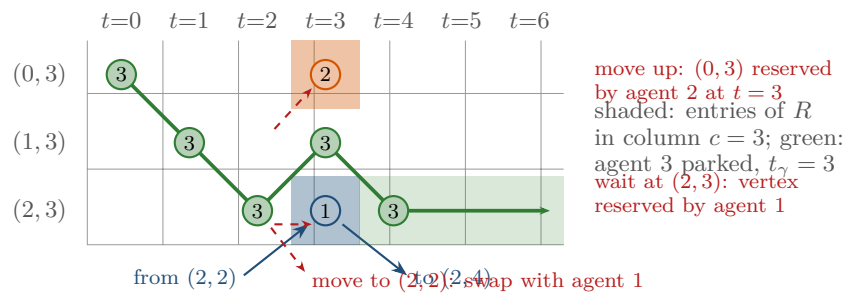
**Figure 8.3.** The reservation table of Table 8.1 restricted to column $c = 3$, drawn in space-time: one row per cell, one column per time step. Shaded squares are entries of R_V (agent 2 at $(0, 3)$ and agent 1 at $(2, 3)$, both at $t = 3$), the arrows below the diagram are agent 1's moves through $(2, 3)$. The green path is the one agent 3 finds; the dashed red arrows are the successors that the table forbids at $t = 2$. From $t = 4$ on agent 3 is parked at its goal (light green).

Table 8.3. Sum of costs of the prioritized plan of [Example 8.4](#) for every priority order (from `code/ch08_prioritized.py`). The optimal sum of costs is 16, and the lower bound from independent shortest paths is 14.

Order	T_1	T_2	T_3	SoC	What happens
(1, 2, 3)	6	6	4	16	agent 3 steps aside once
(1, 3, 2)	6	6	4	16	as above; agent 2 is unaffected
(2, 1, 3)	6	6	4	16	as above
(2, 3, 1)	–	6	2	failure	both corridors closed for agent 1
(3, 1, 2)	10	9	2	21	agent 1 detours, agent 2 dodges and waits
(3, 2, 1)	–	6	2	failure	both corridors closed for agent 1

All six orders. [Table 8.3](#) gives the result of [Algorithm 8.1](#) for every permutation. Whenever agent 1 is planned before agent 3 – the orders (1, 2, 3), (1, 3, 2) and (2, 1, 3) – the plan costs 16, which is also the optimal sum of costs of the instance (the self-test asserts `optimal_soc(inst) == 16`, a brute-force search over the joint space of the three agents). The orders that start with agent 3 are a different story. Agent 3 plans alone, takes two steps down and parks on the bottom corridor at $t = 2$ for ever. In the order (3, 1, 2), agent 1 must now go around through the top corridor: up to (0, 0), right along row 0, and down through (1, 6), which takes 10 steps and reserves the top corridor at the very times agent 2 wants to use it. Agent 2 follows agent 1 to (0, 3), waits there while agent 1 catches up, steps aside into (1, 3) at $t = 5$, lets agent 1 pass, and follows it to (0, 6), arriving at $t = 9$ just after agent 1 has left toward its own goal. The sum of costs is $10 + 9 + 2 = 21$, five more than optimal, because a two-step agent was allowed to cut the corridor that a six-step agent needed. In the order (3, 2, 1) it is even worse: agent 2 parks on (0, 6) at $t = 6$, so the detour through the top corridor is closed as well, and agent 1 has no path at all. The same happens in the order (2, 3, 1). Two of the six orders fail, one is expensive, three are optimal, and nothing in the instance tells you in advance which is which.

8.6 Properties: soundness, incompleteness, suboptimality, complexity

The worked example already displayed the whole character of the method: it is sound, it is fast, and it is greedy. This section states the four properties precisely, gives the two counterexamples that every user of prioritized planning should know by heart, and then proves the one positive result.

Proposition 8.5 (Soundness and per-agent optimality). *If [Algorithm 8.1](#) returns a plan, the plan is valid: no two paths have a vertex or swapping conflict. Moreover, [Algorithm 8.2](#) returns a path whenever an unblocked one with arrival time at most $T_{\max}$ exists. If in addition h is admissible, the path of every agent is a shortest path among all paths that avoid the agents planned before it and arrive no later than $T_{\max}$.*

Proof. Let π_i be planned after π_j . Every state $(\pi_i[t], t)$ was generated as an unblocked successor in [Algorithm 8.2](#), and the goal state satisfies $t > t_\gamma$, so $\pi_i[t] \neq \pi_j[t]$ for all t , including the times at which π_j is parked: no vertex conflict. If $\pi_i[t] = \pi_j[t + 1] = v$ and $\pi_i[t + 1] = \pi_j[t] = u$ with $u \neq v$, then agent j moved $u \rightarrow v$ at time t and $((u, v), t) \in R_E$, so line 7 would have discarded the move $v \rightarrow u$ of agent i at time t : no swapping conflict. For the second claim note that the unblocked space-time states with $t \leq T_{\max}$ form a finite graph with unit edge costs; [Algorithm 8.2](#) is A^* on this graph with a heuristic that is admissible on the static map and hence on the restricted graph, so the optimality theorem of [Chapter 4](#)

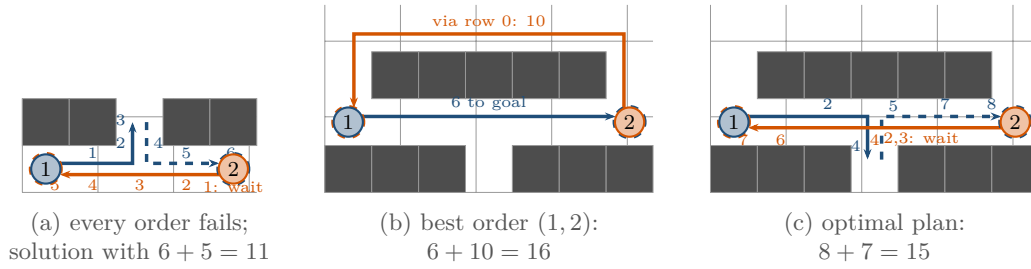


Figure 8.4. Two corridors that defeat prioritized planning. (a) Two agents must exchange the ends of a corridor with a single pocket. Whichever agent is planned first takes the straight path, and the other one can never get past it; the solution shown (agent 1 ducks into the pocket, agent 2 waits one step) has cost 11 but is found by no order. (b) The same corridor with a long bypass. The best order sends agent 1 straight (6) and agent 2 around (10). (c) The optimal plan lets agent 1 use the pocket and agent 2 wait one step: cost 15.

applies. Because the cost of a state (v, t) is t whatever its parent, the first path found to a state is already the cheapest, and no re-opening is needed. $\square$

8.6.1 The order matters, and no order may work

Table 8.3 showed that the order decides between 16, 21 and failure. The next two instances, drawn in Figure 8.4, show that this is not an artifact of a badly chosen order: sometimes there is no good order at all.

Proposition 8.6 (Incompleteness). *Prioritized planning is not complete: there are solvable instances on which the plan fails for every priority order.*

Proof. Consider Figure 8.4(a): a corridor of five cells $(1, 0), \dots, (1, 4)$ with a single pocket $(0, 2)$ above the middle cell. Agent 1 must go from $(1, 0)$ to $(1, 4)$ and agent 2 from $(1, 4)$ to $(1, 0)$. Suppose agent 1 is planned first. Against an empty table its shortest path is the straight one, $\pi_1[t] = (1, t)$ for $t \leq 4$, and it parks on $(1, 4)$ for ever. Agent 2 must leave $(1, 4)$ before $t = 4$ and reach $(1, 0)$; to pass agent 1 it would have to be inside the pocket at the moment agent 1 passes $(1, 2)$, that is at $t = 2$. But $(0, 2)$ can only be entered from $(1, 2)$, and agent 2, which starts two cells away, could reach $(1, 2)$ at the earliest at $t = 2$, where agent 1 already is. Every other path of agent 2 stays in the corridor and collides head-on with agent 1. So agent 2 has no unblocked path and the order $(1, 2)$ fails. The instance is symmetric under reflection, so the order $(2, 1)$ fails for the same reason. The plan in which agent 1 steps into the pocket at $t = 3$ and agent 2 waits one step before setting off is valid, with costs 6 and 5; the brute-force search in `code/ch08_prioritized.py` confirms that 11 is the optimal sum of costs. $\square$

The pattern behind the failure is worth naming: a higher-priority agent has no reason to make room, and the shortest path it takes for itself destroys the only room there was. The same pattern with a bypass produces suboptimality instead of failure.

Proposition 8.7 (Suboptimality). *Even the best priority order can return a plan that is more expensive than the optimal plan.*

Proof. Figure 8.4(b) adds a bypass through row 0 to the corridor of (a): the corridor is now row 2, the pocket is $(3, 3)$, and the two agents exchange the corridor ends $(2, 0)$ and $(2, 6)$. Whichever agent goes first takes the straight path of cost 6 and parks on the other agent's start. The second agent cannot pass it in the corridor, by the argument of Proposition 8.6, so it takes the bypass: two steps up, six along row 0 and two down, cost 10 (the same by the mirror image), for a sum of costs of 16 in both orders. The plan of Figure 8.4(c) instead sends agent 1 into the pocket at $t = 4$ and lets agent 2 wait at $(2, 4)$ during $t = 2, 3$; agent 2 then

passes and arrives at $t = 7$ while agent 1 leaves the pocket at $t = 5$ and arrives at $t = 8$, sum of costs 15. The code verifies the plan and confirms by brute force that 15 is optimal. $\square$

Both counterexamples have two agents and fewer than thirty cells. They are not exotic: narrow aisles with a single passing bay are the everyday geometry of a warehouse, and a drone that is asked to park in the middle of a corridor produces exactly the goal blocking of [Example 8.4](#). The lesson is not to avoid prioritized planning but to know the two conditions under which it is safe, which we turn to next.

8.6.2 Complexity

Proposition 8.8 (Complexity). *Let b be the maximum degree of G plus one (five on a 4-connected grid) and $T_{\max}$ the horizon. With hash sets for R_V and R_E , [Algorithm 8.2](#) runs in time $\mathcal{O}(b|V|T_{\max} \log(|V|T_{\max}))$ and space $\mathcal{O}(|V|T_{\max})$, and [Algorithm 8.1](#) runs in k times that. The reservation table holds $\mathcal{O}(\sum_i T_i)$ entries.*

Proof. There are at most $|V|(T_{\max} + 1)$ space-time states, each is expanded at most once (CLOSED), each expansion generates at most b successors with $\mathcal{O}(1)$ blocking tests, and every insertion into a binary-heap OPEN costs a logarithm of the heap size. Reserving a path of arrival time T_i adds $T_i + 1$ vertex entries, at most T_i move entries and one parked entry. $\square$

In practice the searches are far from this bound because the heuristic keeps them close to a corridor of width $\mathcal{O}(1)$ around the shortest path, and because $T_{\max}$ is replaced by the time at which the table stops changing ([Section 8.8](#)). Forty agents on a 100×100 grid with 20% obstacles take a few seconds in plain Python with the code of this chapter. The self-test plans one such instance and prints its time; `python3 code/ch08_prioritized.py --bench` plans three of them and prints the three times and their median, so that you can compare your own machine. Expect a few seconds per instance, with a spread of about a factor of three between instances of the same size: the spread comes from the instances, not from the machine. A compiled implementation is far faster, and the joint search over all agents would not finish in the lifetime of the reader.

8.6.3 The positive result: well-formed instances

Theorem 8.9 (Completeness on well-formed instances [\[27\]](#)). *Let the instance be well-formed ([Definition 8.3](#)), and for each agent i let L_i be the length of a path from s_i to γ_i that avoids all other endpoints. Then [Algorithm 8.1](#) with the revised rule and a horizon $T_{\max} \geq \sum_i L_i$ returns a valid plan for every priority order σ . The arrival time of the j -th planned agent is at most $\sum_{m \leq j} L_{\sigma(m)}$.*

Proof. We show by induction on j that agent $i = \sigma(j)$ has an unblocked path of arrival time at most $T_j^* + L_i$, where T_j^* is the largest arrival time among agents $\sigma(1), \dots, \sigma(j-1)$ (with $T_1^* = 0$). Since [Algorithm 8.2](#) is complete on the finite space-time graph up to $T_{\max}$ ([Proposition 8.5](#)), it then returns some unblocked path, and the induction can continue. The candidate path is the *wait-then-go* path: wait at s_i for T_j^* steps, then follow the endpoint-free path P_i of length L_i that well-formedness guarantees.

Waiting is unblocked. Every earlier agent $\sigma(m)$, $m < j$, planned with s_i in its static obstacle set B (revised rule, line 5), so no entry (s_i, t) was ever added to R_V , and s_i is not parked on because goals are distinct from starts. Hence (s_i, t) is free for all t , and staying there creates no swap either.

Going is unblocked. After time T_j^* every earlier agent is parked on its own goal, so R contains no timed vertex entry with $t > T_j^*$ and no move entry with $t \geq T_j^*$; the wait-then-go path is still at s_i at time T_j^* , a cell that no earlier agent ever reserved. The only blocked cells at times $t > T_j^*$ are the goals $\gamma_{\sigma(m)}$, $m < j$, which P_i avoids by construction. P_i also avoids the

starts of the lower-priority agents, which are the static obstacles B of agent i . So each step of P_i , taken at a time $\geq T_j^*$, is an unblocked successor.

The goal test passes. The last time at which some earlier agent visits γ_i is at most T_j^* , because after T_j^* nobody moves and nobody is parked on γ_i . Thus $t_\gamma \leq T_j^* < T_j^* + L_i$, and the wait-then-go path is accepted at its arrival. (If $s_i = \gamma_i$ then $L_i = 0$ and the chain reads $t_\gamma = -1 < 0 = T_j^* + L_i$ only because no earlier agent may enter s_i at all; assume $s_i \neq \gamma_i$ otherwise.) Its arrival time is at most $T_j^* + L_i \leq \sum_{m \leq j} L_{\sigma(m)} \leq T_{\max}$, which completes the induction; Algorithm 8.2 may of course return a shorter path. Validity of the plan follows from Proposition 8.5. $\square$

Corollary 8.10. *On a well-formed instance the makespan of the plan of the revised algorithm is at most $\sum_i L_i$ for every order, and the sum of costs is at most $k \sum_i L_i$.*

Remark 8.11. The theorem says nothing about quality, and the bound of the corollary is loose: the wait-then-go path is only a witness that *some* unblocked path exists, and in practice agents rarely wait at all. It also relies on the revised rule in exactly one place, the claim that waiting at s_i is unblocked: without it a higher-priority agent may reserve s_i before agent i has moved, and whether agent i can get out of the way in time then depends on the map. Čáp et al. [27] introduced the rule for this reason. Finally, the two counterexamples above are not well-formed: in both, every path of one agent passes through the start of the other.

8.7 Variants and extensions

8.7.1 Hierarchical Cooperative A*: a true-distance heuristic

The Manhattan distance ignores obstacles. In an open map that costs little, but in a cluttered one it pulls the search into dead ends and makes it explore the whole neighborhood of every wall before it turns back. In space-time this is worse than on a plain grid, because every cell that is explored at time t is explored again at $t + 1, t + 2, \dots$ while the agent waits in front of an obstacle. Silver’s **Hierarchical Cooperative A*** (HCA*) replaces h by the exact static distance to the goal, $h^*(v) = \text{dist}_G(v, \gamma)$, computed on the map without agents. This is the backward search of Chapter 3: one breadth-first search (Dijkstra with unit costs) from the goal gives h^* for every cell at once, and the table is reused at every time step of the space-time search. Silver computes it lazily with a *reverse resumable A** that is paused as soon as the current query is answered and resumed when a new cell is needed; for the maps of this book the full breadth-first search is simpler and costs one pass over the grid per agent.

The true distance is consistent and dominates the Manhattan distance, so for a single agent against a fixed table, writing C^* for the arrival time of the optimal unblocked path, every state with $f < C^*$ that the search with h^* expands is also expanded with the Manhattan distance (Corollary 4.11); the states with $f = C^*$ are left to the tie-breaking rule. Across a whole prioritized run the guarantee is weaker still, because the two heuristics send the early agents along different shortest paths and hence give the later agents different sub-problems. Empirically the gain is large: on twenty random 20×20 grids with 30% obstacles and eight agents each, the self-test of `code/ch08_prioritized.py` counts 32 509 expansions with the Manhattan heuristic and 11 034 with the true distance, a factor of three, on the same instances and orders. Under both heuristics every path is a shortest unblocked one for the agent that plans it; the paths differ only where several shortest paths exist. Note what the true distance does *not* know: the other agents. A corridor that is closed by a parked drone still looks open to h^* , and the search only discovers the closure by running into the reservation, as agent 1 does in the order (3, 1, 2) of Example 8.4. Exercise 8.6 explores a heuristic that accounts for parked agents.

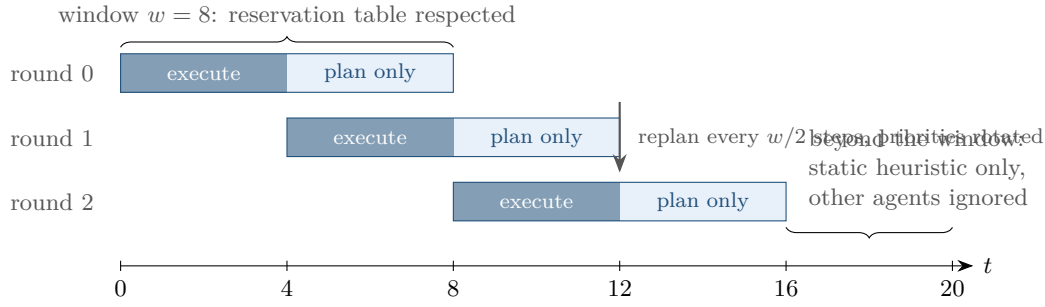


Figure 8.5. WHCA* with window $w = 8$ and execution length $\delta = 4$. In every round each agent plans against the reservation table for the next w steps only (bar), executes the first δ of them (dark), and then all agents replan from where they are. Beyond the window the other agents are invisible and only the static heuristic guides the search.

8.7.2 Windowed Cooperative A*: lifelong and online use

A warehouse never stops: when a drone arrives it receives a new goal, and a plan that reserves cells until the end of time is pointless. Silver’s **Windowed Hierarchical Cooperative A*** (WHCA*) keeps the cooperative search only for the next w time steps, the **window**. Within the window an agent respects the reservation table as before; beyond it, the agents are ignored and the search follows the true-distance heuristic alone, which means it completes the path as if no other agent existed (the static obstacles are still respected, because h^* is the distance on the real map). Every agent is planned in this way, its first δ steps ($\delta = w/2$ is typical) are executed, and then all agents replan from their new positions with a fresh table. An agent whose plan is shorter than δ , because it arrives inside the window, repeats its last cell for the remaining steps – the listing writes `moves += [path[-1]] * (step - len(moves))`, the stay-at-target convention once more. Algorithm 8.3 gives the loop and Figure 8.5 the picture.

Algorithm 8.3. Windowed Cooperative A* (WHCA*)

Input: graph G , starts s_i , goals γ_i , window w , execution length $\delta \leq w$

Output: the executed paths $(\pi_1, \dots, \pi_k)$

```

1 Function WindowedCooperativeAStar( $G, s, \gamma, w, \delta$ ):
2    $p_i \leftarrow s_i$  and  $\pi_i \leftarrow (s_i)$  for every agent  $i$ ;  $h_i^* \leftarrow$  static distances to  $\gamma_i$ 
3   while some agent is not at its goal do
4      $R \leftarrow$  an empty table in which entries with  $t \geq w$  are ignored
5     foreach agent  $i$  in the current order do
6        $\rho_i \leftarrow$  CooperativeAStar( $G, p_i, \gamma_i, R, \emptyset, \infty, h_i^*$ )
7       if  $\rho_i = \text{failure}$  then  $\rho_i \leftarrow (p_i)$  // stay put; no guarantee
8
9       Reserve( $R, \rho_i$ )
10    execute the first  $\delta$  steps of every  $\rho_i$ , append them to  $\pi_i$ , set  $p_i$  to the new cell
11    rotate the order so that a different agent goes first
12  return  $(\pi_1, \dots, \pi_k)$ 
```

Three details of Algorithm 8.3 matter. First, the table of line 4 is relative to the current round: time 0 is now, and entries beyond $t = w$ are invisible. A cell that a higher-priority agent has parked on inside the window is the exception: the goal-stay check of Algorithm 8.2 then reports $t_\gamma = \infty$, the search fails at once, and the agent takes the stay-put branch of line 6: it stays where it is with no guarantee at all, and an agent planned earlier in the same round may

be walking straight into it. This is how the windowed plans of [Exercise 8.7](#) come to contain a collision. The other way is the window itself: beyond $t = w$ the table is empty, so a plan that looks safe now can be invalidated by the next round. Second, line 11 rotates the priority order between rounds so that no agent is always the last to be planned; without it the lowest-priority agent can be pushed around indefinitely. Third, the loop has no termination guarantee. Two agents that approach each other in a corridor with w smaller than the distance to the nearest pocket only see each other when it is too late, back off, and try again; [Exercise 8.7](#) asks you to build this case. In return, WHCA* handles goals that change, agents that appear and disappear, and execution errors, because nothing is planned further ahead than w steps. On [Example 8.4](#) with $w = 6$ and $\delta = 3$ it produces, after rotation, the same costs 6, 6, 4 as the best fixed order. The lifelong MAPF setting in which every arriving agent receives a new task is studied by Ma et al. [112]; windowed prioritized planning is its most common baseline.

8.7.3 Choosing the priority order

Nothing in [Algorithm 8.1](#) says where σ comes from, and [Table 8.3](#) showed how much depends on it. Three families of rules are in use.

- **Longest path first** plans the agents in decreasing order of the length of their independent shortest paths. The intuition: an agent with a long way to go crosses many other paths and is the one most likely to be trapped if it is planned late; a short-haul agent can usually wait a few steps.
- **Most constrained first** computes the independent shortest paths, counts for every agent how many other independent paths it conflicts with, and plans the agents with the most conflicts first, ties broken by path length. This is the discrete cousin of the roadmap heuristic of van den Berg and Overmars [16].
- **Random restarts** run [Algorithm 8.1](#) with several random orders and keep the cheapest plan that succeeds; Bennewitz, Burgard and Thrun [12] go further and hill-climb in the space of orders by swapping two agents at a time. Since each run is cheap, this is often the most effective use of a time budget, and it is embarrassingly parallel.

[Figure 8.6](#) compares the three rules, and a single random order, on random 20×20 grids with 20% obstacles and $k = 4, 8, \dots, 32$ agents, thirty instances per k , all with the true-distance heuristic of HCA* (script `code/figures/gen_ch08_orders.py`). The left plot shows how often each rule returns a plan at all. A single random order fails in 13% of the instances with 32 agents and in 23% of those with 28; longest path first fails in one of the 240 instances, and five random restarts fail in one. The right plot shows the sum of costs divided by the sum of the independent shortest paths, the lower bound of [Chapter 7](#), over the instances that all four rules solve. Here the picture reverses: at $k = 32$ longest path first is 20% above the bound, a single random order 16%, and the best of five random orders 12%. The rule that protects the long-haul agents makes everybody else detour around them. Most constrained first is a disappointment here: it fails in 15 of the 240 instances, almost exactly as often as a single random order (19), and at $k = 32$ its plans are the most expensive of all four rules, 21% above the bound. Counting conflicts between independent paths says little about who will be trapped. Which rule is right therefore depends on what you need: if a plan must be found in one attempt, use longest path first; if you have time for a few attempts, use random restarts, or restarts seeded with the longest-path-first order.

8.7.4 Priority-based search

Random restarts explore the $k!$ orders blindly. **Priority-based search** (PBS) of Ma et al. [111] explores them systematically, and it does so with *partial* orders. A partial priority

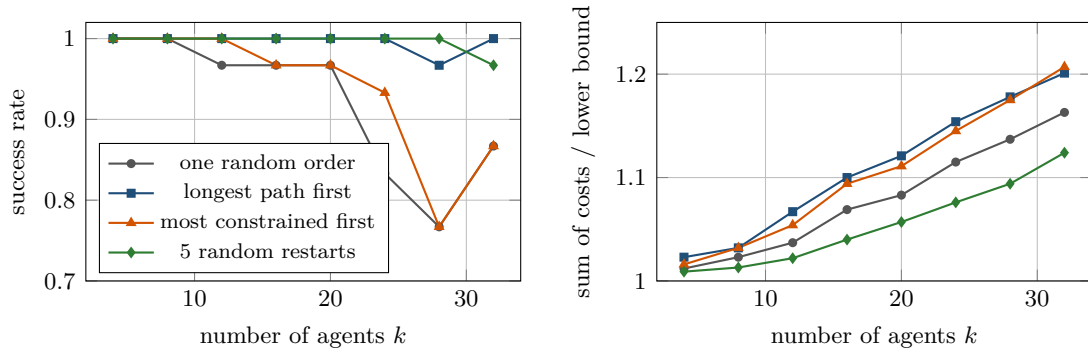


Figure 8.6. Priority ordering rules on random 20×20 grids with 20% obstacles (thirty instances per k , HCA*). Left: fraction of instances solved. Right: sum of costs relative to the sum of the independent shortest paths, over the instances solved by all four rules. Longest path first almost never fails; it and most constrained first produce the most expensive plans (within one point of each other for $k \geq 24$); five random restarts give the cheapest plans and fail once in 240 instances.

order is a set of pairs $i \prec j$, read “ i has priority over j ”, that contains no cycle. PBS runs a two-level search, in the same spirit as the conflict-based search of Chapter 9, but with priorities instead of space-time constraints as the branching decision:

- The **high level** builds a binary tree. The root carries the empty order and a plan in which every agent has its independent shortest path. At every node, the plan is checked for conflicts. If there is none, the plan is a solution and the search stops. Otherwise one conflict, between agents i and j , is picked, and two children are created: one adds $i \prec j$ to the order, the other adds $j \prec i$. Only pairs without an existing relation are ever split, so the orders stay acyclic.
- The **low level** repairs the plan of a child. The agent that just lost priority, and every agent that ranks below it in the partial order, are replanned in a topological order of the partial order, each with Algorithm 8.2 against the reservation table of all agents that rank above it. Agents unrelated to the new pair keep their paths. If some agent finds no path, the child is discarded.
- The tree is searched **depth-first**, trying the child with the smaller sum of costs first and backtracking when a subtree contains no solution.

The depth of the tree is at most $k(k-1)/2$, one relation per pair, and a leaf is exactly a prioritized plan for some total order that extends the partial one. In this sense PBS is prioritized planning that chooses its order lazily, one pair at a time, and only where a conflict forces a choice; a fixed order σ corresponds to one root-to-leaf path of the tree. Compared with CBS, which branches on one constraint $\langle a, v, t \rangle$ at a time and can need a deep tree for two agents that squeeze through a corridor, PBS resolves all present and future conflicts of a pair in one decision. The price is that it inherits the weaknesses of this chapter: PBS is neither complete nor optimal in general, because a solution may require agent i to yield to agent j in one place and j to yield to i in another, which no priority relation can express. In the experiments of Ma et al. it solves far more instances than prioritized planning with a fixed order, in a fraction of the time of CBS, with costs close to the optimum on the instances that both solve. When you need guarantees, Chapters 9 and 10 provide them; when you need a plan for two hundred agents in a second, PBS is the state of the art among priority-based methods.

8.7.5 Beyond grids

The idea of planning in priority order does not depend on grids. Erdmann and Lozano-Pérez [38] formulated it for polygonal robots in configuration space-time; van den Berg and Overmars [16] plan on roadmaps with time as an extra dimension; Čáp et al. [27] prove the well-formedness result for trajectories of robots of arbitrary shape. On grids, safe interval path planning [131] replaces the time axis of Algorithm 8.2 by the maximal intervals during which a cell is free according to the table, which shrinks the state space drastically when reservations are sparse, and prioritized planning with kinematic constraints [66] post-processes the plan into velocity profiles that real drones can follow. In every one of these settings the reservation table is the same object as in Definition 8.1, only the notion of “blocked” changes.

8.8 Implementation notes

The file `code/ch08_prioritized.py` implements everything in this chapter on 4-connected grids: the reservation table, Cooperative A* with either heuristic, the priority loop with the revised rule, a validator, the well-formedness test, the three ordering rules, the windowed variant and a brute-force optimal solver for tiny instances. Its self-test reproduces every number quoted above. The notes below cover the decisions that are easy to get wrong.

The table is three hash sets. Listing 8.1 stores R_V as a set of (cell, t) tuples, R_E as a set of (u, v, t) tuples recording each move as it was made, and the parked goals as a dictionary from cell to arrival time. The three blocking tests of Algorithm 8.2 are then single lookups. The swap test for a move $u \rightarrow v$ at time t asks whether (v, u, t) is in the set, that is, whether somebody moves the other way along the same edge in the same step. The dictionary of parked cells keeps the *earliest* arrival if a cell is parked on twice, which cannot happen with distinct goals but costs nothing to guard against. The goal-stay time is found by a scan over R_V ; for hundreds of agents this is fine, for thousands keep one sorted list of times per cell.

Collapsing time instead of a horizon. Algorithm 8.2 needs a horizon $T_{\max}$ to terminate when no path exists. Chapter 4 recommends $T_{\max} = H + 1 + D$, where H is the last time of any timed entry and D the largest static distance to the goal. Listing 8.2 uses a trick that needs no horizon at all. After time $t_{\text{static}} = H + 1$ the table never changes again: the only remaining entries are parked cells, and they stay parked. Two states (v, t) and (v, t') with $t < t'$, both at or beyond t_{static} , therefore have exactly the same future, shifted in time, and the earlier one dominates. The closed set is keyed by $(v, \min(t, t_{\text{static}}))$, with t_{static} returned by the method `static_time()` of the table, which is what Chapter 4 calls `horizon()`, so every cell is expanded at most $t_{\text{static}} + 1$ times, the search is finite, and it never reports a false failure because a horizon was chosen too small. With a window (Section 8.7.2) the same code uses $t_{\text{static}} = w$.

Order of successors and ties. Wait first, then the four moves in a fixed order; ties in f are broken toward the larger t and then by insertion order. This makes traces reproducible and, together with the true-distance heuristic, makes the search follow one shortest path instead of sweeping the whole $f = C^*$ band.

Validate everything. A plan that leaves the planner is checked by `validate`, an independent function that walks all pairs of paths through time under the stay-at-target convention and reports vertex and swapping conflicts, illegal moves and wrong endpoints. The self-test runs it

on every plan of every order of the examples and on eighty plans of forty random instances; a planner without such a check will eventually ship a swap.

Listing 8.1. The reservation table (from code/ch08_prioritized.py): vertex entries, move entries and parked goals, with the three blocking tests of Algorithm 8.2.

```

1  class ReservationTable:
2      """Space-time entries of the agents planned so far."""
3
4      vertices : set of (cell, t)          -- cell occupied at time t
5      edges    : set of (u, v, t)          -- move u -> v during step t -> t+1
6      parked   : dict cell -> time T       -- cell occupied for all t >= T
7      window   : if not None, entries at t >= window are ignored (WHCA*)
8      """
9
10     def __init__(self, window: Optional[int] = None) -> None:
11         self.vertices: Set[Tuple[Cell, int]] = set()
12         self.edges: Set[Tuple[Cell, Cell, int]] = set()
13         self.parked: Dict[Cell, int] = {}
14         self.window = window
15         self.last_timed = -1              # largest t of any timed entry
16
17     def reserve(self, path: Path) -> None:
18         """Add a path: its cells, its moves and its parked goal."""
19         for t, cell in enumerate(path):
20             self.vertices.add((cell, t))
21             if t + 1 < len(path) and path[t + 1] != cell:
22                 self.edges.add((cell, path[t + 1], t))
23             self.last_timed = max(self.last_timed, len(path) - 1)
24             goal, arrival = path[-1], len(path) - 1
25             self.parked[goal] = min(arrival, self.parked.get(goal, INF))
26
27     def in_window(self, t: int) -> bool:
28         return self.window is None or t < self.window
29
30     def is_blocked(self, cell: Cell, t: int) -> bool:
31         """May an agent NOT be at `cell` at time t?"""
32         if not self.in_window(t):
33             return False
34         return (cell, t) in self.vertices or t >= self.parked.get(cell, INF)
35
36     def is_swap(self, u: Cell, v: Cell, t: int) -> bool:
37         """Would moving u -> v during step t swap with a reserved move?"""
38         return self.in_window(t) and (v, u, t) in self.edges
39
40     def last_reserved(self, cell: Cell) -> float:
41         """Last time at which `cell` is taken: -1 never, inf if parked on."""
42         if cell in self.parked and self.in_window(self.parked[cell]):
43             return INF
44         times = [t for (c, t) in self.vertices if c == cell and
45 self.in_window(t)]
46         return max(times) if times else -1

```

Listing 8.2. The search loop of Cooperative A* (from code/ch08_prioritized.py). The closed set collapses the time index beyond `t_static`, so no horizon is needed.

```

1     t_goal = table.last_reserved(goal)      # arrival must be later than this
2     if t_goal == INF or table.is_blocked(start, 0):

```

```

3     return None
4     t_static = table.static_time()
5     counter = itertools.count()
6     open_heap = [(h(start), 0, next(counter), start, 0)]
7     parent: Dict[Tuple[Cell, int], Tuple[Cell, int]] = {}
8     closed: Set[Tuple[Cell, int]] = set()
9     expansions = 0
10    while open_heap:
11        f, neg_g, _, v, t = heapq.heappop(open_heap)
12        key = (v, min(t, t_static))
13        if key in closed:
14            continue
15        closed.add(key)
16        expansions += 1
17        if v == goal and t > t_goal:
18            if stats is not None:
19                stats["expansions"] = stats.get("expansions", 0) + expansions
20            if trace is not None:
21                trace.append((v, t, t, h(v), f, ["goal"], []))
22            path = [(v, t)]
23            while path[-1] in parent:
24                path.append(parent[path[-1]])
25            return [cell for cell, _ in reversed(path)]
26        generated, blocked = [], []
27        for v2 in [v] + inst.neighbors(v):          # wait first, then moves
28            if v2 in avoid:
29                continue
30            if table.is_blocked(v2, t + 1):
31                blocked.append((v2, "V"))
32                continue
33            if v2 != v and table.is_swap(v, v2, t):
34                blocked.append((v2, "E"))
35                continue
36            if (v2, min(t + 1, t_static)) in closed:
37                continue
38            parent.setdefault((v2, t + 1), (v, t))
39            generated.append(v2)
40            heapq.heappush(open_heap, (t + 1 + h(v2), -(t + 1), next(counter),
v2, t + 1))
41            if trace is not None:
42                trace.append((v, t, t, h(v), f, generated, blocked))
43        if stats is not None:
44            stats["expansions"] = stats.get("expansions", 0) + expansions
45    return None

```

Listing 8.1 is the whole reservation table; Listing 8.2 is the body of `space_time_astar` after the heuristic has been chosen. The optional `avoid` set implements the revised rule, the `trace` list produced Table 8.2, and `stats` counts expansions for the comparison of Section 8.7.1.

Forgetting that agents stay at their goals

The most common bug in a first implementation reserves the goal cell only at the arrival time. A later agent then plans straight through a parked drone, the validator reports a vertex conflict at every time step after the arrival, and the author blames the validator. Reserve the goal for all $t \geq T$ (the parked dictionary), and make the goal test of the later agent wait until its own goal is free for ever ($t > t_goal$). The second half is as

important as the first: without it an agent may “arrive” at a cell that a higher-priority agent will cross two steps later.

Checking the edge in the wrong direction

A swap between u and v at time t is caught only if the table stores the move $u \rightarrow v$ of the earlier agent *and* the later agent’s move $v \rightarrow u$ is tested against it in the reversed orientation. Storing (u, v, t) and testing (u, v, t) again catches only the impossible case of two agents making the same move, which is already a vertex conflict at $t + 1$. Test the reversed tuple, and write a unit test with two agents that exchange the ends of a two-cell corridor: without edge reservations the plan is accepted, with them the second agent fails, which is the correct answer.

A horizon that is too short, or no horizon at all

Without a horizon a blocked agent waits at its start for ever and the search never returns. With a horizon smaller than the time at which the higher-priority agents have cleared the way, the search reports a failure for an agent that only needed to wait longer, and the user concludes that prioritized planning is even less complete than it is. Use $T_{\max} = H + 1 + D$ or collapse the time index beyond $H + 1$ as in [Listing 8.2](#).

8.9 Where this fits in the drone system

In the drone system

Prioritized planning is the fast global planner of a swarm. Forty drones on a 100×100 grid are forty space-time searches instead of one joint search, and with the true-distance heuristic and a good order they finish in a few seconds in plain Python and much faster in a compiled implementation, where the optimal solvers of [Chapters 9](#) and [10](#) may need minutes. In the hybrid architecture of [Chapter 24](#) the global layer is CBS or ECBS because they come with guarantees: CBS is complete and optimal, ECBS is complete and bounded-suboptimal, whereas the plan of this chapter can be arbitrarily worse than optimal and may not exist for the chosen order although the instance is solvable. Prioritized planning is what you fall back on when those solvers time out, what you use for swarms of hundreds, and what you run when a plan must be repaired in place: a drone that has been pushed off its route by the local safety layer can be replanned alone against the reservation table of all the others, which is exactly one call of [Algorithm 8.2](#). Its failure modes are the ones of this chapter: corridors with a single passing bay, goals in the middle of somebody else’s route, and orders that trap a long-haul drone; the well-formedness test of [Definition 8.3](#) tells you in advance whether the layout is safe, and the ordering rules of [Section 8.7.3](#) with a few restarts remove most of the remaining risk. This chapter is the second half of Week 3 of the study plan ([Appendix A](#)): implement prioritized planning for two to five agents ([Exercise 8.8](#)), on the conflict cases you built with the detector of [Chapter 7](#).

8.10 Summary

Chapter summary

- Prioritized planning plans the agents one after another in a priority order; each agent runs space-time A* against a reservation table that holds the (cell, time) and (move, time) entries of the earlier agents and their parked goals. This is Cooperative A*; k agents cost k single-agent searches.
- The goal test must wait until the agent's goal is free for all later times, and the reverse of every reserved move must be forbidden; otherwise parked agents are run over and swaps slip through.
- The plan is always valid and each agent's path is optimal given the earlier ones, but the method is greedy: the order changes the cost, there are solvable instances on which every order fails, and the best order can be worse than the optimum. Two small corridors with one pocket show both defects.
- On well-formed instances, where every agent can reach its goal without touching another agent's start or goal, the revised algorithm (lower-priority starts are static obstacles) succeeds for every order; the proof is the wait-then-go path.
- HCA* replaces the Manhattan heuristic by exact static distances from a backward search, cutting expansions by a factor of about three in clutter; WHCA* plans only w steps ahead and replans, which makes it suitable for lifelong and online use at the price of all guarantees.
- Longest path first is the safest single order, random restarts give the cheapest plans, and priority-based search explores partial orders in a depth-first tree, resolving one conflicting pair per branch.

Further reading. The idea of planning moving objects one at a time in configuration space-time is due to Erdmann and Lozano-Pérez [38]; LaValle's textbook [99] presents it as decoupled planning next to the centralized alternative. Silver [153] introduced Cooperative A*, HCA* and WHCA* for grids; van den Berg and Overmars [16] and Bennewitz et al. [12] study the choice of the order; Čáp et al. [27] prove the completeness result for well-formed infrastructures; Ma et al. [111] introduce priority-based search. The MAPF conventions used here follow Stern et al. [161]; safe interval path planning [131] and planning with kinematic constraints [66] extend the low-level search.

8.11 Exercises

Exercise 8.1 (★ Reservation table by hand). Take [Example 8.4](#) in the order (2, 3, 1). (a) Write down all entries of R_V , R_E and the parked cells after agents 2 and 3 have been planned. (b) What is the goal-stay time t_γ of agent 1? (c) Without running the search, explain why agent 1 has no unblocked path, and how the implementation of [Listing 8.2](#) terminates with a failure although it uses no horizon.

Exercise 8.2 (★ Conventions). (a) Following conflicts are allowed in this book. Which entries would you add to the reservation table to enforce a one-cell safety gap, so that no agent may enter a cell at $t + 1$ that another agent leaves at t , and does the swap test remain necessary? (b) Under the disappear-at-target convention of [Chapter 7](#) there are no parked entries. Determine, for every order of [Table 8.3](#), whether the plan then succeeds and what it costs.

Exercise 8.3 (★★ Trace the expensive order). In the order (3, 1, 2) of [Example 8.4](#), list the entries of the reservation table that lie in the top corridor and in (1, 3) after agents 3 and 1 have been planned, then run [Algorithm 8.2](#) for agent 2 by hand with the Manhattan heuristic in the style of [Table 8.2](#). Verify the path of [Figure 8.2\(b\)](#) and its cost 9, and explain why the wait at (0, 3) at $t = 4$ and the dodge into (1, 3) at $t = 5$ are forced.

Exercise 8.4 (★★ The pocket corridor). (a) Turn the proof of [Proposition 8.6](#) into a table: for the order (1, 2), list the cells agent 2 can occupy at $t = 1, 2, 3, 4$ given the straight path of agent 1, and show that the set is empty at $t = 4$. (b) Add a second pocket (0, 3) above the fourth corridor cell. Which orders succeed now, with which paths and costs? (c) Is the cheapest of these plans optimal for the new instance? Give a lower-bound argument or check with `optimal_soc`.

Exercise 8.5 (★★ Well-formed instances). (a) Show that well-formedness can be tested in $\mathcal{O}(k(|V| + |E|))$ time. (b) Name the single step of the proof of [Theorem 8.9](#) that uses the revised rule, and give a two-agent instance in which, without the rule, the first agent's shortest path reserves the start cell of the second agent. (c) Give a well-formed instance in which the independent shortest path of some agent passes through another agent's goal, and explain what [Algorithm 8.2](#) returns for that agent in each of the two orders.

Exercise 8.6 (★★ Heuristics that know about parked agents). (a) Compute the true distance $h^*(v)$ to $\gamma_1 = (2, 6)$ for every free cell of [Example 8.4](#) and compare it with the Manhattan distance. (b) Use the code to count the expansions of [Algorithm 8.2](#) for agent 1 in the order (3, 1, 2) with both heuristics, and explain why both first explore the bottom corridor. (c) Modify the heuristic so that cells on which a higher-priority agent is parked count as obstacles. Is this heuristic still admissible? Give an instance in which it overestimates, and state a condition under which it is exact.

Exercise 8.7 (★★★ Oscillation and collision in WHCA*). Two agents exchange the ends of a corridor of nine cells (1, 0), ..., (1, 8) whose only pocket is (0, 1). (a) Simulate [Algorithm 8.3](#) by hand with $w = 2$ and $\delta = 1$ from the moment the agents are adjacent, and show that they back off and advance for ever. (b) Run `whca_star` with $\delta = 1$ and windows $w = 2, \dots, 12$ and check every returned plan with `validate`. For $w \leq 5$ the rounds never end and the function returns `None`. For $w \geq 6$ it returns an executed plan that the validator rejects: the two agents pass through each other. Explain both outcomes. Which of the two agents sees the other one at all in a given round? What happens to the agent that is planned second when every cell in front of it is reserved, and which branch of [Algorithm 8.3](#) does it then take? (c) Why does the rotation of [line 11](#) not help here, and what would? (Hint: with the full horizon the order (2, 1) succeeds, agent 2 arriving at $t = 8$ and agent 1 at $t = 15$: agent 1 hides in the pocket while agent 2 passes.)

Exercise 8.8 (★★★ Coding: prioritized planning for two to five agents). This is the second half of the Week 3 coding exercise of the study plan ([Appendix A](#)) — prioritized planning for two to five agents — whose first half, representing instances and detecting their conflicts, is [Exercise 7.9](#). Starting from your space-time A* of [Chapter 4](#), implement a reservation table with vertex, move and parked entries, Cooperative A* with the goal-stay test, and the priority loop with an optional revised rule; the independent validator is the detector you already wrote in [Exercise 7.9](#). Then create deliberate conflict cases with two to five agents on grids of about 10×10 : a head-on meeting in a corridor with one pocket, a crossing at a junction, a goal that lies on another agent's shortest path, and a parked agent that cuts a corridor. For each case run every priority order, report which orders succeed and their sums of costs, and check every plan with the validator. Finally add the well-formedness test and random restarts, and reproduce the left panel of [Figure 8.6](#) for $k \leq 16$. Optional: implement priority-based search ([Section 8.7.4](#)) on top of your code and compare it with the best of ten random orders.

Conflict-Based Search

Learning objectives

- Explain the two-level idea of conflict-based search: a high-level best-first search over a binary *constraint tree* and a low-level single-agent space-time A^* under constraints, and say why this beats search in the joint state space for loosely coupled teams.
- Define constraints, constraint-tree nodes, conflicts and the split rule exactly, and trace CBS by hand on a small instance, drawing the constraint tree with its costs.
- Prove that CBS is complete on solvable instances and returns a plan with the minimum sum of costs, and explain what its running time depends on.
- Implement CBS from scratch in Python, certify its optimality against a brute-force search on tiny instances, and test it on 2, 4 and 8 agents.
- Recognize the classic implementation mistakes, and know the improvements (cardinal conflicts, bypass, high-level heuristics, symmetry breaking) that turn textbook CBS into a practical solver.

9.1 Why this matters for a drone swarm

Twelve delivery drones share the airspace over a city block. Their routes have been reduced, as in [Chapter 2](#), to a grid of cells at a fixed altitude, with buildings as obstacles and a time step of a few seconds. Each drone alone is easy to route: A^* from [Chapter 4](#) finds its shortest path in milliseconds. The difficulty is that the twelve shortest paths cross. Two drones want the same crossing cell at the same time; a third flies up a street while a fourth flies down it. The multi-agent path finding (MAPF) problem of [Chapter 7](#) asks for twelve paths without any vertex or edge conflict, and, because every second of flight costs battery, for paths whose *sum of costs* is as small as possible.

[Chapter 7](#) showed that the obvious exact method, A^* in the joint state space, does not scale: with five actions per drone (four moves and a wait), a joint state has $5^{12} \approx 2.4 \times 10^8$ successors, and the number of joint states is $|V|^{12}$. [Chapter 8](#) showed the opposite trade-off: prioritized planning plans the drones one after another against a reservation table, is very fast, but can fail on solvable instances and returns no guarantee on the cost of its plans. Conflict-based search (CBS) by Sharon, Stern, Felner and Sturtevant [[149](#), [150](#)] sits between these two extremes. It plans every drone alone, exactly as prioritized planning does, but when two paths conflict it does not pick a winner: it tries *both* ways of resolving the conflict, by forbidding one drone or the other from the contested cell at the contested time. It keeps both alternatives and always continues with the cheapest world it has ever created. The result is a plan that is provably optimal and a search that pays only for the conflicts it actually meets,

not for the number of drones. CBS and its bounded-suboptimal sibling ECBS (Chapter 10) are the standard optimal and near-optimal MAPF solvers today and the basis of most of the research literature since 2012.

In the four-layer hybrid architecture of Chapter 24, CBS is the *global planner*: before take-off it computes the nominal time-indexed route of every drone, and these routes are what the prediction, local safety and replanning layers later protect and repair. Whenever a drone has been pushed off its route by an intruder and replans locally, the architecture re-validates the new route against the routes of the other drones with exactly the conflict check that CBS uses at every node. This chapter is Week 4 of the study plan (Appendix A): its coding milestone is a working optimal CBS solver, written from scratch and tested on two, four and eight agents.

The chapter proceeds as follows. Section 9.2 gives the two-level idea, Section 9.3 the exact definitions of constraints and constraint-tree nodes, and Section 9.4 the pseudocode of both levels with a walkthrough. Section 9.5 runs CBS by hand on a crossing of two flight corridors and draws the whole constraint tree. Section 9.6 proves completeness and optimality and discusses what the running time depends on. Section 9.7 presents the improvements that make CBS practical and an experiment on random grids, Section 9.8 the Python implementation and its pitfalls, and Section 9.9 the place of CBS in the drone system.

9.2 The idea in plain words

Imagine the drones planned in separate rooms, each unaware of the others. Every drone comes back with its own shortest route. Now lay the routes on one map and look for the first moment at which two drones, say a_1 and a_2 , occupy the same cell v at the same time t . Somebody has to give way, and we do not know who should. CBS refuses to guess. It creates two *worlds*: in the first world a single new rule says “ a_1 may not be at v at time t ”, in the second world the rule says the same about a_2 . In each world only the drone that received the rule goes back to its room and replans; everybody else keeps their route. A rule can only make a route longer or leave it unchanged, so each world is at least as expensive as its parent.

CBS keeps all worlds it has created in a priority queue, ordered by the sum of the route costs, and always examines the cheapest one. If the cheapest world has no conflict, its routes are the answer, and they are optimal: every other world costs at least as much, and adding rules never makes a world cheaper. If the cheapest world still has a conflict, it is split again, the two new worlds inherit all rules of their parent plus one, and the queue is consulted again. The set of worlds forms a binary tree, the **constraint tree** (CT), whose root is the world without rules. Figure 9.1 shows the first split.

Key idea

CBS separates *who yields* from *how to yield*. The high level decides who yields by searching a binary tree of constraint sets best-first on the sum of costs; the low level decides how to yield by running single-agent space-time A^* under the constraints of one agent. Every split keeps at least one optimal solution, and costs never decrease down the tree, so the first conflict-free node popped is optimal.

Two features of this idea deserve emphasis, because they explain when CBS works well. First, the tree grows with the number of *conflicts*, not with the number of agents: ten drones that never meet cost ten single-agent searches and one node. Second, the high level never looks inside a route. It only asks the low level for the cheapest route under a list of forbidden (cell, time) pairs, which is precisely the space-time A^* of Chapter 4. Any single-agent planner that returns optimal paths under such constraints can serve as the low level, which is why

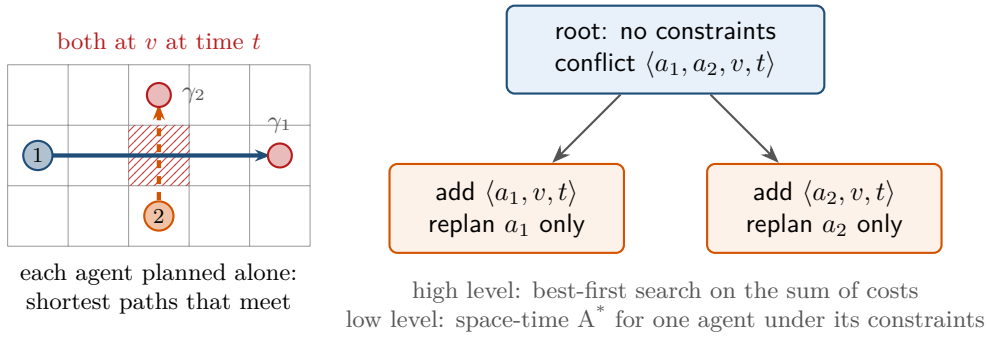


Figure 9.1. The two-level idea. Left: planned alone, two shortest paths meet in cell v at time t . Right: the high level splits the conflict into two children; each child adds one constraint and replans only the constrained agent with the low level. The high level then continues with the cheapest node of the whole queue, which may be either child or a node generated elsewhere in the tree.

CBS generalizes easily to other graphs, action sets and cost functions.

9.3 Problem statement and notation

We use the MAPF model of Chapter 7 (after Stern et al. [161]). A graph $G = (V, E)$, in this chapter a 4-connected grid, is shared by k agents $a_1, \dots, a_k$ with distinct starts s_i and distinct goals γ_i . Time is discrete. In one time step an agent moves along an edge or waits in its cell. A **path** π_i of agent a_i is a sequence of cells $\pi_i[0] = s_i, \pi_i[1], \dots, \pi_i[T_i] = \gamma_i$ with consecutive cells equal or adjacent; the agent stays at γ_i for all $t \geq T_i$, so $\pi_i[t]$ is defined for every $t \geq 0$. Its **cost** $\text{cost}(\pi_i) = T_i$ is the time of the final arrival at the goal, and the **sum of costs** of a plan $\Pi = (\pi_1, \dots, \pi_k)$ is $\text{SoC}(\Pi) = \sum_{i=1}^k \text{cost}(\pi_i)$. Two agents have a **vertex conflict** $\langle a_i, a_j, v, t \rangle$ if $\pi_i[t] = \pi_j[t] = v$, and an **edge conflict** $\langle a_i, a_j, u, v, t \rangle$ if $\pi_i[t] = u, \pi_i[t+1] = v, \pi_j[t] = v$ and $\pi_j[t+1] = u$. This is the swapping conflict of Definition 7.4; the CBS literature calls it an edge conflict because the constraint that forbids it is an edge constraint. The same-direction edge conflict of Chapter 7 needs no separate test here: by Proposition 7.5(a) it implies a vertex conflict. Note that conflicts are checked for all t , including times after one of the agents has arrived: a parked drone still occupies its goal cell. A **solution** is a plan without conflicts; an **optimal** solution minimizes SoC, and C^* denotes its cost.

Definition 9.1 (Constraints). A **vertex constraint** $\langle a_i, v, t \rangle$ forbids agent a_i to occupy cell v at time t . An **edge constraint** $\langle a_i, u, v, t \rangle$ forbids a_i to move from u at time t to v at time $t+1$. A path π_i **respects** a set $\mathcal{C}_i$ of constraints on a_i if it violates none of them, where a vertex constraint $\langle a_i, v, t \rangle$ with $v = \gamma_i$ and $t \geq \text{cost}(\pi_i)$ counts as violated, because the agent is parked at γ_i at that time.

Definition 9.2 (Constraint-tree node). A node N of the **constraint tree** is a triple $N = (N.\mathcal{C}, N.\pi, N.\text{cost})$ where $N.\mathcal{C} = (N.\mathcal{C}_1, \dots, N.\mathcal{C}_k)$ assigns a set of constraints to every agent, $N.\pi = (N.\pi_1, \dots, N.\pi_k)$ assigns to every agent a path of *minimum cost among the paths that respect $N.\mathcal{C}_i$* , and $N.\text{cost} = \sum_i \text{cost}(N.\pi_i)$. The **root** R has $R.\mathcal{C}_i = \emptyset$ for every agent, so $R.\pi_i$ is an unconstrained shortest path. A node is a **goal node** if $N.\pi$ has no conflict. Every non-goal node has *at most* two **children**, one per constraint of the split of one of its conflicts; a child whose low-level problem is infeasible does not exist (Algorithm 9.1 discards it at line 13). The split is defined as follows:

$$\begin{aligned} \text{Split}(\langle a_i, a_j, v, t \rangle) &= \{ \langle a_i, v, t \rangle, \langle a_j, v, t \rangle \}, \\ \text{Split}(\langle a_i, a_j, u, v, t \rangle) &= \{ \langle a_i, u, v, t \rangle, \langle a_j, v, u, t \rangle \}. \end{aligned} \tag{9.1}$$

The child that receives constraint κ on agent a_i has $N'.\mathcal{C}_i = N.\mathcal{C}_i \cup \{\kappa\}$, the same constraint sets as N for all other agents, and the same paths as N except $N'.\pi_i$, which is recomputed.

The wording “minimum cost among the paths that respect $N.\mathcal{C}_i$ ” is the heart of the definition. It makes $N.\text{cost}$ a quantity that the low level can compute exactly, and it is what turns $N.\text{cost}$ into a lower bound on every solution that obeys N ’s constraints (Section 9.6). The problem the low level solves is therefore:

Definition 9.3 (The low-level problem). Given an agent a_i and a finite set $\mathcal{C}_i$ of constraints on it, find a path from s_i to γ_i of minimum cost that respects $\mathcal{C}_i$, or report that none exists.

This is the constrained space-time A^* of Section 4.8 with the constraint set in the role of the reservation table (Chapter 8); Algorithm 9.2 states the differences line by line. Two of its ingredients are worth naming before that. The heuristic is the static shortest-path distance $d_i(v)$ to γ_i , computed once per agent by a backward breadth-first search (or Dijkstra, Chapter 3); it is consistent, so no state is expanded twice. And the goal test is not “ $v = \gamma_i$ ” but “ $v = \gamma_i$ and $t > t_\gamma$ ”. This is the **goal-stay test** of Definition 4.17, also called the goal-occupied-later test; here

$$t_\gamma = \max \{t : \langle a_i, \gamma_i, t \rangle \in \mathcal{C}_i\} \quad (t_\gamma = -1 \text{ if no constraint mentions } \gamma_i), \quad (9.2)$$

because a path that ends at time $t \leq t_\gamma$ would park the agent on its goal at the forbidden time. Since $g(v, t) = t$ in space-time, the search key is simply $f(v, t) = t + d_i(v)$.

9.4 The algorithm

Algorithm 9.1 is the high level. The low level is not a new search: it is the space-time A^* of Chapter 4, printed in full as Algorithm 4.2, reading its filter as the constraint set of one agent, and Algorithm 9.2 lists only the lines in which the two differ. Together the three are a complete description of CBS as published by Sharon et al. [150]; the only freedom left is which conflict to split when a node has several, and how to break ties in OPEN.

9.4.1 The high level, line by line

Line 3 builds the root: one unconstrained low-level search per agent. If an agent cannot reach its goal even alone, no plan exists and CBS stops at once. Line 5 puts the root into OPEN, a priority queue keyed on $N.\text{cost}$.

Line 7 is the best-first choice: the node with the smallest sum of costs is expanded. Among nodes of equal cost, expanding the one with fewer conflicts first is the tie-breaking rule that Sharon et al. recommend [150]; a node without conflicts is a goal node, and one with few conflicts is likely to be close to one. The tie-breaking rule affects only the effort, not the answer.

Line 8 *validates* the node: it scans the k paths, time step by time step, for the earliest vertex or edge conflict (Chapter 7 gives the pairwise detection routine). The scan runs up to the largest path length and treats every agent as parked at its goal after its arrival. If no conflict exists, line 9 returns the paths: the node is a goal node, and Section 9.6 proves that it is optimal. Otherwise the earliest conflict c is split.

Lines 10–15 generate the two children. Each child copies the parent, so it inherits *all* constraints of the parent, and adds the single constraint κ to the set of the agent named in it (line 12). Only that agent is replanned (line 13): the other $k - 1$ paths still respect their unchanged constraint sets and are still of minimum cost, so recomputing them would only waste time and could, with different tie-breaking inside the low level, return different but equally good paths and confuse a trace. If the low level fails, the child is discarded: no path of

Algorithm 9.1. Conflict-based search: the high level

Input: graph G ; agents $a_1, \dots, a_k$ with starts s_i and goals γ_i
Output: a conflict-free plan $\Pi = (\pi_1, \dots, \pi_k)$ of minimum sum of costs, or *failure*

```

1 Function CBS( $G, s_{1..k}, \gamma_{1..k}$ ):
2   foreach agent  $a_i$  do
3      $R.\mathcal{C}_i \leftarrow \emptyset$ ;  $R.\pi_i \leftarrow \text{CbsLowLevel}(a_i, \emptyset)$ 
4   if some  $R.\pi_i$  is failure then return failure // an agent cannot reach its goal at all
5    $R.\text{cost} \leftarrow \sum_i \text{cost}(R.\pi_i)$ ; OPEN  $\leftarrow \{R\}$ 
6   while OPEN  $\neq \emptyset$  do
7      $N \leftarrow$  pop the node with the smallest  $N.\text{cost}$  from OPEN (ties: fewer conflicts first)
8      $c \leftarrow \text{FirstConflict}(N.\pi)$ 
9     if  $c = \text{nil}$  then return  $N.\pi$  // goal node: conflict-free, hence optimal
10    foreach constraint  $\kappa \in \text{Split}(c)$  (Equation (9.1)) do
11       $a_i \leftarrow$  the agent named in  $\kappa$ 
12       $N' \leftarrow$  a copy of  $N$ ;  $N'.\mathcal{C}_i \leftarrow N.\mathcal{C}_i \cup \{\kappa\}$ 
13       $N'.\pi_i \leftarrow \text{CbsLowLevel}(a_i, N'.\mathcal{C}_i)$  // replan only the constrained agent
14      if  $N'.\pi_i \neq \text{failure}$  then
15         $N'.\text{cost} \leftarrow \sum_j \text{cost}(N'.\pi_j)$ ; insert  $N'$  into OPEN
16  return failure

```

a_i satisfies its constraints, so no solution exists below that child. Otherwise the child's cost is recomputed and the child joins OPEN (line 15).

Note what is *not* in the loop. There is no closed list: the constraint tree is a tree, and two nodes with the same constraint sets are possible but rare (Section 9.8). There is no goal test at generation time: a child may be conflict-free but must wait in OPEN until it is the cheapest, because a cheaper node may still lead to a cheaper solution. And there is no explicit stopping rule for unsolvable instances: if the instance has no solution but every agent can reach its goal alone, the loop runs for ever, generating ever more expensive nodes (Section 9.6).

9.4.2 The low level, line by line

Algorithm 9.2 solves the low-level problem of Definition 9.3: it is Section 4.8's space-time A^* written against a constraint set. The search itself is unchanged, and it is worth saying which parts that covers. The state is a pair (v, t) ; the wait action is generated before the moves; the key is $f = t + d_i(v)$ with ties broken toward the larger t (the "deeper" state); and relaxation is replaced by duplicate detection, because $g(v, t) = t$ for every route into a state, so the first time a state is generated it is already the cheapest. Because d_i is consistent, no state is expanded twice.

Four lines are new. Line 3 is the filter: it drops the successors that a vertex or edge constraint of Definition 9.1 forbids, and the agent index is what selects $\mathcal{C}_i$ out of the node's constraints. Line 4 computes t_γ of Equation (9.2), the last time at which the goal is forbidden; a constraint set is finite and forbids single time steps, so, unlike a reservation table, it can never make t_γ infinite. Line 5 is the goal test with the goal-stay rule; a path that reaches γ_i at a time $t \leq t_\gamma$ is not accepted, and the search continues, typically by waiting somewhere and re-entering the goal later, or by leaving the goal and coming back. Line 6 fixes the time horizon: after the last constrained time H the search is unconstrained, and from any cell the goal is then at most D steps away, so an optimal path never needs more than $H + 1 + D$ steps.

Algorithm 9.2. The low level as a delta on space-time A^* : the four lines by which Algorithm 4.2 becomes a search for one agent under its constraints

Input: agent a_i with start s_i and goal γ_i ; constraint set $\mathcal{C}_i$; static distances $d_i(v)$ to γ_i

Output: a minimum-cost path from s_i to γ_i that respects $\mathcal{C}_i$, or *failure*

```

1 Function CbsLowLevel( $a_i, \mathcal{C}_i$ ):
2   run SpaceTimeAStar( $G, s_i, \gamma_i, \mathcal{C}_i, T_{\max}$ ) of Algorithm 4.2 — states  $(v, t)$ , wait
   action first,  $g(v, t) = t$ , key  $(f, -t)$  with  $f(v, t) = t + d_i(v)$  — with the following
   changes
3   filter: the forbidden set is  $\mathcal{C}_i$ , the constraints of this agent; skip the successor  $v'$  if
    $\langle a_i, v', t+1 \rangle \in \mathcal{C}_i$  or  $\langle a_i, v, v', t \rangle \in \mathcal{C}_i$ 
4    $t_\gamma \leftarrow$  largest  $t$  with  $\langle a_i, \gamma_i, t \rangle \in \mathcal{C}_i$ , or  $-1$ ; a constraint set, unlike a reservation table
   (Definition 8.1), never parks an agent, so  $t_\gamma < \infty$  always
5   goal test: return the path as soon as a popped state has  $v = \gamma_i$  and  $t > t_\gamma$ 
6   horizon:  $T_{\max} \leftarrow H + 1 + D$ , with  $H$  the largest time in  $\mathcal{C}_i$  (or  $-1$ ;  $t+1$  for an edge
   constraint  $\langle a_i, u, v, t \rangle$ ) and  $D = \max_v d_i(v)$ 

```

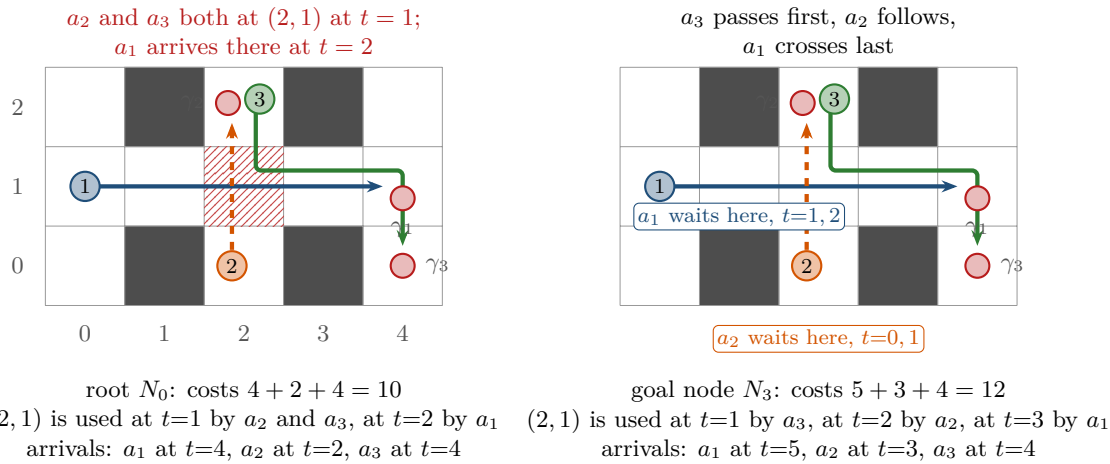


Figure 9.2. The worked example. Left: the root of the constraint tree, in which every drone flies its own shortest path; a_2 and a_3 both want the crossing cell (2, 1) at $t = 1$ (hatched). Right: the optimal plan returned in node N_3 , in which a_2 waits one step at its start and a_1 waits one step at (1, 1); the sum of costs rises from 10 to 12.

The horizon keeps the search finite when no path exists, for instance when the constraints seal the agent in. With these four lines the first path that passes the goal test is a minimum-cost path among those that respect $\mathcal{C}_i$.

9.5 A worked example

Example 9.4 (Two flight corridors that cross). Figure 9.2 (left) shows a 5×3 grid in which a horizontal corridor (row 1) and a vertical corridor (column 2) cross in the cell (2, 1); the four dark cells are buildings. Three drones fly: a_1 from (0, 1) to $\gamma_1 = (4, 1)$ along the horizontal corridor, a_2 from (2, 0) up to $\gamma_2 = (2, 2)$, and a_3 from (2, 2) down through the crossing and then east to $\gamma_3 = (4, 0)$. Cells are written (x, y) with x the column and y the row, y growing upward.

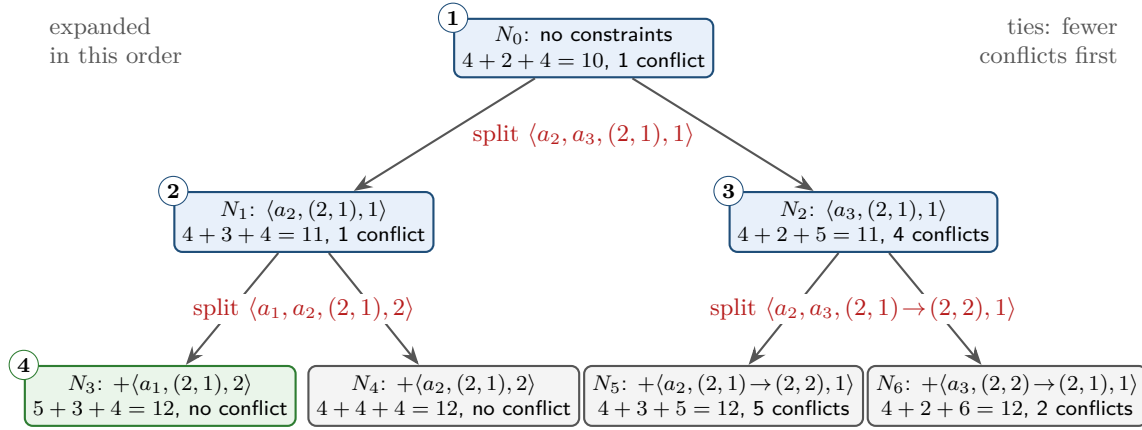


Figure 9.3. The constraint tree of Example 9.4. Every child adds exactly one constraint to its parent's sets (+) and replans one agent. Costs never decrease downward. Nodes N_4 , N_5 and N_6 were generated but never expanded; N_4 is a second optimal solution.

The root. Every number below is produced by `code/ch09_cbs.py`, which prints this trace when run. Planned alone, the drones take

$$\begin{aligned} R.\pi_1 &= (0, 1), (1, 1), (2, 1), (3, 1), (4, 1), & R.\pi_2 &= (2, 0), (2, 1), (2, 2), \\ R.\pi_3 &= (2, 2), (2, 1), (3, 1), (4, 1), (4, 0), \end{aligned}$$

with costs 4, 2 and 4, so $R.\text{cost} = 10$. Validation finds exactly one conflict: a_2 and a_3 are both in $(2, 1)$ at $t = 1$, the vertex conflict $\langle a_2, a_3, (2, 1), 1 \rangle$. (Agent a_1 follows one step behind a_3 from $t = 2$ on — at $t = 2$ it enters the cell $(2, 1)$ that a_3 left at $t = 1$ — but following is not a conflict in our model; this is why the one-step delay of a_3 in N_2 makes the two paths coincide.) The root is split into N_1 with the constraint $\langle a_2, (2, 1), 1 \rangle$ and N_2 with $\langle a_3, (2, 1), 1 \rangle$.

Depth one. In N_1 only a_2 is replanned. The cheapest path that is not in $(2, 1)$ at $t = 1$ waits one step: $N_1.\pi_2 = (2, 0), (2, 0), (2, 1), (2, 2)$, cost 3, so $N_1.\text{cost} = 11$. But now a_2 is in the crossing at $t = 2$, exactly when a_1 arrives there: N_1 has the single conflict $\langle a_1, a_2, (2, 1), 2 \rangle$. In N_2 only a_3 is replanned; it waits one step at its start, $N_2.\pi_3 = (2, 2), (2, 2), (2, 1), (3, 1), (4, 1), (4, 0)$, cost 5, and $N_2.\text{cost} = 11$ as well. This node is worse than it looks: between $t = 1$ and $t = 2$, a_2 moves up into $(2, 2)$ while a_3 moves down into $(2, 1)$, an edge conflict $\langle a_2, a_3, (2, 1) \rightarrow (2, 2), 1 \rangle$, and from $t = 2$ on a_3 occupies the same cells as a_1 at the same times, three more vertex conflicts. OPEN now holds N_1 and N_2 , both of cost 11; the tie-breaking rule picks N_1 (one conflict) before N_2 (four).

Depth two. N_1 is split on $\langle a_1, a_2, (2, 1), 2 \rangle$. Child N_3 adds $\langle a_1, (2, 1), 2 \rangle$: a_1 waits once at $(1, 1)$, $N_3.\pi_1 = (0, 1), (1, 1), (1, 1), (2, 1), (3, 1), (4, 1)$, cost 5, total 12, and the three paths are conflict-free. Child N_4 adds $\langle a_2, (2, 1), 2 \rangle$ to a_2 's set, which now holds two constraints; a_2 waits twice, cost 4, total 12, also conflict-free. Both children go into OPEN. The high level next pops N_2 , because its cost 11 is still smaller than 12: *a conflict-free node is not returned until it is the cheapest node in OPEN*. N_2 is split on its edge conflict into N_5 (constraint $\langle a_2, (2, 1) \rightarrow (2, 2), 1 \rangle$, cost 12, five conflicts) and N_6 ($\langle a_3, (2, 2) \rightarrow (2, 1), 1 \rangle$, cost 12, two conflicts). Now OPEN holds N_3, N_4, N_5, N_6 , all of cost 12; N_3 has no conflict and was generated first, so it is popped, validated, and returned. Figure 9.3 draws the tree and Table 9.1 lists every node; Table 9.2 shows the returned plan.

What the example shows. CBS expanded 4 nodes, generated 7, and called the low level 9 times (three times for the root, once per child); the optimal cost is 12, which the brute-force

Table 9.1. The constraint-tree nodes of [Example 9.4](#) in the order of generation. A child inherits its parent’s constraints and adds the one listed; π_1, π_2, π_3 are the costs of the three paths. “Conflicts” is the number of conflicts among the node’s paths; the last column names the conflict that is split when the node is expanded.

Node	Parent	Constraint added	π_1	π_2	π_3	Total	Conflicts	Expansion
N_0	–	none	4	2	4	10	1	1st: split $\langle a_2, a_3, (2, 1), 1 \rangle$
N_1	N_0	$\langle a_2, (2, 1), 1 \rangle$	4	3	4	11	1	2nd: split $\langle a_1, a_2, (2, 1), 2 \rangle$
N_2	N_0	$\langle a_3, (2, 1), 1 \rangle$	4	2	5	11	4	3rd: split $\langle a_2, a_3, (2, 1) \rightarrow (2, 2), 1 \rangle$
N_3	N_1	$\langle a_1, (2, 1), 2 \rangle$	5	3	4	12	0	4th: goal node, returned
N_4	N_1	$\langle a_2, (2, 1), 2 \rangle$	4	4	4	12	0	left in OPEN (also optimal)
N_5	N_2	$\langle a_2, (2, 1) \rightarrow (2, 2), 1 \rangle$	4	3	5	12	5	left in OPEN
N_6	N_2	$\langle a_3, (2, 2) \rightarrow (2, 1), 1 \rangle$	4	2	6	12	2	left in OPEN

Table 9.2. The plan returned in N_3 : the cell of every drone at every time step. a_3 passes the crossing first, a_2 follows, and a_1 crosses last; after its arrival a drone stays at its goal.

t	0	1	2	3	4	5
a_1	(0, 1)	(1, 1)	(1, 1) wait	(2, 1)	(3, 1)	(4, 1) goal
a_2	(2, 0)	(2, 0) wait	(2, 1)	(2, 2) goal	(2, 2)	(2, 2)
a_3	(2, 2)	(2, 1)	(3, 1)	(4, 1)	(4, 0) goal	(4, 0)

search over the joint state space in `ch09_cbs.py` confirms. Four lessons are worth recording. (i) Resolving one conflict can create another: the wait of a_2 in N_1 produced the conflict with a_1 , a drone that was not involved at the root. (ii) Constraints accumulate: N_4 holds two constraints on a_2 , and its path respects both. (iii) The whole subtree below N_2 , in which a_3 yields, costs at least 12; CBS generated its two children but never had to expand them, because N_3 was found at the same cost first, and it never had to look deeper, because costs cannot decrease. (iv) Two optimal solutions exist (N_3 and N_4); which one is returned depends only on tie-breaking. Finally, every conflict split here was *cardinal*: both children were more expensive than their parent. [Section 9.7](#) explains why this is the case one wants, and why the variant that splits cardinal conflicts first produces the same tree on this instance.

9.6 Properties: correctness, optimality, complexity

Throughout this section the graph is finite, every agent can reach its goal alone, and the low level is [Algorithm 9.2](#), which returns a minimum-cost path respecting its constraints whenever one exists ([Chapter 4](#), together with the horizon argument of [Section 9.4.2](#)). Two lemmas carry the entire theory.

Lemma 9.5 (A split loses no solution). *Let N be a constraint-tree node whose paths have a conflict c between a_i and a_j , and let N'_i and N'_j be the two children obtained by splitting c . Every solution Π whose paths respect the constraints of N respects the constraints of N'_i or those of N'_j (or both).*

Proof. Suppose $c = \langle a_i, a_j, v, t \rangle$ is a vertex conflict. If $\pi_i[t] = v$ and $\pi_j[t] = v$, then Π has a vertex conflict at (v, t) and is not a solution. Hence $\pi_i[t] \neq v$ or $\pi_j[t] \neq v$. In the first case π_i respects $\langle a_i, v, t \rangle$, the only constraint of N'_i that is not already a constraint of N , so Π respects all constraints of N'_i ; in the second case the same holds for N'_j . (The convention that a parked agent occupies its goal is used here: if π_i has arrived at $\gamma_i = v$ before t , then $\pi_i[t] = v$ and Π violates the constraint, exactly as it would conflict in reality.) If $c = \langle a_i, a_j, u, v, t \rangle$ is an edge

conflict, the children forbid a_i to move $u \rightarrow v$ and a_j to move $v \rightarrow u$ at time t . A solution cannot make both moves, because together they are a swapping conflict (the *edge conflict* of Section 9.3), so it respects at least one of the two constraints. $\square$

Lemma 9.6 (The cost of a node is a lower bound). *Let N be a constraint-tree node and Π any plan whose paths respect the constraints of N . Then $\text{SoC}(\Pi) \geq N.\text{cost}$. In particular, every descendant N' of N satisfies $N'.\text{cost} \geq N.\text{cost}$: costs never decrease along a branch of the tree.*

Proof. By Definition 9.2, $N.\pi_i$ has minimum cost among the paths of a_i that respect $N.C_i$, and π_i is such a path, so $\text{cost}(\pi_i) \geq \text{cost}(N.\pi_i)$ for every i ; summing over i gives the first claim. For the second, the constraint sets of a descendant contain those of N , so $N'.\pi$ respects the constraints of N , and the first claim applies with $\Pi = N'.\pi$. $\square$

Lemma 9.7 (Invariant of the high level). *Suppose the instance is solvable and let Π^* be an optimal solution, $C^* = \text{SoC}(\Pi^*)$. At the start of every iteration of the loop of Algorithm 9.1, OPEN contains a node N whose constraints are all respected by Π^* , and every such node has $N.\text{cost} \leq C^*$.*

Proof. The bound $N.\text{cost} \leq C^*$ is Lemma 9.6 applied to $\Pi = \Pi^*$. For the existence, argue by induction over the iterations. Initially OPEN holds the root, which has no constraints. Consider an iteration that pops a node N . If N is not one of the nodes respected by Π^* , those nodes stay in OPEN. If it is, and it is a goal node, the algorithm returns and there is no next iteration. Otherwise N has a conflict c , and by Lemma 9.5 Π^* respects the constraints of at least one child N' . The path π_i^* of the agent constrained in N' respects $N'.C_i$, so a path exists and the low level does not fail; N' is generated and inserted into OPEN. $\square$

Theorem 9.8 (Optimality of CBS). *If Algorithm 9.1 returns a plan Π , then Π is a solution and $\text{SoC}(\Pi) = C^*$.*

Proof. The plan is returned at line 9 only after line 8 found no conflict, so Π is a solution and $\text{SoC}(\Pi) \geq C^*$. Let N be the node that was popped; $N.\text{cost} = \text{SoC}(\Pi)$. At that moment OPEN contained, by Lemma 9.7, a node of cost at most C^* , and N was a node of minimum cost in OPEN, so $N.\text{cost} \leq C^*$. Hence $\text{SoC}(\Pi) = C^*$. $\square$

Theorem 9.9 (Completeness of CBS on solvable instances). *If the instance has a solution, Algorithm 9.1 terminates and returns one.*

Proof. Let C^* be the optimal cost. We first show that only finitely many constraint-tree nodes have cost at most C^* . Let N be such a node. By Lemma 9.6 all its ancestors have cost at most C^* too. Every constraint on the branch from the root to N was created by splitting a conflict of some ancestor M at a time t ; at that time at least one of the two agents had not yet arrived (two parked agents cannot share a cell, because goals are distinct, and an edge conflict involves two moving agents), so $t \leq \max_i \text{cost}(M.\pi_i) \leq M.\text{cost} \leq C^*$. Moreover the constraint was not already in M 's sets, because M 's path violated it while respecting M 's constraints. The constraints along the branch are therefore distinct elements of the finite set of constraints with time at most $C^* + 1$, whose size is at most $\Delta = k(|V| + 2|E|)(C^* + 2)$. So the depth of N is at most Δ , and since every node has at most two children, there are fewer than $2^{\Delta+1}$ nodes of cost at most C^* .

Now run the algorithm. By Lemma 9.7, OPEN is never empty, so the loop can only end by returning at line 9. Every popped node has minimum cost in OPEN and OPEN always contains a node of cost at most C^* , so every popped node has cost at most C^* . Each node is popped at most once, and there are finitely many nodes of cost at most C^* , so the loop performs finitely many iterations and must return. By Theorem 9.8 the returned plan is an optimal solution. $\square$

Remark 9.10 (Unsolvable instances). If every agent can reach its goal alone but no conflict-free plan exists (two agents in a corridor that must swap, for instance), the invariant fails and CBS keeps generating ever more expensive nodes for ever. Sharon et al. [150] state completeness for solvable instances only. In practice one imposes a limit on time or on the number of expanded nodes, or first runs a complete polynomial-time solver such as Push-and-Rotate (Chapter 11) to decide solvability and to obtain an upper bound on C^* , beyond which the constraint tree need not be explored.

9.6.1 What the running time depends on

One iteration of the high level costs one validation, $\mathcal{O}(k^2T)$ for paths of length up to T when done pairwise (or $\mathcal{O}(kT)$ with a hash table of occupied cells per time step), plus two low-level searches, each $\mathcal{O}(|V| T_{\max} \log(|V| T_{\max}))$ for the space-time A^* . The total time is this per-node cost multiplied by the number of expanded constraint-tree nodes, and that number is the crux. The proof of Theorem 9.9 bounds it by $2^{\Delta+1}$, where Δ counts the constraints that could appear; in words, *the effort of CBS is exponential in the number of conflicts it has to resolve, and only polynomial in the number of agents for a fixed number of conflicts*. Sharon et al. [150] phrase the same conclusion as a comparison with A^* on the joint state space (with the operator decomposition and independence detection of Standley [156], see Chapter 11): joint-space search is exponential in k regardless of how the agents interact, CBS is exponential in the depth of the constraint tree regardless of k .

Corridors against open rooms. Which of the two is better on a given instance depends on how many alternative paths a conflict leaves open. Consider first a *bottleneck*: two agents that must pass, one after the other, through a gap of one cell (Exercise 9.2). Each agent has a single shortest path, one constraint delays one agent by one step, and the constraint tree has three nodes; joint-space A^* , in contrast, has to explore the product of the two agents' approaches to the gap. CBS wins easily, and the more free space surrounds the bottleneck, the larger its advantage. Now consider an *open room*: two agents cross an obstacle-free rectangle in perpendicular directions. Both have many shortest paths of equal cost, and, as Section 9.7.5 and Exercise 9.6 show, every pair of them conflicts. A constraint on one shortest path merely diverts the agent to another shortest path that conflicts again at a different cell, at the same cost, so the constraint tree fills a whole level of equal-cost nodes before the cost can rise by one. On the 5×5 room of Figure 9.5 the self-test of `ch09_cbs.py` records 15 expanded nodes for a single cost increase with only two agents; the number grows exponentially with the size of the rectangle, whereas joint-space A^* simply steps around the encounter. The worst case for CBS is a small, crowded map in which agents must shuffle past each other. The self-test contains a 4×3 maze with 8 free cells and three agents whose optimal sum of costs is 24: Dijkstra on the joint state space finds it in 0.01 s, while CBS was still without a conflict-free node after the 2000 constraint-tree nodes (0.3 s) that the self-test allows it, because every extra unit of cost can be spent as a wait of one step by one of several agents at one of several times, and each of these choices is a separate branch. Table 9.3 summarizes the comparison; the improvements of Section 9.7 attack precisely the open-room and corridor cases.

9.7 Variants and extensions

Plain CBS, as defined so far, is a clean algorithm but a slow solver: on open maps it spends most of its time generating equal-cost nodes that resolve nothing. The improvements below, all of which keep optimality, attack this waste at four points: *which* conflict to split, whether to split at all, how to *guide* the high level, and how to recognize whole families of conflicts at once. The bounded-suboptimal relaxation of CBS, ECBS, is the subject of Chapter 10.

Table 9.3. Exact and heuristic MAPF solvers seen so far. “Effort grows with” names the quantity that drives the exponential part of the running time.

Method	Complete	Optimal	Effort grows with	Best on
Joint-space A* (Chapters 7 and 11)	✓	✓	number of agents k	few, tightly coupled agents
Prioritized planning (Chapter 8)	no	no	nothing exponential	many agents, sparse interaction, no guarantees needed
CBS (this chapter)	solvable instances	✓	depth of the constraint tree (conflicts to resolve)	many agents with few, local conflicts
ECBS (Chapter 10)	solvable instances	w -bounded	the same, with far fewer nodes	the same, at larger scale

Table 9.4. The three conflict types of Definition 9.11 and what a split achieves. ICBS works down the last column: split a cardinal conflict if there is one, otherwise a semi-cardinal one.

Type	Children’s costs	MDD test at level t	Split it?
cardinal	both $> N.\text{cost}$	single cell in both MDDs	yes, first: the cost rises
semi-cardinal	exactly one $> N.\text{cost}$	single cell in one MDD	second choice
non-cardinal	both $= N.\text{cost}$	alternatives in both MDDs	avoid; try a bypass

9.7.1 Cardinal, semi-cardinal and non-cardinal conflicts

Definition 9.11 (Conflict types). Let c be a conflict of node N and let N'_i and N'_j be the children that splitting c would produce. The conflict is **cardinal** if $N'_i.\text{cost} > N.\text{cost}$ and $N'_j.\text{cost} > N.\text{cost}$, **semi-cardinal** if exactly one child is more expensive than N , and **non-cardinal** if neither is.

Bojarski et al. [22] observed that the order in which conflicts are split changes the size of the tree dramatically, and that splitting a non-cardinal conflict is almost always wasted work: both children keep the parent’s cost, so OPEN receives two nodes that are as good as their parent and still have conflicts, and the level of equal-cost nodes doubles. Splitting a cardinal conflict, in contrast, raises the cost in both children, so the search makes progress toward C^* with every split. Their algorithm, improved CBS (ICBS), therefore splits a cardinal conflict if the node has one, else a semi-cardinal one, else any conflict. Table 9.4 summarizes the three cases; in Example 9.4 every split was cardinal, which is why ICBS builds the same tree there.

Recognizing the type without generating the children uses the **multi-valued decision diagram** (MDD) of an agent, introduced by Sharon et al. for the increasing cost tree search [151]: for agent a_i with current cost $c = \text{cost}(N.\pi_i)$, the MDD is the layered graph whose level t contains every cell that lies at time t on *some* path of cost c from s_i to γ_i respecting $N.C_i$. Building it costs one breadth-first search forward and one backward in space-time. A vertex conflict at (v, t) is cardinal exactly when v is the only cell at level t of both MDDs [22], semi-cardinal when it is the only cell in one of them, and non-cardinal otherwise; edge conflicts are classified in the same way on the MDD edges. Figure 9.4 shows both situations: the corridor agents of the worked example have chain-shaped MDDs, so their conflict is cardinal, whereas two agents crossing an open room have wide MDDs, and all their conflicts are non-cardinal, the reason for the exponential trees of Section 9.6.1.

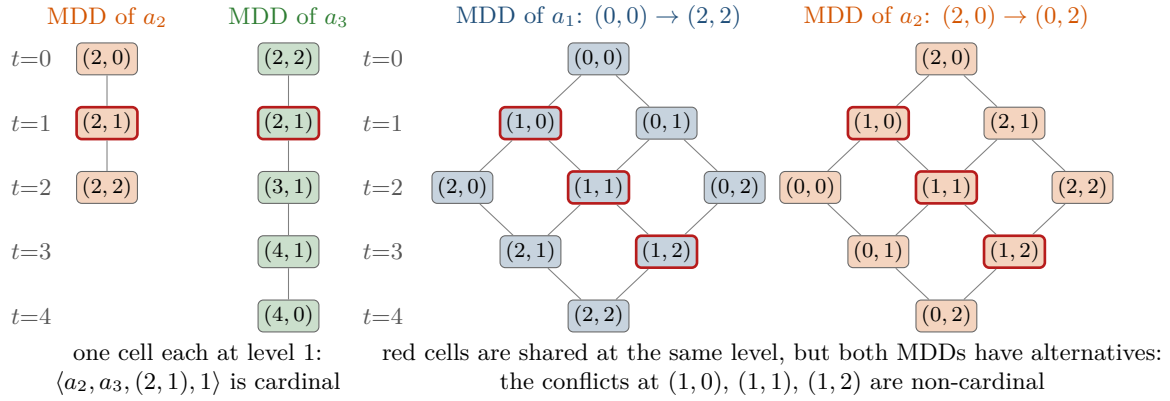


Figure 9.4. Multi-valued decision diagrams. Left: at the root of [Example 9.4](#) the MDDs of a_2 and a_3 are chains that share the cell $(2,1)$ at level 1, so the conflict there is cardinal. Right: in an open 3×3 room the MDDs of two crossing agents are wide; the shared cells (red) have alternatives on both sides, so every conflict is non-cardinal and a split does not raise the cost.

9.7.2 Bypassing a conflict

The second ICBS idea handles the non-cardinal conflicts that remain. Suppose node N is expanded and one of its children, say N' , has the same cost as N but fewer conflicts. Instead of inserting both children, ICBS *adopts* the paths of N' into N ($N.\pi \leftarrow N'.\pi$) and puts N back into OPEN; no split takes place. This is the **bypass**: the agent found an alternative path of the same cost that avoids the conflict, so the tree need not record the choice. The modified N is still a valid constraint-tree node in the sense of [Definition 9.2](#), because $N'.\pi_i$ respects $N.\mathcal{C}_i$ (it respects the larger set $N'.\mathcal{C}_i$) and has the minimum cost under it, because $\text{cost}(N'.\pi_i) = \text{cost}(N.\pi_i)$, which is the minimum under $N.\mathcal{C}_i$; hence [Lemmas 9.5](#) and [9.6](#) and the optimality proof are untouched. If no child qualifies, N is split as usual. Bypasses are cheap, because the children had to be generated anyway, and on open maps they remove most non-cardinal conflicts without growing the tree.

9.7.3 Meta-agent CBS

When two agents keep conflicting, a long chain of one-cell constraints is a poor way to coordinate them. Meta-agent CBS (MA-CBS) by Sharon et al. [\[150\]](#) counts, for every pair of agents, the conflicts split between them in the search so far (some implementations count only along the current branch); when the count exceeds a bound B , the two agents are *merged* into a meta-agent that the low level plans jointly, in the joint space of the pair, with the operator-decomposition A^* of Standley [\[156\]](#). The node is then replanned with the meta-agent under the union of the two agents' external constraints, and the tree continues to branch on conflicts between (meta-)agents. With $B = 0$ every pair is merged at its first conflict, so MA-CBS degenerates to Standley's independence detection [\[156\]](#): each group of interacting agents is solved by a joint-space A^* , while agents that never conflict are still planned alone. With $B = \infty$ it is plain CBS. The bound therefore interpolates between the two regimes of [Section 9.6.1](#): small B for tightly coupled groups in crowded rooms, large B for loosely coupled swarms in corridors.

9.7.4 Admissible heuristics for the high level

The high level of [Algorithm 9.1](#) orders nodes by $N.\text{cost}$ alone, which is g without an h : it is uniform-cost search over the constraint tree. Felner et al. [\[40\]](#) showed how to add an admissible

heuristic and run the high level as A^* (the family is called CBSH, for CBS with heuristics). The quantity to estimate is how much more cost any solution below N must pay, and cardinal conflicts provide a lower bound. Build the **conflict graph** of N : one vertex per agent and one edge per cardinal conflict. Every solution below N resolves every cardinal conflict, and resolving one forces at least one of its two agents to pay at least one extra unit (that is what cardinal means, in unit-cost grids). Set

$$h_{CG}(N) = \text{size of a minimum vertex cover of the conflict graph of } N \quad (9.3)$$

and collect the agents whose cost is larger in a solution below N than in N . Because each cardinal conflict forces at least one of its two agents into that set, the set is a vertex cover of the conflict graph, so it contains at least $h_{CG}(N)$ agents, each paying at least one extra unit. Hence every solution below N costs at least $N.\text{cost} + h_{CG}(N)$: the heuristic h_{CG} is admissible, and the high level can expand nodes in order of $N.\text{cost} + h_{CG}(N)$. Minimum vertex cover is NP-hard in general, but conflict graphs have few edges and a small branch-and-bound solves them in microseconds. At the root of [Example 9.4](#) the conflict graph has the single edge a_2a_3 , so $h_{CG}(N_0) = 1$ and the root's key is 11; in N_1 the conflict $\langle a_1, a_2, (2, 1), 2 \rangle$ is cardinal, so N_1 has key 12, the same as N_2 (cost 11 plus one cardinal edge) and the same as N_3 (cost 12, no conflict). A tie-break toward fewer conflicts then pops N_3 before N_2 , and CBSH expands three nodes instead of four. Later work strengthened the heuristic with dependency graphs (an edge whenever the pair's MDDs admit no conflict-free joint path) and weighted dependency graphs (the edge weight is the pair's true extra cost, and a minimum weighted vertex cover is taken) [102]; these are the heuristics of today's reference implementations.

9.7.5 Symmetry breaking: rectangles and corridors

The exponential trees of [Section 9.6.1](#) come from *symmetries*: families of shortest paths that are interchangeable for the cost but conflict pairwise, so that one-cell constraints exclude them one at a time. Li et al. [104] identified the most common one on grids, the **rectangle symmetry** ([Figure 9.5](#), left): two agents whose shortest paths cross an obstacle-free rectangle in perpendicular directions, each moving monotonically toward its goal. [Exercise 9.6](#) asks you to prove that every shortest path of one agent conflicts with every shortest path of the other. **Rectangle reasoning** detects such a pair from the MDDs and the start-goal bounding boxes, and replaces the two one-cell constraints of the split by two **barrier constraints**: the first child forbids a_1 from every cell of the rectangle's exit border at the time at which a shortest path would reach it, the second child does the same for a_2 . The barriers are a valid split in the sense of [Lemma 9.5](#), since no solution can send both agents across the rectangle at shortest-path speed, so optimality is preserved, and a whole exponential family of conflicts is resolved by one split.

The mirror image of the open room is the **corridor symmetry** ([Figure 9.5](#), right): two agents traverse a corridor of degree-two cells in opposite directions. One of them must wait outside until the other has left, so the optimal cost exceeds the root cost by roughly the length of the corridor, and every intermediate cost level is filled with equal-cost nodes that differ only in where and how long the agents wait. **Corridor reasoning**, introduced with several related techniques by Li et al. [103], computes for each agent the earliest time at which it can leave the corridor if the other agent goes first, and splits with two *range constraints*: the first child forbids a_1 from the far end of the corridor at all times up to that bound, the second child forbids a_2 symmetrically. The same paper treats the *target symmetry*, in which an agent parked at its goal blocks a corridor that another agent must use later, the situation of [Exercise 9.3](#). All these techniques share one principle, which is also the principle behind barrier constraints and MA-CBS: *any pair of constraint sets such that every solution satisfies at least one of them is a legal split*. [Lemma 9.5](#) is the only property a split has to satisfy.

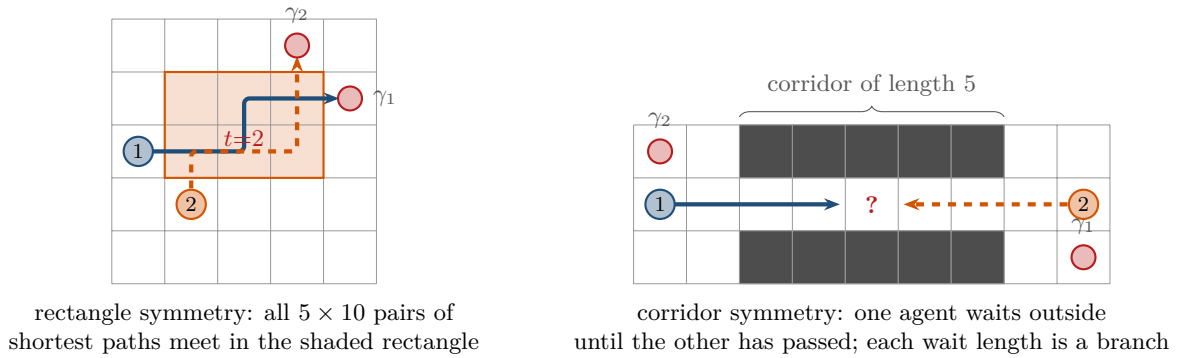


Figure 9.5. Two symmetries. Left: rectangle symmetry in an open 5×5 room; one of the 5 shortest paths of a_1 and one of the 10 of a_2 are drawn, and every pair meets in the shaded rectangle. Right: corridor symmetry; a_1 and a_2 enter a corridor of length 5 from opposite ends, one must wait outside until the other has come through, and plain CBS turns each possible waiting time into a branch.

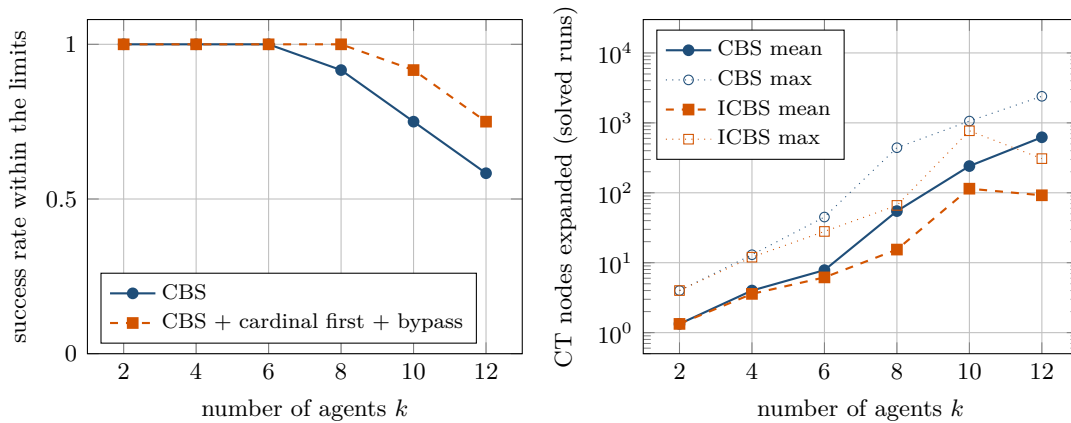


Figure 9.6. CBS on random 8×8 grids (15 % obstacles, 12 instances per k , 5 s and 20 000 CT nodes per run). Left: fraction of instances solved. Right: mean and maximum number of expanded constraint-tree nodes on the solved instances, logarithmic scale. Both curves grow exponentially with k on this small map, because the number of conflicts does; cardinal-first splitting and bypasses divide the node count by two to seven and raise the success rate.

9.7.6 An experiment on random grids

Figure 9.6 shows how CBS scales on random 8×8 grids with 15 % obstacles as the number of agents grows from 2 to 12. For each k the script `code/figures/gen_ch09_scaling.py` draws 12 instances with random distinct starts and goals and runs two solvers from `ch09_cbs.py` on each, with a limit of 5 s and 20 000 expanded nodes per run: plain CBS (split the earliest conflict, ties on fewer conflicts) and CBS with cardinal conflicts first and the bypass, labeled ICBS. The success rate is the fraction of instances solved within the limits; the node counts are averaged over the solved instances only.

Up to six agents every instance is solved in a few milliseconds, with 1.3, 4.0 and 7.8 expanded nodes on average for $k = 2, 4, 6$. From eight agents on the tree explodes on some instances: plain CBS solves 92 % of the instances with $k = 8$, 75 % with $k = 10$ and 58 % with $k = 12$, and the mean number of expanded nodes on the solved instances grows to 55, 241 and 622, with maxima of 441, 1 061 and 2 398. The gap between mean and maximum is the signature of Section 9.6.1: most instances are loosely coupled and cheap, a few contain

a crowded room or a corridor and dominate the running time. Cardinal-first splitting with bypasses cuts the mean node count to 15, 114 and 92 for $k = 8, 10, 12$ and the maxima to 66, 771 and 308; it solves all instances with $k = 8$ and 92 % and 75 % for $k = 10$ and 12. (Its mean at $k = 10$ is larger than at $k = 12$ because at $k = 10$ it still solves an instance that needs 771 nodes – one of those that plain CBS abandons – whereas at $k = 12$ the three hardest instances time out and drop out of the mean altogether: means taken over the solved instances only are not comparable when two solvers, or one solver at two team sizes, solve different sets.) Its price is visible in the low-level statistics: to classify a conflict, our implementation generates the two children of every candidate conflict in time order until it finds a cardinal one, so it performs many more low-level expansions per constraint-tree node: at $k = 12$ a solved run costs 23 600 low-level state expansions against 17 700 for plain CBS, although it expands only 92 instead of 622 CT nodes; an MDD-based classifier avoids this. Twelve instances per setting make the success rates coarse (one instance is 8 %), the 5 s cap makes them depend a little on the machine, and the absolute times are those of pure Python, but the shape of the curves is the one reported by Sharon et al. and Boyarski et al. on benchmark maps: the number of agents that CBS handles is set by the number of conflicts on the map, and the ICBS improvements move the wall by a few agents. Larger teams need the bounded-suboptimal ECBS of [Chapter 10](#).

9.8 Implementation notes

`code/ch09_cbs.py` implements everything in this chapter in about 600 lines, of which some 110 are the self-tests: the grid and the backward breadth-first distances, the low level, conflict detection, the high level with the tie-breaking on conflicts, the cardinal-first and bypass options, a brute-force optimal solver on the joint state space for certification, and the worked example. [Listing 9.1](#) is the low level and [Listing 9.2](#) the body of the high level. The remarks below are the decisions that matter.

Constraint sets. A node stores one immutable set of vertex constraints (v, t) and one of edge constraints (u, v, t) per agent. Immutability (`frozenset`) makes “a child inherits all constraints of its parent and adds one” a single union, and it makes aliasing bugs impossible: a child can never modify its parent’s sets. For very deep trees one stores only the added constraint and a parent pointer, and rebuilds the set on demand, or uses a persistent set structure; the same applies to the k paths, of which only one differs from the parent.

The heuristic of the low level. The static distances $d_i(v)$ to γ_i are computed once per agent by a backward breadth-first search and shared by every low-level call for that agent; the distances also give D for the horizon and tell at once whether an agent can reach its goal at all. With a consistent heuristic no state is expanded twice, so the closed set is a plain `set` of (v, t) pairs.

Which conflict to split. The default is the earliest conflict in time, the choice of Sharon et al.: resolving an early conflict changes the paths from that time on and often removes later conflicts as a side effect, whereas resolving a late conflict first is frequently undone by the resolution of an earlier one. The option `split="cardinal"` implements ICBS’s preference by generating the children of the conflicts in time order and stopping at the first cardinal one; the generated children are reused when the node is expanded. This is exact but costs extra low-level searches ([Section 9.7.6](#)); a production solver classifies conflicts with MDDs instead.

Tie-breaking. The heap key is $(N.\text{cost}, \text{number of conflicts}, \text{node id})$. Fewer conflicts first is the rule of [Algorithm 9.1](#); the id makes the order deterministic (first generated, first expanded), which the worked example and the self-test rely on. Preferring *deeper* nodes among equal keys is another common choice: it dives toward a goal node instead of sweeping a level.

Duplicate nodes. Two branches can reach the same constraint sets, for instance by splitting two independent conflicts in the two possible orders. Duplicate detection by hashing the constraint sets is easy to add, but such coincidences are uncommon; the implementation here does not detect duplicates, and the proofs of [Section 9.6](#) do not need it.

Validation and limits. Conflict detection walks the time axis once, comparing every pair of agents, and treats an agent as parked at its goal after its arrival; on large teams a dictionary from cells to agents per time step replaces the pairwise loop. The solver takes a time limit and a node limit and returns `None` when it hits one, which is the only way to stop on an unsolvable instance ([Remark 9.10](#)). The function `validate` re-checks a returned plan independently (start, goal, adjacency of consecutive cells, obstacles, conflicts); running it on every answer during development catches most bugs of the kinds described in the pitfalls below.

Listing 9.1. The low level: space-time A^* under constraints, with the goal-stay test and the horizon $T_{\max} = H + 1 + D$ (from `code/ch09_cbs.py`).

```

1 def low_level(grid: Grid, start: Cell, goal: Cell, cons: Constraints,
2             dist: Dict[Cell, int], stats: Optional[Stats] = None) ->
   Optional[Path]:
3     """Space-time  $A^*$  for one agent: a shortest path from ``start`` to
4     ``goal`` that respects ``cons``; ``dist`` are true distances to the goal.
5     The agent may finish at (goal, t) only if no vertex constraint touches
6     the goal at a time  $\geq t$ , because it stays at the goal for ever after."""
7     if stats is not None:
8         stats.low_level_calls += 1
9     if start not in dist:
10        return None
11    goal_block = max((t for v, t in cons.vertex if v == goal), default=-1)
12    horizon = cons.last_time() + 1 + max(dist.values()) #  $T_{\max} = H + 1 + D$ 
13    counter = itertools.count()
14    g = {(start, 0): 0}
15    parent: Dict[Tuple[Cell, int], Tuple[Cell, int]] = {}
16    open_heap = [(dist[start], 0, next(counter), (start, 0))]
17    closed = set()
18    while open_heap:
19        f, neg_g, _, state = heapq.heappop(open_heap)
20        if state in closed:
21            continue
22        closed.add(state)
23        if stats is not None:
24            stats.low_level_expanded += 1
25        v, t = state
26        if v == goal and t > goal_block: # goal-stay test
27            path = [v]
28            while state in parent:
29                state = parent[state]
30                path.append(state[0])
31            return path[::-1]
32        if t >= horizon:
33            continue
34        for w in grid.neighbors(v) + [v]: # moves and the wait action

```



```

35         nxt = (w, t + 1)
36         if nxt in closed or nxt in cons.vertex or (v, w, t) in cons.edge:
37             continue
38         if t + 1 < g.get(nxt, float("inf")):
39             g[nxt] = t + 1
40             parent[nxt] = state
41             heapq.heappush(open_heap, (t + 1 + dist[w], -(t + 1),
next(counter), nxt))
42     return None

```

Listing 9.2. The body of the high level `cbs()`: root, best-first loop on the sum of costs, split, replan of one agent, optional bypass (from `code/ch09_cbs.py`; `_make_child` unions the parent's constraints with the new one and calls `low_level` for that agent only).

```

1     t0, stats, trace, ids = time.perf_counter(), Stats(), [], itertools.count()
2     dists = [grid.distances(g) for g in goals] # one BFS per agent
3     paths = [low_level(grid, s, g, Constraints(), d, stats)
4               for s, g, d in zip(starts, goals, dists)] # root: no constraints
5     if any(p is None for p in paths):
6         return Result(None, None, stats)
7     root = CTNode(tuple(Constraints() for _ in starts), paths,
8                     sum_of_costs(paths), len(all_conflicts(paths)), next(ids))
9     open_heap = [(root.cost, root.n_conflicts, root.id, root)]
10    stats.generated += 1
11    while open_heap:
12        _, _, _, node = heapq.heappop(open_heap) # lowest cost, fewest
conflicts
13        stats.expanded += 1
14        conflict, kids = _choose_conflict(node, split, grid, starts, goals,
dists, stats)
15        entry = _record(node, conflict, trace) if keep_trace else None
16        if conflict is None: # goal node: conflict-free
17            stats.seconds = time.perf_counter() - t0
18            return Result(node.paths, node.cost, stats, trace)
19        if (time_limit is not None and time.perf_counter() - t0 > time_limit)
or \
20            (node_limit is not None and stats.expanded >= node_limit):
21            break
22        children = []
23        for kid, (agent, extra) in zip(kids, conflict.constraints()):
24            if kid is None and split == "first": # replan only that agent
25                kid = _make_child(node, agent, extra, grid, starts, goals,
dists, stats)
26            if kid is not None:
27                children.append(kid)
28        if bypass and _bypass(node, children):
29            stats.bypasses += 1
30            if entry is not None:
31                entry["bypass"] = True
32            heapq.heappush(open_heap, (node.cost, node.n_conflicts, node.id,
node))
33        continue
34        for kid in children:
35            kid.id = next(ids)
36            stats.generated += 1
37            if entry is not None:
38                entry["children"].append(_child_summary(kid))

```

```

39         heapq.heappush(open_heap, (kid.cost, kid.n_conflicts, kid.id, kid))
40     stats.seconds = time.perf_counter() - t0
41     return Result(None, None, stats, trace)

```

Replan only the constrained agent, and do replan it

The child of a split must (a) inherit *every* constraint of its parent, (b) add exactly one, and (c) recompute the path of the constrained agent and no other. Forgetting (a), for instance by building the child’s set from the new constraint alone, lets the agent walk back into a cell it was forbidden earlier on the branch, and the tree loops through the same conflicts. Forgetting (c) by copying the parent’s path list by reference, or by replanning the wrong agent of the pair, produces a node whose paths violate its own constraints; the node’s cost is then wrong, and [Lemma 9.6](#) fails, so the returned plan may be suboptimal or conflicting. Replanning *all* agents is not wrong but wastes $k - 1$ searches per node and, with low-level tie-breaking, can shuffle paths that had no reason to change. Immutable constraint sets and an independent `validate` of every returned plan make all three mistakes visible at once.

A parked drone still occupies its goal

Two places must know that an agent stays at its goal for ever. The conflict check must compare agents beyond the end of the shorter path, treating the arrived agent as sitting on its goal cell; otherwise a drone flying through another’s goal after its arrival is never reported and the plan is invalid. And the low level must apply the goal-stay test (line 5): a constraint $\langle a_i, \gamma_i, t \rangle$ with t later than the agent’s arrival is only satisfied by a path that arrives after t or leaves and comes back. A low level that accepts the first arrival returns the parent’s path unchanged, the child has the same conflict as the parent, and the high level splits the same conflict for ever.

An unbounded time horizon

Space-time has infinitely many states, because waiting is always possible. If the constraints seal an agent in, a low level without a horizon runs for ever, expanding (v, t) for larger and larger t ; the same happens when a constraint forbids the goal at a time that no path can avoid. The horizon $T_{\max} = H + 1 + D$ of line 6 keeps every low-level call finite without excluding any optimal constrained path. The high level has the corresponding problem on unsolvable instances ([Remark 9.10](#)); always run it with a time or node limit.

9.9 Where this fits in the drone system

In the drone system

CBS is the global planner, the first of the four layers of the hybrid architecture in [Chapter 24](#). Before take-off it receives the inflated occupancy grid of [Chapter 2](#), one start and one goal cell per drone, and a time step chosen so that a drone crosses one cell per step at cruise speed, and it returns the nominal time-indexed routes $\pi_1, \dots, \pi_k$, conflict-free and of minimum total flight time. In three dimensions nothing changes except the successor function of the low level (a 6-connected grid of altitude layers). The routes are what the other layers work with: the prediction layer ([Chapters 18 and 20](#)) watches for intruders that were not in the plan, the local safety layer ([Chapters 13 and 14](#)) lets a

drone deviate from its route to avoid them, and the replanning layer (Chapter 5) repairs the deviated drone's route, using the routes of the other drones as constraints in exactly the form of the low level here. After any such repair the architecture runs the conflict check of line 8 on the updated plan: if the repaired route conflicts with another drone, the affected pair is re-planned with CBS on the constraints in force, and only then is the new plan released. The optimality guarantee of Theorem 9.8 holds for the nominal plan; after deviations, the re-check is what preserves conflict-freedom. When more than a dozen drones interact, the global planner is ECBS (Chapter 10) with the same two levels. This chapter is Week 4 of the study plan (Appendix A): its milestone is the solver of Exercise 9.9.

9.10 Summary

Chapter summary

- CBS solves MAPF optimally with two levels: the high level searches a binary *constraint tree* best-first on the sum of costs, the low level plans one agent with space-time A^* under that agent's constraints.
- A node holds constraint sets, one minimum-cost path per agent that respects them, and the sum of the path costs. The root has no constraints. A node whose paths are conflict-free is a goal node.
- A conflict $\langle a_i, a_j, v, t \rangle$ is split into two children, one adding $\langle a_i, v, t \rangle$ and one adding $\langle a_j, v, t \rangle$; an edge conflict (the swapping conflict of Definition 7.4) is split into the two opposite edge constraints. Only the constrained agent is replanned; constraints accumulate down the tree.
- Every solution satisfies at least one of the two constraints of a split (Lemma 9.5), and a node's cost lower-bounds every solution below it (Lemma 9.6); hence the first conflict-free node popped is optimal (Theorem 9.8) and CBS terminates on solvable instances (Theorem 9.9).
- The effort is exponential in the number of conflicts that must be resolved, not directly in the number of agents: CBS excels at bottlenecks and corridors with few alternatives and suffers in open rooms and crowded mazes, where joint-space search can be better.
- The low level must obey the constraints *and* the goal-stay test of Chapter 8, and needs the horizon $T_{\max} = H + 1 + D$; a parked agent still conflicts.
- ICBS splits cardinal conflicts first (identified with MDDs) and bypasses non-cardinal ones; MA-CBS merges agents that conflict repeatedly; CBSH adds admissible high-level heuristics such as the conflict-graph vertex cover; rectangle and corridor reasoning replace one-cell constraints by barrier and range constraints. All keep optimality because they are legal splits.
- In the drone system CBS computes the nominal time-indexed routes before flight, and its conflict check re-validates every repaired route.

Further reading. The original conference paper [149] and the journal version [150] contain the algorithm, the proofs, MA-CBS and the corridor-versus-room analysis. ICBS is due to Boyarski et al. [22], the MDD to the increasing cost tree search [151], high-level heuristics to Felner et al. [40] and Li et al. [102], and symmetry reasoning to Li et al. [103, 104]. The MAPF definitions and benchmark instances follow Stern et al. [161]; the joint-space

baseline with operator decomposition and independence detection is Standley’s [156]. The bounded-suboptimal variants start with Barer et al. [8] and are the subject of Chapter 10.

9.11 Exercises

Exercise 9.1 (★). In node N_2 of Example 9.4 the paths are $\pi_1 = (0, 1), (1, 1), (2, 1), (3, 1), (4, 1)$, $\pi_2 = (2, 0), (2, 1), (2, 2)$ and $\pi_3 = (2, 2), (2, 2), (2, 1), (3, 1), (4, 1), (4, 0)$. List all four conflicts of this node (type, agents, cells, time) and, for each, the two constraints that Equation (9.1) would generate. Which of them does Algorithm 9.1 split, and why?

Exercise 9.2 (★★). Consider the 3×3 grid in which the cells $(0, 1)$ and $(2, 1)$ are blocked, so that the only way from the bottom row to the top row is through the gap $(1, 1)$. Agent a_1 flies from $(0, 0)$ to $(2, 2)$ and a_2 from $(2, 0)$ to $(0, 2)$. Build the constraint tree by hand: give the root paths and cost, the first conflict, the two children with their paths and costs, and the returned plan. How does the size of this tree compare with the number of joint states that A^* in the joint space would have to consider?

Exercise 9.3 (★). In the map of Example 9.4 let a_1 fly from $(0, 1)$ to $(4, 1)$ as before, and let a second agent a_2 fly from $(0, 0)$ to $(4, 0)$, which forces it through the whole horizontal corridor and through a_1 ’s goal cell $(4, 1)$ at $t = 5$, after a_1 has arrived at $t = 4$. Explain why the root has a conflict, what constraint each child receives, and what the low level does with the constraint $\langle a_1, (4, 1), 5 \rangle$ in particular. What would happen if the low level accepted the first arrival at the goal?

Exercise 9.4 (★★). (a) Algorithm 9.1 splits the earliest conflict. Show, using only Lemmas 9.5 and 9.6, that CBS stays optimal and complete if it splits *any* conflict of the node, for instance the last one or a random one. (b) Show that discarding a child whose low-level search fails (line 13) never discards a solution. (c) Give an instance in which the choice of the conflict changes the number of expanded nodes.

Exercise 9.5 (★★). Verify, from the child costs listed in Table 9.1, that every conflict split in Example 9.4 is cardinal. Then draw the MDD of a_1 at the root and explain why the conflict $\langle a_1, a_2, (2, 1), 2 \rangle$ of N_1 is cardinal although a_1 ’s corridor is three cells wide at $x = 2$. Finally compute h_{CG} of Equation (9.3) for N_0 , N_1 and N_2 ; for N_2 you may assume that its three a_1 – a_3 conflicts are cardinal, so that you do not have to run six low-level searches by hand.

Exercise 9.6 (★★). In Figure 9.5 (left), a_1 flies from $(0, 2)$ to $(4, 3)$ and a_2 from $(1, 1)$ to $(3, 4)$ on an obstacle-free 5×5 grid. Count the shortest paths of each agent, and prove that every pair of shortest paths has a vertex conflict. Hint: at time t both agents are on the anti-diagonal $x + y = 2 + t$; compare the x -coordinates. Conclude a lower bound on the optimal sum of costs.

Exercise 9.7 (★★). Suppose the high level orders the constraint tree by the makespan $\max_i \text{cost}(N.\pi_i)$ instead of the sum of costs, with the low level unchanged. Is the returned plan makespan-optimal? Prove your answer by adapting Lemma 9.6 and Theorem 9.8, and name one practical difference from the sum-of-costs version.

Exercise 9.8 (★★). A constraint-tree node in `ch09_cbs.py` stores k paths of up to T cells and k constraint sets. Estimate the memory of a tree with 10^5 generated nodes for $k = 20$ and $T = 60$. Then design a node that stores only the replanned path, the added constraint and a parent pointer, and give the cost of reconstructing the k paths and the constraint sets of a node at depth d .

Exercise 9.9 (★★★ Coding). The Week 4 exercise of the study plan. Implement CBS from scratch for a 4-connected grid: a low level with vertex and edge constraints, the goal-stay test and the horizon; conflict detection with parked agents; and the high level with a heap

keyed on the sum of costs. Reproduce the tree of [Table 9.1](#). Then test it on random 8×8 grids with 2, 4 and 8 agents, report the number of expanded nodes and the running time, and certify optimality on tiny instances against a brute-force search over the joint state space (as `ch09_cbs.py` does). Validate every returned plan independently.

Exercise 9.10 (★★★ Coding). Extend your solver from [Exercise 9.9](#) with cardinal-first splitting and the bypass, first by generating both children of each candidate conflict as `ch09_cbs.py` does, then with MDDs built by two breadth-first searches in space-time. Reproduce [Figure 9.6](#), add a curve for the MDD-based classifier, and report how the number of low-level expansions per constraint-tree node changes.

Bounded-Suboptimal Search: ECBS

Learning objectives

- Explain why optimal CBS stops scaling after a dozen interacting drones and what a suboptimality factor w buys: a plan whose cost is provably at most w times the optimum, found in a small fraction of the time.
- Run focal search by hand: keep OPEN ordered by f , keep FOCAL as the band of open nodes with $f \leq w f_{\min}$, choose inside the band by a second heuristic, and prove that the result is w -suboptimal.
- Trace ECBS on a small instance: a low level that returns a path *and* a lower bound, a high level that runs focal search over the constraint tree, and the lower-bound bookkeeping that ties the two together.
- State and prove the theorem $\text{cost} \leq w C^*$ for ECBS, and explain why the algorithm terminates on solvable instances.
- Implement ECBS in Python, benchmark it against CBS, choose w from the measured speed–quality trade-off, and recognize the three classic implementation mistakes.
- Describe what *EECBS* adds: explicit estimation search and online-learned estimates.

10.1 Why this matters for a drone swarm

Picture an inspection swarm of sixteen drones on a warehouse roof cut into an 8×8 grid of cells, a tenth of them blocked by ventilation units. The operator presses *start* and expects the drones to move within a second. Conflict-based search (CBS) from [Chapter 9](#) produces the cheapest conflict-free plan, but on random instances of exactly this kind, in the experiment of [Section 10.7.3](#), it finds that plan for only two of ten instances within ten seconds. The reason is not a weak implementation. Every conflict that CBS resolves splits the constraint tree in two, and the number of nodes with a cost below the optimum grows exponentially with the number of conflicts that the optimal plan has to untangle. Optimal search pays for a certificate that the operator never asked for: the proof that no cheaper plan exists.

The operator asked for something weaker and far cheaper: a plan that is *good enough*, with a number attached that says how good. This is what a **bounded-suboptimal** algorithm delivers. You choose a factor $w \geq 1$, and the algorithm guarantees a solution whose sum of costs is at most w times the optimal sum C^* , without ever computing C^* . Enhanced CBS (ECBS) by Barer, Sharon, Stern and Felner [8] is the bounded-suboptimal version of CBS. On the same sixteen-drone instances, ECBS with $w = 1.5$ solves all ten in about 23 milliseconds each, and its plans are on average 7% more expensive than the optimum (an average over the two instances CBS could solve), far below the guaranteed 50%. The mechanism behind it,

focal search, is thirty years older than CBS: Pearl and Kim’s A^*_ϵ [127] keeps the ordinary A^* queue as a ruler and lets a second heuristic pick, among the nodes that the ruler declares to be within the bound, the one that seems closest to a solution. In ECBS that second heuristic is the number of conflicts.

For the hybrid architecture of [Chapter 24](#) this chapter settles a design decision: the global planner that produces the nominal, time-indexed paths of the swarm is ECBS rather than CBS as soon as more than a handful of drones interact, because the replanning layer calls it again whenever a drone has been pushed off its route, and a planner that sometimes needs minutes is useless in that loop. The factor w is the knob that trades flight time for planning time, and the lower bound that ECBS computes on the way tells you, after each run, how much of the optimum you gave away. Week 5 of the study plan ([Appendix A](#)) asks you to add ECBS to your CBS solver and to measure exactly this trade-off.

Roadmap. [Sections 10.2](#) and [10.3](#) explain and define focal search and the lower bounds; [Section 10.4](#) gives the low level and the high level of ECBS with walkthroughs; [Section 10.5](#) runs CBS and ECBS on the same instance; [Section 10.6](#) proves the bounds; [Section 10.7](#) covers the choice of w , the relatives of ECBS, a benchmark and EECBS; [Sections 10.8](#) and [10.9](#) give the implementation notes and the place of ECBS in the drone system.

10.2 The idea in plain words

10.2.1 A band instead of a point

A^* ([Chapter 4](#)) always expands the open node with the smallest $f = g + h$. That rule is what makes it optimal, and it is also what makes it stubborn: even when a node with a slightly larger f would finish the search in three steps, A^* first works through every node whose f is smaller. Weighted A^* ([Chapter 4](#)) loosens the rule by inflating the heuristic, $f_w = g + wh$. It is fast, and it is w -suboptimal, but its only notion of “promising” is the inflated h : it cannot use any other information about a node.

Focal search separates the two jobs. OPEN stays exactly as in A^* , ordered by an admissible f . Its smallest value, $f_{\min}$, is a lower bound on the optimal cost C^* , because some node on an optimal path is always open and has $f \leq C^*$ ([Section 10.6](#)). Therefore *every* open node with

$$f(n) \leq w f_{\min} \tag{10.1}$$

can be expanded without breaking the bound: if it is a goal, its cost is at most $w f_{\min} \leq w C^*$. These nodes form the **focal list** FOCAL, a band of width w at the front of OPEN ([Figure 10.1](#)). Inside the band any rule may pick the node to expand. The rule is a second heuristic h_{FOCAL} that need not be admissible and need not even estimate cost: it may estimate the number of steps to a solution, or, in ECBS, the number of conflicts. As long as the choice stays inside the band, the solution is within the factor w of the optimum, whatever h_{FOCAL} says.

10.2.2 Two levels, one bound

CBS has two searches, and both can be made focal. The *low level* plans one drone in space-time under a set of constraints. Its OPEN is ordered by $f = t + h(v)$ with the exact grid distance as h , and its FOCAL prefers partial paths that collide with fewer of the other drones’ current paths. The result is a path that is at most w times longer than the drone’s own constrained optimum and that tends to create fewer conflicts for the high level to resolve. The *high level* searches the constraint tree. Its OPEN is ordered by a lower bound on the cost of the best solution below each node, and its FOCAL prefers nodes whose paths contain fewer conflicts, because a node with no conflicts *is* a solution.

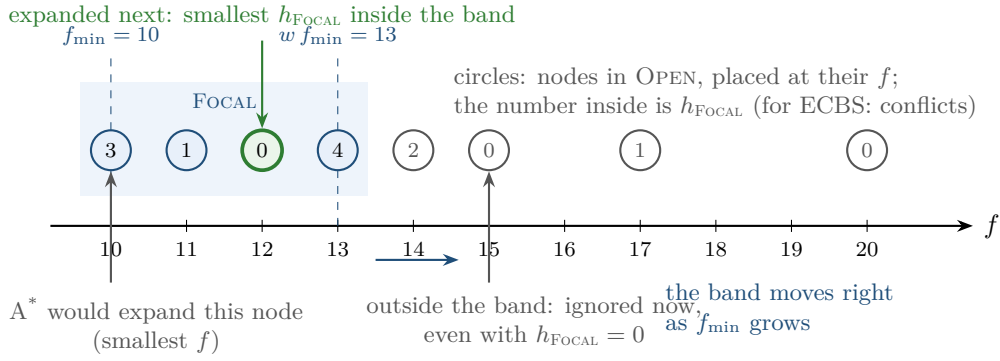


Figure 10.1. Focal search. The open nodes lie on the f axis; the band between $f_{\min}$ and $w f_{\min}$ (here $w = 1.3$) is FOCAL. A^* would expand the leftmost node. Focal search expands the node inside the band with the smallest secondary value h_{FOCAL} , here the node with $f = 12$ and $h_{\text{FOCAL}} = 0$. Nodes outside the band are ignored for now even if their h_{FOCAL} is zero; they enter the band when $f_{\min}$ grows.

The subtle part is the lower bound. In CBS the cost of a node is a lower bound on the cost of every solution below it, because each path is optimal for its constraints. In ECBS the paths are not optimal, so the cost of a node proves nothing. The fix is the piece of bookkeeping that gives the algorithm its correctness: the low level returns, next to the path, the value $f_{\min}$ it saw when it stopped, which is a lower bound on the constrained optimum of that drone. A node stores one such bound per drone, their sum $\text{LB}(N)$ orders OPEN, and the smallest $\text{LB}(N)$ in OPEN, called LB, is a lower bound on C^* . FOCAL is the set of open nodes with $N.\text{cost} \leq w \text{LB}$. When a conflict-free node is taken from FOCAL, its cost is at most $w \text{LB} \leq w C^*$.

Key idea

Keep the admissible ordering of OPEN as a ruler and expand any open node whose f is within the factor w of the smallest one, choosing among them by a second heuristic that need not be admissible. ECBS does this at both levels with the number of conflicts as the second heuristic, and it makes the high-level ruler honest by letting every low-level search return a lower bound together with its path.

10.3 Problem statement and notation

We use the multi-agent path finding (MAPF) notation of Chapter 7. An instance consists of a 4-connected grid with blocked cells, k agents $a_1, \dots, a_k$ with distinct start cells s_i and distinct goal cells γ_i , unit-time move and wait actions, and the stay-at-target convention. A path π_i lists the cell of agent a_i at every time step; $\pi_i[t] = \gamma_i$ for all t after the final arrival, and the cost $\text{cost}(\pi_i)$ is the time of that final arrival. A solution $\Pi = (\pi_1, \dots, \pi_k)$ must be free of vertex conflicts $\langle i, j, v, t \rangle$ and swapping conflicts $\langle i, j, u, v, t \rangle$; its cost is the sum of costs $\text{SoC}(\Pi) = \sum_i \text{cost}(\pi_i)$, and C^* is the smallest cost of any solution. Cells are written (r, c) with row r counted from the top and column c from the left. A vertex constraint $\langle i, v, t \rangle$ forbids agent a_i to be at v at time t ; an edge constraint $\langle i, u, v, t \rangle$ forbids it to move from u to v between t and $t + 1$. A constraint-tree (CT) node N holds a set of constraints $N.\mathcal{C}_i$ for every agent, one path $N.\pi_i$ per agent that respects $N.\mathcal{C}_i$, and the resulting $N.\text{cost} = \sum_i \text{cost}(N.\pi_i)$. This is the setting of CBS in Chapter 9.

Definition 10.1 (Bounded-suboptimal algorithm). Let $w \geq 1$. A solution Π is w -suboptimal if $\text{SoC}(\Pi) \leq w C^*$. An algorithm with parameter w is **bounded-suboptimal**

if, on every solvable instance, it returns a w -suboptimal solution. The number w is the **suboptimality factor**. For $w = 1$ the algorithm is optimal.

The definition asks for a guarantee, not for equality: the returned solution may be much better than $w C^*$, and usually is (Section 10.7.3). Weighted A^* with an admissible heuristic is a bounded-suboptimal algorithm for single-agent search (Chapter 4). So is focal search, which we now define for a general search problem: a finite graph with non-negative edge costs, a start node s , a goal test, and an admissible heuristic h , exactly as in Chapter 4.

Definition 10.2 (Focal search). Let h be admissible, let $f(n) = g(n) + h(n)$, and let $h_{\text{FOCAL}}: V \rightarrow \mathbb{R}$ be any function, the **secondary heuristic**. At every step of a best-first search with open list OPEN, write $f_{\min} = \min_{n \in \text{OPEN}} f(n)$ and

$$\text{FOCAL} = \{n \in \text{OPEN} : f(n) \leq w f_{\min}\}. \quad (10.2)$$

Focal search with factor w expands, at every step, a node of FOCAL with the smallest h_{FOCAL} . Pearl and Kim [127] call it A^*_ε with $w = 1 + \varepsilon$.

Note the two heuristics have different duties. h must be admissible, because $f_{\min}$ has to be a true lower bound; h_{FOCAL} may be anything, because it only chooses inside a band that is safe anyway. Setting $w = 1$ turns FOCAL into the set of nodes with $f = f_{\min}$ and focal search into A^* with h_{FOCAL} as tie-breaker. For ECBS the secondary heuristic is a count of conflicts, at both levels.

Definition 10.3 (Conflict heuristic). For a CT node N , $h_c(N)$ is the number of vertex and swapping conflicts among the paths of N . For a low-level state (v, t) of agent a_i , reached by a partial path from $(s_i, 0)$, $h_c(v, t)$ is the number of conflicts that this partial path has with the current paths of the other agents up to time t ; if (v, t) is a goal state, the conflicts that a_i would have while parked at γ_i from time t on are added.

Definition 10.4 (Lower bounds of a CT node). Let $C_i^*(N)$ be the cost of a cheapest path of agent a_i that respects $N.C_i$ (ignoring the other agents). A **per-agent lower bound** is a number $\text{LB}_i(N) \leq C_i^*(N)$ stored in N . The **lower bound of the node** is $\text{LB}(N) = \sum_{i=1}^k \text{LB}_i(N)$, and the **global lower bound** of the high level is

$$\text{LB} = \min_{N \in \text{OPEN}} \text{LB}(N). \quad (10.3)$$

$\text{LB}(N)$ bounds the cost of every solution in the subtree below N , because every such solution respects the constraints of N and the paths of different agents are bounded separately. Section 10.6 shows that $\text{LB} \leq C^*$ at all times, which is what the high-level FOCAL

$$\text{FOCAL} = \{N \in \text{OPEN} : N.\text{cost} \leq w \text{LB}\} \quad (10.4)$$

relies on.

10.4 The algorithm

10.4.1 Focal search in general

As an algorithm, focal search is the A^* graph search of Chapter 4 with one line changed: instead of the open node with the smallest f , it expands the node of FOCAL with the smallest h_{FOCAL} , breaking ties toward the smaller f and then the larger g , so that with $w = 1$ it degenerates to A^* with the tie-breaking of Chapter 4. Everything else stays. The goal test happens when a node is *chosen*, never when it is generated, because the proof of the bound in Section 10.6 argues about the moment of selection. Successors are relaxed as in A^* , and a closed node is re-opened when a cheaper path to it appears; re-opening is needed in general

even with a consistent heuristic, because focal search expands nodes out of f order and may expand a node before its g is optimal. One property of A^* survives: with a consistent h , $f_{\min}$ never decreases. Whichever node n is expanded, every successor n' it pushes, including a re-opened one, satisfies

$$f(n') = g(n) + c(n, n') + h(n') \geq g(n) + h(n) = f(n) \geq f_{\min},$$

so no node with an f below the current minimum is ever added to OPEN, and the minimum over OPEN can only rise. Section 10.8 shows how this makes FOCAL cheap to maintain.

10.4.2 The low level: focal search in space-time

The low level of ECBS is the space-time A^* of Chapter 4 with the choice rule of Definition 10.2. A state is a pair (v, t) ; the cost of reaching it is $g = t$; the heuristic $h(v)$ is the exact grid distance from v to the goal, computed once by a backward breadth-first search and cached for all time layers and all constrained re-runs. This heuristic is consistent and ignores the other agents, so it never overestimates. Successors are the wait action $(v, t + 1)$ and the moves $(v', t + 1)$ to free neighbors, minus those forbidden by a vertex or edge constraint. Two details of Chapter 4 carry over. The goal test accepts (γ_i, t) only if t is later than every vertex constraint on γ_i , written $t > t_{\gamma}$, so that the agent can stay at its goal for ever. And the search is cut at a time horizon $T_{\max} = \lfloor w(H + 1 + D) \rfloor$, where H is the largest time occurring in a constraint of $\mathcal{C}_i$, or -1 if $\mathcal{C}_i$ is empty, and $D = \max_v h(v)$ over the free cells v . With $w = 1$ this is the horizon of Chapter 4, which keeps the search finite and does not exclude any optimal constrained path; the factor w stretches it so that the band is cut off by the bound and not by the horizon.

Nothing else is new, so Algorithm 10.1 lists only the lines in which the low level departs from the canonical space-time A^* printed in full as Algorithm 4.2: the pop of line 7 becomes a choice from the band, the goal test of line 9 also returns the bound, the horizon of line 11 is stretched by w , and the push of line 18 carries a conflict count; the filter of line 14 reads the constraint set $\mathcal{C}_i$ exactly as in Algorithm 9.2.

Walkthrough. Line 3 prepares the heuristic; in Chapter 9 you saw that the same table serves every low-level call for the same agent. The band at line 6 and the choice at line 7 are focal search (Definition 10.2) with $h_{\text{FOCAL}} = h_c$. Line 8 is the addition that makes ECBS work: the search returns the current $f_{\min}$ together with the path. At that moment the goal state itself is still counted in OPEN, so $f_{\min} \leq f(\gamma_i, t) = t$; Section 10.6 shows that $f_{\min}$ is also a lower bound on the optimal constrained cost, and that $t \leq w f_{\min}$ because the goal was taken from the band. The duplicate test at line 9 is inherited unchanged, and it is worth restating here because a focal search invites the opposite. In space-time with unit costs, every path to $(v', t + 1)$ costs exactly $t + 1$, so a second visit can never be cheaper and re-opening is never needed; the first path to a state wins, even if a later path to the same state would have fewer conflicts. This simplification is standard, and Section 10.5 shows one of its consequences. Line 10 counts the conflicts that the step adds: the other agents that occupy v' at time $t + 1$ (a vertex conflict) and those that move from v' to v while we move from v to v' (a swapping conflict); under stay-at-target, $\pi_j[t]$ is the goal of a_j for every t after its arrival. Line 11 makes the count of a goal state exact: an agent that parks at its goal from time $t + 1$ on collides with every later visit of another agent to that cell, and the high level must know about these conflicts too. Line 12 pushes the successor into OPEN; whether it also belongs to FOCAL follows from Equation (10.2) and is decided by the implementation as described in Section 10.8.

Example 10.5 (The low level on a 3×4 grid). Figure 10.2 shows a 3×4 grid without obstacles. Agent B's path is fixed: $(0, 1) \rightarrow (1, 1) \rightarrow (2, 1)$, where it parks. Agent A starts

Algorithm 10.1. The ECBS low level as a delta on space-time A^* : the ten lines by which [Algorithm 4.2](#) becomes a focal search that also returns a lower bound

Input: agent a_i with start s_i and goal γ_i , constraint set $\mathcal{C}_i$, current paths Π_{-i} of the other agents, factor $w \geq 1$

Output: a path π_i respecting $\mathcal{C}_i$ and a lower bound lb_i with $lb_i \leq C_i^*(\mathcal{C}_i)$ and $\text{cost}(\pi_i) \leq w lb_i$; or *failure*

```

1 Function FocalSpaceTimeAStar( $a_i, \mathcal{C}_i, \Pi_{-i}, w$ ):
2   run SpaceTimeAStar( $G, s_i, \gamma_i, \mathcal{C}_i, T_{\max}$ ) of Algorithm 4.2, with the constraint filter
   and the  $t_\gamma$  of Algorithm 9.2 — states  $(v, t)$ ,  $g(v, t) = t$ ,  $f(v, t) = t + h(v)$ , wait
   action first, one entry per generated state — with the following changes
3    $h(v) \leftarrow$  grid distance from  $v$  to  $\gamma_i$  for all free cells  $v$  (backward BFS, cached across
   every low-level call for  $a_i$ )
4   horizon:  $T_{\max} \leftarrow \lfloor w(H + 1 + D) \rfloor$ , with  $H$  the largest time in  $\mathcal{C}_i$  (or  $-1$  if  $\mathcal{C}_i = \emptyset$ )
   and  $D = \max_v h(v)$  over the free cells
5   conflict count:  $h_c(s_i, 0) \leftarrow$  conflicts of  $(s_i, 0)$  with  $\Pi_{-i}$ ; every generated state carries
   its own  $h_c$  alongside its  $f$ 
6   band:  $f_{\min} \leftarrow \min_{n \in \text{OPEN}} f(n)$ ; FOCAL  $\leftarrow \{n \in \text{OPEN} : f(n) \leq w f_{\min}\}$ 
7   pop:  $(v, t) \leftarrow \arg \min_{n \in \text{FOCAL}} h_c(n)$  (ties: smaller  $f$ , then larger  $t$ ), not the smallest
   key of OPEN
8   goal test: return (path to  $(v, t)$ ,  $f_{\min}$ ) — the bound leaves with the path
9   no re-opening, not even for fewer conflicts: the duplicate test of Algorithm 4.2
   stands; the first path to  $(v', t + 1)$  wins
10  successor:
    $h_c(v', t + 1) \leftarrow h_c(v, t) + |\{j : \pi_j[t + 1] = v'\}| + |\{j : \pi_j[t] = v', \pi_j[t + 1] = v\}|$ 
11  parked goal: if  $v' = \gamma_i$  and  $t + 1 > t_\gamma$  then
    $h_c(v', t + 1) \leftarrow h_c(v', t + 1) + |\{(j, t') : t' > t + 1, \pi_j[t'] = \gamma_i\}|$ 
12  push:  $(v', t + 1)$  enters OPEN with  $f(v', t + 1) = t + 1 + h(v')$ , its  $h_c$  and
   parent( $v', t + 1$ )  $\leftarrow (v, t)$ 

```

at $(1, 0)$ with goal $(1, 3)$, so $h(1, 0) = 3$ and $f_{\min} = 3$ at the start. Run [Algorithm 10.1](#) for A with $w = 1.5$; the instance is `low_level_example()` in `ch10_ecbs.py` and the numbers are checked by its self-test. In step 1 the start is expanded. Its successor $(1, 1)$ at $t = 1$ has $f = 3$ but $h_c = 1$, because B is there at $t = 1$; the wait $(1, 0)$ at $t = 1$ has $f = 4 \leq 1.5 \cdot 3$ and $h_c = 0$; the moves to $(0, 0)$ and $(2, 0)$ have $f = 5 > 4.5$ and stay outside the band. Step 2 expands the wait, because its h_c is smaller, although its f is larger. From there the search moves along row 1 one step behind B ; every state has $f = 4$ and $h_c = 0$, and in step 5 the goal state $(1, 3)$ at $t = 4$ is taken from the band. The returned path costs 4, has no conflict, and comes with the lower bound $f_{\min} = 3$, which is the cost of the dashed path. Run with $w = 1$, the same code expands $(1, 1)$ at $t = 1$ in step 2 and returns the dashed path with one conflict after three expansions. The threshold between the two behaviors is $w = 4/3$ ([Exercise 10.4](#)).

10.4.3 The high level: focal search over the constraint tree

[Algorithm 10.2](#) is the high level. Compared with CBS, three things change: every node carries per-agent lower bounds, OPEN is ordered by $LB(N)$ instead of $N.\text{cost}$, and the node to expand is chosen from FOCAL by h_c .

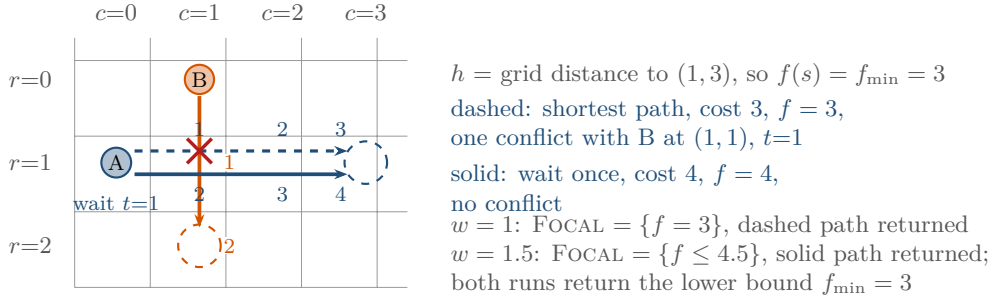


Figure 10.2. The low level on a 3×4 grid. Agent B walks down column 1 and parks at $(2, 1)$; agent A must get from $(1, 0)$ to $(1, 3)$. Small numbers give the time at which a cell is reached; B is at $(1, 1)$ at $t = 1$ and parks at $(2, 1)$ from $t = 2$. The shortest path (dashed) costs 3 but meets B at $(1, 1)$ at $t = 1$. With $w = 1$ the band contains only states with $f = 3$ and the dashed path is returned with one conflict. With $w = 1.5$ the band admits $f \leq 4.5$; the state “wait at $(1, 0)$ until $t = 1$ ” has $f = 4$ and no conflict, so it is expanded first and the solid path of cost 4 is returned. Both runs return the lower bound $f_{\min} = 3$.

Walkthrough. Line 4 builds the root. We plan the agents one after another and hand each low-level call the paths planned so far, so that cheap conflict avoidance happens already at the root: agent a_i steers around $a_1, \dots, a_{i-1}$ where it can within its band, and ignores $a_{i+1}, \dots, a_k$, whose paths do not exist yet. This is prioritized planning (Chapter 8) used as an initialization, but nothing depends on it: the root of plain CBS, with independent shortest paths, would do as well. Line 8 computes the global lower bound Equation (10.3) and the band Equation (10.4); line 9 picks the node with the fewest conflicts inside it. The goal test, with its return at line 11, is the same as in CBS, and the comment states the guarantee proved in Section 10.6. Lines 12 to 19 are the CBS split: the earliest conflict is resolved by one constraint on each of the two agents, and each child re-plans only the constrained agent. The other agents keep their paths and their lower bounds, which remain valid because their constraint sets did not change.

Line 18 is the bookkeeping. The replanned agent’s bound is the larger of two valid lower bounds: the bound it had in the parent, which stays valid because adding a constraint cannot make the constrained optimum cheaper, and the $f_{\min}$ that the new low-level search returned. Barer et al. use the returned value alone; taking the maximum is a safe tightening (Section 10.6) and often lifts the global LB a little earlier. With the maximum of line 18, $\text{LB}(N')$ can be larger than cost of the parent and, unlike the cost, it never decreases along a branch. The cost of a child, on the other hand, may go down: the replanned path is only within w of the agent’s optimum, and the parent’s path may have been a poor one. This is why OPEN must be ordered by $\text{LB}(N)$ and not by cost.

The two rulers. It helps to keep the two bands apart. The low level’s band is $f \leq w f_{\min}$ inside *one* agent’s space-time search; it guarantees $\text{cost}(\pi_a) \leq w \text{lb}_a$ for every path that ever enters a CT node. It holds in every node and not only in the root: an agent that is not replanned inherits its pair (path, bound) unchanged, and for the replanned agent $\text{cost}(\pi) \leq w f_{\min} \leq w \max(N.\text{lb}_a, f_{\min})$, so the maximum taken at line 18 of Algorithm 10.2 preserves it. The high level’s band is $N.\text{cost} \leq w \text{LB}$ over *all* agents and over the whole tree. The low-level guarantee is what keeps the high-level band non-empty: the node $N_{\min}$ that attains LB has $N_{\min}.\text{cost} = \sum_a \text{cost}(\pi_a) \leq w \sum_a \text{lb}_a = w \text{LB}(N_{\min}) = w \text{LB}$, so it always belongs to FOCAL. The high-level guarantee, $\text{cost} \leq w \text{LB} \leq w C^*$, is the one the user sees.

Algorithm 10.2. ECBS: the high level

Input: MAPF instance with agents $a_1, \dots, a_k$, factor $w \geq 1$
Output: a conflict-free solution Π with $\text{SoC}(\Pi) \leq w C^*$, or *failure*

```

1 Function ECBS(instance,  $w$ ):
2    $R.C_i \leftarrow \emptyset$  for  $i = 1, \dots, k$ 
3   foreach  $i = 1, \dots, k$  do
4      $(R.\pi_i, R.lb_i) \leftarrow \text{FocalSpaceTimeAStar}(a_i, \emptyset, (R.\pi_1, \dots, R.\pi_{i-1}), w)$ 
5    $R.\text{cost} \leftarrow \sum_i \text{cost}(R.\pi_i)$ ;  $\text{LB}(R) \leftarrow \sum_i R.lb_i$ ;  $h_c(R) \leftarrow$  number of conflicts among
    $R$ 's paths
6    $\text{OPEN} \leftarrow \{R\}$ 
7   while  $\text{OPEN} \neq \emptyset$  do
8      $\text{LB} \leftarrow \min_{N \in \text{OPEN}} \text{LB}(N)$ ;  $\text{FOCAL} \leftarrow \{N \in \text{OPEN} : N.\text{cost} \leq w \text{LB}\}$ 
9      $N \leftarrow \arg \min_{N' \in \text{FOCAL}} h_c(N')$  (ties: smaller cost, then older node); remove  $N$ 
      from  $\text{OPEN}$ 
10    if  $h_c(N) = 0$  then
11       $\text{return } N$ 's paths //  $N.\text{cost} \leq w \text{LB} \leq w C^*$ 
12     $\chi \leftarrow$  the earliest conflict among  $N$ 's paths, between agents  $a_i$  and  $a_j$ 
13    foreach  $a \in \{a_i, a_j\}$ , with  $\chi_a$  the constraint on  $a$  that resolves  $\chi$  do
14       $N' \leftarrow$  copy of  $N$ ;  $N'.C_a \leftarrow N.C_a \cup \{\chi_a\}$ 
15       $(\pi, lb) \leftarrow \text{FocalSpaceTimeAStar}(a, N'.C_a, \text{paths of the other agents in } N', w)$ 
16      if the low level failed then
17        continue // no path under these constraints: prune
18       $N'.\pi_a \leftarrow \pi$ ;  $N'.lb_a \leftarrow \max(N.lb_a, lb)$ 
19      recompute  $N'.$ cost,  $\text{LB}(N')$  and  $h_c(N')$ ; insert  $N'$  into  $\text{OPEN}$ 
20  return failure

```

10.5 A worked example

Example 10.6 (CBS against ECBS on a 4×4 grid). [Figure 10.3\(a\)](#) shows a 4×4 grid with one blocked cell, (3,2), and three agents: a_1 from (2,2) to (1,1) (grid distance 2), a_2 from (2,3) to (1,0) (distance 4) and a_3 from (3,0) to (1,3) (distance 5). The sum of the distances, 11, is a lower bound on C^* . The instance is `worked_example()` in `ch10_ecbs.py`; in the code the agents are numbered 0, 1, 2, and every number below is produced by its self-test.

The root. Both algorithms start with the same root. Agent a_1 takes the path (2,2), (1,2), (1,1) of cost 2. Agent a_2 , planned against a_1 , walks along row 2 and up: (2,3), (2,2), (2,1), (2,0), (1,0), cost 4, no conflict. Agent a_3 is planned against both. Every path of cost 5 for a_3 meets a_1 or a_2 : the routes through row 2 cross a_2 at (2,1) at $t = 2$, and the routes through row 1 cross the parked a_1 at (1,1). The low level returns (3,0), (2,0), (1,0), (1,1), (1,2), (1,3) with one conflict, $\langle a_1, a_3, (1,1), 3 \rangle$. The root therefore has cost = 11, per-agent bounds (2,4,5), $\text{LB} = 11$ and $h_c = 1$, for CBS ($w = 1$) and for ECBS with $w \leq 1.2$; for $w = 1.5$ the root itself already changes, as the last paragraph of this section shows. With $w = 1.2$ the low level of a_3 may look at states with $f \leq 6$, and a conflict-free path of cost 6 exists: it is the path that CBS finally finds ([Figure 10.3\(b\)](#)). The search does not find it. All conflict-free paths of cost 6 pass through the state (2,1) at $t = 3$, and the search first reaches that state through (2,0) at $t = 2$, where the move (2,0) $\rightarrow$ (2,1) swaps with a_2 ; the conflict-free route through (3,1) arrives at

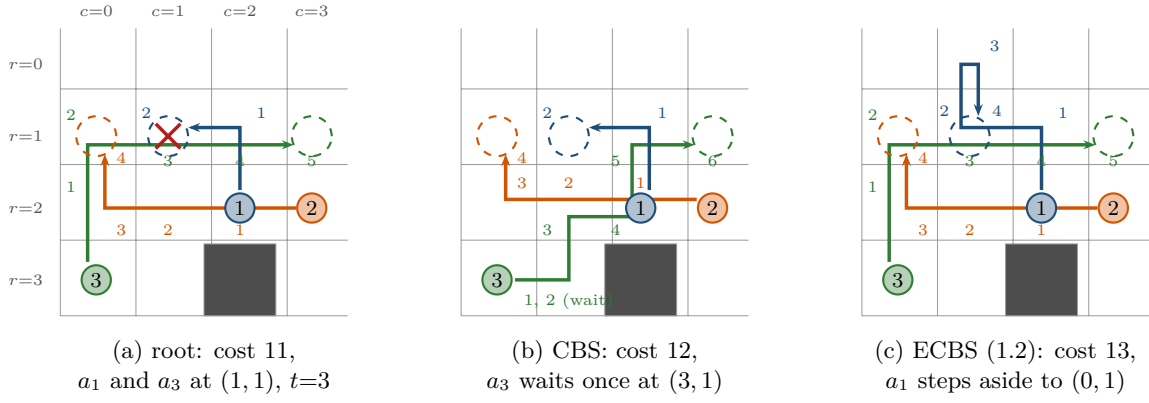


Figure 10.3. Example 10.6. Solid circles are starts, dashed rings goals, small numbers the time at which a cell is reached. (a) The root: three shortest paths of total cost 11; a_1 parks at $(1, 1)$ at $t = 2$ and a_3 walks into it at $t = 3$ (red cross). (b) The optimal solution found by CBS: a_3 waits one step at $(3, 1)$ and passes below the parked a_1 , cost 12. (c) The solution found by ECBS with $w = 1.2$: a_1 reaches its goal, steps aside to $(0, 1)$ while a_3 passes, and returns at $t = 4$, cost 13.

Table 10.1. Trace of CBS on Example 10.6. Children are listed as N_i : cost/ h_c ; a child's constraint is the one drawn on its edge in Figure 10.4(a). OPEN is ordered by cost, then h_c , then age.

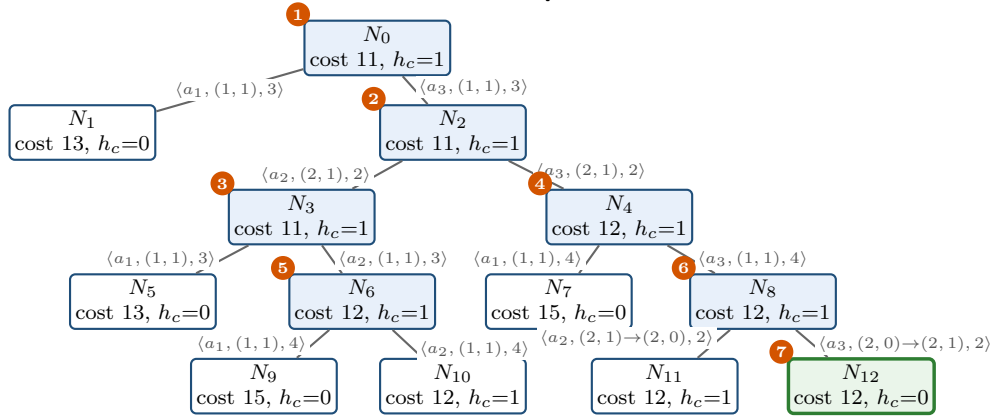
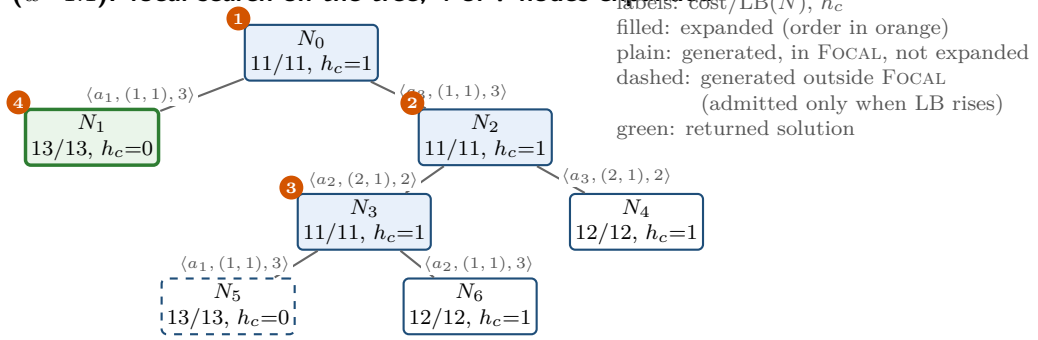
Step	Expanded	cost	h_c	Conflict split	Children generated
1	N_0	11	1	$\langle a_1, a_3, (1, 1), 3 \rangle$	N_1 : 13/0, N_2 : 11/1
2	N_2	11	1	$\langle a_2, a_3, (2, 1), 2 \rangle$	N_3 : 11/1, N_4 : 12/1
3	N_3	11	1	$\langle a_1, a_2, (1, 1), 3 \rangle$	N_5 : 13/0, N_6 : 12/1
4	N_4	12	1	$\langle a_1, a_3, (1, 1), 4 \rangle$	N_7 : 15/0, N_8 : 12/1
5	N_6	12	1	$\langle a_1, a_2, (1, 1), 4 \rangle$	N_9 : 15/0, N_{10} : 12/1
6	N_8	12	1	swap $\langle a_2, a_3, (2, 1), (2, 0), 2 \rangle$	N_{11} : 12/1, N_{12} : 12/0
7	N_{12}	12	0	none: solution of cost 12	

the same state later and is discarded as a duplicate (line 9 of Algorithm 10.1). The FOCAL preference is a heuristic, not a guarantee.

CBS. Table 10.1 traces CBS, which orders OPEN by cost with h_c as tie-breaker, and Figure 10.4(a) draws the tree. The root's conflict is split into N_1 (constraint $\langle a_1, (1, 1), 3 \rangle$) and N_2 ($\langle a_3, (1, 1), 3 \rangle$). In N_1 agent a_1 may not be at its goal at $t = 3$, so under stay-at-target it must arrive at $t = 4$ or later: it goes to $(1, 1)$, steps aside to $(0, 1)$ and comes back, cost 4, and N_1 has cost 13 and *no conflict*. N_1 is a solution, and CBS knows it after a single expansion. But it is not proved optimal, and CBS must expand every node with a smaller cost first: N_2 (cost 11, one conflict), then N_3 (11), then N_4 , N_6 and N_8 (all 12), until N_{12} , of cost 12 and without conflicts, is popped in step 7. Its solution is Figure 10.3(b): a_3 waits at $(3, 1)$ for one step. In total CBS expands 7 nodes, generates 13, and runs 136 low-level state expansions; and N_1 , the solution it found at step 1, is still in OPEN when it stops.

ECBS with $w = 1.1$. Table 10.2 and Figure 10.4(b) show Algorithm 10.2 with $w = 1.1$. After step 1 the global bound is $LB = 11$ and the band is cost ≤ 12.1 : N_2 (cost 11) is in FOCAL, N_1 (cost 13) is in OPEN only. Steps 2 and 3 expand N_2 and N_3 : the band cost ≤ 12.1 admits N_2 , then N_3 and N_4 , all with $h_c = 1$, and the tie-break on cost picks the cost-11 node each time. Their children N_4 and N_6 have $LB = 12$ (the constrained agent's bound rose by one), and N_5 , like N_1 , has cost 13 and $LB = 13$. After step 3, OPEN holds N_1, N_4, N_5, N_6 with

(a) CBS: best-first on the cost; 7 of 13 nodes expanded

(b) ECBS ($w=1.1$): focal search on the tree; 4 of 7 nodes expanded

N_1 (cost 13) stays outside FOCAL while $LB = 11$, since $1.1 \cdot 11 = 12.1 < 13$; N_1 and N_5 are admitted when LB rises to 12 after step 3 ($1.1 \cdot 12 = 13.2$); N_1 is the older and is expanded.

Figure 10.4. The constraint trees of Example 10.6. (a) CBS expands the filled nodes in the orange order and returns N_{12} ; the conflict-free node N_1 of cost 13 is generated in step 1 and never expanded. (b) ECBS with $w = 1.1$ orders OPEN by $LB(N)$ and picks from FOCAL by h_c . N_1 is outside the band until the global bound rises from 11 to 12 after step 3; then $1.1 \cdot 12 = 13.2 \geq 13$ admits it together with N_5 , and of the two conflict-free nodes the older one, N_1 , is expanded.

bounds 13, 12, 13, 12, so LB rises to 12 and the band widens to cost ≤ 13.2 . Now N_1 and N_5 enter FOCAL, both with $h_c = 0$, and step 4 expands the older one, N_1 , and returns its solution of cost 13. The guarantee at that moment is cost $\leq 1.1 \cdot 12 = 13.2$; the true optimum is 12. ECBS expanded 4 nodes, generated 7 and used 61 low-level expansions. Compare the two traces: ECBS performs the same first three expansions as CBS because, while $LB = 11$, every node inside the band still has a conflict and the tie-break on cost selects the cheapest one, exactly what CBS does; and it then stops the moment the bound allows it.

Other values of w . Table 10.3 repeats the run for several factors. With $w = 1.05$ the band cost ≤ 11.55 never contains a node of cost 12 or 13 until LB has reached 12, at which point $1.05 \cdot 12 = 12.6$ admits the cost-12 nodes but not N_1 ; the run is identical to CBS. With $w = 1.2$ the band is cost ≤ 13.2 from the start, N_1 enters FOCAL as soon as it is generated, has $h_c = 0$, and is expanded in step 2: two expansions, cost 13, proven bound $1.2 \cdot 11 = 13.2$. With $w = 1.5$ the run ends at the root: the band of a_3 's low level now reaches $f \leq 7.5$, its horizon $\lfloor 1.5 \cdot 5 \rfloor = 7$ (no constraints, so $H = -1$ and $T_{\max} = \lfloor wD \rfloor$ with $D = 5$) admits the conflict-free detour through row 0, (3, 0), (2, 0), (1, 0), (0, 0), (0, 1), (0, 2), (1, 2), (1, 3) of cost 7, and the conflict count prefers it to every cheaper path. The root has cost $2 + 4 + 7 = 13$ and no conflict, and it is returned after a single expansion with the bound $1.5 \cdot 11 = 16.5$: the same

Table 10.2. Trace of ECBS with $w = 1.1$ on [Example 10.6](#). Children are listed as N_i : cost/LB(N_i)/ h_c , followed by F if the child enters FOCAL at once and O if it waits in OPEN. The global LB and the band w LB are those in force when the node is chosen.

Step	Expanded	cost	LB(N)	h_c	LB	w LB	Children generated
1	N_0	11	11	1	11	12.1	N_1 : 13/13/0 O, N_2 : 11/11/1 F
2	N_2	11	11	1	11	12.1	N_3 : 11/11/1 F, N_4 : 12/12/1 F
3	N_3	11	11	1	11	12.1	N_5 : 13/13/0 O, N_6 : 12/12/1 F
4	N_1	13	13	0	12	13.2	none: solution of cost 13, bound 13.2

Table 10.3. CBS and ECBS on [Example 10.6](#) for several factors w : cost of the returned solution, constraint-tree nodes expanded and generated, global lower bound LB at termination, proven bound w LB, and low-level state expansions. The optimum is 12.

Algorithm	cost	CT expanded	CT generated	LB	bound w LB	low-level exp.
CBS	12	7	13	12	12 (optimal)	136
ECBS $w = 1.05$	12	7	13	12	12.6	136
ECBS $w = 1.1$	13	4	7	12	13.2	61
ECBS $w = 1.2$	13	2	3	11	13.2	37
ECBS $w = 1.5$	13	1	1	11	16.5	20

cost as with $w = 1.2$, reached by a different plan with less work, but with a looser certificate. The table makes the trade-off of the chapter concrete: on this instance a factor $w \geq 13/11$ cuts the constraint tree from 7 to 2 expansions, or to 1 with $w = 1.5$, and the low-level work from 136 to 37 or 20 expansions, at the price of one extra time step in the plan, and the proven bound tells the user honestly that the solution might be up to 13.2, or 16.5, without ever computing the optimum. It also shows that the returned solution can be worse than necessary: a cost-12 solution lies inside every band with $w \geq 12/11$, but the conflict count steered the search to the cost-13 node first.

10.6 Properties: bound, completeness, complexity

Throughout this section graphs are finite, costs are non-negative, $w \geq 1$ and h is admissible. The single-agent results are those of Pearl and Kim [127]; the ECBS results follow Barer et al. [8].

10.6.1 Focal search is w -suboptimal

The proof rests on the same invariant as the optimality proof of A^* in [Chapter 4](#). We restate it because its proof does not use the choice rule at all, which is exactly what focal search needs.

Lemma 10.7 (An optimal-path node is always open). *Consider any search that relaxes edges and re-opens closed nodes exactly as A^* does in [Chapter 4](#), whatever rule it uses to choose the node to expand. Let $P = (s = n_0, n_1, \dots, n_m)$ be a cheapest path from s to a node n_m . At any time before n_m is expanded with $g(n_m) = g^*(n_m)$, OPEN contains a node n_i of P with $g(n_i) = g^*(n_i)$.*

Proof. Let i be the smallest index such that n_i has not yet been expanded while holding $g(n_i) = g^*(n_i)$; it exists because n_m qualifies. If $i = 0$, the start has not been expanded and is still open with $g(s) = 0$. Otherwise n_{i-1} was expanded with its optimal value, and the relaxation of the edge (n_{i-1}, n_i) set $g(n_i) \leq g^*(n_{i-1}) + c(n_{i-1}, n_i) = g^*(n_i)$; since every

g value is the cost of a real path, equality holds from then on. The push that established this value put n_i into OPEN, re-opening it if necessary, and by the choice of i it has not been expanded since. So $n_i \in \text{OPEN}$. $\square$

Theorem 10.8 (Bounded suboptimality of focal search; Pearl and Kim 1982). *If h is admissible and a goal is reachable from s , then focal search (Definition 10.2, with re-opening) terminates and returns a path of cost at most wC^* , for every secondary heuristic h_{FOCAL} .*

Proof. Termination is as for A^* : every push strictly decreases the g of some node, the parent pointers always describe a simple path, and a finite graph has finitely many simple paths, so there are finitely many pushes; and OPEN is not empty until a goal is expanded, by Lemma 10.7 applied to an optimal solution path. Now let γ' be the goal that the search selects from FOCAL, and consider that moment. Since γ' is a goal, admissibility and $h \geq 0$ give $h(\gamma') = 0$, and since γ' was chosen from FOCAL,

$$\text{cost}(\text{returned path}) = g(\gamma') = f(\gamma') \leq w f_{\min}.$$

Let $P = (s, \dots, \gamma)$ be an optimal solution, of cost C^* . If $\gamma' = \gamma$ with $g(\gamma) = C^*$ we are done. Otherwise Lemma 10.7 gives a node $n_i \in \text{OPEN}$ on P with $g(n_i) = g^*(n_i)$, and admissibility gives $h(n_i) \leq h^*(n_i) \leq C^* - g^*(n_i)$, hence $f(n_i) \leq C^*$. Since $f_{\min}$ is the minimum over OPEN, $f_{\min} \leq f(n_i) \leq C^*$. Together, $g(\gamma') \leq wC^*$. $\square$

The proof used h_{FOCAL} nowhere and admissibility of h in one place, to bound $f_{\min}$ by C^* . This is the whole theory of focal search: the ruler must be honest, the choice inside the band is free. Weighted A^* (Chapter 4) can be read as a focal search whose choice rule happens to be $g + wh$; the advantage of the explicit band is that any information may drive the choice.

10.6.2 The low level returns a lower bound

Lemma 10.9 (Lower bound from the low level). *Suppose Algorithm 10.1 returns the path to (γ_i, t) together with $f_{\min}$, and suppose a path respecting $\mathcal{C}_i$ exists within the horizon $T_{\max}$, with optimal cost $C_i^*(\mathcal{C}_i)$. Then*

$$f_{\min} \leq C_i^*(\mathcal{C}_i) \leq t \leq w f_{\min}.$$

Proof. In space-time with unit costs every path to a state (v, t') costs exactly t' , so every generated state carries its optimal g and Lemma 10.7 applies although Algorithm 10.1 never re-opens a state. Let P^* be an optimal path respecting $\mathcal{C}_i$; it ends in a goal state (γ_i, t^*) with $t^* = C_i^*(\mathcal{C}_i) > t_\gamma$ and $t^* \leq T_{\max}$. Every state of P^* passes the constraint tests of Algorithm 10.1, so the successor generation never blocks P^* . At the moment (γ_i, t) is selected, either it is the last state of P^* and $t = t^*$, or by Lemma 10.7 some state (v_j, j) of P^* is open with $f(v_j, j) = j + h(v_j) \leq j + (t^* - j) = t^*$, because the grid distance $h(v_j)$ is at most the number of steps that P^* still needs. In both cases $f_{\min} \leq t^*$. Next, $t^* \leq t$ because the returned path respects $\mathcal{C}_i$. Finally, the goal state was still in OPEN when $f_{\min}$ was taken and it was chosen from FOCAL, so $f_{\min} \leq f(\gamma_i, t) = t \leq w f_{\min}$. $\square$

10.6.3 ECBS is w -suboptimal

Fix, for the rest of the section, one optimal solution $\Pi^* = (\pi_1^*, \dots, \pi_k^*)$ of cost C^* . Call a CT node N **consistent with** Π^* if π_i^* respects $N.\mathcal{C}_i$ for every agent a_i .

Two horizon facts are used below. By Proposition 4.18, if any path respecting $\mathcal{C}_i$ exists at all then one exists that arrives no later than $H + 1 + D \leq \lfloor w(H + 1 + D) \rfloor = T_{\max}$, the horizon of Algorithm 10.1. The H of Chapter 4 also counts the arrival times of parked agents

and uses $t + 1$ for an edge constraint; here no cell of the low-level graph is blocked by another agent, and an edge constraint $\langle i, u, v, t \rangle$ only forbids a move that departs at time t , so from time $H + 1$ on the agent moves freely and the argument of [Proposition 4.18](#) gives arrival by $H + 1 + D$ with the H of [Algorithm 10.1](#). Hence (i) the low-level call of a consistent node never fails, and (ii) the constrained optimum $C_i^*(\mathcal{C}_i)$ is attained within the horizon, so [Lemma 10.9](#) applies. Note that π_i^* itself may be *longer* than $T_{\max}$, because it may wait for other agents to pass; we never need the low level to find π_i^* , only to bound $C_i^*(\mathcal{C}_i) \leq \text{cost}(\pi_i^*)$.

Lemma 10.10 (A consistent node is always open). *At every moment of [Algorithm 10.2](#) before it returns, OPEN contains a node consistent with Π^* .*

Proof. The root has no constraints and is consistent. Suppose the invariant holds and the node removed at line 9 is the consistent one, N ; if it is another node the invariant is untouched. N has a conflict, otherwise the algorithm would have returned. Let it be a vertex conflict $\langle i, j, v, t \rangle$. Since Π^* is conflict-free, π_i^* and π_j^* are not both at v at time t , so at least one of the constraints $\langle i, v, t \rangle$, $\langle j, v, t \rangle$ is respected by Π^* ; for a swapping conflict the same holds for the two edge constraints. The child N' carrying that constraint is consistent with Π^* . Its low-level call at line 15 cannot fail: π_a^* respects $N'.C_a$, so a path exists, and by (i) one exists within the horizon, so [Algorithm 10.1](#) finds one (it only stops with failure when OPEN is empty, which by [Lemma 10.7](#) cannot happen before a goal state is expanded). Hence N' is inserted into OPEN at line 19. $\square$

Theorem 10.11 (Bounded suboptimality of ECBS; Barer et al. 2014). *On a solvable instance, if [Algorithm 10.2](#) returns a solution Π , then $\text{SoC}(\Pi) \leq w C^*$. Moreover, at every moment $\text{LB} \leq C^*$, so the algorithm also returns a valid lower bound on the optimum.*

Proof. First we show that every per-agent bound of a consistent node N satisfies $\text{LB}_i(N) \leq \text{cost}(\pi_i^*)$. The bound $\text{LB}_i(N)$ is the maximum of values $f_{\min}$ returned by low-level searches for agent a_i under constraint sets $\mathcal{C} \subseteq N.C_i$: the set of N itself if a_i was replanned in N , and the sets of ancestors of N otherwise, which are subsets because constraints are only ever added along a branch. For each such set, π_i^* respects $\mathcal{C}$, so $C_i^*(\mathcal{C}) \leq \text{cost}(\pi_i^*)$, and [Lemma 10.9](#) gives $f_{\min} \leq C_i^*(\mathcal{C}) \leq \text{cost}(\pi_i^*)$. The maximum of numbers below $\text{cost}(\pi_i^*)$ is below $\text{cost}(\pi_i^*)$. Summing over the agents, $\text{LB}(N) \leq \sum_i \text{cost}(\pi_i^*) = C^*$ for every consistent node N .

Now consider any moment of the algorithm. By [Lemma 10.10](#) some consistent node N_c is in OPEN, and $\text{LB} = \min_{N \in \text{OPEN}} \text{LB}(N) \leq \text{LB}(N_c) \leq C^*$. In particular, when the solution node N is chosen at line 9, it belongs to FOCAL, so $N.\text{cost} \leq w \text{LB} \leq w C^*$. $\square$

The proof explains every piece of bookkeeping. The low level must return a lower bound under *its own* constraint set, which is what [Lemma 10.9](#) provides. Inherited bounds stay valid because a child has more constraints than its parent. Taking the maximum at line 18 is safe because both numbers bound the same quantity. And the band must be measured against the *global* LB: a node's own $\text{LB}(N)$ may exceed C^* whenever N carries a constraint that the optimal solution violates, so a test of the form $N.\text{cost} \leq w \text{LB}(N)$ proves nothing ([Exercise 10.5](#)).

10.6.4 Termination and completeness

Proposition 10.12 (Termination on solvable instances). *On a solvable instance, [Algorithm 10.2](#) terminates and returns a solution.*

Proof sketch. By [Lemma 10.10](#), OPEN is never empty before a solution is returned, and FOCAL is never empty when OPEN is not, because the node attaining LB lies in the band ([Section 10.4.3](#)). So the loop keeps expanding nodes, and it remains to show that it cannot expand infinitely many. Every expanded node N was in FOCAL when it was chosen, so $N.\text{cost} \leq w \text{LB} \leq w C^*$ at that time. A conflict in N happens at a time t no later than the

makespan of N 's paths, which is at most N . cost; hence every constraint that the algorithm ever creates has $t \leq w C^*$. There are finitely many constraints with $t \leq w C^*$ on a finite grid. Along a branch no constraint is created twice, because once $\langle a, v, t \rangle$ is in $N.C_a$, every path of a in the subtree of N respects it and the corresponding conflict cannot recur. So every expanded node is a set of constraints drawn from a finite universe without repetition along its branch, the tree of expanded nodes has bounded depth and binary branching, and it is finite. $\square$

As for CBS, nothing is claimed for unsolvable instances: the algorithm may run for ever. In practice one imposes a node or time limit, and since LB is valid at every moment (Theorem 10.11), a run that is cut off still reports a lower bound on C^* .

10.6.5 Complexity

The worst case of ECBS is the worst case of CBS: the constraint tree can have a number of nodes exponential in the number of conflicts that must be resolved, and each node costs one or two low-level searches over at most $|V| T_{\max}$ space-time states plus a conflict count in $\mathcal{O}(k^2 T)$ time, or $\mathcal{O}(kT)$ with hashing as in Chapter 7. (Appendix B.6 reviews the asymptotic notation these bounds are written in.) What changes is not the bound but the typical case. Focal search at the high level heads for conflict-free nodes instead of proving that no cheaper node exists, and focal search at the low level produces paths with fewer conflicts, so fewer splits are needed in the first place. The experiment of Section 10.7.3 quantifies the effect: at fourteen agents the mean number of expanded CT nodes drops from about 1 150 for CBS to fewer than six for ECBS with $w = 1.5$.

10.7 Variants and extensions

10.7.1 Choosing w

The factor w has three effects. It widens both bands, so the search becomes more greedy and expands fewer nodes. It loosens the guarantee, which is $w C^*$ in the worst case. And it changes the *proven* bound of a run, $w \text{LB}$, the number you can report to the operator. The actual solution quality moves much less than the guarantee: in the benchmark of Section 10.7.3 the plans of ECBS with $w = 2$ are on average 1–12.5 % more expensive than optimal (Figure 10.5) and, over all instances of that run, never more than 28 %, while the guarantee allows 100 %, because the conflict count steers the search toward solutions and not toward expensive ones; the band permits detours, it does not encourage them. The benchmark samples only $w \in \{1.1, 1.5, 2\}$, but between those three points the runtime drops steeply first and then flattens, which fits the usual picture: a band that admits a couple of extra wait steps already lets the search escape most of the CBS tree, and widening it further buys little. Where exactly the knee lies is instance-dependent; Exercise 10.7(b) asks you to locate it by sweeping w on your own instances. Three rules of thumb follow. Start at $w = 1.1$ for a swarm whose plan must be tight, and go up to 1.5 if the planner still times out. Do not go far beyond 2: the solutions degrade, the guarantee becomes meaningless, and the flight time of the swarm pays. And look at cost/LB after each run: close to 1 means the plan is nearly optimal whatever w was; close to w means the band was fully used and a smaller w would probably buy a better plan. A value slightly above 1 is not free either: with $w = 1.1$ the search sometimes expands *more* nodes than CBS on easy instances, because it accepts suboptimal paths that create new conflicts (Table 10.5, eight agents).

Table 10.4. Suboptimal relatives of CBS. “Focal” means focal search with the number of conflicts as secondary heuristic.

Algorithm	Low level	High level: OPEN by	High level: choice	Guarantee
CBS	A^*	cost	smallest cost	optimal
GCBS	greedy on h_c	single queue	fewest conflicts	none
BCBS(w_H, w_L)	focal, w_L	cost	focal, w_H	$w_H w_L C^*$
ECBS(w)	focal, w , returns lb	LB(N)	focal, w	$w C^*$
EECBS(w)	focal, w , returns lb	LB(N) and $\hat{f}(N)$	explicit estimation	$w C^*$

10.7.2 The family of suboptimal CBS variants

Barer et al. [8] studied several ways of making CBS suboptimal before arriving at ECBS; Table 10.4 lists them together with EECBS. *Greedy CBS* (GCBS) orders both levels purely by conflicts. It is fast but offers no bound. *Bounded CBS* BCBS(w_H, w_L) runs focal search with factor w_L at the low level and focal search with factor w_H at the high level, but the high level orders OPEN by *cost* and uses the band $N.\text{cost} \leq w_H \cdot \min_{N' \in \text{OPEN}} N'.\text{cost}$, without lower bounds. Because the smallest cost in OPEN is not a lower bound on C^* when the low level is suboptimal, the guarantee is only $w_H w_L C^*$ (Exercise 10.6); BCBS($w, 1$) is focal search at the high level only, BCBS($1, w$) at the low level only. ECBS improves on both by carrying the lower bounds: one factor w at both levels, and a guarantee of w , not w^2 .

10.7.3 A benchmark against CBS

Figure 10.5 and Table 10.5 compare CBS with ECBS for $w \in \{1.1, 1.5, 2.0\}$ on random 8×8 grids with 10 % blocked cells, ten instances for each even number of agents k from 2 to 16, each run limited to ten seconds and 50 000 CT expansions (script `gen_ch10_benchmark.py`; all solvers are the implementation of Section 10.8, so the comparison measures the algorithms and not the code). Up to eight agents every solver answers within a few milliseconds. From ten agents on, CBS meets instances with hundreds of conflicts to untangle: at $k = 14$ it solves six of ten instances within the limit, needing on average 1 153 CT expansions and about 2 seconds for those it solves, and at $k = 16$ it solves two. ECBS with $w = 1.5$ solves every instance, at $k = 14$ in 11 milliseconds and 5.7 expansions on average, and its plans cost 6.2 % more than the optimum on average (worst case 10.5 %), with a proven bound of 16 % on average. With $w = 2$ the runtime drops a little further, to 7 milliseconds, and the excess cost rises to 12.5 %. With $w = 1.1$ the plans are within 3 % of optimal, but the band is so narrow that the search inherits much of the CBS tree: it solves seven of ten instances at $k = 14$ and four at $k = 16$. The dashed lines in the right panel are the proven bounds cost/LB; they lie between the actual ratio and w , and they are what a planner can report without knowing C^* .

10.7.4 EECBS: explicit estimation and online learning

ECBS orders OPEN by a lower bound and FOCAL by a conflict count, and neither says how expensive the *best solution below a node* will be. Li, Ruml and Koenig [105] replaced the high-level focal search by **explicit estimation search** (EES) of Thayer and Ruml [167] and called the result **EECBS**. Two ideas are involved; we sketch them without the details, which the paper gives.

Explicit estimation. EES keeps three orderings of the open CT nodes: by LB(N), as in ECBS, whose minimum is the global bound LB; by an *inadmissible estimate* $\hat{f}(N)$ of the cost of the best solution in the subtree of N , a best guess rather than a bound; and a focal list

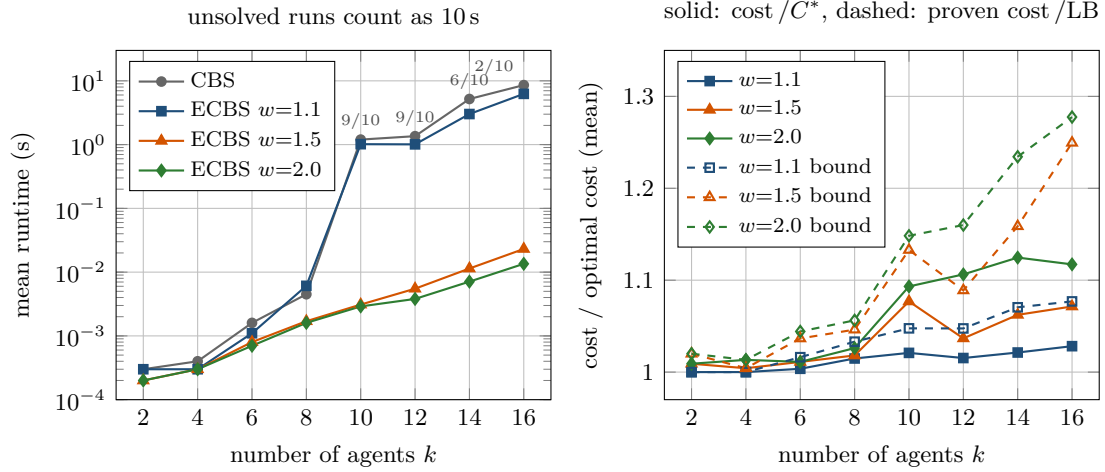


Figure 10.5. CBS against ECBS on random 8×8 grids with 10% obstacles, ten instances per number of agents. Left: mean runtime (log scale); runs that hit the ten-second limit count with ten seconds, so the curves of CBS and *ECBS*(1.1) are lower bounds beyond $k = 10$; the labels on the CBS curve say how many of the ten instances it actually solved, so read the flattening at $k = 16$ as failure and not as speed. Right: mean ratio of the returned cost to the optimal cost over the instances that CBS solved (solid), and the mean proven bound cost/LB over all solved instances (dashed). The guarantees 1.1, 1.5 and 2 are far above the observed ratios.

defined by the estimate, $\text{FOCAL} = \{N : \hat{f}(N) \leq w \hat{f}_{\min}\}$, ordered by the number of conflicts. The selection rule asks, in order: does the node with the fewest conflicts in FOCAL have cost $\leq w \text{LB}$? If so, expand it. (Li et al. state the test on the estimate, $\hat{f}(N) \leq w \text{LB}$; the two agree on the conflict-free nodes that can be returned, where $\hat{f}(N) = N.\text{cost}$. The substitution does change *which* nodes are expanded, however, and not only which ones may be returned, so a solver written from this paragraph is a close relative of EECBS and not EECBS itself.) Otherwise, does the node with the smallest $\hat{f}$? If so, expand it. Otherwise expand the node with the smallest $\text{LB}(N)$, which raises the global bound. Every returned solution still satisfies $\text{cost} \leq w \text{LB} \leq w C^*$, so [Theorem 10.11](#) is untouched; what changes is that a node is judged by where its subtree is expected to end up, not by where it is now.

Online learning of the estimate. EECBS learns $\hat{f}$ during the search: each expansion shows by how much the lower bound of a child exceeds that of its parent and how the number of conflicts changes, and the running averages of these one-step changes, multiplied by the conflicts still present in a node, give the estimate of the cost that resolving them will add. The estimate is inadmissible by design and improves as the search proceeds. EECBS combines this with the CBS improvements of [Chapter 9](#), adapted to the bounded-suboptimal setting: bypassing conflicts, prioritizing cardinal conflicts, symmetry reasoning and admissible high-level heuristics that lift $\text{LB}(N)$ above the sum of the per-agent bounds. In the experiments of Li et al. the combination solves instances with hundreds of agents within a minute where ECBS solves a fraction of them. For swarms of the sizes considered in this book, ECBS as given here is sufficient; EECBS is the upgrade path when it is not.

10.7.5 Other relatives

An anytime ECBS runs [Algorithm 10.2](#) with a decreasing sequence of factors w , in the spirit of ARA^* ([Chapter 6](#)), so that a first plan is available quickly and is improved while the drones are still on the ground. And because the bound never depended on h_{FOCAL} , the conflict count can be changed freely: counting only conflicts inside a time window, or penalizing cells near a

Table 10.5. Speed against quality on the benchmark of Figure 10.5 for 8, 12 and 16 agents. Runtime, CT expansions and low-level expansions are means over the solved instances; cost / C^* is the mean over the instances that CBS solved (two instances at $k = 16$); cost / LB is the mean proven bound.

k	Solver	solved	runtime	CT exp.	low-level exp.	cost / C^*	cost / LB
8	CBS	10/10	4.5 ms	6.9	169	1.000	1.000
8	ECBS $w = 1.1$	10/10	6.1 ms	8.2	255	1.015	1.033
8	ECBS $w = 1.5$	10/10	1.7 ms	2.0	93	1.018	1.046
8	ECBS $w = 2.0$	10/10	1.6 ms	1.8	98	1.026	1.056
12	CBS	9/10	0.40 s	311	8 847	1.000	1.000
12	ECBS $w = 1.1$	9/10	10 ms	6.8	282	1.015	1.048
12	ECBS $w = 1.5$	10/10	5.5 ms	2.9	217	1.037	1.089
12	ECBS $w = 2.0$	10/10	3.8 ms	1.7	190	1.106	1.160
16	CBS	2/10	2.9 s	1 695	33 718	1.000	1.000
16	ECBS $w = 1.1$	4/10	0.62 s	295	17 467	1.028	1.077
16	ECBS $w = 1.5$	10/10	23 ms	7.7	1 018	1.071	1.250
16	ECBS $w = 2.0$	10/10	14 ms	3.8	785	1.117	1.278

predicted intruder, keeps the guarantee. Chapter 11 completes the picture of MAPF solvers with M^* and the push-and-swap family and compares all the methods of Part III.

10.8 Implementation notes

The file `code/ch10_ecbs.py` is a self-contained implementation of everything in this chapter, in about 800 lines: an `Instance` class with cached breadth-first distance tables, conflict detection and counting in the conventions of Chapter 7, constraints, the conflict table `OtherPaths`, the focal low level `focal_space_time_astar`, the high levels `ecbs` and `cbs`, a plan validator, random instances, and the two examples of this chapter. Its self-test runs in well under a second and is described at the end of this section.

10.8.1 Two heaps kept consistent

Algorithm 10.1 recomputes FOCAL at every step, which would cost a scan of OPEN each time. The implementation keeps two binary heaps and makes them agree.

- OPEN is a heap keyed on $(f, -t, \text{tie})$. It serves one purpose only: to read $f_{\min}$. States are never removed from the middle of it; a state that has been expanded is marked closed and is purged lazily when it reaches the top, exactly as the stale entries of Chapter 4.
- FOCAL is a heap keyed on $(h_c, f, -t, \text{tie})$, and a set `in_focal` records which states are in it. A state enters FOCAL either when it is generated, if $f \leq w f_{\min}$ at that moment, or later, when $f_{\min}$ rises.
- *Updating FOCAL when $f_{\min}$ rises.* At the top of each iteration the code purges closed states from OPEN and reads the new minimum. If it is larger than before, the band has widened, and every open state with $f \leq w f_{\min}$ that is not yet in FOCAL is pushed into it. The code scans the whole OPEN heap for them. This looks expensive, but with unit costs $f_{\min}$ takes integer values up to $T_{\max}$ and rises at most that many times, so the scans cost $\mathcal{O}(T_{\max} \cdot |\text{OPEN}|)$ in total; a third heap keyed on f , holding the open states that are not yet in FOCAL, avoids even that and is the usual choice in C++ implementations.

Reading $f_{\min}$ *before* popping from FOCAL is essential: the popped state is still counted, so the returned lower bound satisfies $f_{\min} \leq t$, and [Lemma 10.9](#) holds. The high level uses the same structure with $LB(N)$ in place of f , $N.cost$ in the band test, and expanded node ids in place of closed states; the loop of `ecbs` is the same code pattern as [Listing 10.1](#), the main loop of the low level, with different keys.

Listing 10.1. The main loop of the low level: two heaps, lazy purge of OPEN, widening of the band, goal test with the lower bound, and successor generation with conflict counts (from `code/ch10_ecbs.py`).

```

1  while True:
2      while open_heap and open_heap[0][3] in closed:      # lazy deletion
3          heapq.heappop(open_heap)
4      if not open_heap:
5          return None
6      if open_heap[0][0] > f_min:                          # f_min rose:
7          f_min = open_heap[0][0]                        # admit new states
8          for f, neg_t, tie, st in open_heap:             # to FOCAL
9              if f <= w * f_min and st not in in_focal and st not in closed:
10                 heapq.heappush(focal, (nc[st], f, neg_t, tie, st))
11                 in_focal.add(st)
12             conflicts, f, _, _, state = heapq.heappop(focal)
13             if trace is not None:                       # for trace tables
14                 _snapshot(trace, state, conflicts, f, f_min, focal, open_heap,
closed)
15             closed.add(state)
16             cell, t = state
17             if cell == goal and t > t_goal:
18                 return LowLevelResult(_reconstruct(parent, state), f_min,
conflicts, expansions)
19
20             expansions += 1
21             t2 = t + 1
22             if t2 > max_time:
23                 continue
24             for v in [cell] + inst.neighbors(cell):
25                 if (v, t2) in vcons or (v != cell and (cell, v, t) in econs):
26                     continue
27                 st2 = (v, t2)
28                 if st2 in parent:                        # same g = t2: first path wins
29                     continue
30                 parent[st2] = state
31                 c = conflicts + others.vertex_hits(v, t2)
32                 if v != cell:
33                     c += others.swap_hits(cell, v, t)
34                 if v == goal and t2 > t_goal:            # parked at the goal from t2 on
35                     c += others.future_hits(goal, t2)
36                 nc[st2] = c
37                 f2 = t2 + h[v]
38                 tie = next(counter)
39                 heapq.heappush(open_heap, (f2, -t2, tie, st2))
40                 if f2 <= w * f_min:
41                     heapq.heappush(focal, (c, f2, -t2, tie, st2))
42                     in_focal.add(st2)

```

10.8.2 Tie-breaking

Ties matter more in focal search than in A^* , because h_c takes few distinct values. The FOCAL key orders states by conflicts, then by f (cheaper first), then by larger t (deeper first, the rule of [Chapter 4](#) that makes a search dive along one path), then by insertion order. With $w = 1$ every state in FOCAL has $f = f_{\min}$, and the key reduces to conflicts, depth, age: A^* with conflict-avoiding tie-breaking, which is the low level of our CBS baseline. The high-level FOCAL key is $(h_c, \text{cost}, \text{id})$: fewest conflicts, then cheapest, then oldest. OPEN keys carry the id or the tie counter as their last component so that Python never compares two nodes directly.

10.8.3 Bookkeeping of the lower bounds

[Listing 10.2](#) creates a child: it copies the parent, adds one constraint, rebuilds the conflict table of the other agents, calls the low level with the same w , and stores the new path and the new bound. The `max` on the bound is line 18 of [Algorithm 10.2](#). Note that the child's conflict count is recomputed from scratch with the $\mathcal{O}(k^2T)$ routine of [Chapter 7](#); for large k the hashed variant should be used.

Listing 10.2. Creating a child node: one more constraint, one replanned agent, and the lower-bound bookkeeping (from `code/ch10_ecbs.py`).

```

1 def _child(inst: Instance, parent: CTNode, con: Constraint, w: float,
2           stats: _Stats) -> Optional[CTNode]:
3     """Copy the parent, add one constraint and replan the constrained agent.
4
5     The lower bound of the replanned agent is the larger of the parent's
6     bound (still valid: more constraints cannot make the optimum cheaper)
7     and the f_min returned by the low level.
8     """
9     a = con.agent
10    cons = list(parent.constraints)
11    cons[a] = cons[a] | {con}
12    others = OtherPaths([p for i, p in enumerate(parent.paths) if i != a])
13    res = focal_space_time_astar(inst, a, cons[a], others, w)
14    if res is None:
15        return None
16    stats.low_level_expansions += res.expansions
17    stats.generated += 1
18    paths = list(parent.paths)
19    paths[a] = res.path
20    lbs = list(parent.lbs)
21    lbs[a] = max(lbs[a], res.lower_bound)
22    return CTNode(tuple(cons), paths, lbs, sum_of_costs(paths), sum(lbs),
23                  count_conflicts(paths), next(stats.ids), parent.id, con)

```

10.8.4 Practical details

- *The band test in floating point.* $f \leq w f_{\min}$ compares an integer with the product of a float and an integer, and $3 \cdot 1.3333333333333333$ may or may not equal 4. For a factor $w = p/q$ the exact test is $qf \leq pf_{\min}$; the code compares directly and the self-test allows a tolerance of 10^{-9} .
- *Conflict tables and caching.* `OtherPaths` precomputes, for the other agents' paths, who is where at each time, who moves along each directed edge at each time, the parked goal cells and the sorted visiting times of each cell, so that the counts of lines 10 and 11 of

Algorithm 10.1 cost $\mathcal{O}(1)$ or a binary search; the table is rebuilt per child in $\mathcal{O}(kT)$ time. The distance table of a goal is computed once per instance.

- *Duplicates.* The duplicate test at line 9 keeps the first path to a state. Some implementations, among them the public EECBS code, re-insert a state when a path with the same cost but fewer conflicts reaches it; this finds better paths at the price of some re-expansions and does not affect the bound ([Exercise 10.8](#)).
- *Counting and limits.* The code counts CT expansions and generations and low-level expansions, measures wall-clock time, and stops at a node or time limit with the status of the run and the bound LB reached, which is valid even then ([Theorem 10.11](#)).

The self-test. Running `python3 code/ch10_ecbs.py` checks, on random instances, that the conflict count carried by the low level equals the count of the plan validator; that under constraints the returned bound never exceeds the constrained optimum and the path never exceeds w times the bound; that on 45 random 6×6 instances with four agents ECBS with $w = 1$ returns the CBS cost and ECBS with $w \in \{1.1, 1.5, 2\}$ returns valid plans whose cost lies between C^* and $w C^*$ and below w LB; and that the two examples of this chapter reproduce every number quoted. It then prints the trace of [Table 10.1](#) and the traces of ECBS for $w = 1.05, 1.1$ and 1.2 ; the $w = 1.1$ one is [Table 10.2](#).

An inadmissible heuristic in Open

The bound of [Theorem 10.8](#) needs $f_{\min} \leq C^*$, and that is true only if OPEN is ordered by an admissible f . Two mistakes are common. One is to order OPEN by the inflated $g + wh$ of weighted A^* and to use a band on top of it: the band is then measured against an inflated ruler, and the real guarantee is w^2 . The other is to let the conflict count leak into f , for instance by adding a penalty per conflict to the path cost so that the search avoids conflicts. The ordering of OPEN must see the true cost and an admissible estimate of the cost to go, nothing else; every preference belongs in h_{FOCAL} , where it cannot harm the bound.

Forgetting to return or update the lower bound

The high level of ECBS is only correct because the number it orders OPEN by is a lower bound. Three ways to break this: use the cost of the path returned by the low level as its bound (the cost is an *upper* bound on the constrained optimum); return $f_{\min}$ computed after the goal state has been removed from OPEN, or from a heap top that is a stale entry; or replan an agent in a child without storing its new bound, so that the child inherits the parent's cost as if it were a bound. In each case the global LB can exceed C^* , the band admits nodes it should not, and the returned plan can be worse than $w C^*$ while the program happily reports a “proven” bound. The self-test's check $\text{LB} \leq C^*$ against CBS catches all three.

Applying w per agent instead of to the sum

The low level guarantees $\text{cost}(\pi_i) \leq w \text{lb}_i$ for every agent, and it is tempting to conclude that any node whose paths all satisfy this is within the bound. It is not. The guarantee is about the sum of costs and the *global* bound $\text{LB} = \min_{N \in \text{OPEN}} \text{LB}(N)$: a node with $N.\text{cost} \leq w \text{LB}(N)$ may still have $N.\text{cost} > w C^*$, because $\text{LB}(N)$ counts constraints that the optimal solution does not obey (in [Example 10.6](#), N_7 has $\text{LB}(N_7) = 15 > C^* = 12$). Conversely, the sum test may admit a node in which one agent is far above its own bound and another well below; that is allowed and is often where the cheap solutions are. Test

$N.\text{cost} \leq w \text{LB}$ with the global bound, and nothing else.

10.9 Where this fits in the drone system

In the drone system

In the four-layer architecture of [Chapter 24](#), ECBS is the global swarm planner: it turns the starts and goals of the connected swarm members into nominal, time-indexed paths that are free of planned inter-drone conflicts. It is the practical choice over CBS for three reasons. Time: the nominal plan must exist before take-off and must be recomputed whenever the replanning layer (D^* Lite or A^* from the current state, [Chapter 5](#)) has moved a drone off its route, and a planner that needs minutes instead of milliseconds cannot sit in that loop. The guarantee: the operator gets the plan together with LB and $w \text{LB}$, so the cost of the speed-up is measured, not guessed, which is the Week 5 milestone of the study plan ([Appendix A](#)). The knob: one parameter w moves the planner between optimal and greedy.

The bound and the local safety layer are independent. ECBS guarantees a conflict-free plan for every w ; the factor affects cost, never safety. Safety against *unexpected* moving obstacles is the job of the prediction layer ([Chapters 18 and 20](#)) and of the local safety layer (ORCA or DWA, [Chapters 13 and 14](#)), which deviates from the nominal path when a predicted conflict enters the safety horizon and reconnects afterward. A nominal path that is 10 % longer than optimal costs flight time and battery, but it is neither more nor less safe. Hence a modest w between 1.1 and 1.5 is enough, because the local layer perturbs the paths anyway; after a replanning event the new plan must be re-checked for conflicts with the drones that were not replanned, as the decision logic of [Chapter 24](#) prescribes; and mission knowledge belongs in h_c : penalizing cells near the predicted trajectory of an intruder in h_{FOCAL} steers the nominal paths away from it without touching the bound.

10.10 Summary

Chapter summary

- Optimal CBS pays for a proof of optimality that a swarm operator does not need; a bounded-suboptimal planner returns, for a chosen $w \geq 1$, a solution of cost at most $w C^*$ without computing C^* .
- Focal search keeps OPEN ordered by an admissible f , takes $f_{\min}$ as a lower bound on C^* , and expands any node of the band $\text{FOCAL} = \{n : f(n) \leq w f_{\min}\}$, chosen by a secondary heuristic that need not be admissible. The result is w -suboptimal for every secondary heuristic.
- The ECBS low level is focal space-time A^* with the number of conflicts as secondary heuristic; it returns the path and $f_{\min}$, a lower bound on the agent's constrained optimum, with cost $\leq w f_{\min}$.
- The ECBS high level stores one lower bound per agent in each CT node, orders OPEN by their sum $\text{LB}(N)$, and expands from $\text{FOCAL} = \{N : N.\text{cost} \leq w \text{LB}\}$ the node with the fewest conflicts, where LB is the smallest $\text{LB}(N)$ in OPEN. Because $\text{LB} \leq C^*$ at all times, the returned solution costs at most $w C^*$, and LB is a certificate.
- On random 8×8 grids ECBS with $w = 1.5$ solves sixteen-agent instances in tens of

milliseconds where CBS fails within ten seconds, at a cost excess of about 7 %, far below the guarantee.

- The three classic bugs are an inadmissible ordering of OPEN, a missing or stale lower bound, and a band tested against a node's own $LB(N)$ instead of the global LB.
- EECBS replaces the high-level focal search by explicit estimation search with estimates learned online and adds the CBS improvements of [Chapter 9](#); it keeps the bound w and scales to hundreds of agents.

Further reading. Focal search is the A^*_ϵ of Pearl and Kim [127]; Pearl's book [126] treats it together with weighted A^* [132] and the other semi-admissible searches, and Russell and Norvig [142] give the textbook view of bounded suboptimality. ECBS and its siblings BCBS and GCBS are due to Barer, Sharon, Stern and Felner [8], building on CBS [150] and the improvements of ICBS [22]. Explicit estimation search is by Thayer and Ruml [167], and its use in EECBS by Li, Ruml and Koenig [105], whose paper also surveys the high-level heuristics [40] and symmetry reasoning [104] that the bounded-suboptimal setting inherits. The MAPF definitions and the benchmark format used in this chapter are those of Stern et al. [161]; the anytime idea of decreasing w is the one of ARA^* [108].

10.11 Exercises

Exercise 10.1 (★). OPEN contains eight nodes with (f, h_{FOCAL}) equal to (10, 3), (11, 1), (12, 0), (13, 4), (14, 2), (15, 0), (17, 1) and (20, 0), as in [Figure 10.1](#), and $w = 1.3$. (a) List FOCAL and the node that focal search expands. (b) Its expansion removes it and generates two successors with (12, 2) and (16, 0). Which node is expanded next? (c) For which values of w does the node (15, 0) belong to FOCAL, and is it then expanded before (12, 0)? (d) Suppose that later every node with $f \leq 12$ has been expanded and the smallest remaining f is 13, the nodes of (a) and (b) otherwise unchanged. What is the band, which nodes are in FOCAL, and which one is expanded?

Exercise 10.2 (★). Explain in your own words why the $f_{\min}$ returned by [Algorithm 10.1](#) is a lower bound on the agent's constrained optimum while the cost of the returned path is not. In [Example 10.5](#) the runs with $w = 1$ and $w = 1.5$ return paths of cost 3 and 4 but the same bound 3; explain, with the help of the band, why a larger w tends to return a smaller bound for the same agent, and why that bound is still valid. Finally, explain why $LB(N)$ bounds the cost of every solution in the subtree of N but not, in general, C^* .

Exercise 10.3 (★★). Show that ECBS with $w = 1$ is CBS with conflict-based tie-breaking. First show that the low level with $w = 1$ returns an optimal constrained path and a bound equal to its cost. Then show that with $w = 1$ every node in the high-level FOCAL has $N.\text{cost} = LB(N) = LB$, and conclude that the expanded node always has the smallest cost in OPEN, so that the returned solution is optimal.

Exercise 10.4 (★★). In [Example 10.5](#) the state “wait at (1, 0) until $t = 1$ ” has $f = 4$ and $f_{\min} = 3$. (a) For which factors w does the low level return the waiting path, and for which the conflicting one? (b) What does your implementation do for $w = 4/3$ if the band test is computed in floating point, and how do you make the test exact? (c) Now let agent B wait at (1, 1) for two steps, (0, 1), (1, 1), (1, 1), (2, 1). Determine by hand, and check with `focal_space_time_astar`, which path and which bound the low level returns for $w = 1.5$ and for $w = 2$.

Exercise 10.5 (★★). Show with [Example 10.6](#) that a CT node can have $\text{LB}(N) > C^*$. Then explain why a high level that expands the node with the fewest conflicts among those with $N.\text{cost} \leq w \text{LB}(N)$ (the node's own bound) is not w -suboptimal: exhibit, in the tree of [Figure 10.4\(a\)](#), a node that this rule may return for $w = 1.2$ and compute the ratio of its cost to C^* .

Exercise 10.6 (★★★). $\text{BCBS}(w_H, w_L)$ runs focal search with factor w_L at the low level and, at the high level, orders OPEN by $N.\text{cost}$, uses the band $N.\text{cost} \leq w_H \cdot \min_{N' \in \text{OPEN}} N'.\text{cost}$ and picks the node with the fewest conflicts; no lower bounds are stored. Prove that it returns a solution of cost at most $w_H w_L C^*$. Hint: adapt [Lemma 10.10](#), then show that a node consistent with an optimal solution Π^* has $N.\text{cost} \leq w_L C^*$ by bounding each path separately, including the paths inherited from ancestors. Where exactly does the argument for ECBS save the factor w_L ?

Exercise 10.7 (★★★ Coding). This is the Week 5 exercise of the study plan ([Appendix A](#)). Add ECBS to your CBS solver from [Chapter 9](#), or start from `code/ch10_ecbs.py`, and benchmark it against CBS. (a) Generate random 8×8 and 16×16 grids with 10% obstacles and $k = 2, 4, \dots, 20$ agents, ten instances each; run CBS and ECBS with $w \in \{1.1, 1.5, 2\}$ under a time limit; record success rate, runtime, CT expansions, cost/C^* where CBS succeeded and cost/LB always; validate every plan with a conflict checker; plot the curves of [Figure 10.5](#). (b) Fix $k = 12$, vary $w \in \{1, 1.02, 1.05, 1.1, 1.2, 1.5, 2, 3\}$, plot runtime and cost/C^* against w and locate the knee of the runtime curve. (c) Compare the largest cost/C^* you observed for each w with the guarantee. The milestone is reached when you can say, for your own instances, what a given speed-up costs in solution quality.

Exercise 10.8 (★★★ Coding). (a) Implement $\text{BCBS}(w, 1)$ and $\text{BCBS}(1, w)$ by switching the band off at one level of `ch10_ecbs.py`, and compare CT expansions and cost/C^* with $\text{ECBS}(w)$ on the benchmark of [Exercise 10.7](#) for $w = 1.5$. Which level profits more from the band? (b) Change the duplicate rule of the low level so that a state is re-inserted when a path with the same cost but fewer conflicts reaches it, and check whether the root of [Example 10.6](#) with $w = 1.2$ changes. (c) Measure the effect of (b) on the benchmark of [Exercise 10.7](#).

Coupling and Constructive Methods: M^* , Push-and-Swap, Push-and-Rotate

Learning objectives

- Describe the joint state space of k agents, compute its size and its branching factor, and explain why a plain joint A^* is hopeless beyond a handful of agents.
- Explain operator decomposition and independence detection, and show that independence detection with an optimal group solver returns an optimal plan.
- Trace M^* by hand on a small grid: follow the individual policies, detect a collision, backpropagate the collision set, and expand only the agents that need it.
- State the conditions under which M^* is complete and optimal and sketch why; describe the push, swap and rotate primitives and state what Push-and-Swap and Push-and-Rotate guarantee, and what they do not.
- Place every multi-agent path finding method of the book in one comparison table and choose the right one for a given swarm scenario.

11.1 Why this matters for a drone swarm

Picture twelve inspection drones leaving a hangar through a grid of aisles. Ten of them have routes that never come near each other. Two fly toward each other in the same aisle, and one of them will have to slip into a bay to let the other pass. Every multi-agent path finding (MAPF) solver has to answer the same question for this scene: which agents must be planned *together*? Prioritized planning ([Chapter 8](#)) answers “none”, plans the drones one after another and hopes that the fixed order works. A search in the *joint* state space, where one node holds the positions of all twelve drones, answers “all of them”; it is complete and optimal, but [Section 11.3](#) shows that its state space is astronomically large. Conflict-based search (CBS, [Chapter 9](#)) answers “nobody, until a conflict appears, and then only through constraints”, which is why the global planner of the hybrid architecture in [Chapter 24](#) is CBS or its bounded-suboptimal relative, enhanced CBS (ECBS, [Chapter 10](#)).

This chapter adds the two remaining answers. The first is to **couple** agents explicitly and only where needed: Standley’s independence detection solves groups of agents separately and merges two groups only when their plans conflict, and Wagner and Choset’s M^* searches the joint space but lets every agent follow its own shortest path until a collision proves that a subset of agents must be coordinated. For the ten independent drones this costs almost nothing; for the pair in the aisle it costs a small two-agent search. The second answer gives up optimality. Push-and-Swap and Push-and-Rotate never search the joint space: they move the

agents one at a time with a handful of *primitives* and run in polynomial time; Push-and-Rotate finds a plan whenever one exists and at least two cells are empty (Section 11.7 says what the original Push-and-Swap missed). Their plans can be several times longer than necessary. In the training plan (Appendix A) M^* has priority **Medium** and Push-and-Swap priority **Awareness**: you should be able to trace M^* on a small instance and, above all, know when coupling is worth its price and when a feasible plan is all a swarm needs.

Roadmap. Section 11.2 gives the ideas and Section 11.3 the joint state space, the individual policies and the collision sets. Section 11.4 presents operator decomposition, independence detection and M^* ; Section 11.5 traces M^* on a three-drone instance and compares the effort of the searches; Section 11.6 proves termination and optimality. Section 11.7 covers the push, swap and rotate primitives, Section 11.8 the variants and the comparison table of every MAPF method in the book, Section 11.9 the code and Section 11.10 the place of these methods in the drone system.

11.2 The idea in plain words

Every agent knows its own shortest path. If the agents never met, the optimal joint plan would be the bundle of these paths, and no search would be needed. Now picture the joint state space of two agents as a lattice: one axis measures how far agent 1 has progressed along its own shortest path, the other how far agent 2 has (Figure 11.1). The bundle of individual paths is the diagonal of this lattice, a *one-dimensional* object inside a two-dimensional space. Joint A^* ignores this structure and is prepared to expand any of the $|V|^2$ configurations. M^* starts on the diagonal and stays there: from every configuration it generates a single successor, the one in which every agent takes its next policy step. Only when this successor is a collision does it react. It marks the agents involved as a *collision set*, sends the mark back along the path it came from, and re-expands those configurations with all combinations of moves of the marked agents, while the other agents keep following their policies. The search widens into a small two-dimensional patch, finds a detour, and once the agents have passed each other it is one-dimensional again. Wagner and Choset call this **subdimensional expansion**: the dimension of the search is raised locally, and only where a collision has proved that it must be.

Independence detection applies the same thought one level up: solve every agent alone, check the plans for conflicts, and merge two groups into one joint search only if their plans actually conflict. Where M^* couples agents configuration by configuration, independence detection couples them instance by instance; both are exact, because the agents they leave decoupled are exactly those whose individual plans already fit together. The constructive methods take a different stance. Push-and-Swap moves the agents one at a time, like pebbles on the vertices of the graph: an agent *pushes* along its shortest path and shoves blocking agents into empty vertices, and when it meets an agent that already sits on its goal, the two *swap* places at a vertex with three neighbors. Push-and-Rotate adds a *rotate* primitive for fully occupied cycles. These rules never look at the cost of the plan; their value is that they always terminate, and that Push-and-Rotate finds a plan whenever one exists on a graph with two empty vertices.

Key idea

Couple agents only where a collision proves that you must. Independence detection merges groups whose optimal plans conflict; M^* follows the individual optimal policies through the joint space and branches only over the agents in the collision set of a node. Both keep the guarantees of the full joint search. If you need any plan rather than the best one, the

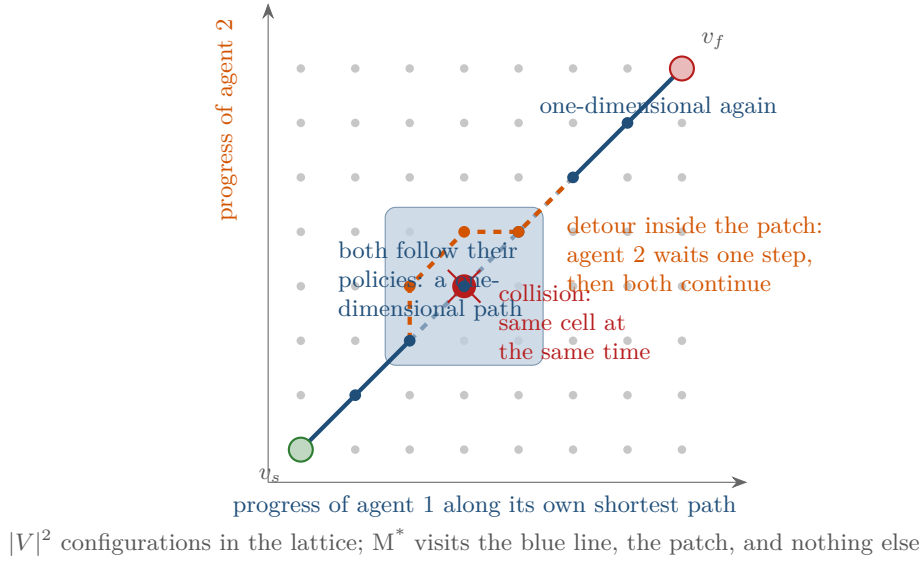


Figure 11.1. Subdimensional expansion for two agents. The gray dots are the $|V|^2$ joint configurations; the diagonal is the path in which both agents follow their individual policies, and M^* expands nothing else until the policy successor is a collision (red cross). The collision set $\{1, 2\}$ is backpropagated along the diagonal, the affected configurations are re-expanded with all joint moves of both agents (the blue patch), the detour in which agent 2 waits one step is found, and after the patch the search follows the diagonal again.

push, swap and rotate primitives build a valid plan in polynomial time, at a cost that can be far from optimal.

11.3 Problem statement and notation

11.3.1 The instance and the joint state space

We use the classical MAPF setting of Chapter 7: an undirected graph $G = (V, E)$, here a 4-connected grid of free cells, and k agents with distinct start vertices $s_1, \dots, s_k$ and distinct goal vertices $\gamma_1, \dots, \gamma_k$. In one time step an agent moves along an edge or waits. Two agents are in a **vertex conflict** if they occupy the same cell at the same time and in a **swap conflict** if they exchange cells along an edge in the same step; following an agent into the cell it is leaving is allowed. An agent that has reached its goal stays there. Agents are numbered $1, \dots, k$ in the text and $0, \dots, k-1$ in the code.

Definition 11.1 (Joint configuration and joint graph). A **joint configuration** is a tuple $v = (v^1, \dots, v^k) \in V^k$ giving the cell v^i of every agent i ; it is *collision-free* if $v^i \neq v^j$ for $i \neq j$. A **joint move** from v to v' moves every agent to $v'^i \in \text{Succ}(v^i) \cup \{v^i\}$. It is legal if v' is collision-free and no pair swaps, that is, $\neg(v'^i = v^j \wedge v'^j = v^i)$ for $i \neq j$. The **joint graph** G^k has the collision-free configurations as vertices and the legal joint moves as edges; the joint start is $v_s = (s_1, \dots, s_k)$ and the joint goal $v_f = (\gamma_1, \dots, \gamma_k)$. A path in G^k from v_s to v_f is exactly a conflict-free plan, one time step per edge.

Definition 11.2 (Cost of a joint move). A joint move from v to v' costs

$$c(v, v') = \sum_{i=1}^k c_i(v^i, v'^i), \quad c_i(u, u') = \begin{cases} 0 & \text{if } u = u' = \gamma_i, \\ 1 & \text{otherwise:} \end{cases} \quad (11.1)$$

every move and every wait away from the goal costs one, resting at the goal is free. The cost

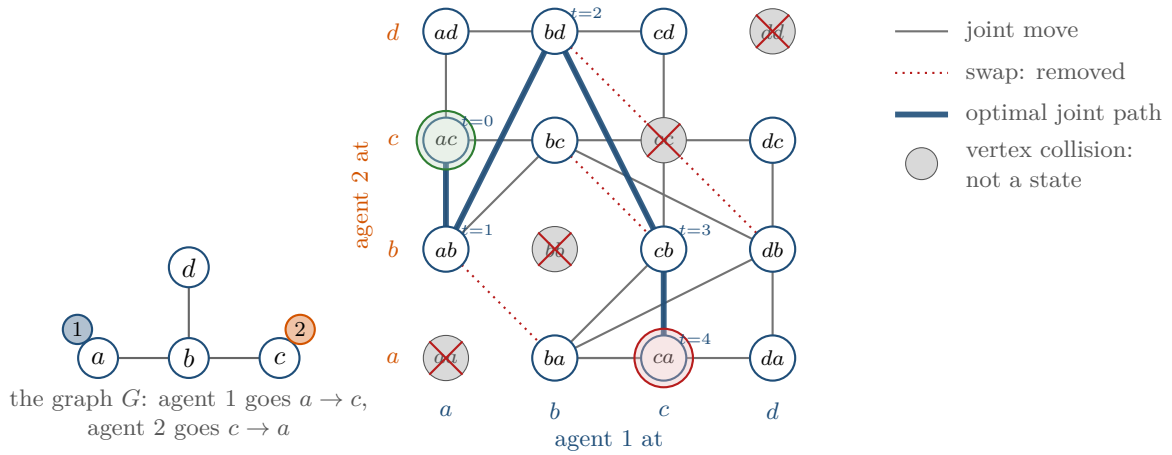


Figure 11.2. The joint graph of two agents on a four-vertex graph. Of the $4^2 = 16$ configurations, the four on the diagonal put both agents on the same vertex and are not states at all; the three dotted edges are joint moves in which the agents would swap along an edge; what remains are 12 states and 18 joint moves (a wait of both agents is a self-loop and is not drawn). The optimal plan from ac to ca has cost 7: agent 2 steps aside into d , agent 1 waits once and passes, and agent 2 comes back.

of a joint path is the sum of its move costs.

This is the convention of Standley [156] and of Wagner and Choset [172]. It agrees with the sum of costs of Chapter 7 on every plan in which no agent leaves its goal after arriving, which includes every plan returned by joint A^* and M^* in this chapter, and is a lower bound on it in general: the push-and-swap plan of Section 11.7.4, in which an agent is displaced from its goal and walks back, costs 26 under Equation (11.1) against a sum of costs of 27. The advantage of Equation (11.1) is that the cost of a move depends only on the two configurations, which is what A^* needs.

Figure 11.2, generated by `gen_ch11_joint_graph.py`, draws the joint graph of two agents on a four-vertex graph: vertex collisions delete states, swap conflicts delete edges, and a plan is a path through what is left. The trouble is the size. With $|V|$ free cells there are $|V|^k$ configurations, of which $|V|(|V| - 1) \cdots (|V| - k + 1) = |V|^k$ are collision-free, and a node has up to $(b + 1)^k$ successors, where b is the number of neighbors of a cell: 5^k on a 4-connected grid with the wait action. Table 11.1 evaluates these numbers for the 8×8 maps of the experiment in Section 11.5.1, which have about 54 free cells, and for a 20×20 map. Ten agents on the small map already have $2 \cdot 10^{17}$ configurations, so many that listing them at a billion per second would take almost seven years, and every expansion of joint A^* has to look at almost ten million joint moves. A good heuristic keeps the search away from most of the space, but each expansion still pays for the full branching factor, and that is what kills joint A^* in practice.

11.3.2 The heuristic and the individual policies

Let $d_i(u)$ be the shortest-path distance in G from cell u to the goal γ_i , computed once per agent by a backward breadth-first search (the backward Dijkstra of Chapter 3 with unit costs). The standard heuristic for the joint search is the sum of the individual distances,

$$h(v) = \sum_{i=1}^k d_i(v^i). \quad (11.2)$$

Table 11.1. Size of the joint state space. $|V|^k$ counts all configurations, $|V|^k$ the collision-free ones, and 5^k the joint moves of one configuration on a 4-connected grid with a wait action (from `gen_ch11_state_space.py`).

k	54^k (8×8 map)	54^k	5^k joint moves	400^k (20×20 map)
1	54	54	5	400
2	$2.9 \cdot 10^3$	$2.9 \cdot 10^3$	25	$1.6 \cdot 10^5$
3	$1.6 \cdot 10^5$	$1.5 \cdot 10^5$	125	$6.4 \cdot 10^7$
5	$4.6 \cdot 10^8$	$3.8 \cdot 10^8$	3 125	$1.0 \cdot 10^{13}$
10	$2.1 \cdot 10^{17}$	$8.7 \cdot 10^{16}$	$9.8 \cdot 10^6$	$1.0 \cdot 10^{26}$
20	$4.4 \cdot 10^{34}$	$7.8 \cdot 10^{32}$	$9.5 \cdot 10^{13}$	$1.1 \cdot 10^{52}$

Proposition 11.3 (The sum heuristic is consistent). *With the costs of Equation (11.1), the heuristic Equation (11.2) is consistent on the joint graph, hence admissible, and $h(v_f) = 0$.*

Proof. $h(v_f) = \sum_i d_i(\gamma_i) = 0$. For a legal joint move from v to v' and an agent i : if v'^i is a neighbor of v^i , then $d_i(v^i) \leq 1 + d_i(v'^i)$ by the triangle inequality of shortest-path distances and $c_i = 1$; if the agent waits away from its goal, d_i is unchanged and $c_i = 1$; if it rests at its goal, $d_i = 0$ on both sides and $c_i = 0$. In every case $d_i(v^i) \leq c_i(v^i, v'^i) + d_i(v'^i)$; summing over i gives $h(v) \leq c(v, v') + h(v')$, the consistency condition of Chapter 4 on the joint graph, and a consistent heuristic is admissible. $\square$

Definition 11.4 (Individual policy). An **individual policy** for agent i is a function $\phi_i: V \rightarrow V$ with $\phi_i(\gamma_i) = \gamma_i$ and, for every other cell u , $\phi_i(u) \in \text{Succ}(u)$ with $d_i(\phi_i(u)) = d_i(u) - 1$. The **policy successor** of a configuration v is $\phi(v) = (\phi_1(v^1), \dots, \phi_k(v^k))$, and the **policy path** from v is $v, \phi(v), \phi(\phi(v)), \dots$ until v_f .

A policy is an optimal single-agent plan in feedback form: it says what to do from *every* cell, which is what M^* needs once a detour has displaced an agent. When several neighbors decrease d_i , the code takes the first one in the neighbor order east, west, north, south; any fixed rule works. Along the policy path $f = g + h$ is constant, because every move that costs one lowers the heuristic by one; the policy path from v costs exactly $h(v)$ and is therefore an optimal continuation whenever it is collision-free.

11.3.3 Collision sets and limited neighbors

Definition 11.5 (Collision set of a move). For a joint move from v to v' in V^k , legal or not, the **collision set of the move** $\Psi(v, v') \subseteq \{1, \dots, k\}$ is the set of agents in a vertex conflict in v' or in a swap conflict along the move. The move is legal if and only if $\Psi(v, v') = \emptyset$.

Definition 11.6 (Collision set of a node and limited neighbors). During the search every generated configuration v carries a **collision set** $C(v) \subseteq \{1, \dots, k\}$, initially empty and only ever growing, and a **back set** $\text{back}(v)$ of all configurations from which v has been generated. The **limited neighbors** of v are

$$N_{C(v)}(v) = \left\{ v' \in V^k \mid v'^i = \phi_i(v^i) \text{ for } i \notin C(v), v'^i \in \text{Succ}(v^i) \cup \{v^i\} \text{ for } i \in C(v) \right\}. \quad (11.3)$$

Agents outside the collision set take their policy step; agents inside it branch over all their moves. With $C(v) = \emptyset$ the only limited neighbor is $\phi(v)$; with $C(v) = \{1, \dots, k\}$ the limited neighbors are all $(b+1)^k$ joint moves. In general there are $(b+1)^{|C(v)|}$ of them, and $|C(v)|$ is the local dimension of the search.

11.4 The algorithms

11.4.1 Operator decomposition and independence detection

Joint A^* is the A^* of Chapter 4 on the joint graph G^k with the costs Equation (11.1) and the heuristic Equation (11.2): the successors of a node are all legal joint moves, generated as the Cartesian product of the individual move lists and filtered by Definition 11.5. Since the heuristic is consistent, joint A^* is complete, optimal, and expands every configuration at most once. It is the baseline against which everything else in this chapter is measured, and Table 11.1 says why it is only a baseline.

Standley [156] attacks the two sources of the blow-up separately. **Operator decomposition** (OD) attacks the branching factor. Instead of choosing the moves of all k agents at once, the search chooses them one agent at a time: from a full configuration it generates the $b + 1$ moves of agent 1, which yield *intermediate* states in which agent 1 has moved and the others have not; from an intermediate state it generates the moves of the next agent, checking vertex and swap conflicts only against the agents that have already chosen; after the move of agent k a full configuration is reached again. A node then has at most $b + 1$ successors instead of $(b + 1)^k$, at the price of a search tree that is k times deeper and a heuristic evaluated on intermediate states (the sum of d_i over the new cell of the agents that have moved and the old cell of the others). Because the intermediate states carry partial g values, A^* prunes an agent's bad move before the moves of the remaining agents are ever enumerated; in Section 11.5.1 OD examines, in the median, about 200 times fewer successor candidates than plain joint A^* with six agents (the ratio of the means is about 120). OD is A^* on a graph with the same paths between full configurations, so it is optimal too.

Independence detection (ID) attacks the exponent k , and it is not a search but a wrapper around one. Algorithm 11.1 starts with every agent in a group of its own, plans each group optimally and independently, and looks for a pair of groups whose plans conflict. Such a pair is merged and re-planned as one group with the joint search; the loop repeats until no two group plans conflict, and the final plan is the union of the group plans.

Algorithm 11.1. Simple independence detection

Input: graph G , starts $s_1, \dots, s_k$, goals $\gamma_1, \dots, \gamma_k$, an optimal joint solver **JointAStar**

Output: an optimal conflict-free plan, or *failure*

```

1 Function IndependenceDetection( $G, s_{1..k}, \gamma_{1..k}$ ):
2    $\mathcal{G} \leftarrow \{\{1\}, \{2\}, \dots, \{k\}\}$ 
3   foreach group  $A \in \mathcal{G}$  do
4      $\pi_A \leftarrow \text{JointAStar}(G, A)$  // single-agent search
5   while there are groups  $A \neq B$  in  $\mathcal{G}$  whose plans  $\pi_A$  and  $\pi_B$  conflict do
6      $M \leftarrow A \cup B$ ; remove  $A$  and  $B$  from  $\mathcal{G}$ ; add  $M$ 
7      $\pi_M \leftarrow \text{JointAStar}(G, M)$  // joint search over the merged group only
8     if  $\pi_M = \text{failure}$  then return failure
9   return  $\bigcup_{A \in \mathcal{G}} \pi_A$ 

```

Walkthrough. Line 4 runs k single-agent searches. Line 5 is the conflict detection of Chapter 7 between the paths of two groups, with an arrived agent blocking its goal cell for ever. Merging (line 6) is irreversible, so the loop runs at most $k - 1$ times, and line 7 is the expensive step: a joint search over $|M|$ agents. In the worst case all agents end up in one group after $k - 1$ useless searches; in the common case the largest group stays small (Figure 11.5,

right). Standley adds two refinements: before merging A and B , try to re-plan A at the same cost while treating π_B as an obstacle, then the other way round, and merge only if both fail; and break ties among equally cheap paths toward paths that conflict with fewer other groups, using a *conflict avoidance table* of the current plans. The chapter code implements the simple version.

Proposition 11.7 (Independence detection is optimal). *If the group solver returns optimal plans and reports failure only when a group has no plan, then Algorithm 11.1 returns a conflict-free plan of minimum cost whenever one exists.*

Proof. When the loop stops, no two group plans conflict, so their union is a conflict-free plan of cost $\sum_A \text{cost}(\pi_A)$ with each π_A optimal for its group. Let π^* be any conflict-free plan of all k agents. Restricting π^* to the agents of a group A gives a conflict-free plan for A , of cost at least $\text{cost}(\pi_A)$, and the cost of π^* is the sum of the costs of its restrictions, so $\text{cost}(\pi^*) \geq \sum_A \text{cost}(\pi_A)$. If a merged group has no plan, the whole instance has none, since a plan for all agents restricts to a plan for the group. $\square$

11.4.2 M^*

Algorithm 11.2 is the basic M^* of Wagner and Choset [172, 173]: joint A^* with three changes. The successors of a node are its limited neighbors Equation (11.3) rather than all joint moves. A successor that collides is not queued; instead its collision set is added to the collision set of the node and *backpropagated* to every node in the back set, recursively. A node whose collision set grows is put back into OPEN so that it is expanded again, now with more neighbors.

Walkthrough. Lines 3 to 6 are A^* with the key $(f, -g)$ of Chapter 4; the goal test happens when v_f is popped. Line 7 is the first difference: with an empty collision set the loop body runs once, for the policy successor. Line 8 records *every* parent of v' , not only the cheapest one, because backpropagation must reach every node from which a collision can be reached along limited neighbors. Line 9 computes the collision set of the move. If the move is illegal, the agents that share a cell in v' are stored in $C(v')$ (line 11): that set is a property of the configuration v' , which can never be entered, whatever the parent. A swap conflict, by contrast, is a property of the move and is not stored on v' , which may be a legal configuration when reached by other moves; it is only backpropagated (line 12). Line 13 lets the parent inherit the collision set of a legal successor that was visited earlier, so that a node always knows about every collision found downstream of it. In **Backpropagate**, line 20 stops the recursion as soon as nothing new arrives, which bounds the work of backpropagation by the number of (node, agent) pairs, and line 22 is the re-expansion rule: a node that has been expanded but whose collision set has just grown goes back into OPEN with its old key, and when it is popped again **LimitedNeighbors** returns more successors than before. Two invariants follow: $C(u) \supseteq C(v)$ whenever $u \in \text{back}(v)$, and along the policy path from any expanded node f is constant, so that path is explored to its end or to its first collision before any node with a larger f is touched. Section 11.6 builds the proof on these two facts.

11.5 A worked example

Example 11.8 (Three drones, two aisles, one bay). Figure 11.3(a) shows a 5×4 map with two aisles. The lower aisle is the row $y = 1$ with a bay at $(1, 0)$; the upper aisle is the row $y = 3$; the two are joined only through the passage $(4, 2)$. Drone 1 flies from $(0, 1)$ to $(4, 1)$, drone 2 from $(4, 1)$ to $(0, 1)$, and drone 3 from $(0, 3)$ to $(4, 3)$. Each drone is four moves from its goal, so $h(v_s) = 12$. Drones 1 and 2 head straight at each other and their policy paths collide in $(2, 1)$ at $t = 2$; drone 3 never meets anyone. The instance is `worked_example()` in

Algorithm 11.2. M^* with collision sets and backpropagation

Input: graph G , joint start v_s , joint goal v_f , optimal individual policies $\phi_1, \dots, \phi_k$, heuristic h from Equation (11.2)

Output: an optimal conflict-free plan, or *failure*

```

1 Function MStar( $G, v_s, v_f, \phi$ ):
2   for every configuration (implicitly):  $g \leftarrow \infty, C \leftarrow \emptyset, back \leftarrow \emptyset$ 
3    $g(v_s) \leftarrow 0$ ; OPEN  $\leftarrow$  priority queue holding  $v_s$  with key  $(h(v_s), 0)$ 
4   while OPEN  $\neq \emptyset$  do
5      $v \leftarrow$  pop the entry with the smallest key  $(f, -g)$  from OPEN, skipping stale
        entries
6     if  $v = v_f$  then return PathFromParents( $v_f$ )
7     foreach  $v' \in \text{LimitedNeighbors}(v, C(v))$  do
8       add  $v$  to  $back(v')$ 
9        $\Psi \leftarrow \text{Collisions}(v, v')$ 
10      if  $\Psi \neq \emptyset$  then
11         $C(v') \leftarrow C(v') \cup \{\text{agents sharing a cell in } v'\}$ 
12        Backpropagate( $v, \Psi \cup C(v')$ ); continue //  $v'$  is never queued
13      Backpropagate( $v, C(v')$ ) //  $v'$  may already carry a collision set
14      if  $g(v) + c(v, v') < g(v')$  then
15         $g(v') \leftarrow g(v) + c(v, v')$ ; parent( $v'$ )  $\leftarrow v$ ; push  $v'$  into OPEN with key
           $(g(v') + h(v'), -g(v'))$ 
16  return failure

17 Function LimitedNeighbors( $v, C$ ):
18  return the Cartesian product of the lists  $L_1, \dots, L_k$  with  $L_i = \text{Succ}(v^i) \cup \{v^i\}$  if
     $i \in C$  and  $L_i = \{\phi_i(v^i)\}$  otherwise

19 Function Backpropagate( $v, C'$ ):
20  if  $C' \not\subseteq C(v)$  then
21     $C(v) \leftarrow C(v) \cup C'$ 
22    if  $v \notin \text{OPEN}$  and  $g(v) < \infty$  then push  $v$  into OPEN again with its current key
23    foreach  $u \in back(v)$  do Backpropagate( $u, C(v)$ )

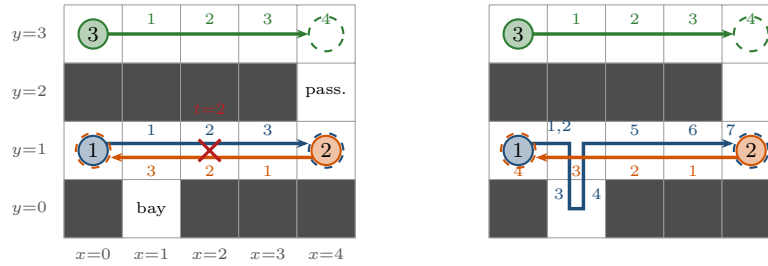
```

■ `ch11_mstar.py`; every number below is produced by its self-test.

Table 11.2 lists the decisive expansions of Algorithm 11.2 on the instance and Figure 11.4 draws the search graph. The run has four phases.

Phase 1: along the policies (steps 1–2). The start has an empty collision set and a single limited neighbor, the policy successor $((1, 1), (3, 1), (1, 3))$ with $f = 12$. That node is expanded next, and its policy successor $((2, 1), (2, 1), (2, 3))$ puts drones 1 and 2 in the same cell. The successor is discarded, $\{1, 2\}$ is stored on the illegal configuration and backpropagated to the node of step 2 and, along its back set, to the start. Both have already been expanded, so both go back into OPEN with their old keys.

Phase 2: widening (steps 3–4). The node of step 2 is popped first because it has the larger g . With $C = \{1, 2\}$ it has $4 \times 3 = 12$ limited neighbors: drone 1 at $(1, 1)$ can go east, west, south into the bay or wait, drone 2 at $(3, 1)$ can go west, east or wait, and drone 3 keeps its policy step to $(2, 3)$. One of the twelve is the known collision; the other eleven enter



(a) individual policies: agents 1 and 2 collide in cell (2, 1) at $t = 2$; drone 3 is independent. (b) plan of cost 15: agent 1 waits at (1, 1), hides in the bay while agent 2 passes, and continues

Figure 11.3. Example 11.8. (a) The individual policies (arrows with arrival times) collide in cell (2, 1) at $t = 2$; drone 3 is independent. (b) The plan returned by M^* , of cost $7 + 4 + 4 = 15$: drone 1 waits one step at (1, 1), ducks into the bay while drone 2 passes through (1, 1), and continues; drones 2 and 3 follow their policies unchanged. The plan is also optimal for the sum of costs and has makespan 7.

OPEN with $f = 13$ or 14. Step 4 re-expands the start with $2 \times 3 = 6$ limited neighbors, five of them new. What has *not* happened matters as much: drone 3 is in no collision set, so it never branches, and every node of the search has drone 3 exactly where its policy puts it. Its dimension has been folded away.

Phase 3: the patch (steps 5–27). A node with $f = 13$ is one in which drones 1 and 2 have together lost one step against the heuristic, for instance because one of them waited. Twelve of the 31 expansions are second expansions, of a node whose collision set grew after it had already been expanded; other nodes inherit $\{1, 2\}$ from a policy successor that already carries it (line 13).

Phase 4: one-dimensional again (steps 28–31). In the node of step 28 ($g = 9$, $f = 15$) drone 1 sits in the bay and drone 2 in (1, 1). Its collision set is empty and its policy successor $((1, 1), (0, 1), (4, 3))$ is legal: drone 1 steps back into the aisle as drone 2 leaves it, a following move. From here on every node has an empty collision set and a single successor; drones 2 and 3 rest at their goals for free and drone 1 walks east. The goal is generated in step 31 with $g = 15$ and popped next, before the three nodes of steps 25–27 that were re-opened at $f = 15$ with a smaller g : the tie-breaking rule $(f, -g)$ makes the search dive to the goal instead of widening those nodes first.

The four searches compared. On the same instance plain joint A^* expands 24 nodes but examines 612 successor candidates, because every expansion enumerates the whole Cartesian product of the agents' move lists—up to $5^3 = 125$ joint moves, and $612/24 \approx 26$ on average on this narrow map, where most cells have only one or two neighbors; operator decomposition expands 100 states, most of them intermediate, and examines 292; independence detection runs three single-agent searches, merges drones 1 and 2 and solves the pair jointly, for 27 expansions and 162 candidates; M^* expands 31 nodes and examines 127 candidates. M^* expands *more* nodes than joint A^* here, because twelve of its expansions are repetitions forced by collision sets that grew later; what it saves is the branching factor, and with three agents the saving is modest. Of the 55 configurations the search touched, 19 end with the collision set $\{1, 2\}$ and none contains drone 3.

Table 11.2. The decisive expansions of Algorithm 11.2 on Example 11.8; `mstar(inst, trace=True)` records all 31 in `result.info`. A node is written as the cells of drones 1, 2, 3; C is its collision set at the moment of the expansion; $\phi(v)$ is its policy successor; “new” counts successors that enter OPEN with a better g . The f column never decreases.

Step	Drone 1	Drone 2	Drone 3	g	h	f	C	What happens
1	(0, 1)	(4, 1)	(0, 3)	0	12	12	$\emptyset$	policy successor generated
2	(1, 1)	(3, 1)	(1, 3)	3	9	12	$\emptyset$	$\phi(v) = (2, 1)(2, 1)(2, 3)$ collides: $C \leftarrow \{1, 2\}$, backpropagated to step 1; both re-opened
3	(1, 1)	(3, 1)	(1, 3)	3	9	12	$\{1, 2\}$	re-expanded: 11 new, 1 known collision
4	(0, 1)	(4, 1)	(0, 3)	0	12	12	$\{1, 2\}$	re-expanded: 5 new
5	(1, 1)	(2, 1)	(2, 3)	6	7	13	$\emptyset$	$\phi(v)$ swaps 1 and 2: $C \leftarrow \{1, 2\}$, re-opened
6	(2, 1)	(3, 1)	(2, 3)	6	7	13	$\emptyset$	$\phi(v)$ swaps 1 and 2: $C \leftarrow \{1, 2\}$, re-opened
7	(1, 1)	(2, 1)	(2, 3)	6	7	13	$\{1, 2\}$	re-expanded: 9 new, 3 collisions
8	(2, 1)	(3, 1)	(2, 3)	6	7	13	$\{1, 2\}$	re-expanded: 3 new, 3 collisions
9–24	sixteen expansions with $f = 13$ and 14: eight nodes expanded first with $C = \emptyset$ (their $\phi(v)$ collides or already carries $\{1, 2\}$) and then again with $C = \{1, 2\}$; drone 2 tries the passage, without success							
25	(1, 1)	(2, 1)	(3, 3)	9	6	15	$\emptyset$	$\phi(v)$ swaps 1 and 2: re-opened
26	(2, 1)	(3, 1)	(3, 3)	9	6	15	$\emptyset$	$\phi(v)$ swaps 1 and 2: re-opened
27	(0, 1)	(1, 1)	(3, 3)	9	6	15	$\emptyset$	$\phi(v)$ swaps 1 and 2: re-opened
28	(1, 0)	(1, 1)	(3, 3)	9	6	15	$\emptyset$	drone 1 in the bay: $\phi(v) = (1, 1)(0, 1)(4, 3)$, $g = 12$
29	(1, 1)	(0, 1)	(4, 3)	12	3	15	$\emptyset$	drones 2, 3 rest for free: successor has $g = 13$
30	(2, 1)	(0, 1)	(4, 3)	13	2	15	$\emptyset$	successor with $g = 14$
31	(3, 1)	(0, 1)	(4, 3)	14	1	15	$\emptyset$	generates v_f with $g = 15$; v_f is popped next

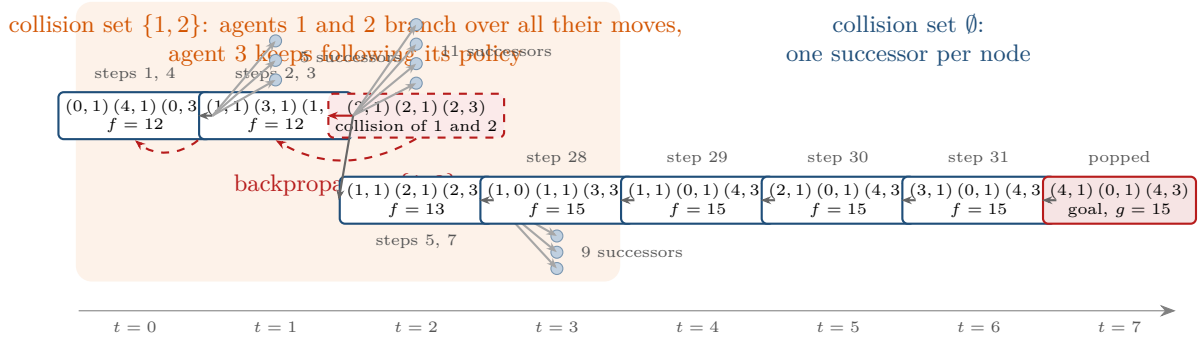


Figure 11.4. The search graph of M^* on Example 11.8, drawn over the time axis. Along the policies it is a chain; the collision at $t = 2$ is backpropagated (red) to the two nodes before it, which are re-expanded with the joint moves of drones 1 and 2 (fans of small nodes; only the successor on the optimal plan is drawn in full). Once drone 1 has left the bay behind drone 2, the collision sets are empty again and the graph is a chain to the goal.

11.5.1 Effort on random maps

Figure 11.5 scales the comparison up. The script `gen_ch11_expansions.py` draws 30 random 8×8 maps with 15% obstacles for each $k = 2, \dots, 6$, places k agents at random and runs the four searches with a cap of 1.5 million successor candidates; the statistics are over the instances that all four solved within the cap. Joint A^* grows from a median of 140 candidates for two agents to 77 000 for six, roughly a factor five per agent, as the branching factor predicts; operator decomposition stays below 360; independence detection and M^* stay between about ten and about 1 000, because on the median instance only two or three agents interact. The means tell a different story: for $k = 6$ they are 132 000 (joint A^*), 1 100 (OD), 14 000 (ID) and

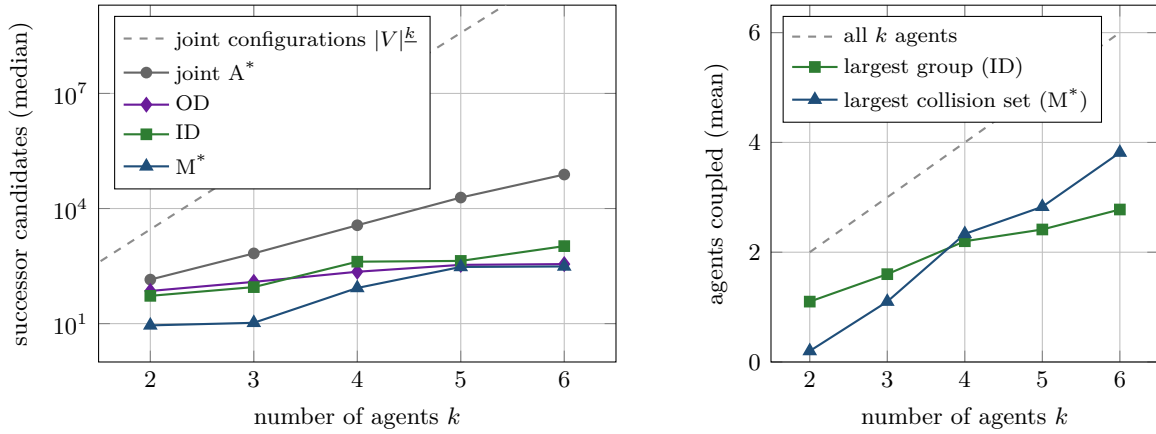


Figure 11.5. Random 8×8 maps, 30 instances per k , of which the statistics use the instances solved by all four methods within the cap of 1.5 million successor candidates: 30, 30, 30, 29, 27 for $k = 2, \dots, 6$. Left: median number of successor candidates examined (log scale), with the number of collision-free joint configurations 54^k as a reference. Right: mean size of the largest group merged by independence detection and of the largest collision set built by M^* ; the dashed line is “all k agents”.

42 000 (M^*), dominated by the few instances on which almost every agent ends up coupled and ID and M^* degenerate into a joint search plus overhead. The common set also creates a survivorship effect that has to be read with care: the mean expansions of joint A^* *fall* from 63.7 at $k = 5$ to 29.6 at $k = 6$, not because the search got easier but because the three hardest instances of $k = 6$ dropped out of the set. The right panel shows the largest group that ID had to merge and the largest collision set that M^* built, on average 2.8 and 3.8 agents out of six: M^* couples agents whenever their *policy* paths collide anywhere in the region it explores, whereas ID merges groups only when their optimal plans conflict. Both stay far below the diagonal k , and that gap is the whole point.

11.6 Properties: termination, completeness, optimality

Throughout this section G is finite, the joint graph is that of Definition 11.1, the policies ϕ_i are optimal (Definition 11.4), the heuristic is admissible (Equation (11.2) is even consistent, Proposition 11.3) and the costs are those of Definition 11.2. The results are those of Wagner and Choset [172, 173].

Theorem 11.9 (Termination). *Algorithm 11.2 terminates on every instance.*

Proof. A node enters OPEN only when its g strictly decreases (line 15) or its collision set strictly grows (line 22). Collision sets are subsets of $\{1, \dots, k\}$ that only grow, so the second event happens at most k times per node. Between two such events anywhere in the search the graph of limited neighbors is fixed, and on a fixed finite graph with non-negative costs the g of a node can decrease only finitely often (the termination argument of Chapter 4). There are finitely many configurations, hence finitely many insertions, and the loop ends. $\square$

Theorem 11.10 (M^* is complete and optimal). *With optimal individual policies and an admissible heuristic, if a conflict-free plan exists, Algorithm 11.2 returns one of minimum cost Equation (11.1); otherwise it returns failure.*

Proof sketch. Let C^* be the optimal cost.

Step 1: M^* is A^* on the final limited graph. Let G^{sub} consist of the generated configurations with the limited-neighbor edges Equation (11.3) for the collision sets at termination. Whenever

a collision set grew, the node was re-inserted, so every expanded node was expanded at least once with its final collision set, and the run is A^* with re-opening on G^{sub} with an admissible heuristic: by the optimality theorem of Chapter 4 it returns a cheapest path of G^{sub} from v_s to v_f if one exists, and by Theorem 11.9 it reports failure otherwise. It remains to show that G^{sub} contains a path of cost C^* whenever a plan exists.

Step 2: everything cheap is expanded. Assume, for the contradiction of Step 3, that the run either fails or returns a cost larger than C^* . Then every node generated with $f \leq C^*$ is expanded before the run ends, because the goal is popped only with f equal to the returned cost, so every such node is popped first. In particular, the policy successor $\phi(v)$ of an expanded node v is a limited neighbor whatever $C(v)$ is, and $f(\phi(v)) = f(v)$; so the policy path from every expanded node with $f \leq C^*$ is explored to v_f or to its first collision, whose agents are then backpropagated into $C(v)$. Moreover $C(u) \supseteq C(w)$ whenever w was generated from u (line 13 and Backpropagate).

Step 3: no optimal plan is cut off. Suppose a plan exists but G^{sub} contains none of cost C^* . Among all optimal plans $\pi^* = (u_0 = v_s, \dots, u_m = v_f)$ choose one that maximizes the number j of leading nodes $u_0, \dots, u_j$ expanded with g equal to the cost of the prefix of π^* ; then $j < m$. Let $A = C(u_j)$ at termination. The move $u_j \rightarrow u_{j+1}$ is not a limited neighbor of u_j , otherwise u_{j+1} would be generated with optimal g and expanded by Step 2, contradicting the choice of j ; so some agent outside A deviates from its policy in this move. Let σ be the continuation from u_j in which the agents in A move as in π^* and the agents outside A follow their policies. Because policy paths are shortest paths, $\text{cost}(\sigma)$ is at most the cost of the suffix of π^* . If σ is conflict-free, the prefix of π^* followed by σ is an optimal plan whose first step after u_j is a limited neighbor of u_j , which contradicts the choice of π^* . Otherwise the first conflict of σ involves an agent outside A , because the agents in A move as in the conflict-free π^* . As long as the nodes of σ carry the collision set A , they are successive limited neighbors, generated and expanded with $f \leq C^*$ by Step 2, the conflict is detected when the last of them is expanded, and backpropagation along σ adds the offending agent to $C(u_j)$: a contradiction with the finality of A . If instead a node w of σ has $C(w) \subsetneq A$, the construction restarts from w with $C(w)$ in place of A , the agents that dropped out of the set switching to their policies. The set A used in the argument is replaced by a strictly smaller one at most k times along this process, and otherwise the first conflict comes one step closer at every repetition, so after finitely many repetitions one of the two contradictions is reached. Hence G^{sub} contains an optimal plan, and by Step 1 M^* returns one. $\square$

The proof uses the optimality of the policies in one place only, when the conflict-free σ is shown to be optimal; with suboptimal policies M^* stays complete but loses optimality. It uses the heuristic only in Step 1, so an inflated heuristic wh gives a plan of cost at most wC^* by the weighted- A^* bound of Chapter 4: that is *inflated* M^* (Section 11.8).

Corollary 11.11 (No collisions, no search). *If the policy path from v_s is conflict-free, M^* expands exactly the T non-goal nodes of that path, one successor each, and then pops v_f and returns: T expansions, where T is the makespan of the individual paths, whatever k is.*

Proof. Every node on the path has an empty collision set and the policy successor as its only limited neighbor; all have $f = h(v_s)$ and no other node is ever generated. The goal is popped with $g = h(v_s)$ after T expansions and returned at once (line 6), so v_f is never expanded; the cost is optimal by admissibility. $\square$

Complexity. Between the two extremes the effort is that of a joint search over the agents in the collision sets: a node with $|C(v)| = m$ has $(b+1)^m$ limited neighbors. In the worst case every collision set becomes $\{1, \dots, k\}$ and M^* is joint A^* plus re-expansions and backpropagation, as on the instances that dominate the means in Section 11.5.1: the exponent is the size of the largest collision set, not k .

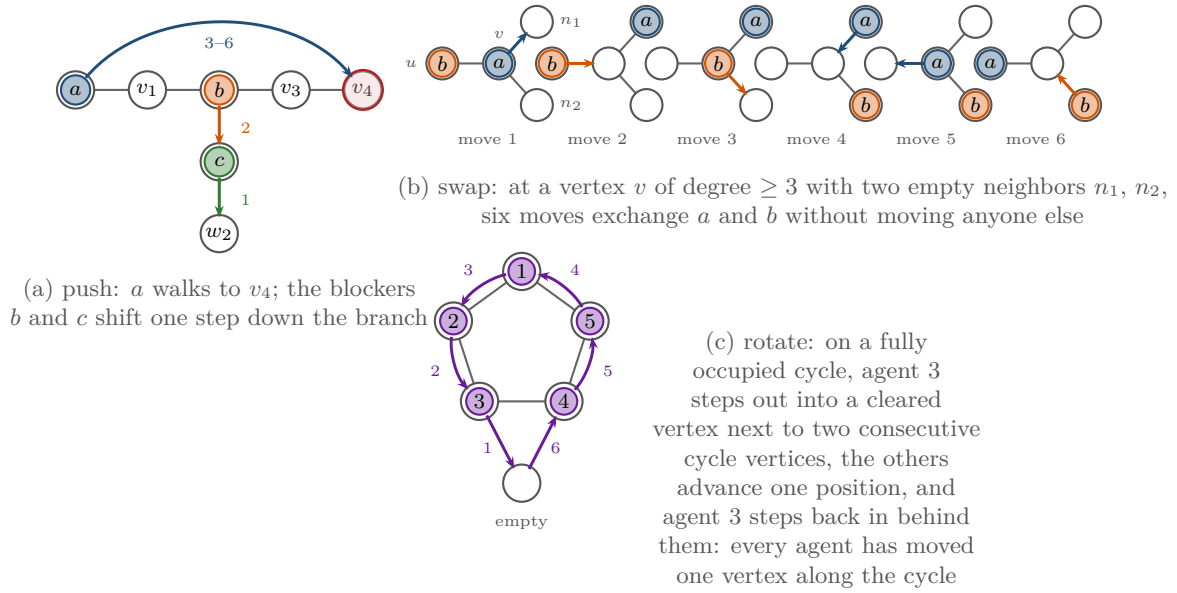


Figure 11.6. The primitives. (a) A push with a chain shift: the two blockers shift and a walks through. (b) The exchange at the heart of a swap: with a at a vertex v of degree three, b behind it and two empty neighbors n_1, n_2 , six moves exchange the two agents and leave the rest of the graph untouched. (c) A rotation: with one cleared vertex next to two consecutive vertices of a fully occupied cycle, every agent on the cycle advances one position in six moves.

11.7 Constructive methods: Push-and-Swap and Push-and-Rotate

11.7.1 Pebbles instead of paths

The constructive methods see the agents as **pebbles** on the vertices of G : one pebble moves at a time, along an edge, into an empty vertex. This is the *pebble motion* problem of Kornhauser, Miller and Spirakis [92], who showed that feasibility is decidable in polynomial time and that $\mathcal{O}(|V|^3)$ moves suffice when a solution exists; for one empty vertex on a biconnected graph, Wilson [180] had already characterized the reachable arrangements (the generalized 15-puzzle). A sequential plan becomes a MAPF plan by letting every move take one time step during which all other agents wait, and it is conflict-free by construction. A *shortest* plan remains NP-hard to find (Chapter 7; [165, 184]), so a polynomial method must give up optimality, and Push-and-Swap and Push-and-Rotate do exactly that.

11.7.2 The primitives

Push(a) moves agent a along its shortest path p toward its goal (Figure 11.6(a)). When the next vertex on p is occupied by an agent b that is not *locked* at its own goal, the algorithm *clears* the vertex: a breadth-first search from it, avoiding the remaining path p and the locked agents, finds the nearest empty vertex e , and every agent on the chain from b to e shifts one step toward e , starting with the agent next to e . Then a steps forward. In the figure b and c shift one step down the side branch and a walks through: $2 + 4 = 6$ single moves. The push fails when the blocking agent is locked or when no empty vertex can be reached without crossing p ; in both cases a swap is needed.

Swap(a, b) exchanges two adjacent agents without changing, in the end, the position of anyone else. The exchange itself needs a vertex v with at least three neighbors and two empty neighbors of v ; Figure 11.6(b) shows the six moves. To get there the algorithm tries the vertices

of degree at least three in order of distance and, for each candidate, performs a *multipush*: the agent nearer to v leads along a shortest path to v , the other follows one step behind, agents in the way are pushed off the path with the clearing rule above, and at v two further neighbors are cleared. After the six exchange moves, the preparation is undone in reverse order, move by move, whoever now stands on the vertex concerned moving back. Because the two agents have exchanged roles, the reversal brings b to where a started, a to where b started, and every other agent to its old vertex. If the leader is d steps from v and nothing has to be cleared, the swap costs $2d$ moves out, 6 for the exchange and $2d$ back: $4d + 6$ moves, verified for $d = 1, \dots, 4$ by the self-test of the chapter code. If no candidate vertex allows the maneuver, the swap fails and so does the algorithm.

Rotate, the addition of Push-and-Rotate, serves a cycle whose vertices are all occupied (Figure 11.6(c)): a vertex next to the cycle is cleared, one agent steps out into it, the remaining agents advance one position each into the vertex just vacated, and the agent that stepped out steps back in behind them — which needs the cleared vertex to touch two consecutive vertices of the cycle, as it does in the figure. Every agent on the cycle has then moved one vertex along it, which no push could achieve because no vertex of the cycle was empty. The primitive that Algorithm 11.4 states is the kernel of this maneuver: once one vertex of a cycle is empty, the agents on it rotate in as many moves as there are agents.

11.7.3 The algorithms and their guarantees

Algorithm 11.3 gives Push-and-Swap [110] at the level of its primitives. Agents are solved one after another: the current agent pushes until it reaches its goal or is blocked by a locked agent, swaps with the blocker, and pushes again; when it arrives it is locked. A locked agent that a swap has displaced is unlocked and solved again later, the *resolve* step of the original paper. Every swap moves the current agent one vertex closer to its goal and leaves everyone else where they were, so the current agent arrives after finitely many primitives and the inner loop terminates. The outer loop is the delicate part, because the resolve step (line 10) puts a displaced agent back into the queue; that it still terminates is the argument of Luna and Bekris [110], corrected by de Wilde et al. [178].

Algorithm 11.3. Push-and-Swap (conceptual)

Input: graph G , starts $s_1, \dots, s_k$, goals $\gamma_1, \dots, \gamma_k$, with at least two empty vertices

Output: a sequence of single-agent moves, or *failure*

```

1 Function PushAndSwap( $G, s_{1..k}, \gamma_{1..k}$ ):
2    $U \leftarrow \emptyset$  // locked agents, already at their goals
3   foreach agent  $a$  taken from a queue  $Q$ , initially all agents in a fixed order do
4     while  $a$  is not at  $\gamma_a$  do
5        $p \leftarrow$  shortest path from the position of  $a$  to  $\gamma_a$ 
6       Push( $a, p, U$ ) // as far as possible; clears unlocked blockers
7       if  $a$  is not at  $\gamma_a$  then
8          $b \leftarrow$  the agent occupying the next vertex of  $p$ 
9         if Swap( $a, b$ ) fails then return failure
10        if  $b \in U$  then remove it from  $U$  and append it to  $Q$ 
11      add  $a$  to  $U$ 
12  return the recorded moves

```

Luna and Bekris state that the algorithm is complete for every instance whose graph has at least two unoccupied vertices, that is, at most $|V| - 2$ agents on a connected graph:

it returns a plan whenever one exists. The two empty vertices are what the exchange of Figure 11.6(b) needs, and a vertex of degree three exists in every connected graph that is not a path or a cycle. De Wilde, ter Mors and Witteveen [178] later found instances in this class on which the published algorithm fails, in particular when solved agents must be moved in a coordinated way or when the agents to be exchanged sit on a fully occupied cycle, and repaired it. Push-and-Rotate keeps push and swap, adds the rotation, and decomposes the graph into *subproblems*: maximal subgraphs within which the agents assigned to them can reach every position, found from the biconnected components and the trees hanging off them. The subproblems are solved in a priority order, and an instance is recognized as unsolvable when an agent cannot be brought into the subproblem of its goal; this is where the theory of Kornhauser, Miller and Spirakis enters the algorithm.

Algorithm 11.4 puts the three primitives together at the level of Algorithm 11.3. **Rotate** is stated for the case that costs nothing to state: a cycle $C = (u_0, \dots, u_{m-1})$ one of whose vertices, u_0 , is empty. Line 2 lets every other agent of C step into the vertex just vacated, so $m - 1$ moves advance each of them one position and leave the empty vertex at u_{m-1} , with nothing outside C touched. What Figure 11.6(c) adds is the work of *producing* that empty vertex on a full cycle: line 12 clears a vertex next to C and parks a cycle agent in it, and line 13 turns the cycle and lets that agent rejoin. No sequence of pushes can do this, because no vertex of C is ever free, and no swap can either, since it cannot clear the two neighbors of Figure 11.6(b). The loop around the primitives is that of Algorithm 11.3 with two further additions: line 5 computes the subproblems and orders the agents, and line 6 is the decision procedure — an agent whose goal lies outside its own subproblem can never get there, so the instance is reported unsolvable rather than failed. The resolve step (line 15) is unchanged.

Algorithm 11.4. Push-and-Rotate (conceptual), after de Wilde, ter Mors and Witteveen

Input: graph G , starts $s_1, \dots, s_k$, goals $\gamma_1, \dots, \gamma_k$, with at least two empty vertices
Output: a sequence of single-agent moves, or *unsolvable*

```

1 Function Rotate(cycle  $C = (u_0, u_1, \dots, u_{m-1})$  whose vertex  $u_0$  is empty):
2   for  $j \leftarrow 1$  to  $m - 1$  do move the agent on  $u_j$  to  $u_{j-1}$ 
3   return  $m - 1$  moves; every agent of  $C$  has advanced one position along it and
    $u_{m-1}$  is now the empty vertex

4 Function PushAndRotate( $G, s_{1..k}, \gamma_{1..k}$ ):
5   decompose  $G$  into subproblems, assign every agent to one, and let  $Q$  hold the
   agents ordered by subproblem;  $U \leftarrow \emptyset$ 
6   if some  $\gamma_a$  lies outside the subproblem of agent  $a$  then return unsolvable
7   foreach agent  $a$  taken from  $Q$  do
8     while  $a$  is not at  $\gamma_a$  do
9        $p \leftarrow$  shortest path from the position of  $a$  to  $\gamma_a$ ; Push( $a, p, U$ )
10      if  $a$  is not at  $\gamma_a$ , blocked by the agent  $b$  on the next vertex of  $p$  then
11        if  $a$  and  $b$  lie on a cycle  $C$  all of whose vertices are occupied then
12          clear a vertex  $e \notin C$  next to  $C$  and park the agent of the adjacent
          cycle vertex in it
13          call Rotate( $C$ ) until  $b$  has left  $p$ , then let the parked agent rejoin  $C$ 
14        else return unsolvable if Swap( $a, b$ ) fails
15        if  $b \in U$  then remove  $b$  from  $U$  and append it to  $Q$ 
16      add  $a$  to  $U$ 
17   return the recorded moves

```

Theorem 11.12 (Completeness of Push-and-Rotate; de Wilde, ter Mors and Witteveen 2014). *For every MAPF instance whose graph has at least two unoccupied vertices, Push-and-Rotate returns a conflict-free plan if the instance is solvable and reports failure otherwise.*

Proof sketch. With two empty vertices any two agents of a biconnected subproblem can be exchanged by a swap or a rotation, so every permutation of its agents is reachable; the decomposition and the priorities reduce the instance to a sequence of such subproblems between which pushes move the agents. The full proof is a long case analysis in [178]. $\square$

11.7.4 Feasibility is not optimality

Both methods are **feasibility solvers**: they never compare the cost of two plans, and nothing in them bounds the cost of the plan they return. On the corridor $c_0c_1c_2c_3c_4$ with a spur s at c_3 (`swap_example()` in the chapter code), agent a starts at c_0 with goal c_1 and agent b at c_1 with goal c_0 . The driver of the chapter code solves it with 14 single-agent moves: b is pushed aside to c_2 , a steps to its goal and is locked, b is blocked by the locked a , the two swap at c_3 (two multipush moves, six exchange moves, two moves back: $4 \cdot 1 + 6 = 10$), b walks to c_0 , and the displaced a is solved again. Executed one move per time step this plan has makespan 14 and sum of costs 27. The optimal joint plan, found by joint A^* , has makespan 7 and sum of costs 14: both agents walk to the junction together, a ducks into the spur, and both walk back. The ratio is two on this toy graph; the $4d + 6$ moves of a swap grow with the distance d to the nearest junction, and a swap is repeated for every locked agent in the way. Both papers therefore post-process their plans, running non-interfering moves of different agents in the same time step and removing redundant moves; even so the plans are typically several times longer than optimal on crowded instances, and no bound is available.

11.8 Variants and extensions

Recursive and inflated M^* . Basic M^* has flat collision sets: if drones 1 and 2 collide in one corner of the map and drones 3 and 4 in another, a node whose descendants see both collisions gets $C = \{1, 2, 3, 4\}$ and 5^4 limited neighbors although the pairs never interact. **Recursive M^*** (rM^*) keeps the collision set as a collection of disjoint subsets, $\{\{1, 2\}, \{3, 4\}\}$, computes the “policy” of each subset by a recursive M^* call over its agents only, and branches over the product of the subsets’ policies; the cost becomes exponential in the size of the largest coupled subset rather than of their union. $ODrM^*$ [43] adds operator decomposition inside the sub-searches and was, at its publication, among the strongest optimal MAPF solvers on the standard benchmarks; Wagner and Choset [173] give the full treatment and the proofs. **Inflated M^*** replaces h by wh with $w > 1$ and returns a plan of cost at most wC^* with far fewer expansions inside the coupled patches; it is to M^* what ECBS is to CBS.

Relatives. Meta-agent CBS [150] merges two agents into a jointly planned meta-agent once they have conflicted more than a threshold number of times, the bridge between CBS and independence detection. No constructive method bounds the cost of its plan; when a bound matters, ECBS or inflated M^* is the right tool.

11.8.1 Every MAPF method of the book in one table

Table 11.3 places the methods of Chapters 8 to 10 and of this chapter side by side. Completeness is the promise to find a plan whenever one exists; only prioritized planning breaks it. The searches of a finite joint space (joint A^* , independence detection, M^*) and Push-and-Rotate also terminate on unsolvable instances and report failure (Theorem 11.12), which CBS and

Table 11.3. The multi-agent path finding methods of this book. “Bounded” means bounded suboptimality with a user-chosen factor w ; “scales with” names the quantity that dominates the running time.

Method	Complete?	Optimal?	Bounded?	Scales with	Typical use	Ch.
Prioritized planning	no in general; yes on well-formed instances	no	no	k single-agent space-time searches	very large swarms, real-time fallback	8
Joint A^* , with OD	yes	yes	–	$ V ^k$ states; $(b+1)^k$ moves per node, $b+1$ with OD	baseline, $k \leq 4$	11
Independence detection	yes, with a complete group solver	yes, with an optimal group solver	inherited	size of the largest conflicting group	wrapper around any solver	11
CBS (ICBS)	yes on solvable instances	yes	–	conflicts resolved: size of the constraint tree	global planner of Chapter 24 , tens of drones	9
ECBS	yes on solvable instances	no	yes, w	conflicts, fewer as w grows	same, hundreds of drones	10
M^* (r M^* , ODr M^*)	yes	yes; inflated: no	inflated: w	size of the largest collision set	sparse interactions, optimal plans	11
Push-and-Swap, Push-and-Rotate	yes with ≥ 2 empty vertices (P&R); reports unsolvable	no	no	polynomial in $ V $ and k	dense, puzzle-like instances; any feasible plan	11

ECBS do not promise. Optimality is paid for in the size of a constraint tree or of the collision sets, and a bound is the middle ground that two methods offer.

11.9 Implementation notes

The file `ch11_mstar.py` contains a small graph class built from an ASCII map, the individual policies, joint A^* with optional operator decomposition, simple independence detection, M^* , a plan validator and the push-and-swap primitives. Every generated configuration owns a record with its g , parent, collision set (a Python `set`; a bit mask is faster for $k > 16$), back set and an `in_open` flag. The heap holds tuples $(f, -g, \text{counter}, \text{configuration})$ as in [Chapter 4](#); a node is pushed when its g improves and when its collision set grows while it is not in the queue, and a stale entry is skipped when popped. One breadth-first search per goal gives the distance tables, and the next policy step is cached. [Listing 11.1](#) shows the three pieces that make M^* differ from joint A^* : the limited neighbors of [Definition 11.6](#) as a Cartesian product of per-agent move lists (wait first, so that ties are broken deterministically), the backpropagation with an explicit stack instead of recursion, and the successor loop; the two `rec` lines record the trace behind [Table 11.2](#).

Listing 11.1. The core of M^* (from `code/ch11_mstar.py`): the limited neighbors, the backpropagation (a local function of `mstar`) and the successor loop of the main loop.

```

1 def limited_neighbors(node, inst, pol):
2     """Agents in the collision set may move anywhere; the rest follow their
   policy."""

```

```

3     options = []
4     for i, v in enumerate(node.config):
5         if i in node.collision_set:
6             options.append(agent_moves(inst.graph, v))
7         else:
8             options.append([pol.step(i, v)])
9     return itertools.product(*options)
10 # --- local function of mstar(); nodes, INF and push are locals of mstar ---
11     def backprop(config, colset):
12         """Add colset to the collision set of config and of all its
13         ancestors."""
14         stack = [(config, colset)]
15         while stack:
16             cfg, cs = stack.pop()
17             node = nodes[cfg]
18             if cs <= node.collision_set:
19                 continue
20             node.collision_set |= cs
21             if not node.in_open and node.g < INF:
22                 push(node) # re-expand with more neighbors
23             for p in node.back_set:
24                 stack.append((p, node.collision_set))
25 # --- inside the main loop of mstar, after the node with configuration cfg was
26 # popped ---
27     for nxt in limited_neighbors(node, inst, pol):
28         generated += 1
29         child = nodes.get(nxt)
30         if child is None:
31             child = nodes[nxt] = MstarNode(nxt)
32         child.back_set.add(cfg)
33         col = collisions(cfg, nxt)
34         if col:
35             # a vertex collision belongs to the configuration nxt, a swap
36             # collision only to the edge cfg -> nxt
37             child.collision_set |= vertex_collisions(nxt)
38             rec["collisions"].append((nxt, frozenset(col)))
39             backprop(cfg, col | child.collision_set)
40             continue
41         backprop(cfg, child.collision_set)
42         new_g = node.g + step_cost(cfg, nxt, goals)
43         if new_g < child.g:
44             child.g = new_g
45             child.parent = cfg
46             push(child)
47             rec["successors"].append((nxt, new_g))

```

A swap is a property of the move, not of the configuration

The first version of the chapter code stored the whole collision set of an illegal move on the successor configuration. For a vertex collision that is right: the configuration itself is impossible. For a swap it is wrong: the configuration after the swap is a legal state that the search may reach later by other moves, and it then dragged a stale collision set into every node that generated it; on [Example 11.8](#) this coupled drones 1 and 2 in nodes where they had long passed each other. The search stayed correct, because larger collision sets never remove an optimal path, but it did more work than necessary. Store vertex

collisions on the configuration and swap collisions on the parent only.

All parents, and always re-open

The back set of a node must contain *every* node from which it was generated, not only the parent of its cheapest path, and a node whose collision set grows must be pushed back into OPEN even though it has already been expanded. Dropping either shortcut makes M^* incomplete: a collision found downstream never reaches the node that needs to branch, its limited neighbors never include the detour, and the search reports failure on solvable instances. The self-test guards against this by comparing the cost of M^* with joint A^* on random instances.

For the constructive methods, the class `PushSwapState` holds the positions and a log of single-agent moves; `clear`, `push`, `swap` and `rotate` implement the primitives of [Section 11.7](#), and a swap is reversed *by position*: for every logged preparatory move, whoever now stands on its target vertex moves back to its source, which is exactly the reversal with the agents exchanged. `rotate` turns a cycle whose first vertex is empty in $m - 1$ moves (line 2 of [Algorithm 11.4](#)), and the self-test runs it both that way and as the six-move maneuver of [Figure 11.6\(c\)](#). The driver `push_and_swap` solves the examples of the chapter but is not the complete algorithm: it has no resolve step, no graph decomposition and no call to `rotate`, so it is Push-and-Swap and not [Algorithm 11.4](#).

11.10 Where this fits in the drone system

In the drone system

The global planner of the hybrid architecture in [Chapter 24](#) is CBS or ECBS, and this chapter explains why: a swarm planner should couple drones only where they interact, and it should do so with a guarantee. CBS couples through constraints and never enumerates a joint space; ECBS adds a bound the operator can set. Independence detection is the cheapest way to make any of these solvers scale, because the largest conflicting group rather than the whole swarm then decides the running time. M^* is the right global planner when interactions are rare but must be resolved optimally, for instance for a few dozen drones on a large map with occasional crossings, and inflated M^* when a bound is enough. A feasible plan is worth accepting when the alternative is no plan at all: a dense hangar in which drones must be reshuffled before take-off, or a replanning request that must be answered within one control cycle, is where Push-and-Rotate's guarantee earns its place as a fallback. What none of these methods does is react: they plan for the swarm's own members on a known map, and the intruder that appears after take-off is the business of the local safety and prediction layers of [Chapters 12, 13 and 18](#). This is the optional second half of Week 5 of the study plan ([Appendix A](#)), which asks for awareness of M^* and Push-and-Swap next to ECBS; its coding exercise is [Exercise 11.7](#).

11.11 Summary

Chapter summary

- The joint state space of k agents has $|V|^k$ configurations and $(b+1)^k$ joint moves per node; joint A^* with the sum-of-distances heuristic is complete and optimal but pays the full branching factor at every expansion. Operator decomposition cuts it to $b+1$ by moving one agent at a time through intermediate states.
- Independence detection solves groups separately and merges two groups only when their optimal plans conflict; with an optimal group solver the result is optimal (Proposition 11.7), and the largest group governs the running time.
- M^* follows the individual optimal policies through the joint space and branches only over the agents in a node's collision set. Collisions found downstream are backpropagated along the back sets, and a node whose collision set grew is expanded again. Vertex collisions belong to the configuration, swap collisions to the move.
- With optimal policies and an admissible heuristic, M^* terminates and is complete and optimal (Theorem 11.10); without collisions it expands only the policy path (Corollary 11.11); in the worst case it is joint A^* plus overhead. Recursive, inflated and OD variants exist.
- Push-and-Swap and Push-and-Rotate move one agent at a time with the push, swap and rotate primitives. Push-and-Rotate is complete for instances with at least two empty vertices; both are feasibility solvers whose plans can be several times longer than optimal.
- Table 11.3 summarizes every MAPF method of the book; the swarm planner of Chapter 24 uses CBS/ECBS, with independence detection and M^* as exact alternatives when interactions are sparse and Push-and-Rotate as a fallback when any feasible plan will do.

Further reading. The joint-space formulation, operator decomposition and independence detection are from Standley [156]; Standley and Korf [157] make them complete and anytime. M^* was introduced by Wagner and Choset [172] and developed, with recursive and inflated variants and the full proofs, in the journal version [173]; ODr M^* is in [43]. Push-and-Swap is [110]; Push-and-Rotate, its completeness proof and the correction of the earlier claims are in [178]; the pebble-motion theory behind them is [92] and [180], and BIBOX is [164]. The definitions of conflicts and objectives follow Stern et al. [161], and Felner et al. [41] survey the search-based optimal solvers, including CBS, ICTS, EPEA * and M^* . LaValle [99] treats the joint state space of this chapter at textbook level.

11.12 Exercises

Exercise 11.1 (★). A warehouse map has $|V| = 300$ free cells and is 4-connected. (a) How many joint configurations, collision-free configurations and joint moves per node does the joint graph have for $k = 3$ and for $k = 12$ agents? (b) Operator decomposition has $b + 1 = 5$ successors per state but needs k intermediate levels to move all agents once; compare the number of states generated by one full joint step of plain joint A^* and of OD for $k = 12$, assuming no pruning. (c) Explain in two sentences why a good heuristic does not rescue plain joint A^* from the branching factor.

Exercise 11.2 (★). Take the four-vertex graph of [Figure 11.2](#) and add a third agent that starts at d with goal d ; agents 1 and 2 are as in the figure. Run [Algorithm 11.2](#) by hand for the first three expansions: give the policy of every agent, the nodes expanded, the collisions found, the collision sets after backpropagation and the limited neighbors of the re-expanded start. At which expansion does agent 3 first enter a collision set, and which move puts it there? What is the collision set of the start when the search stops? What does M^* return on this instance, and which theorem guarantees that it returns at all?

Exercise 11.3 (★★). (a) Prove that $h_{\max}(v) = \max_i d_i(v^i)$ is admissible for the cost [Equation \(11.1\)](#) and dominated by [Equation \(11.2\)](#) in the sense of [Chapter 4](#). (b) Suppose waiting were free everywhere, not only at the goal. Is [Equation \(11.2\)](#) still consistent, and is the joint search still finite? (c) The makespan objective of [Chapter 7](#) charges one per time step regardless of how many agents move. Give a joint move cost and an admissible heuristic for it, and explain why the policy path is still an optimal continuation when it is conflict-free.

Exercise 11.4 (★★). (a) Construct a three-agent instance on which simple independence detection ([Algorithm 11.1](#)) merges all three agents although agent 3 has a path that conflicts with nobody (a group may have several optimal plans). (b) How does Standley’s conflict avoidance table change the outcome? (c) Show that [Proposition 11.7](#) still holds when the group solver is M^* or CBS, and what remains of it when it is ECBS.

Exercise 11.5 (★★). Run `mstar(inst, trace=True)` on [Example 11.8](#) and print the 31 expansions. (a) Twelve nodes are expanded twice; for each, say whether its collision set grew from a collision among its own successors or was handed up from a child (line 13). (b) Explain why M^* cannot know the collision set of a node before searching below it, so that the second expansion of the start (step 4) is unavoidable. (c) Design an instance with three drones in one aisle whose start collision set grows twice, first to $\{1, 2\}$ and later to $\{1, 2, 3\}$, and check your prediction with the code.

Exercise 11.6 (★★). (a) Prove that the swap of [Section 11.7](#) needs exactly $4d + 6$ single-agent moves when the leading agent is d steps from the junction and no other agent has to be cleared. (b) Executed one move per time step, what are the makespan and sum of costs of the swap example of [Section 11.7.4](#) as functions of d , and those of the optimal joint plan? (c) Show that the six exchange moves of [Figure 11.6\(b\)](#) parallelize into exactly three time steps and no fewer, under the convention of [Section 11.3](#) that an agent may follow another into the cell it is leaving, and give the pairing; show also that the multipush parallelizes into d time steps.

Exercise 11.7 (★★★ Coding). The study plan ([Appendix A](#), Week 5) asks only for awareness of M^* and Push-and-Swap, so treat this as an optional extension. (a) Implement joint A^* for k agents on a grid, with [Equation \(11.2\)](#) and [Equation \(11.1\)](#), and check it against `joint_astar` on random 5×4 instances with two to four agents. (b) Add independence detection, with Standley’s refinement that re-plans one group around the other’s at equal cost before merging; count how often that avoids a merge on random 8×8 maps with six agents, and reproduce the right panel of [Figure 11.5](#). (c) Implement the push and swap primitives on an abstract graph, including the reversal by position; solve the swap example of [Section 11.7.4](#) and the push example of [Figure 11.6\(a\)](#), and compare makespan and sum of costs with joint A^* . (d) Optional: implement `rotate` and build an instance with a fully occupied cycle on which push-and-swap fails and push-and-rotate succeeds.

Exercise 11.8 (★★★). (a) Give an instance with two empty vertices that has no solution, and say in one sentence why no algorithm can solve it. (b) Put four agents on a cycle of five vertices, one empty, with goals that advance each agent by one position: pushes alone solve it, so “at least two empty vertices” is sufficient but not necessary. Change the goals so that two agents must exchange places, and argue that the instance is then unsolvable. (c) CBS

(Chapter 9) runs for ever on some unsolvable instances. Explain why M^* always terminates (Theorem 11.9) and why CBS needs an extra argument, using (a) as a test case.

Part IV

Local and Reactive Collision Avoidance

Velocity Obstacles

Learning objectives

- Explain why local collision avoidance is a choice of *velocity* rather than a choice of path, and set up an encounter between two disks in relative coordinates.
- Construct the collision cone and the velocity obstacle of a moving disk by hand: apex, tangent rays and the half-angle $\theta = \arcsin((r_A + r_B) / \|\mathbf{p}_B - \mathbf{p}_A\|)$.
- Compute the time to collision of a candidate velocity from a quadratic equation, and truncate the velocity obstacle with a time horizon τ .
- Choose a velocity outside the union of all velocity obstacles that stays as close as possible to the preferred velocity, by sampling or by projection.
- Implement a velocity-obstacle simulator in Python, measure minimum separation and path length, and recognize the oscillation that appears when two agents avoid each other at the same time.

12.1 Why this matters for a drone swarm

Everything in [Chapters 3 to 11](#) plans *positions*: a path is a sequence of cells, and time enters, if at all, as a discrete index. Such plans are computed before the flight, and they are only as good as the map they were computed on. Now picture the moment that the plan did not foresee. A delivery drone follows its conflict-based-search (CBS) route across a park at 1 m/s when an unknown quadrotor appears 7 m away, flying at 0.9 m/s on a course that crosses the drone's own. The tracker of [Chapter 18](#) reports its position and velocity; nothing says that it will yield. Within five seconds the two aircraft will occupy the same piece of sky. Replanning the whole route through a graph is the wrong tool at this time scale: it takes too long, it needs a map that does not yet contain the intruder, and the intruder will have moved by the time the new route is ready. What the drone needs is an answer to a much smaller question: *which velocity should I fly during the next fraction of a second so that no collision occurs?*

The velocity obstacle (VO) of Fiorini and Shiller [45] answers exactly this question. It looks at the encounter from the drone's own point of view, in which the intruder is a disk that moves with the *relative* velocity of the two aircraft. It then describes, as a single cone in the space of velocities, every velocity of the drone that leads to a collision if the intruder keeps its current velocity. Avoiding the intruder means picking a velocity outside the cone, and picking the one that is closest to the velocity the drone wanted to fly anyway makes the maneuver small. The computation is a handful of dot products per obstacle, which is why VO and its descendants run at hundreds of hertz on board of small vehicles.

In the four-layer architecture of [Chapter 24](#), velocity obstacles are the *local safety layer*. The global planner (CBS or ECBS, enhanced CBS, [Chapters 9 and 10](#)) produces nominal routes;

the prediction layer (Chapters 18 and 20) delivers, for every tracked object, an estimated position and velocity; the local layer takes over the velocity command whenever a predicted collision is closer than a safety horizon and hands control back once the danger has passed. The reciprocal variants of Chapter 13 — reciprocal velocity obstacles (RVO) and optimal reciprocal collision avoidance (ORCA) — are what the swarm members use among themselves; the plain velocity obstacle of this chapter is what a drone uses against an object that does not cooperate.

The chapter proceeds as follows. Section 12.2 gives the idea in one picture. Section 12.3 derives the geometry: relative motion, the collision cone and its tangent half-angle, the velocity obstacle, the time to collision and the truncated velocity obstacle. Section 12.4 turns the geometry into three short algorithms (membership test, velocity selection, simulation), Section 12.5 runs them on the crossing encounter above with every number produced by the chapter’s code, and Section 12.6 states what velocity obstacles guarantee and what they do not. Section 12.7 shows the oscillation that arises when two agents avoid each other simultaneously, together with an experiment on crossing agents; it is the motivation for Chapter 13. Section 12.8 covers multiple and static obstacles and Section 12.9 the cone in three dimensions. Section 12.10 and Section 12.11 discuss the implementation and the place of VO in the drone system.

12.2 The idea in plain words

Two disks move through the plane with constant velocities. Whether they will collide does not depend on where either of them is going in absolute terms; it depends only on how one moves *relative* to the other. So sit on drone A . From there, drone B is an object at the relative position $\mathbf{p}_B - \mathbf{p}_A$ that drifts at the rate $\mathbf{v}_B - \mathbf{v}_A$, the negative of the relative velocity of Definition 12.1. Two more simplifications make the picture crisp. First, by the Minkowski-sum argument of Proposition 2.28, two disks of radii r_A and r_B overlap exactly when their centers are closer than $r_A + r_B$; so we may shrink A to a point and fatten B to a disk of radius $r_A + r_B$. Second, instead of letting the fat disk drift toward the point, let the point move toward the fixed disk with the velocity $\mathbf{v}_A - \mathbf{v}_B$; the two descriptions are the same motion seen from different frames.

The question “will they collide?” has now become a question about a ray: the point starts at the origin and travels along the ray in the direction of the relative velocity. It hits the disk if and only if the ray intersects the disk (Figure 12.1). The directions whose rays hit the disk form a wedge between the two tangent lines from the origin to the disk. This wedge is the **collision cone**. A relative velocity inside it leads to a collision, sooner if the velocity is large and later if it is small; a relative velocity outside it never does, however fast.

The final step translates the picture back to the velocities that A can actually command. Since the relative velocity is $\mathbf{v}_A - \mathbf{v}_B$, the condition “ $\mathbf{v}_A - \mathbf{v}_B$ lies in the cone” is the condition “ $\mathbf{v}_A$ lies in the cone shifted by $\mathbf{v}_B$ ”. The shifted cone is the **velocity obstacle**: a wedge in A ’s velocity space with its apex at $\mathbf{v}_B$. To avoid B , agent A chooses any velocity outside the wedge, and among those it chooses the one closest to the velocity it prefers. Two refinements make this practical. A collision that lies far in the future need not be avoided yet, so the wedge is cut off by a time horizon. And with several obstacles, A avoids the union of their wedges.

Key idea

Look at an encounter from the agent’s own frame: the other agent is a disk of radius $r_A + r_B$, and the relative velocity $\mathbf{v}_A - \mathbf{v}_B$ is dangerous exactly when its ray hits that disk. The dangerous relative velocities form a cone with half-angle $\arcsin((r_A + r_B) / \|\mathbf{p}_B - \mathbf{p}_A\|)$;

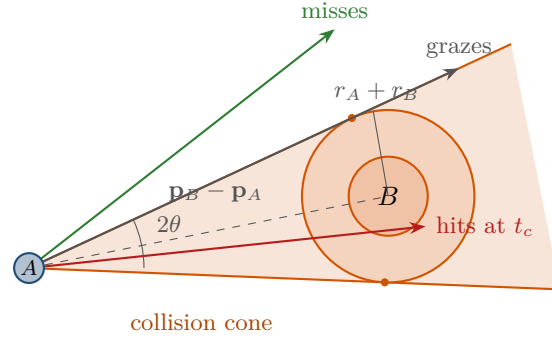


Figure 12.1. The encounter seen from A . Agent A is a point at the origin, agent B is the disk of radius $r_A + r_B$ (its own radius is drawn inside) at the relative position $\mathbf{p}_B - \mathbf{p}_A$. A relative velocity $\mathbf{v}_A - \mathbf{v}_B$ is dangerous exactly when its ray from the origin meets the disk: the red ray hits, the gray ray grazes the disk along a tangent, the green ray misses. The shaded wedge between the tangents is the collision cone.

the velocity obstacle is this cone translated to the apex $\mathbf{v}_B$. Avoiding a collision means choosing a velocity outside the velocity obstacle, as close as possible to the preferred one.

12.3 Problem statement and notation

12.3.1 Relative motion

Throughout, an **agent** is a disk that moves as a point mass with a commanded velocity (the point-mass drone model of Definition 2.21): its center obeys $\mathbf{p}(t) = \mathbf{p} + t\mathbf{v}$ as long as the velocity $\mathbf{v}$ is held. Agent A has center $\mathbf{p}_A$, velocity $\mathbf{v}_A$ and radius r_A ; agent B (a drone, a bird, a static pole) has $\mathbf{p}_B$, $\mathbf{v}_B$, r_B . Velocities may be changed instantaneously; acceleration limits are discussed in Section 12.8 and in Chapter 14.

Definition 12.1 (Relative position and velocity). The **relative position** of B with respect to A and the **relative velocity** of A with respect to B are

$$\mathbf{p}_{\text{rel}} = \mathbf{p}_B - \mathbf{p}_A, \quad \mathbf{v}_{\text{rel}} = \mathbf{v}_A - \mathbf{v}_B. \quad (12.1)$$

The **combined radius** is $R = r_A + r_B$ and the current distance is $d = \|\mathbf{p}_{\text{rel}}\|$. We assume $d > R$: the agents are apart now.

The sign conventions matter and are chosen so that A is the mover. If both agents keep their velocities, the center of B seen from the center of A is

$$\mathbf{p}_B(t) - \mathbf{p}_A(t) = (\mathbf{p}_B + t\mathbf{v}_B) - (\mathbf{p}_A + t\mathbf{v}_A) = \mathbf{p}_{\text{rel}} - t\mathbf{v}_{\text{rel}}, \quad (12.2)$$

which is the position of a *fixed* disk center $\mathbf{p}_{\text{rel}}$ seen from a point that has moved to $t\mathbf{v}_{\text{rel}}$. By the Minkowski-sum argument for two disks (Proposition 2.28), the disks overlap at time t if and only if $\|\mathbf{p}_B(t) - \mathbf{p}_A(t)\| \leq R$, that is,

$$A \text{ and } B \text{ collide at time } t \geq 0 \iff \|\mathbf{p}_{\text{rel}} - t\mathbf{v}_{\text{rel}}\| \leq R \iff t\mathbf{v}_{\text{rel}} \in D(\mathbf{p}_{\text{rel}}, R), \quad (12.3)$$

where $D(\mathbf{c}, r)$ is the closed disk of radius r around $\mathbf{c}$. The disk $D(\mathbf{p}_{\text{rel}}, R) = D(\mathbf{p}_{\text{rel}}, r_B) \oplus D(\mathbf{0}, r_A)$ is the Minkowski sum of B 's disk with A 's disk placed at the origin: A has become a point, B has grown by r_A .

12.3.2 The collision cone

Definition 12.2 (Collision cone). The **collision cone** of A induced by B is the set of relative velocities that lead to a collision at some future time,

$$CC_{A|B} = \left\{ \mathbf{v}_{\text{rel}} \in \mathbb{R}^2 : \exists t > 0 \text{ with } t\mathbf{v}_{\text{rel}} \in D(\mathbf{p}_{\text{rel}}, R) \right\}. \quad (12.4)$$

Equivalently, $\mathbf{v}_{\text{rel}} \in CC_{A|B}$ if and only if the ray $\{t\mathbf{v}_{\text{rel}} : t > 0\}$ from the origin intersects the disk $D(\mathbf{p}_{\text{rel}}, R)$.

The cone depends on $\mathbf{p}_{\text{rel}}$ and R only, not on any velocity. It contains, with any vector, every positive multiple of that vector: if the ray in direction $\mathbf{v}$ hits the disk, so does the ray in direction $2\mathbf{v}$, only sooner. This is what makes $CC_{A|B}$ a cone, and it is why the whole set is described by an angle.

Theorem 12.3 (Tangent half-angle of the collision cone). Let $d = \|\mathbf{p}_{\text{rel}}\| > R$ and let

$$\theta = \arcsin \frac{R}{d} \in \left(0, \frac{\pi}{2}\right). \quad (12.5)$$

Then $CC_{A|B}$ is the closed cone of all non-zero vectors that make an angle of at most θ with $\mathbf{p}_{\text{rel}}$: $CC_{A|B} = \{\mathbf{v} \neq \mathbf{0} : \angle(\mathbf{v}, \mathbf{p}_{\text{rel}}) \leq \theta\}$. Its two boundary rays are the tangent rays from the origin to the circle of radius R around $\mathbf{p}_{\text{rel}}$; they touch the circle at distance $\sqrt{d^2 - R^2}$ from the origin, and their directions are $\mathbf{p}_{\text{rel}}/d$ rotated by $+\theta$ and by $-\theta$.

Proof. Let $\mathbf{v} \neq \mathbf{0}$, write $\mathbf{u} = \mathbf{v} / \|\mathbf{v}\|$ and let $\varphi \in [0, \pi]$ be the angle between $\mathbf{u}$ and $\mathbf{p}_{\text{rel}}$. The ray $\{t\mathbf{u} : t \geq 0\}$ meets the disk if and only if the distance from the center $\mathbf{p}_{\text{rel}}$ to the ray is at most R . If $\varphi \leq \pi/2$, the point of the line $\{t\mathbf{u} : t \in \mathbb{R}\}$ closest to the center is at $t^* = \mathbf{p}_{\text{rel}} \cdot \mathbf{u} = d \cos \varphi \geq 0$, hence on the ray, and its distance to the center is $d \sin \varphi$ (the perpendicular from the center to the line; Definition 2.26). If $\varphi > \pi/2$, the closest point of the ray is the origin itself, at distance $d > R$, and the ray misses. Therefore the ray meets the disk if and only if $\varphi \leq \pi/2$ and $d \sin \varphi \leq R$, i.e., if and only if $\varphi \leq \arcsin(R/d) = \theta$, because $\sin$ is increasing on $[0, \pi/2]$ and $R/d < 1$. This proves the description of the cone. A ray with $\varphi = \theta$ has distance exactly R to the center, so it touches the circle at one point $\mathbf{t}$; the radius to $\mathbf{t}$ is perpendicular to the ray, and the right triangle with vertices $\mathbf{0}$, $\mathbf{t}$, $\mathbf{p}_{\text{rel}}$ has hypotenuse d and legs R and $\|\mathbf{t}\| = \sqrt{d^2 - R^2}$, with $\sin \theta = R/d$ at the origin. These are the tangent points of Proposition 2.29. $\square$

The theorem is the whole geometry of this chapter, so it is worth reading Equation (12.5) twice. The cone is narrow when the obstacle is far (R/d small) and opens as the obstacle approaches; when d shrinks to R the half-angle reaches 90° and every velocity that has any component toward B is dangerous. Forgetting to add r_A to r_B is the most common mistake in this construction; it makes the cone too narrow and the agent grazes the obstacle (Section 12.10).

12.3.3 The velocity obstacle

Definition 12.4 (Velocity obstacle). The **velocity obstacle** of A induced by B moving with velocity $\mathbf{v}_B$ is the collision cone translated by $\mathbf{v}_B$,

$$VO_{A|B}(\mathbf{v}_B) = \mathbf{v}_B \oplus CC_{A|B} = \left\{ \mathbf{v}_B + \mathbf{v} : \mathbf{v} \in CC_{A|B} \right\} = \left\{ \mathbf{v}_A \in \mathbb{R}^2 : \mathbf{v}_A - \mathbf{v}_B \in CC_{A|B} \right\}. \quad (12.6)$$

A velocity $\mathbf{v}_A$ is **dangerous** if $\mathbf{v}_A \in VO_{A|B}(\mathbf{v}_B)$ and **safe** otherwise. When $\mathbf{v}_B$ is clear from the context we write $VO_{A|B}$.

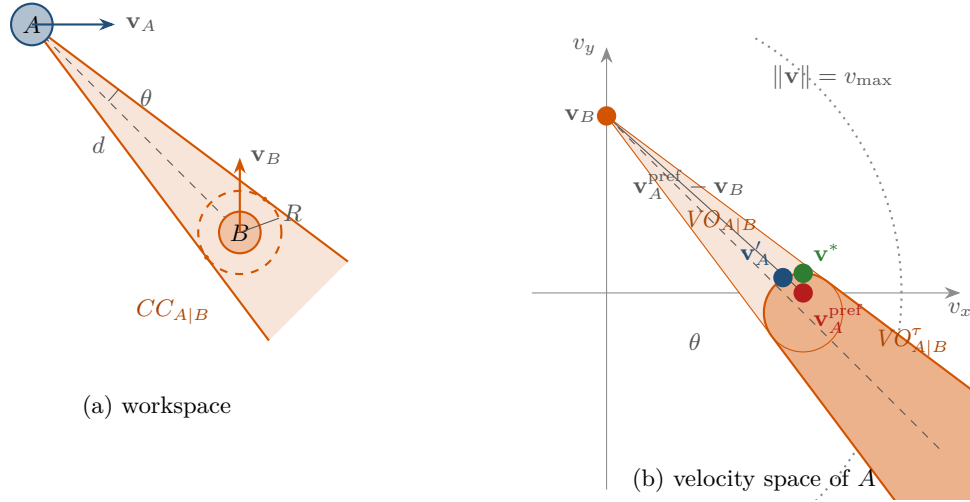


Figure 12.2. The velocity obstacle of the worked example. (a) Workspace: A at $(-5, 0)$ with $r_A = 0.5$ flies east at 1 m/s, B at $(0, -5)$ with $r_B = 0.5$ flies north at 0.9 m/s. The dashed circle is the inflated disk of radius $R = 1$; the tangents from the center of A enclose the collision cone of half-angle $\theta = \arcsin(1/\sqrt{50}) = 8.13^\circ$ around the direction to B . (b) Velocity space of A : the same wedge with its apex at $\mathbf{v}_B = (0, 0.9)$ is $VO_{A|B}$; its legs have the directions $(0.8, -0.6)$ and $(0.6, -0.8)$. The preferred velocity $\mathbf{v}_A^{\text{pref}} = (1, 0)$ lies inside, so A must deviate. The small circle is the truncation disk for $\tau = 5$ s (Section 12.3.5); only the darker region is forbidden. The velocity $\mathbf{v}^*$ is the point of the boundary closest to $\mathbf{v}_A^{\text{pref}}$ and $\mathbf{v}_A'$ is the sampled choice of Section 12.5. The dotted circle bounds the reachable velocities $\|\mathbf{v}\| \leq v_{\max} = 1.5$.

The velocity obstacle lives in the velocity space of A : its points are velocities that A could command, and $VO_{A|B}$ is the set of those that A must not command. It is a wedge with its **apex** at $\mathbf{v}_B$, its axis parallel to $\mathbf{p}_{\text{rel}}$ and the half-angle θ of Theorem 12.3. Unlike the collision cone, it is not closed under scaling unless $\mathbf{v}_B = \mathbf{0}$, and it changes whenever B changes its velocity. Figure 12.2 shows both pictures for the encounter of the introduction, which is also the worked example of Section 12.5: in the workspace on the left, the cone of dangerous relative directions sits at A and the inflated disk around B ; in the velocity space on the right, the same wedge sits at the apex $\mathbf{v}_B$, and the preferred velocity $\mathbf{v}_A^{\text{pref}}$ of A falls inside it.

12.3.4 Time to collision

Membership in the cone says *whether* a velocity collides. For the truncation of the next section, and for choosing between bad options, we need to know *when*.

Definition 12.5 (Time to collision). The **time to collision** of a relative velocity $\mathbf{v}_{\text{rel}}$ is the first time at which the disks touch,

$$t_c(\mathbf{v}_{\text{rel}}) = \min \{t \geq 0 : \|\mathbf{p}_{\text{rel}} - t\mathbf{v}_{\text{rel}}\| \leq R\}, \quad (12.7)$$

with $t_c = \infty$ if the set is empty. For an absolute velocity $\mathbf{v}_A$ of A we write $t_c(\mathbf{v}_A - \mathbf{v}_B)$.

Squaring the condition $\|\mathbf{p}_{\text{rel}} - t\mathbf{v}_{\text{rel}}\| = R$ gives a quadratic equation in t . With the abbreviations $\mathbf{p} = \mathbf{p}_{\text{rel}}$ and $\mathbf{v} = \mathbf{v}_{\text{rel}}$,

$$(\mathbf{v} \cdot \mathbf{v}) t^2 - 2(\mathbf{p} \cdot \mathbf{v}) t + (\mathbf{p} \cdot \mathbf{p} - R^2) = 0, \quad \frac{\Delta}{4} = (\mathbf{p} \cdot \mathbf{v})^2 - (\mathbf{v} \cdot \mathbf{v})(\mathbf{p} \cdot \mathbf{p} - R^2). \quad (12.8)$$

This is the ray–circle intersection of Proposition 2.27 with the ray starting at the origin in the direction $\mathbf{v}$ and the circle centered at $\mathbf{p}$.

Proposition 12.6 (Time to collision). *Assume $d > R$ and $\mathbf{v} \neq \mathbf{0}$. Then $\mathbf{v} \in CC_{A|B}$ if and only if $\Delta \geq 0$ and $\mathbf{p} \cdot \mathbf{v} > 0$, and in that case*

$$t_c(\mathbf{v}) = \frac{\mathbf{p} \cdot \mathbf{v} - \sqrt{\Delta/4}}{\mathbf{v} \cdot \mathbf{v}} > 0, \quad (12.9)$$

the smaller root of Equation (12.8). The disks overlap exactly during the interval between the two roots. If $\mathbf{v} = \mathbf{0}$ there is no collision.

Proof. The function $t \mapsto \|\mathbf{p} - t\mathbf{v}\|^2 - R^2$ is a parabola opening upward with the value $d^2 - R^2 > 0$ at $t = 0$. It is non-positive exactly between its two real roots, if they exist, so the disks overlap exactly on that interval, and a future collision exists if and only if the roots are real ($\Delta \geq 0$) and the smaller one is positive. The product of the roots is $(\mathbf{p} \cdot \mathbf{p} - R^2)/(\mathbf{v} \cdot \mathbf{v}) > 0$, so the roots have the same sign, which is the sign of their sum $2(\mathbf{p} \cdot \mathbf{v})/(\mathbf{v} \cdot \mathbf{v})$. Hence both are positive if and only if $\mathbf{p} \cdot \mathbf{v} > 0$, and then the first contact is at the smaller root Equation (12.9). For $\mathbf{v} = \mathbf{0}$ the distance stays $d > R$. $\square$

The discriminant condition is the cone condition in disguise: with φ the angle between $\mathbf{v}$ and $\mathbf{p}$, a short computation gives $\Delta/4 = \|\mathbf{v}\|^2 (R^2 - d^2 \sin^2 \varphi)$, so $\Delta \geq 0$ is exactly $d \sin \varphi \leq R$ as in the proof of Theorem 12.3, and $\mathbf{p} \cdot \mathbf{v} > 0$ excludes the mirror-image directions that point away from the disk. A velocity with $\Delta = 0$ is grazing: the disks touch at a single instant. The worked example of Section 12.5 evaluates Equation (12.9) on the encounter of Figure 12.2.

12.3.5 The truncated velocity obstacle

Taken literally, Definition 12.4 forbids a velocity that would cause a collision in ten minutes with a drone at the other end of the city. Fiorini and Shiller already distinguished *imminent* collisions from those that lie beyond a time horizon [45], and every practical implementation cuts the cone off.

Definition 12.7 (Truncated velocity obstacle). For a **time horizon** $\tau > 0$, the **truncated velocity obstacle** is the set of velocities that collide within the horizon,

$$VO_{A|B}^\tau(\mathbf{v}_B) = \{\mathbf{v}_A : t_c(\mathbf{v}_A - \mathbf{v}_B) \leq \tau\} \subseteq VO_{A|B}(\mathbf{v}_B). \quad (12.10)$$

Proposition 12.8 (Shape of the truncated velocity obstacle). *In relative-velocity coordinates the truncated cone is a union of disks,*

$$VO_{A|B}^\tau(\mathbf{v}_B) - \mathbf{v}_B = \bigcup_{0 < t \leq \tau} D(\mathbf{p}_{\text{rel}}/t, R/t). \quad (12.11)$$

Every disk $D(\mathbf{p}_{\text{rel}}/t, R/t)$ is inscribed in the collision cone (tangent to both legs), and the disks shrink and move toward the apex as t grows. Consequently $VO_{A|B}^\tau$ is the cone with its tip cut off by the disk $D(\mathbf{v}_B + \mathbf{p}_{\text{rel}}/\tau, R/\tau)$: its boundary consists of the two legs beyond their points of tangency with this disk and of the arc of the disk that faces the apex.

Proof. By Equation (12.3), $\mathbf{v}$ collides at time t if and only if $\|\mathbf{p}_{\text{rel}} - t\mathbf{v}\| \leq R$, i.e., if and only if $\|\mathbf{p}_{\text{rel}}/t - \mathbf{v}\| \leq R/t$, i.e., $\mathbf{v} \in D(\mathbf{p}_{\text{rel}}/t, R/t)$. Since $t_c(\mathbf{v}) \leq \tau$ means that $\mathbf{v}$ collides at some $t \leq \tau$, Equation (12.11) follows. The disk for t is the image of $D(\mathbf{p}_{\text{rel}}, R)$ under the scaling $\mathbf{x} \mapsto \mathbf{x}/t$ about the origin. A scaling about the origin maps every ray from the origin onto itself and preserves tangency, so the two tangent rays of $D(\mathbf{p}_{\text{rel}}, R)$ are also the tangent rays of every scaled disk. It remains to identify the boundary, and for that we use the scaling identity that Equation (12.3) gives at once: for every $s > 0$ the point $t(s\mathbf{v})$ is the point $(st)\mathbf{v}$, so

$$t_c(s\mathbf{v}) = t_c(\mathbf{v})/s. \quad (12.12)$$

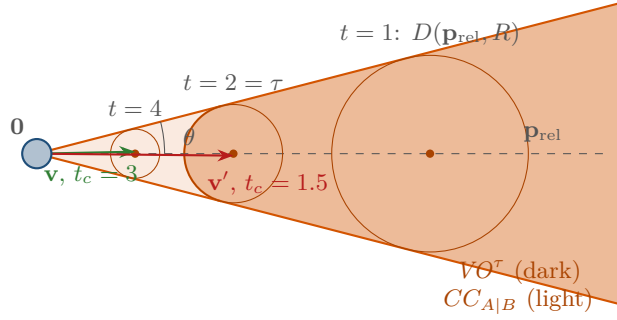


Figure 12.3. Truncation, drawn in relative-velocity coordinates for $\mathbf{p}_{\text{rel}} = (4, 0)$ and $R = 1$ (half-angle 14.5°). The relative velocity $\mathbf{v}$ collides at time t exactly when $\mathbf{v}$ lies in the disk $D(\mathbf{p}_{\text{rel}}/t, R/t)$; the disks for $t = 1, 2$ and 4 are drawn, each inscribed in the cone. With the horizon $\tau = 2$ only the dark region beyond the disk for $t = 2$ is forbidden: the velocity $\mathbf{v}$, which collides at $t_c = 3$, is allowed, $\mathbf{v}'$ with $t_c = 1.5$ is not.

Fix a unit vector $\mathbf{u}$ pointing into the cone, so that $t_c(\mathbf{u}) < \infty$. By Equation (12.12), $t_c(s\mathbf{u}) \leq \tau$ holds if and only if $s \geq t_c(\mathbf{u})/\tau$, so the truncated obstacle meets that ray exactly in the half-line $\{s\mathbf{u} : s \geq t_c(\mathbf{u})/\tau\}$. Its endpoint $(t_c(\mathbf{u})/\tau)\mathbf{u}$ is the point at which the ray enters $D(\mathbf{p}_{\text{rel}}/\tau, R/\tau)$, because $\|\mathbf{p}_{\text{rel}}/\tau - s\mathbf{u}\| \leq R/\tau$ is the same inequality as $\|\mathbf{p}_{\text{rel}} - (\tau s)\mathbf{u}\| \leq R$, whose smallest solution is $\tau s = t_c(\mathbf{u})$. Hence the truncated obstacle is the part of the cone lying at or beyond that disk, and its boundary is the near arc of the disk together with the two legs beyond their points of tangency with it. $\square$

Figure 12.3 shows the construction and Figure 12.2(b) its instance in the worked example, where the truncation disk for $\tau = 5$ s has center $\mathbf{v}_B + \mathbf{p}_{\text{rel}}/5 = (1, -0.1)$ and radius $R/5 = 0.2$. The horizon has a clean interpretation: A ignores collisions that are more than τ seconds away, on the grounds that either agent will have changed its velocity before then and a fresh decision will be made at the next time step. It also has a cost: a velocity just outside VO^τ collides in $\tau + \varepsilon$ seconds, so the horizon must be long enough for A to make the maneuver that the next decision will demand. Practical values are a few seconds for slow indoor vehicles and tens of seconds for fast aircraft; Section 12.10 returns to the choice.

12.3.6 Choosing a velocity

The velocity obstacle tells A what not to do. The decision of what to do instead needs two more ingredients: what A can fly, and what A wants.

Definition 12.9 (Reachable, preferred and feasible velocities). The **reachable velocities** of A are the disk $\mathcal{V}_A = D(\mathbf{0}, v_{\text{max}})$ of speeds up to the limit v_{max} of Definition 2.21. The **preferred velocity** $\mathbf{v}_A^{\text{pref}}$ is the velocity A would fly if nothing were in the way; in this chapter it points from $\mathbf{p}_A$ toward the goal at the nominal speed. Given the obstacles $B_1, \dots, B_k$ with velocities $\mathbf{v}_{B_j}$, the **feasible velocities** are

$$\mathcal{F}_A = \mathcal{V}_A \setminus \bigcup_{j=1}^k VO_{A|B_j}^\tau(\mathbf{v}_{B_j}), \quad (12.13)$$

and A selects the feasible velocity closest to its preferred one,

$$\mathbf{v}_A^* = \arg \min_{\mathbf{v} \in \mathcal{F}_A} \|\mathbf{v} - \mathbf{v}_A^{\text{pref}}\|. \quad (12.14)$$

Figure 12.4 shows the sets. Three remarks complete the definition. First, the union in

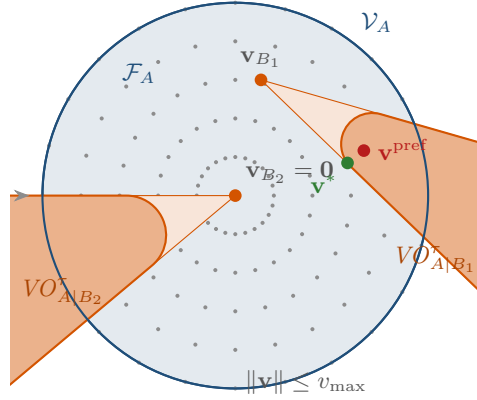


Figure 12.4. Choosing a velocity with two obstacles. The blue disk is the reachable set $\|\mathbf{v}\| \leq v_{\max}$; the dots are the sampled candidate velocities of Algorithm 12.2. The orange wedge with its apex at $\mathbf{v}_{B_1}$ is the truncated velocity obstacle of a moving agent, the wedge with its apex at the origin that of a static obstacle ($\mathbf{v}_{B_2} = \mathbf{0}$). The feasible set is what remains of the disk. The preferred velocity $\mathbf{v}^{\text{pref}}$ is forbidden; the chosen velocity $\mathbf{v}^*$ is the feasible point closest to it, here on the truncation arc of the first obstacle.

Equation (12.13) is how **multiple obstacles** are handled: a velocity is safe only if it is safe with respect to every obstacle, so the forbidden set is the union of the individual wedges, and the feasible set can be small and disconnected. Second, a **static obstacle** is the special case $\mathbf{v}_B = \mathbf{0}$: its velocity obstacle is the collision cone itself, with the apex at the origin, and a polygonal obstacle is handled by the cone spanned by its extreme tangents (Section 12.8). Third, the cost in Equation (12.14) is the simplest sensible one: it prefers small deviations in speed and heading equally. Other costs are possible and easy to use with the sampling algorithm below, for example one that penalizes changes from the *current* velocity to reduce chattering, or one that maximizes the speed toward the goal, which is the heuristic of the original paper [45].

Two things can go wrong with Equation (12.14). The feasible set may be empty, because a fast obstacle is so close that every reachable velocity collides within τ ; then there is no safe choice and A must pick the least dangerous velocity, for example the one that maximizes t_c or the one that minimizes $\|\mathbf{v} - \mathbf{v}^{\text{pref}}\| + \kappa/t_c(\mathbf{v})$ for a weight $\kappa > 0$. And the minimizer may be hard to compute exactly when several wedges overlap; the practical answer is to evaluate the cost on a finite set of candidate velocities, as the algorithm of the next section does, or to project onto one wedge at a time as ORCA does in Chapter 13.

12.4 The algorithm

Velocity obstacles are not a planner with a start and a goal; they are a rule that is applied at every control step. Three pieces make up the rule: the membership test of a velocity (Algorithm 12.1), the selection of a velocity for one agent (Algorithm 12.2), and the loop that applies the selection to every agent and advances the simulation (Algorithm 12.3).

Walkthrough of Algorithm 12.1. The function `TimeToCollision` evaluates Equation (12.9) with the guards that a real implementation needs. The constant term c is negative when the disks already overlap; then the collision is happening now and the function returns 0, leaving it to the caller to decide what to do (the code of Section 12.10 forbids every velocity that approaches the other agent). Line 7 computes a quarter of the discriminant of Equation (12.8);

Algorithm 12.1. Time to collision and membership in the truncated velocity obstacle

Input: relative position $\mathbf{p}$, relative velocity $\mathbf{v}$, combined radius R , horizon τ
Output: $t_c(\mathbf{v})$, and whether $\mathbf{v}_A \in VO_{A|B}^\tau$

```

1 Function TimeToCollision( $\mathbf{p}$ ,  $\mathbf{v}$ ,  $R$ ):
2    $a \leftarrow \mathbf{v} \cdot \mathbf{v}$ ;  $b \leftarrow \mathbf{p} \cdot \mathbf{v}$ ;  $c \leftarrow \mathbf{p} \cdot \mathbf{p} - R^2$ 
3   if  $c \leq 0$  then
4     return 0 // already overlapping
5   if  $a = 0$  then
6     return  $\infty$  // no relative motion
7    $\Delta' \leftarrow b^2 - a c$ 
8   if  $\Delta' < 0$  then
9     return  $\infty$  // the ray misses the disk
10   $t \leftarrow (b - \sqrt{\Delta'})/a$ 
11  if  $t < 0$  then return  $\infty$  // the disk lies behind A
12  return  $t$ 

13 Function InsideVO( $\mathbf{p}_A$ ,  $r_A$ ,  $\mathbf{p}_B$ ,  $\mathbf{v}_B$ ,  $r_B$ ,  $\mathbf{v}_A$ ,  $\tau$ ):
14    $t \leftarrow \text{TimeToCollision}(\mathbf{p}_B - \mathbf{p}_A, \mathbf{v}_A - \mathbf{v}_B, r_A + r_B)$ 
15   return  $t < \infty$  and  $t \leq \tau$ 

```

a negative value means the ray misses the disk and the velocity is safe for ever. Line 10 takes the smaller root; by Proposition 12.6 it is negative exactly when $\mathbf{p} \cdot \mathbf{v} < 0$, i.e., when the velocity points away from B and the “collision” would have happened in the past, so it is reported as no collision. The wrapper `InsideVO` forms the relative quantities of Definition 12.1 in line 14 and compares with the horizon, which is exactly Definition 12.7. The guard $t < \infty$ is what makes the limit case work: without it the convention $\infty \leq \infty$ would report every collision-free velocity as dangerous, and it is precisely this guard that makes $\tau = \infty$ test membership in the untruncated $VO_{A|B}$. The test is the same in three dimensions, where $\mathbf{p}$ and $\mathbf{v}$ are 3-vectors (Section 12.9).

Algorithm 12.2. Sampled velocity selection for one agent

Input: agent A (position, radius, speed limit $v_{\max}$, goal), the other agents $B_1, \dots, B_k$ with their current positions, radii and velocities, horizon τ , penalty weight κ
Output: a new velocity $\mathbf{v}_A$

```

1 Function ChooseVelocity( $A$ ,  $\{B_j\}$ ,  $\tau$ ):
2    $\mathbf{v}^{\text{pref}} \leftarrow \text{PreferredVelocity}(A)$  // toward the goal at nominal speed
3    $S \leftarrow \text{VelocitySamples}(v_{\max}) \cup \{\mathbf{v}^{\text{pref}}\}$ 
4   foreach  $\mathbf{v} \in S$  do
5      $t_{\min}(\mathbf{v}) \leftarrow \min_j \text{TimeToCollision}(\mathbf{p}_{B_j} - \mathbf{p}_A, \mathbf{v} - \mathbf{v}_{B_j}, r_A + r_{B_j})$ 
6    $F \leftarrow \{\mathbf{v} \in S : t_{\min}(\mathbf{v}) > \tau \text{ or } t_{\min}(\mathbf{v}) = \infty\}$ 
7   if  $F \neq \emptyset$  then
8     return  $\arg \min_{\mathbf{v} \in F} \|\mathbf{v} - \mathbf{v}^{\text{pref}}\|$ 
9   return  $\arg \min_{\mathbf{v} \in S} \|\mathbf{v} - \mathbf{v}^{\text{pref}}\| + \kappa/t_{\min}(\mathbf{v})$ 

```

Walkthrough of Algorithm 12.2. Line 3 builds the candidate set: a fixed pattern of velocities that covers the reachable disk, in the book’s code ten concentric rings of 72 directions plus the zero velocity (721 samples), together with the preferred velocity itself so that an unobstructed agent flies exactly where it wants. Line 5 evaluates every candidate against every obstacle; the smallest time to collision over the obstacles is the time to the first collision, and comparing it with τ in line 6, where $t_{\min} = \infty$ (no collision at all) counts as safe, is membership in the union of Equation (12.13). Line 8 is the cost Equation (12.14) restricted to the samples. If no sample is feasible, line 9 chooses the least dangerous one: the penalty $\kappa/t_{\min}$ is small for a collision far away and huge for one that is imminent, so the agent buys time. The whole function costs $\mathcal{O}(|S| k)$ evaluations of Algorithm 12.1, a few dot products each; with NumPy the loop over S is one vectorized expression.

Algorithm 12.3. Synchronous velocity-obstacle simulation

Input: agents $A_1, \dots, A_n$ with positions, velocities, radii and goals; time step Δt ; horizon τ ; number of steps T

Output: the trajectories of all agents

```

1 Function SimulateVO( $\{A_i\}, \Delta t, \tau, T$ ):
2   for  $k \leftarrow 1$  to  $T$  do
3     foreach agent  $A_i$  do
4       if  $A_i$  has reached its goal then  $\mathbf{v}'_i \leftarrow \mathbf{0}$ 
5       else if  $A_i$  avoids then  $\mathbf{v}'_i \leftarrow \text{ChooseVelocity}(A_i, \{A_j : j \neq i\}, \tau)$ 
6       else  $\mathbf{v}'_i \leftarrow \text{PreferredVelocity}(A_i)$  // a non-cooperative agent
7     foreach agent  $A_i$  do
8        $\mathbf{v}_i \leftarrow \mathbf{v}'_i$ ;  $\mathbf{p}_i \leftarrow \mathbf{p}_i + \Delta t \mathbf{v}_i$ 
9     if all agents have reached their goals then break
10  return the recorded positions and velocities

```

Walkthrough of Algorithm 12.3. The simulation is *synchronous*: in line 5 every agent chooses its new velocity from the velocities that the others *currently* fly, and only then, in line 8, are all velocities and positions updated together. This mirrors what happens on real vehicles, which observe each other’s motion and decide at the same time, and it is the source of the oscillation studied in Section 12.7. Agents that do not avoid simply follow their preferred velocity; they model the non-cooperative intruder. The Euler step in line 8 is exact for a point mass with a commanded velocity, because the velocity is constant during the step. The time step must be shorter than the horizon, $\Delta t \leq \tau$, so that a velocity chosen to be safe for τ seconds is safe until the next decision (Theorem 12.11).

12.5 A worked example

Example 12.10 (Two drones on a crossing course). Drone A starts at $\mathbf{p}_A = (-5, 0)$ and wants to reach $(5, 0)$ at its nominal speed of 1 m/s; it can fly up to $v_{\max} = 1.5$ m/s and applies Algorithm 12.2 with $\tau = 5$ s every $\Delta t = 0.1$ s. Drone B starts at $\mathbf{p}_B = (0, -5)$, flies north at $\mathbf{v}_B = (0, 0.9)$ toward $(0, 5)$ and does not react to A . Both have radius 0.5 m, so $R = 1$. This is `crossing_scenario()` in `code/ch12_velocity_obstacles.py`; the function `worked_example()` prints every number quoted below.

The velocity obstacle at $t = 0$. The relative position is $\mathbf{p}_{\text{rel}} = (5, -5)$, at distance $d = \sqrt{50} = 7.071$ in the direction -45° , and the half-angle is $\theta = \arcsin(1/7.071) = 8.13^\circ$.

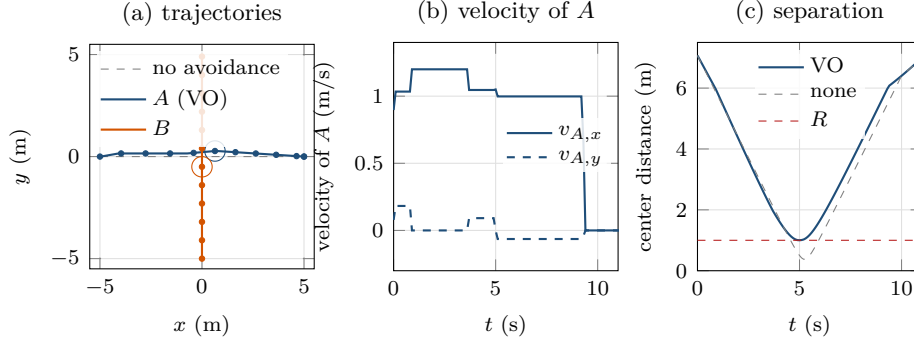


Figure 12.5. Example 12.10 simulated with Algorithm 12.3. (a) Trajectories; dots mark whole seconds, the dashed line is A 's route without avoidance, and the two circles are the disks at $t = 5$ s, where they come closest. A 's detour is barely visible: it passes in front of B by flying a little faster and a little to the left. (b) The velocity components of A over time: it speeds up to 1.2 m/s between $t = 0.9$ and $t = 3.6$ s and returns to its preferred velocity once B has passed. (c) Center distance over time with and without avoidance; the dashed line is $R = 1$. Without avoidance the disks overlap from $t = 4.6$ to 5.9 s; with VO the distance never drops below 1.001.

The two legs of the cone therefore point at -36.87° and -53.13° : the leg directions are the 3–4–5 unit vectors $(0.8, -0.6)$ and $(0.6, -0.8)$, and the tangent points of the circle of radius 1 around $(5, -5)$, seen from the origin, lie at distance $\sqrt{d^2 - R^2} = 7$ along them, at $(5.6, -4.2)$ and $(4.2, -5.6)$. The apex is $\mathbf{v}_B = (0, 0.9)$. The preferred velocity of A is $\mathbf{v}^{\text{pref}} = (1, 0)$, so the relative velocity is $\mathbf{v} = (1, 0) - (0, 0.9) = (1, -0.9)$. Then $\mathbf{v} \cdot \mathbf{v} = 1.81$, $\mathbf{p} \cdot \mathbf{v} = 9.5$ and $\mathbf{p} \cdot \mathbf{p} - R^2 = 49$, so $\Delta/4 = 90.25 - 88.69 = 1.56$ and, by Equation (12.9), $t_c = (9.5 - 1.249)/1.81 = 4.559 \text{ s} < \tau$: the preferred velocity is in $VO_{A|B}^\tau$, and without avoidance the disks would overlap from $t = 4.56$ to $t = 5.94$ s. Indeed, in Figure 12.2(b) the point $(1, 0)$ lies inside the truncation disk of center $(1, -0.1)$ and radius 0.2, at distance 0.1 from its center.

The choice. The nearest point of the boundary of $VO_{A|B}^\tau$ is straight above $\mathbf{v}^{\text{pref}}$ on the truncation arc, $\mathbf{v}^* = (1, 0.1)$, at distance 0.1; its time to collision is exactly $\tau = 5$ s (the method `VelocityObstacle.closest_boundary_point` computes it). The sampled algorithm cannot return this point because it is not a sample. Of its 722 candidates, 695 are feasible, and the feasible one closest to $(1, 0)$ is $\mathbf{v}'_A = (0.897, 0.078)$, the sample at speed 0.9 and heading 5° , at distance 0.130 from $\mathbf{v}^{\text{pref}}$ with $t_c = 5.03$ s. The sampled choice is a little worse than the exact one and, in this case, slower rather than faster; both are legitimate: a velocity just outside the truncated cone is safe for the next five seconds, and the decision is revisited in 0.1 s.

The run. Table 12.1 traces the simulation and Figure 12.5 plots it. In the first step A turns 5° left and slows to 0.9 m/s. One step later the relative geometry has changed slightly and the closest feasible sample is $(1.034, 0.182)$, speed 1.05 at 10° . As B approaches, d shrinks, the half-angle θ grows and the wedge swallows the candidates near $\mathbf{v}^{\text{pref}}$ one after another: the number of feasible samples falls from 695 at $t = 0$ to 122 at $t = 5$ s. From $t = 1$ s the closest feasible sample is $(1.2, 0)$: fly straight but faster. This is the typical VO solution to a crossing: it is cheaper to pass in front of a slower vehicle than to wait for it, and it turns the collision into a miss because the relative velocity $(1.2, -0.9)$ passes above the disk instead of through it (its discriminant is negative); in the workspace this is A crossing in front of B , as Figure 12.5(a) shows. From $t = 3.7$ s the agent adds a small northward component, $\mathbf{v}_A = (1.046, 0.092)$, at $t = 5.0$ s the centers are 1.001 m apart, and from $t = 5.1$ s the preferred velocity is feasible again ($t_c = \infty$): A heads for its goal, arriving after a path of length 10.027 m instead of 10. Without avoidance the centers come within 0.377 m at $t = 5.2$ s; the continuous minimum is

Table 12.1. Trace of [Algorithm 12.2](#) for agent A in [Example 12.10](#). $t_c(\mathbf{v}^{\text{pref}})$ is the time to collision of the preferred velocity, “feasible” the number of feasible candidates among the 722, and $t_c(\mathbf{v}_A)$ the time to collision of the chosen velocity (∞ : the ray misses the disk).

t (s)	$\mathbf{p}_A$	$\mathbf{v}^{\text{pref}} - \mathbf{v}_B$	$t_c(\mathbf{v}^{\text{pref}})$	feasible	chosen $\mathbf{v}_A$	$t_c(\mathbf{v}_A)$
0.0	(−5.000, 0.000)	(1.000, −0.900)	4.559	695	(0.897, 0.078)	5.03
0.1	(−4.910, 0.008)	(1.000, −0.901)	4.465	694	(1.034, 0.182)	∞
0.5	(−4.497, 0.081)	(1.000, −0.909)	4.091	688	(1.034, 0.182)	∞
1.0	(−3.963, 0.154)	(1.000, −0.917)	3.616	679	(1.200, 0.000)	∞
2.0	(−2.763, 0.154)	(1.000, −0.920)	2.578	642	(1.200, 0.000)	∞
3.0	(−1.563, 0.154)	(1.000, −0.923)	1.574	559	(1.200, 0.000)	∞
4.0	(−0.409, 0.181)	(0.999, −0.933)	0.655	299	(1.046, 0.092)	∞
4.5	(0.114, 0.227)	(0.999, −0.946)	0.266	170	(1.046, 0.092)	∞
5.0	(0.637, 0.273)	(0.998, −0.962)	0.013	122	(1.050, 0.000)	∞
6.0	(1.640, 0.215)	(0.998, −0.964)	∞	657	(0.998, −0.064)	∞

0.372 m at the time of closest approach $t^* = \mathbf{p} \cdot \mathbf{v} / (\mathbf{v} \cdot \mathbf{v}) = 9.5/1.81 = 5.25$ s.

Two details of the trace deserve attention. At $t = 5.0$ s the preferred velocity has $t_c = 0.013$ s: the disks are about to touch, and $\mathbf{v}^{\text{pref}}$ would push A into B right now. The chosen velocity (1.05, 0) is nevertheless safe with $t_c = \infty$, because B is at that moment slightly south of A and any velocity that does not point at it misses. This is the untruncated-versus-truncated question in miniature: the half-angle is almost 90° ($d = 1.0012$ m $\approx R$), so the wedge is nearly a half-plane, yet more than a hundred of the samples still lie outside it. Second, the velocity at $t = 6$ s is (0.998, −0.064), which is not on the sample grid: it is the preferred velocity itself, now pointing slightly south from (1.64, 0.22) toward the goal, which is why line 3 adds it to the candidates.

12.6 Properties: safety, feasibility, complexity

Velocity obstacles are a rule, not a search, so the questions are not completeness and optimality but: when is the chosen velocity safe, when does a safe velocity exist, and what does the rule not promise.

12.6.1 Safety of one decision

Theorem 12.11 (One-step safety). *Let $\|\mathbf{p}_B - \mathbf{p}_A\| > R$ at time 0, let A choose a velocity $\mathbf{v}_A \notin VO_{A|B}^\tau(\mathbf{v}_B)$, and let both agents hold their velocities during $[0, \tau]$. Then $\|\mathbf{p}_B(t) - \mathbf{p}_A(t)\| > R$ for all $t \in [0, \tau]$.*

Proof. By [Definition 12.7](#), $\mathbf{v}_A \notin VO_{A|B}^\tau$ means $t_c(\mathbf{v}_A - \mathbf{v}_B) > \tau$ (possibly ∞). By [Definition 12.5](#), t_c is the first time at which $\|\mathbf{p}_{\text{rel}} - t\mathbf{v}_{\text{rel}}\| \leq R$, and by [Equation \(12.2\)](#) this norm is the center distance at time t as long as the velocities are held. Hence the distance exceeds R on $[0, t_c) \supseteq [0, \tau]$. $\square$

Corollary 12.12 (A constant-velocity intruder is never hit). *Let B fly with a constant velocity and let A apply [Algorithm 12.2](#) at times $0, \Delta t, 2\Delta t, \dots$ with $\Delta t \leq \tau$. If at every decision the feasible set is non-empty, A and B never collide.*

Proof. At each decision time $k\Delta t$ the agents are apart (by induction), the chosen velocity is outside $VO_{A|B}^\tau$, and by [Theorem 12.11](#) the distance stays above R until $k\Delta t + \tau \geq (k+1)\Delta t$, when the next decision is made. The theorem may be applied on each interval separately

because the center distance on $[k\Delta t, (k+1)\Delta t]$ depends only on the velocities held during that interval, and both are constant there. $\square$

The corollary is the reason velocity obstacles are the right tool against a non-cooperative object: the guarantee needs nothing from B except that its velocity is known and does not change. Every one of the three assumptions has a practical reading. If B 's velocity is *estimated*, from a Kalman filter for example (Chapter 18), the estimation error must be absorbed by inflating R or by a shorter horizon. If B 's velocity *changes*, the guarantee holds only until it does; the next decision sees the new velocity, and the horizon must be long enough for A to react. And if A cannot change its velocity instantaneously, the decision must leave a margin for the time it takes to reach the new velocity (Section 12.8).

12.6.2 When no safe velocity exists

Proposition 12.13 (Empty feasible set). *Let B fly straight at A , $\mathbf{v}_B = -s \mathbf{p}_{\text{rel}}/d$ with speed $s > v_{\text{max}}$. Then every reachable velocity of A lies in the untruncated $\text{VO}_{A|B}(\mathbf{v}_B)$ if and only if $s \geq v_{\text{max}} d/R$.*

Proof. The apex $\mathbf{v}_B$ lies on the axis of the wedge at distance s from the origin, on the side away from $\mathbf{p}_{\text{rel}}$, so the reachable disk $D(\mathbf{0}, v_{\text{max}})$ is centered on the axis at distance $s > v_{\text{max}}$ from the apex. Seen from the apex it subtends the half-angle $\arcsin(v_{\text{max}}/s)$, by the same right triangle as in Theorem 12.3. The disk lies inside the wedge of half-angle $\theta = \arcsin(R/d)$ if and only if $\arcsin(v_{\text{max}}/s) \leq \theta$, i.e., $v_{\text{max}}/s \leq R/d$. $\square$

The hypothesis $s > v_{\text{max}}$ is not an extra assumption: $d > R$ makes $v_{\text{max}}d/R > v_{\text{max}}$, so the conclusion already forces it. The proposition is therefore a threshold statement about the single quantity s .

In Example 12.10 this threshold is $1.5 \cdot 7.07/1 = 10.6 \text{ m/s}$: an intruder that fast, that close and aimed at A cannot be dodged by any velocity A can fly. Truncation does not help here, because at that speed the collision is less than a second away. The fallback in line 9 of Algorithm 12.2 then chooses the velocity that postpones the collision most, which is the best a purely local rule can do. In dense traffic the feasible set is also empty, more often and less dramatically, when many wedges overlap; the experiment of Section 12.7 counts these events.

12.6.3 Complexity and what is not guaranteed

One evaluation of Algorithm 12.1 costs three dot products. With m candidate velocities and k obstacles, Algorithm 12.2 costs $\mathcal{O}(mk)$, and one step of Algorithm 12.3 with n avoiding agents costs $\mathcal{O}(n^2m)$; for $m = 722$ this is $n(n-1)m \approx 4 \cdot 10^4$ evaluations per step for $n = 8$, which NumPy handles in a few milliseconds. Restricting each agent to its k nearest neighbors, as every real implementation does, makes the step linear in n .

What the rule does *not* guarantee is as important as what it does.

- **No completeness or optimality.** The rule is greedy over one step. A velocity that is safe now can lead to a position from which every velocity is unsafe later, for instance when an agent flies into the narrowing gap between two converging obstacles. Fiorini and Shiller addressed this with a tree search over sequences of avoidance maneuvers [45]; the one-step rule is what practical systems use, backed by a global planner (Chapter 24).
- **No progress guarantee.** Nothing forces the agent toward its goal. Two agents that must pass each other in a corridor narrower than $2R$ stop and stay stopped; the deadlock is a task for the replanning layer, not for the local rule.
- **No guarantee among cooperating agents.** Theorem 12.11 assumes that B holds its velocity. When B is itself avoiding A , the assumption fails at every step, and safety and smoothness both suffer: this is the subject of the next section.

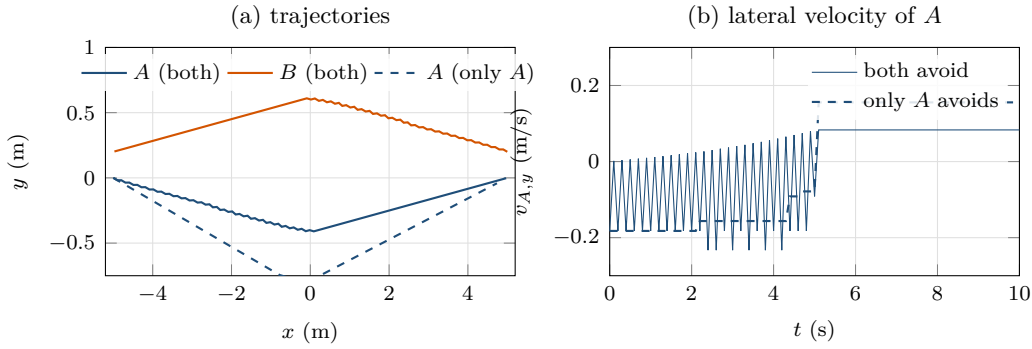


Figure 12.6. Oscillation. Two agents pass each other head-on along lines offset by 0.2. (a) Trajectories when both apply the velocity-obstacle rule (solid) and when only *A* applies it (dashed): with two avoiders the drift is shared and the paths zigzag. (b) The lateral velocity of *A*: with two avoiders it flips sign at almost every step, because each agent reacts to the other’s sidestep by canceling its own; with one avoider it changes sign once. The vertical axis of (a) is stretched.

- **Sampling error.** The sampled minimizer can be off the exact one by the spacing of the sample grid, and a thin feasible corridor between two wedges can be missed entirely, so that the algorithm declares the set empty although it is not (Section 12.10).

12.7 The oscillation problem

12.7.1 Two agents avoiding each other at the same time

Let two identical agents fly head-on along parallel lines offset laterally by 0.2 — *A* from $(-5, 0)$ to $(5, 0)$ and *B* from $(5, 0.2)$ to $(-5, 0.2)$ — both at speed 1 with $R = 1$, and let *both* apply Algorithm 12.2 at every step. At $t = 0$ each sees the other coming straight at it and steps aside: *A* chooses the sample $(1.034, -0.182)$, *B* its mirror image $(-1.034, 0.182)$. Now consider the next decision. *A* assumes that *B* will keep its new, sideways velocity. With *B* moving up and to the left, the relative velocity of the preferred velocity $(1, 0)$ now clears the disk, so *A* returns to $(1, 0)$, and *B*, by symmetry, returns to $(-1, 0)$. One step later both are heading straight at each other again, step aside again, and so on. Figure 12.6 shows the result: the lateral velocity of *A* changes sign 51 times in 100 steps, and although the minimum distance stays at $1.002 \geq R$ the command alternates at every step between a sidestep of about -0.18 m/s (peak 0.23) and a lateral component just above zero, so it crosses zero at almost every decision. A real drone cannot follow such a command, and the maneuver it executes is not the one that was analyzed.

The cause is the assumption behind Definition 12.4: *B* keeps its velocity. When *B* is an avoider too, its velocity at the next step is a *reaction* to *A*’s choice, and the two reactions cancel. The remedy is to let each agent take only half of the responsibility for the avoidance and to assume that the other takes the other half; the reciprocal constructions that do this are listed in Section 12.8 and derived from this chapter’s geometry in Chapter 13. Note that reciprocity is a property of the *pair*: against an intruder that does not react, the plain velocity obstacle with full responsibility is the correct construction, and using a reciprocal one would avoid only half of the collision.

12.7.2 An experiment on crossing agents

The script `code/figures/gen_ch12_sim.py` runs two families of scenarios with the simulator of Algorithm 12.3 ($\Delta t = 0.1$, $\tau = 5$, $R = 1$, nominal speed 1, $v_{\max} = 1.5$). In the *stream*

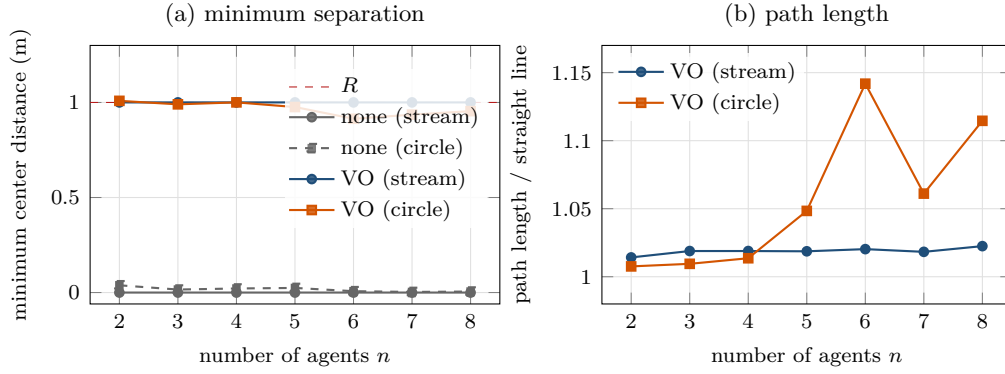


Figure 12.7. No avoidance versus velocity obstacles for $n = 2, \dots, 8$ crossing agents. (a) Minimum center distance; the dashed line is $R = 1$. Without avoidance every scenario collides. One VO agent among non-cooperative intruders (stream) keeps the distance at the guaranteed minimum R (1.0003) for every k , as [Corollary 12.12](#) promises. When all agents apply VO simultaneously (circle) the distance dips below R already for $n = 3$ and stays below it from $n = 5$ on. (b) Path length relative to the straight line: the price of avoidance is 1.4–2.3 % in the stream and up to 14 % in the crowded circle.

Table 12.2. The experiment of [Figure 12.7](#). $d_{\min}$ is the minimum center distance over the run, “pairs $< R$ ” counts the pairs of agents whose center distance dropped below R at some step, and “empty” the number of decisions at which no sampled velocity was feasible.

n	stream: one VO agent, $k = n$ intruders			circle: n VO agents				
	$d_{\min}$ none	$d_{\min}$ VO	path ratio	$d_{\min}$ none	$d_{\min}$ VO	path ratio	pairs $< R$	empty
2	0.000	1.000	1.014	0.038	1.009	1.008	0	0
3	0.000	1.000	1.019	0.016	0.990	1.009	1	0
4	0.000	1.000	1.019	0.021	1.000	1.014	0	0
5	0.000	1.000	1.019	0.024	0.976	1.048	4	1
6	0.000	1.000	1.020	0.007	0.914	1.142	6	5
7	0.000	1.000	1.018	0.003	0.936	1.061	4	4
8	0.000	1.000	1.023	0.005	0.954	1.115	5	1

scenario one avoiding agent flies 12 m east across a picket line of k non-cooperative intruders that fly north at speeds 0.8, 1.1 and 1.4 m/s, each timed to hit the agent exactly where its nominal path crosses theirs. In the *circle* scenario n agents start evenly spaced on a circle of radius 6 and fly to the antipodal points, so every pair is on a crossing course through the center, and *all* of them apply the velocity-obstacle rule (the start positions are perturbed by a fixed-seed noise of standard deviation 0.05 to break the symmetry). [Figure 12.7](#) and [Table 12.2](#) report the minimum center distance over the run and the path length relative to the straight line.

The two halves of the table tell the two stories of this chapter. Against non-cooperative intruders, the velocity-obstacle agent never comes closer than R (the recorded minimum is 1.0003), whatever the number of intruders, and pays 1.4–2.3 % extra path, arriving in 11.3–11.5 s, earlier than the 12 s of the straight flight, because its maneuver is mostly “speed up and pass in front”. Among agents that all apply the rule simultaneously the picture degrades with density: one pair grazes at $n = 3$ (0.990), and from $n = 5$ on the minimum distance falls below R at every density (to 0.914 at $n = 6$), decisions with an empty feasible set appear, and the paths lengthen by up to 14 % as agents oscillate around the crowded center. None of these are head-on crashes, and $n = 8$ agents on a circle of radius 6 with $R = 1$ is a crowded scene; but the guarantee of [Corollary 12.12](#) is gone, because its assumption is violated at every step.

Chapter 13 repeats this experiment with RVO and ORCA.

12.8 Variants and extensions

Static and polygonal obstacles. A static obstacle has $\mathbf{v}_B = \mathbf{0}$, so its velocity obstacle is the collision cone itself with the apex at the origin, and its truncated version is the cone beyond the disk $D(\mathbf{p}_{\text{rel}}/\tau, R/\tau)$. An obstacle that is not a disk, such as a building or a wall, is first inflated by r_A (Proposition 2.9; the distance to a wall segment is Definition 2.26) and then treated point by point: a velocity is dangerous if its ray hits any point of the inflated obstacle, so the cone is the union of the cones of all its points. For a convex obstacle this union is the wedge between the two extreme tangent rays from A (Exercise 12.6). The truncated version is again the union of the scaled copies of the obstacle, $\bigcup_{t \leq \tau} \mathcal{O}/t$, which for a polygon is easy to draw and easy to sample. Truncation matters most for static obstacles: without it a long wall forbids every velocity with any component toward it, however far away the wall is.

Obstacles that do not fly straight. The velocity obstacle assumes that B keeps its velocity. If the prediction layer (Chapter 20) supplies a predicted trajectory $\hat{\mathbf{p}}_B(t)$ instead, Proposition 12.8 generalizes without change of argument: A flying straight with velocity $\mathbf{v}$ is within R of B at time t if and only if $\|\hat{\mathbf{p}}_B(t) - \mathbf{p}_A - t\mathbf{v}\| \leq R$, i.e. $\mathbf{v} \in D((\hat{\mathbf{p}}_B(t) - \mathbf{p}_A)/t, R/t)$, so the forbidden set is the union of these disks over $t \leq \tau$. It is no longer a cone, but the sampled Algorithm 12.2 handles it with the membership test replaced by a scan over the predicted times. This is the non-linear velocity obstacle. Uncertainty in the prediction is absorbed by inflating R with the predicted standard deviation, as the prediction-aware constraints of Chapter 24 do.

Reciprocal variants. For agents that all avoid each other, the reciprocal velocity obstacle [15] shifts the apex to $(\mathbf{v}_A + \mathbf{v}_B)/2$, the hybrid reciprocal velocity obstacle [154] combines one leg of RVO with one leg of VO to fix the side on which the agents pass, and ORCA [14] replaces each wedge by a half-plane so that the selection becomes a small linear program. All three are derived in Chapter 13 from Definition 12.4 and Proposition 12.8.

Dynamics. Definition 12.9 takes every speed below v_{max} to be reachable at once. With an acceleration limit a_{max} the velocities reachable within one step form the disk $D(\mathbf{v}_A, a_{\text{max}}\Delta t)$ intersected with $D(\mathbf{0}, v_{\text{max}})$, the dynamic window of Chapter 14; the sampled algorithm accepts any candidate set, so it is enough to sample that window instead. What does not carry over is the exactness of Theorem 12.11: while the agent accelerates toward its new velocity it flies neither the old nor the new one. A margin on R or on the horizon, of the order of the time $\|\mathbf{v}'_A - \mathbf{v}_A\|/a_{\text{max}}$ needed to reach the new velocity, restores safety in practice.

Exact selection. With a single obstacle, the minimizer of Equation (12.14) is the projection of $\mathbf{v}^{\text{pref}}$ onto the boundary of $VO_{A|B}^\tau$: onto one of the two legs beyond their tangency points or onto the truncation arc, whichever is closest. The method `closest_boundary_point` of the class `VelocityObstacle` does this in a few lines and returned $\mathbf{v}^* = (1, 0.1)$ in Example 12.10. With several obstacles the projection onto one wedge may land inside another, and the exact answer is the closest point of a region bounded by several arcs and legs; ORCA avoids this by using one half-plane per obstacle.

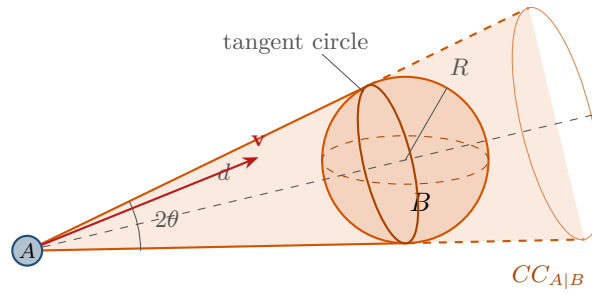


Figure 12.8. The collision cone in three dimensions: a right circular cone with apex at A , axis toward the center of the sphere of radius $R = r_A + r_B$ around B and half-angle $\theta = \arcsin(R/d)$. The cone touches the sphere along a circle. Any plane through the axis shows the planar picture of Figure 12.1; a relative velocity $\mathbf{v}$ is dangerous exactly when it lies inside the cone.

12.9 The cone in three dimensions

Drones fly in three dimensions, and the third dimension is often the cheapest way out of a conflict: climbing over an intruder can be a smaller maneuver than swerving around it. Nothing in the derivation above used the plane. Let $\mathbf{p}_A, \mathbf{v}_A, \mathbf{p}_B, \mathbf{v}_B \in \mathbb{R}^3$ and let the agents be spheres of radii r_A, r_B . The relative quantities of Definition 12.1, the collision condition Equation (12.3) and the quadratic Equation (12.8) are unchanged, with 3-vectors in the dot products, because the Minkowski sum of two balls is again a ball. The ray in direction $\mathbf{v}$ meets the ball $D(\mathbf{p}_{\text{rel}}, R)$ if and only if the distance from the center to the ray, $d \sin \varphi$, is at most R , exactly as in the proof of Theorem 12.3, and the angle φ between $\mathbf{v}$ and $\mathbf{p}_{\text{rel}}$ is now the angle between two vectors in space.

Proposition 12.14 (The 3D collision cone). *For spheres at distance $d > R$, the collision cone $CC_{A|B} \subset \mathbb{R}^3$ is the right circular cone with apex at the origin, axis $\mathbf{p}_{\text{rel}}$ and half-angle $\theta = \arcsin(R/d)$; the velocity obstacle is its translate by $\mathbf{v}_B$, and the truncated velocity obstacle is the cone with its tip cut off by the ball $D(\mathbf{v}_B + \mathbf{p}_{\text{rel}}/\tau, R/\tau)$, which is inscribed in the cone. Every plane through the axis cuts the cone in the planar wedge of Theorem 12.3.*

Proof sketch. The proof is the one of Theorem 12.3 and Proposition 12.8 read with 3-vectors; the only new fact is that the set of directions at angle $\leq \theta$ from a fixed axis in $\mathbb{R}^3$ is a circular cone rather than a wedge. Exercise 12.8 asks for the details. $\square$

Figure 12.8 shows the cone. The tangent points form a circle at distance $(d^2 - R^2)/d$ from the apex along the axis, of radius $R\sqrt{d^2 - R^2}/d$. For the implementation this means that Algorithm 12.1 works verbatim in 3D, which the chapter's code exploits: `time_to_collision` accepts 3-vectors, and its self-test checks the cone for $\mathbf{p}_{\text{rel}} = (3, 4, 12)$, $d = 13$ and $R = 1$ by probing directions just inside and just outside the half-angle $\arcsin(1/13)$. What changes is the candidate set of Algorithm 12.2: the reachable ball must be sampled in speed and in two angles, for instance with a few hundred well-spread directions on the unit sphere times a handful of speeds, and a drone will usually weight vertical deviations differently from horizontal ones in the cost Equation (12.14), because climbing costs energy and altitude bands may be reserved. The reciprocal constructions of Chapter 13 have the same 3D form.

12.10 Implementation notes

The file `code/ch12_velocity_obstacles.py` implements everything in this chapter in about 650 lines: the plane-geometry helpers copied from `code/ch02_toolbox.py`, the time to collision and membership test (Listing 12.1), a `VelocityObstacle` class that exposes the apex, the

tangent directions, the truncation disk and the exact projection onto the boundary, the sampled selection (Listing 12.2), the simulator of Algorithm 12.3 and the scenarios of the figures. Its self-test runs in well under a second.

Numerical care when agents nearly touch. Three situations need explicit handling. When $d \leq R$ the disks already overlap, the constant term of Equation (12.8) is non-positive and every velocity “collides” at $t = 0$; treating all of them as forbidden would freeze the agent inside the other. The code instead forbids the velocities that approach ($\mathbf{p}_{\text{rel}} \cdot \mathbf{v}_{\text{rel}} > 0$) and allows those that separate, so that an overlap, whether from a bad initial placement or a violated assumption, resolves itself. When d is only slightly larger than R , the half-angle is close to 90° and the discriminant of a grazing velocity hovers around zero; rounding then decides between “graze” and “miss”. Do not rely on the exact boundary: the exact projection $\mathbf{v}^* = (1, 0.1)$ of Example 12.10 has $t_c = \tau$ to the last digit, and a real agent should either inflate R by a margin or demand $t_c > \tau + \varepsilon$. Finally, $\mathbf{v}_{\text{rel}} = \mathbf{0}$ (two hovering drones, or two drones in formation) makes the quadratic degenerate; Algorithm 12.1 returns ∞ for it, which is correct because the distance does not change.

The horizon. τ must be longer than the decision interval Δt (Theorem 12.11) and long enough for the maneuver that a velocity just outside VO^τ may require at the next decision: a few times the time needed to fly a distance R , plus the time to change velocity under the acceleration limit. Too short a horizon produces late, sharp maneuvers and empty feasible sets; too long a horizon produces the frozen behavior of the pitfall below. The chapter’s examples use $\tau = 5$ s for $R = 1$ m and speeds around 1 m/s; scale it with R/v .

Sampling versus solving exactly. The default candidate set has 721 velocities on ten rings of 72 directions, i.e., 5° in heading and 0.15 m/s in speed for $v_{\text{max}} = 1.5$. In your own implementation, always add the preferred velocity and, to reduce chattering, the current velocity to the candidates (the code of Listing 12.2 adds only $\mathbf{v}^{\text{pref}}$), and consider a finer ring around $\mathbf{v}^{\text{pref}}$. With a single obstacle, prefer the exact projection of Section 12.8; with many obstacles, sampling is robust and accepts other cost functions and reachable sets without any change to the rule (the dynamic window of Chapter 14). The test of all candidates against one obstacle is one vectorized NumPy expression (`ttc_batch`), which is why the self-test, including a sample of the experiments of Section 12.7.2, finishes in a fraction of a second; the full sweep of Table 12.2 is produced by `code/figures/gen_ch12_sim.py`.

Listing 12.1. Time to collision and membership in the truncated velocity obstacle (from `code/ch12_velocity_obstacles.py`); this is Algorithm 12.1.

```

1 def time_to_collision(p_rel, v_rel, radius):
2     """First time t >= 0 at which |p_rel - t v_rel| <= radius, or math.inf
3     if the disks (spheres) never touch. Works in 2D and in 3D."""
4     p = np.asarray(p_rel, dtype=float)
5     v = np.asarray(v_rel, dtype=float)
6     pp, pv, vv = p @ p, p @ v, v @ v
7     k = pp - radius * radius
8     if k <= 0.0:
9         return 0.0                                # already overlapping
10    if vv == 0.0:
11        return math.inf                            # no relative motion
12    disk = pv * pv - vv * k                        # discriminant / 4
13    if disk < 0.0:
14        return math.inf                            # the ray misses the disk
15    t = (pv - math.sqrt(disk)) / vv                # smaller root = first contact

```



```

16     return t if t >= 0.0 else math.inf    # the disk lies behind A
17
18
19 def in_velocity_obstacle(p_rel, v_rel, radius, tau=math.inf):
20     """True iff the relative velocity v_rel leads to a collision within the
21     horizon tau (tau = inf gives the untruncated velocity obstacle)."""
22     tc = time_to_collision(p_rel, v_rel, radius)
23     return tc < math.inf and tc <= tau

```

Listing 12.1 is Algorithm 12.1 line by line. Note the order of the guards: the overlap test comes before the test for zero relative velocity, because two overlapping agents at rest must still be told that they are colliding. Listing 12.2 is Algorithm 12.2. The candidates are a NumPy array with one row per velocity; for every other agent the relative velocities of all candidates are formed at once and passed to the vectorized `ttc_batch`, and the running minimum over obstacles is the $t_{\min}$ of line 5. The dictionary `info` returns the numbers reported in Table 12.1.

Listing 12.2. Sampled velocity selection (from `code/ch12_velocity_obstacles.py`); this is Algorithm 12.2.

```

1  def choose_velocity(agent, others, tau, dt, samples=None, penalty=1.0):
2      """Sampled velocity-obstacle selection for ``agent`` (Algorithm 12.2).
3
4      Every candidate is tested against the truncated VO of every other
5      agent (others keep their current velocities). Among the candidates
6      that are outside all  $VO^{\tau}$ , the one closest to the preferred velocity
7      wins; if there is none, the penalized cost  $|v - v_{\text{pref}}| + \text{penalty}/t_c$ 
8      picks the least dangerous candidate. Returns (velocity, info)."""
9      v_pref = agent.preferred_velocity(dt)
10     if samples is None:
11         samples = velocity_samples(agent.v_max)
12     cand = np.vstack([samples, np.asarray([v_pref])])
13     tc = np.full(len(cand), np.inf)
14     for other in others:
15         if other is agent:
16             continue
17         p_rel = sub(other.position, agent.position)
18         v_rel = cand - np.asarray(other.velocity, dtype=float)
19         radius = agent.radius + other.radius
20         if norm(p_rel) <= radius:    # already overlapping: forbid
21             tc_j = np.where(v_rel @ np.asarray(p_rel) > 0.0, 0.0, np.inf)
22         else:    # every closing velocity
23             tc_j = ttc_batch(p_rel, v_rel, radius)
24         tc = np.minimum(tc, tc_j)
25     dist = np.linalg.norm(cand - np.asarray(v_pref), axis=1)
26     feasible = (tc > tau) | np.isinf(tc)
27     if feasible.any():
28         idx = int(np.argmin(np.where(feasible, dist, np.inf)))
29     else:
30         idx = int(np.argmin(dist + penalty / np.maximum(tc, 1e-9)))
31     info = {"v_pref": v_pref, "tc_pref": float(tc[-1]),
32            "n_feasible": int(feasible.sum()), "tc": float(tc[idx]),
33            "feasible": bool(feasible[idx]), "dist": float(dist[idx])}
34     return (float(cand[idx, 0]), float(cand[idx, 1])), info

```

Forgetting to add the radii

The disk in [Definition 12.2](#) has radius $r_A + r_B$, not r_B . Using B 's radius alone (or, worse, treating both agents as points) makes the cone too narrow by the missing arcsin term, and the agent then grazes or overlaps the obstacle by up to r_A while its own test reports safety. In a simulation this shows up as a minimum separation between r_B and R ; in the experiment of [Table 12.2](#) it would turn every “1.000” into “0.5”. Add the radii, then add a margin for estimation error and dynamics.

Absolute instead of relative velocity

The cone is tested with $\mathbf{v}_A - \mathbf{v}_B$, and the apex of the velocity obstacle is at $\mathbf{v}_B$. Testing $\mathbf{v}_A$ itself against the cone at the origin is correct only for static obstacles and silently wrong for moving ones: the agent then avoids where the obstacle *is*, not where it will be, and collides with anything that moves across its path. A test that works against walls but fails against other drones is almost always this mistake.

No truncation

With $\tau = \infty$ the velocity obstacle forbids every velocity that would collide at *any* future time. A wall then forbids every velocity with a component toward it, and a slow drone a hundred meters away blocks a whole sector of headings; the agent refuses far-future collisions that a later decision would have handled, creeps, or freezes short of its goal. Truncate with a horizon ([Definition 12.7](#)) and choose it as described above.

Discretization of the velocity samples

A coarse sample set has two failure modes. A thin feasible corridor between two wedges, which is common when two obstacles pass on either side, may contain no sample at all, and the algorithm declares the feasible set empty and falls back to a dangerous velocity although a safe one exists. And when the best sample changes from step to step by one grid cell, the command chatters. Sample finely near the preferred and the current velocity, and always include both as candidates.

12.11 Where this fits in the drone system

In the drone system

Velocity obstacles and their reciprocal descendants form the *local safety layer* of the hybrid architecture in [Chapter 24](#). The executive monitors, for every drone, the time to collision [Equation \(12.9\)](#) against every tracked object, using the positions and velocities estimated by the prediction layer ([Chapters 18 and 20](#)). While the time to collision along the nominal route is larger than the safety horizon, the drone follows the route computed by the global planner ([Chapters 9 and 10](#)). When it drops below the horizon, the local layer takes over the velocity command: the preferred velocity is the velocity along the nominal route, the obstacles are the tracked objects with their estimated velocities, and the radius R is inflated by a margin that grows with the uncertainty of the estimate. Once no collision is predicted within the horizon, control returns to the route, and if the maneuver has pushed the drone too far off its route the replanning layer of [Chapter 5](#) is invoked.

Which construction the layer uses depends on who the obstacle is. A non-cooperative intruder, an unknown drone or a bird, is exactly the case of this chapter: it does not react, so the drone takes the full responsibility and uses VO^τ with the apex at the intruder's estimated velocity (Corollary 12.12). Another member of the swarm reacts with the same rule, and the reciprocal constructions of Chapter 13 apply. Mixing them up is a real error: a reciprocal maneuver against an intruder avoids only half of the collision, and a full-responsibility maneuver among swarm members produces the oscillation of Section 12.7. This chapter is the first half of Week 6 of the study plan (Appendix A); the coding exercise Exercise 12.7 builds the crossing simulation that the second half, Chapter 13, extends with RVO and ORCA.

12.12 Summary

Chapter summary

- Local collision avoidance chooses a *velocity* for the next instant instead of a path. In the frame of agent A , agent B is a disk of radius $R = r_A + r_B$ at $\mathbf{p}_{\text{rel}} = \mathbf{p}_B - \mathbf{p}_A$, and the relative velocity $\mathbf{v}_{\text{rel}} = \mathbf{v}_A - \mathbf{v}_B$ collides exactly when its ray hits that disk.
- The collision cone $CC_{A|B}$ is the wedge of half-angle $\theta = \arcsin(R/d)$ around $\mathbf{p}_{\text{rel}}$, bounded by the tangent rays to the disk; the velocity obstacle $VO_{A|B} = \mathbf{v}_B \oplus CC_{A|B}$ is the same wedge with its apex at $\mathbf{v}_B$.
- The time to collision is the smaller root of $(\mathbf{v} \cdot \mathbf{v})t^2 - 2(\mathbf{p} \cdot \mathbf{v})t + (\mathbf{p} \cdot \mathbf{p} - R^2) = 0$; a collision exists if and only if the discriminant is non-negative and $\mathbf{p} \cdot \mathbf{v} > 0$.
- The truncated velocity obstacle $VO_{A|B}^\tau$ keeps only the velocities that collide within the horizon τ ; it is the wedge with its tip cut off by the inscribed disk $D(\mathbf{v}_B + \mathbf{p}_{\text{rel}}/\tau, R/\tau)$, and it is what makes the method usable near walls and in traffic.
- The agent chooses, among the reachable velocities outside the union of all truncated velocity obstacles, the one closest to its preferred velocity; sampling the velocity disk is the simple and robust way to do so. Static obstacles are the case $\mathbf{v}_B = \mathbf{0}$, and the construction is the same in three dimensions with a circular cone.
- A velocity outside VO^τ is safe for τ seconds if the obstacle holds its velocity; hence a constant-velocity intruder is never hit. When both agents apply the rule at once, the assumption fails, the commands oscillate and, in dense scenes, the separation is no longer guaranteed. That is what the reciprocal methods of Chapter 13 repair.

Further reading. The velocity obstacle was introduced by Fiorini and Shiller [45], whose paper also contains the distinction between imminent and distant collisions and a tree search over sequences of avoidance maneuvers; the geometric background (Minkowski sums, configuration space) is covered in the textbook of Choset et al. [28]. The reciprocal velocity obstacle [15], the hybrid reciprocal velocity obstacle [154] and ORCA [14] are the subject of Chapter 13; the dynamic window approach of Chapter 14 shares the velocity-space view but replaces the collision cone by a stopping-distance test. The non-linear velocity obstacle of Section 12.8, which replaces the cone by the union of the disks swept along a predicted trajectory, is due to Large, Sekhavat, Shiller and Laugier [97]. Implementations that face many neighbors restrict the union of Equation (12.13) to the k nearest ones, the choice made by the ORCA library [14].

12.13 Exercises

Exercise 12.1 (★). Two drones of radii $r_A = 0.3$ m and $r_B = 0.5$ m are 4 m apart. Compute the half-angle of the collision cone. Recompute it with r_A forgotten (radius r_B only) and with both radii forgotten, and give the width of the wedge in degrees in each case. At what distance does the correct half-angle reach 30° , and what happens to the cone as the distance approaches $r_A + r_B$?

Exercise 12.2 (★★ Proof). (a) Using only the fact that a tangent to a circle is perpendicular to the radius at the point of contact, prove that the two tangent rays from a point at distance d to a circle of radius $R < d$ make the angle $\theta = \arcsin(R/d)$ with the direction to the center, and that they touch the circle at distance $\sqrt{d^2 - R^2}$ from the point. (b) Show that $CC_{A|B}$ is closed under multiplication by positive scalars and that $VO_{A|B}(\mathbf{v}_B)$ is closed under it if and only if $\mathbf{v}_B = \mathbf{0}$. (c) Conclude that whether an *untruncated* velocity obstacle contains $\mathbf{v}_A$ depends only on the direction of $\mathbf{v}_A - \mathbf{v}_B$, not on its magnitude, and explain why this is no longer true after truncation.

Exercise 12.3 (★★ Proof). Prove [Proposition 12.8](#) in detail: show that a relative velocity $\mathbf{v}$ collides at time t if and only if $\mathbf{v} \in D(\mathbf{p}_{\text{rel}}/t, R/t)$, that each of these disks is tangent to both legs of the cone, and that the point of the truncated obstacle closest to the apex lies on the axis at distance $(d - R)/\tau$, while the tangency points of the truncation disk lie at distance $\sqrt{d^2 - R^2}/\tau$ along the legs. Verify both distances in [Figure 12.2\(b\)](#), where $d = \sqrt{50}$, $R = 1$ and $\tau = 5$.

Exercise 12.4 (★). Let $\mathbf{p}_{\text{rel}} = (6, 2)$, $R = 2.5$ and $\mathbf{v}_{\text{rel}} = (3, 0)$. Write down the quadratic [Equation \(12.8\)](#), compute the time to collision and the time at which the disks separate again, and compute the time and distance of closest approach. Repeat for $\mathbf{v}_{\text{rel}} = (-3, 0)$ and explain the result with [Proposition 12.6](#). Check both with `time_to_collision`.

Exercise 12.5 (★★). An intruder B flies straight at A with speed s from distance d . Derive the condition of [Proposition 12.13](#), $s \geq v_{\text{max}}d/R$, from the tangent half-angle of the reachable disk seen from the apex, and evaluate it for $v_{\text{max}} = 1.5$ m/s, $d = 7.07$ m, $R = 1$ m. For s just above the threshold, which velocity does the fallback of [Algorithm 12.2](#) choose, and why does truncation with $\tau = 5$ s not create a feasible velocity? Then describe the situation with s well below the threshold but d small: which velocities are feasible, and what does this say about the horizon?

Exercise 12.6 (★★). (a) Show that for a static obstacle the velocity obstacle equals the collision cone and that the truncation disk has center $\mathbf{p}_{\text{rel}}/\tau$. (b) Let $\mathcal{O}$ be a convex obstacle (already inflated by r_A) that does not contain the origin. Show that the set of velocities whose ray meets $\mathcal{O}$ is the wedge between the two extreme tangent rays from the origin to $\mathcal{O}$, i.e., the union of the collision cones of all points of $\mathcal{O}$ is itself a cone. (c) A drone flies parallel to a long straight wall at distance 2 m from its center, with $r_A = 0.5$. Which velocities does the untruncated velocity obstacle of the wall forbid? Which ones does the truncated obstacle with $\tau = 4$ s forbid?

Exercise 12.7 (★★★ Coding). This is the first half of the coding exercise of Week 6 of the study plan ([Appendix A](#)). Build a 2D simulation of n disk agents on crossing courses (the circle and stream scenarios of [Section 12.7.2](#), or your own), either from scratch following [Algorithms 12.1 to 12.3](#) or by extending `code/ch12_velocity_obstacles.py`. Compare no avoidance with velocity obstacles and report, per run, the minimum center distance, the path length relative to the straight line, the travel time, the number of decisions with an empty feasible set and the number of sign changes of the lateral velocity (a measure of oscillation). Then (a) vary the horizon $\tau \in \{1, 2, 5, 10, \infty\}$ and find the values for which the agents freeze or collide; (b) vary the sample resolution (36, 72 and 180 directions) and observe the chattering; (c) add a static obstacle as a wall of disks and check the frozen

behavior without truncation. Keep the harness: in [Chapter 13](#) you will add RVO and ORCA to the comparison.

Exercise 12.8 (★★★ Proof). (a) Prove [Proposition 12.14](#): the set of vectors $\mathbf{v} \in \mathbb{R}^3$ whose ray meets the ball $D(\mathbf{p}_{\text{rel}}, R)$ is the right circular cone of half-angle $\arcsin(R/d)$ about $\mathbf{p}_{\text{rel}}$, and its intersection with any plane through the axis is the planar cone of [Theorem 12.3](#). (b) Show that the cone touches the sphere along a circle and compute its center and radius in terms of d and R . (c) Extend `velocity_samples` to three dimensions (speeds times well-spread directions on the unit sphere) and, with `time_to_collision`, which already accepts 3-vectors, check in a head-on encounter that a drone can escape by climbing, and compare the deviation from the preferred velocity with the planar escape.

Reciprocal Avoidance: RVO and ORCA

Learning objectives

- State precisely why two agents that both avoid each other with plain velocity obstacles oscillate, and why letting each agent take half of the responsibility removes the oscillation.
- Define the reciprocal velocity obstacle $RVO_{A|B}$, show that it is the velocity obstacle with its apex moved to $\frac{1}{2}(\mathbf{v}_A + \mathbf{v}_B)$, and explain why its guarantee does not extend to more than two agents.
- Derive the ORCA half-plane step by step: the truncated velocity obstacle, the current relative velocity, the smallest change $\mathbf{u}$ that reaches its boundary with the outward normal $\mathbf{n}$, and the half-plane $\left\{ \mathbf{v} : (\mathbf{v} - (\mathbf{v}_A + \frac{1}{2}\mathbf{u})) \cdot \mathbf{n} \geq 0 \right\}$, including the full case analysis for $\mathbf{u}$.
- Solve the per-agent program, the velocity closest to the preferred one inside all half-planes and the speed disk, with the incremental two-dimensional linear program, and handle the dense case in which the half-planes have an empty intersection.
- Adapt the construction to non-cooperative obstacles (full responsibility) and to three dimensions (half-spaces), and state exactly what ORCA guarantees and what it does not.
- Implement ORCA in Python, reproduce the worked example, and compare no avoidance, the velocity obstacle (VO) rule and ORCA in a simulation.

13.1 Why this matters for a drone swarm

Eight drones of a survey swarm fly nominal paths planned by conflict-based search (CBS, [Chapter 9](#)). The paths are conflict-free on the grid, but the drones do not fly the grid: they track the paths with a velocity controller, they are late or early by a second or two, the wind pushes them sideways, and a delivery drone that belongs to nobody in the swarm cuts across the survey area. In the last three seconds before two drones would touch, a global replan is too slow and a static plan is worthless. What is needed is a *local* rule that every drone can evaluate a dozen times per second from what it sees around it: the positions, velocities and sizes of its neighbors. The rule must return a velocity that is safe for a short horizon, close to the velocity the nominal path asks for, and computable in microseconds.

Velocity obstacles (VO, [Chapter 12](#)) give such a rule for one agent among moving obstacles that do not react: choose any velocity outside the cone of velocities that lead to a collision. The rule breaks down when the obstacles are other drones running the same rule. Each drone sees the other swerve, concludes that the danger is gone, swerves back, and the pair

performs a *reciprocal dance*: a zig-zag of velocity changes at every time step, sometimes ending in a collision that neither drone intended. This chapter repairs the rule in two steps. The reciprocal velocity obstacle (RVO) [15] lets each agent take half of the avoidance and removes the oscillation for a pair. Optimal reciprocal collision avoidance (ORCA) [14] turns the cone into a half-plane of permitted velocities per neighbor, so that the velocity choice among any number of neighbors becomes a tiny linear program with a provable pairwise guarantee. ORCA is the local safety layer of the hybrid architecture of Chapter 24 and the subject of Week 6 of the study plan (Appendix A). Its priority in the training plan is **Essential**; RVO is **High**.

The chapter proceeds as follows. Section 13.2 shows the dance and the idea of sharing the responsibility. Section 13.3 fixes the notation, recalls the velocity obstacle, states the oscillation precisely and defines RVO. Section 13.4 derives the ORCA half-plane with a figure for every step, including the cases of a non-cooperative obstacle and of three dimensions. Section 13.5 solves the per-agent program and its infeasible variant. Section 13.6 works through one encounter with every number, and Section 13.7 runs eight agents. Section 13.8 proves the guarantee and delimits it, and the remaining sections cover variants, implementation, the drone system and exercises.

13.2 The idea in plain words

Two people walk toward each other in a corridor. If both step to their left at the same moment, they are still on a collision course, so both step back, and the well-known corridor dance begins. The dance stops when both understand that the other one is also going to move: each steps *half* as far as it would have to step alone, and keeps that step. That is the whole content of reciprocity. Figure 13.1 shows the same story for two agents that choose velocities. With a plain velocity obstacle, A at time t sees B on a collision course and swerves by the full amount that takes A out of B's cone; B does the same. One time step later, each sees the other's new velocity, and in the new relative velocity the cone no longer contains the preferred velocity: both return to it, and at $t + 2\Delta t$ they are head-on again. With reciprocity, A swerves only halfway, trusting B to do the other half. The relative velocity changes by the full amount, so the pair is safe, and one step later nothing has changed: the halfway velocities are still the best permitted ones, and the agents keep them.

RVO writes this as a set: A may choose $\mathbf{v}$ only if the velocity $2\mathbf{v} - \mathbf{v}_A$, which is twice as far from A's current velocity as $\mathbf{v}$, lies outside the ordinary velocity obstacle. ORCA goes one step further. It does not ask for a set of velocities outside a cone, which is not convex and makes the choice among several neighbors a combinatorial problem. Instead it computes, for the pair (A, B) , the smallest change $\mathbf{u}$ of the relative velocity that reaches the boundary of the cone, gives half of it to each agent, and permits A every velocity on the far side of the line through $\mathbf{v}_A + \frac{1}{2}\mathbf{u}$ perpendicular to $\mathbf{u}$: a half-plane. B receives the mirror half-plane. Whatever the two agents choose in their half-planes, their relative velocity ends up outside the cone. With many neighbors A intersects all its half-planes and picks the velocity closest to the one it prefers, which is a two-dimensional linear program that is solved in microseconds.

Key idea

Never avoid alone what the other agent will also avoid. RVO lets each agent take half of the velocity change; ORCA computes that change $\mathbf{u}$ once for the pair, gives $\frac{1}{2}\mathbf{u}$ to each agent as a half-plane of permitted velocities, and lets every agent choose, inside all its half-planes and its speed limit, the velocity closest to its preferred one. If the half-planes intersect and all agents use the same horizon, no pair collides within that horizon.

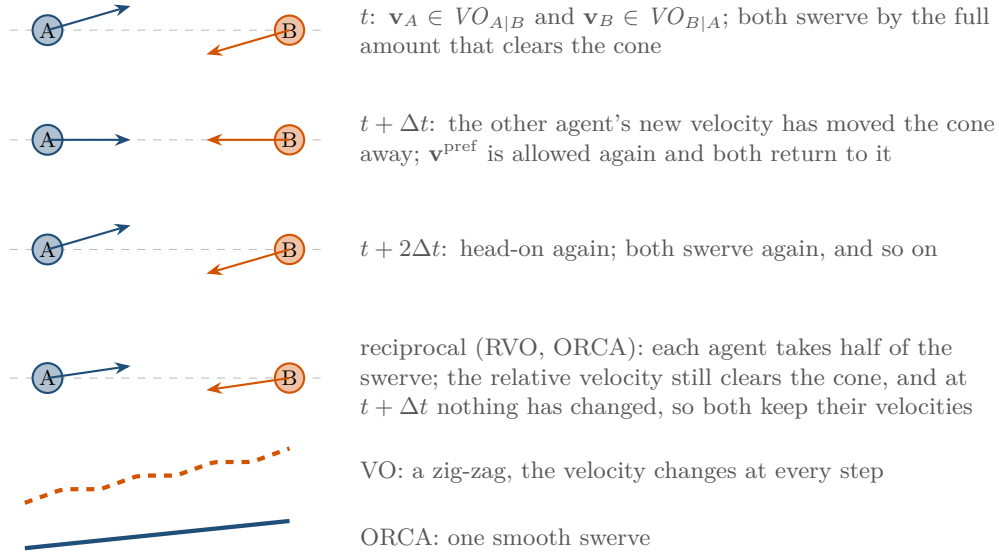


Figure 13.1. The reciprocal dance. Top three rows: with plain velocity obstacles both agents swerve, both return, both swerve again, because each reacts to a neighbor who has already reacted. Fourth row: with reciprocity each agent takes half of the swerve, the relative velocity is the same as if one agent had taken all of it, and at the next step the situation is unchanged. Bottom: the trajectories, a zig-zag against one smooth swerve.

13.3 Problem statement and notation

13.3.1 Agents, relative motion and velocity obstacles

Agents are disks in the plane, as in Chapter 2. Agent A has center $\mathbf{p}_A \in \mathbb{R}^2$, radius r_A , current velocity $\mathbf{v}_A$, a **preferred velocity** $\mathbf{v}_A^{\text{pref}}$ (the velocity it would fly if it were alone, for instance toward the next waypoint of its nominal path at cruise speed) and a speed limit $v_{\max}$. For a pair (A, B) we write

$$\mathbf{p}_{\text{rel}} = \mathbf{p}_B - \mathbf{p}_A, \quad \mathbf{v}_{\text{rel}} = \mathbf{v}_A - \mathbf{v}_B, \quad R = r_A + r_B \quad (13.1)$$

for the relative position, the relative velocity and the **combined radius**, the same symbol R that Chapter 12 uses. Seen from A , agent B is a point at $\mathbf{p}_{\text{rel}}$ that moves with velocity $-\mathbf{v}_{\text{rel}}$, and the two disks overlap at time $t \geq 0$ exactly when $\|\mathbf{p}_{\text{rel}} - t \mathbf{v}_{\text{rel}}\| < R$, that is, when $t \mathbf{v}_{\text{rel}}$ lies in the open disk $D(\mathbf{p}_{\text{rel}}, R)$ of the Minkowski sum (Chapter 2). We use open disks throughout: a relative velocity along which the disks merely touch counts as safe, and the safety margin that every implementation adds to the radii (Section 13.10) covers the difference.

Chapter 12 develops the velocity obstacle in detail; we recall what this chapter needs, in the relative frame, which is where ORCA lives.

Definition 13.1 (Velocity obstacle, truncated velocity obstacle). For a pair (A, B) with $\|\mathbf{p}_{\text{rel}}\| > R$, the **velocity obstacle** of A induced by B is the set of relative velocities that lead to a collision at some future time,

$$VO_{A|B} = \left\{ \mathbf{v} \in \mathbb{R}^2 : \exists t > 0, t \mathbf{v} \in D(\mathbf{p}_{\text{rel}}, R) \right\}, \quad (13.2)$$

and the **truncated velocity obstacle** with horizon $\tau > 0$ is the set of relative velocities that lead to a collision within time τ ,

$$VO_{A|B}^\tau = \left\{ \mathbf{v} \in \mathbb{R}^2 : \exists t \in (0, \tau], t \mathbf{v} \in D(\mathbf{p}_{\text{rel}}, R) \right\} = \bigcup_{t \in (0, \tau]} D(\mathbf{p}_{\text{rel}}/t, R/t). \quad (13.3)$$

In absolute terms, $\mathbf{v}_A$ is dangerous for A given $\mathbf{v}_B$ if $\mathbf{v}_A - \mathbf{v}_B \in VO_{A|B}$; the set of such $\mathbf{v}_A$ is the translate $\mathbf{v}_B + VO_{A|B}$, which Chapter 12 writes as $VO_{A|B}(\mathbf{v}_B)$.

$VO_{A|B}$ is the open cone with apex at the origin whose two boundary rays, the *legs*, are tangent to the disk $D(\mathbf{p}_{\text{rel}}, R)$; its axis is the direction of $\mathbf{p}_{\text{rel}}$, at angle φ from the x -axis, and its half-angle is

$$\theta = \arcsin \frac{R}{\|\mathbf{p}_{\text{rel}}\|}. \quad (13.4)$$

The truncated set $VO_{A|B}^\tau$ is the union of the disks $D(\mathbf{p}_{\text{rel}}/t, R/t)$, which shrink and move toward the apex as t grows. All of them are tangent to the two legs, and the smallest one, $D(\mathbf{p}_{\text{rel}}/\tau, R/\tau)$, cuts the tip off the cone (Figure 13.4(a)). The boundary of $VO_{A|B}^\tau$ therefore consists of three pieces: the arc of the smallest disk that faces the apex, between the two points where the legs touch it, and the two legs from those tangent points outward. Membership is a ray–disc test (Chapter 2): $\mathbf{v} \in VO_{A|B}^\tau$ exactly when the ray $t \mapsto t\mathbf{v}$ enters $D(\mathbf{p}_{\text{rel}}, R)$ at a time at most τ .

13.3.2 The oscillation problem, stated precisely

The velocity obstacle rule of Chapter 12 is: at every time step, choose the velocity closest to $\mathbf{v}^{\text{pref}}$ that lies outside the velocity obstacles induced by the *current* velocities of the neighbors. When the neighbors are also agents running this rule, the current velocities they are reacting to are about to change, and the rule chases a moving target. Figure 13.1 shows the mechanism; the following proposition states it in the simplest symmetric case, which is also the case that occurs most often, because two drones that fly toward each other along a shared corridor are in exactly this situation.

Proposition 13.2 (The reciprocal dance of velocity obstacles). *Let two agents of equal radius fly toward each other with $\mathbf{v}_A^{\text{pref}} = -\mathbf{v}_B^{\text{pref}} = (s, 0)$, far enough apart that the cone half-angle θ does not change noticeably during two time steps, and let both update their velocities synchronously with the velocity obstacle rule (untruncated cones). Suppose that at step k both fly their preferred velocities and that both pass on the same side. Then at step $k+1$ both deviate by $\delta = 2s \sin \theta$ perpendicular to the cone leg, at step $k+2$ both are back at their preferred velocities, and the pattern repeats with period two.*

Proof. At step k the apex of A 's cone is at $\mathbf{v}_B = (-s, 0)$, and $\mathbf{v}_A^{\text{pref}} = (s, 0)$ lies on the axis of the cone at distance $2s$ from the apex, hence at perpendicular distance $2s \sin \theta = \delta$ inside each leg. The closest velocity outside the cone is $\mathbf{v}_A^{\text{pref}} + \delta \mathbf{n}$, where $\mathbf{n}$ is the outward unit normal of the chosen leg; by the symmetry of the configuration B chooses $\mathbf{v}_B^{\text{pref}} - \delta \mathbf{n}$ (passing on the same side means opposite normals in the world frame). At step $k+1$ the apex of A 's cone has moved to $\mathbf{v}_B^{\text{pref}} - \delta \mathbf{n}$, so the signed distance of $\mathbf{v}_A^{\text{pref}}$ to the leg, measured along $\mathbf{n}$, has grown from $-\delta$ to 0 : the preferred velocity lies on the boundary of the cone and is permitted, and it is the closest permitted velocity to itself. Both agents return to their preferred velocities, which restores the situation of step k . $\square$

The relative velocity of the pair alternates between the colliding $(2s, 0)$ and the safe $(2s, 0) + 2\delta \mathbf{n}$, so the agents make lateral progress only every other step, and their velocity changes by δ at every step. In the simulation of Figure 13.3 (sampled velocities, $\Delta t = 0.1$ s) the lateral velocity of agent A under the velocity obstacle rule jumps back and forth between about $+0.05$ and -0.18 m/s for the whole approach (range $[-0.18, +0.05]$ m/s), and its velocity change reverses direction 49 times in 100 steps. For a drone this is a stream of contradictory commands to the attitude controller. Truncating the cones does not help: the two agents still react to each other's stale velocities. What helps is to make the rule anticipate the reaction of the other agent.

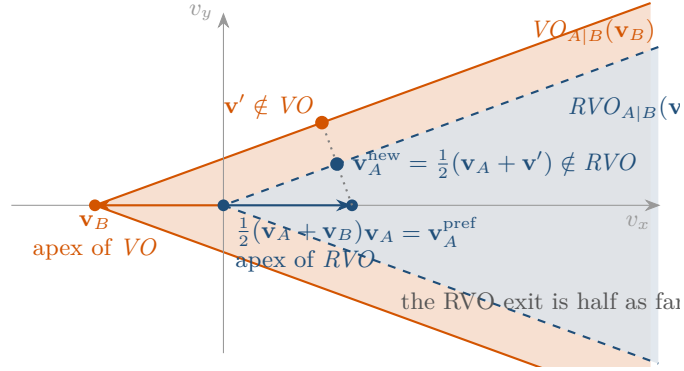


Figure 13.2. Velocity space of agent A in a head-on encounter, $\mathbf{v}_A = (1, 0)$ and $\mathbf{v}_B = (-1, 0)$, cone half-angle 20° . The velocity obstacle (orange) has its apex at $\mathbf{v}_B$; the reciprocal velocity obstacle (blue, dashed) is the same cone with the apex at $\frac{1}{2}(\mathbf{v}_A + \mathbf{v}_B)$. The closest velocity outside the RVO is the midpoint between $\mathbf{v}_A$ and the closest velocity $\mathbf{v}'$ outside the VO: A takes half of the swerve.

13.3.3 Reciprocal velocity obstacles

Definition 13.3 (Reciprocal velocity obstacle). The **reciprocal velocity obstacle** of agent A induced by agent B is

$$RVO_{A|B}(\mathbf{v}_B, \mathbf{v}_A) = \left\{ \mathbf{v} \in \mathbb{R}^2 : 2\mathbf{v} - \mathbf{v}_A \in \mathbf{v}_B + VO_{A|B} \right\}. \quad (13.5)$$

Agent A may choose $\mathbf{v}$ only if $\mathbf{v} \notin RVO_{A|B}(\mathbf{v}_B, \mathbf{v}_A)$, that is, only if $\mathbf{v}$ is the average of its current velocity $\mathbf{v}_A$ and some velocity $\mathbf{v}' = 2\mathbf{v} - \mathbf{v}_A$ that lies outside the velocity obstacle: $\mathbf{v} = \frac{1}{2}(\mathbf{v}_A + \mathbf{v}')$.

The definition says that A goes only halfway from its current velocity to a safe one, and relies on B to go the other half. The set has a simple shape (Figure 13.2).

Proposition 13.4 (The apex moves to the midpoint). $RVO_{A|B}(\mathbf{v}_B, \mathbf{v}_A) = \frac{1}{2}(\mathbf{v}_A + \mathbf{v}_B) + \frac{1}{2}VO_{A|B}$: the reciprocal velocity obstacle is the velocity obstacle cone translated so that its apex lies at the average of the two current velocities.

Proof. $2\mathbf{v} - \mathbf{v}_A \in \mathbf{v}_B + VO_{A|B}$ holds exactly when $\mathbf{v} \in \frac{1}{2}(\mathbf{v}_A + \mathbf{v}_B) + \frac{1}{2}VO_{A|B}$, and $\frac{1}{2}VO_{A|B} = VO_{A|B}$ because a cone with apex at the origin is closed under scaling by positive factors. $\square$

For the truncated cone the scaling argument changes the horizon: $\frac{1}{2}VO_{A|B}^\tau = VO_{A|B}^{2\tau}$, because halving the disk $D(\mathbf{p}_{\text{rel}}/t, R/t)$ gives $D(\mathbf{p}_{\text{rel}}/2t, R/2t)$, so the reciprocal version of a truncated cone with horizon τ is a truncated cone with horizon 2τ at the midpoint apex (Exercise 13.5). Now repeat the argument of Proposition 13.2 with the new rule.

Proposition 13.5 (RVO removes the dance). In the setting of Proposition 13.2, let both agents choose at every step the velocity closest to their preferred one outside their reciprocal velocity obstacle, passing on the same side. Then the velocities chosen at step $k+1$ satisfy $\mathbf{v}_A - \mathbf{v}_B \notin VO_{A|B}$, and they are chosen again at every later step while the geometry is unchanged.

Proof. At step k the apex of A 's reciprocal cone is at the midpoint $\mathbf{m} = \frac{1}{2}(\mathbf{v}_A^{\text{pref}} + \mathbf{v}_B^{\text{pref}}) = \mathbf{0}$, so $\mathbf{v}_A^{\text{pref}}$ lies at distance s from the apex along the axis and at perpendicular distance $s \sin \theta = \delta/2$ inside the leg. A chooses $\mathbf{v}'_A = \mathbf{v}_A^{\text{pref}} + \frac{\delta}{2}\mathbf{n}$ and, by symmetry, B chooses $\mathbf{v}'_B = \mathbf{v}_B^{\text{pref}} - \frac{\delta}{2}\mathbf{n}$. The new relative velocity $\mathbf{v}'_A - \mathbf{v}'_B = (2s, 0) + \delta\mathbf{n}$ lies on the boundary of $VO_{A|B}$, hence outside the open cone. At step $k+1$ the midpoint $\frac{1}{2}(\mathbf{v}'_A + \mathbf{v}'_B) = \mathbf{m}$ is unchanged, so the reciprocal

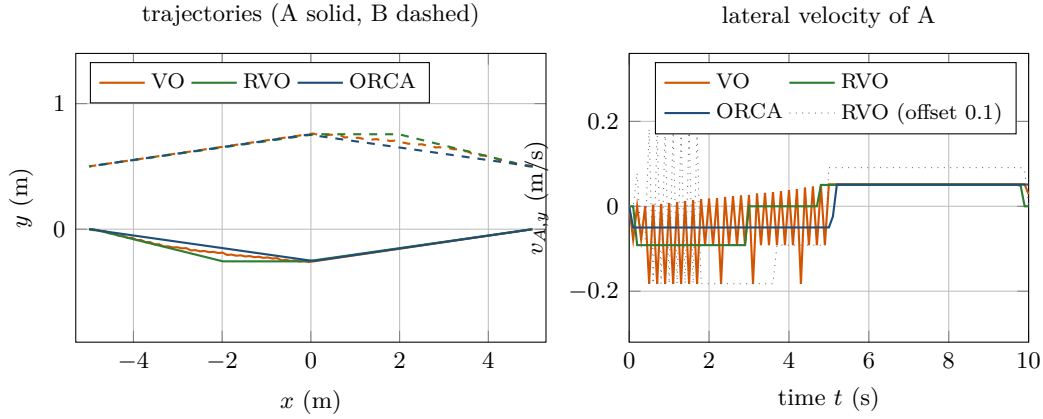


Figure 13.3. Two agents swapping places head-on ($\mathbf{p}_A = (-5, 0)$, $\mathbf{p}_B = (5, 0.5)$, radii 0.5, $\tau = 5$ s, $\Delta t = 0.1$ s), simulated with `ch13_orca.py`. Left: trajectories under sampled VO, sampled RVO and ORCA. Right: the lateral velocity of A. The VO agent alternates between swerving and returning at every step (49 reversals), the RVO agent settles after two steps, the ORCA agent flies one small constant lateral velocity. The dotted gray curve is RVO in an almost symmetric instance (offset 0.1): the agents disagree about the side and switch it 16 times before they settle, the reciprocal dance of RVO.

cone is unchanged; $\mathbf{v}_A^{\text{pref}}$ is still inside by $\delta/2$, and the closest permitted velocity is again $\mathbf{v}'_A$. The same holds for B , and by induction for every later step. $\square$

Van den Berg, Lin and Manocha prove both statements in general, not only in the symmetric case: if both agents choose velocities outside their reciprocal velocity obstacles and pass on the same side, their new relative velocity is outside the velocity obstacle, and if in addition both choose the closest such velocity to their preferred one, the chosen velocities are kept at the next step [15]. In the simulation of Figure 13.3, RVO reverses the direction of its velocity change once in 100 steps against 49 times for the velocity obstacle, and the sum of all velocity changes of agent A drops from 8.32 to 1.44 m/s.

Why RVO is not enough. The guarantee is a two-agent statement, and it rests on two assumptions that fail in a crowd. First, it assumes that B 's reaction mirrors A 's. With a third agent C present, B chooses the closest velocity outside $RVO_{B|A} \cup RVO_{B|C}$, and that velocity may lie on the boundary of $RVO_{B|C}$ alone: B then takes none of the responsibility for A , the relative velocity of A and B moves only halfway out of the cone, and the pair is still on a collision course. Second, it assumes that both agents pass on the same side. The union of several cones is not convex, and the closest velocity outside it jumps from one gap to another when a cone moves a little; two agents that choose different sides do not change their relative velocity at all, they only move the midpoint, and at the next step one or both switch sides. This is the **reciprocal dance** of RVO, visible in the dotted curve of Figure 13.3 even for two agents whose configuration is almost symmetric, and it is the reason why van den Berg et al. state that RVO offers no guarantee for more than two agents [14]. The hybrid reciprocal velocity obstacle (HRVO) of Snape et al. combines one leg of the VO with one leg of the RVO so that passing on the wrong side costs more, which reduces the dance but does not remove it [154]. ORCA removes both problems at the root: the permitted set per neighbor is a half-plane, which is convex, and the share of the responsibility is fixed by construction rather than assumed.

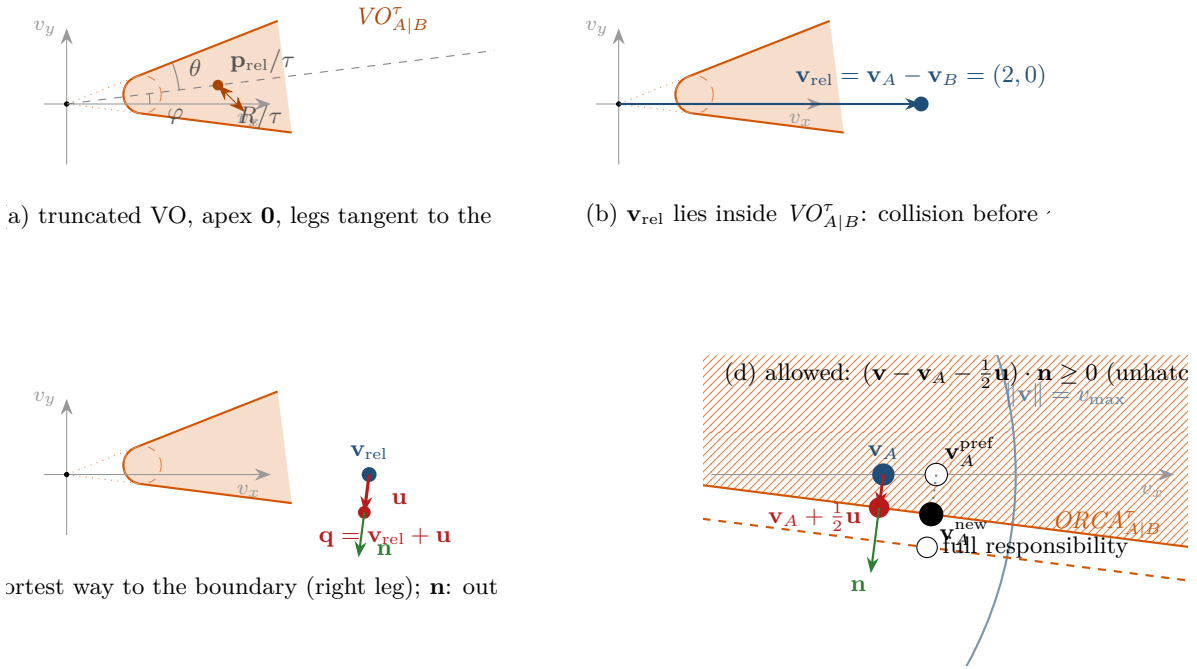


Figure 13.4. The four steps of the ORCA construction on the worked example ($\mathbf{p}_{\text{rel}} = (4, 0.5)$, $R = 1$, $\tau = 4$ s). (a) The truncated velocity obstacle in relative-velocity space: a cone with apex at the origin, half-angle θ around the direction of $\mathbf{p}_{\text{rel}}$, cut off by the disk of center $\mathbf{p}_{\text{rel}}/\tau$ and radius R/τ . (b) The current relative velocity $\mathbf{v}_{\text{rel}} = \mathbf{v}_A - \mathbf{v}_B$ lies inside. (c) $\mathbf{u}$ is the shortest vector from $\mathbf{v}_{\text{rel}}$ to the boundary, here to the right leg, and $\mathbf{n}$ the outward unit normal there. (d) In the velocity space of A, the permitted half-plane is bounded by the line through $\mathbf{v}_A + \frac{1}{2}\mathbf{u}$ perpendicular to $\mathbf{n}$; the hatched side is forbidden. The preferred velocity is forbidden and is projected onto the boundary. The dashed line is the boundary for full responsibility ($\mathbf{v}_A + \mathbf{u}$), which applies against a non-cooperative obstacle.

13.4 The ORCA construction

ORCA replaces the question “which velocities are safe for A?” by the question “what is the smallest change of the *relative* velocity that makes the pair safe for the next τ seconds, and how is it shared?”. The answer is built in four steps, one per panel of Figure 13.4. Throughout, τ is the **time horizon**: ORCA promises nothing beyond it, and every agent recomputes its velocity long before it elapses.

Step 1: the truncated velocity obstacle. Form $VO_{A|B}^\tau$ of Definition 13.1 in relative-velocity space (Figure 13.4(a)). It depends only on $\mathbf{p}_{\text{rel}}$, R and τ , so both agents can compute the same set: B obtains $VO_{B|A}^\tau = -VO_{A|B}^\tau$, the mirror image through the origin, because its relative position is $-\mathbf{p}_{\text{rel}}$.

Step 2: the current relative velocity. Take $\mathbf{v}_{\text{rel}} = \mathbf{v}_A - \mathbf{v}_B$ from the current velocities (Figure 13.4(b)). ORCA calls the velocities around which the construction is centered the *optimization velocities*; using the current velocities is the choice that makes the half-planes of A and B consistent, because both can observe them, and it is the choice of the RVO2 library. If $\mathbf{v}_{\text{rel}} \in VO_{A|B}^\tau$, the agents collide within τ unless one of them changes velocity.

Step 3: the smallest change and the outward normal. Let $\mathbf{q}$ be the point of the boundary of $VO_{A|B}^\tau$ closest to $\mathbf{v}_{\text{rel}}$, let

$$\mathbf{u} = \mathbf{q} - \mathbf{v}_{\text{rel}} \quad (13.6)$$

be the vector that carries $\mathbf{v}_{\text{rel}}$ to it, and let $\mathbf{n}$ be the outward unit normal of $VO_{A|B}^\tau$ at $\mathbf{q}$ (Figure 13.4(c)). If $\mathbf{v}_{\text{rel}}$ is inside the truncated cone, $\mathbf{u}$ points outward along $\mathbf{n}$ and $\|\mathbf{u}\|$ is the smallest velocity change of the pair that makes it safe for τ . If $\mathbf{v}_{\text{rel}}$ is already outside, $\mathbf{u}$ points toward the cone, against $\mathbf{n}$, and $\|\mathbf{u}\|$ is the margin by which the pair may drift toward danger. The boundary has three pieces, and the closest point is found by the following case analysis, which is the heart of every ORCA implementation.

Lemma 13.6 (Closest boundary point of the truncated velocity obstacle). *Let $\|\mathbf{p}_{\text{rel}}\| > R$, let $\mathbf{c} = \mathbf{p}_{\text{rel}}/\tau$ and $\rho = R/\tau$ be the center and radius of the truncating disk, and let $\mathbf{w} = \mathbf{v}_{\text{rel}} - \mathbf{c}$. Write $\mathbf{p} \times \mathbf{w} = p_x w_y - p_y w_x$ for the two-dimensional cross product and $\ell = \sqrt{\|\mathbf{p}_{\text{rel}}\|^2 - R^2}$ for the length of a tangent from the apex to $D(\mathbf{p}_{\text{rel}}, R)$.*

- (i) Arc. *If $\mathbf{w} \cdot \mathbf{p}_{\text{rel}} < 0$ and $(\mathbf{w} \cdot \mathbf{p}_{\text{rel}})^2 > R^2 \|\mathbf{w}\|^2$, that is, if the angle between $\mathbf{w}$ and $-\mathbf{p}_{\text{rel}}$ is smaller than $\arccos(R/\|\mathbf{p}_{\text{rel}}\|) = \pi/2 - \theta$, then the closest boundary point lies on the front arc: $\mathbf{n} = \mathbf{w}/\|\mathbf{w}\|$ and $\mathbf{u} = (\rho - \|\mathbf{w}\|)\mathbf{n}$.*
- (ii) Legs. *Otherwise the closest point is the foot of the perpendicular from $\mathbf{v}_{\text{rel}}$ onto one leg. The unit directions of the legs are $\mathbf{d}_L = (p_x \ell - p_y R, p_x R + p_y \ell)/\|\mathbf{p}_{\text{rel}}\|^2$ (left) and $\mathbf{d}_R = (p_x \ell + p_y R, -p_x R + p_y \ell)/\|\mathbf{p}_{\text{rel}}\|^2$ (right), that is, the direction of $\mathbf{p}_{\text{rel}}$ rotated by $+\theta$ and $-\theta$. If $\mathbf{p}_{\text{rel}} \times \mathbf{w} > 0$ the left leg is closest and $\mathbf{n} = (-d_{L,y}, d_{L,x})$; otherwise the right leg is closest and $\mathbf{n} = (d_{R,y}, -d_{R,x})$. In both cases $\mathbf{q} = (\mathbf{v}_{\text{rel}} \cdot \mathbf{d})\mathbf{d}$ with $\mathbf{d}$ the chosen leg and $\mathbf{u} = \mathbf{q} - \mathbf{v}_{\text{rel}}$.*

If $\|\mathbf{p}_{\text{rel}}\| \leq R$ the disks already overlap, the cone is the whole plane, and the construction is applied instead to the disk $D(\mathbf{p}_{\text{rel}}/\Delta t, R/\Delta t)$ of relative velocities that leave the disks overlapping after one time step: $\mathbf{w} = \mathbf{v}_{\text{rel}} - \mathbf{p}_{\text{rel}}/\Delta t$, $\mathbf{n} = \mathbf{w}/\|\mathbf{w}\|$ and $\mathbf{u} = (R/\Delta t - \|\mathbf{w}\|)\mathbf{n}$.

Proof sketch. The two legs are tangent to the truncating disk at the points where the radii make the angle $\pi/2 - \theta$ with the direction from the center toward the apex, so the front arc spans exactly the directions $\mathbf{w}$ of case (i). For such $\mathbf{w}$ the nearest point of the circle is $\mathbf{c} + \rho \mathbf{w}/\|\mathbf{w}\|$, and no point of a leg is nearer, because the legs lie outside the circle and touch it only at the ends of the arc. For every other $\mathbf{w}$ the nearest boundary point is on the leg on the same side of the axis as $\mathbf{w}$, and it is the foot of the perpendicular, which lies beyond the tangent point precisely because $\mathbf{w}$ is outside the front sector (Exercise 13.3 asks for the details). The overlap case is the ray–disc condition $\|\mathbf{p}_{\text{rel}} - \Delta t \mathbf{v}_{\text{rel}}\| \geq R$ rewritten as $\mathbf{v}_{\text{rel}} \notin D(\mathbf{p}_{\text{rel}}/\Delta t, R/\Delta t)$. $\square$

The normal orientation deserves attention. In case (i), $\mathbf{n}$ points away from the center of the truncating disk, which is outward for the arc whether $\mathbf{v}_{\text{rel}}$ is inside or outside the disk. In case (ii), $\mathbf{n}$ is the leg direction rotated by $+90^\circ$ for the left leg and by -90° for the right leg, which points out of the cone on either side. The vector $\mathbf{u}$ is always parallel to $\mathbf{n}$; its sign says whether $\mathbf{v}_{\text{rel}}$ is inside ($\mathbf{u} \cdot \mathbf{n} > 0$) or outside ($\mathbf{u} \cdot \mathbf{n} < 0$) the truncated cone.

Step 4: the half-plane. Give half of the change to each agent (Figure 13.4(d)).

Definition 13.7 (ORCA half-plane). With $\mathbf{u}$ and $\mathbf{n}$ as in Step 3, the set of velocities permitted to agent A with respect to B for the horizon τ is the closed half-plane

$$ORCA_{A|B}^\tau = \left\{ \mathbf{v} \in \mathbb{R}^2 : (\mathbf{v} - (\mathbf{v}_A + \frac{1}{2}\mathbf{u})) \cdot \mathbf{n} \geq 0 \right\}. \quad (13.7)$$

The half-plane of B with respect to A is obtained by the same construction from B 's point of view; since $VO_{B|A}^\tau = -VO_{A|B}^\tau$ and $\mathbf{v}_B - \mathbf{v}_A = -\mathbf{v}_{\text{rel}}$, its closest point is $-\mathbf{q}$, its change is

$-\mathbf{u}$ and its normal is $-\mathbf{n}$, so

$$ORCA_{B|A}^\tau = \left\{ \mathbf{v} \in \mathbb{R}^2 : (\mathbf{v} - (\mathbf{v}_B - \frac{1}{2}\mathbf{u})) \cdot (-\mathbf{n}) \geq 0 \right\}. \quad (13.8)$$

The boundary of $ORCA_{A|B}^\tau$ passes through $\mathbf{v}_A + \frac{1}{2}\mathbf{u}$, the current velocity shifted by half of the required change, and is perpendicular to $\mathbf{u}$. Agent A must move its velocity at least $\frac{1}{2}\|\mathbf{u}\|$ in direction $\mathbf{n}$ when the pair is on a collision course, and may move it at most $\frac{1}{2}\|\mathbf{u}\|$ against $\mathbf{n}$ when it is not. This is why the method is called *reciprocal*: the responsibility is split half and half by construction, and no assumption about the other agent's choice is needed beyond the fact that it uses the same rule. It is called *optimal* because the pair $(ORCA_{A|B}^\tau, ORCA_{B|A}^\tau)$ is what van den Berg et al. call *reciprocally maximal*: among all pairs of half-planes that are safe in the sense of Theorem 13.10 and treat the two agents symmetrically, no other pair permits more velocities near the optimization velocities; they argue this geometrically and note that the equal split can be replaced by any fixed split that sums to one [14]. Algorithm 13.1 collects Steps 1–4 with a general share α .

Algorithm 13.1. The ORCA half-plane of agent A induced by neighbor B

Input: $\mathbf{p}_A, \mathbf{v}_A, r_A$ and $\mathbf{p}_B, \mathbf{v}_B, r_B$; horizon τ ; time step Δt ; responsibility share α ($\frac{1}{2}$ reciprocal, 1 non-cooperative)

Output: unit normal $\mathbf{n}$ and point $\mathbf{q}_A$ of the half-plane $\{\mathbf{v} : (\mathbf{v} - \mathbf{q}_A) \cdot \mathbf{n} \geq 0\}$

```

1 Function OrcaHalfPlane( $A, B, \tau, \Delta t, \alpha$ ):
2    $\mathbf{p} \leftarrow \mathbf{p}_B - \mathbf{p}_A$ ;  $\mathbf{v} \leftarrow \mathbf{v}_A - \mathbf{v}_B$ ;  $R \leftarrow r_A + r_B$ 
3    $(\mathbf{u}, \mathbf{n}) \leftarrow \text{OrcaClosestBoundaryPoint}(\mathbf{p}, \mathbf{v}, R, \tau, \Delta t)$ 
4   return  $(\mathbf{n}, \mathbf{v}_A + \alpha \mathbf{u})$ 

5 Function OrcaClosestBoundaryPoint( $\mathbf{p}, \mathbf{v}, R, \tau, \Delta t$ ):
6   if  $\|\mathbf{p}\| \leq R$  then
7      $\mathbf{c} \leftarrow \mathbf{p}/\Delta t$ ;  $\rho \leftarrow R/\Delta t$ ;  $\mathbf{w} \leftarrow \mathbf{v} - \mathbf{c}$ ;  $\mathbf{n} \leftarrow \mathbf{w}/\|\mathbf{w}\|$     // overlapping: separate in
      one step
8     return  $((\rho - \|\mathbf{w}\|) \mathbf{n}, \mathbf{n})$ 
9    $\mathbf{c} \leftarrow \mathbf{p}/\tau$ ;  $\rho \leftarrow R/\tau$ ;  $\mathbf{w} \leftarrow \mathbf{v} - \mathbf{c}$     // the disk that truncates the cone
10  if  $\mathbf{w} \cdot \mathbf{p} < 0$  and  $(\mathbf{w} \cdot \mathbf{p})^2 > R^2 \|\mathbf{w}\|^2$  then
11     $\mathbf{n} \leftarrow \mathbf{w}/\|\mathbf{w}\|$     // front sector: closest point on the arc
12    return  $((\rho - \|\mathbf{w}\|) \mathbf{n}, \mathbf{n})$ 
13   $\ell \leftarrow \sqrt{\|\mathbf{p}\|^2 - R^2}$     // length of a tangent from the apex
14  if  $\mathbf{p} \times \mathbf{w} > 0$  then
15     $\mathbf{d} \leftarrow (p_x \ell - p_y R, p_x R + p_y \ell) / \|\mathbf{p}\|^2$ ;  $\mathbf{n} \leftarrow (-d_y, d_x)$     // left leg
16  else
17     $\mathbf{d} \leftarrow (p_x \ell + p_y R, -p_x R + p_y \ell) / \|\mathbf{p}\|^2$ ;  $\mathbf{n} \leftarrow (d_y, -d_x)$     // right leg
18   $\mathbf{q} \leftarrow (\mathbf{v} \cdot \mathbf{d}) \mathbf{d}$     // foot of the perpendicular on the leg
19  return  $(\mathbf{q} - \mathbf{v}, \mathbf{n})$ 
```

Walkthrough. Line 2 moves to the relative frame. Line 7 is the emergency case of Lemma 13.6: the disks overlap, the horizon is replaced by one time step, and the half-plane pushes the agents apart. Line 9 forms the truncating disk and the vector $\mathbf{w}$ from its center to the relative velocity; the test that follows is case (i) of the lemma, with the angle condition written without trigonometric functions. Lines 15–17 select the leg by the sign of the cross product and rotate its direction into the outward normal; the formulas for $\mathbf{d}$ are the

direction of $\mathbf{p}$ rotated by $\pm\theta$, using $\cos\theta = \ell/\|\mathbf{p}\|$ and $\sin\theta = R/\|\mathbf{p}\|$. Line 19 returns the change to the foot of the perpendicular, and line 4 shifts the current velocity by the share α of it.

13.4.1 Non-cooperative obstacles: full responsibility

The half split is right only when B runs the same rule. An intruder that does not know about A , a static obstacle, or a drone whose controller has failed will not take its half. If A nevertheless takes only half of $\mathbf{u}$, the relative velocity moves only halfway to the boundary and the collision course remains. In that case A takes **full responsibility**:

$$ORCA_{A|B}^{\tau,1} = \left\{ \mathbf{v} \in \mathbb{R}^2 : (\mathbf{v} - (\mathbf{v}_A + \mathbf{u})) \cdot \mathbf{n} \geq 0 \right\}, \quad (13.9)$$

the dashed line of Figure 13.4(d), obtained from Algorithm 13.1 with $\alpha = 1$. For a static obstacle, $\mathbf{v}_B = \mathbf{0}$ and the relative velocity is $\mathbf{v}_A$ itself; for an intruder, $\mathbf{v}_B$ is its estimated or predicted velocity (Chapters 18 and 20), and r_B is inflated by the uncertainty of the prediction. In general, if A and B use shares α_A and α_B , the argument of Theorem 13.10 below goes through exactly when $\alpha_A + \alpha_B = 1$: the reciprocal split is $(\frac{1}{2}, \frac{1}{2})$, the non-cooperative case is $(1, 0)$, where B keeps the velocity that A assumed for it, and a priority scheme is any other split. This is the case that matters for the hybrid architecture of Chapter 24: the swarm mates of a drone are reciprocal, the intruder is not, and the two kinds of half-planes go into the same program.

13.4.2 Three dimensions: half-spaces

Drones fly in $\mathbb{R}^3$, and nothing in the construction was specific to the plane. The velocity obstacle becomes the circular cone with apex at the origin and half-angle $\theta = \arcsin(R/\|\mathbf{p}_{\text{rel}}\|)$ around the axis $\mathbf{p}_{\text{rel}}$, truncated by the ball of center $\mathbf{p}_{\text{rel}}/\tau$ and radius R/τ ; its boundary is the front cap of the sphere and the lateral surface of the cone. The truncated cone is a body of revolution about the axis $\mathbf{p}_{\text{rel}}$, so the closest boundary point to $\mathbf{v}_{\text{rel}}$ can be taken in the plane spanned by $\mathbf{p}_{\text{rel}}$ and $\mathbf{v}_{\text{rel}}$ (the truncated cone is invariant under reflection in any plane containing $\mathbf{p}_{\text{rel}}$, so the set of nearest boundary points is invariant too; when $\mathbf{v}_{\text{rel}}$ lies outside the body the nearest point of a convex set is unique and therefore lies in the plane of $\mathbf{p}_{\text{rel}}$ and $\mathbf{v}_{\text{rel}}$, and when it lies inside, at least one nearest boundary point lies in that plane and the construction may take it; if $\mathbf{v}_{\text{rel}}$ is parallel to $\mathbf{p}_{\text{rel}}$ every plane through the axis gives an equally good $\mathbf{u}$ and the code picks one, exactly as the two-dimensional code breaks the leg tie by a sign), and in that plane the body is exactly the two-dimensional truncated cone. Hence $\mathbf{u}$ and $\mathbf{n}$ are computed by Lemma 13.6 in the coordinates of that plane and mapped back, and the permitted set is the **half-space**

$$ORCA_{A|B}^{\tau} = \left\{ \mathbf{v} \in \mathbb{R}^3 : (\mathbf{v} - (\mathbf{v}_A + \frac{1}{2}\mathbf{u})) \cdot \mathbf{n} \geq 0 \right\}. \quad (13.10)$$

The function `orca_half_space_3d` of `ch13_orca.py` does exactly this, spanning the plane by the unit vector along $\mathbf{p}_{\text{rel}}$ and the unit vector along the component of $\mathbf{v}_{\text{rel}}$ perpendicular to it, or, when that component vanishes, along a fixed helper axis; its self-test checks that the result is the two-dimensional half-plane when everything lies in a plane and that it rotates with the configuration. The per-agent program of the next section becomes a three-dimensional linear program over half-spaces and the ball $\|\mathbf{v}\| \leq v_{\text{max}}$; the incremental algorithm extends to three dimensions by recursion (when a half-space is violated, solve the two-dimensional program on its boundary plane) and keeps its expected linear running time [17, 148]. The RVO2-3D library implements this version.

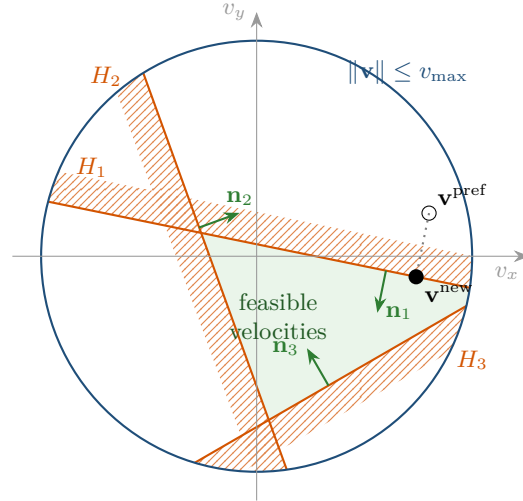


Figure 13.5. The per-agent program in velocity space: the speed disk and three half-planes H_1 , H_2 , H_3 (hatched bands on the forbidden sides, green normals pointing into the permitted sides). The feasible set is convex. The preferred velocity violates H_1 only, and the solution is its projection onto the boundary line of H_1 , which happens to satisfy H_2 , H_3 and the disk. Had the projection violated H_2 , the solution would sit at the intersection of the two boundary lines.

13.5 The per-agent program

13.5.1 The velocity closest to the preferred one

Agent A with neighbors $B_1, \dots, B_m$ has m half-planes $H_i = \{\mathbf{v} : (\mathbf{v} - \mathbf{q}_i) \cdot \mathbf{n}_i \geq 0\}$ from Algorithm 13.1, and it must also respect its speed limit. It chooses

$$\mathbf{v}_A^{\text{new}} = \arg \min_{\mathbf{v}} \|\mathbf{v} - \mathbf{v}_A^{\text{pref}}\| \quad \text{subject to} \quad \mathbf{v} \in H_1 \cap \dots \cap H_m \cap D(\mathbf{0}, v_{\max}). \quad (13.11)$$

The feasible set is an intersection of half-planes and a disk, hence convex, and the objective is the distance to a point, so the program has a unique solution whenever the feasible set is non-empty (Figure 13.5). It is a quadratic program, but a very special one: the solution is either $\mathbf{v}_A^{\text{pref}}$ itself, or its projection onto the disk, or it lies on the boundary line of one of the half-planes (or at the intersection of two of them, or of one with the circle). This structure is what the **incremental linear program** of Algorithm 13.2 exploits; it is the algorithm of Seidel [148] and of de Berg et al. [17], specialized to two dimensions as in the RVO2 library. (Appendix B.4.3 reviews linear programs in two variables.)

Walkthrough. The invariant of Algorithm 13.2 is that after the i -th iteration, $\mathbf{v}$ is the optimum of the program with the first i half-planes. Line 2 solves the program with no half-planes: the preferred velocity, clipped to the disk. If the next half-plane H_i is satisfied by the current optimum, adding it changes nothing, because the feasible set only shrinks (line 4). If H_i is violated, the new optimum must lie on the boundary line of H_i : the old optimum was the closest feasible point to $\mathbf{v}^{\text{pref}}$ in a convex set, the new feasible set is the old one cut by H_i , and by convexity the closest point of the cut set cannot lie strictly inside H_i (Exercise 13.6). So the problem drops to one dimension: **OrcaLineProgram** parameterizes the line as $\mathbf{q}_i + t\mathbf{d}$, finds the interval $[t_-, t_+]$ of the line inside the disk (line 13, from the quadratic $\|\mathbf{q}_i + t\mathbf{d}\|^2 = v_{\max}^2$), clips this interval by every earlier half-plane (line 20: the point $\mathbf{q}_i + t\mathbf{d}$ satisfies H_j when $t \cdot \text{den} \leq \text{num}$), and takes the parameter closest to the projection of $\mathbf{v}^{\text{pref}}$ (line 22). The interval becomes empty exactly when $H_1, \dots, H_i$ and the disk have no common

Algorithm 13.2. Incremental two-dimensional linear program over half-planes and the speed disk

Input: half-planes $H_i = \{\mathbf{v} : (\mathbf{v} - \mathbf{q}_i) \cdot \mathbf{n}_i \geq 0\}$, $i = 1, \dots, m$; speed limit $v_{\max}$; preferred velocity $\mathbf{v}^{\text{pref}}$

Output: $(\mathbf{v}, m+1)$ with $\mathbf{v}$ the velocity closest to $\mathbf{v}^{\text{pref}}$ in $H_1 \cap \dots \cap H_m \cap D(\mathbf{0}, v_{\max})$, or $(\mathbf{v}, i)$ if H_i makes the intersection empty, $\mathbf{v}$ then solving the program for $H_1, \dots, H_{i-1}$

```

1 Function OrcaLinearProgram( $H_1, \dots, H_m, v_{\max}, \mathbf{v}^{\text{pref}}$ ):
2    $\mathbf{v} \leftarrow \mathbf{v}^{\text{pref}}$ , scaled to length  $v_{\max}$  if  $\|\mathbf{v}^{\text{pref}}\| > v_{\max}$ 
3   for  $i \leftarrow 1$  to  $m$  do
4     if  $(\mathbf{v} - \mathbf{q}_i) \cdot \mathbf{n}_i < 0$  then
5        $\mathbf{v}' \leftarrow \text{OrcaLineProgram}(i, H_1, \dots, H_{i-1}, v_{\max}, \mathbf{v}^{\text{pref}})$  //  $H_i$  is violated: the
        optimum lies on its boundary
6       if  $\mathbf{v}' = \text{nil}$  then return  $(\mathbf{v}, i)$ 
7        $\mathbf{v} \leftarrow \mathbf{v}'$ 
8   return  $(\mathbf{v}, m+1)$ 

9 Function OrcaLineProgram( $i, H_1, \dots, H_{i-1}, v_{\max}, \mathbf{v}^{\text{pref}}$ ):
10   $\mathbf{d} \leftarrow (n_{i,y}, -n_{i,x})$  // along the boundary line, with  $H_i$  on its left
11   $b \leftarrow \mathbf{q}_i \cdot \mathbf{d}$ ;  $\Delta \leftarrow b^2 + v_{\max}^2 - \|\mathbf{q}_i\|^2$ 
12  if  $\Delta < 0$  then return nil (the line misses the disk)
13   $t_- \leftarrow -b - \sqrt{\Delta}$ ;  $t_+ \leftarrow -b + \sqrt{\Delta}$  // chord of the line inside the disk
14  for  $j \leftarrow 1$  to  $i-1$  do
15     $\mathbf{d}_j \leftarrow (n_{j,y}, -n_{j,x})$ ;  $\text{den} \leftarrow \mathbf{d} \times \mathbf{d}_j$ ;  $\text{num} \leftarrow \mathbf{d}_j \times (\mathbf{q}_i - \mathbf{q}_j)$ 
16    if  $|\text{den}| \leq \varepsilon$  ( $\varepsilon = 10^{-9}$ , a numerical tolerance) then
17      if  $\text{num} < 0$  then return nil (parallel,  $H_j$  excludes the whole line)
18      else continue
19    if  $\text{den} > 0$  then  $t_+ \leftarrow \min(t_+, \text{num}/\text{den})$ 
20    else  $t_- \leftarrow \max(t_-, \text{num}/\text{den})$ 
21    if  $t_- > t_+$  then return nil
22   $t \leftarrow \min(\max(\mathbf{d} \cdot (\mathbf{v}^{\text{pref}} - \mathbf{q}_i), t_-), t_+)$ 
23  return  $\mathbf{q}_i + t \mathbf{d}$ 

```

point, and then the program is infeasible (line 6). Each call of `OrcaLineProgram` costs $\mathcal{O}(i)$; the total is $\mathcal{O}(m^2)$ in the worst case and, if the half-planes are processed in random order, $\mathcal{O}(m)$ in expectation (Proposition 13.12).

13.5.2 Infeasibility and the dense fallback

In a dense crowd the half-planes may have an empty intersection: every velocity in the disk violates at least one of them, which means that no velocity is safe for the full horizon τ with respect to every neighbor. This is not a failure of the algorithm but a fact about the situation, and the agent still has to choose a velocity. The standard remedy, proposed with ORCA and implemented in RVO2, is to relax all agent half-planes uniformly and choose the velocity that **minimizes the largest violation**,

$$\min_{\mathbf{v} \in D(\mathbf{0}, v_{\max})} \max_i \text{viol}_i(\mathbf{v}), \quad \text{viol}_i(\mathbf{v}) = -(\mathbf{v} - \mathbf{q}_i) \cdot \mathbf{n}_i, \quad (13.12)$$

while the half-planes of static obstacles remain hard. Relaxing every agent half-plane by the same amount δ and taking the smallest δ for which a velocity survives is exactly [Equation \(13.12\)](#); van den Berg et al. call the result the *safest possible velocity*, the one that penetrates the half-planes least [14]. It is a heuristic: it minimizes a penetration, not the time to the first collision. It also ignores $\mathbf{v}^{\text{pref}}$ entirely, because safety comes first. [Equation \(13.12\)](#) is a linear program in the three variables (v_x, v_y, δ) together with the speed disk, the same half-plane-and-disc structure as [Algorithm 13.2](#), and [Algorithm 13.3](#) solves it with the two-dimensional machinery of [Algorithm 13.2](#): it walks through the half-planes in order and, whenever one of them is violated more deeply than the current largest violation, finds the velocity that minimizes the violation of that half-plane subject to the constraint that no earlier half-plane is violated more. The constraint “ H_j is violated no more than H_i ” is itself a half-plane: $\text{viol}_j(\mathbf{v}) \leq \text{viol}_i(\mathbf{v})$ is linear in $\mathbf{v}$, its boundary passes through the intersection point of the two boundary lines, where both violations vanish, and it is the bisector of the two lines. Minimizing viol_i means moving as far as possible in direction $\mathbf{n}_i$, which is [Algorithm 13.2](#) with a linear objective instead of a distance, the variant that the implementation calls with `direction_opt`.

Algorithm 13.3. Dense fallback: minimize the largest violation when the half-planes do not intersect

Input: half-planes $H_1, \dots, H_m$, of which $H_1, \dots, H_k$ (static obstacles) are hard; the index $f > k$ of the half-plane at which [Algorithm 13.2](#) failed; the velocity $\mathbf{v}$ it returned, which satisfies $H_1, \dots, H_{f-1}$; $v_{\max}$

Output: a velocity in the disk that satisfies the hard half-planes and minimizes $\max_{i>k} \text{viol}_i(\mathbf{v})$

```

1 Function OrcaDenseFallback( $H_1, \dots, H_m, k, f, \mathbf{v}, v_{\max}$ ):
2    $\delta \leftarrow 0$  // largest violation so far
3   for  $i \leftarrow f$  to  $m$  do
4     if  $\text{viol}_i(\mathbf{v}) > \delta$  then
5        $\mathcal{P} \leftarrow (H_1, \dots, H_k)$  // hard half-planes stay as they are
6       foreach  $j \in \{k+1, \dots, i-1\}$  do
7         append to  $\mathcal{P}$  the half-plane  $\{\mathbf{v} : \text{viol}_j(\mathbf{v}) \leq \text{viol}_i(\mathbf{v})\}$ , bounded by the
          bisector of the boundary lines of  $H_i$  and  $H_j$ 
8        $\mathbf{v}' \leftarrow$  the point of  $\bigcap \mathcal{P} \cap D(\mathbf{0}, v_{\max})$  furthest in direction  $\mathbf{n}_i$ 
          (OrcaLinearProgram with a linear objective)
9       if  $\mathbf{v}'$  exists then  $\mathbf{v} \leftarrow \mathbf{v}'$ 
10       $\delta \leftarrow \text{viol}_i(\mathbf{v})$ 
11 return  $\mathbf{v}$ 

```

After the loop, δ is the smallest achievable maximum violation and $\mathbf{v}$ attains it. If even the hard half-planes and the disk have no common point, the agent is inside an obstacle and no velocity is safe; the implementation then returns the best velocity for the hard constraints it could satisfy and lets the higher layers ([Chapter 24](#)) react. In the simulations of this chapter the fallback is never needed ([Section 13.7](#)); it becomes essential in crowds of hundreds of agents at close spacing, which is the setting the method was designed for.

13.5.3 The simulation step

One time step for n agents puts the pieces together. Every agent builds its half-planes from the same snapshot of positions and velocities: first the hard half-planes of the static obstacles

Table 13.1. The ORCA construction of [Example 13.8](#), computed by `ch13_orca.py` (four decimals). The half-plane of A is $-0.1260(v_x - 0.9841) - 0.9920(v_y + 0.1250) \geq 0$.

Step	Quantity	Value
1	$\mathbf{p}_{\text{rel}}, R, \tau$	$(4, 0.5), 1, 4$
	$\ \mathbf{p}_{\text{rel}}\ , \varphi, \theta = \arcsin(R/\ \mathbf{p}_{\text{rel}}\)$	$4.0311, 7.13^\circ, 14.36^\circ$
	truncating disk: center $\mathbf{p}_{\text{rel}}/\tau$, radius R/τ	$(1, 0.125), 0.25$
2	$\mathbf{v}_{\text{rel}} = \mathbf{v}_A - \mathbf{v}_B$	$(2, 0)$
3	$\mathbf{w} = \mathbf{v}_{\text{rel}} - \mathbf{p}_{\text{rel}}/\tau$	$(1, -0.125)$
	$\mathbf{w} \cdot \mathbf{p}_{\text{rel}}$ (not < 0 : not the arc)	3.9375
	$\mathbf{p}_{\text{rel}} \times \mathbf{w}$ (< 0 : right leg)	-1
	$\ell = \sqrt{\ \mathbf{p}_{\text{rel}}\ ^2 - R^2}, \mathbf{d}_R$	$3.9051, (0.9920, -0.1260)$
	$\mathbf{q} = (\mathbf{v}_{\text{rel}} \cdot \mathbf{d}_R) \mathbf{d}_R$	$(1.9682, -0.2500)$
	$\mathbf{u} = \mathbf{q} - \mathbf{v}_{\text{rel}}, \ \mathbf{u}\ $	$(-0.0318, -0.2500), 0.2520$
	$\mathbf{n} = (d_{R,y}, -d_{R,x})$	$(-0.1260, -0.9920)$
4	A: point $\mathbf{v}_A + \frac{1}{2}\mathbf{u}$, normal $\mathbf{n}$	$(0.9841, -0.1250), (-0.1260, -0.9920)$
	B: point $\mathbf{v}_B - \frac{1}{2}\mathbf{u}$, normal $-\mathbf{n}$	$(-0.9841, 0.1250), (0.1260, 0.9920)$
LP	violation of $\mathbf{v}_A^{\text{pref}}$	0.1512
	$\ \mathbf{v}_A^{\text{new}} - \mathbf{v}_A\ $, the change of A's velocity	0.2350
	$\mathbf{v}_A^{\text{new}}, \mathbf{v}_B^{\text{new}}$	$(1.1809, -0.1500), (-1.1809, 0.1500)$
	new $\mathbf{v}_{\text{rel}}, \angle(\mathbf{v}_{\text{rel}}^{\text{new}}, \mathbf{p}_{\text{rel}})$	$(2.3618, -0.3000), -14.36^\circ = -\theta$
$\alpha = 1$	point $\mathbf{v}_A + \mathbf{u}$, violation of $\mathbf{v}_A^{\text{pref}}$	$(0.9682, -0.2500), 0.2772$
	$\mathbf{v}_A^{\text{new}}$ with full responsibility	$(1.1651, -0.2750)$

within sensing range, with share 1 and a shorter horizon τ_{obst} , then one half-plane per neighbor, with share $\frac{1}{2}$ if the neighbor runs ORCA and share 1 if it is an intruder. It then solves [Algorithm 13.2](#) for the velocity closest to $\mathbf{v}^{\text{pref}}$ and falls back to [Algorithm 13.3](#) when the program is infeasible. Only when every agent has chosen does the simulation move all of them by Δt times their new velocities (`simulate` in `ch13_orca.py`). The order of the half-planes matters in two places: the obstacle half-planes come first so that [Algorithm 13.3](#) can keep them hard, and a random order of the agent half-planes gives the expected linear running time.

13.6 A worked example

Example 13.8 (Two agents, one half-plane each). Agent A is at $\mathbf{p}_A = (0, 0)$ with velocity $\mathbf{v}_A = (1, 0)$, agent B at $\mathbf{p}_B = (4, 0.5)$ with velocity $\mathbf{v}_B = (-1, 0)$; both have radius 0.5 m, speed limit $v_{\text{max}} = 1.5$ m/s and preferred velocities $\mathbf{v}_A^{\text{pref}} = (1.2, 0)$ and $\mathbf{v}_B^{\text{pref}} = (-1.2, 0)$, slightly faster than they currently fly. The horizon is $\tau = 4$ s and the time step $\Delta t = 0.1$ s. Every number below is printed by `worked_example()` in `ch13_orca.py` and collected in [Table 13.1](#); [Figure 13.4](#) draws the same numbers.

Step 1. $\mathbf{p}_{\text{rel}} = (4, 0.5)$ has length 4.0311 and direction $\varphi = 7.13^\circ$; the cone half-angle is $\theta = \arcsin(1/4.0311) = 14.36^\circ$, so in the world frame the legs point at 21.49° (left) and -7.24° (right). From here on, directions are measured from the axis of the cone: for $\mathbf{v} \neq \mathbf{0}$ write $\angle(\mathbf{v}, \mathbf{p}_{\text{rel}}) = \text{atan2}(v_y, v_x) - \varphi$ for the signed angle from $\mathbf{p}_{\text{rel}}$ to $\mathbf{v}$, positive counterclockwise, so that the legs are at $\pm\theta$ and the cone is the set of directions with $|\angle(\mathbf{v}, \mathbf{p}_{\text{rel}})| < \theta$. The truncating disk has center $(1, 0.125)$ and radius 0.25. *Step 2.* $\mathbf{v}_{\text{rel}} = (2, 0)$ points along the x -axis, that is $\angle(\mathbf{v}_{\text{rel}}, \mathbf{p}_{\text{rel}}) = -7.13^\circ$, between the legs, and at distance 2

Table 13.2. Results of the circle scenario of Figure 13.6 (30 s, 300 steps). “Colliding pairs” counts the pairs whose disks overlapped at some step; “reversals” counts the steps at which the velocity change of agent 0 turned against the previous change, and “velocity variation” is the sum of its velocity changes over the run.

Method	colliding steps	colliding pairs	min. sep. (m), at t (s)	last arrival (s)	mean path length (m)	reversals agent 0	velocity var. (m/s)
no avoidance	33	28	0.010, 9.9	20.3	20.00	1	2.05
sampled VO	25	4	0.393, 11.0	23.6	20.45	62	37.82
sampled RVO	0	0	1.002, 14.2	23.5	20.54	33	15.13
ORCA	0	0	1.000, 14.8	26.2	20.29	2	3.37

it is far beyond the truncating disk: the agents collide in about 1.6 s, well within τ . *Step 3.* $\mathbf{w} = (1, -0.125)$ has $\mathbf{w} \cdot \mathbf{p}_{\text{rel}} = 3.9375 > 0$, so the arc is ruled out; $\mathbf{p}_{\text{rel}} \times \mathbf{w} = -1 < 0$ selects the right leg, whose direction is $\mathbf{d}_R = (0.9920, -0.1260)$. The foot of the perpendicular is $\mathbf{q} = (1.9682, -0.2500)$, hence $\mathbf{u} = (-0.0318, -0.2500)$ with $\|\mathbf{u}\| = 0.2520$ m/s, and the outward normal is $\mathbf{n} = (-0.1260, -0.9920)$: the cheapest way out of the cone is for the relative velocity to turn slightly downward, that is, for A to pass below B. *Step 4.* A’s half-plane has the point $\mathbf{v}_A + \frac{1}{2}\mathbf{u} = (0.9841, -0.1250)$ and the normal $\mathbf{n}$; B’s half-plane has the point $\mathbf{v}_B - \frac{1}{2}\mathbf{u} = (-0.9841, 0.1250)$ and the normal $-\mathbf{n}$.

The program. A’s preferred velocity (1.2, 0) violates its half-plane by 0.1512: it is inside the disk, so Algorithm 13.2 projects it onto the boundary line and returns $\mathbf{v}_A^{\text{new}} = (1.2, 0) + 0.1512\mathbf{n} = (1.1809, -0.1500)$; by symmetry B returns $(-1.1809, 0.1500)$. Each agent ends up 0.1512 m/s away from its preferred velocity (and 0.2350 m/s away from the velocity it currently flies; the half-plane only required $\frac{1}{2}\|\mathbf{u}\| = 0.1260$ m/s along $\mathbf{n}$), and the new relative velocity (2.3618, -0.3000) has $\angle(\mathbf{v}_{\text{rel}}^{\text{new}}, \mathbf{p}_{\text{rel}}) = -14.36^\circ = -\theta$: it lies exactly on the right leg, so the disks will graze and not penetrate. This is the pairwise guarantee of Theorem 13.10 in numbers. *Full responsibility.* If B were an intruder that keeps flying $(-1, 0)$, A would use $\alpha = 1$: the point moves to (0.9682, -0.2500), the violation of the preferred velocity doubles to 0.2772, and A alone chooses (1.1651, -0.2750), whose relative velocity against the unchanged $\mathbf{v}_B$ again lies on the leg. Had A used $\alpha = \frac{1}{2}$ against a non-reacting B, the relative velocity would have been (2.1809, -0.1500), at -11.06° from $\mathbf{p}_{\text{rel}}$ (-3.93° in the world frame), inside the cone of half-angle 14.36° .

13.7 Eight agents on a circle

The classic test of a reciprocal method puts n agents on a circle and sends each to the antipodal point, so that all straight paths cross at the center at the same time. Figure 13.6 shows eight agents of radius 0.5 m on a circle of radius 10 m, cruise speed 1 m/s, speed limit 1.5 m/s, $\tau = 5$ s and $\Delta t = 0.1$ s, simulated for 30 s with `simulate()` of `ch13_orca.py` in four modes: no avoidance; the sampled velocity obstacle rule (the velocity closest to $\mathbf{v}^{\text{pref}}$ among 721 candidates on a polar grid whose time to collision with every neighbor is at least τ); sampled RVO (the same with the reciprocal test $2\mathbf{v} - \mathbf{v}_A$); and ORCA. The preferred velocities are perturbed by fixed offsets of 0.02 m/s so that the instance is not perfectly symmetric, as no real instance is. Table 13.2 gives the numbers, all printed by `gen_ch13_circle.py`.

Three things to notice. Without avoidance every pair collides at the center at $t \approx 10$ s, as it must. The velocity obstacle rule avoids most of that, but the dance of Proposition 13.2 now involves seven neighbors at once: agent 0 reverses its velocity change 62 times and its velocity changes add up to 37.8 m/s over the run, and the stale-velocity problem is severe enough that four pairs still overlap, with a smallest separation of 0.39 m against a threshold of 1 m.

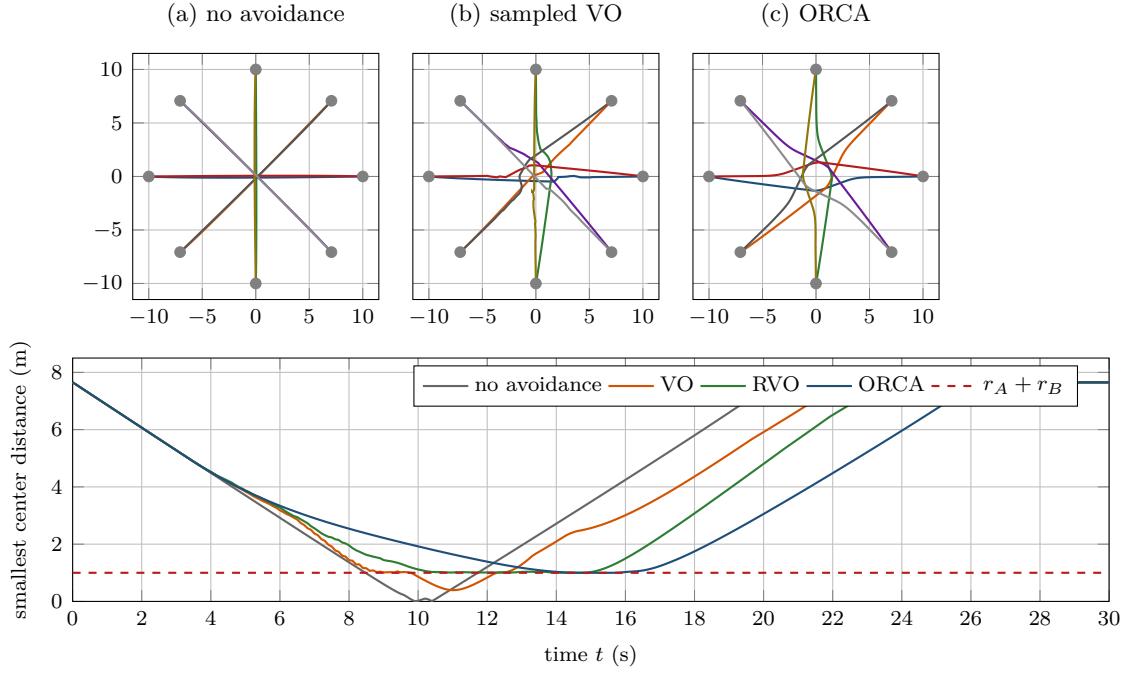


Figure 13.6. Eight agents swapping antipodal positions on a circle. Top: the traces (circles mark the starts) with no avoidance, with the sampled velocity obstacle rule and with ORCA. Bottom: the smallest center distance over all pairs against time, for all four methods (the lower panel also shows sampled RVO, which gets no trace panel); the dashed line is the collision threshold $r_A + r_B = 1$ m. Without avoidance all agents meet at the center; the VO agents zig-zag and four pairs still collide; the ORCA agents spiral gently around the center and the smallest distance touches 1.000 m without ever going below it.

RVO stays collision-free here, but it still reverses 33 times: the reciprocal dance among many agents, which happens to be harmless in this instance and is not guaranteed to be. ORCA is collision-free with two reversals, a velocity variation of 3.4 m/s and paths only 1.5 % longer than the straight line; the linear program was feasible at every one of the 2 400 agent-steps. Its price is time: the agents slow down near the center and the last one arrives at 26.2 s instead of 20 s, because a half-plane never asks an agent to speed up past $v_{\max}$ and the projections of the preferred velocities point partly backward while the cluster passes. That slowing-down is the behavior the local safety layer of Chapter 24 wants: it buys time for the global planner instead of inventing a detour.

13.8 Properties: what ORCA guarantees and what it does not

13.8.1 The pairwise guarantee

Lemma 13.9 (The truncated velocity obstacle is convex). *For $\|\mathbf{p}_{\text{rel}}\| > R$ the set $VO_{A|B}^\tau$ is open and convex, and at every boundary point $\mathbf{q}$ with outward unit normal $\mathbf{n}$ it lies entirely on the inner side of the tangent line: $VO_{A|B}^\tau \subseteq \{\mathbf{v} : (\mathbf{v} - \mathbf{q}) \cdot \mathbf{n} < 0\}$.*

Proof. Openness follows from Equation (13.3), a union of open disks. For convexity let $\mathbf{v}_1, \mathbf{v}_2 \in VO_{A|B}^\tau$ with times $t_1, t_2 \in (0, \tau]$ such that $t_i \mathbf{v}_i \in D(\mathbf{p}_{\text{rel}}, R)$, let $\lambda \in [0, 1]$ and $\mathbf{v} = \lambda \mathbf{v}_1 + (1 - \lambda) \mathbf{v}_2$. Define t by $1/t = \lambda/t_1 + (1 - \lambda)/t_2$; then $t \leq \max(t_1, t_2) \leq \tau$ and

$$t \mathbf{v} = \frac{t\lambda}{t_1} (t_1 \mathbf{v}_1) + \frac{t(1-\lambda)}{t_2} (t_2 \mathbf{v}_2)$$

is a convex combination of two points of the disk, because the two coefficients are non-negative and sum to one. Hence $t\mathbf{v}$ lies in the disk and $\mathbf{v} \in VO_{A|B}^\tau$. The boundary of a convex set with a tangent line at $\mathbf{q}$ is supported by that line, and the boundary of $VO_{A|B}^\tau$ has a tangent line everywhere: on the arc and on the legs it is smooth, and at the two points where they meet the leg is tangent to the circle. Openness turns the weak inequality of the supporting line into a strict one. $\square$

Theorem 13.10 (Pairwise collision avoidance). *Let agents A and B with $\|\mathbf{p}_{\text{rel}}\| > R$ know each other's positions, radii and current velocities exactly, and let both build their half-planes with the same horizon τ from the same current velocities. If $\mathbf{v}_A^{\text{new}} \in \text{ORCA}_{A|B}^\tau$ and $\mathbf{v}_B^{\text{new}} \in \text{ORCA}_{B|A}^\tau$, then $\mathbf{v}_A^{\text{new}} - \mathbf{v}_B^{\text{new}} \notin VO_{A|B}^\tau$: flying these velocities, the disks of A and B do not overlap during the next τ seconds.*

Proof. By Equations (13.7) and (13.8), $(\mathbf{v}_A^{\text{new}} - \mathbf{v}_A - \frac{1}{2}\mathbf{u}) \cdot \mathbf{n} \geq 0$ and $(\mathbf{v}_B^{\text{new}} - \mathbf{v}_B + \frac{1}{2}\mathbf{u}) \cdot (-\mathbf{n}) \geq 0$. Adding the two inequalities gives

$$(\mathbf{v}_A^{\text{new}} - \mathbf{v}_B^{\text{new}} - (\mathbf{v}_A - \mathbf{v}_B) - \mathbf{u}) \cdot \mathbf{n} = (\mathbf{v}_{\text{rel}}^{\text{new}} - \mathbf{q}) \cdot \mathbf{n} \geq 0,$$

where $\mathbf{q} = \mathbf{v}_{\text{rel}} + \mathbf{u}$ is the boundary point of Step 3 and $\mathbf{n}$ its outward normal. By Lemma 13.9 every point of $VO_{A|B}^\tau$ satisfies the opposite strict inequality, so $\mathbf{v}_{\text{rel}}^{\text{new}} \notin VO_{A|B}^\tau$, and by Definition 13.1 the disks do not overlap for $t \in (0, \tau]$. $\square$

The proof uses nothing about how the two agents chose their velocities inside their half-planes, which is what makes the guarantee compose. The same computation with shares α_A, α_B produces $(\alpha_A + \alpha_B)\mathbf{u}$ in place of $\mathbf{u}$, and the conclusion holds exactly when $\alpha_A + \alpha_B = 1$ (Exercise 13.4); with $\alpha_B = 0$ the agent B contributes nothing and must therefore keep the velocity $\mathbf{v}_B$ that A used, which is the non-cooperative case of Section 13.4.1.

Corollary 13.11 (Collision-free motion of n agents). *Let n agents run the step of Section 13.5.3 with the same horizon τ from the same snapshot, with share $\frac{1}{2}$ toward each other and share 1 toward obstacles and intruders that keep their assumed velocities. If every agent's program Equation (13.11) is feasible, then no two agents and no agent and obstacle overlap during the next τ seconds. If the step is repeated every $\Delta t \leq \tau$ and the programs stay feasible, the motion is collision-free for as long as this holds.*

Proof. A feasible program returns a velocity in every half-plane of the agent (Proposition 13.12), so Theorem 13.10 applies to every pair of agents and its non-cooperative variant to every agent-obstacle pair. Each guarantee covers $\tau \geq \Delta t$ seconds, hence the whole step, and the next step starts from the new snapshot. $\square$

13.8.2 The program

Proposition 13.12 (The per-agent program). *If the feasible set of Equation (13.11) is non-empty, the program has a unique solution and Algorithm 13.2 returns it after $\mathcal{O}(m^2)$ operations in the worst case; if the half-planes are processed in uniformly random order, the expected number of operations is $\mathcal{O}(m)$. If the feasible set is empty, the algorithm reports the smallest index i such that $H_1 \cap \dots \cap H_i \cap D(\mathbf{0}, v_{\text{max}}) = \emptyset$.*

Proof sketch. The feasible set is closed and convex and the objective is strictly convex, which gives existence and uniqueness. Correctness is the invariant of the walkthrough: the optimum for $H_1, \dots, H_i$ equals the optimum for $H_1, \dots, H_{i-1}$ if the latter satisfies H_i , and lies on the boundary line of H_i otherwise, where `OrcaLineProgram` finds it exactly by a one-dimensional minimization over an interval. For the running time, iteration i costs $\mathcal{O}(1)$ if H_i is satisfied and $\mathcal{O}(i)$ otherwise. Backward analysis [17, 148]: the optimum for $H_1, \dots, H_i$ is determined by at most two of the half-planes (its boundary point lies on at most two boundary lines, or on one

line and the circle), so if the order is random the probability that H_i is one of the half-planes that determine it, which is the only case in which H_i can be violated by the optimum for the first $i - 1$, is at most $2/i$. The expected cost of iteration i is $\mathcal{O}(1) + \mathcal{O}(i) \cdot 2/i = \mathcal{O}(1)$, and the total is $\mathcal{O}(m)$. $\square$

With k neighbors per agent, one step of [Section 13.5.3](#) costs $\mathcal{O}(k)$ per agent for the half-planes and the program, plus the neighbor query, $\mathcal{O}(\log n + k)$ with a kd-tree, so $\mathcal{O}(n(\log n + k))$ per step for the whole swarm. The RVO2 library simulates thousands of agents in real time on one processor core.

13.8.3 What ORCA does not guarantee

The guarantee of [Corollary 13.11](#) is conditional, and every condition fails somewhere in a real swarm; the honest summary is that ORCA is a *safety filter* for the next τ seconds and nothing more. The half-planes may not intersect, and the fallback of [Algorithm 13.3](#) then returns the least-penetrating velocity, which does not exclude a collision. [Theorem 13.10](#) also assumes $\|\mathbf{p}_{\text{rel}}\| > R$: once two disks already overlap, the overlap branch of [Lemma 13.6](#) carries no τ -guarantee at all and only promises that the pair separates after one time step Δt . Both agents must use the same $\mathbf{p}_{\text{rel}}$, R , τ and current velocities; sensing noise, latency and different horizons make the two half-planes slightly inconsistent, which the radius margin of [Section 13.10](#) absorbs, while the intruder case absorbs the large inconsistencies by taking full responsibility. The agents are assumed to fly the new velocity immediately, whereas a drone tracks it with a delay and an error ([Chapters 2 and 14](#)); again the margin pays for it. Finally, ORCA chooses the closest permitted velocity to the preferred one, step by step, with no memory and no lookahead beyond τ , so it can stop an agent forever.

Proposition 13.13 (ORCA is not a global planner). *There are instances with a collision-free solution in which agents running the step of [Section 13.5.3](#) never reach their goals.*

Proof by example. Two agents of radius r_0 face each other in a corridor of width $3r_0$ with a side bay near one end, so that they can pass only if one agent enters the bay and waits. Both preferred velocities point along the corridor. The wall half-planes bound the lateral velocity by a fixed multiple of the forward velocity ([Exercise 13.8](#) computes the factor 0.75 for one geometry), so a lateral escape is available only while the agent still moves forward. As the two agents approach head-on, the arc case of [Lemma 13.6](#) applies: with $\hat{\mathbf{p}} = \mathbf{p}_{\text{rel}} / \|\mathbf{p}_{\text{rel}}\|$ and the gap $d = \|\mathbf{p}_{\text{rel}}\| - R$ it gives $\mathbf{u} = (d/\tau) \hat{\mathbf{p}} - \mathbf{v}_{\text{rel}}$ and $\mathbf{n} = -\hat{\mathbf{p}}$, and the half-plane through $\mathbf{v}_A + \frac{1}{2}\mathbf{u}$ therefore permits each agent to approach the other at no more than $d/(2\tau)$, whatever its current speed. The gap obeys $d_{k+1} = d_k(1 - \Delta t/\tau)$ and both speeds decay geometrically to zero. Backing into the bay is further from $\mathbf{v}^{\text{pref}}$ than slowing down and no half-plane ever asks for it, so neither agent ever reaches its goal, although a collision-free joint plan exists. $\square$

Perfectly symmetric configurations produce a milder version of the same effect, agents that slow down and hover instead of passing, which is why implementations break symmetry by a small random perturbation of the preferred velocities, as [Section 13.7](#) did. The remedy for deadlocks is not inside ORCA: it is the global planner of the hybrid architecture, which sequences the corridor in the nominal paths, and the replanning layer, which detects that an agent has been stopped for too long and plans around ([Chapter 24](#)).

13.9 Variants and extensions

HRVO. The hybrid reciprocal velocity obstacle [\[154\]](#) attacks the side-choice problem of RVO directly: for the side on which the agent is already passing it uses the RVO leg, for the other side the VO leg, so that crossing over to the wrong side requires the full change instead of

half. It reduces the reciprocal dance markedly on real differential-drive robots, but it keeps the non-convex cone and therefore the sampled velocity choice and the lack of an n -agent guarantee.

Static obstacles. A static disk is an agent with $\mathbf{v}_B = \mathbf{0}$ and share 1. For polygonal obstacles the velocity obstacle of a line segment is the union of the velocity obstacles of its points, whose boundary consists of two legs tangent to the disks around the endpoints and, after truncation, of a stadium-shaped front; RVO2 builds one half-plane per visible segment with full responsibility and a separate, shorter horizon τ_{obst} , and places these half-planes first so that they stay hard in the dense fallback.

Non-holonomic robots. A differential-drive robot cannot fly an arbitrary velocity instantly. Alonso-Mora et al. keep the ORCA half-planes and restrict the velocity choice to the set of holonomic velocities that the robot can track within a bounded error η over the next interval, a non-convex set that they approximate by convex polygons, while inflating the radius by η [2]. The same recipe applies to a multirotor with a velocity controller: measure the tracking error, inflate the radius by it, and clip the velocity change per step to what the controller delivers (Chapter 14 treats the acceleration limits explicitly).

Uncertainty. An intruder's future velocity is a prediction with a covariance (Chapters 18 and 20). Inflating r_B by a multiple of the predicted position standard deviation at the horizon turns Equation (13.9) into a conservative constraint, which is what the drone system of this book does. Zhu and Alonso-Mora formulate the same idea as a chance constraint, a collision probability below a chosen level, inside a model predictive controller for multirotors [187]; Chapter 21 introduces that controller, and the linearized chance constraint has exactly the shape of a half-plane with an uncertainty-dependent margin.

13.10 Implementation notes

The file `ch13_orca.py` implements everything in this chapter with plain tuples and a few NumPy calls; Listings 13.1 and 13.2 show the functions that carry the mathematics, where the code writes the combined radius r for what the text calls R , and the line program of Algorithm 13.2 is `_lp_on_line` in the same file. The geometry helpers (`dot`, `cross`, `ray_circle_intersection`, ...) are copied from `ch02_toolbox.py` so that the file stands alone.

Listing 13.1. The case analysis of Lemma 13.6 (from `code/ch13_orca.py`, docstring omitted). The function returns `u`, `n` and the name of the case.

```

1 def vo_closest_boundary_point(p, v_rel, r, tau, dt):
2     dist2 = dot(p, p)
3     r2 = r * r
4     if dist2 > r2:
5         c = scale(p, 1.0 / tau)           # center of the truncating disk
6         rho = r / tau                     # its radius
7         w = sub(v_rel, c)                 # from that center to v_rel
8         w2 = dot(w, w)
9         wp = dot(w, p)
10        if wp < 0.0 and wp * wp > r2 * w2:
11            # angle(w, -p) < arccos(r / |p|): the closest point is on the arc
12            w_len = math.sqrt(w2)
13            n = scale(w, 1.0 / w_len)
14            return scale(n, rho - w_len), n, "disk"
```

```

15     leg = math.sqrt(dist2 - r2)          # length of a tangent from 0
16     if cross(p, w) > 0.0:
17         d = ((p[0] * leg - p[1] * r) / dist2,
18              (p[0] * r + p[1] * leg) / dist2)  # unit vector, left leg
19         n = (-d[1], d[0])                  # outward: d rotated +90
20         case = "left leg"
21     else:
22         d = ((p[0] * leg + p[1] * r) / dist2,
23              (-p[0] * r + p[1] * leg) / dist2) # unit vector, right leg
24         n = (d[1], -d[0])                  # outward: d rotated -90
25         case = "right leg"
26     q = scale(d, dot(v_rel, d))            # projection onto the leg
27     return sub(q, v_rel), n, case
28     # the disks already overlap: leave the disk of center p/dt, radius r/dt
29     c = scale(p, 1.0 / dt)
30     rho = r / dt
31     w = sub(v_rel, c)
32     w_len = norm(w)
33     if w_len > EPS:
34         n = scale(w, 1.0 / w_len)
35     elif dist2 > 0.0:
36         n = unit(scale(p, -1.0))
37     else:
38         n = (1.0, 0.0)
39     return scale(n, rho - w_len), n, "overlap"

```

Listing 13.2. The half-plane (Algorithm 13.1) and the incremental program (Algorithm 13.2); `HalfPlane.violation(v)` is $-(v - q) \cdot n$ and direction is n rotated by -90° .

```

1  def orca_half_plane(agent, other, tau, dt=0.1, reciprocal=True):
2      p = sub(other.position, agent.position)
3      v_rel = sub(agent.velocity, other.velocity)
4      r = agent.radius + other.radius
5      u, n, case = vo_closest_boundary_point(p, v_rel, r, tau, dt)
6      share = 0.5 if reciprocal else 1.0
7      return OrcaLine(n, add(agent.velocity, scale(u, share)), u, case)
8
9
10 def lp_incremental(lines, v_max, v_opt, direction_opt=False):
11     if direction_opt:
12         v = scale(v_opt, v_max)
13     elif dot(v_opt, v_opt) > v_max * v_max:
14         v = scale(v_opt, v_max / norm(v_opt))
15     else:
16         v = v_opt
17     for i, line in enumerate(lines):
18         if line.violation(v) > 0.0:
19             new_v = _lp_on_line(lines, i, v_max, v_opt, direction_opt)
20             if new_v is None:
21                 return v, i
22             v = new_v
23     return v, len(lines)

```

Libraries. The reference implementation is the RVO2 library of the ORCA authors (C++, with ports to several languages, and RVO2-3D for half-spaces). Its per-agent parameters map

directly onto this chapter: `radius`, `maxSpeed`, `timeHorizon` (τ), `timeHorizonObst` (τ_{obst}), `neighborDist` and `maxNeighbors` (the sensing range and the cap on the number of half-planes), and the global `timeStep` (Δt). Reading its `Agent.cpp` after this chapter is a good exercise: [Algorithms 13.1 to 13.3](#) are its `computeNewVelocity`, `linearProgram2` with `linearProgram1`, and `linearProgram3`.

Neighbor queries. Every agent needs its neighbors within the sensing range, and a linear scan costs $\mathcal{O}(n)$ per agent. RVO2 rebuilds a kd-tree over the agent positions at every step and queries it; the same structure serves the nearest-neighbor searches of [Chapter 16](#). Cap the number of neighbors by distance: the closest 10 to 20 agents determine the program, the rest add half-planes that are never active.

Horizon and time step. The guarantee covers τ seconds and the velocity is recomputed every Δt , so τ must exceed Δt by a comfortable factor, typically $\tau = 10$ to $50 \Delta t$. A larger τ makes agents react earlier and more gently, but it also makes the half-planes more restrictive, since a half-plane forbids velocities that lead to a collision anywhere within τ , and it makes infeasibility and hovering more likely in crowds. Obstacles get a shorter τ_{obst} , because a wall is not to move out of the way and an agent that fears it from far away hugs the middle of every corridor.

The preferred velocity. $\mathbf{v}^{\text{pref}}$ is the interface to the global planner: the vector toward the next waypoint of the nominal path at cruise speed, scaled down near the goal so that the agent does not overshoot in the last step (`preferred_velocity` in the code). It is never the raw direction to the final goal in a cluttered map, which would make ORCA a potential-field method with all its local minima ([Chapter 15](#)).

Radius inflation. The radius that goes into [Algorithm 13.1](#) is not the physical radius. Add the velocity tracking error of the controller over one step, the localization error, half of the position uncertainty of the neighbor and, for an intruder, the predicted position uncertainty at the horizon. The margin also converts the grazing contact that the guarantee permits into a real clearance, and it absorbs the asynchrony between agents that do not update at exactly the same instant.

Mixing reciprocal and full responsibility

The shares must sum to one *per pair*, and both sides must agree. Two common bugs: treating an intruder that does not react as a reciprocal neighbor ($\alpha = \frac{1}{2}$ on one side, 0 on the other), after which the relative velocity moves only halfway out of the cone and the collision course remains, as in the last lines of [Table 13.1](#); and treating cooperative neighbors with full responsibility on both sides ($\alpha = 1$ and 1), which is safe but is exactly the velocity obstacle rule of [Proposition 13.2](#), and the dance returns. Decide the share from the *type* of the neighbor (swarm mate, intruder, obstacle), never from its current behavior.

The normal points the wrong way, or the disk is forgotten

$\mathbf{n}$ must be the *outward* normal of VO^T at the closest point in every case: away from the truncating disk's center on the arc, and the leg direction rotated by $+90^\circ$ on the left leg and by -90° on the right leg. A sign error in one case gives a half-plane on the wrong side, and the program cheerfully returns a velocity that leads into the cone. Test every case against [Lemma 13.6](#) the way the self-test does: the point $\mathbf{v}_{\text{rel}} + \mathbf{u}$ must lie on the

boundary, and $\mathbf{v}_{\text{rel}} + \mathbf{u} \pm \varepsilon \mathbf{n}$ must lie outside and inside. The second bug is to solve the program without the disk: the line program then has an unbounded interval, the returned velocity can exceed v_{max} , and the agent cannot fly it, so the relative velocity that the guarantee is about is never realized.

Horizon, time step and agents that must hold a formation

With $\tau < \Delta t$ the guarantee expires before the next update and agents can overlap within one step; with τ far larger than the time an agent needs to cross the sensing range, every visible neighbor produces an active half-plane and the swarm hovers or the program turns infeasible. Choose τ from the closing speed and the sensing range, and keep τ_{obst} shorter. Finally, do not run ORCA between the members of a tight formation with the formation spacing as their separation: the half-planes are permanently active, they fight the formation controller of [Chapter 23](#), and the formation dissolves into hovering. Keep formation mates apart by the controller, use ORCA between formations and against intruders, or give the leader the right of way with shares $(1, 0)$.

13.11 Where this fits in the drone system

In the drone system

ORCA is the local safety layer of the four-layer architecture of [Chapter 24](#), the Week 6 topic of the study plan ([Appendix A](#)). It receives from the global planner (conflict-based search CBS, or its bounded-suboptimal variant ECBS, [Chapters 9 and 10](#)) the nominal time-indexed path of each drone and turns it into the preferred velocity: toward the next waypoint at cruise speed. While no predicted conflict is inside the safety horizon, the half-planes are inactive and the drone flies its nominal path exactly. When the prediction layer ([Chapters 18 and 20](#)) reports an intruder whose predicted trajectory enters the safety horizon, the trigger of [Chapter 24](#) switches the local layer on: the swarm mates within sensing range become reciprocal half-planes with share $\frac{1}{2}$, and the intruder becomes a half-plane with *full* responsibility, built from its predicted velocity and a radius inflated by the predicted position uncertainty at the horizon. The layer delivers one velocity command per step to the flight controller and reports two things upward: whether its program was feasible, and how far the drone has drifted from the nominal path. When the conflict clears, the preferred velocity points at a waypoint further along the nominal path and ORCA reconnects to it by itself. When the drift is too large, the program has been infeasible or the drone has been stopped for longer than a threshold, the replanning layer (D^* Lite or A^* , [Chapter 5](#)) takes over, because ORCA cannot get out of a deadlock on its own ([Proposition 13.13](#)). The guarantee that survives the integration is the pairwise one of [Theorem 13.10](#), per step and per horizon; everything beyond it, reaching the goal, keeping the formation, staying within communication range, is the job of the other layers.

13.12 Summary

Chapter summary

- Two agents that avoid each other with plain velocity obstacles react to each other's stale velocities and oscillate with period two: both swerve, both return, both swerve.

- RVO makes each agent take half of the velocity change, $RVO_{A|B}(\mathbf{v}_B, \mathbf{v}_A) = \left\{ \mathbf{v} : 2\mathbf{v} - \mathbf{v}_A \in \mathbf{v}_B + VO_{A|B} \right\}$, the cone with its apex moved to $\frac{1}{2}(\mathbf{v}_A + \mathbf{v}_B)$. For two agents that pass on the same side this removes the oscillation; with several agents or disagreeing sides the reciprocal dance returns, and there is no guarantee.
- ORCA builds, for each pair and horizon τ , the truncated cone $VO_{A|B}^\tau$, takes the current relative velocity, computes the smallest change $\mathbf{u}$ to the boundary (arc, left leg, right leg, or the one-step disk when already overlapping) with the outward normal $\mathbf{n}$, and permits A the half-plane $\left\{ \mathbf{v} : (\mathbf{v} - (\mathbf{v}_A + \frac{1}{2}\mathbf{u})) \cdot \mathbf{n} \geq 0 \right\}$; B gets the mirror half-plane with $-\mathbf{u}$ and $-\mathbf{n}$.
- Each agent chooses the velocity closest to its preferred one inside all its half-planes and the disk $\|\mathbf{v}\| \leq v_{\max}$: a two-dimensional convex program solved exactly by the incremental linear program in expected linear time. If the intersection is empty, the dense fallback minimizes the largest violation.
- Against an obstacle or an intruder that does not react, the agent takes full responsibility: the half-plane through $\mathbf{v}_A + \mathbf{u}$. Shares must sum to one per pair. In three dimensions the same construction in the plane of $\mathbf{p}_{\text{rel}}$ and $\mathbf{v}_{\text{rel}}$ gives a half-space.
- If all agents use the same horizon from the same snapshot and every program is feasible, no pair overlaps within τ . ORCA is not a global planner: it has no lookahead, no memory and no completeness, and it deadlocks in corridors and symmetric configurations.
- In the circle scenario, no avoidance collides for all 28 pairs, the velocity obstacle rule still collides for four and reverses its velocity change 62 times, RVO is collision-free but jittery, and ORCA is collision-free with two reversals and paths 1.5 % longer than straight lines.

Further reading. Velocity obstacles are due to Fiorini and Shiller [45]; the reciprocal velocity obstacle to van den Berg, Lin and Manocha [15], and ORCA, with the maximality argument, the dense fallback and the RVO2 library, to van den Berg, Guy, Lin and Manocha [14]. The hybrid reciprocal velocity obstacle is by Snape, van den Berg, Guy and Manocha [154], the non-holonomic extension by Alonso-Mora et al. [2], and the chance-constrained formulation for multirotors by Zhu and Alonso-Mora [187]. The randomized incremental linear program is Seidel’s algorithm [148]; the textbook treatment, including the three-dimensional version and the backward analysis, is Chapter 4 of de Berg, Cheong, van Kreveld and Overmars [17]. LaValle’s book [99] places velocity-space planning among moving obstacles in the wider context of planning in time-varying environments.

13.13 Exercises

Exercise 13.1 (★). Starting from Definition 13.7, show in detail that the half-plane of B with respect to A has the point $\mathbf{v}_B - \frac{1}{2}\mathbf{u}$ and the normal $-\mathbf{n}$: write down $VO_{B|A}^\tau$, its closest boundary point to $\mathbf{v}_B - \mathbf{v}_A$, and the corresponding change and normal. Then add the two half-plane inequalities and show that the sum says that the new relative velocity lies on the outer side of the tangent line at $\mathbf{q}$.

Exercise 13.2 (★). Repeat the worked example of Section 13.6 with B at $\mathbf{p}_B = (4, -0.5)$ instead of $(4, 0.5)$, everything else unchanged. Which leg is closest? Give $\mathbf{u}$, $\mathbf{n}$, the point and normal of A ’s half-plane and $\mathbf{v}_A^{\text{new}}$, and explain why the numbers are the mirror images of Table 13.1. Check with `orca_half_plane`.

Exercise 13.3 (★★). With $\mathbf{p}_{\text{rel}} = (4, 0.5)$, $R = 1$ and $\tau = 4$ as in the worked example, take $\mathbf{v}_{\text{rel}} = (0.4, 0.05)$. (a) Show that this relative velocity is outside $VO_{A|B}^\tau$ and compute the time at which the disks would touch. (b) Apply [Lemma 13.6](#): which case applies, and what are $\mathbf{q}$, $\mathbf{u}$ and $\mathbf{n}$? What does the sign of $\mathbf{u} \cdot \mathbf{n}$ tell you? (c) Prove the claim in the proof sketch of [Lemma 13.6](#) that in the leg case the foot of the perpendicular lies beyond the tangent point, that is, on the boundary of $VO_{A|B}^\tau$ and not on the extension of the leg toward the apex.

Exercise 13.4 (★★). Let A use the share α_A and B the share α_B in [Algorithm 13.1](#). (a) Redo the proof of [Theorem 13.10](#) and show that the new relative velocity is guaranteed to lie outside $VO_{A|B}^\tau$ exactly when $\alpha_A + \alpha_B = 1$; explain why both $\alpha_A + \alpha_B > 1$ and < 1 can fail, using the sign of $\mathbf{u} \cdot \mathbf{n}$. (b) In the worked example, let B be an intruder that keeps $\mathbf{v}_B = (-1, 0)$ while A uses $\alpha_A = \frac{1}{2}$. Compute the new relative velocity and its angle from $\mathbf{p}_{\text{rel}}$ and conclude that the pair is still on a collision course. (c) What happens if two cooperative agents both use $\alpha = 1$? Relate the answer to [Proposition 13.2](#).

Exercise 13.5 (★★). Prove [Proposition 13.4](#) for the truncated cone: show that $\{\mathbf{v} : 2\mathbf{v} - \mathbf{v}_A \in \mathbf{v}_B + VO_{A|B}^\tau\} = \frac{1}{2}(\mathbf{v}_A + \mathbf{v}_B) + VO_{A|B}^{2\tau}$. Why does the horizon double, and what does this mean for an implementation that wants RVO agents to react to collisions within τ seconds?

Exercise 13.6 (★★). Take the three half-planes of [Figure 13.5](#), H_1 through $(0.9, -0.1)$ with normal $(-0.2, -0.98)$, H_2 through $(-0.4, 0.2)$ with normal $(0.94, 0.34)$, H_3 through $(0.5, -0.9)$ with normal $(-0.5, 0.866)$, with $v_{\text{max}} = 1.5$ and $\mathbf{v}^{\text{pref}} = (1.2, 0.3)$. (a) Run [Algorithm 13.2](#) by hand in the order H_1, H_2, H_3 and in the order H_3, H_2, H_1 ; report the calls to `OrcaLineProgram` and the result. (b) Prove the claim of the walkthrough: if the optimum for the first $i - 1$ half-planes violates H_i , the optimum for the first i half-planes lies on the boundary line of H_i . (c) Now let $\mathbf{v}^{\text{pref}} = (1.6, 0.4)$ and show where the disk enters the computation.

Exercise 13.7 (★★). Construct three half-planes whose intersection with the disk $\|\mathbf{v}\| \leq 1.5$ is empty, for instance three lines forming a triangle that lies outside the disk with the permitted sides facing away from it. (a) Solve [Equation \(13.12\)](#) for your instance geometrically: the solution equalizes the violations of the active half-planes. (b) Walk through [Algorithm 13.3](#) and identify the bisector half-planes it constructs. (c) Explain why the result does not depend on $\mathbf{v}^{\text{pref}}$ and whether that is desirable for a drone.

Exercise 13.8 (★★). Two agents of radius 0.5 face each other in a corridor of width 1.5 along the x -axis, walls at $y = \pm 0.75$ represented by disks of radius 0.5 spaced 0.2 apart with their centers at $y = \pm 1.25$. (a) Write down the half-planes that the two nearest wall disks induce on an agent flying $(1, 0)$ at $y = 0$ with $\tau_{\text{obst}} = 2$ and show that together they bound $|v_y|$ by a fixed multiple of v_x that you compute. (b) Show that in the head-on arc case each agent may close the gap $d = \|\mathbf{p}_{\text{rel}}\| - R$ at no more than $d/(2\tau)$, derive $d_{k+1} = d_k(1 - \Delta t/\tau)$ for the gap after one step of [Section 13.5.3](#), and conclude that the configuration converges to a stalled state instead of reaching it in finitely many steps. (c) Propose two remedies, one inside the local layer (a share or priority rule) and one in the architecture of [Chapter 24](#), and discuss what each guarantees. Simulate the instance with `ch13_orca.py` if you want to see the stall.

Exercise 13.9 (★★★ Coding). This is the coding exercise of Week 6. Build a 2D simulation with two groups of four agents whose straight paths cross at right angles (`crossing_scenario` in `ch13_orca.py` gives the instance) and compare no avoidance, the velocity obstacle rule and ORCA. Report for each the number of colliding pairs, the smallest separation, the last arrival time, the mean path length and the number of velocity reversals, as in [Table 13.2](#); plot the traces and the smallest separation over time as in [Figure 13.6](#). Then vary $\tau \in \{2, 5, 10\}$ and the group spacing, and describe when the ORCA program becomes infeasible and what the dense fallback does then. Finally add an intruder that flies straight through the crossing

without avoiding, treat it with full responsibility, and check that no swarm agent collides with it.

Exercise 13.10 (★★★ Coding). Use `orca_half_space_3d` to build the half-spaces of six drones that start on a sphere of radius 10 and fly to their antipodal points, solve the three-dimensional program by sampling velocities in the ball $\|\mathbf{v}\| \leq v_{\max}$ (or by the recursive incremental algorithm), and verify that the smallest center distance never drops below $r_A + r_B$.

The Dynamic Window Approach

Learning objectives

- Explain what the dynamic window approach (DWA) computes: the best of the velocity commands the robot can reach in the next control interval, judged by short simulated rollouts.
- Construct the three velocity sets of DWA (possible, admissible, dynamic window) for a differential-drive robot and for a velocity-controlled drone, and derive the braking condition $v \leq \sqrt{2} \text{dist } a_b$ together with its discrete-time refinement.
- Evaluate the heading, clearance and velocity terms of the objective by hand, explain why they must be normalized, and predict what happens when each weight dominates.
- Prove that an admissible command can always be followed by a safe one, and show by example that DWA is a local method that can get stuck in a U-shaped trap and can oscillate.
- Implement DWA in Python for a drone in a continuous 2D or 3D world, connect its heading term to a global path, and create and repair its failure cases.

14.1 Why this matters for a drone swarm

The local layer of the hybrid architecture in [Chapter 24](#) has one job: when something unexpected enters the safety horizon of a drone, produce a velocity for the next fraction of a second that is safe *and* keeps the drone roughly on its nominal route. [Chapters 12](#) and [13](#) solved this with velocity obstacles and optimal reciprocal collision avoidance (ORCA). Both treat the drone as a point that can change its velocity instantly: ORCA picks any velocity outside a set of forbidden half-planes, and the drone is assumed to adopt it at once. For a light quadrotor at moderate speed this is a fair approximation. For a heavy delivery drone, a fixed-wing aircraft or any vehicle whose acceleration is limited to a fraction of what ORCA asks for, it is not: the drone is told to fly at a velocity it cannot reach within the control interval, and the guarantee of [Chapter 13](#) evaporates.

The dynamic window approach (**DWA**) of Fox, Burgard and Thrun [\[46\]](#) starts from the opposite end. It first asks which velocities the robot can reach in the next Δt seconds given its acceleration limits, discards those from which it could no longer stop before the nearest obstacle, simulates the short trajectory of each remaining candidate, scores the candidates by heading, clearance and speed, and executes the winner for one interval before starting over. DWA was developed for the wheeled robot RHINO, which drove through crowded rooms with its range sensors as its only knowledge of the world, and it has been the default local planner

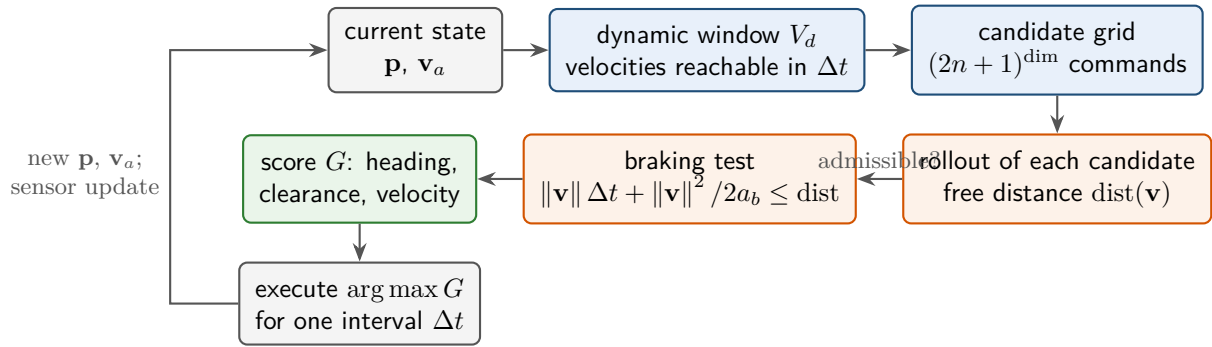


Figure 14.1. One control step of DWA. The current velocity $\mathbf{v}_a$ spans the dynamic window; the candidates in it are rolled out against the obstacles; those that cannot brake in time are discarded; the rest are scored by heading, clearance and velocity; the best one is executed for Δt and the loop repeats. Nothing is remembered between steps except the velocity itself.

of many mobile-robot navigation stacks since.

For the drone system this gives a second local layer, an alternative to ORCA when the drone’s dynamics matter more than reciprocity with the other agent: DWA never asks for a velocity the drone cannot produce, it works against any obstacle that can be sensed or predicted, and it does not assume that the other party shares the effort. The price is that it is a greedy, memoryless method. It can drive into a dead end and stay there, and it can oscillate between two equally attractive commands. The Week 7 milestone of the study plan (Appendix A) is precisely to know when such local methods work and when they fail, and this chapter and the next (Chapter 15) are built around that question.

Sections 14.2 and 14.3 give the idea and the definitions, Sections 14.4 and 14.5 the pseudocode and a worked example with every number taken from `ch14_dwa.py`, Section 14.6 what DWA guarantees and what it does not, Sections 14.7 and 14.8 the weights, the remedies and the comparison with ORCA and potential fields, and Sections 14.9 and 14.10 the implementation and the drone system.

14.2 The idea in plain words

Think of the problem from the point of view of the flight controller rather than the map. Every $\Delta t = 0.25$ seconds it must hand a velocity command to the motors. Whatever the global planner says, the drone is right now flying at some velocity $\mathbf{v}_a$, and in a quarter of a second it can change that velocity only by $a_{\max} \Delta t$ along each axis. The set of commands it can actually execute is therefore a small box around $\mathbf{v}_a$ in velocity space, the **dynamic window**. Everything outside the window is wishful thinking.

Inside the window DWA reasons like a careful driver. For each candidate velocity it imagines flying with that velocity for a short while, a **rollout**, and measures how far it could go before hitting something. If that distance is shorter than the distance needed to brake to a standstill, the candidate is thrown away: choosing it would commit the drone to a collision even if the very next command were full braking. This is the **admissibility** test. The remaining candidates are all safe for at least one more step, and among them DWA simply picks the one with the best score, a weighted sum of three ingredients: does the rollout head toward the goal, how much free space does it have ahead, and how fast is it. The winner is sent to the motors, the drone moves, the sensors deliver a new picture of the obstacles, and the whole computation is repeated from the new velocity. Figure 14.1 shows the loop.

Two things distinguish this from the velocity-obstacle methods of Chapters 12 and 13.

DWA searches the space of *achievable* commands over one control interval, so every command it returns respects the dynamics by construction, whereas ORCA searches all velocities and hopes the robot can follow. And DWA judges a command by simulating it: the rollout is an explicit prediction, an arc for a wheeled robot and a straight segment for a drone, into which any obstacle representation with a “how far is it free in this direction” query can be plugged.

Key idea

Search only the velocities you can reach in the next control interval, keep only those from which you can still stop before the nearest obstacle, simulate each for a short horizon, and execute the one whose rollout best combines heading toward the goal, clearance and speed. Safety comes from the braking test; progress comes from the score; memory comes from nowhere, which is why DWA is fast, respects dynamics, and can still get stuck.

14.3 Problem statement and notation

14.3.1 The velocity space

DWA plans in **velocity space**: the coordinates of the search are not positions but the components of the velocity command that is held constant during the next control interval Δt . Two robot models are used in this chapter.

Definition 14.1 (Velocity space of a differential-drive robot). A **differential-drive robot** has the pose (x, y, θ) and is commanded by a translational velocity v and a rotational velocity ω . Its velocity space is the set of pairs (v, ω) . During an interval in which (v, ω) is constant the robot moves on a circular arc of radius v/ω (a straight line if $\omega = 0$), so that after time t

$$x(t) = x + \frac{v}{\omega}(\sin(\theta + \omega t) - \sin \theta), \quad y(t) = y - \frac{v}{\omega}(\cos(\theta + \omega t) - \cos \theta), \quad \theta(t) = \theta + \omega t. \quad (14.1)$$

Definition 14.2 (Velocity space of a velocity-controlled drone). A **velocity-controlled drone** is the point-mass model of Chapter 2: its state is the position $\mathbf{p} \in \mathbb{R}^3$ and its command is the velocity $\mathbf{v} = (v_x, v_y, v_z)$, held constant for Δt , so that $\mathbf{p}(t) = \mathbf{p} + \mathbf{v}t$ for $0 \leq t \leq \Delta t$. The velocity space is $\mathbb{R}^3$ (or $\mathbb{R}^2$ for flight at constant altitude). The drone has a speed limit $v_{\max}$ and a per-axis acceleration limit $a_{\max}$: two consecutive commands $\mathbf{v}_a$ and $\mathbf{v}$ must satisfy $|v_k - v_{a,k}| \leq a_{\max} \Delta t$ for every axis k .

The drone is **holonomic**: it can move in any direction regardless of its orientation, so a rollout is a straight segment (Figure 14.2, right). The wheeled robot is not: it can only move along its heading, and a rollout is an arc (Figure 14.2, left). Everything else in DWA is the same for both models. We use the drone form from Section 14.4 on and write $\mathbf{v}_a$ for the velocity the drone is currently flying with, exactly as Fox et al. write (v_a, ω_a) for the actual velocity of their robot.

Obstacles are handled in the inflated configuration space of Chapter 2: every obstacle is grown by the drone’s radius r and the drone becomes a point. For a candidate $\mathbf{v} \neq \mathbf{0}$ the **free distance**

$$\text{dist}(\mathbf{v}) = \min(d_{\max}, d_{\text{free}}(\mathbf{p}, \mathbf{v}/\|\mathbf{v}\|)) \quad (14.2)$$

is what the rollout measures, where $d_{\text{free}}(\mathbf{p}, \hat{\mathbf{u}})$ is the distance from $\mathbf{p}$ along the ray in direction $\hat{\mathbf{u}}$ to the first inflated obstacle (∞ if there is none). The cap $d_{\max}$ is the sensing range, or simply the distance beyond which we stop caring. For $\mathbf{v} = \mathbf{0}$ we set $\text{dist}(\mathbf{0}) = d_{\max}$: standing still never runs into a static obstacle. For disks the ray cast is the ray–circle intersection of Chapter 2; for axis-aligned boxes it is a slab test; for a point cloud or an occupancy grid it is a march along the ray.

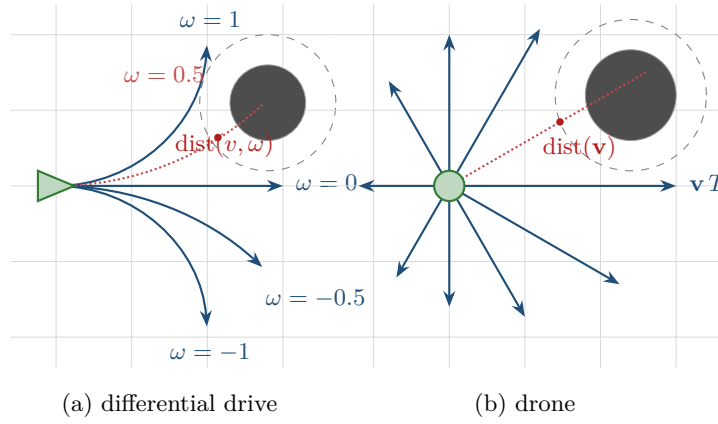


Figure 14.2. Rollouts are the trajectories the robot would follow if it kept a candidate command for a while. Left: a differential-drive robot heading along x at $v = 1$ m/s; the arcs are $\omega = 0, \pm 0.5, \pm 1$ rad/s, with radius v/ω . The free distance $\text{dist}(v, \omega)$ is measured along the arc up to the first contact with the inflated obstacle. Right: a velocity-controlled drone; every rollout is a straight segment $\mathbf{p} + \mathbf{v}t$, and $\text{dist}(\mathbf{v})$ is a ray cast in the direction of $\mathbf{v}$. Dotted rollouts hit the obstacle within the sensing range $d_{\max}$.

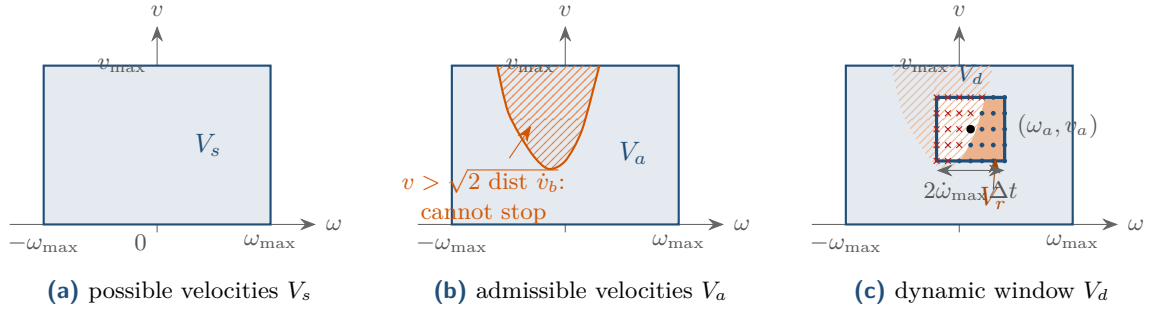


Figure 14.3. The three restrictions in the (ω, v) space of a differential-drive robot. (a) The velocity limits give a rectangle. (b) Velocities from which the robot could not stop before the obstacle on its arc are cut away (hatched); the bite is deepest for the curvatures that lead straight into the obstacle. (c) The dynamic window is the rectangle of velocities reachable from (v_a, ω_a) within one interval; the search set V_r (dark) is its intersection with the first two sets. Only the grid points inside V_r are evaluated; the previous command (ω_a, v_a) itself carries a cross here, because a new obstacle has just come into range and made it inadmissible.

14.3.2 The three restrictions

The search set of DWA is the intersection of three sets in velocity space (Figure 14.3). We state each for the differential drive, as in [46], and for the drone.

Definition 14.3 (Possible velocities). The set of **possible velocities** V_s contains the commands the actuators can produce at all. For the differential drive, $V_s = [0, v_{\max}] \times [-\omega_{\max}, \omega_{\max}]$ (or $[v_{\min}, v_{\max}]$ if driving backward is allowed). For the drone, $V_s = \{\mathbf{v} : \|\mathbf{v}\| \leq v_{\max}\}$, a ball, or a box if the axes have separate limits.

Definition 14.4 (Admissible velocities). A velocity is **admissible** if the robot can still come to a standstill before it reaches the nearest obstacle on its rollout. With the braking decelerations $\dot{v}_b$ and $\dot{\omega}_b$ of the differential drive,

$$V_a = \left\{ (v, \omega) \mid v \leq \sqrt{2 \text{dist}(v, \omega) \dot{v}_b} \text{ and } |\omega| \leq \sqrt{2 \text{dist}(v, \omega) \dot{\omega}_b} \right\}. \quad (14.3)$$

For the drone with braking deceleration a_b , $V_a = \{\mathbf{v} \mid \|\mathbf{v}\| \leq \sqrt{2 \text{dist}(\mathbf{v}) a_b}\}$. In a digital control loop this test is sharpened to Equation (14.6) below, and that discrete form is the one Algorithm 14.1, Theorem 14.12, the tables and the code use.

Where the square root comes from. A robot that moves at speed v and decelerates at the constant rate a_b has speed $v - a_b t$ after t seconds and stops at $t_s = v/a_b$. The distance it covers meanwhile is

$$s_b(v) = \int_0^{t_s} (v - a_b t) dt = v t_s - \frac{1}{2} a_b t_s^2 = \frac{v^2}{2a_b}, \quad (14.4)$$

the **braking distance**. The robot is safe if the braking distance fits into the free distance, $v^2/(2a_b) \leq \text{dist}$, and solving for v gives $v \leq \sqrt{2 \text{dist} a_b}$. The condition on ω is the braking argument applied to the rotation: the turn must be stoppable within the heading change still available before the obstacle, $\omega^2 \leq 2 \Delta\theta(v, \omega) \dot{\omega}_b$ with $\Delta\theta(v, \omega) = |\omega| \text{dist}(v, \omega)/v$, the angle the robot still turns through while it covers the free distance. Fox et al. write the translational free distance $\text{dist}(v, \omega)$ in place of $\Delta\theta$, as Equation (14.3) does; the two sides then do not carry the same units – $\sqrt{\text{m} \cdot \text{rad}}/\text{s}$ on the right against rad/s on the left – and the condition is a heuristic cap on ω rather than a braking guarantee. The sign of ω does not matter – a fast left turn is as hard to stop as a fast right turn – which is why Equation (14.3) needs the bars; Fox et al. omit them, and without them every $\omega < 0$ passes whatever the free distance. Fox et al. measure dist along the arc of the curvature ω/v , which is why it is written $\text{dist}(v, \omega)$. The drone form of Definition 14.4, which is what Algorithm 14.1 and all the code use, has only the translational test, so nothing else in the chapter depends on this.

The discrete-time refinement. Equation (14.4) assumes that braking starts immediately. In a digital control loop the chosen command is held for the whole interval Δt before the next command, at best full braking, can begin. The distance to standstill is then at most

$$s_b^{\Delta t}(v) = v \Delta t + \frac{v^2}{2a_b}, \quad (14.5)$$

and the admissibility test becomes $s_b^{\Delta t}(\|\mathbf{v}\|) \leq \text{dist}(\mathbf{v})$, or, solving the quadratic for the speed,

$$\|\mathbf{v}\| \leq v_{\text{adm}}(\text{dist}) = -a_b \Delta t + \sqrt{a_b^2 \Delta t^2 + 2 a_b \text{dist}}. \quad (14.6)$$

For $\Delta t \rightarrow 0$ this is the condition of Equation (14.3). With $a_b = 2 \text{ m/s}^2$ and $\Delta t = 0.25 \text{ s}$, at 0.608 m from an obstacle Equation (14.3) allows 1.56 m/s and Equation (14.6) 1.14 m/s. The difference is not academic, because braking in a digital loop also proceeds in steps: a drone at 2 m/s that loses $a_b \Delta t = 0.5 \text{ m/s}$ per step covers $0.5 + 0.375 + 0.25 + 0.125 = 1.25 \text{ m}$ before it stands still, not the 1 m of Equation (14.4). One meter in front of a wall, Equation (14.3) still allows 2 m/s and the drone hits the wall in step 3; Equation (14.6) allows 1.56 m/s, so the drone brakes and turns aside 6 cm short of the wall. The self-test of `ch14_dwa.py` flies both versions. Section 14.6 proves that Equation (14.6) is enough, and all code and numbers in this chapter use it.

Definition 14.5 (Dynamic window). The **dynamic window** V_d contains the velocities reachable from the current velocity within one control interval under the acceleration limits. For the differential drive with the maximal accelerations $\dot{v}_{\text{max}}$ and $\dot{\omega}_{\text{max}}$,

$$V_d = \{(v, \omega) \mid v \in [v_a - \dot{v}_{\text{max}} \Delta t, v_a + \dot{v}_{\text{max}} \Delta t] \text{ and } \omega \in [\omega_a - \dot{\omega}_{\text{max}} \Delta t, \omega_a + \dot{\omega}_{\text{max}} \Delta t]\}. \quad (14.7)$$

For the drone with the per-axis limit a_{max} , $V_d = \{\mathbf{v} \mid |v_k - v_{a,k}| \leq a_{\text{max}} \Delta t \text{ for all } k\}$, a cube of side $2a_{\text{max}} \Delta t$ centered on $\mathbf{v}_a$. If the drone's true limit is isotropic, $\|\mathbf{a}\| \leq a_{\text{max}}$, this

per-axis box is optimistic by a factor $\sqrt{\dim}$ at its corners; use the ball $\|\mathbf{v} - \mathbf{v}_a\| \leq a_{\max} \Delta t$ instead, or shrink the box by $\sqrt{\dim}$ (Exercise 14.5).

Definition 14.6 (Search set). The **search set** of one DWA step is $V_r = V_s \cap V_a \cap V_d$. In practice it is discretized: the window is covered by a grid of resolution Δv centered on $\mathbf{v}_a$, and only the grid points in V_r are evaluated. Write $\dim$ for the dimension of the velocity space (2 for the differential drive and for a drone flying at a fixed altitude, 3 for a drone that also climbs). With $n = a_{\max} \Delta t / \Delta v$ the grid has at most $(2n + 1)^{\dim}$ points before the admissibility test; it has fewer when the ball V_s clips the corners of the box.

V_s is fixed. V_d moves with the robot's own velocity and is small (with $a_{\max} = 2 \text{ m/s}^2$ and $\Delta t = 0.25 \text{ s}$ it spans only $\pm 0.5 \text{ m/s}$ around $\mathbf{v}_a$), which is how the dynamics enter. V_a moves with the obstacles and is where the geometry enters. Note that the window constrains the *change* of velocity, not the velocity: a fast robot needs several steps to slow down, and the braking test must look further ahead than one step.

14.3.3 The objective

Among the velocities in V_r , DWA maximizes

$$G(\mathbf{v}) = \sigma(\alpha \cdot \text{heading}(\mathbf{v}) + \beta \cdot \text{clearance}(\mathbf{v}) + \gamma \cdot \text{velocity}(\mathbf{v})), \quad (14.8)$$

where $\alpha, \beta, \gamma \geq 0$ are weights and σ is a smoothing discussed below. Fox et al. call the three terms *target heading*, *clearance* (written *dist*) and *velocity*. In the normalized form used in this chapter all three lie in $[0, 1]$.

Definition 14.7 (Heading term). Let $\mathbf{p}' = \mathbf{p} + \mathbf{v} \Delta t$ be the position predicted after the next interval and $\theta(\mathbf{v}) \in [0, \pi]$ the angle between $\mathbf{v}$ and the direction $\mathbf{t} - \mathbf{p}'$ from $\mathbf{p}'$ to the target point $\mathbf{t}$ (the goal, or a point on the global path, Section 14.8). The **heading term** is

$$\text{heading}(\mathbf{v}) = 1 - \frac{\theta(\mathbf{v})}{\pi}, \quad (14.9)$$

with $\text{heading}(\mathbf{0}) = 0$. It is 1 when the rollout points straight at the target and 0 when it points away. Fox et al. use $180^\circ - \theta$ with θ measured at the position and orientation the robot is predicted to reach after the next interval; for the differential drive the orientation changes along the arc, so this predicted pose matters more than for the drone.

Definition 14.8 (Clearance term). The **clearance term** is the normalized free distance of the rollout,

$$\text{clearance}(\mathbf{v}) = \frac{\min(\text{dist}(\mathbf{v}), d_{\max})}{d_{\max}} \in [0, 1]. \quad (14.10)$$

The min is redundant here, because Equation (14.2) already caps dist at $d_{\max}$; the code keeps it as a guard. Fox et al. use the unnormalized $\text{dist}(v, \omega)$ and set it to a large constant when no obstacle lies on the curvature.

Definition 14.9 (Velocity term). The **velocity term** rewards speed,

$$\text{velocity}(\mathbf{v}) = \frac{\|\mathbf{v}\|}{v_{\max}} \in [0, 1]. \quad (14.11)$$

For the differential drive it is the translational speed v ; the rotation ω is not rewarded.

Why the terms are normalized. The three terms measure an angle, a distance and a speed. In raw units their weighted sum has no meaning: weights that balance heading against clearance on a map in meters let clearance dominate everything on a map in centimeters, and

Table 14.1. Symbols of this chapter and the values used in `ch14_dwa.py` (class `DwaParams`). The drone flies at constant altitude in the examples, so velocities have two components.

Symbol	Meaning	Value
$v_{\max}$	speed limit; V_s is the ball of this radius	2 m/s
$a_{\max}$	acceleration limit per axis (spans the window)	2 m/s ²
a_b	braking deceleration (admissibility), $a_b \leq a_{\max}$	2 m/s ²
Δt	control interval	0.25 s
Δv	resolution of the velocity grid	0.25 m/s
$d_{\max}$	cap on free distances (sensing range)	3 m
r	drone radius (obstacle inflation)	0.2 m
T	rollout horizon used for drawing predicted trajectories	1 s
(α, β, γ)	weights of heading, clearance, velocity	(0.6, 0.2, 0.2)
ρ_{goal}	goal tolerance	0.2 m

must be retuned for a drone that flies ten times faster. Dividing each term by its maximum (π , $d_{\max}$, $v_{\max}$) makes the weights dimensionless fractions that say how much each concern counts. Some implementations normalize each term over the current candidate set instead, so that the best candidate of each term scores 1; this is more portable still, but G then depends on which candidates happen to be admissible.

The smoothing σ . The original objective is smoothed: σ averages the weighted sum over the neighboring cells of the velocity grid, so that commands whose neighbors are also good are preferred and the robot keeps more side clearance. A candidate next to inadmissible cells (score 0) is pulled down. The code offers σ as an option; the examples use no smoothing, to keep the numbers transparent.

14.4 The algorithm

Algorithm 14.1 is one DWA step for the drone; Algorithm 14.2 wraps it into the control loop. Table 14.1 lists the parameters.

Walkthrough of Algorithm 14.1. Line 2 builds the window of Definition 14.5, clipped to the speed limits so that the grid does not leave V_s along an axis; line 3 lays the grid over it and drops the points outside the ball V_s . The grid is centered on $\mathbf{v}_a$, so “keep the current velocity” is always a candidate. Line 4 adds the **braking candidate**: same direction as $\mathbf{v}_a$, speed reduced by $a_b \Delta t$ (or zero). It lies in the window because $a_b \leq a_{\max}$, and Theorem 14.12 shows that it is always admissible when the previous command was; it is the reason why the loop never runs out of safe options and it is also the command returned when nothing else is admissible, which with static obstacles cannot happen. Line 7 is the rollout: for the drone a single ray cast, for the differential drive a march along the arc of Equation (14.1). Line 8 is the discrete admissibility test Equation (14.6), applied to the free distance and to the distance to the goal at once (`brake_for_goal` in the code), so that the drone neither overshoots the goal nor orbits it; the clearance term keeps the free distance d . Line 9 evaluates Equation (14.8) without smoothing, and line 11 keeps the best; ties are broken by grid order.

Walkthrough of Algorithm 14.2. Line 4 is where DWA meets the rest of the system: the obstacle set can be a map, a range scan, or the predicted positions of other drones from Chapters 18 and 20. Line 5 selects the target of the heading term. With $P = \emptyset$ it is the goal

Algorithm 14.1. One step of the dynamic window approach (drone form)

Input: position $\mathbf{p}$, current velocity $\mathbf{v}_a$, target point $\mathbf{t}$, goal $\mathbf{p}_g$, obstacles, parameters of Table 14.1

Output: the velocity command for the next interval

```

1 Function DwaCommand( $\mathbf{p}$ ,  $\mathbf{v}_a$ ,  $\mathbf{t}$ ,  $\mathbf{p}_g$ , obstacles):
2    $\mathbf{lo} \leftarrow \max(\mathbf{v}_a - a_{\max}\Delta t, -v_{\max})$ ;  $\mathbf{hi} \leftarrow \min(\mathbf{v}_a + a_{\max}\Delta t, v_{\max})$  // window  $V_d$ 
3    $C \leftarrow$  grid points  $\mathbf{v}_a + \Delta v (i_1, \dots, i_{\dim})$ ,  $|i_k| \leq a_{\max}\Delta t/\Delta v$ , inside  $[\mathbf{lo}, \mathbf{hi}]$  and with
    $\|\mathbf{v}\| \leq v_{\max}$ 
4    $\mathbf{v}_b \leftarrow \mathbf{v}_a \cdot \max(0, 1 - a_b\Delta t/\|\mathbf{v}_a\|)$ ; add  $\mathbf{v}_b$  to  $C$  // braking candidate, always in  $V_d$ 
5    $G^* \leftarrow -\infty$ ;  $\mathbf{v}^* \leftarrow \mathbf{v}_b$ 
6   foreach  $\mathbf{v} \in C$  do
7      $d \leftarrow \text{FreeDistance}(\mathbf{p}, \mathbf{v}, \text{obstacles}, d_{\max})$  // rollout: ray cast in direction  $\mathbf{v}$ 
8     if  $\|\mathbf{v}\| \Delta t + \|\mathbf{v}\|^2/(2a_b) > \min(d, \|\mathbf{p}_g - \mathbf{p}\|)$  then continue // brake for the
       goal as for a wall
9      $G \leftarrow \alpha (1 - \theta(\mathbf{v})/\pi) + \beta \min(d, d_{\max})/d_{\max} + \gamma \|\mathbf{v}\|/v_{\max}$ 
10    if  $G > G^*$  then
11       $G^* \leftarrow G$ ;  $\mathbf{v}^* \leftarrow \mathbf{v}$ 
12  return  $\mathbf{v}^*$ 

```

Algorithm 14.2. The DWA control loop with a global path and hysteresis

Input: start $\mathbf{p}_0$, goal $\mathbf{p}_g$, obstacles, optional global path P (a polyline), lookahead L , hysteresis margin $\eta \geq 0$

```

1 Function DwaControlLoop( $\mathbf{p}_0$ ,  $\mathbf{p}_g$ , obstacles,  $P$ ):
2    $\mathbf{p} \leftarrow \mathbf{p}_0$ ;  $\mathbf{v}_a \leftarrow \mathbf{0}$ 
3   while  $\|\mathbf{p} - \mathbf{p}_g\| > \rho_{\text{goal}}$  do
4     update the obstacle set from the sensors and the prediction layer
5      $\mathbf{t} \leftarrow \mathbf{p}_g$  if  $P = \emptyset$  else LookaheadPoint( $\mathbf{p}$ ,  $P$ ,  $L$ ) // heading reference
6      $\mathbf{v} \leftarrow \text{DwaCommand}(\mathbf{p}, \mathbf{v}_a, \mathbf{t}, \mathbf{p}_g, \text{obstacles})$ 
7     if  $\eta > 0$  and  $\mathbf{v}_a$  is admissible and  $G(\mathbf{v}_a) \geq G(\mathbf{v}) - \eta$  then
8        $\mathbf{v} \leftarrow \mathbf{v}_a$  // hysteresis: keep the old command
9     execute  $\mathbf{v}$  for  $\Delta t$ ;  $\mathbf{p} \leftarrow \mathbf{p} + \mathbf{v} \Delta t$ ;  $\mathbf{v}_a \leftarrow \mathbf{v}$ 

```

itself, the pure “greedy” DWA of the original paper. With a global path it is a **lookahead point**: the point at arc length L beyond the point of P closest to $\mathbf{p}$, a carrot that the drone chases along the path. Line 8 is the optional hysteresis of Section 14.8. Line 9 moves the drone; on a real drone a velocity controller tracks the command, and the window should be sized for what it actually delivers.

14.5 A worked example

Example 14.10 (A drone approaching an obstacle). A drone of radius $r = 0.2$ m is at $\mathbf{p} = (0, 0)$, flying at $\mathbf{v}_a = (1.0, 0)$ m/s toward the goal $\mathbf{p}_g = (5, 0)$. A disk obstacle of radius 0.4 m is centered at $(1.2, -0.1)$, almost on the line to the goal and slightly below it. The parameters are those of Table 14.1. The instance is `worked_example()` in `ch14_dwa.py`; every number in this section is printed by its self-test and by `gen_ch14_traj.py`.

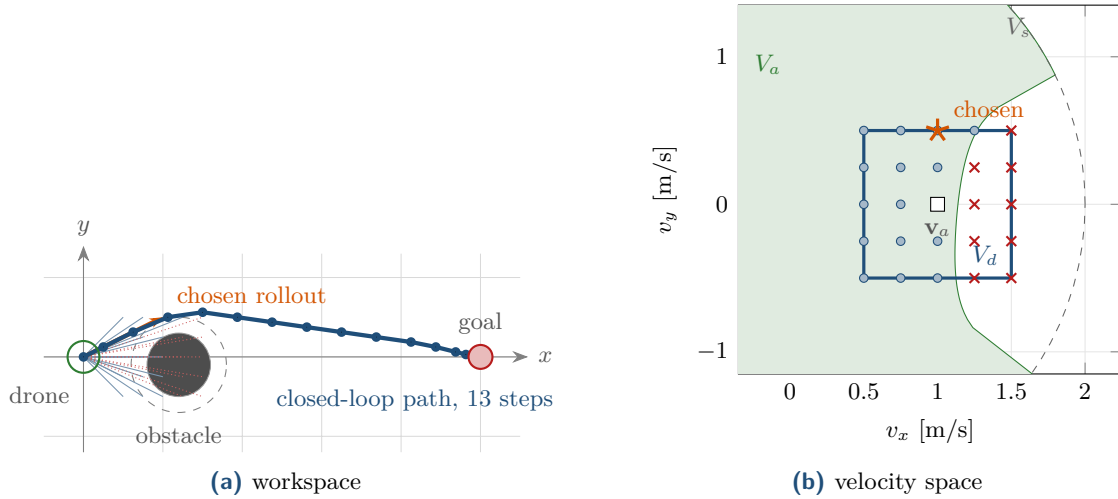


Figure 14.4. Example 14.10. (a) The drone (green circle) with the 25 rollouts of one second: dotted red ones are inadmissible, blue ones admissible, the orange arrow is the chosen command (1.0, 0.5); the dashed circle is the obstacle inflated by the drone radius, and the 14 blue dots are the start and the 13 step end-points of the closed-loop run to the goal. (b) The same step in velocity space: the dashed circle is V_s , the blue box the window V_d , the green region the admissible set V_a of Equation (14.6) (its bite points at the obstacle), the dots the 16 admissible grid points, the crosses the 9 inadmissible ones, the square the current velocity $\mathbf{v}_a$ and the star the chosen command.

The window and the grid. With $a_{\max}\Delta t = 0.5$ m/s the window is $V_d = [0.5, 1.5] \times [-0.5, 0.5]$, and with $\Delta v = 0.25$ m/s the grid has $5 \times 5 = 25$ points, all inside the ball $\|\mathbf{v}\| \leq 2$. The braking candidate (0.5, 0) is a grid point, so C has 25 elements (Figure 14.4b).

The admissibility test. The inflated obstacle has radius 0.6 m. Straight ahead, the ray from the origin along x passes at distance 0.1 from the center and meets the disk at $1.2 - \sqrt{0.6^2 - 0.1^2} = 0.608$ m. By Equation (14.6) the largest admissible speed in that direction is $-0.5 + \sqrt{0.25 + 4 \cdot 0.608} = 1.138$ m/s: keeping (1.0, 0) is admissible (braking distance $0.25 + 0.25 = 0.5 \leq 0.608$), but (1.25, 0) is not ($0.3125 + 0.39 = 0.70 > 0.608$). All ten candidates with $v_x \geq 1.25$ except (1.25, 0.5) are inadmissible: their rays hit the disk within 0.6 to 0.8 m. Sixteen candidates survive; five of them, those that steer clear of the disk at moderate speed, see no obstacle at all within $d_{\max}$ and have $\text{dist} = 3$.

The scores. Table 14.2 lists the three terms for 13 of the 25 candidates. Keeping (1.0, 0) heads perfectly at the goal (heading = 1) but has only $0.608/3 = 0.203$ of clearance, for $G = 0.6 + 0.041 + 0.1 = 0.741$. The winner is (1.0, 0.5): the ray misses the inflated disk, so clearance is 1, and although the heading drops to 0.844 (the rollout points 28° away from the goal seen from the predicted position (0.25, 0.125)) the total is 0.818. The swerve is toward $+y$, the side on which the obstacle leaves more room; the mirror image (1.0, -0.5) still hits the disk at 0.718 m and scores only 0.666. Slower candidates with full clearance, (0.75, 0.5) and (0.5, 0.25), lose on the velocity term. Note what decided the step: the braking test removed every fast candidate that pointed at the obstacle, and among the survivors the clearance term, not the heading term, chose the side.

The closed loop. Table 14.3 continues the run. In step 2 the window has moved to $[0.5, 1.5] \times [0, 1]$ and the drone accelerates to (1.5, 0.75), in step 3 to (1.75, 0.75): with the obstacle no longer in the way, the velocity term takes over. From step 4 the heading term bends

Table 14.2. Thirteen of the 25 candidates of [Example 14.10](#) with their free distance, admissibility and the three normalized terms; $G = 0.6 \text{ heading} + 0.2 \text{ clearance} + 0.2 \text{ velocity}$. The chosen command is in bold.

v_x	v_y	dist [m]	admissible	heading	clearance	velocity	G
1.50	0.00	0.608	no	1.000	0.203	0.750	–
1.25	0.00	0.608	no	1.000	0.203	0.625	–
1.00	0.00	0.608	yes	1.000	0.203	0.500	0.741
0.50	0.00	0.608	yes	1.000	0.203	0.250	0.691
1.50	0.50	0.739	no	0.889	0.246	0.791	–
1.25	0.50	0.812	yes	0.870	0.271	0.673	0.711
1.00	0.50	3.000	yes	0.844	1.000	0.559	0.818
0.75	0.50	3.000	yes	0.805	1.000	0.451	0.773
0.50	0.50	3.000	yes	0.742	1.000	0.354	0.716
0.50	0.25	3.000	yes	0.848	1.000	0.280	0.765
1.00	0.25	0.682	yes	0.918	0.227	0.515	0.699
1.00	–0.50	0.718	yes	0.844	0.239	0.559	0.666
0.50	–0.50	3.000	yes	0.742	1.000	0.354	0.716

the flight back toward the goal line, (1.75, 0.25) and then (1.75, –0.25) for six steps, during which the heading rises to 0.99. Steps 11–13 are the approach to the goal: the goal-braking rule of [Section 14.4](#) makes the fast candidates inadmissible and the drone slows to (1.25, –0.25), (1.0, –0.25) and finally the braking candidate (0.51, –0.13), which is not a grid point. After 13 steps (3.25 s) the drone is at (4.82, 0.03), inside the goal tolerance. The path is 4.98 m long, and the smallest gap between the drone’s hull and the obstacle along the way was 1.6 cm: DWA with a dominant heading term shaves obstacles, because the clearance term only counts what lies ahead on the rollout, not what lies beside it. Raising β , inflating the obstacles by a safety margin, or the smoothing σ all widen the berth.

14.6 Properties: safety, feasibility, locality

14.6.1 Safety with respect to static obstacles

The admissibility test is a promise about the future: whatever the next commands are, the robot can still stop in time. The following lemma and theorem make the promise precise for the discrete control loop of [Algorithm 14.2](#). They are stated for the drone; the differential drive is discussed afterward.

Lemma 14.11 (Braking in discrete steps). *Let the drone fly at speed v along a fixed direction and from the next interval on execute the braking candidates $v - a_b \Delta t$, $v - 2a_b \Delta t$, $\dots$, each for one interval, down to zero. The distance covered after the current interval is at most $v^2/(2a_b)$, and the total distance to standstill, including the current interval, is at most $s_b^{\Delta t}(v) = v\Delta t + v^2/(2a_b)$.*

Proof. The distance after the current interval is $\sum_{k \geq 1} \max(v - ka_b \Delta t, 0) \Delta t$. This is a right Riemann sum of the non-increasing function $f(t) = \max(v - a_b t, 0)$ with step Δt , so it is at most $\int_0^\infty f(t) dt = v^2/(2a_b)$, the continuous braking distance [Equation \(14.4\)](#). Adding the $v\Delta t$ of the current interval gives the second claim. $\square$

Theorem 14.12 (Safety invariant of DWA). *Assume static obstacles, exact free distances in the inflated configuration space, $a_b \leq a_{\max}$, the braking candidate of line 4 always in the*

Table 14.3. Trace of the closed-loop run of [Example 14.10](#): position and velocity at the start of each step, the chosen command and its terms. Every chosen command has $\text{dist} = 3$ m (no obstacle on its ray within $d_{\max}$), so clearance is 1 throughout.

Step	$\mathbf{p}$	$\mathbf{v}_a$	chosen $\mathbf{v}$	heading	velocity	G
1	(0.00, 0.00)	(1.00, 0.00)	(1.00, 0.50)	0.844	0.559	0.818
2	(0.25, 0.12)	(1.00, 0.50)	(1.50, 0.75)	0.830	0.839	0.866
3	(0.62, 0.31)	(1.50, 0.75)	(1.75, 0.75)	0.831	0.952	0.889
4	(1.06, 0.50)	(1.75, 0.75)	(1.75, 0.25)	0.904	0.884	0.919
5	(1.50, 0.56)	(1.75, 0.25)	(1.75, -0.25)	0.994	0.884	0.973
6	(1.94, 0.50)	(1.75, -0.25)	(1.75, -0.25)	0.993	0.884	0.972
7	(2.38, 0.44)	(1.75, -0.25)	(1.75, -0.25)	0.991	0.884	0.971
8	(2.81, 0.38)	(1.75, -0.25)	(1.75, -0.25)	0.989	0.884	0.970
9	(3.25, 0.31)	(1.75, -0.25)	(1.75, -0.25)	0.985	0.884	0.968
10	(3.69, 0.25)	(1.75, -0.25)	(1.75, -0.25)	0.978	0.884	0.964
11	(4.12, 0.19)	(1.75, -0.25)	(1.25, -0.25)	0.993	0.637	0.923
12	(4.44, 0.12)	(1.25, -0.25)	(1.00, -0.25)	0.985	0.515	0.894
13	(4.69, 0.06)	(1.00, -0.25)	(0.51, -0.13)	0.974	0.265	0.838

candidate set, and a starting velocity that is admissible in the sense of [Equation \(14.6\)](#) (for instance $\mathbf{v}_a = \mathbf{0}$). Then at every step of [Algorithm 14.2](#):

- (i) the executed command lies in the dynamic window, so the velocity changes by at most $a_{\max}\Delta t$ per axis per step (dynamic feasibility);
- (ii) at least one candidate is admissible, namely the braking candidate;
- (iii) the drone never touches an obstacle.

Proof. (i) holds by construction of the candidate set: every grid point and the braking candidate satisfy $|v_k - v_{a,k}| \leq a_{\max}\Delta t$, the latter because its per-axis change is $a_b\Delta t |v_{a,k}| / \|\mathbf{v}_a\| \leq a_b\Delta t \leq a_{\max}\Delta t$.

(ii) and (iii) follow by induction over the steps. Suppose the command $\mathbf{v}$ executed in the current step is admissible: $d = \text{dist}(\mathbf{v}) \geq \|\mathbf{v}\| \Delta t + \|\mathbf{v}\|^2 / (2a_b)$. During the step the drone moves the distance $\|\mathbf{v}\| \Delta t < d$ along the ray of $\mathbf{v}$, which is free up to d , so it touches nothing (iii). At the next step the braking candidate $\mathbf{v}_b = \lambda \mathbf{v}$ with $\lambda = \max(0, 1 - a_b\Delta t / \|\mathbf{v}\|)$ points in the same direction, and because the obstacles have not moved and the new ray is a sub-ray of the old one, the free distance along it satisfies $d' \geq d - \|\mathbf{v}\| \Delta t \geq \|\mathbf{v}\|^2 / (2a_b)$ (the cap $d_{\max}$ can only raise it). The distance to the goal obeys the same recursion, since the drone moves $\|\mathbf{v}\| \Delta t$ per step: $\|\mathbf{p}_g - \mathbf{p}'\| \geq \|\mathbf{p}_g - \mathbf{p}\| - \|\mathbf{v}\| \Delta t$. Writing $\tilde{d} = \min(d, \|\mathbf{p}_g - \mathbf{p}\|)$ for the quantity [line 8](#) actually tests, we therefore have $\tilde{d}' \geq \tilde{d} - \|\mathbf{v}\| \Delta t$ as well, and the computation below applies verbatim with $\tilde{d}$ in place of d : the braking candidate passes the stricter test with the goal term included too. Its speed is $v' = \|\mathbf{v}\| - a_b\Delta t$ (or 0; the zero command has $\text{dist}(\mathbf{0}) = d_{\max}$ and is admissible), and

$$v'\Delta t + \frac{v'^2}{2a_b} = \|\mathbf{v}\| \Delta t - a_b\Delta t^2 + \frac{\|\mathbf{v}\|^2 - 2\|\mathbf{v}\| a_b\Delta t + a_b^2\Delta t^2}{2a_b} = \frac{\|\mathbf{v}\|^2}{2a_b} - \frac{a_b\Delta t^2}{2} \leq d',$$

so $\mathbf{v}_b$ is admissible (ii). The algorithm executes some admissible candidate, and the induction hypothesis holds again for the next step. The start of the induction is the assumption on the initial velocity. $\square$

The theorem says what the braking test buys: as long as the world stands still and the free-distance queries are right, a DWA drone can be stopped at any moment without collision,

whatever the weights. Everything the theorem excludes is a real failure mode: obstacles moving toward the drone shrink d' faster than $\|\mathbf{v}\| \Delta t$ (Section 14.8 repairs this with predicted velocities); overestimated free distances, from sensor noise or a coarse grid, break the induction; a velocity controller that lags makes the window larger than what the drone can do. For the differential drive the argument works along the arc if the robot brakes without changing its curvature; Fox et al. test v and ω separately, which is simpler and, in the cases that matter, more restrictive.

14.6.2 DWA is a local method

Proposition 14.13 (DWA is not complete). *Let the drone be at rest at a position where every non-zero candidate $\mathbf{v}$ of its window satisfies $\alpha \text{heading}(\mathbf{v}) + \beta \text{clearance}(\mathbf{v}) + \gamma \text{velocity}(\mathbf{v}) < \beta$. Then Algorithm 14.2 without a global path never moves the drone again, whatever the goal. Such positions exist in environments in which a collision-free path to the goal exists, so DWA is not complete.*

Proof. At rest, $\mathbf{0}$ is a grid point of the window with $\text{dist}(\mathbf{0}) = d_{\max}$; it is admissible and scores $G(\mathbf{0}) = \beta$. By assumption every other candidate scores less, so the $\arg \max$ is $\mathbf{0}$ and the drone stays where it is with the same velocity. DWA keeps no memory beyond the state $(\mathbf{p}, \mathbf{v}_a)$, which is unchanged, so the same decision is taken at every later step. For the existence, let $v_1 = \sqrt{\dim} a_{\max} \Delta t$ be the largest speed of a candidate in the window of a resting drone. If every ray from $\mathbf{p}$ meets an obstacle strictly within $d_{\max}(1 - (\alpha + \gamma v_1/v_{\max})/\beta)$, every non-zero candidate scores at most $\alpha + \gamma v_1/v_{\max} + \beta \text{clearance} < \beta$. A dead-end pocket behind a corner of that size satisfies this (all rays are short, but the corner leads out), and a goal placed outside it is reachable. With the clearance-dominated weights $(\alpha, \beta, \gamma) = (0.2, 0.6, 0.2)$ (not the defaults of Table 14.1, whose $\beta < \alpha$ makes the bound vacuous) and the remaining parameters of Table 14.1, the pocket may be 1.65 m in radius. $\square$

The proposition is the clean form of the failure, a **local minimum** in which the drone rests. With a dominant heading term the drone usually does not rest, because some admissible non-zero candidate then satisfies $\alpha \text{heading} + \gamma \text{velocity} > \beta(1 - \text{clearance})$ and so beats β – a tight pocket, in which every non-zero candidate is either inadmissible or hemmed in, can still deny it – and the failure becomes **oscillation**. Figure 14.5 shows the classical instance, a U-shaped obstacle between the drone and the goal, and Section 14.7 runs it: with the default weights the drone flies into the U, brakes in front of the back wall as Theorem 14.12 promises, and then shuttles between the arms, because every candidate that improves the heading points at the wall and is inadmissible while the candidates along the wall are not. In 300 steps (75 s) it covers 80 m without leaving the U, braking to within 4 cm of the back wall and turning 9 cm short of the arms. The self-test of `ch14_dwa.py` reproduces this run and asserts that the goal is *not* reached; it is the documented failure case of the chapter, and the Week 7 coding exercise asks you to build your own.

A subtler oscillation needs no trap: when two candidates score almost the same, say the two sides of an obstacle exactly on the line to the goal, the winner can flip from step to step and the drone zigzags. It is the same phenomenon as the oscillation of velocity obstacles in Chapter 12 and of potential fields in Chapter 15, and the remedy is the same in spirit: add inertia to the decision (Section 14.8).

14.6.3 Complexity

One step of Algorithm 14.1 evaluates $N_c = (2n + 1)^{\dim}$ grid points plus one, with $n = a_{\max} \Delta t / \Delta v$: 25 candidates in the plane and 125 in space for the parameters of Table 14.1, 121 and 1331 for $\Delta v = 0.1$ m/s. Each candidate needs one free-distance query, $\mathcal{O}(M)$ for

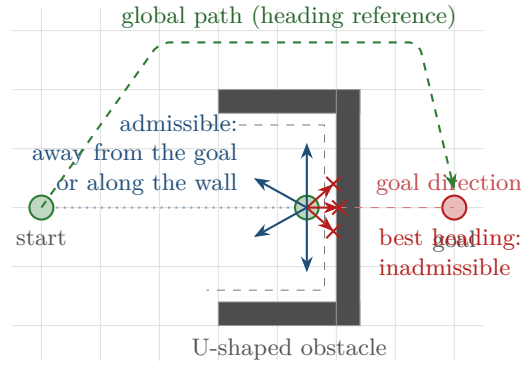


Figure 14.5. The U-shaped trap of `utrap_scene()`. The drone has flown into the U toward the goal behind the back wall. Every candidate that improves the heading (red, crossed) points at the wall and fails the braking test; the admissible candidates (blue) run parallel to the wall and score less than the heading-improving ones would, so the drone shuttles between the arms. The greedy rule cannot value the long way round (dashed green), which a global planner finds at once; feeding that path into the heading term is the standard remedy (Section 14.8).

Table 14.4. Metrics of the runs of Figure 14.6 over at most 300 steps (75 s): outcome, steps, path length, minimum clearance between the hull and the obstacles, and mean speed. The last row uses the global path as heading reference.

Scene	weights (α, β, γ)	outcome	steps	length [m]	clear. [m]	speed [m/s]
corridor	(0.6, 0.2, 0.2) default	reached	16	5.9	0.05	1.48
corridor	(0.8, 0.1, 0.1) heading	reached	16	5.8	0.05	1.46
corridor	(0.2, 0.6, 0.2) clearance	reached from outside	21	8.7	0.09	1.66
corridor	(0.2, 0.2, 0.6) velocity	reached	15	5.9	0.07	1.56
U-trap	(0.6, 0.2, 0.2) default	oscillates in the U	300	80.5	0.04	1.07
U-trap	(0.8, 0.1, 0.1) heading	oscillates in the U	300	79.6	0.03	1.06
U-trap	(0.2, 0.6, 0.2) clearance	flies away	300	146.9	0.09	1.96
U-trap	(0.2, 0.2, 0.6) velocity	wanders off	300	142.5	0.24	1.90
U-trap	(0.6, 0.2, 0.2) + global path	reached	25	10.2	0.61	1.64

M disks or boxes with analytic ray casts, or $\mathcal{O}(d_{\max}/\Delta s)$ steps of Δs for a marched rollout, and constant work for the three terms. The step therefore costs $\mathcal{O}(N_c M)$ time and $\mathcal{O}(N_c)$ memory, with nothing carried over between steps. The self-test performs several hundred control steps with 25 candidates in about a second of plain Python; compiled implementations run at hundreds of hertz.

14.7 Tuning the weights

Figure 14.6 and Table 14.4 show what happens when each weight dominates, in two scenes generated by `gen_ch14_traj.py`: a corridor of half-width 1 m with a pillar in it, and the U-trap of Figure 14.5. Each run uses the parameters of Table 14.1 with the weights changed, starts at rest, and is plotted for its first 20 s.

Heading dominates. With (0.8, 0.1, 0.1) the drone flies the straightest line the braking test allows: through the corridor past the pillar with 5 cm to spare, and straight into the U, where it oscillates for the whole run. Heading-dominant DWA is efficient in open space and blind in clutter: it cannot trade a detour now for progress later.

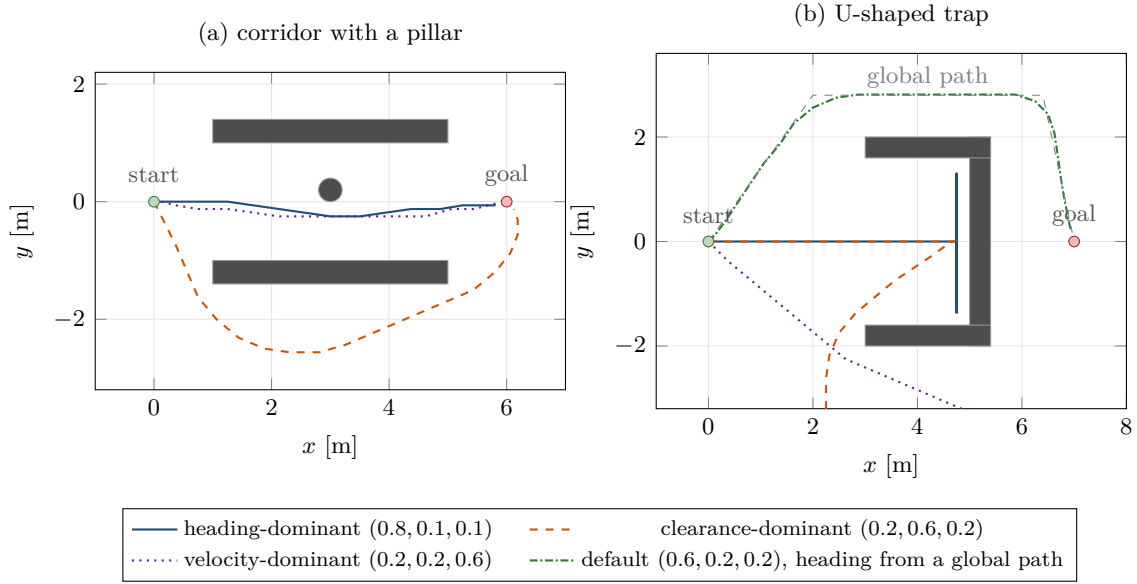


Figure 14.6. Closed-loop trajectories, the first 80 steps (20 s) of each run. (a) The heading- and velocity-dominant runs thread the corridor and pass the pillar; the clearance-dominant run refuses the corridor mouth and flies around the outside of the lower wall. (b) In the U-trap the heading-dominant drone enters and shuttles between the arms for ever; the clearance-dominant drone turns away from all obstacles and leaves the plot; the velocity-dominant drone rushes through, turns at the back wall and disappears at full speed. The fourth curve (dash-dotted green) is the default weights with the heading term aimed at a lookahead point on the global path; it is the only run that reaches the goal. Panel (a) uses the same colors and line styles as the legend of (b).

Clearance dominates. With (0.2, 0.6, 0.2) the drone is timid, and the margin that turns it away is thin. At the start the straight command (0.5, 0) sees the wall and the pillar ahead (dist = 2.65 m, clearance 0.885) and scores $0.2 + 0.531 + 0.05 = 0.781$, while (0.25, ± 0.5), which turn out of the corridor mouth into the open, see nothing within $d_{\max}$ (clearance 1) and score 0.784 despite a heading of only 0.641: β gains $0.6 \cdot 0.115 = 0.069$ and γ another 0.006 against the $0.2 \cdot 0.359 = 0.072$ that α loses. Three thousandths decide it, and the drone turns away, flies around the outside of the lower wall and reaches the goal from below, 47% longer than the 5.9 m of the default run. In the U-trap it turns around before the U and flies away from everything, ending 141 m from the start. Clearance-dominant DWA keeps the widest berth but avoids narrow passages and, given room, prefers it to the goal.

Velocity dominates. With (0.2, 0.2, 0.6) the drone wants to be fast above all. In the corridor this is harmless, the fastest admissible commands also lead to the goal, and it is the quickest run. In the U-trap it enters, is forced by the braking test to turn at the back wall, and then keeps flying at full speed wherever there is room, never returning: the heading term is too weak to bring it back. Velocity-dominant DWA wanders.

The lesson. No fixed weighting suits all scenes. The heading term should be the largest, as in the original paper (Fox et al. report $\alpha = 2.0$, $\beta = 0.2$, $\gamma = 0.2$), and the trap must be fixed elsewhere: with the default weights and the heading term aimed at a lookahead point on a global path around the U, the drone reaches the goal in 25 steps with a minimum clearance of 0.61 m (last row of Table 14.4). Weights tune behavior; the global path fixes topology.

14.8 Variants and extensions

A global path for the heading term. The standard remedy for local minima is not to ask DWA to find the way at all. A global planner (A^* of Chapter 4 on a grid, D^* Lite of Chapter 5 when the map changes, or the nominal multi-agent path finding (MAPF) path of Chapters 9 and 10) produces a path, and the heading term chases a lookahead point on it (line 5 of Algorithm 14.2). DWA then handles only what the global planner did not know about. Brock and Khatib [25] made this precise as the *global dynamic window approach*: the heading term is replaced by the decrease of a navigation function, a grid distance-to-goal computed by a wavefront as in Chapter 3, which has no local minima by construction. The lookahead distance L matters: too short and the drone cuts corners into obstacles the path avoided (the braking test still saves it, but it stalls); too long and the target lies behind a wall so that the heading term again points at the wall. One to two seconds of flight, $L \approx 1\text{--}3$ m here, is a good default.

A longer horizon. DWA evaluates rollouts of a single constant velocity. Simulating each candidate for a longer horizon T and scoring the whole predicted trajectory lets the objective see around the next corner and reduces oscillation, at the price of more computation and of trusting that the command stays constant. Planners that roll out short command sequences, and the model predictive controllers of Chapter 21, continue this idea: MPC is DWA with the window replaced by a constrained optimization over the horizon.

Hysteresis and smoothing. The zigzag between two nearly equal candidates is removed by **hysteresis**: keep the current command unless the best candidate beats it by a margin η (line 8). The smoothing σ of Equation (14.8) acts in the same direction in velocity space, and a penalty on the change $\|\mathbf{v} - \mathbf{v}_a\|$ is a third form. All three damp the oscillation; none removes a trap, since they only change which of two bad commands is chosen.

Moving obstacles. Theorem 14.12 assumes static obstacles, and the rollout of Algorithm 14.1 treats every obstacle as static even when it is a drone crossing at 1.5 m/s. The repair is the relative-velocity view of Chapter 12: if an obstacle moves with velocity $\mathbf{u}$, the rollout of $\mathbf{v}$ collides with it when the ray in direction $\mathbf{v} - \mathbf{u}$ meets the inflated disk, and the free distance along the drone's own path is $\|\mathbf{v}\|$ times that collision time. `free_distance()` in `ch14_dwa.py` does this for disks with a known velocity, which the prediction layer of Chapters 18 and 20 supplies. The self-test contains the corresponding pitfall: an intruder that starts at (3.5, 3.0) and flies down at 1.5 m/s across the drone's route from (0, 0) to (7, 0). With the static view, refreshed every step but never extrapolated, the drone is hit in step 7; with the relative-velocity rollout it slows down, lets the intruder pass and reaches the goal in 18 steps with 4 cm of clearance at the closest point.

Three dimensions. Nothing in the drone form is two-dimensional: the window becomes a cube of $(2n + 1)^3$ candidates, the rollouts are rays in space, and the ray casts against balls and boxes are unchanged. The self-test flies the same open-field scene in 2D and 3D and reaches the goal in 17 and 16 steps.

Guarantees of convergence. Ögren and Leonard [120] showed that a DWA-like scheme can be made *convergent*, that is, provably reach the goal, by taking the objective from a control Lyapunov function and treating the window as the constraint set of a receding-horizon controller, the bridge to the MPC of Chapter 21. Surveys of local planners are found in [152] and [28].

Table 14.5. The three local methods of Part IV compared. M is the number of obstacles, N_c the number of DWA candidates, n the number of ORCA half-planes.

	DWA (this chapter)	ORCA (Chapter 13)	Potential fields (Chapter 15)
Searches over	velocities reachable in one interval (the window)	all velocities outside a union of half-planes	nothing: it follows $-\nabla U$
Assumes	obstacle geometry near the robot; acceleration and speed limits; obstacles static during the rollout unless predicted	the others run the same rule and share the effort; velocities change instantly; their positions, velocities and radii are known	a potential over positions with obstacles as repulsive sources
Guarantees	no collision with static obstacles while a braking command is admissible (Theorem 14.12); feasible commands; not complete	pairwise collision-free for the horizon τ if all reciprocate and the program is feasible; not complete	none in general; local minima and oscillation
Cost per step	$\mathcal{O}(N_c M)$, $N_c = 25\text{--}1\,331$	a linear program in n half-planes	$\mathcal{O}(M)$ gradient evaluation
Best for	tight dynamics, unknown or non-cooperative obstacles, sensor-based avoidance	many cooperative agents of one kind	a cheap steering term, or a first cut

Table 14.5 summarizes: ORCA is the tool when the obstacles are other ORCA agents, DWA when they are anything else and the vehicle cannot jump in velocity, and potential fields when you need a steering term in a hurry and can live without guarantees.

14.9 Implementation notes

Resolution of the velocity grid. The grid spacing Δv sets the cost, $(2n+1)^{\dim}$ candidates, and the quality: a narrow admissible corridor between two inadmissible regions can fall between grid points, and the drone then stops in front of a gap it could pass. Keep $2n+1 \geq 5$ and refine the grid near the boundary of the admissible set if narrow passages matter.

Cost of rollouts and free-distance queries. For disks and boxes the ray cast is exact and costs $\mathcal{O}(M)$; it is the right choice for the obstacle lists of the prediction layer. For an occupancy grid or a point cloud, march along the ray in steps smaller than the smallest obstacle feature, or read dist off a distance transform computed once per scan. The differential drive marches along the arc of Equation (14.1) (`diffdrive_free_distance()`, checked against the ray cast for $\omega = 0$ in the self-test). Whatever the method, the query must be *conservative*: Theorem 14.12 tolerates underestimated free distances, the drone is merely timid, but not overestimated ones. Inflating boxes as boxes, as the code does, is conservative at the corners.

The global path as heading reference. Keep the path as a polyline, project the current position onto it and take the point at arc length L beyond the projection as the target (`lookahead_point()`). The projection follows a drone that has been pushed off the path, so it reconnects further along instead of backtracking; if the path is replanned (Chapter 5), replace the polyline: DWA has no state to update.

Listing 14.1. The candidate grid over the dynamic window (from `code/ch14_dwa.py`); the helpers `dynamic_window()` and `braking_candidate()` implement Equation (14.7) and line 4 directly.

```

1 def window_candidates(v_a, prm):
2     """Velocity grid over the dynamic window, intersected with V_s, plus
3     the braking candidate (appended if it is not a grid point already)."""
4     lo, hi = dynamic_window(v_a, prm)
5     n = int(round(prm.a_max * prm.dt / prm.v_res))
6     axes = [v_a[k] + prm.v_res * np.arange(-n, n + 1) for k in range(len(v_a))]
7     grid = np.array(np.meshgrid(*axes, indexing="ij")).reshape(len(v_a), -1).T
8     inside = np.all((grid >= lo - 1e-9) & (grid <= hi + 1e-9), axis=1)
9     grid = grid[inside]
10    grid = grid[np.linalg.norm(grid, axis=1) <= prm.v_max + 1e-9]
11    v_b = braking_candidate(v_a, prm)
12    if not np.any(np.all(np.abs(grid - v_b) < 1e-9, axis=1)):
13        grid = np.vstack([grid, v_b])
14    return grid

```

Listing 14.1 builds the candidate set of line 3. Listing 14.2 scores the candidates and picks the command: the admissible speed of Equation (14.6) is `admissible_speed(dist, a_brake, dt)`, called with the delay `prm.brake_delay`, which equals Δt unless it is set to 0 to reproduce the continuous test of Fox et al. (the self-test does so for the wall of Section 14.3.2). The free-distance function on line 12 ray-casts against disks and boxes and switches to the relative-velocity ray when a disk carries a velocity.

Listing 14.2. Scoring and selection (from `code/ch14_dwa.py`).

```

1 def evaluate(p, v_a, obstacles, target, prm, goal=None):
2     """Score every candidate of the window; returns a list of Candidate."""
3     alpha, beta, gamma = prm.weights
4     a_b, delay = prm.a_brake, prm.brake_delay # braking test parameters
5     d_goal = None if goal is None else float(np.linalg.norm(goal - p))
6     out = []
7     for v in window_candidates(v_a, prm):
8         speed = float(np.linalg.norm(v))
9         if speed < 1e-9:
10            dist = prm.d_max # standing still is safe (static obstacles)
11        else:
12            dist = free_distance(p, v / speed, obstacles, prm.radius,
13                               prm.d_max, v)
14            limit = admissible_speed(dist, a_b, delay)
15            if prm.brake_for_goal and d_goal is not None: # and for the goal
16                limit = min(limit, admissible_speed(d_goal, a_b, delay))
17            adm = speed <= limit + 1e-9
18            h = heading_term(p, v, target, prm)
19            c = min(dist, prm.d_max) / prm.d_max
20            s = speed / prm.v_max
21            g = alpha * h + beta * c + gamma * s if adm else 0.0
22            out.append(Candidate(v, dist, adm, h, c, s, g))
23    if prm.smooth:
24        _smooth_scores(out, prm)
25    return out
26
27
28 def dwa_command(p, v_a, obstacles, target, prm, goal=None):
29     """One DWA step: the admissible window velocity with the largest G, or
30     the braking candidate if nothing is admissible (emergency braking).
31     Returns (v_best, candidates)."""

```



```

32     cands = evaluate(p, v_a, obstacles, target, prm, goal)
33     adm = [c for c in cands if c.admissible]
34     if not adm:
35         return braking_candidate(v_a, prm), cands
36     best = max(adm, key=lambda c: c.score)
37     if prm.hysteresis > 0.0:  # keep v_a unless beaten by the margin
38         for c in adm:
39             if np.allclose(c.v, v_a) and c.score >= best.score - prm.hysteresis:
40                 return c.v, cands
41     return best.v, cands

```

Forgetting the braking constraint

The heading and velocity terms both reward flying fast toward the goal, and an implementation that scores every window candidate without the admissibility test flies the drone into the first obstacle on its line: the clearance term only lowers a score, it forbids nothing. The braking test is the only hard constraint in DWA. Implement it first, test it with a wall directly ahead, and use its discrete-time form [Equation \(14.6\)](#).

Scoring in different units

Adding an angle in radians, a distance in meters and a speed in meters per second with weights like (0.6, 0.2, 0.2) produces a number whose meaning changes with the map units and the vehicle. Normalize each term to $[0, 1]$ by its maximum (π , $d_{\max}$, $v_{\max}$) before weighting, and note that $d_{\max}$ is part of the tuning: doubling it halves every clearance score. [Exercise 14.6](#) shows that in centimeters the goal stops mattering altogether: the heading term contributes less than one percent of the score.

A window too small to react

The window spans $\pm a_{\max} \Delta t$ around the current velocity. With a short control interval or a small acceleration limit this can be less than one grid step, and the only candidate left is $\mathbf{v}_a$ itself: the drone keeps its velocity whatever happens. Even with a fine grid, a drone at $v_{\max}$ needs $v_{\max}/(a_{\max} \Delta t)$ steps to stop, four steps or one second in the examples, which is why the braking test looks $v \Delta t + v^2/(2a_b)$ ahead and not one step. Check $a_{\max} \Delta t \geq \Delta v$ at start-up ([Exercise 14.2](#)), and size $a_{\max}$ by what the velocity controller really delivers, not by the datasheet.

Treating moving obstacles as static during the rollout

A rollout against a snapshot of the obstacles is a rollout against a world that has stopped. An intruder crossing the route at 1.5 m/s is far from the ray when the command is chosen and in the way when it is executed; the self-test shows the collision in step 7. Use the predicted velocity of every tracked obstacle in the rollout (relative-velocity ray casting, or time-indexed rollouts), inflate obstacles by their prediction uncertainty as [Chapter 24](#) describes, and keep the horizon short, since predictions degrade quickly.

14.10 Where this fits in the drone system

In the drone system

In the four-layer architecture of [Chapter 24](#), DWA is the alternative local safety layer to ORCA. It is the better choice when the drone's acceleration limits matter more than reciprocity: a heavy or fast drone whose velocity controller cannot jump to an ORCA velocity within one interval, or an intruder that does not cooperate at all. In the drone form the window is a cube of achievable velocities (v_x, v_y, v_z) , the rollouts are straight segments, and the braking test guarantees that the drone can always stop for the obstacles it knows about.

DWA receives three things. From the global layer—conflict-based search (CBS) or its bounded-suboptimal variant ECBS, [Chapters 9](#) and [10](#)—it receives the nominal path, which supplies the heading term through a lookahead point; this keeps a locally avoiding drone heading back to its route and out of the U-trap. From the prediction layer ([Chapters 18](#) and [20](#)) it receives the positions and velocities of the tracked obstacles, which enter the rollouts as relative-velocity ray casts, inflated by their uncertainty. From the trigger logic of [Chapter 24](#) it receives control when a predicted conflict enters the safety horizon, and hands it back once the conflict has cleared and the drone has reconnected to the path. If the lookahead point becomes unreachable, DWA reports it and the replanning layer (D^* Lite or A^* , [Chapters 4](#) and [5](#)) plans anew from the current state. The study plan covers DWA and potential fields in Week 7 ([Appendix A](#)); its coding exercise is [Exercise 14.8](#).

14.11 Summary

Chapter summary

- DWA searches the velocity space, not the workspace: candidates are velocity commands held for one control interval, and each is judged by a short simulated rollout (an arc for a differential drive, a straight segment for a velocity-controlled drone).
- The search set is $V_r = V_s \cap V_a \cap V_d$: possible velocities, admissible velocities (the braking distance $v^2/(2a_b)$, in discrete time $v\Delta t + v^2/(2a_b)$, fits into the free distance), and the dynamic window of velocities reachable within Δt under the acceleration limits.
- The objective $G = \sigma(\alpha \text{ heading} + \beta \text{ clearance} + \gamma \text{ velocity})$ must be computed from normalized terms; the heading term is aimed at the goal or, better, at a lookahead point on a global path.
- With static obstacles and the braking candidate always available, DWA never collides and every command is dynamically feasible ([Theorem 14.12](#)). It is not complete: it can rest in a local minimum ([Proposition 14.13](#)) or oscillate in a U-trap.
- Weights tune behavior (heading: straight and bold; clearance: timid; velocity: fast and wandering); a global path fixes the topology; hysteresis and smoothing damp oscillation; predicted obstacle velocities repair the rollout for moving obstacles. Compared with ORCA, DWA respects dynamics and needs no cooperation but is greedy; compared with potential fields it has a hard safety constraint.

Further reading. The original paper [46] is short and readable; the global dynamic window approach [25] adds the navigation function and [120] the convergence proof. Textbook treatments of DWA and the other local methods are in [28] and [152]; the potential-field method that DWA was designed to improve on is due to Khatib [83] (Chapter 15), and ORCA to van den Berg et al. [14].

14.12 Exercises

Exercise 14.1 (★). A drone flies at 2 m/s straight at a wall and can brake at $a_b = 2 \text{ m/s}^2$. (a) How far does it travel before it stops if braking starts immediately? (b) How far if the current command is held for $\Delta t = 0.25 \text{ s}$ first? (c) The wall is 0.608 m away: what is the largest admissible speed under Equation (14.3) and under Equation (14.6)? (d) Explain in one sentence why the answer to (c) differs although the physics is the same.

Exercise 14.2 (★). With $a_{\max} = 2 \text{ m/s}^2$, $\Delta t = 0.25 \text{ s}$ and $\Delta v = 0.25 \text{ m/s}$, how many grid points does the window have in 2D and in 3D? How many with $\Delta v = 0.1 \text{ m/s}$? Now set $\Delta t = 0.1 \text{ s}$ and keep $\Delta v = 0.25 \text{ m/s}$: what does the candidate set of Listing 14.1 contain, and what does the drone do?

Exercise 14.3 (★★). Using the terms in Table 14.2, recompute G for the candidates (1.0, 0), (1.0, 0.5) and (0.75, 0.5) with the weights (0.8, 0.1, 0.1) and with (0.2, 0.6, 0.2). Which of the three wins in each case? Explain the result in terms of the behaviors of Section 14.7, and then explain why a heading-dominant drone that keeps (1.0, 0) still does not crash.

Exercise 14.4 (★★). Derive Equation (14.1) from $\dot{x} = v \cos \theta$, $\dot{y} = v \sin \theta$, $\dot{\theta} = \omega$ with constant v and ω , and show that the trajectory is a circle of radius v/ω through the start pose. Compute the pose after 2 s for $(v, \omega) = (1 \text{ m/s}, 0.5 \text{ rad/s})$ from (0, 0, 0) and compare with `diffdrive_rollout()`. Why does the heading term of Fox et al. need the *predicted* pose for a differential drive but hardly for a drone?

Exercise 14.5 (★★★). (a) Show that the braking candidate also lies in a Euclidean-ball window $\|\mathbf{v} - \mathbf{v}_a\| \leq a_{\max} \Delta t$. (b) Where exactly does the proof of Theorem 14.12 use that the obstacles are static? Construct an obstacle motion for which the braking candidate is inadmissible in the next step. (c) If the free-distance query overestimates `dist` by up to ε , how must Equation (14.6) change so that the theorem survives?

Exercise 14.6 (★★). An implementation forgets to normalize: it scores $G = 0.6 \text{ heading} + 0.2 \text{ dist} + 0.2 \|\mathbf{v}\|$ with `dist` in meters and $\|\mathbf{v}\|$ in m/s. (a) Which of the 16 admissible candidates of Example 14.10 wins? (`evaluate()` gives you all the numbers.) (b) Now the map is in centimeters and speeds in cm/s: which candidate wins, and what fraction of its score comes from the heading term? Construct a scene in which this drone flies away from its goal. (c) Show in general that changing the length unit by a factor λ multiplies the effective weights β and γ by λ but leaves α unchanged, and explain why the normalized form Equation (14.8) is invariant.

Exercise 14.7 (★★). A drone flies straight at a disk obstacle centered exactly on the line to the goal. (a) Explain why the two mirror-image candidates $(v_x, \pm v_y)$ have identical scores and why, after the first step breaks the tie, small changes of the geometry can make the winner alternate from step to step. (b) Show that the hysteresis rule of line 8 with margin η stops the alternation as soon as the two scores differ by less than η . (c) Explain why no value of η makes the drone leave the U-trap of Figure 14.5, and check your claim with `DwaParams(hysteresis=0.05)` in `utrap_scene()`.

Exercise 14.8 (★★★ Coding). This is the Week 7 exercise of the study plan (Appendix A). Implement DWA in a continuous 2D environment, from `ch14_dwa.py` or from scratch, and

create failure cases: (i) a U-trap of your own dimensions; (ii) a gap between two obstacles that the velocity grid misses at $\Delta v = 0.25$ m/s and finds at 0.05 m/s; (iii) an obstacle exactly on the goal line, to provoke oscillation; (iv) an intruder crossing the route, treated as static in the rollout. For each case record whether the goal is reached, the path length, the minimum clearance and the number of steps; then repair each case with one remedy from [Section 14.8](#) (a global path from A^* of [Chapter 4](#), a finer grid, hysteresis, the relative-velocity rollout) and report the same metrics. Finally extend the controller to candidates (v_x, v_y, v_z) , fly around a sphere, and measure the time per step for $\Delta v = 0.25$ and 0.1 m/s.

Artificial Potential Fields

Learning objectives

- Explain how a potential that is low at the goal and high near obstacles turns navigation into gradient descent, and write down Khatib's attractive and repulsive potentials with their forces.
- Derive the forces from the potentials, evaluate them by hand at a given point, and trace the discrete-time controller with a velocity limit.
- Reproduce the four classic failure modes – local minima, goals that cannot be reached next to obstacles, oscillation, and blocked passages – and explain each with a formula, not only with a picture.
- Apply the standard remedies, state what navigation functions and harmonic potentials guarantee, and use a potential field as a local layer under a global plan.
- Extend the field to several drones with inter-agent repulsion and relate it to Reynolds's flocking rules and to consensus.

15.1 Why this matters for a drone swarm

A drone of the swarm is following the path that the global planner of [Chapter 9](#) assigned to it when a bird, or a drone that is not part of the swarm, appears three meters ahead. The on-board computer has perhaps ten milliseconds per control cycle, and in that time it must produce a velocity command that moves the drone away from the intruder and, as soon as it can, back toward its goal. The oldest and simplest answer is the **artificial potential field** (APF) of Khatib [83]: imagine a landscape over the map that slopes down toward the goal and rises steeply around every obstacle, and let the drone roll downhill. The command is the negative gradient of that landscape; per obstacle it costs one distance query and one square root, with no search, no queue and no optimization.

The method has an equally old list of defects. Five years after Khatib, Koren and Borenstein [91] cataloged the situations in which a potential field traps the robot, refuses to let it through a doorway, or makes it oscillate against a wall. Every one of these failures has a precise mathematical cause, and in this chapter you will produce each of them on purpose, in a few lines of Python, and see what the cause is. That is what the Week 7 milestone of the study plan ([Appendix A](#)) asks for: *know when local methods work and when they fail*. The week pairs this chapter with the dynamic window approach (DWA) of [Chapter 14](#), and its coding exercise ([Exercise 15.9](#)) asks you to implement one of the two and to construct its failure cases. In the hybrid architecture of [Chapter 24](#) the safety layer is ORCA or the dynamic window approach, not a potential field; [Section 15.11](#) explains why, and where potential fields survive.

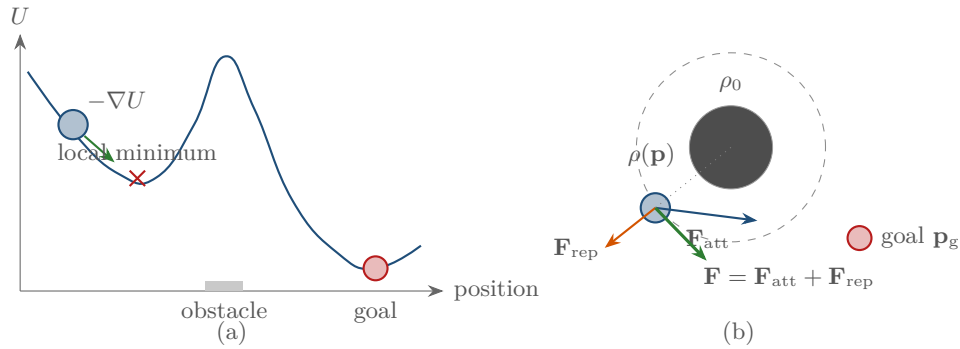


Figure 15.1. (a) A one-dimensional potential: the goal is the global minimum, the obstacle a peak, and the drone follows the negative gradient. The dip to the left of the obstacle is a local minimum; a drone released there never leaves it. (b) In the plane the attractive force $\mathbf{F}_{\text{att}}$ points at the goal, the repulsive force $\mathbf{F}_{\text{rep}}$ points away from the nearest point of the obstacle, and the command is their sum; the repulsion is zero outside the dashed circle of radius ρ_0 .

The chapter proceeds as follows. Sections 15.2 and 15.3 give the picture, the potentials and the derivations; Sections 15.4 and 15.5 state the controller and evaluate the forces by hand; Section 15.6 collects what can be proved. Section 15.7 is the heart of the chapter: the four failure modes, each with a runnable example, and the Koren–Borenstein critique. Sections 15.8 and 15.9 present the remedies, the basin experiment and the extension to swarms, and Sections 15.10 and 15.11 cover the implementation and the place of the method in the drone system.

15.2 The idea in plain words

Think of a marble on a hilly table (Figure 15.1a). The table is shaped so that the goal is the lowest point and every obstacle sits on top of a steep hill. Released anywhere, the marble rolls downhill: away from the hills, toward the goal. The height of the table is the **potential** $U(\mathbf{p})$, a scalar function of the position; the direction of steepest descent at $\mathbf{p}$ is the negative gradient $-\nabla U(\mathbf{p})$, and this vector is the **force** that the field exerts on the drone. The drone does not plan a path; at every control cycle it evaluates one vector and follows it.

The potential is built from two kinds of terms (Figure 15.1b). The *attractive* term is a bowl centered at the goal; its gradient points straight at the goal, and the further away the drone is, the harder it pulls. Each *repulsive* term is a wall around one obstacle; it is zero beyond an *influence distance* ρ_0 , grows as the drone approaches the obstacle, and becomes infinite on its boundary, so that the drone can never touch it. Forces are vectors, so they add: the command is the sum of the attraction and of the repulsions of every obstacle within range. That is the whole method. Everything else in this chapter is about the two words that the picture hides. Downhill is a *local* notion: the marble finds the lowest point of the valley it is in, which need not be the goal. And the drone is not a marble in continuous time but a computer that takes finite steps.

Key idea

Add a bowl at the goal to a wall around every obstacle and move against the gradient of the sum. The command is cheap and smooth, and in continuous time it never touches an obstacle. But the gradient only sees the slope under the drone's feet: the drone stops wherever attraction and repulsion cancel, and that point is the goal only if you were lucky with the scene.

15.3 Problem statement and notation

Definition 15.1 (Potential-field navigation problem). The drone is a point moving in a workspace $\mathcal{W} \subseteq \mathbb{R}^2$ (or $\mathbb{R}^3$); its physical radius is absorbed by inflating the obstacles, as in [Chapter 2](#). The obstacles $O_1, \dots, O_m$ are closed sets, the free space is $\mathcal{C}_{\text{free}} = \mathcal{W} \setminus \bigcup_i O_i$, and the goal is a point $\mathbf{p}_g \in \mathcal{C}_{\text{free}}$. The **clearance** to obstacle i is the distance

$$\rho_i(\mathbf{p}) = \min_{\mathbf{o} \in O_i} \|\mathbf{p} - \mathbf{o}\|, \quad (15.1)$$

and $\nabla \rho_i(\mathbf{p})$ is its gradient, the unit vector that points from the closest point of O_i toward $\mathbf{p}$ (defined wherever that closest point is unique). A **potential** is a continuously differentiable function $U: \mathcal{C}_{\text{free}} \rightarrow \mathbb{R}_{\geq 0}$; its **force** is $\mathbf{F}(\mathbf{p}) = -\nabla U(\mathbf{p})$. The task is to choose U so that a drone that follows $\mathbf{F}$ from a given start reaches $\mathbf{p}_g$ without leaving $\mathcal{C}_{\text{free}}$.

For a disk with center $\mathbf{c}$ and radius r the clearance is $\rho(\mathbf{p}) = \|\mathbf{p} - \mathbf{c}\| - r$ and $\nabla \rho = (\mathbf{p} - \mathbf{c}) / \|\mathbf{p} - \mathbf{c}\|$; for a segment it is the point-to-segment distance of [Chapter 2](#) and $\nabla \rho$ points away from the closest point. The code of this chapter uses disks; any convex obstacle is handled by the same two quantities, a distance and a unit vector.

Definition 15.2 (Attractive potential). Let $d(\mathbf{p}) = \|\mathbf{p} - \mathbf{p}_g\|$ and $k_{\text{att}} > 0$. The **quadratic attractive potential** and its force are

$$U_{\text{att}}(\mathbf{p}) = \frac{1}{2} k_{\text{att}} d(\mathbf{p})^2, \quad \mathbf{F}_{\text{att}}(\mathbf{p}) = -\nabla U_{\text{att}}(\mathbf{p}) = -k_{\text{att}} (\mathbf{p} - \mathbf{p}_g). \quad (15.2)$$

The **conic** potential is $U_{\text{att}} = k_{\text{att}} d^* d$ for a constant $d^* > 0$, with a force of constant magnitude $k_{\text{att}} d^*$. The **hybrid** potential is quadratic for $d \leq d^*$ and conic beyond:

$$U_{\text{att}}(\mathbf{p}) = \begin{cases} \frac{1}{2} k_{\text{att}} d^2 & d \leq d^*, \\ k_{\text{att}} d^* d - \frac{1}{2} k_{\text{att}} (d^*)^2 & d > d^*, \end{cases} \quad \mathbf{F}_{\text{att}}(\mathbf{p}) = \begin{cases} -k_{\text{att}} (\mathbf{p} - \mathbf{p}_g) & d \leq d^*, \\ -k_{\text{att}} d^* (\mathbf{p} - \mathbf{p}_g) / d & d > d^*. \end{cases} \quad (15.3)$$

The gradient of $\frac{1}{2}d^2$ is $\mathbf{p} - \mathbf{p}_g$ and the gradient of d is the unit vector $(\mathbf{p} - \mathbf{p}_g)/d$, which gives the two forces. Each form has one defect: the quadratic force grows without bound with the distance, so that a drone far from its goal but next to an obstacle is pulled through the repulsion, while the conic force is bounded but discontinuous at the goal, where the unit vector flips, so a drone under it chatters across the goal. The hybrid form takes the bounded force far away and the smooth bowl near the goal; the constant $-\frac{1}{2}k_{\text{att}}(d^*)^2$ makes U_{att} continuous at $d = d^*$ and both branches give a force of magnitude $k_{\text{att}}d^*$ there ([Exercise 15.2](#)); it is the form recommended by Choset et al. [28]. Away from obstacles, clipping the quadratic force at v_{max} , as the controller of [Section 15.4](#) does, gives the same command as the hybrid potential with $d^* = v_{\text{max}}/k_{\text{att}}$ ([Exercise 15.2](#)); inside an influence region the two differ, because the controller clips the *sum* while the hybrid caps the attraction before the repulsion is added: from (0, 4.5) in [Example 15.5](#) the clipped field keeps a clearance of 0.519, the hybrid one with $d^* = 1$ a clearance of 0.840. The examples use the quadratic bowl, $k_{\text{att}} = 1$, $v_{\text{max}} = 1$.

Definition 15.3 (Repulsive potential, Khatib 1986). Let $k_{\text{rep}} > 0$ and let $\rho_0 > 0$ be the **influence distance**. The **repulsive potential** of obstacle i and its force are

$$U_{\text{rep},i}(\mathbf{p}) = \begin{cases} \frac{1}{2} k_{\text{rep}} \left(\frac{1}{\rho_i(\mathbf{p})} - \frac{1}{\rho_0} \right)^2 & \rho_i(\mathbf{p}) \leq \rho_0, \\ 0 & \rho_i(\mathbf{p}) > \rho_0, \end{cases} \quad (15.4)$$

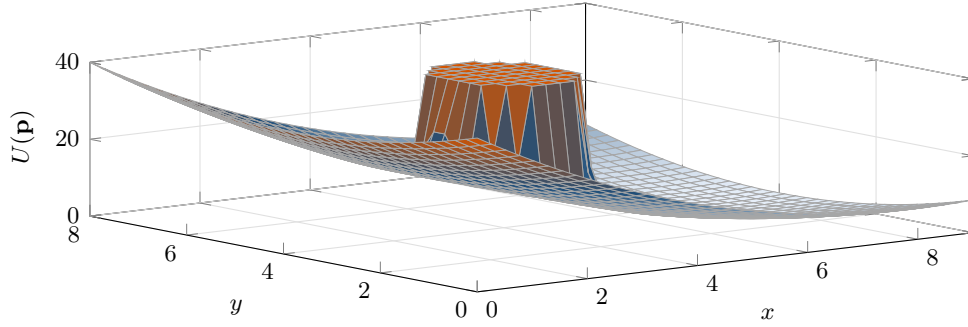


Figure 15.2. The total potential U of the one-disc scene (goal at $(8, 4)$, disk of radius 1 at $(4, 4)$, $k_{\text{att}} = k_{\text{rep}} = 1$, $\rho_0 = 2$), clipped at $U = 40$. The quadratic bowl is centered at the goal; the repulsive term is zero beyond $\rho_0 = 2$ from the disk and rises to infinity at its boundary. Notice how gentle the bowl is compared with the chimney: close to the obstacle the attraction is negligible, far from it the obstacle is invisible.

$$\mathbf{F}_{\text{rep},i}(\mathbf{p}) = -\nabla U_{\text{rep},i}(\mathbf{p}) = \begin{cases} k_{\text{rep}} \left(\frac{1}{\rho_i(\mathbf{p})} - \frac{1}{\rho_0} \right) \frac{1}{\rho_i(\mathbf{p})^2} \nabla \rho_i(\mathbf{p}) & \rho_i(\mathbf{p}) \leq \rho_0, \\ \mathbf{0} & \rho_i(\mathbf{p}) > \rho_0. \end{cases} \quad (15.5)$$

The derivation is one application of the chain rule. Inside the influence region write $g(\mathbf{p}) = 1/\rho_i(\mathbf{p}) - 1/\rho_0$, so that $U_{\text{rep},i} = \frac{1}{2}k_{\text{rep}}g^2$ and $\nabla U_{\text{rep},i} = k_{\text{rep}}g \nabla g$. Since $\nabla(1/\rho_i) = -\rho_i^{-2}\nabla \rho_i$,

$$\nabla U_{\text{rep},i} = k_{\text{rep}} \left(\frac{1}{\rho_i} - \frac{1}{\rho_0} \right) \left(-\frac{1}{\rho_i^2} \right) \nabla \rho_i,$$

and the force is the negative of this. It points along $\nabla \rho_i$, straight away from the closest point of the obstacle, and its magnitude $k_{\text{rep}}(1/\rho_i - 1/\rho_0)/\rho_i^2$ behaves like k_{rep}/ρ_i^3 as $\rho_i \rightarrow 0$: the wall is infinitely high and infinitely steep at the obstacle boundary. At $\rho_i = \rho_0$ both the potential and the force are zero and join continuously with the zero outside. Khatib called this the FIRAS potential (*force involving artificial repulsion from the surface*); the subtraction of $1/\rho_0$ limits the influence of an obstacle to its neighborhood.

Definition 15.4 (Total potential, critical points). The total potential and the total force are

$$U(\mathbf{p}) = U_{\text{att}}(\mathbf{p}) + \sum_{i=1}^m U_{\text{rep},i}(\mathbf{p}), \quad \mathbf{F}(\mathbf{p}) = \mathbf{F}_{\text{att}}(\mathbf{p}) + \sum_{i=1}^m \mathbf{F}_{\text{rep},i}(\mathbf{p}). \quad (15.6)$$

A point with $\mathbf{F}(\mathbf{p}) = \mathbf{0}$ is a **critical point** of U . It is a **local minimum** if U does not decrease in any direction from it, and a **saddle** if U decreases in some directions and increases in others. The **basin** of a local minimum is the set of starts from which the descent ends there.

Figure 15.2 shows the total potential of the scene used throughout the chapter: one disk of radius 1 centered at $(4, 4)$, the goal at $(8, 4)$, $k_{\text{att}} = k_{\text{rep}} = 1$ and $\rho_0 = 2$. What the surface does not show at this scale is the feature that matters most: on the line from the far side of the disk to the goal, the slopes of the bowl and of the chimney cancel at one point, a critical point that is not the goal (Section 15.7.1).

15.4 The algorithm

The method has two parts: the evaluation of the force at a point (Algorithm 15.1) and the loop that turns the force into motion (Algorithm 15.2). Algorithm 15.1 is Equations (15.3),

(15.5) and (15.6) written out, with one addition at line 12: the optional factor d^n of Ge and Cui [52] that cures the second failure mode of Section 15.7. With $n = 0$ the line is skipped and the function computes Khatib's original force.

Algorithm 15.1. The potential-field force at a point: hybrid attraction, Khatib repulsion, optional GNRON factor

Input: position $\mathbf{p}$, goal $\mathbf{p}_g$, obstacles $O_1, \dots, O_m$ with distance queries; gains $k_{\text{att}}, k_{\text{rep}}$, influence distance ρ_0 , switch distance d^* (∞ for a purely quadratic bowl), exponent $n \geq 0$ ($n = 0$ for Khatib's original)

Output: the force $\mathbf{F}(\mathbf{p}) = -\nabla U(\mathbf{p})$

```

1 Function ApfForce( $\mathbf{p}, \mathbf{p}_g, O_{1..m}$ ):
2    $\mathbf{d} \leftarrow \mathbf{p} - \mathbf{p}_g$ ;  $d \leftarrow \|\mathbf{d}\|$ 
3   if  $d \leq d^*$  then
4      $\mathbf{F} \leftarrow -k_{\text{att}} \mathbf{d}$                                 // quadratic bowl
5   else
6      $\mathbf{F} \leftarrow -k_{\text{att}} d^* \mathbf{d}/d$                         // conic far away: constant magnitude
7   foreach obstacle  $O_i$  do
8      $(\rho, \nabla \rho) \leftarrow \text{Distance}(O_i, \mathbf{p})$            // clearance and unit vector away from  $O_i$ 
9     if  $\rho < \rho_0$  then
10       $\mathbf{F}_i \leftarrow k_{\text{rep}} (1/\rho - 1/\rho_0) \rho^{-2} \nabla \rho$ 
11      if  $n \geq 1$  then
12         $\mathbf{F}_i \leftarrow d^n \mathbf{F}_i - \frac{n}{2} k_{\text{rep}} (1/\rho - 1/\rho_0)^2 d^{n-1} \mathbf{d}/d$  // GNRON correction,
13         $\mathbf{F} \leftarrow \mathbf{F} + \mathbf{F}_i$                                Section 15.7.2
14 return  $\mathbf{F}$ 

```

Walkthrough. Line 4 of Algorithm 15.1 is the attraction; with $d^* = \infty$ it is always the quadratic branch. Line 8 is the only geometric operation of the whole method, a query for the clearance to one obstacle and the unit vector away from its closest point; an obstacle further away than ρ_0 contributes nothing, so a spatial index that returns only the obstacles within ρ_0 makes the loop independent of the size of the map. Line 13 adds the vectors, which is where all the trouble of Section 15.7 originates, because a sum of vectors can be zero. Algorithm 15.2 is explicit Euler integration of $\dot{\mathbf{p}} = \mathbf{F}(\mathbf{p})$ with step Δt (line 12), guarded by three tests: the goal tolerance ε_g (line 4), needed because the quadratic bowl makes the drone approach the goal exponentially and never arrive exactly (Exercise 15.2); the collision test (line 6), which cannot fire in continuous time (Theorem 15.7) but can in discrete time (Section 15.7.3); and the **local-minimum detector** of line 8, which declares the drone stuck if it has moved less than δ during the last W steps – the only way a gradient method can notice a local minimum, since the force there is zero and the drone simply stops.

Line 11 is the **velocity limit**. The force has the units of a velocity only by convention: Khatib's original applied $\mathbf{F}$ to a mass with viscous damping, $m\dot{\mathbf{p}} = \mathbf{F} - b\dot{\mathbf{p}}$, while almost every mobile-robot implementation uses the first-order form $\mathbf{v} = \mathbf{F}$ and sends $\mathbf{v}$ to the velocity controller of the vehicle. The magnitude of $\mathbf{F}$ is then arbitrary and can be enormous, k_{rep}/ρ^3 at a clearance of a few centimeters. The clip keeps the direction, which is the information the field carries, caps the speed at what the drone can fly, and bounds every step by $\Delta t v_{\text{max}}$, which Section 15.7.3 shows to be the difference between a controller that chatters and one that crashes. The chapter uses $k_{\text{att}} = k_{\text{rep}} = 1$, $\rho_0 = 2$, $v_{\text{max}} = 1$, $\Delta t = 0.01$, $\varepsilon_g = 0.05$, $W = 100$ and $\delta = 10^{-3}$ unless stated otherwise; once the speed is clipped only the ratio of the two gains

Algorithm 15.2. Gradient-descent controller with a velocity limit and local-minimum detection

Input: start $\mathbf{p}_0$, goal $\mathbf{p}_g$, obstacles $O_{1..m}$; speed limit $v_{\max}$, control period Δt , goal tolerance ε_g , horizon $K_{\max}$, stuck window W (steps) and stuck tolerance δ

Output: the path $\mathbf{p}_0, \mathbf{p}_1, \dots$ and a status in {reached, collision, stuck, timeout}

```

1 Function ApfDescend( $\mathbf{p}_0, \mathbf{p}_g, O_{1..m}$ ):
2   for  $k \leftarrow 0$  to  $K_{\max} - 1$  do
3     if  $\|\mathbf{p}_k - \mathbf{p}_g\| \leq \varepsilon_g$  then
4       return (reached,  $\mathbf{p}_{0..k}$ )
5     if  $\min_i \rho_i(\mathbf{p}_k) \leq 0$  then
6       return (collision,  $\mathbf{p}_{0..k}$ )
7     if  $k \geq W$  and  $\|\mathbf{p}_k - \mathbf{p}_{k-W}\| < \delta$  then
8       return (stuck,  $\mathbf{p}_{0..k}$ )                                // no progress: a local minimum
9      $\mathbf{v} \leftarrow \text{ApfForce}(\mathbf{p}_k, \mathbf{p}_g, O_{1..m})$ 
10    if  $\|\mathbf{v}\| > v_{\max}$  then
11       $\mathbf{v} \leftarrow v_{\max} \mathbf{v} / \|\mathbf{v}\|$                         // velocity limit: keep the direction, cap the speed
12     $\mathbf{p}_{k+1} \leftarrow \mathbf{p}_k + \Delta t \mathbf{v}$ 
13  return (timeout,  $\mathbf{p}_{0..K_{\max}}$ )

```

shapes the trajectory, ρ_0 decides how early an obstacle is felt, and Δt and $v_{\max}$ set the step length.

15.5 A worked example

Example 15.5 (One drone, one obstacle). The scene is that of [Figure 15.2](#): a disk obstacle of radius 1 centered at (4, 4), the goal $\mathbf{p}_g = (8, 4)$, the gains $k_{\text{att}} = k_{\text{rep}} = 1$ and the influence distance $\rho_0 = 2$, so the repulsion acts within 3 units of the disk center (the dashed circle in [Figure 15.3](#)). We evaluate the forces at three probe points, $A = (0.5, 4)$ far to the left, $B = (2.5, 4)$ on the axis half a unit from the disk, and $C = (3, 5.3)$ above and to the left of it, and then run [Algorithm 15.2](#) from the start (0, 4.5). The instance is `worked_example()` in `ch15_potential_fields.py`, and every number below is printed by its self-test.

Point A = (0.5, 4). The distance to the disk center is 3.5, so the clearance is $\rho = 2.5 > \rho_0$ and the repulsion is zero. The attraction is $\mathbf{F}_{\text{att}} = -(\mathbf{p} - \mathbf{p}_g) = -(0.5 - 8, 4 - 4) = (7.5, 0)$, and $U = \frac{1}{2} \cdot 7.5^2 = 28.125$.

Point B = (2.5, 4). Now $\rho = 1.5 - 1 = 0.5$ and $\nabla \rho = (-1, 0)$. The attraction is (5.5, 0). The repulsion has the magnitude $(1/0.5 - 1/2)/0.5^2 = 1.5/0.25 = 6$, so $\mathbf{F}_{\text{rep}} = (-6, 0)$ and the total force is $(-0.5, 0)$: the drone is pushed *back*, away from the goal, although the goal is straight ahead. The potential is $\frac{1}{2} \cdot 5.5^2 + \frac{1}{2} \cdot 1.5^2 = 16.25$. Somewhere between *A* and *B* the horizontal force changes sign, and at that point a drone on the axis stops ([Section 15.7.1](#)).

Point C = (3, 5.3). The offset from the center is $(-1, 1.3)$, of length $\sqrt{2.69} = 1.6401$, so $\rho = 0.6401$ and $\nabla \rho = (-0.6097, 0.7926)$. The attraction is (5, -1.3). The repulsion has magnitude $(1/0.6401 - 0.5)/0.6401^2 = 2.5928$ and equals $(-1.5805, 2.0547)$; the total force is (3.4195, 0.7547), of magnitude 3.5018. The drone is deflected upward, over the obstacle, while still making progress toward the goal. [Table 15.1](#) collects the three evaluations.

[Figure 15.3](#) shows the trajectory from (0, 4.5). The drone flies straight until it enters the influence circle, is deflected over the top of the disk, and curves back down to the goal, which it reaches after 1 099 steps, that is, 10.99 time units, with a minimum clearance of 0.519. The

Table 15.1. The forces of Example 15.5 at the three probe points of Figure 15.3 ($k_{\text{att}} = k_{\text{rep}} = 1$, $\rho_0 = 2$). At B the repulsion beats the attraction although the goal is straight ahead; at C the two forces are at an angle and their sum turns the drone around the obstacle.

Point	$\mathbf{p}$	ρ	$\mathbf{F}_{\text{att}}$	$\mathbf{F}_{\text{rep}}$	$\mathbf{F}$	U
A	(0.5, 4)	2.5	(7.5, 0)	(0, 0)	(7.5, 0)	28.125
B	(2.5, 4)	0.5	(5.5, 0)	(-6, 0)	(-0.5, 0)	16.250
C	(3, 5.3)	0.6401	(5, -1.3)	(-1.5805, 2.0547)	(3.4195, 0.7547)	13.909

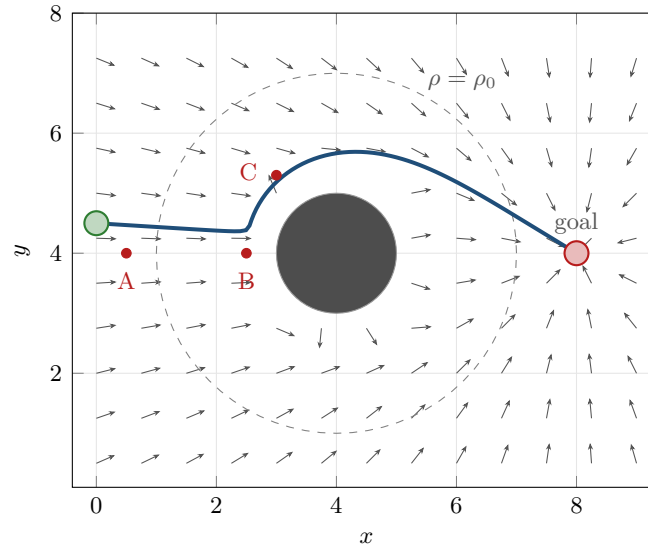


Figure 15.3. The force field of Example 15.5: arrows show the direction of $\mathbf{F}$, the dashed circle is the influence boundary $\rho = \rho_0$, the red dots are the probe points of Table 15.1, and the blue curve is the trajectory from (0, 4.5), which passes the disk with a minimum clearance of 0.519 and reaches the goal after 1 099 steps. Notice the arrows on the axis between $x = 1$ and $x = 3$, which first point right and then left: there the two forces cancel at one point.

path is 8.957 units long against a straight-line distance of 8.016; the detour costs about 12 %. The drone flies at the speed limit until it is one unit from the goal; the last stretch is the exponential approach of the quadratic bowl, which is why the goal tolerance is needed. The same run with the hybrid potential ($d^* = 2$) reaches the goal in 1 171 steps with a larger clearance, 0.695, because next to the disk the conic pull has the magnitude $k_{\text{att}}d^* = 2$ instead of about 5: the ratio of the two forces near an obstacle, not their absolute size, shapes the path.

Table 15.2 traces the same run at seven steps. The first rows are the straight approach: the force is long, the speed limit binds, and the command is the unit vector along $\mathbf{F}$. By $k = 800$ the drone is 1.007 from the goal, $\|\mathbf{F}\|$ has dropped to the speed limit, and from there the clip releases: the last stretch is the exponential tail of the bowl, $d(t) = d(8)e^{-k_{\text{att}}(t-8)}$, which needs $\ln(1.007/0.05) = 3.00$ time units to reach the tolerance – the 299 steps that the table shows between the last two rows.

15.6 Properties: continuity, descent, safety, and where the minimum is

A potential field is not a search algorithm, and the usual questions get unusual answers. It is not complete (Section 15.7 exhibits instances with a free path on which it fails) and not

Table 15.2. Trace of [Algorithm 15.2](#) from (0, 4.5) in the scene of [Example 15.5](#), printed by `worked_example()`: step k , position, clearance, force, its magnitude, the clipped velocity command $\mathbf{v}_k$ and the distance to the goal. The speed limit binds in every row but the last, so the command is a unit vector while the force is not; it releases inside the last unit, where the drone crosses the goal tolerance $\varepsilon_g = 0.05$ after 1 099 steps.

k	$\mathbf{p}_k$	ρ	$\mathbf{F}(\mathbf{p}_k)$	$\ \mathbf{F}\ $	$\mathbf{v}_k$	$\ \mathbf{p}_k - \mathbf{p}_g\ $
0	(0.000, 4.500)	3.031	(8.000, -0.500)	8.016	(0.998, -0.062)	8.016
100	(0.998, 4.438)	2.034	(7.002, -0.438)	7.016	(0.998, -0.062)	7.016
200	(1.996, 4.377)	1.039	(5.583, -0.298)	5.591	(0.999, -0.053)	6.016
300	(2.713, 4.826)	0.529	(1.104, 1.859)	2.162	(0.510, 0.860)	5.351
500	(4.410, 5.688)	0.737	(3.962, -0.156)	3.965	(0.999, -0.039)	3.967
800	(7.140, 4.524)	2.183	(0.860, -0.524)	1.007	(0.854, -0.520)	1.007
1 099	(7.957, 4.026)	2.957	(0.043, -0.026)	0.050	(0.043, -0.026)	0.050

optimal in any sense (nothing in it measures path length). Its complexity is its strength: one evaluation of [Algorithm 15.1](#) costs $\mathcal{O}(m)$ distance queries for m obstacles, or $\mathcal{O}(m_{\text{loc}})$ with a spatial index that returns the m_{loc} obstacles within ρ_0 , with no map representation, no queue and no memory beyond the obstacle list. What can be proved is what the marble picture promises: the drone always goes downhill, never touches an obstacle in continuous time, and stops at a critical point.

Proposition 15.6 (Continuity of the field). *On the subset of $\mathcal{C}_{\text{free}}$ where every $\nabla\rho_i$ is defined, the total potential U of [Equation \(15.6\)](#) is continuously differentiable and the force $\mathbf{F} = -\nabla U$ is continuous, in particular across the influence boundaries $\rho_i = \rho_0$.*

Proof. U_{att} is a polynomial in $\mathbf{p}$ (in the hybrid form, two pieces with equal value and gradient at $d = d^*$). Inside its influence region each $U_{\text{rep},i}$ is a smooth function of ρ_i , which is differentiable where its closest point is unique. At $\rho_i = \rho_0$ the inside expressions [Equations \(15.4\)](#) and [\(15.5\)](#) contain the factor $(1/\rho_i - 1/\rho_0)$ and vanish, which is the value of the outside branch. A sum of C^1 functions is C^1 . $\square$

The second derivative does jump at $\rho_i = \rho_0$: along $\nabla\rho_i$ the inside branch has the curvature $k_{\text{rep}}(3/\rho_i^4 - 2/(\rho_0\rho_i^3))$, equal to k_{rep}/ρ_0^4 at the boundary, while the outside branch has none; [Section 15.7.3](#) uses this curvature.

Theorem 15.7 (Descent, safety and convergence in continuous time). *Let $\mathbf{p}(t)$ solve $\dot{\mathbf{p}} = \mathbf{F}(\mathbf{p}) = -\nabla U(\mathbf{p})$ from a start $\mathbf{p}(0) \in \mathcal{C}_{\text{free}}$ with $U_0 = U(\mathbf{p}(0))$, and assume that ∇U is locally Lipschitz along the trajectory. Then:*

- (i) $U(\mathbf{p}(t))$ is non-increasing, with $\frac{d}{dt}U(\mathbf{p}(t)) = -\|\nabla U(\mathbf{p}(t))\|^2$;
- (ii) the drone never touches an obstacle: $\rho_i(\mathbf{p}(t)) \geq \rho_{\min} > 0$ for all t and i , where $\rho_{\min} = (1/\rho_0 + \sqrt{2U_0/k_{\text{rep}}})^{-1}$;
- (iii) $\mathbf{p}(t)$ stays in a bounded neighborhood of the goal – within $\sqrt{2U_0/k_{\text{att}}}$ with the quadratic attraction [Equation \(15.2\)](#), within $U_0/(k_{\text{att}}d^*) + d^*/2$ with the hybrid one [Equation \(15.3\)](#) – and converges to the set of critical points of U ; if these are isolated, it converges to one of them.

Proof. (i) By the chain rule, $\frac{d}{dt}U(\mathbf{p}(t)) = \nabla U(\mathbf{p})^\top \dot{\mathbf{p}} = -\|\nabla U(\mathbf{p})\|^2 \leq 0$. (ii) Every term of U is non-negative, so $U \geq U_{\text{rep},i} = \frac{1}{2}k_{\text{rep}}(1/\rho_i - 1/\rho_0)^2$ whenever $\rho_i \leq \rho_0$. Along the trajectory $U \leq U_0$, hence $1/\rho_i - 1/\rho_0 \leq \sqrt{2U_0/k_{\text{rep}}}$, which rearranges to $\rho_i \geq \rho_{\min}$; the bound is trivially true when $\rho_i > \rho_0$. (iii) Likewise $U_{\text{att}}(\mathbf{p}) \leq U \leq U_0$ bounds the trajectory: with the quadratic attraction $\frac{1}{2}k_{\text{att}}d^2 \leq U_0$ gives $d \leq \sqrt{2U_0/k_{\text{att}}}$, and with the hybrid attraction

the branch $k_{\text{att}}d^*d - \frac{1}{2}k_{\text{att}}(d^*)^2 \leq U_0$ gives $d \leq U_0/(k_{\text{att}}d^*) + d^*/2$ for $d > d^*$, while $d \leq d^*$ on the other branch. The sublevel set $S = \{\mathbf{p}: U(\mathbf{p}) \leq U_0\}$ is closed, bounded and, by (ii), at positive distance from every obstacle, so it is a compact subset of $\mathcal{C}_{\text{free}}$ that the trajectory never leaves. LaSalle’s invariance principle [82] then says that $\mathbf{p}(t)$ converges to the largest invariant subset of $\{\mathbf{p} \in S: \nabla U(\mathbf{p}) = \mathbf{0}\}$, the critical points of U in S ; if these are isolated, a continuous trajectory converging to the set converges to one point of it. $\square$

The theorem is the good news and the bad news in one statement: in continuous time the method is *safe*, with an explicit bound $\rho_{\min}$, and the drone always arrives *somewhere* – at a critical point of U , which is the goal only under a condition, and never the only one.

Proposition 15.8 (The goal as the minimum, and when it is not). *Consider U of Equation (15.6) with the quadratic or hybrid attraction.*

- (a) *If $\rho_i(\mathbf{p}_g) \geq \rho_0$ for every obstacle, then $U(\mathbf{p}_g) = 0 \leq U(\mathbf{p})$ for all $\mathbf{p}$: the goal is the global minimum of U and a critical point.*
- (b) *If $\rho_i(\mathbf{p}_g) < \rho_0$ for some obstacle and the repulsions do not cancel exactly at the goal, then $\mathbf{F}(\mathbf{p}_g) = \sum_i \mathbf{F}_{\text{rep},i}(\mathbf{p}_g) \neq \mathbf{0}$: the goal is not a critical point, and no trajectory of Theorem 15.7 ends there.*
- (c) *If every repulsive term is multiplied by d^n with $d = \|\mathbf{p} - \mathbf{p}_g\|$ and $n \geq 1$, the goal is again the global minimum, and for $n \geq 2$ it is a critical point.*

Proof. (a) All terms are non-negative and vanish at the goal, which lies outside every influence region; a global minimum of a C^1 function is critical. (b) $\mathbf{F}_{\text{att}}(\mathbf{p}_g) = \mathbf{0}$ by Equation (15.2), and the repulsive terms with $\rho_i < \rho_0$ are non-zero by Equation (15.5). (c) The modified terms $U_{\text{rep},i}d^n$ are non-negative and vanish at $d = 0$, so $U(\mathbf{p}_g) = 0$ is the global minimum. Their gradient is $d^n \nabla U_{\text{rep},i} + n d^{n-1} U_{\text{rep},i} \nabla d$ (Section 15.7.2), and both summands contain a positive power of d when $n \geq 2$. $\square$

Remark 15.9 (Critical points other than the goal cannot be avoided). Koditschek and Rimon [87] showed, using Morse theory, that on a sphere world (a disk with M disjoint disk obstacles removed) every smooth Morse potential that has the goal as its only minimum and is maximal on the obstacle boundaries has at least M other critical points. The reason is topological: the free space has Euler characteristic $1 - M$, and the count “minima minus saddles plus maxima” of a Morse function equals it, so one minimum and no interior maxima force M saddles. The best one can hope for is that all of them are *saddles*, from which almost every start escapes, rather than *minima*, which capture a whole basin. Khatib’s field does not achieve even that (Section 15.8.1); the navigation functions of Section 15.8 do.

15.7 The failure modes

Koren and Borenstein [91] listed four problems that they had met on their own robots: traps due to local minima, no passage between closely spaced obstacles, oscillations in the presence of obstacles, and oscillations in narrow passages. Each subsection below takes one failure, produces it with a function of `ch15_potential_fields.py` (the self-test asserts that the failure occurs), shows it in a figure, and gives the formula that explains it. Every number is printed by the self-test.

15.7.1 Local minima: the obstacle between the drone and the goal

Put the drone at $(0, 4)$ in the scene of Example 15.5, so that the disk sits exactly between it and the goal. By symmetry the force on the axis $y = 4$ is horizontal, and for $1 \leq x < 3$, inside

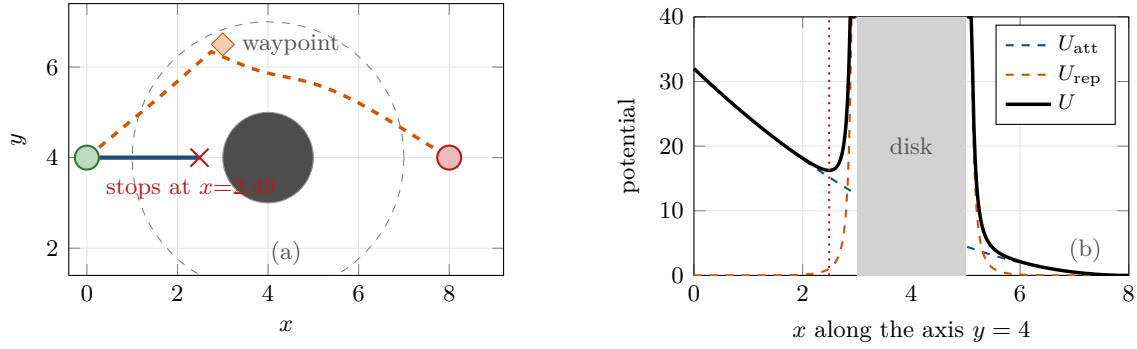


Figure 15.4. Failure mode 1: local minimum. (a) From (0, 4) the drone stops at $x = 2.49$, where the attraction $(8 - x)$ and the repulsion cancel; the dashed orange path is the same drone steered through a waypoint at (3, 6.5) from a global planner (Section 15.8). (b) The potentials along the axis $y = 4$: attractive bowl (blue), repulsive chimney (orange) and their sum (black), with a dip at $x = 2.49$ (dotted red line).

the influence region and left of the disk, its x -component is

$$F_x(x) = (8 - x) - \left(\frac{1}{3 - x} - \frac{1}{2} \right) \frac{1}{(3 - x)^2}, \quad (15.7)$$

because the clearance is $\rho = 3 - x$ and $\nabla \rho = (-1, 0)$. At $x = 1$ the repulsion is zero and $F_x = 7$; as $x \rightarrow 3$ the repulsion tends to $-\infty$. F_x is continuous, so it has a root, and bisection puts it at $x^* = 2.4872$. The run `local_minimum_case()` confirms it: the drone flies along the axis, slows down, and is declared stuck at $x = 2.4872$ after 354 steps (Figure 15.4a). Along the axis the potential has a genuine dip at x^* (Figure 15.4b): U falls from 32 at the start to about 16 at x^* and then climbs the chimney. This is the **classic local minimum**: the attraction says “forward”, the repulsion says “back”, and where they cancel the drone stops, although a free path over or under the disk exists: the shortest one, two tangents and an arc, is 8.25 units against the straight-line 8, only 3 % longer, and even one that keeps the clearance 0.519 of Figure 15.3 costs 7 %.

The dip is a minimum *along the axis* only. The Hessian of U at $(x^*, 4)$, computed by central differences of the analytic force, has the eigenvalues 36.96 in the x -direction and -2.64 in the y -direction: the point is a **saddle**, and a drone slightly off the axis is pushed further off by the repulsion faster than the attraction pulls it back. For a single convex obstacle the unavoidable critical point of Remark 15.9 is a saddle and only the starts exactly on the axis are trapped – little comfort in practice, because a drone near the axis lingers next to the saddle, and any asymmetry (a second obstacle, a wall, a non-convex shape) turns the saddle into a true minimum. The basin experiment of Section 15.8.1 uses two overlapping disks, and there 41 % of the starts end in a genuine local minimum with two positive Hessian eigenvalues.

15.7.2 Goals non-reachable with obstacles nearby (GNRON)

Move the goal to (5.4, 4), 0.4 units from the right edge of the disk and therefore well inside its influence region, and start the drone at (8, 6). The run `gnron_case()` stops at (5.968, 4.001), 0.568 units short of the goal, and stays there (Figure 15.5). The cause is Proposition 15.8(b): at the goal the attraction vanishes but the repulsion does not; with $\rho = 0.4$ it has the magnitude $(1/0.4 - 1/2)/0.4^2 = 12.5$ and points away from the disk. The minimum of U is where the pull toward the goal balances the push away from the disk, $k_{att} d = k_{rep}(1/\rho - 1/\rho_0)/\rho^2$ with $\rho = 0.4 + d$, whose solution $d = 0.568$ is exactly where the drone stops. Ge and Cui [52] named the phenomenon **GNRON**, *goals non-reachable with obstacles nearby*; a landing pad next to a building or a formation slot beside a neighbor is enough to produce it.

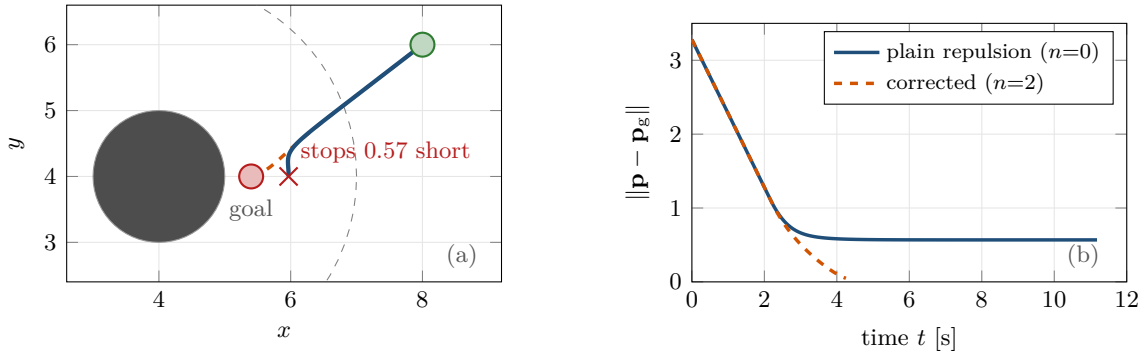


Figure 15.5. Failure mode 2: the goal lies 0.4 from the disk, inside the influence region. (a) With Khatib’s repulsion (solid blue) the drone from (8,6) stops 0.57 short of the goal, where attraction and repulsion balance; with the repulsion multiplied by d^2 (dashed orange) it arrives. (b) Distance to the goal against time: the plain field levels off at 0.57, the corrected one reaches the tolerance 0.05 after 4.25 time units.

The standard fix multiplies each repulsive term by a power of the distance to the goal, so that the repulsion fades out exactly where the goal is:

$$U'_{\text{rep},i}(\mathbf{p}) = U_{\text{rep},i}(\mathbf{p}) d(\mathbf{p})^n, \quad d(\mathbf{p}) = \|\mathbf{p} - \mathbf{p}_g\|, \quad n \geq 1. \quad (15.8)$$

The product rule with $\nabla d = (\mathbf{p} - \mathbf{p}_g)/d$ gives the corrected force, line 12 of Algorithm 15.1:

$$\mathbf{F}'_{\text{rep},i} = -\nabla(U_{\text{rep},i} d^n) = d^n \mathbf{F}_{\text{rep},i} - \frac{n}{2} k_{\text{rep}} \left(\frac{1}{\rho_i} - \frac{1}{\rho_0} \right)^2 d^{n-1} \frac{\mathbf{p} - \mathbf{p}_g}{d}. \quad (15.9)$$

The first term is the old repulsion, scaled down near the goal; the second is a new pull *toward* the goal, present only inside influence regions. By Proposition 15.8(c) the goal is now the global minimum, and for $n \geq 2$ also a critical point; with $n = 1$ the second term keeps a non-zero magnitude at the goal whose direction depends on the side of approach, a conic pull that makes the drone chatter with the amplitude $\Delta t v_{\text{max}}$ of one step. Here that is 0.01, well below the goal tolerance $\varepsilon_g = 0.05$, so the run is declared reached and the chatter appears only when the tolerance is tightened below it (Exercise 15.4); this is why Ge and Cui recommend $n = 2$, which is the default of the code. With $n = 2$ the run of Figure 15.5 reaches the goal after 425 steps. Two warnings: the factor changes the field everywhere (d^2 is 100 ten units away, so k_{rep} must be retuned for the size of the scene), and it cures only this failure – from (0,4) the corrected field still stops the drone on the axis, at a different x .

Waypoints next to walls

The most common way to meet GNRON in a drone system is not a goal next to a building but a *waypoint* next to one: a grid planner (Chapter 4) happily routes through cells that touch obstacles, and a potential field asked to reach such a waypoint with a tolerance smaller than the offset of Proposition 15.8(b) hovers in front of it for ever. Either inflate the obstacles by more than the drone radius before planning, so that waypoints stay outside ρ_0 , or accept a waypoint as reached at a tolerance of the order of ρ_0 , or use Equation (15.9) for the last leg.

15.7.3 Oscillation in narrow passages and with discrete time steps

Two disks of radius 1 centered at (4,2.5) and (4,5.5) leave a passage of width 1.0 on the axis $y = 4$; the drone starts at (0,4.3) and its goal is (8,4.2), slightly off the axis as in any

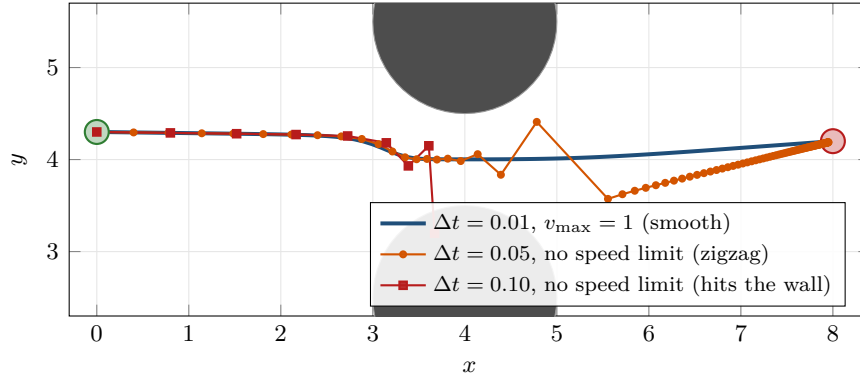


Figure 15.6. Failure mode 3: oscillation in a passage of width 1.0, crossed with three control periods. With $\Delta t = 0.01$ and the speed limit (blue) the drone glides through; with $\Delta t = 0.05$ and no limit (orange, one mark per step) it zigzags between the walls; with $\Delta t = 0.10$ and no limit (red) the lateral error changes sign and grows at every step inside the passage, and the third step there overshoots the axis by 0.80, more than the half-width 0.5, so the drone ends inside the lower disk after 8 steps. The lateral stiffness at the midpoint is 81, so $\Delta t \lambda_{\max}$ is 0.81, 4.05 and 8.1.

real scene. Figure 15.6 shows three runs of `corridor_case()` through this passage. With $\Delta t = 0.01$ and $v_{\max} = 1$ the drone glides through, reverses its lateral direction once, never deviates more than 0.161 from the axis and keeps a clearance of 0.498; with $\Delta t = 0.05$ and no speed limit it zigzags (8 reversals inside the passage, amplitude 0.410, clearance 0.342); with $\Delta t = 0.10$ and no speed limit it hits the lower wall after 8 steps. The field is the same in all three runs; what differs is the step.

Proposition 15.10 (Discrete-time stability near a minimum). *Let $\mathbf{p}^*$ be a strict local minimum of a C^2 potential U , with Hessian $\mathbf{H} = \nabla^2 U(\mathbf{p}^*)$ and eigenvalues $0 < \lambda_1 \leq \dots \leq \lambda_{\max}$. The update $\mathbf{p}_{k+1} = \mathbf{p}_k - \Delta t \nabla U(\mathbf{p}_k)$ (line 12 of Algorithm 15.2 while the speed limit is inactive) is locally asymptotically stable at $\mathbf{p}^*$ if $\Delta t \lambda_{\max} < 2$ and unstable if $\Delta t \lambda_{\max} > 2$; at $\Delta t \lambda_{\max} = 2$ the linearization is marginally stable and decides nothing. Along the eigenvector of λ_i the error is multiplied by $1 - \Delta t \lambda_i$ per step: it shrinks monotonically for $\Delta t \lambda_i < 1$, alternates in sign for $1 < \Delta t \lambda_i < 2$, and grows for $\Delta t \lambda_i > 2$.*

Proof. Write $\mathbf{e}_k = \mathbf{p}_k - \mathbf{p}^*$. Since $\nabla U(\mathbf{p}^*) = \mathbf{0}$, Taylor's theorem gives $\mathbf{e}_{k+1} = (\mathbf{I} - \Delta t \mathbf{H})\mathbf{e}_k + o(\|\mathbf{e}_k\|)$. The symmetric matrix $\mathbf{I} - \Delta t \mathbf{H}$ has the eigenvalues $1 - \Delta t \lambda_i$, and the linear recursion is asymptotically stable when all of them have modulus below 1, that is, when $\Delta t \lambda_i < 2$ for all i , and unstable when one of them has modulus above 1; the nonlinear system inherits both properties from its linearization (Lyapunov's indirect method [82]), which says nothing when a modulus equals 1. The statements about monotone and alternating decay are the sign and size of the factor. $\square$

In a passage the potential is stiff in the lateral direction. At the midpoint of a passage of width w between two walls at clearance $\rho = w/2$, differentiating Equation (15.4) twice gives the lateral curvature

$$\lambda_y = k_{\text{att}} + 2k_{\text{rep}} \left(\frac{3}{\rho^4} - \frac{2}{\rho_0 \rho^3} \right), \quad \rho = \frac{w}{2}, \quad (15.10)$$

the sum of the bowl and of the two walls. For $w = 1$ this is $1 + 2(48 - 8) = 81$, the **stiffness** that the code computes numerically as the largest Hessian eigenvalue, and Proposition 15.10 explains the three runs: $\Delta t \lambda_y = 0.81 < 1$ decays monotonically, $4.05 > 2$ diverges, and 8.1 diverges so fast that the second step leaves the passage. The stiffness grows roughly like the inverse fourth power of the width, 204 for $w = 0.8$ and 668 for $w = 0.6$: a step that is safe in open space is unstable in every passage narrower than some width, and no single Δt serves all

Table 15.3. The passage experiment of `corridor_case()` with the speed limit $v_{\max} = 1$ active: width w , control period Δt , outcome, steps, lateral reversals inside the passage, largest lateral excursion, stiffness Equation (15.10) and the stability number $\Delta t \lambda_y$. Widths 0.6 and 0.8 are never entered (Section 15.7.4); with $\Delta t = 0.05$ the blocked drone chatters in front of the entrance.

w	Δt	Outcome	Steps	Reversals	Excursion	λ_y	$\Delta t \lambda_y$
0.6	0.01	stuck	446	0	0.065	667.7	6.68
0.6	0.05	stuck	88	24	0.067	667.7	33.4
0.8	0.01	stuck	619	0	0.122	204.1	2.04
0.8	0.05	stuck	120	51	0.117	204.1	10.2
1.0	0.01	reached	1003	1	0.161	81.0	0.81
1.0	0.05	reached	200	1	0.155	81.0	4.05
1.2	0.01	reached	1002	1	0.190	38.0	0.38
1.2	0.05	reached	200	1	0.183	38.0	1.90

scenes.

Table 15.3 repeats the experiment with the speed limit switched on. The limit is a strong medicine: with $v_{\max} = 1$ the run with $\Delta t = 0.05$ and $w = 1$ crosses with a single reversal although its stability number is 4.05, because the clipped step is at most $\Delta t v_{\max} = 0.05$ long and cannot overshoot the axis by more than that; what remains of the instability is chatter of that amplitude (the 51 reversals of the blocked run with $w = 0.8$). Koren and Borenstein’s “oscillations in the presence of obstacles” also arise in continuous time when the drone has inertia: the second-order model $m\ddot{\mathbf{p}} = \mathbf{F} - b\dot{\mathbf{p}}$ oscillates unless the damping b is large compared with $\sqrt{m\lambda}$, a velocity controller with latency behaves like a lightly damped mass, and noisy distances produce a jittering force.

15.7.4 No passage between closely spaced obstacles

Narrow the passage to $w = 0.6$ (disks centered at (4, 2.7) and (4, 5.3)). The axis stays 0.3 clear of both disks, so the passage is free, and a drone that starts inside it is carried through: on the axis at $x = 4$ the factor $4 - x$ of Equation (15.11) below kills the horizontal repulsion, so $F_x(4) = 8 - 4 = 4 > 0$ points at the goal, and the run `simulate((4,4), (8,4.2), corridor(0.6), ApfParams())` reaches it in 599 steps with a minimum clearance of 0.300. Yet the drone from (0, 4.3) never gets in: it stops at (3.163, 4.005), in front of the entrance, after 446 steps (Figure 15.7a). On the axis the vertical components of the two repulsions cancel and their horizontal components add. With $r(x) = \sqrt{(4-x)^2 + 1.3^2}$ the distance to either center and $\rho = r - 1$,

$$F_x(x) = (8 - x) - 2k_{\text{rep}} \left(\frac{1}{\rho} - \frac{1}{\rho_0} \right) \frac{1}{\rho^2} \frac{4 - x}{r}, \quad (15.11)$$

for $\rho \leq \rho_0$, with $F_x = 8 - x$ beyond the influence region; it is plotted in Figure 15.7b. It is positive far away, dips to -6.01 just before the entrance, has its first root at $x = 3.163$ (a second, unstable root sits near $x = 3.91$), and is positive again inside the passage. The drone stops at the first root: two walls that each say “keep away” add up to “go back” in front of the gap, although neither wall is in the way. Table 15.3 shows that $w = 0.8$ is blocked as well. Raising k_{att} or lowering k_{rep} or ρ_0 opens the gap at the price of a smaller clearance everywhere else (Exercise 15.6); no setting passes every doorway and keeps every margin, which is what makes potential fields unsuitable as the *only* planner in cluttered spaces.

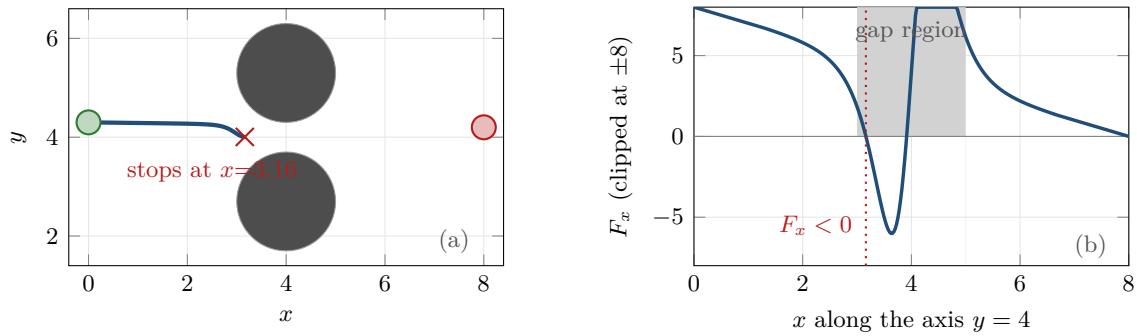


Figure 15.7. Failure mode 4: a free passage of width 0.6 that the drone does not enter. (a) The trajectory from (0,4.3) ends at $x = 3.16$, in front of the gap. (b) The horizontal force Equation (15.11) along the axis, clipped at ± 8 : the two repulsions add up in front of the entrance and push the drone back before it reaches the point where they cancel.

15.7.5 The Koren–Borenstein critique, summarized honestly

Koren and Borenstein [91] tested their own implementation, the *virtual force field*, on a mobile robot with sonar sensors, the repulsion being summed over the occupied cells of a certainty grid. They found the four problems above, concluded that potential fields are unsuitable for real-time obstacle avoidance, and replaced them with the **vector field histogram** (VFH) of Borenstein and Koren [20], which picks a steering direction from a polar histogram instead of adding vectors. Three decades later the honest summary is this.

- *Traps and blocked passages are structural.* They follow from the algebra of Equations (15.7) and (15.11), and Remark 15.9 shows that critical points other than the goal cannot be avoided by any potential; gains and influence distances move the failures around, they do not remove them.
- *The oscillations are partly implementation-dependent.* The first-order controller with exact distances never oscillates in continuous time (Theorem 15.7); what oscillated in their experiments was discretization, robot dynamics and sonar noise. But the stiffness Equation (15.10) leaves every discrete controller a passage too narrow for it.
- *The verdict was about a role, not about the idea.* It concerned the potential field as the *whole* obstacle-avoidance layer of a robot driven by raw sonar readings, not potential terms inside a larger system (Section 15.11) or navigation functions.

15.8 Remedies

Navigation functions. Rimon and Koditschek [87, 140] asked for the best a potential can be: a **navigation function** is a smooth potential on a bounded free space that is maximal on the obstacle boundaries, has the goal as its unique minimum, and whose other critical points are non-degenerate saddles. By Theorem 15.7 and Remark 15.9 this is the strongest possible guarantee: every start outside a set of measure zero reaches the goal, and no start collides. For a **sphere world** – a disk of radius r_0 centered at $\mathbf{c}_0$ with M disjoint disk obstacles $(\mathbf{c}_i, r_i)$ inside – they construct one explicitly:

$$\varphi_\kappa(\mathbf{p}) = \frac{\|\mathbf{p} - \mathbf{p}_g\|^2}{(\|\mathbf{p} - \mathbf{p}_g\|^{2\kappa} + \beta(\mathbf{p}))^{1/\kappa}}, \quad \beta(\mathbf{p}) = (r_0^2 - \|\mathbf{p} - \mathbf{c}_0\|^2) \prod_{i=1}^M (\|\mathbf{p} - \mathbf{c}_i\|^2 - r_i^2), \quad (15.12)$$

and prove that φ_κ is a navigation function for every sufficiently large κ : β is positive in the free space and zero on every boundary, so φ_κ is 0 at the goal and 1 on the obstacles, and the

large exponent sharpens the goal term until the obstacle terms can only produce saddles. The result matters for what it says: local minima are not a law of nature but a defect of the sum in Equation (15.6). It is rarely used on drones, because it needs the complete obstacle geometry in closed form, a large κ makes the field extremely flat far from the goal and extremely steep near it, and moving obstacles break the construction.

Random walks and simulated annealing. Barraquand and Latombe [10] accepted the local minima and added an escape: when the descent stalls, execute a **random walk** of random length, then descend again, and repeat until the goal is reached. The escape used by `simulate_many()` is the simplest version, a straight move of random direction and random length between 0.5 and 2 units at full speed, abandoned early if it would come within 0.1 of an obstacle. **Simulated annealing** is the principled form of the same idea: accept an uphill move with probability $\exp(-\Delta U/T)$ and lower the temperature T over time. Both trade determinism and path quality for a probabilistic guarantee, and Section 15.8.1 measures what the trade buys.

Harmonic potentials. Connolly, Burns and Weiss [30] proposed to compute U as a solution of Laplace’s equation, $\nabla^2 U = 0$ in the free space, with $U = 1$ on the obstacle boundaries and $U = 0$ at the goal. A **harmonic potential** has no interior extrema by the maximum principle, so it has no local minima, and the descent reaches the goal from every start. The price is a global computation: Laplace’s equation must be solved numerically on a grid covering the workspace and re-solved whenever the map changes. Its discrete cousin, the wavefront planner of Choset et al. [28], assigns every free cell its number of steps to the goal by a breadth-first sweep – Dijkstra’s algorithm of Chapter 3 under another name. A potential without local minima costs a global search.

A potential field as the local layer under a global plan. The remedy that matters for this book is to stop asking the field to do global planning. A global planner – A^* on a grid (Chapter 4), D^* Lite when the map changes (Chapter 5), CBS for the swarm (Chapter 9) – delivers waypoints along a path that is known to exist, and the field descends to each waypoint in turn. Each leg is short, its goal is close and in view, and the local minima of the field with respect to the far-away goal are simply not there. The dashed path of Figure 15.4a is the local-minimum case of Section 15.7.1 steered through the single waypoint (3, 6.5): the drone reaches the waypoint, then the goal, in 1188 steps, without any change to the field; `follow_waypoints()` in `ch15_potential_fields.py` is the whole mechanism, a loop that runs Algorithm 15.2 toward each waypoint with a loose tolerance and aborts when a leg does not end with “reached”. The field keeps its job as the reactive layer – obstacles that were not on the map, and other drones, still repel – and two rules keep the arrangement honest: waypoints must be reached with a tolerance of the order of ρ_0 , or moved away from walls, because of GNRON; and a stalled leg (line 8) must trigger a replanning of the global path, which is the loop of Chapter 24.

15.8.1 A generated experiment: the basin of a local minimum

How much of the start space does a local minimum capture, and how much does a random walk recover? `basin_experiment()` answers this for a scene with two obstacles: disks of radius 1 centered at (4, 3.2) and (4, 5.0), which overlap into a single peanut-shaped obstacle with a concave waist, and the goal at (8, 4.1). Drones are released from 441 starts on a 21×21 grid covering $x \in [0, 2]$, $y \in [0.5, 7.7]$, and Algorithm 15.2 runs from all of them at once (Figure 15.8a). With the plain field, 262 starts reach the goal (59.4%) and 179 are declared stuck (40.6%); none collides. All 179 end at the *same* point, (2.676, 4.1), in front of the waist.

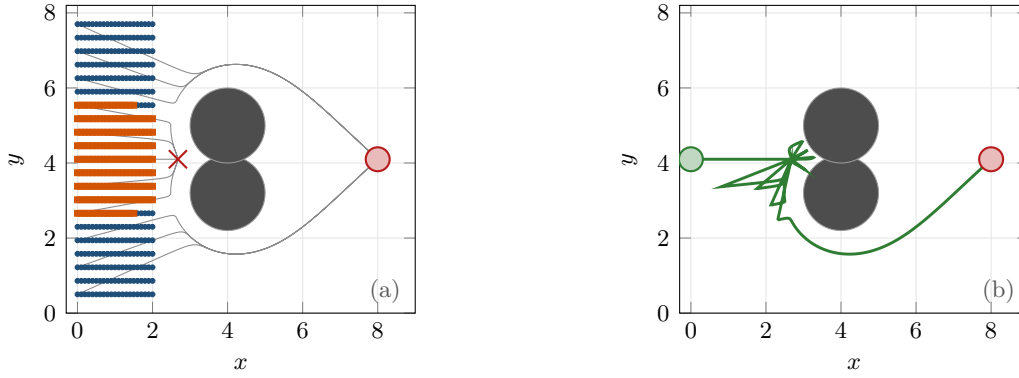


Figure 15.8. The basin experiment. (a) 441 starts in front of a peanut-shaped obstacle with the goal at (8, 4.1): blue dots reach the goal, orange squares end in the local minimum at (2.676, 4.1) (red cross), gray curves are the trajectories from the column $x = 0$; the basin is a band around the symmetry axis holding 41 % of the starts. (b) A single-drone run from the trapped start (0, 4.1) with random-walk escapes (green), drawn from its own random stream: after 8 walks and 3810 steps the drone finds the way round the upper lobe.

Unlike the saddle of [Section 15.7.1](#), this is a true minimum: the two repulsions form a valley along the symmetry axis, and the Hessian of U there has the eigenvalues 9.87 and 24.89, both positive. Its basin is the band of starts within about 1.4 of the axis, narrowing to about 1.1 for starts already inside the influence region.

With the random-walk escape switched on, 386 starts reach the goal (87.5 %), none is left stuck, and 55 run into the horizon of 4000 steps while still walking and descending; the drones that do arrive need on average 1.24 escapes and 1428 steps, against 1018 for the plain runs (averaged over all 441 starts the escape count is 2.48, because the 55 drones that never arrive keep walking). [Figure 15.8b](#) shows a separate single-drone run from the trapped start (0, 4.1) on the axis, with its own random stream: eight random walks, 3810 steps, and a final route over the upper lobe. In the batch run that start is still walking when the horizon expires, because the outcome of a random escape is a random variable and not a property of the start. The random walk converts a certain failure into a probable, slow success with an unpredictable path – a crutch worth having in a local layer, and no substitute for a global plan.

The failures of this chapter are properties of *regions* of the start space, not of the scene alone, and nothing in a run signals them except the stuck detector: a single test from a start outside the band would have passed. Sweep a grid of starts as `basin_experiment()` does, report the fractions that arrive and that stall, and keep the detector of [Algorithm 15.2](#) in the deployed code, because it is the only component that can tell the rest of the system that the field has failed; [Chapter 25](#) turns this into an experiment protocol.

15.9 Swarms: inter-agent repulsion, flocking and consensus

Nothing in [Algorithm 15.1](#) requires an obstacle to be static. The other drones of the swarm are obstacles too, disks of radius r that move, and the natural extension is a pairwise repulsion with the clearance $\rho_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\| - 2r$ between drones i and j :

$$\mathbf{F}_i = \mathbf{F}_{\text{att},i} + \sum_k \mathbf{F}_{\text{rep},k}(\mathbf{p}_i) + \sum_{j \neq i, \rho_{ij} < \rho_a} k_a \left(\frac{1}{\rho_{ij}} - \frac{1}{\rho_a} \right) \frac{1}{\rho_{ij}^2} \frac{\mathbf{p}_i - \mathbf{p}_j}{\|\mathbf{p}_i - \mathbf{p}_j\|}, \quad (15.13)$$

with its own gain k_a and range ρ_a (ρ_{ij} is a clearance here, not the correlation of [Chapter 2](#)). This **inter-agent repulsion** is symmetric – the term that j exerts on i is the negative of the

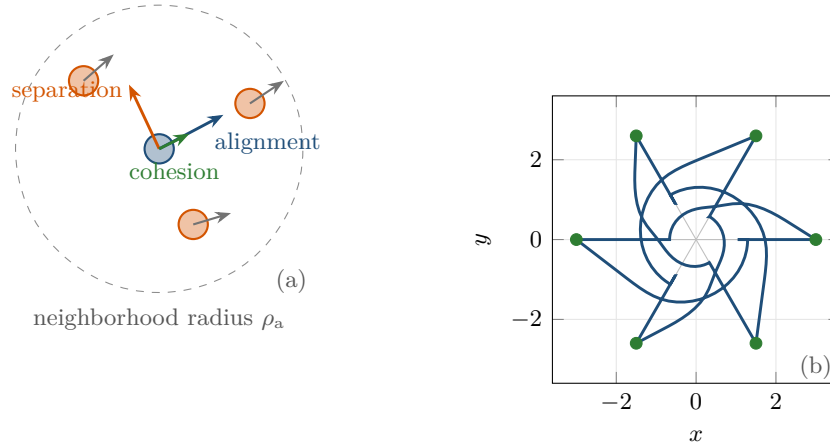


Figure 15.9. Inter-agent repulsion and flocking. (a) Reynolds's three rules seen from the focal drone (blue): separation pushes it away from close neighbors, alignment turns its velocity toward the neighbors' average heading (gray arrows), cohesion pulls it toward their centroid; all three use only the neighbors within the radius ρ_a . (b) Six drones on a ring of radius 3 swap places with their opposite points. With pairwise repulsion (blue) the closest approach is 1.044; the gray lines are the paths without it, which all pass through the center at the same time.

term i exerts on j – so the pair shares one potential. `simulate_swarm()` runs it for six drones on a ring of radius 3 whose goals are the opposite points of the ring, so that all six nominal paths cross at the center at the same moment. Without the pair term the minimum separation over the run is 0.000: all six meet. With $k_a = 1$, $\rho_a = 1.5$, $r = 0.25$ and $\Delta t = 0.02$ it is 1.044, twice the touching distance $2r = 0.5$, and after 30 time units every drone is within 0.3 of its goal (Figure 15.9b). The symmetric scene is also the swarm version of the local minimum: two drones exactly head-on would stop facing each other, since their attractions and repulsions cancel. Symmetric repulsion has no notion of who yields; the reciprocal velocity obstacles of Chapter 13 exist precisely to supply one.

Flocking and consensus. Reynolds [138] animated flocks with three local rules that every agent applies to the neighbors within a radius (Figure 15.9a): **separation**, steer away from neighbors that are too close; **alignment**, steer toward the average velocity of the neighbors; and **cohesion**, steer toward their centroid. The three rules are potential fields and one thing more. Separation is Equation (15.13). Cohesion is an attractive potential whose goal moves, $\frac{1}{2}k_c \|\mathbf{p}_i - \bar{\mathbf{p}}\|^2$ with $\bar{\mathbf{p}}$ the centroid of the neighbors. Alignment is not the gradient of any potential over positions; it acts on velocities. Write $N_i = \{j \neq i : \|\mathbf{p}_i - \mathbf{p}_j\| \leq \rho_a\}$ for the neighbors of drone i within the interaction radius ρ_a , and let $\alpha > 0$ be the alignment gain, small enough that $\alpha |N_i| < 1$ for every drone, so that each update is a convex combination of the velocities:

$$\mathbf{v}_i \leftarrow \mathbf{v}_i + \alpha \sum_{j \in N_i} (\mathbf{v}_j - \mathbf{v}_i), \quad (15.14)$$

and Equation (15.14) is exactly the consensus update of Chapter 23 applied to the velocities: on a connected neighborhood graph all velocities converge to a common value and the flock moves as one. Olfati-Saber [122] made the correspondence precise and proved that a pairwise potential (separation and cohesion in one smooth term), a velocity-consensus term and a navigation term together produce stable flocking without collisions. Formation keeping is the same construction with a prescribed geometry: a spring of rest length d_{ij} between designated pairs, $\frac{1}{2}k_s(\|\mathbf{p}_i - \mathbf{p}_j\| - d_{ij})^2$, is the displacement-based formation controller of Chapter 23 read as a potential (`simulate_swarm()` offers such springs through its `k_spring` argument), and Chapter 23 supplies the convergence proofs that this chapter, with its local minima, cannot.

15.10 Implementation notes

`ch15_potential_fields.py` implements everything in this chapter for a point drone in the plane with disk obstacles. Positions are NumPy arrays of shape $(2,)$ or $(N,2)$, and every function broadcasts over the leading axis, so that `simulate_many()` advances N drones with one call per step and the basin experiment moves its 441 drones for 4000 steps in about a second. [Listing 15.1](#) is the repulsive force, a direct transcription of [Equations \(15.5\) and \(15.9\)](#); the self-test checks the analytic forces against central differences of the potential at 200 random points for five parameter settings.

Listing 15.1. The repulsive force (from `code/ch15_potential_fields.py`). `prm` holds the gains; `obs.distance(p)` returns the clearance ρ and $\nabla\rho$; `prm.n_gnron` is the exponent n of [Equation \(15.9\)](#), 0 for Khatib’s original; `EPS` is 10^{-12} .

```

1 def repulsive_force(p, goal, obstacles, prm):
2     """F_rep = -grad U_rep, summed over the obstacles within rho0.
3
4     Plain (n = 0): k_rep (1/rho - 1/rho0) / rho^2 * grad rho.
5     GNRON (n >= 1): the same times d^n, minus
6                     (n/2) k_rep (1/rho - 1/rho0)^2 d^(n-1) * grad d,
7     where d = ||p - goal|| and grad d = (p - goal)/d points away from the goal.
8     """
9     diff, d = _goal_offset(p, goal)
10    n = prm.n_gnron
11    f = np.zeros_like(diff)
12    for obs in obstacles:
13        rho, grad_rho = obs.distance(p)
14        rho_c = np.maximum(rho, EPS)
15        gap = 1.0 / rho_c - 1.0 / prm.rho0
16        term = prm.k_rep * gap / rho_c ** 2 * grad_rho
17        if n > 0:
18            grad_d = diff / np.maximum(d, EPS)
19            term = term * d ** n - 0.5 * n * prm.k_rep * gap ** 2 * d ** (n -
20            1) * grad_d
21        f = f + np.where(rho < prm.rho0, term, 0.0)
22    return f

```

Step size. [Proposition 15.10](#) gives the rule: $\Delta t \lambda_{\max} < 1$ at the stiffest point the drone is meant to visit, with $\lambda_{\max}$ from [Equation \(15.10\)](#) for the narrowest passage in the scene; `stiffness()` computes the largest Hessian eigenvalue at a point by differencing the analytic force. A second rule protects against the crash of [Figure 15.6](#): with the speed limit active a step moves the drone by at most $\Delta t v_{\max}$, so keeping that product below the smallest clearance you accept rules out the crash, though not the chatter.

Force clipping. Clip the *total* force, as line 11 does, not its terms: scaling the vector keeps the direction that the field prescribes, while clipping components or individual repulsions changes the direction and can steer the drone into the very obstacle it is avoiding. The magnitude $k_{\text{rep}}(1/\rho - 1/\rho_0)/\rho^2$ overflows as $\rho \rightarrow 0$; [Listing 15.1](#) floors ρ at `EPS`, the controller reports a collision when $\rho \leq 0$, and a drone already inside an obstacle (a map error, a moving intruder) must be handled explicitly, because the formula changes sign there.

Distance queries. Line 8 of [Algorithm 15.1](#) is where the implementation meets the map. For disks it is a subtraction; for a polygon, the minimum over its edges of the point-to-segment

distance of [Chapter 2](#); for a grid map, a Euclidean distance transform computed once, with the gradient read off by central differences. With many obstacles, keep them in a uniform grid of buckets or a KD-tree ([Chapter 16](#)) and query only the ones within ρ_0 . Where two obstacles are equally close, $\nabla\rho$ of the obstacle set is undefined and a drone feels the sum of both repulsions, which is the mechanism of [Section 15.7.4](#).

Discrete-time instability and the controller loop. The loop of `simulate_many()` is [Algorithm 15.2](#) with the three tests, the clipped step and the optional random-walk escape of [Section 15.8](#), vectorized over the drones with boolean masks. The stuck detector compares the current position with the position W steps ago; W should cover several periods of any chatter, and the tolerance δ should exceed the chatter amplitude $\Delta t v_{\max}$, otherwise a chattering drone is never declared stuck. A random walk is committed for a whole leg of steps, so that the descent does not cancel it at the next step.

A force is not a velocity

The formula $\mathbf{v} = \mathbf{F}$ hides two unit conversions that a real drone does not forgive. First, the magnitude of $\mathbf{F}$ is unbounded, far from the goal and close to an obstacle, and a velocity controller that receives such a command saturates its motors, loses the direction information and, in the discrete-time model, takes the crash step of [Figure 15.6](#); the speed limit of [line 11](#) is a necessity, not a refinement. Second, a drone has inertia and its velocity loop has latency, so the effective model is second order and lightly damped, and [Proposition 15.10](#) underestimates the oscillation. Either low-pass filter the command, $\mathbf{v}_{k+1} = (1 - \beta)\mathbf{v}_k + \beta \mathbf{F}$ with $\beta < 1$, or hand the force to a controller that knows the dynamics: the dynamic window of [Chapter 14](#) or the model predictive controller of [Chapter 21](#).

15.11 Where this fits in the drone system

In the drone system

A potential field is the cheapest reactive layer there is: one distance query per nearby obstacle or drone, one vector sum, no search. In the hybrid architecture of [Chapter 24](#) it is nevertheless *not* the local safety layer. It reacts to positions, not velocities: an intruder approaching head-on is repelled only once it is within ρ_0 , whereas the velocity obstacles of [Chapter 12](#) and ORCA ([Chapter 13](#)) reason about relative velocity and time to collision and yield earlier and reciprocally; it has no notion of the drone's dynamics, whereas the dynamic window approach of [Chapter 14](#) evaluates only commands the drone can execute; and it has the four failure modes of [Section 15.7](#), which no tuning removes from every scene.

The book therefore keeps potential fields for what they do well: the inter-agent repulsion of [Equation \(15.13\)](#) is the *soft spacing* term that runs at every control cycle between swarm members on top of the time-indexed paths of CBS or ECBS ([Chapters 9 and 10](#)), the spring terms of [Section 15.9](#) are the *formation-keeping* controller of [Chapter 23](#), and the same quadratic penalties reappear as soft constraints in the model predictive controller of [Chapter 21](#). The waypoint-following arrangement of [Section 15.8](#) is the pattern every local layer of [Chapter 24](#) follows, with a stalled leg triggering replanning ([Chapters 4 and 5](#)). Week 7 of the study plan ([Appendix A](#)) covers this chapter with [Chapter 14](#); its coding exercise is [Exercise 15.9](#).

15.12 Summary

Chapter summary

- A potential field adds a bowl at the goal, $U_{\text{att}} = \frac{1}{2}k_{\text{att}} \|\mathbf{p} - \mathbf{p}_g\|^2$ (conic beyond d^* in the hybrid form), to a wall around every obstacle, $U_{\text{rep}} = \frac{1}{2}k_{\text{rep}}(1/\rho - 1/\rho_0)^2$ for $\rho \leq \rho_0$, and commands the negative gradient of the sum, [Equations \(15.2\)](#) and [\(15.5\)](#).
- The controller is explicit Euler with a speed limit and a stuck detector. In continuous time the potential decreases monotonically, the drone never touches an obstacle, and it converges to a critical point of U , which is the goal only when nothing cancels the attraction first.
- The four failure modes have formulas: a local minimum where $\mathbf{F}_{\text{att}} + \mathbf{F}_{\text{rep}} = \mathbf{0}$; GNRON, a goal inside ρ_0 where $\mathbf{F}_{\text{rep}} \neq \mathbf{0}$, cured by the factor d^n ; oscillation once $\Delta t \lambda_{\text{max}} > 1$ and divergence once $\Delta t \lambda_{\text{max}} > 2$, with a stiffness growing like the inverse fourth power of a passage width; and blocked passages, where two repulsions add up in front of a free gap.
- Critical points other than the goal cannot be avoided by any potential; navigation functions make them all saddles on sphere worlds, harmonic potentials need a global grid computation, random walks cost time and path quality. The remedy that scales is to use the field as a local layer under a global plan, with a stalled leg triggering replanning.
- Inter-agent repulsion is Reynolds's separation rule, cohesion a spring to the neighbors' centroid, alignment a velocity consensus. Potential fields survive in the swarm as soft spacing and formation keeping, while ORCA and DWA take the local safety layer.

Further reading. Khatib [83] introduced the artificial potential field with the FIRAS repulsion; Koren and Borenstein [91] is the critique of [Section 15.7.5](#), still worth reading for its experiments, and Borenstein and Koren [20] is the vector field histogram they proposed instead; Ge and Cui [52] named GNRON and proposed the d^n correction. Koditschek and Rimon [87] and Rimon and Koditschek [140] define navigation functions, prove the count of saddles and construct [Equation \(15.12\)](#); Connolly, Burns and Weiss [30] introduced harmonic potentials and Barraquand and Latombe [10] the random-walk escape. Reynolds [138] is the flocking paper and Olfati-Saber [122] its control-theoretic form; Choset et al. [28, ch. 4] and LaValle [99] give the textbook treatments, and Khalil [82] the two stability results this chapter borrows: LaSalle's invariance principle and the indirect method.

15.13 Exercises

Exercise 15.1 (★★ Derivation). (a) Derive the repulsive force [Equation \(15.5\)](#) from the potential [Equation \(15.4\)](#) and show that the force is continuous at $\rho = \rho_0$. (b) Give ρ and $\nabla\rho$ for a disk and for a segment obstacle ([Chapter 2](#)), and say where $\nabla\rho$ is discontinuous. (c) Derive the corrected force [Equation \(15.9\)](#) from [Equation \(15.8\)](#) and show that at the goal it vanishes for $n \geq 2$ but not for $n = 1$.

Exercise 15.2 (★). (a) Verify that the hybrid potential [Equation \(15.3\)](#) and its force are continuous at $d = d^*$. (b) Show that clipping the quadratic force at the speed v_{max} gives the same velocity command as the hybrid potential with $d^* = v_{\text{max}}/k_{\text{att}}$. (c) Without obstacles and without a speed limit, solve $\dot{\mathbf{p}} = -k_{\text{att}}(\mathbf{p} - \mathbf{p}_g)$ and show that the drone never reaches

the goal exactly; how long does it take to get within ε_g from a distance D ?

Exercise 15.3 (★★). Take the scene of [Example 15.5](#) and the axis $y = 4$. (a) Derive [Equation \(15.7\)](#), show that it has a root in $(1, 3)$, and compute the root by bisection. (b) Estimate the Hessian of U at the root by central differences of the analytic force and classify the critical point. Which starts are trapped? (c) Repeat (b) for the stopping point $(2.676, 4.1)$ of the basin experiment and explain why the second obstacle turns a saddle into a minimum.

Exercise 15.4 (★★). (a) Show that with the factor d^n the total potential is non-negative with its global minimum at the goal, and that for $n \geq 2$ the goal is a critical point. (b) Run the corrected field with $n = 1$ and a goal tolerance below the chatter amplitude $\Delta t v_{\max}$, `simulate((8,6), (5.4,4), DISK, replace(prm, n_gnron=1, goal_tol=1e-3))`, describe the motion near the goal and explain it with [Exercise 15.1\(c\)](#); repeat with $n = 2$ and with the chapter's tolerance $\varepsilon_g = 0.05$ and say what each run reports. (c) Run the local-minimum case of [Section 15.7.1](#) with $n = 2$ to show that the correction does not remove local minima elsewhere, and explain why the stopping point moves.

Exercise 15.5 (★★). (a) Without obstacles and without a speed limit, write the update of [Algorithm 15.2](#) for the quadratic bowl as a linear recursion for the error $\mathbf{p}_k - \mathbf{p}_g$ and find the values of $k_{\text{att}}\Delta t$ for which it converges monotonically, with alternating sign, or diverges. (b) Generalize to an arbitrary minimum with Hessian $\mathbf{H}$ and derive the condition of [Proposition 15.10](#). (c) Derive [Equation \(15.10\)](#), evaluate it for the widths 0.6, 0.8 and 1.0, and explain which runs of [Table 15.3](#) the linear analysis does and does not describe.

Exercise 15.6 (★★). (a) Derive [Equation \(15.11\)](#) for the passage of [Section 15.7.4](#) and locate its root numerically. (b) With $k_{\text{rep}} = 1$ and $\rho_0 = 2$ fixed, find by experiment the smallest k_{att} for which the drone from $(0, 4.3)$ enters the passage of width 0.6. (c) Rerun [Example 15.5](#) with that k_{att} and report the minimum clearance from $(0, 4.5)$; what does the comparison say about tuning one pair of gains for a whole map? (d) Explain why a waypoint inside the passage removes the problem without touching the gains.

Exercise 15.7 (★). (a) Explain why the force $\mathbf{F}$ is continuous but not continuously differentiable across the influence boundary $\rho = \rho_0$ ([Proposition 15.6](#)), and what that costs a controller that estimates the stiffness of [Equation \(15.10\)](#) by finite differences of the force. (b) The stuck detector of line 8 fires when the drone has moved less than δ in the last W steps. Explain why it cannot distinguish a local minimum from a very flat region of U , give one scene of each kind, and argue why the caller of [Algorithm 15.2](#) should treat both in the same way.

Exercise 15.8 (★★★★ Coding). Extend `simulate_swarm()` with Reynolds's alignment and cohesion rules, the velocity update [Equation \(15.14\)](#) over the neighbors within ρ_a and a spring toward their centroid. Release twenty drones with random positions and velocities and plot the spread of the velocities and of the positions against time. Show that the velocities converge to a common value when the neighborhood graph stays connected, construct an initial configuration in which the swarm splits into two flocks that never meet, and relate both outcomes to the consensus results of [Chapter 23](#).

Exercise 15.9 (★★★★ Coding). The Week 7 coding exercise of the study plan ([Appendix A](#)): implement a potential-field controller in the plane and create its failure cases, without reading `ch15_potential_fields.py` until you are stuck. (a) Implement [Algorithms 15.1](#) and [15.2](#) for disk and segment obstacles and check your forces against central differences of your potential. (b) Reproduce the four failure modes of [Section 15.7](#) in scenes of your own – a local minimum, a goal inside ρ_0 (then fix it with $n = 2$), an oscillating and a crashing run in a passage, a free passage never entered – each with a figure and the number that explains it (the root of F_x , the offset d , the stability number $\Delta t \lambda_{\max}$). (c) Sweep a grid of starts as in [Section 15.8.1](#) for a U-shaped obstacle and measure the fraction that arrives, with and

without escapes. (d) Plan a path on a grid with A^* ([Chapter 4](#)), inflate the obstacles, use your controller as the local layer along the waypoints, compare the arrival fraction with (c), and report what happens when a waypoint lies within ρ_0 of a wall.

Part V

Sampling-Based Motion Planning

Rapidly-Exploring Random Trees

Learning objectives

- Explain why grids fail in large or high-dimensional continuous spaces, and what a sampling-based planner does instead.
- State the four primitives of RRT exactly (SAMPLE with goal bias, NEAREST, STEER with step size η , COLLISIONFREE by subdivision) and trace RRT by hand on a small map.
- Explain the Voronoi bias, prove that RRT is probabilistically complete, and explain why it is not optimal.
- Use RRT-Connect, kinodynamic RRT and path shortcutting, and estimate the cost of nearest-neighbor search and collision checking.
- Implement RRT in Python for 2D and 3D box worlds, run it in a 3D obstacle field, and decide when to use grid search and when to sample.

16.1 Why this matters for a drone swarm

Every planner of [Chapters 3 to 6](#) and every multi-agent planner of [Chapters 7 to 11](#) searches a graph that somebody built first, usually a grid. On a 100×100 city map with one-meter cells that is a fine choice. Now let the drone fly through a $100 \times 100 \times 50$ m volume between buildings, cranes and trees, with a required accuracy of ten centimeters. The grid has 5×10^8 cells ([Table 16.1](#)); it needs half a gigabyte to store one byte per cell, and A^* has to touch a good fraction of it before it reaches the goal. Add the velocity of the drone to the state, because a fast drone cannot turn a corner in place, and the grid has 5×10^{14} cells: no computer holds it. This is the **curse of dimensionality**: the number of cells grows as N^d with the resolution N per axis and the dimension d , and so does the work of any method that must visit them one by one.

Sampling-based planning turns the problem around. Instead of enumerating the space it draws random points from it, keeps the collision-free ones and connects them by short collision-free segments. The memory is proportional to the number of samples, not to the size of the space, and the same code runs in two, three or twelve dimensions. The rapidly-exploring random tree (RRT) of LaValle [98] is the simplest and most used member of this family: it grows a tree from the start, one random sample at a time, and stops when the tree reaches the goal. In the hybrid architecture of [Chapter 24](#) it is the continuous planner that takes over when the global grid is too coarse for the local geometry or too large to build ([Section 16.10](#)). Week 8 of the study plan ([Appendix A](#)) asks you to implement a sampling-based planner in a 3D obstacle field and to compare it with grid A^* ; this chapter and [Chapter 17](#) are its material.

Table 16.1. The curse of dimensionality. Sizes of a uniform grid over a 100×100 m map, a $100 \times 100 \times 50$ m volume, and the same volume with the velocity (± 5 m/s per axis) added to the state. The last column is the number of neighbors of a cell in the fully connected lattice, $3^d - 1$. A tree of 10^4 vertices with three coordinates each takes about 240 kB.

Space	Resolution	Cells per axis	Cells	Memory	Neighbors
2D map	0.5 m	200	4.0×10^4	40 kB	8
2D map	0.1 m	1000	1.0×10^6	1 MB	8
3D volume	0.5 m	200, 200, 100	4.0×10^6	4 MB	26
3D volume	0.1 m	1000, 1000, 500	5.0×10^8	500 MB	26
6D position+velocity	0.5 m, 0.5 m/s	as above, 20	3.2×10^{10}	32 GB	728
6D position+velocity	0.1 m, 0.1 m/s	as above, 100	5.0×10^{14}	500 TB	728

Section 16.2 gives the idea, Section 16.3 the exact primitives, Section 16.4 the algorithm and Section 16.5 a worked example. Section 16.6 proves probabilistic completeness and shows why RRT is not optimal. Section 16.7 covers RRT-Connect, kinodynamic RRT, shortcutting and planning in 3D, Section 16.8 the collision checks, nearest-neighbor search and parameters, and Section 16.9 the choice between grid search and sampling.

16.2 The idea in plain words

Imagine throwing darts at a map. The tree, which starts as the single start point, reacts to each dart in the same way: it finds the vertex closest to the dart, moves from that vertex a fixed distance η toward the dart, and, if that short step does not cross an obstacle, adds the end of the step as a new vertex with the old vertex as its parent. The dart itself is thrown away. After a few hundred darts the tree has branches everywhere, and as soon as one branch enters a small region around the goal, the chain of parents from that vertex back to the root is a collision-free path.

Why does this explore so quickly, without any heuristic pointing to the goal? Because of the way the nearest vertex is chosen. Figure 16.1 shows a small tree in an empty square together with the **Voronoi region** of every vertex, the set of points closer to this vertex than to any other. A dart that lands in the region of vertex $\mathbf{v}$ makes $\mathbf{v}$ the parent of the new vertex, so with uniform darts the probability that $\mathbf{v}$ is *selected* equals the area of its region divided by the area of the map; whether the step is then accepted is up to the collision check. The tips of branches that point into empty space own the largest regions, so they are selected most often, and every accepted extension pushes the tree further into the empty space. In the figure the largest region covers 20.6 % of the square and the three largest together 59.3 %, while an even share would be 6.7 % per vertex; the smallest region, deep inside the tree, covers 0.7 %. This is the **Voronoi bias** of RRT: the tree is pulled toward the largest unexplored regions until they have been filled.

Key idea

Throw random darts at the space; each dart pulls the nearest vertex of the tree one step of length η toward it. A vertex is selected with probability equal to the size of its Voronoi region, and pulled if the step is collision-free, so the tree rushes into the largest unexplored regions without any heuristic. Nothing, however, makes its paths short: RRT finds *a* path, not the best one.

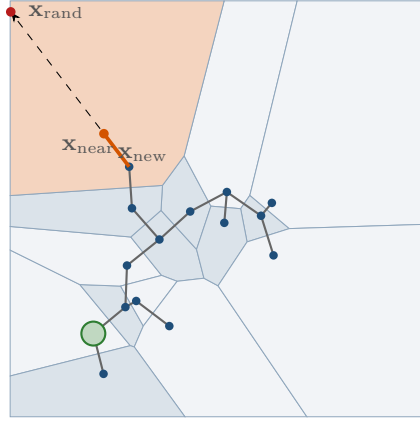


Figure 16.1. The Voronoi bias. A 15-vertex RRT in an empty 10×10 square (step size $\eta = 1$) and the Voronoi regions of its vertices. The vertex $\mathbf{x}_{\text{near}}$ at the tip of the longest branch owns the orange region, 20.6 % of the square, so one uniform sample in five extends it; the sample $\mathbf{x}_{\text{rand}}$ shown does, and the orange step of length η pushes the branch further into the empty part of the map. The small regions inside the tree are almost never chosen.

16.3 Problem statement and notation

16.3.1 The space and the query

Chapter 2 introduced the configuration space $\mathcal{C}$, its obstacle region $\mathcal{C}_{\text{obs}}$ and its free space $\mathcal{C}_{\text{free}}$. The sampling-based planning literature writes the same objects as $\mathcal{X}$, $\mathcal{X}_{\text{obs}}$ and $\mathcal{X}_{\text{free}}$, because the space is often a *state* space that contains velocities as well as positions (Section 16.7.2); this chapter and Chapter 17 follow that usage. For a drone planned as a sphere of radius r , a configuration is the position of its center, $\mathbf{x} \in \mathbb{R}^d$ with $d = 2$ or 3 , and $\mathcal{X}_{\text{obs}}$ is the union of the obstacles inflated by r (the Minkowski sum of Chapter 2).

Definition 16.1 (Sampling-based planning problem). Let $\mathcal{X} \subset \mathbb{R}^d$ be a bounded box, the **configuration space**, let $\mathcal{X}_{\text{obs}} \subset \mathcal{X}$ be a closed set, the **obstacle region**, and let $\mathcal{X}_{\text{free}} = \mathcal{X} \setminus \mathcal{X}_{\text{obs}}$ be the **free space**. A query consists of an initial configuration $\mathbf{x}_{\text{init}} \in \mathcal{X}_{\text{free}}$ and a **goal region** $X_{\text{goal}} \subseteq \mathcal{X}_{\text{free}}$; in this book the goal region is the ball $X_{\text{goal}} = \{\mathbf{x} : \|\mathbf{x} - \mathbf{x}_{\text{goal}}\| \leq r_{\text{goal}}\}$ of radius $r_{\text{goal}} > 0$ around a goal point $\mathbf{x}_{\text{goal}}$. A **solution** is a continuous path $\sigma : [0, 1] \rightarrow \mathcal{X}_{\text{free}}$ with $\sigma(0) = \mathbf{x}_{\text{init}}$ and $\sigma(1) \in X_{\text{goal}}$. The planners of this chapter return polygonal paths, sequences of configurations $\mathbf{x}_0 = \mathbf{x}_{\text{init}}, \mathbf{x}_1, \dots, \mathbf{x}_k$ whose connecting segments lie in $\mathcal{X}_{\text{free}}$; the **length** of such a path is $\sum_i \|\mathbf{x}_i - \mathbf{x}_{i-1}\|$, the path length of Chapter 2.

The tree $\mathcal{T} = (V, E)$ built by RRT has a vertex set $V \subset \mathcal{X}_{\text{free}}$ rooted at $\mathbf{x}_{\text{init}}$ and one directed edge $(\mathbf{x}_{\text{parent}}, \mathbf{x})$ per vertex except the root, each a collision-free segment; the path from the root to any vertex is therefore a solution as soon as the vertex lies in X_{goal} .

16.3.2 The four primitives

RRT is defined by four operations, and every variant in this chapter and the next changes one of them. Figure 16.2 shows them together.

Definition 16.2 (Sample, Nearest, Steer, CollisionFree). Let $\eta > 0$ be the **step size** and $p_{\text{goal}} \in [0, 1)$ the **goal bias**.

- **SAMPLE()** returns $\mathbf{x}_{\text{goal}}$ with probability p_{goal} and otherwise a configuration drawn uniformly at random from $\mathcal{X}$ (not from $\mathcal{X}_{\text{free}}$: the sample may lie inside an obstacle, the collision check below takes care of it). Successive samples are independent. (Ap-

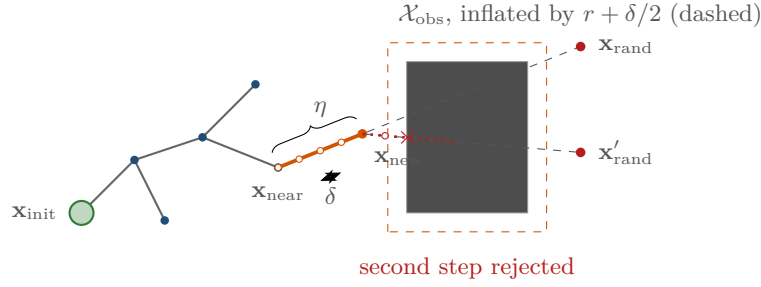


Figure 16.2. The primitives of one iteration. The sample $\mathbf{x}_{\text{rand}}$ selects the nearest vertex $\mathbf{x}_{\text{near}}$; STEER moves at most η toward the sample and proposes $\mathbf{x}_{\text{new}}$; COLLISIONFREE tests points spaced δ apart along the new segment against the obstacle inflated by $r + \delta/2$ (dashed), which is conservative by Proposition 16.3. The second attempt, from $\mathbf{x}_{\text{new}}$ toward $\mathbf{x}'_{\text{rand}}$, is rejected because its second test point falls inside the inflated region; no vertex is added.

pendix B.3 reviews random vectors, densities and expectations, the probability the completeness argument of Section 16.6 rests on.)

- $\text{Nearest}(V, \mathbf{x}) = \arg \min_{\mathbf{v} \in V} \|\mathbf{v} - \mathbf{x}\|$ is the vertex of the tree closest to $\mathbf{x}$ in the Euclidean norm, ties broken arbitrarily.
- $\text{Steer}(\mathbf{x}_{\text{near}}, \mathbf{x}_{\text{rand}})$ moves from $\mathbf{x}_{\text{near}}$ toward $\mathbf{x}_{\text{rand}}$ by at most η :

$$\text{Steer}(\mathbf{x}_{\text{near}}, \mathbf{x}_{\text{rand}}) = \begin{cases} \mathbf{x}_{\text{rand}} & \text{if } \|\mathbf{x}_{\text{rand}} - \mathbf{x}_{\text{near}}\| \leq \eta, \\ \mathbf{x}_{\text{near}} + \eta \frac{\mathbf{x}_{\text{rand}} - \mathbf{x}_{\text{near}}}{\|\mathbf{x}_{\text{rand}} - \mathbf{x}_{\text{near}}\|} & \text{otherwise.} \end{cases} \quad (16.1)$$

- $\text{CollisionFree}(\mathbf{a}, \mathbf{b})$ is true if and only if the closed segment $[\mathbf{a}, \mathbf{b}] = \{\mathbf{a} + t(\mathbf{b} - \mathbf{a}) : t \in [0, 1]\}$ lies in $\mathcal{X}_{\text{free}}$.

A segment has infinitely many points, so COLLISIONFREE is implemented by testing finitely many of them. The following proposition says how to do this without ever missing an obstacle. Write $\text{dist}(\mathbf{x}, \mathcal{O})$ for the Euclidean distance from $\mathbf{x}$ to the workspace obstacles $\mathcal{O}$; for axis-aligned boxes and spheres it has a closed form (Section 16.8.1).

Proposition 16.3 (Collision checking by subdivision). *Let $r \geq 0$ be the robot radius, so that a point $\mathbf{x}$ is free if and only if $\text{dist}(\mathbf{x}, \mathcal{O}) > r$, and let $\delta > 0$ be the **check resolution**. Let $\mathbf{p}_0 = \mathbf{a}, \mathbf{p}_1, \dots, \mathbf{p}_n = \mathbf{b}$ be points on the segment $[\mathbf{a}, \mathbf{b}]$ with $\|\mathbf{p}_i - \mathbf{p}_{i-1}\| \leq \delta$ for all i . If $\text{dist}(\mathbf{p}_i, \mathcal{O}) > r + \delta/2$ for every i , then every point $\mathbf{p}$ of $[\mathbf{a}, \mathbf{b}]$ has $\text{dist}(\mathbf{p}, \mathcal{O}) > r$, that is, the segment is collision-free.*

Proof. Take any $\mathbf{p} \in [\mathbf{a}, \mathbf{b}]$. It lies between two consecutive test points, so its distance to the nearer of them, $\mathbf{p}_i$ say, is at most $\delta/2$. The function $\mathbf{x} \mapsto \text{dist}(\mathbf{x}, \mathcal{O})$ is 1-Lipschitz (an infimum of distances, each of which changes by at most $\|\mathbf{p} - \mathbf{p}_i\|$ when $\mathbf{p}_i$ is replaced by $\mathbf{p}$), so $\text{dist}(\mathbf{p}, \mathcal{O}) \geq \text{dist}(\mathbf{p}_i, \mathcal{O}) - \|\mathbf{p} - \mathbf{p}_i\| > r + \delta/2 - \delta/2 = r$. $\square$

The test rejects some free segments, those that pass within $r + \delta/2$ of an obstacle, and needs $\lceil \|\mathbf{b} - \mathbf{a}\| / \delta \rceil + 1$ point tests. It never accepts a colliding segment, whatever the shape or thickness of the obstacles, which is what makes it safe for a drone. Without the inflation by $\delta/2$ a wall thinner than δ can slip between two test points (Section 16.8.4); the self-test of `ch16_rrt.py` checks exactly this case, a wall of thickness 0.02 with $\delta = 0.05$, at forty segment angles through a point inside the wall and at forty parallel crossings of it.

Algorithm 16.1. Rapidly-exploring random tree with goal bias [98, 100]

Input: $\mathbf{x}_{\text{init}}$, $\mathbf{x}_{\text{goal}}$, goal radius r_{goal} , step size η , goal bias p_{goal} , iteration budget N
Output: a collision-free polygonal path from $\mathbf{x}_{\text{init}}$ into X_{goal} , or *failure*

```

1 Function RRT( $\mathbf{x}_{\text{init}}$ ,  $\mathbf{x}_{\text{goal}}$ ,  $N$ ):
2    $V \leftarrow \{\mathbf{x}_{\text{init}}\}; E \leftarrow \emptyset$ 
3   for  $i \leftarrow 1$  to  $N$  do
4      $\mathbf{x}_{\text{rand}} \leftarrow \text{Sample}()$            //  $\mathbf{x}_{\text{goal}}$  with probability  $p_{\text{goal}}$ , else uniform in  $\mathcal{X}$ 
5      $\mathbf{x}_{\text{near}} \leftarrow \text{Nearest}(V, \mathbf{x}_{\text{rand}})$ 
6      $\mathbf{x}_{\text{new}} \leftarrow \text{Steer}(\mathbf{x}_{\text{near}}, \mathbf{x}_{\text{rand}})$  // at most  $\eta$  from  $\mathbf{x}_{\text{near}}$ , Equation (16.1)
7     if  $\text{CollisionFree}(\mathbf{x}_{\text{near}}, \mathbf{x}_{\text{new}})$  then
8        $V \leftarrow V \cup \{\mathbf{x}_{\text{new}}\}; E \leftarrow E \cup \{(\mathbf{x}_{\text{near}}, \mathbf{x}_{\text{new}})\}$ 
9       if  $\|\mathbf{x}_{\text{new}} - \mathbf{x}_{\text{goal}}\| \leq r_{\text{goal}}$  then
10        return  $\text{ExtractPath}(V, E, \mathbf{x}_{\text{new}})$  // follow the parents back to  $\mathbf{x}_{\text{init}}$ 
11  return failure

```

16.4 The algorithm

Algorithm 16.1 is the whole algorithm. Line 2 starts the tree with the root. Each iteration draws one sample (line 4); the goal bias makes every twentieth sample (for $p_{\text{goal}} = 0.05$) the goal itself, so that the tree is pulled toward the goal once in a while instead of waiting for a uniform sample to land in the small goal region. Line 5 is the expensive step, a search over all vertices (Section 16.8.2). Line 6 limits the new edge to length η : without the limit a far-away sample would produce a long edge that is almost always blocked. Line 7 is the only place where the obstacles enter, and it tests the whole segment, not just the new vertex. If the segment is free, the new vertex joins the tree with $\mathbf{x}_{\text{near}}$ as its parent (line 8); if not, the sample is simply forgotten. The goal test of line 9 asks whether the new vertex lies in the goal ball: exact equality with $\mathbf{x}_{\text{goal}}$ never happens with continuous samples, which is why a goal region is part of the problem statement. The implementation of Section 16.8 finally appends $\mathbf{x}_{\text{goal}}$ itself when the segment from the entry vertex to it is free, so that the path ends exactly at the goal.

Kuffner and LaValle [94] package lines 5–8 as one operation $\text{EXTEND}(\mathcal{T}, \mathbf{x})$ with three outcomes: TRAPPED if the segment was blocked, ADVANCED if a new vertex was added short of $\mathbf{x}$, and REACHED if the new vertex is $\mathbf{x}$ itself, which happens exactly when $\|\mathbf{x} - \mathbf{x}_{\text{near}}\| \leq \eta$. Algorithm 16.2 is written in these terms.

The Voronoi bias, quantified. With $p_{\text{goal}} = 0$ the sample is uniform on $\mathcal{X}$, so the probability that vertex $\mathbf{v}$ is chosen by line 5 is

$$\mathbb{P}(\mathbf{x}_{\text{near}} = \mathbf{v}) = \frac{\text{vol}(\text{Vor}(\mathbf{v}) \cap \mathcal{X})}{\text{vol}(\mathcal{X})}, \quad \text{Vor}(\mathbf{v}) = \{\mathbf{x} : \|\mathbf{x} - \mathbf{v}\| \leq \|\mathbf{x} - \mathbf{w}\| \text{ for all } \mathbf{w} \in V\}, \quad (16.2)$$

the relative volume of its Voronoi region. LaValle [98] observed that the distribution of the vertices converges to the sampling distribution, so a uniformly sampled RRT eventually covers $\mathcal{X}_{\text{free}}$ uniformly. Goal samples always extend the vertex nearest to the goal, which hurts when that vertex sits against an obstacle (Section 16.8.4).

Table 16.2. Trace of the first ten iterations of Algorithm 16.1 on Example 16.4. Vertices are numbered in the order of insertion, vertex 0 being the start. Iteration 4 drew the goal (goal bias); from iteration 7 on every step is rejected by the wall $[3, 5] \times [0, 6]$, as the text explains.

Iteration	$\mathbf{x}_{\text{rand}}$	Nearest vertex	$\mathbf{x}_{\text{near}}$	$\mathbf{x}_{\text{new}}$	Segment free?
1	(9.50, 1.44)	0	(1.00, 1.00)	(1.50, 1.03)	yes, vertex 1 added
2	(3.12, 4.23)	1	(1.50, 1.03)	(1.72, 1.47)	yes, vertex 2
3	(4.09, 5.50)	2	(1.72, 1.47)	(1.98, 1.90)	yes, vertex 3
4	(9.00, 9.00) goal	3	(1.98, 1.90)	(2.33, 2.26)	yes, vertex 4
5	(5.38, 3.30)	4	(2.33, 2.26)	(2.80, 2.42)	yes, vertex 5
6	(3.03, 4.53)	5	(2.80, 2.42)	(2.86, 2.92)	yes, vertex 6
7	(4.03, 2.03)	5	(2.80, 2.42)	(3.28, 2.27)	no, enters the wall
8	(7.50, 2.80)	6	(2.86, 2.92)	(3.36, 2.90)	no
9	(9.81, 9.62)	6	(2.86, 2.92)	(3.22, 3.26)	no
10	(5.41, 2.77)	6	(2.86, 2.92)	(3.36, 2.89)	no

16.5 A worked example

Example 16.4 (RRT on a map with two walls). The configuration space is the square $\mathcal{X} = [0, 10]^2$ with two rectangular obstacles, $[3, 5] \times [0, 6]$ and $[6, 8] \times [4, 10]$, so that the only way from the bottom-left to the top-right corner passes between them. The robot is a point ($r = 0$), the start is $\mathbf{x}_{\text{init}} = (1, 1)$, the goal $\mathbf{x}_{\text{goal}} = (9, 9)$ with $r_{\text{goal}} = 0.5$, the step size $\eta = 0.5$, the goal bias $p_{\text{goal}} = 0.05$ and the check resolution $\delta = 0.05$. The random generator is seeded with 1. The instance is `worked_example_world()` in `ch16_rrt.py`, and every number below is printed by its self-test or by the figure script `gen_ch16_tree.py`.

Table 16.2 traces the first ten iterations. The first sample lies far to the right, so STEER produces a step of exactly $\eta = 0.5$ from the start. The next samples lie toward the north-east, and the tree grows as a single chain of six vertices in that direction, exactly what the Voronoi bias predicts for a tree with one branch: its tip owns almost the whole map. At iteration 7 the chain has reached $x \approx 2.9$ and the wall at $x = 3$ blocks every step toward the right half of the map. The tree is now stuck in an unproductive regime: samples on the right, which are most of them, keep selecting the two vertices closest to the wall, and COLLISIONFREE keeps rejecting the step; only samples on the left of the wall, probability about 0.3, add vertices. This is why the first panel of Figure 16.3 shows only 17 vertices after 50 iterations: 16 of the 50 extensions succeeded and 34 were rejected.

Figure 16.3 shows the tree after 50, 200 and 1000 iterations. By iteration 200 the tree has 103 vertices and has found the gap between the walls; the goal region is entered at iteration 402, when the tree has 245 vertices, by a vertex at (9.14, 8.60) at distance $0.42 < r_{\text{goal}}$ from the goal. The final segment to $\mathbf{x}_{\text{goal}}$ is free and is appended, which gives a path of 50 vertices and length 23.85. Of the 402 iterations, 158 were rejected by the collision check, and the checks tested 3965 points (a counter the self-test prints), about ten per iteration, as expected for $\eta/\delta = 10$. The path is far from optimal: the shortest path in $\mathcal{X}_{\text{free}}$ hugs the corners (3, 6), (5, 6), (6, 4) and (8, 4) and has length 16.72 in the limit of zero clearance, and 17.20 when the corners are rounded off by the clearance 0.075 of the collision checker; the self-test prints both. The RRT path is 43% longer than the zero-clearance optimum and 39% longer than the 17.20 reference. Section 16.6 explains why this is not bad luck, and Section 16.7.3 how most of the excess is removed afterward.

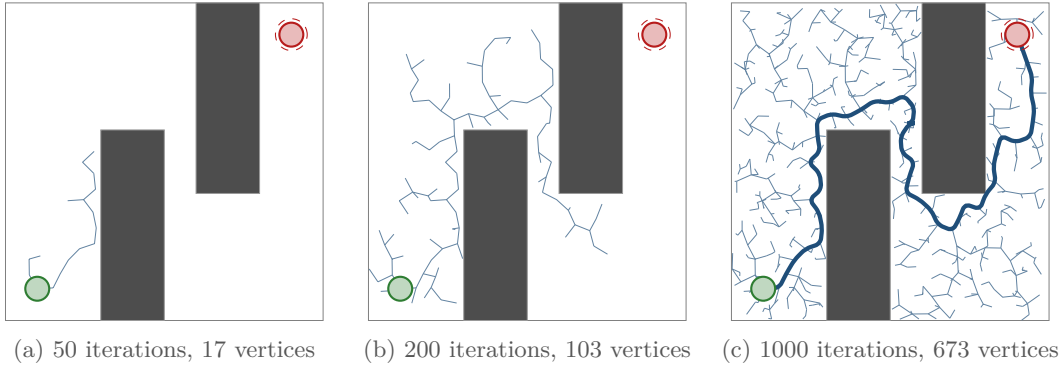


Figure 16.3. The tree of Example 16.4 after (a) 50, (b) 200 and (c) 1000 iterations, grown past the first solution for the last panel. The dashed circle is the goal region of radius $r_{\text{goal}} = 0.5$. In (a) the tree has hit the wall $x = 3$ and grows only on the left; in (b) it has found the gap between the two walls; in (c) the whole free space is covered and the blue path, found at iteration 402, is the chain of parents from the vertex that first entered the goal region. The path has 50 vertices, length 23.85, and visibly wanders: the shortest path has length 16.72.

16.6 Properties: completeness, optimality, complexity

16.6.1 Probabilistic completeness

RRT is a randomized algorithm, and its guarantee is a statement about probabilities. A planner is **probabilistically complete** if, whenever a solution with positive clearance exists – a *robustly feasible* query, the hypothesis made precise in Theorem 16.5 – the probability that it has found one after n iterations tends to 1 as $n \rightarrow \infty$. This is weaker than the completeness of A^* on a finite graph, and it says nothing when no solution exists: RRT then spends its budget N and reports failure, without knowing whether the query was infeasible or merely hard.

Theorem 16.5 (Probabilistic completeness of RRT; LaValle and Kuffner 2001). *Let $\mathcal{X}$ be a bounded box, $\eta > 0$, $0 \leq p_{\text{goal}} < 1$ and $r_{\text{goal}} > 0$. Suppose there is a path σ of finite length from $\mathbf{x}_{\text{init}}$ to $\mathbf{x}_{\text{goal}}$ with **clearance** $\delta_c > 0$: every segment that lies within distance δ_c of σ is accepted by COLLISIONFREE. Then the probability that Algorithm 16.1 has not found a solution after n iterations is at most $m/(np)$ for constants m and $p > 0$ that depend on σ , η , δ_c , r_{goal} and $\mathcal{X}$ but not on n ; in particular it tends to 0 as $n \rightarrow \infty$.*

Proof. Let $\nu = \min(\eta/3, \delta_c/4, r_{\text{goal}})$ and choose points $\mathbf{x}_0 = \mathbf{x}_{\text{init}}, \mathbf{x}_1, \dots, \mathbf{x}_m = \mathbf{x}_{\text{goal}}$ along σ with $\|\mathbf{x}_{k+1} - \mathbf{x}_k\| \leq \nu$ for all k ; a path of finite length admits such a sequence with $m \leq \lceil \text{length}(\sigma)/\nu \rceil + 1$. Finite length is no restriction: the image of a path with clearance δ_c is compact, so finitely many of the balls of radius $\delta_c/2$ centered on its points cover it, and joining the centers of consecutive overlapping balls gives a polygonal path of finite length inside the same tube, with clearance at least $\delta_c/2$. Let B_k be the open ball of radius ν around $\mathbf{x}_k$; since B_k lies within δ_c of σ it is contained in $\mathcal{X}_{\text{free}} \subseteq \mathcal{X}$, so a uniform sample lands in it with probability $p = (1 - p_{\text{goal}}) \text{vol}(B_k)/\text{vol}(\mathcal{X}) > 0$, the same for every k .

Claim: if the tree has a vertex $\mathbf{v} \in B_k$ and the current sample satisfies $\mathbf{x}_{\text{rand}} \in B_{k+1}$, then the iteration adds a vertex in B_{k+1} . Let $\mathbf{v}' = \text{Nearest}(V, \mathbf{x}_{\text{rand}})$. Then $\|\mathbf{v}' - \mathbf{x}_{\text{rand}}\| \leq \|\mathbf{v} - \mathbf{x}_{\text{rand}}\| \leq \|\mathbf{v} - \mathbf{x}_k\| + \|\mathbf{x}_k - \mathbf{x}_{k+1}\| + \|\mathbf{x}_{k+1} - \mathbf{x}_{\text{rand}}\| < 3\nu \leq \eta$, so STEER returns $\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{rand}}$ itself. Both endpoints of the segment $[\mathbf{v}', \mathbf{x}_{\text{rand}}]$ lie within $3\nu + \nu = 4\nu \leq \delta_c$ of $\mathbf{x}_{k+1} \in \sigma$, and a ball is convex, so the whole segment lies within δ_c of σ and COLLISIONFREE accepts it. Hence $\mathbf{x}_{\text{rand}} \in B_{k+1}$ becomes a vertex.

Let K_n be the largest k such that the tree after n iterations has a vertex in B_k ; $K_0 \geq 0$ because $\mathbf{x}_0 \in B_0$ is the root. By the claim, whenever $K_n = k < m$ and the next sample lands

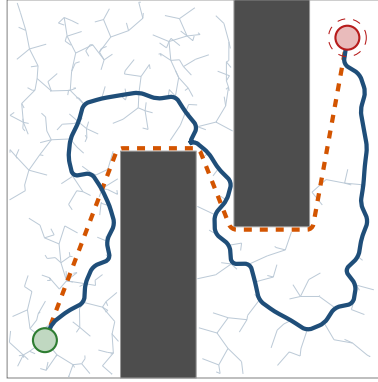


Figure 16.4. RRT is not optimal. The worst of 100 runs of [Example 16.4](#) (seed 14, solution at iteration 746): the blue RRT path has length 28.35, the dashed orange shortest path (clearance 0.075) 17.20. The path wanders because the parent of every vertex is fixed at insertion and because every edge points toward a random sample. The gray tree shows that the tree *had* vertices close to the shortest path; they were simply not on the chain of parents that reached the goal first.

in B_{k+1} , $K_{n+1} \geq k + 1$. Samples are independent of the past and each lands in the required ball with probability at least p , so the number of iterations T until $K_n = m$ is stochastically at most a sum of m independent geometric variables with success probability p , whence $\mathbb{E}[T] \leq m/p$ and, by Markov's inequality, $\mathbb{P}(T > n) \leq m/(np)$. When $K_n = m$ the tree has a vertex within $\nu \leq r_{\text{goal}}$ of $\mathbf{x}_{\text{goal}}$, that is, in X_{goal} , and line 9 of [Algorithm 16.1](#) has returned a path. $\square$

Three things in the proof deserve attention.

- *The role of η .* The step size only enters through $\nu \leq \eta/3$: a smaller η means smaller balls, a smaller p and more iterations, but any $\eta > 0$ gives completeness. A larger η does not hurt this bound, because STEER moves *at most* η and reaches nearby samples exactly.
- *The role of the goal bias.* Only $p_{\text{goal}} < 1$ is needed; goal samples are simply wasted in the argument. With $p_{\text{goal}} = 1$ the algorithm is deterministic and fails on any map where the straight line from start to goal is blocked ([Exercise 16.4](#)).
- *Narrow passages.* The ball volume is proportional to ν^d , so $p \approx (1 - p_{\text{goal}}) \zeta_d \nu^d / \text{vol}(\mathcal{X})$ with ζ_d the volume of the unit ball. A passage of width w forces $\delta_c \leq w/2$ and $\nu \leq w/8$: the expected number of iterations grows like $(1/w)^d$. This **narrow-passage problem** is the known weakness of every uniform sampler and one motivation for the bidirectional search of [Section 16.7.1](#). In [Example 16.4](#) the gap is one unit wide on a ten-unit map and is found within a few hundred iterations.

The bound $m/(np)$ is loose; [Exercise 16.4](#) sharpens it to an exponential decay in n . The original argument is in LaValle and Kuffner [100]; Kleinbort et al. [86] give a modern, careful proof that also covers the kinodynamic variant of [Section 16.7.2](#), and [Chapter 17](#) reuses the ball construction for RRT*.

16.6.2 Not optimal, and not even eventually optimal

[Figure 16.3](#) already showed a path 43% longer than necessary. [Figure 16.4](#) shows the worst of 100 runs of [Example 16.4](#) with different seeds: its path has length 28.35, 65% above the 17.20 reference. Over the 100 runs the lengths range from 20.55 to 28.35, with mean 23.87 and standard deviation 1.84. Two features of [Algorithm 16.1](#) are responsible. The parent of a vertex is fixed forever at insertion, so a vertex reached early by a detour keeps its detour, and every later vertex behind it inherits it. And the path zigzags at the scale of η , because each edge points toward a random sample, not toward the goal.

Could one let the tree grow and wait for a better path to appear? No. Karaman and Frazzoli [77] proved that this never happens.

Theorem 16.6 (RRT converges to a suboptimal path; Karaman and Frazzoli 2011). *Let c^* be the length of a shortest solution and let Y_n be the length of the shortest path from $\mathbf{x}_{\text{init}}$ into X_{goal} contained in the tree after n iterations of Algorithm 16.1 (continued past the first solution). Assume, with Karaman and Frazzoli [77], that X_{goal} has non-empty interior, that some optimal path has weak clearance (it is the limit of paths of positive clearance) and that η is held fixed. Then, for $d \geq 2$,*

$$\mathbb{P}\left(\lim_{n \rightarrow \infty} Y_n = c^*\right) = 0.$$

The best path in the tree converges almost surely to a value strictly larger than the optimum.

The proof analyses the vertices nearest to the start along an optimal path and shows that, with probability one, the tree’s first edges in that direction are not aligned with the optimal path, and no later sample can repair them since edges are never changed. The remedy is to let the tree *rewire*: RRT* (Chapter 17) reconsiders the parent of every new vertex among its neighbors within a shrinking radius and reconnects existing vertices through it when that is cheaper, which restores optimality in the limit at the price of more collision checks. For RRT itself the practical remedy is post-processing (Section 16.7.3), which removes most of the excess length but cannot carry the path to the other side of an obstacle.

16.6.3 Complexity

Let n be the number of vertices. One iteration costs one nearest-neighbor query, $\mathcal{O}(n)$ with a linear scan and $\mathcal{O}(\log n)$ on average with a kd-tree (Section 16.8.2), plus one collision check of $\lceil \eta/\delta \rceil + 1$ point tests, each $\mathcal{O}(n_{\text{obs}})$ in the number of obstacles. After N iterations the total is $\mathcal{O}(N^2)$ with the scan and $\mathcal{O}(N \log N)$ with a kd-tree, plus $\mathcal{O}(N n_{\text{obs}} \eta/\delta)$ for the checks, which dominate in cluttered scenes. The memory is $\mathcal{O}(N)$ vertices with d coordinates and one parent index each, whatever the size or dimension of $\mathcal{X}$, which is the whole point of Table 16.1. What no bound tells you is *how many* iterations are needed; that depends on the narrowest passage the path must cross.

16.7 Variants and extensions

16.7.1 RRT-Connect: two trees and a greedy connection

A single tree rooted at the start has to discover the goal region by chance or by goal bias. Kuffner and LaValle [94] grow two trees, one from the start and one from the goal, and add a greedy step: whenever one tree gains a new vertex, the other tree tries to reach that vertex by repeating EXTEND as long as it makes progress. Algorithm 16.2 gives both ingredients.

Line 17 extends the current tree $\mathcal{T}_a$ toward a uniform sample exactly as RRT does. If that produced a vertex, the other tree $\mathcal{T}_b$ tries to connect to it (line 19): the CONNECT heuristic of line 10 repeats EXTEND toward the *same* target, adding one edge of length η per repetition, until the target is reached or an obstacle is hit. In open space this is a straight run of edges across the map, and this greedy extension is what makes the method fast. The trees swap roles every iteration (line 21), which keeps them balanced; no goal bias is needed, because the goal is the root of the second tree. When CONNECT returns REACHED, $\mathbf{x}_{\text{new}}$ belongs to both trees and the two root-to-vertex chains form the path, one of them reversed; it is collision-free because every edge of either tree is. Figure 16.5 shows the two trees on Example 16.4: with seed 1 they meet at iteration 164 with $68 + 29$ vertices, against 402 iterations and 245 vertices

Algorithm 16.2. RRT-Connect [94]

Input: $\mathbf{x}_{\text{init}}$, $\mathbf{x}_{\text{goal}}$, step size η , budget N
Output: a collision-free path from $\mathbf{x}_{\text{init}}$ to $\mathbf{x}_{\text{goal}}$, or *failure*

```

1 Function Extend( $\mathcal{T}$ ,  $\mathbf{x}$ ):
2    $\mathbf{x}_{\text{near}} \leftarrow \text{Nearest}(V_{\mathcal{T}}, \mathbf{x})$ ;  $\mathbf{x}_{\text{new}} \leftarrow \text{Steer}(\mathbf{x}_{\text{near}}, \mathbf{x})$ 
3   if CollisionFree( $\mathbf{x}_{\text{near}}$ ,  $\mathbf{x}_{\text{new}}$ ) then
4     add  $\mathbf{x}_{\text{new}}$  to  $\mathcal{T}$  with parent  $\mathbf{x}_{\text{near}}$ 
5     if  $\mathbf{x}_{\text{new}} = \mathbf{x}$  then return REACHED
6     return ADVANCED
7   return TRAPPED
8 Function Connect( $\mathcal{T}$ ,  $\mathbf{x}$ ):
9   repeat
10     $s \leftarrow \text{Extend}(\mathcal{T}, \mathbf{x})$ 
11  until  $s \neq \text{ADVANCED}$ 
12  return  $s$ 
13 Function RRT-Connect( $\mathbf{x}_{\text{init}}$ ,  $\mathbf{x}_{\text{goal}}$ ,  $N$ ):
14    $\mathcal{T}_a \leftarrow$  tree with root  $\mathbf{x}_{\text{init}}$ ;  $\mathcal{T}_b \leftarrow$  tree with root  $\mathbf{x}_{\text{goal}}$ 
15   for  $i \leftarrow 1$  to  $N$  do
16      $\mathbf{x}_{\text{rand}} \leftarrow$  uniform sample in  $\mathcal{X}$  // no goal bias
17     if Extend( $\mathcal{T}_a$ ,  $\mathbf{x}_{\text{rand}}$ )  $\neq$  TRAPPED then
18        $\mathbf{x}_{\text{new}} \leftarrow$  the vertex just added to  $\mathcal{T}_a$ 
19       if Connect( $\mathcal{T}_b$ ,  $\mathbf{x}_{\text{new}}$ ) = REACHED then
20         return the path  $\text{root}(\mathcal{T}_a) \rightarrow \mathbf{x}_{\text{new}}$  in  $\mathcal{T}_a$  joined with  $\mathbf{x}_{\text{new}} \rightarrow \text{root}(\mathcal{T}_b)$  in
           $\mathcal{T}_b$ , reversed if  $\mathcal{T}_a$  is the goal tree
21     Swap( $\mathcal{T}_a$ ,  $\mathcal{T}_b$ )
22   return failure

```

Table 16.3. Iterations to the first solution over 100 seeds on [Example 16.4](#), budget 3 000 iterations, $\eta = 0.5$; all runs succeed. Vertices and path lengths are averages, the lengths before any smoothing. RRT-Connect needs 2.4 times fewer iterations than RRT with the default bias and stores 2.7 times fewer vertices, but its paths are no shorter.

Planner	Median iterations	Mean iterations	Vertices	Path length
RRT, $p_{\text{goal}} = 0$	518	537	352	24.31 ± 2.00
RRT, $p_{\text{goal}} = 0.05$	425	453	275	23.87 ± 1.84
RRT, $p_{\text{goal}} = 0.5$	745	783	245	23.61 ± 1.86
RRT-Connect	178	192	101	23.69 ± 1.98

for the single tree, and the path has length 25.11, no better than before.

One run proves nothing about a randomized algorithm, so [Figure 16.6](#) and [Table 16.3](#) report 100 runs with seeds 0 to 99. RRT-Connect solves every instance within 375 iterations; by iteration 500 plain RRT with the default goal bias has solved 73 %, and its median is 425 iterations against 178. The self-test of `ch16_rrt.py` asserts this ordering of medians and means over 40 seeds. The table also isolates the goal bias: a small bias helps ($518 \rightarrow 425$ median iterations), a large one hurts badly (745), for the reason given in [Section 16.8.4](#). The tree with $p_{\text{goal}} = 0.5$ has *fewer* vertices after *more* iterations, the signature of wasted, trapped extensions.

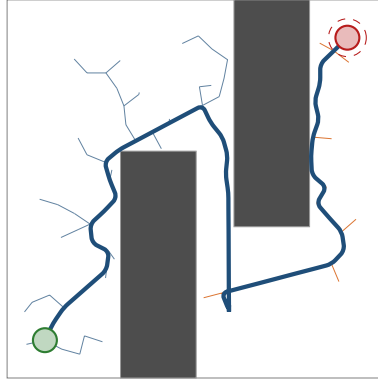


Figure 16.5. RRT-Connect on Example 16.4 (seed 1). The blue tree grows from the start, the orange tree from the goal; the trees meet at iteration 164 with $68 + 29$ vertices. The long straight runs of edges are the work of CONNECT: each one is a sequence of ADVANCED results toward one target vertex of the other tree. The path (thick blue) is as jagged as that of the single tree.

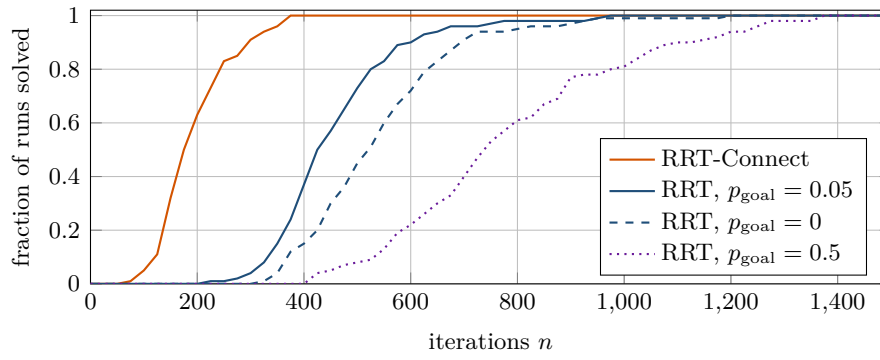


Figure 16.6. Fraction of 100 runs of Example 16.4 (seeds 0 to 99) that have found a path within a given number of iterations, for RRT with goal bias 0, 0.05 and 0.5 and for RRT-Connect. The empirical distribution of RRT-Connect lies far to the left; a goal bias of 0.5 is worse than no bias at all on this map.

16.7.2 Kinodynamic RRT

The paths of Algorithm 16.1 are polygons, and a drone cannot fly a polygon at speed: it has bounded acceleration, so its velocity is part of its state. LaValle and Kuffner [100] extended RRT to systems with dynamics; the version below uses the double integrator of Chapter 2, with state $\mathbf{x} = (\mathbf{p}, \mathbf{v}) \in \mathbb{R}^{2d}$, control $\mathbf{a}$ with $\|\mathbf{a}\| \leq a_{\max}$ held constant over a step Δt , and speed limit $\|\mathbf{v}\| \leq v_{\max}$. Two primitives change. STEER becomes forward simulation: the planner cannot move “toward” a sampled state along a straight line, because the straight line is not a feasible motion. Instead it tries m random controls, integrates each for one step, keeps the result that ends closest to the sample, and stores the control on the edge. NEAREST becomes a question rather than a formula: the Euclidean distance between two states is not the time or effort needed to move between them, since a state whose velocity points away from the sample is “far” even when its position is close. The exact answer, the cost of the optimal motion between two states, requires solving a two-point boundary value problem for every candidate vertex, far too expensive inside a nearest-neighbor query. The usual compromise is the weighted metric

$$d_w(\mathbf{x}, \mathbf{x}') = \sqrt{\|\mathbf{p} - \mathbf{p}'\|^2 + w_v^2 \|\mathbf{v} - \mathbf{v}'\|^2}, \quad (16.3)$$

with a weight w_v (units of time) that says how much a velocity mismatch counts against a position mismatch. Algorithm 16.3 is the result; line 10 is the exact zero-order-hold update of

Chapter 2. The arc flown during one step is a parabola, and the collision check samples it at spacing δ , using the fact that the speed along a constant-acceleration arc is largest at one of its ends.

Algorithm 16.3. Kinodynamic RRT for the double integrator [100]

Input: $\mathbf{x}_{\text{init}} = (\mathbf{p}_{\text{init}}, \mathbf{0})$, $\mathbf{p}_{\text{goal}}$, r_{goal} , v_{max} , a_{max} , Δt , controls per extension m , goal bias p_{goal} , velocity weight w_v , budget N

Output: a sequence of states and constant accelerations that obeys the dynamics, or *failure*

```

1 Function KinodynamicRRT( $\mathbf{x}_{\text{init}}$ ,  $\mathbf{p}_{\text{goal}}$ ,  $N$ ):
2    $V \leftarrow \{\mathbf{x}_{\text{init}}\}$ 
3   for  $i \leftarrow 1$  to  $N$  do
4     if  $\text{rand}() < p_{\text{goal}}$  then  $\mathbf{x}_{\text{rand}} \leftarrow (\mathbf{p}_{\text{goal}}, \mathbf{0})$ 
5     else  $\mathbf{x}_{\text{rand}} \leftarrow (\mathbf{p}, \mathbf{v})$  with  $\mathbf{p}$  uniform in  $\mathcal{X}$  and  $\mathbf{v}$  uniform in the ball  $\|\mathbf{v}\| \leq v_{\text{max}}$ 
6      $\mathbf{x}_{\text{near}} \leftarrow \arg \min_{\mathbf{x} \in V} d_w(\mathbf{x}, \mathbf{x}_{\text{rand}})$  // weighted metric Equation (16.3)
7      $\text{best} \leftarrow \text{nil}$ ;  $\mathbf{a}_{\text{best}} \leftarrow \text{nil}$  // convention  $d_w(\text{nil}, \cdot) = +\infty$ 
8     for  $j \leftarrow 1$  to  $m$  do
9        $\mathbf{a}_j \leftarrow$  uniform in the ball  $\|\mathbf{a}\| \leq a_{\text{max}}$ 
10       $\mathbf{x}_j \leftarrow \text{Integrate}(\mathbf{x}_{\text{near}}, \mathbf{a}_j, \Delta t)$  //  $\mathbf{p} + \Delta t \mathbf{v} + \frac{1}{2} \Delta t^2 \mathbf{a}_j$ ,  $\mathbf{v} + \Delta t \mathbf{a}_j$ 
11      if  $\|\mathbf{v}_j\| \leq v_{\text{max}}$  and the arc from  $\mathbf{x}_{\text{near}}$  to  $\mathbf{x}_j$  is collision-free and
12         $d_w(\mathbf{x}_j, \mathbf{x}_{\text{rand}}) < d_w(\text{best}, \mathbf{x}_{\text{rand}})$  then
13           $\text{best} \leftarrow \mathbf{x}_j$ ;  $\mathbf{a}_{\text{best}} \leftarrow \mathbf{a}_j$ 
14      if  $\text{best} \neq \text{nil}$  then
15        add best to  $V$  with parent  $\mathbf{x}_{\text{near}}$  and edge label  $\mathbf{a}_{\text{best}}$ 
16        if  $\|\mathbf{p}_{\text{best}} - \mathbf{p}_{\text{goal}}\| \leq r_{\text{goal}}$  then return states and controls from the root to best
17   return failure

```

The implementation `kinodynamic_rrt` in `ch16_rrt.py` solves [Example 16.4](#) for a drone with $v_{\text{max}} = 2$, $a_{\text{max}} = 2$, $\Delta t = 0.5$, $m = 8$ controls per extension, $w_v = 0.5$ and seed 2: it finds a trajectory at iteration 680 with 512 vertices, 41 steps or 20.5 s of flight, arriving at speed 0.54. The self-test verifies that consecutive states satisfy $\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{a}_k$ with the matrices of [Chapter 2](#) and that no control or velocity exceeds its limit. Three lessons carry over to every planner with dynamics: the tree needs more vertices (512 against 245) because velocities add dimensions; the solution is a trajectory that can be flown as it is; and the metric of line 6 strongly affects the growth of the tree. Kinodynamic planning has a large literature [99]; in the hybrid architecture of [Chapter 24](#) the geometric RRT with post-processing, followed by the controller of [Chapter 21](#), is usually the simpler choice.

16.7.3 Post-processing: shortcutting and smoothing

A raw RRT path is safe but ugly. **Shortcutting** repairs most of the damage with the same collision checker: pick two points on the path, and if the straight segment between them is free, replace the part of the path in between by the segment. By the triangle inequality the length can only decrease. [Algorithm 16.4](#) gives the randomized version of Geraerts and Overmars [53] and a greedy variant.

[Figure 16.7](#) shows the effect on the worked example: 100 random attempts bring the length from 23.85 to 19.91, and greedy pruning to 18.39 with only 7 vertices, within 7% of the shortest path with the same clearance. The self-test asserts that neither method ever

Algorithm 16.4. Path post-processing: random shortcutting and greedy pruning**Input:** a collision-free polygonal path $\sigma = (\mathbf{x}_0, \dots, \mathbf{x}_k)$, number of attempts K **Output:** a collision-free path with the same endpoints that is no longer than σ

```

1 Function Shortcut( $\sigma, K$ ):
2   for  $i \leftarrow 1$  to  $K$  do
3     draw  $s_1 < s_2$  uniformly from  $[0, \text{length}(\sigma)]$  and let  $\mathbf{a} = \sigma(s_1)$ ,  $\mathbf{b} = \sigma(s_2)$  be the
       points at these arc lengths
4     if  $\mathbf{a}$  and  $\mathbf{b}$  lie on different segments of  $\sigma$  and CollisionFree( $\mathbf{a}, \mathbf{b}$ ) then
5       replace the part of  $\sigma$  between  $\mathbf{a}$  and  $\mathbf{b}$  by the segment  $[\mathbf{a}, \mathbf{b}]$ 
6   return  $\sigma$ 
7 Function Prune( $\sigma$ ):
8    $\sigma' \leftarrow (\mathbf{x}_0)$ ;  $i \leftarrow 0$ 
9   while  $i < k$  do
10     $j \leftarrow$  the largest index  $> i$  with CollisionFree( $\mathbf{x}_i, \mathbf{x}_j$ )
11    append  $\mathbf{x}_j$  to  $\sigma'$ ;  $i \leftarrow j$ 
12  return  $\sigma'$ 

```

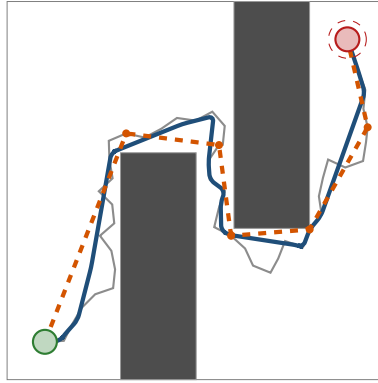


Figure 16.7. Post-processing of the path of Example 16.4. Gray: the raw RRT path (50 vertices, length 23.85). Blue: after 100 random shortcut attempts (23 vertices, 19.91). Dashed orange with dots: after greedy pruning (7 vertices, 18.39). The shortest path with clearance 0.075 has length 17.20. Both results are collision-free, and the pruned path hugs the obstacle corners, which is why the obstacles should be inflated by a safety margin before planning.

increases the length and that the results are collision-free. Pruning hugs the obstacle corners as tightly as the checker allows, good for length and bad for safety, so plan with obstacles inflated by a margin beyond the drone radius. Both methods improve the path locally: every shortcut segment must itself be free, so a path that takes the long way around a large obstacle keeps taking it (Exercise 16.7), and only a new search, or RRT* (Chapter 17), finds the better route. The polygonal result is turned into a flyable trajectory by the controller of Chapter 21, or by fitting a smooth curve with bounded velocity and acceleration through the waypoints, which must then be collision-checked again. In practice the corners are rounded first, each waypoint being replaced by a short arc whose radius respects the speed and acceleration limits of Chapter 2; the arc is then sampled at the resolution δ and checked again, because cutting a corner moves the path toward the inside of the turn, which is exactly where the obstacle that caused the corner sits. Chapter 21 takes the better route of optimizing the trajectory subject to the dynamics and the obstacles instead of repairing it afterward.

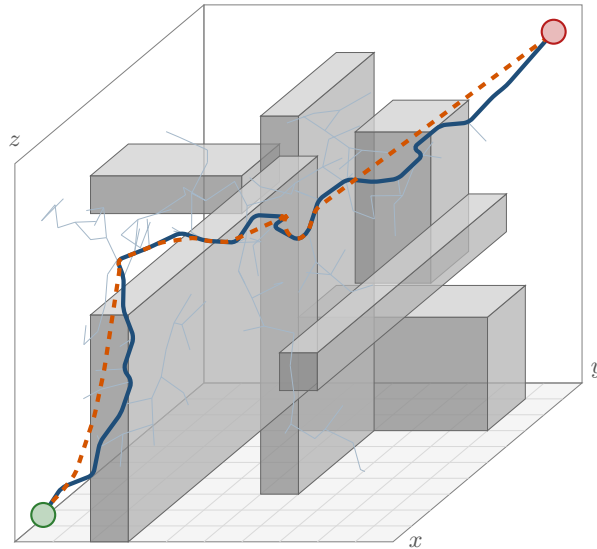


Figure 16.8. RRT in the 3D box field `scene_3d()`, drawn in an oblique projection (y recedes into the page). The light tree has 158 vertices; the blue path (length 24.48) flies over the wall on the left and around the pillar, and the dashed orange path is its shortcut (20.40). The code is the same as in 2D; only the bounds of $\mathcal{X}$ have three components.

16.7.4 Planning in 3D

Nothing in [Algorithm 16.1](#) refers to the dimension. The code of [Section 16.8](#) takes the bounds of $\mathcal{X}$ as two vectors, and the only difference between the 2D and the 3D planner is that they have three components: `World([0, 0, 0], [10, 10, 10], boxes=...)` instead of `World([0, 0], [10, 10], boxes=...)`, and the same `rrt()` call. [Figure 16.8](#) shows the 3D box field `scene_3d()`, a wall to fly over, a full-height pillar, a slab to fly under or over and three blocks, for a drone of radius 0.3 with $\delta = 0.1$ and $\eta = 0.8$. With seed 3, RRT reaches the goal at iteration 321 with 158 vertices and a path of length 24.48; shortcutting brings it to 20.40, against a straight-line distance of 15.59. RRT-Connect solves the same query at iteration 117 with 59 vertices. A 0.1 m grid of this $10 \times 10 \times 10$ scene has a million cells; RRT has looked at a few hundred points.

16.7.5 Relatives

The **probabilistic roadmap** (PRM) of Kavraki et al. [80] is the other founding method of sampling-based planning: sample configurations, keep the free ones, connect each to its k nearest neighbors by collision-checked segments, and answer any number of queries by graph search ([Chapters 3 and 4](#)) on the roadmap. PRM is a *multi-query* method that spends its effort once per map; RRT is a *single-query* method that spends it once per query and never builds a graph of the whole space, which suits a drone whose map and queries change every time and is the only option when the dynamics allow no exact steering. RRT* and Informed RRT* [49, 77] keep the tree but make it optimal in the limit; they are the subject of [Chapter 17](#), together with BIT* [50]. Both books [28, 99] survey the many other variants.

16.8 Implementation notes

16.8.1 Collision checking for drones

The obstacles of this chapter are axis-aligned boxes and spheres, which cover buildings, no-fly volumes and other drones well enough for planning and have closed-form distance functions.

- *Sphere versus box.* The distance from a point $\mathbf{x}$ to the box $[\mathbf{l}, \mathbf{u}]$ is the length of the vector from $\mathbf{x}$ to its *clamped* copy, $\|\mathbf{x} - \text{clamp}(\mathbf{x}, \mathbf{l}, \mathbf{u})\|$, with `clamp` applied per coordinate; the sphere of radius r around $\mathbf{x}$ is free of the box if and only if this distance exceeds r . `World.box_distances` computes it for all points and all boxes at once with NumPy broadcasting.
- *Segment versus sphere.* A sphere obstacle of radius R at $\mathbf{c}$ meets the sphere robot moving along $[\mathbf{a}, \mathbf{b}]$ if and only if the distance from $\mathbf{c}$ to the segment is at most $R + r$; the point-to-segment distance of [Chapter 2](#) (project $\mathbf{c}$ onto the line, clamp the parameter to $[0, 1]$) answers this exactly, with no subdivision.
- *Conservative inflation.* Boxes are checked by subdivision with the margin $r + \delta/2$ of [Proposition 16.3](#), so a path that passes the check keeps a clearance of at least r from every obstacle. Any extra safety margin, for the position uncertainty of [Chapter 18](#) for example, is simply added to r .

[Listing 16.1](#) is the segment test; the vertex test `point_free` is the special case of a segment of length zero.

Listing 16.1. The segment test of `World` (from `code/ch16_rrt.py`): subdivision with the conservative margin $r + \delta/2$ for the boxes, the exact segment–sphere test for the spheres.

```

1  def segment_free(self, a, b):
2      """CollisionFree(a, b): subdivision check of the straight segment.
3
4      Points spaced at most delta apart are tested against the boxes
5      inflated by radius + delta/2 (conservative: every point of the
6      segment is within delta/2 of a tested point, so the true clearance
7      is at least radius). Spheres use the exact segment-sphere test."""
8      a = np.asarray(a, dtype=float)
9      b = np.asarray(b, dtype=float)
10     n = max(1, int(math.ceil(np.linalg.norm(b - a) / self.delta)))
11     pts = a + np.linspace(0.0, 1.0, n + 1)[:, None] * (b - a)
12     if not np.all(self.points_free(pts, self.radius + 0.5 * self.delta,
13                                     spheres=False)):
14         return False
15     if len(self.sph_c):
16         ab = b - a
17         t = (self.sph_c - a) @ ab / max(float(ab @ ab), 1e-300)
18         closest = a + np.clip(t, 0.0, 1.0)[:, None] * ab
19         d = np.linalg.norm(self.sph_c - closest, axis=1)
20         if np.any(d <= self.sph_r + self.radius):
21             return False
22     return True

```

16.8.2 Nearest neighbors and kd-trees

The tree of `ch16_rrt.py` keeps its vertices in one NumPy array and answers NEAREST by a vectorized linear scan: one subtraction, one squared norm per row and an `argmin`. That is $\mathcal{O}(n)$ per query with a small constant, about 0.04 ms for $n = 10^3$ vertices, 0.3 ms for 10^4 and 3 ms for 10^5 in three dimensions: means of 200 queries, timed and printed by the self-test of `ch16_rrt.py` on the machine that produced this book, and reproducible to within a factor of two on any other. For the trees of this chapter the scan is not the bottleneck, and it needs no

Table 16.4. The parameters of RRT and what happens when they are badly chosen. Values are for the map size L of [Example 16.4](#) ($L = 10$).

Parameter	Typical value	Too small	Too large
Step size η	$0.02L$ to $0.1L$ (here 0.5)	slow growth, many vertices, long nearest-neighbor scans	long segments rejected in clutter, jagged paths; never a safety problem when the whole segment is checked
Goal bias p_{goal}	0.05 to 0.1	no pull toward the goal; the region is found by chance	greedy, trapped behind obstacles (Table 16.3)
Goal radius r_{goal}	about η	the goal region is never entered	the path ends far from the goal; always append the final segment if free
Check resolution δ	$\eta/10$; at most half the thinnest obstacle	slow collision checks	obstacles missed unless inflated by $\delta/2$; then over-conservative in tight spaces
Budget N	10^3 to 10^5	failure reported on feasible queries	time wasted on infeasible queries
Random seed	fixed per experiment	a result from one seed is an anecdote; report statistics over seeds	

rebuilding when a vertex is added. Beyond 10^4 vertices use a **kd-tree** [13]: a binary tree that splits the points along one coordinate axis at each level, so that a query descends to the leaf containing the query point and then backtracks only into branches whose bounding region is closer than the best distance found so far. In low dimension the expected cost is $\mathcal{O}(\log n)$ per query after an $\mathcal{O}(n \log n)$ build. Since RRT inserts a vertex every iteration and a balanced kd-tree does not take insertions well, implementations rebuild it every few hundred insertions and scan the vertices added since (`scipy.spatial.cKDTree` is the standard Python choice), or use a bucket grid over $\mathcal{X}$. In high dimension the backtracking visits most branches and a kd-tree degrades to a scan; beyond ten or so dimensions approximate methods are used instead.

16.8.3 Parameters

[Table 16.4](#) lists the parameters. Two deserve a word beyond the table: seed the generator (`numpy.random.default_rng(seed)`) and report statistics over many seeds, as in [Table 16.3](#), never a single run; and choose the budget N from the time you can afford, since RRT cannot recognize an infeasible query and failure means only “no path found within the budget”. [Listing 16.2](#) is the main loop of the implementation; it matches [Algorithm 16.1](#) line by line and adds the bookkeeping for the figures. `extend`, `connect` and `rrt_connect` implement [Algorithm 16.2](#) in the same file. The self-test runs in a few seconds – it prints its own runtime, three to five seconds on the machines used for this book – and re-checks in one place every claim of this chapter that a program can check: the collision tests, the worked example and its two reference lengths, the smoothing bounds, the RRT-Connect ordering over 40 seeds, the 3D scene, the kinodynamic limits and the nearest-neighbor timings quoted above.

Listing 16.2. The loop of `rrt(world, start, goal, eta, p_goal, r_goal, max_iters, seed, ...)` (from `code/ch16_rrt.py`). Tree stores the vertices in a growing array with parent indices; `steer` is Equation (16.1); the goal itself is appended to the path when the last segment is free.

```

1  rng = np.random.default_rng(seed)
2  start = np.asarray(start, dtype=float)
3  goal = np.asarray(goal, dtype=float)
4  tree = Tree(start, max_iters + 2)
5  goal_index, solution_iter, snaps = -1, None, {}
6  it = 0
7  for it in range(1, max_iters + 1):
8      q_rand = goal if rng.random() < p_goal else world.sample(rng)
9      i_near = tree.nearest(q_rand)
10     q_near = tree.V[i_near]
11     q_new = steer(q_near, q_rand, eta)
12     free = world.segment_free(q_near, q_new)
13     if trace is not None:
14         trace.append((it, q_rand.copy(), i_near, q_new.copy(), free))
15     if free and np.linalg.norm(q_new - q_near) > 1e-12:
16         i_new = tree.add(q_new, i_near)
17         if goal_index < 0 and np.linalg.norm(q_new - goal) <= r_goal:
18             goal_index, solution_iter = i_new, it
19             if stop_at_goal:
20                 break
21         if it in snapshots:
22             snaps[it] = tree.n
23     path = None
24     if goal_index >= 0:
25         path = tree.path_to(goal_index)
26         if (np.linalg.norm(path[-1] - goal) > 0
27             and world.segment_free(path[-1], goal)):
28             path = np.vstack([path, goal])
29     return Result(path=path, iterations=it, solution_iter=solution_iter,
30                  tree=tree, snapshots=snaps, n_vertices=tree.n)

```

16.8.4 Pitfalls

Testing the vertices but not the segments

The most common bug in a first RRT is to test only $\mathbf{x}_{\text{new}}$, or the endpoints of the edge, for collision. The tree then tunnels through every obstacle thinner than η , and the bug is invisible on maps with thick walls. A step size that is “too large” is dangerous only in this sense; with the subdivision test of Proposition 16.3 a long segment is simply rejected more often. The same holds for the resolution: test points spaced δ apart *without* the inflation by $\delta/2$ miss any wall thinner than δ , at a rate that depends on the angle of the segment; this is the trap that the self-test of `ch16_rrt.py` reproduces with the thin wall of Section 16.3.2.

A goal bias that is too high

Goal bias looks like a free lunch: more samples at the goal, faster solutions. Table 16.3 shows the opposite for $p_{\text{goal}} = 0.5$: the median number of iterations rises from 425 to 745, above the value without any bias. Every goal sample extends the vertex nearest to the goal, and until the tree has passed the last obstacle that vertex sits against a wall and

Table 16.5. Grid A*, RRT and RRT* compared. “Optimal” for grid A* means optimal among grid paths, which are longer than the true shortest path by up to 8% on an 8-connected grid.

Property	Grid A* (Chapter 4)	RRT	RRT* (Chapter 17)
Completeness	complete on the grid; blind to gaps narrower than a cell	probabilistically complete (Theorem 16.5)	probabilistically complete
Optimality	optimal on the grid	no, and never in the limit (Theorem 16.6)	asymptotically optimal
Memory	the whole grid, N^d cells	n vertices	n vertices
Anytime behavior	no; ARA* (Chapter 6)	first solution only; improve by shortcutting	the path improves with every iteration
Dynamics	only via a state lattice	kinodynamic RRT (Section 16.7.2)	needs an exact steering function; hard
Cost driven by	N^d and the heuristic	narrow passages; nearest-neighbor and collision checks	the same plus the neighbor queries and rewiring
Prefer when	2D or coarse 3D maps; many agents on one graph; exact answers	large or high-dimensional spaces; a feasible path fast	path quality matters and time allows

the extension is rejected; the tree becomes greedy and stuck. Keep p_{goal} between 0.05 and 0.1, or use RRT-Connect, which needs no bias.

Forgetting the goal region

With continuous samples the tree never lands exactly on $\mathbf{x}_{\text{goal}}$, so a test $\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{goal}}$ succeeds only through a goal sample within η of its nearest vertex, and never when $p_{\text{goal}} = 0$. Define the goal as a region of radius r_{goal} (Definition 16.1), test membership on every new vertex, and append the final segment to $\mathbf{x}_{\text{goal}}$ when it is free so that the path ends where the mission requires. In the kinodynamic planner the goal state should also allow a range of arrival velocities.

16.9 Grid search or sampling?

The Week 8 milestone of the study plan is to choose between graph search and sampling-based planning; Table 16.5 collects the criteria and its last row answers in one line. Two entries deserve a comment: only a grid lets many agents share one discretization, which the multi-agent planners of Chapters 7 to 11 all need, and only sampling in the state space respects the dynamics during planning. Exercise 16.8 asks you to measure the trade-off yourself.

16.10 Where this fits in the drone system

In the drone system

In the hybrid architecture of [Chapter 24](#) the global planner (conflict-based search, CBS, or its bounded-suboptimal variant ECBS) plans time-indexed paths for the whole swarm on a coarse grid, and the replanning layer repairs a path with the incremental D^* Lite or a fresh A^* search on the same grid. RRT is the **continuous 3D planner** that steps in when the grid is not enough: when the cells are too coarse for the local geometry (a gap between two cranes narrower than a cell), when the volume is too large to discretize at the accuracy the mission needs ([Table 16.1](#)), or when an unknown obstacle detected by the tracker of [Chapter 18](#) blocks the nominal route and a detour has to be found in three dimensions within a fraction of a second.

Combining a global grid plan with a local RRT. When a segment between two consecutive waypoints of the grid path becomes infeasible, the local planner runs [Algorithm 16.1](#) or [Algorithm 16.2](#) from the current position to the next reachable waypoint, with a goal region of about one cell and the obstacles inflated by the drone radius, the tracking error and the prediction uncertainty of [Chapters 19](#) and [20](#). The result is shortcut ([Algorithm 16.4](#)), turned into a trajectory by the controller of [Chapter 21](#) and checked against the reservations of the other drones; if it violates them, or if the local planner fails within its budget, the event is escalated to a global replan, the decision logic of [Chapter 24](#). In the other direction, the waypoints of the grid path can seed the goal bias of the tree, turning the global plan into a heuristic for the local sampler.

What it receives and delivers. A start, a goal region, a map of boxes and the predicted volumes of moving obstacles come in; a collision-free polygonal path with a known clearance, or a failure within a known time, goes out. RRT guarantees nothing about optimality ([Theorem 16.6](#)) and nothing at all when no path exists, which is why its budget must be chosen from the available time. Week 8 of the study plan ([Appendix A](#)) covers this chapter and [Chapter 17](#); its coding exercise is [Exercise 16.8](#).

16.11 Summary

Chapter summary

- Grids cost N^d cells; a $100 \times 100 \times 50$ m volume at 0.1 m has 5×10^8 of them, and adding velocities makes the grid impossible. Sampling-based planners store only the samples they draw, and the same code runs in any dimension.
- RRT repeats the four primitives of [Definition 16.2](#): SAMPLE (uniform in $\mathcal{X}$, the goal with probability p_{goal}), NEAREST (Euclidean nearest vertex), STEER (at most η toward the sample) and COLLISIONFREE (the whole segment, by subdivision with the margin $r + \delta/2$). A vertex that enters the goal region ends the search.
- The Voronoi bias explains the fast exploration: a vertex is *selected* with probability equal to the relative volume of its Voronoi region ([Equation \(16.2\)](#)), and extended when the step is collision-free, so the tree is pulled into the largest unexplored regions.
- RRT is probabilistically complete ([Theorem 16.5](#)) for any $\eta > 0$ and $p_{\text{goal}} < 1$, with an expected number of iterations that grows like $(1/w)^d$ for a passage of width w . It is not optimal, and its best path converges almost surely to a suboptimal one ([Theorem 16.6](#)); RRT* fixes this.

- RRT-Connect grows two trees and connects them greedily; on the worked example it needs 2.4 times fewer iterations than RRT. Kinodynamic RRT replaces STEER by forward simulation of sampled controls and needs a state-space metric for NEAREST.
- Shortcutting and pruning never increase the length and remove most of the excess ($23.85 \rightarrow 18.39$ on the worked example) but cannot carry a path across an obstacle.
- Nearest-neighbor search is $\mathcal{O}(n)$ by a scan and $\mathcal{O}(\log n)$ with a kd-tree; collision checks dominate in clutter. Fix the seed, report statistics over seeds, and never test only the vertices.

Further reading. RRT was introduced by LaValle in a technical report [98], and the journal version with Kuffner [100] contains the kinodynamic formulation and the completeness argument, which Kleinbort et al. [86] revisited with a rigorous proof. RRT-Connect is due to Kuffner and LaValle [94]. LaValle’s book [99], freely available online, treats sampling-based planning in depth, including collision detection, metric spaces and kinodynamic planning, and Choset et al. [28] give a compact textbook chapter. The probabilistic roadmap of Kavraki et al. [80] is the multi-query counterpart. Karaman and Frazzoli [77] proved [Theorem 16.6](#) and introduced RRT*, the subject of [Chapter 17](#) together with Informed RRT* [49]. The kd-tree is due to Bentley [13], and Geraerts and Overmars [53] compare shortcutting with other path improvement methods.

16.12 Exercises

Exercise 16.1 (★). A survey drone must plan through a $200 \times 200 \times 60$ m volume with an accuracy of 0.25 m. How many cells does the grid have, how much memory does one byte per cell take, and how many neighbors does a cell have in the 26-connected lattice? Repeat for a state that also contains the velocity (± 4 m/s per axis at 0.25 m/s). Compare with the memory of an RRT with 20 000 vertices in three dimensions (three 8-byte coordinates and one 4-byte parent index per vertex).

Exercise 16.2 (★). Continue [Table 16.2](#) by hand for iterations 11 and 12 of [Example 16.4](#), given the samples $\mathbf{x}_{\text{rand}} = (9.70, 5.16)$ and $(6.23, 7.77)$: find the nearest vertex among vertices 0 to 6 (their coordinates are in the table), compute $\mathbf{x}_{\text{new}}$ from [Equation \(16.1\)](#) with $\eta = 0.5$, and decide whether the segment enters the wall $[3, 5] \times [0, 6]$. Then explain, using the Voronoi bias, why almost every sample in iterations 7 to 16 selects vertex 5 or 6.

Exercise 16.3 (★★). (a) Give a segment and a wall of thickness 0.04 such that test points spaced $\delta = 0.05$ apart, tested *without* the inflation by $\delta/2$, all lie outside the wall although the segment crosses it. (b) The margin $r + \delta/2$ of [Proposition 16.3](#) is sufficient but not the smallest: show that the margin $\sqrt{r^2 + \delta^2/4}$ also guarantees a collision-free segment, and that no smaller margin does. (c) Let $d_0 = \text{dist}(\mathbf{a}, \mathcal{O}) - r$ be the free distance of the first endpoint. Explain why no point of the segment within d_0 of $\mathbf{a}$ needs to be tested, and design an adaptive check that uses this idea at every test point (exact, no δ , fast in open space).

Exercise 16.4 (★★). This exercise completes [Theorem 16.5](#). (a) Let T be the number of iterations until the tree reaches the last ball B_m . Using the binomial distribution of the number of samples that land in “the next ball” among n iterations, show that $\mathbb{P}(T > n) \leq \mathbb{P}(\text{Bin}(n, p) < m)$ and deduce, with a Chernoff bound, that the failure probability decays exponentially in n once $n > 2m/p$. (b) Give a map on which [Algorithm 16.1](#) with $p_{\text{goal}} = 1$ never finds a path although one exists, and explain which step of the proof fails. (c) Where exactly does the proof use $\eta > 0$, and what would go wrong for an algorithm whose step size shrinks like $\eta_i = 1/i$ at iteration i ?

Exercise 16.5 (★★). Consider the tree with vertices $\mathbf{v}_0 = (0, 0)$, $\mathbf{v}_1 = (1, 0)$ and $\mathbf{v}_2 = (2, 0)$ in the square $\mathcal{X} = [-1, 3] \times [-2, 2]$, with $p_{\text{goal}} = 0$. Draw the Voronoi regions of the three vertices and compute the probability that each one is chosen as $\mathbf{x}_{\text{near}}$ by the next sample, using Equation (16.2). Then show in general that for a tree that is a single straight chain of $k + 1$ vertices spaced η apart, centered in a large square map of side L with $L \gg k\eta$, each of the two tips is chosen with probability close to $1/2$ and every interior vertex with probability at most η/L . Finally explain why the picture changes when the chain starts near a corner and points into the map, as in Example 16.4: the region of the single tip is then almost the whole square, so that tip takes almost all of the probability.

Exercise 16.6 (★★ Coding). Using `ch16_rrt.py`, run RRT on Example 16.4 for $p_{\text{goal}} \in \{0, 0.02, 0.05, 0.1, 0.2, 0.5, 0.9\}$ with 50 seeds each and plot the median and the quartiles of the number of iterations to the first solution. Explain the shape of the curve with the arguments of Section 16.8.4. Then move the goal to $(9, 1)$, where the straight line from the start is blocked by the first wall only, and repeat: does the best bias change?

Exercise 16.7 (★★★). (a) Prove that line 5 of Algorithm 16.4 never increases the length of the path and that the result is collision-free whenever the input is. (b) Prove that PRUNE returns a path with at most as many vertices as the input and give an example where its greedy jump of line 10 returns a longer path than a different choice of jumps would. (c) Take the wall $[4, 6] \times [2, 9.5]$ in $[0, 10]^2$, the start $(1, 5)$, the goal $(9, 5)$ and the path $(1, 5), (1, 9.75), (9, 9.75), (9, 5)$ over the top of the wall. Prove that no sequence of shortcuts can produce a path that passes below the wall, although that route is shorter.

Exercise 16.8 (★★★ Coding). This is the first half of the Week 8 coding exercise of the study plan (Appendix A); Chapter 17 completes it. Build a 3D obstacle field of about 20 boxes in a $50 \times 50 \times 20$ m volume (or reuse `scene_3d()`) and plan between random start-goal pairs with (a) RRT and RRT-Connect from `ch16_rrt.py` followed by shortcutting, (b) RRT* from Chapter 17 or a library, and (c) grid A^* from Chapter 4 on a 26-connected lattice at resolutions 2, 1 and 0.5 m with inflated obstacles. Over 30 queries record success rate, path length relative to the best found by any method, computation time and memory (vertices or cells). Plot length against time and state in one paragraph, with your numbers, when you would use graph search and when sampling: the Week 8 milestone of Appendix A. Finally scale the map to $200 \times 200 \times 60$ m at 0.25 m and report which methods still run.

Optimal Sampling-Based Planning: RRT* and Informed RRT*

Learning objectives

- Explain what RRT* adds to RRT (NEAR, CHOOSEPARENT, REWIRE), trace one iteration by hand, and say why each addition is needed for optimality.
- State the connection radius $r_n = \min\{\gamma_{RRT^*}(\log n/n)^{1/d}, \eta\}$ exactly, explain every symbol, compute an admissible γ_{RRT^*} for a given map, and explain why the radius must shrink slowly.
- State the asymptotic-optimality theorem with its assumptions, sketch the ball-covering proof, and say what the theorem does *not* promise.
- Derive the informed set of Informed RRT* (an ellipse in 2D, an ellipsoid in 3D), sample it uniformly with the scale–rotate–translate transformation, and predict when it helps.
- Implement RRT* and Informed RRT* in Python in 2D and 3D, compare them with RRT and with grid A*, and use them as anytime planners while a drone is flying.

17.1 Why this matters for a drone swarm

The rapidly-exploring random tree of [Chapter 16](#) solves a problem that grid search cannot touch: it finds a collision-free route through a continuous 3D volume with a few thousand samples, whatever the resolution a grid would have needed. But look at the route it returns. It is a chain of short, randomly oriented segments, it wanders, and on the map of [Chapter 16](#) it comes out somewhere between 20 and 27 units long when the shortest route has length about 16.9 ([Chapter 16](#)). Smoothing helps, but it can only pull the path taut inside the corridor the tree happened to find. Worse, more samples do not help at all: the parent of every vertex is fixed at the moment the vertex is created, so the first path that reaches the goal is, up to the random order of insertion, the path you keep. A drone that flies a route thirty percent longer than necessary spends thirty percent more battery, and a swarm spends it over and over.

RRT*, introduced by Karaman and Frazzoli in 2011 [77], repairs this with two extra steps per iteration: the new vertex takes as its parent the neighbor that gives it the cheapest route from the start (CHOOSEPARENT), and it then becomes the parent of any neighbor whose route it shortens (REWIRE). The tree keeps re-optimizing itself, its best path converges to the optimum as samples accumulate—RRT* is *asymptotically optimal*—and the price is a logarithmic number of extra collision checks per iteration. Informed RRT* [49] then adds the observation that, once a path of cost c_{best} is known, every point that could improve it lies

inside an ellipsoid with the start and the goal as foci, and that sampling this ellipsoid directly makes the convergence much faster in large free spaces.

In the hybrid architecture of [Chapter 24](#) these planners live in two places. The global layer plans the swarm on a grid with conflict-based search (CBS) or its bounded-suboptimal variant ECBS ([Chapters 9](#) and [10](#)); when a drone must leave that grid, because it flies through a 3D volume too large or too finely structured to discretize, RRT* plans the continuous segment, in *anytime* fashion: the first path is available after a few hundred samples, and the tree keeps improving it while the drone flies. The replanning layer can seed Informed RRT* with the grid path it already has, so that the sampler concentrates on the ellipsoid around that path from the first iteration. This is Week 8 of the study plan ([Appendix A](#)): its milestone is choosing between graph search and sampling-based planning, and its coding exercise is [Exercise 17.7](#).

The chapter proceeds as follows. [Sections 17.2](#) and [17.3](#) give the idea and the definitions; [Section 17.4](#) the three new primitives, the pseudocode and the connection radius; [Section 17.5](#) the worked example; [Section 17.6](#) the asymptotic-optimality theorem; [Section 17.7](#) Informed RRT* and [Section 17.8](#) the family up to BIT*; and [Sections 17.9](#) and [17.10](#) the implementation, its pitfalls and the drone system.

17.2 The idea in plain words

Think of the RRT tree as a road network that is built one road at a time. RRT builds each new road from the nearest existing junction, whichever junction that is. If the nearest junction happens to lie at the end of a long detour, the new road inherits the detour, and so does every road built from it later. Nothing in RRT ever revisits this decision.

RRT* makes two changes, both local to the new vertex $\mathbf{x}_{\text{new}}$ and both restricted to a ball of radius r_n around it ([Figure 17.1](#)).

- **Choose the parent.** Connect $\mathbf{x}_{\text{new}}$ to the neighbor that gives it the cheapest collision-free route from the start, not merely to the nearest vertex.
- **Rewire the neighbors.** Make $\mathbf{x}_{\text{new}}$ the parent of every neighbor whose route it shortens by a collision-free segment, and pass the saving on to that neighbor's descendants.

Because a vertex's parent can change long after the vertex was created, the tree is no longer a record of the order in which space was explored; it is an ever-improving *shortest-path tree* over the sampled points, in the same sense that Dijkstra's algorithm ([Chapter 3](#)) maintains a shortest-path tree over a graph. As samples accumulate, the sampled points become dense, the shortest paths through them approach the shortest paths through free space, and the cost of the best tree path to the goal converges to the optimum.

The radius r_n is the one delicate quantity: constant, it makes every iteration do $\mathcal{O}(n)$ collision checks; shrinking too fast, it leaves the balls empty and the tree stops improving. The schedule $r_n \propto (\log n/n)^{1/d}$ shrinks as slowly as it can while keeping the expected number of neighbors logarithmic ([Section 17.4.3](#)).

Key idea

RRT* is RRT with a shortest-path-tree discipline: every new vertex takes the cheapest parent within radius r_n and becomes the parent of every neighbor it can serve more cheaply. With $r_n \propto (\log n/n)^{1/d}$ and a large enough constant, the best path converges to the optimum almost surely. Once a path of cost c_{best} exists, Informed RRT* keeps the same tree but samples only $\|\mathbf{x} - \mathbf{x}_{\text{init}}\| + \|\mathbf{x} - \mathbf{x}_{\text{goal}}\| \leq c_{\text{best}}$, the only region a better path can cross.

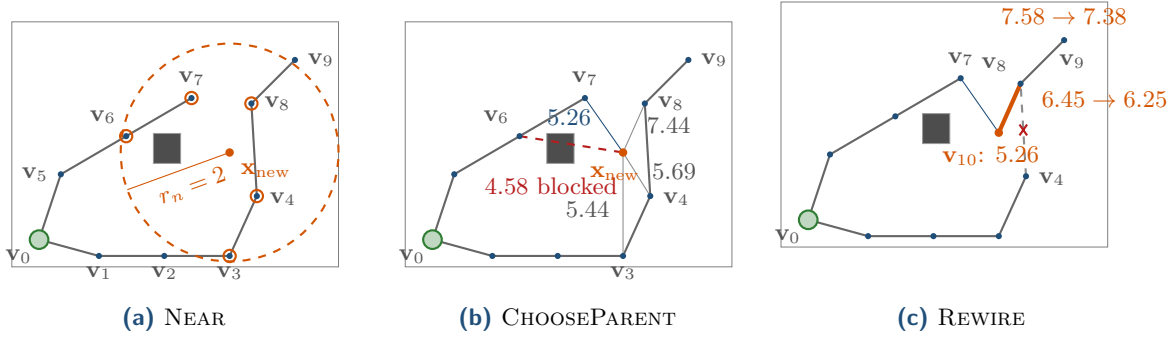


Figure 17.1. One RRT* iteration on the ten-vertex tree of Example 17.8. (a) The ball of radius $r_n = 2$ around $\mathbf{x}_{\text{new}}$ contains $\mathbf{v}_3, \mathbf{v}_4, \mathbf{v}_6, \mathbf{v}_7, \mathbf{v}_8$; $\mathbf{v}_4$ is the nearest vertex. (b) The cost-to-come through each candidate: $\mathbf{v}_6$ would be the cheapest parent (4.58) but the segment crosses the obstacle, so $\mathbf{v}_7$ wins with 5.26, not the nearest $\mathbf{v}_4$ (5.69). (c) $\mathbf{v}_8$ is cheaper through $\mathbf{x}_{\text{new}}$ (6.25 instead of 6.45), so its edge from $\mathbf{v}_4$ is cut and the saving reaches its child $\mathbf{v}_9$.

17.3 Problem statement and notation

We keep the setting of Chapter 16. The planning space is a box $\mathcal{X} \subset \mathbb{R}^d$ with $d \in \{2, 3\}$ (a robot position, or a configuration after the obstacles have been inflated by the drone radius as in Chapter 2), $\mathcal{X}_{\text{obs}}$ is its obstacle region and $\mathcal{X}_{\text{free}} = \mathcal{X} \setminus \mathcal{X}_{\text{obs}}$ the free space. The start is $\mathbf{x}_{\text{init}} \in \mathcal{X}_{\text{free}}$ and the goal $\mathbf{x}_{\text{goal}} \in \mathcal{X}_{\text{free}}$. A path is a continuous map $\sigma : [0, 1] \rightarrow \mathcal{X}$; it is **collision-free** if $\sigma(s) \in \mathcal{X}_{\text{free}}$ for all s , and its cost $c(\sigma)$ is its Euclidean length. Throughout, $\mu(\cdot)$ denotes volume (Lebesgue measure), $D(\mathbf{x}, r)$ the closed ball of radius r around $\mathbf{x}$, and $\zeta_d = \mu(D(\mathbf{0}, 1))$ the volume of the unit ball: $\zeta_2 = \pi$, $\zeta_3 = 4\pi/3$. (Appendix B.3 reviews the probability these volumes are used with: random vectors, densities and expectations.)

Definition 17.1 (Optimal planning problem). Given $(\mathcal{X}_{\text{free}}, \mathbf{x}_{\text{init}}, \mathbf{x}_{\text{goal}})$, find a collision-free path σ^* from $\mathbf{x}_{\text{init}}$ to $\mathbf{x}_{\text{goal}}$ with $c(\sigma^*) = c^* := \inf \{c(\sigma) : \sigma \text{ collision-free from } \mathbf{x}_{\text{init}} \text{ to } \mathbf{x}_{\text{goal}}\}$. We write $c_{\min} = \|\mathbf{x}_{\text{goal}} - \mathbf{x}_{\text{init}}\|$ for the straight-line distance; every path has $c(\sigma) \geq c_{\min}$.

Sampling-based planners can only find paths that have some room around them, so the theory needs a notion of clearance.

Definition 17.2 (Clearance and robust optimum). A path σ has **strong δ -clearance** if $D(\sigma(s), \delta) \subseteq \mathcal{X}_{\text{free}}$ for all $s \in [0, 1]$. An optimal path σ^* is a **robustly optimal solution** if there are collision-free paths σ_n with strong δ_n -clearance, $\delta_n > 0$, such that $c(\sigma_n) \rightarrow c^*$.

The optimum of the map in Chapter 16 hugs the corners of the two walls, so it has no clearance itself; but pushing it away from the corners by any $\delta > 0$ costs only $\mathcal{O}(\delta)$, so it is robustly optimal. A route through a gap of zero width is *not* robust: no path with positive clearance approximates it, and no sampling-based planner finds it.

Definition 17.3 (Tree, cost-to-come, best cost). The planner maintains a tree $\mathcal{T} = (V, E)$ rooted at $\mathbf{x}_{\text{init}}$ with $n = |V|$ vertices; $\text{parent}(\mathbf{v})$ is the parent of $\mathbf{v} \neq \mathbf{x}_{\text{init}}$. The **cost-to-come** $\text{Cost}(\mathbf{v})$ is the length of the tree path from $\mathbf{x}_{\text{init}}$ to $\mathbf{v}$, so $\text{Cost}(\mathbf{v}) = \text{Cost}(\text{parent}(\mathbf{v})) + \|\mathbf{v} - \text{parent}(\mathbf{v})\|$ and $\text{Cost}(\mathbf{x}_{\text{init}}) = 0$. The symbol n is reserved for $|V|$ and iterations are counted by i ; the two differ, because a blocked sample adds no vertex. The **best cost** after i iterations is $c_{\text{best}} = \text{Cost}(\mathbf{x}_{\text{goal}})$ if $\mathbf{x}_{\text{goal}} \in V$ and $c_{\text{best}} = \infty$ otherwise.

The primitives $\text{Nearest}(V, \mathbf{x})$, $\text{Steer}(\mathbf{x}, \mathbf{y})$ (move from $\mathbf{x}$ toward $\mathbf{y}$ by at most the step η) and $\text{CollisionFree}(\mathbf{x}, \mathbf{y})$ (the segment from $\mathbf{x}$ to $\mathbf{y}$ lies in $\mathcal{X}_{\text{free}}$) are those of Chapter 16; RRT* adds one more. The vertex returned by Nearest is written $\mathbf{x}_{\text{nearest}}$ here, not $\mathbf{x}_{\text{near}}$ as in Chapter 16, because this chapter needs the name *near* for the set X_{near} of Definition 17.4.

Definition 17.4 (Near and the connection radius). $\text{Near}(V, \mathbf{x}, r) = \{\mathbf{v} \in V : \|\mathbf{v} - \mathbf{x}\| \leq r\}$ is the set of vertices within distance r of $\mathbf{x}$. RRT* calls it with the **connection radius**

$$r_n = \min\left\{\gamma_{\text{RRT}^*} \left(\frac{\log n}{n}\right)^{1/d}, \eta\right\}, \quad \gamma_{\text{RRT}^*} > \gamma_{\text{RRT}^*}^* := 2\left(1 + \frac{1}{d}\right)^{1/d} \left(\frac{\mu(\mathcal{X}_{\text{free}})}{\zeta_d}\right)^{1/d}, \quad (17.1)$$

where $n = |V|$ is the current number of vertices, d the dimension, η the step size of Steer and $\log$ the natural logarithm.

Definition 17.5 (Asymptotic optimality). Let Y_i be the cost of the best solution in the tree after i iterations ($Y_i = \infty$ if there is none). A planner is **asymptotically optimal** if, for every problem with a robustly optimal solution of cost c^* , $\mathbb{P}(\lim_{i \rightarrow \infty} Y_i = c^*) = 1$.

Finally, the set that Informed RRT* samples.

Definition 17.6 (Informed set). For a current best cost $c_{\text{best}} \geq c_{\min}$, the **informed set** is

$$\mathcal{X}_f(c_{\text{best}}) = \{\mathbf{x} \in \mathcal{X} : \|\mathbf{x} - \mathbf{x}_{\text{init}}\| + \|\mathbf{x} - \mathbf{x}_{\text{goal}}\| \leq c_{\text{best}}\}. \quad (17.2)$$

17.4 The algorithm

17.4.1 The three new primitives

An RRT* iteration begins exactly like an RRT iteration: draw $\mathbf{x}_{\text{rand}}$, find the nearest vertex $\mathbf{x}_{\text{nearest}}$, steer to $\mathbf{x}_{\text{new}}$, and give up on the sample if the segment from $\mathbf{x}_{\text{nearest}}$ to $\mathbf{x}_{\text{new}}$ is blocked. If it is free, RRT would insert the edge $(\mathbf{x}_{\text{nearest}}, \mathbf{x}_{\text{new}})$ and stop. RRT* instead runs three steps, illustrated in Figure 17.1 on the small tree that Example 17.8 works through in numbers.

Near. $X_{\text{near}} = \text{Near}(V, \mathbf{x}_{\text{new}}, r_n)$ collects every vertex within the connection radius of the new point. It is the candidate set for both of the following steps; $\mathbf{x}_{\text{nearest}}$ is added to it if the radius happens to exclude it, so that a parent always exists.

ChooseParent. For every candidate $\mathbf{v} \in X_{\text{near}}$ compute the cost-to-come that $\mathbf{x}_{\text{new}}$ would have through $\mathbf{v}$, namely $\text{Cost}(\mathbf{v}) + \|\mathbf{x}_{\text{new}} - \mathbf{v}\|$, and connect $\mathbf{x}_{\text{new}}$ to the candidate with the smallest value whose segment $\text{CollisionFree}(\mathbf{v}, \mathbf{x}_{\text{new}})$ holds.

Rewire. For every candidate $\mathbf{v} \in X_{\text{near}}$ other than the chosen parent, compute the cost $\mathbf{v}$ would have through the new vertex, $\text{Cost}(\mathbf{x}_{\text{new}}) + \|\mathbf{v} - \mathbf{x}_{\text{new}}\|$. If it is smaller than $\text{Cost}(\mathbf{v})$ and $\text{CollisionFree}(\mathbf{x}_{\text{new}}, \mathbf{v})$ holds, replace the edge from $\text{parent}(\mathbf{v})$ to $\mathbf{v}$ by the edge from $\mathbf{x}_{\text{new}}$ to $\mathbf{v}$. The cost of $\mathbf{v}$ drops, and so does the cost of every descendant of $\mathbf{v}$ by the same amount; UPDATESUBTREE propagates the change through the subtree so that the invariant $\text{Cost}(\mathbf{u}) = \text{Cost}(\text{parent}(\mathbf{u})) + \|\mathbf{u} - \text{parent}(\mathbf{u})\|$ holds again for every vertex.

17.4.2 Pseudocode

Algorithm 17.1 is the main loop and Algorithm 17.2 its three helpers. The goal is handled as in Chapter 16 by goal bias: with probability p_{goal} the sample is $\mathbf{x}_{\text{goal}}$ itself, and since Steer returns the target when it is within η , the goal point eventually becomes a vertex. From then on the tree path to that vertex is the current best path, its cost is c_{best} , and rewiring keeps shortening it; the bias is switched off because the goal needs no second copy.

Walkthrough. Lines 4–6 are RRT. Line 7 computes the radius from the *current* number of vertices, which is why the radius is recomputed in every iteration, and line 8 collects the

Algorithm 17.1. RRT* (Karaman and Frazzoli)

Input: free space $\mathcal{X}_{\text{free}}$ (via CollisionFree), start $\mathbf{x}_{\text{init}}$, goal $\mathbf{x}_{\text{goal}}$, step η , constant γ_{RRT^*} , goal bias p_{goal} , budget N

Output: a tree $\mathcal{T}$; the tree path to $\mathbf{x}_{\text{goal}}$ of cost c_{best} , or *failure*

```

1 Function RRTstar( $\mathbf{x}_{\text{init}}, \mathbf{x}_{\text{goal}}, N$ ):
2    $V \leftarrow \{\mathbf{x}_{\text{init}}\}; E \leftarrow \emptyset; \text{Cost}(\mathbf{x}_{\text{init}}) \leftarrow 0; c_{\text{best}} \leftarrow \infty$ 
3   for  $i \leftarrow 1$  to  $N$  do
4      $\mathbf{x}_{\text{rand}} \leftarrow \mathbf{x}_{\text{goal}}$  with probability  $p_{\text{goal}}$  if  $\mathbf{x}_{\text{goal}} \notin V$ , else a uniform sample of  $\mathcal{X}$ 
5      $\mathbf{x}_{\text{nearest}} \leftarrow \text{Nearest}(V, \mathbf{x}_{\text{rand}}); \mathbf{x}_{\text{new}} \leftarrow \text{Steer}(\mathbf{x}_{\text{nearest}}, \mathbf{x}_{\text{rand}})$ 
6     if CollisionFree( $\mathbf{x}_{\text{nearest}}, \mathbf{x}_{\text{new}}$ ) then
7        $n \leftarrow |V|; r_n \leftarrow \min\{\gamma_{\text{RRT}^*}(\log n/n)^{1/d}, \eta\}$ 
8        $X_{\text{near}} \leftarrow \text{Near}(V, \mathbf{x}_{\text{new}}, r_n) \cup \{\mathbf{x}_{\text{nearest}}\}$ 
9        $\mathbf{x}_{\text{parent}} \leftarrow \text{ChooseParent}(X_{\text{near}}, \mathbf{x}_{\text{nearest}}, \mathbf{x}_{\text{new}})$ 
10       $V \leftarrow V \cup \{\mathbf{x}_{\text{new}}\}; E \leftarrow E \cup \{(\mathbf{x}_{\text{parent}}, \mathbf{x}_{\text{new}})\};$ 
11       $\text{Cost}(\mathbf{x}_{\text{new}}) \leftarrow \text{Cost}(\mathbf{x}_{\text{parent}}) + \|\mathbf{x}_{\text{new}} - \mathbf{x}_{\text{parent}}\|$ 
12      Rewire( $X_{\text{near}}, \mathbf{x}_{\text{new}}$ )
13      if  $\mathbf{x}_{\text{goal}} \in V$  then  $c_{\text{best}} \leftarrow \text{Cost}(\mathbf{x}_{\text{goal}})$ 
14  return failure

```

candidates; $\mathbf{x}_{\text{nearest}}$ is added so that CHOOSEPARENT always has one whose segment is known to be free, and adding it for CHOOSEPARENT only, as Listing 17.1 does, costs at most the rare rewire of $\mathbf{x}_{\text{nearest}}$ itself. In Algorithm 17.2, CHOOSEPARENT starts from the nearest vertex as the default ($\mathbf{x}_{\text{par}}, c_{\text{par}}$ in line 2), whose segment is already known to be free, and improves on it only with a candidate whose segment passes the check in line 5; the early **break** after the sorted loop means that no candidate is tested once it cannot beat the best value found. The new vertex is inserted with its cost (line 10 of Algorithm 17.1) *before* rewiring, because REWIRE reads $\text{Cost}(\mathbf{x}_{\text{new}})$ in line 10 of Algorithm 17.2. Its line 11 tests the improvement first and the collision second, so segments are checked only when they would be used; line 12 swaps the parent and line 13 pushes the new cost down the subtree. Line 12 of Algorithm 17.1 reads c_{best} from the tree; it can only decrease, since REWIRE never increases any cost (Proposition 17.10).

17.4.3 The connection radius

Every symbol in Equation (17.1) has a job.

- $n = |V|$ is the number of vertices *in the tree*, not the number of iterations; blocked samples add no vertex and do not shrink the radius.
- d is the dimension. Volume scales like r^d , so a ball must have radius $\propto (\cdot)^{1/d}$ to contain a prescribed fraction of the space.
- $(\log n/n)^{1/d}$ is the schedule. A ball of radius r_n has volume $\zeta_d r_n^d = \zeta_d \gamma^d \log n/n$, so with n vertices spread over $\mu(\mathcal{X}_{\text{free}})$ the expected number of vertices in it is $n \cdot \zeta_d \gamma^d \log n / (n \mu(\mathcal{X}_{\text{free}})) = \zeta_d \gamma^d \log n / \mu(\mathcal{X}_{\text{free}})$, which grows only logarithmically.
- γ_{RRT^*} is the constant that decides *how many* neighbors that logarithm holds. The threshold $\gamma_{\text{RRT}^*}^*$ makes the expected number of neighbors at least $2^d(1 + 1/d) \log n$; with this many, the ball-covering argument of Section 17.6 goes through.
- $\mu(\mathcal{X}_{\text{free}})$ is the volume of the free space; the larger it is, the sparser the vertices, and the larger the radius must be to find neighbors. Any upper bound, such as $\mu(\mathcal{X})$, is a safe

Algorithm 17.2. The helpers of RRT*

```

1 Function ChooseParent( $X_{\text{near}}, \mathbf{x}_{\text{nearest}}, \mathbf{x}_{\text{new}}$ ):
2    $\mathbf{x}_{\text{par}} \leftarrow \mathbf{x}_{\text{nearest}}; c_{\text{par}} \leftarrow \text{Cost}(\mathbf{x}_{\text{nearest}}) + \|\mathbf{x}_{\text{new}} - \mathbf{x}_{\text{nearest}}\|$ 
3   foreach  $\mathbf{v} \in X_{\text{near}}$  in increasing order of  $\text{Cost}(\mathbf{v}) + \|\mathbf{x}_{\text{new}} - \mathbf{v}\|$  do
4     if  $\text{Cost}(\mathbf{v}) + \|\mathbf{x}_{\text{new}} - \mathbf{v}\| \geq c_{\text{par}}$  then break
5     if CollisionFree( $\mathbf{v}, \mathbf{x}_{\text{new}}$ ) then
6        $\mathbf{x}_{\text{par}} \leftarrow \mathbf{v}; c_{\text{par}} \leftarrow \text{Cost}(\mathbf{v}) + \|\mathbf{x}_{\text{new}} - \mathbf{v}\|$ ; break
7   return  $\mathbf{x}_{\text{par}}$ 

8 Function Rewire( $X_{\text{near}}, \mathbf{x}_{\text{new}}$ ):
9   foreach  $\mathbf{v} \in X_{\text{near}} \setminus \{\text{parent}(\mathbf{x}_{\text{new}})\}$  do
10     $c \leftarrow \text{Cost}(\mathbf{x}_{\text{new}}) + \|\mathbf{v} - \mathbf{x}_{\text{new}}\|$ 
11    if  $c < \text{Cost}(\mathbf{v})$  and CollisionFree( $\mathbf{x}_{\text{new}}, \mathbf{v}$ ) then
12       $E \leftarrow E \setminus \{(\text{parent}(\mathbf{v}), \mathbf{v})\} \cup \{(\mathbf{x}_{\text{new}}, \mathbf{v})\}$ ;  $\text{parent}(\mathbf{v}) \leftarrow \mathbf{x}_{\text{new}}$ 
13      UpdateSubtree( $\mathbf{v}$ )

14 Function UpdateSubtree( $\mathbf{v}$ ):
15    $\text{Cost}(\mathbf{v}) \leftarrow \text{Cost}(\text{parent}(\mathbf{v})) + \|\mathbf{v} - \text{parent}(\mathbf{v})\|$ 
16   foreach child  $\mathbf{u}$  of  $\mathbf{v}$  do UpdateSubtree( $\mathbf{u}$ )           // depth-first over the subtree

```

substitute because a larger γ still satisfies the condition.

- ζ_d converts the radius into a volume.
- η caps the radius at the step size, so that connections are never longer than the segments Steer produces. For small n the logarithmic term is far larger than η and the cap is active; the shrinking phase begins only when $\gamma(\log n/n)^{1/d}$ falls below η .

Example 17.7 (The constants of the two maps). The map of Chapter 16 is the square $[0, 10]^2$ with the walls $[3, 5] \times [0, 6]$ and $[6, 8] \times [4, 10]$, so $\mu(\mathcal{X}_{\text{free}}) = 100 - 12 - 12 = 76$ and, with $d = 2$ and $\zeta_2 = \pi$, $\gamma_{\text{RRT}^*}^* = 2\sqrt{1.5} \sqrt{76/\pi} = 12.05$ (the code's Monte-Carlo estimate $\hat{\mu} = 75.9$ gives 12.04). We use $\gamma_{\text{RRT}^*} = 13.3$, ten percent above the threshold. With $\eta = 1$ the cap is active until $13.3\sqrt{\log n/n} < 1$, that is until $n = 1\,264$ vertices (between iterations 1 500 and 2 000 of the run in Table 17.2); at $n = 2\,193$ the radius is 0.79. For the 3D box field of Chapter 16, $\mu(\mathcal{X}) = 1\,000$ is a safe upper bound for $\mu(\mathcal{X}_{\text{free}})$ and gives $\gamma_{\text{RRT}^*}^* = 2(4/3)^{1/3}(1000/\zeta_3)^{1/3} = 13.66$; the code uses 15.0.

Why the radius must shrink, and shrink slowly. The optimality proof needs two things at once. The tree must keep finding neighbors everywhere along a near-optimal path, which pushes the radius up; and the cost per iteration must stay bounded by a slowly growing function of n , which pushes it down. If r_n shrank like $n^{-1/d}$ or faster, the count of the third bullet above would lose its $\log n$ and stay bounded, so a ball of radius r_n would be empty with a probability bounded away from zero, iteration after iteration. Somewhere along the optimal path a ball would then stay empty for long stretches and the tree path would keep its detour: this is the sense in which *a radius that is too small kills optimality* (Section 17.9). The factor $\log n$ is exactly what makes the expected count grow without bound, and the constant in front decides how fast: with γ above the threshold, the probability that a given ball is empty decays like n^{-c} with c large enough that, summed over all n and over the $\mathcal{O}(n^{1/d})$ balls needed to cover the path, the sum is finite. The schedule $(\log n/n)^{1/d}$ is the slowest shrinking radius that keeps the work per iteration at $\mathcal{O}(\log n)$; the same $\log n/n$ is the threshold at which a

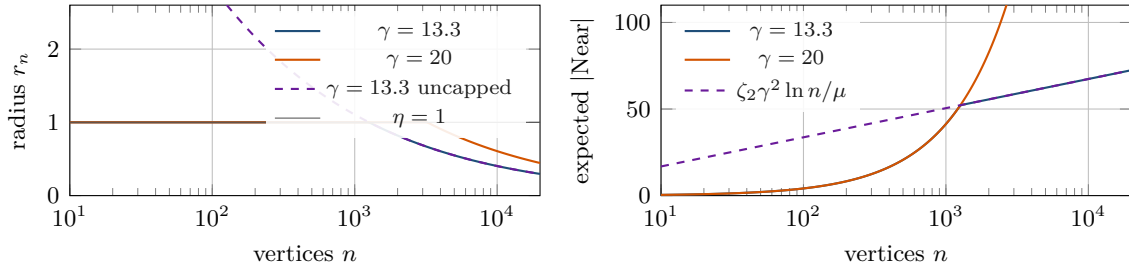


Figure 17.2. Left: the connection radius of Equation (17.1) for the map of Example 17.7 ($\mu = 76$, $d = 2$, $\eta = 1$) with the chapter’s $\gamma = 13.3$ (blue) and with $\gamma = 20$ (orange). The cap η is active while the tree is small, and for longer with the larger constant; the dashed curve is the uncapped schedule for $\gamma = 13.3$. Right: the expected size of the NEAR set, $n\zeta_2 r_n^2 / \mu$, for the same two constants. It grows linearly while the cap is active and then follows $\zeta_2 \gamma^2 \ln n / \mu$ (dashed, $\gamma = 13.3$): the $\gamma = 20$ curve sits a factor $(20/13.3)^2 = 2.3$ above it, which is what more rewiring per iteration costs.

random geometric graph on n uniform points becomes connected [130], and RRT* succeeds because the graph of all pairs within r_n is, with high probability, connected along every path with clearance.

17.4.4 The k -nearest variant

Instead of a ball, NEAR may return the k_n nearest vertices, with

$$k_n = \lceil k_{\text{RRG}} \log n \rceil, \quad k_{\text{RRG}} > e \left(1 + \frac{1}{d}\right), \quad (17.3)$$

where e is Euler’s number [77]. The threshold has the practical merit of not depending on $\mu(\mathcal{X}_{\text{free}})$, which is often unknown; $k_{\text{RRG}} = 2e \approx 5.44$ is valid in every dimension, and the code uses $1.1e(1 + 1/d)$. The trade is a different behavior in non-uniform regions: in a dense cluster the k nearest vertices lie inside a tiny ball and rewiring becomes very local, while in a sparse region the k -th neighbor may be far away and its segment is likely to be blocked. On the map of Example 17.7, $k_n = 33$ at $n = 1438$ vertices, and after 2000 iterations the k -nearest version reaches a best cost of 17.44 against 17.45 for the ball version with the same seed; on this map the two are interchangeable.

17.4.5 One iteration by hand

Example 17.8 (One RRT* iteration on a ten-vertex tree). Figure 17.1 shows a tree with root $\mathbf{v}_0 = (0.5, 0.5)$ in the box $[0, 6] \times [0, 4.5]$ with one obstacle $[2.6, 3.1] \times [1.9, 2.45]$; the first block of Table 17.1 lists the vertices, their parents and their costs. The sample is $\mathbf{x}_{\text{rand}} = (4.0, 2.1)$ and $\eta = 1$. The nearest vertex is $\mathbf{v}_4 = (4.5, 1.3)$ at distance $0.943 < \eta$, so $\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{rand}}$, and the segment $\mathbf{v}_4 \mathbf{x}_{\text{new}}$ is free. With $r_n = 2$, Near returns $\{\mathbf{v}_3, \mathbf{v}_4, \mathbf{v}_6, \mathbf{v}_7, \mathbf{v}_8\}$; the distances to $\mathbf{x}_{\text{new}}$ are 1.900, 0.943, 1.924, 1.221 and 0.985. CHOOSEPARENT sorts the candidates by $\text{Cost}(\mathbf{v}) + \|\mathbf{x}_{\text{new}} - \mathbf{v}\|$: $\mathbf{v}_6$ ($2.654 + 1.924 = 4.578$), $\mathbf{v}_7$ (5.264), $\mathbf{v}_3$ (5.440), $\mathbf{v}_4$ (5.692), $\mathbf{v}_8$ (7.436). The segment from $\mathbf{v}_6$ crosses the obstacle and is rejected; the segment from $\mathbf{v}_7$ is free, so $\mathbf{x}_{\text{new}}$ becomes $\mathbf{v}_{10}$ with parent $\mathbf{v}_7$ and cost 5.264, 0.43 less than through the nearest vertex. REWIRE then tests the remaining candidates: $\mathbf{v}_3$, $\mathbf{v}_4$ and $\mathbf{v}_6$ are cheaper as they are, but $\mathbf{v}_8$ would cost $5.264 + 0.985 = 6.249 < 6.451$, and its segment is free. Its parent changes from $\mathbf{v}_4$ to $\mathbf{v}_{10}$, and UPDATESUBTREE lowers its child $\mathbf{v}_9$ from 7.583 to 7.380 (second block of Table 17.1). Every number is produced by `tiny_example()` in `code/ch17_rrt_star.py`.

Table 17.1. Trace of [Example 17.8](#): the tree before and after one iteration of [Algorithm 17.1](#). Costs are rounded to three decimals; the costs that change are in bold, and the comment column names the two vertices whose parent changes.

Vertex	before			after		comment
	position	parent	cost	parent	cost	
$\mathbf{v}_0$	(0.5, 0.5)	–	0	–	0	root
$\mathbf{v}_1$	(1.6, 0.2)	$\mathbf{v}_0$	1.140	$\mathbf{v}_0$	1.140	
$\mathbf{v}_2$	(2.8, 0.2)	$\mathbf{v}_1$	2.340	$\mathbf{v}_1$	2.340	
$\mathbf{v}_3$	(4.0, 0.2)	$\mathbf{v}_2$	3.540	$\mathbf{v}_2$	3.540	near; 7.164 via $\mathbf{v}_{10}$, no
$\mathbf{v}_4$	(4.5, 1.3)	$\mathbf{v}_3$	4.748	$\mathbf{v}_3$	4.748	nearest; 6.207 via $\mathbf{v}_{10}$, no
$\mathbf{v}_5$	(0.9, 1.7)	$\mathbf{v}_0$	1.265	$\mathbf{v}_0$	1.265	
$\mathbf{v}_6$	(2.1, 2.4)	$\mathbf{v}_5$	2.654	$\mathbf{v}_5$	2.654	near; cheapest parent but blocked
$\mathbf{v}_7$	(3.3, 3.1)	$\mathbf{v}_6$	4.043	$\mathbf{v}_6$	4.043	near; chosen parent
$\mathbf{v}_8$	(4.4, 3.0)	$\mathbf{v}_4$	6.451	$\mathbf{v}_{10}$	6.249	near; rewired
$\mathbf{v}_9$	(5.2, 3.8)	$\mathbf{v}_8$	7.583	$\mathbf{v}_8$	7.380	updated with its parent
$\mathbf{v}_{10}$	(4.0, 2.1)	–	–	$\mathbf{v}_7$	5.264	$\mathbf{x}_{\text{new}}$

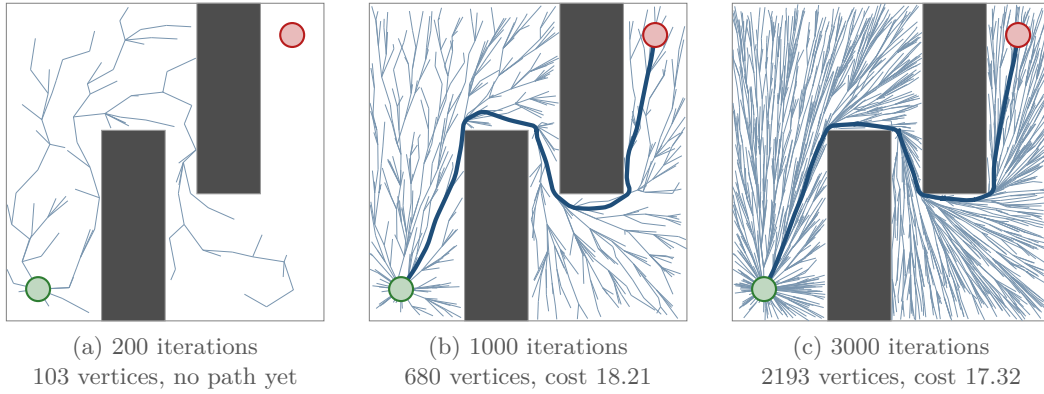


Figure 17.3. RRT* on [Example 17.9](#) after (a) 200, (b) 1 000 and (c) 3 000 iterations, with the current best path in blue. Unlike the RRT tree of [Chapter 16](#) on the same map, the edges point radially away from the start, because every vertex hangs from the parent that gives it the shortest route; the best path straightens as rewiring continues, from 18.21 to 17.32 against the optimum 16.91.

17.5 A worked example

Example 17.9 (RRT* on the map of [Chapter 16](#)). The map is the square $[0, 10]^2$ with the walls $[3, 5] \times [0, 6]$ and $[6, 8] \times [4, 10]$, the start is (1, 1) and the goal (9, 9); the robot is a point and segments are checked at spacing $\delta = 0.05$ with the conservative inflation of [Chapter 16](#). The only route passes over the first wall and under the second, and the optimum, found by Dijkstra on the visibility graph of the wall corners pushed out by the checker's inflation ($\delta/2 = 0.025$ for a point robot) plus a margin of 0.005, has length $c^* = 16.909$. That number says as much about the clearance as about the map: [Chapter 16](#) quotes 16.72 for the same corridor in the limit of zero clearance and 17.20 for a clearance of 0.075. We run [Algorithm 17.1](#) with $\eta = 1$, $\gamma_{\text{RRT}^*} = 13.3$, $p_{\text{goal}} = 0.05$ and seed 1 for 3 000 iterations, recording the tree and the best path at iterations 200, 1 000 and 3 000 ([Figure 17.3](#)) and c_{best} throughout ([Table 17.2](#)).

The first path appears at iteration 351, when the goal point becomes the 204th vertex, and

Table 17.2. Best cost against iterations for [Example 17.9](#) (seed 1), with the number of vertices and the connection radius, which stays at the cap $\eta = 1$ until 1 264 vertices and then follows $13.3\sqrt{\log n/n}$. The last column is plain RRT with the same seed: its first path is its only path.

Iteration	$ V $	r_n	c_{best} (RRT*)	c_{best} (RRT)
200	103	1.000	–	–
351	204	1.000	21.003	25.721
500	311	1.000	19.726	25.721
1 000	680	1.000	18.212	25.721
1 500	1 058	1.000	17.711	25.721
2 000	1 438	0.946	17.452	25.721
3 000	2 193	0.788	17.324	25.721
optimum c^*			16.909	

costs 21.003; RRT with the same seed reaches the goal at the same iteration, because both planners insert the same vertices, but its path costs 25.721 and never changes. From then on RRT* improves at every rewiring that touches the goal’s branch: 19.73 at 500 iterations, 18.21 at 1 000, 17.32 at 3 000, that is 2.5% above the optimum. The improvement slows down as the tree densifies, for the reason [Section 17.6.2](#) gives. Over twenty seeds the final cost of RRT* is 17.33 ± 0.08 (range 17.18 to 17.49) while that of RRT is 23.51 ± 1.67 (range 20.75 to 26.74): the random tree of RRT makes the path quality a lottery, RRT* makes it a matter of budget.

The Week 8 plan also asks for a comparison with grid A^* . On the same map discretized at $h = 0.1$ with 8-connected moves and no corner cutting, A^* ([Chapter 4](#)) returns a path of length 18.29: optimal on the grid, but 8% longer than the continuous optimum because of the octile detours. String pulling (the greedy pruning of [Chapter 16](#)) shortens it to 17.23 and no further, because the grid keeps the path one cell away from every corner. RRT* is still at 17.40 after 2 500 iterations and at 17.32 after 3 000; with the same seed it matches the string-pulled grid path (17.23) at iteration 4 301 and keeps improving (17.05 at 12 000), which no grid refinement can do. It needed 7 997 segment checks for 3 000 iterations, 2.7 times the 3 000 of RRT, with a NEAR set of 23.6 vertices on average.

17.6 Properties: correctness, optimality, complexity

17.6.1 What the tree always satisfies

Proposition 17.10 (Invariants of RRT*). *After every iteration of [Algorithm 17.1](#): (i) $\mathcal{T}$ is a tree rooted at $\mathbf{x}_{\text{init}}$ and every edge is collision-free; (ii) $\text{Cost}(\mathbf{v})$ is the length of the tree path from $\mathbf{x}_{\text{init}}$ to $\mathbf{v}$ for every $\mathbf{v} \in V$; (iii) no cost increases during an iteration, so c_{best} is non-increasing in the number of iterations; (iv) the vertex set is the one that RRT would have built from the same sequence of samples.*

Proof. (i) Every edge is inserted only after CollisionFree has accepted it (line 6 of [Algorithm 17.1](#), lines 5 and 11 of [Algorithm 17.2](#)). A rewire could create a cycle only if $\mathbf{x}_{\text{new}}$ were a descendant of the rewired vertex $\mathbf{v}$; but then the tree path to $\mathbf{x}_{\text{new}}$ passes through $\mathbf{v}$, so $\text{Cost}(\mathbf{x}_{\text{new}}) \geq \text{Cost}(\mathbf{v}) + \|\mathbf{x}_{\text{new}} - \mathbf{v}\|$ by the triangle inequality, and the test $c < \text{Cost}(\mathbf{v})$ in line 11 of [Algorithm 17.2](#) fails. (ii) The new vertex receives $\text{Cost}(\mathbf{x}_{\text{parent}}) + \|\mathbf{x}_{\text{new}} - \mathbf{x}_{\text{parent}}\|$, and after a rewire UPDATESUBTREE recomputes exactly the vertices whose tree path changed, namely $\mathbf{v}$ and its descendants. (iii) CHOOSEPARENT touches no existing cost, and REWIRE lowers $\text{Cost}(\mathbf{v})$ and, by the same amount, the cost of every descendant. (iv) Whether a sample becomes a vertex, and where, depends only on Nearest, Steer and the check of line 6

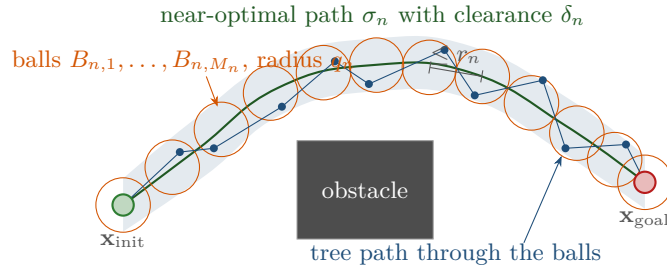


Figure 17.4. The ball-covering argument. A near-optimal path σ_n with clearance δ_n (green, inside its blue tube) is covered by balls of radius q_n (orange) whose centers are spaced so that any two points in consecutive balls are at most r_n apart. If every ball contains a tree vertex, the vertices form a chain (blue) whose consecutive members were in each other's NEAR sets, so the tree connects them; the chain stays inside the tube and is therefore collision-free.

of Algorithm 17.1; these are the steps of RRT, and the extra steps only change edges. $\square$

Part (iv) has a consequence: RRT* contains the same vertices as RRT, so it inherits its probabilistic completeness (Chapter 16).

17.6.2 Asymptotic optimality

Theorem 17.11 (Asymptotic optimality of RRT*; Karaman and Frazzoli 2011). *Let $d \geq 2$, let $\mathcal{X}$ be bounded with $\mu(\mathcal{X}_{\text{free}}) > 0$, let the cost be Euclidean length, let the samples be uniform on $\mathcal{X}$ apart from the goal-biased draws, which stop once $\mathbf{x}_{\text{goal}}$ is in V , let $\mathbf{x}_{\text{goal}}$ enter the tree through the goal bias of Algorithm 17.1, and suppose the problem has a robustly optimal solution (Definition 17.2) of cost $c^* < \infty$. If $\gamma_{\text{RRT}^*} > \gamma_{\text{RRT}^*}^*$ in Equation (17.1), then the best cost Y_i in the tree satisfies $\mathbb{P}(\lim_{i \rightarrow \infty} Y_i = c^*) = 1$.*

Read the statement carefully; it is easy to hear more in it than it says.

- It promises convergence of the *cost*, with probability one, as $n \rightarrow \infty$. It gives no rate: after any fixed budget N the gap $Y_N - c^*$ can be large, and it typically shrinks slowly, because the last few percent are corner cuts that each need a sample in a small region (Table 17.2).
- It does not promise convergence of the *path*: the tree path may keep switching between routes of almost equal cost.
- It needs a robustly optimal solution. An optimum that squeezes through a gap of zero width has no clearance and is never found; the same holds for any planner that checks collisions at points.
- It needs γ_{RRT^*} above a threshold that depends on $\mu(\mathcal{X}_{\text{free}})$, which is usually unknown; any upper bound is safe (Section 17.4.3).
- Nothing is said about RRT: Karaman and Frazzoli also prove that, under the same assumptions, the best cost of RRT converges with probability one to a limit *strictly larger* than c^* [77], because the root of an RRT tree acquires only finitely many children (once vertices surround it, new samples are steered from them instead), so every tree path leaves $\mathbf{x}_{\text{init}}$ along one of finitely many random segments, none of which points along the optimal path.

Proof idea. The argument is illustrated in Figure 17.4; we sketch it in five steps and say where the full proof works hardest.

Step 1 (a path with room). Fix $\varepsilon > 0$. By robust optimality there is a collision-free path σ with strong δ -clearance for some $\delta > 0$ and $c(\sigma) \leq c^* + \varepsilon$. (The full proof uses a sequence σ_n

with $\delta_n \rightarrow 0$ slowly; one path is enough for the idea.)

Step 2 (cover it with balls). Take n so large that $r_n < \delta$. Choose a small $\theta > 0$ and cover σ with M_n balls of radius $q_n = r_n/(2 + \theta)$ whose centers lie on σ at spacing θq_n . Two points in consecutive balls are then at most $\theta q_n + 2q_n = r_n$ apart, and the segment between them stays inside the δ -tube, so it is collision-free. The number of balls is $M_n \approx c(\sigma)/(\theta q_n)$, which grows like $(n/\log n)^{1/d}$.

Step 3 (a ball is rarely empty). A ball of radius q_n has volume $\zeta_d q_n^d = \zeta_d \gamma^d \log n / ((2 + \theta)^d n)$. If the n vertices were independent uniform points in $\mathcal{X}_{\text{free}}$, the ball would be empty with probability $(1 - \zeta_d q_n^d / \mu(\mathcal{X}_{\text{free}}))^n \leq e^{-n \zeta_d q_n^d / \mu(\mathcal{X}_{\text{free}})} = n^{-a}$ with

$$a = \frac{\zeta_d \gamma^d}{(2 + \theta)^d \mu(\mathcal{X}_{\text{free}})}.$$

The vertices of RRT* are not uniform, since Steer produces them; the full proof spends most of its length showing that, once the tree has a vertex within η of every point of the tube, a sample falling into a ball becomes a vertex in that ball, so the estimate holds up to constants.

Step 4 (all balls, for all large n). By the union bound the probability that *some* ball is empty at time n is at most $M_n n^{-a} \propto n^{1/d-a} (\log n)^{-1/d}$. Summed over n this is finite exactly when $a > 1 + 1/d$, that is when $\gamma > (2 + \theta) (1 + 1/d)^{1/d} (\mu(\mathcal{X}_{\text{free}})/\zeta_d)^{1/d}$; because $\gamma > \gamma_{RRT}^*$ strictly, such a $\theta > 0$ exists. This is the origin of the constant $2(1 + 1/d)^{1/d} (\mu(\mathcal{X}_{\text{free}})/\zeta_d)^{1/d}$: the factor 2 is the two ball radii that must fit inside one connection radius, and $(1 + 1/d)^{1/d}$ is the exponent that beats the $n^{1/d}$ balls. The Borel–Cantelli lemma (if $\sum_n \mathbb{P}(A_n) < \infty$, then with probability one only finitely many A_n occur) now says that, from some random n_0 on, *every* ball contains a vertex.

Step 5 (the chain is cheap). Pick one vertex in each ball. Consecutive vertices are within r_n , so the later one found the earlier in its NEAR set when it was inserted (the radius was then even larger), and CHOOSEPARENT and REWIRE connect them at least as cheaply as the direct segment. The chain zigzags inside the tube, so this crude bound only gives a constant factor above $c(\sigma)$; the full proof removes the factor using the $\Theta(\log n)$ vertices that every ball eventually contains, most of which lie very close to the center line, so the best chain straightens as n grows and its cost tends to $c(\sigma) \leq c^* + \varepsilon$. Since ε was arbitrary and $Y_n \geq c^*$ always, $Y_n \rightarrow c^*$ with probability one. $\square$

Remark 17.12 (The fine print of the radius). Equation (17.1) is the constant Karaman and Frazzoli prove, for the rapidly-exploring random *graph* (RRG), which keeps every edge between vertices within r_n , and for RRT* alike. Implementations use a smaller one: Gammell, Srinivasa, and Barfoot [49] and the OMPL library take $\gamma_{RRT}^* = (2(1 + 1/d) \mu(\mathcal{X}_{\text{free}})/\zeta_d)^{1/d}$, which in 2D is $1.73\sqrt{\mu/\pi}$ against the proved $2.45\sqrt{\mu/\pi}$, about 30 % below the threshold. It works well in practice but is not covered by the theorem, so Equation (17.1) is the safe choice. Step 5 is where the tree makes life difficult, because the cost of a vertex depends on the order in which its neighbors arrived: Solovey et al. [155] found a gap there in the original argument and closed it with a radius that shrinks slightly more slowly, $\gamma'(\log n/n)^{1/(d+1)}$. In practice everyone uses Equation (17.1) with γ above the threshold, and it converges as Figure 17.7 shows.

17.6.3 Cost per iteration

An RRT iteration does one nearest-neighbor query and one collision check. An RRT* iteration adds a range query and up to $1 + 2|X_{\text{near}}|$ collision checks (one per candidate in CHOOSEPARENT, one per candidate in REWIRE). By the computation in Section 17.4.3 the expected size of X_{near} is $\zeta_d \gamma^d \log n / \mu(\mathcal{X}_{\text{free}})$, so the number of candidates is $\mathcal{O}(\log n)$ in expectation, and with a kd-tree (Chapter 16) the nearest-neighbor query costs $\mathcal{O}(\log n)$ and the range query $\mathcal{O}(\log n + |X_{\text{near}}|)$,

which is $\mathcal{O}(\log n)$ again because the reported set is itself logarithmic. A run of n iterations therefore costs $\mathcal{O}(n \log n)$, the same order as RRT with a kd-tree; Karaman and Frazzoli show that the ratio of the expected running times of the two planners is bounded by a constant [77]. The constant comes almost entirely from collision checks: in [Example 17.9](#) the NEAR set had 23.6 members on average, but the ordering trick of [Algorithm 17.2](#) kept the count at 2.7 checks per iteration instead of the worst case of 48.

17.7 Informed RRT*: sampling where a better path can pass

17.7.1 The informed set

Once the tree contains a path of cost c_{best} , most of the free space can no longer matter. Take any point $\mathbf{x}$ and any path σ from $\mathbf{x}_{\text{init}}$ to $\mathbf{x}_{\text{goal}}$ through $\mathbf{x}$. The cheapest way into $\mathbf{x}$ is the straight segment from the start and the cheapest way out is the straight segment to the goal, so

$$c(\sigma) \geq \hat{f}(\mathbf{x}) := \|\mathbf{x} - \mathbf{x}_{\text{init}}\| + \|\mathbf{x}_{\text{goal}} - \mathbf{x}\|. \quad (17.4)$$

A path that improves on the current one has $c(\sigma) < c_{\text{best}}$, hence every point on it satisfies $\hat{f}(\mathbf{x}) < c_{\text{best}}$: it lies in the informed set $\mathcal{X}_{\hat{f}(c_{\text{best}})}$ of [Definition 17.6](#). Samples outside that set cannot improve the solution, and a planner that keeps drawing them wastes its iterations. You have met $\hat{f}$ before: it is $g + h$ with the straight-line heuristic, and the informed set is the region $f \leq C^*$ outside which A^* with a consistent heuristic never expands ([Chapter 4](#)).

The set $\{\mathbf{x} : \|\mathbf{x} - \mathbf{x}_{\text{init}}\| + \|\mathbf{x} - \mathbf{x}_{\text{goal}}\| \leq c_{\text{best}}\}$ is, by the string-and-two-pins definition, an ellipse in 2D and an ellipsoid of revolution in 3D; the general name is **prolate hyperspheroid**. Its foci are $\mathbf{x}_{\text{init}}$ and $\mathbf{x}_{\text{goal}}$, its center is $\mathbf{x}_{\text{center}} = (\mathbf{x}_{\text{init}} + \mathbf{x}_{\text{goal}})/2$, and its **transverse diameter** (along the focal axis) is c_{best} . Its **conjugate diameter** follows from the point $\mathbf{x}$ on the conjugate axis at distance b from the center: both foci are at distance $\sqrt{b^2 + c_{\text{min}}^2}/4$ from it, so $\hat{f}(\mathbf{x}) = 2\sqrt{b^2 + c_{\text{min}}^2}/4 = c_{\text{best}}$ gives $2b = \sqrt{c_{\text{best}}^2 - c_{\text{min}}^2}$. In summary,

$$\text{semi-axes} \quad r_1 = \frac{c_{\text{best}}}{2}, \quad r_2 = \dots = r_d = \frac{\sqrt{c_{\text{best}}^2 - c_{\text{min}}^2}}{2}, \quad \mu(\mathcal{X}_{\hat{f}}) = \zeta_d r_1 r_2^{d-1}. \quad (17.5)$$

The volume tells you when informed sampling pays. On the 10×10 map of [Example 17.9](#), $c_{\text{min}} = 11.31$ and the first solution has $c_{\text{best}} = 21.0$, so the ellipse has area 292 and contains the whole map. It keeps containing it until $c_{\text{best}} = 18.11$ ([Exercise 17.6](#)) and then loses only two thin corners: 99.5 % of the map is still inside at $c_{\text{best}} = 17.32$ (the code rejected 8 of 2657 informed draws) and 98.7 % at the optimum, where the ellipse has area 167. Uniform sampling of the map is therefore essentially informed sampling there, which is why the two planners coincide in [Figure 17.7\(a\)](#). In the field $[-10, 20]^2$ with the same two walls, the same start and the same goal (`wide_example_world()`, [Figure 17.5](#)) the free space is nine times larger, and the ellipse of the first solution covers 46 % of the field and that of the final one 10 %, so nine of ten uniform samples are wasted. In 3D the effect is stronger, because the conjugate axes enter the volume with the power $d - 1$: in the box field of [Chapter 16](#) ($c_{\text{min}} = 15.59$) a solution of cost 17.0 has an ellipsoid of volume 41 % of the box, one of cost 16.0 only 11 %.

17.7.2 Sampling the ellipsoid uniformly

Rejection sampling (draw uniformly in $\mathcal{X}$, keep the sample if $\hat{f}(\mathbf{x}) \leq c_{\text{best}}$) is correct but wasteful in exactly the cases where the informed set helps, because its acceptance rate is the volume ratio just computed. Gammell, Srinivasa, and Barfoot [49] sample the hyperspheroid

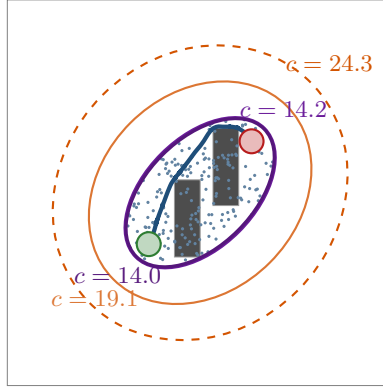


Figure 17.5. Informed RRT* on the 30×30 field $[-10, 20]^2$ with the walls of [Example 17.9](#) (seed 1): the informed set at the first solution ($c_{\text{best}} = 24.3$, iteration 102, dashed) and after 1 000, 2 000 and 3 000 iterations (19.1, 14.2, 14.0; optimum 13.88). The dots are 200 samples from the final set: all of them land where a better path could still pass. The ellipse shrinks toward the segment between the foci as $c_{\text{best}} \rightarrow c_{\text{min}}$.

directly by mapping the unit ball onto it ([Figure 17.6](#)):

$$\mathbf{x} = \mathbf{C}\mathbf{L}\mathbf{x}_{\text{ball}} + \mathbf{x}_{\text{center}}, \quad \mathbf{L} = \text{diag}(r_1, r_2, \dots, r_d), \quad \mathbf{C}\mathbf{e}_1 = \mathbf{a}_1 := \frac{\mathbf{x}_{\text{goal}} - \mathbf{x}_{\text{init}}}{c_{\text{min}}}. \quad (17.6)$$

Here $\mathbf{x}_{\text{ball}}$ is uniform in the unit ball, $\mathbf{L}$ stretches the ball into the axis-aligned spheroid with the semi-axes of [Equation \(17.5\)](#), $\mathbf{C}$ is a rotation matrix that turns the first coordinate axis into the start-to-goal direction, and the translation puts the center between the foci. The map is affine with a constant Jacobian determinant $\det \mathbf{L} = r_1 r_2^{d-1} > 0$, so a uniform density on the ball becomes a uniform density on its image ([Exercise 17.3](#) asks you to verify this and the axes). Two ingredients remain.

- *A uniform point of the unit ball.* Draw $\mathbf{u} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and $w \sim U(0, 1)$ and return $w^{1/d}\mathbf{u}/\|\mathbf{u}\|$: the normalized Gaussian is uniform on the sphere (the standard Gaussian density depends on $\mathbf{u}$ only through $\|\mathbf{u}\|$ and is therefore rotation-invariant; [Exercise 17.3\(c\)](#)), and the radius $w^{1/d}$ has the density $\propto \rho^{d-1}$ that the volume of a shell demands.
- *The rotation to the world frame.* In 2D, $\mathbf{C}$ is the rotation by the angle of $\mathbf{a}_1$, $\mathbf{C} = \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}$ with $(\cos \phi, \sin \phi) = \mathbf{a}_1$. In any dimension, Gammell et al. compute it from the singular value decomposition $\mathbf{U}\Sigma\mathbf{V}^T$ of the rank-one matrix $\mathbf{M} = \mathbf{a}_1\mathbf{e}_1^T$ as

$$\mathbf{C} = \mathbf{U} \text{diag}(1, \dots, 1, \det \mathbf{U} \det \mathbf{V}) \mathbf{V}^T, \quad (17.7)$$

which is orthogonal with determinant +1 and satisfies $\mathbf{C}\mathbf{e}_1 = \mathbf{a}_1$. In 3D any rotation with first column $\mathbf{a}_1$ will do, such as $[\mathbf{a}_1 \quad \mathbf{b} \quad \mathbf{a}_1 \times \mathbf{b}]$ with $\mathbf{b} \perp \mathbf{a}_1$, because the spheroid is symmetric about its transverse axis.

[Algorithm 17.3](#) is the complete planner. Before a solution exists, $c_{\text{best}} = \infty$ and the sampler falls back to the goal-biased uniform sampling of [Algorithm 17.1](#); from the first solution on every sample is drawn inside the current ellipsoid, so the tree grows toward it—a new vertex can still land just outside, because Steer starts from a vertex that may predate the solution—and the ellipsoid shrinks whenever REWIRE improves the goal’s branch. NEAR, CHOOSEPARENT and REWIRE are unchanged, and so is the theory: the samples remain uniform on a set that contains every point that can still improve the solution, so Informed RRT* keeps the probabilistic completeness and the asymptotic optimality of RRT* [\[49\]](#). Samples of the hyperspheroid outside $\mathcal{X}$ are rejected (line 14); when the hyperspheroid is larger than $\mathcal{X}$, the code does the opposite and samples $\mathcal{X}$, rejecting by $\hat{f}$.

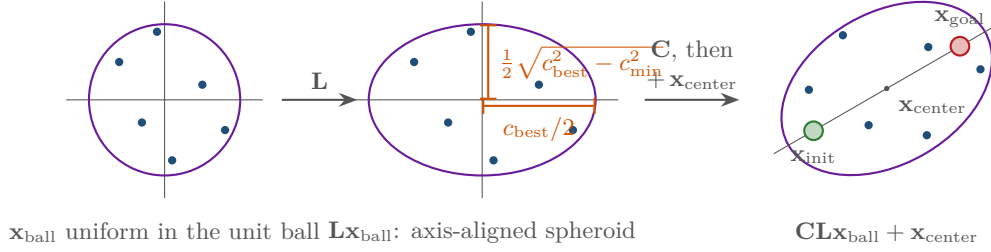


Figure 17.6. Sampling the informed set: the same six points of the unit ball after scaling by $\mathbf{L}$ (semi-axes $c_{\text{best}}/2$ and $\frac{1}{2}\sqrt{c_{\text{best}}^2 - c_{\text{min}}^2}$) and after rotation by $\mathbf{C}$ and translation to $\mathbf{x}_{\text{center}}$; the foci of the result are the start and the goal.

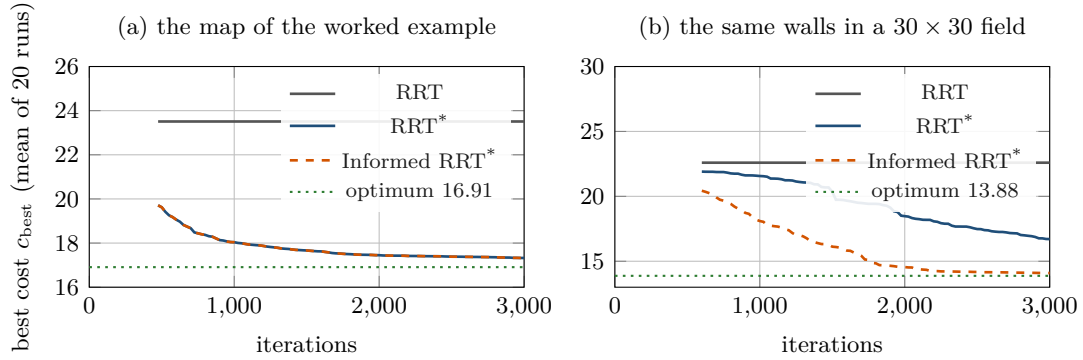


Figure 17.7. Best cost against iterations, mean over 20 seeds, on (a) the map of [Example 17.9](#) and (b) the same walls, start and goal in the 30×30 field $[-10, 20]^2$. Each curve starts at the first iteration at which all 20 runs have a solution. RRT (gray) never improves; RRT* and Informed RRT* coincide on the small map, and on the wide field Informed RRT* (dashed) reaches in 1 000 iterations what RRT* has not reached in 3 000.

17.7.3 What it buys

[Figure 17.7](#) compares RRT, RRT* and Informed RRT* over 20 seeds on both maps; the code is `code/figures/gen_ch17_convergence.py`. On the small map (a) the informed set is essentially the whole map ([Section 17.7.1](#)), so RRT* and Informed RRT* coincide, both reaching the 17.33 ± 0.08 of [Section 17.5](#) while RRT stays at 23.51 ± 1.67 . On the wide field (b) the difference appears: after 1 000 iterations RRT* stands at a mean of 21.57 and Informed RRT* at 18.10; after 3 000 they stand at 16.71 ± 1.23 and 14.09 ± 0.12 against the optimum 13.88. RRT* is still converging, just slowly, because most of its samples land far from the corridor; Informed RRT* has put every sample after the first solution to use, at the price of 8 486 segment checks per run against 7 399, because samples inside the ellipsoid see more neighbors. Gammell, Srinivasa, and Barfoot [49] also shrink the connection radius with the informed set, replacing $\mu(\mathcal{X}_{\text{free}})$ in [Equation \(17.1\)](#) by $\mu(\mathcal{X}_{\hat{f}} \cap \mathcal{X}_{\text{free}})$; keeping $\mu(\mathcal{X}_{\text{free}})$ is admissible ([Section 17.4.3](#)) and those extra checks are what it costs.

17.8 Variants and extensions

The graph and the batch. The rapidly-exploring random graph (RRG) keeps *all* edges between vertices within r_n ; RRT* is its tree, and PRM* is the probabilistic roadmap of [Chapter 16](#) built with the same radius on a batch of n samples; both are asymptotically optimal. Fast marching trees (FMT*) [71] process one batch of samples in a single Dijkstra-like sweep with lazy collision checks, often faster than RRT* for a fixed budget, but not anytime:

Algorithm 17.3. Informed RRT* (Gammell, Srinivasa and Barfoot)**Input:** as in Algorithm 17.1**Output:** the tree path to $\mathbf{x}_{\text{goal}}$ of cost c_{best} , or *failure*

```

1 Function InformedRRTstar( $\mathbf{x}_{\text{init}}, \mathbf{x}_{\text{goal}}, N$ ):
2    $V \leftarrow \{\mathbf{x}_{\text{init}}\}; E \leftarrow \emptyset; c_{\text{best}} \leftarrow \infty; c_{\text{min}} \leftarrow \|\mathbf{x}_{\text{goal}} - \mathbf{x}_{\text{init}}\|; \mathbf{x}_{\text{center}} \leftarrow (\mathbf{x}_{\text{init}} + \mathbf{x}_{\text{goal}})/2$ 
3    $\mathbf{C} \leftarrow \text{RotationToWorldFrame}((\mathbf{x}_{\text{goal}} - \mathbf{x}_{\text{init}})/c_{\text{min}})$ 
4   for  $i \leftarrow 1$  to  $N$  do
5      $\mathbf{x}_{\text{rand}} \leftarrow \text{SampleInformed}(c_{\text{best}})$ 
6     run lines 5–12 of Algorithm 17.1 on  $\mathbf{x}_{\text{rand}}$  (Nearest, Steer, Near, ChooseParent,
7       Rewire)
8   if  $c_{\text{best}} < \infty$  then return the tree path from  $\mathbf{x}_{\text{init}}$  to  $\mathbf{x}_{\text{goal}}$ 
9   return failure

9 Function SampleInformed( $c_{\text{best}}$ ):
10  if  $c_{\text{best}} = \infty$  then return  $\mathbf{x}_{\text{goal}}$  with probability  $p_{\text{goal}}$ , else a uniform sample of  $\mathcal{X}$ 
11   $r_1 \leftarrow c_{\text{best}}/2; r_2, \dots, r_d \leftarrow \sqrt{c_{\text{best}}^2 - c_{\text{min}}^2}/2; \mathbf{L} \leftarrow \text{diag}(r_1, \dots, r_d)$ 
12  repeat
13     $\mathbf{u} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}); w \sim U(0, 1); \mathbf{x}_{\text{ball}} \leftarrow w^{1/d} \mathbf{u} / \|\mathbf{u}\|$ 
14     $\mathbf{x}_{\text{rand}} \leftarrow \mathbf{C} \mathbf{L} \mathbf{x}_{\text{ball}} + \mathbf{x}_{\text{center}}$ 
15  until  $\mathbf{x}_{\text{rand}} \in \mathcal{X}$ 
16  return  $\mathbf{x}_{\text{rand}}$ 

17 Function RotationToWorldFrame( $\mathbf{a}_1$ ):
18   $\mathbf{U} \Sigma \mathbf{V}^T \leftarrow$  singular value decomposition of  $\mathbf{a}_1 \mathbf{e}_1^T$ 
19  return  $\mathbf{U} \text{diag}(1, \dots, 1, \det \mathbf{U} \det \mathbf{V}) \mathbf{V}^T$ 

```

the batch size is chosen before the run.

Anytime RRT*. Karaman et al. [78] run RRT* on a moving robot: the tree keeps growing while the robot follows the current best path, the robot commits to the first segment of that path and the tree is re-rooted at its end, and **branch-and-bound pruning** deletes every vertex $\mathbf{v}$ with $\text{Cost}(\mathbf{v}) + \|\mathbf{x}_{\text{goal}} - \mathbf{v}\| \geq c_{\text{best}}$, which can never lie on a better path. Pruning is the tree-side twin of the informed set, and the two are usually combined (Section 17.9).

BIT* and the batch-informed family. Batch informed trees (BIT*) of Gammell, Srinivasa, and Barfoot [50] take the informed set one step further. Samples are drawn in *batches* inside the current ellipsoid, the batch together with the tree is viewed as an implicit random geometric graph, and that graph is searched with an edge queue ordered by $\hat{f} = \hat{g} + \hat{h}$, exactly as $\mathbf{A}^*$ orders its nodes; collision checks are made only when an edge is popped, and the search is reused across batches in the manner of LPA* (Chapter 5). Each batch shrinks the ellipsoid, and the planner is anytime and asymptotically optimal: it is the point where sampling-based planning and heuristic graph search, the two halves of this book, meet. Later refinements by the same group (ABIT*, AIT*) improve the ordering and the heuristic, and Gammell, Barfoot, and Srinivasa [48] generalize informed sets to other cost functions; Table 17.3 lists every sampling-based planner of the book.

Table 17.3. The sampling-based planners of Chapter 16 and this chapter. “Complete” means probabilistically complete; “optimal” means asymptotically optimal; the work per iteration is for a kd-tree and excludes the constant of the collision checker.

Planner	Where	Complete	Optimal	Work per iteration
RRT (with goal bias)	Chapter 16	✓	no; limit $> c^*$ a.s.	$\mathcal{O}(\log n)$, 1 check
RRT-Connect	Chapter 16	✓	no	$\mathcal{O}(\log n)$, greedy checks
Kinodynamic RRT	Chapter 16	✓	no	forward simulation
PRM (k fixed)	Chapter 16	✓	no	multi-query
RRT*	Section 17.4	✓	✓	$\mathcal{O}(\log n)$ neighbors, ≈ 3 checks
k -nearest RRT*	Section 17.4.4	✓	✓	$k_n = \mathcal{O}(\log n)$, no $\mu(\mathcal{X}_{\text{free}})$
Informed RRT*	Section 17.7	✓	✓	as RRT*, fewer wasted samples
PRM*, FMT*	pointer	✓	✓	one batch, not anytime
BIT*	pointer	✓	✓	batches, A*-ordered lazy edges

17.9 Implementation notes

The file `code/ch17_rrt_star.py` implements everything in this chapter for $d = 2$ and $d = 3$ with NumPy only: the world and the collision checker are copied from Chapter 16, so the two chapters plan on the same maps. Listing 17.1 is the body of one iteration.

Listing 17.1. One iteration of RRT* (from `code/ch17_rrt_star.py`): NEAREST, STEER and the collision check as in RRT, then NEAR, CHOOSEPARENT with the candidates sorted by cost, and REWIRE with the improvement test before the segment check. `tree.set_parent` propagates the new cost to the subtree; `radius` is r_n or k_n .

```

1      # --- Nearest, Steer, CollisionFree (as in RRT) -----
2      i_nearest = tree.nearest(x_rand)
3      x_new = steer(tree.V[i_nearest], x_rand, eta)
4      d_nearest = float(np.linalg.norm(x_new - tree.V[i_nearest]))
5      radius = (knn_count(tree.n, d, k_rrg) if knn
6                else connection_radius(tree.n, d, gamma, eta))
7      if d_nearest > 1e-12 and world.segment_free(tree.V[i_nearest], x_new):
8          # --- Near -----
9          if knn:
10             near = tree.knearest(x_new, radius)
11         else:
12             near = tree.near(x_new, radius)
13         near_total += len(near)
14         dists = np.linalg.norm(tree.V[near] - x_new, axis=1)
15         # --- ChooseParent (c_par is local; c_min is the -----
16         #      start-goal distance of the informed set) -----
17         i_par, c_par = i_nearest, tree.cost[i_nearest] + d_nearest
18         if choose_parent and len(near):
19             cand = tree.cost[near] + dists
20             for k in np.argsort(cand):          # cheapest first
21                 if cand[k] >= c_par:
22                     break
23                 if world.segment_free(tree.V[near[k]], x_new):
24                     i_par, c_par = int(near[k]), float(cand[k])
25                     break
26         i_new = tree.add(x_new, i_par, c_par - tree.cost[i_par])
27         # --- Rewire -----
28         if rewire:
29             for k in range(len(near)):

```

```

30         j = int(near[k])
31         if j == i_par:
32             continue
33         via = c_par + dists[k]
34         if via < tree.cost[j] - 1e-12 and \
35             world.segment_free(x_new, tree.V[j]):
36             tree.set_parent(j, i_new, dists[k])
37         if goal_index < 0 and np.linalg.norm(x_new - goal) < 1e-9:
38             goal_index, solution_iter = i_new, it

```

Choosing γ_{RRT^*} . $\mu(\mathcal{X}_{\text{free}})$ is rarely known; the code estimates it by Monte Carlo for the text but plans with $\gamma = 1.1 \gamma^*(\mu(\mathcal{X}))$ (Section 17.4.3): on the 3D scene of Chapter 16 that is $\gamma = 15.0$ against a threshold of 12.2 for the true free volume of about 716. Overshooting costs work, not optimality (Figure 17.2): doubling γ multiplies the expected NEAR set by 2^d .

The cost of rewiring. Collision checks dominate. Three habits keep them rare: sort the candidates of CHOOSEPARENT by cost and stop at the first free segment; in REWIRE test the improvement before the segment; and reuse the check of the parent segment for the edge to the nearest vertex, which is already known to be free.

Propagating costs to a subtree. Every candidate's cost-to-come must be available in $\mathcal{O}(1)$, so the implementation stores $\text{Cost}(\mathbf{v})$ and keeps it consistent: Listing 17.2 stores, per vertex, its parent, its parent-edge length and its children; `set_parent` moves the vertex, then walks its subtree with a stack and recomputes Cost from the parent, exactly UPDATESUBTREE. The lazy alternative, summing edge lengths on demand, costs $\mathcal{O}(\text{depth})$ per candidate and was slower in every experiment; the self-test asserts the invariant after every run.

Listing 17.2. UPDATESUBTREE (from `code/ch17_rrt_star.py`): rewiring vertex i under p and recomputing the cost of every descendant with an explicit stack.

```

1     def set_parent(self, i, p, edge_len):
2         """Rewire vertex i under p and update the costs of its subtree."""
3         self.children[self.parent[i]].remove(i)
4         self.parent[i], self.edge[i] = p, edge_len
5         self.children[p].append(i)
6         stack = [i]
7         while stack:
8             v = stack.pop()
9             self.cost[v] = self.cost[self.parent[v]] + self.edge[v]
10            stack.extend(self.children[v])

```

Nearest and near. The implementation uses vectorized linear scans, because its trees have a few thousand vertices and a run takes about a second. Beyond 10^4 vertices use a kd-tree (Chapter 16): NEAR becomes a range query, and since points only ever get inserted, a tree rebuilt whenever the vertex count doubles keeps the amortized cost logarithmic. The class `InformedSampler` of the file implements Algorithm 17.3 line by line, with the rotation computed once per problem from the SVD of Equation (17.7).

Anytime use. A drone does not wait for N iterations. The pattern of Karaman et al. [78] is: plan until the first solution, start flying its first segment, keep iterating at a fixed rate, and hand over the new path whenever c_{best} improves by more than a tolerance. At each waypoint re-root the tree there: the waypoint's subtree keeps its structure, its costs drop by the cost of

the waypoint, and the other branches are discarded. With branch-and-bound pruning the tree stays small and focused; [Exercise 17.8](#) builds it.

A radius that is too small kills optimality

RRT* with a constant small radius, or with γ below the threshold, still finds paths and still rewires, so nothing looks broken; it just stops improving at a cost above the optimum, because balls along the optimal route stay empty ([Section 17.4.3](#)). Two ways to get there by accident: forgetting the root, $r_n = \gamma \log n/n$ instead of $\gamma(\log n/n)^{1/d}$, and reusing a γ tuned in 2D for a 3D problem, where ζ_3 and the exponent $1/3$ change the threshold.

Rewiring without propagating

If REWIRE updates $\text{Cost}(\mathbf{v})$ but not the costs of $\mathbf{v}$'s descendants, the tree still returns valid paths, but Cost of those descendants is now too high: CHOOSEPARENT undervalues them as parents and REWIRE rewires them when it should not. Convergence slows down and c_{best} , read from $\text{Cost}(\mathbf{x}_{\text{goal}})$, may be reported wrongly. Keep children lists and propagate ([Listing 17.2](#)), and assert [Proposition 17.10\(ii\)](#) in your tests; the bug shows up only there.

Sampling the ellipse before a solution exists

With $c_{\text{best}} = \infty$ the transformation produces `inf` and `nan`; with $c_{\text{best}} < c_{\text{min}}$, which happens when the goal is a region and c_{min} is measured to its center, the square root is imaginary. Sample uniformly until the goal is in the tree, measure c_{min} to the point actually reached, and clamp $c_{\text{best}}^2 - c_{\text{min}}^2$ at zero.

Rewiring through obstacles

The temptation is to skip the collision check in REWIRE because “ $\mathbf{x}_{\text{new}}$ was checked when it was inserted”. Only the segment to its parent was. Every rewired edge is a new segment and needs its own check (line 11 of [Algorithm 17.2](#)); the same holds for the candidate segments of CHOOSEPARENT, and a checker that is not symmetric (a drone with a heading) must run in the direction the edge will be flown.

17.10 Where this fits in the drone system

In the drone system

RRT* and Informed RRT* are the continuous planners behind the global planner and the replanning layer of [Chapter 24](#). The swarm as a whole is planned on a grid by CBS or ECBS ([Chapters 9](#) and [10](#)), because conflicts between drones are easiest to resolve in discrete space-time. A single drone calls RRT* instead of A* when (i) it must cross a 3D volume whose grid would be far larger than the tree that solves it, (ii) its constraints are not grid-shaped, such as a turning radius handled by kinodynamic steering, or (iii) an unexpected obstacle has invalidated a long stretch of its nominal route and the replanning layer wants a short, smooth detour rather than a staircase. It keeps A* or D* Lite ([Chapters 4](#) and [5](#)) when the map is small, when the plan must be time-indexed for the conflict re-check, or when the map keeps changing and incremental repair is cheaper than a new tree. Three habits make the planner fit the architecture. *Anytime operation*: return the first path after a few hundred iterations, hand it to the local layer

(optimal reciprocal collision avoidance, ORCA, or the dynamic window approach, DWA, [Chapters 13 and 14](#)), and keep improving it while the drone flies, re-rooting at every waypoint. *Seeding*: string-pull the grid path from CBS ([Chapter 16](#)), insert its waypoints as the initial tree and set c_{best} to its length, so that Informed RRT* samples its ellipsoid from the first iteration (on the map of [Example 17.9](#) that path costs 17.23, the first random solution 21.0). *Re-check*: a continuous path is still a single-agent path; after any replan, run the conflict check of [Chapter 24](#) against the other drones' reservations before flying it. This is Week 8 of the study plan ([Appendix A](#)); its milestone is being able to say, for a given mission, which planner of [Table 17.3](#) or which grid search to call.

17.11 Summary

Chapter summary

- RRT* adds NEAR, CHOOSEPARENT and REWIRE to RRT: the new vertex takes the cheapest collision-free parent within r_n , then becomes the parent of every neighbor it can serve more cheaply, and the saving is propagated to all descendants.
- The connection radius $r_n = \min\{\gamma_{\text{RRT}^*}(\log n/n)^{1/d}, \eta\}$ with $\gamma_{\text{RRT}^*} > 2(1 + 1/d)^{1/d}(\mu(\mathcal{X}_{\text{free}})/\zeta_d)^{1/d}$ shrinks as slowly as it can while keeping $\mathcal{O}(\log n)$ neighbors; the k -nearest variant uses $k_n = \lceil k_{\text{RRG}} \log n \rceil$ and needs no volume.
- With that radius the best cost converges to the optimum almost surely (ball covering plus Borel–Cantelli); the theorem gives no rate, needs a robustly optimal solution and says nothing about the path itself. The limit of RRT is suboptimal.
- An iteration costs $\mathcal{O}(\log n)$ expected neighbors and a few collision checks; a run costs $\mathcal{O}(n \log n)$, a constant factor above RRT.
- Once a solution of cost c_{best} exists, only the prolate hyperspheroid with foci $\mathbf{x}_{\text{init}}, \mathbf{x}_{\text{goal}}$, transverse diameter c_{best} and conjugate diameter $\sqrt{c_{\text{best}}^2 - c_{\text{min}}^2}$ can contain a better path; Informed RRT* samples it uniformly (scale the unit ball, rotate, translate) and converges much faster in large spaces.
- BIT* searches batches of informed samples with an A*-ordered edge queue; anytime RRT* re-roots the tree while the drone flies and prunes vertices that cannot help.

Further reading. The original paper is Karaman and Frazzoli [77], which introduces RRG, RRT* and PRM*, proves that RRT is not asymptotically optimal and contains the full proof; anytime use is described by Karaman et al. [78], and the proof is revisited by Solovey et al. [155]. Informed RRT* is due to Gammell, Srinivasa, and Barfoot [49] and was generalized by Gammell, Barfoot, and Srinivasa [48]; BIT* is due to Gammell, Srinivasa, and Barfoot [50] and FMT* to Janson et al. [71]. The textbook treatment of sampling-based planning is LaValle [99], and the tree itself goes back to LaValle [98]. The connectivity threshold of random geometric graphs, the reason the radius has the form it has, is the subject of Penrose [130].

17.12 Exercises

Exercise 17.1 (★). Repeat [Example 17.8](#) on the tree of [Table 17.1](#) (before the iteration) with the sample $\mathbf{x}_{\text{rand}} = (5.0, 2.6)$, $\eta = 1$ and $r_n = 2$. Give the nearest vertex, $\mathbf{x}_{\text{new}}$, the NEAR set, the candidates of CHOOSEPARENT in the order they are tested with their costs, the chosen parent, and every rewiring with the costs before and after. Check with `tiny_example(x_rand=(5.0, 2.6))`.

Exercise 17.2 (★). (a) For the map of [Example 17.7](#) ($\gamma = 13.3$, $\eta = 1$), find the number of vertices at which the cap in [Equation \(17.1\)](#) stops being active, and the expected size of the NEAR set at $n = 10^4$. (b) The 3D scene of [Chapter 16](#) has $\mu(\mathcal{X}_{\text{free}}) \approx 716$. Compute γ_{RRT}^* with this value and with $\mu(\mathcal{X}) = 1000$, and the number of vertices at which the cap $\eta = 1.5$ stops being active for $\gamma = 15$. (c) Explain, with the computation of [Section 17.4.3](#), what goes wrong with $r_n = \gamma \log n / n$ in 2D.

Exercise 17.3 (★★). Derive the sampling transformation of [Equation \(17.6\)](#). (a) Show that the set $\hat{f}(\mathbf{x}) \leq c_{\text{best}}$ is the image of the unit ball under $\mathbf{x} = \mathbf{C}\mathbf{L}\mathbf{x}_{\text{ball}} + \mathbf{x}_{\text{center}}$ with the semi-axes of [Equation \(17.5\)](#); start from the frame in which the foci are $(\pm c_{\text{min}}/2, 0, \dots, 0)$. (b) Show that an affine map with invertible matrix $\mathbf{A}$ sends a uniform density to a uniform density divided by $|\det \mathbf{A}|$. (c) Show that $w^{1/d} \mathbf{u} / \|\mathbf{u}\|$ is uniform in the unit ball. (d) Verify that [Equation \(17.7\)](#) is orthogonal, has determinant +1 and maps $\mathbf{e}_1$ to $\mathbf{a}_1$; in 2D reduce it to a rotation by the angle of $\mathbf{a}_1$.

Exercise 17.4 (★★). Prove [Proposition 17.10\(i\)](#) in detail: show that a rewire can never create a cycle, and construct a small tree, sample and near set in which REWIRE considers a candidate that would create one and rejects it. Then show that the *order* in which REWIRE processes the candidates does not change the final tree of the iteration (except for exact ties), even when one candidate is a descendant of another, but can change the propagation work; which order minimizes it?

Exercise 17.5 (★★). Explain why the root of an RRT tree has only finitely many children with probability one, and why this implies that the best cost of RRT converges to a random limit above c^* . Then test it: run `rrt_plain` on [Example 17.9](#) for 20 seeds and 10000 iterations, keeping the goal bias on and recording the best of *all* goal-reaching paths, and compare the final costs with [Figure 17.7\(a\)](#).

Exercise 17.6 (★★). (a) For the wide field of [Figure 17.5](#) ($\mathcal{X} = [-10, 20]^2$) compute the fraction of the field inside the informed set at $c_{\text{best}} = 24.3, 19.1$ and 14.0 , and the expected number of uniform draws per accepted sample if rejection sampling were used. (b) Do the same for the 3D scene ($c_{\text{min}} = 15.59$, $\mathcal{X} = [0, 10]^3$) at $c_{\text{best}} = 23.45, 21.07, 17.0$ and 16.0 . (c) At which c_{best} does the ellipse on the 10×10 map of [Example 17.9](#) stop containing the whole map?

Exercise 17.7 (★★★ Coding). This completes the Week 8 coding exercise of the study plan ([Appendix A](#)), whose first half is [Exercise 16.8](#). Starting from your RRT of [Chapter 16](#), implement RRT* in the API of `code/ch17_rrt_star.py` (read that file only when stuck): a tree with parents, edge lengths, costs and children lists, NEAR by radius and by k nearest, CHOOSEPARENT with sorted candidates, REWIRE with subtree propagation. (a) Reproduce [Table 17.1](#) and [Table 17.2](#). (b) Run it in the 3D box field of [Chapter 16](#) with $\eta = 1.5$, record c_{best} against iterations, and compare with 26-connected grid A^* at $h = 0.25$ ([Chapter 4](#)) after string pulling, in path length and in computation time. (c) Add informed sampling with [Equation \(17.6\)](#) and show, with a plot like [Figure 17.7](#), on which of your maps it helps. (d) Write the self-test: paths collision-free at a finer subdivision, costs consistent, c_{best} non-increasing, informed samples inside the ellipsoid, final cost within 5% of the grid reference.

Exercise 17.8 (★★★). Build the anytime planner of [Section 17.9](#). (a) Implement re-rooting: given a waypoint $\mathbf{w}$ on the current best path, make $\mathbf{w}$ the root, subtract $\text{Cost}(\mathbf{w})$ from its subtree and discard the rest. (b) Implement branch-and-bound pruning and measure how many vertices it removes on the wide field after the first solution. (c) Seed the tree with the string-pulled grid path of [Example 17.9](#) as its initial branch, set c_{best} to its length and compare the convergence with an unseeded Informed RRT* over 20 seeds. (d) Simulate a drone flying one unit per 50 iterations along the current path and report the length flown against the first path and the optimum.

Part VI

Tracking and Prediction

The Kalman Filter

Learning objectives

- Write down the linear-Gaussian state-space model of a moving drone, $\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{B}\mathbf{u}_k + \mathbf{w}_k$ and $\mathbf{z}_k = \mathbf{H}\mathbf{x}_k + \mathbf{v}_k$, and say what each of $\mathbf{F}$, $\mathbf{H}$, $\mathbf{Q}$ and $\mathbf{R}$ encodes.
- Derive the predict and update steps of the Kalman filter from two facts about Gaussians, and explain the Kalman gain as a precision-weighted compromise between prediction and measurement.
- Build the constant-velocity model in 2D and 3D with the white-noise-acceleration process noise, derive its $\mathbf{Q}$, and run the filter by hand for a few steps.
- Tune $\mathbf{Q}$ and $\mathbf{R}$ with the innovation statistics, gate outliers with the Mahalanobis distance, initialize a track from two measurements and cope with missing measurements.
- Propagate the covariance several steps ahead to obtain the growing uncertainty ellipses that the local safety layer uses, implement the filter in Python, and verify its accuracy and consistency on a simulation.

18.1 Why this matters for a drone swarm

The planners of Parts II and III know where every drone of the swarm is, because the swarm reports its own positions. An intruder reports nothing. A foreign drone that crosses the swarm’s airspace is seen only through a sensor — a radar, a camera with a depth estimate, a receiver of broadcast position messages — that delivers a noisy position a few times per second. Suppose the sensor reports the intruder’s position every $\Delta t = 0.1$ s with an error of about 1 m in each coordinate. The velocity obstacle of [Chapter 12](#), used by the local safety layer of [Chapter 13](#), needs the intruder’s *velocity*, and the obvious way to obtain it is to difference two consecutive reports. That divides the difference of two 1-meter errors by 0.1 s: the velocity estimate carries a noise of about $\sqrt{2} \cdot 1/0.1 \approx 14$ m/s, several times the speed of the intruder itself. [Figure 18.1](#) shows what this looks like. The raw positions jitter by meters from one report to the next, and the differenced velocity is useless. No avoidance rule can work with that input: ORCA would see a different collision cone every tenth of a second, and a formation would scatter in response to noise.

The Kalman filter turns this stream of jittery reports into a stable estimate of the intruder’s state — position *and* velocity — together with a covariance matrix that says how far the estimate can be trusted. It does so by carrying a model of how the intruder moves, predicting where it will be at the next report, and correcting the prediction by the new measurement, weighting the two by their uncertainties. In the experiment of [Figure 18.1](#), produced by the

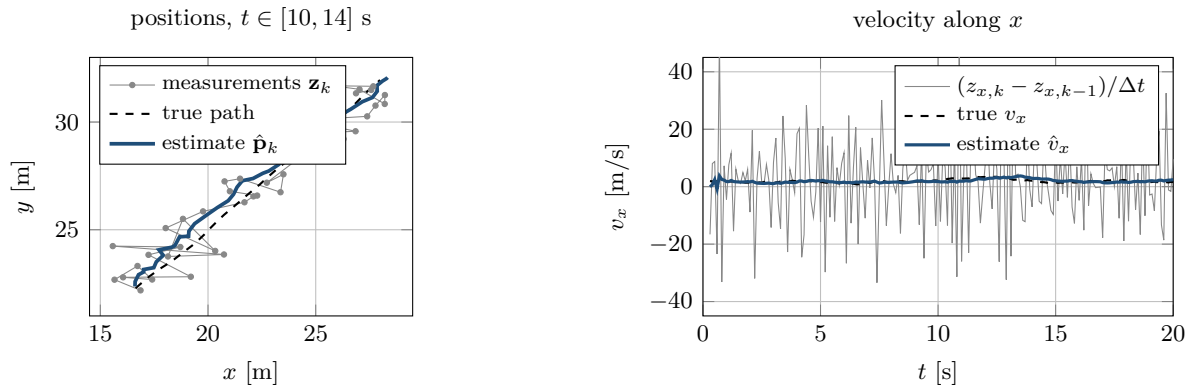


Figure 18.1. Why raw measurements are unusable for planning. Left: four seconds of the experiment of Section 18.9, position reports at 10 Hz with noise 1 m along x and 0.5 m along y (gray), the true path (dashed) and the Kalman estimate (blue). Right: the velocity v_x from differencing consecutive reports (gray) swings by tens of meters per second, while the filter’s estimate (blue) follows the true velocity (dashed) to within a fraction of a meter per second.

code of this chapter, the filter reduces the position error from 1.18 m to 0.52 m (root-mean-square error, RMSE) and the velocity error from 16.8 m/s to 0.79 m/s, at the cost of a few small matrix products per report.

In the hybrid architecture of Chapter 24 the filter is the core of the *prediction layer*, and it is the Week 9 topic of the study plan (Appendix A); Section 18.12 sets out the interface it offers the rest of the system.

The chapter proceeds as follows. Sections 18.2 and 18.3 give the idea, the linear-Gaussian model and the constant-velocity model of a drone; Section 18.4 derives the two steps and explains the gain; Section 18.5 runs the filter by hand; Section 18.6 proves what it guarantees; Section 18.7 covers tuning, gating, missing measurements and initialization; Section 18.8 propagates uncertainty into the future; Section 18.9 reports an experiment; and Sections 18.10 to 18.12 close with variants, implementation notes and the filter’s place in the drone system.

18.2 The idea in plain words

You have two sources of information about the intruder, and both are imperfect. The first is a *motion model*: a drone that was at some position with some velocity a tenth of a second ago is now, very probably, a tenth of a second further along that velocity. The model is not exact, because the intruder may accelerate, so its prediction becomes less certain the longer you rely on it. The second is the *sensor*, which reports the position with an error that does not accumulate but is large at every single report. Neither source alone is good enough; combined, they are.

The Kalman filter is the combination. It keeps a Gaussian belief about the state, a mean $\hat{\mathbf{x}}$ and a covariance $\mathbf{P}$, and alternates two steps (Figure 18.2). In the **predict step** it pushes the belief through the motion model: the mean moves along the velocity and the covariance grows, because the model’s own noise is added. In the **update step** it compares the new measurement with the predicted measurement; the difference is the *innovation*, the news that the measurement brings. The state is corrected by a fraction of the innovation, and that fraction, the *Kalman gain*, is large when the prediction is uncertain and the sensor accurate, small in the opposite case. The covariance shrinks, because two independent pieces of information are worth more than one. Then the cycle repeats.

Behind the fraction is the arithmetic of a weighted average: two readings of the same

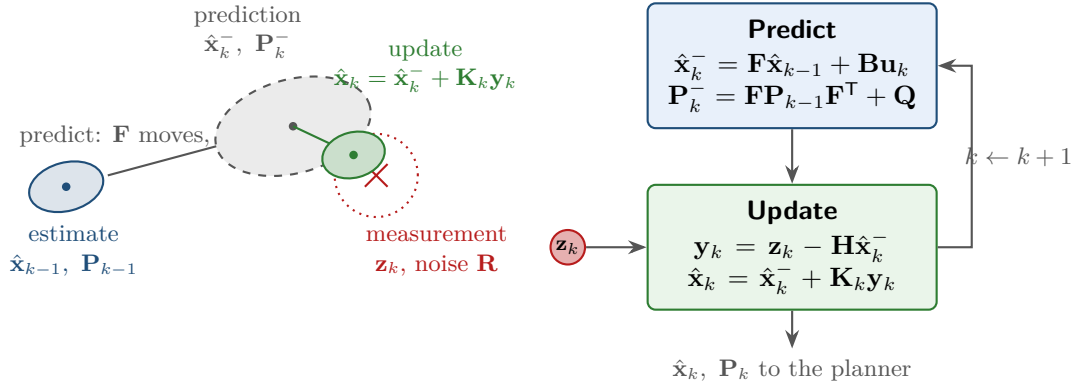


Figure 18.2. One cycle of the Kalman filter. Left, in the plane of positions: the estimate at step $k - 1$ (blue ellipse) is moved by the motion model and inflated by the process noise into the prediction (gray, dashed); the measurement $\mathbf{z}_k$ (red cross, with its noise circle) pulls the mean toward itself, and the updated estimate (green) is both shifted and tighter than the prediction. Right: the same cycle as a loop; the planner reads $\hat{\mathbf{x}}_k$ and $\mathbf{P}_k$ after every update.

quantity are best combined with weights proportional to their precisions (inverse variances), and the precision of the result is the sum of the two. The rest of the chapter is the bookkeeping that keeps this rule exact when the state has several components, the sensor sees only some of them, and time passes between readings.

Key idea

Keep a Gaussian belief $\mathcal{N}(\hat{\mathbf{x}}, \mathbf{P})$ over the state. Predict by pushing it through the motion model: the mean moves, the covariance grows by $\mathbf{Q}$. Update by moving the mean toward the measurement with a gain that weights prediction and measurement by their precisions: the covariance shrinks. For a linear model with Gaussian noise this recursion is exact and optimal — no estimator, linear or not, has a smaller mean-square error.

18.3 Problem statement and notation

18.3.1 The linear-Gaussian state-space model

Definition 18.1 (Linear-Gaussian state-space model). A **linear-Gaussian state-space model** for a state $\mathbf{x}_k \in \mathbb{R}^n$ observed through measurements $\mathbf{z}_k \in \mathbb{R}^m$ at steps $k = 1, 2, \dots$ consists of a **motion model** and a **measurement model**,

$$\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{B}\mathbf{u}_k + \mathbf{w}_k, \quad \mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}), \quad (18.1)$$

$$\mathbf{z}_k = \mathbf{H}\mathbf{x}_k + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}), \quad (18.2)$$

with an initial belief $\mathbf{x}_0 \sim \mathcal{N}(\hat{\mathbf{x}}_0, \mathbf{P}_0)$. $\mathbf{F} \in \mathbb{R}^{n \times n}$ is the **state transition matrix**, $\mathbf{B} \in \mathbb{R}^{n \times l}$ the control matrix acting on a known input $\mathbf{u}_k \in \mathbb{R}^l$ (absent for an intruder, $\mathbf{B}\mathbf{u}_k = \mathbf{0}$), $\mathbf{H} \in \mathbb{R}^{m \times n}$ the **measurement matrix**, $\mathbf{Q}$ the **process noise covariance** and $\mathbf{R}$ the **measurement noise covariance**, assumed positive definite ($\mathbf{R} \succ \mathbf{0}$) so that the innovation covariance $\mathbf{H}\mathbf{P}_k^-\mathbf{H}^T + \mathbf{R}$ met below is invertible at every step. The noise vectors $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{v}_1, \mathbf{v}_2, \dots$ and $\mathbf{x}_0$ are mutually independent. The matrices may depend on k ; the index is dropped when they do not.

$\mathbf{F}$ carries the deterministic part of the motion (a drone keeps its velocity), $\mathbf{w}_k$ collects what

the model cannot know (the intruder's accelerations) and $\mathbf{Q}$ says how large these unknowns are per step; $\mathbf{H}$ selects the linear combinations of the state that the sensor sees and $\mathbf{R}$ says how noisy it is. The independence assumption is what makes a recursion possible: once the current state is known, the past measurements carry no further information about the future.

Notation: $\mathbf{F}$ and $\mathbf{H}$, or $\mathbf{A}$ and $\mathbf{C}$?

This book writes the transition matrix as $\mathbf{F}$ and the measurement matrix as $\mathbf{H}$, the convention of the tracking literature [7]. Control texts, and the model predictive controller of Chapter 21, write the same matrices as $\mathbf{A}$ and $\mathbf{C}$; the double-integrator model of Chapter 2 uses $\mathbf{A}$ and $\mathbf{B}$; Thrun, Burgard, and Fox [168] use $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, and Welch and Bishop [175] use $\mathbf{A}$ and $\mathbf{H}$. Beware also that $\mathbf{Q}$ and $\mathbf{R}$ are noise covariances here but cost weights in Chapter 21; the notation table in the front matter lists both meanings. One collision is internal to this chapter: $\mathbf{v}_k$, with a time index, is the measurement noise of Equation (18.2), while $\mathbf{v}$ without one is the velocity block of the state (Definition 18.4).

18.3.2 What the filter computes

Definition 18.2 (Filtering, prediction, estimate). Write $\mathbf{z}_{1:k} = (\mathbf{z}_1, \dots, \mathbf{z}_k)$ for the measurements received up to step k . The **filtering problem** is to compute, at every step, the posterior density $p(\mathbf{x}_k | \mathbf{z}_{1:k})$. Under Definition 18.1 this density is Gaussian, and the filter represents it by its mean and covariance, the **estimate**

$$\hat{\mathbf{x}}_k = \mathbb{E}[\mathbf{x}_k | \mathbf{z}_{1:k}], \quad \mathbf{P}_k = \text{Cov}(\mathbf{x}_k | \mathbf{z}_{1:k}). \quad (18.3)$$

The density $p(\mathbf{x}_k | \mathbf{z}_{1:k-1})$, before $\mathbf{z}_k$ has been used, is the **prediction**; it is Gaussian too, with mean $\hat{\mathbf{x}}_k^-$ and covariance $\mathbf{P}_k^-$. As in the notation table, the superscript minus marks quantities computed before the current measurement is used.

For general densities the recursive Bayes filter [168, ch. 2] propagates the posterior through the motion model and multiplies it by the likelihood $p(\mathbf{z}_k | \mathbf{x}_k)$, two integrals without closed form; for the linear-Gaussian model both reduce to matrix arithmetic, because of two facts about Gaussians.

18.3.3 Two facts about Gaussians

Lemma 18.3 (Affine transformation and conditioning of Gaussians). *Let $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ in $\mathbb{R}^n$.*

- (i) *If $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b} + \mathbf{w}$ with $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$ independent of $\mathbf{x}$ ($\mathbf{Q}$ may be singular, and $\mathbf{A}$ need not have full rank), then $\mathbf{y} \sim \mathcal{N}(\mathbf{A}\boldsymbol{\mu} + \mathbf{b}, \mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top + \mathbf{Q})$.*
- (ii) *If $\mathbf{a}$ and $\mathbf{b}$ are jointly Gaussian,*

$$\begin{pmatrix} \mathbf{a} \\ \mathbf{b} \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_a \\ \mu_b \end{pmatrix}, \begin{pmatrix} \boldsymbol{\Sigma}_{aa} & \boldsymbol{\Sigma}_{ab} \\ \boldsymbol{\Sigma}_{ba} & \boldsymbol{\Sigma}_{bb} \end{pmatrix}\right)$$

with $\boldsymbol{\Sigma}_{bb}$ invertible, then $\mathbf{a}$ given $\mathbf{b}$ is Gaussian with

$$\mathbb{E}[\mathbf{a} | \mathbf{b}] = \mu_a + \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}(\mathbf{b} - \mu_b), \quad \text{Cov}(\mathbf{a} | \mathbf{b}) = \boldsymbol{\Sigma}_{aa} - \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}\boldsymbol{\Sigma}_{ba}. \quad (18.4)$$

Both facts are proved in Appendix B — (ii) is the conditioning theorem there, whose covariance is the Schur complement of $\boldsymbol{\Sigma}_{bb}$ — and in Thrun, Burgard, and Fox [168, ch. 3].

Fact (i) is the predict step in disguise: the motion model Equation (18.1) is an affine map of $\mathbf{x}_{k-1}$ plus independent noise. Fact (ii) is the update step: the state and the measurement

are jointly Gaussian, and conditioning on the observed $\mathbf{z}_k$ gives the posterior. Note that the conditional covariance in Equation (18.4) does not depend on the value of $\mathbf{b}$: the covariances of the filter can be computed before any measurement arrives, which Section 18.8 exploits.

18.3.4 Motion models for a drone

Definition 18.4 (Constant-velocity model). The **constant-velocity (CV) model** of a drone in d dimensions ($d = 2$ or 3) has the state $\mathbf{x} = (\mathbf{p}, \mathbf{v}) \in \mathbb{R}^{2d}$, position and velocity, and, for a sampling interval Δt ,

$$\mathbf{F} = \begin{pmatrix} \mathbf{I}_d & \Delta t \mathbf{I}_d \\ \mathbf{0} & \mathbf{I}_d \end{pmatrix}, \quad \mathbf{Q} = q \begin{pmatrix} \frac{\Delta t^3}{3} \mathbf{I}_d & \frac{\Delta t^2}{2} \mathbf{I}_d \\ \frac{\Delta t^2}{2} \mathbf{I}_d & \Delta t \mathbf{I}_d \end{pmatrix}, \quad \mathbf{H} = \begin{pmatrix} \mathbf{I}_d & \mathbf{0} \end{pmatrix} \text{ or } \mathbf{H} = \mathbf{I}_{2d}, \quad (18.5)$$

where $q > 0$ is the **white-noise-acceleration intensity** in m^2/s^3 . The first $\mathbf{H}$ describes a sensor that measures positions only, the second one that measures positions and velocities; $\mathbf{R}$ is then $\sigma_p^2 \mathbf{I}_d$, or $\text{diag}(\sigma_p^2 \mathbf{I}_d, \sigma_v^2 \mathbf{I}_d)$ for a sensor with independent errors of standard deviation σ_p in position and σ_v in velocity.

$\mathbf{F}$ is the double-integrator matrix of Chapter 2 with the acceleration removed: $\mathbf{p}_k = \mathbf{p}_{k-1} + \Delta t \mathbf{v}_{k-1}$ and $\mathbf{v}_k = \mathbf{v}_{k-1}$. The intruder does accelerate, of course; the model treats its acceleration as noise, and the shape of $\mathbf{Q}$ is what makes that treatment consistent.

Proposition 18.5 (Process noise of the constant-velocity model). *Suppose that between two samples the acceleration of the drone along one axis is a continuous-time white noise $a(t)$ with $\mathbb{E}[a(t)] = 0$ and $\mathbb{E}[a(t)a(s)] = q \delta(t - s)$. Then the disturbance of position and velocity along that axis over a step of length Δt has covariance $q \begin{pmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{pmatrix}$, and the disturbances of different axes are independent, which gives the $\mathbf{Q}$ of Equation (18.5).*

Proof. Integrate the continuous-time model $\dot{\mathbf{p}} = \mathbf{v}$, $\dot{\mathbf{v}} = a(t)$ over $[0, \Delta t]$ along one axis. A unit impulse of acceleration at time $\tau \in [0, \Delta t]$ changes the velocity at time Δt by 1 and the position by the remaining time $\Delta t - \tau$, because it acts on the velocity, which then integrates into the position. Superposing all instants,

$$\mathbf{w} = \begin{pmatrix} w_p \\ w_v \end{pmatrix} = \int_0^{\Delta t} \begin{pmatrix} \Delta t - \tau \\ 1 \end{pmatrix} a(\tau) d\tau.$$

Its covariance is $\mathbb{E}[\mathbf{w}\mathbf{w}^T] = \int_0^{\Delta t} \int_0^{\Delta t} \begin{pmatrix} \Delta t - \tau \\ 1 \end{pmatrix} \begin{pmatrix} \Delta t - \tau' & 1 \end{pmatrix} \mathbb{E}[a(\tau)a(\tau')] d\tau d\tau'$, and the delta function collapses the double integral to a single one. With $s = \Delta t - \tau$,

$$\mathbb{E}[\mathbf{w}\mathbf{w}^T] = q \int_0^{\Delta t} \begin{pmatrix} s^2 & s \\ s & 1 \end{pmatrix} ds = q \begin{pmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{pmatrix}.$$

Independence across axes follows from independent accelerations along the axes. In matrix form, $\mathbf{w} = \int_0^{\Delta t} e^{\mathbf{A}(\Delta t - \tau)} \mathbf{G} a(\tau) d\tau$ with $\mathbf{A} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ and $\mathbf{G} = (0, 1)^T$, and $e^{\mathbf{A}s} = \mathbf{I} + s\mathbf{A}$ because $\mathbf{A}^2 = \mathbf{0}$; the self-test of `ch18_kalman.py` checks the closed form against this integral numerically. $\square$

Two features of $\mathbf{Q}$ matter in practice. First, q is a *rate*: it does not change when the sampling interval changes, because $\mathbf{Q}$ carries all the dependence on Δt ; a sensor that suddenly reports at 20 Hz instead of 10 Hz needs a new $\mathbf{Q}$ but the same q . Second, the off-diagonal term couples position and velocity, because an unknown acceleration moves both in the same direction. Dropping the off-diagonal term is a common shortcut and a mild error; using $\mathbf{Q} = q \mathbf{I}$ regardless of Δt is just as common and much worse (Section 18.11). As a starting value, if the

intruder's accelerations have magnitude about $a_{\max}$ and persist for about T_a seconds, take $q \approx a_{\max}^2 T_a$, so that the model's velocity uncertainty after T_a , $\sqrt{qT_a}$, matches the velocity change $a_{\max}T_a$ of a real maneuver; [Section 18.7](#) refines the value from data.

The constant-acceleration model. When the intruder maneuvers persistently — long turns, sustained climbs — a model that carries the acceleration as a state reacts faster. The **constant-acceleration (CA) model** has the state $(\mathbf{p}, \mathbf{v}, \mathbf{a}) \in \mathbb{R}^{3d}$, the transition $\mathbf{p}_k = \mathbf{p}_{k-1} + \Delta t \mathbf{v}_{k-1} + \frac{1}{2} \Delta t^2 \mathbf{a}_{k-1}$, $\mathbf{v}_k = \mathbf{v}_{k-1} + \Delta t \mathbf{a}_{k-1}$, $\mathbf{a}_k = \mathbf{a}_{k-1}$, a position-selecting $\mathbf{H}$, and a white-noise *jerk* of intensity q (in m^2/s^5) as its process noise; the derivation of [Proposition 18.5](#) with the vector $((\Delta t - \tau)^2/2, \Delta t - \tau, 1)^\top$ gives a per-axis $\mathbf{Q}$ with entries $q\Delta t^5/20, q\Delta t^4/8, q\Delta t^3/6$ in the first row, $q\Delta t^3/3, q\Delta t^2/2$ in the second and $q\Delta t$ in the corner ([Exercise 18.4](#)). The price is a larger state whose acceleration the sensor sees only through two integrations, so the estimate is noisier during straight flight. Bar-Shalom, Li, and Kirubarajan [7] treat both models in detail, together with turning models, and [Chapter 19](#) compares filters on a turning target. This chapter uses the CV model throughout; `ch18_kalman.py` builds both.

18.4 The algorithm

The filter is the two facts of [Lemma 18.3](#) applied in turn, once per measurement. This section derives the two steps, explains the gain, and writes the result as pseudocode.

18.4.1 The predict step

Suppose that after step $k-1$ the belief is $\mathbf{x}_{k-1} \mid \mathbf{z}_{1:k-1} \sim \mathcal{N}(\hat{\mathbf{x}}_{k-1}, \mathbf{P}_{k-1})$. The motion model [Equation \(18.1\)](#) is an affine map of $\mathbf{x}_{k-1}$, with $\mathbf{A} = \mathbf{F}$ and $\mathbf{b} = \mathbf{B}\mathbf{u}_k$, plus the independent noise $\mathbf{w}_k$. Fact (i) gives the prediction directly:

$$\hat{\mathbf{x}}_k^- = \mathbf{F}\hat{\mathbf{x}}_{k-1} + \mathbf{B}\mathbf{u}_k, \quad \mathbf{P}_k^- = \mathbf{F}\mathbf{P}_{k-1}\mathbf{F}^\top + \mathbf{Q}. \quad (18.6)$$

The mean moves as the noise-free model says. The covariance is transformed by $\mathbf{F}$ and then inflated by $\mathbf{Q}$, and the transformation is not a mere rescaling: for the CV model along one axis, with $\mathbf{P}_{k-1} = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$ (position variance a , velocity variance c , covariance b),

$$\mathbf{P}_k^- = \begin{pmatrix} a + 2\Delta t b + \Delta t^2 c + q\Delta t^3/3 & b + \Delta t c + q\Delta t^2/2 \\ b + \Delta t c + q\Delta t^2/2 & c + q\Delta t \end{pmatrix}, \quad (18.7)$$

so the position variance absorbs the velocity variance ($\Delta t^2 c$): a drone whose velocity is uncertain has an uncertain position a moment later, even if its position was known exactly. This coupling is what makes a position sensor informative about the velocity in the update step.

18.4.2 The update step

Now the measurement $\mathbf{z}_k$ arrives. Before it is read, the state and the measurement are jointly Gaussian given $\mathbf{z}_{1:k-1}$: the state has mean $\hat{\mathbf{x}}_k^-$ and covariance $\mathbf{P}_k^-$, and $\mathbf{z}_k = \mathbf{H}\mathbf{x}_k + \mathbf{v}_k$ is an affine function of it plus independent noise. Fact (i) applied to the stacked vector $(\mathbf{x}_k, \mathbf{z}_k)$ gives the joint distribution

$$\begin{pmatrix} \mathbf{x}_k \\ \mathbf{z}_k \end{pmatrix} \mid \mathbf{z}_{1:k-1} \sim \mathcal{N}\left(\begin{pmatrix} \hat{\mathbf{x}}_k^- \\ \mathbf{H}\hat{\mathbf{x}}_k^- \end{pmatrix}, \begin{pmatrix} \mathbf{P}_k^- & \mathbf{P}_k^- \mathbf{H}^\top \\ \mathbf{H}\mathbf{P}_k^- & \mathbf{H}\mathbf{P}_k^- \mathbf{H}^\top + \mathbf{R} \end{pmatrix}\right), \quad (18.8)$$

where the cross-covariance is $\text{Cov}(\mathbf{x}_k, \mathbf{H}\mathbf{x}_k + \mathbf{v}_k) = \mathbf{P}_k^- \mathbf{H}^\top$ because $\mathbf{v}_k$ is independent of $\mathbf{x}_k$. Conditioning on the observed value of $\mathbf{z}_k$ with fact (ii), $\mathbf{a} = \mathbf{x}_k$ and $\mathbf{b} = \mathbf{z}_k$, is the whole update step. Written out with the standard names, it reads

$$\mathbf{y}_k = \mathbf{z}_k - \mathbf{H}\hat{\mathbf{x}}_k^-, \quad \text{innovation} \quad (18.9)$$

$$\mathbf{S}_k = \mathbf{H}\mathbf{P}_k^- \mathbf{H}^\top + \mathbf{R}, \quad \text{innovation covariance} \quad (18.10)$$

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}^\top \mathbf{S}_k^{-1}, \quad \text{Kalman gain} \quad (18.11)$$

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + \mathbf{K}_k \mathbf{y}_k, \quad \text{updated mean} \quad (18.12)$$

$$\mathbf{P}_k = \mathbf{P}_k^- - \mathbf{K}_k \mathbf{S}_k \mathbf{K}_k^\top = (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_k^-, \quad \text{updated covariance.} \quad (18.13)$$

The innovation is the part of the measurement that the prediction did not already know: if the sensor reported exactly what the filter expected, $\mathbf{y}_k = \mathbf{0}$ and the mean does not move. Its covariance $\mathbf{S}_k$ is what the filter *believes* the innovation's spread to be, and the two together are the raw material of every diagnostic in [Section 18.7](#). The two expressions for $\mathbf{P}_k$ agree because $\mathbf{K}_k \mathbf{S}_k \mathbf{K}_k^\top = \mathbf{P}_k^- \mathbf{H}^\top \mathbf{S}_k^{-1} \mathbf{H} \mathbf{P}_k^- = \mathbf{K}_k \mathbf{H} \mathbf{P}_k^-$.

The Joseph form. A third expression for the updated covariance, the **Joseph form**,

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_k^- (\mathbf{I} - \mathbf{K}_k \mathbf{H})^\top + \mathbf{K}_k \mathbf{R} \mathbf{K}_k^\top, \quad (18.14)$$

is the one to implement. Expanding it gives $(\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^- - \mathbf{P}^- \mathbf{H}^\top \mathbf{K}^\top + \mathbf{K}\mathbf{S}\mathbf{K}^\top$ for any matrix $\mathbf{K}$, and the last two terms cancel exactly when $\mathbf{K} = \mathbf{P}^- \mathbf{H}^\top \mathbf{S}^{-1}$, so [Equation \(18.14\)](#) equals [Equation \(18.13\)](#) for the Kalman gain. The difference is what happens when $\mathbf{K}$ is *not* exactly the Kalman gain, because of rounding or because a fixed approximate gain is used on purpose: the Joseph form is the covariance of the error $\mathbf{x}_k - \hat{\mathbf{x}}_k = (\mathbf{I} - \mathbf{K}\mathbf{H})(\mathbf{x}_k - \hat{\mathbf{x}}_k^-) - \mathbf{K}\mathbf{v}_k$ for *any* gain, a sum of two symmetric positive semidefinite matrices and hence symmetric positive semidefinite whatever $\mathbf{K}$ is, while the short form can lose symmetry and even positive definiteness (the self-test constructs such a case). [Section 18.6](#) uses the same expansion to prove that the Kalman gain is the best gain.

18.4.3 The gain as a weighting between prediction and measurement

Take the simplest case, a scalar position measured directly: $n = m = 1$, $\mathbf{H} = 1$, prediction $\hat{x}^-$ with variance P^- , measurement z with variance R . Then $S = P^- + R$ and

$$K = \frac{P^-}{P^- + R} \in (0, 1), \quad \hat{x} = (1 - K) \hat{x}^- + K z, \quad \frac{1}{P} = \frac{1}{P^-} + \frac{1}{R}. \quad (18.15)$$

The updated mean is a weighted average of the prediction and the measurement, and the weights are proportional to the *precisions* $1/P^-$ and $1/R$: $K = (1/R)/(1/P^- + 1/R)$. The precision of the result is the sum of the two precisions, the weighted-average rule of [Section 18.2](#). Three limits are worth remembering. If the sensor is perfect, $R \rightarrow 0$, then $K \rightarrow 1$ and the filter simply takes the measurement. If the prediction is certain, $P^- \rightarrow 0$, then $K \rightarrow 0$ and the measurement is ignored. If the measurement is missing, treat it as infinitely noisy, $R \rightarrow \infty$: $K \rightarrow 0$ and $P = P^-$, so a missing measurement is handled by skipping the update ([Section 18.7.4](#)).

Example 18.6 (One-dimensional update). The prediction says the intruder is at $\hat{x}^- = 2$ m with variance $P^- = 1$ m² (standard deviation 1 m); the sensor reports $z = 3$ m with variance $R = 0.25$ m² (standard deviation 0.5 m). Then $S = 1.25$, $K = 1/1.25 = 0.8$, the innovation is $y = 1$ m, and

$$\hat{x} = 2 + 0.8 \cdot 1 = 2.8 \text{ m}, \quad P = (1 - 0.8) \cdot 1 = 0.2 \text{ m}^2 \quad (\text{standard deviation } 0.447 \text{ m}).$$

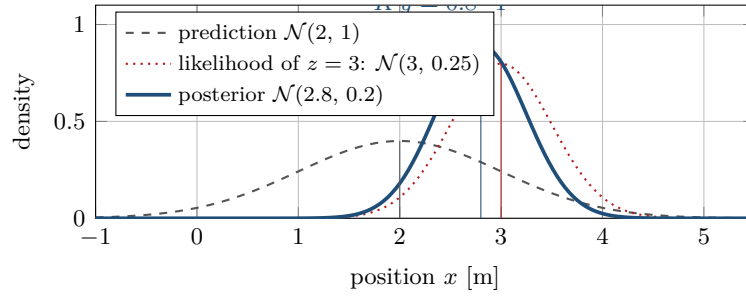


Figure 18.3. The one-dimensional update of [Example 18.6](#). The prediction (gray, dashed) and the likelihood of the measurement (red, dotted) multiply to the posterior (blue), which is narrower than both and centered at $\hat{x} = \hat{x}^- + Ky$ with $K = 0.8$. The wider density contributes less to the position of the peak: the arrow is 80% of the way from $\hat{x}^-$ to z .

The precision form gives the same numbers: $1/P = 1 + 4 = 5$ and $\hat{x} = P(\hat{x}^-/P^- + z/R) = 0.2(2 + 12) = 2.8$. The estimate lies four times closer to the measurement than to the prediction, because the sensor is four times more precise, and it is more precise than either: 0.447 m against 1 m and 0.5 m. [Figure 18.3](#) shows the three densities. The posterior is proportional to the product of the prediction density and the likelihood, and the product of two Gaussians is a Gaussian whose peak lies between the two and whose width is smaller than both.

In the matrix case the same reading survives. Invert the updated covariance of [Equation \(18.13\)](#) with the Woodbury identity of [Appendix B](#):

$$\mathbf{P}_k^{-1} = (\mathbf{P}_k^-)^{-1} + \mathbf{H}^T \mathbf{R}^{-1} \mathbf{H}, \quad \hat{\mathbf{x}}_k = \mathbf{P}_k((\mathbf{P}_k^-)^{-1} \hat{\mathbf{x}}_k^- + \mathbf{H}^T \mathbf{R}^{-1} \mathbf{z}_k). \quad (18.16)$$

Precisions add, and the updated mean is the precision-weighted average of the prediction and of the measurement mapped into state space: the **information form** of the update ([Section 18.10](#)). The gain form [Equation \(18.11\)](#) is preferred in practice because it inverts the $m \times m$ matrix $\mathbf{S}_k$ rather than $n \times n$ matrices and works when $\mathbf{P}_k^-$ is singular.

One feature of the matrix gain has no scalar analog. $\mathbf{K}_k$ is $n \times m$: it corrects *every* component of the state, including those the sensor does not see. For the CV model with a position sensor along one axis, $\mathbf{H} = (1 \ 0)$, $\mathbf{P}^- = \begin{pmatrix} a^- & b^- \\ b^- & c^- \end{pmatrix}$ and $\mathbf{R} = r$,

$$S = a^- + r, \quad \mathbf{K} = \frac{1}{a^- + r} \begin{pmatrix} a^- \\ b^- \end{pmatrix} = \begin{pmatrix} k_p \\ k_v \end{pmatrix}. \quad (18.17)$$

The velocity is corrected by $k_v y$ with $k_v = b^-/S$: an innovation in the position is attributed partly to a velocity error, in proportion to the covariance b^- between the two, which the predict step created in [Equation \(18.7\)](#). This is how a filter fed with positions alone produces a velocity estimate, and why that estimate is so much better than a finite difference: the filter divides by the innovation covariance, not by Δt .

18.4.4 Pseudocode

[Algorithm 18.1](#) states the two steps as functions of the current belief; [Algorithm 18.2](#) is the loop that a tracker runs on a measurement stream, with the practical additions — initialization, missing measurements, gating and prediction ahead — that [Sections 18.7](#) and [18.8](#) explain in detail.

Lines 2–3 are [Equation \(18.6\)](#); for an intruder $\mathbf{B}\mathbf{u}$ is dropped. Lines 6–7 compute the innovation and its covariance, which the implementation also returns, because the tracking loop needs them for gating and the engineer for tuning. Line 8 is [Equation \(18.11\)](#) written as

Algorithm 18.1. The Kalman filter: one predict step and one update step

Input: belief $(\hat{\mathbf{x}}, \mathbf{P})$; model $\mathbf{F}, \mathbf{B}, \mathbf{H}, \mathbf{Q}, \mathbf{R}$; control $\mathbf{u}$; measurement $\mathbf{z}$
Output: the predicted belief $(\hat{\mathbf{x}}^-, \mathbf{P}^-)$, or the updated belief $(\hat{\mathbf{x}}, \mathbf{P})$

```

1 Function KalmanPredict( $\hat{\mathbf{x}}, \mathbf{P}, \mathbf{u}$ ):
2    $\hat{\mathbf{x}}^- \leftarrow \mathbf{F}\hat{\mathbf{x}} + \mathbf{B}\mathbf{u}$ 
3    $\mathbf{P}^- \leftarrow \mathbf{F}\mathbf{P}\mathbf{F}^\top + \mathbf{Q}$ 
4   return  $(\hat{\mathbf{x}}^-, \mathbf{P}^-)$ 

5 Function KalmanUpdate( $\hat{\mathbf{x}}^-, \mathbf{P}^-, \mathbf{z}$ ):
6    $\mathbf{y} \leftarrow \mathbf{z} - \mathbf{H}\hat{\mathbf{x}}^-$  // innovation
7    $\mathbf{S} \leftarrow \mathbf{H}\mathbf{P}^-\mathbf{H}^\top + \mathbf{R}$ 
8    $\mathbf{K} \leftarrow \mathbf{P}^-\mathbf{H}^\top\mathbf{S}^{-1}$  // solve  $\mathbf{KS} = \mathbf{P}^-\mathbf{H}^\top$ ; do not invert  $\mathbf{S}$ 
9    $\hat{\mathbf{x}} \leftarrow \hat{\mathbf{x}}^- + \mathbf{K}\mathbf{y}$ 
10   $\mathbf{P} \leftarrow (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^-(\mathbf{I} - \mathbf{K}\mathbf{H})^\top + \mathbf{K}\mathbf{R}\mathbf{K}^\top$  // Joseph form
11   $\mathbf{P} \leftarrow \frac{1}{2}(\mathbf{P} + \mathbf{P}^\top)$  // remove round-off asymmetry
12  return  $(\hat{\mathbf{x}}, \mathbf{P})$ 

```

a linear solve with the symmetric positive definite $\mathbf{S}$: cheaper and better conditioned than an explicit inverse, a habit that pays off in the larger filters of Chapter 19. Line 10 is the Joseph form and line 11 keeps $\mathbf{P}$ exactly symmetric over millions of steps (Section 18.11). A predict step costs $\mathcal{O}(n^3)$; an update costs $\mathcal{O}(n^2m + m^3)$ in the short form of Equation (18.13) and $\mathcal{O}(n^3)$ in the Joseph form of line 10, which multiplies the $n \times n$ matrix $\mathbf{I} - \mathbf{K}\mathbf{H}$ by $\mathbf{P}^-$ and again by its transpose. For the CV model in 3D, $n = 6$ and $m = 3$, that is about two thousand floating-point operations per cycle, so a filter runs at kilohertz rates on a flight computer.

Algorithm 18.2 wraps the two steps in the loop that the prediction layer of Chapter 24 runs; Sections 18.7.3 to 18.7.5 and 18.8 fill in its gating, missing-measurement, initialization and prediction lines. After $M_{\max}$ consecutive misses (line 16) the covariance has grown so much that any measurement would pass the gate, and the track is dropped and restarted.

18.5 A worked example

Example 18.7 (Five steps of a two-dimensional constant-velocity filter). A drone flies in the plane with the true state $(1, 0.5)$ m/s of velocity starting at the origin, so that its true position at step k is $(k, 0.5k)$. The sensor reports its position once per second, $\Delta t = 1$ s, with independent errors of standard deviation $\sigma_p = 0.5$ m per axis, $\mathbf{R} = 0.25 \mathbf{I}_2$. The filter uses the CV model with $q = 0.1$ m²/s³ and starts from a deliberately poor initial belief: $\hat{\mathbf{x}}_0 = (0, 0, 0.5, 0)$ — the velocity is half the true one and points along x — with $\mathbf{P}_0 = \mathbf{I}_4$. The five measurements, drawn once with a fixed seed and rounded, are

$$\mathbf{z}_1 = (1.28, 0.31), \mathbf{z}_2 = (1.71, 0.92), \mathbf{z}_3 = (3.03, 1.81), \mathbf{z}_4 = (4.31, 1.79), \mathbf{z}_5 = (5.34, 2.88).$$

Because $\mathbf{F}$, $\mathbf{Q}$, $\mathbf{H}$, $\mathbf{R}$ and $\mathbf{P}_0$ are block-diagonal with identical blocks for the x and y axes, the four-dimensional filter is two identical two-dimensional filters run side by side, sharing one 2×2 covariance $\begin{pmatrix} a & b \\ b & c \end{pmatrix}$ and one gain (k_p, k_v) , and the whole computation fits on a sheet of paper. With $\Delta t = 1$ the per-axis recursion, from Equation (18.7) and Equation (18.17), is

$$\begin{aligned}
 \text{predict: } a^- &= a + 2b + c + \frac{q}{3}, & b^- &= b + c + \frac{q}{2}, & c^- &= c + q, \\
 \text{update: } S &= a^- + r, & k_p &= \frac{a^-}{S}, & k_v &= \frac{b^-}{S}, \\
 a &= (1 - k_p) a^-, & b &= (1 - k_p) b^-, & c &= c^- - k_v b^-,
 \end{aligned} \tag{18.18}$$

Algorithm 18.2. Tracking an intruder from a stream of position measurements

Input: sampling interval Δt ; CV model $\mathbf{F}, \mathbf{H}, \mathbf{Q}$ of Equation (18.5); sensor noise $\mathbf{R}$; gate threshold γ (Table 18.3); prediction horizon J steps

Output: after every step, the estimate $(\hat{\mathbf{x}}_k, \mathbf{P}_k)$ and the predictions $(\hat{\mathbf{x}}_{k+j|k}, \mathbf{P}_{k+j|k})$, $j = 1, \dots, J$

```

1 Function TrackIntruder(stream of measurements  $\mathbf{z}_1, \mathbf{z}_2, \dots$  (some missing)):
2   wait for two measurements  $\mathbf{z}_a, \mathbf{z}_b$  received one step apart
3    $(\hat{\mathbf{x}}, \mathbf{P}) \leftarrow \text{InitFromTwoMeasurements}(\mathbf{z}_a, \mathbf{z}_b, \Delta t, \mathbf{R})$  // Equation (18.22)
4    $\text{misses} \leftarrow 0$ 
5   foreach following step  $k$  do
6      $(\hat{\mathbf{x}}^-, \mathbf{P}^-) \leftarrow \text{KalmanPredict}(\hat{\mathbf{x}}, \mathbf{P}, 0)$ 
7     if  $\mathbf{z}_k$  is missing then
8        $(\hat{\mathbf{x}}, \mathbf{P}) \leftarrow (\hat{\mathbf{x}}^-, \mathbf{P}^-)$ ;  $\text{misses} \leftarrow \text{misses} + 1$  // predict-only step
9     else
10       $\mathbf{y} \leftarrow \mathbf{z}_k - \mathbf{H}\hat{\mathbf{x}}^-$ ;  $\mathbf{S} \leftarrow \mathbf{H}\mathbf{P}^-\mathbf{H}^\top + \mathbf{R}$ ;  $d^2 \leftarrow \mathbf{y}^\top \mathbf{S}^{-1} \mathbf{y}$  // reused in
        KalmanUpdate
11      if  $d^2 > \gamma$  then
12         $(\hat{\mathbf{x}}, \mathbf{P}) \leftarrow (\hat{\mathbf{x}}^-, \mathbf{P}^-)$ ;  $\text{misses} \leftarrow \text{misses} + 1$  // gated out: treat as missing
13      else
14         $(\hat{\mathbf{x}}, \mathbf{P}) \leftarrow \text{KalmanUpdate}(\hat{\mathbf{x}}^-, \mathbf{P}^-, \mathbf{z}_k)$ ;  $\text{misses} \leftarrow 0$ 
15      if  $\text{misses} > M_{\max}$  then
16        drop the track and restart from line 3
17      deliver  $(\hat{\mathbf{x}}, \mathbf{P})$  and  $\text{PredictAhead}(\hat{\mathbf{x}}, \mathbf{P}, J)$  to the local layer
        // Equation (18.23)

```

with $r = \sigma_p^2 = 0.25$; the updated a, b, c follow from $(\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^-$, whose first column is scaled by $1 - k_p$ and whose $(2, 2)$ entry loses $k_v b^-$.

Step 1 by hand. Predict: $\hat{\mathbf{x}}_1^- = \mathbf{F}\hat{\mathbf{x}}_0 = (0.5, 0, 0.5, 0)$, and from $a = c = 1, b = 0$: $a^- = 1 + 0 + 1 + 0.0333 = 2.0333, b^- = 0 + 1 + 0.05 = 1.05, c^- = 1.1$. The position variance has doubled because the velocity was uncertain. Update: $S = 2.0333 + 0.25 = 2.2833, k_p = 0.8905, k_v = 0.4599$. The innovation is $\mathbf{y}_1 = \mathbf{z}_1 - \hat{\mathbf{p}}_1^- = (0.78, 0.31)$, so

$$\begin{aligned} \hat{\mathbf{x}}_1 &= (0.5 + 0.8905 \cdot 0.78, 0 + 0.8905 \cdot 0.31, 0.5 + 0.4599 \cdot 0.78, 0 + 0.4599 \cdot 0.31) \\ &= (1.195, 0.276, 0.859, 0.143), \end{aligned}$$

and $a = 0.1095 \cdot 2.0333 = 0.2226, b = 0.1095 \cdot 1.05 = 0.1150, c = 1.1 - 0.4599 \cdot 1.05 = 0.6172$. One position measurement has moved the velocity estimate from $(0.5, 0)$ to $(0.86, 0.14)$, toward the true $(1, 0.5)$, through the covariance term b^- . The position standard deviation is now $\sqrt{0.2226} = 0.47$ m, slightly *better* than the sensor's 0.5 m, because the prior still contributed a little.

Table 18.1 continues the computation and Figure 18.4 draws it. Three things are worth noticing. The gain *decreases*, $k_p = 0.89, 0.82, 0.76, 0.71, 0.68$, as the covariance settles: at first the filter trusts the sensor because it knows nothing, later the prediction is worth weighting; the gain converges to a steady value that depends only on $q, \mathbf{R}$ and Δt (Section 18.6), not on the measurements. The velocity estimate recovers from its wrong start, and both final errors fall inside the standard deviations the filter reports. And the last column lists $d_k^2 = \mathbf{y}_k^\top \mathbf{S}_k^{-1} \mathbf{y}_k$, the squared Mahalanobis distance of each measurement from its prediction, which for a correctly

Table 18.1. Trace of [Algorithm 18.1](#) on [Example 18.7](#) (`worked_example()` in `ch18_kalman.py`): a predict row and an update row per step. The mean is (p_x, p_y, v_x, v_y) ; (a, b, c) are the per-axis entries of $\mathbf{P}_k^-$ or $\mathbf{P}_k$, and S_k, k_p, k_v are per axis too. The true state at step k is $(k, 0.5k, 1, 0.5)$.

k	phase	mean (p_x, p_y, v_x, v_y)	(a, b, c)	measurement, innovation, gain
1	predict	(0.500, 0.000, 0.500, 0.000)	(2.033, 1.050, 1.100)	$\mathbf{z}_1 = (1.28, 0.31)$, $\mathbf{y}_1 = (0.780, 0.310)$, $S_1 = 2.283$
	update	(1.195, 0.276, 0.859, 0.143)	(0.223, 0.115, 0.617)	$k_p = 0.891$, $k_v = 0.460$, $d_1^2 = 0.31$
2	predict	(2.053, 0.419, 0.859, 0.143)	(1.103, 0.782, 0.717)	$\mathbf{z}_2 = (1.71, 0.92)$, $\mathbf{y}_2 = (-0.343, 0.501)$, $S_2 = 1.353$
	update	(1.773, 0.827, 0.660, 0.432)	(0.204, 0.145, 0.265)	$k_p = 0.815$, $k_v = 0.578$, $d_2^2 = 0.27$
3	predict	(2.434, 1.260, 0.660, 0.432)	(0.791, 0.460, 0.365)	$\mathbf{z}_3 = (3.03, 1.81)$, $\mathbf{y}_3 = (0.596, 0.550)$, $S_3 = 1.041$
	update	(2.887, 1.678, 0.923, 0.675)	(0.190, 0.110, 0.162)	$k_p = 0.760$, $k_v = 0.441$, $d_3^2 = 0.63$
4	predict	(3.810, 2.353, 0.923, 0.675)	(0.606, 0.323, 0.262)	$\mathbf{z}_4 = (4.31, 1.79)$, $\mathbf{y}_4 = (0.500, -0.563)$, $S_4 = 0.856$
	update	(4.164, 1.954, 1.112, 0.463)	(0.177, 0.094, 0.141)	$k_p = 0.708$, $k_v = 0.377$, $d_4^2 = 0.66$
5	predict	(5.276, 2.418, 1.112, 0.463)	(0.539, 0.285, 0.241)	$\mathbf{z}_5 = (5.34, 2.88)$, $\mathbf{y}_5 = (0.064, 0.462)$, $S_5 = 0.789$
	update	(5.320, 2.734, 1.135, 0.630)	(0.171, 0.090, 0.138)	$k_p = 0.683$, $k_v = 0.361$, $d_5^2 = 0.28$

tuned filter is chi-square with $m = 2$ degrees of freedom, mean 2; the five values average 0.43, far below the expected $m = 2$ — for a correctly initialized filter a sum this small would have probability $\mathbb{P}(\chi_{10}^2 \leq 2.15) \approx 0.005$. The cause is the deliberately pessimistic $\mathbf{P}_0 = \mathbf{I}_4$: while the filter is still shedding that excess covariance, $\mathbf{S}_k$ is larger than the innovations warrant. This is why the NIS test of [Section 18.7.2](#) is run over a long record and after the transient, not over five steps. The self-test of `ch18_kalman.py` reproduces every number of the table to four decimals and checks the step-1 gain against the hand computation.

18.6 Properties: optimality, consistency, observability

Theorem 18.8 (The Kalman filter is exact and optimal for the linear-Gaussian model). *Under [Definition 18.1](#), the posterior $p(\mathbf{x}_k | \mathbf{z}_{1:k})$ is the Gaussian $\mathcal{N}(\hat{\mathbf{x}}_k, \mathbf{P}_k)$ whose parameters are produced by [Equation \(18.6\)](#) and [Equation \(18.9\)](#)–[Equation \(18.13\)](#), started from $(\hat{\mathbf{x}}_0, \mathbf{P}_0)$. Consequently $\hat{\mathbf{x}}_k$ is the **minimum mean-square-error (MMSE) estimator** of $\mathbf{x}_k$: for every function g of the measurements, $\mathbb{E}[\|\mathbf{x}_k - g(\mathbf{z}_{1:k})\|^2] \geq \mathbb{E}[\|\mathbf{x}_k - \hat{\mathbf{x}}_k\|^2] = \text{tr } \mathbf{P}_k$, and $\mathbf{P}_k = \mathbb{E}[(\mathbf{x}_k - \hat{\mathbf{x}}_k)(\mathbf{x}_k - \hat{\mathbf{x}}_k)^\top]$ is the covariance of the estimation error.*

Proof. Induction on k . The claim holds for $k = 0$ by the choice of the initial belief. Assume $\mathbf{x}_{k-1} | \mathbf{z}_{1:k-1} \sim \mathcal{N}(\hat{\mathbf{x}}_{k-1}, \mathbf{P}_{k-1})$. The noise $\mathbf{w}_k$ is independent of $\mathbf{x}_{k-1}$ and of $\mathbf{z}_{1:k-1}$, so fact (i) of [Lemma 18.3](#) gives $\mathbf{x}_k | \mathbf{z}_{1:k-1} \sim \mathcal{N}(\hat{\mathbf{x}}_k^-, \mathbf{P}_k^-)$ with [Equation \(18.6\)](#). The noise $\mathbf{v}_k$ is independent of everything before it, so, conditionally on $\mathbf{z}_{1:k-1}$, the pair $(\mathbf{x}_k, \mathbf{z}_k)$ is jointly Gaussian with the parameters [Equation \(18.8\)](#), and fact (ii) gives $\mathbf{x}_k | \mathbf{z}_{1:k} \sim \mathcal{N}(\hat{\mathbf{x}}_k, \mathbf{P}_k)$ with [Equation \(18.12\)](#)–[Equation \(18.13\)](#); conditioning on $\mathbf{z}_{1:k-1}$ and then on $\mathbf{z}_k$ is conditioning on $\mathbf{z}_{1:k}$. This proves the first claim, and it identifies $\hat{\mathbf{x}}_k$ with the conditional mean $\mathbb{E}[\mathbf{x}_k | \mathbf{z}_{1:k}]$ and $\mathbf{P}_k$ with the conditional covariance, which is the covariance of the error $\mathbf{x}_k - \hat{\mathbf{x}}_k$ because the error has conditional mean zero.

For the MMSE property, let g be any estimator and write $\mathbf{x}_k - g = (\mathbf{x}_k - \hat{\mathbf{x}}_k) + (\hat{\mathbf{x}}_k - g)$. The second term is a function of $\mathbf{z}_{1:k}$, and the first has conditional mean zero given $\mathbf{z}_{1:k}$, so the conditional expectation of their inner product vanishes, and $\mathbb{E} \|\mathbf{x}_k - g\|^2 = \mathbb{E} \|\mathbf{x}_k - \hat{\mathbf{x}}_k\|^2 +$

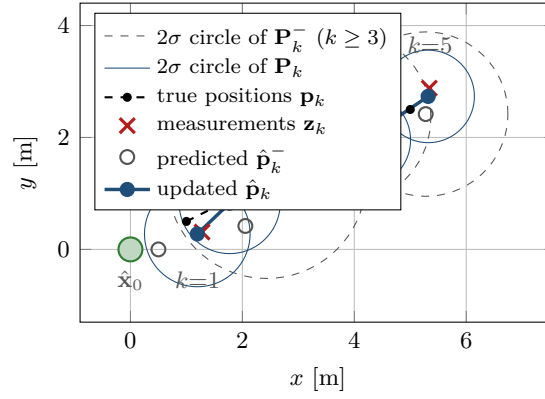


Figure 18.4. The five steps of [Example 18.7](#) in the plane. The predicted positions (gray circles) continue the previous velocity estimate; each measurement (red cross) pulls the estimate (blue) most of the way toward itself at first ($k_p = 0.89$) and less later ($k_p = 0.68$). The 2σ circles of the prediction (dashed, drawn for $k \geq 3$; those of steps 1 and 2 exceed the frame) shrink after every update (solid), to less than the sensor's 2σ radius of 1 m from the first step on.

$\mathbb{E} \|\hat{\mathbf{x}}_k - g\|^2 \geq \mathbb{E} \|\mathbf{x}_k - \hat{\mathbf{x}}_k\|^2$, with equality only for $g = \hat{\mathbf{x}}_k$ almost surely. $\square$

The optimality is over *all* estimators, linear or not, because the posterior is exactly Gaussian and the conditional mean is the MMSE estimator for any distribution. When the noises are not Gaussian but still have zero mean, the stated covariances and the stated independence, the posterior is no longer Gaussian and a nonlinear estimator may do better — but no *linear* one can.

Proposition 18.9 (The Kalman gain is the best gain). *Let $\hat{\mathbf{x}}^-$ be an unbiased prediction with error covariance $\mathbf{P}^-$, let $\mathbf{z} = \mathbf{H}\mathbf{x} + \mathbf{v}$ with $\mathbf{v} \sim (\mathbf{0}, \mathbf{R})$ independent of the prediction error, and consider the estimators $\hat{\mathbf{x}}(\mathbf{K}) = \hat{\mathbf{x}}^- + \mathbf{K}(\mathbf{z} - \mathbf{H}\hat{\mathbf{x}}^-)$ for an arbitrary $n \times m$ matrix $\mathbf{K}$. Their error covariance is*

$$\mathbf{P}(\mathbf{K}) = \underbrace{\mathbf{P}^- - \mathbf{P}^- \mathbf{H}^T \mathbf{S}^{-1} \mathbf{H} \mathbf{P}^-}_{\mathbf{P}(\mathbf{K}^*)} + (\mathbf{K} - \mathbf{K}^*) \mathbf{S} (\mathbf{K} - \mathbf{K}^*)^T, \quad \mathbf{K}^* = \mathbf{P}^- \mathbf{H}^T \mathbf{S}^{-1}, \quad (18.19)$$

with $\mathbf{S} = \mathbf{H} \mathbf{P}^- \mathbf{H}^T + \mathbf{R}$. Hence $\mathbf{P}(\mathbf{K}) - \mathbf{P}(\mathbf{K}^*)$ is positive semidefinite for every $\mathbf{K}$, and the Kalman gain minimizes every diagonal entry of the error covariance, in particular its trace, the mean-square error. No Gaussian assumption is needed: the recursion is the **best linear unbiased estimator** for any noise with the given second moments. The statement extends from one update to the whole recursion by induction over k , since the prediction $\hat{\mathbf{x}}_{k+1}^- = \mathbf{F} \hat{\mathbf{x}}_k$ is a linear unbiased function of the previous estimate and [Equation \(18.6\)](#) propagates its error covariance exactly.

Proof. The error is $\mathbf{x} - \hat{\mathbf{x}}(\mathbf{K}) = (\mathbf{I} - \mathbf{K}\mathbf{H})(\mathbf{x} - \hat{\mathbf{x}}^-) - \mathbf{K}\mathbf{v}$, a sum of two independent zero-mean terms, so its covariance is the Joseph form $\mathbf{P}(\mathbf{K}) = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^-(\mathbf{I} - \mathbf{K}\mathbf{H})^T + \mathbf{K}\mathbf{R}\mathbf{K}^T$, which expands to $\mathbf{P}^- - \mathbf{K}\mathbf{H}\mathbf{P}^- - \mathbf{P}^- \mathbf{H}^T \mathbf{K}^T + \mathbf{K}\mathbf{S}\mathbf{K}^T$. Expanding $(\mathbf{K} - \mathbf{K}^*)\mathbf{S}(\mathbf{K} - \mathbf{K}^*)^T$ with $\mathbf{K}^* \mathbf{S} = \mathbf{P}^- \mathbf{H}^T$ gives $\mathbf{K}\mathbf{S}\mathbf{K}^T - \mathbf{K}\mathbf{H}\mathbf{P}^- - \mathbf{P}^- \mathbf{H}^T \mathbf{K}^T + \mathbf{P}^- \mathbf{H}^T \mathbf{S}^{-1} \mathbf{H} \mathbf{P}^-$, and subtracting the two expressions leaves exactly $\mathbf{P}^- - \mathbf{P}^- \mathbf{H}^T \mathbf{S}^{-1} \mathbf{H} \mathbf{P}^-$, which is [Equation \(18.19\)](#). The last term of [Equation \(18.19\)](#) is positive semidefinite because $\mathbf{S}$ is, and it vanishes only for $\mathbf{K} = \mathbf{K}^*$ since $\mathbf{S} \succ \mathbf{0}$ when $\mathbf{R} \succ \mathbf{0}$. $\square$

This is the argument of Kalman's original paper, which derives the filter by orthogonal projection and never mentions Gaussians [76]; the self-test verifies [Equation \(18.19\)](#) for a gain 50% too large.

Proposition 18.10 (Data-independent covariance and steady state). *The sequences $\mathbf{P}_k^-$, $\mathbf{S}_k$, $\mathbf{K}_k$ and $\mathbf{P}_k$ are determined by $\mathbf{F}$, $\mathbf{H}$, $\mathbf{Q}$, $\mathbf{R}$ and $\mathbf{P}_0$ alone and can be computed before any measurement arrives. If the model is time-invariant, $(\mathbf{F}, \mathbf{H})$ is observable, $\mathbf{Q} \succ \mathbf{0}$ and $\mathbf{R} \succ \mathbf{0}$, then $\mathbf{P}_k^-$ converges for every $\mathbf{P}_0 \succeq \mathbf{0}$ to the unique positive definite solution $\mathbf{P}_\infty^-$ of the *discrete algebraic Riccati equation**

$$\mathbf{P}_\infty^- = \mathbf{F}\mathbf{P}_\infty^-\mathbf{F}^\top + \mathbf{Q} - \mathbf{F}\mathbf{P}_\infty^-\mathbf{H}^\top(\mathbf{H}\mathbf{P}_\infty^-\mathbf{H}^\top + \mathbf{R})^{-1}\mathbf{H}\mathbf{P}_\infty^-\mathbf{F}^\top, \quad (18.20)$$

the gain converges to $\mathbf{K}_\infty = \mathbf{P}_\infty^-\mathbf{H}^\top\mathbf{S}_\infty^{-1}$, and the filter with this constant gain is stable.

Proof sketch. The measurement enters Equation (18.6) and Equation (18.10)–Equation (18.13) only through the mean. Substituting the update into the predict step gives the Riccati recursion whose fixed points solve Equation (18.20); observability bounds $\mathbf{P}_k^-$ from above, $\mathbf{Q} \succ \mathbf{0}$ from below, and a monotonicity argument gives convergence and uniqueness [7, ch. 5]. The conditions weaken to detectability of $(\mathbf{F}, \mathbf{H})$ and stabilizability of $(\mathbf{F}, \mathbf{Q}^{1/2})$. $\square$

Observability. The pair $(\mathbf{F}, \mathbf{H})$ is **observable** if the observability matrix

$$\mathcal{O} = \begin{pmatrix} \mathbf{H} \\ \mathbf{HF} \\ \vdots \\ \mathbf{HF}^{n-1} \end{pmatrix}$$

has rank n : n consecutive noise-free measurements then determine the state. For the CV model along one axis with a position sensor, $\mathbf{H} = (1 \ 0)$ and $\mathbf{HF} = (1 \ \Delta t)$: rank 2, observable, and the filter can recover the velocity from positions. With a velocity-only sensor, $\mathbf{H} = (0 \ 1)$ and $\mathbf{HF} = (0 \ 1)$: rank 1. The velocity is then estimated well, but the position is a velocity integrated from an unknown start, and its variance grows without bound (the self-test runs this case for 300 steps). A filter on an unobservable model does not fail loudly: it runs, reports a mean, and the only warning is a covariance that never stops growing. Check the rank once, when the model is built; `observability_rank` does it.

Complexity and memory. The filter is recursive: memory $\mathcal{O}(n^2)$ and time $\mathcal{O}(n^3)$ per step, independent of k (Section 18.4.4); it is nevertheless the batch least-squares fit to all measurements so far, reorganized so that each measurement is absorbed once and forgotten. The equivalence is easiest to see in the information form of Section 18.10, where each measurement adds $\mathbf{H}^\top\mathbf{R}^{-1}\mathbf{H}$ to the accumulated normal equations; see Särkkä [144, ch. 4] for the proof.

18.7 Tuning, diagnostics, gating, and imperfect sensors

The theory assumes that $\mathbf{Q}$ and $\mathbf{R}$ are the true noise covariances. In practice $\mathbf{R}$ is roughly known and $\mathbf{Q}$ is a modeling choice, sensors drop reports and occasionally return garbage, and there may be several intruders. Algorithm 18.2 already contains the corresponding logic; this section explains it.

18.7.1 What the two noise covariances do

$\mathbf{R}$ is the easier of the two: a datasheet gives the sensor’s error standard deviation, and a static calibration — a few hundred reports on a target at a known position — gives the sample covariance directly, with a check that the errors are unbiased and uncorrelated in time. An $\mathbf{R}$ wrong by a factor of two costs a little accuracy, not the unbiasedness, and the diagnostics

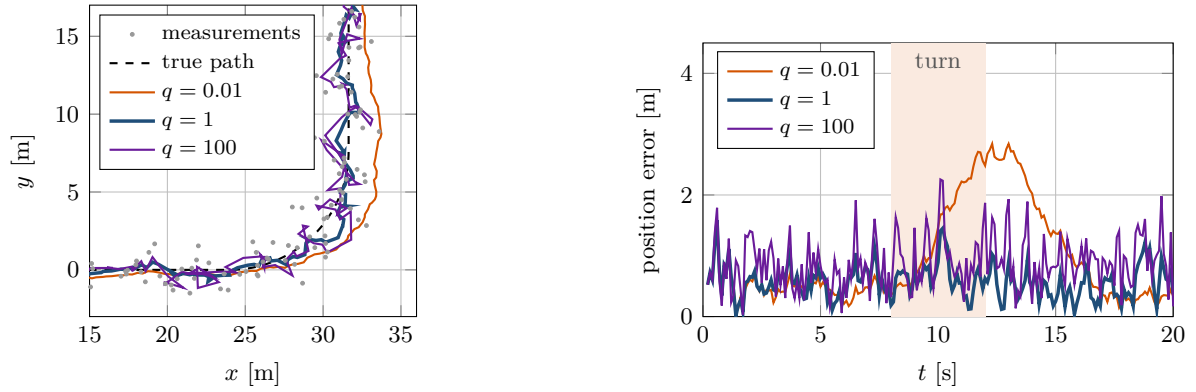


Figure 18.5. Three process-noise intensities on the same data. Left: the drone turns by 90° between $t = 8$ and 12 s. The filter with $q = 0.01$ (orange) cuts the corner and needs seconds to rejoin the path; $q = 100$ (purple) follows every measurement, noise included; $q = 1$ (blue) does neither. Right: position error against time. The small- q filter has the smallest error on the straight legs and by far the largest in the turn (shaded); the large- q filter is uniformly mediocre.

Table 18.2. The three filters of Figure 18.5 (experiment C of `gen_ch18_tracking.py`; $N = 199$ steps; raw RMSE 1.50 m); ρ_x, ρ_y are the lag-one autocorrelations of the innovations. With the whiteness bound $1.96/\sqrt{N} = 0.14$ and the 95% NIS band $[1.72, 2.28]$: $q = 0.01$ fails both tests, $q = 100$ fails whiteness with negative correlation, $q = 1$ passes both.

q [m^2/s^3]	RMSE [m]	in the turn	straight legs	mean NIS	ρ_x	ρ_y
0.01	1.24	1.89	0.92	3.60	0.25	0.26
1	0.63	0.70	0.60	2.11	-0.12	-0.04
100	0.98	1.13	0.93	1.85	-0.24	-0.18

below reveal it. $\mathbf{Q}$ is a statement about the intruder, which comes with no datasheet. The intensity q says how fast the velocity wanders: after a time T without measurements the velocity uncertainty grows by $\sqrt{qT}$, whence the rule of thumb $q \approx a_{\max}^2 T_a$ of Section 18.3.4. Whatever the starting value, its effect is a trade-off between two failure modes, shown in Figure 18.5 and Table 18.2:

- **$\mathbf{Q}$ too small: sluggish.** The filter believes its own model, the gain becomes small, and the estimate follows the true motion with a lag: straight flight is tracked beautifully, but in a turn the estimate overshoots the corner and takes seconds to catch up. The innovations are large and, tellingly, *correlated in time*.
- **$\mathbf{Q}$ too large: noisy.** The filter believes the sensor, the gain stays large, and the estimate copies the measurement noise; the velocity estimate in particular becomes almost as bad as a finite difference. The innovations are small and *negatively* correlated: each measurement overcorrects, and the next one corrects back.

In Figure 18.5 the small value pays for its smoothness in the turn, the large value pays everywhere: a factor of 100 above the best q costs 56% in position error and turns the velocity estimate into noise, a factor of 100 below doubles the error and misses the turn.

18.7.2 The innovation as a diagnostic

You cannot compare the estimate with the truth, because the truth is what you are estimating. You *can* compare the innovations with what the filter predicted for them: under Definition 18.1, with the correct $\mathbf{Q}$ and $\mathbf{R}$, the innovation sequence is zero-mean, Gaussian with covariance $\mathbf{S}_k$,

Table 18.3. Gate thresholds γ for $\mathbb{P}(\chi_m^2 \leq \gamma) = P_G$ (m = dimension of the measurement). A gate at $P_G = 0.99$ rejects one genuine measurement in a hundred and every measurement whose Mahalanobis distance exceeds $\sqrt{\gamma}$, about three standard deviations for $m = 2$.

P_G	$m = 1$	$m = 2$	$m = 3$	$m = 4$	$m = 6$
0.95	3.84	5.99	7.81	9.49	12.59
0.99	6.63	9.21	11.34	13.28	16.81
0.999	10.83	13.82	16.27	18.47	22.46

and *white* — $\mathbf{y}_k$ is uncorrelated with $\mathbf{y}_j$ for $j \neq k$ — because each innovation is the part of the measurement that the past could not predict. A filter whose innovations pass these tests is called **consistent**; the tests are, for a run of N steps:

1. **Normalized innovation squared (NIS)**. $d_k^2 = \mathbf{y}_k^\top \mathbf{S}_k^{-1} \mathbf{y}_k$ is chi-square with m degrees of freedom, mean m and variance $2m$, so the time average $\bar{d}^2 = \frac{1}{N} \sum_k d_k^2$ should lie in

$$m \pm 1.96\sqrt{2m/N} \quad (18.21)$$

with probability 95% (normal approximation of χ_{Nm}^2/N). A mean above the band says the innovations are larger than the filter expects, so **Q** or **R** is too small — usually **Q**, since **R** is measured; a mean below the band says the opposite.

2. **Whiteness**. The lag-one autocorrelation $\rho = \sum_k y_k y_{k+1} / \sum_k y_k^2$ of each innovation component should not exceed $1.96/\sqrt{N}$ in absolute value. Positive ρ is the signature of a sluggish filter, negative ρ of a nervous one, and the test is more sensitive to a wrong **Q** than the NIS: in Table 18.2 the $q = 100$ filter passes the NIS test and fails whiteness by a wide margin.
3. **Normalized estimation error squared (NEES)**, in simulation only: $(\mathbf{x}_k - \hat{\mathbf{x}}_k)^\top \mathbf{P}_k^{-1} (\mathbf{x}_k - \hat{\mathbf{x}}_k)$ has mean n if the reported covariance is honest. The self-test checks it over 300 Monte-Carlo runs.

To tune q : start from the rule of thumb, run the filter on recorded data, and move q up by factors of two or three while the mean NIS is high or ρ positive, down while ρ is negative, until both tests pass. A sensor whose errors are correlated in time (a drifting camera calibration, say) fails the whiteness test for every q ; the remedy is a state augmented with the drift (Section 18.10).

18.7.3 Gating with the Mahalanobis distance

Real sensors occasionally return a measurement that has nothing to do with the intruder: a reflection, a bird, a corrupted packet. Fed to Algorithm 18.1, an outlier 50 m from the predicted position moves the estimate by **K** times 50 m in one step, the velocity estimate with it, and the local safety layer reacts to a phantom. The defense is the same statistic as the first diagnostic, the squared **Mahalanobis distance** $d^2 = \mathbf{y}^\top \mathbf{S}^{-1} \mathbf{y}$ of the measurement from its prediction, which under the model is χ_m^2 -distributed.

Definition 18.11 (Validation gate). For a gate probability P_G let γ be the P_G -quantile of the χ_m^2 distribution, $\mathbb{P}(\chi_m^2 \leq \gamma) = P_G$. The **validation gate** of a track is the ellipsoid $\{\mathbf{z} : (\mathbf{z} - \mathbf{H}\hat{\mathbf{x}}^-)^\top \mathbf{S}^{-1} (\mathbf{z} - \mathbf{H}\hat{\mathbf{x}}^-) \leq \gamma\}$; a measurement outside it is **gated out** and treated as missing.

The price of a gate is the fraction $1 - P_G$ of genuine measurements it rejects, the benefit is immunity to anything beyond $\sqrt{\gamma}$ standard deviations; $P_G = 0.99$ is a common compromise.

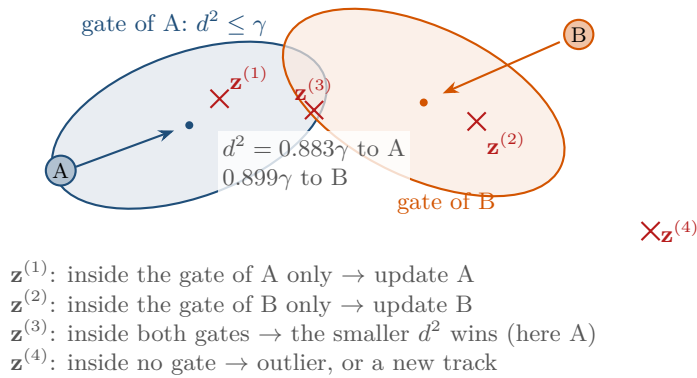


Figure 18.6. Gating and data association with two tracked intruders. Each track predicts a position (dot) and a validation gate (ellipse of $\mathbf{S}$, scaled by $\sqrt{\gamma}$); the gates are elongated along the directions of motion, where the prediction is least certain. Four measurements arrive: two fall in exactly one gate, one in both, one in none.

In the self-test a 50-meter outlier is rejected with $d^2 \gg 9.21$ while the ungated filter jumps by meters.

Resetting the filter on every outlier

A tempting reaction to a gated-out measurement is to reinitialize the track from it: “the filter has lost the target”. It has not. A single outlier is exactly what the gate is for; skip the update and let the covariance grow a little. Resetting throws away the velocity estimate and the covariance, both of which took seconds to build, and replaces them with a track started on the outlier itself. Drop a track only after *several consecutive* rejections, when the gate has grown so wide that the track no longer says anything (line 16 of Algorithm 18.2), and report the count of rejections to the local safety layer as a health signal.

Several intruders: data association. With several tracks and several measurements per scan, the gate also answers *which measurement belongs to which track*, the **data association** problem of Figure 18.6. **Nearest-neighbor association** computes d^2 for every (track, measurement) pair, discards the pairs outside the gates, and assigns the rest greedily in increasing order of d^2 , each track and each measurement at most once (`nearest_neighbor_association` in the code). Leftover measurements are outliers or new intruders (a track is started when two consecutive leftovers are consistent with a plausible speed); leftover tracks take a predict-only step. The rule fails when two gates overlap and both measurements fall in the overlap — two intruders crossing — where a wrong assignment swaps the tracks; the probabilistic data association filter and multiple-hypothesis tracking are the standard answers [7].

18.7.4 Missing measurements

A sensor that drops reports — an occluded camera, a lost packet, an empty radar scan — needs no special rule: the predict step runs at every sampling instant and the update only when a measurement is available (line 8 of Algorithm 18.2). The estimate after a miss is the prediction, whose position variance has grown by $2\Delta t b + \Delta t^2 c + q\Delta t^3/3$ per axis, and nothing is lost by this bookkeeping: the larger covariance is the honest description of what the filter knows, the local safety layer receives a larger ellipse to keep clear of, the gate widens so that the next genuine measurement is accepted, and the next update shrinks the covariance back (the self-test checks all three, and a sensor that drops 30% of its reports still beats the raw

measurements). Only a long run of misses is a problem, the same as a long run of rejections: at some point the track is a guess, and it is better to drop it and wait for two fresh measurements.

18.7.5 Track initialization from two measurements

The filter needs a starting belief. One position measurement fixes the position but not the velocity; two measurements $\mathbf{z}_a, \mathbf{z}_b$ received Δt apart fix both, the velocity as the difference quotient, and the covariance of this linear combination follows from $\text{Cov}(\mathbf{z}_a) = \text{Cov}(\mathbf{z}_b) = \mathbf{R}$ with independent errors,

$$\hat{\mathbf{x}}_0 = \begin{pmatrix} \mathbf{z}_b \\ (\mathbf{z}_b - \mathbf{z}_a)/\Delta t \end{pmatrix}, \quad \mathbf{P}_0 = \begin{pmatrix} \mathbf{R} & \mathbf{R}/\Delta t \\ \mathbf{R}/\Delta t & 2\mathbf{R}/\Delta t^2 \end{pmatrix}. \quad (18.22)$$

The velocity block is the finite-difference variance of [Section 18.1](#), $(14 \text{ m/s})^2$ for $\sigma_p = 1 \text{ m}$ at 10 Hz, and the off-diagonal block records that the position and velocity errors are correlated, since both contain the error of $\mathbf{z}_b$ — the correlation that lets the next measurement correct the velocity as well as the position. Two-point initialization is exact for the CV model, needs no tuning, and its huge velocity variance is honest: the right panel of [Figure 18.1](#) shows the velocity estimate starting from a wild value and converging within about two seconds, the time the filter needs to see a few meters of travel. `init_two_point` implements [Equation \(18.22\)](#).

18.8 Predicting ahead: uncertainty propagation

The local safety layer of [Chapter 24](#) asks where the intruder *will be* during the next few seconds and how sure the tracker is. Both questions have the same answer, the predict step run j times without an update, and since mean and covariance evolve independently of the measurements the result is closed form:

$$\hat{\mathbf{x}}_{k+j|k} = \mathbf{F}^j \hat{\mathbf{x}}_k, \quad \mathbf{P}_{k+j|k} = \mathbf{F}^j \mathbf{P}_k (\mathbf{F}^j)^\top + \sum_{i=0}^{j-1} \mathbf{F}^i \mathbf{Q} (\mathbf{F}^i)^\top. \quad (18.23)$$

The notation $\hat{\mathbf{x}}_{k+j|k}$ reads “the estimate of the state at step $k+j$ given the measurements up to step k ”; $\hat{\mathbf{x}}_k^-$ is the case $j=1$ of the previous step. For the CV model the sum collapses. The matrix $\mathbf{F}$ built for a step Δt , call it $\mathbf{F}(\Delta t)$, satisfies $\mathbf{F}(\Delta t)^j = \mathbf{F}(j\Delta t)$, and the accumulated process noise satisfies $\sum_{i=0}^{j-1} \mathbf{F}(\Delta t)^i \mathbf{Q} (\Delta t) \mathbf{F}(\Delta t)^{i\top} = \mathbf{Q}(j\Delta t)$ (the self-test checks this identity, which is the reason q is a rate and $\mathbf{Q}$ is not a free matrix). With the horizon $h = j\Delta t$,

$$\mathbf{P}_{k+j|k} = \mathbf{F}(h) \mathbf{P}_k \mathbf{F}(h)^\top + \mathbf{Q}(h), \quad \text{position variance per axis: } a(h) = a + 2hb + h^2c + \frac{q}{3}h^3, \quad (18.24)$$

where (a, b, c) are the entries of $\mathbf{P}_k$ for that axis. The three growing terms have three meanings: $2hb$ is the correlation between the current position and velocity errors; h^2c is the velocity uncertainty turned into position uncertainty, the dominant term for horizons of a second or two and the reason a good velocity estimate matters more than a good position estimate for avoidance; and $qh^3/3$ is the maneuver the intruder may make meanwhile, which dominates for long horizons and cannot be reduced by any sensor.

[Figure 18.7](#) shows the ellipses for the experiment of [Section 18.9](#), from the estimate at $t = 10 \text{ s}$: the position standard deviation grows from $(0.39, 0.23) \text{ m}$ now to $(0.84, 0.64) \text{ m}$ one second ahead and $(1.46, 1.21) \text{ m}$ at two, half the semi-axes drawn in the figure. These ellipses are what the local layer inflates the intruder’s radius with: if the intruder has radius r and the ellipse at horizon h has largest standard deviation $\sigma_{\max}(h)$ (square root of the largest eigenvalue of the position block of $\mathbf{P}_{k+j|k}$), keeping the distance $r + \kappa \sigma_{\max}(h)$ from the predicted position guarantees separation with the probability of the κ -sigma ellipse — and in

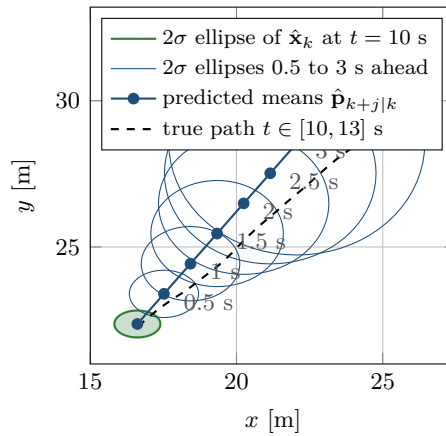


Figure 18.7. Predicting ahead from the estimate of the experiment of [Section 18.9](#) at $t = 10$ s. The 2σ ellipse of the estimate (green) has semi-axes 0.79×0.46 m; the predicted ellipses (blue) at 0.5, 1, ..., 3 s ahead grow to 4.4×3.8 m, roughly linearly in the horizon at first (velocity uncertainty) and faster later (possible maneuvers). The dashed line is the true path over the next three seconds; it stays inside the ellipses.

two dimensions the 2σ ellipse contains the true position with probability $1 - e^{-2} = 86.5\%$ only; 95% needs $\kappa = \sqrt{5.99} = 2.45$, by the same chi-square table as the gate. Since the inflation grows with the horizon, a local safety layer that plans over 3 s must keep clear of a disk several meters larger than the intruder: the concrete cost of sensor noise and unknown intentions, and the reason [Chapter 24](#) triggers avoidance on a short horizon and leaves long horizons to the replanning layer. [Chapter 20](#) uses the same propagated Gaussian as its constant-velocity baseline, and [Chapter 13](#) describes the radius inflation on the ORCA side.

Confusing the estimation covariance with the prediction covariance

$\mathbf{P}_k$ says how well the tracker knows where the intruder is *now*; while measurements keep arriving it converges to a small steady value and never grows, however long you track. $\mathbf{P}_{k+j|k}$ says how well anyone can know where the intruder will be j steps from now; it grows without bound in j , because it contains the intruder's future decisions. A planner that inflates the radius by the current 2σ of 0.8 m when the collision is two seconds away and the honest figure is 2.9 m is over-confident by a factor that grows with the horizon, and the collision rate in the experiments of [Chapter 25](#) shows it. The converse mistake, gating a measurement with $\mathbf{P}_{k+j|k}$ instead of $\mathbf{S}_k$, accepts every outlier.

18.9 An experiment on simulated data

The script `code/figures/gen_ch18_tracking.py` runs four experiments on the filter of `ch18_kalman.py`; every number quoted in this chapter comes from it. Experiment A, the one behind [Figure 18.1](#), [Figure 18.7](#) and the left panel of [Figure 18.8](#), simulates an intruder that starts at the origin with velocity (2, 1) m/s and follows the CV model with white-noise acceleration of intensity $q = 0.2 \text{ m}^2/\text{s}^3$ for 20 s. A sensor reports its position at 10 Hz with independent errors of standard deviation 1 m along x and 0.5 m along y . The filter is initialized from the first two reports by [Equation \(18.22\)](#) and uses the true q and $\mathbf{R}$.

The results are the ones quoted in [Section 18.1](#): the position RMSE drops by more than half and the velocity RMSE by a factor of twenty. The error bars are honest: the final standard deviations are 0.39 m along x , 0.23 m along y and 0.48 m/s in v_x , and the mean NIS over the run is 2.21, inside the 95% band $[1.72, 2.28]$ of [Equation \(18.21\)](#). The ellipses in the left panel

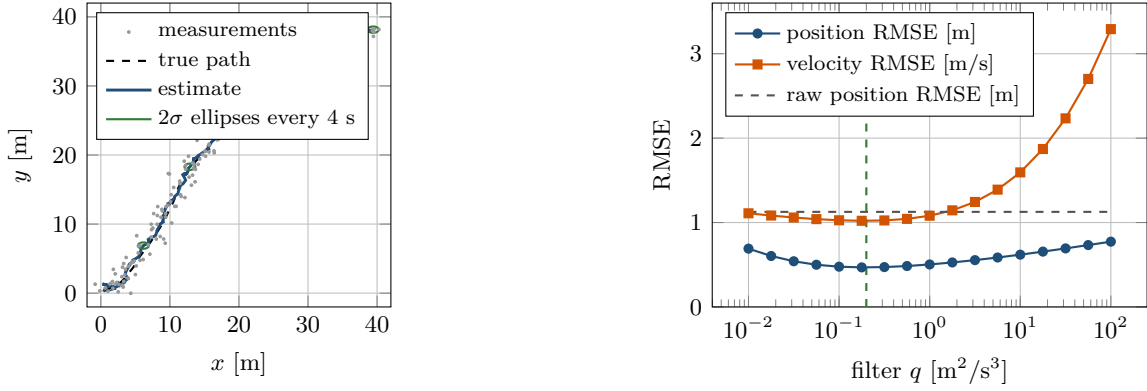


Figure 18.8. Left, experiment A: true path (dashed), the 198 measurements (gray), the Kalman estimate (blue) and its 2σ ellipses every 4 s (green), wider along x , the noisier axis. Right, experiment B: position (blue) and velocity (orange) RMSE against the q used by the filter, over 20 Monte-Carlo runs; dashed gray is the raw-measurement RMSE. The minimum lies at the true q , the curve is flat within a factor of two below and five above it, and the velocity suffers far more from a q that is too large than from one that is too small.

are elongated along x , the axis measured twice as noisily, and the true path threads through all of them.

Experiment B, the right panel, asks how much the tuning of q matters: the scenario is simulated 20 times with fresh noise and each data set is filtered with 17 values of q from 0.01 to 100, the true value being 0.2. The position RMSE has its minimum, 0.469 m, at $q = 0.178$, the grid point nearest the truth; it is 0.477 m at $q = 0.1$ and 0.503 m at $q = 1$, so a factor of two below or five above the optimum costs less than 8%, while a factor of five below already costs 15% (0.541 m at $q = 0.032$): under-sizing q is punished sooner than over-sizing it on this straight-flying target. Far away the price is severe, but the position RMSE stays below the raw 1.126 m over the whole grid. The velocity RMSE is asymmetric the other way, and far more sharply, because an over-large q lets the sensor noise straight into the velocity, while an under-sized q merely smooths a velocity that changes slowly in this scenario (on a turning target, Figure 18.5, the small- q side pays instead). The mean NIS falls monotonically from 2.74 at $q = 0.01$ to 1.58 at $q = 100$ and crosses $m = 2$ at $q \approx 0.3$, next to the truth: choosing the q at which the mean NIS equals m , the rule of Section 18.7.2, lands within a factor of two of the optimum without ever seeing the true trajectory.

18.10 Variants and extensions

Constant gain. By Proposition 18.10 the gain converges; running the CV filter with the constant gain (k_p, k_v) per axis from the start gives the **alpha-beta filter**, $\hat{p} \leftarrow \hat{p}^- + \alpha y$, $\hat{v} \leftarrow \hat{v}^- + (\beta/\Delta t)y$ with $\alpha = k_p$, $\beta = k_v\Delta t$: no covariance, no matrix arithmetic, the same steady-state accuracy, but no transient phase of large gains, so it converges slowly from a poor start, and no covariance output for the safety layer.

The information filter. Carrying $\mathbf{P}^{-1}$ and $\mathbf{P}^{-1}\hat{\mathbf{x}}$ turns the update Equation (18.16) into two additions and makes the predict step the expensive one; it is the natural form when several sensors observe the same intruder, since their contributions $\mathbf{H}_i^T \mathbf{R}_i^{-1} \mathbf{H}_i$ add in any order and a silent sensor adds zero. With a diagonal $\mathbf{R}$ the gain form achieves the same by processing the components of $\mathbf{z}_k$ one at a time as scalar updates.

Smoothing. Offline, the measurements *after* step k are available too, and the Rauch–Tung–Striebel smoother [133] runs a backward pass over the stored filter outputs that roughly halves the error in the interior of a recorded trajectory: the right tool for cleaning the trajectories on which the predictors of Chapter 20 are trained, not a tracker, because it needs the future [144].

Richer states and maneuvering targets. Anything the noise model cannot absorb can be added to the state — the acceleration (the CA model), a slowly varying sensor bias, a time-correlated disturbance — at the price of a larger n , a slower response to what the extra states must first learn, and a fresh observability check. For an intruder that alternates between straight flight and turns, the interacting multiple model (IMM) estimator runs a CV and a CA or coordinated-turn filter in parallel and mixes them by their likelihoods [7].

Nonlinear models. A radar that reports range and bearing, a camera that reports pixels, a turn model with an unknown rate: each breaks the linearity of $\mathbf{H}$ or $\mathbf{F}$. The extended Kalman filter linearizes around the current estimate, the unscented filter propagates a handful of chosen points, and the particle filter abandons the Gaussian altogether; Chapter 19 presents all three and compares them on a turning target. Everything in this chapter survives with Jacobians in place of $\mathbf{F}$ and $\mathbf{H}$.

18.11 Implementation notes

Listing 18.1 is the heart of `ch18_kalman.py`. The rest of the file builds the CV and CA models, initializes tracks, simulates measurements with misses and outliers, runs the diagnostics of Section 18.7.2, associates measurements with several tracks and predicts ahead; its self-test checks the worked example to four decimals, the process-noise integrals, the identity Equation (18.19), gating, misses, observability and the consistency statistics on a 2000-step simulation, in about three seconds.

Listing 18.1. The predict and update steps of the `KalmanFilter` class (from `code/ch18_kalman.py`). `predict` accepts $\mathbf{F}$ and $\mathbf{Q}$ built for the actual time step; `update` returns the innovation, its covariance, the gain and the NIS, rejects the measurement when a gate probability is given and the NIS exceeds the chi-square threshold, and uses the Joseph form by default.

```

1  def predict(self, u=None, F=None, Q=None):
2      """Predict:  $\hat{x} = F x + B u$ ,  $\hat{P} = F P F^T + Q$ .
3      Pass  $F$  and  $Q$  built for the actual time step if  $dt$  varies."""
4      F = self.F if F is None else np.asarray(F, float)
5      Q = self.Q if Q is None else np.asarray(Q, float)
6      self.x = F @ self.x
7      if u is not None and self.B is not None:
8          self.x = self.x + self.B @ np.asarray(u, float)
9      self.P = symmetrize(F @ self.P @ F.T + Q)
10     return self.x.copy(), self.P.copy()
11
12     def update(self, z, R=None, gate_prob=None, joseph=True):
13         """Update with measurement  $z$ . With  $gate\_prob$ ,  $z$  is rejected (state
14         unchanged) when its NIS exceeds the chi-square threshold."""
15         R = self.R if R is None else np.asarray(R, float)
16         y, S = self.innovation(z, R)
17         d2 = float(y @ np.linalg.solve(S, y))
18         K = np.linalg.solve(S, self.H @ self.P).T      #  $K = P H^T S^{-1}$ 
19         if gate_prob is not None and d2 > chi2_threshold(self.m, gate_prob):
20             return UpdateResult(y, S, K, d2, False)

```



```

21     self.x = self.x + K @ y
22     I_KH = np.eye(self.n) - K @ self.H
23     if joseph:      # valid for any K, keeps P symmetric positive definite
24         self.P = I_KH @ self.P @ I_KH.T + K @ R @ K.T
25     else:          # the short form, exact only for the optimal K
26         self.P = I_KH @ self.P
27     self.P = symmetrize(self.P)
28     return UpdateResult(y, S, K, d2, True)

```

Keep the covariance symmetric and positive definite. $\mathbf{P}$ is symmetric in exact arithmetic and slightly asymmetric after a few thousand floating-point steps, and the asymmetry feeds back into the gain and grows; symmetrizing after every step (line 11 of Algorithm 18.1, `symmetrize` in the listing) costs n^2 additions and removes the problem. Positive definiteness is protected by the Joseph form: the short form $(\mathbf{I} - \mathbf{KH})\mathbf{P}^-$, a difference of two matrices, can go indefinite when $\mathbf{P}^-$ has entries of very different sizes, such as a position variance of 10^4 m^2 after a long gap next to a velocity variance of $10^{-2} \text{ m}^2/\text{s}^2$. Solve $\mathbf{KS} = \mathbf{P}^- \mathbf{H}^\top$ rather than forming $\mathbf{S}^{-1}$. Square-root implementations that propagate a Cholesky factor of $\mathbf{P}$ [114] are the remedy for filters with hundreds of states or in single precision; for the six-state tracker of this chapter, symmetrization and the Joseph form suffice, and the self-test confirms that $\mathbf{P}$ stays symmetric positive definite over every step of a 2000-step run.

Units and scales. Keep the state in SI units, meters and meters per second, so that q is in m^2/s^3 and $\mathbf{R}$ in m^2 ; a mixed state (meters and degrees, say) is legal but makes $\mathbf{Q}$ and $\mathbf{P}$ unreadable and their tuning guesswork. The gate threshold and the NIS band are unit-free, which is the point of normalizing by $\mathbf{S}$.

Time-varying sampling. Sensors do not deliver at exactly 10 Hz. Timestamp every report, and build $\mathbf{F}(\Delta t)$ and $\mathbf{Q}(\Delta t)$ from the *actual* interval since the last report before each predict step (`predict(F=..., Q=...)` in the listing, `cv_model` to build them); a nominal Δt used with reports that arrive early or late puts a bias into the velocity estimate that no tuning removes. When the sensor is silent, predict at the planner’s cycle time with the same rule.

A process noise that does not scale with the step

Setting $\mathbf{Q} = q\mathbf{I}$ or $\mathbf{Q} = \text{diag}(q_p, q_v)$ with fixed numbers, independent of Δt , is the most common Kalman bug in student code. It works at the rate at which it was tuned and breaks when the rate changes: at twice the rate the filter injects twice as much noise per second and becomes nervous, at half the rate it becomes sluggish, and with jittering timestamps it is inconsistent from step to step. The white-noise-acceleration $\mathbf{Q}$ of Equation (18.5) scales correctly because it is the integral of a continuous-time noise over the step, and q is a property of the intruder, not of the sensor rate; a diagonal shortcut should at least scale the position term with Δt^3 and the velocity term with Δt .

Feeding finite differences to the filter as velocity measurements

When the sensor reports positions only, it is tempting to compute $(\mathbf{z}_k - \mathbf{z}_{k-1})/\Delta t$ and hand it to the filter as a velocity measurement with $\mathbf{H} = \mathbf{I}$. Do not. The differenced “measurement” is a function of two position reports the filter is also using, so its error is correlated with the position measurements of the same and the previous step, which violates the independence assumption of Definition 18.1: the filter counts the same sensor

error twice, reports a covariance that is too small, and fails the NIS test. The filter already extracts the velocity from the positions, optimally, through the gain k_v of Equation (18.17). Use a velocity measurement only when a sensor measures the velocity independently — a Doppler radar, or a broadcast containing the intruder’s own velocity estimate — with its own σ_v in $\mathbf{R}$, as Exercise 18.8 does.

18.12 Where this fits in the drone system

In the drone system

The Kalman filter is the **prediction layer** of the hybrid architecture of Chapter 24, the only layer that deals with agents the swarm does not control. It *receives* timestamped sensor reports about unknown drones (positions, or positions and velocities) with their noise levels, and the safety horizon h from the decision logic. It *maintains* one track per intruder — a CV filter with gating and two-point initialization (Algorithm 18.2), nearest-neighbor association when several intruders are visible, and a health count of consecutive misses — and *delivers*, at every cycle, $(\hat{\mathbf{x}}_k, \mathbf{P}_k)$ and the predictions $(\hat{\mathbf{x}}_{k+j|k}, \mathbf{P}_{k+j|k})$ up to the horizon (Equation (18.23)). The local safety layer (ORCA in Chapter 13, the dynamic window approach, DWA, in Chapter 14) takes the predicted position and velocity as the obstacle and inflates its radius by $\kappa \sigma_{\max}(h)$; the time-to-collision trigger of Chapter 24 is evaluated on the same prediction; the constant-velocity predictor of Chapter 20 is Equation (18.23) itself, and the learned predictors of that chapter take the filtered history as input; the replanning layer (Chapters 5 and 24) receives the predicted occupancy when a detour must be planned; and the health count tells the decision logic when an intruder is no longer tracked. This is the Week 9 topic of the study plan (Appendix A), whose milestone is exactly this interface: a stable estimated state with a covariance instead of raw noisy observations.

18.13 Summary

Chapter summary

- The linear-Gaussian model $\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{B}\mathbf{u}_k + \mathbf{w}_k$, $\mathbf{z}_k = \mathbf{H}\mathbf{x}_k + \mathbf{v}_k$, $\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$, $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$, describes a drone whose accelerations are unknown and a sensor whose errors are independent.
- The filter keeps $\mathcal{N}(\hat{\mathbf{x}}_k, \mathbf{P}_k)$ and alternates predict, $\hat{\mathbf{x}}^- = \mathbf{F}\hat{\mathbf{x}}$, $\mathbf{P}^- = \mathbf{F}\mathbf{P}\mathbf{F}^\top + \mathbf{Q}$, and update, $\mathbf{y} = \mathbf{z} - \mathbf{H}\hat{\mathbf{x}}^-$, $\mathbf{S} = \mathbf{H}\mathbf{P}^-\mathbf{H}^\top + \mathbf{R}$, $\mathbf{K} = \mathbf{P}^-\mathbf{H}^\top\mathbf{S}^{-1}$, $\hat{\mathbf{x}} = \hat{\mathbf{x}}^- + \mathbf{K}\mathbf{y}$, $\mathbf{P} = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^-$ (implemented in the Joseph form); both steps are the affine and conditioning properties of Gaussians.
- The gain is a precision weighting, $K = P^-/(P^- + R)$ and $1/P = 1/P^- + 1/R$ in one dimension; through the position–velocity covariance a position sensor also corrects the velocity.
- The constant-velocity model has $\mathbf{F} = \begin{pmatrix} \mathbf{I} & \Delta t \mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}$ and $\mathbf{Q} = q \begin{pmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{pmatrix}$ per axis; q is a rate, and $\mathbf{Q}$ must be rebuilt when Δt changes.
- For the linear-Gaussian model the filter is the exact posterior and the MMSE estimator, and the best linear estimator otherwise; the covariance sequence is data-independent and converges when the model is observable.

- Tune q with the innovations (mean NIS in $m \pm 1.96\sqrt{2m/N}$, white innovations); gate with the Mahalanobis distance and a chi-square threshold; treat gated-out and missing measurements alike as predict-only steps; initialize from two measurements; drop a track only after several consecutive misses.
- Predicting j steps ahead gives $\mathbf{F}^j \hat{\mathbf{x}}_k$ and a covariance that grows without bound; its ellipses, not $\mathbf{P}_k$, are what the local safety layer inflates the intruder's radius with.

Further reading. The filter is Kalman's [76], derived by orthogonal projection without a Gaussian assumption. The tutorial of Welch and Bishop [175] is the shortest complete introduction and the source of the $\mathbf{A}$, $\mathbf{H}$ notation found in much code; Thrun, Burgard, and Fox [168, ch. 3] derive the filter as a Bayes filter with Gaussian beliefs, the route taken in Section 18.4, and continue with the nonlinear filters of Chapter 19. Bar-Shalom, Li, and Kirubarajan [7] is the reference for tracking: kinematic models and their process noise, consistency tests, gating, data association and maneuvering-target estimators. Särkkä [144] gives the modern Bayesian treatment with smoothers, Maybeck [114] the engineering view including the numerical forms, and Rauch, Tung, and Striebel [133] the smoother.

18.14 Exercises

Exercise 18.1 (★). Starting from the update row of step 1 in Table 18.1, $\hat{\mathbf{x}}_1 = (1.195, 0.276, 0.859, 0.143)$ and $(a, b, c) = (0.2226, 0.1150, 0.6172)$, compute step 2 by hand with the recursion Equation (18.18): the predicted mean and (a^-, b^-, c^-) , then S , k_p , k_v , the innovation for $\mathbf{z}_2 = (1.71, 0.92)$, the updated mean and (a, b, c) . Compare with the table. Then compute the predicted mean of step 3 and explain why its x -component is smaller than the true x -position 3.

Exercise 18.2 (★). Two independent unbiased readings z_1 and z_2 of the same quantity have variances σ_1^2 and σ_2^2 . (a) Among the estimates $\lambda z_1 + (1 - \lambda)z_2$, find the λ that minimizes the variance, and show that the minimum variance σ^2 satisfies $1/\sigma^2 = 1/\sigma_1^2 + 1/\sigma_2^2$. (b) Show that this is exactly the one-dimensional Kalman update Equation (18.15) with z_1 as the prediction and z_2 as the measurement, and identify K . (c) Evaluate for $z_1 = 2$, $\sigma_1 = 1$, $z_2 = 3$, $\sigma_2 = 0.5$ and compare with Example 18.6.

Exercise 18.3 (★★). Derive the Kalman gain without Gaussians. Let $\hat{\mathbf{x}}(\mathbf{K}) = \hat{\mathbf{x}}^- + \mathbf{K}(\mathbf{z} - \mathbf{H}\hat{\mathbf{x}}^-)$ and let $\mathbf{P}(\mathbf{K})$ be its error covariance, the Joseph form. (a) Expand $\mathbf{P}(\mathbf{K})$ and differentiate $\text{tr } \mathbf{P}(\mathbf{K})$ with respect to $\mathbf{K}$ (use $\partial \text{tr}(\mathbf{K}\mathbf{A})/\partial \mathbf{K} = \mathbf{A}^\top$ and $\partial \text{tr}(\mathbf{K}\mathbf{S}\mathbf{K}^\top)/\partial \mathbf{K} = 2\mathbf{K}\mathbf{S}$ for symmetric $\mathbf{S}$) to find the stationary point. (b) Show with Equation (18.19) that it is the global minimizer, not merely a stationary point, and that it minimizes every diagonal entry of $\mathbf{P}(\mathbf{K})$, not just the trace. (c) Show that at the optimum $\mathbf{P}(\mathbf{K}^*) = \mathbf{P}^- - \mathbf{K}^* \mathbf{S} \mathbf{K}^{*\top} = (\mathbf{I} - \mathbf{K}^* \mathbf{H}) \mathbf{P}^-$.

Exercise 18.4 (★★). (a) Repeat the derivation of Proposition 18.5 for the constant-acceleration model, where the jerk is white noise of intensity q : show that the disturbance of (p, v, a) over a step is $\int_0^{\Delta t} ((\Delta t - \tau)^2/2, \Delta t - \tau, 1)^\top j(\tau) d\tau$ and obtain the $\mathbf{Q}$ of Section 18.3.4. (b) Prove the semigroup property used in Section 18.8: for the CV model, $\mathbf{F}(\Delta t) \mathbf{Q}(\Delta t) \mathbf{F}(\Delta t)^\top + \mathbf{Q}(\Delta t) = \mathbf{Q}(2\Delta t)$, and by induction $\sum_{i < j} \mathbf{F}(\Delta t)^i \mathbf{Q}(\Delta t) \mathbf{F}(\Delta t)^{i\top} = \mathbf{Q}(j\Delta t)$. (c) Check both results numerically with `process_noise_numeric` and `cv_model`.

Exercise 18.5 (★★). (a) Derive $\mathbf{P}_0$ in Equation (18.22) from $\text{Cov}(\mathbf{z}_a) = \text{Cov}(\mathbf{z}_b) = \mathbf{R}$ and independence. (b) Show that two-point initialization is a Kalman filter in disguise: start from the single measurement $\mathbf{z}_a$ with the belief $\hat{\mathbf{p}} = \mathbf{z}_a$, $\hat{\mathbf{v}} = \mathbf{0}$, $\text{Cov}(\mathbf{p}) = \mathbf{R}$, $\text{Cov}(\mathbf{v}) = \mathbf{V}\mathbf{I}$, uncorrelated, predict one step with $q = 0$ and update with $\mathbf{z}_b$; show with the per-axis formulas Equation (18.7) and Equation (18.17) that as $V \rightarrow \infty$ the result converges to

Equation (18.22). (c) What is lost if instead the filter starts from $\mathbf{z}_b$ with $\hat{\mathbf{v}} = \mathbf{0}$ and $\text{Cov}(\mathbf{v}) = v_{\max}^2 \mathbf{I}$? Try both starts in the code on experiment A and compare the velocity error over the first two seconds.

Exercise 18.6 (★★). (a) Write the observability matrix of the CV model along one axis for a velocity-only sensor, $\mathbf{H} = (0 \ 1)$, and confirm its rank. (b) Do the same for the CA model with a position sensor, $\mathbf{H} = (1 \ 0 \ 0)$, and show that the rank is 3. (c) A 2D CV model is observed by a sensor that reports only the x -coordinate: which state components are observable? Verify with `observability_rank`, run the filter on a simulation and describe how $\mathbf{P}$ behaves.

Exercise 18.7 (★★★ Coding). Simulate two intruders whose straight paths cross at 30° at the same instant, observed by one sensor that returns both positions unlabeled, with $\sigma_p = 1$ m at 10 Hz, plus one spurious measurement per second at a random location. Track both with [Algorithm 18.2](#) and `nearest_neighbor_association` at $P_G = 0.99$. (a) Measure over 200 runs how often the tracks swap identities after the crossing. (b) Repeat for crossing angles of 90° and 10° and for $\sigma_p = 0.3$ m. (c) Propose and test one improvement, for instance forbidding an assignment whose implied velocity change exceeds $a_{\max} \Delta t$, and report what it costs when the intruders really do maneuver.

Exercise 18.8 (★★★ Coding). The coding exercise of Week 9. Simulate an external drone that flies a piecewise-constant-velocity path with a few turns, observed at 10 Hz by a sensor that reports its position *and* velocity with independent errors $\sigma_p = 1$ m and $\sigma_v = 0.5$ m/s ($\mathbf{H} = \mathbf{I}_4$, $\mathbf{R} = \text{diag}(\sigma_p^2, \sigma_p^2, \sigma_v^2, \sigma_v^2)$), dropping 10% of the reports and replacing 2% by outliers. Track it with the filter of this chapter. (a) Report the position and velocity RMSE of the filter, of the raw measurements and of a position-only filter; check the mean NIS ($m = 4$) and the whiteness of the innovations, and tune q until both pass. (b) Implement the interface of the prediction layer: a function that, at every cycle, returns $(\hat{\mathbf{x}}_k, \mathbf{P}_k)$, the predictions $(\hat{\mathbf{x}}_{k+j|k}, \mathbf{P}_{k+j|k})$ for $j\Delta t \leq 3$ s and a track-health flag. (c) Demonstrate the milestone: feed the raw reports and the filtered state, in turn, to a 3-second constant-velocity extrapolation (or to the velocity-obstacle construction of [Chapter 12](#)) and measure how much the predicted position 3 s ahead changes from one cycle to the next in each case.

Exercise 18.9 (★★★). Repeat experiment B, the RMSE-versus- q curve of [Figure 18.8](#), on the turning trajectory of experiment C with 20 noise realizations. (a) Where is the minimum now, and how does the curve differ? (b) Plot the mean NIS and the lag-one autocorrelation against q ; which q do the diagnostics of [Section 18.7.2](#) select, and is it the RMSE-optimal one? (c) Add the CA model with a white-noise jerk of intensity q_j and find the q_j that minimizes the RMSE on the same data. Which model wins in the turn, which on the straight legs, and what does that suggest for the IMM estimator of [Section 18.10](#)?

Nonlinear Filtering: EKF, UKF and Particle Filters

Learning objectives

- Explain why a turning intruder and a range-bearing sensor break the linear Kalman filter of Chapter 18, and write down both models with their Jacobians.
- Run one step of the extended Kalman filter (EKF) by hand, wrapping the bearing innovation, and say where the linearization error comes from.
- Construct the $2n + 1$ sigma points and weights of the unscented transform and recombine them into the unscented Kalman filter (UKF).
- Implement the bootstrap particle filter with systematic resampling, compute the effective sample size, and recognize degeneracy and sample impoverishment.
- Choose between EKF, UKF and particle filter for a given intruder model and sensor, and hand their output, including a multimodal particle cloud, to the planner of Chapter 24.

19.1 Why this matters for a drone swarm

The Kalman filter of Chapter 18 gives the prediction layer of the hybrid architecture (Chapter 24) a stable estimate of an intruder’s state and a covariance that says how much to trust it. It rests on two assumptions: the intruder moves according to a *linear* model, and the sensor reports a *linear* function of the state. Both fail in the situations that matter most. An unknown quadrotor that enters a survey site flies a straight leg, banks into a turn around a crane and straightens out; a constant-velocity model lags in every turn, and the natural model for a banked drone, the **coordinated-turn model** with speed, heading and turn rate in the state, contains the sine and cosine of the heading. The sensor our drone carries, a radar, a lidar or a camera with a rangefinder, reports a range and a bearing, the square root and the arctangent of the position. The Kalman filter has no place for either.

This chapter presents the three standard answers in increasing order of generality and cost. The **extended Kalman filter** (EKF) linearizes the models around the current estimate and runs the Kalman equations on the linearization. The **unscented Kalman filter** (UKF) avoids derivatives: it pushes a handful of chosen points through the true functions and fits a Gaussian to what comes out. The **particle filter** (PF) drops the Gaussian assumption and represents the belief by thousands of weighted samples, which lets it keep “went left around the building” and “went right” alive at once. All three live in the prediction layer of Chapter 24, turning raw measurements into a state estimate with its uncertainty for the local safety layer

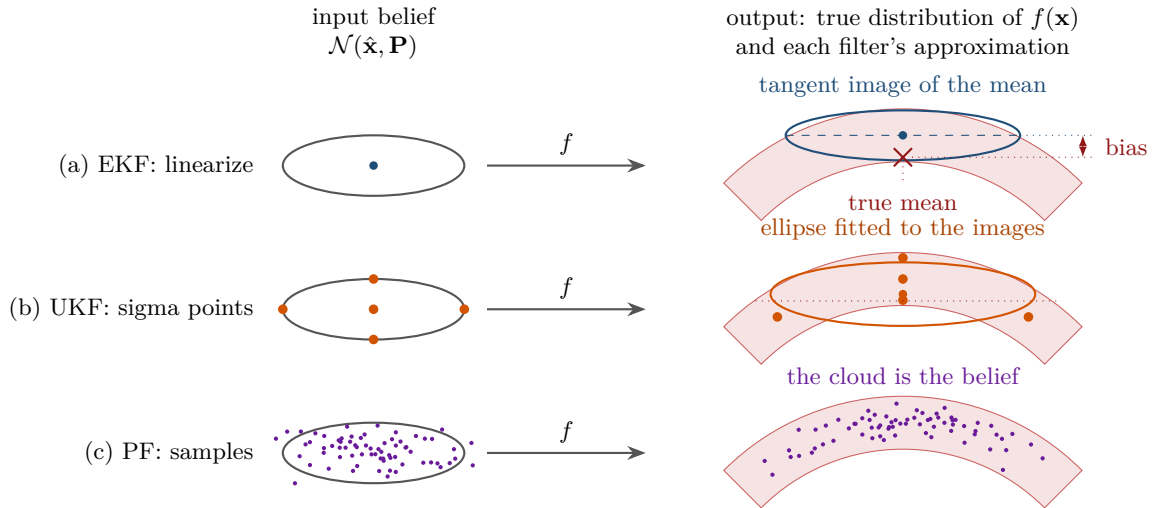


Figure 19.1. Three ways of pushing a Gaussian belief (left) through a nonlinear function f into the curved true output distribution (shaded banana). (a) The EKF moves the ellipse with the tangent at the mean; the ellipse sits on the image of the mean, not on the mean of the banana (the red cross), and cannot bend: the gap between the two is the arrow marked *bias*. (b) The UKF pushes $2n + 1$ sigma points through the true f and fits an ellipse to the images; the fitted mean lands on the dotted true-mean level, inside the banana. (c) The particle filter pushes a cloud of random samples through f and keeps the cloud as the belief, curvature included.

(Chapters 13 and 14), the trajectory predictor (Chapter 20) and the planner’s prediction-aware constraints; in the study plan (Appendix A) they complete Week 9. Sections 19.2 and 19.3 give the idea and the models, Sections 19.4 to 19.6 develop the three filters and compare them on a turning target, and the remaining sections cover properties, variants, implementation and the place in the drone system.

19.2 The idea in plain words

Every filter step pushes a Gaussian belief through a function and describes the result by a mean and a covariance. For an affine function the result is Gaussian (Chapter 2) and the Kalman filter is exact. As soon as the function bends, the output is no longer Gaussian: push a Gaussian in range and bearing through the polar-to-Cartesian map and you get a *banana*, a curved region that no ellipse describes well (Figure 19.1). The three filters are three approximations of that banana. The EKF replaces the bent function by its tangent at the current estimate: one Jacobian per step, and an error that grows with the curvature of the function within the width of the belief. The UKF keeps the true function and changes the representation of the input: $2n + 1$ *sigma points* whose mean and covariance are exactly those of the belief are mapped through the function, and mean and covariance are read off the images; no derivative is needed, and the mean is correct up to the second-order Taylor term, precisely the term the EKF drops. The particle filter represents the belief by many weighted random samples, moves each through the true function with its own noise and re-weights it by how well it explains the measurement; the belief can be curved, skewed or split in two, and the price is the number of samples.

Key idea

A nonlinear filter has to push a belief through a bent function. The EKF bends the belief with the tangent (one Jacobian), the UKF pushes $2n + 1$ representative points through the real function (no Jacobian, second-order accurate mean), and the particle filter pushes thousands of random samples through it (no Gaussian at all, but $\mathcal{O}(N)$ work and the need to resample). Choose the cheapest one whose assumptions your intruder and your sensor satisfy.

19.3 Problem statement and notation

Definition 19.1 (Nonlinear state-space model). A **nonlinear state-space model** has a state $\mathbf{x}_k \in \mathbb{R}^n$ at times $k = 0, 1, \dots$ spaced by Δt , measurements $\mathbf{z}_k \in \mathbb{R}^m$, a motion model f and a measurement model h with

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}) + \mathbf{w}_k, \quad \mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k, \quad (19.1)$$

where $\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k)$ and $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_k)$ are independent of each other and over time, and $\mathbf{x}_0 \sim \mathcal{N}(\hat{\mathbf{x}}_0, \mathbf{P}_0)$. The **filtering problem** is to compute, at every k , the posterior $p(\mathbf{x}_k | \mathbf{z}_{1:k})$ or at least its mean $\hat{\mathbf{x}}_k$ and covariance $\mathbf{P}_k$.

For $f(\mathbf{x}) = \mathbf{F}\mathbf{x}$ and $h(\mathbf{x}) = \mathbf{H}\mathbf{x}$ this is the linear-Gaussian model of [Chapter 18](#), whose notation we keep: $\hat{\mathbf{x}}_k^-$, $\mathbf{P}_k^-$ are the *predicted* mean and covariance before $\mathbf{z}_k$ is used, $\hat{\mathbf{x}}_k$, $\mathbf{P}_k$ the *updated* ones, $\mathbf{y}_k$ the innovation, $\mathbf{S}_k$ its covariance and $\mathbf{K}_k$ the gain. All three filters approximate the **Bayes filter** [168],

$$p(\mathbf{x}_k | \mathbf{z}_{1:k-1}) = \int p(\mathbf{x}_k | \mathbf{x}_{k-1}) p(\mathbf{x}_{k-1} | \mathbf{z}_{1:k-1}) d\mathbf{x}_{k-1}, \quad p(\mathbf{x}_k | \mathbf{z}_{1:k}) \propto p(\mathbf{z}_k | \mathbf{x}_k) p(\mathbf{x}_k | \mathbf{z}_{1:k-1}), \quad (19.2)$$

which pushes yesterday's posterior through the motion model and then applies Bayes' rule ([Chapter 2](#)) with the likelihood $p(\mathbf{z}_k | \mathbf{x}_k) = \mathcal{N}(\mathbf{z}_k; h(\mathbf{x}_k), \mathbf{R}_k)$; for the models below neither step has a closed form. ([Appendix B.3](#) reviews the multivariate Gaussian and [Appendix B.3.4](#) the Monte Carlo estimate the particle filter builds on.) We write $\partial g / \partial \mathbf{x}$ for the **Jacobian** of g , the matrix of partial derivatives $\partial g_i / \partial x_j$, which gives the best affine approximation $g(\mathbf{x} + \mathbf{d}) \approx g(\mathbf{x}) + \frac{\partial g}{\partial \mathbf{x}}(\mathbf{x}) \mathbf{d}$.

19.3.1 The coordinated-turn model

A drone flying at constant speed with a constant bank angle follows a circular arc. The **coordinated-turn model** has the state ([Figure 19.2](#))

$$\mathbf{x} = (x, y, v, \psi, \omega)^\top, \quad (19.3)$$

with position (x, y) , speed $v \geq 0$, heading ψ (counterclockwise from the x axis) and turn rate $\omega = \dot{\psi}$; $\omega > 0$ turns left, $\omega = 0$ is straight flight. Integrating $\dot{x} = v \cos \psi$, $\dot{y} = v \sin \psi$, $\dot{\psi} = \omega$ over one step gives the exact discrete model

$$f(\mathbf{x}) = \begin{pmatrix} x + \frac{v}{\omega}(\sin(\psi + \omega\Delta t) - \sin \psi) \\ y - \frac{v}{\omega}(\cos(\psi + \omega\Delta t) - \cos \psi) \\ v \\ \psi + \omega\Delta t \\ \omega \end{pmatrix} \quad (\omega \neq 0), \quad f(\mathbf{x}) = \begin{pmatrix} x + v\Delta t \cos \psi \\ y + v\Delta t \sin \psi \\ v \\ \psi \\ 0 \end{pmatrix} \quad (\omega = 0), \quad (19.4)$$

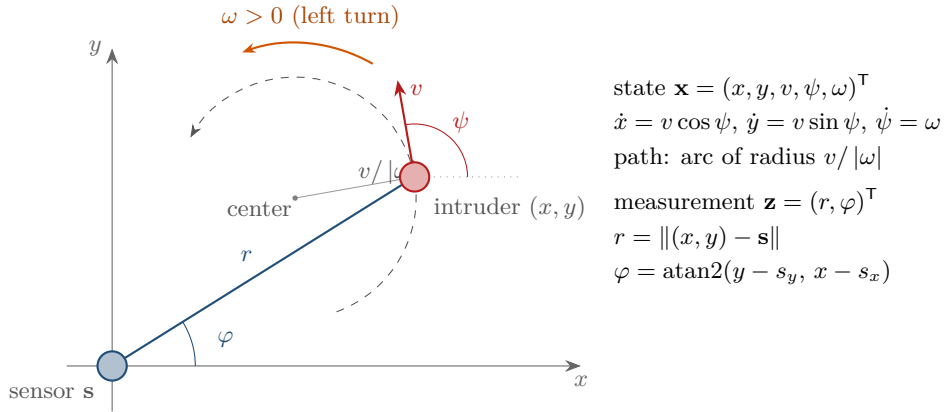


Figure 19.2. The two models of the chapter. The intruder at (x, y) flies at speed v along the heading ψ and turns left at the rate $\omega > 0$ on a circle of radius $v/|\omega|$; the sensor at $\mathbf{s}$ measures the range r and the bearing φ of the intruder, not its position.

an arc of radius $v/|\omega|$; the second form is the limit of the first, used below a small threshold on $|\omega|$. The model is nonlinear through $v \sin \psi$, $v \cos \psi$ and the division by ω (Bar-Shalom, Li and Kirubarajan [7] write it with Cartesian velocities; the polar form has the more compact Jacobian and delivers the speed and heading the local safety layer wants). Real intruders change speed and turn rate. We model a random longitudinal acceleration $a_k \sim \mathcal{N}(0, \sigma_a^2)$ and a random turn acceleration $\gamma_k \sim \mathcal{N}(0, \sigma_\gamma^2)$, constant during the step, so that, to first order in Δt (the construction ignores how the turn acceleration γ_k bends the path within one step),

$$\mathbf{w}_k = \mathbf{G}(\mathbf{x}) \begin{pmatrix} a_k \\ \gamma_k \end{pmatrix}, \quad \mathbf{G}(\mathbf{x}) = \begin{pmatrix} \frac{1}{2}\Delta t^2 \cos \psi & 0 \\ \frac{1}{2}\Delta t^2 \sin \psi & 0 \\ \Delta t & 0 \\ 0 & \frac{1}{2}\Delta t^2 \\ 0 & \Delta t \end{pmatrix}, \quad \mathbf{Q}(\mathbf{x}) = \mathbf{G}(\mathbf{x}) \begin{pmatrix} \sigma_a^2 & 0 \\ 0 & \sigma_\gamma^2 \end{pmatrix} \mathbf{G}(\mathbf{x})^\top, \quad (19.5)$$

the white-noise-acceleration construction of Chapter 18 applied along the heading and to the turn rate. $\mathbf{Q}$ depends on ψ , so the filters evaluate it at the current estimate; σ_a and σ_γ say how abruptly the intruder may maneuver.

19.3.2 The range–bearing sensor and angles

Definition 19.2 (Range–bearing measurement). A sensor at the known position $\mathbf{s} = (s_x, s_y)$ measures the **range** r and the **bearing** $\varphi \in (-\pi, \pi]$ of the intruder,

$$h(\mathbf{x}) = \begin{pmatrix} r \\ \varphi \end{pmatrix} = \begin{pmatrix} \sqrt{(x - s_x)^2 + (y - s_y)^2} \\ \text{atan2}(y - s_y, x - s_x) \end{pmatrix}, \quad \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \text{diag}(\sigma_r^2, \sigma_\varphi^2)). \quad (19.6)$$

h depends only on the position; the filter learns v , ψ and ω from how the position changes over time.

A bearing error of σ_φ radians at range r displaces the position by about $r\sigma_\varphi$ across the line of sight; the error region is a thin arc, the banana of Figure 19.1. Headings and bearings live on a circle: the difference between the bearings 3.10 and -3.10 is not 6.20 but -0.083 . We write

$$\text{wrap}(\theta) = \theta - 2\pi \left\lceil \frac{\theta - \pi}{2\pi} \right\rceil \in (-\pi, \pi] \quad (19.7)$$

for the representative of θ on the circle; every difference of two angles in this chapter, in an innovation, a residual or a likelihood, is $\text{wrap}(\theta_1 - \theta_2)$, and $\mathbf{z} \ominus \hat{\mathbf{z}}$ denotes a componentwise difference whose angular components are wrapped. The heading of an estimate is wrapped after every update.

19.4 The extended Kalman filter

19.4.1 Linearize, then run the Kalman filter

The EKF replaces f by its tangent at the current estimate and h by its tangent at the prediction,

$$f(\mathbf{x}) \approx f(\hat{\mathbf{x}}_{k-1}) + \mathbf{F}_k(\mathbf{x} - \hat{\mathbf{x}}_{k-1}), \quad h(\mathbf{x}) \approx h(\hat{\mathbf{x}}_k^-) + \mathbf{H}_k(\mathbf{x} - \hat{\mathbf{x}}_k^-), \quad \mathbf{F}_k = \frac{\partial f}{\partial \mathbf{x}}(\hat{\mathbf{x}}_{k-1}), \quad \mathbf{H}_k = \frac{\partial h}{\partial \mathbf{x}}(\hat{\mathbf{x}}_k^-), \quad (19.8)$$

and applies the Kalman equations of [Chapter 18](#) to the tangents, with the true f and h for the means:

$$\hat{\mathbf{x}}_k^- = f(\hat{\mathbf{x}}_{k-1}), \quad \mathbf{P}_k^- = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^\top + \mathbf{Q}_k, \quad (19.9)$$

$$\mathbf{y}_k = \mathbf{z}_k \ominus h(\hat{\mathbf{x}}_k^-), \quad \mathbf{S}_k = \mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^\top + \mathbf{R}_k, \quad (19.10)$$

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}_k^\top \mathbf{S}_k^{-1}, \quad \hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + \mathbf{K}_k \mathbf{y}_k, \quad \mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^-. \quad (19.11)$$

Only the covariances and the gain use the linearization. The Jacobians depend on the estimate, hence on the data: the covariance cannot be computed in advance, and a wrong estimate produces a wrong gain ([Section 19.4.4](#)).

Algorithm 19.1. One step of the extended Kalman filter

Input: previous estimate $\hat{\mathbf{x}}_{k-1}$, $\mathbf{P}_{k-1}$; measurement $\mathbf{z}_k$ or **nil**; models f , h with Jacobians; $\mathbf{Q}_k$, $\mathbf{R}_k$

Output: updated estimate $\hat{\mathbf{x}}_k$, $\mathbf{P}_k$

```

1 Function EkfStep( $\hat{\mathbf{x}}_{k-1}$ ,  $\mathbf{P}_{k-1}$ ,  $\mathbf{z}_k$ ):
2    $\mathbf{F}_k \leftarrow \partial f / \partial \mathbf{x}$  at  $\hat{\mathbf{x}}_{k-1}$ ;  $\hat{\mathbf{x}}_k^- \leftarrow f(\hat{\mathbf{x}}_{k-1})$ , heading wrapped;  $\mathbf{P}_k^- \leftarrow \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^\top + \mathbf{Q}_k$ 
3   if  $\mathbf{z}_k = \mathbf{nil}$  then
4     return ( $\hat{\mathbf{x}}_k^-$ ,  $\mathbf{P}_k^-$ )                                     // missing measurement: predict only
5    $\mathbf{H}_k \leftarrow \partial h / \partial \mathbf{x}$  at  $\hat{\mathbf{x}}_k^-$ 
6    $\mathbf{y}_k \leftarrow \mathbf{z}_k - h(\hat{\mathbf{x}}_k^-)$ ;  $\mathbf{y}_k[j] \leftarrow \text{wrap}(\mathbf{y}_k[j])$  for every angular component  $j$ 
7    $\mathbf{S}_k \leftarrow \mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^\top + \mathbf{R}_k$ ;  $\mathbf{K}_k \leftarrow \mathbf{P}_k^- \mathbf{H}_k^\top \mathbf{S}_k^{-1}$ 
8    $\hat{\mathbf{x}}_k \leftarrow \hat{\mathbf{x}}_k^- + \mathbf{K}_k \mathbf{y}_k$ , heading wrapped
9    $\mathbf{P}_k \leftarrow (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^- (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^\top + \mathbf{K}_k \mathbf{R}_k \mathbf{K}_k^\top$                                      // Joseph form
10  return ( $\hat{\mathbf{x}}_k$ ,  $\mathbf{P}_k$ )

```

Line 2 of [Algorithm 19.1](#) is the predict step, with $\mathbf{F}_k$ at the *previous* estimate; line 4 handles a step without measurement, as when an obstacle hides the intruder, by taking the prediction as the estimate. The update linearizes h at the *predicted* estimate (line 5); the innovation on line 6 has its bearing wrapped ([Section 19.4.3](#)). Line 9 uses the Joseph form of the covariance update, algebraically equal to $(\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^-$ but symmetric and positive definite under rounding.

19.4.2 The Jacobians of the two models

Proposition 19.3 (Jacobian of the coordinated-turn model). *Let $\psi' = \psi + \omega\Delta t$. For $\omega \neq 0$ the Jacobian of Equation (19.4) is*

$$\mathbf{F} = \begin{pmatrix} 1 & 0 & \frac{\sin \psi' - \sin \psi}{\omega} & \frac{v}{\omega}(\cos \psi' - \cos \psi) & \frac{v\Delta t \cos \psi'}{\omega} - \frac{v(\sin \psi' - \sin \psi)}{\omega^2} \\ 0 & 1 & -\frac{\cos \psi' - \cos \psi}{\omega} & \frac{v}{\omega}(\sin \psi' - \sin \psi) & \frac{v\Delta t \sin \psi'}{\omega} + \frac{v(\cos \psi' - \cos \psi)}{\omega^2} \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}, \quad (19.12)$$

and for $\omega = 0$ the first two rows become $F_{13} = \Delta t \cos \psi$, $F_{14} = -v\Delta t \sin \psi$, $F_{15} = -\frac{1}{2}v\Delta t^2 \sin \psi$, $F_{23} = \Delta t \sin \psi$, $F_{24} = v\Delta t \cos \psi$, $F_{25} = \frac{1}{2}v\Delta t^2 \cos \psi$.

Proof sketch. Rows three to five are the derivatives of v , $\psi + \omega\Delta t$ and ω . In the first row, v enters linearly with the factor $(\sin \psi' - \sin \psi)/\omega$; differentiating with respect to ψ uses $\partial\psi'/\partial\psi = 1$, and with respect to ω the quotient rule on $1/\omega$ together with $\partial\psi'/\partial\omega = \Delta t$; the second row is the same computation for $-\frac{v}{\omega}(\cos \psi' - \cos \psi)$. The $\omega = 0$ entries are limits: $\sin \psi' = \sin \psi + \omega\Delta t \cos \psi - \frac{1}{2}\omega^2\Delta t^2 \sin \psi + \mathcal{O}(\omega^3)$ gives $x' = x + v\Delta t \cos \psi - \frac{1}{2}v\omega\Delta t^2 \sin \psi + \mathcal{O}(\omega^2)$, whose ω -derivative at 0 is $-\frac{1}{2}v\Delta t^2 \sin \psi$. [Exercise 19.2](#) completes the limits; the self-test of the chapter code checks every entry against finite differences. $\square$

Proposition 19.4 (Jacobian of the range-bearing model). *With $d_x = x - s_x$, $d_y = y - s_y$ and $r = \sqrt{d_x^2 + d_y^2} > 0$,*

$$\mathbf{H} = \frac{\partial h}{\partial \mathbf{x}} = \begin{pmatrix} d_x/r & d_y/r & 0 & 0 & 0 \\ -d_y/r^2 & d_x/r^2 & 0 & 0 & 0 \end{pmatrix}. \quad (19.13)$$

Proof. The chain rule on the square root gives the first row; for the bearing, $\varphi = \text{atan2}(d_y, d_x)$ has $\partial\varphi/\partial x = -d_y/(d_x^2 + d_y^2)$ and $\partial\varphi/\partial y = d_x/(d_x^2 + d_y^2)$ wherever $r > 0$, on every branch of the arctangent. The last three columns vanish because h ignores v , ψ , ω . $\square$

19.4.3 A worked example: one EKF step by hand

Example 19.5 (One EKF step with a range-bearing measurement). Our drone hovers at $\mathbf{s} = (0, 0)$ and observes an intruder once per second ($\Delta t = 1$ s). After step $k-1$,

$$\hat{\mathbf{x}}_{k-1} = (-100, -4, 10, \pi, 0.3)^\top, \quad \mathbf{P}_{k-1} = \text{diag}(4, 4, 1, 0.04, 0.01),$$

in meters, meters per second, radians and radians per second: the intruder is 100 m to the west, flies west at 10 m/s and turns left at 0.3 rad/s, so its path curves toward the south. The process noise has $\sigma_a = 1$ m/s² and $\sigma_\gamma = 0.1$ rad/s², the sensor $\sigma_r = 2$ m and $\sigma_\varphi = 0.03$ rad, and the new measurement is $\mathbf{z}_k = (111.5 \text{ m}, 3.12 \text{ rad})$. The instance is `worked_example()` in `ch19_nonlinear_filters.py`; every number below is printed by its self-test.

Predict. With $v/\omega = 33.33$ and $\psi' = \pi + 0.3$, Equation (19.4) gives $x^- = -100 + 33.33 \sin \psi' = -109.85$ and $y^- = -4 - 33.33(\cos \psi' + 1) = -5.49$: 9.85 m west and 1.49 m south. The heading $\pi + 0.3$ wraps to -2.8416 ; speed and turn rate are unchanged. The Jacobian Equation (19.12) at $\hat{\mathbf{x}}_{k-1}$ has $F_{13} = -0.985$, $F_{14} = 1.489$, $F_{15} = 0.991$, $F_{23} = -0.149$, $F_{24} = -9.851$ and

Table 19.1. One step of [Algorithm 19.1](#) on [Example 19.5](#) and the same step of the UKF ([Section 19.5](#)). The two filters agree to about a decimeter because the heading uncertainty is small compared with the curvature of the arc; [Section 19.5.4](#) shows cases where they do not.

Quantity	x [m]	y [m]	v [m/s]	ψ [rad]	ω [rad/s]
$\hat{\mathbf{x}}_{k-1}$	-100.000	-4.000	10.000	3.1416	0.3000
EKF predicted $\hat{\mathbf{x}}_k^-$	-109.851	-5.489	10.000	-2.8416	0.3000
EKF updated $\hat{\mathbf{x}}_k$	-110.941	-2.134	10.190	-3.0313	0.2787
UKF predicted $\hat{\mathbf{x}}_k^-$	-109.641	-5.456	10.000	-2.8416	0.3000
UKF updated $\hat{\mathbf{x}}_k$	-110.834	-2.176	10.214	-3.0284	0.2784
std. dev. before	2.000	2.000	1.000	0.2000	0.1000
std. dev. predicted (EKF)	2.306	2.854	1.414	0.2291	0.1414
std. dev. updated (EKF)	1.513	2.157	1.326	0.2047	0.1410
std. dev. predicted (UKF)	2.355	2.810	1.414	0.2291	0.1414
std. dev. updated (UKF)	1.527	2.137	1.329	0.2059	0.1410

$F_{25} = -4.888$. The predicted covariance $\mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^\top + \mathbf{Q}_k$ has the position block $\begin{pmatrix} 5.319 & -0.488 \\ -0.488 & 8.143 \end{pmatrix}$ (standard deviations 2.31 and 2.85 m, larger across the westward flight) and now correlates position with heading (-0.443 between y and ψ) and speed, which the update will use to correct both from a position measurement.

Update. The predicted measurement is $h(\hat{\mathbf{x}}_k^-) = (109.99, -3.0917)$: the intruder is expected just south of the negative x axis, so its bearing is just above $-\pi$, while the measured bearing 3.12 is just below $+\pi$. Subtracting naively gives 6.2117, an innovation of 356° that would send the estimate to the other side of the sensor; wrapping gives -0.0715 rad, so $\mathbf{y}_k = (1.51, -0.0715)$: 1.5 m further away and 4.1° clockwise of the prediction. With $\mathbf{H}_k = \begin{pmatrix} -0.9988 & -0.0499 & 0 & 0 & 0 \\ 0.00045 & -0.00908 & 0 & 0 & 0 \end{pmatrix}$ from [Proposition 19.4](#), the innovation covariance is $\mathbf{S}_k \approx \text{diag}(9.28, 0.0016)$, and the gain maps the range innovation mostly into x and the bearing innovation mostly into y : $K_{11} = -0.569$ and $K_{22} = -47.1$, no typo, since a radian of bearing at 110 m is 110 m across the line of sight. The rows for the unmeasured components come from the correlations in $\mathbf{P}_k^-$: $K_{42} = 2.56$ turns a bearing innovation into a heading correction, $K_{31} = 0.16$ a range innovation into a speed correction. [Table 19.1](#) lists the result: the position moved 1.09 m west and 3.35 m north, almost entirely from the bearing; the heading turned by -0.19 rad toward due west (an intruder found north of the predicted arc is best explained by a less southerly heading); the speed rose by 0.19 m/s. The updated position standard deviations are 1.51 m along the line of sight, set by the range noise, and 2.16 m across it, set by the bearing noise at this range.

19.4.4 Where the EKF fails

Strong nonlinearity. [Figure 19.3](#) makes the linearization error visible. A Gaussian belief in range and bearing, $r \sim \mathcal{N}(100, 5^2)$ and $\varphi \sim \mathcal{N}(\pi/2, 0.3^2)$, is pushed through $(r, \varphi) \mapsto (r \cos \varphi, r \sin \varphi)$, as when a track is initialized from one measurement. The true output, estimated from 2000 samples, has its mean at $(-0.2, 95.6)$; the exact mean is $(0, 100 e^{-0.3^2/2}) = (0, 95.6)$, because r and φ are independent and $\mathbb{E}[\sin \varphi] = e^{-\sigma_\varphi^2/2}$ for $\varphi \sim \mathcal{N}(\pi/2, \sigma_\varphi^2)$ ([Exercise 19.5](#)). The picture to keep is the arc: the cloud bends away from the tangent, and the average of points on an arc lies *inside* it. The linearized map puts the mean at the image of the input mean, $(0, 100)$, 4.4 m too far out, and its ellipse is a straight bar that cannot follow the curved tails: its standard deviation along y is 5.0 m where the samples spread by 7.7 m, and its 2σ ellipse contains 74.6 % of the samples instead of the nominal 86.5 %.

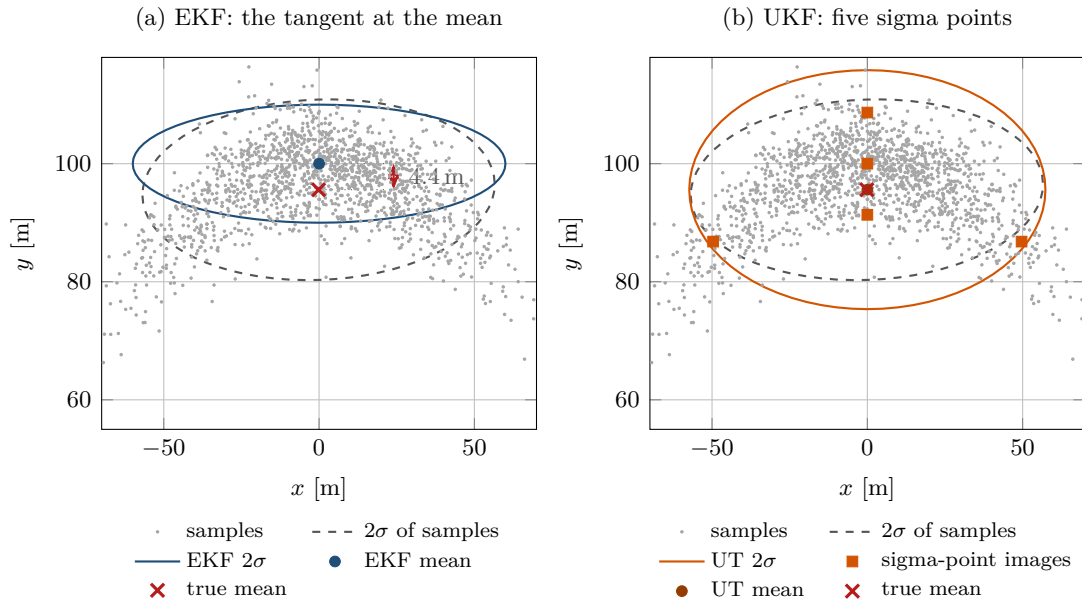


Figure 19.3. The linearization error of the EKF on the polar-to-Cartesian map. Gray dots: 2000 samples of a Gaussian in (r, φ) mapped to the plane; they form a banana. Left: the EKF's 2σ ellipse (blue) is centered on the image of the mean, 4.4m beyond the true mean (cross), and cannot bend. Right: the five sigma points of the unscented transform mapped through the same function (orange squares) and the ellipse fitted to them, centered inside the banana. The dashed ellipse is the 2σ ellipse of the samples themselves. The transform uses $n = 2$, $\alpha = 1$, $\beta = 2$ and $\kappa = 1$, so $\lambda = 1$ and the five points sit at $\sqrt{3}$ standard deviations (Exercise 19.5 repeats the computation by hand; the chapter's default elsewhere is $\kappa = 0$). The two axes are not drawn to the same scale: at equal scaling the 120m width of the ellipses would flatten the panels until the 4.4m offset disappeared.

Nothing in the EKF signals the error: it reports a confident covariance around a biased mean (Proposition 19.10 quantifies the bias). The unscented transform of Section 19.5 puts the mean at (0.0, 95.6) with five function evaluations and no derivative, and its ellipse contains 91.6% of the samples. The mean is right; the ellipse is not exact either, only conservative—the transform reports a spread along y of 10.1m where the samples spread by 7.7, so it covers *more* than the nominal 86.5%. Over-covering is the safer error of the two, but 91.6% is not “better” than 86.5%.

A poor initial estimate. If the estimate is far from the truth, $\mathbf{F}_k$ and $\mathbf{H}_k$ describe the wrong tangent, the gain is computed for a fictitious geometry, and an update can push the estimate further away while the covariance keeps shrinking: the filter has **diverged**. A range-bearing sensor measures the position directly, and in the experiment of Section 19.5.4 the EKF recovers from a 90° heading error in every run; divergence is the real danger with weak or bearing-only measurements, long gaps, and an initial covariance small enough to make the filter ignore its innovations. The cures are an honest $\mathbf{P}_0$, a filter that does not depend on a tangent, or a particle filter that keeps several hypotheses alive.

19.5 The unscented Kalman filter

19.5.1 The unscented transform

Julier and Uhlmann [74, 75] start from one observation: it is easier to approximate a probability distribution than an arbitrary nonlinear function. Instead of replacing f by its tangent, keep f and replace the Gaussian by a small set of points that carry its mean and covariance exactly, push the *points* through the true f , and read mean and covariance off the images.

Definition 19.6 (Sigma points and weights). Let $\bar{\mathbf{x}} \in \mathbb{R}^n$ and $\mathbf{P}$ be a mean and a covariance, let α, β, κ be parameters and $\lambda = \alpha^2(n + \kappa) - n$. Let $\mathbf{L}$ be a **matrix square root** of $(n + \lambda)\mathbf{P}$, that is $\mathbf{L}\mathbf{L}^\top = (n + \lambda)\mathbf{P}$, with columns $\mathbf{l}_1, \dots, \mathbf{l}_n$. The $2n + 1$ **sigma points** and their weights are

$$\begin{aligned} \mathcal{X}_0 &= \bar{\mathbf{x}}, & W_0^{(m)} &= \frac{\lambda}{n + \lambda}, & W_0^{(c)} &= \frac{\lambda}{n + \lambda} + 1 - \alpha^2 + \beta, \\ \mathcal{X}_i &= \bar{\mathbf{x}} + \mathbf{l}_i, \quad \mathcal{X}_{n+i} = \bar{\mathbf{x}} - \mathbf{l}_i, & W_i^{(m)} &= W_i^{(c)} = \frac{1}{2(n + \lambda)}, & i &= 1, \dots, n, \end{aligned} \quad (19.14)$$

where $W^{(m)}$ weights the mean and $W^{(c)}$ the covariance; the weights $W^{(m)}$ sum to one.

The points sit on the principal axes of the ellipse, $\sqrt{n + \lambda} = \alpha\sqrt{n + \kappa}$ standard deviations from the mean. α sets that spread, κ is a secondary scaling, and β enters only $W_0^{(c)}$ to inject knowledge of the shape of the distribution, $\beta = 2$ being optimal for a Gaussian [174]. The standard choices are $\alpha \in [10^{-3}, 1]$, $\beta = 2$ and $\kappa = 0$ or $\kappa = 3 - n$. With $\alpha = 1$ and $\kappa = 0$ (the chapter code), $\lambda = 0$, $W_0^{(m)} = 0$ and the points sit at $\sqrt{n}$ standard deviations; $\kappa = 3 - n$ puts them at $\sqrt{3}$, which matches the fourth moment of a Gaussian [74], at the price of a negative $W_0^{(m)}$ when $n > 3$. A small α keeps the points close to the mean, but makes $W_0^{(m)}$ a large negative number and the recombination a difference of large terms (Exercise 19.4). The square root is computed by the **Cholesky factorization** $(n + \lambda)\mathbf{P} = \mathbf{L}\mathbf{L}^\top$, $\mathbf{L}$ lower triangular, in $\mathcal{O}(n^3)$ operations; it exists when $\mathbf{P}$ is symmetric and positive definite. Any other square root gives a different set of points with the same mean and covariance and a slightly different result.

Definition 19.7 (Unscented transform). For a function g , the **unscented transform** (UT) of $\mathcal{N}(\bar{\mathbf{x}}, \mathbf{P})$ maps every sigma point, $\mathcal{Y}_i = g(\mathcal{X}_i)$, and returns

$$\bar{\mathbf{y}} = \sum_{i=0}^{2n} W_i^{(m)} \mathcal{Y}_i, \quad \mathbf{P}_{yy} = \sum_{i=0}^{2n} W_i^{(c)} (\mathcal{Y}_i \ominus \bar{\mathbf{y}})(\mathcal{Y}_i \ominus \bar{\mathbf{y}})^\top, \quad \mathbf{P}_{xy} = \sum_{i=0}^{2n} W_i^{(c)} (\mathcal{X}_i \ominus \bar{\mathbf{x}})(\mathcal{Y}_i \ominus \bar{\mathbf{y}})^\top, \quad (19.15)$$

the approximate mean, covariance and **cross-covariance** of $\mathbf{x}$ and $g(\mathbf{x})$.

Angular components are handled as in Section 19.3.2: the differences in Equation (19.15) are wrapped, and the mean of an angular component is $\mathcal{Y}_{0,j} + \sum_i W_i^{(m)} \text{wrap}(\mathcal{Y}_{i,j} - \mathcal{Y}_{0,j})$, wrapped again, so that images on both sides of the $\pm\pi$ seam average correctly. Figure 19.3(b) shows the transform at work: five evaluations of the polar-to-Cartesian map, and the mean lands inside the banana; Proposition 19.11 proves why.

19.5.2 The filter

The UKF replaces the two linearizations of the EKF by two unscented transforms and the product $\mathbf{P}_k^- \mathbf{H}_k^\top$ by the cross-covariance $\mathbf{C}_k = \mathbf{P}_{xz}$ between state and predicted measurement (equal to $\mathbf{P}_k^- \mathbf{H}_k^\top$ in the linear case), so that $\mathbf{K}_k = \mathbf{C}_k \mathbf{S}_k^{-1}$ and $\mathbf{P}_k = \mathbf{P}_k^- - \mathbf{K}_k \mathbf{S}_k \mathbf{K}_k^\top$ need no $\mathbf{H}_k$ at all. Algorithm 19.2 gives the version for additive noise, the one that fits Equation (19.1).

Lines 2–4 build the points of Definition 19.6; line 7 is the only place where the model is called, so the UKF works with any f and h you can evaluate, a learned model included. The prediction (line 12) adds $\mathbf{Q}_k$ after the transform, because additive noise does not pass

Algorithm 19.2. One step of the unscented Kalman filter (additive noise)

Input: previous estimate $\hat{\mathbf{x}}_{k-1}$, $\mathbf{P}_{k-1}$; measurement $\mathbf{z}_k$ or **nil**; models f , h ; $\mathbf{Q}_k$, $\mathbf{R}_k$; parameters α , β , κ

Output: updated estimate $\hat{\mathbf{x}}_k$, $\mathbf{P}_k$

```

1 Function SigmaPoints( $\bar{\mathbf{x}}$ ,  $\mathbf{P}$ ):
2    $\lambda \leftarrow \alpha^2(n + \kappa) - n$ ;  $\mathbf{L} \leftarrow \text{chol}((n + \lambda)\mathbf{P})$  with columns  $\mathbf{l}_1, \dots, \mathbf{l}_n$ 
3    $\mathcal{X}_0 \leftarrow \bar{\mathbf{x}}$ ;  $\mathcal{X}_i \leftarrow \bar{\mathbf{x}} + \mathbf{l}_i$  and  $\mathcal{X}_{n+i} \leftarrow \bar{\mathbf{x}} - \mathbf{l}_i$  for  $i = 1, \dots, n$ 
4    $W_0^{(m)} \leftarrow \lambda/(n + \lambda)$ ;  $W_0^{(c)} \leftarrow W_0^{(m)} + 1 - \alpha^2 + \beta$ ;  $W_i^{(m)} = W_i^{(c)} \leftarrow 1/(2(n + \lambda))$  for  $i \geq 1$ 
5   return ( $\mathcal{X}, W^{(m)}, W^{(c)}$ )
6 Function UnscentedTransform( $\mathcal{X}, W^{(m)}, W^{(c)}, g$ ):
7    $\mathcal{Y}_i \leftarrow g(\mathcal{X}_i)$  for  $i = 0, \dots, 2n$ 
8    $\bar{\mathbf{y}} \leftarrow \sum_i W_i^{(m)} \mathcal{Y}_i$ ;  $\mathbf{P}_{yy} \leftarrow \sum_i W_i^{(c)} (\mathcal{Y}_i \ominus \bar{\mathbf{y}})(\mathcal{Y}_i \ominus \bar{\mathbf{y}})^\top$ ;
    $\mathbf{P}_{xy} \leftarrow \sum_i W_i^{(c)} (\mathcal{X}_i \ominus \mathcal{X}_0)(\mathcal{Y}_i \ominus \bar{\mathbf{y}})^\top$ 
9   return ( $\bar{\mathbf{y}}, \mathbf{P}_{yy}, \mathbf{P}_{xy}$ )
10 Function UkfStep( $\hat{\mathbf{x}}_{k-1}$ ,  $\mathbf{P}_{k-1}$ ,  $\mathbf{z}_k$ ):
11   ( $\mathcal{X}, W^{(m)}, W^{(c)}$ )  $\leftarrow$  SigmaPoints( $\hat{\mathbf{x}}_{k-1}$ ,  $\mathbf{P}_{k-1}$ )
12   ( $\hat{\mathbf{x}}_k^-, \mathbf{P}_{xx}, \cdot$ )  $\leftarrow$  UnscentedTransform( $\mathcal{X}, W^{(m)}, W^{(c)}, f$ );  $\mathbf{P}_k^- \leftarrow \mathbf{P}_{xx} + \mathbf{Q}_k$ 
13   if  $\mathbf{z}_k = \text{nil}$  then
14     return ( $\hat{\mathbf{x}}_k^-, \mathbf{P}_k^-$ )
15   ( $\mathcal{X}, W^{(m)}, W^{(c)}$ )  $\leftarrow$  SigmaPoints( $\hat{\mathbf{x}}_k^-, \mathbf{P}_k^-$ ) // redraw around the prediction
16   ( $\hat{\mathbf{z}}_k, \mathbf{P}_{zz}, \mathbf{C}_k$ )  $\leftarrow$  UnscentedTransform( $\mathcal{X}, W^{(m)}, W^{(c)}, h$ );  $\mathbf{S}_k \leftarrow \mathbf{P}_{zz} + \mathbf{R}_k$ 
17    $\mathbf{K}_k \leftarrow \mathbf{C}_k \mathbf{S}_k^{-1}$ ;  $\hat{\mathbf{x}}_k \leftarrow \hat{\mathbf{x}}_k^- + \mathbf{K}_k(\mathbf{z}_k \ominus \hat{\mathbf{z}}_k)$ , heading wrapped;  $\mathbf{P}_k \leftarrow \mathbf{P}_k^- - \mathbf{K}_k \mathbf{S}_k \mathbf{K}_k^\top$ 
18   return ( $\hat{\mathbf{x}}_k, \mathbf{P}_k$ )

```

through f . The update redraws the sigma points around the prediction (line 15): reusing the propagated points would ignore $\mathbf{Q}_k$, which for a maneuvering intruder is the larger part of $\mathbf{P}_k^-$. Line 16 produces the predicted measurement, its covariance and the cross-covariance in one call, and line 17 is the Kalman update with $\mathbf{C}_k$ in place of $\mathbf{P}_k^- \mathbf{H}_k^\top$.

19.5.3 Worked example: the same step with sigma points

Run Algorithm 19.2 on Example 19.5. With $n = 5$, $\alpha = 1$, $\kappa = 0$ we get $\lambda = 0$, $W_0^{(m)} = 0$, $W_0^{(c)} = 2$ and $W_i^{(m)} = W_i^{(c)} = 0.1$ for $i \geq 1$. Since $\mathbf{P}_{k-1}$ is diagonal, each sigma point differs from the mean in one component by $\sqrt{5} \approx 2.236$ standard deviations (4.472 m in x or y , 2.236 m/s in v , 0.447 rad in ψ , 0.224 rad/s in ω). The center image is the EKF prediction, $(-109.851, -5.489)$. The two heading points map to $(-108.238, -9.602)$ and $(-109.526, -1.082)$: they straddle the arc, and their average x is -108.882 , almost a meter east of the center image, because the arc bends and the chord lies inside it. The weighted mean of all eleven images, $\hat{\mathbf{x}}_k^- = (-109.641, -5.456, 10, -2.8416, 0.3)$, is 0.21 m east of the EKF prediction: the second-order term $\frac{1}{2} P_{\psi\psi} \partial^2 x' / \partial \psi^2 = \frac{1}{2} \cdot 0.04 \cdot 9.85 = 0.197$ m, plus a little from the ω points. The update redraws eleven points around $\hat{\mathbf{x}}_k^-$, maps them through h , and forms the wrapped innovation and the gain from the cross-covariance. The result (Table 19.1) lies within 0.11 m of the EKF's: on this step the two filters agree because the heading uncertainty (0.2 rad) is small compared with the curvature of the arc. The next section shows what happens when the belief is wide compared with the bend.

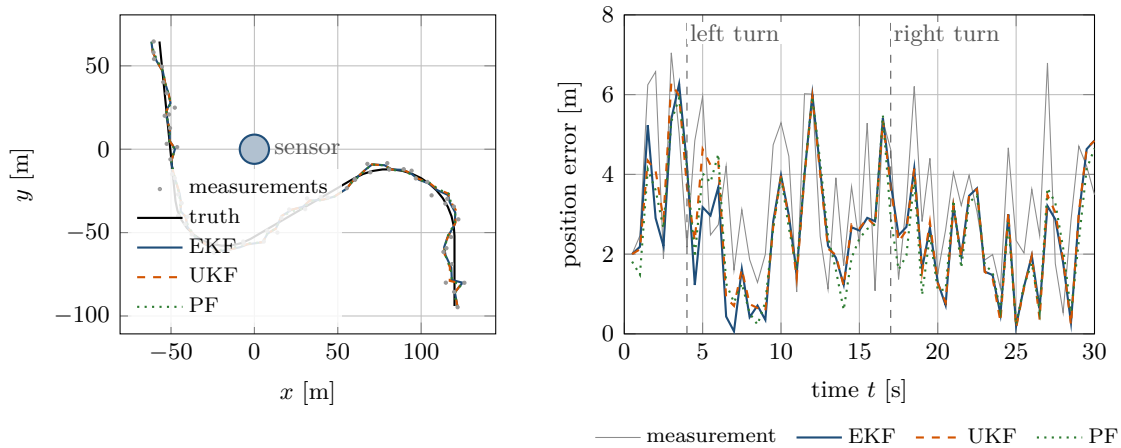


Figure 19.4. One run of the baseline scenario: the true path, the converted measurements (gray dots) and the three estimates (left); the position error over time, with the raw measurement error in gray (right). The errors rise at the onset of each turn, while the turn rate in the state catches up, and settle within a few seconds. RMSE on this run: 3.61 m for the measurements, 2.80 (EKF), 2.89 (UKF) and 2.76 m (PF).

Table 19.2. Position RMSE in meters after a 5 s settling period, mean and median over 100 Monte Carlo runs per scenario, the same measurement sequences for the three filters (`code/figures/gen_ch19_compare.py`). “Lost” counts runs whose final position error exceeds 30 m; one lost run dominates the particle filter’s mean in two scenarios. The close pass moves the sensor to (35, −20), within 10 m of the path; the poor start rotates the initial heading by 90° with a standard deviation of only 0.3 rad.

Scenario	mean RMSE				median RMSE			lost
	meas.	EKF	UKF	PF	EKF	UKF	PF	
baseline, $\sigma_\varphi = 2^\circ$	3.78	2.74	2.76	4.04	2.72	2.73	2.77	0/0/1
wide bearing noise, $\sigma_\varphi = 8^\circ$	10.11	6.88	6.21	7.62	6.23	5.95	6.02	0/0/0
close pass, minimum range 10 m	3.92	3.09	2.71	2.68	2.63	2.65	2.64	0/0/0
poor start, 90° heading error	3.89	3.10	2.90	5.74	2.78	2.78	3.08	0/0/1

19.5.4 EKF, UKF and particle filter on a turning target

Figure 19.4 and Table 19.2 compare the three filters on a target that starts at (120, −100) heading north at 12 m/s and flies 4 s straight, 7 s at $\omega = 0.3 \text{ rad/s}$, 6 s straight, 5 s at $\omega = -0.4 \text{ rad/s}$ and 8 s straight, sampled at $\Delta t = 0.5 \text{ s}$. The sensor at the origin has $\sigma_r = 3 \text{ m}$ and $\sigma_\varphi = 2^\circ$; the filters use the coordinated-turn model with $\sigma_a = 1.5 \text{ m/s}^2$ and $\sigma_\gamma = 0.3 \text{ rad/s}^2$ and are initialized from the first two measurements (position, speed and heading from the displacement, $\omega = 0$, a covariance that follows from the measurement noise at that range). The particle filter uses $N = 2000$ particles and resamples below $N/2$. The error measure is the root mean square (RMSE) of the position error after a settling period of ten steps; the raw measurement, converted to Cartesian coordinates, is the baseline any filter must beat.

Four things to notice. First, in the baseline scenario the EKF and the UKF are indistinguishable: at ranges of 45–160 m with 2° of bearing noise the belief is narrow compared with the curvature, and the tangent is as good as the sigma points. Second, when the belief widens the UKF pulls ahead: with 8° of bearing noise its RMSE is 10 % below the EKF’s, and on the close pass, where $\mathbf{H}$ changes fastest, 12 % below, with the particle filter matching the UKF. Third, the poor start is survived by *both Gaussian filters* in every run, because

the range-bearing measurement pins the position at each step; the UKF recovers faster than the EKF and the particle filter more slowly, and one of its hundred runs never recovers: a cloud whose initial spread does not contain the true heading has nothing to reweight, whereas the Gaussian filters extrapolate along the tangent (the lost baseline run has the same cause at a turn onset). Fourth, the cost: a UKF step costs roughly 1.8 times an EKF step, and a particle-filter step with 2000 particles roughly 4.5 times, a factor that grows linearly with N (Section 19.7 gives the absolute times).

19.6 The particle filter

19.6.1 Particles, weights and the bootstrap filter

When the intruder disappears behind a building and may come out on either side, no single ellipse describes where it is: the belief has two lobes, and their mean is a point inside the building where the intruder cannot be. The particle filter drops the Gaussian.

Definition 19.8 (Particle set). A **particle set** is a collection $\{(\mathbf{x}^{(i)}, \omega^{(i)})\}_{i=1}^N$ of states, the **particles**, with non-negative weights summing to one. It represents the distribution $\sum_i \omega^{(i)} \delta(\mathbf{x} - \mathbf{x}^{(i)})$, so that $\mathbb{E}[g(\mathbf{x})] \approx \sum_i \omega^{(i)} g(\mathbf{x}^{(i)})$ for any g ; the estimate is the weighted mean $\hat{\mathbf{x}} = \sum_i \omega^{(i)} \mathbf{x}^{(i)}$ and the weighted covariance.

Two remarks on the letters, because this section is where several of them meet. The weights are written with a Greek ω , not a Latin w , because $\mathbf{w}_k$ is already the process noise of Equation (19.1) and the two appear side by side in Equation (19.16); the bracketed superscript counts particles and is never a power. And from here to the end of the chapter N is the number of particles— N_{eff} , N_{thr} and N_i are all counts of particles—while the dimension of the state stays n and the Gaussian keeps its own symbol $\mathcal{N}$.

The weights come from **importance sampling**: if we cannot draw from the posterior p but can draw from a proposal q , then samples $\mathbf{x}^{(i)} \sim q$ with weights $\omega^{(i)} \propto p(\mathbf{x}^{(i)})/q(\mathbf{x}^{(i)})$ represent p . Done sequentially with the motion model as the proposal, this is the **bootstrap filter** of Gordon, Salmond and Smith [57], also called **sequential importance resampling** (SIR) [6]: each particle is moved through f with its own draw of the process noise, which samples $p(\mathbf{x}_k | \mathbf{x}_{k-1}^{(i)})$, and its weight is multiplied by the likelihood of the new measurement,

$$\mathbf{x}_k^{(i)} = f(\mathbf{x}_{k-1}^{(i)}) + \mathbf{w}_k^{(i)}, \quad \mathbf{w}_k^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k), \quad \omega_k^{(i)} \propto \omega_{k-1}^{(i)} p(\mathbf{z}_k | \mathbf{x}_k^{(i)}), \quad (19.16)$$

followed by normalization. This is the Bayes filter Equation (19.2) with the integral replaced by a sample and the multiplication by the likelihood applied to each sample. For our models the noise sample is $\mathbf{G}(\mathbf{x}^{(i)})(a^{(i)}, \gamma^{(i)})^\top$ from Equation (19.5) and the log-likelihood is

$$\log p(\mathbf{z}_k | \mathbf{x}) = -\frac{(r_k - \hat{r})^2}{2\sigma_r^2} - \frac{\text{wrap}(\varphi_k - \hat{\varphi})^2}{2\sigma_\varphi^2} + \text{const}, \quad (\hat{r}, \hat{\varphi}) = h(\mathbf{x}), \quad (19.17)$$

with the bearing difference wrapped. A step without a measurement skips the weighting, and any other knowledge that can be written as a likelihood joins in the same way: a map, for instance, gives weight zero to a particle inside a building.

19.6.2 Degeneracy, effective sample size and resampling

Multiplying weights step after step makes their variance grow until one particle carries almost all the weight and the others contribute nothing, a state called **degeneracy**. The standard diagnostic is the **effective sample size**

$$N_{\text{eff}} = \frac{1}{\sum_{i=1}^N (\omega^{(i)})^2} \in [1, N], \quad (19.18)$$

which equals N when all weights are equal and 1 when one particle has all the weight (Exercise 19.6). When N_{eff} falls below a threshold N_{thr} , typically $N/2$, the filter **resamples**: it draws N particles from the current set with probabilities $\omega^{(i)}$, with replacement, and gives the copies equal weights $1/N$: heavy particles are duplicated, light ones disappear. Drawing N times independently (multinomial resampling) costs $\mathcal{O}(N \log N)$ with one binary search per draw (or $\mathcal{O}(N)$ if the N uniforms are generated in sorted order) and adds unnecessary randomness; **systematic resampling** [6, 85], Algorithm 19.3, uses one uniform draw and a grid of N evenly spaced points walked along the cumulative weights, in $\mathcal{O}(N)$ time; its per-particle counts obey Proposition 19.12, and in practice its variance is far below multinomial resampling's, although—unlike **stratified resampling**, which draws one uniform per interval $[(m-1)/N, m/N)$ and provably never does worse than multinomial—systematic resampling carries no such guarantee, and its result depends on the order in which the particles are stored [35].

Algorithm 19.3. Systematic resampling

Input: normalized weights $\omega^{(1)}, \dots, \omega^{(N)}$

Output: indices $j_1, \dots, j_N$ of the particles to copy

```

1 Function SystematicResample( $\omega$ ):
2   draw  $u \sim \mathcal{U}[0, 1/N)$ ;  $c \leftarrow \omega^{(1)}$ ;  $i \leftarrow 1$ 
   // the test  $i < N$  guards against rounding:  $\sum_i \omega^{(i)}$  may be  $1 - \epsilon$ 
3   for  $m = 1, \dots, N$  do
4      $u_m \leftarrow u + (m-1)/N$ 
5     while  $u_m > c$  and  $i < N$  do
6        $i \leftarrow i + 1$ ;  $c \leftarrow c + \omega^{(i)}$ 
7      $j_m \leftarrow i$ 
8   return ( $j_1, \dots, j_N$ )

```

Resampling cures degeneracy and creates a second problem. The survivors are copies of a few ancestors; if the process noise is small, the copies stay together and the cloud collapses onto a handful of distinct states, which is **sample impoverishment**. Resampling at every step makes it worse, because each round discards diversity the noise has not yet restored. The remedies are to resample only when N_{eff} demands it, to keep the process noise honest, and to jitter the copies after resampling (*roughening* [57]) or draw them from a kernel around each particle (the regularized particle filter [6]).

19.6.3 The algorithm

Algorithm 19.4 collects the pieces. It keeps **log-weights** $\ell^{(i)} = \log \omega^{(i)}$: the likelihood of a particle ten standard deviations off is e^{-50} , and a product of such factors underflows double precision within a few steps, whereas sums of logarithms do not.

Line 3 is the prediction: no Jacobian, no sigma points, just the model and a noise draw per particle, so the filter accepts any model you can simulate, even one with discrete decisions such as “turn left or right with equal probability”. Line 5 is Bayes’ rule, particle by particle; line 6 normalizes in the log domain by subtracting the largest log-weight before exponentiating. The estimate on line 7 is computed *before* resampling, from the weighted set; it serves consumers that want a Gaussian, but the particle set itself is the belief (Section 19.10). Lines 8–10 resample only when N_{eff} has dropped below the threshold.

Algorithm 19.4. One step of the bootstrap particle filter with systematic resampling

Input: particles $\mathbf{x}_{k-1}^{(i)}$ with log-weights $\ell^{(i)}$, $i = 1, \dots, N$; measurement $\mathbf{z}_k$ or **nil**;
 motion model f with process noise $\mathbf{Q}_k$; likelihood $p(\mathbf{z} | \mathbf{x})$; threshold N_{thr}

Output: particles and log-weights at time k ; estimate $\hat{\mathbf{x}}_k$, $\mathbf{P}_k$; N_{eff}

```

1 Function ParticleFilterStep( $\{\mathbf{x}_{k-1}^{(i)}, \ell^{(i)}\}, \mathbf{z}_k$ ):
2   for  $i = 1, \dots, N$  do
3     draw  $\mathbf{w}^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k)$ ;  $\mathbf{x}_k^{(i)} \leftarrow f(\mathbf{x}_{k-1}^{(i)}) + \mathbf{w}^{(i)}$ , heading wrapped
4     if  $\mathbf{z}_k \neq \text{nil}$  then
5        $\ell^{(i)} \leftarrow \ell^{(i)} + \log p(\mathbf{z}_k | \mathbf{x}_k^{(i)})$ 
6    $\omega^{(i)} \leftarrow \exp(\ell^{(i)} - \max_j \ell^{(j)})$ ;  $\omega^{(i)} \leftarrow \omega^{(i)} / \sum_j \omega^{(j)}$ ;  $\ell^{(i)} \leftarrow \log \omega^{(i)}$ 
7    $\hat{\mathbf{x}}_k \leftarrow \sum_i \omega^{(i)} \mathbf{x}_k^{(i)}$  (circular mean for the heading);
8    $\mathbf{P}_k \leftarrow \sum_i \omega^{(i)} (\mathbf{x}_k^{(i)} \ominus \hat{\mathbf{x}}_k)(\mathbf{x}_k^{(i)} \ominus \hat{\mathbf{x}}_k)^\top$ 
9    $N_{\text{eff}} \leftarrow 1 / \sum_i (\omega^{(i)})^2$ 
10  if  $N_{\text{eff}} < N_{\text{thr}}$  then
11     $(j_1, \dots, j_N) \leftarrow \text{SystematicResample}(\omega)$ ;  $\mathbf{x}_k^{(i)} \leftarrow \mathbf{x}_k^{(j_i)}$  and  $\ell^{(i)} \leftarrow -\log N$  for all  $i$ 
12  return  $\{\mathbf{x}_k^{(i)}, \ell^{(i)}\}, \hat{\mathbf{x}}_k, \mathbf{P}_k, N_{\text{eff}}$ 

```

19.6.4 Worked example: an intruder hidden behind a building

Example 19.9 (A multimodal posterior). An intruder starts at $(0, 5)$ flying north at 8 m/s toward a building that occupies $x \in [-10, 10]$, $y \in [50, 70]$. Our sensor at the origin ($\sigma_r = 2$ m, $\sigma_\varphi = 0.05$ rad, $\Delta t = 0.5$ s) loses sight of it while $y \in [35, 75]$, a band of urban canyon in which the intruder, unknown to us, turns left for one second at 1 rad/s, flies straight for two, turns back and continues north on the west side of the building. The particle filter uses $N = 2000$ particles, $N_{\text{thr}} = N/2$, the coordinated-turn model with $\sigma_a = 0.5$ m/s² and $\sigma_\gamma = 0.3$ rad/s², and the map as a likelihood: a particle inside the building gets weight zero. A UKF with the same models runs alongside. The instance is `occlusion_example()` in the chapter code.

Figure 19.5 tells the story. While measurements arrive, the cloud is a tight ellipse and behaves like a Gaussian filter. Once they stop, the process noise spreads the particles, the map removes those whose noise draws point them into the building, and the survivors flow round both sides: at $k = 15$, 68.3% of the particles are west of the building and 27.8% east of it; the remaining 3.9% are still level with the building in x , and 3.4% of the whole cloud sits inside its footprint carrying weight zero, waiting to be removed by the next resampling: dead bookkeeping, not a third hypothesis. The UKF, whose belief must be one ellipse, carries its mean straight through the building, $(-3.0, 61.4)$ at $k = 15$, a point where the intruder cannot be. At $k = 20$ the intruder reappears on the west side. The likelihood of that first measurement is tiny for every eastern particle and for most western ones: N_{eff} collapses to 17, the filter resamples, the eastern lobe vanishes and 99.9% of the new particles lie within 15 m of the truth; one step later the process noise has separated the copies and N_{eff} is back at 432. The filter resamples in 13 of the 29 steps. Resampling resets the weights to $1/N$, but the very next measurement re-concentrates them at once, and N_{eff} is plotted *before* resampling (line 8), so the curve climbs back only to between $0.2N$ and $0.8N$: that sawtooth between the threshold and a partial recovery is the signature of a healthy filter with informative measurements. A curve pinned near N , as in the measurement-free band, means the data are telling the filter nothing; one pinned near 1 means a lost track. For the planner, the two lobes are two

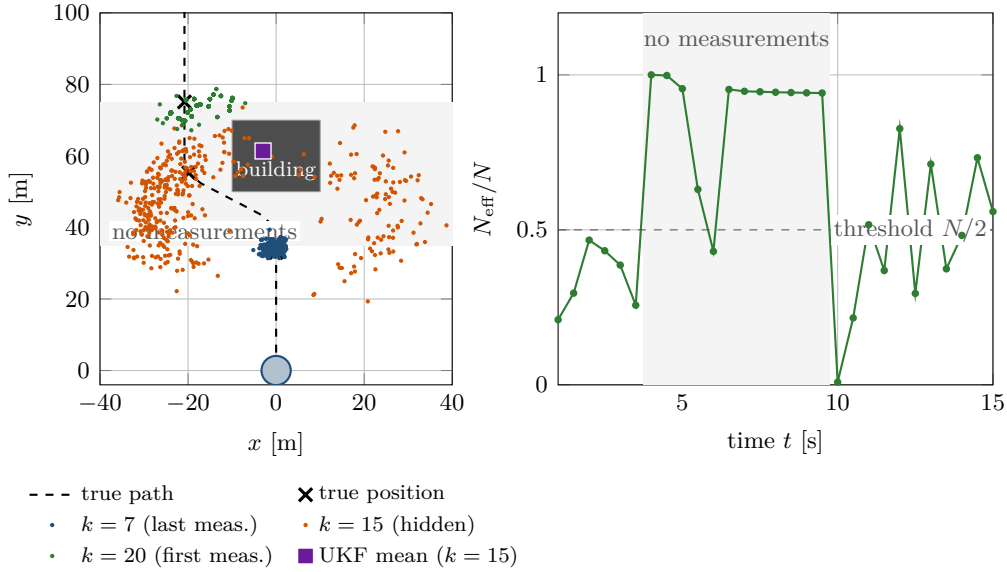


Figure 19.5. The particle cloud of Example 19.9 (400 of the 2000 particles) at the last measurement before the canyon ($k = 7$, blue), while hidden ($k = 15$, orange) and at the first measurement after it ($k = 20$, green); crosses mark the true positions. Behind the building the cloud splits into two lobes, while the UKF mean (purple square) sits inside the building. Right: N_{eff}/N over time; the filter resamples below $1/2$. Without measurements the weights stay flat except where the map kills particles entering the building; the first measurement after the gap collapses N_{eff} to 17.

hypotheses with probabilities 0.68 and 0.28, each with its own predicted path; Section 19.10 says what to do with them.

19.7 Properties: accuracy, convergence and cost

None of the three filters is optimal for a nonlinear model; each approximates the Bayes filter, and what can be proved is how good the approximation is and what it costs.

Proposition 19.10 (The EKF and the second-order term). *If f and h are affine, the EKF is the Kalman filter and is exact. For a twice differentiable g and $\mathbf{x} \sim \mathcal{N}(\bar{\mathbf{x}}, \mathbf{P})$,*

$$\mathbb{E}[g(\mathbf{x})] = g(\bar{\mathbf{x}}) + \frac{1}{2} \sum_{i,j} P_{ij} \frac{\partial^2 g}{\partial x_i \partial x_j}(\bar{\mathbf{x}}) + (\text{fourth and higher order}), \quad (19.19)$$

and the EKF keeps only the first term: its mean error is of second order in the spread of the belief, proportional to curvature times covariance.

Proof. For affine g the Jacobian is the constant matrix of the model, so the equations of Algorithm 19.1 are those of the Kalman filter. Write $\mathbf{x} = \bar{\mathbf{x}} + \mathbf{d}$ with $\mathbb{E}[\mathbf{d}] = \mathbf{0}$, $\mathbb{E}[\mathbf{d}\mathbf{d}^\top] = \mathbf{P}$, and expand g to second order; the linear term has zero mean, the quadratic term has mean $\frac{1}{2} \sum_{i,j} P_{ij} \partial^2 g / \partial x_i \partial x_j$, and the odd terms of a symmetric distribution vanish. The EKF mean is $g(\bar{\mathbf{x}})$. $\square$

Proposition 19.11 (Properties of the unscented transform). *The sigma points of Definition 19.6 satisfy $\sum_i W_i^{(m)} \mathcal{X}_i = \bar{\mathbf{x}}$ and $\sum_i W_i^{(c)} (\mathcal{X}_i - \bar{\mathbf{x}})(\mathcal{X}_i - \bar{\mathbf{x}})^\top = \mathbf{P}$. For affine g the unscented transform returns the exact mean, covariance and cross-covariance. For any three times differentiable g it returns the mean of Equation (19.19) correctly through second order.*

Proof. The weights $W^{(m)}$ sum to $\lambda/(n + \lambda) + 2n/(2(n + \lambda)) = 1$, and the points come in symmetric pairs, so $\sum_i W_i^{(m)} \mathcal{X}_i = \bar{\mathbf{x}}$. The covariance sum has no contribution from $\mathcal{X}_0$ and gives $\frac{1}{2(n+\lambda)} \sum_{i=1}^n 2\mathbf{l}_i \mathbf{l}_i^\top = \frac{1}{n+\lambda} \mathbf{L} \mathbf{L}^\top = \mathbf{P}$. For $g(\mathbf{x}) = \mathbf{B}\mathbf{x} + \mathbf{b}$ every image is $g(\bar{\mathbf{x}}) \pm \mathbf{B}\mathbf{l}_i$, and the three sums of Equation (19.15) reduce to $\mathbf{B}\bar{\mathbf{x}} + \mathbf{b}$, $\mathbf{B}\mathbf{P}\mathbf{B}^\top$ and $\mathbf{P}\mathbf{B}^\top$ by the same computation. For general g , expand each image about $\bar{\mathbf{x}}$: the odd terms cancel between $\bar{\mathbf{x}} + \mathbf{l}_i$ and $\bar{\mathbf{x}} - \mathbf{l}_i$, the constant term is weighted by $\sum_i W_i^{(m)} = 1$, and the quadratic terms give $\frac{1}{2(n+\lambda)} \sum_{i=1}^n \mathbf{l}_i^\top \nabla^2 g \mathbf{l}_i = \frac{1}{2(n+\lambda)} \text{tr}(\nabla^2 g \mathbf{L} \mathbf{L}^\top) = \frac{1}{2} \text{tr}(\nabla^2 g \mathbf{P})$, the second-order term of Equation (19.19) (for each component of g). $\square$

The covariance is also correct through second order, with β tuning part of the fourth-order term [75]; the chapter's self-test checks the affine exactness numerically. What the proposition does not say matters as much: the UKF still fits one Gaussian, and its error grows with the curvature within the belief just as the EKF's does, only more slowly.

Proposition 19.12 (Systematic resampling). *Let N_i be the number of copies of particle i produced by Algorithm 19.3. Then $N_i \in \{\lfloor N\omega^{(i)} \rfloor, \lceil N\omega^{(i)} \rceil\}$, $\mathbb{E}[N_i] = N\omega^{(i)}$, and the algorithm runs in $\mathcal{O}(N)$ time.*

Proof. Particle i owns the interval $(c_{i-1}, c_i]$ of the cumulative weights, of length $\omega^{(i)}$, and N_i is the number of grid points $u + (m-1)/N$ in it; a half-open interval of length ω contains $\lfloor N\omega \rfloor$ or $\lceil N\omega \rceil$ points of a grid with spacing $1/N$. Each u_m is uniform on $[(m-1)/N, m/N)$, so it lands in the interval with probability N times the overlap, and summing over m gives $N\omega^{(i)}$. The inner loop advances i at most N times in total. $\square$

Theorem 19.13 (Convergence of the bootstrap filter; Crisan and Doucet 2002). *Under mild conditions on the model (a bounded, continuous likelihood), for every bounded function g and every time k there is a constant c_k , independent of N , such that*

$$\mathbb{E}\left[\left(\sum_i \omega_k^{(i)} g(\mathbf{x}_k^{(i)}) - \mathbb{E}[g(\mathbf{x}_k) \mid \mathbf{z}_{1:k}]\right)^2\right] \leq \frac{c_k \|g\|_\infty^2}{N}.$$

Proof sketch. Each sampling step (prediction, resampling) contributes a mean-square error of order $1/N$ that the next step propagates, and the constants compound over time; see Crisan and Doucet [32]. $\square$

The theorem is reassuring and misleading at once: the rate $1/\sqrt{N}$ does not depend on the dimension, but c_k does, and badly, because the cloud must cover a region whose volume grows exponentially with the state dimension (Section 19.8).

Cost per step. With n states and m measurements, the EKF evaluates one Jacobian pair and does $\mathcal{O}(n^3 + n^2m)$ matrix work; the UKF adds two Cholesky factorizations and $2(2n+1)$ model evaluations; the particle filter costs $\mathcal{O}(Nn)$ for propagation and likelihoods and $\mathcal{O}(N)$ for resampling, with N in the thousands. On the machine used for Table 19.2, one step of the five-dimensional model in NumPy takes about 0.3 ms (EKF), 0.5 ms (UKF) and 1.2 ms (particle filter, $N = 2000$); absolute times move with the machine, the ratios, roughly 1 : 1.8 : 4.5, much less. Table 19.3 summarizes.

Which filter for which intruder model.

- Linear motion and sensor (constant velocity, position measurements): the Kalman filter of Chapter 18.
- Mild nonlinearity, unimodal belief (coordinated turn at long range, small bearing noise, a good initialization): the EKF, within 1 % of the UKF in Table 19.2 at the lowest cost.
- Strong nonlinearity, still unimodal (short range, wide bearing noise, a poor start, a model without closed-form derivatives): the UKF.

Table 19.3. The three filters side by side. Costs per step are given relative to one EKF step of the five-dimensional coordinated-turn model in NumPy (Section 19.7).

	EKF	UKF	Particle filter
Assumptions	Gaussian belief; Jacobians of f , h in closed form; mild curvature across the belief	Gaussian belief; f , h evaluable, no derivatives; $\mathbf{P}$ positive definite	f can be simulated and $p(\mathbf{z} \mid \mathbf{x})$ evaluated; N large enough to cover the posterior
Cost per step	one Jacobian pair, $\mathcal{O}(n^3)$; $1\times$	$2(2n+1)$ model calls, two Cholesky factorizations; $1.8\times$	N model calls and likelihoods, $\mathcal{O}(N)$ resampling; $4.5\times$ at $N = 2000$
Multimodality	no	no	yes (and any likelihood: maps, outliers)
Tuning burden	$\mathbf{Q}, \mathbf{R}, \mathbf{P}_0$	$\mathbf{Q}, \mathbf{R}, \mathbf{P}_0, \alpha, \beta, \kappa$	$\mathbf{Q}, \mathbf{R}, \mathbf{P}_0, N, N_{\text{thr}}$, roughening
Typical use	mild maneuvers at long range; embedded targets	short range, wide bearing noise, poor starts; models without Jacobians	occlusions, maps, unknown maneuver onset; input to multimodal prediction

- Multimodal or non-Gaussian beliefs (occlusions, buildings, an unknown maneuver onset, outliers, hard constraints): the particle filter, with thousands of particles in five dimensions and more in 3D.
- An intruder that switches between straight flight and turns: a bank of EKFs or UKFs mixed by the IMM scheme of Section 19.8.

19.8 Variants and extensions

Better Gaussian filters. The *iterated EKF* re-linearizes h at the updated estimate and repeats the update until it stops moving; the *second-order EKF* adds the Hessian term of Equation (19.19) explicitly [7]. The *augmented UKF* appends the noise to the state so that non-additive noise passes through f ; the *square-root UKF* propagates the Cholesky factor of $\mathbf{P}$ and cannot lose positive definiteness [174]; the *cubature Kalman filter* uses $2n$ equally weighted points at $\sqrt{n}$ standard deviations, a UKF without α, β, κ [5].

Switching models. With ω in the state, the coordinated-turn model adapts to a turn only as fast as σ_γ allows, hence the error spikes at every turn onset in Figure 19.4. The **interacting multiple model** (IMM) filter of Blom and Bar-Shalom [7, 19] runs one filter per maneuver mode (straight, left turn, right turn), mixes their estimates with mode probabilities updated from the innovations, and switches within a step or two.

Better particle filters, and three dimensions. The *Rao–Blackwellized* particle filter runs a Kalman filter for the linear part of the state inside every particle and samples only the nonlinear part, which is what makes particle methods workable in higher dimensions [6, 36]. For a drone the state gains altitude and vertical speed, $\mathbf{x} = (x, y, z, v, \psi, \omega, v_z)$ with $z' = z + v_z \Delta t$, and the sensor an elevation angle; the Jacobians gain a row and two columns, the UKF two sigma points, and the particle filter roughly an order of magnitude in N .

19.9 Implementation notes

Keeping $\mathbf{P}$ a covariance. The Cholesky factorization fails as soon as $\mathbf{P}$ loses symmetry or positive definiteness by rounding. Symmetrize, $\mathbf{P} \leftarrow \frac{1}{2}(\mathbf{P} + \mathbf{P}^T)$, after every update; use the Joseph form for the EKF; add a small multiple of the identity before factorizing; and if all else fails clip the negative eigenvalues or move to the square-root UKF.

Angles. Wrap every difference of angles: bearing innovations, the residuals of sigma-point images and of particles, and the differences from which a sigma-point mean is computed; wrap the heading of every estimate and every particle after prediction and update; and compute the heading of a particle cloud as the circular mean $\text{atan2}(\sum_i \omega^{(i)} \sin \psi^{(i)}, \sum_i \omega^{(i)} \cos \psi^{(i)})$. The code centralizes this in `wrap_angle`, `residual` and `weighted_mean`.

Listing 19.1 shows the resampling and the update of the particle filter; `sigma_points` and `unscented_transform` in the same file transcribe Algorithm 19.2 line by line.

Listing 19.1. Systematic resampling and the update of the bootstrap particle filter with log-weights (from `code/ch19_nonlinear_filters.py`).

```

1  def systematic_resample(weights, rng):
2      """Indices to keep: one uniform draw, N evenly spaced points."""
3      n = len(weights)
4      positions = (rng.random() + np.arange(n)) / n
5      cumulative = np.cumsum(weights)
6      cumulative[-1] = 1.0                                # guard against rounding
7      return np.searchsorted(cumulative, positions)
8
9  def update(self, z=None):
10     """Weight by the likelihood, normalize, estimate, resample."""
11     if z is not None:
12         self.logw += self.sensor.log_likelihood(z, self.X)
13     if self.constraint is not None:
14         self.logw[~self.constraint(self.X)] = -np.inf
15     top = np.max(self.logw)
16     if not np.isfinite(top):                             # every particle died: flat
17         self.logw[:] = -math.log(self.n)
18         top = self.logw[0]
19     w = np.exp(self.logw - top)                          # log-sum-exp normalization
20     w /= np.sum(w)
21     with np.errstate(divide="ignore"):
22         self.logw = np.log(w)
23     self.n_eff = effective_sample_size(w)
24     self.x, self.P = self.estimate(w)
25     if self.n_eff < self.threshold:
26         idx = systematic_resample(w, self.rng)
27         self.X = self.X[idx]
28         # optional jitter of the copies
29         if self.roughening is not None:
30             jitter = self.rng.normal(size=self.X.shape)
31             self.X = self.X + jitter * self.roughening
32             self.logw = np.full(self.n, -math.log(self.n))
33             self.resample_count += 1
34     return self.x, self.P

```

Vectorizing the particle filter. Store the particles as an (N, n) array and write f , h and the log-likelihood for arrays of states, so that a step is a handful of NumPy calls; systematic

resampling is a cumulative sum and one `searchsorted`. With $N = 2000$ this brings a step down to about a millisecond, a hundred times faster than a Python loop over particles. Keep log-weights, normalize with the log-sum-exp trick, and detect a dead cloud (all log-weights $-\infty$) explicitly, restarting the weights flat rather than dividing by zero.

Pitfall

The Jacobian at the wrong point, and the angle that is not wrapped. $\mathbf{F}_k$ is the derivative of f at the *previous updated* estimate $\hat{\mathbf{x}}_{k-1}$, from which the prediction starts; $\mathbf{H}_k$ is the derivative of h at the *predicted* estimate $\hat{\mathbf{x}}_k^-$, at which the innovation is formed. Swapping the two points costs a fraction of a meter per step in [Example 19.5](#) and, when the state moves fast, a divergent filter; evaluating $\mathbf{Q}(\mathbf{x})$ at the wrong point is the same mistake in disguise. Just as silent: an intruder crossing the negative x axis produces bearings that jump from $+3.1$ to -3.1 , and an unwrapped innovation of 6.2rad times a gain of -47 moves the estimate 290m ([Exercise 19.3](#)); the same happens to sigma-point images that straddle the seam and to particle likelihoods. Wrap in one function and call it everywhere.

Pitfall

Resampling every step, and too few particles. Resampling discards diversity; with a small process noise a filter that resamples every step collapses onto a few distinct states within seconds ([Section 19.6.2](#)). Resample when $N_{\text{eff}} < N/2$ and watch the sawtooth of [Figure 19.5](#): a curve that stays near N means uninformative measurements, one that stays near 1 a lost track. And a cloud of 200 particles that tracks a target in the plane fails in the five-dimensional coordinated-turn state, because the fraction of particles near the truth shrinks with every dimension ([Theorem 19.13](#) hides this in c_k). Start with N in the low thousands for five or six states, check that the RMSE has stopped improving with N ([Exercise 19.8](#)), and use a Rao–Blackwellized filter or an IMM when it has not.

19.10 Where this fits in the drone system

In the drone system

The three filters are the prediction layer’s answer to *maneuvering* intruders in the hybrid architecture of [Chapter 24](#): one filter per tracked drone turns its range–bearing or position measurements into an estimate with speed, heading and turn rate for the local safety layer ([Chapters 13 and 14](#)) and a covariance for the trajectory predictor of [Chapter 20](#). The EKF is the default for a turning intruder at long range; the UKF replaces it when the sensor is close or noisy or the motion model has no Jacobians. The particle filter is the natural input to the *prediction-aware* constraints of [Chapter 24](#): its cloud, pushed H steps through the motion model, is a set of weighted predicted trajectories, and “stay clear of the intruder with probability $1 - \delta$ ” becomes “stay clear of the $(1 - \delta)$ fraction of the particles”, which handles two lobes behind a building without inflating one ellipse over the whole block. In the study plan ([Appendix A](#)) this chapter completes Week 9; its coding exercise is [Exercise 19.8](#).

19.11 Summary

Chapter summary

- A turning intruder (coordinated-turn model, $\mathbf{x} = (x, y, v, \psi, \omega)$) and a range-bearing sensor make f and h nonlinear; a Gaussian pushed through a bent function is a banana, not an ellipse.
- The EKF linearizes, $\mathbf{F}_k = \partial f / \partial \mathbf{x}$ at $\hat{\mathbf{x}}_{k-1}$ and $\mathbf{H}_k = \partial h / \partial \mathbf{x}$ at $\hat{\mathbf{x}}_k^-$, then runs the Kalman equations; it is cheap and accurate when the belief is narrow compared with the curvature, its mean error is the dropped second-order term, and a poor start can make it diverge.
- The UKF pushes $2n + 1$ sigma points through the true models and recombines them with the weights $W^{(m)}$, $W^{(c)}$: no Jacobians, exact for affine models, second-order accurate mean, less than twice the cost.
- The bootstrap particle filter moves N samples through the motion model with sampled noise, weights them by the likelihood and resamples systematically when $N_{\text{eff}} = 1 / \sum_i (\omega^{(i)})^2 < N/2$; it represents multimodal beliefs and constraints at a cost linear in N and fails by degeneracy, impoverishment and too few particles.
- Wrap every angle difference, evaluate Jacobians and $\mathbf{Q}$ at the right point, keep $\mathbf{P}$ symmetric positive definite, keep log-weights, vectorize; and choose the cheapest filter whose assumptions hold: KF for linear models, EKF for mild nonlinearity, UKF for strong nonlinearity, particle filter for multimodality, IMM for switching maneuvers.

Further reading. Thrun, Burgard and Fox [168] develop all three filters in the notation of this chapter; Särkkä [144] is the concise mathematical treatment. The unscented transform is due to Julier and Uhlmann [74, 75], the parameters α , β , κ and the square-root form to Wan and van der Merwe [174]. The bootstrap filter is Gordon, Salmond and Smith [57]; Arulampalam et al. [6] is the standard tutorial with systematic resampling and the variants; Doucet, de Freitas and Gordon [36] collect the theory. Bar-Shalom, Li and Kirubarajan [7] cover the coordinated-turn model and the IMM filter.

19.12 Exercises

Exercise 19.1 (★). For each combination say whether f , h , both or neither is nonlinear and which filter of Chapter 18 or this chapter you would use, with one reason: (a) constant-velocity motion, position measurements; (b) constant-velocity motion, range-bearing measurements; (c) coordinated-turn motion, position measurements; (d) coordinated-turn motion, range-bearing measurements; (e) a turn with a *known, fixed* rate ω and state $(x, \dot{x}, y, \dot{y})$, position measurements.

Exercise 19.2 (★★). Derive the Jacobian Equation (19.12) of the coordinated-turn model (Proposition 19.3) from Equation (19.4), including the six entries for $\omega \rightarrow 0$ as limits of the general expressions. Then check your result numerically: implement the model, compute a central finite difference with step 10^{-5} at $\mathbf{x} = (10, 20, 12, 0.7, \omega)$ for $\omega = 0.3$ rad/s and for $\omega = 0$, and compare with `CoordinatedTurnModel.jacobian`.

Exercise 19.3 (★). In Example 19.5 the unwrapped bearing innovation is 6.2117 rad. With the gain printed by `worked_example()` ($K_{12} = 3.21$, $K_{22} = -47.05$, $K_{42} = 2.56$), compute where the updated position and heading would have landed, and explain in one sentence why K_{22} is negative.

Exercise 19.4 (★★). For $n = 5$ compute λ , the distance of the sigma points from the mean in standard deviations, and the weights $W_0^{(m)}$, $W_0^{(c)}$ and $W_i^{(m)}$ for $(\alpha, \beta, \kappa) = (1, 2, 0)$, $(1, 2, -2)$ and $(10^{-3}, 2, 0)$. Which choices give a negative $W_0^{(m)}$? Verify that the weights $W^{(m)}$ sum to one in every case, and explain why the third choice is numerically dangerous when the images are computed in floating point.

Exercise 19.5 (★★). Let $r \sim \mathcal{N}(\mu_r, \sigma_r^2)$ and $\varphi \sim \mathcal{N}(\pi/2, \sigma_\varphi^2)$ be independent. Show that the exact mean of $(r \cos \varphi, r \sin \varphi)$ is $(0, \mu_r e^{-\sigma_\varphi^2/2})$ and evaluate it for the numbers of [Figure 19.3](#). Then compute the unscented-transform mean by hand with $n = 2$, $\alpha = 1$, $\beta = 2$, $\kappa = 1$ (five points, $\lambda = 1$) and compare it with the EKF mean $(0, \mu_r)$ and with the exact value.

Exercise 19.6 (★★). (a) Compute N_{eff} for the weight vectors $(0.25, 0.25, 0.25, 0.25)$, $(0.7, 0.1, 0.1, 0.1)$ and $(1, 0, 0, 0)$. (b) Prove that $1 \leq N_{\text{eff}} \leq N$ for any normalized weights and characterize the two equality cases. (c) Give a particle set with $N_{\text{eff}} = N$ whose every particle is 100 m from the truth, and say what this tells you about N_{eff} as a diagnostic. (d) Run [Algorithm 19.3](#) by hand on $(0.7, 0.1, 0.1, 0.1)$ with $u = 0.1$ and check the result against [Proposition 19.12](#).

Exercise 19.7 (★★★ Coding). Modify `occlusion_example()` in three ways and report, for each, the fraction of particles west and east of the building at $k = 15$ and N_{eff} at $k = 20$: (a) remove the map constraint; (b) resample at every step instead of below $N/2$; (c) use $N = 200$ particles. Explain each change in the light of [Section 19.6.2](#).

Exercise 19.8 (★★★ Coding). Track a turning drone with all three filters: simulate an intruder that flies the legs of [Section 19.5.4](#) (or your own), observe it with a range-bearing sensor, and run the EKF, the UKF and the bootstrap particle filter on the same measurement sequences. (a) Reproduce the baseline row of [Table 19.2](#) within Monte Carlo error. (b) Vary the bearing noise from 1° to 15° and plot the RMSE of the three filters against it; where does the UKF start to pay off? (c) Vary $N \in \{100, 300, 1\,000, 3\,000, 10\,000\}$ and plot the particle filter's RMSE and time per step against N . (d) Add a five-second occlusion during the second turn and report which filters recover. Optional: extend the state to three dimensions ([Section 19.8](#)) and repeat (c).

Trajectory Prediction: From Constant Velocity to LSTMs and Transformers

Learning objectives

- State the trajectory-prediction problem (observe T_{obs} positions, predict the next T_{pred}), define ADE, FDE, minADE_K and collision-based metrics exactly, and design an evaluation that cannot fool you.
- Implement and tune the baselines a learned model must beat: constant velocity, constant acceleration and Kalman-filter extrapolation, and explain why constant velocity is so hard to beat at short horizons.
- Write down the LSTM cell with its three gates, trace one step by hand, and build a sequence-to-sequence predictor with an agent-centered input frame, a step-by-step decoder, a Gaussian output, Adam and early stopping.
- Explain scaled dot-product attention, multi-head attention and positional encodings, and how attention across agents makes a predictor interaction-aware.
- Turn a predicted distribution into a time-indexed occupancy region and feed it to the local safety layer, the replanning layer and the controller of the hybrid system.

20.1 Why this matters for a drone swarm

The tracker of [Chapter 18](#) hands the planner a clean estimate of where an unknown drone *is*. That is not enough. A drone flying at 3 m/s covers 7.5 m in the 2.5 s that a swarm member needs to brake, turn and re-establish its formation, and the collision that matters is the one at the end of those 2.5 s. The local safety layer of [Chapter 24](#) therefore asks a different question: where will the intruder be at $t + 0.2$ s, $t + 0.4$ s, $\dots$, $t + 2.4$ s, and how sure are we? This is *trajectory prediction*, the second half of the prediction layer of the hybrid architecture.

The simplest answer, “where it is now plus its velocity times the elapsed time”, is the constant-velocity extrapolation that the Kalman filter of [Chapter 18](#) produces when its prediction step is run several times in a row. For a drone in steady flight it is exact, and at short horizons it is remarkably hard to beat: a widely discussed study [\[146\]](#) found that a carefully tuned constant-velocity model matches or beats many published neural predictors on the standard pedestrian benchmarks, and the survey of Rudenko et al. [\[141\]](#) places that result in context. The reason is not that learning is useless but that most of a trajectory *is* straight flight, and that learned models were often compared against a badly tuned baseline. The interesting cases are the others: the intruder banks into a turn, begins an evasive maneuver, or is one of several drones whose paths interact. There the last second of history reveals what

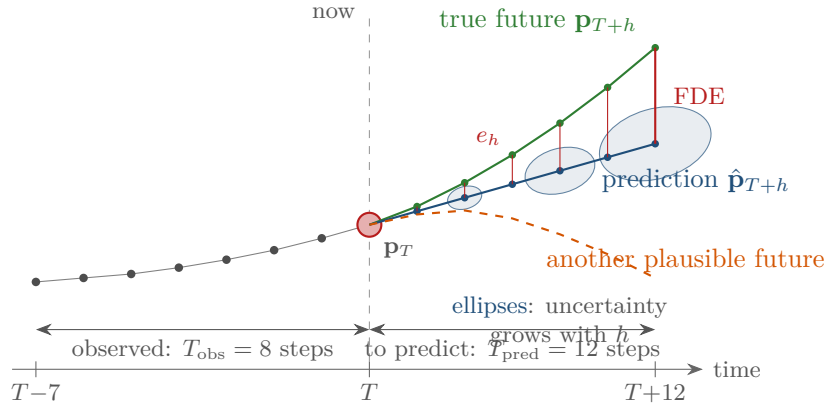


Figure 20.1. The prediction problem. The last $T_{\text{obs}} = 8$ positions of an intruder are observed; the next $T_{\text{pred}} = 12$ must be predicted. Blue is a prediction, green the future that actually happens, and the red segments are the displacement errors e_h whose average is the ADE and whose last value is the FDE. The ellipses grow with the horizon, and the dashed orange line is a second future that was just as plausible given the history: predictions are distributions, not lines.

happens next, and a model that has learned the typical shapes of maneuvers predicts the turn while the straight-line model flies off along the tangent.

The Week 10 milestone of the study plan ([Appendix A](#)) therefore needs a strong baseline and an honest measurement: you should be able to *quantify* whether learning improves prediction, per horizon and per kind of maneuver, before you let a learned predictor set the safety margins of a swarm. The chapter builds both, and it prepares what [Chapter 24](#) needs: not a single predicted path but a distribution over paths, turned into a growing ellipse per time step, so that ORCA ([Chapter 13](#)) can inflate the intruder’s radius, D* Lite ([Chapter 5](#)) can raise the cost of the cells it may occupy, and the MPC of [Chapter 21](#) can impose a chance constraint. [Section 20.8](#) is the experiment, run with the book’s own NumPy code, and [Section 20.9](#) proves the few results that make it interpretable.

20.2 The idea in plain words

[Figure 20.1](#) shows the situation. A drone has been observed at eight time steps, 0.2 s apart; we want the next twelve positions. Three things are visible in the picture, and they organize the chapter.

The past constrains the future — the drone will not jump, so its next position and velocity are near its last, which is all the constant-velocity model uses and for the first few steps enough. The past also reveals the maneuver: steadily rotating displacements mean a turn that will continue, a rotation that began two steps ago may be an evasive burst that lasts about a second, and which patterns occur and how they end is a property of the *population* of trajectories, learnable from recorded data by a recurrent network or by a Transformer. And the future is genuinely uncertain, part noise and part a choice the intruder has not made yet, so a single predicted line is wrong twice: it lies between the branches and it does not say how unsure it is ([Section 20.7](#)).

Key idea

Predict the next T_{pred} positions from the last T_{obs} , in the intruder’s own frame, and measure the error at every horizon. Constant velocity is the baseline that must be beaten; learned sequence models beat it where the history reveals a maneuver; nothing beats it

on straight flight at the long horizons that set the safety margin, or on a maneuver that has not started yet. Always predict a distribution, because the planner needs the spread as much as the mean.

20.3 Problem statement, metrics and evaluation protocol

Definition 20.1 (Trajectory prediction). A **trajectory** is a sequence of positions $\mathbf{p}_t \in \mathbb{R}^2$ (or $\mathbb{R}^3$) sampled every Δt seconds. At the current time T the **observation window** of length T_{obs} contains the positions $\mathbf{p}_{T-T_{\text{obs}}+1}, \dots, \mathbf{p}_T$ (in practice noisy measurements of them). A **predictor** maps the observation window, and possibly the windows of other agents and a map, to predicted positions $\hat{\mathbf{p}}_{T+1}, \dots, \hat{\mathbf{p}}_{T+T_{\text{pred}}}$ over the **prediction horizon** T_{pred} . The integer $h \in \{1, \dots, T_{\text{pred}}\}$ is the **horizon step**; $h \Delta t$ is the look-ahead time.

Three distinctions matter. A **single-agent** predictor sees one history at a time; an **interaction-aware** predictor also sees the neighbors, because drones and people avoid each other. A **deterministic** predictor outputs one trajectory; a **probabilistic** one outputs a distribution, as a mean and covariance per step or as K sampled trajectories, and a **multimodal** one can put mass on several distinct futures. Throughout the chapter $\Delta t = 0.2$ s, $T_{\text{obs}} = 8$ (1.6 s) and $T_{\text{pred}} = 12$ (2.4 s), the 8/12-step convention of the pedestrian benchmarks ETH [128] and UCY [101] at a sampling rate suited to drones.

Definition 20.2 (Displacement errors). For one trajectory with true future $\mathbf{p}_{T+h}$ and prediction $\hat{\mathbf{p}}_{T+h}$, the **displacement error** at step h is $e_h = \|\hat{\mathbf{p}}_{T+h} - \mathbf{p}_{T+h}\|$. The **average displacement error** and the **final displacement error** are

$$\text{ADE} = \frac{1}{T_{\text{pred}}} \sum_{h=1}^{T_{\text{pred}}} e_h, \quad \text{FDE} = e_{T_{\text{pred}}}, \quad (20.1)$$

averaged over the trajectories of a test set. The **per-horizon** versions are $\text{ADE}_h = \frac{1}{h} \sum_{j \leq h} e_j$ and $\text{FDE}_h = e_h$. For a predictor that outputs K sampled trajectories with errors $e_h^{(1)}, \dots, e_h^{(K)}$,

$$\text{minADE}_K = \min_k \frac{1}{T_{\text{pred}}} \sum_h e_h^{(k)}, \quad \text{minFDE}_K = \min_k e_{T_{\text{pred}}}^{(k)}, \quad (20.2)$$

again averaged over the test set. The **miss rate** is the fraction of trajectories with $\text{FDE} > \tau_{\text{miss}}$ for a threshold such as $\tau_{\text{miss}} = 1$ m.

The ADE rewards a prediction that is close on average; the FDE isolates the end of the horizon, where errors are largest and the planner's safety margin is decided. minADE_K is generous by construction: it scores a sampler by its *best* of K guesses, which measures whether the samples cover the true future but not how a planner should choose among them, so a minADE_{20} next to the ADE of a deterministic baseline compares two different things. Displacement errors also ignore physics: a prediction that passes through a wall has a small ADE if it ends near the truth. **Collision-based metrics** count the fraction of predicted trajectories that come closer than d_{min} to a static obstacle or to the true trajectory of another agent at the same time step:

$$\text{CR}_{d_{\text{min}}} = \frac{1}{|\mathcal{D}|} \sum_{n \in \mathcal{D}} \mathbb{I}[\exists h \leq T_{\text{pred}}, \exists j \neq i : \|\hat{\mathbf{p}}_{T+h}^i - \mathbf{p}_{T+h}^j\| < d_{\text{min}}], \quad (20.3)$$

where $\mathcal{D}$ is the test set and i the agent predicted in sample n . Scored against the true $\mathbf{p}^j$ the metric measures the predictor; scored against the other agents' *predictions* it measures a joint

sampler's self-consistency instead. [Chapter 25](#) measures the collision rate of the *planner* that consumes the predictions, the metric that finally matters.

An honest evaluation protocol. Prediction results are easy to inflate, mostly by accident. The protocol of this chapter has five rules.

1. Split the *scenes* (recordings) into training, validation and test sets, then cut every trajectory into windows of $T_{\text{obs}} + T_{\text{pred}}$ samples. Cutting first and splitting the shuffled windows puts the future of a training window into the past of a test window: the model has seen the answers. This is **data leakage**; split by scene, or by time for one long recording, and keep scenes with the maneuvers you care about in the test set.
2. Compute normalization statistics on the training set only: whole-data statistics leak the test set into the model, and the error then surfaces only when the drone flies.
3. Tune the baselines (the velocity window k , the process noise q) on the validation set with the same care as the learning rate; both change the ADE by a large factor ([Section 20.8](#)).
4. Select epochs and hyperparameters of the learned models on the validation set, never on the test set.
5. Report ADE_h and FDE_h as curves over h , per subset of the test set (maneuver type, density), with the miss rate and, for samplers, minADE_K with K stated; repeat over seeds and report the spread. An ADE at one unstated horizon hides whether a model is good at 0.4 s or at 2.4 s.

20.4 The baselines: constant velocity, constant acceleration, Kalman extrapolation

Algorithm 20.1. Physics baselines for a horizon of H steps

Input: observed positions $\mathbf{z}_1, \dots, \mathbf{z}_{T_{\text{obs}}}$ (noisy), step Δt , horizon H , window k , filter matrices $\mathbf{F}, \mathbf{Q}$

Output: predicted positions $\hat{\mathbf{p}}_1, \dots, \hat{\mathbf{p}}_H$ (and covariances $\Sigma_1, \dots, \Sigma_H$ for the filter)

1 **Function** ConstantVelocity($\mathbf{z}, H, k$):

2 $\hat{\mathbf{v}} \leftarrow (\mathbf{z}_{T_{\text{obs}}} - \mathbf{z}_{T_{\text{obs}}-k}) / (k \Delta t)$ // mean velocity over the last k intervals

3 **return** $\hat{\mathbf{p}}_h = \mathbf{z}_{T_{\text{obs}}} + h \Delta t \hat{\mathbf{v}}$ for $h = 1, \dots, H$

4 **Function** ConstantAcceleration($\mathbf{z}, H, k$):

5 $\mathbf{v}_1 \leftarrow (\mathbf{z}_{T_{\text{obs}}} - \mathbf{z}_{T_{\text{obs}}-k}) / (k \Delta t)$; $\mathbf{v}_0 \leftarrow (\mathbf{z}_{T_{\text{obs}}-k} - \mathbf{z}_{T_{\text{obs}}-2k}) / (k \Delta t)$

6 $\hat{\mathbf{a}} \leftarrow (\mathbf{v}_1 - \mathbf{v}_0) / (k \Delta t)$; $\hat{\mathbf{v}} \leftarrow \mathbf{v}_1 + \frac{1}{2} \hat{\mathbf{a}} k \Delta t$ // $\mathbf{v}_1$ is the velocity at the midpoint

7 **return** $\hat{\mathbf{p}}_h = \mathbf{z}_{T_{\text{obs}}} + h \Delta t \hat{\mathbf{v}} + \frac{1}{2} (h \Delta t)^2 \hat{\mathbf{a}}$ for $h = 1, \dots, H$

8 **Function** KalmanExtrapolate($\mathbf{z}, H, \mathbf{F}, \mathbf{Q}$):

9 initialize $\hat{\mathbf{x}} \leftarrow (\mathbf{z}_2, (\mathbf{z}_2 - \mathbf{z}_1) / \Delta t)$ and $\mathbf{P}$ from the measurement noise $\sigma^2 \mathbf{I}$

10 **for** $t = 3, \dots, T_{\text{obs}}$ **do**

11 run one predict and one update step of the constant-velocity Kalman filter of [Chapter 18](#) with $\mathbf{F}, \mathbf{Q}$ of [Equation \(20.4\)](#), $\mathbf{H} = (\mathbf{I} \ 0)$, $\mathbf{R} = \sigma^2 \mathbf{I}$ and measurement $\mathbf{z}_t$

12 **for** $h = 1, \dots, H$ **do**

13 $\hat{\mathbf{x}} \leftarrow \mathbf{F} \hat{\mathbf{x}}$; $\mathbf{P} \leftarrow \mathbf{F} \mathbf{P} \mathbf{F}^\top + \mathbf{Q}$ // predict step only: no measurement

14 $\hat{\mathbf{p}}_h \leftarrow \mathbf{H} \hat{\mathbf{x}}$; $\Sigma_h \leftarrow \mathbf{H} \mathbf{P} \mathbf{H}^\top$

15 **return** $(\hat{\mathbf{p}}_h, \Sigma_h)_{h=1}^H$

Algorithm 20.1 lists the three predictors that every learned model must beat; Figure 20.2 shows them on a gently turning test trajectory of the experiment in Section 20.8.

Constant velocity (CV). The velocity is estimated from the last k intervals (line 2) and held constant. With $k = 1$ the estimate uses the last two measurements only and is very noisy: the velocity estimate inherits an error of standard deviation $\sigma\sqrt{2}/(k\Delta t)$ per axis, which the look-ahead $h\Delta t$ turns into a position error growing like $h\sigma/k$ — the step Δt cancels (Proposition 20.8). A larger k averages the noise but lags behind a turn. The window k is the tuning parameter of the baseline and must be chosen on the validation set; in the experiment below the naive $k = 1$ is 88 % worse than the tuned $k = 2$ on straight flight. This is the *poorly tuned baseline* that many papers beat.

Constant acceleration (CA). Two successive velocity estimates give an acceleration (line 6). On a circular turn the acceleration points to the center and rotates with the drone, so extrapolating it as a constant is right for a step or two and then overshoots: the CA prediction in Figure 20.2 cuts inside the turn and ends 4 m from the truth, further than either CV prediction. The second difference also amplifies the measurement noise: its stencil $(1, -2, 1)$ has squared norm 6 against the first difference’s 2, and extrapolated to $h = 12$ this becomes a root-mean-square position error 4.7 times that of constant velocity at the same window $k = 3$, so Proposition 20.8 predicts a mean final error of $\sqrt{\pi}/2 \cdot 4.7 \cdot 0.45 = 1.88$ m on straight flight against the 1.869 m measured in Table 20.2, and CA loses there (0.830 against 0.335 m ADE). “More physics” in the extrapolation is not automatically better.

Kalman extrapolation (KF). The constant-velocity Kalman filter of Chapter 18, with state $\mathbf{x} = (\mathbf{p}, \mathbf{v}) \in \mathbb{R}^4$, transition $\mathbf{F} = \begin{pmatrix} \mathbf{I} & \Delta t \mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}$, measurement $\mathbf{H} = (\mathbf{I} \ \mathbf{0})$, measurement noise $\mathbf{R} = \sigma^2 \mathbf{I}$ and the white-noise-acceleration process noise

$$\mathbf{Q} = q \begin{pmatrix} \frac{1}{3}\Delta t^3 \mathbf{I} & \frac{1}{2}\Delta t^2 \mathbf{I} \\ \frac{1}{2}\Delta t^2 \mathbf{I} & \Delta t \mathbf{I} \end{pmatrix}, \quad (20.4)$$

is run over the observed window and then *extrapolated*: the predict step is repeated H times without an update (line 13). The mean is a constant-velocity extrapolation of the filtered state, so the filter is a CV model whose velocity estimate weighs the whole window optimally under its noise model; under that model the filtered mean is the conditional mean of the state given the measurements (Chapter 18), hence the minimum-mean-square-error predictor (Theorem 20.9), and its covariance Σ_h , which grows with h , is the first prediction in this chapter that comes with an uncertainty. Small q trusts the straight-line model and smooths the noise, large q follows the last measurements and reacts to turns. None of the three baselines knows that turns end, that evasive maneuvers have a typical duration, or that two drones on a collision course will both deviate. That knowledge has to come from data.

20.5 Sequence models: the LSTM and sequence-to-sequence prediction

20.5.1 The LSTM cell

A **recurrent neural network** reads a sequence one element at a time and keeps a state vector that summarizes what it has read. The simplest form updates a hidden state $\mathbf{h}_t \in \mathbb{R}^D$ as $\mathbf{h}_t = \tanh(\mathbf{W}[\mathbf{x}_t; \mathbf{h}_{t-1}] + \mathbf{b})$, where $[\mathbf{x}_t; \mathbf{h}_{t-1}]$ is the concatenation of the current input and the previous state. Training it means propagating the loss gradient backward through every

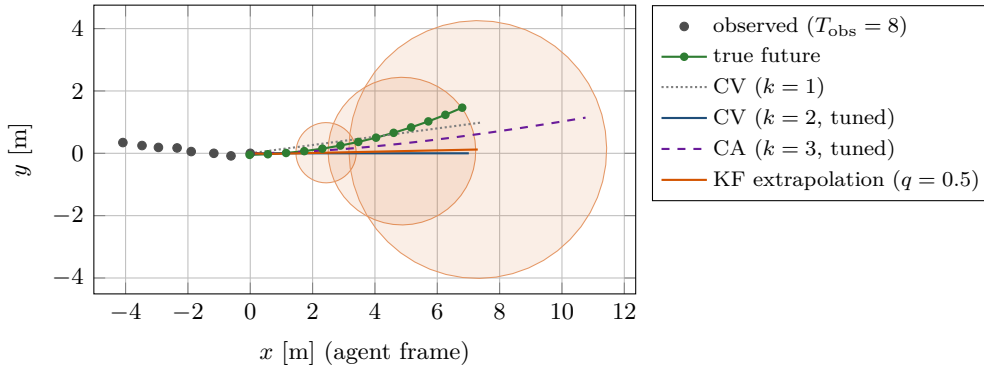


Figure 20.2. The physics baselines on a gently turning test trajectory (turn rate 0.17 rad/s, speed 2.9 m/s), plotted in the agent frame of Section 20.5.2. Both CV predictions leave the turn along a tangent; the constant-acceleration prediction (dashed) extrapolates the rotating centripetal acceleration as a constant and overshoots, ending 4.0 m from the truth against 1.5 m for the tuned CV. The Kalman extrapolation is a smoothed CV prediction with a 95 % ellipse at $h = 4, 8, 12$. On this one trajectory the naive $k = 1$ happens to beat the tuned $k = 2$; single examples prove nothing, which is why Table 20.2 averages over 800.

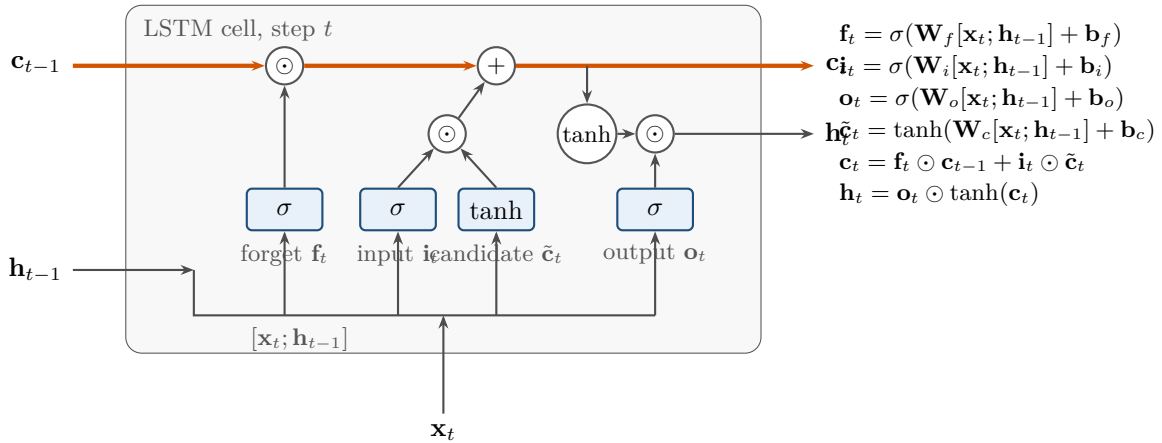


Figure 20.3. One step of the LSTM cell. The cell state $\mathbf{c}$ runs along the top and is changed only by a multiplication with the forget gate and an addition of the gated candidate; the hidden state $\mathbf{h}$ is a gated, squashed read-out of it. All four gate vectors are computed from the same concatenated input $[\mathbf{x}_t; \mathbf{h}_{t-1}]$ with their own weights.

step (*backpropagation through time*, BPTT [176]), and each step multiplies the gradient by the Jacobian of the update, a matrix whose norm is typically far from 1. After twenty steps the product has either vanished or exploded, and the network cannot learn that something it saw twenty steps ago matters now. Hochreiter and Schmidhuber [64] solved this with the **long short-term memory** (LSTM) cell of Figure 20.3: a second state, the **cell state** $\mathbf{c}_t$, that is changed only by *gated* additions, so that the gradient can flow through it unchanged. The forget gate, used throughout, was added by Gers, Schmidhuber and Cummins [54].

Definition 20.3 (LSTM cell). An LSTM cell with input size M and D hidden units has weights $\mathbf{W}_f, \mathbf{W}_i, \mathbf{W}_o, \mathbf{W}_c \in \mathbb{R}^{D \times (M+D)}$ and biases $\mathbf{b}_f, \mathbf{b}_i, \mathbf{b}_o, \mathbf{b}_c \in \mathbb{R}^D$. Given the input $\mathbf{x}_t \in \mathbb{R}^M$ and the previous states $\mathbf{h}_{t-1}, \mathbf{c}_{t-1} \in \mathbb{R}^D$, one step computes, with $\mathbf{z}_t = [\mathbf{x}_t; \mathbf{h}_{t-1}]$,

the logistic function $\sigma(a) = 1/(1 + e^{-a})$ and $\odot$ the element-wise product,

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{z}_t + \mathbf{b}_f) \quad \text{forget gate,} \quad (20.5)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{z}_t + \mathbf{b}_i) \quad \text{input gate,} \quad (20.6)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{z}_t + \mathbf{b}_o) \quad \text{output gate,} \quad (20.7)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{z}_t + \mathbf{b}_c) \quad \text{candidate values,} \quad (20.8)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad \text{cell state,} \quad (20.9)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \text{hidden state.} \quad (20.10)$$

We write $(\mathbf{h}_t, \mathbf{c}_t) = \text{LSTMCell}(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1})$.

In Equations (20.5) to (20.10), and only there, $\sigma(\cdot)$ is this squashing function applied entry by entry; everywhere else in the chapter σ is a standard deviation, as in the measurement noise σ of Section 20.4 and the predicted σ_h of Equation (20.13).

The two nonlinearities have different jobs. The logistic function maps every pre-activation into $(0, 1)$, so the gates are soft switches: a forget-gate entry near 1 keeps that component of the memory, near 0 erases it; the input gate decides how much of the candidate is written and the output gate how much of the memory is exposed. The hyperbolic tangent maps into $(-1, 1)$ and produces the *values*: the candidate that may be written and the squashed memory that may be read. Because Equation (20.9) is a gated sum rather than a squashed product, the derivative of $\mathbf{c}_t$ with respect to $\mathbf{c}_{t-1}$ along the direct path is exactly $\text{diag}(\mathbf{f}_t)$ (Proposition 20.11); with forget gates near 1 the gradient neither vanishes nor explodes, which is why gating helps.

Example 20.4 (One LSTM step by hand). Take a cell with $M = 2$ inputs and $D = 2$ hidden units, the weights

$$\begin{aligned} \mathbf{W}_f &= \begin{pmatrix} 0.5 & -0.5 & 0.3 & 0.1 \\ 0.2 & 0.4 & -0.3 & 0.6 \end{pmatrix}, & \mathbf{W}_i &= \begin{pmatrix} 1.0 & 0.0 & 0.2 & -0.4 \\ -0.5 & 0.8 & 0.1 & 0.3 \end{pmatrix}, \\ \mathbf{W}_o &= \begin{pmatrix} 0.3 & 0.3 & -0.6 & 0.2 \\ 0.7 & -0.2 & 0.4 & 0.5 \end{pmatrix}, & \mathbf{W}_c &= \begin{pmatrix} 0.9 & -0.6 & 0.0 & 0.8 \\ -0.4 & 0.5 & 0.7 & -0.1 \end{pmatrix}, \end{aligned}$$

$\mathbf{b}_f = (1, 1)$ (the usual initialization that starts by remembering) and all other biases zero. The input is $\mathbf{x}_t = (1.0, -0.5)$, the previous states are $\mathbf{h}_{t-1} = (0.2, -0.1)$ and $\mathbf{c}_{t-1} = (0.5, -0.3)$, so $\mathbf{z}_t = (1.0, -0.5, 0.2, -0.1)$. Table 20.1 traces the step; every number is produced by `lstm_cell_example()` in `ch20_prediction.py`. For the first unit the forget pre-activation is $0.5 \cdot 1.0 + (-0.5)(-0.5) + 0.3 \cdot 0.2 + 0.1 \cdot (-0.1) + 1 = 1.80$, so $f_1 = \sigma(1.80) = 0.8581$: the unit keeps 86% of its old memory 0.5 and adds $i_1 \tilde{c}_1 = 0.6029$, giving $c_1 = 1.0319$, of which the output gate lets half through as $h_1 = 0.3893$. The second unit has a small input gate (0.2870) and a negative candidate, so its memory moves only from -0.3 to -0.3447 .

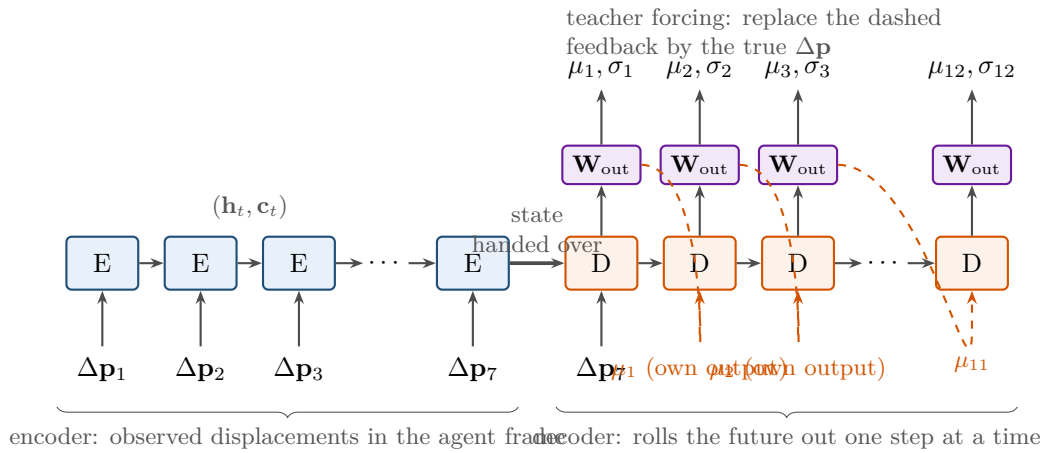
20.5.2 A sequence-to-sequence predictor

Figure 20.4 shows how LSTM cells become a trajectory predictor. An **encoder** LSTM reads the observed history one step at a time; its final states $(\mathbf{h}, \mathbf{c})$ summarize the history. A **decoder** LSTM starts from those states and rolls the future out one step at a time: at every step a linear read-out turns its hidden state into the next displacement, which is fed back as the next input. Algorithm 20.2 is the forward pass; four design decisions make it work.

Input normalization: the agent-centered frame. The network should not have to learn that trajectories near $x = 40$ m behave like those near $x = -10$ m, or that a drone

Table 20.1. Trace of one LSTM step on [Example 20.4](#): pre-activations, gate values (all in $(0, 1)$, the candidate in $(-1, 1)$) and the new states.

	pre-activation a		activation		state	
	unit 1	unit 2	unit 1	unit 2	unit 1	unit 2
forget gate $\mathbf{f}_t$	1.80	0.88	0.8581	0.7068		
input gate $\mathbf{i}_t$	1.08	-0.91	0.7465	0.2870		
output gate $\mathbf{o}_t$	0.01	0.83	0.5025	0.6964		
candidate $\tilde{\mathbf{c}}_t$	1.12	-0.50	0.8076	-0.4621		
cell state $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$					1.0319	-0.3447
$\tanh(\mathbf{c}_t)$					0.7747	-0.3316
hidden state $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$					0.3893	-0.2309

**Figure 20.4.** The sequence-to-sequence predictor unrolled in time. The encoder cells (blue) read the seven observed displacements in the agent frame; their final state is handed to the decoder cells (orange), which produce, through the read-out $\mathbf{W}_{\text{out}}$, the mean and standard deviation of the next displacement. Each decoder step is fed the mean of the previous one (dashed); teacher forcing replaces this feedback by the true displacement during training.

heading north turns like one heading east; positions are arbitrary and the physics is invariant to translation and rotation. The inputs are therefore *displacements* in a frame whose origin is the last observed position and whose x -axis points along the last observed heading φ_T , divided by a typical step length s so that they are of order one:

$$\mathbf{x}_t = \frac{1}{s} \mathbf{R}(-\varphi_T) (\mathbf{p}_t - \mathbf{p}_{t-1}), \quad t = T - T_{\text{obs}} + 2, \dots, T, \quad (20.11)$$

with $\mathbf{R}(\cdot)$ a rotation matrix. The targets, the future displacements, are transformed the same way; predicted displacements are summed, rescaled and rotated back into world positions. Constant velocity is then a fixed, simple pattern: the inputs are all $\approx (\bar{x}, 0)$ and the answer is to repeat them. Line 8 makes this explicit by adding the mean observed displacement $\bar{\mathbf{x}}$ to every decoder output, so that a decoder that outputs zero reproduces the constant-velocity prediction and the network only learns the *deviation* from it. In [Section 20.8](#) the same network trained on absolute positions is worse than the constant-velocity baseline.

Rolling the decoder out. The decoder's input at step h is a displacement, and where it comes from defines three modes of the same network. In **free-running** mode it is the mean μ_{h-1} the decoder produced one step earlier (line 14); this is how the predictor is used at run

Algorithm 20.2. Sequence-to-sequence prediction with an encoder and a decoder LSTM

Input: normalized observed displacements $\mathbf{x}_1, \dots, \mathbf{x}_{T_{\text{obs}}-1}$, horizon H , mode $\in \{\text{free-running, teacher forcing, sampling}\}$, true future $\mathbf{y}_{1:H}$ (teacher forcing only)

Output: means $\mu_{1:H}$ and standard deviations $\sigma_{1:H}$ of the future displacements

```

1 Function Seq2SeqPredict( $\mathbf{x}_{1:T_{\text{obs}}-1}, H, \text{mode}$ ):
2    $\mathbf{h}, \mathbf{c} \leftarrow \mathbf{0}$ ;  $\bar{\mathbf{x}} \leftarrow \text{mean of the } \mathbf{x}_t$  // the constant-velocity displacement
3   for  $t = 1, \dots, T_{\text{obs}} - 1$  do
4      $(\mathbf{h}, \mathbf{c}) \leftarrow \text{LSTMCell}_{\text{enc}}(\mathbf{x}_t, \mathbf{h}, \mathbf{c})$ 
5    $\mathbf{u} \leftarrow \mathbf{x}_{T_{\text{obs}}-1}$  // first decoder input: the last observed displacement
6   for  $h = 1, \dots, H$  do
7      $(\mathbf{h}, \mathbf{c}) \leftarrow \text{LSTMCell}_{\text{dec}}(\mathbf{u}, \mathbf{h}, \mathbf{c})$ 
8      $(\mu_h, \log \sigma_h) \leftarrow \mathbf{W}_{\text{out}} \mathbf{h} + \mathbf{b}_{\text{out}}$ ;  $\mu_h \leftarrow \mu_h + \bar{\mathbf{x}}$  // residual over constant velocity
9     if  $\text{mode} = \text{teacher forcing}$  then
10       $\mathbf{u} \leftarrow \mathbf{y}_h$ 
11     else if  $\text{mode} = \text{sampling}$  then
12       $\mathbf{u} \leftarrow \mu_h + \sigma_h \odot \varepsilon, \varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
13     else
14       $\mathbf{u} \leftarrow \mu_h$ 
15   return  $(\mu_{1:H}, \sigma_{1:H})$ ; positions are the cumulative sums of the means, mapped back
      to the world frame
  
```

time. In **teacher forcing** mode, used during training, it is the *true* displacement $\mathbf{y}_{h-1}$: the decoder is told the correct answer for the previous step and only has to predict the next one, which makes early training fast and stable because errors do not accumulate along the rollout. Its weakness is **exposure bias**: trained on perfect inputs and evaluated on its own imperfect ones, the decoder can learn to copy its input, a shortcut that works under teacher forcing and fails in free-running mode (Figure 20.8). The remedies are to train in free-running mode with the gradient flowing through the feedback, or to move from one to the other with a schedule [11]. In **sampling** mode the input is a sample from the predicted Gaussian: one plausible future per rollout.

The loss: squared error or Gaussian likelihood. With a read-out of two numbers per step the natural loss is the **mean squared error** of the predicted positions,

$$\mathcal{L}_{\text{MSE}} = \frac{1}{H} \sum_{h=1}^H \|\hat{\mathbf{p}}_{T+h} - \mathbf{p}_{T+h}\|^2, \quad (20.12)$$

averaged over the batch. It trains a predictor of the *conditional mean* (Theorem 20.9) and says nothing about uncertainty. With four numbers per step, a mean μ_h and a log standard deviation $\log \sigma_h$ of the next displacement, the loss becomes the **Gaussian negative log-likelihood** of the true displacements $\mathbf{y}_h$,

$$\mathcal{L}_{\text{NLL}} = \frac{1}{H} \sum_{h=1}^H \sum_{d \in \{x, y\}} \left[\log \sigma_{h,d} + \frac{(y_{h,d} - \mu_{h,d})^2}{2\sigma_{h,d}^2} \right] + \text{const}, \quad (20.13)$$

which is minimized by the conditional mean *and* the conditional standard deviation: the network learns to say “I do not know” where the training data are unpredictable, for instance

right after the onset of an evasive maneuver. Predicting $\log \sigma$ keeps σ positive without a constraint; a full covariance per step (a third number for the correlation) is a small extension, and the original Social LSTM used exactly such a bivariate Gaussian output [1]. The position covariance at horizon h is the sum of the displacement covariances up to h , which assumes independent step errors and, as Section 20.8 shows, underestimates the true spread.

Training. One epoch draws the training windows in random order, normalizes each mini-batch to the agent frame of Equation (20.11), runs the decoder in free-running (or teacher-forcing) mode, and takes one optimizer step per batch; the learning rate is interpolated geometrically from η_0 to η_E over the run. (Appendix B.4.6 reviews gradient descent and what the step size does to it.) The gradient is computed by BPTT through the decoder, the hand-over of the state and the encoder, and in free-running mode also through the fed-back means. The weights are updated with Adam [84], which rescales every parameter’s step by a running estimate of its gradient’s magnitude; the gradient is clipped to a maximum norm because one badly conditioned batch can otherwise throw the weights far away. **Early stopping** evaluates the free-running validation ADE after every epoch, keeps the best epoch’s weights and stops after p epochs without improvement: the one regularizer every trajectory predictor should have, and where the validation split earns its keep. `train_predictor` in `ch20_prediction.py` is the loop, line for line.

20.5.3 Interaction: the Social LSTM

A drone in a swarm, like a pedestrian in a crowd, moves according to its neighbors. Alahi et al. [1] made the LSTM predictor interaction-aware with **social pooling** (Figure 20.5). Every agent i has its own LSTM with hidden state $\mathbf{h}_{i,t}$. Around agent i lay a grid of $N_o \times N_o$ cells of side $2\rho/N_o$ and, for every neighbor j inside it, add its hidden state of the previous step to the cell in which it lies:

$$\mathbf{S}_{i,t}[m, n, :] = \sum_{j \neq i} \mathbb{I}[\mathbf{p}_{j,t-1} - \mathbf{p}_{i,t-1} \in \text{cell}(m, n)] \mathbf{h}_{j,t-1}. \quad (20.14)$$

The states of the *previous* step are pooled, so every agent’s cell can be stepped once per time step in any order; pooling the current states would make the recurrence circular. The tensor $\mathbf{S}_{i,t} \in \mathbb{R}^{N_o \times N_o \times D}$ is flattened, embedded and concatenated with the embedding of the agent’s own displacement as the input of its cell. Because every $\mathbf{h}_{j,t-1}$ summarizes the history of agent j , agent i sees not only where its neighbors are but where they are going; because the grid is centered on i the representation is translation invariant; because cells are summed, any number of neighbors fits. Social pooling is coarse: a neighbor on a cell boundary jumps between cells and the grid has a fixed range. Its successors replaced the grid by a maximum over all neighbors (Social GAN [58]), by a graph whose edges carry the relative state of each pair (Trajectron++ [143]) and, most generally, by attention.

20.6 Transformers and attention over agents

An LSTM reads the history in order and compresses it into one state vector; whatever the decoder needs from step 3 of the history must survive five updates. **Attention** removes the bottleneck: every output looks directly at every input and decides how much each one matters. Vaswani et al. [169] showed that a network built from attention alone, the **Transformer**, outperforms recurrent networks on sequence transduction, and Giuliani et al. [55] showed that the same architecture, applied to one trajectory at a time, beats the LSTM predictors on the pedestrian benchmarks.

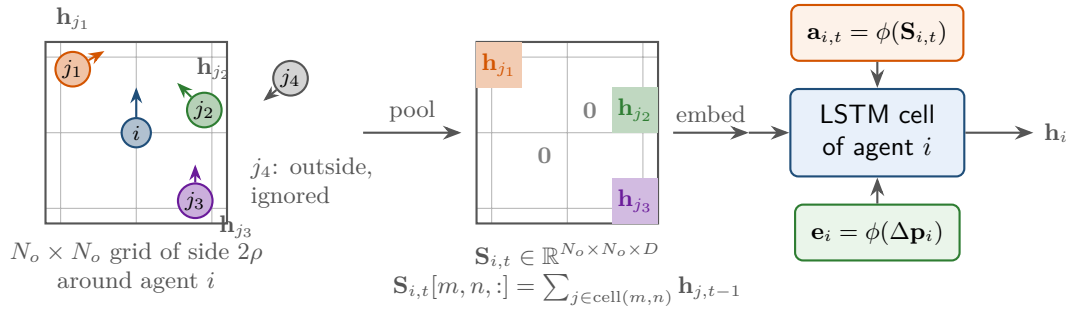


Figure 20.5. Social pooling. Left: a grid centered on agent i ; neighbors inside it (j_1, j_2, j_3) contribute their LSTM hidden states, a neighbor outside it (j_4) is ignored. Middle: the pooling tensor $\mathbf{S}_{i,t}$ holds, per cell, the sum of the previous hidden states $\mathbf{h}_{j,t-1}$ of the neighbors in it. Right: the embedded tensor is fed to agent i 's cell with the embedding of its own displacement.

Definition 20.5 (Scaled dot-product attention). Let $\mathbf{Q} \in \mathbb{R}^{n_q \times d_k}$ hold n_q **queries**, $\mathbf{K} \in \mathbb{R}^{n_k \times d_k}$ hold n_k **keys** and $\mathbf{V} \in \mathbb{R}^{n_k \times d_v}$ the **values** attached to the keys, one per row. Then

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \quad \alpha_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k})}{\sum_{j'} \exp(\mathbf{q}_i^\top \mathbf{k}_{j'} / \sqrt{d_k})}, \quad (20.15)$$

where the softmax acts on every row: the output for query i is $\sum_j \alpha_{ij} \mathbf{v}_j$, a convex combination of the values with the **attention weights** $\alpha_{ij} \geq 0$, $\sum_j \alpha_{ij} = 1$.

Three letters that mean something else in Part VI

In this section, and nowhere else in the book, $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ are the query, key and value matrices of attention: they have nothing to do with the process-noise covariance $\mathbf{Q}$ of Equation (20.4) and Chapters 18 and 19, with the measurement-noise covariance $\mathbf{R}$, with the Kalman gain $\mathbf{K}$, or with the cost weights $\mathbf{Q}$ and $\mathbf{R}$ of Chapter 21. The three attention matrices are data that the network builds from its own input; the covariances are properties of a motion model and a sensor.

The dot product $\mathbf{q}_i^\top \mathbf{k}_j$ scores how relevant input j is to output i ; the division by $\sqrt{d_k}$ keeps the scores of order one for random vectors, so that the softmax does not saturate at initialization. **Multi-head attention** runs n_h such attentions in parallel on learned projections of the same inputs,

$$\text{head}_m = \text{Attention}(\mathbf{X}\mathbf{W}_m^Q, \mathbf{X}\mathbf{W}_m^K, \mathbf{X}\mathbf{W}_m^V), \quad \text{MultiHead}(\mathbf{X}) = [\text{head}_1, \dots, \text{head}_{n_h}] \mathbf{W}^O, \quad (20.16)$$

with $\mathbf{W}_m^Q, \mathbf{W}_m^K, \mathbf{W}_m^V \in \mathbb{R}^{d \times d/n_h}$ and $\mathbf{W}^O \in \mathbb{R}^{d \times d}$, so that one head can attend to the last step, another to the onset of the current maneuver, a third to the nearest neighbor. A Transformer layer wraps the attention block in a position-wise feed-forward network, residual connections and layer normalization; see Vaswani et al. [169] and Goodfellow, Bengio, and Courville [56].

Positional encodings. Attention is a set operation: permuting the inputs permutes the outputs, and nothing in Equation (20.15) knows that step 7 comes after step 6. The order is injected by adding to the embedding of the input at step t a **positional encoding**

$$\text{PE}(t, 2i) = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad \text{PE}(t, 2i+1) = \cos\left(\frac{t}{10000^{2i/d}}\right), \quad i = 0, \dots, d/2 - 1, \quad (20.17)$$

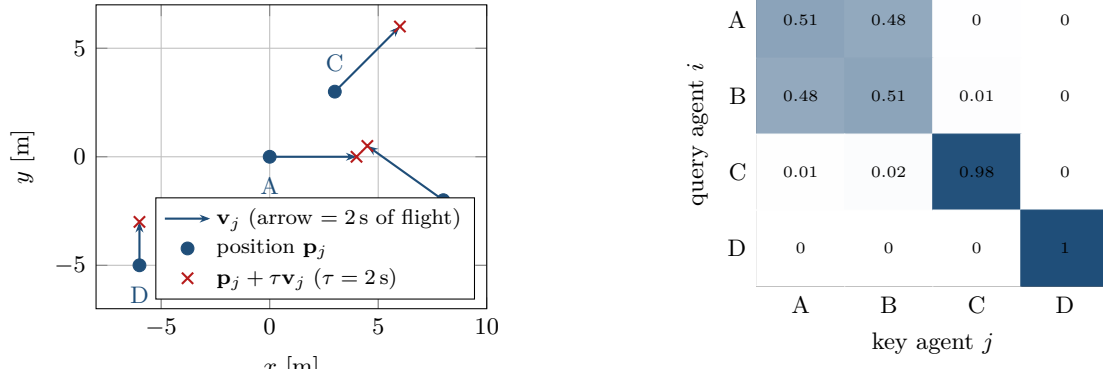


Figure 20.6. Attention across agents. Left: four drones with their velocities (arrows of 2 s of flight) and their positions extrapolated 2 s ahead (crosses); A and B are on a collision course, C passes nearby, D is far away. Right: the attention weights α_{ij} of Equation (20.15) with hand-set query and key maps that score the distance between the extrapolated positions (rows sum to one). Each drone attends to itself and to the drones it will meet; D attends to nobody but itself.

a vector of sinusoids of geometrically spaced frequencies. No two steps have the same vector, and the encoding of step $t + \delta$ is a fixed linear function of the encoding of step t (Exercise 20.5), so a head can learn to attend “two steps back” regardless of t .

An encoder-decoder Transformer for one trajectory. Giuliari et al. [55] use the original encoder-decoder layout: a stack of encoder layers with self-attention over the embedded, positionally encoded observed displacements, and an autoregressive decoder that at step k applies *masked* self-attention to the displacements predicted so far (a token attends only to earlier ones, so that training processes all H steps in one pass while staying causal), then *cross-attention* with the encoder outputs as keys and values, and a read-out. There is no recurrence, so every observed step is one attention hop away from every predicted step.

Attention across agents. Nothing in Equation (20.15) requires the keys to be time steps of the *same* agent. Let the tokens be the encoded histories of all N agents in the scene. The query of agent i scored against the keys of all agents gives weights α_{ij} that say how much agent j matters for predicting agent i , and $\sum_j \alpha_{ij} \mathbf{v}_j$ is a learned summary of the neighborhood: social pooling with a continuous notion of “nearby”. The graph attention of Trajectron++ [143] and joint attention over agents and time steps are the current state of the art for interaction-aware prediction. Figure 20.6 shows the mechanism on four drones. To keep the numbers reproducible without training, the query and key maps are set by hand in `toy_agent_attention`: with $\mathbf{x} = \mathbf{p} + \tau \mathbf{v}$ the position extrapolated $\tau = 2$ s ahead, the features $\mathbf{k}_j = (\mathbf{x}_j, \|\mathbf{x}_j\|^2, 1)$ and $\mathbf{q}_i = (\mathbf{x}_i, -\frac{1}{2}, -\frac{1}{2} \|\mathbf{x}_i\|^2) \sqrt{d_k}/\ell^2$ make the scaled dot product equal to $-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / (2\ell^2)$, so the weight is large for agents whose extrapolated positions are close. Drone A splits its attention almost equally between itself and drone B, which is on a collision course ($\alpha_{AB} = 0.48$), gives drone C, near but diverging, 0.003 and the distant drone D nothing. A trained model learns such maps from data, one per head.

What attention buys. Attention helps at long horizons because the decoder can consult any observed step directly instead of a compressed state, and it helps with many agents because the same mechanism ranks neighbors by learned relevance rather than by a grid. The next subsection builds the smallest predictor that uses both ideas and measures what they are worth here.

20.6.1 A Transformer predictor end to end

The primitives above are enough to build a predictor, and `ch20_prediction.py` contains one: an encoder-decoder Transformer in just under two hundred lines of NumPy, its gradient written by hand and checked against finite differences like the LSTM's. It is trained on the same 1 600 trajectories of [Example 20.6](#), with the same Adam, batches and budget, so that its row of [Table 20.2](#) differs from the LSTM's in the model and in nothing else.

The encoder embeds each of the $T_{\text{obs}} - 1 = 7$ observed displacements linearly into $\mathbb{R}^d$ with $d = 24$, adds the positional encoding of [Equation \(20.17\)](#), and applies one multi-head self-attention layer with $n_h = 2$ heads ([Equation \(20.16\)](#)) and a residual connection, followed by a residual position-wise block $\tanh(\cdot \mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2$ of width $2d$. It returns one context vector per observed step.

The decoder has neither recurrence nor autoregression. The $H = 12$ future steps are represented by their own positional encodings, mapped by a learned matrix into one query per predicted step,

$$\mathbf{Q}_{\text{dec}} = [\text{PE}(T_{\text{obs}}), \dots, \text{PE}(T_{\text{obs}} + H - 1)]^T \mathbf{W}_{\text{dec}}^Q \in \mathbb{R}^{H \times d}; \quad (20.18)$$

one cross-attention layer reads the encoder outputs with these queries, and a linear head maps each of the H results to one displacement, added to the constant-velocity displacement exactly as in [Section 20.5.2](#). All H displacements leave in a single pass: this is a **non-autoregressive decoder**, which has no exposure bias to pay but also no way to let step k see what step $k - 1$ predicted. Two things are left out on purpose: the layer is one layer, not a stack, and there is no layer normalization, which a stack needs and a single residual layer can do without.

Listing 20.1. The forward pass of the Transformer predictor (from `code/ch20_prediction.py`): embedding plus positional encoding, one encoder self-attention layer, the feed-forward block, the cross-attention decoder with one query per predicted step, and the linear head. `_split_heads` reshapes (B, T, d) into $(B, n_h, T, d/n_h)$ and `_merge_heads` inverts it.

```

1 Z = x @ self.W_emb + self.b_emb + self.pe[:T]      # embed + positional encoding
2 Q, K, V = Z @ self.Wq, Z @ self.Wk, Z @ self.Wv    # encoder self-attention
3 qh, kh, vh = _split_heads(Q, nh), _split_heads(K, nh), _split_heads(V, nh)
4 A = softmax(np.einsum("bhtd,bhsd->bhts", qh, kh) / math.sqrt(dh))
5 C = _merge_heads(np.einsum("bhts,bhsd->bhtd", A, vh))
6 Z1 = Z + C @ self.Wo                                # residual connection
7 Hff = np.tanh(Z1 @ self.Wf1 + self.bf1)
8 Z2 = Z1 + Hff @ self.Wf2 + self.bf2                 # position-wise feed-forward
9 Q2 = self.pe[T_OBS:T_OBS + P] @ self.Wq2           # one query per predicted step
10 K2, V2 = Z2 @ self.Wk2, Z2 @ self.Wv2             # decoder cross-attention
11 q2h = Q2.reshape(P, nh, dh).transpose(1, 0, 2)
12 k2h, v2h = _split_heads(K2, nh), _split_heads(V2, nh)
13 A2 = softmax(np.einsum("hpd,bhtd->bhpt", q2h, k2h) / math.sqrt(dh))
14 C2 = _merge_heads(np.einsum("bhpt,bhtd->bhpd", A2, v2h))
15 Z3 = Q2 + C2 @ self.Wo2
16 out = Z3 @ self.W_out.T + self.b_out + self.baseline(x)[: , None]
```

The model has 7 106 weights against the LSTM's 9 026, and trains with the same learning rate, batches, clipping and epoch budget; only the patience of the early stopping is 15 epochs rather than 10, because without layer normalization the validation curve is noisy enough that 10 stopped this initialization at epoch 12 while it was still improving. The run keeps epoch 24 of 39 and takes nine seconds.

The result is the last row of [Tables 20.2](#) and [20.3](#) and the pentagon curve of [Figure 20.7](#): an ADE of 0.831 m and an FDE of 1.758 m over the test set, 27% below the tuned constant velocity, one centimeter above LSTM-NLL's 0.821 m and four above LSTM-MSE's 0.789 m.

The pattern per subset is the LSTM's: it halves the error on turns (0.504 against the tuned CV's 1.125 m), takes a third off the visible evasive maneuvers (1.594 against 2.351 m), loses to constant velocity on straight flight (0.470 against 0.335 m) and is level with everything else on the hidden maneuvers (1.639 m). Per horizon it tracks the LSTM within three centimeters, and its miss rate of 0.512 sits between LSTM-NLL's 0.520 and LSTM-MSE's 0.436.

What this experiment shows, and what it does not. It shows that the attention block, the positional encoding and a linear head are already a complete predictor: an exact hand-written gradient, nine seconds of training, and a model that beats every physics baseline on the maneuvers it can see. It does *not* show that Transformers predict trajectories better than recurrent networks. Three reasons not to read it that way.

- *Measure the spread before believing a gap.* `seed_study()` retrains all three learned models with three initializations on the same split: LSTM-NLL 0.824 ± 0.004 m, Transformer 0.821 ± 0.012 m, LSTM-MSE 0.794 ± 0.005 m. The Transformer and the Gaussian LSTM are indistinguishable, and the squared-error LSTM is ahead of both by more than twice the largest spread. The single-seed column of Table 20.2 would have supported either conclusion.
- *The bottleneck this architecture removes is not binding here.* Attention pays when the decoder must reach far back into a long history; ours is seven steps, which one LSTM state carries easily. Giuliani et al. [55] report their advantage with longer histories, thousands of recorded scenes and a stack of layers — three differences, none of them reproduced here.
- *The model is small on purpose:* one layer, no layer normalization, a parallel decoder and a squared-error head, so it has no uncertainty to report and its calibration column is empty. Each of those is a small change to the file and each costs data; Exercise 20.9(d) asks you to make them one at a time and to check whether any moves the numbers beyond that spread.

The general bills are the same. *Data:* more parameters and fewer built-in assumptions than an LSTM means more trajectories are needed to fit them, so on a few hundred recorded maneuvers a small LSTM, or constant velocity, remains the better choice. *Compute:* the quadratic term of Proposition 20.12 is harmless for the 7×7 and 12×7 score matrices here and bites for a planner that attends over all agents and all time steps every 0.1 s. *Engineering:* one layer fits in a listing, a stack wants a framework (Section 20.11). What attention buys even here is readable weights: averaged over the test set the decoder queries put 85 % of theirs on the first four of the seven observed displacements. The last displacements are already in the constant-velocity term the head adds, so what the attention has to supply is how the heading has *changed* across the window.

20.7 Multimodality and uncertainty

Theorem 20.9 states the problem precisely: a predictor trained with the squared error converges to the conditional mean of the future, so when the future has two distinct modes, left and right around an obstacle, the prediction runs through the obstacle. The Gaussian output of Equation (20.13) reports the disagreement as a large σ , which is honest but still unimodal. Three families of remedies exist.

Mixture density outputs. The read-out can produce, per step, the parameters of a **mixture of Gaussians** [18]: C weights π_c (a softmax over C logits), means μ_c and covariances Σ_c ,

with the loss

$$\mathcal{L}_{\text{MDN}} = -\log \sum_{c=1}^C \pi_c \mathcal{N}(\mathbf{y}; \mu_c, \Sigma_c), \quad (20.19)$$

computed with the log-sum-exp trick. At a fork the network learns two components with weights $\approx \frac{1}{2}$; on straight flight one takes all the weight. Mixtures are the cheapest form of multimodality and are used by Trajectron++ and many Transformer predictors.

Sampling-based models. A second approach feeds the decoder a random latent vector $\mathbf{z}$ and trains the network so that different $\mathbf{z}$ give different plausible futures. Social GAN [58] does this adversarially, with a discriminator that tells generated futures from real ones and a *variety loss* that back-propagates only the best of K samples; Trajectron++ [143] uses a conditional variational autoencoder with a discrete latent variable, so that each value of $\mathbf{z}$ is an interpretable mode, and integrates sampled controls through a dynamics model so that every sample is feasible. Both are scored with minADE_K and need a planner that can consume a set of samples. The sampling mode of Algorithm 20.2 is the poor man’s version: diverse rollouts from a unimodal per-step Gaussian.

Ensembles. Training several copies with different seeds and averaging their outputs improves accuracy, and their spread estimates the *model* uncertainty that the per-step σ cannot see [95].

From a distribution to an occupancy region. Whatever produces the distribution, the planner needs it as geometry. For a Gaussian prediction at step h with mean $\hat{\mathbf{p}}_{T+h}$ and covariance Σ_h , the set of positions the intruder occupies with probability p is the covariance ellipse of Definition 2.33,

$$\mathcal{E}_h(p) = \left\{ \mathbf{x} : (\mathbf{x} - \hat{\mathbf{p}}_{T+h})^\top \Sigma_h^{-1} (\mathbf{x} - \hat{\mathbf{p}}_{T+h}) \leq k_p^2 \right\}, \quad k_p = \sqrt{-2 \ln(1-p)}, \quad (20.20)$$

where k_p follows from the χ^2 distribution with two degrees of freedom ($k_{0.95} = 2.448$). The sequence $\mathcal{E}_1(p), \dots, \mathcal{E}_H(p)$ is the **time-indexed occupancy region** of the intruder. For a mixture, take the union of the ellipses of the components whose weight exceeds a threshold; for K samples, the smallest disks or grid cells that contain a fraction p of them at each step. Before any of this is trusted its **calibration** must be checked on held-out data: the fraction of true positions inside $\mathcal{E}_h(p)$ should be about p at every h . Learned predictors are often over-confident, the experiment below quantifies by how much, and the repair is a per-horizon inflation factor fitted on the validation set (Exercise 20.10).

20.8 A worked example: constant velocity against a learned predictor

The experiment of this chapter is the Week 10 exercise of the study plan, run end to end with `ch20_prediction.py` and the script `gen_ch20_results.py`; every number in this section is printed by that code.

Example 20.6 (Synthetic drone trajectories). A drone flies at a constant speed drawn from $[1.5, 4]$ m/s with a random initial heading; its heading changes at a turn rate that depends on the maneuver class: *straight flight* (40 % of the trajectories, turn rate 0), a *gentle turn* (30 %, constant rate $|\omega| \in [0.15, 0.45]$ rad/s) or an *evasive maneuver* (30 %, a burst of $|\omega| \in [0.8, 1.5]$ rad/s lasting 4–6 steps with an onset uniform in $\{2, \dots, 9\}$, straight otherwise). Positions are sampled every $\Delta t = 0.2$ s for 20 steps; the first $T_{\text{obs}} = 8$ are observed with Gaussian measurement noise of $\sigma = 0.05$ m, the last $T_{\text{pred}} = 12$ are the noise-free ground truth. A maneuver is *visible* if its onset lies inside the observation window and *hidden* if it

Table 20.2. ADE/FDE in meters at the full horizon ($h = 12, 2.4$ s) on the 800 test trajectories and on the four subsets; best value per column in bold. LSTM-abs is LSTM-NLL trained on absolute coordinates, LSTM-TF is LSTM-NLL trained with teacher forcing only. On the 60 hidden-maneuver trajectories all predictors lie within noise of each other, so the bold entry of that column singles out no winner. The last row is the Transformer of [Section 20.6.1](#).

Predictor	all	straight	turn	evasive, visible	evasive, hidden
CV, $k = 1$	1.232/2.623	0.632/1.125	1.163/2.690	2.142/4.417	1.838/4.687
CV, tuned $k = 2$	1.141/2.492	0.335/0.575	1.125/2.721	2.351/4.814	1.696/4.479
CA, tuned $k = 3$	1.322/3.056	0.830/1.869	0.939/2.233	2.495/5.617	1.768/4.588
KF, tuned $q = 0.5$	1.125/2.461	0.312/0.539	1.112/2.694	2.337/4.772	1.698/4.500
LSTM-NLL	0.821/1.751	0.427/0.959	0.549/1.037	1.585/3.204	1.598/4.209
LSTM-MSE	0.789/1.668	0.407/0.859	0.487/0.914	1.554/3.168	1.608/4.238
LSTM-abs	1.315/2.921	0.596/1.415	1.289/3.016	2.486/4.972	1.569/4.113
LSTM-TF	1.069/2.382	0.628/1.257	0.773/1.831	1.984/4.307	1.715/4.491
Transformer	0.831/1.758	0.470/0.951	0.504/0.968	1.594/3.317	1.639/4.277

Table 20.3. The same predictors at three horizons (ADE_{*h*} in meters on all test trajectories), the miss rate (FDE > 1 m), and the calibration of the two probabilistic predictors: the fraction of true positions inside their 95 % ellipse at $h = 4, 8$ and 12.

Predictor	ADE ₄ (0.8 s)	ADE ₈ (1.6 s)	ADE ₁₂ (2.4 s)	miss rate	inside 95 % ellipse, $h = 4/8/12$
CV, $k = 1$	0.360	0.751	1.232	0.775	–
CV, tuned	0.304	0.676	1.141	0.618	–
CA, tuned	0.320	0.744	1.322	0.860	–
KF, tuned	0.298	0.665	1.125	0.611	0.850/0.807/0.787
LSTM-NLL	0.227	0.496	0.821	0.520	0.762/0.632/0.544
LSTM-MSE	0.227	0.481	0.789	0.436	–
LSTM-abs	0.364	0.774	1.315	0.870	–
LSTM-TF	0.281	0.624	1.069	0.681	–
Transformer	0.252	0.511	0.831	0.512	–

begins after it. One fixed seed draws 1600 training, 400 validation and 800 test trajectories; the test set contains 317 straight flights, 236 turns, 187 visible and 60 hidden maneuvers.

The baselines follow [Algorithm 20.1](#), with parameters tuned on the validation set by minimizing the ADE over a small grid: CV window $k = 2$ (the naive $k = 1$ is kept as a second row), CA window $k = 3$, Kalman process noise $q = 0.5$ with $r = \sigma = 0.05$ m. The learned predictor is [Algorithm 20.2](#) with $D = 32$ hidden units in each cell, 9 026 weights, trained for 40 epochs with batches of 64, learning rate $5 \cdot 10^{-3}$ decaying to $5 \cdot 10^{-4}$, clipping at norm 5 and patience 10; each training takes about ten seconds on one processor core. **LSTM-NLL** has the Gaussian read-out of [Equation \(20.13\)](#) and **LSTM-MSE** the loss of [Equation \(20.12\)](#); both use the agent frame, the residual over constant velocity and free-running training. **LSTM-abs** is LSTM-NLL trained on absolute world positions and **LSTM-TF** is LSTM-NLL trained with teacher forcing in every epoch: two pitfalls with numbers. The last row of each table is the Transformer of [Section 20.6.1](#). All numbers below come from the one fixed seed of [Example 20.6](#); [Exercise 20.9](#) asks you to repeat the training over seeds and report the spread, which rule 5 of [Section 20.3](#) demands of any published comparison. [Tables 20.2](#) and [20.3](#) and [Figure 20.7](#) contain the results; read them subset by subset.

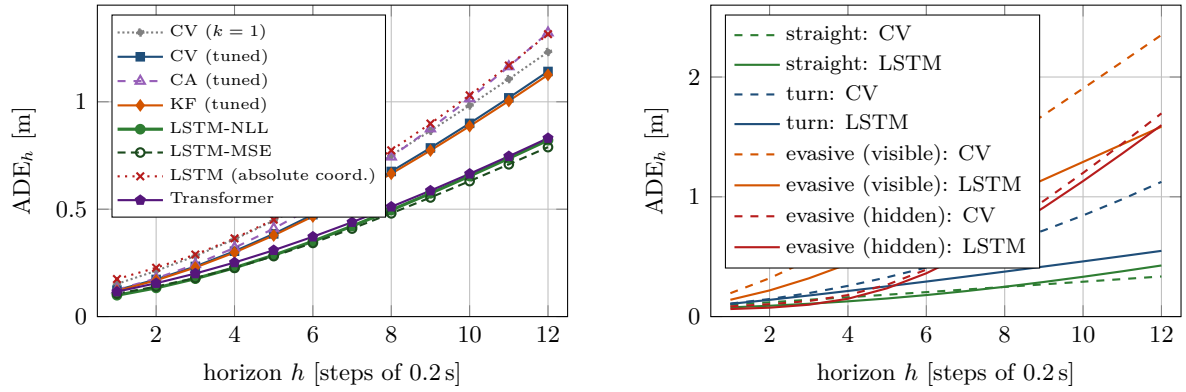


Figure 20.7. Left: ADE_h on all 800 test trajectories against the horizon. The two LSTMs and the Transformer of Section 20.6.1 (pentagons) run together, below every baseline from the first step on, and the gap widens with h ; the same network trained on absolute coordinates stays above both constant-velocity baselines and the Kalman filter at every horizon. Right: the tuned CV (dashed) against LSTM-NLL (solid) per subset. On straight flight the two cross between $h = 7$ and $h = 8$ and the tuned CV ends 0.09 m ahead at 2.4 s. On turns the LSTM halves the error and on visible evasive maneuvers it removes a third of it; on hidden maneuvers the two curves coincide, because nothing in the observation window announces the maneuver.

Straight flight: constant velocity wins the long horizons. On the 317 straight trajectories the tuned CV has an ADE of 0.335 m and the Kalman extrapolation 0.312 m, against 0.427 m for LSTM-NLL and 0.407 m for LSTM-MSE. The advantage is a long-horizon one: up to $h = 7$ (1.4 s) the two are within 3.5 cm of each other and the LSTM is in fact slightly ahead, and only from $h = 8$ on does the tuned CV pull away (Figure 20.7, right). This is the expected order, not an accident of training: under constant-velocity motion the CV prediction is exact and its only error is amplified measurement noise (Proposition 20.8), which the Kalman filter averages almost optimally, while the learned model can only add noise-driven corrections and the *hedge* it has learned against maneuvers that begin after the window. The naive $k = 1$ baseline is twice as bad (0.632 m): its velocity uses two noisy points 0.2 s apart, and Proposition 20.8 predicts a root-mean-square final error of 1.25 m, hence a mean error of $\sqrt{\pi}/2 \cdot 1.25 = 1.11$ m for an isotropic Gaussian error, against the measured 1.125 m. A paper that compares against *this* baseline would report a learned model that “halves the error on straight flight”.

Turns and evasive maneuvers: the history reveals the curvature. On the 236 gentle turns the CV and KF predictions leave along the tangent and end 2.7 m from the truth, the CA prediction overshoots to 2.2 m, and the LSTMs, which see the steady rotation of the observed displacements, end at 0.9–1.0 m and halve the ADE (0.55 against 1.12 m). On the 187 maneuvers that start inside the window the LSTMs cut the ADE from 2.35 m (tuned CV) to 1.59 m; the rest is large because how long the burst continues is only partly predictable from its onset. Here the naive $k = 1$ CV (2.14 m) beats the tuned $k = 2$: during a sharp turn the most recent velocity is the best one. On the 60 maneuvers that start *after* the window every predictor scores 1.6–1.8 m and none is ahead by more than the noise of 60 trajectories: the irreducible, multimodal part of the problem, where the honest report is that the learned model has lost nothing.

Horizons, pitfalls and calibration. Table 20.3 shows the advantage growing with the horizon: at 0.8 s the LSTM’s ADE_4 of 0.227 m is 25 % below the tuned CV, at 2.4 s it is 28 %

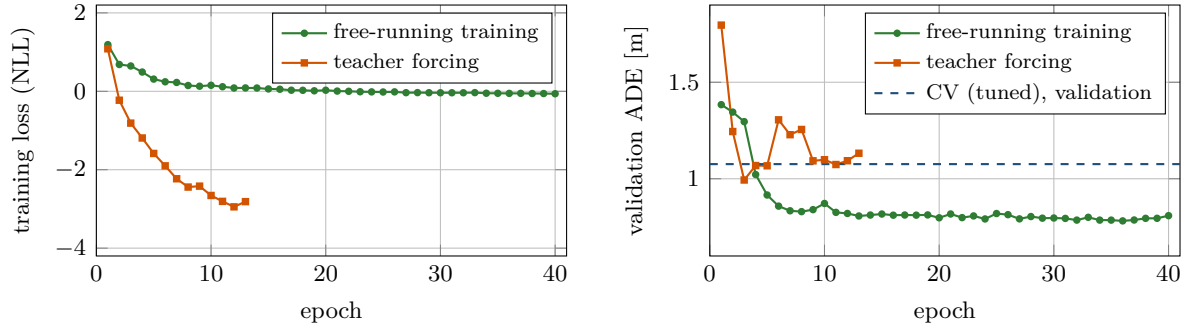


Figure 20.8. Exposure bias, measured. Left: the training loss of the teacher-forced model (orange) falls far below the free-running model’s (green) and keeps falling to epoch 12. Right: its free-running validation ADE, the quantity it will be judged by, is best at epoch 3 (0.99 m) and drifts upward afterward, so early stopping ends the run at epoch 13; the free-running model settles near 0.79 m. From epoch 3 on, teacher forcing makes the network better at the task it is trained on and worse at the task it is used for. The dashed line is the tuned CV baseline on the same validation split (1.076 m); no test-set number appears here.

below and 0.32 m in absolute terms; the 0.82 m over “all” hides a 0.43 m and a 1.60 m. LSTM-TF pays the exposure bias of Section 20.5.2 in full: its free-running validation ADE is best at epoch 3 and then drifts upward while its training loss keeps falling, so early stopping ends the run after 13 epochs and on the test set it is barely better than CV (Figure 20.8). LSTM-MSE is slightly more accurate than LSTM-NLL (0.789 against 0.821 m) but knows nothing about its own uncertainty. LSTM-NLL’s 20 sampled rollouts reach $\text{minADE}_{20} = 0.695$ m and $\text{minFDE}_{20} = 1.415$ m, below its own ADE and FDE, as minADE_K always is. Its calibration is poor: the 95 % ellipse contains 76 % of the true positions at $h = 4$ and only 54 % at $h = 12$, while the Kalman ellipse contains 79–85 %. The learned σ_h has the right shape, growing with h and larger during maneuvers, but the step errors of a free-running rollout are strongly correlated, and summing per-step variances as if they were independent underestimates the spread.

Figure 20.9 shows one test trajectory, chosen so that the LSTM’s final error is the median of the visible-maneuver subset. The drone flies at 2.62 m/s and starts a right turn of -1.28 rad/s at step 4 lasting six steps, three of them observed. The tuned CV and the Kalman extrapolation continue along the last heading, with errors $e_4 = 1.34$ and 1.41 m, $e_8 = 3.35$ and 3.47 m, $e_{12} = 5.40$ and 5.55 m; the LSTM, which has seen the onset, continues the turn for a while and then straightens out: 0.70, 1.82 and 2.64 m. Its 95 % ellipses grow with the horizon but stay far too small; at $h = 12$ the semi-axes are 0.84 and 0.51 m while the error is 2.64 m.

20.9 Properties: when the baseline is optimal, and what the models cost

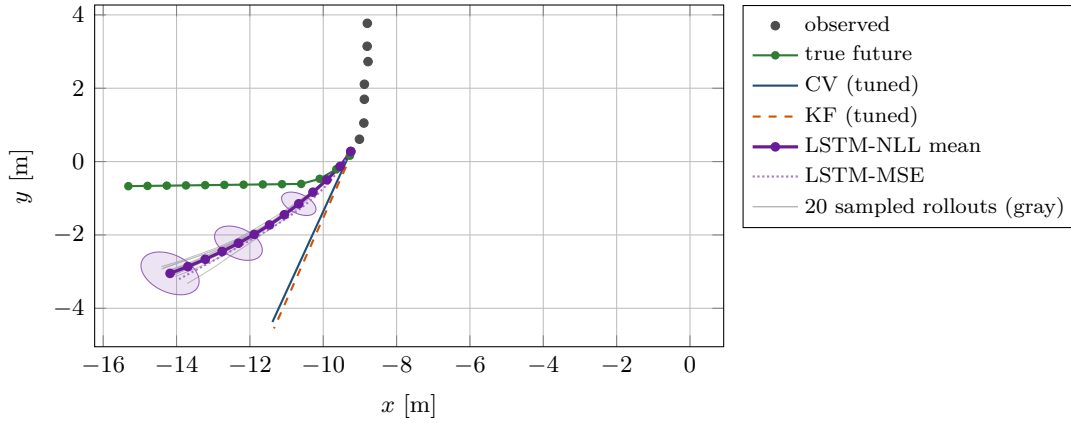


Figure 20.9. An evasive maneuver, step by step. Black dots: the eight observed positions; green: the true future; blue: the tuned CV; orange dashed: the Kalman extrapolation; purple: the mean of LSTM-NLL with its 95 % ellipses at $h = 4, 8, 12$; dotted purple: LSTM-MSE; gray: 20 sampled rollouts. The learned model turns with the drone and halves the final error, but its ellipses are over-confident and the samples, drawn from a unimodal per-step Gaussian, do not cover the true future either.

Proposition 20.7 (Error of constant-velocity extrapolation). *Let the observations be noise-free. (i) If the agent moves with constant velocity, the CV prediction of Algorithm 20.1 has $e_h = 0$ for every h and every window k . (ii) If the agent moves with constant acceleration $\mathbf{a}$, then, for the exact velocity at time T ,*

$$e_h = \frac{1}{2} \|\mathbf{a}\| (h \Delta t)^2, \quad \text{FDE} = \frac{1}{2} \|\mathbf{a}\| (H \Delta t)^2, \quad \text{ADE} = \frac{1}{2} \|\mathbf{a}\| \Delta t^2 \frac{(H+1)(2H+1)}{6},$$

while the CA predictor has $e_h = 0$. A circular turn at speed v and rate ω has centripetal acceleration $v\omega$, so (ii) describes the CV error on turns for $\omega h \Delta t \ll 1$.

Proof. (i) With $\mathbf{p}_t = \mathbf{p}_0 + t \Delta t \mathbf{v}$, line 2 gives $(\mathbf{p}_T - \mathbf{p}_{T-k}) / (k \Delta t) = \mathbf{v}$ and $\hat{\mathbf{p}}_{T+h} = \mathbf{p}_T + h \Delta t \mathbf{v} = \mathbf{p}_{T+h}$. (ii) With $\mathbf{p}_{T+h} = \mathbf{p}_T + h \Delta t \mathbf{v}_T + \frac{1}{2} (h \Delta t)^2 \mathbf{a}$ and the CV prediction $\mathbf{p}_T + h \Delta t \mathbf{v}_T$, the difference is $\frac{1}{2} (h \Delta t)^2 \mathbf{a}$; summing h^2 from 1 to H gives $H(H+1)(2H+1)/6$, and dividing by H gives the ADE. The CA predictor of line 6 recovers $\mathbf{a}$ and $\mathbf{v}_T$ exactly from three noise-free points of a parabola. $\square$

Proposition 20.8 (Noise amplification of constant velocity). *Let the agent move with constant velocity and let the observations be $\mathbf{z}_t = \mathbf{p}_t + \varepsilon_t$ with independent $\varepsilon_t \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$ in $\mathbb{R}^2$. Then the CV prediction with window k has*

$$\mathbb{E}[e_h^2] = 2\sigma^2 \left[\left(1 + \frac{h}{k}\right)^2 + \left(\frac{h}{k}\right)^2 \right].$$

For $\sigma = 0.05$ m and $h = 12$: $\sqrt{\mathbb{E}[e_h^2]} = 1.25$ m for $k = 1$, 0.65 m for $k = 2$ and 0.23 m for $k = 7$.

Proof. The prediction is $\hat{\mathbf{p}}_{T+h} = (1 + \frac{h}{k})\mathbf{z}_T - \frac{h}{k}\mathbf{z}_{T-k}$. Substituting $\mathbf{z}_t = \mathbf{p}_t + \varepsilon_t$ and using $\mathbf{p}_T + \frac{h}{k}(\mathbf{p}_T - \mathbf{p}_{T-k}) = \mathbf{p}_{T+h}$ (case (i) of Proposition 20.7) leaves the error $(1 + \frac{h}{k})\varepsilon_T - \frac{h}{k}\varepsilon_{T-k}$, a sum of two independent Gaussian vectors; its expected squared norm is the sum of the variances of its two coordinates. $\square$

The proposition explains the first two rows of Table 20.2: the noise term falls with k , but on a turn a large k averages a rotating velocity and lags, so the best k depends on the mixture of maneuvers.

Theorem 20.9 (The conditional mean minimizes the expected squared error). *Let $\mathbf{x}$ be the observation and $\mathbf{y}$ the future, jointly distributed with $\mathbb{E} \|\mathbf{y}\|^2 < \infty$. For every measurable predictor g ,*

$$\mathbb{E} \|\mathbf{y} - g(\mathbf{x})\|^2 = \mathbb{E} \|\mathbf{y} - \mathbb{E}[\mathbf{y} | \mathbf{x}]\|^2 + \mathbb{E} \|\mathbb{E}[\mathbf{y} | \mathbf{x}] - g(\mathbf{x})\|^2,$$

so the expected squared error is minimized by $g^(\mathbf{x}) = \mathbb{E}[\mathbf{y} | \mathbf{x}]$ and, up to modification on a set of measure zero, by nothing else. A predictor trained with Equation (20.12) on enough data therefore approximates the conditional mean of the future, and a Gaussian read-out trained with Equation (20.13) approximates the conditional mean and the conditional variance.*

Proof. Write $\mathbf{m} = \mathbb{E}[\mathbf{y} | \mathbf{x}]$ and expand $\|\mathbf{y} - g(\mathbf{x})\|^2 = \|\mathbf{y} - \mathbf{m}\|^2 + 2(\mathbf{y} - \mathbf{m})^\top (\mathbf{m} - g(\mathbf{x})) + \|\mathbf{m} - g(\mathbf{x})\|^2$. Conditional on $\mathbf{x}$ the cross term has expectation zero, because $\mathbb{E}[\mathbf{y} - \mathbf{m} | \mathbf{x}] = \mathbf{0}$ while $\mathbf{m} - g(\mathbf{x})$ is fixed. Taking the expectation over $\mathbf{x}$ gives the identity; the first term does not depend on g and the second is non-negative and zero exactly when $g = \mathbf{m}$ almost surely. For the Gaussian read-out, $\mathbb{E}[\log \sigma + (y - \mu)^2 / (2\sigma^2) | \mathbf{x}]$ is minimized over μ by the conditional mean and then over σ by $\sigma^2 = \text{Var}(y | \mathbf{x})$, by setting the derivatives to zero. $\square$

Corollary 20.10 (Mode averaging). *If, given the observation, the future is $\mathbf{y}^{(1)}$ with probability π or $\mathbf{y}^{(2)}$ with probability $1 - \pi$, the squared-error-optimal prediction is $\pi \mathbf{y}^{(1)} + (1 - \pi) \mathbf{y}^{(2)}$, which for $\pi = \frac{1}{2}$ is the midpoint between the two, a trajectory that never occurs. A two-sample predictor that outputs both has $\text{minADE}_2 = 0$.*

This is the whole case for multimodal outputs, and why the learned model of Section 20.8 hedges on straight flight: some straight-looking histories are followed by a maneuver, and the conditional mean shortens the predicted progress accordingly.

Proposition 20.11 (The constant error carousel). *In the LSTM cell of Definition 20.3, the Jacobian of $\mathbf{c}_t$ with respect to $\mathbf{c}_{t-1}$ along the direct path (holding $\mathbf{h}_{t-1}$ fixed) is $\text{diag}(\mathbf{f}_t)$, and over τ steps the direct-path product is $\prod_{s=1}^{\tau} \text{diag}(\mathbf{f}_{t-s+1})$, whose entries stay close to 1 as long as the forget gates do. For the simple recurrence $\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{z}_t + \mathbf{b})$ the corresponding product is $\prod_s \text{diag}(1 - \mathbf{h}_{t-s+1}^2) \mathbf{W}_h$, with $\mathbf{W}_h$ the recurrent block of $\mathbf{W}$, which shrinks or grows geometrically unless $\mathbf{W}_h$ has unit gain.*

Proof. $\mathbf{c}_{t-1}$ enters Equation (20.9) only through the product $\mathbf{f}_t \odot \mathbf{c}_{t-1}$, so with the gates held fixed $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \text{diag}(\mathbf{f}_t)$, and the chain rule multiplies these diagonal matrices along the path. For the simple recurrence the chain rule gives a product of $\text{diag}(\tanh'(\cdot)) \mathbf{W}_h$ factors, whose norm is bounded by $\|\mathbf{W}_h\|^\tau$ and, since $\tanh' \leq 1$ with equality only at 0, is typically much smaller. $\square$

The full Jacobian also contains the indirect paths through $\mathbf{h}_{t-1}$, which behave like the simple recurrence, so the LSTM does not *guarantee* stable gradients; it guarantees a path along which the error signal survives when the forget gates are near 1. The initialization $\mathbf{b}_f = \mathbf{1}$ starts there, and clipping handles the explosions the indirect paths can still cause.

Proposition 20.12 (Cost of the sequence models). *(i) One LSTM step with M inputs and D hidden units costs $\Theta(D(M + D))$ multiply-adds; encoding T_{obs} steps and decoding H steps costs $\Theta((T_{\text{obs}} + H) D(M + D))$ and is inherently sequential in time. (ii) One scaled dot-product attention over n tokens of dimension d costs $\Theta(nd^2)$ for the query, key and value projections and $\Theta(n^2d)$ for the scores and the weighted sum, with all n outputs computable in parallel; multi-head attention with n_h heads of dimension d/n_h has the same total. (iii) Attention over all N agents and T time steps of a scene as one token set has $n = NT$ and costs $\Theta(N^2T^2d + NTd^2)$ per layer, against $\Theta(NT D(M + D))$ for one LSTM per agent.*

Proof. (i) One step multiplies a $4D \times (M + D)$ matrix by a vector; the rest is element-wise on vectors of length D , and each step needs the states of the previous one. (ii) Each of $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ is an $(n \times d)(d \times d)$ product; $\mathbf{Q}\mathbf{K}^\top$ is $(n \times d)(d \times n)$, the softmax is $\Theta(n^2)$ and the product

Table 20.4. The predictors of this chapter compared. “Data” is the number of trajectories typically needed; “compute” is the cost per prediction for N agents, from [Proposition 20.12](#).

Predictor	Interaction	Uncertainty	Data	Compute	Use it when
constant velocity	no	no	none	$\Theta(N)$	always, as the baseline; short horizons
Kalman extrapolation	no	Gaussian, growing	none	$\Theta(N)$	noisy tracks; when the planner needs an ellipse now
seq2seq LSTM	no (Social LSTM: grid)	Gaussian per step	10^3 – 10^4	$\Theta(NTD^2)$	recorded maneuvers exist; turns and evasions matter
Transformer	attention over agents	mixture or samples	10^4 – 10^6	$\Theta(N^2T^2d)$	long horizons, dense scenes, large data sets
GAN / CVAE	via pooling or attention	K samples	10^4 and more	K rollouts	the planner can use sample sets; forks are common

with $\mathbf{V}$ is $(n \times n)(n \times d)$. With n_h heads each projection costs nd^2/n_h and each score matrix n^2d/n_h ; summing over heads restores the totals. (iii) Substitute $n = NT$ in (ii); for (i) run N independent encoders and decoders. $\square$

20.10 Variants and extensions

[Table 20.4](#) places the predictors of this chapter side by side. *Goal-conditioned* predictors first predict where the agent is heading and then how it gets there, which turns multimodality into a small classification problem. *Map-aware* predictors add a rasterized map or a graph of corridors as an input; in a warehouse the aisles decide the modes. *Joint* prediction outputs the futures of all agents together, so that the samples are consistent. *Dynamics integration*, as in Trajectron++, predicts controls and integrates them through the drone model of [Definition 2.22](#), which makes every sample feasible. Everything here carries over to three dimensions: the agent frame rotates about the vertical axis, displacements have three coordinates and ellipses become ellipsoids. Rudenko et al. [141] survey the field along these axes.

20.11 Implementation notes

The reference implementation `ch20_prediction.py` is about 1200 lines of NumPy: the generator of [Example 20.6](#), the baselines, the metrics, the LSTM cell with its backward pass ([Listing 20.2](#) is the forward step), the sequence-to-sequence model with BPTT, Adam, the training loop, attention, the Transformer of [Section 20.6.1](#) with its own backward pass, the ellipse helpers and the experiment. Its self-test checks that the metrics vanish for a perfect predictor, that CV is exact on straight flight and CA on a parabola, that the Kalman covariance grows with the horizon, that all gate outputs lie in $(0, 1)$, that the BPTT gradients agree with finite differences to 10^{-6} in all three decoder modes and the Transformer’s to 10^{-5} , that training lowers the loss and beats the constant-velocity start for both models, and that attention weights are probability vectors; it then runs the experiment and asserts the claims of [Section 20.8](#).

Listing 20.2. One step of the LSTM cell (from `code/ch20_prediction.py`); the backward pass follows it in the file.

```
1 def forward(self, x, h_prev, c_prev):
```

```

2      """One step.  x (B, n_in), h_prev and c_prev (B, n_hidden)."""
3      H = self.n_hidden
4      z = np.concatenate([x, h_prev], axis=1)          # [x_t ; h_{t-1}]
5      a = z @ self.W.T + self.b                       # pre-activations
6      f = sigmoid(a[:, :H])                           # forget gate
7      i = sigmoid(a[:, H:2 * H])                      # input gate
8      o = sigmoid(a[:, 2 * H:3 * H])                 # output gate
9      g = np.tanh(a[:, 3 * H:])                      # candidate values
10     c = f * c_prev + i * g                          # new cell state
11     tc = np.tanh(c)
12     h = o * tc                                       # new hidden state
13     cache = (z, f, i, o, g, c_prev, tc)
14     return h, c, cache

```

Normalization, sequence lengths, batches and seeds. The model stores the observed displacements of a batch as an array of shape $(B, T_{\text{obs}} - 1, 2)$ and the targets as $(B, T_{\text{pred}}, 2)$, both in the agent frame divided by $s = 0.5$ m. Fixed window lengths keep the batches rectangular; variable-length histories need padding and a mask. Shuffle windows, not scenes, keep whole scenes in one split, and use one seeded generator for the data and another for the training, so that a change to the model does not change the data. Evaluate on held-out maneuver types at least once: a test set whose maneuvers all occur in training measures interpolation, not prediction. Initialize the forget-gate biases to 1 and the read-out to small values, so that the untrained network starts as the constant-velocity model, and clamp the log standard deviations: a network that drives $\sigma \rightarrow 0$ on a few easy examples produces huge gradients on the next hard one. The finite-difference check in the self-test is the only way to be sure a hand-written backward pass is right.

The PyTorch version. For real data, and for a stack of Transformer layers, use a framework. The same sequence-to-sequence model is written in PyTorch in `ch20_prediction_torch.py` (not run by the self-test): an `nn.LSTM` encodes the batch in one call, an `nn.LSTMCell` is stepped by hand so that the mean can be fed back, an `nn.Linear` read-out produces $(\mu_k, \log \sigma_k)$ and autograd replaces the hand-written BPTT — a dozen lines of forward pass against the two hundred of the NumPy model, at the price of not seeing the gradient. Replacing the encoder and decoder by `nn.TransformerEncoder` and `nn.TransformerDecoder` with the positional encoding of Equation (20.17) gives the model of Giuliani et al. [55].

Predicting in absolute coordinates

A network trained on world positions has to learn that the physics is the same everywhere and in every direction. With 1600 trajectories it did not (Table 20.2: 1.315 against the tuned CV's 1.141 m overall, and worse on every subset except the hidden maneuvers, where no predictor is separated from the others). Predict displacements in the agent-centered frame of Equation (20.11), and let the network predict the correction to constant velocity rather than the motion itself.

Ignoring multimodality at intersections

At a junction the conditional mean runs through the pillar (Corollary 20.10). A deterministic predictor scored by ADE is rewarded for exactly this, and a planner that inflates the mean by a Gaussian ellipse blocks the pillar and nothing else. Where forks occur, predict a mixture or a set of samples, and hand the planner every mode whose weight is

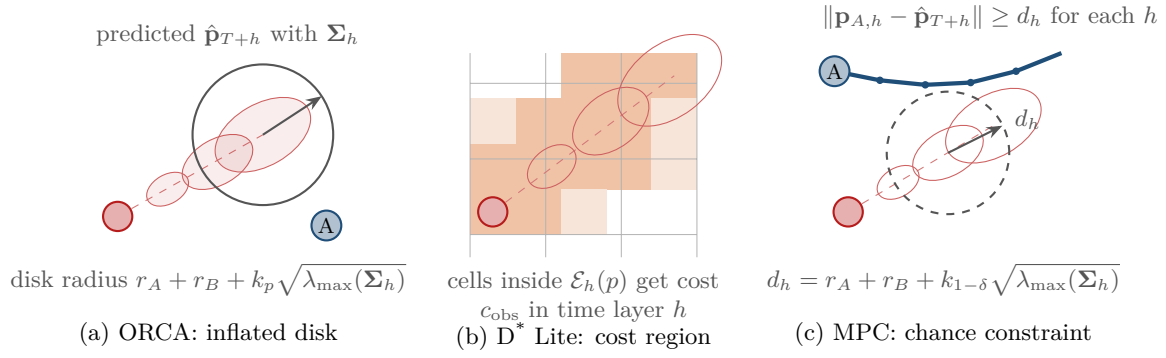


Figure 20.10. One prediction, three consumers. (a) ORCA inflates the intruder’s disk to the radius d_h^{safe} of Equation (20.21) at the step h that matches the time to collision. (b) D* Lite raises the cost of every cell inside $\mathcal{E}_h(p)$ in time layer h of the space-time graph. (c) The MPC keeps its planned position $\mathbf{p}_{A,h}$ at least d_h^{safe} from the predicted mean. The same sequence of ellipses is read three ways — one radius, a set of cells, a per-step margin — so a predictor that reports a covariance serves all three layers unchanged.

not negligible.

20.12 Where this fits in the drone system: predictions as constraints

The three consumers of a prediction use the same quantity, derived from the time-indexed occupancy region of Equation (20.20): at step h , a disk of radius

$$d_h^{\text{safe}} = r_A + r_B + k_p \sqrt{\lambda_{\max}(\Sigma_h)} \quad (20.21)$$

around the predicted mean, which contains the intruder (radius r_B) with probability at least p when the ellipse is calibrated, because the disk of radius $k_p \sqrt{\lambda_{\max}}$ contains the p -ellipse, and which adds the own drone’s radius r_A as in the Minkowski sum of Definition 2.8. `inflated_radius` and `occupancy_cells` compute the radius and the cells inside the ellipse; Figure 20.10 shows the three interfaces side by side.

In the drone system

Receives. From the tracker of Chapter 18 (or the nonlinear filters of Chapter 19) the filtered history of every unknown drone; the planned positions of the swarm members come from the global planner, are known exactly and need no prediction.

Delivers. For every unknown drone and every step h of the horizon, a mean $\hat{\mathbf{p}}_{T+h}$ and a covariance Σ_h , or a set of samples, calibrated on validation data and recomputed at every cycle from the newest observation.

Local safety layer (Chapter 13, Chapter 14). ORCA and DWA work with disks. The intruder enters as a moving disk whose radius is inflated by Equation (20.21) at the step h that matches the time-to-collision horizon of Chapter 12, and whose velocity is the predicted displacement per Δt rather than the last measured one, so that a turn already in progress is taken into account. The prediction also decides *whether* the local layer acts: the trigger of Chapter 24 fires when the predicted region intersects the own path within the safety horizon.

Replanning layer (Chapter 5). Every cell inside $\mathcal{E}_h(p)$ gets a high traversal cost at time layer h of the space-time graph of Chapter 4, or, in a static graph, the union over the horizon does. The prediction is recomputed every cycle, so the costs change every cycle:

exactly the situation D^* Lite is built for.

Controller (Chapter 21). The MPC plans positions $\mathbf{p}_{A,h}$ over its own horizon; the constraint $\|\mathbf{p}_{A,h} - \hat{\mathbf{p}}_{T+h}\| \geq d_h^{\text{safe}}$ with $p = 1 - \delta$ in Equation (20.21) enforces a collision probability of at most δ per step under a Gaussian prediction, the chance constraint of Zhu and Alonso-Mora [187].

Study plan. Week 10 of Appendix A. Prediction-aware constraints that carry the uncertainty rather than the mean alone are one of the open research directions of Chapter 24.

20.13 Summary

Chapter summary

- Trajectory prediction maps the last T_{obs} positions to the next T_{pred} ; ADE averages the displacement error over the horizon, FDE takes the last one, minADE_K scores the best of K samples and is not comparable to a deterministic ADE. Report curves per horizon and per subset, split by scene, and tune the baselines.
- Constant velocity is exact on straight flight and its only error is amplified noise, $\mathbb{E}[e_h^2] = 2\sigma^2[(1 + h/k)^2 + (h/k)^2]$; the Kalman extrapolation is its optimal linear version with a growing ellipse; constant acceleration overshoots on turns.
- The LSTM cell keeps a gated memory $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$ whose direct gradient path is $\text{diag}(\mathbf{f}_t)$; an encoder-decoder pair of cells predicts displacements in the agent frame one step at a time, with a Gaussian read-out trained by the negative log-likelihood, Adam and early stopping, in the mode in which it is used.
- Social pooling sums the neighbors' previous hidden states per grid cell; attention $\text{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d_k})\mathbf{V}$ generalizes it to learned relevance over time steps and over agents at $\Theta(n^2d)$ per layer.
- Squared-error training converges to the conditional mean, which averages the modes; mixtures, samplers and ensembles predict several futures, and a calibrated ellipse per step turns any of them into the inflated radius, cost region or chance constraint that the planners consume.
- In the experiment the LSTM beats the tuned baselines by 28 % overall, halves the error on turns and removes a third of it on visible maneuvers, loses on straight flight beyond 1.4 s, ties on hidden maneuvers, fails on absolute coordinates or under teacher forcing alone, and its ellipses cover 54 % instead of 95 % at 2.4 s. A one-layer Transformer trained on the same data matches it and does not beat it.

Further reading. The LSTM cell is due to Hochreiter and Schmidhuber [64], the forget gate to Gers, Schmidhuber and Cummins [54] and backpropagation through time to Werbos [176]; Goodfellow, Bengio and Courville [56] give the textbook treatment of recurrent networks, attention, mixture density outputs and Adam [84]. Social LSTM [1], Social GAN [58] and Trajectron++ [143] are the three papers to read on interaction-aware and multimodal prediction; the Transformer is Vaswani et al. [169] and its first use for trajectories Giuliari et al. [55]. Rudenko et al. [141] organize the field, Schöller et al. [146] made the community tune its constant-velocity baselines, and the ETH [128] and UCY [101] recordings are the standard benchmarks; mixture density networks are from Bishop [18], deep ensembles from Lakshminarayanan, Pritzel, and Blundell [95] and scheduled sampling from Bengio et al. [11].

20.14 Exercises

Exercise 20.1 (★). A predictor outputs $\hat{\mathbf{p}}_{T+1} = (1, 0)$, $\hat{\mathbf{p}}_{T+2} = (2, 0)$, $\hat{\mathbf{p}}_{T+3} = (3, 0)$ while the truth is $(1, 0.1)$, $(2, 0.5)$, $(3, 1.2)$. Compute the ADE and the FDE. A second sample of the same predictor is $(1, 0.2)$, $(2, 0.4)$, $(3, 0.8)$; compute minADE_2 and minFDE_2 and explain why they cannot exceed the ADE and FDE of the first sample.

Exercise 20.2 (★). A drone decelerates at $\|\mathbf{a}\| = 2 \text{ m/s}^2$. Using [Proposition 20.7](#), compute the FDE and the ADE of the constant-velocity prediction for $H = 12$ and $\Delta t = 0.2 \text{ s}$. At which horizon step does the error first exceed the 0.5 m safety margin of a small drone?

Exercise 20.3 (★★). Prove [Proposition 20.8](#) for a velocity estimated by a least-squares line fit through the last $k + 1$ observations instead of the endpoint difference, and compare the two at $h = 12$, $k = 7$. Which $k \in \{1, \dots, 7\}$ minimizes the endpoint-difference error at $h = 12$, and why is the answer different on a turn?

Exercise 20.4 (★★). Run one more step of the cell of [Example 20.4](#) with the input $\mathbf{x}_{t+1} = (-1.0, 0.5)$ and the states $\mathbf{h}_t$, $\mathbf{c}_t$ of [Table 20.1](#). Which unit forgets most, and which gate would have to change for unit 2 to keep its memory unchanged? Check your numbers with `LSTMCell.forward`.

Exercise 20.5 (★★). Show that for every offset δ there is a matrix $\mathbf{M}_\delta$, independent of t , with $\text{PE}(t + \delta) = \mathbf{M}_\delta \text{PE}(t)$ for the encoding of [Equation \(20.17\)](#). (Hint: treat each pair $(\sin \theta_i t, \cos \theta_i t)$ separately.) Why does this let an attention head learn “attend to the step two before me”?

Exercise 20.6 (★★). Derive the time and memory complexity of scaled dot-product attention over n tokens of dimension d , of multi-head attention with n_h heads, and of attention over all N agents and T time steps of a scene. Compare with an LSTM per agent with D hidden units, and find the scene size (N, T) at which attention with $d = 64$ costs more than LSTMs with $D = 64$.

Exercise 20.7 (★★). At a T-junction a drone turns left with probability π and right otherwise; the two futures end at $(-2, 4)$ and $(2, 4)$ relative to the junction. Give the prediction that minimizes the expected squared final error and its expected FDE as functions of π ; compute both for $\pi = \frac{1}{2}$ and $\pi = 0.9$. What does a Gaussian read-out trained with [Equation \(20.13\)](#) report as its σ in the x direction, and what is the minFDE_2 of a predictor that outputs both end points?

Exercise 20.8 (★). A report states: “We cut all recordings into windows of 20 steps with a stride of 1, shuffled them, used 80 % for training and 20 % for testing, normalized all positions by the mean and standard deviation of the data set, and obtained an ADE of 0.31 m against 0.58 m for constant velocity (velocity from the last two positions) and a minADE_{20} of 0.12 m.” List every violation of the protocol of [Section 20.3](#) and say, for each, in which direction it biases the comparison.

Exercise 20.9 (★★★ Coding). The Week 10 exercise: compare constant velocity against an LSTM on trajectory sequences and measure ADE/FDE. Starting from `ch20_prediction.py`, (a) reproduce [Table 20.2](#), plot ADE_h and FDE_h for every predictor, and repeat the training with three seeds to report each learned ADE as a mean and a spread; (b) repeat with measurement noise $\sigma = 0.02$ and 0.15 m and with 400 and 6400 training trajectories, and describe how the gap between CV and the LSTM changes; (c) add a least-squares constant-velocity baseline and a scheduled-sampling variant of the trainer; (d) train the `TransformerPredictor` of [Section 20.6.1](#) on the same splits, compare ADE_h and FDE_h with the LSTM at every horizon and on every subset, and then change one thing at a time — a second encoder layer, an autoregressive masked decoder instead of the parallel one, a Gaussian read-out — and say which of them, if any, moves the numbers beyond

the seed spread of (a); (e) if you have the ETH recordings [128], run the PyTorch model of `ch20_prediction_torch.py` with a leave-one-scene-out split at $\Delta t = 0.4$ s and report whether it beats the tuned CV, per horizon.

Exercise 20.10 (★★★ Coding). (a) Fit on the validation set one inflation factor γ_h per horizon step such that the ellipse of LSTM-NLL with covariance $\gamma_h^2 \Sigma_h$ contains 95 % of the true positions, and check it on the test set. (b) Feed the inflated radius of Equation (20.21) into the ORCA simulation of Chapter 13 as the radius of an intruder flying the evasive trajectories of Example 20.6, and compare the minimum separation and the number of triggered maneuvers against the same simulation with the Kalman ellipses and with the last measured velocity alone. (c) Extend the generator to two drones that cross, implement social pooling (Equation (20.14)) or attention over the two agents, and measure the gain there.

Part VII

Optimization and Control

Model Predictive Control

Learning objectives

- Explain the receding-horizon principle and write down the finite-horizon optimal control problem that a model predictive controller solves at every step: double-integrator model, quadratic cost, input and velocity bounds.
- Turn the constraints that matter for a swarm (intruder avoidance, formation keeping, communication range) into linear inequalities, and decide when a constraint should be hard and when it should be soft with a slack variable.
- Condense the horizon into one quadratic program: derive the prediction matrices, write $\mathbf{H}$, $\mathbf{f}$, $\mathbf{G}$ and the bounds, and solve the program with a small ADMM solver.
- Trace the MPC loop on a worked example in which an intruder crosses the reference path, identify the active constraint and explain the resulting deviation.
- State what terminal costs, terminal sets and slack variables buy in terms of stability and feasibility, and recognize the failure modes: infeasible hard constraints, stale linearizations, weights in inconsistent units and horizons too short to stop.
- Implement an MPC in Python that follows a path while keeping a formation offset and a communication distance, with prediction-aware, uncertainty-inflated avoidance constraints.

21.1 Why this matters for a drone swarm

Picture four quadrotors that fly in a diamond formation along the corridor of a warehouse. The global planner of [Chapter 9](#) has given each of them a time-indexed path, and the paths are free of conflicts among the four. That is not the end of the story. Each drone still has to *execute* its path: turn a list of waypoints into accelerations that respect its limits $a_{\max}$ and $v_{\max}$, stay within a tolerance $e_{\max}$ of its slot in the diamond, stay within the radio range R_{comm} of the drone it relays for, and, when the prediction layer of [Chapter 20](#) reports a forklift-mounted drone that crosses the corridor, keep at least r_{safe} away from its predicted positions. All of these requirements are *constraints*, and the milestone of Week 11 of the study plan ([Appendix A](#)) states the problem exactly: constraints must be enforced explicitly, not repaired after they have been violated.

The reactive methods of Part IV do part of this job. Optimal reciprocal collision avoidance (ORCA, [Chapter 13](#)) and the dynamic window approach ([Chapter 14](#)) choose one velocity for the next step, and they are fast and robust. But they look one step or one fixed time τ ahead, they treat one constraint at a time, and they know nothing about a formation. When the corridor narrows, a one-step method notices too late that it should have slowed

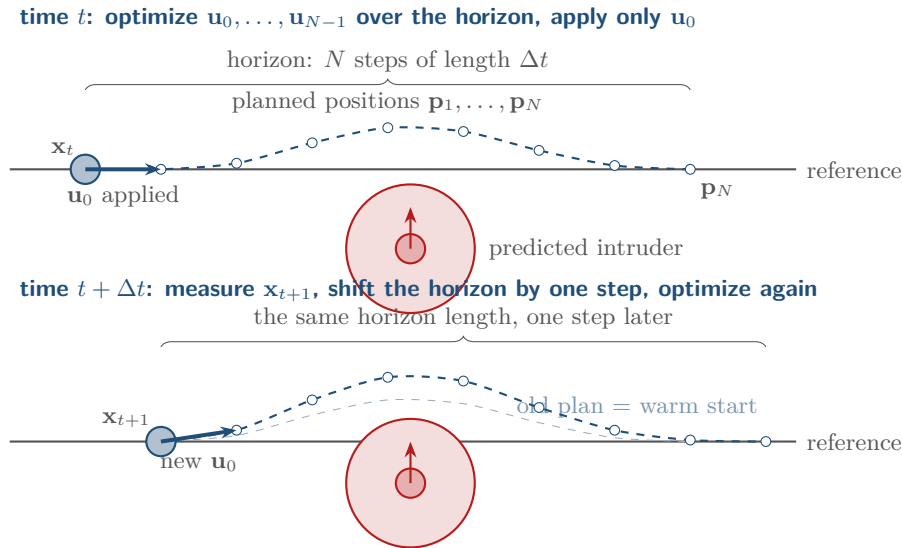


Figure 21.1. The receding-horizon principle. At time t the controller predicts the next N positions from the measured state x_t , optimizes the N inputs so that the plan tracks the reference and stays outside the predicted intruder disk, and applies only u_0 . One step later the state is measured again, the horizon has moved by one step, and a new plan is optimized; the old plan, shifted by one step, serves as the starting guess (warm start).

down two seconds ago. **Model predictive control** (MPC) closes this gap. At every control step it optimizes a whole trajectory over the next N steps, subject to the drone’s dynamics and to *every* constraint at once, applies only the first input, and repeats from the newly measured state. The optimizer sees the narrowing corridor N steps ahead and brakes in time; the formation tolerance and the radio range are inequalities in the same problem; and if an intruder’s predicted disk intersects the plan, the plan bends around it. MPC is the standard tool of process control since the 1980s [51] and of quadrotor control since the 2010s; its theory is mature [115, 134], and its computational core, a quadratic program, is a solved problem [24].

In the hybrid architecture of Chapter 24, MPC is the tracker that sits between the planners and the flight controller: it follows the conflict-based search (CBS) path of Chapter 9, takes the ORCA velocity as its reference when the local safety layer is active, and is the one component that can promise, before the fact, that the formation and communication constraints will hold. This chapter builds it from the ground up. Section 21.2 explains the receding-horizon principle, Section 21.3 states the optimal control problem and the swarm constraints, Section 21.4 condenses it into a quadratic program and solves it, Section 21.5 works through an intruder crossing with real numbers, Section 21.6 discusses stability and feasibility honestly, and Section 21.7 covers tuning, prediction-aware constraints and a generated experiment. Everything is backed by `code/ch21_mpc.py`.

21.2 The idea in plain words

Suppose you drive on a winding road at night. Your headlights show the next hundred meters. You do not plan the whole trip; you plan the next hundred meters, taking into account the speed you have, the curve you see, the car ahead and the limits of your brakes, and then you execute the first fraction of a second of that plan. A moment later the headlights show a slightly different picture, and you plan again. Everything that made the first plan sensible is still there, and everything you could not know (the car ahead braking, the pothole) is corrected as soon as it appears.

MPC is this behavior written as an algorithm (Figure 21.1). The headlights are the **prediction horizon** of N steps of length Δt ; the road you see is the reference trajectory and the predicted obstacles for the times $t + \Delta t, \dots, t + N\Delta t$; the plan is a sequence of N inputs $\mathbf{u}_0, \dots, \mathbf{u}_{N-1}$ chosen to minimize a cost (distance to the reference, effort) subject to the dynamics and the constraints; “executing the first fraction” is applying $\mathbf{u}_0$ and discarding the rest; and “planning again” is the **receding horizon**: the window moves forward by one step at every step. Three points deserve emphasis.

- *The model is inside the optimizer.* The plan is not a curve drawn on the map; it is a sequence of states that the double integrator can actually reach with the chosen inputs. A plan can therefore not ask for more acceleration than the drone has.
- *Constraints are part of the problem, not a filter after it.* If the plan satisfies $\|\mathbf{p}_k - \hat{\mathbf{o}}_k\| \geq r_{\text{safe}}$ for every predicted intruder position $\hat{\mathbf{o}}_k$, and the model is right, the next real state will satisfy it too. This is what “explicitly enforced” means.
- *Feedback comes from re-measuring.* Because only the first input is applied and the next problem starts from the measured state, model errors, wind and a mispredicted intruder are corrected at the next step. MPC is a feedback controller that happens to compute its feedback by optimization.

The reader will recognize the pattern. Windowed prioritized planning (Chapter 8) plans a window of the path and re-plans as the window moves; D* Lite (Chapter 5) repairs a plan when the world changes. MPC does the same thing in continuous state space with dynamics, and it does it at the control rate, ten to a hundred times per second.

Key idea

At every step, solve a small optimization problem over the next N steps that contains the dynamics and every constraint you care about; apply the first input, throw the rest away, and repeat from the measured state. With a linear model, a quadratic cost and linear (or linearized) constraints, that problem is a quadratic program that a few dozen matrix–vector products solve.

21.3 Problem statement and notation

21.3.1 The drone model

We use the discrete-time double integrator of Chapter 2. The state is $\mathbf{x} = (\mathbf{p}, \mathbf{v}) \in \mathbb{R}^{2n}$ with $n \in \{2, 3\}$, the input is the acceleration $\mathbf{u} = \mathbf{a} \in \mathbb{R}^n$, and with a zero-order hold over a step of length Δt the exact update is $\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k$. In the plane,

$$\mathbf{A} = \begin{pmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \frac{1}{2}\Delta t^2 & 0 \\ 0 & \frac{1}{2}\Delta t^2 \\ \Delta t & 0 \\ 0 & \Delta t \end{pmatrix}, \quad (21.1)$$

and in three dimensions the same blocks appear with 3×3 identities. The physical limits $\|\mathbf{a}\| \leq a_{\text{max}}$ and $\|\mathbf{v}\| \leq v_{\text{max}}$ are disks, which are convex but not linear. Throughout this chapter we replace them by the per-axis boxes $|u_{k,i}| \leq a_{\text{max}}$ and $|v_{k,i}| \leq v_{\text{max}}$, which are linear. The box is a *relaxation*: it contains the disk and admits $\|\mathbf{a}\| \leq \sqrt{n} a_{\text{max}}$ along a diagonal, so a_{max} must be declared per axis and set from what the vehicle can do on one axis — the worked example of Section 21.5 accordingly takes $a_{\text{max}} = 2 \text{ m/s}^2$ per axis. If you need the disk or ball exactly, use the largest box inside it, of half-width $a_{\text{max}}/\sqrt{n}$ (and $v_{\text{max}}/\sqrt{n}$ for the

velocity), or the inscribed polygon of [Proposition 21.4](#) below. Time is indexed by k *within* the horizon ($k = 0$ is now, $k = N$ is the end of the window) and by t in the closed loop.

21.3.2 The finite-horizon optimal control problem

Definition 21.1 (Finite-horizon optimal control problem). Given the current state $\mathbf{x}_0 = \mathbf{x}_t$, reference states $\mathbf{x}_k^{\text{ref}} = (\mathbf{p}^{\text{ref}}(t + k\Delta t), \mathbf{v}^{\text{ref}}(t + k\Delta t))$ for $k = 1, \dots, N$, symmetric weight matrices $\mathbf{Q} \succeq \mathbf{0}$, $\mathbf{R} \succ \mathbf{0}$ and $\mathbf{P}_f \succeq \mathbf{0}$, the problem is

$$\begin{aligned} \min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} \quad & J = \sum_{k=1}^{N-1} (\mathbf{x}_k - \mathbf{x}_k^{\text{ref}})^\top \mathbf{Q} (\mathbf{x}_k - \mathbf{x}_k^{\text{ref}}) + \sum_{k=0}^{N-1} \mathbf{u}_k^\top \mathbf{R} \mathbf{u}_k + (\mathbf{x}_N - \mathbf{x}_N^{\text{ref}})^\top \mathbf{P}_f (\mathbf{x}_N - \mathbf{x}_N^{\text{ref}}) \\ \text{subject to} \quad & \mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k, \quad k = 0, \dots, N-1, \\ & |u_{k,i}| \leq a_{\max}, \quad k = 0, \dots, N-1, \\ & |v_{k,i}| \leq v_{\max}, \quad \mathbf{p}_k \in \mathcal{X}, \quad k = 1, \dots, N, \\ & \text{the swarm constraints of Section 21.3.3.} \end{aligned} \tag{21.2}$$

The **stage cost** weighs the tracking error with $\mathbf{Q}$ and the effort with $\mathbf{R}$; the **terminal cost** $\mathbf{P}_f$ weighs the last predicted state; $\mathcal{X}$ is an optional polytope (a geofence). The **MPC control law** applies the first element of the minimizer: $\mathbf{u}_t = \mathbf{u}_0^*(\mathbf{x}_t)$.

A warning about letters. The matrices $\mathbf{Q}$ and $\mathbf{R}$ here are *cost weights* chosen by the designer: $\mathbf{Q}$ says how much a meter of position error or a meter per second of velocity error costs, and $\mathbf{R}$ how much a meter per second squared of acceleration costs. They have nothing to do with the process and measurement noise covariances that the Kalman filter of [Chapter 18](#) also calls $\mathbf{Q}$ and $\mathbf{R}$; the letters coincide by tradition, not by meaning. A natural choice is $\mathbf{Q} = \text{diag}(q_p \mathbf{I}, q_v \mathbf{I})$ with $q_p \gg q_v$ and $\mathbf{R} = r \mathbf{I}$, and the terminal weight $\mathbf{P}_f$ is best taken from the Riccati equation ([Section 21.6](#)). That weight carries the subscript of the terminal cost V_f and of the terminal set $\mathcal{X}_f$ precisely so that it is not read as the estimate covariance $\mathbf{P}_k$ that the filters of [Chapters 18](#) and [19](#) propagate: $\mathbf{P}_k$ says how uncertain an estimate is, $\mathbf{P}_f$ says how expensive the last predicted state is. Increasing $\mathbf{Q}$ relative to $\mathbf{R}$ makes the drone track harder and use more acceleration; the absolute scale of the cost is irrelevant.

Why a quadratic cost? Three reasons. It penalizes large errors more than small ones, which matches what we want; with the linear model it makes the problem convex, so the solver finds the global minimum every time; and without constraints its solution is the classical linear-quadratic regulator (LQR), whose properties are completely understood [134]. MPC is LQR plus constraints plus a moving reference, and the constraints are the point. ([Appendix B.4.2](#) reviews convex problems and the KKT conditions that characterize their solutions.)

21.3.3 Constraints that matter for the swarm

[Table 21.1](#) lists the constraints a swarm drone has to respect. The input and velocity boxes are linear and always satisfiable. The other three involve other agents, and each needs a small piece of geometry before it fits into a convex program.

(1) Obstacle and intruder avoidance. The prediction layer provides, for every step k of the horizon, a predicted intruder position $\hat{\mathbf{o}}_k$ ([Chapters 18](#) and [20](#)); static obstacles are the special case $\hat{\mathbf{o}}_k = \mathbf{o}$. With the radii of the drone and the intruder and a margin folded into r_k (in the simplest case $r_k = r_{\text{safe}}$ for all k), the requirement is $\|\mathbf{p}_k - \hat{\mathbf{o}}_k\| \geq r_k$. This set, the outside of a disk, is *not convex*: the midpoint of two safe positions on opposite sides of the intruder is the intruder. A quadratic program cannot express it. The standard remedy replaces the disk by a half-plane that touches it ([Figure 21.2](#)).

Table 21.1. The constraints of a swarm drone, whether they are convex in the drone’s own positions, how they enter the quadratic program, and whether they are usually hard or soft. The last column is the recommendation of [Section 21.3.3](#), not a description of every implementation: `ch21_mpc.py` attaches slacks to the avoidance rows only and keeps the keep-in rows of [Equation \(21.9\)](#) hard. The predicted positions of others ($\hat{\mathbf{o}}_k, \mathbf{p}_k^j$) are data from the prediction layer or from the neighbor’s communicated plan.

Constraint	Form	Convex	In the QP	Hard/soft
Input limit	$ u_{k,i} \leq a_{\max}$	yes	box on $\mathbf{U}$	hard
Velocity limit	$ v_{k,i} \leq v_{\max}$	yes	box on $\mathbf{S}_u^v \mathbf{U}$	hard
Avoidance	$\ \mathbf{p}_k - \hat{\mathbf{o}}_k\ \geq r_k$	no	half-plane per step, Definition 21.2	soft
Formation	$\ \mathbf{p}_k^i - \mathbf{p}_k^j - \mathbf{d}_{ij}\ \leq e_{\max}$	yes	polygon, $M = 8$, Proposition 21.4	soft
Communication	$\ \mathbf{p}_k^i - \mathbf{p}_k^j\ \leq R_{\text{comm}}$	yes	polygon, $M = 8$, Proposition 21.4	soft

Definition 21.2 (Linearized avoidance constraint). Let $\bar{\mathbf{p}}_k$ be a guess of the drone’s position at step k (the **linearization point**) with $\bar{\mathbf{p}}_k \neq \hat{\mathbf{o}}_k$, and let $\mathbf{n}_k = (\bar{\mathbf{p}}_k - \hat{\mathbf{o}}_k) / \|\bar{\mathbf{p}}_k - \hat{\mathbf{o}}_k\|$ be the unit vector from the predicted intruder toward the guess. The **linearized avoidance constraint** at step k is the half-plane

$$\mathbf{n}_k^T(\mathbf{p}_k - \hat{\mathbf{o}}_k) \geq r_k. \quad (21.3)$$

Proposition 21.3 (The half-plane is conservative). *Every $\mathbf{p}_k$ that satisfies [Equation \(21.3\)](#) satisfies $\|\mathbf{p}_k - \hat{\mathbf{o}}_k\| \geq r_k$.*

Proof. By the Cauchy–Schwarz inequality and $\|\mathbf{n}_k\| = 1$, $\|\mathbf{p}_k - \hat{\mathbf{o}}_k\| = \|\mathbf{n}_k\| \|\mathbf{p}_k - \hat{\mathbf{o}}_k\| \geq \mathbf{n}_k^T(\mathbf{p}_k - \hat{\mathbf{o}}_k) \geq r_k$. $\square$

The half-plane is the tangent to the disk at the point closest to the guess, and it excludes more than the disk: everything “behind” the disk as seen from $\bar{\mathbf{p}}_k$. This is harmless when the guess is on the correct side of the intruder and harmful when it is not, which is why the choice of $\bar{\mathbf{p}}_k$ matters. The MPC loop uses the previous plan shifted by one step ([Section 21.4.4](#)); at the very first solve, or after a failure, it uses the reference. An alternative with the same flavor is a **convex corridor**: a planner (for instance the path of [Chapter 4](#) inflated as in [Chapter 2](#)) provides for every step k a convex polygon $\mathcal{C}_k$ of free space, and the constraint $\mathbf{p}_k \in \mathcal{C}_k$ is a handful of linear inequalities. This $\mathcal{C}_k$ is one polygon per step of the horizon, not the configuration space $\mathcal{C}$ of [Chapter 2](#) and not the constraint set $\mathcal{C}_i$ that [Chapter 9](#) attaches to an agent. Corridors are the method of choice for static clutter; half-planes are the method of choice for a moving intruder, whose predicted disk moves with k .

(2) Formation keeping. Drone i should stay near its slot relative to drone j : with the desired offset $\mathbf{d}_{ij}$ and the tolerance $e_{\max}$, $\|\mathbf{p}_k^i - \mathbf{p}_k^j - \mathbf{d}_{ij}\| \leq e_{\max}$. This is the *inside* of a disk of radius $e_{\max}$ around the moving center $\mathbf{c}_k = \mathbf{p}_k^j + \mathbf{d}_{ij}$ and it is convex; when drone j ’s plan (or its reference) is known and communicated, $\mathbf{c}_k$ is data and the constraint is convex in $\mathbf{p}_k^i$ alone. A disk is still not linear, and we approximate it from inside by a polygon.

Proposition 21.4 (Inscribed polygon). *Let $\mathbf{n}_1, \dots, \mathbf{n}_M$ be the unit vectors at angles $2\pi m/M$ and $e > 0$. If $\mathbf{n}_m^T(\mathbf{p} - \mathbf{c}) \leq e \cos(\pi/M)$ for all m , then $\|\mathbf{p} - \mathbf{c}\| \leq e$.*

Proof. The M inequalities describe the regular M -gon whose apothem (distance from the center to a side) is $e \cos(\pi/M)$; its vertices lie at distance e from $\mathbf{c}$, on the circle. A convex polygon is the convex hull of its vertices, all of which lie in the disk, so the polygon lies in the disk. $\square$

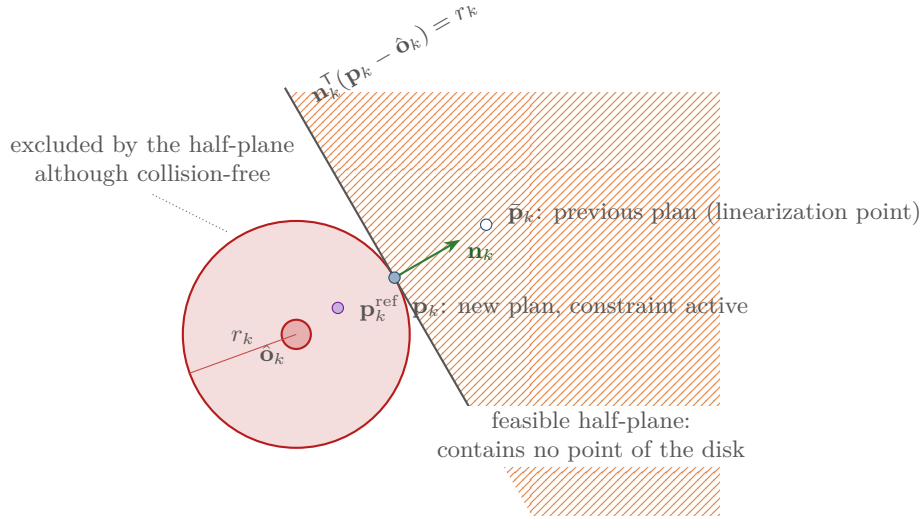


Figure 21.2. Convexifying the avoidance constraint at one step k . The non-convex requirement is to stay outside the disk of radius r_k around the predicted intruder position $\hat{o}_k$. The unit vector $\mathbf{n}_k$ points from $\hat{o}_k$ toward the previous plan $\bar{\mathbf{p}}_k$; the half-plane $\mathbf{n}_k^T(\mathbf{p}_k - \hat{o}_k) \geq r_k$ (hatched) touches the disk and contains none of it, so every position that satisfies it is safe. The price is the region to the left, which is collision-free but excluded. The new plan point $\mathbf{p}_k$ lands on the boundary when the constraint is active.

With $M = 8$ the polygon loses $1 - \cos(\pi/8) \approx 7.6\%$ of the radius at the flat sides, with $M = 16$ only 1.9% . In three dimensions the same idea uses a polyhedron; the book's code handles the planar case with $M = 8$.

(3) Communication range. Drone i must stay within radio range of its relay j : $\|\mathbf{p}_k^i - \mathbf{p}_k^j\| \leq R_{\text{comm}}$. This is the formation constraint with $\mathbf{d}_{ij} = \mathbf{0}$ and $e_{\text{max}} = R_{\text{comm}}$, and it enters the program through the same polygon. The two constraints differ in their role, not their form: formation keeping is about a slot, the communication range is about connectivity of the whole swarm graph (Chapter 23), and a drone typically has one formation partner but several communication partners.

(4) Soft constraints and slack variables. Every constraint that involves a prediction of somebody else can become impossible to satisfy: an intruder that appears at short range, a partner that brakes hard, a reference that runs through a newly detected obstacle. A **hard constraint** that cannot be satisfied makes the whole program infeasible and the solver returns nothing, at the worst possible moment. A **soft constraint** replaces $g_k(\mathbf{z}) \leq 0$ by

$$g_k(\mathbf{z}) \leq s_k, \quad s_k \geq 0, \quad \text{and adds } w_1 s_k + w_2 s_k^2 \text{ to the cost,} \quad (21.4)$$

with a **slack variable** s_k and penalty weights $w_1, w_2 \geq 0$. The program is then always feasible, and when the hard version is feasible and w_1 is large enough, the soft version returns the same solution with all slacks at zero (Proposition 21.7). When it is not, the solver returns the input that violates the constraints *least*, in the sense of the penalty, which is exactly what a drone should do when a collision can no longer be ruled out: minimize the damage rather than give up. The rule of thumb that follows is in Table 21.1: input and velocity bounds are hard, because they are on the drone's own variables and can always be met; avoidance, formation and communication constraints are soft, with weights large enough that the slack is used only in emergencies. The linear term $w_1 s_k$ is what makes the penalty *exact*; the quadratic term

$w_2 s_k^2$ keeps the program strictly convex and spreads a violation over several steps instead of concentrating it in one.

21.4 The algorithm

21.4.1 Stacking the horizon: the condensed form

The dynamics constraint in Equation (21.2) is an equality that we can eliminate. Applying $\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k$ repeatedly from $\mathbf{x}_0$ gives $\mathbf{x}_k = \mathbf{A}^k \mathbf{x}_0 + \sum_{j=0}^{k-1} \mathbf{A}^{k-1-j} \mathbf{B} \mathbf{u}_j$, and stacking the N predicted states into one vector,

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_N \end{pmatrix} = \underbrace{\begin{pmatrix} \mathbf{A} \\ \mathbf{A}^2 \\ \vdots \\ \mathbf{A}^N \end{pmatrix}}_{\mathbf{S}_x} \mathbf{x}_0 + \underbrace{\begin{pmatrix} \mathbf{B} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{AB} & \mathbf{B} & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}^{N-1}\mathbf{B} & \mathbf{A}^{N-2}\mathbf{B} & \cdots & \mathbf{B} \end{pmatrix}}_{\mathbf{S}_u} \underbrace{\begin{pmatrix} \mathbf{u}_0 \\ \mathbf{u}_1 \\ \vdots \\ \mathbf{u}_{N-1} \end{pmatrix}}_{\mathbf{U}}. \quad (21.5)$$

The **prediction matrices** $\mathbf{S}_x \in \mathbb{R}^{2nN \times 2n}$ and $\mathbf{S}_u \in \mathbb{R}^{2nN \times nN}$ depend only on the model and the horizon, so they are computed once. The block (k, j) of $\mathbf{S}_u$ is $\mathbf{A}^{k-1-j} \mathbf{B}$ for $j < k$ and zero otherwise: the input at step j influences the states after it, and the influence propagates through $\mathbf{A}$. This is the **condensed form** of the problem: the only decision variables are the nN inputs, and the states are affine functions of them. We write $\mathbf{S}_x^p, \mathbf{S}_x^v, \mathbf{S}_u^p, \mathbf{S}_u^v$ for the rows that produce the positions and the velocities, and $\mathbf{S}_{u,k}^p$ for the n rows that produce $\mathbf{p}_k$. One warning on the fonts: $\mathbf{X}$ and $\mathbf{U}$ are stacked *vectors* printed in the same bold as the matrices, whereas the calligraphic $\mathcal{X}$ of Definition 21.1 is the geofence, a set.

21.4.2 The quadratic program

Let $\bar{\mathbf{Q}} = \text{diag}(\mathbf{Q}, \dots, \mathbf{Q}, \mathbf{P}_f)$ (N blocks, the last one $\mathbf{P}_f$), $\bar{\mathbf{R}} = \text{diag}(\mathbf{R}, \dots, \mathbf{R})$ and $\mathbf{X}^{\text{ref}}$ the stacked reference. Substituting Equation (21.5) into the cost of Equation (21.2),

$$J = \mathbf{U}^T (\mathbf{S}_u^T \bar{\mathbf{Q}} \mathbf{S}_u + \bar{\mathbf{R}}) \mathbf{U} + 2 (\mathbf{S}_x \mathbf{x}_0 - \mathbf{X}^{\text{ref}})^T \bar{\mathbf{Q}} \mathbf{S}_u \mathbf{U} + \text{const}, \quad (21.6)$$

a quadratic function of $\mathbf{U}$ (Exercise 21.2 asks you to verify it). Collect the inputs and the slacks in $\mathbf{z} = (\mathbf{U}, \mathbf{s})$. Every constraint of Section 21.3.3 is linear in $\mathbf{z}$, and the whole problem is the **quadratic program** (QP) in standard form

$$\min_{\mathbf{z}} \frac{1}{2} \mathbf{z}^T \mathbf{H} \mathbf{z} + \mathbf{f}^T \mathbf{z} \quad \text{subject to} \quad \mathbf{l} \leq \mathbf{G} \mathbf{z} \leq \mathbf{u}_{\max}, \quad (21.7)$$

with the cost matrices

$$\mathbf{H} = \begin{pmatrix} 2(\mathbf{S}_u^T \bar{\mathbf{Q}} \mathbf{S}_u + \bar{\mathbf{R}}) & \mathbf{0} \\ \mathbf{0} & 2w_2 \mathbf{I} \end{pmatrix}, \quad \mathbf{f} = \begin{pmatrix} 2\mathbf{S}_u^T \bar{\mathbf{Q}} (\mathbf{S}_x \mathbf{x}_0 - \mathbf{X}^{\text{ref}}) \\ w_1 \mathbf{1} \end{pmatrix}, \quad (21.8)$$

and the constraint rows, one block per constraint type,

$$\mathbf{G} = \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{S}_u^v & \mathbf{0} \\ \mathbf{N}^c \mathbf{S}_u^p & \mathbf{0} \\ -\mathbf{N}^o \mathbf{S}_u^p & -\mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}, \quad \mathbf{l} = \begin{pmatrix} -a_{\max} \mathbf{1} \\ -v_{\max} \mathbf{1} - \mathbf{S}_x^v \mathbf{x}_0 \\ -\infty \\ -\infty \\ \mathbf{0} \end{pmatrix}, \quad \mathbf{u}_{\max} = \begin{pmatrix} a_{\max} \mathbf{1} \\ v_{\max} \mathbf{1} - \mathbf{S}_x^v \mathbf{x}_0 \\ \mathbf{c} \\ \mathbf{b} \\ +\infty \end{pmatrix}. \quad (21.9)$$

Here $\mathbf{l}$ and $\mathbf{u}_{\max}$ are the lower and upper bounds of the constraint rows. The QP literature writes that upper bound $\mathbf{u}$; this book cannot, because $\mathbf{u}$ is the control input, so the bound carries a subscript here and in [Algorithm 21.1](#) (the implementation still calls the two arrays $\mathbf{l}$ and $\mathbf{u}$, as [Listing 21.2](#) shows). ([Appendix B.4.4](#) reviews quadratic programs in this standard form.)

Read the blocks from top to bottom. The input box is a bound on $\mathbf{U}$ itself. The velocity box is a bound on the velocity rows of the prediction, $\mathbf{S}_u^v \mathbf{U}$, shifted by the free response $\mathbf{S}_x^v \mathbf{x}_0$. The keep-in block has one row per step k and direction m of the polygon: $\mathbf{N}^c$ stacks the row vectors $\mathbf{n}_m^T$ applied to the position block of step k (a bold $\mathbf{N}$ is a matrix of normals; the plain N stays the prediction horizon), and the bound is $c_{k,m} = e \cos(\pi/M) - \mathbf{n}_m^T ((\mathbf{S}_x \mathbf{x}_0)_k^p - \mathbf{c}_k)$. The avoidance block has one row per step and obstacle: the row is $-\mathbf{n}_k^T \mathbf{S}_{u,k}^p$, the bound is $b_k = \mathbf{n}_k^T ((\mathbf{S}_x \mathbf{x}_0)_k^p - \hat{\mathbf{o}}_k) - r_k$, and the $-\mathbf{I}$ next to it subtracts the slack s_k , so that the row reads $\mathbf{n}_k^T (\mathbf{p}_k - \hat{\mathbf{o}}_k) \geq r_k - s_k$. The last block is $\mathbf{s} \geq \mathbf{0}$. For a hard avoidance constraint delete the slack column and the last block. Written this way the keep-in rows are hard, which is what the book's code does; softening them is the same construction as for avoidance (append a $-\mathbf{I}$ slack column and a non-negativity block, and add $w_1 s + w_2 s^2$ to the cost), and [Exercise 21.9](#) asks you to do it. Because $\bar{\mathbf{R}} \succ \mathbf{0}$ and $w_2 > 0$, $\mathbf{H}$ is positive definite, the program is strictly convex, and its minimizer is unique.

The size of the program is what makes MPC practical. For the planar drone with $N = 15$ and one intruder the hard program has 30 variables (the inputs) and $30 + 30 + 15 = 75$ rows: the input box, the velocity box, and one avoidance half-plane per step. Softening the avoidance rows adds 15 slacks and their 15 non-negativity rows, so 45 variables and 90 rows; [Section 21.8](#) reports the solve times.

21.4.3 Solving the quadratic program

Three families of methods solve [Equation \(21.7\)](#) [[24](#), [134](#)]. **Active-set methods** guess which constraints hold with equality, solve the resulting equality-constrained problem (a linear system), and add or drop constraints until the guess is right; they are exact and warm-start beautifully, but each change of the active set costs a linear solve. **Interior-point methods** follow a central path with Newton steps; they are robust and converge in a few dozen iterations regardless of the problem, but they cannot exploit a warm start. **First-order splitting methods** such as the alternating direction method of multipliers (ADMM) [[23](#)] take many cheap steps, each a matrix–vector product, and they warm-start well. The book uses a small dense ADMM solver written in NumPy, the iteration of the OSQP solver [[158](#)], for four reasons: its iteration is twenty lines that you can read in full (the solver around it, with scaling, restart and polishing, is some seventy-five); it needs no library; the shifted previous plan is a warm start it can use; and it detects an infeasible program instead of looping forever. For a real vehicle you would call OSQP itself, or a general nonlinear solver such as `scipy.optimize.minimize` with SLSQP when you prototype with non-linear constraints, but the algorithm is the same.

[Algorithm 21.1](#) works with a copy $\mathbf{w}$ of $\mathbf{Gz}$ that is forced into the box $[\mathbf{l}, \mathbf{u}_{\max}]$ by the componentwise projection $\Pi_{[\mathbf{l}, \mathbf{u}_{\max}]}(\mathbf{a}) = \min(\max(\mathbf{a}, \mathbf{l}), \mathbf{u}_{\max})$, and a vector of **Lagrange multipliers** $\mathbf{y}$ that pushes the two copies together. Line 5 minimizes the cost plus a quadratic penalty for the disagreement between $\mathbf{Gz}$ and $\mathbf{w}$; this is a linear system with the fixed matrix $\mathbf{M} = \mathbf{H} + \sigma \mathbf{I} + \rho \mathbf{G}^T \mathbf{G}$, so the factorization (in the book's code, for a matrix with at most a few hundred rows, simply the inverse: `Kinv` in [Listing 21.2](#) is $\mathbf{M}^{-1}$) is computed once and every iteration costs two matrix–vector products. The letter $\mathbf{M}$ is used here only for this matrix; $\mathbf{K}$ is reserved for the LQR gain of [Equation \(21.11\)](#). Line 7 clips the disagreeing copy into the box, the cheapest operation imaginable, and line 8 updates the multipliers by the remaining disagreement. Row scaling (line 2) matters more than it looks: an input-bound row has entries of size one, an avoidance row has entries of size Δt^2 , and one penalty ρ cannot

Algorithm 21.1. ADMM for the quadratic program Equation (21.7) (the OSQP iteration)

Input: $\mathbf{H}$, $\mathbf{f}$, $\mathbf{G}$, $\mathbf{l}$, $\mathbf{u}_{\max}$; warm start $\mathbf{z}$, $\mathbf{y}$; penalty ρ , regularization σ , relaxation α , tolerance ε

Output: minimizer $\mathbf{z}$, multipliers $\mathbf{y}$, status

```

1 Function AdmmQP( $\mathbf{H}, \mathbf{f}, \mathbf{G}, \mathbf{l}, \mathbf{u}_{\max}, \mathbf{z}, \mathbf{y}$ ):
2   scale every row of  $\mathbf{G}$  and of  $\mathbf{l}, \mathbf{u}_{\max}$  to unit infinity norm
3    $\mathbf{w} \leftarrow \Pi_{[\mathbf{l}, \mathbf{u}_{\max}]}(\mathbf{G}\mathbf{z})$ ;  $\mathbf{M} \leftarrow \mathbf{H} + \sigma\mathbf{I} + \rho\mathbf{G}^\top\mathbf{G}$ , factorized once
4   for  $i = 1, 2, \dots, i_{\max}$  do
5      $\tilde{\mathbf{z}} \leftarrow \mathbf{M}^{-1}(\sigma\mathbf{z} - \mathbf{f} + \mathbf{G}^\top(\rho\mathbf{w} - \mathbf{y}))$ 
6      $\hat{\mathbf{w}} \leftarrow \alpha\mathbf{G}\tilde{\mathbf{z}} + (1 - \alpha)\mathbf{w}$ ;  $\mathbf{z} \leftarrow \alpha\tilde{\mathbf{z}} + (1 - \alpha)\mathbf{z}$ 
7      $\mathbf{w}^+ \leftarrow \Pi_{[\mathbf{l}, \mathbf{u}_{\max}]}(\hat{\mathbf{w}} + \mathbf{y}/\rho)$ 
8      $\mathbf{y} \leftarrow \mathbf{y} + \rho(\hat{\mathbf{w}} - \mathbf{w}^+)$ ;  $\mathbf{w} \leftarrow \mathbf{w}^+$ 
9     if  $\|\mathbf{G}\mathbf{z} - \mathbf{w}\|_\infty \leq \varepsilon$  and  $\|\mathbf{H}\mathbf{z} + \mathbf{f} + \mathbf{G}^\top\mathbf{y}\|_\infty \leq \varepsilon$  then
10      | polish: re-solve the KKT system on the active rows; return ( $\mathbf{z}, \mathbf{y}$ , solved)
11      if the change of  $\mathbf{y}$  certifies infeasibility then
12        | return ( $\mathbf{z}, \mathbf{y}$ , infeasible)
13      every 30 iterations: rebalance  $\rho$  from the ratio of the two residuals and
        refactorize  $\mathbf{M}$ 
14 return ( $\mathbf{z}, \mathbf{y}$ , max_iter)

```

serve both unless the rows are equilibrated. The two residuals in the stopping test are the **primal residual** (are the constraints satisfied?) and the **dual residual** (is the gradient of the cost balanced by the constraint forces?); at a solution both are zero; the code tests them every ten iterations against the mixed tolerances $\varepsilon_{\text{abs}} + \varepsilon_{\text{rel}}s_{\text{prim}}$ and $\varepsilon_{\text{abs}} + \varepsilon_{\text{rel}}s_{\text{dual}}$, where $s_{\text{prim}} = \max(\|\mathbf{G}\mathbf{z}\|_\infty, \|\mathbf{w}\|_\infty)$ and $s_{\text{dual}} = \max(\|\mathbf{H}\mathbf{z}\|_\infty, \|\mathbf{f}\|_\infty, \|\mathbf{G}^\top\mathbf{y}\|_\infty)$ are the largest of the terms that make up the residual being tested (**s_prim** and **s_dual** in Listing 21.2); the letter σ stays with the regularization of line 3. ADMM converges to a modest accuracy quickly and to a high accuracy slowly, so line 10 **polishes** the answer: it reads the **active set** (the rows whose multiplier is non-zero) off $\mathbf{y}$ and solves the equality-constrained problem on those rows exactly, which makes the active constraints hold to machine precision. Finally, if the program is infeasible the multipliers grow along a direction $\delta\mathbf{y}$ with $\mathbf{G}^\top\delta\mathbf{y} = \mathbf{0}$ and $\mathbf{u}_{\max}^\top\delta\mathbf{y}^+ + \mathbf{l}^\top\delta\mathbf{y}^- < 0$, where $\delta\mathbf{y}^+ = \max(\delta\mathbf{y}, \mathbf{0})$ and $\delta\mathbf{y}^- = \min(\delta\mathbf{y}, \mathbf{0})$ are the positive and negative parts, which is a certificate that no $\mathbf{z}$ satisfies the bounds [158]; line 12 checks for it, so that a hard constraint that cannot be met is reported rather than iterated on forever.

The multipliers are more than a by-product. The multiplier y_i of a constraint row tells you how much the optimal cost would drop if that row were relaxed by one unit; a row with $y_i > 0$ is **active** and shapes the solution, a row with $y_i = 0$ could be deleted without changing anything. In the worked example we read off which avoidance step is active from exactly these numbers.

21.4.4 The MPC loop

Algorithm 21.2 is the receding-horizon principle of Figure 21.1 with the quadratic program inside. Everything that depends only on the model is computed once (line 2): the prediction matrices and the Hessian $\mathbf{H}$, which does not change from step to step because the cost weights do not. At every step the loop measures the state and collects the data for the N future times

Algorithm 21.2. Model predictive control loop with linearized avoidance and keep-in constraints

Input: model $(\mathbf{A}, \mathbf{B})$, weights $(\mathbf{Q}, \mathbf{R}, \mathbf{P}_f)$, horizon N , limits $a_{\max}, v_{\max}$, reference $\mathbf{x}^{\text{ref}}(\cdot)$, predicted obstacles $\hat{\mathbf{o}}_k$ with radii r_k , keep-in centers $\mathbf{c}_k$ with radii e

Output: the input $\mathbf{u}_t$ applied at every step

```

1 Function MpcLoop():
2   precompute  $\mathbf{S}_x, \mathbf{S}_u$  and  $\mathbf{H}$  of Equation (21.8);  $\mathbf{U}^{\text{prev}} \leftarrow \mathbf{0}$ 
3   for  $t = 0, \Delta t, 2\Delta t, \dots$  do
4     measure  $\mathbf{x}_t$ ; fetch  $\mathbf{x}_k^{\text{ref}}, \hat{\mathbf{o}}_k, \mathbf{c}_k$  for the times  $t + k\Delta t, k = 1, \dots, N$ 
5      $(\mathbf{u}_t, \mathbf{U}^{\text{prev}}) \leftarrow \text{MpcStep}(\mathbf{x}_t, \text{references}, \text{obstacles}, \text{keep-in disks}, \mathbf{U}^{\text{prev}})$ 
6     apply  $\mathbf{u}_t$  to the drone for one step
7 Function MpcStep( $\mathbf{x}_0, \text{references}, \text{obstacles}, \text{keep-in disks}, \mathbf{U}^{\text{prev}}$ ):
8    $\bar{\mathbf{U}} \leftarrow \mathbf{U}^{\text{prev}}$  without its first input, with the last input repeated
9    $\bar{\mathbf{X}} \leftarrow \mathbf{S}_x \mathbf{x}_0 + \mathbf{S}_u \bar{\mathbf{U}}$  // linearization point
10   $\mathbf{f} \leftarrow 2\mathbf{S}_u^T \mathbf{Q}(\mathbf{S}_x \mathbf{x}_0 - \mathbf{X}^{\text{ref}})$ , extended by  $w_1 \mathbf{1}$  for the slacks
11  add the input and velocity boxes to  $(\mathbf{G}, \mathbf{l}, \mathbf{u}_{\max})$ 
12  foreach obstacle and step  $k = 1, \dots, N$  do
13     $\mathbf{n}_k \leftarrow (\bar{\mathbf{p}}_k - \hat{\mathbf{o}}_k) / \|\bar{\mathbf{p}}_k - \hat{\mathbf{o}}_k\|$  (use  $\mathbf{p}_k^{\text{ref}}$  instead of  $\bar{\mathbf{p}}_k$  if the two coincide)
14    add the row  $-\mathbf{n}_k^T \mathbf{S}_{u,k}^p \mathbf{U} - s_k \leq \mathbf{n}_k^T ((\mathbf{S}_x \mathbf{x}_0)_k^p - \hat{\mathbf{o}}_k) - r_k$ 
15  foreach keep-in disk, step  $k$  and direction  $m = 1, \dots, M$  do
16    add the row  $\mathbf{n}_m^T \mathbf{S}_{u,k}^p \mathbf{U} \leq e \cos(\pi/M) - \mathbf{n}_m^T ((\mathbf{S}_x \mathbf{x}_0)_k^p - \mathbf{c}_k)$ 
17   $(\mathbf{z}, \mathbf{y}, \text{status}) \leftarrow \text{SolveQP}(\mathbf{H}, \mathbf{f}, \mathbf{G}, \mathbf{l}, \mathbf{u}_{\max}, \text{warm start } (\bar{\mathbf{U}}, \mathbf{0}) \text{ and the shifted multipliers})$ 
18  if status  $\neq$  solved then
19    return  $(\Pi_{[-a_{\max}, a_{\max}]}(-\mathbf{v}_0/\Delta t), \bar{\mathbf{U}})$  // fallback: brake
20   $\mathbf{U}^* \leftarrow$  the first  $nN$  entries of  $\mathbf{z}$ 
21  return  $(\mathbf{u}_0^*, \mathbf{U}^*)$ 

```

(line 4): the reference from the path the drone is following, the intruder predictions from the prediction layer, the keep-in centers from the communicated plans of its partners. The step function first builds the **warm start** (line 8): the previous plan minus the input that has just been applied, with the last input repeated to fill the vacated slot. This shifted plan is the linearization point $\bar{\mathbf{X}}$ for the avoidance half-planes (line 13) and the initial guess of the solver (line 17). Only the linear term $\mathbf{f}$ of the cost depends on the current state and reference (line 10). Lines 11 to 16 assemble the constraint blocks of Equation (21.9). The solver returns either a plan, of which line 21 applies the first input and keeps the rest for the next warm start, or a failure, in which case line 19 brakes as hard as the limits allow. The fallback is a design decision; Section 21.8 discusses the alternatives. With soft constraints the solver essentially never fails, which is the strongest argument for them.

21.5 A worked example

Example 21.5 (An intruder crosses the reference path). A planar drone with $a_{\max} = 2 \text{ m/s}^2$ (per axis), $v_{\max} = 2 \text{ m/s}$ and $\Delta t = 0.1 \text{ s}$ starts at the origin with velocity $(1, 0) \text{ m/s}$ and follows the reference $\mathbf{p}^{\text{ref}}(t) = (t, 0) \text{ m}$, $\mathbf{v}^{\text{ref}} = (1, 0) \text{ m/s}$. An intruder starts at $(4.5, -3.0) \text{ m}$ and moves with the constant velocity $(0, 0.8) \text{ m/s}$, so that it crosses the reference line at $t = 3.75 \text{ s}$, when the drone would be 0.75 m behind the crossing point. The required

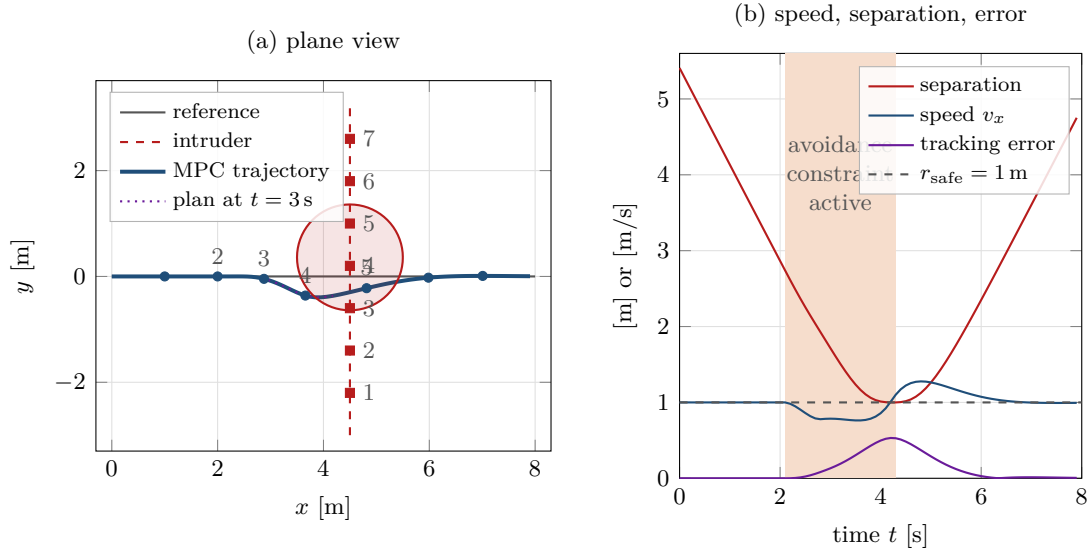


Figure 21.3. Example 21.5. (a) The reference (gray), the MPC trajectory (blue, dots every second), the intruder's path (red dashed, squares every second) and the safety disk at the moment of closest approach, $t = 4.2$ s. The dotted curve is the plan computed at $t = 3$ s: it slows down and dips below the reference line, on the side the intruder comes from, so that the intruder passes in front. (b) Speed, separation and tracking error over time; the shaded band marks the steps at which an avoidance constraint is active. The separation touches $r_{\text{safe}} = 1$ m and never goes below it.

separation is $r_{\text{safe}} = 1$ m. The prediction layer reports the intruder's constant-velocity extrapolation exactly. The controller uses $N = 15$ (a horizon of 1.5 s), $\mathbf{Q} = \text{diag}(1, 1, 0.1, 0.1)$, $\mathbf{R} = 0.1 \mathbf{I}$, the terminal weight from the Riccati equation,

$$\mathbf{P}_f = \begin{pmatrix} 9.078 & 0 & 3.166 & 0 \\ 0 & 9.078 & 0 & 3.166 \\ 3.166 & 0 & 2.766 & 0 \\ 0 & 3.166 & 0 & 2.766 \end{pmatrix},$$

and hard avoidance constraints. The simulation runs for 8 s (80 steps) with the exact model Equation (21.1) as the plant.

Figure 21.3 and Table 21.2 show what happens. For the first two seconds the drone is exactly on the reference and every input is zero: the predicted intruder positions are far from every plan point, all avoidance rows have zero multipliers, and the unconstrained minimizer of the cost is “do nothing”. At $t = 2.1$ s the last point of the shifted plan, $\mathbf{p}_{15}$ at time 3.6 s, is only 0.91 m from the intruder's predicted position (4.5, -0.12) m at that time. The half-plane for $k = 15$ becomes active (its multiplier is 16.2 at $t = 2.5$ s): from now on the constraint, not the cost, shapes the end of the plan. Its normal $\mathbf{n}_{15}$ points from the predicted intruder back toward the drone, and as the predicted positions of the later steps climb above the line, their normals tilt downward; the cheapest way to satisfy them is to arrive later and lower. The drone brakes ($a_x = -0.416$ m/s² at $t = 2.5$ s) and starts to move below the line ($a_y < 0$).

As the horizon slides forward, the active step moves down the horizon: $k = 12$ and 13 at $t = 3.0$ s, $k = 7$ and 8 at $t = 3.5$ s, $k = 2$ and 3 at $t = 4.0$ s. The drone is now at the boundary of the safety disk and follows it: the separation is 1.012 m at $t = 4.0$ s, reaches its minimum of exactly 1.000 m at $t = 4.2$ s (the polished solution satisfies the active row to machine precision, and the plant is the model), and grows again as the intruder moves on. The

Table 21.2. Trace of Algorithm 21.2 on Example 21.5: every half second from $t = 2$ s to $t = 5$ s, then every second (the first two seconds are identical to the row $t = 2.0$, all inputs zero). State, applied input, separation from the intruder, tracking error, the horizon steps k whose avoidance constraint is active, and the largest multiplier. All numbers are printed by `ch21_mpc.py`.

t [s]	x	y	v_x	v_y	a_x	a_y	sep.	error	active k	max y_i
2.0	2.000	0.000	1.000	0.000	0.000	0.000	2.865	0.000	–	0
2.5	2.477	0.000	0.861	−0.010	−0.416	−0.214	2.257	0.023	15	16.2
3.0	2.876	−0.045	0.786	−0.199	−0.016	−0.447	1.716	0.132	12, 13	14.2
3.5	3.264	−0.192	0.763	−0.360	−0.001	−0.080	1.236	0.304	7, 8	14.7
4.0	3.658	−0.362	0.858	−0.254	0.640	0.817	1.012	0.498	2, 3	14.7
4.5	4.183	−0.366	1.223	0.220	0.318	0.458	1.016	0.484	–	0
5.0	4.815	−0.222	1.264	0.300	−0.150	−0.140	1.262	0.289	–	0
6.0	5.983	−0.023	1.074	0.092	−0.140	−0.166	2.351	0.029	–	0
7.0	7.009	0.010	0.998	−0.001	−0.019	−0.025	3.606	0.013	–	0

intruder passes in front and above: at closest approach the drone is at (3.84, −0.39) m and the intruder at (4.50, 0.36) m, and the drone never dips below $y = -0.395$ m. It never stops ($v_x \geq 0.763$ m/s) because it is cheaper, under **Q** and **R**, to keep moving on the far side of the disk than to wait. After $t = 4.2$ s no avoidance row is active any more. The drone is now 0.53 m behind the reference and accelerates to 1.277 m/s at $t = 4.8$ s to catch up; the velocity bound $v_{\max} = 2$ m/s is never reached because the quadratic cost prefers a gentle recovery. By $t = 6.5$ s the tracking error is below 1 cm. Over the whole run the RMS tracking error is 0.208 m and the largest error 0.532 m, both at the price of the intruder.

Two details of the trace are worth a second look. First, two consecutive steps are usually active at once (12 and 13, 7 and 8): the plan touches the disk along a short arc, not at a point, because the half-planes of neighboring steps are nearly parallel and the drone slides along the disk for two steps. Second, the multipliers stay between 14 and 19 while the constraint is active, and the largest over the whole run is 18.7. This number is the price, in units of the cost, of one more meter of clearance, and it is the number that decides how large the slack weight w_1 of a soft constraint must be (Proposition 21.7, Section 21.7.1).

21.6 Properties: stability, feasibility and cost

MPC has a clean theory for the *nominal* case: a fixed reference, a correct model, and constraints that do not move. This section states that theory at the level of a first course, following Mayne et al. [115] and Rawlings, Mayne, and Diehl [134], and then says honestly which parts survive when an intruder moves and a half-plane is linearized around a guess.

21.6.1 Terminal cost, terminal set and stability

Consider regulation to the origin ($\mathbf{x}^{\text{ref}} = \mathbf{0}$), the stage cost $\ell(\mathbf{x}, \mathbf{u}) = \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{u}^\top \mathbf{R} \mathbf{u}$ with $\mathbf{Q}, \mathbf{R} \succ \mathbf{0}$, the terminal cost $V_f(\mathbf{x}) = \mathbf{x}^\top \mathbf{P}_f \mathbf{x}$, and constraint sets $\mathbf{x}_k \in \mathcal{X}$, $\mathbf{u}_k \in \mathcal{U}$ that contain the origin. Write $V_N(\mathbf{x})$ for the optimal cost of Equation (21.2) from the state $\mathbf{x}$ (the **value function**). For this section we let the stage sum start at $k = 0$, that is $J = \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + V_f(\mathbf{x}_N)$; because $\mathbf{x}_0$ is fixed data, the extra term $\mathbf{x}_0^\top \mathbf{Q} \mathbf{x}_0$ is a constant and the minimizer of Equation (21.2) is unchanged, but the bookkeeping of the proof below becomes the familiar one. Two **terminal ingredients** turn MPC into a provably stable controller:

- (i) a *terminal set* $\mathcal{X}_f \subseteq \mathcal{X}$ containing the origin and a *terminal controller* κ_f with $\kappa_f(\mathbf{x}) \in \mathcal{U}$ and $\mathbf{A}\mathbf{x} + \mathbf{B}\kappa_f(\mathbf{x}) \in \mathcal{X}_f$ for every $\mathbf{x} \in \mathcal{X}_f$ (the set is *invariant* under κ_f);
- (ii) a terminal cost that decreases at least as fast as the stage cost under κ_f : $V_f(\mathbf{A}\mathbf{x} + \mathbf{B}\kappa_f(\mathbf{x})) - V_f(\mathbf{x}) \leq -\ell(\mathbf{x}, \kappa_f(\mathbf{x}))$ for every $\mathbf{x} \in \mathcal{X}_f$;

together with the *terminal constraint* $\mathbf{x}_N \in \mathcal{X}_f$ added to the problem.

Theorem 21.6 (Recursive feasibility and stability of nominal MPC). *Under (i) and (ii), if the problem is feasible at $\mathbf{x}_t$, then it is feasible at $\mathbf{x}_{t+1} = \mathbf{A}\mathbf{x}_t + \mathbf{B}\mathbf{u}_0^*(\mathbf{x}_t)$, and*

$$V_N(\mathbf{x}_{t+1}) \leq V_N(\mathbf{x}_t) - \ell(\mathbf{x}_t, \mathbf{u}_0^*(\mathbf{x}_t)). \quad (21.10)$$

*Consequently the problem stays feasible forever (**recursive feasibility**) and the origin is asymptotically stable for every initially feasible state.*

Proof sketch. Let $\mathbf{u}_0^*, \dots, \mathbf{u}_{N-1}^*$ be the optimal sequence at $\mathbf{x}_t$ with predicted states $\mathbf{x}_1^*, \dots, \mathbf{x}_N^*$. At $\mathbf{x}_{t+1} = \mathbf{x}_1^*$ consider the *shifted candidate* $(\mathbf{u}_1^*, \dots, \mathbf{u}_{N-1}^*, \kappa_f(\mathbf{x}_N^*))$. Its first $N-1$ states are $\mathbf{x}_2^*, \dots, \mathbf{x}_N^*$, which satisfy the constraints because they did before; its last state is $\mathbf{A}\mathbf{x}_N^* + \mathbf{B}\kappa_f(\mathbf{x}_N^*)$, which lies in $\mathcal{X}_f$ by (i) because $\mathbf{x}_N^* \in \mathcal{X}_f$; and its last input $\kappa_f(\mathbf{x}_N^*)$ is admissible by (i). So the candidate is feasible, which proves recursive feasibility. Its cost is the old optimal cost minus the first stage, minus the old terminal cost, plus the new stage at $\mathbf{x}_N^*$ and the new terminal cost: $V_N(\mathbf{x}_t) - \ell(\mathbf{x}_t, \mathbf{u}_0^*) - V_f(\mathbf{x}_N^*) + \ell(\mathbf{x}_N^*, \kappa_f(\mathbf{x}_N^*)) + V_f(\mathbf{A}\mathbf{x}_N^* + \mathbf{B}\kappa_f(\mathbf{x}_N^*))$, and (ii) says that the last three terms add up to at most zero. The optimal cost at $\mathbf{x}_{t+1}$ is no larger than the cost of this candidate, which is Equation (21.10). Since $\ell(\mathbf{x}, \mathbf{u}) \geq \lambda_{\min}(\mathbf{Q}) \|\mathbf{x}\|^2$, V_N strictly decreases away from the origin; it is a Lyapunov function, and the standard argument gives asymptotic stability [134, Ch. 2]. $\square$

The natural terminal ingredients for the double integrator come from LQR. Let $\mathbf{P}_f$ be the solution of the discrete algebraic **Riccati equation**

$$\mathbf{P}_f = \mathbf{Q} + \mathbf{A}^\top \mathbf{P}_f \mathbf{A} - \mathbf{A}^\top \mathbf{P}_f \mathbf{B} (\mathbf{R} + \mathbf{B}^\top \mathbf{P}_f \mathbf{B})^{-1} \mathbf{B}^\top \mathbf{P}_f \mathbf{A}, \quad \mathbf{K} = (\mathbf{R} + \mathbf{B}^\top \mathbf{P}_f \mathbf{B})^{-1} \mathbf{B}^\top \mathbf{P}_f \mathbf{A}, \quad (21.11)$$

which the code computes by iterating the right-hand side until it stops changing (**dare**). With $\kappa_f(\mathbf{x}) = -\mathbf{K}\mathbf{x}$ condition (ii) holds *with equality*: $(\mathbf{A} - \mathbf{B}\mathbf{K})^\top \mathbf{P}_f (\mathbf{A} - \mathbf{B}\mathbf{K}) - \mathbf{P}_f = -(\mathbf{Q} + \mathbf{K}^\top \mathbf{R}\mathbf{K})$ is a rearrangement of Equation (21.11). In words, $\mathbf{x}^\top \mathbf{P}_f \mathbf{x}$ is the infinite-horizon cost of the unconstrained LQR from $\mathbf{x}$, so the finite-horizon problem with this terminal cost “knows” what happens after the horizon, and without active constraints MPC *is* LQR. The terminal set is then any invariant set of $\mathbf{A} - \mathbf{B}\mathbf{K}$ inside which the LQR law respects the input and velocity bounds; for a double integrator it is a sizable ellipsoid around the origin. Most implementations, including the book’s code, drop the explicit terminal constraint and rely on the terminal cost and a reasonably long horizon; the theory says that for a long enough horizon the terminal constraint would be inactive anyway on a large region of states [134], and the practice says that with the Riccati $\mathbf{P}_f$ the closed loop behaves. What you should not do is take $\mathbf{P}_f = \mathbf{Q}$ with a short horizon: such a controller is myopic and can be unstable even without constraints.

21.6.2 Feasibility, soft constraints and the exact penalty

Theorem 21.6 gives feasibility for free because the shifted plan is always a feasible candidate. That argument needs the constraints of step $k+1$ at time t to be the constraints of step k at time $t+1$, which is true for a geofence and for the drone’s own bounds and *false* for an intruder that moves differently from its prediction, for an obstacle that appears, and for a partner that changes its plan. In those cases a hard avoidance constraint can become infeasible from one step to the next, and soft constraints are the practical answer. They cost nothing when the hard problem is feasible:

Proposition 21.7 (Exact penalty). *Let $\mathbf{z}^*$ solve the hard QP with multipliers $\lambda_k^* \geq 0$ for the rows $g_k(\mathbf{z}) \leq 0$, and let the soft QP replace them by Equation (21.4) with $w_1 \geq \max_k \lambda_k^*$ and $w_2 \geq 0$. Then $(\mathbf{z}^*, \mathbf{s} = \mathbf{0})$ solves the soft QP.*

Proof. The soft problem is convex, so the Karush–Kuhn–Tucker conditions are sufficient. Take the multipliers λ_k^* for the rows $g_k(\mathbf{z}) - s_k \leq 0$ and $\mu_k = w_1 - \lambda_k^* \geq 0$ for the rows $-s_k \leq 0$. Stationarity in $\mathbf{z}$ is the stationarity of the hard problem; stationarity in s_k reads $w_1 + 2w_2s_k - \lambda_k^* - \mu_k = 0$, which holds at $s_k = 0$; primal feasibility, dual feasibility and complementary slackness hold because they held for the hard problem and $s_k = 0$ is active with $\mu_k \geq 0$. $\square$

The MPC form of this classical exact-penalty result (exact penalties themselves go back to the nonlinear-programming literature of the nineteen sixties and seventies) is due to Kerrigan and Maciejowski [81]; it tells you how to choose the slack weight: look at the multipliers of the hard problem in normal operation and take w_1 safely above the largest one. In the worked example the multipliers never exceed 19, so $w_1 = 50$ reproduces the hard trajectory exactly, whereas $w_1 = 1$ is below the price of a meter of clearance and the solver buys a violation (Section 21.7.1). The quadratic term $w_2s_k^2$ does not affect the threshold; it only shapes *how* a violation is spread when one is unavoidable.

21.6.3 What can go wrong

The honest list, each item taken up in Section 21.8:

- *The half-plane is an approximation* built around the previous plan, so a plan on the wrong side of the intruder can cost a long detour or make the drone alternate between the two sides (Section 21.8, pitfall “Linearizing around a stale trajectory”).
- *Moving obstacles break the shifted-candidate argument*: a hard avoidance constraint can be infeasible at step $k = 1$ when an intruder is detected late, and then only soft constraints or a fallback produce an input at all (Section 21.8, pitfall “Hard constraints that become infeasible”).
- *The solver is numerical*. The constraints hold to the solver’s tolerance (a fraction of a millimeter after polishing, 10^{-4} relative without), at the sample times only, and for the model. Put the discretization error, the model error and the solver tolerance into the margin of r_k .
- *Stability is nominal*. With a moving reference and moving constraints, Theorem 21.6 is a statement about the design (the terminal cost is the right one) rather than a guarantee for the flight. Robust and tube MPC [134] recover guarantees for bounded disturbances at the price of conservatism; the hybrid architecture of Chapter 24 instead layers a reactive safety net under the MPC.

21.6.4 The cost of a step

Building the condensed problem once costs $\mathcal{O}(N^3n^3)$ for $\mathbf{S}_u^T \bar{\mathbf{Q}} \mathbf{S}_u$ and is negligible. Per step, forming $\mathbf{f}$ and the constraint rows costs $\mathcal{O}((nN)^2)$; each ADMM iteration costs one product with $\mathbf{M}^{-1}$ and two with $\mathbf{G}$, i.e. $\mathcal{O}((nN)^2 + mnN)$ for m rows; and every change of ρ costs a new factorization, $\mathcal{O}((nN)^3)$. The number of iterations does not grow with N in any simple way; it grows with how far the warm start is from the solution and with how ill-conditioned the active constraints are. For a horizon of a few hundred steps, or for a centralized problem over many drones, the condensed Hessian becomes dense and large, and the **sparse formulation** that keeps the states as variables and the dynamics as equality constraints is preferable: its KKT matrix is banded and one iteration costs $\mathcal{O}(N)$ [134, 158].

Table 21.3. The knobs of the controller and what turning them does.

Knob	Effect
Horizon N	Longer: constraints are seen earlier, maneuvers are gentler, closer to the infinite-horizon optimum; but the dense condensed Hessian grows quadratically with N and a refactorization cubically, and predictions of others far ahead are unreliable. Must exceed the stopping time $v/a_{\max}$ and the reaction time of the prediction layer.
Time step Δt	Smaller: finer control and constraints checked more often, more steps for the same horizon length; the model stays exact. Bounded below by the solve time and the flight controller's rate.
q_p (position weight)	Larger: tighter tracking, harder accelerations, larger multipliers (raise w_1 with it).
q_v (velocity weight)	Larger: the drone matches the reference speed even when its position lags; too large and it never catches up.
r (input weight)	Larger: smoother, lazier; the recovery after a disturbance takes longer.
$\mathbf{P}_f$ (terminal weight)	Riccati solution: consistent with the infinite horizon, stable design. $\mathbf{P}_f = \mathbf{Q}$: myopic, avoid with short horizons.
w_1, w_2 (slack weights)	w_1 above the largest multiplier makes the penalty exact; $w_2 > 0$ spreads an unavoidable violation over the horizon.
$r_k, e_{\max}, M$	Margins absorb discretization, model and solver error; more polygon directions M reduce the loss $1 - \cos(\pi/M)$.

21.7 Variants and extensions

21.7.1 Tuning the horizon, the weights and the time step

Table 21.3 summarizes the effect of each parameter, and Figure 21.4 shows the two that matter most on Example 21.5. Panel (a) varies the horizon with hard constraints. With $N = 20$ the controller sees the intruder 2 s ahead and reduces its speed gently to 0.76 m/s; with $N = 10$ it reacts later and brakes to 0.56 m/s; with $N = 5$ it notices the conflict only 0.5 s before it, brakes almost to a standstill (0.22 m/s) and needs longer to recover (RMS error 0.227 m against 0.207 to 0.209 m for $N \geq 10$). All three keep the separation at 1.000 m, because the hard constraint is feasible in each case; with $N = 3$ it is not, and Section 21.7.3 shows what happens then. Panel (b) varies the slack weight with soft constraints and $N = 15$. With $w_1 = w_2 = 100$ the trajectory is the hard one (minimum separation 1.000 m); with $w = 10$ the penalty is slightly below the multipliers of 14 to 19 and the drone gives up 2 mm of separation (0.998 m); with $w = 1$ the penalty is cheap, the drone barely deviates (RMS error 0.126 m) and cuts 0.22 m into the safety disk (0.777 m). The lesson is the one of Proposition 21.7: the slack weight is not a tuning knob for comfort, it must dominate the multipliers, and the trajectory should then be identical to the hard one.

A word on units, because it is the most common tuning mistake. The entry q_p multiplies squared meters, q_v squared meters per second, r squared meters per second squared; the numbers 1, 0.1 and 0.1 of the example are therefore not “ten times more weight on position” but a statement that a position error of 1 m costs as much as a velocity error of 3.2 m/s or an acceleration of 3.2 m/s². If you rescale positions to centimeters, q_p must shrink by 10^4 to leave the controller unchanged (Exercise 21.4). Choose the weights by asking what error you are willing to trade for what effort, and check the answer on a plot such as Figure 21.4.

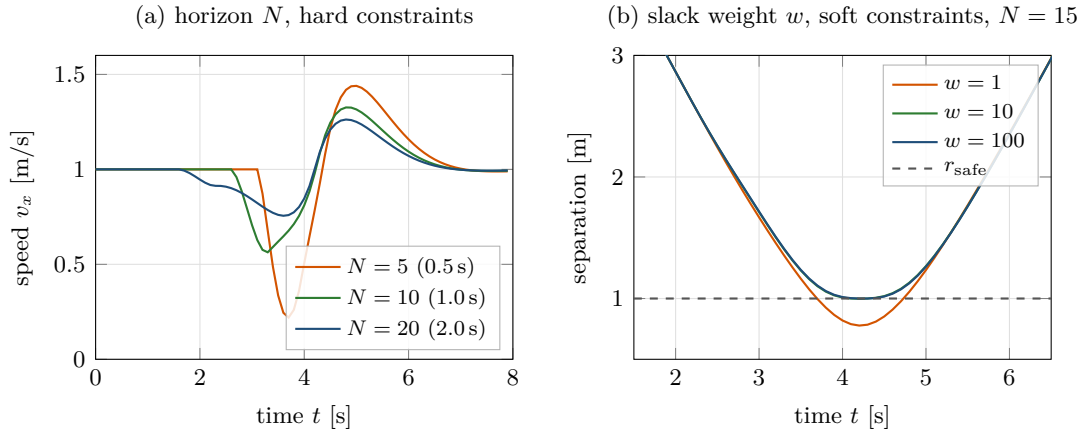


Figure 21.4. Tuning on Example 21.5. (a) Speed over time for three horizons with hard constraints: a short horizon reacts late and violently, a long one early and gently. (b) Separation over time for three slack weights with soft constraints and $N = 15$: with $w = 1$ the drone cuts into the safety disk (dashed line), with $w = 100$ the soft controller reproduces the hard solution.

21.7.2 Prediction-aware constraints

So far the intruder’s predicted position $\hat{\mathbf{o}}_k$ entered the constraint as if it were exact. It is not: the Kalman filter of Chapter 18 delivers a mean and a covariance, and the predictors of Chapter 20 deliver a distribution whose spread grows with the prediction horizon. Treating the mean as the truth is the pitfall that the training plan singles out as a research topic (“prediction-aware constraints that include uncertainty rather than treating the predicted path as exact”), and the half-plane form makes it easy to avoid.

Let the intruder’s position at step k be Gaussian, $\mathbf{o}_k \sim \mathcal{N}(\hat{\mathbf{o}}_k, \Sigma_k)$, and demand that the half-plane constraint hold with probability at least $1 - \delta$:

$$\mathbb{P}(\mathbf{n}_k^\top(\mathbf{p}_k - \mathbf{o}_k) \geq r_{\text{safe}}) \geq 1 - \delta. \quad (21.12)$$

The scalar $\mathbf{n}_k^\top(\mathbf{p}_k - \mathbf{o}_k)$ is Gaussian with mean $\mathbf{n}_k^\top(\mathbf{p}_k - \hat{\mathbf{o}}_k)$ and variance $\mathbf{n}_k^\top \Sigma_k \mathbf{n}_k$ (Chapter 2), so Equation (21.12) is equivalent to the *linear* constraint

$$\mathbf{n}_k^\top(\mathbf{p}_k - \hat{\mathbf{o}}_k) \geq r_{\text{safe}} + \kappa_\delta \sqrt{\mathbf{n}_k^\top \Sigma_k \mathbf{n}_k}, \quad \kappa_\delta = \Phi^{-1}(1 - \delta), \quad (21.13)$$

where Φ is the standard normal distribution function ($\kappa_{0.1} = 1.28$, $\kappa_{0.05} = 1.64$, $\kappa_{0.01} = 2.33$). This is the **chance constraint** of Zhu and Alonso-Mora [187], who use it for micro aerial vehicles among uncertain obstacles: the same half-plane as before with the radius inflated by the standard deviation of the prediction *in the direction of the normal*, times a confidence factor (Figure 21.5). By Proposition 21.3 the disk constraint then holds with at least the same probability. Two remarks. The guarantee is per step and per obstacle; if you want the whole horizon safe with probability $1 - \delta$, Boole’s inequality suggests δ/N per step, which is conservative but simple. And the inflation grows with k : for a constant-velocity prediction with position covariance Σ_0 and velocity covariance Σ_v , $\Sigma_k = \Sigma_0 + (k\Delta t)^2 \Sigma_v$, so the disk at the end of the horizon is much larger than the one at its start; the Kalman filter propagates Σ_k exactly with its own $\mathbf{A}$ and process noise. In the code, `MPC(delta=0.05)` together with an obstacle that carries its covariances applies Equation (21.13). On Example 21.5 with $\Sigma_0 = (0.05 \text{ m})^2 \mathbf{I}$ and $\Sigma_v = (0.2 \text{ m/s})^2 \mathbf{I}$, the inflation reaches 0.50 m at $k = 15$ and the minimum separation rises from 1.000 m to 1.143 m. The price of caution is visible and adjustable, which is the whole point.

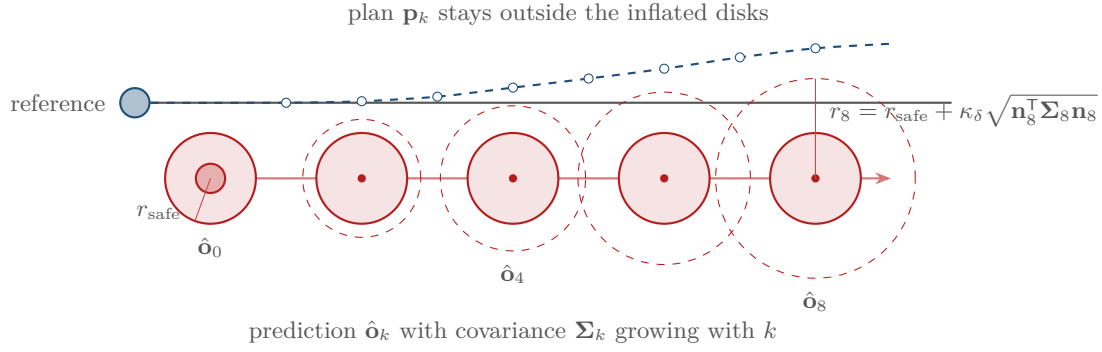


Figure 21.5. Prediction-aware avoidance. The intruder’s predicted positions $\hat{o}_k$ carry covariances Σ_k that grow with k ; the chance constraint Equation (21.13) inflates the safety radius by κ_δ standard deviations along the normal (dashed circles), and the plan keeps clear of the inflated disks. The reference, safe against the nominal disks, would violate the chance constraint late in the horizon.

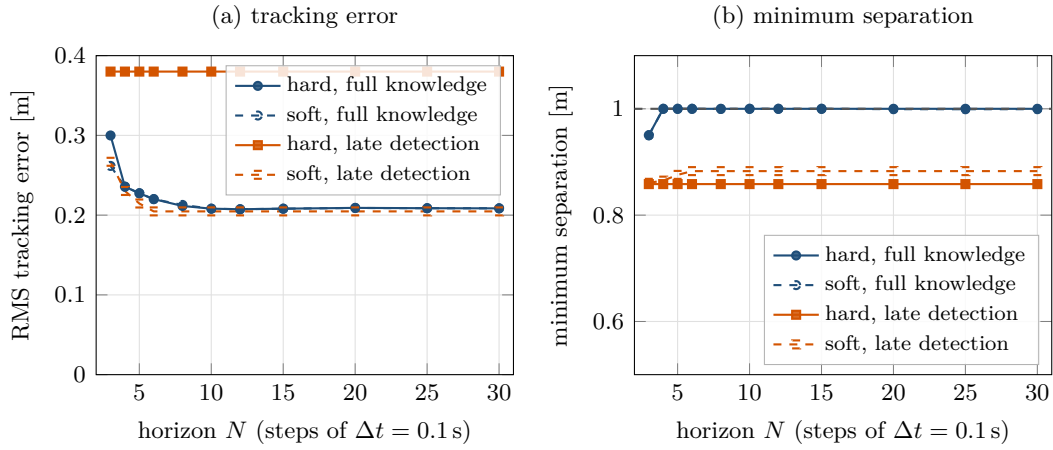


Figure 21.6. RMS tracking error (a) and minimum separation (b) as functions of the horizon, for hard and soft avoidance constraints, with full knowledge of the intruder and with late detection at 1.2 m. With full knowledge hard and soft coincide for $N \geq 4$; at $N = 3$ the horizon is shorter than the stopping time and both miss the safety disk. With late detection the hard problem is infeasible on nine steps for every N and the braking fallback loses both separation and tracking, whereas the soft problem returns the least-violation input and does better on both counts.

21.7.3 A generated experiment: hard versus soft constraints

Figure 21.6 and Table 21.4 report the experiment produced by `gen_ch21_mpc.py`: Example 21.5 run for eleven horizons from $N = 3$ to $N = 30$, with hard and with soft avoidance constraints, and in two information regimes. With *full knowledge* the intruder’s prediction is available from the start. With *late detection* the drone learns about the intruder only when it is within 1.2 m, a stand-in for a short sensor range or a late track initialization (Chapter 18).

Three things stand out. First, with full knowledge, hard and soft constraints give the same closed loop for every $N \geq 4$, to within the solver tolerance: the exact penalty at work. Second, $N = 3$ is a horizon of 0.3 s, shorter than the 0.5 s the drone needs to stop from 1 m/s; the hard problem is infeasible on seven steps, the drone falls back on braking, and both variants end up 5 cm inside the safety disk. Beyond $N \approx 10$ nothing improves: the intruder’s prediction is exact here, but with a real predictor the far end of a long horizon would be pure noise. Third, and most important for the swarm, late detection makes the hard problem infeasible on nine consecutive steps for *every* horizon, because the violation is already inside the first steps when

Table 21.4. Selected rows of the experiment of Figure 21.6: RMS tracking error, minimum separation and number of failed (infeasible) solves out of 80.

N	full knowledge						late detection (1.2 m)					
	hard			soft			hard			soft		
	RMS	sep.	fail	RMS	sep.	fail	RMS	sep.	fail	RMS	sep.	fail
3	0.300	0.950	7	0.262	0.950	0	0.380	0.858	9	0.267	0.861	0
5	0.227	1.000	0	0.228	1.000	0	0.380	0.858	9	0.215	0.876	0
10	0.208	1.000	0	0.208	1.000	0	0.380	0.858	9	0.205	0.883	0
20	0.209	1.000	0	0.209	0.999	0	0.380	0.858	9	0.205	0.883	0
30	0.208	1.000	0	0.209	0.999	0	0.380	0.858	9	0.205	0.883	0

the intruder appears. The braking fallback keeps the drone 0.86 m from the intruder and costs an RMS error of 0.38 m. The soft problem never fails: it returns the input that violates the constraints least, keeps 0.88 m of separation and tracks with an RMS error of 0.205 m, better on both counts. This is what “degrades gracefully” means, and it is the reason Table 21.1 makes every constraint that depends on someone else’s prediction soft.

21.7.4 Beyond the linear quadratic program

Nonlinear MPC keeps the full quadrotor model (attitude, thrust, drag) and solves a nonlinear program at every step, usually by sequential quadratic programming, in which each iteration is a QP of the kind above; for trajectories that the double integrator cannot represent, the differential-flatness result of Mellinger and Kumar [117] lets you plan in position space and track with the full model. *Explicit MPC* [21] precomputes the piecewise-affine control law $\mathbf{u}_0^*(\mathbf{x})$ offline for small problems, so that the online work is a table lookup. *Robust and stochastic MPC* [134] extend the chance constraint idea to the drone’s own disturbances. *Centralized multi-drone MPC* stacks the inputs of all m drones into one program; the formation and communication constraints then couple variables of different drones but stay convex, and the program grows to mnN variables. The *sequential* alternative, which the book’s code uses for a leader–follower pair, solves one QP per drone in priority order with the plans of the higher-priority drones as data; it is cheaper and distributed but, like prioritized planning (Chapter 8), it can fail where a joint solution exists. Finally, when a decision is genuinely discrete (pass left or right, which corridor), the linearized half-plane is a heuristic, and the mixed-integer formulation of Chapter 22 makes the choice explicit at a much higher computational price.

21.8 Implementation notes

Warm starting. The shifted plan is a good guess for the primal variables and, block by block, for the multipliers: the multiplier of step $k + 1$ becomes the guess for step k . On Example 21.5 the warm-started ADMM needs 26 iterations per step on average (at most 280, while the constraint is active), a cold start 47 (at most 420). The primal warm start is what matters; a stale dual warm start can even slow the iteration down, which is why the solver resets the multipliers once if it has not converged after 2 500 iterations.

Solve time. With 30 variables and 75 rows (the hard program of Example 21.5; 45 variables and 90 rows once the avoidance rows are softened), one step of the planar example takes a few milliseconds in NumPy on a laptop: about 1.3 ms on average and under 10 ms at worst on the machine that produced the numbers of this chapter. The figure is hardware dependent, but

it is an order of magnitude below a Δt of 0.1 s. Row scaling and polishing are not optional: without scaling the same problems take thousands of iterations, and without polishing the active constraint holds only to 10^{-4} relative accuracy. Compute $\mathbf{S}_x$, $\mathbf{S}_u$ and $\mathbf{H}_{uu}$ once; only $\mathbf{f}$ and the constraint rows change per step.

When the solver fails. A QP solver fails in two ways: it reports infeasibility (a hard constraint cannot be met), or it hits its iteration cap. The code’s policy is the simplest safe one, to brake, and to keep the shifted plan as the next warm start. Better policies exist: apply the next input of the previous plan (it was feasible one step ago), re-solve with the constraints softened, or hand control to the local safety layer of [Chapter 24](#). Whatever the policy, log every failure: a controller that fails silently is the classic cause of an “unexplained” collision in a simulation study ([Chapter 25](#)).

Per drone or centralized. Running one MPC per drone with communicated plans scales linearly with the swarm and matches the architecture of [Chapter 24](#), in which each drone executes its own CBS path. It requires that a drone receives its partner’s plan for the same horizon at every step; if communication is delayed, use the partner’s reference instead and widen $e_{\max}$. A centralized MPC is worth it for small tightly coupled groups, or as the ground truth against which the sequential scheme is measured.

[Listing 21.1](#) shows how the avoidance rows of [Equation \(21.9\)](#) are assembled in `MPC.step`, including the chance-constraint inflation, and [Listing 21.2](#) the core of the ADMM solver, [Algorithm 21.1](#) in some twenty lines.

Listing 21.1. Linearized avoidance rows in `MPC.step` (from `code/ch21_mpc.py`). `X_guess` is the shifted previous plan, `free` the free response $\mathbf{S}_x \mathbf{x}_0$, `pos_idx[k]` the indices of $\mathbf{p}_k$ in the stacked state.

```

1  # (3) one linearized half-plane per obstacle and step
2  avoid_rows, avoid_lo, avoid_up = [], [], []
3  avoid_k, normals = [], []
4  for obs in obstacles:
5      for k in range(N):
6          pk = self.pos_idx[k]
7          d = X_guess[pk] - obs.centers[k]
8          dist = float(np.linalg.norm(d))
9          if dist < 1e-9:      # on the center: use the reference
10             d = xref[pk] - obs.centers[k]
11             dist = float(np.linalg.norm(d))
12             if dist < 1e-9:
13                 d = np.eye(n)[-1]
14                 dist = 1.0
15             nk = d / dist
16             radius = obs.radius
17             if obs.covs is not None and self.kappa > 0.0: # chance
18                 sig = math.sqrt(float(nk @ obs.covs[k] @ nk))
19                 radius += self.kappa * sig
20             # nk'(p_k - o_k) >= radius, written as a row in U
21             bound = float(nk @ (free[pk] - obs.centers[k])) - radius
22             avoid_rows.append(-nk @ self.Su[pk])
23             avoid_lo.append(-INF)
24             avoid_up.append(bound)
25             avoid_k.append(k)
26             normals.append(nk)

```

Listing 21.2. The ADMM iteration of `solve_qp` (from `code/ch21_mpc.py`): the linear system with the precomputed inverse, the relaxed update, the projection onto the bounds, the multiplier update, and every ten iterations the residual test and the infeasibility certificate.

```

1  for it in range(1, max_iter + 1):
2      zt = Kinv @ (sigma * z - f + G.T @ (rho * w - y))
3      w_hat = alpha * (G @ zt) + (1.0 - alpha) * w
4      z = alpha * zt + (1.0 - alpha) * z
5      w_new = np.clip(w_hat + y / rho, l, u)
6      y = y + rho * (w_hat - w_new)
7      w = w_new
8      if it % 10 == 0:
9          Gz, Hz, Gty = G @ z, H @ z, G.T @ y
10         r_prim = _norm_inf(Gz - w)
11         r_dual = _norm_inf(Hz + f + Gty)
12         s_prim = max(_norm_inf(Gz), _norm_inf(w), 1e-12)
13         s_dual = max(_norm_inf(Hz), _norm_inf(Gty), f_norm, 1e-12)
14         ok_prim = r_prim <= eps_abs + eps_rel * s_prim
15         ok_dual = r_dual <= eps_abs + eps_rel * s_dual
16         if ok_prim and ok_dual:
17             status = "solved"
18             break
19         if _primal_infeasible(G, l, u, y - y_check):
20             status = "infeasible"
21             break
22         y_check = y.copy()

```

Linearizing around a stale trajectory

The half-plane normals come from the previous plan. If the intruder’s prediction jumps (a track re-initialization, a maneuver) or if the drone’s own plan was rejected and the warm start is old, the normals point the wrong way and the QP forbids the very region the drone should move into. Symptoms are a detour on the wrong side, or a plan that flips sides every step. Re-linearize around the reference when the shifted plan is invalid, and consider a second QP solve with the normals recomputed from the first solution when a prediction changes abruptly.

Hard constraints that become infeasible

A hard avoidance, formation or communication constraint promises what the drone cannot always deliver; the late-detection experiment of [Section 21.7.3](#) is the standard case, and a hard-constrained controller then does whatever its fallback says. Keep the drone’s own input and velocity bounds hard, make every constraint that depends on another agent soft with an exact penalty, and log the slacks: a non-zero slack is the event you want to count in [Chapter 25](#).

Weights in inconsistent units

Q, **R** and the slack weights multiply quantities in different units, and a slack weight that was “large” when positions were in meters is tiny when they are in centimeters. Fix a unit system, derive the weights from the trade-off you want ([Section 21.7.1](#)), and check the multipliers of the hard problem before choosing w_1 .

A horizon too short to stop

The horizon must cover the stopping time $v_{\max}/a_{\max}$ plus the time the prediction layer needs to notice an intruder; otherwise an avoidance constraint enters the problem at a step the drone can no longer influence enough, and the hard problem is infeasible while the soft one violates. In the example the stopping time from cruising speed is 0.5 s and $N = 3$ (0.3 s) fails; from $v_{\max} = 2$ m/s it is 1 s, so $N \geq 10$ is the sensible minimum at $\Delta t = 0.1$ s.

21.9 Where this fits in the drone system

In the drone system

In the hybrid architecture of [Chapter 24](#), MPC is the constraint-enforcing tracker of each drone. Its reference is the time-indexed path of the global planner: CBS ([Chapter 9](#)) or its bounded-suboptimal variant enhanced CBS (ECBS, [Chapter 10](#)), converted into a trajectory as in [Chapter 2](#); while the local safety layer is active, the ORCA velocity of [Chapter 13](#) replaces the path as the reference for the next steps, and the MPC turns that velocity into accelerations that respect $a_{\max}$, $v_{\max}$ and the formation and communication limits, which ORCA knows nothing about. From the prediction layer ([Chapters 18 and 20](#)) it receives the intruder's mean and covariance for the next N steps and turns them into uncertainty-inflated half-planes. From its partners it receives their plans for the same horizon and keeps its formation slot and its radio link as explicit constraints. Back to the decision logic it reports two things: the slack values, which say that a constraint is under pressure before it is violated, and solver failures, which are a trigger for replanning with D^* Lite ([Chapter 5](#)). The evaluation chapter ([Chapter 25](#)) reads formation error and communication violations straight from the MPC's constraint rows. Week 11 of the study plan ([Appendix A](#)) builds this controller; its milestone, constraints enforced before violation rather than repaired after it, is [Exercise 21.9](#).

21.10 Summary

Chapter summary

- MPC solves a finite-horizon optimal control problem at every step, applies the first input and repeats from the measured state; the constraints hold in the plan and therefore, if the model is right, in the next state.
- With the double integrator, a quadratic cost with weights $\mathbf{Q}$, $\mathbf{R}$ (not the Kalman covariances) and $\mathbf{P}_f$, and linear constraints, the problem is a quadratic program. Eliminating the dynamics with $\mathbf{X} = \mathbf{S}_x \mathbf{x}_0 + \mathbf{S}_u \mathbf{U}$ gives the condensed form [Equation \(21.7\)](#), whose Hessian is fixed and whose linear term and rows change per step.
- Avoidance is non-convex; one half-plane per step, linearized around the previous plan, is a conservative convex replacement. Formation keeping and communication range are convex disks, approximated from inside by polygons.
- Constraints on the drone's own variables are hard; constraints that depend on others are soft with slack variables. With a linear slack weight above the largest multiplier the soft solution equals the hard one whenever that exists, and gives the least violation otherwise.

- A small ADMM solver with row scaling, warm start, polishing and an infeasibility certificate solves the planar problem in about a millisecond.
- With a terminal cost from the Riccati equation and an invariant terminal set, nominal MPC is recursively feasible and stable; moving obstacles break the feasibility argument, which is why soft constraints and margins matter. Horizons must cover the stopping time; weights carry units.
- Prediction uncertainty enters as an inflated radius $\kappa_\delta \sqrt{\mathbf{n}_k^T \Sigma_k \mathbf{n}_k}$: a chance constraint that stays linear.

Further reading. The textbook of Rawlings, Mayne, and Diehl [134] is the reference for the theory, the stability results and the sparse and condensed formulations; the survey of Mayne et al. [115] introduced the terminal ingredients used in [Theorem 21.6](#); García, Prett, and Morari [51] is the classic account of MPC in process control; Borrelli, Bemporad, and Morari [21] covers constrained systems, invariant sets and explicit MPC; convex optimization and QP solvers are treated by Boyd and Vandenberghe [24], ADMM by Boyd et al. [23] and the OSQP iteration by Stellato et al. [158]; soft constraints and exact penalties are due to Kerrigan and Maciejowski [81]; Zhu and Alonso-Mora [187] develop chance-constrained avoidance for micro aerial vehicles, and Mellinger and Kumar [117] is the entry point to quadrotor trajectory generation beyond the double integrator.

21.11 Exercises

Exercise 21.1 (★). Explain in your own words why applying only $\mathbf{u}_0^*$ and re-solving at the next step turns an open-loop plan into a feedback controller. Then measure the difference: run `simulate(..., wind=(0.0,0.3))`, which adds a constant 0.3 m/s^2 to the plant but not to the model, and compare the tracking error with the same run in open loop, in which the plan is solved once and its stored inputs $\mathbf{u}_0^*, \dots, \mathbf{u}_{N-1}^*$ are applied one after another (two lines inside `simulate`: keep the first `info["U"]` and index it with the step counter instead of calling `mpc.step` again).

Exercise 21.2 (★★). Derive the condensed quadratic program. Prove $\mathbf{x}_k = \mathbf{A}^k \mathbf{x}_0 + \sum_{j < k} \mathbf{A}^{k-1-j} \mathbf{B} \mathbf{u}_j$ by induction, write out $\mathbf{S}_x$ and $\mathbf{S}_u$ for $N = 3$, substitute into the cost of [Equation \(21.2\)](#) and identify $\mathbf{H}$ and $\mathbf{f}$ of [Equation \(21.8\)](#). Show that $\mathbf{H}$ is positive definite whenever $\mathbf{R} \succ \mathbf{0}$, and write the rows and bounds of the velocity constraint $|v_{k,i}| \leq v_{\max}$ explicitly.

Exercise 21.3 (★★). (a) Prove [Proposition 21.3](#) and describe the collision-free positions that the half-plane excludes. (b) Prove [Proposition 21.4](#) and compute the relative loss $1 - \cos(\pi/M)$ for $M = 4, 8, 16, 32$. (c) Propose a set of directions and the corresponding apothem for the three-dimensional version of the keep-in constraint.

Exercise 21.4 (★). Positions are re-expressed in centimeters and velocities in cm/s while accelerations stay in m/s^2 . How must q_p, q_v, r, w_1 and w_2 change so that the controller's behavior is unchanged? Which of the multipliers change, and by what factor?

Exercise 21.5 (★★). Derive a lower bound on the horizon length $N\Delta t$ from the stopping distance of [Proposition 2.24](#), for a drone at speed v that must be able to stop before an obstacle that appears at the end of its horizon. Evaluate it for the example and for $v_{\max}$, then run `horizon_experiment` with $N = 3, 4, 5$ and explain the failures at $N = 3$.

Exercise 21.6 (★★). Prove [Proposition 21.7](#) directly from the KKT conditions of the soft problem. Then, on [Example 21.5](#), run the soft controller for $w_1 = w_2 \in \{1, 2, 5, 10, 20, 50\}$,

record the minimum separation, and find the threshold above which the soft trajectory equals the hard one. Compare it with the largest multiplier of the hard problem.

Exercise 21.7 (★★★). (a) Give a complete proof of the recursive feasibility part of [Theorem 21.6](#) for a static geofence $\mathcal{X}$, the input box, and the terminal set $\mathcal{X}_f = \{\mathbf{x} : \mathbf{x}^\top \mathbf{P}_f \mathbf{x} \leq c\}$ with the LQR controller, stating what c must satisfy. (b) Construct an explicit example with a moving intruder in which the hard problem is feasible at time t and infeasible at time $t + \Delta t$ although the intruder moves exactly as predicted. Hint: let the linearization point change sides.

Exercise 21.8 (★★). Derive [Equation \(21.13\)](#) from [Equation \(21.12\)](#). Compute κ_δ for $\delta = 0.2, 0.1, 0.05, 0.01$ with `normal_quantile`, run [Example 21.5](#) with `MPC(delta=δ)` and the growing covariance $\Sigma_k = \Sigma_0 + (k\Delta t)^2 \Sigma_v$, which you pass to the simulator as `simulate(..., cov_growth=(Σ0, Σv))`, and plot the minimum separation and the RMS tracking error against δ . Then estimate the empirical collision probability by sampling the intruder's true trajectory from the same Gaussian and compare it with δ .

Exercise 21.9 (★★★ Coding). The Week-11 exercise of the study plan. Build a controller for a leader and two followers in a V formation with offsets $\mathbf{d}_{ij}$, tolerance $e_{\max} = 0.3$ m and communication range $R_{\text{comm}} = 2$ m, in which each follower runs [Algorithm 21.2](#) with the leader's communicated plan as the center of its keep-in constraints (the code's `Follower` shows the interface for one follower). Fly the formation along a path while an intruder crosses. Measure the formation error and the number of communication violations per step, and compare with a controller that tracks the path and only corrects the formation after the error exceeds $e_{\max}$. Then make the formation constraint soft and report the slacks. The milestone is met when the formation and range constraints are satisfied at every step of the constrained run and violated in the reactive one.

Exercise 21.10 (★★★). Write the centralized QP for two drones: stack both input sequences, add the formation constraint as a polygon in the difference of their positions, and solve it with `solve_qp`. Compare cost, formation error and solve time with the sequential leader–follower scheme on the same scenario, and find a scenario in which the sequential scheme violates the formation tolerance while the centralized one does not.

Mixed-Integer Linear Programming for Planning

Learning objectives

- Write a planning question as a linear program (LP), explain why its feasible set is a polyhedron and why the optimum sits at a vertex, and say what integer and binary variables add: choices, either-or conditions, on/off switches and assignments.
- Derive the big- M encoding of “stay outside this rectangle” and of “stay at least $d_{\min}$ apart”, choose M tightly, and count how many binaries a multi-vehicle problem needs.
- Assemble the complete mixed-integer linear program (MILP) for minimum-fuel and minimum-time multi-vehicle trajectories with double-integrator dynamics, and linearize 1-norm and ∞ -norm objectives with auxiliary variables.
- Trace branch-and-bound by hand: LP relaxation, branching on a fractional binary, bounding with the incumbent, and why the tree can grow exponentially.
- Solve a two-vehicle instance with `scipy.optimize.milp`, check the solution, formulate multi-agent path finding (MAPF) as an integer program, and judge from a scaling experiment when exactness is worth its price.

22.1 Why this matters for a drone swarm

Three inspection drones must cross a courtyard in which a 2×4 m building block stands in the way. They start at rest, must be at rest at their goals six seconds later, may accelerate at no more than 2 m/s^2 , must never come closer than one meter to each other, and the battery budget asks for the plan with the smallest total velocity change. Every planner of the earlier chapters answers a version of this question, and every one of them compromises somewhere. CBS (conflict-based search, [Chapter 9](#)) returns optimal paths, but on a grid with unit moves, not trajectories with accelerations. The model predictive controller of [Chapter 21](#) handles the dynamics and the input limits exactly, but a collision constraint is *non-convex*: “stay out of the block” is the union of four half-planes, and the quadratic program of [Chapter 21](#) can only keep a linearized version of it around the current guess. The local planners of [Chapters 12 to 14](#) look one step ahead. None of them can tell you that no cheaper safe plan exists.

Mixed-integer linear programming (MILP) can. It keeps the linear dynamics and the linear bounds of an LP and adds *binary variables* that switch constraints on and off: one binary per side of the block per time step says which side the drone passes on, and one per axis direction per pair of drones says on which side of each other they fly. The solver then searches

the combinations of these switches with a bound that lets it discard most of them unseen, and it returns the global optimum together with a proof of optimality. This is exactly the formulation with which Schouwenaars, De Moor, Feron and How [147] and Richards and How [139] planned aircraft and spacecraft trajectories, and it is the reference solution against which the hybrid planner of Chapter 24 is benchmarked in Chapter 25. The price is that, in the worst case, the search is exponential in the number of binaries: MILP is the tool for small, safety-critical or offline problems, for verifying that a heuristic is close to optimal, and for scheduling take-off and landing slots, not for replanning a swarm of fifty drones ten times a second.

The chapter recalls linear programs and what integer variables buy (Sections 22.2 and 22.3), derives the big- M encodings and the full multi-vehicle MILP (Section 22.4), explains and traces branch-and-bound (Section 22.5), solves a two-vehicle instance with `scipy.optimize.milp` (Section 22.6), proves what the method guarantees and where it is exponential (Section 22.7), formulates MAPF as an integer program (Section 22.8), measures the cost of exactness (Section 22.9) and closes with implementation notes, pitfalls and the place of MILP in the drone system (Sections 22.10 and 22.11).

22.2 The idea in plain words

A **linear program** asks for the point that minimizes a linear function subject to linear inequalities. Each inequality cuts the space with a half-plane, so the feasible set is a **polyhedron**: a convex region with flat sides and sharp corners (Figure 22.1). Sliding the level line of the objective across the polyhedron shows that the best value is always reached at a corner, a *vertex*. The simplex method walks from vertex to neighboring vertex, always improving; interior-point methods travel through the inside and converge to a point of the optimal face, from which a *crossover* step recovers a vertex when one is wanted. Both solve LPs with hundreds of thousands of variables in seconds, and both are reviewed in Section 22.3.1.

Now insist that some variables be integers, for instance $b \in \{0, 1\}$ meaning “the drone passes to the left” or “to the right” of a block. The feasible set is no longer a polyhedron but the set of lattice points inside one, and it is not convex: the midpoint of two feasible points is usually infeasible. Sliding the level line now stops at a lattice point that need not be near the LP vertex. Figure 22.1 shows both: the LP optimum is at the fractional vertex $(2.5, 3.75)$, rounding it gives an infeasible point, and the integer optimum $(3, 3)$ lies on a different side of the polyhedron. Dropping the integrality requirement is called the **LP relaxation**; its value is a bound on the integer optimum, and this bound is what makes an exact search affordable. This example *maximizes* $x + 2y$ (equivalently, it *minimizes* $-x - 2y$), so its relaxation value 10 is an upper bound on the integer optimum 9; for the minimization of Definition 22.1, the convention of the rest of this chapter, the relaxation value is a *lower* bound.

The second ingredient is the **big- M trick**. A drone that stays out of a rectangular block must satisfy *at least one* of four inequalities: left of the block, right of it, below it or above it. An “or” of linear inequalities is not linear. But if we add to each inequality a term Mb_j with a large constant M and a binary b_j , then $b_j = 1$ relaxes the inequality so much that it holds everywhere in the workspace (it is switched off), while $b_j = 0$ enforces it. Demanding $\sum_j b_j \leq 3$ keeps at least one inequality switched on. The same trick encodes “stay at least $d_{\min}$ apart” for every pair of drones. Everything else in the trajectory problem, the double-integrator dynamics, the acceleration and speed limits, the start and goal conditions, is already linear, and the objective can be made linear with auxiliary variables. The result is one MILP, and a solver that combines LP relaxations with a systematic search over the binaries, **branch-and-bound**, finds its global optimum.

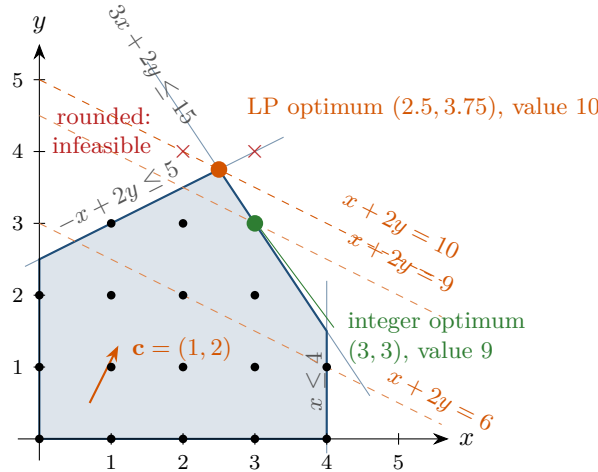


Figure 22.1. A linear program with two variables. The feasible polyhedron (blue) is cut by three half-planes; the level lines of the objective $x + 2y$ (dashed) show that the LP optimum is the vertex $(2.5, 3.75)$ with value 10. If x and y must be integers, only the lattice points inside the polyhedron (dots) are feasible. Rounding the LP vertex to $(2, 4)$ or $(3, 4)$ leaves the polyhedron; the integer optimum is $(3, 3)$ with value 9, and the LP value 10 is an upper bound on it because this example maximizes $x + 2y$; for a minimization, as in Definition 22.1, the relaxation value is a lower bound.

Key idea

Binary variables turn “either this or that” into linear constraints: with the big- M trick, $b_j = 1$ switches an inequality off and $b_j = 0$ keeps it on. Planning with dynamics, obstacles and separation then becomes one mixed-integer linear program. Its LP relaxation is a bound; branch-and-bound fixes fractional binaries one at a time and uses the bound to discard whole subtrees, so it finds the exact optimum, at a cost that is exponential in the number of binaries in the worst case.

22.3 Problem statement and notation

22.3.1 Linear programs

Definition 22.1 (Linear program). A **linear program (LP)** in the variables $\mathbf{z} \in \mathbb{R}^n$ is

$$\min_{\mathbf{z}} \mathbf{c}^T \mathbf{z} \quad \text{subject to} \quad \mathbf{A}_{\text{eq}} \mathbf{z} = \mathbf{b}_{\text{eq}}, \quad \mathbf{A} \mathbf{z} \leq \mathbf{b}, \quad \mathbf{l} \leq \mathbf{z} \leq \mathbf{u}, \quad (22.1)$$

with a linear objective $\mathbf{c}^T \mathbf{z}$, linear equality and inequality constraints, and bounds $\mathbf{l} \leq \mathbf{z} \leq \mathbf{u}$ in which entries may be $\pm\infty$. The **feasible set** is the polyhedron $P = \{\mathbf{z} : \text{all constraints of Equation (22.1) hold}\}$. The LP is **infeasible** if $P = \emptyset$, **unbounded** if the objective has no lower bound on P , and otherwise it has an optimal value J^* attained at a vertex of P (if P has vertices).

The geometry behind the last sentence is reviewed in Appendix B: a polyhedron is convex, a linear function on a convex polyhedron with vertices takes its minimum at a vertex, and every vertex is determined by n of the constraints holding with equality. The **simplex method** exploits this: it starts at a vertex, moves along an edge to a neighboring vertex with a smaller objective, and stops when no neighbor is better, which by convexity means optimal. Its worst case is exponential in n , but in practice its step count grows roughly linearly with the number of constraints. **Interior-point methods** follow a path through the interior of P toward the

Table 22.1. What integer variables buy. Every gadget is linear in the variables of Definition 22.1, of which b and z_j are binary; M is a constant large enough to make a switched-off inequality hold everywhere.

Need	Encoding	Example in this book
choice among r options	$z_1 + \dots + z_r = 1, z_j \in \{0, 1\}$	arrival step of a drone
either $f(\mathbf{z}) \leq 0$ or $g(\mathbf{z}) \leq 0$	$f(\mathbf{z}) \leq Mb, g(\mathbf{z}) \leq M(1 - b)$	pass left or right of a block
on/off switch	$0 \leq u \leq u_{\max}z$	pad in use or not
assignment of i to j	$\sum_j z_{ij} = 1, \sum_i z_{ij} \leq 1$	drones to landing slots
implication, at most one	$z_1 \leq z_2, \sum_j z_j \leq 1$	one drone per vertex and time

optimal face in a number of iterations that is polynomial in the worst case. Both are treated in Nocedal and Wright [119] and, with the convex geometry, in Boyd and Vandenberghe [24]. For this chapter it is enough to know that infeasibility is reported reliably, and that the LP solver is called thousands of times inside a MILP solver; HiGHS, the solver behind `scipy.optimize.milp`, uses a dual simplex method for these calls [69].

22.3.2 Integer and binary variables

Definition 22.2 (Mixed-integer linear program). A **mixed-integer linear program (MILP)** is an LP Equation (22.1) together with an index set $I \subseteq \{1, \dots, n\}$ of variables that must be integers, $z_j \in \mathbb{Z}$ for $j \in I$. A variable with $z_j \in \{0, 1\}$ is **binary**. The **LP relaxation** of a MILP is the LP obtained by dropping the integrality requirement. If all variables are integers the program is an **integer program (IP)**.

Integer variables buy four kinds of modeling power, listed in Table 22.1. A binary can select one of several *choices*; it can enforce an *either-or* between constraints with the big- M trick of the next section; it can act as an *on/off switch* that forces a continuous quantity to zero (a motor that is either off or runs within its limits, a landing pad that is either free or used); and a matrix of binaries with row and column sums encodes an *assignment* of drones to slots or goals. Logical relations between binaries are linear too: “ z_1 implies z_2 ” is $z_1 \leq z_2$, “at most one of” is $\sum_j z_j \leq 1$, and “at least one of” is $\sum_j z_j \geq 1$. Any finite combinatorial decision can be written this way; the modeling craft, surveyed by Vielma [171], lies in writing it so that the LP relaxation is as tight as possible.

22.4 Modeling obstacles, separation and the trajectory problem

22.4.1 Obstacle avoidance with the big- M trick

Consider one drone with position $\mathbf{p}_k = (x_k, y_k)$ at time step k and one rectangular obstacle $O = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$, already inflated by the drone radius as in Chapter 2, inside a rectangular workspace $W = [X_{\min}, X_{\max}] \times [Y_{\min}, Y_{\max}]$ that bounds all positions. The drone is outside O at step k if and only if at least one of the four inequalities

$$x_k \leq x_{\min}, \quad x_k \geq x_{\max}, \quad y_k \leq y_{\min}, \quad y_k \geq y_{\max} \quad (22.2)$$

holds (Figure 22.2). Introduce four binaries $b_{k,1}, \dots, b_{k,4} \in \{0, 1\}$ and a constant $M > 0$, and write

$$\begin{aligned} x_k &\leq x_{\min} + Mb_{k,1}, & x_k &\geq x_{\max} - Mb_{k,2}, \\ y_k &\leq y_{\min} + Mb_{k,3}, & y_k &\geq y_{\max} - Mb_{k,4}, \\ b_{k,1} + b_{k,2} + b_{k,3} + b_{k,4} &\leq 3. \end{aligned} \quad (22.3)$$

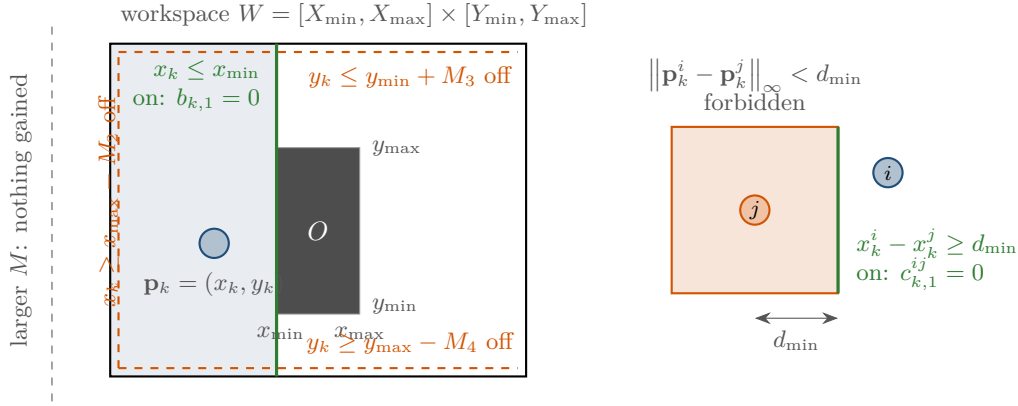


Figure 22.2. The big- M encoding of “stay outside the rectangle O ”. The drone at $\mathbf{p}_k$ satisfies $x_k \leq x_{\min}$, so $b_{k,1} = 0$ keeps that inequality on and the other three may be switched off ($b_{k,j} = 1$). A switched-off inequality, here $x_k \geq x_{\max} - M$ with the tight $M = x_{\max} - X_{\min}$, moves its boundary (dashed) to the edge of the workspace W and is satisfied by every point of W . A larger M moves the dashed line further out without gaining anything.

If $b_{k,j} = 0$ the j -th inequality of Equation (22.3) is the j -th inequality of Equation (22.2): the constraint is *on*. If $b_{k,j} = 1$ it reads $x_k \leq x_{\min} + M$, and if M is large enough this holds for every x_k in the workspace: the constraint is *off*. The sum constraint forbids switching all four off, so at least one side condition is enforced. The binaries are the drone’s *discrete* decision, on which side of the block it is at step k ; the solver, not the modeler, makes it.

The meaning of M . For each inequality, M must be at least the largest amount by which it can be violated inside the workspace; this makes the switched-off inequality vacuous and nothing more. The smallest such values are

$$M_1 = X_{\max} - x_{\min}, \quad M_2 = x_{\max} - X_{\min}, \quad M_3 = Y_{\max} - y_{\min}, \quad M_4 = y_{\max} - Y_{\min}, \quad (22.4)$$

and Proposition 22.3 below shows that with these values Equation (22.3) is *exactly* equivalent to “outside O at step k ”. Nothing forbids a larger M , and textbooks often write a single huge constant, but a larger M hurts twice. First, it weakens the LP relaxation. In the relaxation the binaries are fractional, and a point x_k inside the obstacle satisfies $x_k \leq x_{\min} + Mb_{k,1}$ with $b_{k,1} = (x_k - x_{\min})/M$; the larger M , the closer this is to zero, so the relaxation can place the drone inside the obstacle at almost no cost in the sum constraint and its bound tells branch-and-bound nothing. Second, it damages the numerics. A solver accepts a binary as integral when it is within a tolerance ε of 0 or 1, typically $\varepsilon = 10^{-6}$; a value $b_{k,1} = \varepsilon$ is then reported as 0 although it relaxes the constraint by $M\varepsilon$, which is one meter when $M = 10^6$. Coefficients that differ by six orders of magnitude within one row also make the simplex pivots ill-conditioned. Use Equation (22.4), computed from the actual workspace, and let the position bounds of the workspace be explicit variable bounds so that the solver’s presolve can verify them.

Proposition 22.3 (Exactness of the big- M encoding). *Let $\mathbf{p}_k \in W$ and let $M_1, \dots, M_4$ be given by Equation (22.4) (or any larger values). Then $\mathbf{p}_k \notin \text{int } O$ if and only if there exist $b_{k,1}, \dots, b_{k,4} \in \{0, 1\}$ that satisfy Equation (22.3).*

Proof. If $\mathbf{p}_k$ is not in the interior of O , one of the four inequalities Equation (22.2) holds, say the j -th. Set $b_{k,j} = 0$ and the other three binaries to 1. The j -th row of Equation (22.3) is the j -th inequality, which holds; a row with $b = 1$ holds because $\mathbf{p}_k \in W$, for instance $x_k \leq X_{\max} = x_{\min} + M_1$; and the sum is 3. Conversely, if binaries satisfying Equation (22.3) exist, the sum constraint forces some $b_{k,j} = 0$, whose row is the j -th inequality of Equation (22.2),

Table 22.2. Number of binary variables $4N(mn_O + \binom{m}{2})$ for m drones, one obstacle ($n_O = 1$) and N time steps. The pair term $\binom{m}{2}$ dominates from about four drones on.

Drones m	Pairs $\binom{m}{2}$	$N = 10$	$N = 20$	$N = 50$
2	1	120	240	600
3	3	240	480	1 200
5	10	600	1 200	3 000
10	45	2 200	4 400	11 000
20	190	8 400	16 800	42 000

so $\mathbf{p}_k$ is not in the interior of O . $\square$

Boundary points count as outside; if you need a strict margin, inflate the obstacle. A convex polygon with r edges is handled in the same way with one binary per edge and $\sum_j b_j \leq r - 1$, a box in three dimensions with six binaries, and a moving obstacle with a predicted position at every step (Chapter 20) with a different rectangle O_k per step. The constraint is imposed at the sampled instants $k = 1, \dots, N$ only; what happens between them is the subject of Proposition 22.9 and of the pitfalls in Section 22.10.

22.4.2 Inter-vehicle separation

Two drones i and j with positions $\mathbf{p}_k^i$ and $\mathbf{p}_k^j$ are separated by at least $d_{\min}$ in the ∞ -norm, $\|\mathbf{p}_k^i - \mathbf{p}_k^j\|_{\infty} \geq d_{\min}$, if and only if one of

$$x_k^i - x_k^j \geq d_{\min}, \quad x_k^j - x_k^i \geq d_{\min}, \quad y_k^i - y_k^j \geq d_{\min}, \quad y_k^j - y_k^i \geq d_{\min} \quad (22.5)$$

holds: drone i is far enough to the right, left, above or below drone j . This is the obstacle condition again, with a square of side $2d_{\min}$ centered on the other drone, and it gets the same treatment with four binaries $c_{k,1}^{ij}, \dots, c_{k,4}^{ij}$ per pair and step:

$$\begin{aligned} x_k^i - x_k^j &\geq d_{\min} - M_x c_{k,1}^{ij}, & x_k^j - x_k^i &\geq d_{\min} - M_x c_{k,2}^{ij}, \\ y_k^i - y_k^j &\geq d_{\min} - M_y c_{k,3}^{ij}, & y_k^j - y_k^i &\geq d_{\min} - M_y c_{k,4}^{ij}, \\ c_{k,1}^{ij} + c_{k,2}^{ij} + c_{k,3}^{ij} + c_{k,4}^{ij} &\leq 3. \end{aligned} \quad (22.6)$$

The tight constants are $M_x = d_{\min} + (X_{\max} - X_{\min})$ for the x -rows and $M_y = d_{\min} + (Y_{\max} - Y_{\min})$ for the y -rows, because $x_k^i - x_k^j$ can be as small as $-(X_{\max} - X_{\min})$. The ∞ -norm square is a conservative stand-in for the Euclidean disk of radius $d_{\min}$ (it contains the disk); the disk itself can be approximated by a polygon with more binaries per pair.

Table 22.2 counts the binaries. With m drones, n_O rectangular obstacles and N steps there are $4Nmn_O$ obstacle binaries and $4N\binom{m}{2}$ separation binaries; the pair term grows quadratically in m , and every binary can in principle double the size of the search tree. Two drones and twenty steps are a problem of seconds; three interacting drones are already past what thirty seconds can prove optimal (Section 22.9); ten drones and fifty steps are out of reach without further structure. This table is the quantitative reason for the verdict of Section 22.9.

22.4.3 The full multi-vehicle trajectory problem

Definition 22.4 (Multi-vehicle trajectory MILP). Given m drones with start positions $\mathbf{p}_s^i$ and goals $\mathbf{p}_g^i$, obstacles $O_1, \dots, O_{n_O}$ and a workspace W , a horizon of N steps of length Δt , acceleration and speed limits $a_{\max}$ and $v_{\max}$ and a separation $d_{\min}$, find accelerations

$\mathbf{u}_k^i \in \mathbb{R}^2$, $k = 0, \dots, N-1$, that minimize a cost J subject to

$$\mathbf{x}_{k+1}^i = \mathbf{A} \mathbf{x}_k^i + \mathbf{B} \mathbf{u}_k^i, \quad i = 1..m, \ k = 0..N-1, \quad (22.7a)$$

$$\mathbf{x}_0^i = (\mathbf{p}_s^i, \mathbf{0}), \quad \mathbf{x}_N^i = (\mathbf{p}_g^i, \mathbf{0}), \quad i = 1..m, \quad (22.7b)$$

$$\|\mathbf{u}_k^i\|_\infty \leq a_{\max}, \quad \|\mathbf{v}_k^i\|_\infty \leq v_{\max}, \quad \mathbf{p}_k^i \in W, \quad \text{all } i, k, \quad (22.7c)$$

$$\text{Equation (22.3) for every } i, \text{ every } O_o \text{ and } k = 1..N, \quad b_{k,j}^{i,o} \in \{0, 1\}, \quad (22.7d)$$

$$\text{Equation (22.6) for every pair } i < j \text{ and } k = 1..N, \quad c_{k,q}^{i,j} \in \{0, 1\}, \quad (22.7e)$$

where $\mathbf{x}_k^i = (\mathbf{p}_k^i, \mathbf{v}_k^i) \in \mathbb{R}^4$ and $\mathbf{A}, \mathbf{B}$ are the zero-order-hold double-integrator matrices of Chapter 2,

$$\mathbf{A} = \begin{pmatrix} \mathbf{I} & \Delta t \mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \frac{1}{2} \Delta t^2 \mathbf{I} \\ \Delta t \mathbf{I} \end{pmatrix}.$$

Every constraint in Equation (22.7) is linear: the dynamics are equalities in the states and inputs, the norm bounds are boxes on single components (the ∞ -norm box is a linear inner approximation of the Euclidean disk $\|\mathbf{u}\|_2 \leq a_{\max}$ only if $a_{\max}$ is divided by $\sqrt{2}$; the code keeps the box, as Richards and How [139] do), and the obstacle and separation constraints are the big- M rows. The states could be eliminated by substituting the dynamics, but keeping them as variables with sparse equality rows is what every modern solver prefers.

Linear objectives. A quadratic cost as in Chapter 21 is not allowed, but the two norms that matter for a drone are. The **minimum-fuel** cost, the total velocity change $J_{\text{fuel}} = \Delta t \sum_{i,k} \|\mathbf{u}_k^i\|_1$, is linearized with one auxiliary variable $s_{k,d}^i \geq 0$ per input component:

$$J_{\text{fuel}} = \Delta t \sum_{i,k,d} s_{k,d}^i, \quad -s_{k,d}^i \leq u_{k,d}^i \leq s_{k,d}^i. \quad (22.8)$$

At the optimum $s_{k,d}^i = |u_{k,d}^i|$, because a larger s would only cost more. The **minimum-peak** cost $J_{\text{peak}} = \sum_i \max_k \|\mathbf{u}_k^i\|_\infty$, the sum of the drones' peak accelerations, uses one auxiliary s^i per drone with $-s^i \leq u_{k,d}^i \leq s^i$ for all k, d : an ∞ -norm needs one variable where the 1-norm needs one per component. (A single s shared by all drones would bound only the largest peak and leave the inputs of the other drones undetermined below it.) The **minimum-time** cost of Richards and How [139] needs binaries again: $w_k^i \in \{0, 1\}$ marks the arrival step of drone i ,

$$J_{\text{time}} = \sum_i \sum_{k=1}^N k \Delta t w_k^i, \quad \sum_{k=1}^N w_k^i = 1, \quad |p_{k,d}^i - p_{g,d}^i| \leq M_g \left(1 - \sum_{k' \leq k} w_{k'}^i\right) \quad \text{for all } k, d, \quad (22.9)$$

with M_g the workspace diameter: once the drone has arrived ($\sum_{k' \leq k} w_{k'}^i = 1$) it must stay at the goal, before that the row is switched off; the letter w keeps the arrival binary apart from the y -coordinate of Equation (22.6). A small multiple of J_{fuel} is added as a tie-breaker among plans with the same arrival times. All three objectives are implemented in `code/ch22_milp.py`.

22.5 The algorithm: branch-and-bound

Branch-and-bound (Land and Doig [96]) is the search that every MILP solver runs at its core. Algorithm 22.1 is its textbook form for a minimization problem P with binaries.

The algorithm maintains an **incumbent**, the best solution with integral binaries found so far, and a set of open subproblems. Each subproblem is the original MILP with some binaries fixed to 0 or 1. Line 5 selects the open node with the smallest parent bound (*best-first*);

Algorithm 22.1. Branch-and-bound for a MILP with binary variables**Input:** a MILP P (Definition 22.2) with binaries indexed by I **Output:** an optimal solution $\mathbf{z}^*$ or *infeasible*

```

1 Function BranchAndBound( $P$ ):
2    $J_{\text{inc}} \leftarrow \infty$ ;  $\mathbf{z}^* \leftarrow \text{nil}$            // incumbent: best integral solution so far
3    $\text{OPEN} \leftarrow \{(P, -\infty)\}$            // nodes: a subproblem and the bound of its parent
4   while  $\text{OPEN} \neq \emptyset$  do
5      $(Q, \beta) \leftarrow$  node of  $\text{OPEN}$  with the smallest  $\beta$ 
6     if  $\beta \geq J_{\text{inc}}$  then
7       continue           // pruned by the parent's bound
8      $(\mathbf{z}, J_{\text{LP}}) \leftarrow \text{SolveRelaxation}(Q)$ 
9     if the relaxation is infeasible then
10      continue           // pruned by infeasibility
11     if  $J_{\text{LP}} \geq J_{\text{inc}}$  then
12      continue           // pruned by bound
13     if  $z_j \in \{0, 1\}$  for all  $j \in I$  then
14        $J_{\text{inc}} \leftarrow J_{\text{LP}}$ ;  $\mathbf{z}^* \leftarrow \mathbf{z}$            // integral: new incumbent
15       continue
16      $j \leftarrow$  a binary with fractional  $z_j$ , e.g., the one closest to  $\frac{1}{2}$ 
17     add  $(Q \cup \{z_j = 0\}, J_{\text{LP}})$  and  $(Q \cup \{z_j = 1\}, J_{\text{LP}})$  to  $\text{OPEN}$ 
18   return  $\mathbf{z}^*$  (infeasible if  $\mathbf{z}^* = \text{nil}$ )

```

depth-first selection, which finds an incumbent sooner, is the common alternative, and solvers mix both. Line 8 solves the LP relaxation of the node, which is cheap and gives J_{LP} , a *lower bound* on every solution of the node with integral binaries (Theorem 22.7). Three things can happen. The relaxation is infeasible, so no integral solution exists in the node either. Or the bound is not better than the incumbent (line 12): nothing in this node can beat what we have, and the whole subtree is discarded unseen; this is the **bounding** that makes the method work. Or the relaxation happens to be integral, and it is the best solution of the node, so it becomes the incumbent (line 14). Otherwise some binary is fractional, and line 17 **branches**: the node is replaced by two children in which that binary is fixed to 0 and to 1. The union of the children contains every integral solution of the parent, so nothing is lost, and each child's relaxation is at least as constrained as the parent's, so the bounds can only rise. Which binary to branch on (line 16) is a heuristic decision; “most fractional” is the simplest, and production solvers estimate which branching raises the bound most.

Example 22.5 (Branch-and-bound on a tiny instance). One drone must fly from $(0, 0)$ to $(4, 0)$, at rest at both ends, in $N = 4$ steps of $\Delta t = 1$ with $a_{\max} = 1$, $v_{\max} = 2$, the workspace $[-1, 5] \times [-3, 3]$ and the inflated obstacle $O = [1.5, 2.5] \times [-0.5, 0.5]$ on the straight line, with the minimum-fuel objective. The MILP has 16 binaries, four per step. Its LP relaxation flies straight through the block with cost 4 (the accelerations $(1, 1, -1, -1)$ along x) and sets every binary of step $k = 2$, where the drone is at $(2, 0)$ inside O , to $b_{2,j} = 0.143 = 0.5/3.5$: each side inequality is switched off by one seventh, which the relaxation allows and the drone cannot do. (The other groups already contain a binary at 0, which certifies their disjunction; the code sets their remaining binaries to 1 at no cost and branches only on step 2.) Table 22.3 and Figure 22.3 show the search. Fixing $b_{2,1} = 0$ demands $x_2 \leq 1.5$, which the dynamics cannot reach while still arriving at $x_4 = 4$ at rest, so that child is infeasible; $b_{2,2} = 0$ demands $x_2 \geq 2.5$, also infeasible. The branch $b_{2,3} = 0$ (pass below, $y_2 \leq -0.5$) has an integral

Table 22.3. Trace of Algorithm 22.1 on Example 22.5 (best-first, branching on the most fractional binary of the smallest index). J_{LP} is the value of the node's LP relaxation.

Node	Parent	Fixed	J_{LP}	Action	Incumbent
1	–	–	4	fractional $b_{2,1} = 0.143$: branch	∞
2	1	$b_{2,1} = 0$	infeasible	pruned	∞
3	1	$b_{2,1} = 1$	4	fractional $b_{2,2} = 0.143$: branch	∞
4	3	$b_{2,2} = 0$	infeasible	pruned	∞
5	3	$b_{2,2} = 1$	4	fractional $b_{2,3} = 0.143$: branch	∞
6	5	$b_{2,3} = 0$	6	integral: new incumbent	6
7	5	$b_{2,3} = 1$	6	$J_{LP} \geq$ incumbent: pruned	6

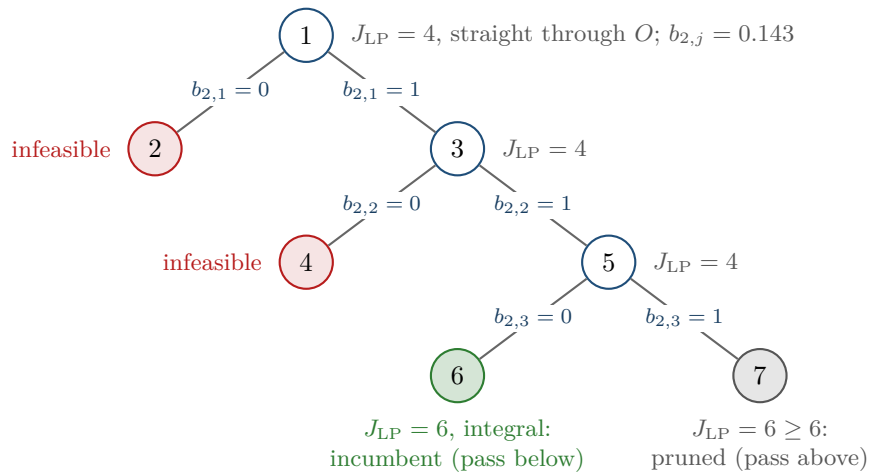


Figure 22.3. The branch-and-bound tree of Example 22.5. Each node shows the value of its LP relaxation. The two left children are infeasible, node 6 is integral and becomes the incumbent, and node 7, the mirror-image plan above the block, is discarded because its bound 6 cannot beat the incumbent 6. The root bound 4 is the cost of flying straight through the block: the LP relaxation of a big- M model is weak.

relaxation of cost 6: the extra 2 pays for the lateral excursion $y = (0, -0.333, -0.5, -0.167, 0)$. Its sibling $b_{2,3} = 1$ forces $b_{2,4} = 0$ through the sum constraint, “pass above”, a mirror image with the same cost 6, and it is pruned by the bound. Seven nodes, one LP each, settle the problem; enumerating the $4^4 = 256$ ways of choosing one active side per step, which the self-test does as a check, gives the same optimum 6.

The example shows in miniature what makes these trees grow: the root relaxation of a big- M model is nearly the unconstrained problem, and the mirror-image plan is a second leaf of the same cost. Production solvers such as HiGHS add *presolve* (tightening M from the variable bounds, removing redundant rows), *cutting planes* (extra valid inequalities that cut off fractional vertices; the combination is called **branch-and-cut**), and *primal heuristics* that find incumbents early; see Wolsey [181]. They change the constants, not the worst case of Proposition 22.8.

Table 22.4. Size of the MILP of [Example 22.6](#) and the effort of `scipy.optimize.milp` (HiGHS, one core) for the three objectives, on the machine and software named in the footnote of [Section 22.6](#). The minimum-time optimum is the pair of arrival times; the code prints the objective 9.016, which adds the small fuel tie-breaker of [Equation \(22.9\)](#).

Objective	Variables	Constraints	Binaries	B&B nodes	Time [s]	Optimum
minimum fuel	344	388	144	366	1.8	12.02 m/s
minimum peak	298	388	144	1	0.14	$0.889 + 0.889$ m/s ²
minimum time	368	486	168	1	0.53	$4.5 + 4.5$ s

22.6 A worked example

Example 22.6 (Two drones crossing past a block). Drone A flies from (1, 4) to (9, 4) and drone B from (9, 4.5) to (1, 4.5) (meters), both at rest at both ends, in a workspace $[0, 10] \times [0, 8]$. A building block occupies, after inflation by the drone radius, the rectangle $O = [4, 6] \times [1.5, 5.5]$, which cuts both straight lines. The horizon is $N = 12$ steps of $\Delta t = 0.5$ s, $a_{\max} = 2$ m/s², $v_{\max} = 3$ m/s and $d_{\min} = 1$ m. The passage above the block ($y \geq 5.5$) is closer to both drones than the passage below ($y \leq 1.5$), so both want to use it, head-on. The instance is built and solved by `code/ch22_milp.py` ([Section 22.10](#)); every number below is printed by its self-test.

[Table 22.4](#) lists the size of the MILP for the three objectives and [Figure 22.4](#) shows the minimum-fuel and the minimum-time trajectories. With the fuel objective there are $2 \cdot 4 \cdot 13 = 104$ state variables, 48 inputs, 48 slacks, 96 obstacle binaries and 48 separation binaries, 344 variables in all, and 388 constraint rows: 8 initial and 8 terminal conditions, 96 dynamics rows, 96 slack rows, 120 obstacle rows (2 drones $\times$ 12 steps $\times$ 5 rows) and 60 separation rows. The tight big- M values [Equation \(22.4\)](#) of this instance are $M_1 = 6$, $M_2 = 6$, $M_3 = 6.5$ and $M_4 = 5.5$ m, and the separation rows use $M_x = 11$ m and $M_y = 9$ m ([Exercise 22.2](#)). HiGHS solves the fuel model in about 1.8 s¹ after 366 branch-and-bound nodes. The optimal fuel is $J_{\text{fuel}} = 12.02$ m/s of total velocity change, and [Table 22.5](#) lists the resulting pair of trajectories step by step. Both drones go over the top; A climbs to $y = 6.5$ while B stays on the edge $y = 5.5$, and at step $k = 6$ ($t = 3$ s) they pass each other with $\|\mathbf{p}_6^A - \mathbf{p}_6^B\|_{\infty} = 1.000$: the separation constraint is active, and the binary $c_{6,3} = 0$ (“A above B”) is the solver’s discrete decision. Raising $d_{\min}$ shrinks the feasible set and can only raise the optimum: the self-test finds $J_{\text{fuel}} = 10.77, 12.02, 12.56$ and 12.56 m/s for $d_{\min} = 0.5, 1, 1.5$ and 2 m. The last two are equal, so an optimal plan for 1.5 m already keeps 2 m at every sampled instant.

The minimum-peak objective gives a smooth plan in which neither drone exceeds 0.889 m/s² (objective $0.889 + 0.889$) and whose relaxation is already integral: the block hardly matters when the drones fly slowly. The minimum-time objective (right panel) brings both drones home in 4.5 s each, the discrete optimum of the arrival binaries, at the price of 16.0 m/s of velocity change and of a flaw: at $t = 2$ s A is at $x = 4.375$ and B at $x = 5.625$, both on the edge $y = 5.5$, and at $t = 2.5$ s they have exchanged these positions. The sampled ∞ -distance is 1.25 m at both instants, so the MILP is satisfied, but the continuous-time separation, evaluated by `continuous_check` on the exact zero-order-hold motion, drops to 0: the drones fly through each other. The minimum-fuel plan is not clean either: between samples its separation drops to 0.856 m and drone B cuts the corner of the block by 0.146 m between $t = 3$ and 3.5 s.

¹Every solve time quoted in this chapter was measured on one core of an Intel Xeon at 2.8 GHz under Python 3.11 with SciPy 1.17.1 and its bundled HiGHS. Times vary by a few tenths of a second between runs of the same instance and by a factor of two between machines; the node counts are reproducible.

Table 22.5. The minimum-fuel plan of [Example 22.6](#) around the crossing, as printed by the self-test of `code/ch22_milp.py`, which lists all thirteen steps; the omitted ones ($k = 1\text{--}3$ and $k = 9\text{--}11$) continue the pattern of the neighboring rows. The separation constraint is tight at $k = 6$, where the two drones pass each other with exactly $d_{\min} = 1$ m; everywhere else it is slack.

k	t [s]	$\mathbf{p}_k^A$ [m]	$\mathbf{p}_k^B$ [m]	$\ \mathbf{p}_k^A - \mathbf{p}_k^B\ _\infty$ [m]
0	0.0	(1.000, 4.000)	(9.000, 4.500)	8.000
4	2.0	(3.444, 5.750)	(6.455, 5.295)	3.010
5	2.5	(4.222, 6.250)	(5.636, 5.500)	1.414
6	3.0	(5.000, 6.500)	(4.818, 5.500)	1.000
7	3.5	(5.778, 6.250)	(4.000, 5.318)	1.778
8	4.0	(6.556, 5.750)	(3.182, 5.136)	3.374
12	6.0	(9.000, 4.000)	(1.000, 4.500)	8.000

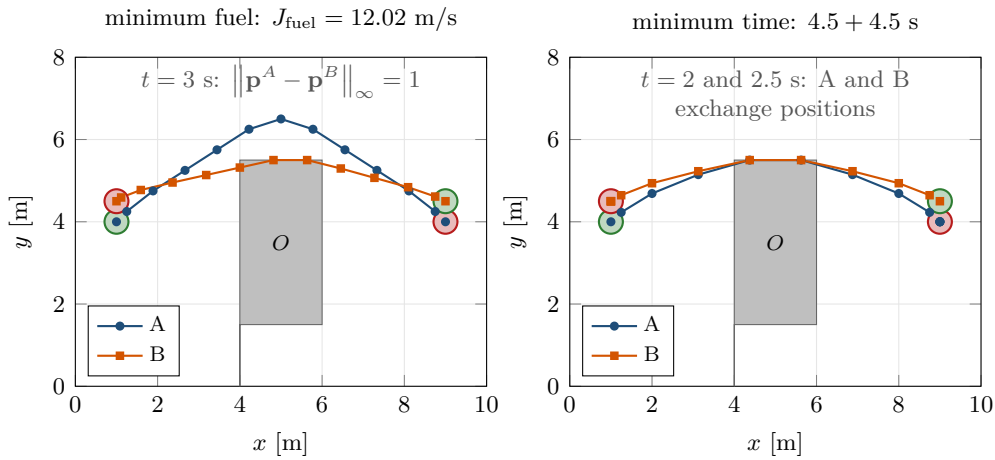


Figure 22.4. Optimal trajectories of [Example 22.6](#), sampled every $\Delta t = 0.5$ s (dots), for the minimum-fuel objective (left) and the minimum-time objective (right); the gray rectangle is the inflated block. Left: A (blue) climbs over B (orange) and the two pass at $t = 3$ s with exactly $d_{\min} = 1$ m of ∞ -norm separation. Right: both drones arrive after 4.5 s at full acceleration, hug the top edge of the block, and *swap* their x -positions between $t = 2$ and $t = 2.5$ s: every sampled separation is at least 1.25 m, yet the continuous paths cross. The constraints hold only where they are imposed.

[Proposition 22.9](#) gives the margins that make the sampled constraints imply the continuous ones; [Exercise 22.4](#) applies them.

22.7 Properties: correctness, optimality, complexity

Three questions remain open after the walkthrough: does branch-and-bound always find the optimum, how badly can it behave, and what does the optimum of the sampled model say about the motion between the samples. This section answers them in turn.

22.7.1 Branch-and-bound is exact

Theorem 22.7 (Correctness of branch-and-bound). *Let P be a MILP with binaries indexed by I whose LP relaxations are bounded and solved exactly. [Algorithm 22.1](#) terminates after visiting at most $2^{|I|+1} - 1$ nodes and returns an optimal solution of P , or infeasible if P has no feasible solution.*

Proof. Call a point *integral* if all its binaries are 0 or 1. Three facts carry the proof.

Bound. The feasible set of the LP relaxation of a node Q contains every integral point that is feasible for Q , so its minimum $J_{LP}(Q)$ is at most the cost of every integral solution of Q . A child of Q adds one equation, so its bound is at least $J_{LP}(Q)$.

Invariant. At the start of every iteration, every integral solution of P that is cheaper than the incumbent is feasible for some node in OPEN. This holds initially, when $OPEN = \{P\}$ and $J_{inc} = \infty$. Consider the node Q popped in an iteration. If it is pruned by its parent's bound $\beta \geq J_{inc}$, every integral solution of Q costs at least β by the first fact, so none is cheaper than the incumbent and the invariant survives; the same holds when the relaxation is infeasible (then Q has no integral solution) or when $J_{LP} \geq J_{inc}$. If the relaxation's solution $\mathbf{z}$ is integral, it is feasible for Q and costs $J_{LP}(Q)$, the least of all integral solutions of Q ; it becomes the incumbent, and no integral solution of Q is cheaper than it. Otherwise the algorithm branches on z_j : every integral solution of Q has $z_j = 0$ or $z_j = 1$, so it is feasible for one of the two children, and both are pushed.

Termination. Each child fixes one binary more than its parent. A node with all $|I|$ binaries fixed has an integral relaxation, or an infeasible one, and is never branched on. The search tree is therefore binary with depth at most $|I|$, it has at most $2^{|I|+1} - 1$ nodes, and every iteration pops one node, so the loop ends.

When OPEN is empty, the invariant says that no integral solution of P is cheaper than the incumbent, and the incumbent is itself a feasible integral solution: it is optimal. If no incumbent was ever found, P has no integral solution at all. $\square$

The trajectory MILP of Definition 22.4 satisfies the hypothesis: every variable has finite bounds, so every relaxation is a bounded LP with an optimal vertex.

Solvers relax the last step. They stop as soon as the incumbent is within a relative **MIP gap** of the smallest bound in OPEN, $(J_{inc} - \min_{OPEN} \beta) / |J_{inc}| \leq \varepsilon_{gap}$, with $\varepsilon_{gap} = 10^{-4}$ by default in HiGHS, and they accept a binary as integral within 10^{-6} . Both tolerances turn “optimal” into “optimal up to a known margin”, which is still far more than any planner of the earlier chapters offers: the gap is a certificate that the reader of a benchmark can check.

22.7.2 The worst case is exponential

The node bound of Theorem 22.7 is not a pessimistic artifact of the proof.

Proposition 22.8 (Exponential branch-and-bound trees). *Deciding whether a MILP with binaries has a feasible solution is NP-hard [79], and there are families of programs with n binaries on which Algorithm 22.1 visits at least $2^{(n-1)/2}$ nodes, whatever branching rule and node order it uses [72].*

Proof sketch. Jeroslow's family is the single constraint $2(z_1 + \dots + z_n) = n$ with n odd and $z_j \in \{0, 1\}$. It has no integral solution, because the binaries would have to sum to $n/2$. Yet a node in which $q < n/2$ binaries are fixed, f of them to 1, has a feasible relaxation: the $n - q$ free variables must sum to $n/2 - f$, which lies between 0 and $n - q$ because $f \leq q < n/2$ and $q - f \leq q < n/2$. So no node above depth $(n - 1)/2$ is pruned by infeasibility, no incumbent ever exists to prune by bound, and the algorithm must visit every one of the $2^{(n-1)/2}$ nodes at that depth. A cutting plane, here the valid inequality $z_1 + \dots + z_n \leq (n - 1)/2$, settles the instance at the root, which is why modern solvers are not pure branch-and-bound; the NP-hardness says that no such trick works for all instances unless $P = NP$. $\square$

In practice the effort is the number of nodes times the cost of one LP, and the dual simplex method re-solves a child from its parent's basis in a few pivots. What decides the number of nodes is the number of binaries and the tightness of the relaxation: the weaker the root bound, the deeper the tree before bounding bites (Table 22.2 and Section 22.9).

22.7.3 What the sampled constraints guarantee between samples

Proposition 22.9 (Inter-sample safety). *Let a drone follow Equation (22.7a) with $\|\mathbf{v}_k\|_\infty \leq v_{\max}$ at every sampled instant, and let $\mathbf{p}(t)$ be its exact zero-order-hold motion, with the constant acceleration $\mathbf{u}_k$ on $[k\Delta t, (k+1)\Delta t]$. Then*

- (i) *every point of the motion is within ∞ -distance $\delta = v_{\max}\Delta t/2$ of a sampled position $\mathbf{p}_k$;*
- (ii) *if every sampled position has ∞ -distance at least δ from a rectangle O , that is, lies outside O inflated by δ on every side, then no point of the motion lies in the interior of O ;*
- (iii) *if two drones satisfy $\|\mathbf{p}_k^i - \mathbf{p}_k^j\|_\infty \geq d_{\min} + v_{\max}\Delta t$ at every sampled instant, then $\|\mathbf{p}^i(t) - \mathbf{p}^j(t)\|_\infty \geq d_{\min}$ at every time t .*

Proof. On $[k\Delta t, (k+1)\Delta t]$ the velocity $\mathbf{v}(t) = \mathbf{v}_k + (t - k\Delta t)\mathbf{u}_k$ is linear in t , so each of its components lies between the corresponding components of $\mathbf{v}_k$ and $\mathbf{v}_{k+1}$, both of magnitude at most $v_{\max}$; hence $\|\mathbf{v}(t)\|_\infty \leq v_{\max}$ throughout the step. Integrating, $\|\mathbf{p}(t) - \mathbf{p}_k\|_\infty \leq v_{\max}(t - k\Delta t)$ and $\|\mathbf{p}(t) - \mathbf{p}_{k+1}\|_\infty \leq v_{\max}((k+1)\Delta t - t)$. The smaller of the two right-hand sides is at most $v_{\max}\Delta t/2$, which is (i). For (ii), let $\mathbf{p}(t)$ be within δ of $\mathbf{p}_k$; the ∞ -distance to O obeys the triangle inequality, so $\text{dist}(\mathbf{p}(t), O) \geq \text{dist}(\mathbf{p}_k, O) - \delta \geq 0$, and a point at distance 0 from O is on its boundary or outside. For (iii), write $s = t - k\Delta t$. Then

$$\|\mathbf{p}^i(t) - \mathbf{p}^j(t)\|_\infty \geq \|\mathbf{p}_k^i - \mathbf{p}_k^j\|_\infty - \|\mathbf{p}^i(t) - \mathbf{p}_k^i\|_\infty - \|\mathbf{p}^j(t) - \mathbf{p}_k^j\|_\infty \geq d_{\min} + v_{\max}\Delta t - 2v_{\max}s,$$

and the same argument from the end of the step gives $d_{\min} + v_{\max}\Delta t - 2v_{\max}(\Delta t - s)$. One of s and $\Delta t - s$ is at most $\Delta t/2$, so the larger of the two bounds is at least $d_{\min}$. $\square$

The margins are cheap to apply: inflate every obstacle by δ and raise $d_{\min}$ by $v_{\max}\Delta t$ before building the model. In Example 22.6 this means $\delta = 0.75$ m and a sampled separation of 2.5 m, and the self-test solves this “safe” instance: the optimal fuel rises from 12.02 to 16.78 m/s, and the continuous motion, checked against the original block and the original $d_{\min}$, never enters the block and never comes closer than 4.08 m (the margins assume top speed in the worst direction at every step, so they are conservative). Inflating the block alone is not enough: with the original $d_{\min}$ the plan is 0.06 m from a mid-step collision. The alternatives, a finer Δt and an *a posteriori* check of the continuous motion, are discussed in Section 22.10.

22.8 Variants and extensions

22.8.1 MAPF as an integer program

The multi-agent path finding (MAPF) problem of Chapter 7 lives on a graph rather than in continuous space, but it is an integer program too, and a particularly clean one. Yu and LaValle [185] write it as a **multi-commodity flow** on the **time-expanded graph** G_T : the space-time states (v, t) of Chapter 2 for $t = 0, \dots, T$, with a directed edge from (v, t) to $(v', t+1)$ whenever v' is v itself (a wait) or a neighbor of v (Figure 22.5). Agent i is one unit of flow that enters at $(s_i, 0)$ and leaves at (γ_i, T) , and the binary f_e^i says whether agent i uses

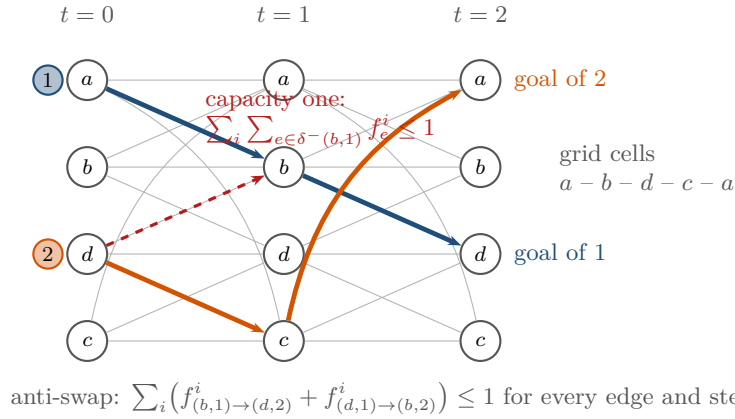


Figure 22.5. MAPF as a multi-commodity flow on the time-expanded graph of a 2×2 grid of cells a, b, d, c (a 4-cycle) with $T = 2$. Thin edges are waits and moves; the thick blue and orange paths are the unit flows of agents 1 ($a \rightarrow d$) and 2 ($d \rightarrow a$). The dashed red edge is the cheaper move of agent 2 through b , excluded by the capacity of state $(b, 1)$, which agent 1 already uses. The anti-swap row forbids the two agents to exchange cells along one edge in the same step.

the edge e . With $\delta^+(v, t)$ and $\delta^-(v, t)$ the edges leaving and entering (v, t) , the program is

$$\sum_{e \in \delta^+(v, t)} f_e^i - \sum_{e \in \delta^-(v, t)} f_e^i = \begin{cases} 1 & (v, t) = (s_i, 0), \\ -1 & (v, t) = (\gamma_i, T), \\ 0 & \text{otherwise,} \end{cases} \quad \text{all } i, (v, t), \quad (22.10a)$$

$$\sum_i \sum_{e \in \delta^-(v, t)} f_e^i \leq 1, \quad \text{all } (v, t), t \geq 1, \quad (22.10b)$$

$$\sum_i (f_{(u, t) \rightarrow (v, t+1)}^i + f_{(v, t) \rightarrow (u, t+1)}^i) \leq 1, \quad \text{all edges } \{u, v\}, t. \quad (22.10c)$$

Equation (22.10a) is flow conservation: what enters a state leaves it, one unit is injected at the start and absorbed at the goal, so the edges with $f_e^i = 1$ form a path in G_T , that is, a plan for agent i with wait actions. Equation (22.10b) gives every state capacity one and forbids the vertex conflicts of Chapter 7; Equation (22.10c) forbids the edge (swap) conflicts, because two agents cannot traverse the same edge in opposite directions during the same step. Yu and LaValle obtain the same effect by splitting every edge of G_T with a small gadget, so that their program stays a network flow with capacities. For the makespan objective the program is a feasibility problem: solve it for increasing T , starting from the largest single-agent distance, and the first feasible T is optimal. The sum of costs is the linear objective $\sum_i \sum_{e \neq ((\gamma_i, t) \rightarrow (\gamma_i, t+1))} f_e^i$: every edge of agent i counts one, except the wait edges at its own goal, so the sum is the number of steps the agent spends away from that goal, which is the definition of Chapter 7 whenever no agent leaves its goal again. Excluding all of $\delta^-(\gamma_i, \cdot)$ instead would drop the arrival move as well and undercount every agent by one step.

For a single agent, Equation (22.10a) alone is a shortest-path problem whose LP relaxation is always integral (a network-flow matrix is totally unimodular), and Dijkstra's algorithm of Chapter 3 solves it faster than any LP; all the difficulty comes from the coupling rows Equations (22.10b) and (22.10c). This tells you how the ILP compares with CBS (Chapter 9). Both are exact. CBS works on the conflicts: its constraint tree grows with the conflicts of the low-level paths, whatever the size of the map. The ILP works on the volume: it has $m \cdot T \cdot \sum_v (\deg v + 1)$ binaries whether the agents interact or not, so its size grows with the map, the horizon and the number of agents, but a conflict costs it nothing beyond a row. On a 32×32 grid with 20 agents and $T = 60$ that is about six million binaries (Exercise 22.7),

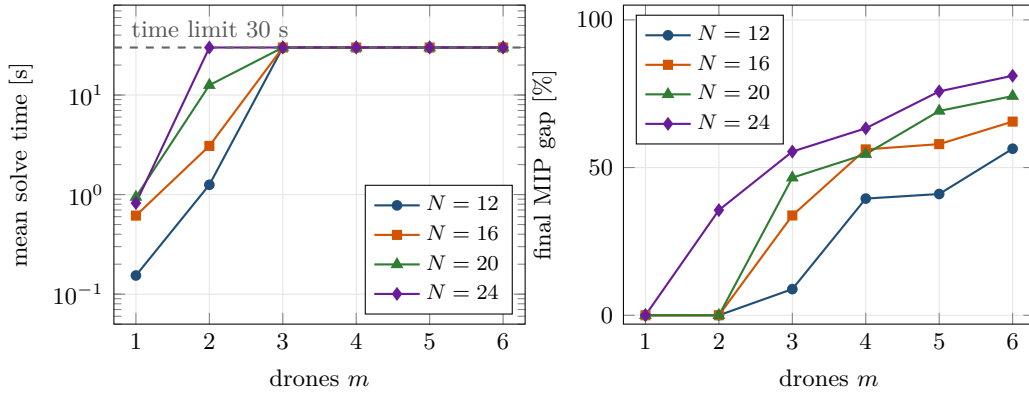


Figure 22.6. The cost of exactness on the crossing instance. Left: mean solve time (log scale) against the number of drones for four horizons; runs stopped by the 30 s limit are plotted at the limit. Right: the relative MIP gap between the incumbent and the best bound when the solver stopped; zero means proven optimal. One drone is easy at every horizon, two drones are solved in seconds up to $N = 20$, and from three drones on the solver returns an incumbent without a proof of optimality, with a gap that grows with the horizon.

far beyond branch-and-bound, while CBS solves such instances routinely when the conflicts are few. On small, densely populated graphs the picture reverses: the constraint tree of CBS explodes while the flow program stays small, and Yu and LaValle report optimal solutions for a hundred or more robots on grids of a few hundred cells, using the splitting heuristics of their paper rather than the plain flow ILP. The ILP also inherits everything a MILP solver offers: an incumbent with a gap at any time, a warm start, and any linear side constraint that CBS would need a new branching rule for.

22.8.2 Other extensions

The trajectory MILP of Definition 22.4 extends in the directions a drone swarm needs. Moving obstacles with predicted positions, boxes in three dimensions, polygonal obstacles and polygonal separation disks were listed at the end of Sections 22.4.1 and 22.4.2; each costs binaries and nothing else. The *minimum-time* objective Equation (22.9) and the time-window constraints of Exercises 22.5 and 22.6 turn the MILP into a scheduler. Run in a *receding horizon*, the MILP becomes the mixed-integer variant of the MPC of Chapter 21, treated by Borrelli, Bemporad, and Morari [21] under the name hybrid MPC: only the first input is applied, the plan is shifted by one step and re-solved, and the previous plan serves as the warm start. Its solve time must then fit into one control period, which is what Section 22.9 measures.

22.9 The cost of exactness

How far does exactness scale? The script `gen_ch22_scaling.py` solves the *crossing instance* of `ch22_milp.py`: m drones start on a circle of radius 4 m around a 2×2 m block in the center of a 10×10 m workspace and fly to their antipodes, with a small random jitter of the start angles, $d_{\min} = 1$ m, $\Delta t = 0.5$ s, $a_{\max} = 2$ m/s², $v_{\max} = 3$ m/s and the minimum-fuel objective, so that every pair of straight lines crosses near the block at about the same time. For $m = 1, \dots, 6$ and $N = 12, 16, 20, 24$ two seeded instances are solved by HiGHS on one core with a time limit of 30 s. Figure 22.6 shows the mean solve time and the MIP gap left when the limit stops the solver.

One drone is an LP with $4N$ obstacle binaries whose relaxation is close to integral: HiGHS needs less than a second for every horizon. Two drones add the $4N$ separation binaries, and

Table 22.6. When exactness is worth its price. The verdict follows from [Figure 22.6](#) and [Table 22.2](#): the solve time grows exponentially with the number of interacting drones and with the horizon, and it is unpredictable from one instance to the next.

MILP is the right tool for	... because
small, safety-critical missions planned offline	a proof of optimality and feasibility is worth minutes of computing
reference solutions for benchmarks (Chapter 25)	the gap of a heuristic to the optimum can be measured, not guessed
verifying a heuristic	an incumbent plus a bound certifies how far the heuristic is from optimal
scheduling take-off and landing slots, assignments	few binaries, tight relaxations, natural linear constraints
MILP is the wrong tool for	... because
online replanning at several hertz	the solve time is unbounded and varies by orders of magnitude
swarms of tens of drones in one problem	the pair binaries grow as m^2N and the tree as 2^{binaries}
long horizons at fine time steps	each halving of Δt doubles the binaries and multiplies the nodes

the time rises from about 1 s at $N = 12$ through 3 s at $N = 16$ to 12 s at $N = 20$; at $N = 24$ the solver hits the limit with a gap of 30–40 %. From three drones on the limit is hit at every horizon, and the gap left when the solver stops grows with m and with N : 9 % at $m = 3$, $N = 12$, more than 50 % at $m = 3$, $N = 24$, and 81 % at $m = 6$, $N = 24$, where 30 s buy only two nodes because a single LP relaxation, with about 2 000 binaries among 3 200 variables, takes seconds. The number of binaries, [Table 22.2](#), is not the whole story; what the solver fights is the weak relaxation of the big- M rows and the symmetry of a crossing, where the mirror image of every plan is another plan of the same cost. Less entangled instances (a formation flying one way, a two-lane corridor) and symmetry-breaking constraints ([Section 22.10](#)) gain an order of magnitude, not a change of shape.

[Table 22.6](#) states the verdict: MILP is not a competitor of the planners in Parts III and IV of this book but their judge, computing the exact optimum offline for a two- or three-drone scenario of [Chapter 25](#), so that the gap of the hybrid planner’s plan becomes a measured number instead of a guess.

22.10 Implementation notes

The file `code/ch22_milp.py` builds the MILP of [Definition 22.4](#) as sparse matrices. The dataclass `Instance` carries the geometry, the horizon, the limits and the field `objective` ("fuel", "peak" or "time"); the class `MilpModel` allocates blocks of variable indices (states, inputs, slacks, obstacle, separation and arrival binaries) behind the index functions `x(i, k, d)`, `u(i, k, d)`, `b(i, o, k, j)` and `cpair(pair, k, j)`, and `add_row` collects a constraint row as a dictionary of coefficients with two bounds. [Listing 22.1](#) is the big- M block of [Equation \(22.3\)](#) with the tight values [Equation \(22.4\)](#); the separation rows follow the same pattern. [Listing 22.2](#) assembles the rows into a `scipy.sparse` matrix and calls `scipy.optimize.milp`, whose `integrality` vector marks the binaries, whose `bounds` carry the workspace and the input limits and whose `options` accept a time limit; with `relax=True` it solves the LP relaxation of a node for the textbook `branch_and_bound` that produced [Table 22.3](#). The result carries

the status (0 optimal, 1 stopped by the limit with an incumbent, 2 infeasible) and, in `res.mip_node_count` and `res.mip_gap`, the node count and the relative gap, which `Solution` exposes as `sol.nodes` and `sol.gap`. Listing 22.3 is all a user writes.

Listing 22.1. The big- M obstacle rows of Equation (22.3) with the tight M of Equation (22.4) (from `code/ch22_milp.py`).

```

1     def _obstacle_rows(self):
2         """Big-M disjunction: at every step at least one side constraint
           holds."""
3         inst = self.inst
4         ws = inst.workspace
5         for o, (xmin, xmax, ymin, ymax) in enumerate(inst.obstacles):
6             # tight M: the largest violation possible inside the workspace
7             tight = (ws[1] - xmin, xmax - ws[0], ws[3] - ymin, ymax - ws[2])
8             M = [self.big_m] * 4 if self.big_m is not None else list(tight)
9             for i in range(len(inst.vehicles)):
10                 for k in range(1, inst.horizon + 1):
11                     px, py = self.x(i, k, 0), self.x(i, k, 1)
12                     b = [self.b(i, o, k, j) for j in range(4)]
13                     self.add_row({px: 1.0, b[0]: -M[0]}, -np.inf, xmin) # left
14                     self.add_row({px: -1.0, b[1]: -M[1]}, -np.inf, -xmax) #
15                                     right
16                     self.add_row({py: 1.0, b[2]: -M[2]}, -np.inf, ymin) #
17                                     below
18                     self.add_row({py: -1.0, b[3]: -M[3]}, -np.inf, -ymax) #
19                                     above
20                     self.add_row({bj: 1.0 for bj in b}, -np.inf, 3.0)

```

Listing 22.2. Assembling the sparse matrix and calling `scipy.optimize.milp` (from `code/ch22_milp.py`).

```

1     def matrix(self):
2         return coo_matrix((self.vals, (self.rows, self.cols)),
3                             shape=(self.n_cons, self.n_var)).tocsr()
4
5     def solve(self, time_limit=None, lb=None, ub=None, relax=False):
6         """Solve with scipy.optimize.milp (HiGHS); relax=True drops
           integrality."""
7         options = {"disp": False}
8         if time_limit is not None:
9             options["time_limit"] = time_limit
10        constraints = LinearConstraint(self.matrix(), self.row_lb, self.row_ub)
11        bounds = Bounds(self.lb if lb is None else lb,
12                        self.ub if ub is None else ub)
13        integrality = None if relax else self.integrality
14        t0 = time.perf_counter()
15        res = milp(self.c, integrality=integrality, bounds=bounds,
16                  constraints=constraints, options=options)
17        return self._unpack(res, time.perf_counter() - t0)

```

Listing 22.3. Solving and checking the two-drone instance of Example 22.6 (from the self-test of `code/ch22_milp.py`).

```

1     inst = example_instance("fuel")
2     model, sol = solve_instance(inst)
3     assert sol.status == 0, sol.message

```



```

4     viol = check_solution(inst, sol)
5     assert is_valid(inst, sol), viol
6     print("example (fuel): vars %d, cons %d, binaries %d, objective %.4f, "
7           "%.3f s, %d B&B nodes" % (sol.n_var, sol.n_cons, sol.n_bin,
8                                     sol.objective, sol.solve_time, sol.nodes))

```

Tight M and explicit bounds. Compute every M from the workspace as in Listing 22.1 and give the positions explicit bounds, so that the presolve of the solver can tighten coefficients on its own. In Example 22.6 HiGHS is robust: the self-test solves the instance with $M = 10^2$, 10^4 and 10^6 instead of the tight values and finds the same optimum after the same 366 nodes, because presolve reduces the large coefficients from the variable bounds. Do not count on that in larger models or with other solvers; Section 22.4.1 explains what a loose M does to the relaxation and to the tolerances.

Symmetry. Two drones that cross can pass left-or-right of each other at the same cost, and a plan and its mirror image are different leaves of the tree. Break the symmetry when the instance has one: forbid one of the four disjuncts for one pair ($c_{k,1}^{12} = 1$ for all k switches off the row “drone 1 at least $d_{\min}$ to the right of drone 2”, so the pair must be separated in one of the three other directions at every step), or add an ordering constraint on the drones’ arrival binaries. Fixing $c_{k,1}^{12} = 1$ removes the mirror-image subtree, and every symmetry removed halves the tree; but it is a *valid* constraint only when a plan of the remaining shape is known to exist, so compare the optimum with and without it before trusting the faster run. (The opposite value $c_{k,1}^{12} = 0$ is a much stronger demand: it enforces $x_k^1 - x_k^2 \geq d_{\min}$, that is, drone 1 stays to the right of drone 2 at every step.)

Time discretization. The step Δt trades three things: the accuracy of the zero-order-hold model, the size of the corner-cutting margins of Proposition 22.9, which shrink linearly with Δt , and the number of binaries, which grows as $1/\Delta t$. Halving Δt in Example 22.6 ($N = 24$, $\Delta t = 0.25$ s) reduces the mid-step penetration of the block from 0.146 to 0.017 m and leaves the fuel almost unchanged (12.04 m/s), but the solve takes about 21 s and 4 100 nodes instead of 1.8 s and 366. A coarse step with the margins of Proposition 22.9, followed by an *a posteriori* check of the continuous motion with `continuous_check`, is usually the better bargain.

Warm starts, time limits, incumbents. A MILP solver profits enormously from a good incumbent: everything with a bound above it is pruned at once. In a receding-horizon loop the previous plan, shifted by one step, is such an incumbent, and solver interfaces such as `highspy`, Gurobi or CPLEX accept it as a starting solution; `scipy.optimize.milp` has no argument for it, which is one reason to move to a solver interface for anything online. Always set a time limit. When it strikes, `milp` returns status 1 with the incumbent and the gap, which is exactly the information a supervisor needs: in Figure 22.6 the three-drone instances stopped after 30 s hold plans within 9 % of optimal at $N = 12$. Check every returned plan with `check_solution`, never with the status alone: the tolerances of the solver are yours to accept or to reject.

A huge M

The textbook “ $M = 10^6$ ” costs twice, once in the relaxation and once in the tolerances (Section 22.4.1). Compute M from the workspace, keep all coefficients within a few orders of magnitude of one, and verify the returned plan.

Safe at the samples, unsafe in between

The MILP enforces the obstacle and separation constraints at $k = 1, \dots, N$ and nowhere else. Both plans of [Example 22.6](#) violate the continuous constraint although every sampled constraint holds. Inflate the obstacles by $v_{\max}\Delta t/2$ and the separation by $v_{\max}\Delta t$ ([Proposition 22.9](#)), or use a finer Δt , and check the continuous motion before you fly it.

Infeasible, and no one tells you why

When the horizon is too short, the workspace too small or the margins too large, the solver answers “infeasible” and nothing else: the self-test shortens [Example 22.6](#) to $N = 8$ and gets status 2 in a few milliseconds, with no hint which row is to blame. Diagnose by relaxation: drop the separation rows, then the obstacles, then the terminal condition, and see when the problem becomes feasible; compare the horizon with the minimum-time bound of a single drone; or add elastic slacks with a large linear penalty to the suspect rows, as in the soft constraints of [Chapter 21](#), and read off which slacks are non-zero.

22.11 Where this fits in the drone system

In the drone system

The hybrid architecture of [Chapter 24](#) is built from heuristics: CBS plans on a grid, ORCA and DWA look one step ahead, the replanner repairs paths locally. None of them knows how far its plan is from the best plan. MILP is the *offline reference solution*: for the small scenarios of the experiment matrix of [Chapter 25](#), two to four drones, one or two obstacles, a horizon of a few seconds, [Definition 22.4](#) yields the exact optimum of fuel, peak acceleration or arrival time, with a proof, against which the collision rate, path length and travel time of the hybrid planner are measured. The second use is *scheduling*: with the assignment gadget of [Table 22.1](#), take-off and landing slots, charging pads and corridor time windows are assigned to a fleet under separation and deadline constraints, a problem with dozens of binaries that branch-and-bound solves instantly and that no local planner can express. The study plan places MILP in Week 11 next to MPC ([Appendix A](#)) as a Medium-priority topic.

22.12 Summary

Chapter summary

- A linear program minimizes a linear objective over a polyhedron and attains its optimum at a vertex; simplex and interior-point methods solve it reliably. Integer and binary variables add choices, either-or conditions, on/off switches and assignments; the LP relaxation bounds the integer optimum.
- The big- M trick turns “outside the rectangle” into four switched inequalities with at most three switches on, and “at least $d_{\min}$ apart” into the same with four binaries per pair. M is the largest possible violation inside the workspace; larger values weaken the relaxation and endanger the tolerances.
- The multi-vehicle trajectory problem with double-integrator dynamics, input and speed bounds, obstacles, separation and a 1-norm, ∞ -norm or arrival-time objective is one MILP with $4N(mn_O + \binom{m}{2})$ binaries.

- Branch-and-bound solves LP relaxations, branches on fractional binaries and prunes by bound and infeasibility. It is exact ([Theorem 22.7](#)) and exponential in the worst case ([Proposition 22.8](#)); solvers add presolve, cuts and heuristics and report an incumbent with a gap.
- The constraints hold at the sampled instants only; the margins $v_{\max}\Delta t/2$ and $v_{\max}\Delta t$ of [Proposition 22.9](#) make them hold in between.
- MAPF is an integer program on the time-expanded graph, exact like CBS but scaling with the map and the horizon rather than with the conflicts.
- MILP is for small, safety-critical, offline problems, reference solutions, verification and scheduling; not for online replanning of large swarms.

Further reading. The formulation of this chapter is that of Schouwenaars, De Moor, Feron and How [147] and of Richards and How [139], who introduced the minimum-time encoding; Yu and LaValle [185] give the MAPF flow formulation. Branch-and-bound goes back to Land and Doig [96]; Wolsey [181] is the standard textbook on integer programming, and Vielma [171] surveys formulation techniques, including tighter alternatives to big- M . Linear programs and the interior-point method are in Boyd and Vandenberghe [24], the simplex method in Nocedal and Wright [119], and the HiGHS solver in Huangfu and Hall [69]. Borrelli, Bemporad and Morari [21] treat receding-horizon mixed-integer models as hybrid MPC.

22.13 Exercises

Exercise 22.1 (★). The polyhedron of [Figure 22.1](#) is $3x + 2y \leq 15$, $-x + 2y \leq 5$, $x \leq 4$, $x, y \geq 0$, and the objective is to maximize $x + 2y$. List the vertices and their objective values and confirm the LP optimum. Then run [Algorithm 22.1](#) by hand (for a maximization, prune a node whose bound is at most the incumbent), once branching on x first and once on y first, and draw both trees.

Exercise 22.2 (★). Compute the tight constants [Equation \(22.4\)](#) for the block and the workspace of [Example 22.6](#), and the tight M of the separation rows [Equation \(22.6\)](#). Show by an explicit point $\mathbf{p}_k$ that an M smaller than the tight value cuts off positions that are outside the block. What goes wrong in the model when $M_1 = 0$?

Exercise 22.3 (★★). Prove the analog of [Proposition 22.3](#) for the separation encoding [Equation \(22.6\)](#): two positions in W satisfy $\|\mathbf{p}_k^i - \mathbf{p}_k^j\|_\infty \geq d_{\min}$ if and only if binaries satisfying [Equation \(22.6\)](#) exist. Then write the encoding of “outside a convex polygon with r edges” and of “outside a box in three dimensions”, and count the binaries of a problem with four drones, two hexagonal obstacles and $N = 20$.

Exercise 22.4 (★★). Apply [Proposition 22.9](#) to [Example 22.6](#): compute the obstacle margin δ and the sampled separation that guarantee the continuous motion, and explain why the block must be inflated on all four sides. Solve the safe instance with `example_instance("fuel", d_min=2.5, inflate=0.75)` and compare its fuel, its solve time and its continuous-time minimum separation with the unsafe plan. Then inflate the block only and report what happens to the separation between samples.

Exercise 22.5 (★★). Show that the arrival rows of [Equation \(22.9\)](#) force a drone to be at its goal at every sampled instant from its arrival step on, and give an example of two plans with the same arrival times but different fuel to explain why the objective adds a small multiple of J_{fuel} . Extend the encoding so that drone i must arrive within a time window $[t_a^i, t_b^i]$, and so that two drones may not arrive in the same step.

Exercise 22.6 (★★ Modeling). A fleet of m drones returns to a base with two landing pads. Landings are scheduled in slots of 30 s; drone i must have landed before its battery deadline T_i , two landings on the same pad must be at least two slots apart, and the cost is the total waiting time. Formulate the schedule as an integer program with assignment binaries $z_{i,p,q}$ (drone i , pad p , slot q): write the objective and every constraint family, count the binaries and the rows for $m = 6$ drones and $q = 1, \dots, 8$, and say which gadgets of Table 22.1 you used.

Exercise 22.7 (★★). Write out Equation (22.10) for the 4-cycle of Figure 22.5 with $T = 2$ and the two agents shown: list the binaries of agent 1, its flow-conservation rows at $(a, 0)$, $(b, 1)$ and $(d, 2)$, the capacity row of $(b, 1)$ and the anti-swap row of the edge $\{a, b\}$ at $t = 0$. Check that the drawn flows satisfy every row and that the dashed alternative violates one. Then count the binaries and the capacity and anti-swap rows for an obstacle-free 32×32 grid with 20 agents and $T = 60$.

Exercise 22.8 (★★★ Coding). Formulate and solve the courtyard problem of Section 22.1 with `ch22_milp.py`: three drones in a 10×8 m workspace, A from $(1, 1)$ to $(9, 7)$, B from $(1, 4)$ to $(9, 4)$ and C from $(1, 7)$ to $(9, 1)$, a block $[4, 6] \times [2, 6]$, $N = 12$ steps of $\Delta t = 0.5$ s, $a_{\max} = 2$ m/s², $v_{\max} = 3$ m/s and $d_{\min} = 1$ m. (a) Build the `Instance` and solve it with the fuel and the time objective as `solve_instance(inst, time_limit=120)`; report `sol.status` (and `sol.gap`, if the limit strikes) as well as the objective, the binaries, the nodes and the time in a table like Table 22.4, and check both solutions with `check_solution` and `continuous_check`. Expect a solve of tens of seconds: three drones are already a hard instance (Section 22.9). (b) Plot the trajectories and mark the steps at which a separation constraint is active. (c) Solve the three-drone crossing instance `crossing_instance(3, 12)` with a time limit of 10 s and report the incumbent and the gap. (d) Add the symmetry-breaking constraint of Section 22.10 to the pair (drone 1, drone 2), measure the change in nodes and time, and check whether the optimum changed.

Consensus and Formation Control

Learning objectives

- Build the communication graph of a swarm from the drone positions and the radio range, write down its adjacency, degree and Laplacian matrices, and read connectivity and convergence speed off the Laplacian spectrum.
- State the consensus protocol $\dot{\mathbf{x}} = -\mathbf{L}\mathbf{x}$, prove that on a connected graph it drives every drone to the average of the initial states at the rate λ_2 , and choose a safe step size for its discrete-time version.
- Turn consensus into a formation controller by prescribing displacements, pin a leader to a reference path, and explain why the formation is reached only up to a translation.
- Define the formation error and the algebraic connectivity as the two quantities that a formation-flying swarm monitors, and express the communication radius as a hard constraint or as a potential.
- Extend the controller to double-integrator drones and to flocking with separation, alignment and navigation terms, and implement and test all of it in Python.

23.1 Why this matters for a drone swarm

Four camera drones film a moving vehicle from four sides. The images are only useful if the drones keep a square of side 2 m around the target, and the drones can only coordinate if every one of them stays within the radio range R_{comm} of at least one other drone. The global planner of the hybrid architecture (Chapter 24) has produced a conflict-free path for the square as a whole with CBS or ECBS (Chapters 9 and 10); the reactive layer (ORCA, Chapter 13; DWA, Chapter 14) will push single drones off that path when a bird crosses the scene. Between these two layers something has to hold the square together: a controller that lets each drone use only what it can hear from its neighbors, that keeps the relative positions, that keeps the communication graph connected, that yields to an avoidance maneuver and that pulls the square back together afterward. That controller is the subject of this chapter, and its two constraints, formation preservation and the communication radius, are the last two constraints of the capstone system of Chapter 24; Chapter 25 measures how well they are met with the formation error and the communication violations defined here.

The controller is built from a single rule, **consensus**: every drone moves its state toward the average of its neighbors' states. Consensus is one of the few results in this book with a complete, short and beautiful theory. Its convergence is decided by one number, the second-smallest eigenvalue λ_2 of the graph Laplacian, which is positive exactly when the communication graph

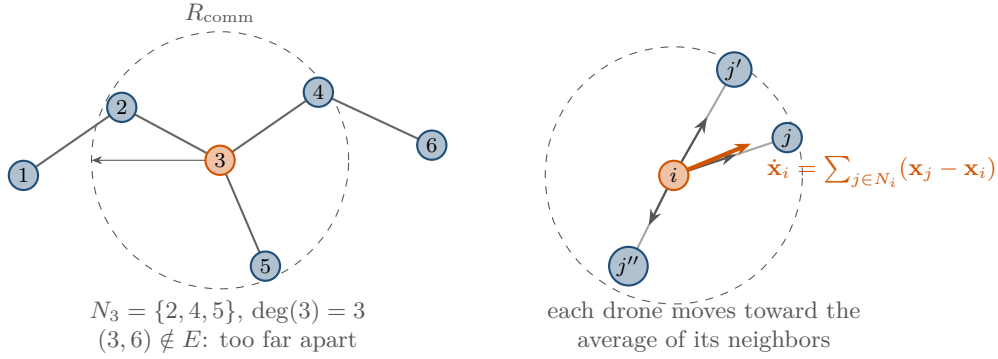


Figure 23.1. Left: the communication graph of six drones with radio range R_{comm} ; drone 3 hears drones 2, 4 and 5 and nobody else, so its update uses three neighbors. Right: the consensus rule for one drone. Each gray arrow is the difference $\mathbf{x}_j - \mathbf{x}_i$ to a neighbor; their sum (orange) is $|N_i|$ times the vector from $\mathbf{x}_i$ to the neighbors' average, so drone i moves toward the average of the drones it can hear.

is connected and which is also the rate at which the drones agree. Everything else in the chapter, leader–follower tracking, displacement-based formation control, second-order control for double-integrator drones and flocking, is consensus applied to a shifted or extended state.

The chapter follows the usual order: the rule and its picture, the graph matrices and the two error metrics, the controllers (consensus, its discrete-time form, a pinned leader, formations, the second-order version), a worked example of four drones, the convergence theorems, and finally flocking, connectivity maintenance and the implementation. Everything after [Section 23.4](#) rests on the single observation of [Definition 23.5](#): a formation is consensus on a shifted state.

23.2 The idea in plain words

Imagine that each drone holds a number, say its altitude, and that the swarm wants to agree on a common altitude without a leader and without anybody knowing all the numbers. The rule is: at every instant, move your number toward the numbers of the drones you can hear ([Figure 23.1](#)). A drone whose neighbors are all higher climbs, a drone in the middle of its neighbors stays. Two things follow immediately. Whatever one drone gains, its neighbor loses, so the *sum* of the numbers never changes; and the *spread* of the numbers can only shrink, because the largest number can only go down and the smallest can only go up. If the communication graph is connected, the spread shrinks to zero and the swarm settles on the average of the initial numbers. How fast it does so depends on how well the graph is connected: a long chain of drones agrees slowly, a dense cluster agrees fast.

Formation control is the same rule applied to a shifted number. Give every drone a slot $\mathbf{o}_i$ in a template of the formation, say the four corners of a square, and let the drones run consensus not on their positions but on $\mathbf{p}_i - \mathbf{o}_i$, the position of the template's origin as drone i sees it. When they agree on that origin, every drone is in its slot and the swarm is in formation. The template can be placed anywhere: consensus decides *where*, the offsets decide the *shape*. If one drone is told where the origin should be, for example by the CBS path, the whole formation follows it.

Key idea

Consensus is the rule $\dot{\mathbf{x}}_i = \sum_{j \in N_i} (\mathbf{x}_j - \mathbf{x}_i)$, in matrix form $\dot{\mathbf{x}} = -\mathbf{L}\mathbf{x}$ with the graph Laplacian $\mathbf{L}$. On a connected graph it drives every drone to the average of the initial states, and the disagreement decays like $e^{-\lambda_2 t}$, where λ_2 is the algebraic connectivity of the graph. A formation is consensus on the shifted positions $\mathbf{p}_i - \mathbf{o}_i$: the drones agree on where the template sits, and the offsets give the shape.

23.3 Problem statement and notation

23.3.1 The communication graph and its matrices

Definition 23.1 (Communication graph). A swarm of n drones with positions $\mathbf{p}_1, \dots, \mathbf{p}_n \in \mathbb{R}^m$ ($m = 2$ or 3) forms the **communication graph** $G = (V, E)$ with $V = \{1, \dots, n\}$ and an edge $\{i, j\} \in E$ whenever drones i and j can exchange messages. For a radio of range R_{comm} this is the **radius graph**

$$\{i, j\} \in E \iff \|\mathbf{p}_i - \mathbf{p}_j\| \leq R_{\text{comm}}, \quad i \neq j. \quad (23.1)$$

The **neighborhood** of drone i is $N_i = \{j \mid \{i, j\} \in E\}$ and its **degree** is $d_i = |N_i|$. The graph is **directed** when the links are one-way: then $(j, i) \in E$ means that i receives from j , and N_i is the set of drones that i hears. A graph that changes with time is a **switching topology**; the radius graph switches whenever a pair of drones crosses the distance R_{comm} .

The mathematics refresher (Appendix B) collects the linear algebra used below: symmetric matrices have real eigenvalues and an orthonormal basis of eigenvectors, and a symmetric matrix is positive semidefinite exactly when its quadratic form is non-negative.

Definition 23.2 (Adjacency, degree and Laplacian matrices). The **adjacency matrix** $\mathbf{W} \in \mathbb{R}^{n \times n}$ has $w_{ij} = 1$ if $\{i, j\} \in E$ and $w_{ij} = 0$ otherwise (in particular $w_{ii} = 0$); weighted graphs use $w_{ij} > 0$ for the strength of the link. The **degree matrix** is $\mathbf{D} = \text{diag}(d_1, \dots, d_n)$ with $d_i = \sum_j w_{ij}$, and the **graph Laplacian** is

$$\mathbf{L} = \mathbf{D} - \mathbf{W}, \quad \ell_{ij} = \begin{cases} d_i & i = j, \\ -w_{ij} & i \neq j. \end{cases} \quad (23.2)$$

Its eigenvalues are written in increasing order, $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$; λ_2 is the **algebraic connectivity** of the graph [44]. $\mathbf{1}$ denotes the vector of n ones.

Proposition 23.3 (Properties of the Laplacian of an undirected graph). *Let G be undirected with n vertices and let $d_{\max}$ and $d_{\min}$ be its largest and smallest degree. Then*

- (i) every row of $\mathbf{L}$ sums to zero: $\mathbf{L}\mathbf{1} = \mathbf{0}$ and $\mathbf{1}^\top \mathbf{L} = \mathbf{0}^\top$;
- (ii) $\mathbf{L}$ is symmetric and $\mathbf{x}^\top \mathbf{L} \mathbf{x} = \sum_{\{i,j\} \in E} w_{ij} (x_i - x_j)^2 \geq 0$ for every $\mathbf{x} \in \mathbb{R}^n$, so $\mathbf{L}$ is positive semidefinite and $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$;
- (iii) the multiplicity of the eigenvalue 0 equals the number of connected components of G ; in particular $\lambda_2 > 0$ if and only if G is connected;
- (iv) $\lambda_n \leq 2d_{\max}$;
- (v) if all weights are 1 and G is not complete then $\lambda_2 \leq d_{\min}$, and the complete graph K_n with unit weights has $\lambda_2 = n$.

Proof. (i) The diagonal entry d_i of row i equals the sum of the w_{ij} that are subtracted in the same row. (ii) Symmetry is inherited from $\mathbf{W}$. Expand $\mathbf{x}^\top \mathbf{L} \mathbf{x} = \sum_i d_i x_i^2 - \sum_{i \neq j} w_{ij} x_i x_j$ and

group the terms of each edge: $w_{ij}(x_i^2 + x_j^2 - 2x_i x_j) = w_{ij}(x_i - x_j)^2$. A symmetric matrix with a non-negative quadratic form has non-negative eigenvalues, and $\lambda_1 = 0$ by (i). (iii) By (ii), $\mathbf{L}\mathbf{x} = \mathbf{0}$ holds if and only if $\mathbf{x}^\top \mathbf{L}\mathbf{x} = 0$, that is, $x_i = x_j$ on every edge, that is, $\mathbf{x}$ is constant on every connected component. The indicator vectors of the components therefore span the null space, whose dimension is the number of components. (iv) By Gershgorin's theorem every eigenvalue lies in a disk centered at some diagonal entry d_i with radius $\sum_{j \neq i} |\ell_{ij}| = d_i$, hence in $[0, 2d_{\max}]$. (v) is Fiedler's theorem [44]: for a non-complete graph λ_2 is at most the vertex connectivity, which is at most $d_{\min}$. For K_n , $\mathbf{L} = n\mathbf{I} - \mathbf{1}\mathbf{1}^\top$ has the eigenvalue n on the whole space orthogonal to $\mathbf{1}$. $\square$

Two examples fix the scale of λ_2 . A path of n drones in a line, each hearing only its two neighbors, has $\lambda_2 = 2(1 - \cos(\pi/n))$, which is 0.27 for $n = 6$ and shrinks like π^2/n^2 ; a complete graph on the same six drones has $\lambda_2 = 6$. Connectivity is a matter of degree, and λ_2 measures it on a continuous scale from 0 (disconnected) upward.

23.3.2 Consensus

Definition 23.4 (Consensus). Each drone i holds a state $\mathbf{x}_i \in \mathbb{R}^m$ (a scalar, a position, a velocity, an estimate). The swarm **reaches consensus** if $\mathbf{x}_i(t) - \mathbf{x}_j(t) \rightarrow \mathbf{0}$ for all pairs i, j as $t \rightarrow \infty$, and **average consensus** if the common limit is the average $\bar{\mathbf{x}} = \frac{1}{n} \sum_i \mathbf{x}_i(0)$ of the initial states. Stacking the states of a scalar problem into $\mathbf{x} = (x_1, \dots, x_n)^\top$, the **disagreement vector** is $\delta = \mathbf{x} - \bar{\mathbf{x}}\mathbf{1}$, the component of $\mathbf{x}$ orthogonal to $\mathbf{1}$; the swarm is in consensus exactly when $\delta = \mathbf{0}$. Vector states are handled one coordinate at a time.

23.3.3 Formations and the formation error

Definition 23.5 (Displacement-based formation). A **formation** is described by a template of offsets $\mathbf{o}_1, \dots, \mathbf{o}_n \in \mathbb{R}^m$ in a common frame, one slot per drone. The **desired displacement** from drone i to drone j is

$$\mathbf{d}_{ij} = \mathbf{o}_j - \mathbf{o}_i, \quad \text{so that} \quad \mathbf{d}_{ji} = -\mathbf{d}_{ij} \quad \text{and} \quad \mathbf{d}_{ij} + \mathbf{d}_{jk} + \mathbf{d}_{ki} = \mathbf{0}. \quad (23.3)$$

The **formation graph** $G_F = (V, E_F)$ lists the pairs whose displacement is prescribed and measured; it is connected and is usually the communication graph, a spanning subgraph of it, or the complete graph. The swarm is **in formation** if $\mathbf{p}_j - \mathbf{p}_i = \mathbf{d}_{ij}$ for every $\{i, j\} \in E_F$; because G_F is connected this holds if and only if $\mathbf{p}_i = \mathbf{o}_i + \mathbf{c}$ for all i and one common translation $\mathbf{c}$.

A formation is therefore defined only up to translation, which is exactly what the leader supplies. The shifted variable $\mathbf{y}_i = \mathbf{p}_i - \mathbf{o}_i$ is the position of the template origin as seen by drone i , and the swarm is in formation if and only if all $\mathbf{y}_i$ agree. This is the observation that turns consensus into formation control in Section 23.4.4.

Definition 23.6 (Formation error). The **formation error** of the positions $\mathbf{p} = (\mathbf{p}_1, \dots, \mathbf{p}_n)$ with respect to the formation $(\mathbf{o}_i)$ and the formation graph G_F is the root-mean-square violation of the desired displacements over the edges of G_F ,

$$e_F(\mathbf{p}) = \sqrt{\frac{1}{|E_F|} \sum_{\{i,j\} \in E_F} \|\mathbf{p}_j - \mathbf{p}_i - \mathbf{d}_{ij}\|^2} = \sqrt{\frac{1}{|E_F|} \sum_{\{i,j\} \in E_F} \|\mathbf{y}_j - \mathbf{y}_i\|^2}. \quad (23.4)$$

The **centered formation error** removes the best translation first: with $\bar{\mathbf{y}} = \frac{1}{n} \sum_i \mathbf{y}_i$,

$$e_c(\mathbf{p}) = \sqrt{\frac{1}{n} \sum_{i=1}^n \|\mathbf{y}_i - \bar{\mathbf{y}}\|^2}. \quad (23.5)$$

Both errors have the unit of a length, both are invariant under a translation of the whole swarm, and both are zero exactly when the swarm is in formation. e_F is the metric used by Chapter 25: it only needs the pairs of the formation graph, so a drone can compute its own contribution from its neighbors' messages. The translation $\bar{\mathbf{y}}$ that e_c removes is the least-squares fit of the template to the current positions, and for the complete formation graph the two metrics are proportional, $e_F^2 = \frac{2n}{n-1} e_c^2$ (Exercise 23.2). Two more quantities are monitored alongside e_F : the **edge lengths** $\ell_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\|$ of the edges that must stay in range, and the number of **communication violations**, the edges of a chosen set $E_{\text{keep}} \subseteq E_F$ that are longer than R_{comm} at a given time.

23.4 The algorithm: consensus and its extensions

23.4.1 The consensus protocol

The **consensus protocol** of Olfati-Saber and Murray [124] is the rule of Section 23.2 written as a differential equation. Drone i moves its state with the velocity

$$\dot{\mathbf{x}}_i = \sum_{j \in N_i} (\mathbf{x}_j - \mathbf{x}_i) = \sum_{j=1}^n w_{ij} (\mathbf{x}_j - \mathbf{x}_i), \quad i = 1, \dots, n. \quad (23.6)$$

The right-hand side is $-d_i \mathbf{x}_i + \sum_j w_{ij} \mathbf{x}_j$, which is row i of $-\mathbf{L}\mathbf{x}$, so the whole swarm obeys the linear system

$$\dot{\mathbf{x}} = -\mathbf{L}\mathbf{x}. \quad (23.7)$$

Three features of Equation (23.6) matter for a swarm. The rule is *local*: drone i needs only the states of the drones it hears. It is *leaderless* and *symmetric*: no drone is special and every message is used the same way. And it *conserves the sum*: by Proposition 23.3(i), $\frac{d}{dt}(\mathbf{1}^\top \mathbf{x}) = -\mathbf{1}^\top \mathbf{L}\mathbf{x} = 0$, so the average $\bar{x}$ is a constant of the motion. Every drone that runs Equation (23.6) is therefore computing the same number, the average of the initial states, without anybody ever collecting them. A **consensus gain** $k_c > 0$ in front of the sum only rescales time ($\mathbf{x}(t)$ becomes $\mathbf{x}(k_c t)$) and is omitted in the analysis. The letter k is reserved for the time step and the mode index.

23.4.2 Discrete-time consensus and the step size

A drone updates its command at a control period Δt , so what it really runs is the Euler step of Equation (23.7) with the step size $h = k_c \Delta t$:

$$\mathbf{x}_{k+1} = (\mathbf{I} - h\mathbf{L}) \mathbf{x}_k, \quad \text{that is} \quad \mathbf{x}_{i,k+1} = \mathbf{x}_{i,k} + h \sum_{j \in N_i} (\mathbf{x}_{j,k} - \mathbf{x}_{i,k}). \quad (23.8)$$

The matrix $\mathbf{P} = \mathbf{I} - h\mathbf{L}$ is called the Perron matrix of the graph [123]. Its rows sum to one, and its entries $1 - hd_i$ on the diagonal and hw_{ij} off it are all non-negative if and only if $h \leq 1/d_{\max}$. In that case every drone's new state is a weighted average of its own and its neighbors' states, in which it keeps the weight $1 - hd_i$ on itself, and the maximum can never grow nor the minimum shrink. The **step-size condition**

$$0 < h < \frac{1}{d_{\max}} \quad (23.9)$$

is the sufficient condition every drone can check from local information (it knows its own degree, and a bound on the largest degree is a design parameter of the swarm). The exact condition is $h < 2/\lambda_n$, which by Proposition 23.3(iv) is implied by Equation (23.9); both are proved in Theorem 23.10. Above the exact bound the iteration diverges, with the fastest mode of the graph flipping sign at every step; the worked example shows this happening.

23.4.3 Leader–follower consensus

Average consensus agrees on a value that nobody chose. To make the swarm follow a reference $\mathbf{r}(t)$, for example the CBS path of the formation, one or more drones are **pinned** to it [136]:

$$\dot{\mathbf{x}}_i = \sum_{j \in N_i} (\mathbf{x}_j - \mathbf{x}_i) + b_i k_l (\mathbf{r} - \mathbf{x}_i) + \dot{\mathbf{r}}, \quad b_i = \begin{cases} 1 & \text{drone } i \text{ is pinned,} \\ 0 & \text{otherwise,} \end{cases} \quad (23.10)$$

with a pinning gain $k_l > 0$. The pinned drones are the **leaders**; the others follow them through the graph and never see $\mathbf{r}$. With $\mathbf{B} = \text{diag}(b_1, \dots, b_n)$ the tracking error $\mathbf{e} = \mathbf{x} - \mathbf{1} \otimes \mathbf{r}$ (every drone compared with the reference) obeys

$$\dot{\mathbf{e}} = -(\mathbf{L} + k_l \mathbf{B}) \mathbf{e}, \quad (23.11)$$

and Proposition 23.12 shows that $\mathbf{L} + k_l \mathbf{B}$ is positive definite whenever the graph is connected and at least one drone is pinned, so that $\mathbf{e} \rightarrow \mathbf{0}$. The **feed-forward** term $\dot{\mathbf{r}}$ in Equation (23.10) is the velocity of the reference, which the leader knows from its path and the followers can receive from it through the graph. Without it the followers still converge, but to a formation that trails the reference by a constant lag proportional to the speed (Exercise 23.7); Figure 23.4 shows the difference.

23.4.4 Formation control by displacements

The **displacement-based formation controller** [121, 136] replaces the difference of the states by the difference minus the desired displacement:

$$\dot{\mathbf{p}}_i = \sum_{j \in N_i} (\mathbf{p}_j - \mathbf{p}_i - \mathbf{d}_{ij}). \quad (23.12)$$

Drone i moves toward the point $\mathbf{p}_j - \mathbf{d}_{ij}$ where it should be according to each neighbor j , and it stops when it is there for all of them. With $\mathbf{d}_{ij} = \mathbf{o}_j - \mathbf{o}_i$ and the shifted variables $\mathbf{y}_i = \mathbf{p}_i - \mathbf{o}_i$ the summand is $\mathbf{y}_j - \mathbf{y}_i$, so that

$$\dot{\mathbf{y}}_i = \sum_{j \in N_i} (\mathbf{y}_j - \mathbf{y}_i), \quad \dot{\mathbf{y}} = -\mathbf{L} \mathbf{y} \text{ in every coordinate :} \quad (23.13)$$

the shifted variables run the plain consensus protocol Equation (23.7). The offsets never appear in the dynamics of $\mathbf{y}$; they only decide what the agreement means. Consequently every theorem about consensus is a theorem about formation control: on a connected graph the $\mathbf{y}_i$ converge to their initial average $\bar{\mathbf{y}} = \frac{1}{n} \sum_i (\mathbf{p}_i(0) - \mathbf{o}_i)$, and the drones settle at $\mathbf{p}_i = \mathbf{o}_i + \bar{\mathbf{y}}$, the template translated by the initial centroid shift (Theorem 23.13). Pinning a leader to $\mathbf{r} + \mathbf{o}_i$ as in Equation (23.10) pins the template origin to $\mathbf{r}$ and makes the formation follow the path.

Inconsistent displacement vectors

Every $\mathbf{d}_{ij}$ must satisfy Equation (23.3). If drone i wants j two meters to its east but j wants i one meter to its west, the pair never settles: the sum of the displacement terms over the graph is no longer zero, and the centroid of the swarm drifts with a constant velocity while the relative positions converge to a compromise (Exercise 23.5). The same happens when the vectors do not close around a cycle, $\mathbf{d}_{12} + \mathbf{d}_{23} + \mathbf{d}_{31} \neq \mathbf{0}$: no set of positions satisfies all edges, the controller minimizes the residual and a formation error remains for ever. The safe way is to store the offsets $\mathbf{o}_i$ and to compute $\mathbf{d}_{ij} = \mathbf{o}_j - \mathbf{o}_i$ on the fly, which makes both conditions automatic; never store the $\mathbf{d}_{ij}$ themselves.

Algorithm 23.1. One control period of the displacement-based formation controller on drone i

Input: own position $\mathbf{p}_i$ and offset $\mathbf{o}_i$; messages $(\mathbf{p}_j, \mathbf{o}_j)$ from every neighbor $j \in N_i$, i.e., every drone within R_{comm} ; gains k_c, k_l ; control period Δt ; speed limit v_{max} ; the reference velocity $\dot{\mathbf{r}}$, forwarded from the leader through the graph to every drone (use $\mathbf{0}$ if unknown); if drone i is pinned ($b_i = 1$), also the reference $\mathbf{r}$; optional extra velocity $\mathbf{v}_i^{\text{extra}}$ (avoidance, range keeping)

Output: the velocity command $\mathbf{v}_i$ and the new position $\mathbf{p}_i$

```

1 Function FormationStep( $i$ ):
2    $\mathbf{v}_i \leftarrow \mathbf{0}$ 
3   foreach  $j \in N_i$  do
4      $\mathbf{d}_{ij} \leftarrow \mathbf{o}_j - \mathbf{o}_i$ 
5      $\mathbf{v}_i \leftarrow \mathbf{v}_i + k_c(\mathbf{p}_j - \mathbf{p}_i - \mathbf{d}_{ij})$            // consensus on  $\mathbf{p} - \mathbf{o}$ 
6   if  $b_i = 1$  then
7      $\mathbf{v}_i \leftarrow \mathbf{v}_i + k_l(\mathbf{r} + \mathbf{o}_i - \mathbf{p}_i)$            // pin the leader to the reference
8    $\mathbf{v}_i \leftarrow \mathbf{v}_i + \dot{\mathbf{r}} + \mathbf{v}_i^{\text{extra}}$            // feed-forward and extra terms
9   if  $\|\mathbf{v}_i\| > v_{\text{max}}$  then
10     $\mathbf{v}_i \leftarrow v_{\text{max}} \mathbf{v}_i / \|\mathbf{v}_i\|$ 
11    $\mathbf{p}_i \leftarrow \mathbf{p}_i + \Delta t \mathbf{v}_i$            // or hand  $\mathbf{v}_i$  to the autopilot
12   broadcast  $(\mathbf{p}_i, \mathbf{o}_i)$ 
13   return  $\mathbf{v}_i, \mathbf{p}_i$ 

```

Algorithm 23.1 is one control period of drone i with all of the above combined. It is written from the point of view of a single drone, because that is how it runs: there is no central computer that assembles $\mathbf{L}$.

Line 4 derives the displacement from the offsets, which is why the message carries $\mathbf{o}_j$ (or a slot number from which $\mathbf{o}_j$ is looked up). Line 5 is Equation (23.12) with the gain k_c ; with all offsets zero it is the consensus protocol Equation (23.6), and over the whole swarm lines 5 and 11 are the discrete-time protocol Equation (23.8) with $h = k_c \Delta t$, so the gain and the period must satisfy $k_c \Delta t < 1/d_{\text{max}}$. Line 7 is the pinning term of Equation (23.10) applied to the template origin: the leader is pulled to $\mathbf{r} + \mathbf{o}_i$, its own slot of a template whose origin is on the reference. Line 8 adds the reference velocity, which is known to the leader from its path and is forwarded to the followers, and the extra velocities of Section 23.7.3: the safety term of the reactive layer and the range-keeping term. Line 10 enforces the speed limit of Chapter 2 without changing the direction, and line 11 is the Euler step, which on a real drone is replaced by sending $\mathbf{v}_i$ to the velocity controller of the autopilot. Line 12 closes the loop: the position that drone i broadcasts now is what its neighbors use in their next period. The whole period costs $\mathcal{O}(|N_i| m)$ operations, one message in from each neighbor and one message out.

23.4.5 Second-order consensus for double-integrator drones

A quadrotor is not a first-order system that can set its velocity at will: to a good approximation it is the double integrator $\ddot{\mathbf{p}}_i = \mathbf{u}_i$ of Chapter 2, whose input is an acceleration. Feeding the velocity of Equation (23.12) through an inner velocity loop works when that loop is fast; when the formation loop and the dynamics interact, one uses the **second-order consensus** protocol [123, 136]

$$\mathbf{u}_i = \sum_{j \in N_i} [k_p(\mathbf{p}_j - \mathbf{p}_i - \mathbf{d}_{ij}) + k_v(\mathbf{v}_j - \mathbf{v}_i)] - k_d(\mathbf{v}_i - \dot{\mathbf{r}}) + b_i [k_p(\mathbf{r} + \mathbf{o}_i - \mathbf{p}_i) + k_v(\dot{\mathbf{r}} - \mathbf{v}_i)]. \quad (23.14)$$

The first sum is the displacement term of Equation (23.12) used as a spring. The second sum is consensus on the velocities: it damps the relative motion of neighbors. The term $-k_d(\mathbf{v}_i - \dot{\mathbf{r}})$ is absolute damping toward the reference velocity (with $\dot{\mathbf{r}} = \mathbf{0}$ it brings the swarm to rest); like the feed-forward of Equation (23.10) it is local only once every follower has been told $\dot{\mathbf{r}}$, forwarded from the leader through the graph. The last bracket is the pinning of the leader. Without any velocity term ($k_v = k_d = 0$) the drones are masses connected by springs and oscillate for ever; with $k_v > 0$ or $k_d > 0$ every mode of the formation decays (Proposition 23.14). In the shifted variables, with $\dot{\mathbf{r}}$ constant, the swarm obeys $\ddot{\mathbf{y}} = -k_p \mathbf{L} \mathbf{y} - (k_v \mathbf{L} + k_d \mathbf{I}) \dot{\mathbf{y}} + k_d \mathbf{1} \otimes \dot{\mathbf{r}}$ plus the pinning terms, which is again the Laplacian acting on every coordinate. The forcing term $k_d \mathbf{1} \otimes \dot{\mathbf{r}}$ is what makes the swarm travel with the reference instead of coming to rest; it is the same on every drone, so it moves the centroid and leaves the shape alone. Algorithm 23.2 is the corresponding control period.

Algorithm 23.2. One control period of the second-order formation controller on drone i

Input: own $\mathbf{p}_i, \mathbf{v}_i, \mathbf{o}_i$; messages $(\mathbf{p}_j, \mathbf{v}_j, \mathbf{o}_j)$ from $j \in N_i$; gains k_p, k_v, k_d ; period Δt ; the reference velocity $\dot{\mathbf{r}}$ on every drone (use $\mathbf{0}$ if unknown), the reference $\mathbf{r}$ if pinned

Output: acceleration command $\mathbf{u}_i$; updated $\mathbf{v}_i, \mathbf{p}_i$

```

1 Function SecondOrderFormationStep( $i$ ):
2    $\mathbf{u}_i \leftarrow -k_d(\mathbf{v}_i - \dot{\mathbf{r}})$ 
3   foreach  $j \in N_i$  do
4      $\mathbf{u}_i \leftarrow \mathbf{u}_i + k_p(\mathbf{p}_j - \mathbf{p}_i - (\mathbf{o}_j - \mathbf{o}_i)) + k_v(\mathbf{v}_j - \mathbf{v}_i)$ 
5   if  $b_i = 1$  then
6      $\mathbf{u}_i \leftarrow \mathbf{u}_i + k_p(\mathbf{r} + \mathbf{o}_i - \mathbf{p}_i) + k_v(\dot{\mathbf{r}} - \mathbf{v}_i)$ 
7    $\mathbf{v}_i \leftarrow \mathbf{v}_i + \Delta t \mathbf{u}_i$ ;  $\mathbf{p}_i \leftarrow \mathbf{p}_i + \Delta t \mathbf{v}_i$  // semi-implicit Euler
8   broadcast  $(\mathbf{p}_i, \mathbf{v}_i, \mathbf{o}_i)$ 
9   return  $\mathbf{u}_i, \mathbf{v}_i, \mathbf{p}_i$ 

```

Line 4 needs the neighbors' velocities as well as their positions, so the messages double in size, and line 7 updates the velocity before the position (semi-implicit Euler), which keeps the oscillatory modes of the spring system stable for the step sizes used in practice.

23.5 A worked example

Example 23.7 (Four drones, one Laplacian, three controllers). Four drones hover at the planar positions $\mathbf{p}_1 = (0, 0)$, $\mathbf{p}_2 = (2, 0)$, $\mathbf{p}_3 = (1, 1.5)$ and $\mathbf{p}_4 = (3.5, 3)$ (meters) with a radio range $R_{\text{comm}} = 3$ m. The pairwise distances are $\ell_{12} = 2$, $\ell_{13} = \ell_{23} = 1.80$, $\ell_{34} = 2.92$, $\ell_{24} = 3.35$ and $\ell_{14} = 4.61$, so the radius graph has the four edges $\{1, 2\}, \{1, 3\}, \{2, 3\}, \{3, 4\}$ (Figure 23.2a, gray), the degrees $(2, 2, 3, 1)$ and

$$\mathbf{W} = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad \mathbf{L} = \mathbf{D} - \mathbf{W} = \begin{pmatrix} 2 & -1 & -1 & 0 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix}.$$

Every row sums to zero. The eigenvalues are $\lambda_1 = 0$, $\lambda_2 = 1$, $\lambda_3 = 3$ and $\lambda_4 = 4$ (their sum, 8, is the trace of $\mathbf{L}$), with the eigenvectors $\mathbf{1}$, $(1, 1, 0, -2)^T$, $(1, -1, 0, 0)^T$ and $(1, 1, -3, 1)^T$; multiply $\mathbf{L}$ by each to check. The graph is connected ($\lambda_2 > 0$), its **Fiedler**

Table 23.1. Consensus on the altitudes of [Example 23.7](#): the exact solution of $\dot{\mathbf{z}} = -\mathbf{L}\mathbf{z}$, the disagreement norm and the bound $e^{-\lambda_2 t} \|\delta(0)\|$ with $\lambda_2 = 1$. The average 5.5 m is conserved in every row.

t [s]	z_1	z_2	z_3	z_4	$\ \delta(t)\ $	bound
0	10.000	4.000	7.000	1.000	6.708	6.708
0.5	7.315	5.976	5.703	3.006	3.127	4.069
1	6.376	6.077	5.527	4.019	1.815	2.468
2	5.778	5.763	5.501	4.958	0.663	0.908
3	5.600	5.599	5.500	5.301	0.244	0.334
5	5.513	5.513	5.500	5.473	0.033	0.045

vector $(1, 1, 0, -2)^\top$ separates the leaf drone 4 from drones 1 and 2 across the bottleneck edge $\{3, 4\}$, and the **time constant** of consensus is $1/\lambda_2 = 1$ s. The bounds on the discrete step size are $1/d_{\max} = 1/3$ and $2/\lambda_4 = 1/2$.

Consensus on the altitudes. Let the drones agree on a common altitude starting from $\mathbf{z}(0) = (10, 4, 7, 1)^\top$ m, whose average is 5.5 m. [Table 23.1](#) lists the exact solution of $\dot{\mathbf{z}} = -\mathbf{L}\mathbf{z}$ and the disagreement norm $\|\delta(t)\|$ next to the bound $e^{-\lambda_2 t} \|\delta(0)\|$ of [Corollary 23.9](#). Drone 4, which hears only drone 3, is the slow one; after 3 s the bound leaves 5% of the initial disagreement and after 5 s 0.7%, and the actual disagreement is smaller still because its components along the faster modes λ_3 and λ_4 have long vanished.

Discrete time. [Table 23.2](#) iterates [Equation \(23.8\)](#) from the same altitudes with $h = 0.25 < 1/3$ and with $h = 0.6 > 1/2$. The first run is a slowed-down copy of the continuous one (after the first step drone 3 lands exactly on the average of its three neighbors, $(7.75 + 6.25 + 2.5)/3 = 5.5$, so its update term is zero and it never moves again, while the others keep converging toward it); the second run overshoots at every step, the mode $(1, 1, -3, 1)^\top$ of λ_4 is multiplied by $1 - 0.6 \cdot 4 = -1.4$ each time, and the altitudes fly apart.

Formation. Now let the drones form the square with the offsets $\mathbf{o}_1 = (0, 0)$, $\mathbf{o}_2 = (2, 0)$, $\mathbf{o}_3 = (2, 2)$, $\mathbf{o}_4 = (0, 2)$ on the same fixed graph, which also serves as the formation graph. The shifted variables are $\mathbf{y} = \mathbf{p} - \mathbf{o} = ((0, 0), (0, 0), (-1, -0.5), (3.5, 1))$; drones 1 and 2 already agree, drone 4 is far off. Their average, the centroid shift, is $\bar{\mathbf{y}} = (0.625, 0.125)$, so [Theorem 23.13](#) predicts the final positions $\mathbf{o}_i + \bar{\mathbf{y}}$: $(0.625, 0.125)$, $(2.625, 0.125)$, $(2.625, 2.125)$ and $(0.625, 2.125)$, the square translated by $\bar{\mathbf{y}}$ ([Figure 23.2a](#), circles). The initial formation error over the four edges is $e_F(0) = 2.5$ m (the edge $\{3, 4\}$ alone contributes $\|(4.5, 1.5)\|^2 = 22.5$ m² to the mean of squares) and the centered error is $e_c(0) = 1.794$ m. [Table 23.3](#) follows both errors under [Equation \(23.12\)](#); they decay with the time constant 1 s of λ_2 , and e_F drops below 1% of its initial value after 4.06 s. All numbers in this example are produced by `worked_example()` in `code/ch23_consensus.py`.

23.6 Properties: convergence, rate and step size

23.6.1 Average consensus on a connected graph

Theorem 23.8 (Average consensus, Olfati-Saber and Murray [124]). *Let the communication graph be undirected and connected. Then for every initial state $\mathbf{x}(0) \in \mathbb{R}^n$ the solution of $\dot{\mathbf{x}} = -\mathbf{L}\mathbf{x}$ converges to the average of the initial states, $\mathbf{x}(t) \rightarrow \bar{x}\mathbf{1}$ with $\bar{x} = \frac{1}{n}\mathbf{1}^\top \mathbf{x}(0)$, and the disagreement vector satisfies*

$$\|\delta(t)\| \leq e^{-\lambda_2 t} \|\delta(0)\| \quad \text{for all } t \geq 0. \quad (23.15)$$

Table 23.2. Discrete-time consensus $\mathbf{z}_{k+1} = (\mathbf{I} - h\mathbf{L})\mathbf{z}_k$ on [Example 23.7](#). Left: $h = 0.25$ satisfies $h < 1/d_{\max} = 1/3$ and converges. Right: $h = 0.6$ exceeds the exact bound $2/\lambda_4 = 0.5$ and diverges; the sign of the pattern $(1, 1, -3, 1)$ flips at every step.

k	$h = 0.25$				$h = 0.6$			
	z_1	z_2	z_3	z_4	z_1	z_2	z_3	z_4
0	10.000	4.000	7.000	1.000	10.000	4.000	7.000	1.000
1	7.750	6.250	5.500	2.500	4.600	9.400	3.400	4.600
2	6.812	6.438	5.500	3.250	6.760	2.920	8.440	3.880
3	6.391	6.297	5.500	3.812	5.464	8.536	1.384	6.616
4	6.145	6.121	5.500	4.234	4.859	2.402	11.262	3.477

Table 23.3. Formation error of [Example 23.7](#) under the displacement-based controller [Equation \(23.12\)](#) with $k_c = 1$: the edge RMS error e_F , the centered error e_c and the bound $e_c(0) e^{-\lambda_2 t}$ of [Theorem 23.13](#).

t [s]	$e_F(t)$ [m]	$e_c(t)$ [m]	$e_c(0) e^{-\lambda_2 t}$ [m]
0	2.500	1.794	1.794
1	0.542	0.541	0.660
2	0.197	0.197	0.243
3	0.072	0.072	0.089
5	0.010	0.010	0.012

Proof. By [Proposition 23.3](#), $\mathbf{L}$ is symmetric, so it has an orthonormal basis of eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_n$ with eigenvalues $0 = \lambda_1 < \lambda_2 \leq \dots \leq \lambda_n$, where $\mathbf{v}_1 = \mathbf{1}/\sqrt{n}$ and $\lambda_2 > 0$ because the graph is connected. Expand the state in this basis, $\mathbf{x}(t) = \sum_k q_k(t) \mathbf{v}_k$ with $q_k(t) = \mathbf{v}_k^T \mathbf{x}(t)$. Each coefficient obeys $\dot{q}_k = \mathbf{v}_k^T \dot{\mathbf{x}} = -\mathbf{v}_k^T \mathbf{L} \mathbf{x} = -\lambda_k \mathbf{v}_k^T \mathbf{x} = -\lambda_k q_k$, using $\mathbf{v}_k^T \mathbf{L} = \lambda_k \mathbf{v}_k^T$ (symmetry). Hence $q_k(t) = e^{-\lambda_k t} q_k(0)$ and

$$\mathbf{x}(t) = q_1(0) \mathbf{v}_1 + \sum_{k=2}^n e^{-\lambda_k t} q_k(0) \mathbf{v}_k. \quad (23.16)$$

The first term is $\frac{\mathbf{1}^T \mathbf{x}(0)}{\sqrt{n}} \cdot \frac{1}{\sqrt{n}} = \bar{x} \mathbf{1}$, and it does not move: the average is conserved. The sum is orthogonal to $\mathbf{1}$, so it is exactly the disagreement vector $\delta(t)$, and every one of its terms decays because $\lambda_k \geq \lambda_2 > 0$. By orthonormality, $\|\delta(t)\|^2 = \sum_{k \geq 2} e^{-2\lambda_k t} q_k(0)^2 \leq e^{-2\lambda_2 t} \sum_{k \geq 2} q_k(0)^2 = e^{-2\lambda_2 t} \|\delta(0)\|^2$. $\square$

Corollary 23.9 (Convergence rate). *Under the consensus protocol the disagreement decays at least as fast as $e^{-\lambda_2 t}$, with the **time constant** $1/\lambda_2$, and no faster bound holds for all initial states: if $\delta(0)$ is a Fiedler vector (an eigenvector of λ_2), [Equation \(23.15\)](#) is an equality. With a gain k_c in [Equation \(23.6\)](#) the rate is $k_c \lambda_2$, and for vector states [Equation \(23.15\)](#) holds coordinate by coordinate.*

The corollary turns λ_2 into a design quantity: it says how long the swarm needs to agree and how strongly it resists being pulled apart. [Figure 23.3](#) shows the effect for six drones in a line whose initial states $(12, 3, 9, 1, 6, 11)$ have the average 7. With a short radio range each drone hears only its two neighbors, the graph is a path, $\lambda_2 = 0.27$ and the time constant is 3.7 s; with a longer range the graph has nine edges, $\lambda_2 = 1.19$, and the time constant is 0.84 s. The disagreement falls below 1% of its initial value after 6.3 s on the path and after 1.9 s on the denser graph, and the slopes of the two decays in the logarithmic plot are the two values

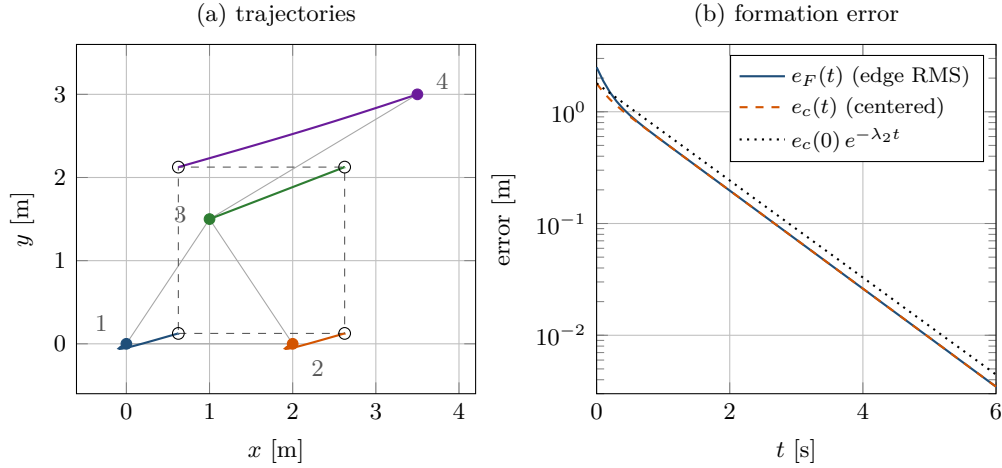


Figure 23.2. The four drones of Example 23.7 converging into a square under Equation (23.12). (a) Trajectories from the initial positions (dots) along the fixed communication graph (gray) to the final square (circles), which is the template translated by the centroid shift $\bar{\mathbf{y}} = (0.625, 0.125)$; drones 1 and 2 move although they already agree with each other, because the template origin they end up on is the average of all four. (b) The formation error decays with the time constant $1/\lambda_2 = 1$ s; the dotted line is the bound of Theorem 23.13.

of λ_2 .

A disconnected graph agrees in clusters, not as a swarm

Nothing in Equation (23.6) tells a drone that the graph is disconnected; each component runs its own consensus and settles on its own average. Two pairs of drones 10 m apart with $R_{\text{comm}} = 2$ m and the initial states (1, 3, 10, 14) converge to (2, 2, 12, 12), the two cluster averages, and the disagreement norm settles at 10 instead of decaying to zero: every drone sees its neighbors agreeing with it and is satisfied. The only local symptom is that $\lambda_2 = 0$, which is why Section 23.7.3 monitors it, and why the formation controller of a swarm must keep the graph connected rather than hope that it is.

23.6.2 Discrete time and the step size

Theorem 23.10 (Discrete-time consensus). *Let the graph be undirected and connected and let $0 < h < 2/\lambda_n$. Then the iteration $\mathbf{x}_{m+1} = (\mathbf{I} - h\mathbf{L})\mathbf{x}_m$ of Equation (23.8), whose time step is written m here because k indexes the modes in the proof, conserves the average and converges to it geometrically,*

$$\|\delta_m\| \leq \rho^m \|\delta_0\|, \quad \rho = \max(|1 - h\lambda_2|, |1 - h\lambda_n|) < 1. \quad (23.17)$$

In particular the local condition $h < 1/d_{\max}$ of Equation (23.9) suffices, because $\lambda_n \leq 2d_{\max}$.

Proof sketch. $\mathbf{P} = \mathbf{I} - h\mathbf{L}$ has the eigenvectors of $\mathbf{L}$ with the eigenvalues $\mu_k = 1 - h\lambda_k$; $\mu_1 = 1$ keeps the average, and expanding the initial disagreement in the modes, $\delta_0 = \sum_{k \geq 2} c_k \mathbf{v}_k$, gives $\delta_m = \mathbf{P}^m \delta_0 = \sum_{k \geq 2} c_k \mu_k^m \mathbf{v}_k$ after m steps: mode k is damped by the factor μ_k^m . All the μ_k with $k \geq 2$ lie strictly inside $(-1, 1)$ exactly when $0 < h\lambda_k < 2$ for $k \geq 2$, that is, when $h < 2/\lambda_n$ (the lower end is $\lambda_2 > 0$). The details, including the Gershgorin step that gives $\lambda_n \leq 2d_{\max}$, are Exercise 23.3. $\square$

Remark 23.11 (The best step size). ρ is smallest when $1 - h\lambda_2 = -(1 - h\lambda_n)$, that is, for $h^* = 2/(\lambda_2 + \lambda_n)$ [182]. For the graph of Example 23.7 this is $h^* = 0.4$ with $\rho = 0.6$ per step,

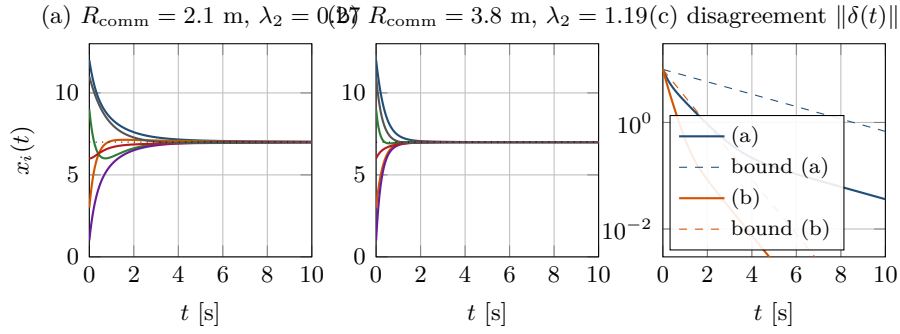


Figure 23.3. Consensus of six drones on two radius graphs with different algebraic connectivity. (a) With $R_{\text{comm}} = 2.1$ m the graph is a path ($\lambda_2 = 0.27$): the end drones are slow and the states creep toward the average 7 (dotted). (b) With $R_{\text{comm}} = 3.8$ m the graph has nine edges ($\lambda_2 = 1.19$) and agreement is reached four times faster. (c) The disagreement norms on a logarithmic scale with the bounds $e^{-\lambda_2 t} \|\delta(0)\|$ (dashed): the bound is tight in slope, and after the faster modes have died out the curves run parallel to it.

while the local bound $h = 1/3$ gives $\rho = 2/3$. The optimum needs the spectrum of the whole graph, so in a swarm one settles for a safe margin below $1/d_{\text{max}}$, or for the weighted averages of Xiao and Boyd [182].

23.6.3 Directed graphs, switching topologies and delays

On a directed graph, drone i still runs Equation (23.6) over the drones it hears, and $\mathbf{L} = \mathbf{D} - \mathbf{W}$ with $d_i = \sum_j w_{ij}$ still satisfies $\mathbf{L}\mathbf{1} = \mathbf{0}$, but $\mathbf{L}$ is no longer symmetric and the sum of the states is no longer conserved. Ren and Beard [135] proved the decisive result: the protocol reaches consensus for every initial state if and only if the digraph contains a **directed spanning tree**, a root drone from which every other drone can be reached by following directed edges. Only the root's information can propagate everywhere; two drones that hear nobody can never agree. The agreed value is $\mathbf{w}^T \mathbf{x}(0)$, where $\mathbf{w} \geq \mathbf{0}$ is the left eigenvector of $\mathbf{L}$ for the eigenvalue 0 scaled to $\mathbf{w}^T \mathbf{1} = 1$: a weighted average that gives weight only to the drones that can reach all the others. If the digraph is **balanced**, with in-degree equal to out-degree at every drone, then $\mathbf{w} = \mathbf{1}/n$ and the average is recovered [124]. For **switching topologies**, when the graph changes as the drones move or as packets are lost, consensus is still reached if the union of the graphs over each of a sequence of bounded, consecutive time intervals contains a spanning tree [135]; for undirected graphs this is the condition that the union over bounded intervals is connected, first proved by Jadbabaie, Lin, and Morse [70] for the alignment rule of Section 23.7.2. Finally, if every message arrives with the same delay τ , Olfati-Saber and Murray [124] showed that the protocol still converges to the average if and only if $\tau < \pi/(2\lambda_n)$: a dense graph (large λ_n) tolerates less delay, because it reacts faster to information that is already stale.

23.6.4 Leader–follower tracking and formation convergence

Proposition 23.12 (Leader–follower consensus). *Let the graph be undirected and connected, let at least one drone be pinned and $k_l > 0$. Then $\mathbf{M} = \mathbf{L} + k_l \mathbf{B}$ is positive definite, and under Equation (23.10) every state converges to the reference: the tracking error obeys $\|\mathbf{e}(t)\| \leq e^{-\mu_1 t} \|\mathbf{e}(0)\|$, where $\mu_1 > 0$ is the smallest eigenvalue of $\mathbf{M}$. Without the feed-forward term the followers converge to a lagged reference, $\mathbf{e}(t) \rightarrow -(\mathbf{M}^{-1} \mathbf{1}) \otimes \dot{\mathbf{r}}$ for a constant $\dot{\mathbf{r}}$.*

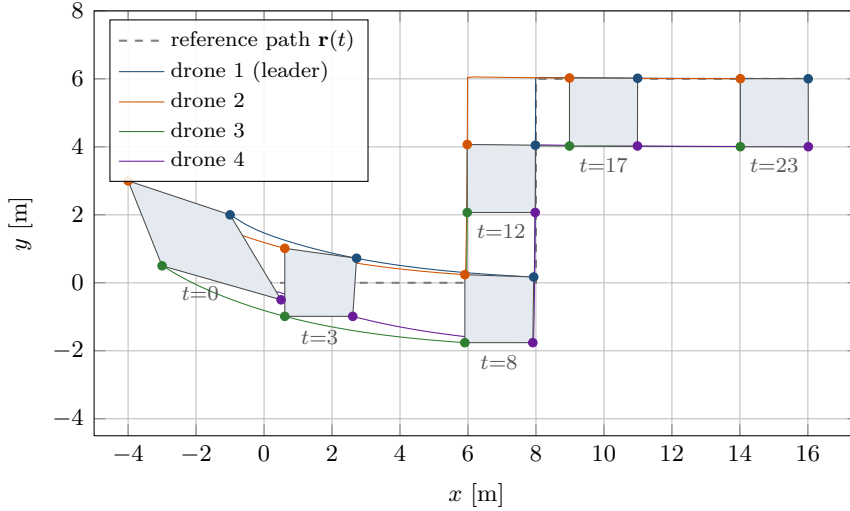


Figure 23.4. Four drones forming a square of side 2 m behind drone 1, which is pinned to a reference point (dashed) that moves along three waypoints at 1 m/s; $R_{\text{comm}} = 3.5$ m, $k_c = 1$, $k_l = 1.5$, $v_{\text{max}} = 3$ m/s. The polygons are snapshots of the formation. The formation error falls from 1.72 m to 0.12 m after 5 s and to 0.0025 m after 20 s, and the switching radius graph stays connected throughout ($\lambda_2 \geq 1$). Without the feed-forward velocity $\dot{\mathbf{r}}$ the same run settles at a formation error of 1.00 m: the followers trail the leader, and the leader trails the reference.

Proof. $\mathbf{M}$ is symmetric and $\mathbf{x}^\top \mathbf{M} \mathbf{x} = \sum_{\{i,j\} \in E} w_{ij} (x_i - x_j)^2 + k_l \sum_{i: b_i=1} x_i^2 \geq 0$. If the form is zero, the first sum forces $\mathbf{x}$ to be constant on the connected graph and the second forces that constant to be zero at a pinned drone, so $\mathbf{x} = \mathbf{0}$: $\mathbf{M}$ is positive definite. The error dynamics Equation (23.11) then give $\frac{d}{dt} \|\mathbf{e}\|^2 = -2\mathbf{e}^\top \mathbf{M} \mathbf{e} \leq -2\mu_1 \|\mathbf{e}\|^2$, and Grönwall's inequality yields the bound. Without feed-forward, $\dot{\mathbf{e}} = -\mathbf{M} \mathbf{e} - \mathbf{1} \otimes \dot{\mathbf{r}}$ has the stable equilibrium $-(\mathbf{M}^{-1} \mathbf{1}) \otimes \dot{\mathbf{r}}$, since $\mathbf{M}$ is invertible. $\square$

For the graph of Example 23.7 with drone 1 pinned and $k_l = 1$, $\mu_1 = 0.18$, so a pinned formation tracks a step in the reference with the time constant 5.6 s, slower than the 1 s of consensus among the drones: the reference has to make its way through the graph. Without feed-forward at 1 m/s the lags are 4 m for the leader and up to 6.67 m for the tail drone, and a formation error of 1.19 m remains for ever (Exercise 23.7). Figure 23.4 shows a formation forming behind a leader that follows a waypoint path with a switching radius graph, with and without feed-forward.

Theorem 23.13 (Formation convergence up to translation). *Let the communication graph be undirected and connected with unit weights, and let the displacements come from offsets, $\mathbf{d}_{ij} = \mathbf{o}_j - \mathbf{o}_i$. Under Equation (23.12) the positions converge to the formation translated by the initial centroid shift,*

$$\mathbf{p}_i(t) \rightarrow \mathbf{o}_i + \bar{\mathbf{y}}, \quad \bar{\mathbf{y}} = \frac{1}{n} \sum_{j=1}^n (\mathbf{p}_j(0) - \mathbf{o}_j), \quad (23.18)$$

the centered formation error satisfies $e_c(t) \leq e^{-\lambda_2 t} e_c(0)$, and when the formation graph is the communication graph the edge error is squeezed between the two,

$$\sqrt{\frac{n\lambda_2}{|E|}} e_c(t) \leq e_F(t) \leq \sqrt{\frac{n\lambda_n}{|E|}} e_c(t), \quad (23.19)$$

so $e_F(t) \rightarrow 0$ at the rate λ_2 .

Proof. By Equation (23.13) the shifted variables $\mathbf{y}_i = \mathbf{p}_i - \mathbf{o}_i$ obey $\dot{\mathbf{y}} = -\mathbf{L}\mathbf{y}$ in every coordinate, and Theorem 23.8 applied to each coordinate gives $\mathbf{y}_i(t) \rightarrow \bar{\mathbf{y}}$, which is Equation (23.18). The disagreement of coordinate c is $\delta^{(c)} = \mathbf{y}^{(c)} - \bar{y}^{(c)}\mathbf{1}$ and $n e_c^2 = \sum_c \|\delta^{(c)}\|^2$, so Equation (23.15) in every coordinate gives $e_c(t) \leq e^{-\lambda_2 t} e_c(0)$. For the sandwich, Proposition 23.3(ii) gives $\sum_{\{i,j\} \in E} (y_i^{(c)} - y_j^{(c)})^2 = \delta^{(c)\top} \mathbf{L} \delta^{(c)}$, which lies between $\lambda_2 \|\delta^{(c)}\|^2$ and $\lambda_n \|\delta^{(c)}\|^2$ because $\delta^{(c)} \perp \mathbf{1}$; summing over the coordinates and dividing by $|E|$ gives $\lambda_2 n e_c^2 \leq |E| e_F^2 \leq \lambda_n n e_c^2$. $\square$

In Example 23.7, $n = |E| = 4$, so $e_c \leq e_F \leq 2e_c$; the table shows $e_F \approx e_c$ from $t = 1$ s on, because by then only the λ_2 mode is left and the lower bound is attained. The theorem also answers the question where the formation ends up: exactly where the initial positions put the template's centroid. A pinned leader overrides this, and Proposition 23.12 applied to $\mathbf{y}$ shows that the formation then converges to $\mathbf{o}_i + \mathbf{r}$.

Proposition 23.14 (Second-order formation control). *Let the graph be undirected and connected and let the drones be double integrators under Equation (23.14) without pinning, with a constant reference velocity $\dot{\mathbf{r}}$. In the eigenbasis of $\mathbf{L}$ the shifted dynamics decouple into the disagreement modes*

$$\ddot{q}_k + (k_v \lambda_k + k_d) \dot{q}_k + k_p \lambda_k q_k = 0, \quad k = 2, \dots, n. \quad (23.20)$$

The centroid mode $k = 1$ ($\lambda_1 = 0$, $\mathbf{v}_1 = \mathbf{1}/\sqrt{n}$) carries the whole forcing and obeys $\ddot{q}_1 + k_d \dot{q}_1 = k_d \sqrt{n} \dot{r}$ instead, where $\dot{r}$ is the component of $\dot{\mathbf{r}}$ in the coordinate considered, so the centroid velocity converges to $\dot{r}$ when $k_d > 0$ and is conserved when $k_d = 0$. Every mode with $\lambda_k > 0$ decays if and only if $k_p > 0$ and $k_v \lambda_k + k_d > 0$, so the formation is reached for any $k_p > 0$ with $k_v > 0$ or $k_d > 0$; with $k_v = k_d = 0$ every mode with $\lambda_k > 0$ oscillates forever at the angular frequency $\sqrt{k_p \lambda_k}$, while the centroid keeps whatever velocity it started with.

Proof sketch. $\ddot{\mathbf{y}} = -k_p \mathbf{L}\mathbf{y} - (k_v \mathbf{L} + k_d \mathbf{I})\dot{\mathbf{y}} + k_d \mathbf{1} \otimes \dot{\mathbf{r}}$ is diagonalized by the eigenvectors of $\mathbf{L}$, which are shared by $\mathbf{I}$; the forcing is a multiple of $\mathbf{1} = \sqrt{n} \mathbf{v}_1$, so it appears in the mode $k = 1$ alone and every mode $k \geq 2$ is homogeneous. Each such mode is a damped oscillator with the characteristic polynomial $s^2 + (k_v \lambda_k + k_d)s + k_p \lambda_k$, which has both roots in the open left half-plane exactly when both coefficients are positive (Exercise 23.7). $\square$

The mode analysis is also the tuning rule. The damping ratio of mode k is $\zeta_k = (k_v \lambda_k + k_d)/(2\sqrt{k_p \lambda_k})$; for the graph of Example 23.7 with $k_p = 1$ the undamped frequencies are 1, $\sqrt{3}$ and 2 rad/s, and $k_v = 1$, $k_d = 0.5$ give $\zeta = 0.75, 1.01$ and 1.13, close to critical for all three. In the simulation of `code/ch23_consensus.py` these gains bring the formation error below 10^{-3} m in 25 s with the drones at rest, while $k_v = k_d = 0$ leaves an error above 0.1 m that never decays.

23.7 Variants and extensions

23.7.1 Distance-based formations and rigidity

The controller of Section 23.4.4 needs every drone to express $\mathbf{p}_j - \mathbf{p}_i$ in a frame whose orientation all drones share: a compass, a GPS heading or the map frame of the global planner. Where no such frame exists, **distance-based formation control** prescribes only the lengths $d_{ij} = \|\mathbf{d}_{ij}\|$ and lets drone i move along the line to each neighbor, $\dot{\mathbf{p}}_i = \sum_j (\|\mathbf{p}_j - \mathbf{p}_i\|^2 - d_{ij}^2)(\mathbf{p}_j - \mathbf{p}_i)$, which every drone can compute in its own body frame [93, 121]. The price is high. Distances define a shape only up to rotation and reflection, and only if the formation graph is **rigid**: a square with four edges can fold into a rhombus, so a diagonal must be added, and in general the graph must be infinitesimally rigid, a condition on the rank of a rigidity matrix [3]. Moreover

Table 23.4. The three families of formation control [121]. The displacement-based family is the one used in this book.

Family	Each drone measures	Frame needed	Shape fixed up to	Graph condition
Position-based	its own absolute position	global frame and heading	nothing (unique)	none (interaction optional)
Displacement-based	relative positions $\mathbf{p}_j - \mathbf{p}_i$	common heading only	translation	connected
Distance-based	distances $\ \mathbf{p}_j - \mathbf{p}_i\ $	none (body frames)	translation, rotation, reflection	rigid

the dynamics are nonlinear, convergence is guaranteed only from nearby configurations, and mirrored formations are equilibria too. Table 23.4 places the three families of Oh, Park, and Ahn [121] side by side. Since every drone in this book shares the frame of the global planner, displacement-based control is the right choice: it is linear, it converges from everywhere on any connected graph, and the CBS reference enters it through one pinned drone.

23.7.2 Flocking

A formation fixes every relative position. A **flock** fixes none of them and still moves as a group: the drones keep a comfortable distance from each other, move in the same direction and follow a common goal. Reynolds [138] produced this behavior in computer graphics with three rules for his *boids*: separation (steer away from crowded neighbors), alignment (steer toward the average heading of the neighbors) and cohesion (steer toward their average position). Olfati-Saber [122] turned the rules into a controller with a proof. Each double-integrator drone applies the acceleration

$$\mathbf{u}_i = \underbrace{\sum_{j \in N_i} \phi(\|\mathbf{p}_j - \mathbf{p}_i\|) \frac{\mathbf{p}_j - \mathbf{p}_i}{\|\mathbf{p}_j - \mathbf{p}_i\|}}_{\text{gradient: separation and cohesion}} + \underbrace{\sum_{j \in N_i} w_{ij}(\mathbf{p}) (\mathbf{v}_j - \mathbf{v}_i)}_{\text{velocity alignment}} + \underbrace{(-c_1(\mathbf{p}_i - \mathbf{p}_\gamma) - c_2(\mathbf{v}_i - \mathbf{v}_\gamma))}_{\text{navigation feedback}}, \quad (23.21)$$

with the three terms of Figure 23.5. The *gradient term* is minus the gradient of a smooth pairwise potential (Figure 23.6 is its relative): the action function $\phi(\ell)$ is negative below the desired distance d (repulsion), positive between d and the interaction range r (attraction) and exactly zero beyond r , where it is faded out by a bump function so that neighbors can enter and leave the range without a jump in the force; Olfati-Saber evaluates the distances through a smoothed norm $\|\mathbf{z}\|_\sigma = (\sqrt{1 + \sigma \|\mathbf{z}\|^2} - 1)/\sigma$ with a small $\sigma > 0$, so that the potential is differentiable everywhere; he calls this parameter ε , which this chapter writes σ . The *alignment term* is consensus on the velocities with the same distance-dependent weights $w_{ij}(\mathbf{p}) \in [0, 1]$, so Theorem 23.8 explains why the velocities become equal. The *navigation feedback* is a PD term toward a virtual leader (the γ -agent) at $\mathbf{p}_\gamma$ with velocity $\mathbf{v}_\gamma$, for us the CBS reference of the group. It is not an afterthought: without it the flock fragments into small groups that drift apart, which is one of the main results of Olfati-Saber [122] and a warning against the pure boids rules for a swarm that must stay together. Obstacles enter through virtual β -agents on their boundary, exactly the mechanism of the repulsive potentials of Chapter 15.

The connections to the rest of the book are direct. The gradient term is an artificial potential field between agents in the sense of Chapter 15 (Khatib's repulsion [83] with a cohesive part added), and it inherits the weaknesses of potential fields: it is soft, it can be overpowered, and it makes no promise about the minimum distance. In the hybrid architecture

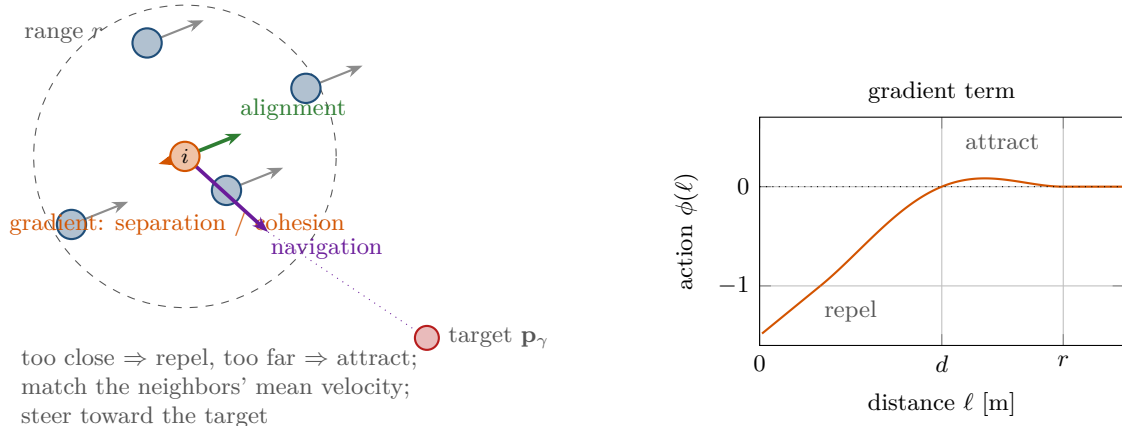


Figure 23.5. Left: the three terms of Equation (23.21) acting on drone i ; the neighbors within the range r produce the gradient term (repel when too close, attract when too far) and the alignment term (match their mean velocity), the target produces the navigation term. Right: the action function of the gradient term, $\phi(\ell) < 0$ below the desired distance d , $\phi(\ell) > 0$ between d and r , and zero beyond r , faded out smoothly by the bump function.

(Chapter 24) the separation is therefore delegated to the local safety layer: ORCA (Chapter 13) takes the velocity computed by the consensus or formation terms as its *preferred velocity* and returns the closest velocity that is collision-free for the ORCA horizon τ of Chapter 13, a hard guarantee that a potential cannot give. Consensus decides where the swarm wants to go, ORCA decides what it may do this instant.

Historical note

Reynolds' boids (1987) came from computer animation and had no proof. Vicsek et al. [170] studied the alignment rule alone as a physics model and observed a phase transition to ordered motion; Jadbabaie, Lin, and Morse [70] gave the first convergence proof for it, using products of stochastic matrices on switching graphs, which is the discrete-time consensus of Theorem 23.10 in disguise. Olfati-Saber and Murray [124] set up the Laplacian framework with λ_2 as the rate and treated delays and switching, Ren and Beard [135] settled the directed case with the spanning-tree condition, Olfati-Saber [122] and Tanner, Jadbabaie, and Pappas [166] proved flocking, and the algebraic connectivity itself goes back to Fiedler [44].

23.7.3 Communication-range constraints and connectivity maintenance

Every theorem above assumes a connected graph, and the radius graph of a moving swarm is connected only as long as the drones keep it so. **Connectivity maintenance** selects a set of edges E_{keep} that spans the graph, typically the edges of the formation graph, and keeps each of them shorter than R_{comm} . There are two ways to do this, and a quantity to watch.

As a hard constraint. The condition $\|\mathbf{p}_i - \mathbf{p}_j\| \leq R_{\text{comm}}$ is convex in the pair of positions (a disk of relative positions), so it can be handed to an optimization-based controller. In the MPC of Chapter 21 it becomes one constraint per kept edge and per step of the horizon, either as a second-order cone or, for a quadratic program, replaced by a polygon inscribed in the disk: $\mathbf{n}_q^\top (\mathbf{p}_i - \mathbf{p}_j) \leq R_{\text{comm}} \cos(\pi/Q)$ for Q unit directions $\mathbf{n}_q$, a slightly conservative inner approximation that stays linear (Exercise 23.6). The MILP formulations of Chapter 22 use the same linear inequalities. The constraint is exact and can be combined with the collision

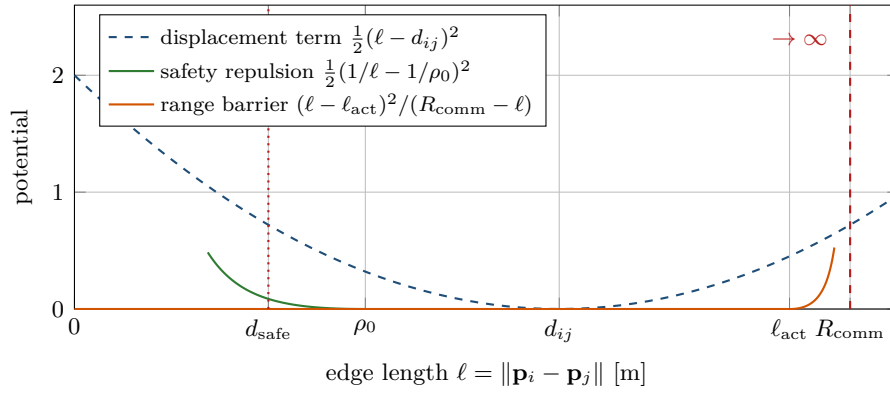


Figure 23.6. The pairwise potentials acting on one edge as functions of its length. The displacement term (dashed) is a quadratic bowl around the desired length; the safety repulsion (green) is Khatib’s potential, active below ρ_0 and infinite at contact; the range-keeping barrier (orange) is zero up to ℓ_{act} and infinite at R_{comm} . Formation control lives in the bowl, and the two barriers keep the edge out of the forbidden ends.

constraints, but it needs a planner that solves a program at every period; for the distributed controller of [Algorithm 23.1](#) one uses the second way.

As a potential. Add to the displacement potential a **range-keeping barrier** on each kept edge that is zero while the edge is comfortably short and grows without bound as the length approaches R_{comm} [[73](#), [186](#)]:

$$V_{\text{range}}(\ell) = \begin{cases} 0 & \ell \leq \ell_{\text{act}}, \\ \frac{(\ell - \ell_{\text{act}})^2}{R_{\text{comm}} - \ell} & \ell_{\text{act}} < \ell < R_{\text{comm}}, \end{cases} \quad (23.22)$$

and let both drones of the edge descend its gradient ([Figure 23.6](#)). Because the barrier is infinite at the range boundary, a finite disturbance cannot stretch the edge beyond it in the continuous-time ideal; in discrete time the barrier must be paired with the velocity limit so that one period cannot jump across the last few centimeters, and the activation distance ℓ_{act} (say $0.9 R_{\text{comm}}$) leaves the formation controller alone until an edge is actually at risk. The safety repulsion of the local safety layer is the same construction at the other end of the scale, infinite at the contact distance. The velocity of every term is added on line 8 of [Algorithm 23.1](#).

What to monitor. The barrier protects the edges of E_{keep} ; λ_2 of the current radius graph tells whether the whole graph is still connected and how robustly. It is the natural **connectivity measure** of a swarm: zero means split, small means one edge away from splitting, and it is also the convergence rate the formation controller currently enjoys. A ground station with all positions computes it with an eigensolver in $\mathcal{O}(n^3)$. On board, a drone can estimate it by power iteration on the matrix $s\mathbf{I} - \mathbf{L} - \frac{s}{n}\mathbf{1}\mathbf{1}^\top$ with $s = 2d_{\text{max}}$ (not the pinning matrix $\mathbf{B}$ of [Equation \(23.10\)](#)), whose largest eigenvalue is $s - \lambda_2$; each iteration needs only products with $\mathbf{L}$ (one message exchange) and the average of the iterate (a consensus), which is the basis of the distributed estimators of Yang et al. [[183](#)]. The function `fiedler_power_iteration()` in `code/ch23_consensus.py` is the centralized version and agrees with NumPy’s eigenvalues to 10^{-6} in the self-test. Together with the lengths of the kept edges, λ_2 is the connectivity constraint that [Chapter 24](#) monitors and [Chapter 25](#) reports.

23.8 Implementation notes

Step size. The gain and the control period enter only as the product $h = k_c \Delta t$, and [Theorem 23.10](#) needs $h < 1/d_{\max}$ with margin. At 20 Hz ($\Delta t = 0.05$ s) with $k_c = 1 \text{ s}^{-1}$ and at most five neighbors, $h = 0.05$ is far below 0.2; the bound bites when the gain is raised for a faster response, or when a slow link forces a long period. The range barrier and the safety repulsion are stiffer than the consensus term near their poles, so the velocity limit $v_{\max}$ on [line 10](#) is what keeps the Euler step from jumping over a barrier.

An unstable step size looks like a bug in the messages

With h above $2/\lambda_n$ the swarm does not merely converge slowly: it diverges, and it does so in the highest-frequency pattern of the graph, so the symptom is neighboring drones lurching in opposite directions at every period (right half of [Table 23.2](#)). Because λ_n depends on the graph, a gain that was fine in a sparse test can fail when the drones cluster and the degrees grow. Also beware of the boundary: a 4-cycle has $\lambda_4 = 4 = 2d_{\max}$, so $h = 1/d_{\max} = 0.5$ exactly gives the eigenvalue -1 and the pattern $(+, -, +, -)$ flips for ever without decaying. Use a strict margin, and $\log \max_i d_i$ together with the control period.

Asynchronous updates and packet loss. Real drones do not update in lockstep. Using the latest position received from each neighbor is a bounded delay and a lost packet is a switching topology, so [Section 23.6.3](#) already covers both. What it does not cover is an *asymmetric* loss: if i hears j but j misses i , the graph is momentarily directed and unbalanced, and the agreed value drifts. A pinned leader corrects that for a formation; a swarm that averages sensor readings needs a protocol that retransmits or that exchanges mass explicitly.

Switching topologies and hysteresis. The radius graph switches whenever a pair crosses R_{comm} , and a pair hovering at exactly that distance would switch at every period. Use hysteresis: add an edge when the pair comes within $0.9 R_{\text{comm}}$ and drop it only when it exceeds R_{comm} . Keep the formation graph G_F separate from the communication graph: the controller uses whichever edges exist, the formation error is always measured on G_F , and the range barrier guards $E_{\text{keep}} \subseteq E_F$.

The code. `code/ch23_consensus.py` implements the chapter: the graph tools of [Listing 23.1](#), the exact and RK4 continuous-time consensus, the discrete protocol, the pinned matrix, the formation controllers of [Listing 23.2](#) (first order) and `formation_acceleration()` (second order), the error metrics of [Listing 23.3](#), the barrier and repulsion velocities, a first-order simulator with a switching radius graph, a second-order simulator, the flocking controller and the worked example. Positions are the rows of an (n, m) array, and the desired displacements are never stored: the functions receive the offsets and compute $\mathbf{d}_{ij} = \mathbf{o}_j - \mathbf{o}_i$ themselves. The self-test checks the Laplacian identities and the power-iteration monitor against NumPy on random radius graphs, the spectrum $\{0, 1, 3, 4\}$ and the numbers of [Example 23.7](#), convergence to the average on a connected graph and to cluster averages on a disconnected one, the step-size bounds on the 4-cycle and on random graphs, leader–follower convergence and its failure when the pinned part is cut off, the formation-error identities and the convergence of the first- and second-order controllers, the range barrier under a stretching disturbance, and velocity alignment without collisions in the flock. It runs in a few seconds.

Listing 23.1. The communication graph from positions and a radius, its Laplacian, its edge list and λ_2 (from `code/ch23_consensus.py`).

```

1 def radius_graph(positions, r_comm):
2     """Return the 0/1 adjacency matrix of the communication graph.
3
4     Drones  $i \neq j$  are neighbors if and only if  $\|p_i - p_j\| \leq r_{\text{comm}}$ .
5     """
6     P = np.asarray(positions, dtype=float)
7     dist = np.linalg.norm(P[:, None, :] - P[None, :, :], axis=2)
8     W = (dist <= r_comm).astype(float)
9     np.fill_diagonal(W, 0.0)
10    return W
11
12
13 def laplacian(W):
14     """Graph Laplacian  $L = D - W$ ; every row of  $L$  sums to zero."""
15     W = np.asarray(W, dtype=float)
16     return np.diag(W.sum(axis=1)) - W
17
18
19 def edges_of(W):
20     """Undirected edges  $(i, j)$  with  $i < j$  and  $w_{ij} > 0$ ."""
21     W = np.asarray(W)
22     n = W.shape[0]
23     return [(i, j) for i in range(n) for j in range(i + 1, n) if W[i, j] > 0]
24
25
26 def algebraic_connectivity(L):
27     """ $\lambda_2(L)$ : the second-smallest eigenvalue, zero iff disconnected."""
28     return float(np.linalg.eigvalsh(np.asarray(L, dtype=float))[1])

```

Listing 23.2. The first-order displacement-based formation controller with leader pinning and feed-forward: [Algorithm 23.1](#) for the whole swarm at once, using $\mathbf{p}_j - \mathbf{p}_i - \mathbf{d}_{ij} = \mathbf{y}_j - \mathbf{y}_i$ (from `code/ch23_consensus.py`).

```

1 def formation_velocity(P, offsets, W, gain=1.0, ref=None, ref_vel=None,
2                       pinned=(), gain_ref=1.0):
3     """First-order displacement-based formation controller (velocities).
4
5      $v_i = \text{gain} * \sum_j w_{ij} (p_j - p_i - d_{ij})$ 
6         +  $b_i \text{gain\_ref} (r + o_i - p_i)$       pinning of the leader(s)
7         +  $\text{rdot}$                                 feed-forward if known
8     Because  $p_j - p_i - d_{ij} = y_j - y_i$  with  $y = p - o$ , the sum is  $-L y$ .
9     """
10    P = np.asarray(P, dtype=float)
11    O = np.asarray(offsets, dtype=float)
12    V = -gain * (laplacian(W) @ (P - O))
13    if ref is not None:
14        r = np.asarray(ref, dtype=float)
15        for i in pinned:
16            V[i] += gain_ref * (r + O[i] - P[i])
17    if ref_vel is not None:
18        V += np.asarray(ref_vel, dtype=float)
19    return V

```

Listing 23.3. The formation error of [Definition 23.6](#) over the edges of the formation graph, and its centered variant (from `code/ch23_consensus.py`).

```

1 def formation_error(P, offsets, W_form):

```

```

2      """RMS of  $\| (p_j - p_i) - d_{ij} \|$  over the edges of the formation graph."""
3      Y = np.asarray(P, dtype=float) - np.asarray(offsets, dtype=float)
4      edges = edges_of(W_form)
5      if not edges:
6          return float("nan")
7      sq = [float(np.sum((Y[j] - Y[i]) ** 2)) for i, j in edges]
8      return float(np.sqrt(np.mean(sq)))
9
10
11 def formation_error_centered(P, offsets):
12     """RMS deviation from the template after removing the best translation."""
13     Y = np.asarray(P, dtype=float) - np.asarray(offsets, dtype=float)
14     Y = Y - Y.mean(axis=0)
15     return float(np.sqrt(np.mean(np.sum(Y ** 2, axis=1))))

```

23.9 Where this fits in the drone system

In the drone system

Consensus is the **low-level formation controller** of the hybrid architecture of [Chapter 24](#), and the communication radius is the last of its constraints. It sits between two layers and talks to both.

From the global planner. CBS or ECBS ([Chapters 9 and 10](#)) plan one path for the formation as a whole, which MPC ([Chapter 21](#)) may smooth; sampled at the current time it is the reference $\mathbf{r}(t)$, $\dot{\mathbf{r}}(t)$ of [Algorithm 23.1](#). One drone is pinned to $\mathbf{r}$ and $\dot{\mathbf{r}}$ is forwarded through the graph to the others, which need nothing else but their neighbors' positions and the offsets $\mathbf{o}_i$ — the shape of the mission (a line for a survey, a square for a camera rig), with the graph connected and λ_2 as large as the radio range allows.

To and from the local safety layer. The velocity of [Algorithm 23.1](#) is the *preferred velocity* handed to ORCA ([Chapter 13](#)) or DWA ([Chapter 14](#)) for the unexpected obstacles that the prediction layer ([Chapters 18 to 20](#)) reports. The safety layer may change it, and when it does the formation breaks temporarily. That is intended: separation from an intruder has priority over the shape. The consensus term keeps acting on the drones that were not pushed, so they move to accommodate the one that was, and it pulls the formation back together as soon as the intruder has passed. [Figure 23.7](#) quantifies this on the square of [Example 23.7](#).

Monitored constraints. Two quantities are logged at every period and reported by the evaluation of [Chapter 25](#): the formation error e_F of [Definition 23.6](#) and the connectivity, as λ_2 of the current radius graph together with the lengths of the kept edges and the count of communication violations. The range barrier of [Section 23.7.3](#) acts on the same edges, and a drop of λ_2 toward zero, or a kept edge beyond ℓ_{act} , is a trigger for the replanning layer (D^* Lite, A^* ; [Chapters 4 and 5](#)) to slow the reference down or to route the formation around the obstacle as a group. This is the Week 11 material of the study plan ([Appendix A](#)); its coding exercise is [Exercise 23.8](#).

Avoidance breaks the formation, and must be allowed to

A tempting fix for a high formation error during an avoidance maneuver is to raise the formation gain until the shape holds. This turns the formation controller into an adversary of the local safety layer: the consensus term pulls the avoiding drone back toward the

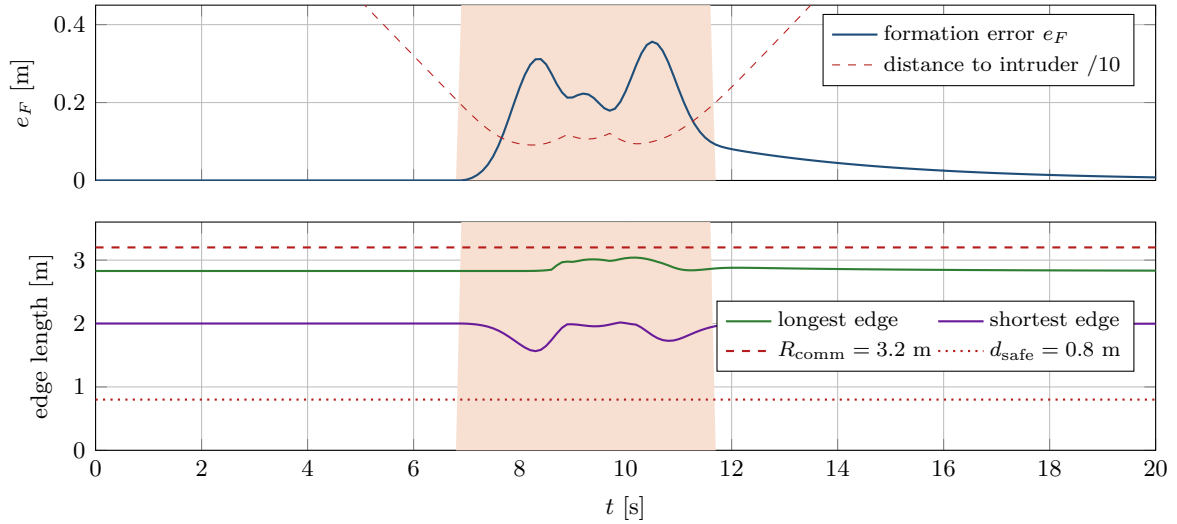


Figure 23.7. The square formation of Example 23.7 following a straight reference at 1 m/s while an intruder crosses its path from above at 1 m/s; $R_{\text{comm}} = 3.2$ m, $\ell_{\text{act}} = 2.95$ m, safety distance $d_{\text{safe}} = 0.8$ m, repulsion within 2 m. The shaded band marks the interval $[6.9, 11.7]$ s during which the repulsion is active. Top: the formation error rises to a peak of 0.36 m at $t = 10.5$ s and is back below 0.05 m at $t = 13.6$ s; the distance to the intruder is drawn divided by ten so that both curves fit one axis, and its smallest value is a closest approach of 0.91 m. Bottom: the longest edge reaches 3.04 m, held below R_{comm} by the range barrier, and the shortest edge stays at 1.56 m; the graph never loses an edge ($\lambda_2 = 4$ throughout) and there are no communication violations.

intruder, the safety term pushes it out again, and the drone oscillates at the edge of the safety distance. Treat the formation error during avoidance as a cost to be reported (Chapter 25), not a constraint to be enforced, keep the gains such that the safety term dominates near contact, and judge the controller by the recovery time afterward, 3 s in Figure 23.7. What must be enforced is the other constraint: the range barrier, which keeps the stretched edges below R_{comm} so that the swarm stays connected while it is out of shape.

23.10 Summary

Chapter summary

- The communication graph of a swarm is the radius graph of the drone positions; its Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{W}$ is symmetric, positive semidefinite, has $\mathbf{L}\mathbf{1} = \mathbf{0}$, and its second eigenvalue λ_2 is positive exactly when the graph is connected.
- The consensus protocol $\dot{\mathbf{x}} = -\mathbf{L}\mathbf{x}$ is local, leaderless and conserves the sum. On a connected undirected graph it converges to the average of the initial states, and the disagreement decays like $e^{-\lambda_2 t}$ (Theorem 23.8).
- In discrete time, $\mathbf{x}_{k+1} = (\mathbf{I} - h\mathbf{L})\mathbf{x}_k$ converges for $h < 2/\lambda_n$, and $h < 1/d_{\text{max}}$ is the sufficient condition every drone can check (Theorem 23.10).
- On a directed graph consensus needs a directed spanning tree and reaches a weighted average; on switching graphs it needs the union over bounded windows to be connected.

- Pinning one drone to a reference makes the whole graph follow it; the feed-forward $\dot{\mathbf{r}}$ removes the lag.
- Displacement-based formation control is consensus on the shifted positions $\mathbf{p}_i - \mathbf{o}_i$; it converges to the formation up to a translation ([Theorem 23.13](#)). The formation error e_F is the RMS violation of the desired displacements over the formation graph.
- Double-integrator drones need velocity consensus or absolute damping, otherwise the formation oscillates for ever.
- Flocking adds a gradient term for separation and cohesion, velocity alignment and navigation feedback; in the drone system the separation is delegated to ORCA.
- Connectivity is kept by a hard constraint in MPC or by a barrier potential at R_{comm} , and monitored through λ_2 and the edge lengths; avoidance breaks the formation temporarily, and the controller is judged by its recovery.

Further reading. The history note above traces the original papers; this paragraph adds what it does not name. The survey of Olfati-Saber, Fax, and Murray [123] is the standard entry point and covers everything in [Section 23.6](#) with the Perron-matrix view of discrete time. The book of Ren and Beard [136] and the tutorial of Ren, Beard, and Atkins [137] treat leader-follower and second-order consensus and formation control for vehicles; Mesbahi and Egerstedt [118] and Bullo, Cortés, and Martínez [26] are the graph-theoretic textbooks. Oh, Park, and Ahn [121] organizes formation control into the three families of [Table 23.4](#), with Anderson et al. [3] and Krick, Broucke, and Francis [93] for the rigidity theory of the distance-based family. Connectivity maintenance is Ji and Egerstedt [73] and Zavlanos, Egerstedt, and Pappas [186], the distributed estimation of λ_2 is Yang et al. [183], and the optimal step size is Xiao and Boyd [182].

23.11 Exercises

Exercise 23.1 (★). Four drones communicate over the edges $\{1, 2\}$, $\{2, 3\}$, $\{3, 4\}$, $\{4, 1\}$ and $\{1, 3\}$ (a square with one diagonal). Write down $\mathbf{W}$, $\mathbf{D}$ and $\mathbf{L}$, verify $\mathbf{L}\mathbf{1} = \mathbf{0}$, and compute $\mathbf{x}^T \mathbf{L} \mathbf{x}$ for $\mathbf{x} = (1, 0, 0, 1)^T$ both from the matrix and from the sum over the edges. Find the four eigenvalues (guess eigenvectors orthogonal to $\mathbf{1}$ and check the trace), and give λ_2 , the time constant of consensus, and the two step-size bounds $1/d_{\max}$ and $2/\lambda_4$.

Exercise 23.2 (★). (a) Show that the discrete protocol [Equation \(23.8\)](#) conserves the average for every h , and that the maximum state never grows if $h \leq 1/d_{\max}$. (b) Four drones sit on a line at $x = 0, 1, 10$ and 11 m, so that $R_{\text{comm}} = 2$ m splits them into the two components that never agree with each other. How large must R_{comm} be before they agree globally? Give λ_2 and the time constant $1/\lambda_2$ at that radius and at $R_{\text{comm}} = 10$ m, and compare them with the $\lambda_2 = 1$ of [Example 23.7](#). (c) Prove the identity $\sum_{i < j} \|\mathbf{y}_i - \mathbf{y}_j\|^2 = n \sum_i \|\mathbf{y}_i - \bar{\mathbf{y}}\|^2$ and deduce $e_F^2 = \frac{2n}{n-1} e_c^2$ for the complete formation graph; check it on $e_F(0) = 2.5$ m and $e_c(0) = 1.794$ m of [Example 23.7](#), remembering that the example measures e_F on four edges, not six.

Exercise 23.3 (★★). Prove [Theorem 23.10](#) in full. Show that $\mathbf{P} = \mathbf{I} - h\mathbf{L}$ has the eigenvalues $1 - h\lambda_k$ with the eigenvectors of $\mathbf{L}$, that the average is conserved, that the disagreement vector stays orthogonal to $\mathbf{1}$ and contracts by the factor ρ of [Equation \(23.17\)](#) at every step, and that $\rho < 1$ when the graph is connected and $h < 2/\lambda_n$. Then prove $\lambda_n \leq 2d_{\max}$ with Gershgorin's theorem and conclude that $h < 1/d_{\max}$ suffices. Where exactly does connectedness enter?

Exercise 23.4 (★★). Three drones communicate over one-way links: drone 2 hears drone 1 and drone 3 hears drone 2, nobody hears anybody else. (a) Write $\mathbf{L}$ for the convention that $w_{ij} = 1$ when i hears j , check that it has a directed spanning tree, solve $\dot{\mathbf{x}} = -\mathbf{L}\mathbf{x}$ and show that all states converge to $x_1(0)$; identify the left eigenvector $\mathbf{w}$. (b) Add the link by which drone 1 hears drone 3; show that the digraph is now balanced and that the states converge to the average. (c) Replace the links by “drone 2 hears drones 1 and 3” only; show that there is no spanning tree, compute the limit of each state, and explain in words why no protocol can reach consensus on this graph.

Exercise 23.5 (★★). (a) Sum Equation (23.12) over all drones and show that the centroid moves with the constant velocity $-\frac{1}{n} \sum_i \sum_{j \in N_i} \mathbf{d}_{ij}$; conclude that it is stationary whenever $\mathbf{d}_{ji} = -\mathbf{d}_{ij}$ on every edge, and give an example that is antisymmetric on no edge at all and still leaves the centroid fixed. (b) With antisymmetric displacements, show that the controller is gradient descent on $J(\mathbf{p}) = \frac{1}{2} \sum_{\{i,j\} \in E} \|\mathbf{p}_j - \mathbf{p}_i - \mathbf{d}_{ij}\|^2$ and that J decreases monotonically. For a triangle with $\mathbf{d}_{12} = \mathbf{d}_{23} = (1, 0)^\top$ and $\mathbf{d}_{31} = (-1, 0)^\top$ (the vectors do not close), find the configuration the swarm settles in and its residual formation error. (c) Explain why computing $\mathbf{d}_{ij}$ from offsets makes both problems impossible.

Exercise 23.6 (★★). (a) Show that the set of pairs $(\mathbf{p}_i, \mathbf{p}_j)$ with $\|\mathbf{p}_i - \mathbf{p}_j\| \leq R_{\text{comm}}$ is convex and write the constraint for one kept edge and one step of the MPC horizon of Chapter 21. (b) Show that the Q linear inequalities $\mathbf{n}_q^\top (\mathbf{p}_i - \mathbf{p}_j) \leq R_{\text{comm}} \cos(\pi/Q)$, with $\mathbf{n}_q$ equally spaced unit vectors, describe a polygon inside the disk, so that a plan that satisfies them keeps the edge in range; how much range is given away for $Q = 8$? (c) Compute the derivative of the barrier Equation (23.22) and check that the resulting velocity is continuous at ℓ_{act} . (d) Explain why λ_2 alone cannot tell which edge is about to break, and what to log instead.

Exercise 23.7 (★★★). (a) Derive the modal equation Equation (23.20) from Equation (23.14) without pinning, prove the stability condition of Proposition 23.14, give the oscillation frequencies for the graph of Example 23.7 with $k_p = 1$ and $k_v = k_d = 0$, and find the k_v that makes the slowest mode critically damped for $k_d = 0$. (b) For the first-order leader–follower controller without feed-forward and a reference moving at the constant velocity $v\mathbf{e}$, derive the steady-state error $-v((k_c\mathbf{L} + k_l\mathbf{B})^{-1}\mathbf{1}) \otimes \mathbf{e}$ of Proposition 23.12. Evaluate it for the graph of Example 23.7 with drone 1 pinned, $k_c = k_l = 1$ and $v = 1$ m/s: how far does each drone trail its slot, and what formation error remains?

Exercise 23.8 (★★★ Coding). This is the Week 11 coding exercise of the study plan (Appendix A): simulate formation control with a communication radius and measure the formation error during an avoidance maneuver. Work in the API of `code/ch23_consensus.py` but write the functions yourself. (a) Implement the radius graph, the Laplacian and λ_2 , and reproduce the spectrum $\{0, 1, 3, 4\}$ of Example 23.7. (b) Implement Algorithm 23.1 for the whole swarm and reproduce Table 23.3 and the final square; then pin drone 1 to the waypoint path of Figure 23.4 with and without feed-forward and compare the formation errors after 20 s. (c) Send the square of Figure 23.7 along a straight reference at 1 m/s with $R_{\text{comm}} = 3.2$ m and let an intruder cross its path; add the safety repulsion of Chapter 15 (or ORCA from Chapter 13 if you have it) and the range barrier Equation (23.22); log at every period the formation error, the shortest and longest edge, the distance to the intruder, λ_2 and the number of communication violations, and plot them. Reproduce the peak error of 0.36 m, the recovery below 0.05 m about 3 s later, and a longest edge below R_{comm} . (d) Reduce R_{comm} to 2.5 m so that the diagonals of the square (2.83 m) are out of range and the graph switches between the 4-cycle and the complete graph during the maneuver; report what happens to λ_2 , to the recovery time and to the violations, and then remove the range barrier and show a run in which the formation splits. (e) Optional: replace the formation term by the flocking controller Equation (23.21) and check that the velocities align and that the group stays together only with the navigation term switched on.

Part VIII

Putting It Together

The Hybrid Collision-Avoidance Architecture

Learning objectives

- Draw the four-layer architecture as a block diagram with its data flows and rates, and state the inputs, outputs and guarantees of every layer.
- Describe the world model that the layers share, and convert a time-indexed grid path into a velocity reference and a continuous state back into a space-time start.
- Run the decision logic by hand: the state machine, the safety-horizon trigger built from a predicted trajectory and its uncertainty, the reconnection search, the replanning trigger and the conflict re-check.
- Say precisely which guarantees of CBS, ORCA and space-time A* survive the integration, which do not, and which experiment of [Chapter 25](#) measures the gap.
- Build the capstone prototype of the study plan ([Appendix A](#), Week 12) from the modules of the earlier chapters, and recognize the five pitfalls that break it.

24.1 Why this matters for a drone swarm

Every earlier chapter solved one problem well. CBS and ECBS ([Chapters 9 and 10](#)) compute conflict-free, time-indexed routes for the drones you control, but only for the world they were given before take-off. The Kalman filter and the learned predictors ([Chapters 18 and 20](#)) tell you where an unknown drone will be in the next few seconds, with an honest covariance, but they do not decide anything. ORCA and DWA ([Chapters 13 and 14](#)) choose a safe velocity for the next tenth of a second, but they have no memory, no plan and no notion of a swarm. D* Lite and A* ([Chapters 4 and 5](#)) repair a route on a changed map, but a route for one drone is worthless if it crosses the routes of the others. The consensus and MPC controllers ([Chapters 21 and 23](#)) keep a formation and a radio link, as long as nobody has to swerve.

The picture that motivates this chapter is the one of [Chapter 1](#): three delivery drones fly a planned mission through a district, and a fourth drone that nobody planned for crosses their lanes. The nominal plan must not be thrown away at the first sign of trouble: it is the only thing that keeps the three from colliding with *each other*. It must not be followed blindly either, because it knows nothing about the intruder. The **hybrid architecture** of the training plan ([Appendix A](#)) resolves this with one rule: *a global planner for the routes, a local method for the surprises, a replanner only when the local response is insufficient*. This chapter turns that sentence into an executable design: a block diagram with rates, a shared world model, a state machine and four precise decision rules with pseudocode. The design is then flown in a fully worked scenario, run by the book's own code. The chapter closes with an honest account of what the composed system does and does not guarantee.

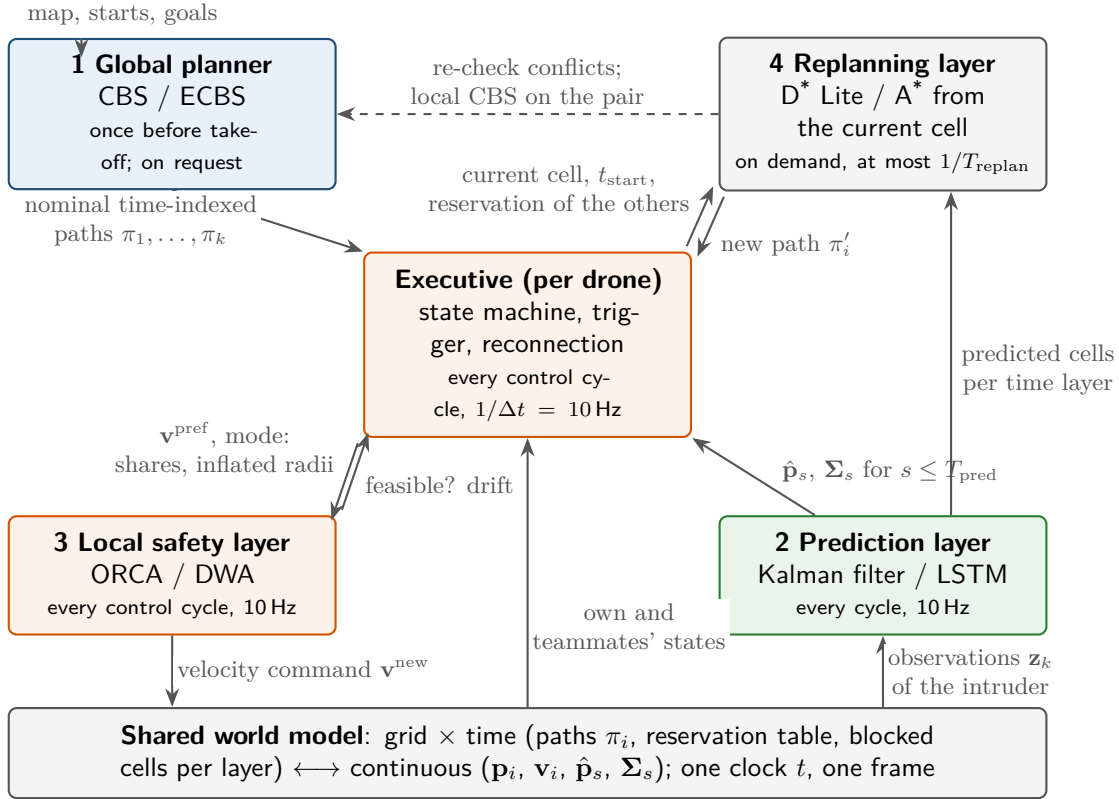


Figure 24.1. The four layers, the executive and the shared world model, with the data flows and the rate of each block. What to notice: the global planner and the replanner work on the grid-and-time side of the world model, the local layer and the predictor on the continuous side, and the executive is the only block that converts between the two. The dashed arrow is the conflict re-check that closes the loop.

The architecture, the world model and the four decision rules come first. They are then flown, with the book's own code, in the worked scenario of [Section 24.10](#) and audited in [Section 24.11](#). The exercises end with the capstone build of Week 12.

24.2 The idea in plain words

Think of the four layers as four people with different time budgets ([Figure 24.1](#) and [Table 24.1](#)). The *global planner* has minutes before take-off, and it writes the contract that the drones fly by: one time-indexed path π_i per drone. The *prediction layer* watches the sensors ten times a second. The *local safety layer*, also ten times a second, returns the safe velocity closest to the one it is asked for. The *replanning layer* runs on demand only, when a drone cannot get back to its route safely.

The glue between them is the **executive**: a small state machine that runs in every drone at the control rate and decides which layer is in charge. While no predicted conflict lies inside a short *safety horizon*, it follows the nominal path and the local layer is idle. When the prediction says that the intruder will come too close within the horizon, the local layer takes the velocity command and the nominal path is kept in memory. When the danger has passed, the executive tries to *reconnect* at a waypoint a little further along the path. Only if that fails does it call the replanner. After any replan it re-checks the new route against the routes of the other drones: a route that is safe against the intruder may now cross a teammate.

Table 24.1. Responsibilities and interfaces of the layers. Each layer is a module with the inputs and outputs listed; the rates are those of the worked scenario of [Section 24.10](#).

Layer (chapter)	Inputs	Outputs	Guarantee	Rate
1 Global planner: CBS, ECBS (Chapters 9 and 10)	inflated grid, starts, goals, swarm constraints	one time-indexed path π_i per drone; SoC, MS	conflict-free; optimal or w -suboptimal (Propositions 24.5 and 24.6)	once before take-off; again on request
2 Prediction layer: KF, EKF/UKF/PF, LSTM (Chapters 18 to 20)	noisy observations $\mathbf{z}_k$ of every unknown object	estimate $\hat{\mathbf{x}}_k$; means $\hat{\mathbf{p}}_{T+h}$ and covariances Σ_h for $h\Delta t \leq T_{\text{pred}}$	calibrated ellipse per step, if the model holds	every control cycle, 10 Hz
3 Local safety layer: ORCA, DWA (Chapters 13 and 14)	own and teammates' states, predicted intruder disk, $\mathbf{v}^{\text{pref}}$	velocity command $\mathbf{v}^{\text{new}}$; feasibility flag	pairwise safety for τ (Proposition 24.7)	every control cycle, 10 Hz
4 Replanning layer: D* Lite, A* (Chapters 4 and 5)	current cell and t_{start} , reservation of the others, blocked cells per layer	new path π'_i from the current cell, or failure	conflict-free against the reserved paths (Proposition 24.8)	on demand, at most one per T_{replan}
Executive (this chapter)	all of the above, formation offsets, R_{comm}	state, $\mathbf{v}^{\text{pref}}$, triggers, re-check	the composition of Section 24.11	every control cycle, 10 Hz

Key idea

Keep the global plan as the contract between the drones and treat every deviation from it as temporary. A deviation is handled by the cheapest layer that can handle it. The order of escalation is: a velocity change for a few seconds; a reconnection to the plan; a new plan for one drone; and only then a new plan for several. Each escalation is triggered by a condition you can write down, and each returns the drone to the contract or replaces the contract with one that has been checked again.

[Table 24.1](#) is the architecture as a set of interfaces. The rates are those of the code of this chapter and of a typical small quadrotor. The control cycle is $\Delta t = 0.1$ s, the prediction horizon a few seconds, and the grid time step one second, the time a drone needs to cross one cell at cruise speed.

24.3 The world model shared by the layers

The global and replanning layers live on a grid with discrete time; the local and prediction layers live in continuous space and time. The two descriptions must refer to the same world, the same clock and the same frame, and the executive must convert between them in both directions.

Definition 24.1 (Hybrid world model). The **hybrid world model** consists of

1. a grid $G = (V, E)$ of square cells of side ℓ ([Chapter 2](#)), inflated by the drone radius, with a time step $\Delta T = \ell/v_{\text{cruise}}$; the cell (x, y) has its center at $\mathbf{c}(x, y) = ((x + \frac{1}{2})\ell, (y + \frac{1}{2})\ell)$;
2. for every drone a_i a **time-indexed path** π_i with a **start time** $t_0^i \in \mathbb{N}$: $\pi_i[j]$ is the cell

occupied at grid time $t_0^i + j$, and the drone parks at $\pi_i[T_i]$ from $t_0^i + T_i$ on (stay-at-target, Chapter 7, in the conventions of Stern et al. [161]);

3. the **reservation table** R of the other drones, holding the pairs (cell, grid time), the reversed moves and the parked goals of their paths in absolute time, exactly as in Chapters 4 and 8;
4. the **blocked layers** $\mathcal{B}_t \subseteq V$: the cells that the predicted intruder may occupy at grid time t ;
5. the continuous states: own position $\mathbf{p}_i(t)$ and velocity $\mathbf{v}_i(t)$, the teammates' states, and for each intruder the estimate $\hat{\mathbf{x}}$ with predicted means $\hat{\mathbf{p}}_{T+h}$ and covariances Σ_h at the look-ahead $s = h \Delta t$ for $0 \leq s \leq T_{\text{pred}}$ (Chapter 20).

One clock t (seconds since mission start) and one frame serve all of them; grid time $t_{\text{grid}} = t/\Delta T$.

In the worked scenario $\ell = 1$ m, $v_{\text{cruise}} = 1$ m/s, so $\Delta T = 1$ s and grid time equals time in seconds; the control cycle is $\Delta t = 0.1$ s.

From a path to a velocity reference. The flight controller needs a continuous reference. The **reference position** of drone a_i at time t is the linear interpolation between the cells of its path,

$$\mathbf{p}_i^{\text{ref}}(t) = \mathbf{c}(\pi_i[j]) + (s - j)(\mathbf{c}(\pi_i[j + 1]) - \mathbf{c}(\pi_i[j])), \quad s = \frac{t}{\Delta T} - t_0^i, \quad j = \lfloor s \rfloor, \quad (24.1)$$

clamped to the first cell for $s \leq 0$ and to the last cell for $j \geq T_i$. The **preferred velocity** that the executive hands to the local layer is a pure pursuit of the reference a short lookahead t_{la} ahead,

$$\mathbf{v}_i^{\text{pref}}(t) = \frac{\mathbf{p}_i^{\text{ref}}(t + t_{\text{la}}) - \mathbf{p}_i(t)}{t_{\text{la}}}, \quad \text{clipped to } \|\mathbf{v}_i^{\text{pref}}\| \leq v_{\text{cruise}} \text{ while the path is unfinished,} \quad (24.2)$$

with $t_{\text{la}} = 0.5$ s in the code. A drone that sits on its reference receives exactly the velocity of the reference; a drone that has been pushed off receives a velocity that steers it back, and the pull grows with the distance. The **drift** $d_i(t) = \|\mathbf{p}_i(t) - \mathbf{p}_i^{\text{ref}}(t)\|$ is the distance from the reference and is the quantity the executive monitors while the local layer is in charge.

From a continuous state back to the grid. The replanner needs a space-time start. The executive uses the cell that contains the drone, $c_0 = (\lfloor p_x/\ell \rfloor, \lfloor p_y/\ell \rfloor)$, or its nearest free neighbor if that cell is blocked. The **start time** is $t_{\text{start}} = \lceil t/\Delta T + \frac{1}{2} \rceil$, the next grid time that leaves at least half a step to fly to the center of c_0 . The new path is stored as $\pi'_i = (c_0, \pi'_i[1], \dots)$ with $t_0^i = t_{\text{start}} - 1$, so that Equation (24.1) interpolates from the current cell into the plan. Everything a teammate reserves is expressed in absolute grid time through its own t_0 . The executive never stores a path without its start time; that is the mistake of the third pitfall of Section 24.12.

The intruder on the grid. The prediction is a sequence of disks. At every grid time $t' \geq t_{\text{start}}$ inside the prediction horizon, the executive blocks the cells whose center lies within $R_{\text{safe}} + \kappa_p \sigma(h) + \ell/2$ of the predicted mean, where h is the prediction step that matches t' . Section 24.9 defines the inflation. These are the layers $\mathcal{B}_{t'}$; they change every cycle, which is the situation D* Lite was built for (Chapter 5).

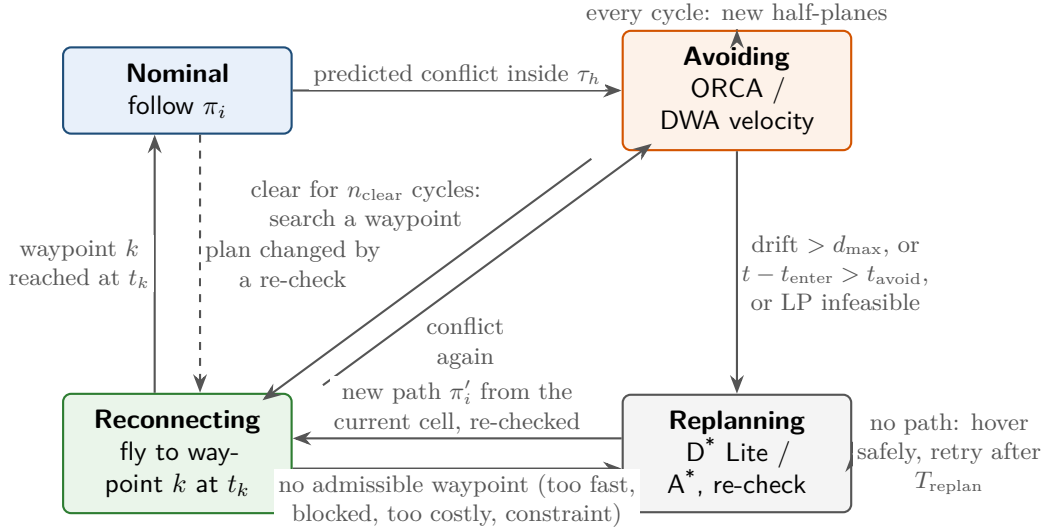


Figure 24.2. The per-drone state machine. What to notice: every transition is labeled with a condition the executive can evaluate from the world model; the two self-loops are where the time is spent; and Reconnecting is the state through which every deviation returns to Nominal, whether the plan was kept, shifted or replaced.

24.4 The decision logic: a state machine and a control loop

The five steps of the training plan (follow, avoid, reconnect, replan, re-check) become four states of the executive and the transitions of Figure 24.2. Each drone runs its own copy; the only shared data are the paths in the world model.

In Nominal the drone follows π_i through Equation (24.2), with the formation and communication terms of Section 24.8. The local layer is armed but sees only the teammates, and their half-planes are inactive while everybody follows a conflict-free plan. In Avoiding the local layer commands the velocity: the intruder is a disk of full responsibility and inflated radius, the teammates are reciprocal disks. The formation term is off and the nominal path is kept. In Reconnecting the drone flies a straight segment to a waypoint $\pi_i[j]$, timed to arrive at grid time $t_0^i + j + \Delta$ with a delay $\Delta \geq 0$. In Replanning the replanning layer computes π'_i from the current cell and the result is re-checked; while no path exists the drone hovers under the protection of the local layer and retries. Table 24.2 lists the transitions.

Algorithm 24.1 implements the table; it is called once per cycle for every drone with the freshest prediction. Its three decisions `InsideHorizon`, `Reconnect` and `ReplanAndRecheck` are Algorithms 24.2 to 24.4, the subject of the next three sections, and `LocalVelocity` is the ORCA program of Chapter 13 (or the DWA search of Chapter 14) with the half-planes chosen by the argument list.

Three details of the loop deserve a comment. First, the trigger on line 2 is evaluated in every state, so that a reconnecting drone that flies into a new prediction returns to Avoiding on line 20 before it is hit. Second, the bounded exit of Avoiding, line 8, is the important one. Without a bound on drift and time the local layer can carry the drone arbitrarily far from its plan. The reservation table of the others assumes the plan, so it becomes fiction (first pitfall of Section 24.12). Third, every velocity returned on lines 15, 18 and 22 passes through the local layer. Even the nominal flight is filtered by the teammates' half-planes. That costs nothing while the plan is conflict-free, and it protects the drone when a teammate has deviated.

Table 24.2. Transitions of the state machine, their conditions and the action taken. The parameters are listed in Table 24.3.

From	To	Condition	Action
Nominal	Avoiding	a predicted conflict is inside the horizon: $t_c \leq \tau_h$ (Definition 24.2)	remember $t_{\text{enter}} = t$; keep π_i
Avoiding	Reconnecting	no predicted conflict for n_{clear} consecutive cycles, and Reconnect finds (j, Δ)	target $\pi_i[j]$ at $t_0^i + j + \Delta$; if $\Delta > 0$ shift the remainder of π_i and re-check
Avoiding	Replanning	drift $d_i > d_{\text{max}}$, or $t - t_{\text{enter}} > t_{\text{avoid}}$, or the local program was infeasible, or Reconnect fails	hover safely until a path exists
Reconnecting	Nominal	the target waypoint time has been reached	clear the target
Reconnecting	Avoiding	a predicted conflict is inside the horizon again	as Nominal $\rightarrow$ Avoiding
Reconnecting	Replanning	the segment to the target is no longer free	as above
Replanning	Reconnecting	ReplanAndRecheck returned a path and the re-check passed (possibly after a local CBS)	target $\pi_i'[0]$ at t_{start}
Nominal	Reconnecting	another drone's re-check changed π_i (partial replanning)	target the new $\pi_i[0]$

24.5 The safety horizon and the time-to-collision trigger

The first decision is whether an intruder is a problem *now*. Compare where the drone intends to be with where the intruder is predicted to be, over a short window, with the prediction uncertainty taken into account.

The intended motion. Let $\tilde{\mathbf{p}}_i(s)$ be the **intended position** of drone a_i a look-ahead $s \geq 0$ from now. It is the point reached by flying at cruise speed along a polyline: the polyline starts at $\mathbf{p}_i(t)$, passes through the reconnection target if there is one, and then follows the remaining waypoints $\mathbf{c}(\pi_i[j_0]), \mathbf{c}(\pi_i[j_0 + 1]), \dots$ of the plan. Here $j_0 = \lceil t/\Delta T - t_0^i \rceil$ is the first waypoint not yet passed. For a drone on its reference this is the reference itself; for a drone that has deviated it is the path it would take home. The prediction layer supplies $\hat{\mathbf{p}}_{T+h}$ and Σ_h at the same look-aheads $s = h \Delta t$ (Chapter 20), and we write $\sigma(h) = \sqrt{\lambda_{\text{max}}(\Sigma_h)}$ for the largest standard deviation of the predicted position.

Definition 24.2 (Predicted conflict inside the safety horizon). Let $R_{\text{safe}} = r_A + r_B + \Delta r$ be the required separation between a drone of radius r_A and an intruder of radius r_B , with a margin $\Delta r \geq 0$, and let κ_p be the confidence factor of Section 24.9. The **margin** at look-ahead $s = h \Delta t$ is

$$m_i(s) = \|\tilde{\mathbf{p}}_i(s) - \hat{\mathbf{p}}_{T+h}\| - \kappa_p \sigma(h) - R_{\text{safe}}. \quad (24.3)$$

A predicted conflict is **inside the safety horizon** τ_h if $m_i(s) < 0$ for some $s \in [0, \tau_h]$, and the **predicted time to collision** is the first such look-ahead,

$$t_c = \min \{s \in [0, \tau_h] : m_i(s) < 0\}. \quad (24.4)$$

In practice s runs over the multiples of Δt up to τ_h . Note that t_c is a time to loss of *margin*: it is strictly earlier than any impact, because the margin includes the inflation and R_{safe} .

Algorithm 24.1. The per-drone control cycle of the executive

Input: drone a_i with state, path π_i , start time t_0^i ; time t ; prediction $(\hat{\mathbf{p}}_{T+h}, \Sigma_h)_h$ or **nil**; states of the teammates

Output: the velocity command for this cycle

```

1 Function ControlCycle( $a_i, t$ ):
2   ( $inside, t_c$ )  $\leftarrow$  InsideHorizon( $a_i, t, prediction$ )
3   if  $state = Nominal$  and  $inside$  then
4      $state \leftarrow Avoiding$ ;  $t_{enter} \leftarrow t$ ;  $n_{ok} \leftarrow 0$ 
5   if  $state = Avoiding$  then
6      $n_{ok} \leftarrow 0$  if  $inside$  else  $n_{ok} + 1$ 
7     if  $d_i(t) > d_{max}$  or  $t - t_{enter} > t_{avoid}$  or the last local program was infeasible then
8        $state \leftarrow Replanning$ 
9     else if  $n_{ok} \geq n_{clear}$  then
10      ( $j, \Delta$ )  $\leftarrow Reconnect(a_i, t, prediction)$ 
11      if ( $j, \Delta$ ) = failure then  $state \leftarrow Replanning$ 
12      else
13         $state \leftarrow Reconnecting$ ;  $target \leftarrow (c(\pi_i[j]), t_0^i + j + \Delta)$ 
14        if  $\Delta > 0$  then  $\pi_i \leftarrow \pi_i[j:]$ ;  $t_0^i \leftarrow t_0^i + j + \Delta$ ; re-check as in Algorithm 24.4
15      else return LocalVelocity( $\mathbf{v}_i^{pref}$  without formation term; teammates
        reciprocal; intruder with full share, inflated radius at  $t_c$ )
16    if  $state = Replanning$  then
17      if  $t - t_{replan} \geq T_{replan}$  and ReplanAndRecheck( $a_i, t, prediction$ ) succeeds then
18         $t_{replan} \leftarrow t$ ;  $state \leftarrow Reconnecting$ ;  $target \leftarrow (c(c_0), t_{start})$ 
19      else return LocalVelocity( $\mathbf{0}$ ; teammates and intruder as in Avoiding)
        // hover safely, retry next cycle
20    if  $state = Reconnecting$  then
21      if  $inside$  then  $state \leftarrow Avoiding$ ; return the velocity of line 15
22      if  $t \geq target\ time$  then  $state \leftarrow Nominal$ ; clear the target
23    return LocalVelocity( $\mathbf{v}_i^{pref}$  with formation and communication terms;
      teammates reciprocal; no intruder)

```

The definition admits $t_c = 0$: the drone is already inside the inflated margin. The executive then enters Avoiding with $t_c = 0$ and builds the ORCA half-plane at $h_c = 0$, that is, from the current relative position. That is the most aggressive maneuver the local layer can produce. It is a recovery, not a prevention, and the drone stays in Avoiding until the margin is positive again for n_{clear} cycles.

Subtracting the inflation from the separation is the same as adding it to the radius; Section 24.9 explains why $\kappa_p \sigma(h)$ is the right amount. Algorithm 24.2 is the test, and Figure 24.3 shows it at the moment it fires in the worked scenario. The predicted separation of drone A falls to 1.53 m at $s = 3.0$ s, while the threshold $R_{safe} + \kappa_p \sigma$ has grown to $0.70 + 0.86 = 1.56$ m. One cycle earlier the margin at the same look-ahead was still +0.01 m.

Choosing the horizon. Three quantities constrain τ_h . First, the horizon τ of the local layer: ORCA's half-plane of Chapter 13 forbids the relative velocities that collide within τ , so the local layer only starts to steer when $t_c \leq \tau$. With $\tau_h \geq \tau$ control is handed over before the local layer has to act, and its first correction is small; with $\tau_h < \tau$ the hand-over comes late

Table 24.3. Parameters of the executive, their meaning, their value in the worked scenario and their effect. Distances in meters, times in seconds.

Parameter	Value	Meaning and effect
$\Delta t; \Delta T$	0.1; 1	control cycle; grid time step = ℓ/v_{cruise}
$v_{\text{cruise}}; v_{\text{max}}; a_{\text{max}}$	1; 1.5; 1	cruise and maximum speed in m/s; acceleration limit in m/s ²
$r_A, r_B, \Delta r$	0.3, 0.3, 0.1	drone radius, intruder radius, safety margin; $R_{\text{safe}} = r_A + r_B + \Delta r = 0.7$
τ_h	3	safety horizon = ORCA horizon τ ; larger = earlier, gentler, more false alarms
κ_p	2.45	inflation factor of the 95 % disk; larger = safer and more conservative
T_{pred}	5	prediction horizon; must cover τ_h and the replanning layers
n_{warm}	10	filter updates before a prediction is trusted (Chapter 18)
n_{clear}	5	clear cycles before reconnection is attempted; hysteresis against chattering
$K; \Delta_{\text{max}}$	6; 1	waypoints searched forward; largest admissible delay in grid steps
$d_{\text{max}}; t_{\text{avoid}}$	3.5; 8	drift and time bounds of Avoiding; the escalation to Replanning
T_{replan}	1	shortest interval between two replans of one drone
<code>period_prediction</code>	0.1	period of the prediction layer in the rate schedule of Listing 24.1 ; equal to Δt here, which is why Table 24.1 reads “every control cycle”
$R_{\text{comm}}; \ell_{\text{act}}$	8.5; 7.5	communication range; link length at which the range term acts
k_F	0.5	formation correction gain of Equation (24.6)
k_C	1	gain of the soft range term of Equation (24.7)

and the first correction is a jump. The code uses $\tau_h = \tau$. Second, the dynamics. A drone with acceleration limit a_{max} needs v/a_{max} seconds to stop from speed v , and it covers the stopping distance $d_{\text{stop}}(v) = v^2/(2a_{\text{max}})$ of [Chapter 2](#) while doing so. Changing from one velocity of the disk $\|\mathbf{v}\| \leq v_{\text{max}}$ to any other takes at most $2v_{\text{max}}/a_{\text{max}}$. The horizon must leave time for the local layer’s command to be executed,

$$\tau_h \geq \Delta t + \frac{v_{\text{max}}}{a_{\text{max}}} \quad (\text{a stop suffices}), \quad \tau_h \geq \Delta t + \frac{2v_{\text{max}}}{a_{\text{max}}} \quad (\text{a reversal may be needed}). \quad (24.5)$$

Third, the prediction: $\tau_h \leq T_{\text{pred}}$, and $\kappa_p \sigma(\tau_h)$ must stay well below the spacing of the lanes, otherwise the trigger fires for intruders in the next lane. In the scenario $v_{\text{max}}/a_{\text{max}} = 1.5$ s, $\tau = 3$ s and $\kappa_p \sigma(3 \text{ s}) = 0.86$ m. So $\tau_h = 3$ s clears the hand-over condition and the stop form of [Equation \(24.5\)](#) (1.6 s) with room to spare, and falls 0.1 s short of the reversal form (3.1 s). The single-integrator drones of the scenario are never asked to reverse, but a double-integrator drone would need $\tau_h \geq 3.1$ s ([Exercise 24.7](#)). The inflation is still below the lane spacing of 2 m. A last condition ties the horizon to the sensor: the intruder must be detected and tracked before it enters the horizon. That needs a detection range of at least $(v_{\text{cruise}} + v_B)(\tau_h + n_{\text{warm}}\Delta t) + R_{\text{safe}} + \kappa_p \sigma(\tau_h)$, about 8 m in the scenario against a sensor range of 6 m ([Exercise 24.1](#) computes such a range). The scenario gets away with it because the tracks are shared: drone B detects the intruder first and A inherits the track, so A’s trigger still fires with the full 3 s to act.

Algorithm 24.2. The safety-horizon trigger

Input: drone a_i , time t , prediction $(\hat{\mathbf{p}}_{T+h}, \Sigma_h)$ for $h = 0, \dots, \lfloor T_{\text{pred}}/\Delta t \rfloor$ or **nil**
Output: (inside, t_c)

```

1 Function InsideHorizon( $a_i, t, \text{prediction}$ ):
2   if  $\text{prediction} = \text{nil}$  then return (false, nil)    // no intruder, or the filter is still
   warming up
3    $H \leftarrow \min(\lfloor \tau_h/\Delta t \rfloor, \lfloor T_{\text{pred}}/\Delta t \rfloor)$ 
4    $(\tilde{\mathbf{p}}_i(0), \dots, \tilde{\mathbf{p}}_i(H\Delta t)) \leftarrow \text{IntendedPositions}(a_i, t)$ 
5   for  $h \leftarrow 0$  to  $H$  do
6      $m \leftarrow \|\tilde{\mathbf{p}}_i(h\Delta t) - \hat{\mathbf{p}}_{T+h}\| - \kappa_p \sqrt{\lambda_{\max}(\Sigma_h)} - R_{\text{safe}}$ 
7     if  $m < 0$  then return (true,  $h\Delta t$ )
8   return (false, nil)

```

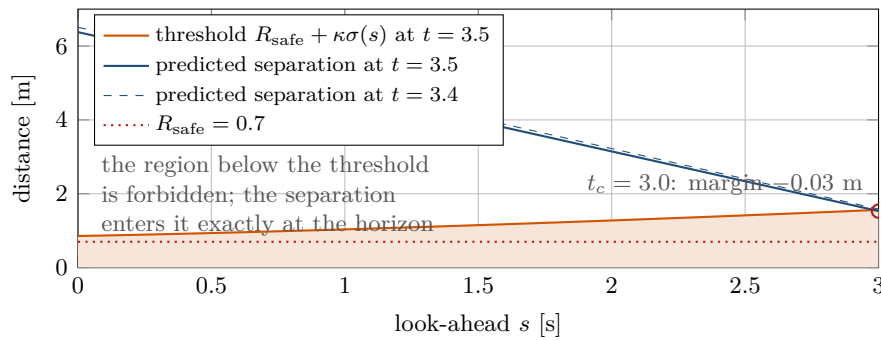


Figure 24.3. The trigger of drone A in the worked scenario at $t = 3.5$ s. The predicted separation between A’s intended positions and the intruder’s predicted mean (solid blue) decreases with the look-ahead; the threshold $R_{\text{safe}} + \kappa_p \sigma(s)$ (orange) grows with it because the prediction becomes less certain. What to notice: the two curves meet exactly at the horizon $s = \tau_h = 3$ s, where the margin is -0.03 m; one control cycle earlier (dashed) the separation was 0.05 m larger and the trigger stayed silent. The trigger fires at the horizon, not at the collision: three seconds remain to act.

A trigger without hysteresis chatters

The margin at the horizon changes by a few centimeters per cycle, and noise in the estimate moves it back and forth across zero. If the executive left Avoiding in the first cycle without a predicted conflict, it would re-enter it in the next one. The drone would then alternate between the ORCA velocity and the pursuit of its plan ten times a second. The counter n_{clear} in Algorithm 24.1 is the remedy: the drone leaves Avoiding only after n_{clear} consecutive clear cycles. A second hysteresis is built into the trigger itself, because entering uses the inflated radius at t_c while the local layer keeps the intruder’s half-plane for the whole of τ .

24.6 Reconnecting to the nominal path

When the conflict has cleared, the cheapest repair is no repair at all: fly back to the plan and resume it. The question is *where* to rejoin, and the answer is a search along the plan.

Definition 24.3 (Admissible reconnection). A **reconnection** of drone a_i at time t is a pair (j, Δ) with $j \geq j_0$ a waypoint index and $\Delta \in \{0, \dots, \Delta_{\text{max}}\}$ a delay in grid steps. It is **admissible** when the straight flight from $\mathbf{p}_i(t)$ to $\mathbf{c}(\pi_i[j])$, arriving at $t_j = (t_0^i + j + \Delta)\Delta T$,

meets four conditions. It (i) needs a speed $\|\mathbf{c}(\pi_i[j]) - \mathbf{p}_i(t)\| / (t_j - t) \leq v_{\max}$, and (ii) keeps a clearance of at least r_A from the static obstacles. It also (iii) keeps the margin of Equation (24.3) non-negative against the prediction at every sampled time. Finally it (iv) stays at least $R_{\text{mate}} = 2r_A + \Delta r$ from the intended positions of every teammate at the same times. Its **extra cost** is Δ : the arrival time of a_i increases by Δ grid steps, and SoC and MS by at most that.

Algorithm 24.3. Reconnection search along the nominal path

Input: drone a_i at $\mathbf{p}_i(t)$ with path π_i , start time t_0^i and last index T_i ; prediction; look-ahead K ; largest delay $\Delta_{\max}$

Output: (j, Δ) or *failure*

```

1 Function Reconnect( $a_i, t, \text{prediction}$ ):
2    $j_0 \leftarrow \max(0, \lceil t/\Delta T - t_0^i \rceil)$ 
3   for  $\Delta \leftarrow 0$  to  $\Delta_{\max}$  do
4     for  $j \leftarrow j_0$  to  $\min(j_0 + K, T_i)$  do
5        $t_j \leftarrow (t_0^i + j + \Delta) \Delta T$ ;  $\mathbf{w} \leftarrow \mathbf{c}(\pi_i[j])$ 
6       if  $t_j - t < \Delta t$  or  $\|\mathbf{w} - \mathbf{p}_i(t)\| / (t_j - t) > v_{\max}$  then continue // too fast
7       if SegmentFree( $\mathbf{p}_i(t), t, \mathbf{w}, t_j, \text{prediction}, \text{teammates}$ ) then return  $(j, \Delta)$ 
8   return failure
9 Function SegmentFree( $\mathbf{p}, t_a, \mathbf{w}, t_b, \text{prediction}, \text{teammates}$ ):
10  foreach sample time  $t' = t_a + (t_b - t_a)u/n, u = 0, \dots, n$ , with  $n = \lceil (t_b - t_a)/\Delta t \rceil$ 
11    do
12       $\mathbf{q} \leftarrow \mathbf{p} + (\mathbf{w} - \mathbf{p})(t' - t_a)/(t_b - t_a)$ 
13      if clearance of  $\mathbf{q}$  to the obstacles  $< r_A$  then return false
14      if  $\|\mathbf{q} - \hat{\mathbf{p}}_{T+h}\| - \kappa_p \sigma(h) < R_{\text{safe}}$  for  $h = (t' - t_a)/\Delta t$  then return false
15      if  $\|\mathbf{q} - \tilde{\mathbf{p}}_m(t' - t_a)\| < R_{\text{mate}}$  for some teammate  $a_m$  then return false
16  return true

```

The search order of Algorithm 24.3 encodes the preferences. Delay zero comes first: a reconnection with $\Delta = 0$ returns the drone to its own reservation, and nothing else in the world model changes. Among equal delays the earliest waypoint comes first, because it minimizes the time off the plan. The look-ahead K bounds the work to $(\Delta_{\max} + 1)(K + 1)$ segment tests of $\mathcal{O}(nk)$ distance computations each, and bounds the detour to a segment of at most $(K + \Delta_{\max})\ell$.

Two consequences matter for the rest of the design. A reconnection with $\Delta = 0$ needs no conflict re-check: the plan in force is unchanged, and condition (iv) has verified the segment against the teammates. A reconnection with $\Delta > 0$ shifts the tail of π_i by Δ steps (line 14 of Algorithm 24.1). The shifted reservation may now collide with a teammate that was timed to cross behind the drone. The executive must therefore run the re-check of Section 24.7 after every delayed reconnection. In the worked scenario this is exactly what happens to drone C at $t = 4.9$ s: its reconnection with $\Delta = 1$ succeeds, and the re-check finds the conflict $\langle B, C, (6, 2), 7 \rangle$ that the shift created.

When is a reconnection “too costly” or “impossible”? Impossible when every candidate fails line 6 or the segment test. That is what happens to drone A at $t = 9.1$ s. Seven of the eight candidates would need more than $v_{\max}$ (up to 4.61 m/s), and the only slow enough one, $j = 13$ with $\Delta = 1$, crosses the predicted tube. Too costly when only delays beyond $\Delta_{\max}$ would work or, with a formation, when the segment would stretch a link beyond R_{comm} or leave the slot for longer than the mission tolerates (Section 24.8). All of these end on line 8, and Algorithm 24.1 escalates to Replanning.

24.7 Replanning and the conflict re-check

The replanning trigger. The executive replans in four situations, all listed in Table 24.2. The reconnection search failed. The drift exceeded $d_{\max}$ while Avoiding, so that the plan has lost contact with the drone. The drone has been Avoiding for longer than t_{avoid} , which is how a deadlock of the local layer (Chapter 13) is detected. Or the local program was infeasible, so that the dense fallback of Chapter 13 is choosing the least bad velocity. A fifth trigger comes from the constraints of Section 24.8. None of them fires because the plan is *suboptimal*. A safe, connected drone keeps its plan even if a shorter one has appeared: every replan costs a re-check and risks a cascade (second pitfall of Section 24.12).

What the replanner searches. The replanning layer searches the time-expanded grid of Chapter 4 from the space-time start (c_0, t_{start}) of Section 24.3 to the drone’s goal, with three kinds of obstacles:

1. the **reservation table** R of the teammates’ *current* paths, in absolute grid time: every (cell, time) pair, every reversed move (against swaps) and every parked goal from its arrival time on, exactly as in the low level of CBS and in prioritized planning (Chapters 8 and 9);
2. the teammates’ current cells, blocked at t_{start} , because a teammate that is itself Avoiding is not where its plan says;
3. the **blocked layers** $\mathcal{B}_v$ of the predicted intruder for $t_{\text{start}} \leq t' \leq t + T_{\text{pred}}$, the cells within $R_{\text{safe}} + \kappa_p \sigma(h) + \ell/2$ of the predicted mean at the matching step h ; beyond the prediction horizon the intruder is unknown and nothing is blocked.

A layer may be *blocked* or merely *expensive*. Blocking gives the strongest statement, namely that the path is clear of the p -disc at every step, but it can make the problem infeasible in a corridor. A high traversal cost keeps the search complete and lets the drone cross the tube when there is no alternative. The code of this chapter blocks; Exercise 24.5 asks for the soft version. The search itself is D* Lite [88] (Chapter 5) or space-time A* (Chapter 4). D* Lite is the natural choice when the layers change every cycle and the drone keeps moving. The changed cells are edge-cost changes, the moving start is what the key modifier k_m absorbs, and the previous search tree is reused. The simulation of this chapter calls a fresh space-time A* for every replan, because its instances have a few hundred states and a fresh search costs a millisecond. The interface (current cell, start time, reservation, blocked layers) is the same, so the two are interchangeable.

Why a re-check is needed. The replanned path respects the reservation table, so why check it again? Because the table is not the world. First, it holds *plans*, and a teammate that is Avoiding is off its plan; its reservation protects the cells it will occupy once it is back, not the ones it flies through now. Second, two drones may replan in the same cycle, each against the other’s *old* path, and the two new paths can collide although each respected what it saw. Third, a delayed reconnection shifts a path without any search at all. The re-check therefore runs after every change of a path in force: the conflict detector of Chapter 7 over the padded paths of all drones from the current grid time on. If it finds a conflict, the executive does not restart the global planner. It runs a **local CBS**, also called partial replanning, on the two agents in conflict, from t_{start} , with the paths of all other drones reserved. A conflict between the pair and a third drone is impossible by construction, and the pair’s own conflicts are resolved by the constraint tree of Chapter 9. Algorithm 24.4 states the whole procedure.

Algorithm 24.4. Replanning from the current state and the conflict re-check

Input: drone a_i at $\mathbf{p}_i(t)$, goal γ_i , time t , prediction, the paths (π_j, t_0^j) of all drones
Output: success (paths updated) or *failure*

```

1 Function ReplanAndRecheck( $a_i, t, prediction$ ):
2    $c_0 \leftarrow$  the cell containing  $\mathbf{p}_i(t)$ , or its nearest free neighbor;  $t_{\text{start}} \leftarrow \lceil t/\Delta T + \frac{1}{2} \rceil$ 
3    $R \leftarrow$  reservation of  $(\pi_j, t_0^j)$  for all  $j \neq i$ , plus the cell of every  $\mathbf{p}_j(t)$  at  $t_{\text{start}}$ 
4   foreach grid time  $t' \geq t_{\text{start}}$  with  $t'\Delta T - t \leq T_{\text{pred}}$ , step  $h = (t'\Delta T - t)/\Delta t$  do
5      $\lfloor$  add to  $R$  the layer  $\mathcal{B}_{t'} = \{v \in V : \|\mathbf{c}(v) - \hat{\mathbf{p}}_{T+h}\| < R_{\text{safe}} + \kappa_p \sigma(h) + \ell/2\}$ 
6    $\pi' \leftarrow \text{SpaceTimeSearch}(G, c_0, \gamma_i, t_{\text{start}}, R)$  //  $D^*$  Lite or space-time  $A^*$  with the
     goal-stay test
7   if  $\pi' = \text{failure}$  then return failure
8    $\pi_i \leftarrow (c_0, \pi'[0], \pi'[1], \dots)$ ;  $t_0^i \leftarrow t_{\text{start}} - 1$ 
9    $\text{conflict} \leftarrow \text{FirstConflict}((\pi_j, t_0^j)_j, \text{from } \lfloor t/\Delta T \rfloor)$  // detector of Chapter 7
10  if  $\text{conflict} = \langle a_i, a_j, \dots \rangle$  then
11     $R' \leftarrow$  reservation of every drone except  $a_i, a_j$ 
12     $(\pi_i, \pi_j) \leftarrow \text{LocalCBS}(\text{starts: cells of } a_i, a_j \text{ at } t_{\text{start}}; \text{goals } \gamma_i, \gamma_j; t_{\text{start}}; R')$ 
13    if failure then escalate: CBS on a larger subset, or on all drones from their
      current cells
14     $t_0^i \leftarrow t_0^j \leftarrow t_{\text{start}}$ ;  $a_j \leftarrow$  Reconnecting toward  $\mathbf{c}(\pi_j[0])$  at  $t_{\text{start}}$ 
15  return success

```

Reservations without their time offsets

A path that starts at $t_0 = 10$ occupies its first cell at grid time 10, not at time 0. An executive that stores paths without their start times sees conflicts that never happen: a drone that “occupies” its current cell during the whole past. So does one that pads a replanned path backward to time zero before checking. Both also miss the conflicts that do happen, because a reservation is shifted by the age of the plan. The same error appears when grid time and clock time drift apart, for instance after a delayed reconnection. Keep one clock, store (π_i, t_0^i) together, convert with Equation (24.1) only, and run the detector from the current grid time, never from zero.

24.8 Formation and communication constraints during avoidance and replanning

A swarm that must keep a formation or a radio link has two more constraints, and the architecture handles them in three places.

In the preferred velocity. In the Nominal and Reconnecting states the executive adds to Equation (24.2) the displacement-based formation correction of Chapter 23: for a follower a_i with leader a_ℓ and desired displacement $\mathbf{d}_{\ell i}$,

$$\mathbf{v}_i^{\text{pref}} \leftarrow \mathbf{v}_i^{\text{pref}} + k_F(\mathbf{p}_\ell + \mathbf{d}_{\ell i} - \mathbf{p}_i), \quad (24.6)$$

which is one term of the consensus protocol on the shifted positions $\mathbf{p}_i - \mathbf{o}_i$. The general form sums over the formation neighbors. The communication range enters as a soft barrier that acts before the link breaks: if the nearest teammate a_j is farther than an activation length

$$\ell_{\text{act}} < R_{\text{comm}},$$

$$\mathbf{v}_i^{\text{pref}} \leftarrow \mathbf{v}_i^{\text{pref}} + k_C \frac{\mathbf{p}_j - \mathbf{p}_i}{\|\mathbf{p}_j - \mathbf{p}_i\|} (\|\mathbf{p}_j - \mathbf{p}_i\| - \ell_{\text{act}}), \quad (24.7)$$

with a gain $k_C > 0$ in units of 1/s, like k_F in Equation (24.6) ($k_C = 1$ in the code). The result is clipped to v_{max} . Both terms are inputs to the local layer, not overrides of it: ORCA may still change the velocity, and when it does, safety wins.

What is relaxed while Avoiding, and restored while Reconnecting. Separation from the intruder has priority over the shape of the swarm. In the Avoiding state the executive drops the formation term Equation (24.6). Keeping the slot would pull the drone back into the conflict it is leaving. It would also drag the whole formation after a single avoiding drone, through the followers’ corrections. The range term Equation (24.7) stays, because a broken link is a mission failure rather than a shape error, and it is soft enough not to fight the half-planes. The teammates that are not avoiding keep their formation terms and therefore move to accommodate the one that is, as Chapter 23 describes. When the drone enters Reconnecting the formation gain returns: at once in the code, ramped over a second in a smoother design. The formation error e_F of Chapter 23 then decays with the consensus rate. With the MPC tracker of Chapter 21 the same policy is expressed in the constraint rows. The formation slot and the range are soft constraints with slacks. The slack weight of the formation is set to zero while Avoiding and restored while Reconnecting, and the range constraint is kept hard whenever the program stays feasible. The recovery time of e_F after the intruder has passed is one of the metrics of Chapter 25.

In the replanning trigger and the re-check. A reconnection or a replanned path is a single-drone decision. It can be admissible in every geometric sense and still break the swarm. The straight segment of Algorithm 24.3 may take the drone beyond R_{comm} of every teammate, and a replanned path may keep it there for ten steps. The executive therefore adds a constraint test to both. Over the horizon of the prediction, at every grid time t' , the padded paths must keep at least one link of every drone shorter than R_{comm} . For a formation, they must instead keep the formation graph connected. The formation error of the planned positions must also return below its tolerance e_{max} within the recovery time. A candidate that fails the test counts as “violates the constraints” in the training plan’s rule (iv). A reconnection that fails is rejected and the drone replans. A replanned path that fails is rejected too, and the executive escalates to line 13 of Algorithm 24.4 with the formation planned as a group. One way to plan it as a group is the virtual leader of Chapter 23, whose footprint is the formation’s. In the worked scenario the range is monitored only. The longest link between a drone and its nearest teammate over the run is 7.33 m, below $\ell_{\text{act}} = 7.5$ m, so Equation (24.7) never acts and the communication graph stays connected throughout.

24.9 Prediction-aware constraints with uncertainty

Every decision above uses the prediction through one number, the inflation $\kappa_p \sigma(h)$. This section says where it comes from and what it guarantees.

From a covariance to a radius. The prediction layer delivers, at step h , a Gaussian $\mathcal{N}(\hat{\mathbf{p}}_{T+h}, \Sigma_h)$ for the intruder’s position (Chapters 18 and 20). The set of positions within Mahalanobis distance κ of the mean is the ellipse $\mathcal{E}_h(\kappa)$ of Chapter 2. In two dimensions the probability mass inside it is $1 - e^{-\kappa^2/2}$, because the squared Mahalanobis distance of a two-dimensional Gaussian is exponentially distributed with mean 2. Hence the ellipse of

confidence p has

$$\kappa_p = \sqrt{-2 \ln(1 - p)}, \quad \kappa_{0.95} = 2.45, \quad \kappa_{0.99} = 3.03. \quad (24.8)$$

The disk of radius $\kappa_p \sigma(h)$, with $\sigma(h) = \sqrt{\lambda_{\max}(\Sigma_h)}$, contains this ellipse (Chapter 2: the semi-axes are $\kappa_p \sqrt{\lambda_i}$). So the intruder's center lies inside it with probability at least p . The **inflated safety radius**

$$d_h^{\text{safe}} = R_{\text{safe}} + \kappa_p \sigma(h) = r_A + r_B + \Delta r + \kappa_p \sqrt{\lambda_{\max}(\Sigma_h)} \quad (24.9)$$

is the distance from the predicted mean at which the drone's disk is clear of the intruder's disk with probability at least p . This is the formula of Chapter 20, where the factor is written k_p . The chapter's code uses $p = 0.95$.

The chance-constraint reading. Requiring $\|\mathbf{p}_A - \hat{\mathbf{p}}_{T+h}\| \geq d_h^{\text{safe}}$ is a conservative version of the **chance constraint** $\mathbb{P}(\|\mathbf{p}_A - \mathbf{p}_B\| \geq R_{\text{safe}}) \geq p$ of Zhu and Alonso-Mora [187]. Chapter 21 imposes that constraint in its linear form, with the standard deviation taken along the normal of the half-plane and the one-dimensional factor $\kappa_\delta = 1.64$ for $\delta = 0.05$. The disk form is what the trigger and the replanner need, because neither has a normal direction to project on. It costs a factor 2.45/1.64 in radius and guarantees the same probability. In both forms the guarantee is per step and per obstacle. Over H steps and n intruders the risk adds up to at most $Hn(1 - p)$. That is why p is chosen close to one and why the safety horizon is short.

Three consumers. The same disks are read three ways. The trigger inflates at every look-ahead. Once it has fired, the ORCA half-plane of the intruder is built with the radius $r_B + \Delta r + \kappa_p \sigma(h_c)$ at the step h_c of the predicted collision. It carries full responsibility, because the intruder does not reciprocate (Chapter 13). The replanner reads one layer of cells per grid time (Algorithm 24.4, line 5). The replan of drone A at $t = 9.1$ s blocks 70 predicted cells over five layers, seven of them in the first, plus the two teammates' current cells. The MPC tracker of Chapter 21 reads one half-plane per step. A wrong covariance corrupts all three. The LSTM of Chapter 20 covered 54 % instead of 95 % at 2.4 s before calibration. An over-confident predictor makes the trigger late, the layers thin and the MPC margin small. Calibrate the ellipses per horizon before they enter this chapter.

Particles instead of ellipses. When the prediction is a set of N weighted trajectories, the disk becomes a count. Such a set comes from the particle filter of Chapter 19 or from a multimodal predictor of Chapter 20. The trigger fires at look-ahead s when the weight of the particles $\mathbf{p}_h^{(n)}$ with $\|\tilde{\mathbf{p}}_i(s) - \mathbf{p}_h^{(n)}\| < R_{\text{safe}}$ exceeds $1 - p$. A cell enters $\mathcal{B}_t$ when the weight within $R_{\text{safe}} + \ell/2$ of its center exceeds the same threshold. The ORCA disk is built around the mode that threatens the drone. An intruder that may pass on either side of a building is then handled without one ellipse over the whole block, at a cost linear in N (Exercise 24.2).

How fast the inflation grows. For the constant-velocity Kalman model with white-noise acceleration of intensity q (Chapter 18), the extrapolated position variance is $\sigma_p^2 + 2s \sigma_{pv} + s^2 \sigma_v^2 + \frac{1}{3}qs^3$ in the filter's position, cross and velocity variances. The tracker of Example 24.4 has $q = 0.005 \text{ m}^2/\text{s}^3$. The velocity term $s^2 \sigma_v^2$ dominates at the look-aheads that matter here, so the inflation grows almost linearly. The cubic term takes over only once the velocity is well estimated and the horizon is long. At the trigger $\sigma_p^2 = 0.0041$, $\sigma_{pv} = 0.0035$ and $\sigma_v^2 = 0.0060$ (SI units). They give a variance of 0.004, 0.019, 0.056, 0.124 m^2 at $s = 0, 1, 2, 3$ s. Of the 0.124 at $s = 3$ s, 0.054 is the velocity term and 0.045 the cubic one. The tracker of the scenario

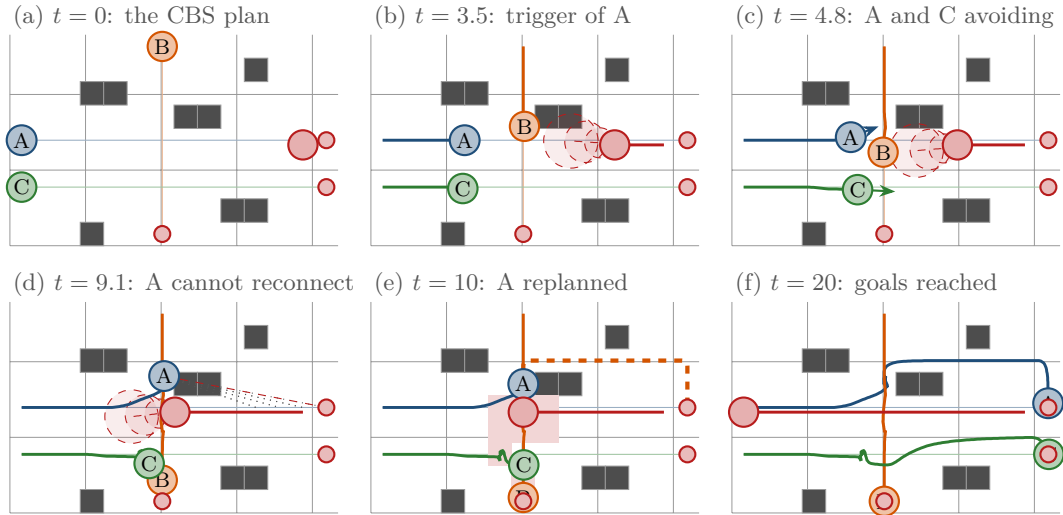


Figure 24.4. Six frames of the worked scenario (thin lines: nominal paths; thick lines: flown trajectories; red: the intruder, its predicted mean over τ_h and the 95 % disks at 1, 2, 3 s; small red dots: the goals). (a) The CBS plan; B waits once at (6, 3) to let C pass. (b) At $t = 3.5$ s the predicted disk at 3 s touches A's intended position: A switches to Avoiding. (c) At $t = 4.8$ s A has climbed north and C, whose lane is grazed by the inflated tube, has slowed and dipped; the arrows are the ORCA velocities. (d) At $t = 9.1$ s the conflict has cleared for A, but every candidate segment back to its lane is either too fast (gray dotted) or crosses the predicted tube (red dashed). (e) A's replanned path (orange dashed) from (6, 5) at $t_{\text{start}} = 10$ along row 6; the shaded cells are the layer $\mathcal{B}_{10}$. (f) At the end all three drones sit at their goals and the formation is restored.

reports $\sigma = 0.06, 0.14, 0.24, 0.35$ m, an inflation of 0.86 m at the horizon, larger than R_{safe} itself. The uncertainty, not the geometry, decides the trigger at the horizon. A longer horizon with a constant-velocity prediction would mostly add false alarms. What a longer horizon buys is a calibrated predictor with a slower-growing covariance (Section 24.13).

24.10 A worked scenario

The book's own code, `code/ch24_hybrid.py`, combines condensed copies of the modules of Chapters 4, 7, 9, 13 and 18 with the executive of Algorithm 24.1. Every number below is printed by it or by `code/figures/gen_ch24_scenario.py`.

Example 24.4 (Three drones, one intruder). The grid has 14×9 cells of 1 m with eight blocked cells (Figure 24.4). Drone A flies from (0, 4) to (13, 4), B from (6, 8) to (6, 0), C from (0, 2) to (13, 2); C is a follower of A with the displacement $\mathbf{d}_{AC} = (0, -2)$. An intruder of radius 0.3 m starts at (12.5, 4.3) and flies west at 0.6 m/s along A's lane. It is unknown to the swarm until it comes within 6 m of a drone. From then on it is observed at 10 Hz with a position noise of $\sigma_z = 0.15$ m and tracked by the constant-velocity Kalman filter of Chapter 18. The filter is used in its discrete white-noise-acceleration form, with acceleration variance $\sigma_a^2 = 0.05 \text{ m}^2/\text{s}^4$ per axis; that is a continuous intensity $q = \sigma_a^2 \Delta t = 0.005 \text{ m}^2/\text{s}^3$ at the 10 Hz observation rate. The drones are single integrators with $v_{\text{cruise}} = 1 \text{ m/s}$, $v_{\text{max}} = 1.5 \text{ m/s}$ and $a_{\text{max}} = 1 \text{ m/s}^2$; the other parameters are those of Table 24.3.

Table 24.4 is the trace of the executive, Figure 24.5 the separations, states, formation error and drift over the run, and Figure 24.6 the conflict re-check in detail.

What the run shows. The smallest distance between two drones over the run is 0.80 m against $R_{\text{mate}} = 0.70$ m, and none of the three ever approaches R_{safe} of the intruder (Figure 24.5).

Table 24.4. Trace of the worked scenario: every transition of the state machine with the numbers that caused it. Times in seconds, distances in meters; t_0 is the start time of a path.

t	Drone: transition	What happened
0.0	all: Nominal	CBS expands 2 constraint-tree nodes: A costs 13, B 9 (one wait at (6, 3), $t = 5, 6$, to let C cross (6, 2)), C 13; SoC = 35, MS = 13.
1.3	—	B is 6.0 m from the intruder and detects it; the filter is trusted after $n_{\text{warm}} = 10$ updates, at $t = 2.2$.
3.5	A: Nominal → Avoiding	$t_c = 3.0$: separation 1.53, inflation 0.86, $R_{\text{safe}} = 0.70$, margin -0.03 . Estimate (10.37, 4.35), velocity $(-0.61, 0.04)$; truth (10.40, 4.30), $(-0.60, 0)$.
4.3	C: Nominal → Avoiding	$t_c = 2.9$: A's climb and the growing inflation reach C's lane.
4.9	C: Avoiding → Reconnecting; B: Nominal → Reconnecting	five clear cycles; $j = 5$, $\Delta = 1$; the tail of π_C is shifted ($t_0 = 6$). Re-check: $\langle B, C, (6, 2), 7 \rangle$. Local CBS on $\{B, C\}$ from $t_{\text{start}} = 6$ with A reserved: B waits one more step at (6, 3) and reaches (6, 2) at 8; C unchanged; B's plan changed (partial replanning).
5.0	C: Reconnecting → Avoiding	$t_c = 2.8$: the repaired plan leads C back into the tube.
6.0	B: Reconnecting → Nominal; C: Avoiding → Reconnecting → Replanning → Reconnecting	B reaches waypoint 0 of its new path. C is clear again but every segment is blocked; replan from (5, 2) at $t_{\text{start}} = 7$: wait until 8, pass under the tube along row 1 from (5, 1) to (8, 1), back to row 2 at $t = 13$, goal at 18. Re-check: no conflict.
7.0	C: Reconnecting → Nominal	C reaches (5, 2), the first cell of its new path.
9.1	A: Avoiding → Reconnecting → Replanning → Reconnecting	clear after 5.6 s of avoiding, drift $3.22 < d_{\text{max}}$; $j = 10, \dots, 13$ with $\Delta = 0$ and $j = 10, 11, 12$ with $\Delta = 1$ are too fast, $j = 13$ with $\Delta = 1$ crosses the tube. Replan from (6, 5) at $t_{\text{start}} = 10$ with 72 blocked cells: hover, climb to row 6 at $t = 11$, east to (13, 6) at 18, down to the goal at 20. Re-check: no conflict.
10.0	A: Reconnecting → Nominal	all drones Nominal.
10, 18, 20	—	arrivals of B, C, A (nominal 9, 13, 13): SoC = 48, MS = 20.

In two of the 201 cycles the ORCA program was infeasible and the dense fallback of [Chapter 13](#) chose the least violated velocity. The price of safety is time: thirteen extra steps of SoC, over half of them A's detour along row 6, the rest C's descent to row 1 and B's extra wait. Whether a local-only or a replan-only policy would have paid less, or collided, is the question of the benchmark of [Chapter 25](#). The hybrid policy's answer on this instance is no collision, one re-check that mattered, and nothing global recomputed.

24.11 What guarantees survive

The composed system inherits guarantees from its parts, but only where the parts' assumptions still hold: what survives, the one result that belongs to the integration itself, and what is lost.

Proposition 24.5 (The nominal plan). *The plan $\Pi = (\pi_1, \dots, \pi_k)$ delivered by the global layer is free of vertex and swapping conflicts. It is optimal in the sum of costs if computed by CBS [150] (Chapter 9), and within a factor w of optimal if computed by ECBS [8] (Chapter 10). These statements hold on the grid; the next proposition carries them to the physical drones.*

Proposition 24.6 (Physical separation of a conflict-free plan). *Let Π be a plan on a 4-connected grid of cell side ℓ without vertex and swapping conflicts, executed by drones that*

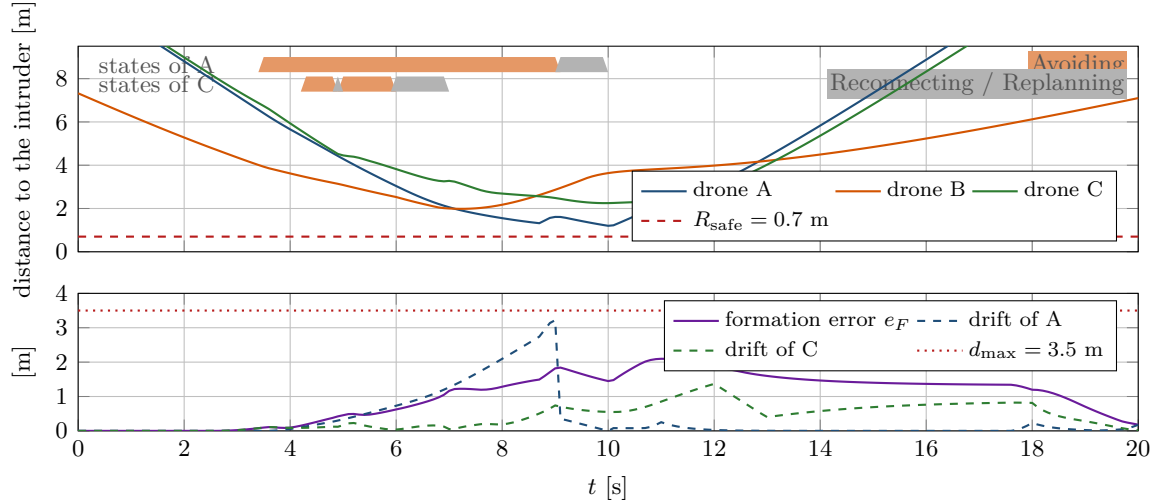


Figure 24.5. The run in numbers. Top: distance of each drone to the intruder, with the states of A and C as bands (orange: Avoiding; gray: Reconnecting or Replanning). What to notice: A's distance bottoms out at 1.20m, well above $R_{\text{safe}} = 0.7\text{m}$, while it is Avoiding; C never comes closer than 2.25m although it too triggered, because the trigger fires on the inflated prediction, not on the geometry. Bottom: the formation error grows while A avoids and C is repaired, peaks at 2.10m at $t = 11\text{s}$ and decays once A is Nominal again; A's drift stays below d_{max} and is reset to zero by the replan.

fly the reference Equation (24.1) exactly. Then at every instant every two drones are at least $\ell/\sqrt{2}$ apart, and the bound is attained. In particular the execution is collision-free whenever $r_i + r_j \leq \ell/\sqrt{2}$ for every pair.

Proof. Fix a pair and a grid step $[\tau, \tau + 1]$, and measure positions in cells. The relative position is $\Delta(u) = \Delta_0 + u(\Delta_1 - \Delta_0)$ for $u \in [0, 1]$, where $\Delta_0, \Delta_1 \in \mathbb{Z}^2$ are the relative cell offsets at τ and $\tau + 1$. Both are non-zero because there is no vertex conflict. The difference $\Delta_1 - \Delta_0$ is the difference of two moves, each $\mathbf{0}$ or a unit axis vector, so its length is 0, 1, $\sqrt{2}$ or 2. The norm of $\Delta(u)$ is convex in u . If its minimum is at an endpoint it is at least 1. Otherwise the minimum is the distance from the origin to the line through Δ_0 and Δ_1 , which is $|\Delta_0 \times \Delta_1| / \|\Delta_1 - \Delta_0\|$. The cross product of two integer vectors is an integer. If it is zero, the line passes through the origin and the interior minimum would be zero. That requires Δ_0 and Δ_1 on opposite sides of the origin at distance at most 2 apart, that is, $\Delta_1 = -\Delta_0$ with

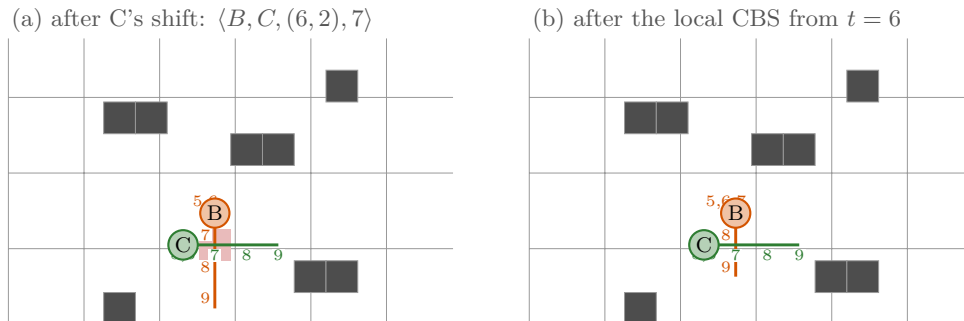


Figure 24.6. The conflict re-check at $t = 4.9\text{s}$ (grid times 5 to 9 written next to the cells). (a) C's delayed reconnection shifted its path by one step, and the shifted path meets B's plan at $(6, 2)$ at $t = 7$: a vertex conflict that the detector of Chapter 7 reports. (b) The local CBS on $\{B, C\}$ from $t_{\text{start}} = 6$, with A's path reserved, adds one wait to B and leaves C alone; the plan in force is conflict-free again and A never knew.

Δ_0 a unit vector. This is the swapping conflict, which is excluded, so in this case the minimum is at an endpoint. If the cross product is non-zero, its absolute value is at least 1. When the moves are opposite along one axis, $\Delta_1 - \Delta_0 = \pm 2\mathbf{e}$ and $|\Delta_0 \times \Delta_1| = 2|\Delta_0 \times \mathbf{e}| \geq 2$, giving a distance of at least 1. When the moves are perpendicular, $\|\Delta_1 - \Delta_0\| = \sqrt{2}$ and the distance is at least $1/\sqrt{2}$. When one drone waits, the length is at most 1 and the distance at least 1. The bound $1/\sqrt{2}$ is attained when a drone enters, along one axis, the cell that its neighbor leaves along the other axis: $\Delta_0 = (1, 0)$, $\Delta_1 = (0, 1)$, closest at $u = \frac{1}{2}$. Multiplying by ℓ gives the claim. $\square$

In the scenario $R_{\text{mate}} = 0.70 \leq 1/\sqrt{2} = 0.707$ m, so the nominal plan is physically safe with seven millimeters to spare. The local layer's reciprocal half-planes use exactly R_{mate} . They add nothing while the drones are on their plan, and everything once one of them is not.

Proposition 24.7 (The local layer). *While a drone is Avoiding, the pairwise guarantee of ORCA [14] from Chapter 13 applies. Suppose every drone builds its half-planes from the same snapshot with the same horizon τ and holds the chosen velocity constant over τ . Suppose also that every drone is velocity-controlled (a single integrator, as in the scenario, which a real quadrotor is not) and that every program is feasible. Then no two drones come closer than R_{mate} within τ . Suppose in addition that the intruder's true position stays within the p -disc of its prediction, and its velocity within the predicted one. Then the drone, which takes full responsibility, stays at least R_{safe} from it within τ with probability at least p per step. Nothing is guaranteed when a program is infeasible, which the executive treats as a replanning trigger.*

Proposition 24.8 (Replanning restores conflict-freedom against known agents). *Suppose Algorithm 24.4 returns success for drone a_i at time t without reaching line 13. Then the plans in force, $(\pi_j, t_0^j)_{j=1}^k$, are free of vertex and swapping conflicts from grid time $\lfloor t/\Delta T \rfloor$ on.*

Proof sketch. The search on line 6 rejects every state and move in the reservation R (Chapter 4), so π'_i has no conflict with the reserved paths as they were when R was built. The padded detector on line 9 examines every pair at every grid time from $\lfloor t/\Delta T \rfloor$ to the common horizon (Chapter 7), so any conflict that survives, including one created by a teammate's simultaneous change, is found. If one is found between a_i and a_j , the local CBS of line 12 searches with the reservation R' of all other drones as low-level constraints. By the completeness of CBS (Chapter 9) it returns a pair of paths that is conflict-free between a_i and a_j . Because R' is respected at the low level, that pair is conflict-free against everyone else too. A conflict between the repaired pair and a third drone is impossible by construction, because every non-pair path is reserved in R' . Hence no conflict remains. $\square$

The proposition is about plans, not about drones in flight: a teammate that is itself Avoiding is not on its plan, and the physical separation during that interval rests on Proposition 24.7. It is also silent about optimality, because the replanned path is optimal only for a_i under the others' reservations, the same one-agent-at-a-time quality as prioritized planning (Chapter 8).

What is not guaranteed.

- *Deadlock-freedom.* ORCA deadlocks in symmetric and corridor configurations (Chapter 13); t_{avoid} turns a deadlock into a replan, but the replan may find no path and the drone then hovers and retries. Nothing forces the escalation to succeed.
- *Optimality after a deviation.* Reconnections add delays, replans are single-agent optima, partial replans are optimal for the pair only; in the scenario the sum of costs rose by 37%.
- *Safety against an adversarial or maneuvering intruder.* Every layer assumes the intruder stays inside the p -disc of a prediction whose covariance grows with the modeled process

Table 24.5. Guarantees of the hybrid system, their assumptions, and how the evaluation of Chapter 25 measures what happens when the assumptions do not hold.

Claim	Assumption	Measured by
nominal plan conflict-free and (bounded-)optimal (Proposition 24.5)	static map, no intruder, drones on their references	sum of costs, makespan versus the nominal values
physical separation $\geq \ell/\sqrt{2}$ on the plan (Proposition 24.6)	exact tracking of Equation (24.1)	minimum separation between drones
pairwise safety for τ while Avoiding (Proposition 24.7)	common snapshot, feasible programs, intruder inside its p -disc	collision rate, minimum separation, count of infeasible cycles
conflict-free plans after a replan (Proposition 24.8)	the local CBS succeeds within its budget	collision rate between drones, replanning count, computation time
no deadlock; arrival	— (not guaranteed)	travel time, fraction of runs that time out
formation and link recovery	connected formation graph, soft range term	formation error, communication violations

noise; an intruder that turns or accelerates toward a drone faster than that defeats every finite horizon, and the risk is bounded only per step and per intruder.

- *Completeness of the replanner* within the horizon: blocked layers can disconnect a corridor; soft layers keep the search complete at the price of crossing the tube.
- *Formation and connectivity.* The range term is soft and the formation is deliberately relaxed while Avoiding; recovery has no bound during a maneuver.

Table 24.5 pairs every claim with the assumption it needs and with the metric of Chapter 25 that measures the gap when the assumption fails. The experiment matrix there varies exactly these assumptions: prediction quality, intruder behavior, number of drones, constraints.

24.12 Implementation architecture

Modules and interfaces. The code of this chapter is one file so that it can be read in one sitting. It is nevertheless organized as the modules of Table 24.1, each behind the interface the table names: `cbs`, `IntruderTracker` (`update`, `predict_horizon`), `orca_half_plane` and `orca_velocity`, `space_time_astar` over a `Reservation`, and `HybridSimulation`.`decide` for the executive. A real system replaces each module by the full implementation of its chapter without touching the executive, provided the interfaces are kept.

Threads and rates. Three loops run at three rates. The *control loop* runs the executive and the local layer once per Δt for every drone and must never block. The *prediction loop* runs one filter per tracked object at the sensor rate and publishes a snapshot (means and covariances over the horizon) that the control loop reads without waiting. The *planning loop* runs the replanner and the local CBS on demand in a worker with a deadline. The executive stays in Replanning, hovering under the local layer, until the result arrives. An anytime planner (Chapter 6) is the right choice when the deadline is tight. Within one cycle all drones decide from one snapshot of everybody’s state and the commands are applied afterward, so the outcome does not depend on the processing order. Listing 24.1 is the schedule of the simulation with the interface map of its modules.

Listing 24.1. The module map and the rate schedule (from `code/ch24_hybrid.py`). Each module runs when its period has elapsed; the executive asks the schedule once per control cycle.

```

1  #   module (layer)           interface                               rate
2  #   global planner (1)      cbs(grid, starts, goals)                once, before take-off
3  #   tracker (2)             update(z); predict_horizon(n)          period_prediction
4  #   local layer (3)         local_layer(d, v_pref, pred)           every cycle DT
5  #   replanner (4)           replan(d, pred); recheck(d)            on demand, >=
6  #                               replan_period
7
6  #   executive               decide(d, pred) -> velocity           every cycle DT
7
8
9  class RateSchedule:
10     """A module with period T runs in the first cycle at or after its due
11     time; the executive asks ``due(name, t)`` once per cycle."""
12
13     def __init__(self, periods: Dict[str, float]):
14         self.periods = dict(periods)
15         self.next_due = {name: 0.0 for name in periods}
16
17     def due(self, name: str, t: float) -> bool:
18         if t + EPS < self.next_due[name]:
19             return False
20         self.next_due[name] = t + self.periods[name]
21         return True

```

The executive in code. Listing 24.2 is the control cycle of Algorithm 24.1 line by line. In the file, `conflict_profile` and `predicted_conflict` implement Algorithm 24.2, `reconnect` and `segment_free` implement Algorithm 24.3 and `try_reconnect` installs their result, and `replan` and `recheck` complete Algorithm 24.4.

Listing 24.2. The per-drone control cycle (from `code/ch24_hybrid.py`), the state machine of Algorithm 24.1.

```

1  def decide(self, d: Drone, pred) -> np.ndarray:
2     """One pass of the per-drone control loop; returns the velocity."""
3     p = self.p
4     inside, t_c, _ = self.predicted_conflict(d, pred)
5     if d.state == NOMINAL and inside:
6         self.transition(d, AVOIDING, "predicted conflict at t_c=%.1f s" % t_c)
7     if d.state == AVOIDING:
8         drift = float(np.linalg.norm(d.pos - d.plan_position(self.t)))
9         too_long = self.t - d.t_state > p["t_avoid_max"]
10        d.clear_count = 0 if inside else d.clear_count + 1
11        if drift > p["drift_max"] or too_long:
12            self.transition(d, REPLANNING, "drift %.2f m or time" % drift)
13        elif d.clear_count >= p["n_clear"]:
14            self.transition(d, RECONNECTING, "conflict cleared")
15            reason = self.try_reconnect(d, pred)
16            if reason != "ok":
17                self.transition(d, REPLANNING, "reconnection failed: " + reason)
18        else:
19            v_pref = self.preferred_velocity(d, with_formation=False)
20            return self.local_layer(d, v_pref, pred, t_c, with_intruder=True)[0]
21    if d.state == REPLANNING:
22        due = self.t - d.t_last_replan >= p["replan_period"]
23        if due and self.replan(d, pred):

```



```

24         d.t_last_replan = self.t
25         self.recheck(d)
26         self.transition(d, RECONNECTING, "new path from the current cell")
27         inside, t_c, _ = self.predicted_conflict(d, pred)
28     else:                                     # no path yet: hover, retry
29         return self.local_layer(d, np.zeros(2), pred, t_c, True)[0]
30     if d.state == RECONNECTING:
31         if inside:
32             self.transition(d, AVOIDING, "conflict again, t_c=%.1f s" % t_c)
33             v_pref = self.preferred_velocity(d, with_formation=False)
34             return self.local_layer(d, v_pref, pred, t_c, with_intruder=True)[0]
35         if self.t >= d.target_time - DT / 2:
36             self.transition(d, NOMINAL, "back on the plan at waypoint %d" %
d.target_index)
37         v_pref = self.preferred_velocity(d, with_formation=True)
38         return self.local_layer(d, v_pref, pred, t_c, with_intruder=False)[0]

```

Letting the local layer run unbounded

ORCA has no memory and no goal. Left in charge, it will happily carry a drone along an intruder's flank for a minute, ten meters from a plan that the reservation table still believes it is flying. The other drones' guarantees then quietly evaporate. Bound the Avoiding state twice, by drift and by time, and escalate to a replan when either bound is hit. Those two bounds are the parameters of [Table 24.3](#) that most affect the collision rate between teammates in the experiments of [Chapter 25](#).

Replanning too often

A replan is cheap; its consequences are not. Every replan changes a path in force, every change triggers a re-check, and every re-check may change a second path and send a second drone into Reconnecting. With several intruders the swarm can spend its time re-planning around its own re-plans. Replan only on the triggers of [Table 24.2](#), never because a shorter path has appeared; rate-limit replans per drone (T_{replan}); and give the trigger and the Avoiding exit hysteresis (n_{clear}). The replanning count is a metric of [Chapter 25](#) for exactly this reason.

Mixing frames and time bases

The four layers use four coordinate conventions: cell indices, cell centers offset by $\ell/2$, world-frame positions and the relative frame of ORCA. They also use three time bases: grid time in steps of ΔT , control time in steps of Δt , and the prediction step h counted from the latest observation. Every conversion belongs in one place, the world model of [Section 24.3](#), and every function should say which convention it expects. The symptoms of a mix-up are subtle: a drone aiming half a cell beside its waypoint, a prediction one step stale, a layer $\mathcal{B}_t$ built for the wrong second.

24.13 Research directions

The components of this chapter are established; the training plan places the research in how they are combined, constrained, predicted and evaluated. Each direction comes with a formulation and a way to evaluate it.

Non-cooperative, uncertain intruders. Replace the Gaussian around a constant-velocity prediction by a *set*. The positions reachable under a bound a_B on the intruder’s acceleration form a tube that grows quadratically, and a robust trigger fires when the tube meets the intended path. A second option is a mixture over maneuver modes (straight, left, right) from the interacting-model filter of Chapter 19, with one disk per mode. Evaluate on intruders of increasing hostility, constant velocity, random turns, an adversary steering toward the nearest drone at a_B , and report collision rate and minimum separation against a_B , and the path length that robustness costs.

Coordinating CBS with real-time local avoidance without losing the global guarantees unnecessarily. Every deviation costs a re-check and every delayed reconnection may cost a partial replan. Plan with slack instead. Let the global layer reserve a tube of cells around each path and a time window around each step, so that a deviation inside the tube and a delay within the window need no re-check. This is a robust MAPF problem, solvable by CBS with enlarged constraints at a known price in cost. Measure re-checks, partial replans and SoC as functions of the slack, and find where the re-checks vanish for a given intruder density.

Formation and communication constraints during avoidance and replanning. Section 24.8 relaxes the formation and tests the range after the fact. Keep both inside the decision instead, with the multi-drone MPC of Chapter 21 and prioritized slacks: safety, then range, then formation. Keep them one level up as well, with a formation-level replan in which the global layer routes the virtual leader of Chapter 23 through the layers $\mathcal{B}_l$ with the formation’s footprint. Evaluate by the recovery time of e_F , the communication violations and the minimum separation for formations of 4 to 16 drones with one or two intruders.

Prediction-aware constraints with uncertainty. The per-step risk $1 - p$ is allocated uniformly here. Allocate it across steps and intruders instead, calibrate the predictors so that their ellipses hold their coverage, and use the particle tests of Section 24.9. Evaluate all three by the trade-off curve between collision rate and path length as p varies, for the Kalman prediction and for a learned predictor of Chapter 20. Report the calibration error of each as well.

Deciding automatically between local and global. The thresholds $d_{\max}$, t_{avoid} , n_{clear} and τ_h are set by hand. Formulate the choice among *continue avoiding*, *reconnect* and *replan* as a decision with an estimated cost for each. The three costs are the expected extra flight time of staying local, the delay of a reconnection, and the cost of a replan plus its expected re-checks. Estimate them from the prediction (how long the tube covers the lane) or learn them from runs. Evaluate by the regret against an oracle that knows the intruder’s future, on the scenario generator of Chapter 25.

Computation for larger swarms in dynamic 3D environments. CBS scales exponentially in the conflicts, the detector quadratically in the drones, ORCA linearly in the neighbors, the replanner with the horizon and the layers. For tens of drones in three dimensions, combine ECBS with independence detection (Chapters 10 and 11), windowed replanning as in WHCA* (Chapter 8) and incremental search (Chapter 5) for layers that change every cycle. Add lattices or sampling (Chapters 16 and 17) for continuous detours, and neighbor lists for the local layer. Report the computation time per control cycle against the number of drones (2 to 64), and the fraction of cycles that miss the deadline.

Benchmarks comparing local-only, replan-only and hybrid. The claim that the hybrid policy beats either extreme must be measured under identical scenarios, seeds, intruders and prediction quality. The three policies to compare are a local-only policy (ORCA for everything), a replan-only policy (every intruder triggers a replan) and the hybrid of this chapter. [Chapter 25](#) defines the experiment matrix and the metrics. The research contribution is a scenario generator that spans easy to hard instances, and statistics that make the comparison reproducible.

24.14 Where this fits in the drone system

In the drone system

This chapter *is* the drone system: the executive that the dronebox of every other chapter of the book has been pointing to. Grouped by the layer they feed, those chapters are the following. *Global planner*: [Chapter 3](#) for the distance tables, [Chapters 7 to 11](#) for the nominal plan, [Chapter 22](#) for its offline optimum. *Prediction layer*: [Chapters 18 to 20](#). *Local safety layer*: [Chapters 12 to 14](#), with [Chapter 15](#) as the cheaper reactive alternative it warns against. *Replanning layer*: [Chapters 4 to 6](#) on the grid and [Chapters 16 and 17](#) in continuous space. *Across all four*: [Chapter 2](#) for the geometry, grids and uncertainty, [Chapter 21](#) for the tracker that enforces $a_{\max}$ and the constraints, and [Chapter 23](#) for formation and communication. [Chapter 1](#) states the interfaces and [Chapter 25](#) measures the result. The executive receives the nominal plan from CBS or ECBS, the predictions from the tracker and the predictor, one velocity per cycle from ORCA or DWA, and repaired paths from D^* Lite or A^* . It delivers the decisions: which layer is in charge, when to hand over, reconnect, replan and re-check. In the study plan ([Appendix A](#)) this is Week 12, the capstone: the coding exercise is [Exercise 24.8](#), and its milestone is a research prototype with an experiment matrix suitable for a thesis chapter.

24.15 Summary

Chapter summary

- Four layers, one executive: CBS/ECBS plan the contract before take-off; the tracker and predictor deliver $\hat{\mathbf{p}}_{T+h}$, Σ_h ; ORCA or DWA chooses a safe velocity for seconds; D^* Lite or A^* replans from the current cell; the executive decides which is in charge, ten times a second, from one world model with one clock and one frame, which converts paths into references ([Equations \(24.1\) and \(24.2\)](#)) and states into space-time starts, and stores every path with its start time t_0 .
- The state machine Nominal / Avoiding / Reconnecting / Replanning has conditions on every edge ([Table 24.2](#)); Avoiding is bounded by drift and time; Reconnecting is the only way back.
- A conflict is inside the safety horizon when the predicted separation minus the inflation $\kappa_p \sigma(h)$ drops below R_{safe} within τ_h ([Definition 24.2](#)); $\tau_h \geq \tau$ and $\tau_h \geq \Delta t + v_{\max}/a_{\max}$.
- Reconnection searches waypoints forward with delay zero first; a delayed reconnection shifts the reservation and needs a re-check. Replanning searches the time-expanded grid with the others' paths reserved and the predicted cells blocked per layer, and is followed by the detector of [Chapter 7](#) and, if needed, a local CBS on the pair.
- Formation terms are dropped while Avoiding and restored while Reconnecting; the range term stays; candidates that break a link are rejected. Uncertainty enters as

the p -disc radius $\kappa_p \sqrt{\lambda_{\max}(\Sigma_h)}$, a per-step chance constraint; particles replace the disk by a count.

- What survives: a conflict-free, (bounded-)optimal nominal plan with physical separation $\ell/\sqrt{2}$; ORCA's pairwise safety for τ ; conflict-free plans after every replan. What does not: deadlock-freedom, optimality after deviations, safety against an adversarial intruder. In the worked scenario the intruder never came closer than 1.20 m, and the swarm paid 13 steps of sum of costs.

Further reading. The components: CBS [150] and ECBS [8] for the global layer, with the MAPF definitions of Stern et al. [161]; ORCA [14] and DWA [46] for the local layer; D* Lite [88] for replanning; the Kalman filter [76] and social LSTM prediction [1] for the prediction layer; chance constraints under uncertain predictions [187]; consensus and formation control [123]. Turning grid plans into flyable quadrotor trajectories, the step that Equation (24.1) takes in its simplest form, is treated by Hönig et al. [67].

24.16 Exercises

Exercise 24.1 (★). A drone has $v_{\max} = 2$ m/s and $a_{\max} = 2$ m/s², the local layer uses $\tau = 2$ s and the control cycle is $\Delta t = 0.05$ s. (a) Give the smallest safety horizon that satisfies both conditions of Equation (24.5) in the stop form and in the reversal form, and the constraint from τ . (b) The intruder flies at up to 3 m/s and the tracker needs $n_{\text{warm}} = 20$ updates; what detection range does the sensor need for $\tau_h = 2$ s if the drone cruises at $v_{\text{cruise}} = 1.5$ m/s, $R_{\text{safe}} = 1$ m and $\kappa_p \sigma(\tau_h) = 0.5$ m? (c) Explain what goes wrong when $\tau_h < \tau$, and what goes wrong when τ_h is much larger than the look-ahead at which $\kappa_p \sigma$ reaches the lane spacing.

Exercise 24.2 (★★). Write the safety-horizon trigger for a prediction given as N weighted particles $(\mathbf{p}_h^{(n)}, w^{(n)})$: the trigger fires at look-ahead s when the weight of the particles within R_{safe} of $\tilde{\mathbf{p}}_i(s)$ exceeds $1 - p$. Construct a bimodal prediction: an intruder that passes a building on either side with equal probability. Compare, for $p = 0.95$, the look-ahead at which the particle trigger fires with the look-ahead at which the Gaussian trigger of Definition 24.2 fires when the particles are summarized by one mean and one covariance. Which one blocks the lane between the two modes, and why?

Exercise 24.3 (★★). (a) Prove that a reconnection (j, Δ) returned by Algorithm 24.3 increases the drone's arrival time by exactly Δ grid steps and the sum of costs of the swarm by at most Δ , provided the re-check finds no conflict. (b) Prove that with $\Delta = 0$ no re-check is needed, stating exactly which lines of the algorithm and which property of the plan in force you use. (c) Reproduce with the code the delayed reconnection of drone C at $t = 4.9$ s in the worked scenario. Show, by calling `first_conflict` on the shifted paths, that the shift creates the conflict $\langle B, C, (6, 2), 7 \rangle$. Then explain why Algorithm 24.3's teammate test (line 14) did not prevent it.

Exercise 24.4 (★). In the worked scenario drone A's replanned path starts with (6, 5) at $t_0 = 9$ and drone B's nominal path passes (6, 5) at $t = 3$. Show that a conflict detector started at grid time 0 over the padded paths reports the false conflict $\langle A, B, (6, 5), 3 \rangle$, and that the same detector started at $\lfloor t \rfloor = 9$ reports none. Then list every place in Algorithms 24.1, 24.3 and 24.4 where a start time t_0 is read or written, and check that each one is in absolute grid time.

Exercise 24.5 (★★). Extend the state machine in two ways and draw the new diagram. (a) Make the layers *expensive* instead of *blocked*: a cell in $\mathcal{B}_t$ costs $c_B > 1$. Which guarantee of Section 24.11 changes, and how should c_B depend on $\kappa_p \sigma(h)$? (b) Add an *Emergency* state,

entered when the local program has been infeasible for m consecutive cycles, in which the drone brakes or climbs, and specify its exits. Implement (a) in `ch24_hybrid.py` and report the change in A's replanned path and arrival time.

Exercise 24.6 (★★). In the worked scenario C is a follower of A. (a) Explain, using Equation (24.6), why C triggered at $t = 4.3\text{s}$ although the intruder never came closer to it than 2.25 m. Say also what would have happened had the formation term stayed active while A was Avoiding. (b) Run the code with $k_F \in \{0, 0.5, 1.0\}$ and with the gain ramped over one second when Reconnecting begins; report the peak and recovery time of e_F and the minimum separations. (c) Propose a rule for a formation whose *leader* is the drone that must avoid.

Exercise 24.7 (★★). Replace ORCA by the dynamic window approach of Chapter 14 in the local layer. Specify the interface. Say what the rollouts receive from the prediction layer (predicted intruder velocity, inflated radius per rollout step), where the heading term points while Avoiding and while Reconnecting, and which DWA output replaces ORCA's feasibility flag in Algorithm 24.1. Implement it for a double-integrator drone with $a_{\max} = 1\text{ m/s}^2$ and compare, on the worked scenario, the minimum separation, the time in Avoiding and the number of replans.

Exercise 24.8 (★★★ Coding: the capstone). Build the hybrid prototype of Week 12 of the study plan and run controlled experiments with it. (a) Assemble the four layers from your own implementations of the earlier weeks behind the interfaces of Table 24.1, and reproduce the trace of Table 24.4 with `ch24_hybrid.py` as the reference. (b) Replace the fresh space-time A^* of the replanner by D^* Lite (Chapter 5) driven by the changing layers $\mathcal{B}_t$, and compare the expansions per replan over a run with several intruders. (c) Add the communication and formation tests of Section 24.8 to the reconnection and to the re-check, and a scenario in which they reject a reconnection. (d) Implement the local-only and replan-only policies as two more executives with the same interfaces. (e) Run the experiment matrix of Chapter 25: 2, 4 and 8 drones; 0, 1 and 2 intruders; perfect, noisy and constant-velocity predictions; the three policies; with and without formation and range constraints; at least 20 seeds per cell. Report the ten metrics of Chapter 25 with confidence intervals, and state in one page which policy wins where and why.

Evaluating a Hybrid Planner: Experiments, Metrics and Benchmarks

Learning objectives

- Define the ten metrics of the training plan precisely, with a formula, a unit, an estimator over runs and a list of what each one is sensitive to, and sort them into safety, efficiency, effort and constraint metrics.
- Turn the five factors of the plan into an experiment matrix, choose between a full factorial and a fractional design, and name the baselines that every comparison needs.
- Generate controlled scenarios from seeds, with intruder trajectory families and difficulty knobs, so that every strategy sees exactly the same scenarios.
- Choose the right statistics: how many runs, mean or median, confidence intervals, paired comparisons, effect sizes, success rates under a cap, and the three plots that show them.
- Run a small paired study with the book's code, read its table and figures cautiously, and write a results section that a reviewer can reproduce and trust.

25.1 Why this matters for a drone swarm

Every algorithm in this book is established: A^* , D^* Lite, CBS and ECBS, ORCA, DWA, the Kalman filter and MPC each have their original paper, their proofs and their decades of use, and the training plan says plainly that your contribution will not be a new planner built from zero. It will be a *combination*, the hybrid architecture of [Chapter 24](#), and a combination has no theorem that says it is better than its parts. The only way to know whether the hybrid planner is safer than local avoidance alone, cheaper than replanning everything, or whether a learned predictor earns its cost, is to measure it — in a way that another researcher can repeat and believe.

This is where research contributions are made or lost. A reviewer who reads that “the hybrid planner reduced collisions by 40 %” will ask: on which scenarios, against which baselines, over how many runs, with what spread, and did the baselines see the same scenarios? If the answers are missing, the claim is worth nothing, however good the code. Conversely, a modest result reported with its scenarios, seeds, intervals and failure cases is a result that can be built on. The last row of the training plan, Week 12 ([Appendix A](#)), asks for exactly this: “a research prototype and an experiment matrix suitable for a paper or thesis chapter”.

The chapter proceeds as follows. [Section 25.2](#) shows the evaluation pipeline in one

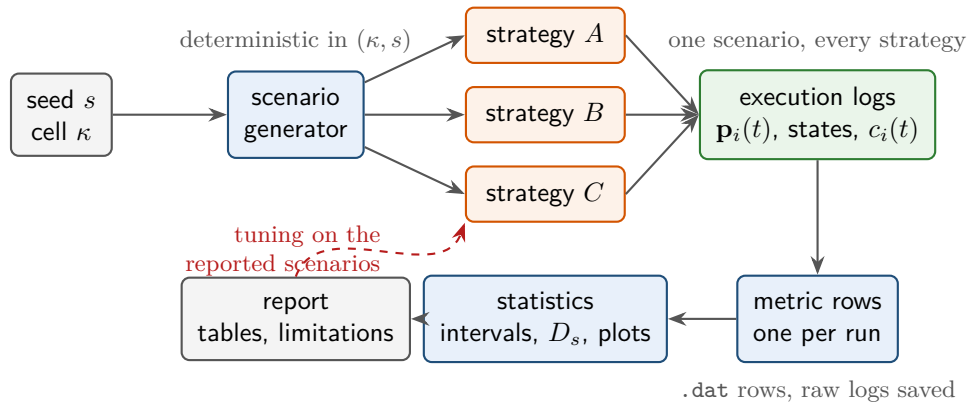


Figure 25.1. The evaluation pipeline. The seed and the cell of the experiment matrix fix the scenario; every strategy runs on the same scenario (paired design); logs become metric rows, rows become estimates with intervals, and only the last box is prose. The dashed arrow is the one to avoid: tuning a strategy on the scenarios that will be reported.

picture; [Section 25.3](#) defines the metrics, [Section 25.4](#) the experiment matrix and the baselines, [Section 25.5](#) the scenario generator and the paired design, and [Section 25.6](#) the statistics and the plots, each shown on data from the book’s own toy simulator. [Section 25.7](#) runs a complete mini-study and interprets it; [Sections 25.8 to 25.10](#) say what to save, what to write and which benchmarks and simulators to connect to, and [Section 25.11](#) discusses the code. This chapter has no theorem about a planner; its two small propositions are about experiments.

25.2 The idea in plain words

An evaluation is a pipeline ([Figure 25.1](#)). A **scenario generator** turns a seed and a cell of the experiment matrix into a concrete world: map, starts, goals, intruder trajectories, even the measurement noise the predictor will see. A **runner** executes every strategy on that same scenario and writes a log of positions, decisions and timings. **Metric functions** reduce each log to a row of numbers. **Statistics** reduce the rows of a cell to estimates with intervals and compare strategies *pairwise*, *scenario by scenario*. Only then comes the **report**: tables, three kinds of plots, and the limitations. Nothing in the pipeline is clever; what makes it scientific is that every arrow is deterministic given the seed, so that the whole picture can be regenerated by anyone who has the code and the configuration file.

Key idea

Fix the scenarios by seeds, run every strategy on the same scenarios, report every metric with its spread and its failures, and let the seeds regenerate the whole study. A difference you cannot show on paired runs with a confidence interval is not a result yet.

25.3 Metrics: what to measure and how

25.3.1 The execution log

All metrics are functions of one object, the log of a run.

Definition 25.1 (Execution log). A **run** executes one scenario under one strategy for $T + 1$ sampling instants $t = 0, 1, \dots, T$ of length Δt . Its **execution log** records four things for every t . The first two are positions: $\mathbf{p}_i(t) \in \mathbb{R}^2$ (or $\mathbb{R}^3$) of the k controlled drones $i = 1, \dots, k$, and

$\mathbf{q}_j(t)$ of the m intruders $j = 1, \dots, m$. The third is the state of each drone's state machine (Nominal, Avoiding, Reconnecting or Replanning, [Chapter 24](#)) with the computation time $c_i(t)$ of the decision taken for drone i at t . The fourth is the events of the run, in particular every replan. It also stores the nominal plan $\mathbf{p}_i^{\text{nom}}(t)$ of every drone, the goals $\mathbf{g}_i$ and the radii r_i , so that the metrics can be recomputed later from the log alone.

The separation of two agents at time t is $d_{ij}(t) = \|\mathbf{p}_i(t) - \mathbf{p}_j(t)\|$ as in [Chapter 2](#), and two agents of radii r_i, r_j collide when $d_{ij}(t) < R = r_i + r_j$. Positions are sampled, so a collision between two samples can be missed. The code of this chapter therefore evaluates the minimum of d_{ij} on every interval $[t, t + 1]$ under the assumption of linear motion, using the closest-approach formula of [Chapter 2](#). Write $\mathbf{r}_0 = \mathbf{p}_i(t) - \mathbf{p}_j(t)$ and $\mathbf{w} = \mathbf{p}_i(t + 1) - \mathbf{p}_j(t + 1) - \mathbf{r}_0$. The minimum over the interval is then $\|\mathbf{r}_0 + s^* \mathbf{w}\|$ with $s^* = \text{clip}(-\mathbf{r}_0 \cdot \mathbf{w} / \|\mathbf{w}\|^2, 0, 1)$. If $\mathbf{w} = \mathbf{0}$ the two agents move identically over the interval, the quotient is undefined and the minimum is $\|\mathbf{r}_0\|$. The code takes that branch, and [Exercise 25.2](#) needs it. With $\Delta t = 0.1$ s and relative speeds up to 4 m/s a sampled minimum can be off by 0.2 m, a third of the collision distance of two 0.3 m drones.

25.3.2 Four families of metrics

The plan lists ten metrics. They answer four different questions, and a results table should group them that way. Did anything go wrong (**safety**)? How much did avoidance cost the mission (**efficiency**)? How hard did the planner work (**effort**)? Were the swarm constraints respected (**constraints**)?

Definition 25.2 (Safety metrics). The **collision indicator** of a run is $X_{\text{coll}} = 1$ if some pair of agents (drone–drone or drone–intruder) has $d_{ij}(t) < r_i + r_j$ at some time, and 0 otherwise; the **collision count** n_{coll} is the number of such pairs. The **collision rate** of a cell is the fraction of its N runs with $X_{\text{coll}} = 1$,

$$\hat{p}_{\text{coll}} = \frac{1}{N} \sum_{s=1}^N X_{\text{coll}}^{(s)}. \quad (25.1)$$

The **minimum separation** of a run is

$$d_{\min} = \min_t \min_{i \neq j} d_{ij}(t), \quad (25.2)$$

reported separately for drone–drone pairs ($d_{\min}^{\text{dd}}$) and drone–intruder pairs ($d_{\min}^{\text{di}}$), in meters. A run is a **near miss** if $d_{\min} < R_{\text{safe}}$, the safety radius the local safety layer of [Chapter 24](#) tries to keep.

Definition 25.3 (Efficiency metrics). The **path length** of drone i is the flown length $L_i = \sum_{t=0}^{T-1} \|\mathbf{p}_i(t+1) - \mathbf{p}_i(t)\|$ in meters, and its **path-length ratio** is $\rho_i = L_i / L_i^{\text{nom}}$, where L_i^{nom} is the length of the nominal plan; a run reports the mean ratio $\rho_L = \frac{1}{k} \sum_i \rho_i$. The **travel time** of drone i is the first instant after which it stays within the goal tolerance ε_g of $\mathbf{g}_i$,

$$T_i = \Delta t \cdot \min \{ \tau : \|\mathbf{p}_i(t) - \mathbf{g}_i\| \leq \varepsilon_g \text{ for all } t \geq \tau \}, \quad (25.3)$$

in seconds, and $T_i = \infty$ if the drone is not at its goal at the end of the log. The **makespan** and the **sum of costs** of the run are

$$\text{MS} = \max_i T_i, \quad \text{SoC} = \sum_{i=1}^k T_i, \quad (25.4)$$

the continuous-time forms of [Chapter 7](#): a MAPF path cost of c steps is a travel time of $c \Delta t$ seconds, and trailing waits at the goal are free in both.

Definition 25.4 (Effort metrics). The **replanning count** n_{rp} is the number of entries into the Replanning state summed over the drones of a run; the number of steps spent in Avoiding or Reconnecting, n_{av} , is reported next to it. The **computation time** is recorded per decision, $c_i(t)$ in milliseconds, and summarized by its mean $\bar{c}$, its 99th percentile c_{99} (the deadline behavior), its maximum, the number of **deadline misses** $|\{(i, t) : c_i(t) > \Delta t\}|$, and the total $C_{tot} = \sum_{i,t} c_i(t)$ per run in seconds.

Definition 25.5 (Constraint metrics). The **formation error** at time t is $e_F(t)$ of Chapter 23: the root-mean-square violation of the desired displacements $\mathbf{d}_{ij}(t)$ over the edges of the formation graph G_F ,

$$e_F(t) = \sqrt{\frac{1}{|E_F|} \sum_{\{i,j\} \in E_F} \|\mathbf{p}_j(t) - \mathbf{p}_i(t) - \mathbf{d}_{ij}(t)\|^2}, \quad (25.5)$$

where the template is either a fixed formation or, for a coordinated mission, the relative geometry of the nominal plan, $\mathbf{d}_{ij}(t) = \mathbf{p}_j^{\text{nom}}(t) - \mathbf{p}_i^{\text{nom}}(t)$. A run reports the time average $\bar{e}_F$ and the peak e_F^{max} , in meters. The **communication violations** of a run are the number of time steps at which a kept edge is longer than the communication range or the radius graph of Chapter 23 is disconnected,

$$n_{cv} = \left| \left\{ t : \exists \{i, j\} \in E_{\text{keep}} \text{ with } d_{ij}(t) > R_{\text{comm}}, \text{ or } \lambda_2(\mathbf{L}(t)) = 0 \right\} \right|, \quad (25.6)$$

where $\mathbf{L}(t)$ is the Laplacian of the radius graph at time t and λ_2 its algebraic connectivity.

25.3.3 Estimators, units and sensitivities

A metric is a number per run; a result is a number per *cell*, estimated from the N runs of the cell, and the estimator must fit the metric. Rates such as Equation (25.1) are proportions with a binomial (Wilson) interval. Lengths, times and errors are averaged with a t interval when their distribution is roughly symmetric and reported as a median with the interquartile range (IQR) when it is skewed. The minimum separation is the worst case of a run, so its distribution across runs has a heavy left tail. Report its median, its 5th percentile and the near-miss rate, never only its mean. Computation times are skewed to the right and need their 99th percentile. Table 25.1 collects the ten metrics; its last column is the one to reread before you believe a number.

Collision rate without minimum separation

A collision rate of 0/20 says that the true rate is below about 14 % (Proposition 25.9; the Wilson interval of Equation (25.7) gives 16 %) and nothing more. Two strategies with 0/20 each can differ enormously in how close they came. In the mini-study of Section 25.7 the local-only and the replan-only strategies both score 0/20 collisions with two drones. But their median minimum separations are 1.10 m and 1.65 m, and 40 % of the local-only runs are near misses against 10 %. Always report $d_{\min}$ (median, 5th percentile, near-miss rate) next to the collision rate, and use the segment-based minimum, not the sampled one.

Comparing computation times across machines

A decision that takes 0.12 ms in the toy simulator of this chapter takes a different time on a flight computer, under a different Python version, with a different NumPy, or with the logger switched on. Computation times are only comparable *within* one study on

Table 25.1. The ten metrics of the training plan. “Estimator” is the quantity reported for a cell of N runs; “sensitive to” lists what can change the number without changing the planner.

Metric	Formula per run	Unit	Estimator over runs	Sensitive to
<i>Safety</i>				
Collision rate	$X_{\text{coll}} = [\exists t, i \neq j : d_{ij}(t) < r_i + r_j]$	–	proportion Equation (25.1) with Wilson interval; n_{coll} alongside	sampling interval (missed contacts), radii, N (granularity $1/N$), which pairs count
Minimum separation	$d_{\min} = \min_{t, i \neq j} d_{ij}(t)$, split dd/di	m	median, IQR, 5th percentile; near-miss rate [$d_{\min} < R_{\text{safe}}$]	Δt , intruder aim and speed, prediction error; one bad run dominates a mean
<i>Efficiency</i>				
Path length	$L_i = \sum_t \ \mathbf{p}_i(t+1) - \mathbf{p}_i(t)\ $; ratio $\rho_i = L_i/L_i^{\text{nom}}$	m, –	mean of ρ_L with t interval; paired difference to a baseline	controller jitter and noise inflate L_i ; arena size (use the ratio)
Travel time	T_i of Equation (25.3)	s	mean with t interval; censored at the cap $T_{\max}$	goal tolerance ε_g , intruders that hit a hovering drone, the cap
Makespan	$\max_i T_i$	s	mean or median; report with the success rate	the slowest drone only; cap; nominal delays of the plan
Sum of costs	$\sum_i T_i$	s	mean with t interval	k (normalize per drone when k varies); cap
<i>Effort</i>				
Replanning count	n_{rp} (Replanning entries); n_{av} steps in Avoiding	–	mean with interval; distribution across runs	trigger thresholds, hysteresis, prediction noise (thrashing)
Computation time	$c_i(t)$ per decision; $\bar{c}$, c_{99} , misses, C_{tot}	ms, s	mean and 99th percentile; cactus plot of C_{tot}	machine, language, load, logging; comparable only within one machine and run
<i>Constraints</i>				
Formation error	$e_F(t)$ of Equation (25.5); $\bar{e}_F$, $e_F^{\max}$	m	mean with interval; peak distribution	template choice (fixed or nominal), formation graph E_F
Communication violations	n_{cv} of Equation (25.6)	steps	mean; fraction of steps; runs with $n_{\text{cv}} > 0$	R_{comm} , kept edges, k ; the nominal plan may already violate

one machine, and even then they should be reported with the hardware and software (Section 25.9). When you compare with a published method, compare counts that do not depend on the machine — expanded nodes, replans, low-level searches — or run the published code yourself on your machine. Never copy a runtime from a paper into your table.

25.4 The experiment matrix and the baselines

25.4.1 Factors and levels

The training plan names five **factors** to vary, each with a few **levels** (Table 25.2). A **cell** is one choice of a level for every factor; the mini-study of Section 25.7 occupies four cells. The **full factorial** design runs every cell: with the levels of Table 25.2 that is $3 \times 4 \times 4 \times 3 \times 4 = 576$ cells and, with 20 seeds per cell, 11 520 runs — a day of computing for the toy simulator, weeks for a physics simulator, and most of the cells answer no question anyone asked.

Where the two constraint levels are enforced. The last row of Table 25.2 holds two independent switches. Neither of them is a property of the scenario: both live inside the

Table 25.2. The experiment matrix of the training plan. The base cell (bold) is the reference around which a fractional design varies one factor at a time; the two starred factors interact and deserve a small full factorial of their own. The count $3 \times 4 \times 4 \times 3 \times 4 = 576$ reads the two binary constraint choices as the one four-level factor of the last row, and the strategy factor as three levels, with “no avoidance” kept outside the design as a sanity check.

Factor	Levels
Controlled drones k	2, 4 , 8 (more if computationally feasible)
Unknown intruders m	0, 1 , 2, several crossing trajectories
Prediction quality*	perfect state, noisy state (KF) , constant-velocity prediction, learned prediction (Chapter 20)
Avoidance strategy*	local-only, replan-only, hybrid (plus the sanity baseline “none”)
Constraints	no formation requirement vs formation-preserving; no communication radius vs bounded range R_{comm}

planner. *Formation-preserving* means that the formation term of Chapter 24, Equation (24.6), is switched on. Each drone then adds to its preferred velocity a soft consensus correction: the gain k_F times the error between its current displacement from a formation neighbor and the nominal one, summed over the neighbors. The formation graph E_F comes from the scenario, and the runner takes either the chain of consecutive drones or every pair. The term is dropped while a drone is Avoiding and restored while it is Reconnecting, exactly as Section 24.8 prescribes. The level therefore measures how well the shape *recovers*, not how rigidly it is held. The second switch, a bounded communication range, is a monitor rather than a controller in the toy: a run is scored by the number of steps at which the radius graph falls apart or a kept link exceeds R_{comm} .

Switching the formation level on changes the numbers, so the level belongs in the description of the cell. Take the $(k, m) = (4, 1)$ cell of the mini-study, with the hybrid strategy, E_F every pair and the same twenty seeds. The mean peak formation error falls from 2.06 to 1.17 m; the paired difference is -0.89 m, with a 95% interval of $[-1.41, -0.37]$ and $d_z = -0.81$. Safety and efficiency barely move: the drone–intruder minimum separation changes by -0.01 m $[-0.04, +0.02]$ and the path-length ratio by $+0.000$ $[-0.013, +0.014]$. The price is paid in effort, where the mean number of replans rises from 1.50 to 1.95. Holding the slot and leaving the conflict pull in opposite directions, which is why Chapter 24 drops the term while Avoiding. Section 25.11 names the arguments that switch the level on, and `code/figures/gen_ch25_study.py` writes these rows to `ch25-study-levels.dat`.

The learned prediction level. Row three of Table 25.2 ends with a level the toy cannot supply by itself: the trajectory predictor of Chapter 20, which is the deliverable of Week 10. The runner takes it as a callable, and Section 25.11.1 is the adapter that turns the reader’s trained model into that callable. The book’s own runs of the level use the constant-velocity stand-in built into the same adapter, so that no chapter imports another one’s code. What this chapter reports under “learned” is therefore a constant-velocity predictor without a calibrated covariance, and it is labeled as such wherever it appears.

A **fractional design** runs a chosen subset. The most readable one is *one-factor-at-a-time*: fix a base cell (bold in Table 25.2) and vary one factor over its levels while the others stay at the base, $3 + 4 + 4 + 3 + 4 - 4 = 14$ cells, each a clean answer to one question. It cannot see *interactions*, such as whether the value of the hybrid strategy depends on the prediction quality; for two factors suspected to interact, run their full factorial at the base levels of the others (strategy $\times$ prediction is $3 \times 4 = 12$ cells). State the design: a reader told “14 cells around a base configuration plus the 3×4 strategy–prediction factorial” can judge what was

Table 25.3. Baselines. Every one of them runs on exactly the scenarios of the strategy under test.

Baseline	What it does	Purpose
No avoidance	follows the nominal plan of CBS/ECBS and ignores intruders	sanity: collides with aimed intruders, never without
Local-only	ORCA/DWA with horizon τ , never replans; rejoins the moving reference	lower bound on effort, reference for deviation
Replan-only	no local safety layer; D* Lite/A* replan whenever a conflict enters the horizon	reference for replanning count and path quality
Hybrid	the state machine of Chapter 24: local first, replan when reconnection fails	the method under test
MILP oracle	offline optimum with full knowledge of the intruder (Chapter 22), tiny instances	gap to the optimum on path length and time

and was not tested.

25.4.2 Baselines

Every number needs a reference run on the same scenarios (Section 25.5); Table 25.3 lists five baselines. The three avoidance strategies are the ones the plan names. “No avoidance” is a *sanity* baseline: it must collide often when an intruder is aimed at a drone and never without one, because the nominal plan is conflict-free; if either fails, the generator or the plan is broken and the study is meaningless. The fifth is an *oracle*: on tiny instances a mixed-integer program (Chapter 22) that knows the intruder’s whole trajectory computes the shortest collision-free mission offline. No online strategy can beat it, so it turns a path-length ratio into a gap to the optimum; it costs minutes per instance, so it runs on a handful of scenarios and gets its own line. The straight line of the nominal plan is the free lower bound and the denominator of ρ_L .

Tuning on the test scenarios

The hybrid strategy has thresholds — the safety horizon τ_h , the deviation that triggers a replan, the minimum gap between replans — and so do the baselines. If you adjust them while watching the results table, the table measures your patience, not the method. Split the seeds: tune on seeds 0–9, freeze every parameter in the configuration file, and report seeds 100–119 that no one has looked at. Tune the baselines with the same care as your own method, on the same seeds, and say so; a baseline that was “run with default parameters” is a straw man.

25.5 Scenario generation

25.5.1 What a scenario contains

Definition 25.6 (Scenario). A **scenario** is the complete input of a run except the strategy. It holds the map, the starts $\mathbf{s}_i$ and goals $\mathbf{g}_i$ of the k drones, and the nominal plan. It also holds the trajectories $\mathbf{q}_j(t)$ of the m intruders for the whole horizon, the measurement noise sequence the predictor will see, and the constraint data (formation template, kept edges,

R_{comm}). It is a deterministic function `MakeScenario(cell, seed)` of the cell and an integer seed.

The noise is part of the scenario on purpose: if the predictor drew it from a global random state, two strategies would see different measurements and part of their difference would be noise. The same holds for randomness inside a strategy (RRT samples, particles): seed each drone’s generator from the scenario seed and the drone index. A scenario carries a **signature**, a hash of its arrays, with which the runner asserts that every strategy of a cell and seed received the same object.

25.5.2 Maps, starts and goals

For the grid layer, take maps and start–goal sets from the MAPF benchmark in the `.map` and `.scen` formats of [Chapter 7](#), so that the global planner is tested on instances others can download. Name the map and scenario file in the report. Random maps with an obstacle density and random start–goal pairs are the alternative when you need a difficulty knob; then the map is regenerated from the seed. The toy world of this chapter is obstacle-free: k drones start on the left edge of a 20×20 m arena, spread over 8 m, and fly to goals on the right edge whose rows are a random permutation of the start rows, so that their missions cross. The nominal plan is a prioritized plan ([Chapter 8](#)) giving each drone the smallest departure delay that keeps it 1.5 m from every higher-priority drone; it stands in for CBS, whose output has the same shape, and the code says so.

25.5.3 Intruder trajectory families and difficulty

An intruder that wanders randomly rarely threatens anyone, and a study of avoidance then measures nothing. The generator therefore *aims*. It picks a target drone i and an instant t_x in the middle third of its mission, and reads $\mathbf{p}_x = \mathbf{p}_i^{\text{nom}}(t_x)$ off the nominal plan. It then launches the intruder so that it passes through $\mathbf{p}_x$ at t_x , at a chosen **crossing angle** θ to the drone’s heading and a chosen speed. Three **trajectory families** are drawn around this aim ([Figure 25.2](#)). **Straight crossing** flies at constant velocity, where a constant-velocity predictor is exact and the case measures the reaction itself. **Turning** follows an arc anchored at the aim point at a constant rate ω of $5\text{--}15^\circ/\text{s}$, where a constant-velocity prediction over 3 s is off by $\frac{1}{2}v\omega\tau_h^2 \approx 1$ m. **Evasive** flies straight until 1–2.5 s before t_x and then makes a sudden heading change of $30\text{--}70^\circ$ that no predictor foresees; it may take the intruder away from its target or into another drone. Two knobs control difficulty. The first is the **density**, the lateral spread of the starts relative to k : eight drones in 8 m need long delays and see several neighbors at once. The second is the crossing angle (90° broadside, 180° head-on with the largest closing speed, 30° an overtaking intruder in the blind spot of a truncated velocity obstacle). Report which angles and families a study used; “random intruders” is not a description.

25.5.4 The paired design

Definition 25.7 (Paired design). An evaluation is **paired** if every strategy is run on the same set of scenarios $\{\text{MakeScenario}(\text{cell}, s) : s = 1, \dots, N\}$, and every comparison between two strategies A and B is made through the per-scenario differences $D_s = X_s^A - X_s^B$ of a metric X .

Proposition 25.8 (Pairing reduces variance when scenarios matter). Let X_s^A and X_s^B be the values of a metric for two strategies on scenario s , with scenarios drawn independently, variances σ_A^2 , σ_B^2 and correlation ρ between X_s^A and X_s^B across scenarios. The paired estimate $\bar{D} = \frac{1}{N} \sum_s (X_s^A - X_s^B)$ of the mean difference has variance $(\sigma_A^2 + \sigma_B^2 - 2\rho\sigma_A\sigma_B)/N$, whereas the unpaired estimate from two independent sets of N scenarios has variance $(\sigma_A^2 + \sigma_B^2)/N$.

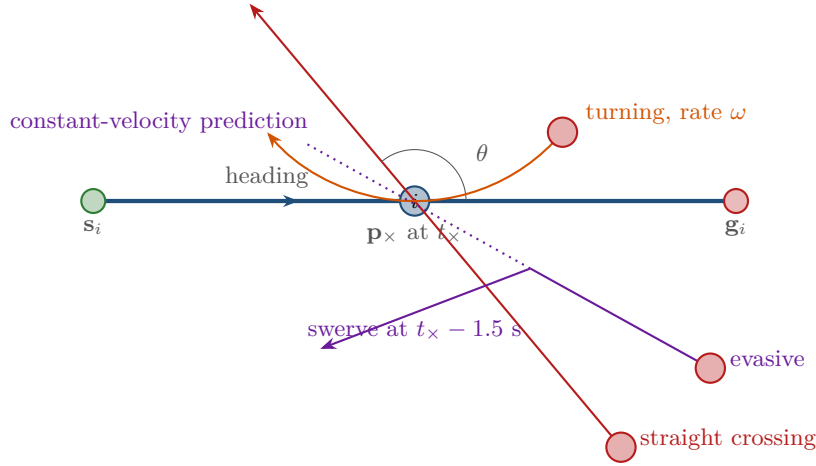


Figure 25.2. Aimed intruders. The drone (blue) follows its nominal plan and reaches $\mathbf{p}_x$ at t_x . All three families pass near $\mathbf{p}_x$ at t_x : the straight crossing at angle θ , the turning intruder along an arc anchored at the aim point, and the evasive intruder, which swerves shortly before t_x so that its constant-velocity prediction (dotted) points to the wrong place.

Pairing is more precise exactly when $\rho > 0$, and for $\sigma_A = \sigma_B$ it shrinks the variance by the factor $1 - \rho$.

Proof. For one scenario, $\text{Var}(X_s^A - X_s^B) = \text{Var}(X_s^A) + \text{Var}(X_s^B) - 2 \text{Cov}(X_s^A, X_s^B) = \sigma_A^2 + \sigma_B^2 - 2\rho\sigma_A\sigma_B$, and the mean of N independent such differences divides the variance by N . In the unpaired design the two sample means are independent, so their variances add: $(\sigma_A^2 + \sigma_B^2)/N$. The difference between the two expressions is $2\rho\sigma_A\sigma_B/N$, positive exactly when $\rho > 0$. $\square$

Scenarios matter: a head-on intruder at 2 m/s lengthens every strategy's path, one that misses shortens none, so ρ is positive and usually large. In the mini-study the path-length ratios of the hybrid and local-only strategies have $\rho = 0.90$ across the twenty scenarios of the four-drone cell, so pairing cuts the variance of their comparison tenfold. Algorithm 25.1 is the runner that enforces the design.

Algorithm 25.1. Paired evaluation of a set of strategies on one cell

Input: cell κ (levels of the five factors), seeds $s = 1, \dots, N$, strategies $\mathcal{S}$, configuration file $\mathcal{F}$

Output: one metric row per (seed, strategy), the raw logs, and per-strategy summaries

```

1 Function PairedStudy( $\kappa, N, \mathcal{S}, \mathcal{F}$ ):
2   rows  $\leftarrow \emptyset$ 
3   for  $s \leftarrow 1$  to  $N$  do
4      $\sigma \leftarrow \text{MakeScenario}(\kappa, s)$  // deterministic in  $(\kappa, s)$ 
5     foreach strategy  $A \in \mathcal{S}$  do
6        $\ell \leftarrow \text{RunEpisode}(\sigma, A, \mathcal{F})$  // log: positions, states,  $c_i(t)$ , events
7       save  $\ell$  to disk under  $(\kappa, s, A, \text{git hash of the code})$ 
8       assert signature( $\sigma$ ) unchanged // the strategy must not mutate the scenario
9       rows  $\leftarrow \text{rows} \cup \{(\kappa, s, A, \text{Metrics}(\ell))\}$ 
10  foreach metric  $X$  and strategies  $A, B \in \mathcal{S}$  do
11    PairedCompare( $(X_s^A)_s, (X_s^B)_s$ ) // differences  $D_s$ , interval, effect size, tests
12  return rows and summaries

```

Walkthrough. Line 4 builds the scenario once per seed, line 6 runs each strategy on it, and line 8 checks that the strategy did not modify it (a replanner that edits the intruder array it was handed silently unpairs the design). Logs are saved before metrics are computed, so a metric can be redefined later without rerunning anything; line 11 makes every comparison on the differences D_s . The loop wrapped in an outer loop over cells is the whole study, and `run_study()` in `ch25_evaluation.py` is this algorithm.

Unpaired comparisons

The commonest flaw in student evaluations is to run strategy A on twenty random scenarios, then strategy B on twenty *other* random scenarios, and compare the means. The scenario draw is then a hidden factor: if B happened to get an easier intruder in two of its runs, B wins. Proposition 25.8 says how much precision this throws away, but the worse effect is bias in the small samples that are typical of simulation studies. Seeds are free; use the same ones.

25.6 Statistics: from runs to claims

25.6.1 How many runs

The number of runs N per cell decides what the study can see. For a mean with standard deviation σ the half-width of a 95 % interval is about $2\sigma/\sqrt{N}$: halving it costs four times the runs. (Appendix B.3.4 reviews the Monte Carlo standard error behind this $1/\sqrt{N}$ rule.) For a rate the situation is harsher, because rare events need many runs to occur at all.

Proposition 25.9 (The rule of three). *If N independent runs show no collision, the one-sided 95 % upper confidence bound on the collision probability p is $1 - 0.05^{1/N} \approx 3/N$.*

Proof. The probability of seeing no collision in N runs is $(1 - p)^N$. The largest p that is still consistent with this observation at level 95 % satisfies $(1 - p)^N = 0.05$, that is $p = 1 - 0.05^{1/N}$. Since $0.05^{1/N} = e^{\ln(0.05)/N} \approx 1 + \ln(0.05)/N$ for large N and $-\ln 0.05 = 2.996$, the bound is approximately $3/N$. $\square$

With $N = 20$ the bound is 0.139 (the Wilson interval below gives 0.161): a perfect 0/20 rules out collision rates above about one in six, and no more. To claim a rate below 1 % you need 300 clean runs; to *measure* 1 % with any precision, thousands. This is why safety is also reported through $d_{\min}$, which is observed in every run. For continuous metrics, $N = 20$ seeds per cell is a reasonable minimum for a first study and 50–100 for a paper.

25.6.2 Mean and standard deviation, or median and IQR

The mean and the sample standard deviation $s = \sqrt{\frac{1}{N-1} \sum_s (x_s - \bar{x})^2}$ describe a symmetric, light-tailed distribution well: path-length ratios and formation errors are such. The median and the interquartile range describe skewed ones: minimum separations with their tail of near misses, computation times, and makespans with an occasional very late drone. Plot the distribution first (Figure 25.3), and if mean and median disagree visibly, report the median and say why. In the mini-study the mean makespan of local-only with two drones is 14.4 s with a t interval from 12.3 to 16.6 s, the hybrid's 14.4 s within 13.9 to 15.0 s. The means agree, but the wide interval hides one run in which a drone was pushed off its route and needed 34 s. The paired sign test, which sees only the run-by-run order, says the hybrid is faster in one run and slower in ten.

25.6.3 Confidence intervals

Three intervals cover almost every need.

- The ***t* interval** for a mean: $\bar{x} \pm t_{N-1, 0.975} s / \sqrt{N}$, with $t_{19, 0.975} = 2.093$ for $N = 20$. It assumes an approximately normal sampling distribution of the mean, which the central limit theorem grants for $N \geq 20$ unless the tail is extreme.
- The **bootstrap interval** for any statistic [37]: resample the N values with replacement $B = 2000$ times, compute the statistic each time, and take the 2.5th and 97.5th percentiles. It needs no distributional assumption and works for medians, percentiles and ratios; its randomness is seeded like everything else.
- The **Wilson interval** for a proportion $\hat{p} = x/N$ [179]:

$$\frac{\hat{p} + \frac{z^2}{2N} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}}, \quad z = 1.96. \quad (25.7)$$

Unlike the textbook $\hat{p} \pm z \sqrt{\hat{p}(1-\hat{p})/N}$, it does not collapse to a width of zero at $\hat{p} = 0$: for 0/20 it gives [0, 0.161], for 1/20 [0.009, 0.236], for 17/20 [0.640, 0.948].

25.6.4 Paired comparisons and effect sizes

Given the per-scenario differences $D_s = X_s^A - X_s^B$, report three things.

1. **The mean difference with its *t* interval**, in the units of the metric. “The hybrid’s paths are 2.1 % longer than local-only, 95 % interval 0.6 to 3.6 %” gives size, direction and uncertainty at once; an interval that excludes zero is significance at the 5 % level, and it says more than a *p*-value.
2. **A standardized effect size**: Cohen’s $d_z = \bar{D}/s_D$, the mean difference in units of the standard deviation of the differences [29]. It compares metrics with different units and studies with different N ; 0.2, 0.5 and 0.8 are conventionally small, medium and large. Report it next to the raw difference, not instead of it.
3. **A rank test** when the differences are skewed or N is small: the Wilcoxon signed-rank test [177], which ranks $|D_s|$ and asks whether the positive ranks dominate, and the sign test, which only counts how often A beat B . Print the counts “better / worse / tie” in any case; “16 of 20 scenarios” is understood faster than any *p*-value. Compute both *p*-values from the *exact* null distribution. With $N \leq 20$ scenarios — fewer once the ties are dropped — the usual normal approximation to the signed-rank statistic is wrong by a factor of several in the tail, which is the only part anyone reads; it needs N of about 25. Enumerating the 2^n sign patterns costs microseconds: the polynomial $\prod_i (1 + x^{r_i})$ counts them by rank sum, and its coefficients are the null distribution (`wilcoxon_signed_rank` in `ch25_evaluation.py`).

Every comparison is a chance to be fooled by noise: with 10 metrics and 3 strategy pairs you run 30 tests, and at the 5 % level one or two will be “significant” by accident. Either name the two or three primary comparisons in advance and call the rest exploratory, or adjust: Holm’s procedure [65] sorts the M *p*-values and compares the j -th smallest with $0.05/(M - j + 1)$, a step-down Bonferroni correction that loses little power.

25.6.5 Success rate with a runtime cap

Some runs do not finish: a planner exceeds its time budget, a drone never reaches its goal, the simulation hits the cap $T_{\max}$. The MAPF literature reports the **success rate**, the fraction of instances solved within the cap, as a function of the number of agents [8, 150, 161]. It

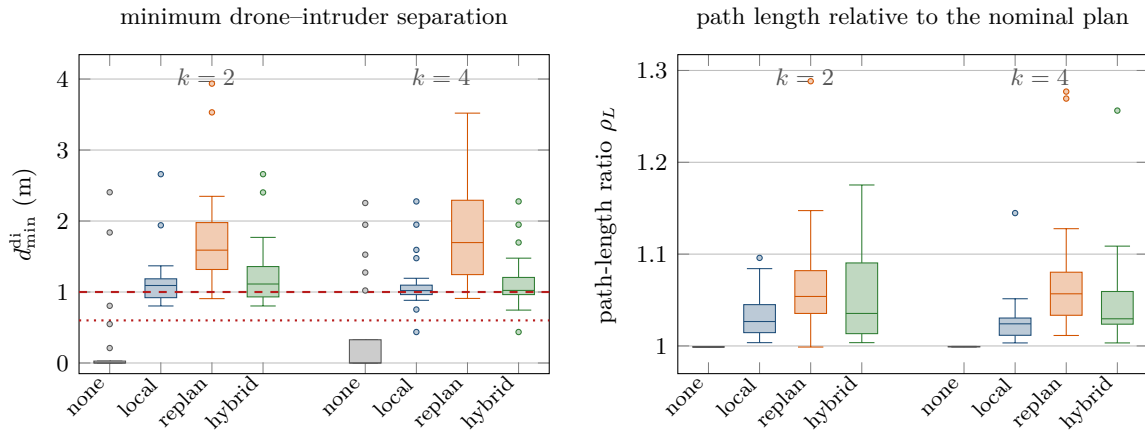


Figure 25.3. Box plots from the mini-study of Section 25.7 (one intruder, 20 seeds per box). Left: minimum drone–intruder separation $d_{\min}^{\text{di}}$; the dashed line is the safety radius $R_{\text{safe}} = 1$ m and the dotted line the collision distance 0.6 m; without avoidance every median is at zero. Right: path-length ratio ρ_L . Replan-only buys its larger separation with the longest paths; the hybrid sits between the two.

averages every finished-run metric (path length, makespan) over the successful runs only, *with the success rate printed next to it*. Two rules keep this honest. First, compare two strategies on the scenarios that *both* finished, or the better one is penalized for finishing the hard cases. Second, when one number is needed, use a penalized average such as PAR-2, in which an unfinished run counts as twice the cap. The cactus plot of Figure 25.4a shows the whole distribution instead.

Reporting means without spread, and hiding failed runs

A table of means with no interval, no N and no success rate is not a result; it is an anecdote with decimals. Every entry needs its interval or IQR and the number of runs it is based on, and every cell needs its success rate. Runs that crashed, timed out or collided are the most informative ones: keep them in the raw results file, count them in the table, and describe the worst of them in the text. Dropping a “pathological seed” after the fact is the same thing as tuning on the test set.

25.6.6 Three plots

Box plots per factor level. A **box plot** shows the median (line), the interquartile range (box) and whiskers at 1.5 IQR, with outliers as dots: five numbers that a mean and a standard deviation cannot replace for a skewed metric. Draw one box per strategy for every level of the factor under study, as in Figure 25.3, and put the safety threshold in as a horizontal line so that the eye can count the boxes that cross it. There the local-only and hybrid boxes sit on the safety radius $R_{\text{safe}} = 1$ m, because a velocity-obstacle avoider chooses velocities on the boundary of the forbidden set. The replan-only boxes sit 0.55–0.7 m higher, because its detours are offset by at least 2 m. The path-length panel shows the price.

Cactus and survival plots for runtime. A **cactus plot** sorts the runs of each strategy by their effort (total computation time, or expansions) and plots, against the effort x , the number of runs finished with effort at most x . A curve further to the right is a slower method; a curve that stops below N has unsolved runs, and its final height is the success rate. Its mirror image, the **survival plot**, shows the fraction still unfinished at x ; both show the whole

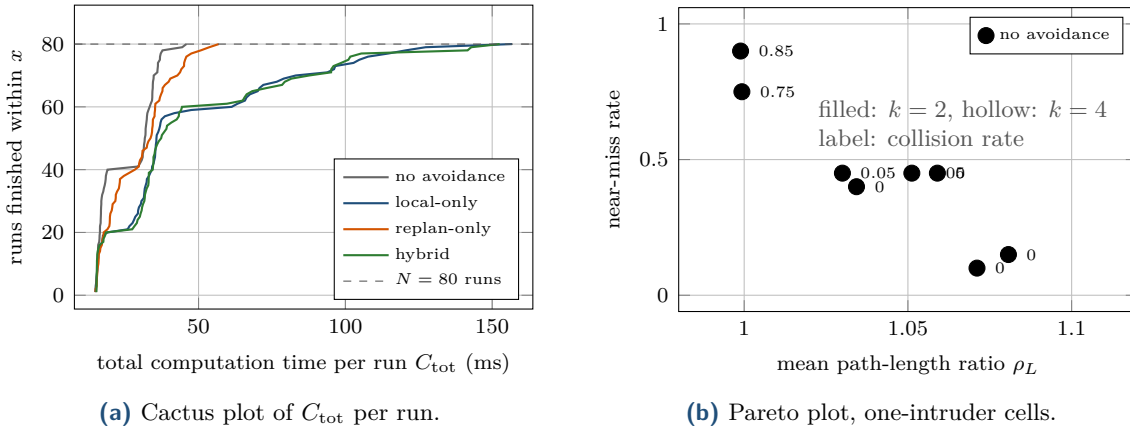


Figure 25.4. Effort and trade-off plots of the mini-study. (a) Total computation time per run (milliseconds of Python on one machine) for the 80 runs of each strategy; every curve reaches 80, so no run exceeded the cap. The reactive strategies are the slowest because their velocity sampling runs at every Avoiding step; the ordering is meaningful, the absolute values are not. (b) Mean path-length ratio (right is worse) against the near-miss rate (up is worse), one marker per strategy and cell; filled markers are the two-drone cells, hollow markers the four-drone cells, labels give the collision rate. No-avoidance (top left) is dominated; the three avoidance strategies form the front.

distribution, including the tail a mean hides. In Figure 25.4a the no-avoidance curve is the cost of prediction and tracking alone. The strategies with a local safety layer pay for their velocity sampling at every Avoiding step, and replanning is cheap here because it happens a few times per run.

Pareto plots of safety against efficiency. Safety and efficiency pull in opposite directions, and a strategy that is best in one column and worst in another is not “better”. A **Pareto plot** puts one point per strategy and cell into the plane (efficiency loss, safety loss), here ρ_L against the near-miss rate, and marks the points that no other point dominates. In Figure 25.4b the three avoidance strategies form a front — local-only the most efficient, replan-only the safest, the hybrid between them — and no-avoidance is dominated by all of them. The plot states the trade-off; the text must say which point the application wants. Both panels of Figure 25.4 are drawn from the 320 runs of Section 25.7, so the same study can be read three ways: as a table, as a distribution and as a front.

25.7 A worked mini-study

Example 25.10 (Two and four drones, zero or one intruder, three strategies, twenty seeds). The study occupies the four cells $(k, m) \in \{2, 4\} \times \{0, 1\}$ of Table 25.2 with the other factors fixed. Prediction is by a constant-velocity Kalman filter on position measurements with $\sigma = 0.3$ m of noise, whose covariance inflates the safety radius (capped at 1.5 m). There is no formation requirement, and the communication range $R_{\text{comm}} = 10$ m is monitored through λ_2 only. The strategies are no-avoidance, local-only, replan-only and hybrid, and the seeds are 0–19. The intruder family cycles with the seed (seven straight, seven turning, six evasive per cell), with a random crossing angle in $[45^\circ, 135^\circ]$ and a speed in $[1, 2]$ m/s. The world is the toy of Section 25.5: drones of radius 0.3 m at 1.5 m/s (limit 2 m/s), $\Delta t = 0.1$ s, safety radius $R_{\text{safe}} = 1$ m, planned separation 1.5 m, safety horizon $\tau_h = 3$ s, mission cap 40 s. The local layer is a sampled velocity-obstacle avoider (97 sampled velocities plus the preferred one, constant over the horizon) and the replanner a detour planner with waits. The hybrid is the state machine of Chapter 24 with a deviation threshold of 3 m, at most 4 s

Table 25.4. Results of [Example 25.10](#), $N = 20$ runs per row. Collision rate with its Wilson interval; median (IQR) of the drone–intruder minimum separation; mean path-length ratio with its t interval; mean makespan and sum of costs (s); mean replans per run; mean computation time per decision and mean of the per-run 99th percentile (ms, one machine); mean formation error against the nominal geometry (m); mean number of steps with a communication violation; success rate (all drones at their goals, no collision). Without an intruder the four strategies produce identical runs, so one row serves. Means are over all 20 runs of a row, collisions included: in the toy a collision does not stop a run and every drone reaches its goal, so there is nothing to censor; the rule of [Section 25.6](#) to average over successful runs only applies as soon as a run can fail to finish. The $\bar{c}$ and c_{99} columns are wall-clock times on one machine; they are the only columns that will differ when you re-run the study, and only their ordering is meaningful.

Cell	Strategy	Coll. rate [CI]	$d_{\min}^{\text{di}}$ (IQR)	ρ_L [CI]	MS	SoC	n_{rp}	$\bar{c}$	c_{99}	$\bar{e}_F$	n_{cv}	Succ.
$k=2, m=0$	all four	0.00 [0.00, 0.16]	–	0.999	13.4	25.2	0.00	0.056	0.11	0.03	0.0	1.00
$k=2, m=1$	none	0.85 [0.64, 0.95]	0.00 (0.00–0.07)	0.999	13.4	25.2	0.00	0.060	0.11	0.03	0.0	0.15
	local	0.00 [0.00, 0.16]	1.10 (0.93–1.21)	1.034 [1.022, 1.047]	14.4	27.4	0.00	0.123	0.39	0.67	0.0	1.00
	replan	0.00 [0.00, 0.16]	1.65 (1.32–2.01)	1.071 [1.042, 1.100]	14.8	27.9	4.15	0.070	0.58	1.41	3.5	1.00
	hybrid	0.00 [0.00, 0.16]	1.12 (0.94–1.36)	1.059 [1.033, 1.085]	14.4	26.8	0.85	0.117	0.41	1.03	1.6	1.00
$k=4, m=0$	all four	0.00 [0.00, 0.16]	–	0.999	13.2	48.1	0.00	0.059	0.12	0.03	0.0	1.00
$k=4, m=1$	none	0.75 [0.53, 0.89]	0.00 (0.00–0.50)	0.999	13.2	48.1	0.00	0.060	0.11	0.03	0.0	0.25
	local	0.05 [0.01, 0.24]	1.02 (0.97–1.12)	1.030 [1.016, 1.044]	13.4	48.4	0.00	0.150	0.70	0.31	0.0	0.95
	replan	0.00 [0.00, 0.16]	1.73 (1.31–2.31)	1.081 [1.047, 1.115]	14.7	52.8	7.45	0.068	0.43	1.48	0.0	1.00
	hybrid	0.05 [0.01, 0.24]	1.04 (0.97–1.21)	1.051 [1.025, 1.078]	14.2	50.0	1.50	0.152	0.75	0.74	0.0	0.95

in Avoiding and a reconnection lead of 2 s. All three are stand-ins for the modules of the earlier chapters, documented as such in `ch25_evaluation.py`. The 320 runs take about 16 s on the machine that produced the numbers below, and produce [Tables 25.4](#) and [25.5](#) and [Figures 25.3](#), [25.4a](#) and [25.4b](#). Every number below is produced by the code. The summary rows and the paired comparisons come from `gen_ch25_study.py`, which prints the $\text{\LaTeX}$ of both tables. The metric definitions and the determinism of the runner are checked by the self-test of `ch25_evaluation.py`, which also prints the per-cell summary, the paired rows and the family breakdown. The per-seed facts of the discussion come from the raw rows in `ch25-study-runs.dat`.

Sanity first. Without an intruder the four strategies produce runs that are identical to the last digit; only the computation times differ, by microseconds. The trigger never fires, because the nominal plan keeps 1.5 m between drones and the trigger radius is 1 m, and nothing collides. With an intruder and no avoidance, 17 of 20 two-drone runs and 15 of 20 four-drone runs end in a collision. Broken down by family, every straight and every turning intruder hits its target (28/28), while only 4 of 12 evasive intruders do: the swerve, drawn 1–2.5 s before the aim time, takes the intruder off its aim more often than not. The generator does what it should, and the ratio 0.999 instead of 1 is the goal tolerance: a run ends when every drone is within 0.3 m of its goal.

Safety. All three avoidance strategies remove almost every collision: 0/20 in five of the six one-intruder rows. The single collision is seed 8 of the four-drone cell, an evasive intruder. It is shared by local-only and hybrid with the same minimum separation, 0.44 m, in both: the collision happened while the hybrid was still in its local phase, acting exactly as local-only does. Replan-only had taken that drone 3.4 m away by then. Yet 0/20 against 1/20 is not a difference the study can see: the intervals are [0, 0.16] and [0.01, 0.24]. What the study does see is the minimum separation. Local-only and hybrid keep a median of 1.02–1.12 m, that

Table 25.5. Paired comparisons of [Example 25.10](#), hybrid minus the other strategy, one-intruder cells. Mean difference with its 95 % t interval, Cohen’s d_z , the number of scenarios in which the hybrid value was smaller / larger / equal, and the two-sided p -values of the Wilcoxon signed-rank test and of the sign test. Both are *exact* (the null distribution is enumerated, not approximated by a normal), which matters here: the $d_{\min}$ rows against local-only have only 7 non-zero differences, far too few for a normal approximation. For $d_{\min}$ larger is better; for the other three smaller is better.

Metric	Cell	Difference [CI]	d_z	smaller/larger/equal	p_w	p_{sign}
<i>Hybrid minus local-only</i>						
ρ_L	$k=2$	+0.025 [+0.004, +0.045]	0.57	5 / 9 / 6	0.042	0.424
ρ_L	$k=4$	+0.021 [+0.006, +0.036]	0.66	4 / 10 / 6	0.009	0.180
$d_{\min}^{\text{di}}$ (m)	$k=2$	+0.082 [+0.004, +0.159]	0.49	0 / 7 / 13	0.016	0.016
$d_{\min}^{\text{di}}$ (m)	$k=4$	+0.023 [−0.003, +0.050]	0.41	2 / 5 / 13	0.078	0.453
n_{rp}	$k=2$	+0.85 [+0.54, +1.16]	1.27	0 / 14 / 6	0.0001	0.0001
n_{rp}	$k=4$	+1.50 [+0.87, +2.14]	1.11	0 / 14 / 6	0.0001	0.0001
$\bar{e}_F$ (m)	$k=2$	+0.36 [−0.19, +0.91]	0.31	3 / 11 / 6	0.025	0.057
$\bar{e}_F$ (m)	$k=4$	+0.43 [+0.20, +0.66]	0.88	0 / 14 / 6	0.0001	0.0001
<i>Hybrid minus replan-only</i>						
ρ_L	$k=2$	−0.012 [−0.049, +0.025]	−0.15	14 / 6 / 0	0.452	0.115
ρ_L	$k=4$	−0.029 [−0.053, −0.006]	−0.58	16 / 4 / 0	0.006	0.012
$d_{\min}^{\text{di}}$ (m)	$k=2$	−0.56 [−0.84, −0.28]	−0.94	17 / 3 / 0	0.0003	0.003
$d_{\min}^{\text{di}}$ (m)	$k=4$	−0.71 [−1.05, −0.36]	−0.96	17 / 3 / 0	0.0001	0.003
n_{rp}	$k=2$	−3.30 [−4.05, −2.56]	−2.07	20 / 0 / 0	< 0.0001	< 0.0001
n_{rp}	$k=4$	−5.95 [−7.59, −4.31]	−1.70	20 / 0 / 0	< 0.0001	< 0.0001
$\bar{e}_F$ (m)	$k=2$	−0.38 [−0.80, +0.04]	−0.42	13 / 7 / 0	0.114	0.263
$\bar{e}_F$ (m)	$k=4$	−0.73 [−1.02, −0.44]	−1.19	17 / 3 / 0	0.0001	0.003

is, the safety radius itself, with 40–45 % near misses per cell; replan-only keeps 1.65–1.73 m with 10–15 %. The paired differences are −0.56 and −0.71 m with intervals far from zero and $d_z \approx -0.95$: a large, real effect that the collision counts alone would have hidden.

Efficiency. Local-only flies the shortest paths (3.0–3.4 % above the nominal plan), the hybrid 5.1–5.9 %, replan-only 7.1–8.1 %. The paired differences of two to three percentage points have intervals that exclude zero in three of the four rows, so they are real; whether they matter is a question for the application. The mean makespans of the three avoidance strategies lie within 1.3 s of each other (local-only 13.4 s against replan-only 14.7 s with four drones), and the hybrid is within 0.8 s of either baseline in every cell. The two-drone local-only row shows why the median belongs in the table. Its mean makespan, 14.4 s, equals the hybrid’s, but its interval [12.3, 16.6] s is four times wider. The reason is seed 13, a turning intruder, which pushed one drone off its route for 465 Avoiding steps, drove the formation error to a peak of 9 m and delayed the mission to 33.7 s. The hybrid’s deviation trigger is designed to prevent exactly this, and its longest run took 17.2 s.

Effort and constraints. Replan-only replans 4.15 and 7.45 times per run, the hybrid 0.85 and 1.50, local-only never. Under noisy prediction the replan-only strategy thrashes: every change of the predicted trajectory re-triggers a replan a second later, and each replan re-times the mission. The ordering of the computation times is the part that transfers to another machine: a step with the local safety layer costs about twice a replanning step and about twice a pure prediction-and-tracking step, because the velocity sampling runs at every Avoiding step ([Figure 25.4a](#)). On the machine that produced the table those steps are 0.12 ms with two

drones and 0.15 ms with four, against 0.07 ms for a replanning step and 0.06 ms for prediction and tracking alone; on yours all three will move together. The two timing columns tell different stories. Replan-only is the cheapest on average, but its c_{99} , 0.58 and 0.43 ms, is of the same order as the reactive strategies' 0.39–0.75 ms, because a replan is rare and expensive while a velocity sample is frequent and cheap. A deadline is missed by the tail, not by the mean. Detours distort the swarm geometry more than dodges: the mean formation error against the nominal plan is 0.3–0.7 m for local-only, 0.7–1.0 m for the hybrid and 1.4–1.5 m for replan-only. Communication violations occur in the two-drone cell only, where a lagging drone can fall 10 m behind its partner, and the means of 3.5 and 1.6 steps each come from a single run. A mean of a quantity that is zero in 19 of 20 runs should be reported differently: “1 of 20 runs, for 69 steps” for replan-only and “1 of 20 runs, for 32 steps” for the hybrid.

A cautious interpretation. The study supports the following statements, and no more.

1. In this toy world the three strategies form a trade-off: local-only is the cheapest and closest, replan-only the widest and dearest, the hybrid in between on every metric, with 4–6 replans fewer per run than replan-only and the local-only's runaway run avoided.
2. The hybrid does *not* improve the minimum separation over local-only, because its local phase is the same avoider, and the avoider chooses velocities on the boundary of the safe set.
3. The separation advantage of replan-only comes from a parameter, the 2 m minimum offset of its detours, not from a principle; a local layer with a margin above R_{safe} would move that row, and a fair follow-up study varies the margin for both.
4. With $N = 20$ the collision rates of the avoidance strategies are indistinguishable; only d_{min} separates them.
5. The evasive family causes the only collision and the only misses of the sanity baseline; a real study breaks its results down by family and by crossing angle.
6. The layers are stand-ins, the intruders are aimed, the drones are point masses without acceleration limits, and the timings are those of one machine: none of the numbers transfers to a physics simulator or a flight test, but the pipeline does.

25.8 Reproducibility

A study is reproducible when a reader with the code can regenerate every table and figure from a command line and trace every number back to a run [129]. Five habits achieve this.

1. **One configuration file.** Every constant — radii, speeds, Δt , horizons, thresholds, noise levels, R_{comm} , the cells and the seeds — lives in one file (JSON or YAML) that the runner reads and copies into every results file. The toy keeps them as named constants at the top of `ch25_evaluation.py`; a larger study moves them out of the code.
2. **Seeds everywhere.** The scenario, the measurement noise, every sampler inside a strategy and the bootstrap take their seeds from the scenario seed and a fixed offset, never from the clock or a global state. The self-test checks that two runs of one seed give identical logs (wall-clock timings excepted).
3. **Log every decision.** Per step and drone: the state, the layer that produced the command, the command, the predicted time to collision and separation that the trigger saw, and $c_i(t)$; per event: replans with their reason (deviation, time-out, reconnection failure, constraint violation). A log of positions alone can compute the metrics but cannot explain a failure, and explaining the worst run is half of a results section.
4. **Version the code and the data.** Write the commit hash, the Python and NumPy

Table 25.6. Template of a results table. Replace the placeholders; keep the columns. The caption of the real table states the machine (CPU, cores, RAM), the software versions, the commit hash, the seeds and the configuration file.

Column	Content	Estimator
Cell	levels of every factor (k , m , prediction, strategy, constraints)	–
N , success	runs per cell; fraction finished within the cap without collision	Wilson interval
Collision rate	$\hat{p}_{\text{coll}}$ [interval]; n_{coll} in a footnote	Wilson interval
d_{min}	median (IQR), 5th percentile, near-miss rate; drone–drone and drone–intruder	bootstrap interval if needed
ρ_L , MS, SoC	mean [interval] over finished runs; per drone when k varies	t interval
n_{rp} , n_{av}	mean [interval]; runs with $n_{\text{rp}} > 0$	t interval
$\bar{c}$, c_{99} , misses	per decision, ms; total per run in a cactus plot	–
$\bar{e}_F$, e_F^{max}	mean [interval]; template named	t interval
n_{cv}	fraction of steps; runs with $n_{\text{cv}} > 0$	Wilson interval on runs
Paired rows	difference to the primary baseline [interval], d_z , better/worse/tie	paired t , sign test

versions and the machine into the header of every results file; tag the commit that produced the reported numbers; pin the dependencies.

5. **Save the raw per-run results.** One row per run in a plain text table (the `.dat` files of this chapter), one compressed log per run, and summary tables derived from them by a script under version control. Never edit a results file by hand.

Table 25.6 is a results-table template that follows the rules of Section 25.6: every cell carries N and the success rate, every rate an interval, every skewed metric a median and IQR, and the caption states the hardware. Fill it by script from the raw rows.

25.9 Reporting the results

What goes into a results section. Seven parts, in this order. (1) The setup: cells and design, baselines, parameters (a table), seeds, hardware and software. (2) The sanity results, in one paragraph, because they establish that the apparatus works. (3) The primary comparison, with the paired statistics of Table 25.5 and the box plots. (4) The trade-off, with the Pareto plot and a sentence on which point the application wants. (5) The scaling with k and m , with success rates and the cactus plot. (6) The failure analysis: the worst runs by name (seed, cell), what happened and what it suggests. (7) The limitations. The classical MAPF papers are good models for (5). CBS [150] and ECBS [8] report success rates within a time limit as functions of the number of agents on named benchmark maps. ORCA [14] reports timings per simulation step against the number of agents on one described machine.

How to state limitations. Say what was simulated and what was not (dynamics, sensing, communication delays, wind), which modules are stand-ins, and what the intruder model assumes. Say how large N was and what it therefore cannot resolve (the rule of three), which parameters were tuned and on which seeds, and which scenarios were never tried (dense swarms, 3D, many intruders). State what is *not* claimed: 0/20 collisions is not “collision-free”. A limitations paragraph that names concrete gaps reads as competence; “future work will

Table 25.7. The completion checklist of the training plan restated as evaluation questions.

Checklist item	Evaluation question	Evidence
A* and space-time A*	Are the single-agent paths optimal and constraint-respecting?	cost vs Dijkstra on random grids (Chapter 4)
CBS constraint tree	Is every nominal plan conflict-free and optimal?	conflict detection of Chapter 7; SoC vs a brute-force optimum on tiny instances
ECBS trade-off	How much cost does the factor w buy in runtime?	success rate and cost /LB against k (Chapter 10)
VO/RVO/ORCA	Does the local layer prevent predicted collisions?	$d_{\min}$ box plots, near-miss rate, oscillation count
Tracking an unknown drone	Is the estimate stable and calibrated?	innovation statistics, error vs covariance (Chapter 18)
Prediction baseline vs LSTM	Does learning reduce ADE/FDE, and does that reach the planner?	ADE/FDE (Chapter 20); $d_{\min}$ and ρ_L per prediction level
Time-to-collision trigger	Does the trigger fire early enough and not too often?	$d_{\min}$ and n_{av} as functions of τ_h
Reconnection and replanning	When is a replan needed, and how often is it triggered?	n_{tp} , deviation at trigger time, replan reasons from the log
Formation and range constraints	Are constraints respected during avoidance?	$\bar{e}_F$, $e_F^{\max}$, n_{cv} per constraint level
Reproducible evaluation	Can a reader regenerate every number?	seeds, configuration, commit hash, raw rows, scripts

address remaining issues” reads as evasion.

How to describe the hardware and software. CPU model and clock, cores used (and whether the code is single-threaded), memory, operating system, and Python and NumPy versions. Then the timer (`time.perf_counter`), whether the timed code includes logging, and whether the machine was otherwise idle. Finally, whether times are per decision, per step or per run, and, for a physics simulator, its version, physics rate and controller rate.

The completion checklist as evaluation questions. The training plan ends with ten boxes to tick (Appendix A). Table 25.7 restates them as questions an experiment can answer, each with the evidence that answers it; every row is one experiment with a yes-or-no outcome, and the table is a usable outline for the evaluation chapter of a thesis.

25.10 The benchmark landscape and physics simulators

MAPF benchmarks. The global layer is evaluated on the MAPF benchmark of Stern et al. [161], described in Chapter 7. It has grid maps in the `.map` format of the single-agent pathfinding benchmarks [162] and scenario files with start–goal pairs. Its protocol is a success rate within a time limit as a function of the number of agents. Use it: it makes the CBS/ECBS part of your system comparable with the literature at no cost. It says nothing about intruders, continuous motion or constraints; those are what your scenario generator adds, and the honest way to present a hybrid planner is “the global layer on the standard maps, the full system on our scenarios, both released”.

gym-pybullet-drones. The natural step after the toy is a physics simulator, and the suggested stack of the training plan is `gym-pybullet-drones` [125]. It simulates quadrotors with PyBullet physics, including motor dynamics, drag and downwash between vehicles. It exposes a Gym-style interface with observations and actions per drone, and ships PID controllers that turn a velocity or position setpoint into motor commands. The study of Section 25.7 should be repeated in it before any of its numbers is quoted. Flight tests are the step after that. Hönig et al. [67] show what a hardware evaluation of swarm trajectory planning looks like, with downwash-aware separation constraints and tens of small quadrotors. Their evaluation section is a good model for reporting on hardware.

Connecting the book’s code to a simulator. The planner of Chapter 24 needs four things from a simulator: to reset it to a scenario, to observe positions and velocities (and noisy intruder measurements), to apply one velocity setpoint per drone, and to advance time. Listing 25.1 sketches the interface; the toy simulator implements it in memory, and a PyBullet wrapper implements it by calling the environment’s `step()` in an inner loop, because the physics runs at 240 Hz and the planner at 10 Hz. The evaluation loop then reads the observation, times the planner’s decision, records both in the log, applies the commands and steps the world. Three things change with the switch. The commanded velocity is no longer the flown one: a low-level controller tracks it with lag and overshoot, so $d_{\min}$ drops and the safety radius may need a margin. Downwash adds a vertical separation rule that 2D metrics do not see; log the vertical distance. And the computation time now contains the simulator, so time the planner’s decision separately from the physics step. The metrics, the paired design and the statistics do not change at all: they read the log, and the log has the same columns.

Listing 25.1. Interface between the evaluation loop and a simulator (a sketch, not part of `ch25_evaluation.py`). The toy simulator and a PyBullet wrapper both fit it; the loop and the metrics never see the difference.

```

1  class Simulator:
2      """What the planner needs from any world, toy or physics."""
3
4      def reset(self, scenario):
5          """Place drones and intruders; return the first observation."""
6
7      def observe(self):
8          """Positions and velocities of the drones (true state) and the
9             noisy position measurements of the intruders at this instant."""
10
11     def command(self, velocity_setpoints):
12         """One (vx, vy[, vz]) per drone, tracked by the low-level control."""
13
14     def step(self, dt):
15         """Advance the world by dt (a PyBullet wrapper loops its physics
16            step 24 times at 240 Hz for dt = 0.1 s)."""

```

25.11 Implementation notes

`ch25_evaluation.py` has the five parts named at its top: metrics, scenarios, the toy simulator, the paired runner and the statistics. Three choices deserve a word, and the one level of the experiment matrix that the toy cannot supply by itself — the learned predictor — deserves a subsection of its own.

Metrics read arrays, not objects. Every metric function takes plain NumPy arrays (positions of shape $(T + 1, k, 2)$, computation times of shape $(T + 1, k)$), so that the same functions serve the toy, a PyBullet log and a flight log. The collision test uses the segment-based minimum of [Section 25.3](#). Metrics that do not exist for a run (no intruder, one drone) return `nan`, which the summaries skip: a run without intruders must not report a drone–intruder separation of ∞ that later averages into nonsense. [Listing 25.2](#) shows the safety and arrival metrics.

Listing 25.2. Safety and arrival metrics (from `code/ch25_evaluation.py`). Minimum separations use linear motion between samples.

```

1  def collision_summary(P, Q=None, r_drone=R_DRONE, r_intruder=R_INTRUDER):
2      """Collision indicator and count, and the minimum separations."""
3      dd, di = pair_min_distances(P, Q)
4      k = P.shape[1]
5      iu = np.triu_indices(k, 1)
6      n_dd = int(np.sum(dd[iu] < 2 * r_drone)) if k > 1 else 0
7      n_di = int(np.sum(di < r_drone + r_intruder)) if di.size else 0
8      return {
9          "collision": int(n_dd + n_di > 0),
10         "n_collisions": n_dd + n_di,
11         "dmin_dd": float(np.min(dd[iu])) if k > 1 else float("nan"),
12         "dmin_di": float(np.min(di)) if di.size else float("nan"),
13     }
14
15
16  def path_lengths(P):
17      """Flown length of every drone (m), shape (k,)."""
18      if P.shape[0] < 2:
19          return np.zeros(P.shape[1])
20      return np.sum(np.linalg.norm(np.diff(P, axis=0), axis=2), axis=0)
21
22
23  def travel_times(P, goals, dt=DT, tol=GOAL_TOL):
24      """First time after which each drone stays within tol of its goal;
25      inf if it is not there at the end of the log."""
26      away = np.linalg.norm(P - np.asarray(goals)[None], axis=2) > tol
27      out = []
28      for i in range(P.shape[1]):
29          idx = np.nonzero(away[:, i])[0]
30          if away[-1, i]:
31              out.append(float("inf"))
32          elif idx.size == 0:
33              out.append(0.0)
34          else:
35              out.append(float((idx[-1] + 1) * dt))
36      return np.array(out)

```

The runner is the paired design. [Listing 25.3](#) is [Algorithm 25.1](#) in Python: one scenario per seed, every strategy on it, the signature stored in every row so that the self-test can assert that the rows of a seed share it. The paired comparison works on the per-scenario differences and returns the raw difference with its interval, d_z , the counts and the two rank tests. Pairs whose metric is undefined — $d_{\min}^{\text{di}}$ in a cell without an intruder — are dropped before the tests, so a comparison with no data reports $n = 0$ and $p = 1$ instead of a p -value computed from a sample of zeros. `summarize()` produces the per-cell statistics of [Table 25.4](#), and `write_dat()`

the plain-text tables the figures read.

Listing 25.3. The paired runner and the paired comparison (from `code/ch25_evaluation.py`; the docstrings are abridged at the ellipses). The two levels that [Section 25.4](#) adds to the runner are its `predictor` and `formation` arguments.

```

1 def run_study(cells, strategies, seeds, prediction="noisy", family="mixed",
2               predictor=None, formation=None, r_comm=R_COMM, edges_keep=(),
3               **kw):
4     """Paired design: for every cell (k, m) and seed one scenario is built
5     and every strategy runs on it. Returns one row (dict) per run.
6     ...
7     """
8     rows = []
9     for (k, m) in cells:
10         for seed in seeds:
11             sc = make_scenario(seed, k, m, family=family, **kw)
12             sig = sc.signature()
13             edges_F = formation_edges(k, formation) if isinstance(formation,
14 str) else formation
15             for strat in strategies:
16                 log = run_episode(sc, strat, prediction, predictor, edges_F)
17                 row = {"k": k, "m": m, "seed": seed, "strategy": strat,
18                       "formation": formation if formation else "-",
19                       "family": sc.families[0] if sc.m else "-", "signature":
20 sig}
21                 row.update(compute_metrics(log, r_comm=r_comm,
22 edges_keep=edges_keep))
23                 rows.append(row)
24     return rows
25
26 def paired_compare(a, b):
27     """Paired comparison of metric a against metric b (same scenarios):
28     mean difference with its t interval, Cohen's d_z, Wilcoxon and sign tests.
29     ...
30     """
31     d = np.asarray(a, float) - np.asarray(b, float)
32     d = d[np.isfinite(d)] # an undefined metric is not evidence
33     if d.size == 0:
34         nan = float("nan")
35         return {"mean_diff": nan, "lo": nan, "hi": nan, "d_z": nan, "n": 0,
36               "better": 0, "worse": 0, "tie": 0,
37               "p_wilcoxon": 1.0, "p_exact": True, "p_sign": 1.0}
38     ci = mean_ci(d)
39     dz = ci["mean"] / ci["sd"] if ci["sd"] > 0 else float("inf") if ci["mean"]
40 != 0 else 0.0
41     out = {"mean_diff": ci["mean"], "lo": ci["lo"], "hi": ci["hi"], "d_z": dz,
42           "n": d.size, "better": int(np.sum(d < 0)), "worse": int(np.sum(d >
43 0)),
44           "tie": int(np.sum(d == 0))}
45     w = wilcoxon_signed_rank(d) # exact null distribution for n <= 20
46     out["p_wilcoxon"], out["p_exact"] = w["p"], w["exact"]
47     out["p_sign"] = sign_test(d)["p"]
48     return out

```

Determinism and timing. The simulator uses no global random state: every random quantity is drawn in `make_scenario()` from a generator seeded with the scenario seed and stored in the scenario. The self-test runs a cell twice and compares every column except the wall-clock ones, checks that all strategies of a seed share the signature, and runs the whole mini-study (320 runs in about 16 s on that machine). It then asserts its sanity relations: no collisions without intruders, at least half of the no-avoidance runs colliding with one, and no avoidance strategy worse than no avoidance. Computation times come from `time.perf_counter()` around the decision of one drone, which resolves fractions of a microsecond but is disturbed by whatever else the machine does. The 99th percentile and the maximum are the columns that show it. The six pitfalls collected in the five boxes above — unpaired comparisons, tuning on the test scenarios, means without spread, hidden failures, runtimes across machines and collision rates without $d_{\min}$ — are the ones most often committed in student evaluations. The classic essays of Hooker [68] and Barr et al. [9] list many more.

25.11.1 Plugging in the Week 10 predictor

The *learned prediction* level of Table 25.2 is the one level of the matrix that this chapter cannot produce on its own. Its model is the deliverable of Week 10, the sequence predictor of Chapter 20, and `predict_intruders()` therefore takes it as an argument: a callable that receives the noisy history of one intruder and returns its predicted positions. Everything between that model and this chapter’s arrays is an adapter, and each of the three conversions it makes is a place where a prediction can silently come out one step, one frame or one second wrong. *Rate and history length:* the predictor of Chapter 20 reads `PRED_OBS = 8` samples spaced `PRED_DT = 0.2` s apart, while this chapter logs one sample per $\Delta t = 0.1$ s, so the history must be resampled. *Frame:* the model predicts *displacements* in the agent frame of Chapter 20, whose origin is the last observation and whose $+x$ axis is the last observed heading, divided by `PRED_SCALE = 0.5` m; the adapter builds that frame, converts into it and converts the answer back. *Horizon:* the model returns `PRED_STEPS = 12` steps, that is 2.4 s, while the trigger needs $H + 1 = 31$ positions out to $\tau_h = 3$ s, so the answer is interpolated onto this chapter’s grid, anchored at the current position and continued at constant velocity past its last step. Listing 25.4 is the whole adapter.

Listing 25.4. The Week-10 adapter (from `code/ch25_evaluation.py`; the docstring is abridged at the ellipsis). `model` is the reader’s trained predictor of Chapter 20; with `model=None` the same code is the constant-velocity stand-in.

```

1 def week10_predictor(model=None, horizon=TAU_H, dt=DT):
2     """Adapter for the trajectory predictor of Chapter 20 (Week 10).
3     ...
4     """
5     n_out = int(round(horizon / dt)) + 1
6
7     def predictor(hist):
8         obs = _resample(hist, dt, PRED_DT, PRED_OBS)
9         origin = obs[-1]
10        d = obs[-1] - obs[-3]
11        phi = math.atan2(d[1], d[0])
12        c, s = math.cos(phi), math.sin(phi)
13        R = np.array([[c, s], [-s, c]]) # world -> agent frame
14        x = np.diff((obs - origin) @ R.T, axis=0) / PRED_SCALE
15        std = None
16        if model is None:
17            step = x[-1:] if np.any(x[-1:]) else np.zeros((1, 2))
18            pos = np.cumsum(np.repeat(step, PRED_STEPS, axis=0), axis=0)
19        else:

```

```

20         pos, std = model.predict_frame(x[None], PRED_STEPS)
21         pos, std = np.asarray(pos)[0], np.asarray(std)[0]
22         world = origin + (pos * PRED_SCALE) @ R          # back to world,
(PRED_STEPS, 2)
23         src = np.vstack([origin[None], world])
24         t_src = PRED_DT * np.arange(PRED_STEPS + 1)
25         t_out = dt * np.arange(n_out)
26         out = np.empty((n_out, 2))
27         for a in (0, 1):
28             out[:, a] = np.interp(t_out, t_src, src[:, a])
29         late = t_out > t_src[-1]
30         if np.any(late):
31             v_end = (src[-1] - src[-2]) / PRED_DT
32             out[late] = src[-1] + (t_out[late][:, None] - t_src[-1]) * v_end
33         if std is None:
34             return out
35         sig = np.concatenate([[0.0], np.max(std, axis=1) * PRED_SCALE])
36         infl = np.minimum(KAPPA * np.interp(t_out, t_src, sig), INFL_MAX)
37         return out, infl
38
39     return predictor

```

Pass it to the runner as `prediction="learned"` together with `predictor= week10_predictor(model)`. It imports nothing from [Chapter 20](#): the model arrives as an object with one method, so the two files stay independent and any predictor with that method fits.

The covariance is where the level earns or loses its safety. A Gaussian model of [Chapter 20](#) returns a standard deviation per step next to its mean, and the adapter turns it into the inflation of the safety radius with the same κ_p and the same cap as the Kalman level of [Section 25.7](#). A model that returns means only gets no inflation at all. The difference is not cosmetic. Take the $(k, m) = (4, 1)$ cell with the hybrid strategy and the same twenty seeds. The uninflated stand-in reaches a mean drone–intruder separation of 0.87 m against 1.16 m for the Kalman level, a paired difference of -0.29 m $[-0.46, -0.12]$, and its collision rate is 0.40 against 0.05. An uncalibrated predictor is worse than a calibrated linear one, however good its mean error looks: [Chapter 20](#) calibrates the ellipses per horizon for exactly this reason, and [Exercise 25.7\(d\)](#) asks for the comparison.

25.12 Where this fits in the drone system

In the drone system

The experiment matrix of [Table 25.2](#) is a set of switches on the architecture of [Chapter 24](#), and [Figure 25.5](#) shows where each switch sits on its state machine.

Avoidance strategy selects transitions. *Local-only* removes every transition into Replanning: the drone dodges in Avoiding, reconnects in Reconnecting, and if reconnection fails it keeps dodging and chasing its reference. *Replan-only* removes Avoiding and Reconnecting: the time-to-collision trigger leads from Nominal straight into Replanning. The *hybrid* keeps all four states; *no avoidance* disables the trigger. Implement the switches as flags of one state machine, not as three planners, so that everything else stays identical across the rows.

Prediction quality selects what feeds the trigger and the layers — the true future, a Kalman filter on noisy measurements ([Chapter 18](#)), constant-velocity extrapolation or a learned predictor ([Chapter 20](#)) — and, through the covariance that inflates the

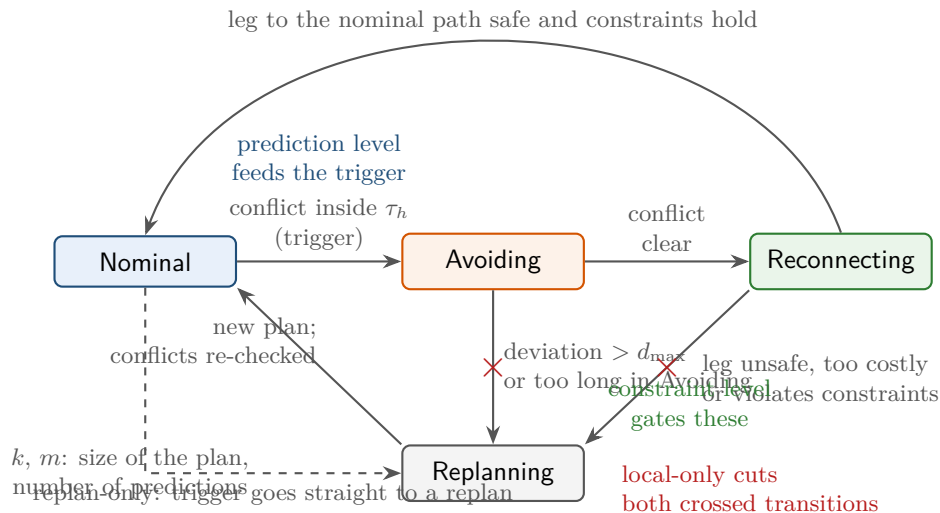


Figure 25.5. The state machine of Chapter 24 with the switches of the experiment matrix. Local-only cuts the two transitions into Replanning (crossed), replan-only cuts Avoiding and takes the dashed transition instead, and the hybrid keeps everything. The prediction level feeds the trigger, the constraint level gates the transitions out of Reconnecting and Replanning.

safety radius, how conservative every layer is. **Drones and intruders** set the size of the nominal plan and reservation table (Chapters 9 and 10) and the number of predicted trajectories each drone must check; both drive the effort columns. **Constraints** gate transitions and shape commands: a formation term in the preferred velocity of ORCA and in the MPC cost (Chapters 21 and 23), and the kept edges checked before Reconnecting may succeed and before a replanned path is accepted.

This chapter is the Week 12 milestone of the study plan (Appendix A), whose coding exercise — build the capstone and run controlled experiments — is Exercise 25.7 below.

25.13 Summary

Chapter summary

- The contribution of a hybrid planner lives in its evaluation: no theorem says that the combination is better than its parts, so the experiments must, and they must be repeatable.
- Ten metrics in four families: safety (collision rate with a Wilson interval, minimum separation with median, 5th percentile and near-miss rate), efficiency (path-length ratio, travel time, makespan, sum of costs), effort (replanning count, computation time per decision and per run) and constraints (formation error e_F , communication violations through kept edges or $\lambda_2 = 0$). Each has a unit, an estimator and things it is sensitive to (Table 25.1).
- Five factors make the experiment matrix; a full factorial is hundreds of cells, so use one-factor-at-a-time around a base cell plus a small factorial of the factors that interact. Baselines: no avoidance (sanity), local-only, replan-only, hybrid, and a MILP oracle on tiny instances.
- Scenarios are deterministic functions of a seed, including intruder trajectories

(straight, turning, evasive, aimed at a drone at a crossing angle) and measurement noise; every strategy sees the same scenarios, and comparisons use per-scenario differences (Proposition 25.8).

- Report rates with Wilson intervals and the rule of three (Proposition 25.9), means with t intervals, skewed metrics with medians and IQR, paired differences with intervals and d_z , success rates with the cap; draw box plots per level, cactus plots for effort, Pareto plots for the safety–efficiency trade-off.
- The mini-study shows the pattern: local-only cheapest and closest, replan-only widest and dearest with many replans, the hybrid in between; collision counts of 0/20 and 1/20 are indistinguishable, minimum separations are not.
- Reproducibility is a configuration file, seeds everywhere, a log of every decision, versioned code and raw per-run results; the report states the design, the hardware, the failures and the limitations.

Further reading. Stern et al. [161] define the MAPF benchmark and its protocol, and Sturtevant [162] the grid maps it uses; the CBS and ECBS papers [8, 150] and the ORCA paper [14] are worth rereading for their evaluation sections alone. Panerati et al. [125] describe the simulator `gym-pybullet-drones`, and Hönig et al. [67] a flight-tested evaluation of swarm trajectory planning. On experimental method, Hooker [68] and Barr et al. [9] are the classic critiques of how algorithms are compared, and McGeoch [116] the textbook of experimental algorithmics. The bootstrap is Efron and Tibshirani [37], the Wilson interval [179], the signed-rank test [177], effect sizes [29], Holm’s correction [65], and reproducible computational research Peng [129].

25.14 Exercises

Exercise 25.1 (★). Design an experiment matrix for the question “does a learned predictor make the hybrid planner safer than a constant-velocity predictor when four drones face one intruder?”. Say which factors are fixed and at which level, which cells are run, how many seeds and which baselines. Say which *primary* metric and paired comparison decide the question, and which secondary metrics are reported. Then add a sixth factor, the crossing angle with levels 45° , 90° and 135° , and count the cells of the full factorial of Table 25.2 with it and of a one-factor-at-a-time design around the base cell.

Exercise 25.2 (★★). A log with $\Delta t = 1$ s holds two drones and one intruder at four instants: drone 1 at $(0, 0)$, $(1, 0)$, $(2, 0)$, $(3, 0)$ with goal $(3, 0)$; drone 2 at $(0, 2)$, $(0, 2)$, $(1, 2)$, $(1, 2)$ with goal $(1, 2)$; the intruder at $(2, 4)$, $(2, 3)$, $(2, 2)$, $(2, 1)$. Drone radii are 0.3 m, the intruder’s 0.8 m, the goal tolerance 0.3 m. Compute by hand: (a) the drone–drone and drone–intruder minimum separations, from the samples and with linear motion between samples; (b) the collision indicator and count; (c) the path lengths, travel times, makespan and sum of costs; (d) $e_F(t)$ for the template $\mathbf{d}_{12} = (0, 2)$ with the single edge $\{1, 2\}$, its mean and its peak; (e) the communication violations for $R_{\text{comm}} = 2.5$ m and for 2.2 m. Check every answer with `ch25_evaluation.py`, whose self-test uses this log.

Exercise 25.3 (★★). A draft thesis chapter contains the following table, captioned “Results averaged over successful runs, 10 random scenarios per method”:

Method	Collisions	Path length	Runtime
Ours (hybrid)	0 %	17.3 m	12 ms
ORCA (as reported in the ORCA paper)	4 %	16.9 m	3 ms
Replan-only	0 %	18.1 m	40 ms

List at least six flaws, each with the pitfall it commits and the fix, and rewrite the table in the form of [Table 25.6](#) (placeholders where numbers are missing). What is the largest collision rate that “0 % of 10 runs” is consistent with?

Exercise 25.4 (★★). (a) With $\sigma_A = \sigma_B$ and a correlation $\rho = 0.9$ between the two strategies across scenarios, how many unpaired runs per strategy give the precision of 20 paired runs? (b) Construct a situation in which pairing *hurts*, and explain why it is rare in planner evaluations. (c) The hybrid has the shorter path in 16 of 20 scenarios and the longer one in 4: compute the two-sided sign-test p -value exactly, find the row of [Table 25.5](#) it belongs to, and explain why the Wilcoxon p -value of that row is smaller. (d) Why does the table report $d_{\min}$ as “0/7/13” for hybrid against local-only, and what does the 13 say about the two strategies?

Exercise 25.5 (★). (a) How many collision-free runs are needed to claim, at the 95 % level, that the collision probability is below 0.5 %? Give the exact number from $(1 - p)^N = 0.05$ and the rule-of-three approximation. (b) The hybrid’s path-length ratio in the four-drone cell has a t interval of half-width 0.026 with $N = 20$; how many seeds halve it, and how many bring it to 0.005? (c) Which goal is more expensive, and why does the answer differ between a rate and a mean?

Exercise 25.6 (★★). Two planners ran on the same ten scenarios with a cap of 100 ms of computation per run. Planner A finished in 12, 15, 18, 20, 22, 25, 31, 40, 55, 70 ms; planner B in 5, 6, 7, 8, 9, 10, 12, 15 ms and timed out twice. (a) Draw the cactus plot of both. (b) Compute the success rates, the mean runtime over finished runs and the PAR-2 scores. (c) Which planner is “faster”? Write the two sentences a results section should contain instead of that word. (d) Which numbers in (b) change if the cap is 200 ms, and what does that say about a mean runtime reported without its cap?

Exercise 25.7 (★★★ Coding). This is the Week 12 coding exercise of the study plan ([Appendix A](#)): build the capstone and run controlled experiments. (a) With `ch25_evaluation.py`, run the one-factor-at-a-time design around the base cell of [Table 25.2](#) as far as your machine allows: $k \in \{2, 4, 8\}$, $m \in \{0, 1, 2\}$, prediction in `{perfect, cv, noisy}`, all four strategies, 20 seeds, with the crossing angle as a knob. Budget the run first: the full grid of those levels is $3 \times 3 \times 3 = 27$ cells, so $27 \times 4 \times 20 = 2160$ runs. Because the eight-drone cells cost more per run than the two-drone ones, that is roughly ten times the mini-study, a few minutes; the one-factor-at-a-time design around the base cell is a quarter of it. (b) Add the constraint factor. Switch on the formation-preserving level of [Section 25.4](#) (the `formation` argument of the runner) and give the drones a kept edge set E_{keep} with a bounded R_{comm} , and report $\bar{e}_F$, $e_F^{\max}$ and n_{cv} next to the safety metrics. (c) Produce the box, cactus and Pareto figures and a results table with Wilson and t intervals from the raw rows by script, and write the results section of [Section 25.9](#), limitations included. (d) Run one cell of the prediction-quality factor at its learned level. Train the predictor of [Chapter 20](#) on the intruder families of [Section 25.5](#), wrap it with `week10_predictor()` ([Section 25.11.1](#)), and run the $(k, m) = (4, 1)$ cell with the same twenty seeds under `prediction="noisy"` and under `prediction="learned"`. Report the paired difference in $d_{\min}^{\text{di}}$ and in the collision rate, once with your model’s covariance feeding the inflation and once without it. Then say which of the two — the better mean or the calibrated covariance — your numbers make responsible for the difference. (e) Replace the stand-in strategies by the real modules — the CBS plan,

the Kalman filter, ORCA, the reconnection and D* Lite replanning of [Chapter 24](#) — behind the same `run_episode()` interface, rerun the same seeds, and report what changed and what did not.

Exercise 25.8 (★★★). Implement the interface of [Listing 25.1](#) for a physics simulator: `gym-pybullet-drones` [\[125\]](#) if you have it, otherwise a double-integrator model with acceleration limits and a proportional velocity controller with a lag of 0.3 s. Run the two-drone, one-intruder cell of [Example 25.10](#) with the same 20 seeds through it. Compare $d_{\min}$, ρ_L , the makespan and the computation time per decision with [Table 25.4](#), paired by seed. Explain every difference by a property of the simulator (tracking lag, acceleration limit, physics rate), and state which margin on R_{safe} would restore the toy’s near-miss rate.

Part IX

Appendices

The Twelve-Week Study Plan

This appendix is the book’s answer to a practical question: *in which order, and at what pace, should I work through these chapters if I want a working hybrid swarm planner in three months?* It reproduces the twelve-week training plan that the book was built from, maps every week to the chapters that teach it, and adds study notes, a capstone description, a completion checklist and a traceability table. Read it once before you start [Chapter 1](#), and return to it at the beginning of every week.

Learning objectives

After working through this appendix you will be able to

- choose a twelve-week route through this book and a weekly rhythm that fits the plan’s 6–8 hours of study;
- say, for every week, what to build and how to tell that it works, using the week’s “Done when” test rather than a feeling of understanding;
- name the four layers of the capstone architecture, the interface of each layer, and the decision logic that switches between them at run time;
- design the capstone experiment matrix, name the metrics to report, and say what the headline metrics hide;
- use the traceability table in both directions: from an algorithm of the plan to its chapter and week, and from a chapter to the depth the plan expects of you there.

A.1 How to use this plan

Goal. The goal of the plan is to build enough theory and implementation skill to design a **hybrid swarm planner** that handles two very different kinds of trouble: *planned* conflicts between the drones you control, and *unexpected* moving obstacles such as an unknown drone crossing your formation. The target outcome after twelve weeks is a working research prototype that combines a global multi-agent path finding (MAPF) solver (conflict-based search, CBS, or its bounded-suboptimal variant enhanced CBS (ECBS), [Chapters 9](#) and [10](#)), a local safety layer (optimal reciprocal collision avoidance, ORCA, or the dynamic window approach, DWA, [Chapters 13](#) and [14](#)), a prediction layer (a Kalman filter or a long short-term memory network, LSTM, [Chapters 18](#) and [20](#)) and a replanning layer (D^* Lite or A^* , [Chapters 4](#) and [5](#)). [Chapter 24](#) describes how these pieces fit together and [Chapter 25](#) how to evaluate the result.

The core idea of the whole plan

Use a global planner for planned routes, a local collision-avoidance method for surprises, and a replanner only when the local response is insufficient. Every week of the plan builds one of these three abilities, or the tracking and prediction that feeds them.

Study load. Plan for approximately 6–8 hours per week. The plan prioritizes coding over passive reading, and so does this appendix: a good split is at most two hours of reading and four to six hours at the keyboard. Each week has a *coding exercise* and a *milestone*. The milestone is a sentence you should be able to say truthfully on the last day of the week; if you cannot, spend the first hour of the next week closing the gap rather than moving on. Over twelve weeks that is 12×6 to 12×8 , or 72–96 hours in total, which is what you should budget before you start.

A weekly rhythm. On the first day, read the week’s chapters up to the worked example and trace that example by hand. On the next three days, write the coding exercise, using the chapter’s code file as an oracle. On the fifth day, compare your implementation with the reference on the same instance, write the milestone sentence into a lab notebook, and note what you would do differently; keep the weekend as a buffer. The notebook matters more than it looks: the capstone of Week 12, and any paper you write afterward, will draw on the numbers, failure cases and plots you produce in Weeks 1–11.

Suggested stack. The plan assumes the following tools. All of them are free, and the book’s own code uses only the first two.

- **Python 3.10+ and NumPy.** Every reference implementation in this book is plain Python with NumPy (SciPy only for the quadratic program (QP) and mixed-integer linear program (MILP) solvers of [Chapters 21](#) and [22](#)). Write your own code the same way; you will read and debug it faster.
- **NetworkX** [\[59\]](#) for quick graph experiments and for checking your graph code against an independent shortest-path implementation.
- **Matplotlib** for the pictures the exercises ask for: OPEN and CLOSED sets, trajectories, velocity obstacles, benchmark curves. A plot you look at every ten minutes is worth more than a printout you look at once.
- **PyBullet or gym-pybullet-drones** [\[125\]](#) for a 3D simulator with quadrotor dynamics. You need it from Week 8 onward and for the capstone; until then a 2D NumPy simulator is enough and much faster to iterate on.
- **Your own MAPF codebase**, if you have one. The plan was written for a reader who already owns a swarm or MAPF simulator; reuse its grid, instance and visualization code so that each week’s exercise is an addition, not a rewrite.

How the book’s code files support the exercises

Every chapter ships a Python file `code/chNN_*.py` (NN is the chapter number), and every listing in a chapter is a verbatim excerpt of that file. Each file ends with a self-test under `if __name__ == "__main__":` that runs in a few seconds and fails loudly with an `assert` if the implementation is wrong. Use the files in this order: *run* the self-test to see that the reference works; *write* your own version for the week’s exercise without looking at the reference; then *compare* both implementations on the same instance. The reference is your oracle, not your starting point. When the two disagree, the chapter’s worked

example, whose numbers were produced by that same file, tells you which one is wrong.

A.2 The schedule at a glance

Table A.1 is the twelve-week schedule of the training plan, with two columns added: *Read* names the chapters to read and *Exercise* the exercise of this book that is the week's deliverable. The *Learn*, *Coding exercise* and *Milestone* columns are the plan's own words. The chapters in the *Read* column are listed in the order in which you should read them; a chapter in parentheses is background or optional depth.

Table A.1. The twelve-week schedule, mapped to the chapters of this book and to the exercise that is the week's deliverable.

Wk	Topic	Read	Learn	Coding exercise	Exercise	Milestone
1	Foundations: Dijkstra and A*	Chapters 3 and 4 (Chapters 1 and 2)	Graph search, admissible/consistent heuristics, space-time representation.	Implement grid A*. Add time as a state variable. Visualize open/closed sets.	Exercises 3.7 and 4.8	You can explain exactly why A* returns an optimal path under the required conditions.
2	Dynamic replanning: LPA* and D* Lite	Chapter 5 (Chapter 6)	Incremental search and what changes when obstacles appear after planning.	Implement or adapt D* Lite. Change obstacles during execution and compare runtime with rerunning A*.	Exercise 5.7	Your agent can repair a path rather than restarting every search.
3	MAPF basics and conflict types	Chapters 7 and 8	Vertex conflicts, edge/swap conflicts, time-indexed paths, sum-of-costs and makespan.	Implement prioritized planning for 2–5 agents. Create deliberate conflict cases.	Exercises 7.9 and 8.8	You can represent and detect MAPF conflicts correctly.
4	Conflict-Based Search (CBS)	Chapter 9	Constraint tree, high-level search, low-level A*, branching on conflicts.	Implement basic CBS from scratch for a grid. Test 2, 4, and 8 agents.	Exercise 9.9	A working optimal CBS solver on small instances.
5	ECBS and practical MAPF	Chapter 10 (Chapter 11)	Bounded suboptimality, focal search, performance/optimality trade-off.	Add ECBS or reproduce an open implementation. Benchmark against CBS.	Exercise 10.7	You can measure the cost of faster suboptimal solutions.
6	Velocity Obstacles, RVO, and ORCA	Chapters 12 and 13	Collision cones, relative velocity, reciprocal avoidance.	Build a 2D simulation with agents crossing. Compare no avoidance, VO, and ORCA.	Exercises 12.7 and 13.9	You understand how a local velocity choice prevents a predicted collision.
7	DWA and Artificial Potential Fields	Chapters 14 and 15	Short-horizon control, local minima, oscillation, robot dynamics.	Implement DWA or APF in a continuous environment. Create failure cases.	Exercise 14.8 or Exercise 15.9	You know when local methods work and when they fail.

continued on the next page

Table A.1 (continued)

Wk	Topic	Read	Learn	Coding exercise	Exercise	Milestone
8	RRT, RRT*, and continuous 3D planning	Chapters 16 and 17	Sampling, steering, collision checks, path quality in continuous space.	Implement RRT* or use a library in a 3D obstacle field. Compare to grid A*.	Exercises 16 and 17.7	You can choose between graph search and sampling-based planning.
9	Tracking unknown moving obstacles	Chapters 18 and 19	Kalman/EKF tracking, state estimation, uncertainty.	Simulate an external drone observed with noisy position/velocity measurements. Track it with a Kalman filter.	Exercise 18.8	Your planner receives a stable estimated state instead of raw noisy observations.
10	Trajectory prediction	Chapter 20	Constant-velocity baselines, LSTM prediction, horizon and uncertainty.	Compare constant velocity vs LSTM on trajectory sequences. Measure ADE/FDE or comparable prediction errors.	Exercise 20.9	You can quantify whether learning actually improves prediction.
11	MPC and formation/-communication constraints	Chapters 21 and 23 (Chapter 22)	Receding-horizon optimization, hard/soft constraints, formation error, communication radius.	Create an MPC or simplified constrained controller that follows a path while preserving a formation or distance limit.	Exercises 21 and 23.8	Constraints are explicitly enforced rather than handled only after violation.
12	Hybrid system integration	Chapters 24 and 25	Global MAPF + reactive avoidance + prediction + replanning.	Build the capstone architecture (Section A.5) and run controlled experiments.	Exercises 24 and 25.7	A research prototype and an experiment matrix suitable for a paper/thesis chapter.

A.3 The twelve weeks on one timeline

The plan groups its algorithms into six families, labeled A to F, and asks for a different depth in each: *learn deeply* (A, B, C), *know well* (D, E) and *learn enough to integrate* (F). Figure A.1 shows how the twelve weeks are distributed over the families. Notice that seven of the twelve weeks – more than half – go to the three “learn deeply” families, and that the capstone gets a full week of its own. If you are short of time, the places to compress are Week 5 (benchmark two values of w instead of three), Week 8 (use a library for RRT*) and the MILP part of Week 11 (read Chapter 22, do not implement it). Do not compress Weeks 1, 4, 6, 9 or 12: each of them builds a layer of the capstone that the later weeks assume.

A.4 Study notes, week by week

The notes below add to each row of Table A.1 what a table cell cannot say: where to put your attention, what to build first, how to know that you are done, which mistakes cost the most time, and which chapter exercises to attempt. The *load* rating, from ★ (light) to ★★★ (heavy), estimates how much of the 6–8 hours the coding exercise takes for a reader who meets the prerequisites of Chapter 2; plan lighter weeks around heavier ones. When these notes say

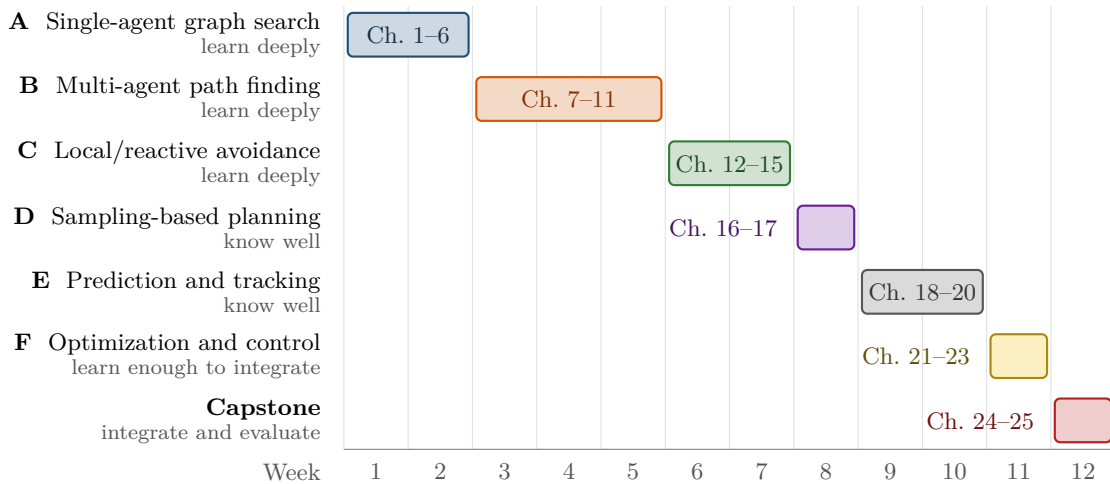


Figure A.1. The twelve weeks grouped by algorithm family. The length of a bar is the number of weeks the plan gives to the family; the label names the chapters read in those weeks (Chapters 1 and 2 are the foundations read in Week 1). Families A–C, which the plan asks you to learn deeply, take seven of the twelve weeks (2, 3 and 2); the capstone has a week of its own.

“the coding exercise of” a chapter, the exercise meant is the one named in the *Exercise* column of Table A.1; several chapters carry more than one exercise marked *Coding*, and that column says which of them is the week’s deliverable.

Moving on with a red test

The most expensive mistake in a self-study plan is to carry an unfinished week forward. A space-time A^* that mishandles waiting will break the prioritized planner of Week 3, the CBS low level of Week 4 and the replanning layer of Week 12, and each time it will look like a new bug. Figure A.2 draws the artifacts that later weeks reuse, so that you can see how far a red test travels before it is found. If the milestone sentence is not true on the last day of the week, spend the first hours of the next week on it and shorten that week’s exercise instead; Section A.3 names the parts that can be compressed.

A.4.1 Week 1: Foundations – Dijkstra and A^*

Load ★★ Read Chapters 3 and 4, with Chapters 1 and 2 for the vocabulary and the toolbox.

Focus. Dijkstra’s settled-node invariant on graphs with non-negative edge costs (a node popped from OPEN with the smallest g has its final cost), and what the heuristic buys in A^* : an admissible one makes the first expansion of the goal optimal, a consistent one makes a CLOSED set safe. Give real time to the space-time state (v, t) with a wait action; it is the low-level search of every MAPF solver in Part III.

Build. Grid A^* with a pluggable heuristic (Manhattan, octile, Euclidean, zero); the same search over (v, t) with a wait action and vertex and edge constraints; a plot that colors cells by their OPEN/CLOSED status.

Done when. You can state the optimality theorem with its conditions and say what breaks when each is dropped; your space-time search waits or detours around a cell reserved at a given time; with the zero heuristic both searches expand the same nodes as Dijkstra, up to tie-breaking.

Traps. Unit diagonal cost on an 8-connected grid makes the octile and Euclidean heuristics inadmissible; a space-time search without a time bound never terminates when the goal is

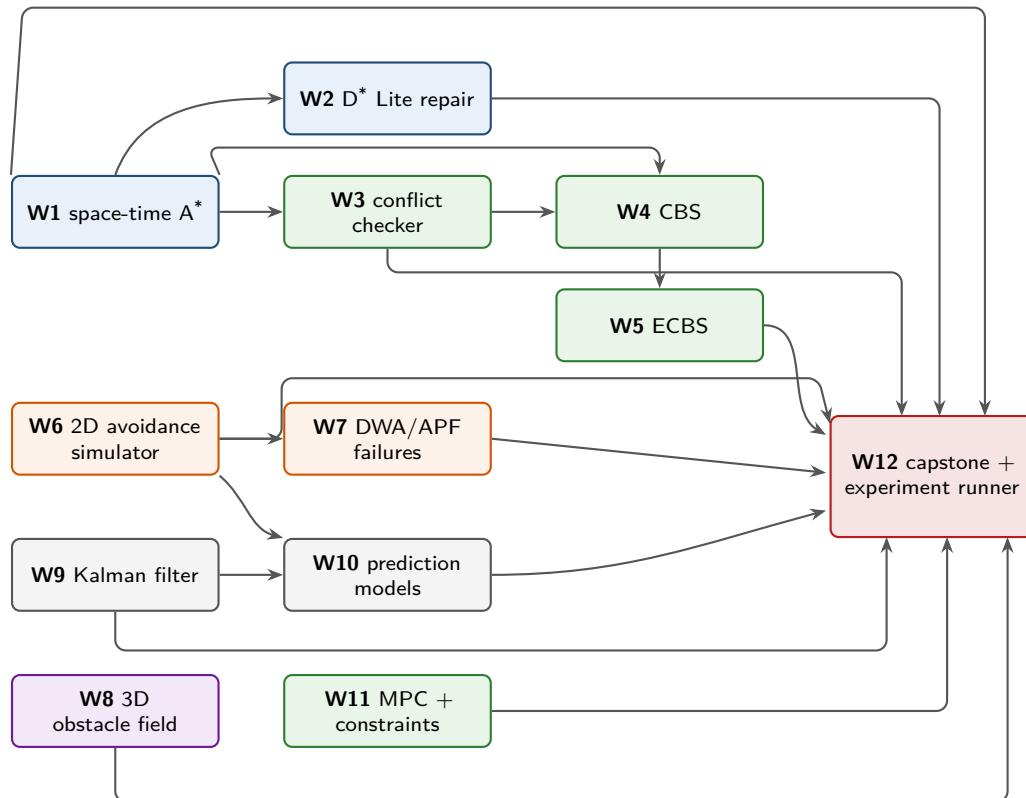


Figure A.2. What each week hands to the weeks after it. Only the reuse that the week notes assert is drawn: the space-time A^* of Week 1 is the low level of Weeks 3, 4 and of the replanning layer, the Week-3 checker is the conflict detector of CBS and of the capstone’s re-check step, and the Week-6 simulator produces both the failure cases of Week 7 and the trajectories that Week 10 learns from. Notice how much hangs on Weeks 1 and 3: four arrows leave Week 1 and two leave Week 3, so a bug carried out of either reappears in every week it feeds, which is why Section A.3 refuses to let you compress them.

unreachable; a search that stops the instant it reaches the goal cannot honor a later constraint on the goal cell.

Exercises. The coding exercise of Chapter 4 is the deliverable. Add the coding exercise of Chapter 3; that chapter’s backward Dijkstra becomes the low-level heuristic of CBS in Week 4.

A.4.2 Week 2: Dynamic replanning – LPA* and D* Lite

Load ★★★ **Read** Chapter 5; Chapter 6 if time permits.

Focus. Each vertex carries g and rhs and is locally inconsistent when they differ; keys, `UpdateVertex` and `ComputeShortestPath` only repair the inconsistent vertices, in the right order. D* Lite searches backward from the goal so that a moving start needs only the key modifier k_m instead of a re-sort of the queue.

Build. Turn the Week-1 search into D* Lite; move the agent one step at a time while obstacles appear and disappear ahead of it; count expansions and time per repair against a fresh A^* from the current position.

Done when. Expansions after a small change are a small fraction of a fresh A^* , and the path cost after every repair equals the fresh A^* cost; that equality is the correctness test.

Traps. Not adding to k_m when the start moves, which leaves stale keys in the queue; updating only one endpoint of a changed edge; timing grids so small that Python overhead dominates (count expansions too); changing the map inside `ComputeShortestPath`.

Exercises. The coding exercise of [Chapter 5](#); then, if time permits, the coding exercise of [Chapter 6](#), since an anytime search gives the replanning layer a path within a time budget, which D^* Lite alone does not promise.

A.4.3 Week 3: MAPF basics and conflict types

Load ★★ Read [Chapters 7](#) and [8](#).

Focus. The formal problem of [Chapter 7](#): agents, time steps, wait actions, the conflict types (vertex, edge, swapping, following, cycle) and the two objectives (sum of costs, makespan). Keep the two edge-like types apart: an edge conflict is the same edge in the same direction, a swap is the two agents exchanging ends. A swap is invisible if you compare positions only. Then [Chapter 8](#): prioritized planning as one space-time A^* per agent against a reservation table, fast but incomplete and suboptimal in general.

Build. A conflict checker that lists every conflict of a set of time-indexed paths with its type and time; prioritized planning for 2–5 agents; three deliberate conflict instances (head-on in a corridor, a crossing, an agent parked on another agent’s route).

Done when. The checker finds every planted conflict and nothing else; the planner solves the well-formed instances; you own an instance that one priority order fails and another solves.

Traps. Off-by-one in time (the position at time 0 is the start; a swap is agent i moving $u \rightarrow v$ while agent j moves $v \rightarrow u$ over the same step $t \rightarrow t + 1$); an agent that has arrived stays at its goal and must be reserved there for all later times; a reservation table that stores vertices but not edges.

Exercises. The coding exercises of [Chapters 7](#) and [8](#). Keep the checker: it is the conflict detector of your CBS high level next week and the “re-check inter-agent conflicts” step of the capstone.

A.4.4 Week 4: Conflict-Based Search

Load ★★★ Read [Chapter 9](#).

Focus. The constraint tree: a node holds constraints, one path per agent and the sum of costs; the high level expands the cheapest node, takes its first conflict and creates two children, each forbidding it for one agent; the low level replans only that agent with the space-time A^* of Week 1. Optimality follows because every conflict-free solution satisfies the constraint of at least one child.

Build. CBS from scratch on a grid: vertex constraints (agent, vertex, time), edge constraints (agent, edge, time), a constrained low level and the high-level search; test with 2, 4 and 8 agents on an open grid and on the Week-3 corridors.

Done when. Your plans pass the Week-3 checker; the sum of costs equals that of the reference implementation of [Chapter 9](#) on every instance; you can draw the constraint tree of a two-agent instance by hand and match it against your program’s log.

Traps. An edge constraint has a direction and a time; a low level that returns when it first pops the goal ignores later constraints on the goal cell; sorting the high level by makespan while reporting sum of costs; missing conflicts after the shorter path has ended.

Exercises. The coding exercise of [Chapter 9](#). Read about cardinal conflicts and bypass in the same chapter, but do not implement them yet; they matter when eight agents become twenty.

A.4.5 Week 5: ECBS and practical MAPF

Load ★★ Read [Chapter 10](#); [Chapter 11](#) for contrast.

Focus. Bounded suboptimality: a focal search may expand any node with $f \leq w \cdot f_{\min}$ and picks the one with the fewest conflicts; ECBS does this at both levels and guarantees a

cost within a factor w of optimal. Know where the lower bound comes from (each agent's low-level $f_{\min}$, summed).

Build. Turn your CBS into ECBS, or reproduce an open implementation as the plan allows; benchmark both for $w \in \{1.1, 1.5, 2\}$ with $w = 1$ (plain CBS) as the reference, as [Exercise 10.7\(a\)](#) asks: success rate under a time limit, runtime, high-level nodes and sum of costs, over several seeds.

Done when. A plot of cost against runtime as w varies, with $w = 1$ reproducing CBS's optimal cost, and a paragraph that says with numbers what suboptimality buys.

Traps. Using the current node's f instead of the minimum over OPEN as the focal bound; taking the returned path cost instead of $f_{\min}$ as the lower bound; not growing the focal list when $f_{\min}$ rises; concluding from one instance.

Exercises. The coding exercise of [Chapter 10](#). Read [Chapter 11](#) to know how M^* and Push-and-Swap differ from CBS; the plan asks for awareness, not an implementation.

A.4.6 Week 6: Velocity Obstacles, RVO and ORCA

Load ★★★ Read [Chapters 12](#) and [13](#).

Focus. Everything is relative: relative position, relative velocity and a Minkowski disk whose radius is the sum of the two radii. The velocity obstacle (VO) written $VO_{A|B}^\tau$ without an argument ([Chapter 13](#), [Definition 13.1](#)) is the set of *relative* velocities $\mathbf{v}_{\text{rel}} = \mathbf{v}_A - \mathbf{v}_B$ that bring the two disks into contact within the horizon τ ; its apex is at the origin, and ORCA reasons in exactly this form. *Adding* $\mathbf{v}_B$ gives the absolute set, the velocities of A that collide if B holds its current velocity, which [Chapter 12](#) writes with the argument, $VO_{A|B}^\tau(\mathbf{v}_B)$ ([Definition 12.7](#)), and whose relative form is recovered by subtracting $\mathbf{v}_B$ again ([Proposition 12.8](#)). Getting the direction of that translation the wrong way round is the commonest reading error of the week. The reciprocal velocity obstacle (RVO) shares the avoidance between cooperating agents; ORCA turns it into one half-plane per neighbor, so that the new velocity is a small linear program.

Build. A 2D simulation of crossing agents (two head-on, then eight on a circle exchanging places) in three modes, no avoidance, VO and ORCA, recording collisions, minimum separation and time to goal.

Done when. You can draw the cone and the ORCA half-plane for one pair and say which velocities each of them forbids. Across the three modes you should see no avoidance collide, VO avoid but oscillate or detour widely, and ORCA stay smooth and collision-free at the same speeds; if your radii and speeds put VO closer to ORCA than that, say so and report the separation numbers rather than doubting a correct implementation.

Traps. The orientation of the half-plane (it must exclude the velocity obstacle); the truncated cone for finite τ is a disk plus two tangent legs, not a triangle; when the linear program is infeasible in a crowd, choose the least-violating velocity rather than stopping; symmetric instances deadlock unless you break the symmetry.

Exercises. The coding exercises of [Chapters 12](#) and [13](#). Keep the simulator; [Week 7](#) and the capstone reuse it.

A.4.7 Week 7: DWA and Artificial Potential Fields

Load ★ Read [Chapters 14](#) and [15](#).

Focus. DWA searches velocity space: the dynamic window holds the velocities reachable within one control interval, admissible velocities are those from which the robot can still stop, and a weighted sum of heading, clearance and velocity terms picks the command. Artificial potential fields (APF) have no search, only forces, attractive to the goal and repulsive from obstacles, which is why APF is fast and why it has local minima and oscillation.

Build. One of the two (DWA if your drones have real acceleration limits, APF as a two-hour baseline) in the Week-6 simulator; then failure cases: a U-shaped obstacle, a narrow passage, a dead end that a short DWA horizon cannot see.

Done when. A picture of each failure with the reason, and a tuning table of how the DWA weights or the APF gains change the behavior.

Traps. DWA terms of different units summed without normalization; skipping the braking (admissibility) check; repulsive forces that grow without bound near obstacles (clamp them); comparing the methods at different control rates.

Exercises. The coding exercise of [Chapter 14](#) or of [Chapter 15](#), plus the conceptual exercises of the other chapter. DWA is the plan's alternative to ORCA in the local safety layer.

A.4.8 Week 8: RRT, RRT* and continuous 3D planning

Load ★★ Read [Chapters 16](#) and [17](#).

Focus. Sample, nearest, steer and collision-free plus a goal bias make RRT; the near set, parent choice and rewiring with a shrinking connection radius make RRT* asymptotically optimal; informed sampling restricts samples to the ellipsoid that can still improve the solution. Know what probabilistic completeness and asymptotic optimality promise, and that neither says anything about finite time.

Build. RRT*, yours or a library's, in a 3D field of box or sphere obstacles; grid A* on the same field at two or three resolutions; compare path length, time and memory.

Done when. A convergence plot of the RRT* cost against iterations, a table of A* per resolution against RRT* per sample budget, and a one-paragraph rule for when you would choose each.

Traps. Checking the new vertex but not the segment to it; a quadratic nearest-neighbor search (use a KD-tree above a few thousand nodes); a rewire that changes a parent without updating the descendants' costs; comparing raw paths without smoothing either.

Exercises. The coding exercises of [Chapters 16](#) and [17](#), which halve the week between them: [Exercise 16.8](#) is the first half, the RRT and the 3D obstacle field, and [Exercise 17.7](#) completes it with rewiring, the convergence plot and the comparison against grid A*. The 3D field you build here is the map of the capstone.

A.4.9 Week 9: Tracking unknown moving obstacles

Load ★★ Read [Chapters 18](#) and [19](#).

Focus. Predict with the motion model (the state moves, the covariance grows), update with the measurement (the gain weighs model against measurement). The constant-velocity model with white-noise-acceleration process noise is the workhorse; gating rejects outliers; a missing measurement is a predict without update. [Chapter 19](#) adds the extended and unscented Kalman filters (EKF, UKF) and the particle filter for targets that turn.

Build. An external drone on a curved path, observed with noisy position (optionally velocity) measurements at 10–20 Hz with dropouts; a Kalman filter with tuned **Q** and **R**; the error of the estimate against the truth next to the error of the raw measurements.

Done when. The estimate beats the raw measurements, the velocity estimate extrapolates one or two seconds ahead without jitter, and the Week-6 simulator consumes the estimate and its covariance instead of the observation.

Traps. **R** below the real noise (the filter trusts noise) or **Q** too small (it lags on turns and becomes overconfident); a wrong Δt in the transition matrix; trusting the covariance when the model is violated (check the normalized innovations); tuning on one trajectory.

Exercises. The coding exercise of [Chapter 18](#); the coding exercise of [Chapter 19](#) if your intruder turns sharply.

A.4.10 Week 10: Trajectory prediction

Load ★★★ Read [Chapter 20](#).

Focus. Average and final displacement error (ADE, FDE) as the yardsticks; constant-velocity and Kalman baselines; the LSTM as a sequence model that reads past positions and writes future ones; horizon, uncertainty and the evaluation protocol.

Build. Trajectories from the Week-6 simulator (agents that avoid one another are not constant-velocity, which is what makes learning worthwhile), split by trajectory rather than by time step; both baselines; an LSTM (a deep-learning framework is fine; the book’s own code stays in NumPy); ADE/FDE at 1, 2 and 3 s.

Done when. A table of ADE/FDE per horizon and method over several training seeds, and an honest conclusion: “the LSTM did not beat constant velocity on this data” is a legitimate and common result at short horizons.

Traps. Leakage between training and test trajectories; predicting absolute coordinates instead of displacements in the agent’s frame; a weak baseline that uses two raw points instead of a smoothed velocity; comparing at different horizons; tuning on the test set.

Exercises. The coding exercise of [Chapter 20](#). Save the model and the baselines; the “prediction quality” axis of the capstone uses all of them.

A.4.11 Week 11: MPC and formation/communication constraints

Load ★★★ Read [Chapters 21](#) and [23](#); [Chapter 22](#) as background.

Focus. Receding horizon: solve a short optimal control problem, apply the first input, repeat. With the double-integrator model and a quadratic cost it is a quadratic program; velocity and acceleration bounds are linear; separation constraints are nonconvex and are linearized around the previous solution; slack variables keep the program feasible. From [Chapter 23](#), displacement-based formation and the formation error. Mixed-integer linear programming (MILP, [Chapter 22](#)) is the exact alternative; read it for the cost of exactness.

Build. A model predictive controller (MPC), or the simplified constrained controller the plan allows, that follows a nominal path while keeping the formation error bounded or every pairwise distance below the communication radius; log violations and slack.

Done when. Zero violations with hard constraints, logged and small slack with soft ones, a plot of the formation error over time, and a visible difference when the constraint is switched off.

Traps. A horizon too long for the time budget or too short to see a conflict coming; hard constraints that turn infeasible when an intruder closes in (use slack); linearizing around a stale trajectory; a model Δt that differs from the control interval.

[Chapter 22](#) is the book’s addition to the plan’s Week 11: the plan gives MILP a Medium priority in its algorithm list but no week of its own, so it is read here, as background for the cost of exactness, and its coding exercise is optional.

Exercises. The coding exercises of [Chapters 21](#) and [23](#); the coding exercise of [Chapter 22](#) is optional.

A.4.12 Week 12: Hybrid system integration

Load ★★★ Read [Chapters 24](#) and [25](#).

Focus. The architecture and decision logic of [Section A.5](#): the safety horizon and the time-to-collision trigger, reconnection to the nominal path, the replanning triggers, the conflict re-check, and the experiment matrix and metrics of [Chapter 25](#).

Build. The capstone: the four layers you already own, connected behind clean interfaces, and an experiment runner that takes a scenario and a seed and writes one row of metrics per run.

Done when. The experiment matrix runs unattended from one script, every run is reproducible from its seed, and a table and plots compare local-only, replan-only and hybrid strategies under the same conditions.

Traps. One file that mixes the layers; different time steps for the discrete global plan and the continuous local layer (convert in one place); measuring success rate only; running experiments before each layer passes its own tests.

Exercises. The coding exercises of [Chapters 24](#) and [25](#).

A.5 The capstone: a hybrid collision-avoidance prototype

The capstone is the plan’s target outcome: a research prototype that combines a global planner, a prediction layer, a local safety layer and a replanning layer. Everything you built in Weeks 1–11 is a component of it. [Chapter 24](#) designs the architecture in full, with its state machine, its safety horizon and a discussion of which guarantees survive; [Chapter 25](#) shows how to evaluate it. This section restates the plan’s specification so that you can check your prototype against it.

A.5.1 The four layers

[Table A.2](#) lists the four layers in the plan’s words and [Figure A.3](#) draws them with the data that crosses each interface. The numbering is the plan’s; it is neither the order in which the layers act at run time (that is the decision logic below) nor the order in which you build them (that is the schedule).

Table A.2. The four layers of the capstone architecture, in the plan’s words, with the chapters that teach each layer and the weeks in which you build it.

Layer	Job	Chapters	Weeks
1 Global planner (CBS/ECBS)	Plan nominal, time-indexed paths for the drones under your control. Resolve conflicts among connected swarm members before execution. (Week 3 builds the conflict checker it needs.)	Chapters 9 and 10	4–5
2 Prediction layer (Kalman filter / LSTM)	Observe an unknown external drone. Estimate its current state and predict a short future trajectory with uncertainty.	Chapters 18 to 20	9–10
3 Local safety layer (ORCA or DWA)	When a predicted collision enters a short safety horizon, generate a temporary safe velocity or local maneuver without immediately discarding the global route.	Chapters 13 and 14	6–7
4 Replanning layer (D^* Lite or A^*)	If the local maneuver cannot safely return the drone to its nominal route, replan from the current state. Then re-check swarm conflicts and formation/communication constraints. (Week 11 adds the formation and communication constraints this re-check must test.)	Chapters 4 and 5	1–2, 11

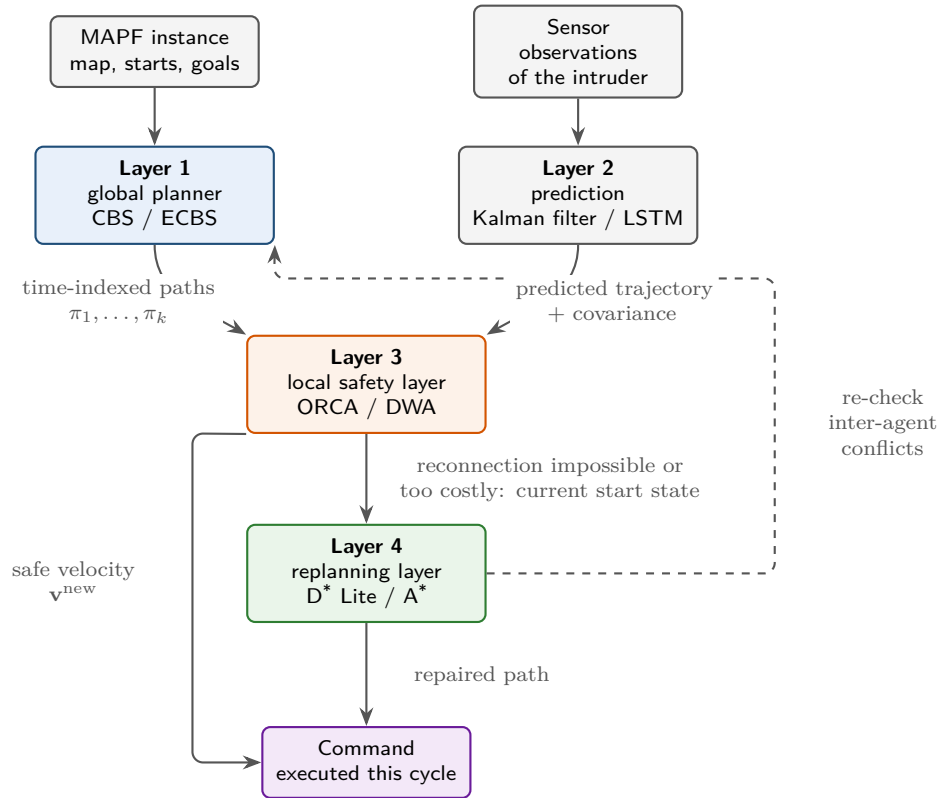


Figure A.3. The four layers of the capstone and the data that crosses each interface. Notice that every arrow carries one named object – time-indexed paths, a predicted trajectory with its covariance, a velocity, a repaired path – which is what makes the layers separable and the “avoidance strategy” axis of the experiments a switch rather than a rewrite. The dashed edge is the one feedback path: a replan invalidates the swarm plan, so Layer 1’s conflict check runs again.

A.5.2 The decision logic

The plan suggests five steps for every controlled drone, at every control cycle. [Chapter 24](#) turns them into a state machine with explicit triggers (a time to collision below a threshold, a reconnection cost above a bound, a violated constraint) and explains what happens to the optimality of the CBS plan once a drone leaves it. [Figure A.4](#) draws the five steps as a flowchart.

1. Follow the nominal CBS/ECBS path while no predicted conflict is inside the safety horizon.
2. If an unknown moving object creates a near-term risk, invoke ORCA/DWA to produce a local avoidance action.
3. After the conflict clears, attempt to reconnect to the nominal path at a safe future waypoint.
4. If reconnection is impossible, too costly, or violates formation/communication constraints, run D* Lite/A* from the current state.
5. Re-check inter-agent conflicts after any significant replan.

Keep the layers separable

Give each layer one entry point: the global planner takes an instance and returns time-indexed paths; the predictor takes observations and returns a predicted trajectory with

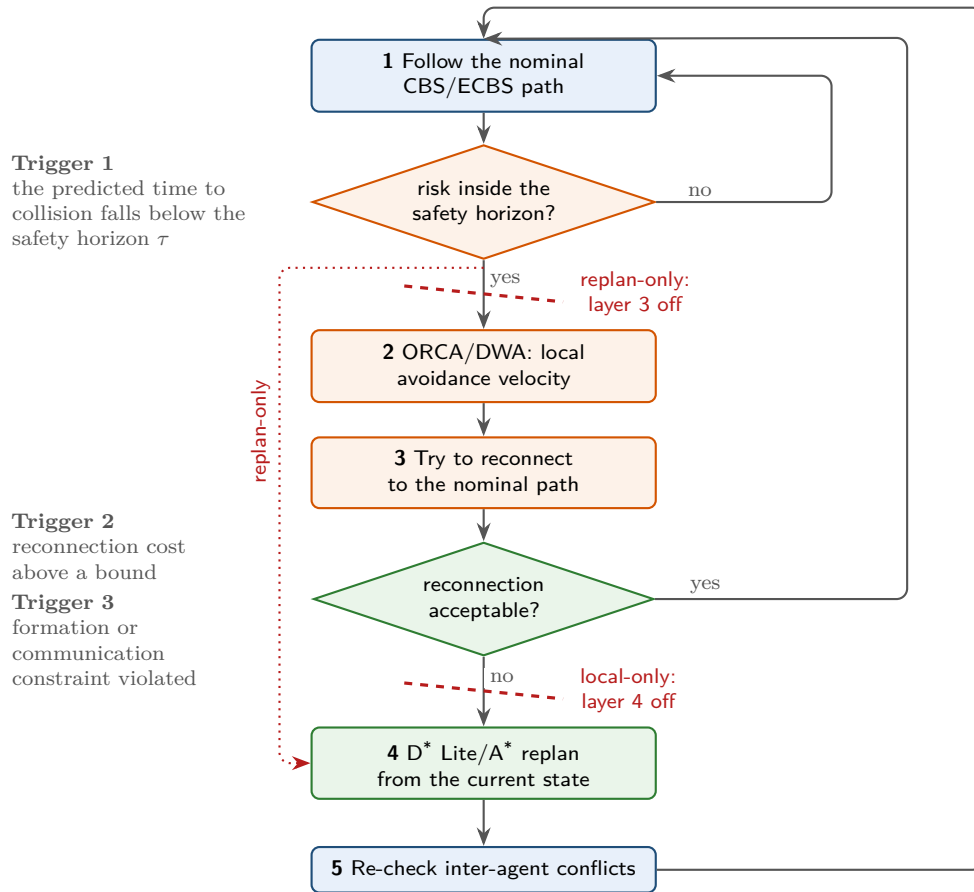


Figure A.4. The decision logic of the capstone, one pass per control cycle. Notice the three triggers on the left: the two diamonds are the only places where a drone leaves or rejoins its nominal CBS/ECBS path, so they are also the only places to instrument when you count replans. The two dashed cuts are the “avoidance strategy” axis of the experiment matrix: cutting layer 4 leaves the local-only strategy, cutting layer 3 leaves the replan-only strategy, and the whole chart is the hybrid. In replan-only the *yes* branch goes straight to step 4 (dotted); in local-only step 4 is never reached and the drone stays on the local layer until it can reconnect.

covariance; the local layer takes states and predictions and returns a velocity; the replanner takes a start state and returns a path. The “avoidance strategy” axis of the experiments is then a switch, not a rewrite: local-only disables layer 4, replan-only disables layer 3, the hybrid uses both.

A.5.3 Experiments and metrics

Table A.3 lists the factors the plan asks you to vary; Chapter 25 gives the scenario generator and the statistics to report. As a full factorial design the five factors – three swarm sizes, four intruder counts, four prediction qualities, three avoidance strategies and four constraint settings (two formation options crossed with two communication options) – have $3 \times 4 \times 4 \times 3 \times 4 = 576$ cells before seeds, and 11 520 runs at the 20 seeds per cell that Chapter 25 asks for; you will not run all of them. This is the count of Chapter 25 (Section 25.4 and Table 25.2): the swarm size is read as the three levels 2, 4, 8, with “and more if computationally feasible” an invitation rather than a fourth level, while the intruder count keeps four levels because “multiple crossing trajectories” is a level of its own. Fix every factor at a default, vary one or two at a time, and let the research question of Chapter 24 decide which.

Table A.3. The experiment factors of the capstone and their levels, in the plan’s words.

Factor	Levels	Where in this book
Controlled drones	2, 4, 8, and more if computationally feasible	Chapter 25 ; scaling of CBS/ECBS in Chapters 9 and 10
Unknown external drones	0, 1, 2, and multiple crossing trajectories	Chapter 25
Prediction quality	perfect state, noisy state, constant-velocity prediction, learned prediction	Chapters 18, 20 and 25
Avoidance strategy	local-only, global-replan-only, and hybrid local + replanning	Chapters 24 and 25
Constraints	no formation requirement vs formation-preserving; no communication radius vs bounded communication range	Chapters 21 and 23 ; during avoidance in Chapter 24

Metrics. Report, for every run, the metrics the plan lists: **collision rate**, **minimum separation**, **path length**, **travel time**, **makespan**, **sum of costs**, **replanning count**, **computation time**, **formation error** and **communication violations**. [Chapter 25](#) defines each of them and the statistics to report over seeds; makespan and sum of costs are the MAPF objectives of [Chapter 7](#), and the replanning count is the direct measure of the decision logic of [Chapter 24](#), since it counts how often the local layer was not enough. Two of them deserve a warning: a collision rate of zero hides near misses, which is what minimum separation is for, and a mean computation time hides the cycle that missed its deadline, so report the maximum as well.

Benchmarking on one instance

One instance, one seed and one trajectory split are the three cheapest ways to reach a conclusion that does not survive contact with a second one. Week 5 warns against concluding from one instance and Week 10 against tuning on the test set; the capstone inherits both traps at once, because a hybrid strategy that wins on a single crossing geometry can lose on the next. Fix the factors of [Table A.3](#) at a default, run every cell you report over several seeds, give the spread and not only the mean, and keep the instances you tune on apart from the instances you report on.

A.6 Completion checklist

The plan ends with ten statements. [Table A.4](#) repeats them and names the chapters where each ability is taught. Tick a line only when you can do what it says without opening the book; together, the ten lines are a fair summary of what a thesis committee or a reviewer will expect you to know.

An eight-question self-check. The chapters carry the graded exercises; the questions below are the plan’s own “Done when” tests turned into prompts, graded like those exercises from ★ to ★★★, and you should be able to answer all eight from your own lab notebook without opening a chapter.

1. ★ State the optimality theorem of A^* with its conditions, and say what breaks when each condition is dropped (Week 1).

Table A.4. The plan’s completion checklist, with the chapters that teach each item.

Statement	Where in this book
☐ I can implement and explain A^* and space-time A^* .	Chapter 4; reservation tables in Chapter 8
☐ I can explain the CBS constraint tree and implement vertex and edge constraints.	Chapter 9; conflict types in Chapter 7
☐ I understand the speed/optimalty trade-off in ECBS.	Chapter 10
☐ I can explain VO/RVO/ORCA using relative position and velocity.	Chapters 12 and 13
☐ I can track an unknown drone from noisy observations.	Chapters 18 and 19
☐ I can compare a simple prediction baseline with LSTM prediction.	Chapter 20
☐ I can trigger local avoidance based on a time-to-collision or safety-horizon rule.	time to collision in Chapter 12; the trigger in Chapter 24
☐ I can reconnect to the global route or trigger D^* Lite/ A^* replanning.	Chapter 5; reconnection in Chapter 24
☐ I can enforce formation and/or communication-range constraints.	Chapters 21 and 23; during avoidance in Chapter 24
☐ I can evaluate the hybrid system with reproducible metrics and controlled scenarios.	Chapter 25

2. ★★ Show a log in which the path cost after every D^* Lite repair equals the cost of a fresh A^* from the same position (Week 2).
3. ★ Give an instance on which one priority order fails and another succeeds, and name the conflict type that appears (Week 3).
4. ★★ Draw the constraint tree of a two-agent instance by hand and match it against your CBS log (Week 4).
5. ★★ Show your cost-against-runtime plot as w varies and say in numbers what suboptimality bought (Week 5).
6. ★★ Draw the VO cone and the ORCA half-plane for one crossing pair and say which velocities each of them forbids (Week 6).
7. ★★ Show the filtered estimate beating the raw measurements on one run, and the ADE/FDE table per horizon for constant velocity and the LSTM (Weeks 9–10).
8. ★★★ Name one run of your capstone in which the local layer was not enough, and say which trigger fired and what the replanning count was (Week 12).

A.7 Traceability: from the plan to the chapters

Table A.5 maps every algorithm of the training plan to the chapter that teaches it and the week that uses it, with the plan’s priority tag; where the plan qualified a tag, the qualification is kept next to it. The plan’s one-line reason for each algorithm is in the algorithm map of Chapter 1. Use the table in two directions: to check that the book covers everything the plan asks for, and, when you open a chapter, to see how much depth the plan expects of you there.

Table A.5. Every algorithm of the training plan, its priority, and where this book teaches it.

Algorithm	Priority in the plan	Chapter	Wk
A. Single-agent graph search – learn deeply			
Dijkstra	High	Chapter 3	1
A*	Essential	Chapter 4	1
D* Lite	Essential	Chapter 5	2
Lifelong Planning A* (LPA*)	Medium	Chapter 5	2
Anytime Repairing A* (ARA*)	Medium	Chapter 6	2
B. Multi-agent path finding – learn deeply			
Prioritized Planning	High	Chapter 8	3
Conflict-Based Search (CBS)	Essential	Chapter 9	4
Enhanced CBS (ECBS)	Essential	Chapter 10	5
M*	Medium	Chapter 11	5
Push-and-Swap / Push-and-Rotate	Awareness	Chapter 11	5
C. Local/reactive collision avoidance – learn deeply			
Velocity Obstacles (VO)	Essential	Chapter 12	6
Reciprocal Velocity Obstacles (RVO)	High	Chapter 13	6
ORCA	Essential	Chapter 13	6
Dynamic Window Approach (DWA)	High	Chapter 14	7
Artificial Potential Fields (APF)	High	Chapter 15	7
D. Sampling-based motion planning – know well			
RRT	High	Chapter 16	8
RRT*	High	Chapter 17	8
Informed RRT*	Medium	Chapter 17	8
E. Prediction and tracking – know well			
Kalman Filter	Essential	Chapter 18	9
Extended Kalman Filter (EKF)	High	Chapter 19	9
Unscented Kalman Filter (UKF)	Medium	Chapter 19	9
Particle Filter	Medium	Chapter 19	9
LSTM trajectory prediction	Essential for your planned research	Chapter 20	10
Transformer trajectory prediction	Awareness to High	Chapter 20	10
F. Optimization and control – learn enough to integrate			
Model Predictive Control (MPC)	Essential	Chapter 21	11
Mixed-Integer Linear Programming (MILP)	Medium	Chapter 22	11
Consensus / formation control	Essential for formation-preserving work	Chapter 23 (Chapter 21)	11

A.8 Reading order and primary sources

The plan also gives a reading order for the literature, reproduced in Table A.6 together with the chapter that covers each item and the primary source to read next to it. The first six items are the plan’s highest priority: they are three of the four capstone layers – the global planner (item 3), the local safety layer (items 5–6) and the replanning layer (items 1–2) – plus the MAPF definitions (item 4) that make the conflict re-check precise; the prediction layer’s sources come later, at item 12. To read a paper next to its chapter: read the chapter up

to the end of its worked example; then read the paper’s algorithm section and compare its pseudocode line by line with the chapter’s; then read the paper’s experiments and ask which of its claims your implementation of that week reproduces. Item 10 is a book rather than a paper: LaValle’s *Planning Algorithms* [99] is the reference to keep open for anything about configuration spaces, sampling and collision checking that [Chapters 2, 16 and 17](#) treat only as far as this book needs.

Table A.6. The plan’s reading order, mapped to chapters and to the primary sources.

#	Read	Chapter(s)	Primary source
1	A*	Chapter 4	Hart, Nilsson and Raphael [62]
2	D* Lite	Chapter 5	Koenig and Likhachev [88]
3	CBS	Chapter 9	Sharon et al. [150]
4	MAPF overview	Chapter 7	Stern et al. [161]
5	Velocity Obstacles	Chapter 12	Fiorini and Shiller [45]
6	ORCA	Chapter 13	van den Berg et al. [14]
7	DWA	Chapter 14	Fox, Burgard and Thrun [46]
8	RRT	Chapter 16	LaValle [98]
9	RRT*	Chapter 17	Karaman and Frazzoli [77]
10	Planning reference	Chapters 2, 16 and 17	LaValle, <i>Planning Algorithms</i> [99]
11	MPC	Chapter 21	Rawlings, Mayne and Diehl [134]
12	Trajectory prediction	Chapter 20	Rudenko et al. (survey) [141]; Alahi et al. (Social LSTM) [1]

Appendix summary

- The plan builds three abilities and the perception that feeds them: a global planner for planned routes, a local method for surprises, and a replanner for when the local response is not enough.
- The rhythm is 6–8 hours a week – at most two reading, four to six coding – and the milestone rule: if the week’s sentence is not true on the last day, close the gap in the first hours of the next week instead of moving on.
- If time runs short, compress Week 5 (two values of w instead of three), Week 8 (use a library for RRT*) and the MILP part of Week 11. Do not compress Weeks 1, 4, 6, 9 or 12: each builds a layer that the later weeks assume, as [Figure A.2](#) shows.
- The capstone has four layers – global planner, prediction, local safety, replanning ([Figure A.3](#)) – joined by the five-step decision logic of [Figure A.4](#), and is measured by the factor matrix of [Table A.3](#) and its ten metrics.
- The ten statements of [Table A.4](#) are the exit test. Tick a line only when you can do what it says without opening the book.

After Week 12. The plan is explicit that the individual algorithms above are established, and that the research lies in how they are combined, constrained, predicted and evaluated: unknown non-cooperative drones, local avoidance that does not needlessly destroy the guarantees of the global plan, formation and communication constraints during avoidance, prediction-aware constraints that carry uncertainty, an automatic decision between local action and global replan, computation for larger swarms, and benchmarks that compare local-only, replan-only and hybrid approaches under the same conditions. [Chapter 24](#) discusses these directions and [Chapter 25](#) the benchmark; the prototype and the experiment matrix you finish in Week 12

are the starting point for both.

Mathematical Refresher

Learning objectives

After working through this appendix you will be able to

- use the matrix identities (transpose, inverse, eigen-decomposition, definiteness, block matrices, Schur complement, least squares) that the filtering, control and consensus chapters rely on;
- linearize a nonlinear map with its Jacobian, bound the error, and step a double integrator forward exactly;
- write down a multivariate Gaussian, transform it affinely, condition it on a measurement, and gate with the Mahalanobis distance;
- tell a convex problem from a non-convex one, read the KKT conditions, and say what linear, quadratic and mixed-integer programs are and how each is solved;
- form the Laplacian of a graph and explain why its second eigenvalue measures connectivity;
- read a complexity bound and choose the right Python data structure for a search.

This appendix collects the mathematics that the chapters use without proving it. It is a refresher, not a course: every result is stated exactly, short proofs are given in full, long ones are sketched with a pointer to a textbook, and each result names the chapters that depend on it. Read a section when a chapter sends you here; nothing has to be read in order. The notation is that of the notation table at the front of the book, and every number quoted in the examples is produced by the script `code/figures/gen_appB_numbers.py`, so you can check your own hand calculations against it.

Key idea

Four moves carry almost every derivation in this book: diagonalize a symmetric matrix ($\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$), linearize a nonlinear map (keep the Jacobian, bound the rest), push a Gaussian through an affine map and condition it on what was measured (the Kalman filter is nothing else), and check convexity before trusting a local minimum.

B.1 Vectors and matrices

B.1.1 Vectors, norms and products

A vector $\mathbf{x} \in \mathbb{R}^n$ is a column of n real numbers $x_1, \dots, x_n$; its transpose $\mathbf{x}^T$ is the corresponding row. Positions, velocities and filter states are vectors of this kind, with $n = 2$ or 3 for a point

in the plane or in space and a larger n for a stacked state such as $\mathbf{x} = (\mathbf{p}, \mathbf{v})$.

Definition B.1 (Norms, dot product, cross product). For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ the **dot product** is $\mathbf{x} \cdot \mathbf{y} = \mathbf{x}^\top \mathbf{y} = \sum_i x_i y_i$. The **Euclidean norm** is $\|\mathbf{x}\| = \sqrt{\mathbf{x}^\top \mathbf{x}}$, the **1-norm** is $\|\mathbf{x}\|_1 = \sum_i |x_i|$ and the **∞ -norm** is $\|\mathbf{x}\|_\infty = \max_i |x_i|$. Every norm satisfies $\|\mathbf{x}\| \geq 0$ with equality only for $\mathbf{x} = \mathbf{0}$, $\|\alpha \mathbf{x}\| = |\alpha| \|\mathbf{x}\|$, and the triangle inequality $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$. For $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$ the **cross product** is

$$\mathbf{a} \times \mathbf{b} = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)^\top, \quad (\text{B.1})$$

and for $\mathbf{a}, \mathbf{b} \in \mathbb{R}^2$ the scalar $\mathbf{a} \times \mathbf{b} = a_x b_y - a_y b_x = \mathbf{a}^\perp \cdot \mathbf{b}$, with $\mathbf{a}^\perp = (-a_y, a_x)^\top$ the quarter turn of $\mathbf{a}$.

Three facts about these products are used all over the geometry of Chapters 2, 12 and 13. First, $\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta$ with θ the angle between the vectors, so $|\mathbf{x} \cdot \mathbf{y}| \leq \|\mathbf{x}\| \|\mathbf{y}\|$ (the **Cauchy–Schwarz inequality**) and two vectors are orthogonal exactly when their dot product is zero. Second, the **projection** of $\mathbf{x}$ onto a unit vector $\mathbf{u}$ is $(\mathbf{x} \cdot \mathbf{u}) \mathbf{u}$, which is how the distance from a point to a segment is computed in Chapter 2. Third, $\mathbf{a} \times \mathbf{b}$ in $\mathbb{R}^3$ is orthogonal to both factors with length $\|\mathbf{a}\| \|\mathbf{b}\| \sin \theta$, and the planar scalar $\mathbf{a} \times \mathbf{b}$ is positive exactly when $\mathbf{b}$ lies counterclockwise from $\mathbf{a}$: this sign is the left-or-right test behind the half-plane and cone tests of Chapters 12 and 13. The grid heuristics of Chapter 4 are norms of the displacement $\mathbf{d}$ to the goal: Manhattan $\|\mathbf{d}\|_1$, Euclidean $\|\mathbf{d}\|$, Chebyshev $\|\mathbf{d}\|_\infty$.

B.1.2 Matrices, transpose, inverse and rank

A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ maps $\mathbb{R}^n$ to $\mathbb{R}^m$ by $\mathbf{x} \mapsto \mathbf{A}\mathbf{x}$; the product $\mathbf{AB}$, with entries $(\mathbf{AB})_{ij} = \sum_k a_{ik} b_{kj}$, is the composition “first $\mathbf{B}$, then $\mathbf{A}$ ” and is not commutative in general. The **transpose** $\mathbf{A}^\top$ swaps rows and columns (a square $\mathbf{A}$ with $\mathbf{A}^\top = \mathbf{A}$ is **symmetric**); the **inverse** $\mathbf{A}^{-1}$ of a square matrix satisfies $\mathbf{AA}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$, when it exists; the **rank** is the number of linearly independent columns, which equals the number of linearly independent rows; the **trace** $\text{tr } \mathbf{A} = \sum_i a_{ii}$ and the **determinant** $\det \mathbf{A}$ are defined for square matrices; and an **orthogonal** matrix, $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$, such as a rotation, preserves norms and dot products. The following equivalences are the working definition of invertibility.

Proposition B.2 (Invertibility). *For a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ the following are equivalent: $\mathbf{A}$ is invertible; $\text{rank } \mathbf{A} = n$; $\det \mathbf{A} \neq 0$; $\mathbf{A}\mathbf{x} = \mathbf{0}$ only for $\mathbf{x} = \mathbf{0}$; $\mathbf{A}$ has no zero eigenvalue. In that case $\mathbf{A}\mathbf{x} = \mathbf{b}$ has the unique solution $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$.*

Table B.1 lists the algebra of transposes, inverses, traces and determinants that the derivations of Chapters 18, 21 and 23 use without comment; the block-matrix rows are explained in Appendix B.1.4.

B.1.3 Eigenvalues, symmetric matrices and definiteness

Definition B.3 (Eigenvalues and eigenvectors). A scalar λ and a non-zero vector $\mathbf{q}$ with $\mathbf{A}\mathbf{q} = \lambda \mathbf{q}$ are an **eigenvalue** and an **eigenvector** of the square matrix $\mathbf{A}$. The eigenvalues are the roots of $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$.

For a general real matrix the eigenvalues may be complex. The matrices that carry uncertainty, cost and connectivity in this book are symmetric, and for them everything is real and orthogonal.

Theorem B.4 (Spectral theorem). *A symmetric matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ has n real eigenvalues*

Table B.1. Matrix identities used in the book. All inverses are assumed to exist; λ_i are the eigenvalues; $\mathbf{S} = \mathbf{A} - \mathbf{B}\mathbf{D}^{-1}\mathbf{C}$ is the Schur complement of $\mathbf{D}$ in the block matrix $\mathbf{M}$ of [Theorem B.9](#).

Identity	Remark
$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$, $(\mathbf{AB})^{-1} = \mathbf{B}^{-1} \mathbf{A}^{-1}$	the order reverses
$(\mathbf{A}^\top)^{-1} = (\mathbf{A}^{-1})^\top$	written $\mathbf{A}^{-\top}$
$\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$, $\text{tr} \mathbf{A} = \sum_i \lambda_i$	the trace is cyclic
$\det(\mathbf{AB}) = \det \mathbf{A} \det \mathbf{B}$, $\det \mathbf{A}^\top = \det \mathbf{A}$, $\det \mathbf{A} = \prod_i \lambda_i$	$\det \mathbf{A}^{-1} = 1/\det \mathbf{A}$
$\mathbf{x}^\top \mathbf{A} \mathbf{x} = \frac{1}{2} \mathbf{x}^\top (\mathbf{A} + \mathbf{A}^\top) \mathbf{x}$	only the symmetric part matters
$\mathbf{A} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top$, $\mathbf{A}^{-1} = \mathbf{Q} \mathbf{\Lambda}^{-1} \mathbf{Q}^\top$ ($\mathbf{A}$ symmetric)	Theorem B.4
$\mathbf{A} = \mathbf{L} \mathbf{L}^\top$ ($\mathbf{A} \succ \mathbf{0}$, $\mathbf{L}$ lower triangular)	Cholesky factorization
$\det \mathbf{M} = \det \mathbf{S} \det \mathbf{D}$, $(\mathbf{M}^{-1})_{11} = \mathbf{S}^{-1}$	block matrices, Theorem B.9
$(\mathbf{A} + \mathbf{UCV})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1} \mathbf{U} (\mathbf{C}^{-1} + \mathbf{VA}^{-1} \mathbf{U})^{-1} \mathbf{VA}^{-1}$	Woodbury identity, Proposition B.10
$\nabla_{\mathbf{x}}(\mathbf{b}^\top \mathbf{x}) = \mathbf{b}$, $\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = (\mathbf{A} + \mathbf{A}^\top) \mathbf{x}$	derivatives, Appendix B.2

$\lambda_1 \geq \dots \geq \lambda_n$ with orthonormal eigenvectors $\mathbf{q}_1, \dots, \mathbf{q}_n$, so that

$$\mathbf{A} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top = \sum_{i=1}^n \lambda_i \mathbf{q}_i \mathbf{q}_i^\top, \quad \mathbf{Q} = [\mathbf{q}_1 \cdots \mathbf{q}_n], \quad \mathbf{Q}^\top \mathbf{Q} = \mathbf{I}, \quad \mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n). \quad (\text{B.2})$$

The proof is in every linear-algebra text [[24](#), Appendix A]. The decomposition says that a symmetric matrix is a pure scaling by λ_i along each of n orthogonal axes $\mathbf{q}_i$; hence $\mathbf{A}^k = \mathbf{Q} \mathbf{\Lambda}^k \mathbf{Q}^\top$ and, if no eigenvalue vanishes, $\mathbf{A}^{-1} = \mathbf{Q} \mathbf{\Lambda}^{-1} \mathbf{Q}^\top$. The **quadratic form** $\mathbf{x}^\top \mathbf{A} \mathbf{x} = \sum_i \lambda_i (\mathbf{q}_i^\top \mathbf{x})^2$ is a weighted sum of squared coordinates along the axes, and its sign decides whether a covariance is valid, a cost is convex and a consensus protocol converges.

Definition B.5 (Positive definite and semidefinite matrices). A symmetric matrix $\mathbf{A}$ is **positive semidefinite** (PSD), written $\mathbf{A} \succeq \mathbf{0}$, if $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$ for all $\mathbf{x}$, and **positive definite** (PD), written $\mathbf{A} \succ \mathbf{0}$, if $\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0$ for all $\mathbf{x} \neq \mathbf{0}$. $\mathbf{A} \succeq \mathbf{B}$ means $\mathbf{A} - \mathbf{B} \succeq \mathbf{0}$.

Proposition B.6 (Tests and closure properties). *Let $\mathbf{A}$ be symmetric.*

- (i) $\mathbf{A} \succeq \mathbf{0}$ iff all eigenvalues are ≥ 0 , and $\mathbf{A} \succ \mathbf{0}$ iff all are > 0 . A PD matrix is invertible and its inverse is PD.
- (ii) $\mathbf{A} \succeq \mathbf{0}$ iff $\mathbf{A} = \mathbf{B}^\top \mathbf{B}$ for some $\mathbf{B}$. If $\mathbf{A} \succ \mathbf{0}$, $\mathbf{B}^\top$ can be chosen lower triangular with positive diagonal, $\mathbf{A} = \mathbf{L} \mathbf{L}^\top$ (the **Cholesky factorization**), and this $\mathbf{L}$ is unique.
- (iii) If $\mathbf{A}, \mathbf{B} \succeq \mathbf{0}$ and $\alpha, \beta \geq 0$ then $\alpha \mathbf{A} + \beta \mathbf{B} \succeq \mathbf{0}$; for any $\mathbf{C}$ of compatible size $\mathbf{C}^\top \mathbf{A} \mathbf{C} \succeq \mathbf{0}$; and $\mathbf{A} + \varepsilon \mathbf{I} \succ \mathbf{0}$ for every $\varepsilon > 0$.

Proof sketch. (i) and (iii) are read off the quadratic form $\sum_i \lambda_i (\mathbf{q}_i^\top \mathbf{x})^2$ and the definition, using $\mathbf{x}^\top \mathbf{C}^\top \mathbf{A} \mathbf{C} \mathbf{x} = (\mathbf{C} \mathbf{x})^\top \mathbf{A} (\mathbf{C} \mathbf{x})$; the eigenvalues of $\mathbf{A}^{-1}$ are $1/\lambda_i$. For (ii), $\mathbf{x}^\top \mathbf{B}^\top \mathbf{B} \mathbf{x} = \|\mathbf{B} \mathbf{x}\|^2 \geq 0$; conversely $\mathbf{B} = \mathbf{\Lambda}^{1/2} \mathbf{Q}^\top$ works, and Gaussian elimination without pivoting produces the unique triangular factor [[24](#), Appendix C]. $\square$

Geometrically, the level set $\mathbf{x}^\top \mathbf{A} \mathbf{x} = 1$ of a PD quadratic form is an ellipsoid with axes $\mathbf{q}_i$ and semi-axes $1/\sqrt{\lambda_i}$; the covariance ellipse of a Gaussian in [Appendix B.3](#) is the level set of the form built from $\mathbf{\Sigma}^{-1}$, so its semi-axes are $\sqrt{\lambda_i(\mathbf{\Sigma})}$.

Example B.7 (A covariance matrix taken apart). The matrix

$$\mathbf{\Sigma} = \begin{bmatrix} 2.0 & 1.2 \\ 1.2 & 1.0 \end{bmatrix} \quad (\text{B.3})$$

is symmetric with $\text{tr} \mathbf{\Sigma} = 3$ and $\det \mathbf{\Sigma} = 0.56$. Its eigenvalues solve $\lambda^2 - 3\lambda + 0.56 = 0$

and are $\lambda_1 = 2.8$ and $\lambda_2 = 0.2$: both positive, so $\Sigma \succ \mathbf{0}$. The eigenvector of λ_1 is $\mathbf{q}_1 = (3, 2)^\top / \sqrt{13} = (0.832, 0.555)^\top$, at an angle of 33.7° to the x -axis, and $\mathbf{q}_2 = (-2, 3)^\top / \sqrt{13}$ is orthogonal to it. The ellipse $\mathbf{x}^\top \Sigma^{-1} \mathbf{x} = 1$ has semi-axes $\sqrt{2.8} = 1.673$ along $\mathbf{q}_1$ and $\sqrt{0.2} = 0.447$ along $\mathbf{q}_2$; it is drawn in Figure B.1. The Cholesky factor is $\mathbf{L} = \begin{bmatrix} 1.4142 & 0 \\ 0.8485 & 0.5292 \end{bmatrix}$, and $\Sigma^{-1} = \frac{1}{0.56} \begin{bmatrix} 1.0 & -1.2 \\ -1.2 & 2.0 \end{bmatrix} = \begin{bmatrix} 1.786 & -2.143 \\ -2.143 & 3.571 \end{bmatrix}$. Appendix B.3 uses this Σ as the covariance of a Gaussian.

B.1.4 Block matrices and the Schur complement

The large matrices of this book come in blocks: the covariance of $(\mathbf{p}, \mathbf{v})$, the joint covariance of a state and a measurement, the KKT matrix of a quadratic program. Blocks multiply like scalars as long as their order is kept,

$$\begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{C} & \mathbf{D} \end{bmatrix} \begin{bmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{G} & \mathbf{H} \end{bmatrix} = \begin{bmatrix} \mathbf{AE} + \mathbf{BG} & \mathbf{AF} + \mathbf{BH} \\ \mathbf{CE} + \mathbf{DG} & \mathbf{CF} + \mathbf{DH} \end{bmatrix}, \quad (\text{B.4})$$

a block-diagonal matrix is inverted block by block, and the transpose of a block matrix transposes both the arrangement and the blocks.

Definition B.8 (Schur complement). Let $\mathbf{M} = \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{C} & \mathbf{D} \end{bmatrix}$ with $\mathbf{A}$ and $\mathbf{D}$ square. If $\mathbf{D}$ is invertible, the **Schur complement** of $\mathbf{D}$ in $\mathbf{M}$ is $\mathbf{S} = \mathbf{A} - \mathbf{BD}^{-1}\mathbf{C}$; if $\mathbf{A}$ is invertible, the Schur complement of $\mathbf{A}$ in $\mathbf{M}$ is $\mathbf{S}' = \mathbf{D} - \mathbf{CA}^{-1}\mathbf{B}$.

Theorem B.9 (Block factorization, determinant and inverse). If $\mathbf{D}$ is invertible and $\mathbf{S} = \mathbf{A} - \mathbf{BD}^{-1}\mathbf{C}$, then

$$\mathbf{M} = \begin{bmatrix} \mathbf{I} & \mathbf{BD}^{-1} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{S} & \mathbf{0} \\ \mathbf{0} & \mathbf{D} \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{D}^{-1}\mathbf{C} & \mathbf{I} \end{bmatrix}, \quad \det \mathbf{M} = \det \mathbf{S} \det \mathbf{D}, \quad (\text{B.5})$$

and $\mathbf{M}$ is invertible iff $\mathbf{S}$ is, with

$$\mathbf{M}^{-1} = \begin{bmatrix} \mathbf{S}^{-1} & -\mathbf{S}^{-1}\mathbf{BD}^{-1} \\ -\mathbf{D}^{-1}\mathbf{CS}^{-1} & \mathbf{D}^{-1} + \mathbf{D}^{-1}\mathbf{CS}^{-1}\mathbf{BD}^{-1} \end{bmatrix}. \quad (\text{B.6})$$

If instead $\mathbf{A}$ is invertible and $\mathbf{S}' = \mathbf{D} - \mathbf{CA}^{-1}\mathbf{B}$, the same statements hold with the roles of the blocks exchanged; in particular $\det \mathbf{M} = \det \mathbf{A} \det \mathbf{S}'$, $(\mathbf{M}^{-1})_{22} = \mathbf{S}'^{-1}$ and $(\mathbf{M}^{-1})_{11} = \mathbf{A}^{-1} + \mathbf{A}^{-1}\mathbf{BS}'^{-1}\mathbf{CA}^{-1}$.

Proof. Multiplying out the right-hand side of Equation (B.5) with Equation (B.4) gives back $\mathbf{M}$. The outer factors are block-triangular with identity diagonal blocks, so they have determinant 1 and are inverted by negating the off-diagonal block; the middle factor has determinant $\det \mathbf{S} \det \mathbf{D}$ and is inverted block by block. Inverting the three factors in reverse order and multiplying out gives Equation (B.6). The second statement is the first applied to $\mathbf{M}$ with its block rows and block columns swapped. $\square$

In Example B.7 the Schur complement of the $(2, 2)$ entry is $2.0 - 1.2^2/1.0 = 0.56$, which is both $\det \Sigma / \Sigma_{22}$ and the reciprocal of the top-left entry 1.786 of Σ^{-1} , as the theorem says. Reading the theorem in both directions gives the identity with which Chapter 18 passes between the covariance and the information form of the Kalman update, and which turns the inversion of a large matrix into that of a small one whenever the update has low rank.

Proposition B.10 (Matrix inversion lemma, Woodbury identity). For $\mathbf{A} \in \mathbb{R}^{n \times n}$, $\mathbf{U} \in \mathbb{R}^{n \times k}$, $\mathbf{C} \in \mathbb{R}^{k \times k}$ and $\mathbf{V} \in \mathbb{R}^{k \times n}$, whenever all inverses below exist,

$$(\mathbf{A} + \mathbf{UCV})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{U}(\mathbf{C}^{-1} + \mathbf{VA}^{-1}\mathbf{U})^{-1}\mathbf{VA}^{-1}. \quad (\text{B.7})$$

Proof. Apply [Theorem B.9](#) to $\mathbf{M} = \begin{bmatrix} \mathbf{A} & \mathbf{U} \\ \mathbf{V} & -\mathbf{C}^{-1} \end{bmatrix}$: eliminating the (2, 2) block gives $(\mathbf{M}^{-1})_{11} = (\mathbf{A} + \mathbf{UCV})^{-1}$, eliminating the (1, 1) block gives $(\mathbf{M}^{-1})_{11} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{U}(\mathbf{C}^{-1} + \mathbf{VA}^{-1}\mathbf{U})^{-1}\mathbf{VA}^{-1}$, and the two expressions are equal. $\square$

B.1.5 Least squares and the normal equations

Theorem B.11 (Normal equations). *Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$. A vector $\mathbf{x}^*$ minimizes $\|\mathbf{Ax} - \mathbf{b}\|^2$ iff it satisfies the **normal equations***

$$\mathbf{A}^\top \mathbf{A} \mathbf{x}^* = \mathbf{A}^\top \mathbf{b}. \quad (\text{B.8})$$

If $\mathbf{A}$ has full column rank n , then $\mathbf{A}^\top \mathbf{A} \succ \mathbf{0}$ and the minimizer is unique, $\mathbf{x}^ = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}$. For a weight matrix $\mathbf{W} \succ \mathbf{0}$ the minimizer of $(\mathbf{Ax} - \mathbf{b})^\top \mathbf{W} (\mathbf{Ax} - \mathbf{b})$ solves $\mathbf{A}^\top \mathbf{W} \mathbf{A} \mathbf{x}^* = \mathbf{A}^\top \mathbf{W} \mathbf{b}$.*

Proof. With $\mathbf{r} = \mathbf{Ax}^* - \mathbf{b}$ and $\mathbf{d} = \mathbf{x} - \mathbf{x}^*$, $\|\mathbf{Ax} - \mathbf{b}\|^2 = \|\mathbf{r}\|^2 + 2\mathbf{d}^\top \mathbf{A}^\top \mathbf{r} + \|\mathbf{Ad}\|^2$. If [Equation \(B.8\)](#) holds, the middle term vanishes for every $\mathbf{d}$ and $\mathbf{x}^*$ is a minimizer; if not, $\mathbf{d} = -t\mathbf{A}^\top \mathbf{r}$ with a small $t > 0$ makes the last two terms sum to a negative number. Full column rank gives $\|\mathbf{Ad}\|^2 > 0$ for $\mathbf{d} \neq \mathbf{0}$, hence uniqueness; the weighted case is the unweighted one for $\mathbf{W}^{1/2}\mathbf{A}$ and $\mathbf{W}^{1/2}\mathbf{b}$, with $\mathbf{W}^{1/2} = \mathbf{Q}\mathbf{\Lambda}^{1/2}\mathbf{Q}^\top$ from [Theorem B.4](#). $\square$

Example B.12 (Fitting a constant-velocity line). A drone is seen at the positions 0.0, 1.1 and 1.9 m at the times $t = 0, 1, 2$ s along one axis. The model $p(t) = p_0 + vt$ is linear in $\mathbf{x} = (p_0, v)^\top$, with rows $(1, t)$ in $\mathbf{A}$ and $\mathbf{b} = (0.0, 1.1, 1.9)^\top$, so $\mathbf{A}^\top \mathbf{A} = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}$, $\mathbf{A}^\top \mathbf{b} = (3.0, 4.9)^\top$, and [Equation \(B.8\)](#) gives $p_0 = 0.05$ m and $v = 0.95$ m/s. The residuals $\mathbf{Ax}^* - \mathbf{b} = (0.05, -0.10, 0.05)^\top$ have squared norm 0.015 and are orthogonal to both columns of $\mathbf{A}$, which is what the normal equations say.

Least squares is the quiet workhorse of Parts VI and VII: the Kalman update of [Chapter 18](#) is a weighted least-squares compromise between a prediction and a measurement, weighted by the inverses of their covariances, and an MPC problem without inequality constraints ([Chapter 21](#)) is a least-squares problem in the stacked inputs, which the constraints turn into the quadratic program of [Appendix B.4.4](#).

Do not form the inverse

The formulas above are written with $\mathbf{A}^{-1}$ because that is how they are proved, not how they should be computed. Solving $\mathbf{Ax} = \mathbf{b}$ with `numpy.linalg.solve`, or with a Cholesky factor when $\mathbf{A} \succ \mathbf{0}$, is faster and more accurate than forming `numpy.linalg.inv(A)` and multiplying. Two more habits pay off in [Chapters 18](#) and [19](#): re-symmetrize a covariance as $\frac{1}{2}(\mathbf{P} + \mathbf{P}^\top)$ after each update, and check its smallest eigenvalue before a Cholesky factorization, because rounding can turn a PSD matrix into one with a slightly negative eigenvalue.

B.2 Calculus for motion

B.2.1 Gradient, Jacobian, Hessian and the chain rule

Definition B.13 (Derivatives of vector functions). Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ and $\mathbf{g}: \mathbb{R}^n \rightarrow \mathbb{R}^m$ be differentiable. The **gradient** $\nabla f(\mathbf{x}) \in \mathbb{R}^n$ has the entries $\partial f / \partial x_i$; the **Jacobian** $\mathbf{J}_\mathbf{g}(\mathbf{x}) \in \mathbb{R}^{m \times n}$ has the entries $\partial g_i / \partial x_j$, one row per output and one column per input, so that the gradient is the transpose of the Jacobian of a scalar function; the **Hessian** $\nabla^2 f(\mathbf{x}) \in \mathbb{R}^{n \times n}$ has the entries $\partial^2 f / \partial x_i \partial x_j$ and is symmetric when f is twice continuously differentiable.

The gradient points in the direction of steepest ascent and is orthogonal to the level set of f through $\mathbf{x}$; the derivative of f along a unit vector $\mathbf{u}$ is $\nabla f(\mathbf{x})^\top \mathbf{u}$. The **chain rule** for a composition is a matrix product: if $\mathbf{h}(\mathbf{x}) = \mathbf{g}(\mathbf{f}(\mathbf{x}))$ then

$$\mathbf{J}_{\mathbf{h}}(\mathbf{x}) = \mathbf{J}_{\mathbf{g}}(\mathbf{f}(\mathbf{x})) \mathbf{J}_{\mathbf{f}}(\mathbf{x}), \quad (\text{B.9})$$

which is how the gradients of a neural network in [Chapter 20](#) are propagated backward through its layers. The derivatives that recur in this book are

$$\begin{aligned} \nabla(\mathbf{b}^\top \mathbf{x}) &= \mathbf{b}, & \nabla\left(\frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x}\right) &= \mathbf{A} \mathbf{x} \quad (\mathbf{A} \text{ symmetric}), \\ \nabla\left(\frac{1}{2} \|\mathbf{x} - \mathbf{c}\|^2\right) &= \mathbf{x} - \mathbf{c}, & \nabla \|\mathbf{x} - \mathbf{c}\| &= \frac{\mathbf{x} - \mathbf{c}}{\|\mathbf{x} - \mathbf{c}\|}, \end{aligned} \quad (\text{B.10})$$

together with $\mathbf{J}_{\mathbf{A}\mathbf{x}} = \mathbf{A}$ and $\nabla^2(\frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x}) = \mathbf{A}$. The fourth expression holds only for $\mathbf{x} \neq \mathbf{c}$, because $\|\mathbf{x} - \mathbf{c}\|$ is not differentiable at its own center. The attractive potential of [Chapter 15](#) is the third expression with $\mathbf{c}$ the goal, so its force $-\nabla U$ points at the goal; the repulsive potential is a function of the clearance to an obstacle, and by the chain rule its gradient is a multiple of the unit vector in the fourth expression, which points away from the obstacle—a formula that the code must guard at zero clearance, where the gradient is undefined.

Proposition B.14 (Taylor expansion). *If $\mathbf{g}: \mathbb{R}^n \rightarrow \mathbb{R}^m$ is twice continuously differentiable near $\mathbf{x}$, then for small δ*

$$\mathbf{g}(\mathbf{x} + \delta) = \mathbf{g}(\mathbf{x}) + \mathbf{J}_{\mathbf{g}}(\mathbf{x}) \delta + \mathbf{r}(\delta), \quad \|\mathbf{r}(\delta)\| \leq c \|\delta\|^2 \quad (\text{B.11})$$

for a constant c that depends on the second derivatives of $\mathbf{g}$ near $\mathbf{x}$. For a scalar f the next term is explicit: $f(\mathbf{x} + \delta) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \delta + \frac{1}{2} \delta^\top \nabla^2 f(\mathbf{x}) \delta + \mathcal{O}(\|\delta\|^3)$.

Dropping $\mathbf{r}$ in [Equation \(B.11\)](#) is **linearization**. The extended Kalman filter of [Chapter 19](#) replaces its motion and measurement models by their Jacobians $\mathbf{F}$ and $\mathbf{H}$ at the current estimate, and the size of $\mathbf{r}$ over the spread of the estimate decides whether that is good enough or whether the unscented or particle filters of the same chapter are needed; the second-order expansion underlies Newton's method and the convexity test of [Appendix B.4](#).

Example B.15 (Range and bearing of a target). A radar at the origin measures the range r and the bearing φ of a drone with state $\mathbf{x} = (p_x, p_y, v_x, v_y)^\top$:

$$\mathbf{h}(\mathbf{x}) = \begin{bmatrix} \sqrt{p_x^2 + p_y^2} \\ \text{atan2}(p_y, p_x) \end{bmatrix}, \quad \mathbf{H} = \mathbf{J}_{\mathbf{h}}(\mathbf{x}) = \begin{bmatrix} p_x/r & p_y/r & 0 & 0 \\ -p_y/r^2 & p_x/r^2 & 0 & 0 \end{bmatrix}.$$

At $p_x = 3$, $p_y = 4$ (so $r = 5$) the Jacobian is $\mathbf{H} = \begin{bmatrix} 0.6 & 0.8 & 0 & 0 \\ -0.16 & 0.12 & 0 & 0 \end{bmatrix}$ and $\mathbf{h} = (5, 0.9273)^\top$. For the displacement $\delta = (0.1, -0.2, 0, 0)^\top$ the linear prediction is $\mathbf{h} + \mathbf{H}\delta = (4.9000, 0.8873)^\top$, and the exact value $(4.9041, 0.8865)^\top$ exceeds it by $(0.0041, -0.0008)^\top$: an error of second order in $\|\delta\| = 0.22$, as [Equation \(B.11\)](#) promises. The zero columns say that the measurement carries no direct information about the velocity; a filter learns the velocity only through the motion model.

Check every Jacobian numerically

Hand-derived Jacobians are the most common source of silent errors in an extended Kalman filter. Compare each column against the central difference $(\mathbf{h}(\mathbf{x} + \varepsilon \mathbf{e}_j) - \mathbf{h}(\mathbf{x} - \varepsilon \mathbf{e}_j))/2\varepsilon$ with $\varepsilon \approx 10^{-6}$ before you trust it; the script of this appendix does exactly that for [Example B.15](#). Two details bite in practice: `atan2` takes the y argument first, and a bearing difference must be wrapped into $(-\pi, \pi]$ before it is used as an innovation, because

a linearization across the $\pm\pi$ cut is meaningless.

B.2.2 The double-integrator step

The drone model of [Chapter 2](#) treats the acceleration $\mathbf{a}$ as the input and integrates it twice. If $\mathbf{a}$ is held constant during a step of length Δt , the velocity grows linearly and the position quadratically,

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \mathbf{a} \Delta t, \quad \mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \mathbf{v}(t) \Delta t + \frac{1}{2} \mathbf{a} \Delta t^2, \quad (\text{B.12})$$

because $\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \int_0^{\Delta t} (\mathbf{v}(t) + \mathbf{a} s) ds$. This is exact, not an approximation, for a piecewise-constant input, which is what a digital controller produces. With the state $\mathbf{x} = (\mathbf{p}, \mathbf{v}) \in \mathbb{R}^{2d}$ and the input $\mathbf{u} = \mathbf{a} \in \mathbb{R}^d$ the step is the linear system $\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k$ with

$$\mathbf{A} = \begin{bmatrix} \mathbf{I} & \Delta t \mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} \frac{1}{2} \Delta t^2 \mathbf{I} \\ \Delta t \mathbf{I} \end{bmatrix}, \quad (\text{B.13})$$

where $\mathbf{I}$ is the $d \times d$ identity. [Chapter 18](#) takes $\mathbf{A}$ as the transition matrix $\mathbf{F}$ of a constant-velocity target and turns the unknown acceleration into process noise through $\mathbf{B}$, which is where its white-noise-acceleration covariance $\mathbf{Q}$ comes from; [Chapter 21](#) stacks N copies of [Equation \(B.13\)](#) into the prediction matrices of its quadratic program; [Chapter 14](#) bounds the velocities reachable in one step by $\|\mathbf{v}_{k+1} - \mathbf{v}_k\| \leq a_{\max} \Delta t$, its dynamic window. The explicit Euler step $\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k \Delta t$ drops the term $\frac{1}{2} \mathbf{a} \Delta t^2$ (with $\Delta t = 0.1$ s, $v = 2$ m/s and $a = 1$ m/s²: 0.200 m instead of the exact 0.205 m, at 2.1 m/s afterward), an error that accumulates over a trajectory.

B.3 Probability and Gaussians

B.3.1 Random vectors, expectation and covariance

A **random vector** $\mathbf{x} \in \mathbb{R}^n$ is a vector whose entries are random variables; it is described by a density $p(\mathbf{x})$ with $p \geq 0$ and $\int p(\mathbf{x}) d\mathbf{x} = 1$, and the probability that $\mathbf{x}$ falls in a region $\mathcal{R}$ is $\mathbb{P}(\mathbf{x} \in \mathcal{R}) = \int_{\mathcal{R}} p(\mathbf{x}) d\mathbf{x}$.

Definition B.16 (Expectation, covariance, cross-covariance). The **expectation** of $\mathbf{x}$ is $\mu = \mathbb{E}[\mathbf{x}] = \int \mathbf{x} p(\mathbf{x}) d\mathbf{x}$, taken entry by entry, and $\mathbb{E}[\mathbf{g}(\mathbf{x})] = \int \mathbf{g}(\mathbf{x}) p(\mathbf{x}) d\mathbf{x}$ for a function $\mathbf{g}$. The **covariance** is

$$\text{Cov}(\mathbf{x}) = \mathbb{E}[(\mathbf{x} - \mu)(\mathbf{x} - \mu)^T] \in \mathbb{R}^{n \times n}, \quad (\text{B.14})$$

whose diagonal entries are the variances $\text{Var}(x_i) = \sigma_i^2$ and whose off-diagonal entries σ_{ij} say how x_i and x_j vary together; $\rho_{ij} = \sigma_{ij}/(\sigma_i \sigma_j) \in [-1, 1]$ is the correlation coefficient. The **cross-covariance** of two random vectors is $\text{Cov}(\mathbf{x}, \mathbf{y}) = \mathbb{E}[(\mathbf{x} - \mu_x)(\mathbf{y} - \mu_y)^T]$, and $\mathbf{x}, \mathbf{y}$ are **independent** if $p(\mathbf{x}, \mathbf{y}) = p(\mathbf{x})p(\mathbf{y})$, in which case $\text{Cov}(\mathbf{x}, \mathbf{y}) = \mathbf{0}$.

Proposition B.17 (Rules for means and covariances). *Expectation is linear, $\mathbb{E}[\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} + \mathbf{c}] = \mathbf{A}\mathbb{E}[\mathbf{x}] + \mathbf{B}\mathbb{E}[\mathbf{y}] + \mathbf{c}$, with no assumption about dependence. Every covariance matrix is symmetric and PSD. For fixed $\mathbf{A}$ and $\mathbf{b}$,*

$$\text{Cov}(\mathbf{A}\mathbf{x} + \mathbf{b}) = \mathbf{A} \text{Cov}(\mathbf{x}) \mathbf{A}^T, \quad (\text{B.15})$$

and if $\mathbf{x}$ and $\mathbf{y}$ are independent, $\text{Cov}(\mathbf{x} + \mathbf{y}) = \text{Cov}(\mathbf{x}) + \text{Cov}(\mathbf{y})$.

Proof. Linearity is linearity of the integral. For any fixed $\mathbf{a}$, $\mathbf{a}^T \text{Cov}(\mathbf{x}) \mathbf{a} = \mathbb{E}[(\mathbf{a}^T (\mathbf{x} - \mu))^2] = \text{Var}(\mathbf{a}^T \mathbf{x}) \geq 0$, which is the definition of PSD; symmetry is read off [Equation \(B.14\)](#). Since

$\mathbf{Ax} + \mathbf{b} - \mathbb{E}[\mathbf{Ax} + \mathbf{b}] = \mathbf{A}(\mathbf{x} - \mu)$, Equation (B.15) follows by pulling $\mathbf{A}$ and $\mathbf{A}^\top$ out of the expectation. For the sum, expand $(\mathbf{x} + \mathbf{y} - \mu_x - \mu_y)(\mathbf{x} + \mathbf{y} - \mu_x - \mu_y)^\top$; the two cross terms are the cross-covariances, which vanish under independence. $\square$

Equation (B.15) is the single most used formula of Chapters 18 and 19: pushing a covariance through the linear model $\mathbf{F}$ gives $\mathbf{F}\mathbf{P}\mathbf{F}^\top$, and adding independent process noise adds $\mathbf{Q}$. Note that it needs no Gaussian assumption at all.

B.3.2 The multivariate Gaussian

Definition B.18 (Multivariate Gaussian). A random vector $\mathbf{x} \in \mathbb{R}^n$ is **Gaussian** with mean μ and covariance $\Sigma \succ \mathbf{0}$, written $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$, if its density is

$$\mathcal{N}(\mathbf{x}; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} \sqrt{\det \Sigma}} \exp\left(-\frac{1}{2} d^2(\mathbf{x})\right), \quad d^2(\mathbf{x}) = (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu), \quad (\text{B.16})$$

where $d(\mathbf{x})$ is the **Mahalanobis distance** of $\mathbf{x}$ from μ . Then $\mathbb{E}[\mathbf{x}] = \mu$ and $\text{Cov}(\mathbf{x}) = \Sigma$.

The density is constant on the ellipsoids $d^2(\mathbf{x}) = k^2$. By Theorem B.4 applied to $\Sigma = \mathbf{Q}\mathbf{A}\mathbf{Q}^\top$, the ellipsoid $\mathcal{E}_k$ of Chapter 2 is centered at μ , its axes are the eigenvectors $\mathbf{q}_i$ and its semi-axes are $k\sqrt{\lambda_i}$: a large eigenvalue is a direction of large uncertainty. In two dimensions the probability mass inside $\mathcal{E}_k$ is $1 - e^{-k^2/2}$, that is 39.3% for $k = 1$ and 86.5% for $k = 2$, a consequence of Theorem B.23 below. A singular $\Sigma \succeq \mathbf{0}$ has no density; Theorem B.19 still holds for it, because a degenerate Gaussian is defined by its mean and covariance through its characteristic function, while the conditioning formula of Theorem B.20 needs $\Sigma_{bb} \succ \mathbf{0}$ (or the Moore–Penrose pseudo-inverse Σ_{bb}^+ in place of Σ_{bb}^{-1}).

Theorem B.19 (Affine transformations of Gaussians). If $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$ in $\mathbb{R}^n$ and $\mathbf{y} = \mathbf{Ax} + \mathbf{b}$ with $\mathbf{A} \in \mathbb{R}^{m \times n}$, then $\mathbf{y} \sim \mathcal{N}(\mathbf{A}\mu + \mathbf{b}, \mathbf{A}\Sigma\mathbf{A}^\top)$.

Proof. The mean and the covariance follow from Proposition B.17. For the shape, take first $m = n$ and $\mathbf{A}$ invertible: substituting $\mathbf{x} = \mathbf{A}^{-1}(\mathbf{y} - \mathbf{b})$ into Equation (B.16) and multiplying by the change-of-variables factor $|\det \mathbf{A}^{-1}|$ gives exactly the density $\mathcal{N}(\mathbf{y}; \mathbf{A}\mu + \mathbf{b}, \mathbf{A}\Sigma\mathbf{A}^\top)$, because $(\mathbf{A}\Sigma\mathbf{A}^\top)^{-1} = \mathbf{A}^{-\top}\Sigma^{-1}\mathbf{A}^{-1}$ and $\det(\mathbf{A}\Sigma\mathbf{A}^\top) = (\det \mathbf{A})^2 \det \Sigma$. A non-square $\mathbf{A}$ of full row rank is extended to an invertible matrix, and the extra coordinates are integrated out with the marginalization half of Theorem B.20, whose proof does not use this theorem; the remaining cases follow with characteristic functions [144]. $\square$

In particular the sum of independent Gaussians is Gaussian with the sums of the means and of the covariances (apply $\mathbf{A} = [\mathbf{I} \ \mathbf{I}]$ to the joint vector, whose covariance is block-diagonal), which is how process noise enters the prediction step of Chapter 18.

Theorem B.20 (Marginal and conditional of a joint Gaussian). Let $\mathbf{x} = (\mathbf{x}_a, \mathbf{x}_b)$ be jointly Gaussian with a non-singular covariance,

$$\begin{bmatrix} \mathbf{x}_a \\ \mathbf{x}_b \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu_a \\ \mu_b \end{bmatrix}, \begin{bmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{bmatrix}\right), \quad \Sigma_{ba} = \Sigma_{ab}^\top, \quad \Sigma \succ \mathbf{0}. \quad (\text{B.17})$$

Then the **marginal** of $\mathbf{x}_b$ is $\mathcal{N}(\mu_b, \Sigma_{bb})$ (and likewise for $\mathbf{x}_a$), and the **conditional** density of $\mathbf{x}_a$ given $\mathbf{x}_b$ is Gaussian,

$$\mathbf{x}_a | \mathbf{x}_b \sim \mathcal{N}\left(\mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(\mathbf{x}_b - \mu_b), \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}\right). \quad (\text{B.18})$$

The conditional mean is affine in the observed value; the conditional covariance is the Schur complement of Σ_{bb} , does not depend on the observed value, and is never larger than Σ_{aa} .

Proof. Write $\mathbf{d}_a = \mathbf{x}_a - \mu_a$, $\mathbf{d}_b = \mathbf{x}_b - \mu_b$, $\mathbf{S} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$ and $\mathbf{\Lambda} = \Sigma^{-1}$ with blocks $\mathbf{\Lambda}_{aa}, \mathbf{\Lambda}_{ab}, \mathbf{\Lambda}_{ba}, \mathbf{\Lambda}_{bb}$. By [Theorem B.9](#) with $\mathbf{D} = \Sigma_{bb}$,

$$\mathbf{\Lambda}_{aa} = \mathbf{S}^{-1}, \quad \mathbf{\Lambda}_{ab} = -\mathbf{S}^{-1}\Sigma_{ab}\Sigma_{bb}^{-1}, \quad \det \Sigma = \det \mathbf{S} \det \Sigma_{bb}.$$

The exponent of the joint density is $-\frac{1}{2}(\mathbf{d}_a^\top \mathbf{\Lambda}_{aa} \mathbf{d}_a + 2\mathbf{d}_a^\top \mathbf{\Lambda}_{ab} \mathbf{d}_b + \mathbf{d}_b^\top \mathbf{\Lambda}_{bb} \mathbf{d}_b)$. Completing the square in $\mathbf{d}_a$ with $\mathbf{m} = -\mathbf{\Lambda}_{aa}^{-1}\mathbf{\Lambda}_{ab}\mathbf{d}_b = \Sigma_{ab}\Sigma_{bb}^{-1}\mathbf{d}_b$ turns it into

$$-\frac{1}{2}(\mathbf{d}_a - \mathbf{m})^\top \mathbf{S}^{-1}(\mathbf{d}_a - \mathbf{m}) - \frac{1}{2}\mathbf{d}_b^\top (\mathbf{\Lambda}_{bb} - \mathbf{\Lambda}_{ba}\mathbf{\Lambda}_{aa}^{-1}\mathbf{\Lambda}_{ab}) \mathbf{d}_b,$$

and the matrix in the second term is the Schur complement of $\mathbf{\Lambda}_{aa}$ in $\mathbf{\Lambda}$, which by [Theorem B.9](#) applied to $\mathbf{\Lambda}$ (whose inverse is Σ) equals Σ_{bb}^{-1} . Splitting the normalizing constant with $\det \Sigma = \det \mathbf{S} \det \Sigma_{bb}$ and $(2\pi)^n = (2\pi)^{n_a}(2\pi)^{n_b}$ gives the exact factorization

$$p(\mathbf{x}_a, \mathbf{x}_b) = \mathcal{N}(\mathbf{x}_a; \mu_a + \mathbf{m}, \mathbf{S}) \cdot \mathcal{N}(\mathbf{x}_b; \mu_b, \Sigma_{bb}). \quad (\text{B.19})$$

Integrating over $\mathbf{x}_a$ removes the first factor, which is a density in $\mathbf{x}_a$, and leaves the marginal of $\mathbf{x}_b$. Dividing by that marginal gives $p(\mathbf{x}_a | \mathbf{x}_b) = p(\mathbf{x}_a, \mathbf{x}_b)/p(\mathbf{x}_b)$, the first factor, which is [Equation \(B.18\)](#). Finally $\Sigma_{aa} - \mathbf{S} = \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba} \succeq \mathbf{0}$ by [Proposition B.6\(iii\)](#). $\square$

Corollary B.21 (Linear-Gaussian measurement update). *Let $\mathbf{x} \sim \mathcal{N}(\mu, \mathbf{P})$ with $\mathbf{P} \succ \mathbf{0}$, and let $\mathbf{z} = \mathbf{H}\mathbf{x} + \mathbf{v}$ with $\mathbf{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$, $\mathbf{R} \succ \mathbf{0}$, independent of $\mathbf{x}$ (so the joint covariance below is non-singular, as [Theorem B.20](#) requires). Then $(\mathbf{x}, \mathbf{z})$ is jointly Gaussian with $\text{Cov}(\mathbf{z}) = \mathbf{H}\mathbf{P}\mathbf{H}^\top + \mathbf{R} =: \mathbf{S}$ and $\text{Cov}(\mathbf{x}, \mathbf{z}) = \mathbf{P}\mathbf{H}^\top$, and*

$$\mathbf{x} | \mathbf{z} \sim \mathcal{N}(\mu + \mathbf{K}(\mathbf{z} - \mathbf{H}\mu), \mathbf{P} - \mathbf{K}\mathbf{H}\mathbf{P}), \quad \mathbf{K} = \mathbf{P}\mathbf{H}^\top \mathbf{S}^{-1}. \quad (\text{B.20})$$

Proof. $(\mathbf{x}, \mathbf{z})$ is the image of the Gaussian vector $(\mathbf{x}, \mathbf{v})$, whose covariance is block-diagonal, under the linear map $\begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{H} & \mathbf{I} \end{bmatrix}$, hence Gaussian by [Theorem B.19](#); its blocks follow from [Proposition B.17](#). Insert $\Sigma_{ab} = \mathbf{P}\mathbf{H}^\top$ and $\Sigma_{bb} = \mathbf{S}$ into [Equation \(B.18\)](#). $\square$

[Equation \(B.20\)](#) is the update step of the Kalman filter with gain $\mathbf{K}$: [Chapter 18](#) derives it from this corollary, and [Proposition B.10](#) turns $\mathbf{P} - \mathbf{K}\mathbf{H}\mathbf{P}$ into the information form $(\mathbf{P}^{-1} + \mathbf{H}^\top \mathbf{R}^{-1} \mathbf{H})^{-1}$, in which the precisions of prior and measurement simply add. [Table B.2](#) collects these identities in the form in which [Chapters 18](#) and [19](#) use them.

Example B.22 (Conditioning the Gaussian of [Example B.7](#)). Let $(x, y) \sim \mathcal{N}(\mu, \Sigma)$ with $\mu = (2, 1)^\top$ and Σ from [Equation \(B.3\)](#), and suppose $y = 2$ is observed. [Equation \(B.18\)](#) with $a = x$ and $b = y$ gives

$$\mathbb{E}[x | y = 2] = 2 + \frac{1.2}{1.0}(2 - 1) = 3.2, \quad \text{Var}(x | y = 2) = 2.0 - \frac{1.2^2}{1.0} = 0.56.$$

Before the observation $x \sim \mathcal{N}(2, 2.0)$, with standard deviation 1.414; afterward $x | y \sim \mathcal{N}(3.2, 0.56)$, with standard deviation 0.748. The positive correlation $\rho = 1.2/\sqrt{2.0 \cdot 1.0} = 0.85$ means that a y above its mean shifts the belief about x upward, and the variance shrinks by the factor $1 - \rho^2 = 0.28$. [Figure B.1](#) shows the ellipses, the slice $y = 2$ and both densities.

B.3.3 Bayes' rule, total probability and the recursive filter

For two random vectors the **chain rule** $p(\mathbf{x}, \mathbf{z}) = p(\mathbf{z} | \mathbf{x})p(\mathbf{x})$ defines the conditional density, and integrating it over $\mathbf{x}$ gives the law of **total probability**, $p(\mathbf{z}) = \int p(\mathbf{z} | \mathbf{x})p(\mathbf{x}) d\mathbf{x}$. Reading the chain rule in both directions gives **Bayes' rule**

$$p(\mathbf{x} | \mathbf{z}) = \frac{p(\mathbf{z} | \mathbf{x})p(\mathbf{x})}{p(\mathbf{z})} = \frac{p(\mathbf{z} | \mathbf{x})p(\mathbf{x})}{\int p(\mathbf{z} | \mathbf{x}')p(\mathbf{x}') d\mathbf{x}'}, \quad (\text{B.21})$$

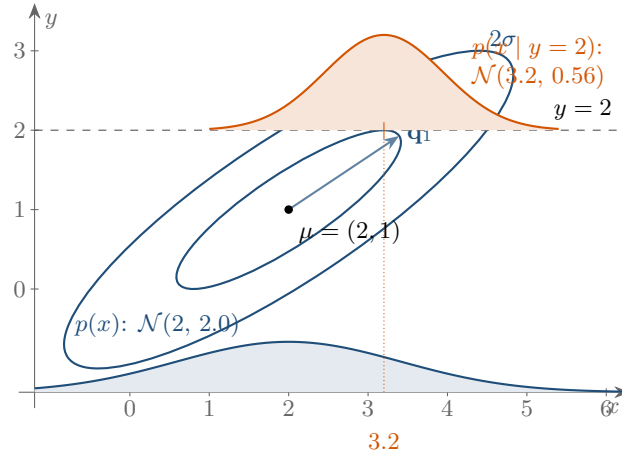


Figure B.1. Conditioning the Gaussian of Example B.22. The 1σ and 2σ ellipses of the joint density are tilted along the eigenvector $\mathbf{q}_1$ of Σ . Observing $y = 2$ restricts the density to the dashed slice; along it, the conditional density of x (orange) is centered at 3.2, to the right of the prior mean 2, and is narrower than the marginal density of x (blue, drawn along the bottom) because knowing y removes the part of the uncertainty of x that is correlated with y .

Table B.2. Gaussian identities. $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$, blocks as in Equation (B.17); “ $\propto$ ” means equality up to a factor that does not depend on $\mathbf{x}$.

Identity	Remark
$\mathbf{Ax} + \mathbf{b} \sim \mathcal{N}(\mathbf{A}\mu + \mathbf{b}, \mathbf{A}\Sigma\mathbf{A}^\top)$	Theorem B.19; prediction step
$\mathbf{x}_a \sim \mathcal{N}(\mu_a, \Sigma_{aa})$	marginal: drop the other blocks
$\mathbf{x}_a \mathbf{x}_b \sim \mathcal{N}(\mu_{a b}, \Sigma_{a b}), \mu_{a b} = \mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(\mathbf{x}_b - \mu_b),$ $\Sigma_{a b} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$	Theorem B.20; update step
$\mathbf{x} + \mathbf{y} \sim \mathcal{N}(\mu_x + \mu_y, \Sigma_x + \Sigma_y)$	$\mathbf{x}, \mathbf{y}$ independent
$\mathcal{N}(\mathbf{x}; \mathbf{a}, \mathbf{A})\mathcal{N}(\mathbf{x}; \mathbf{b}, \mathbf{B}) \propto \mathcal{N}(\mathbf{x}; \mathbf{c}, \mathbf{C}), \mathbf{C} = (\mathbf{A}^{-1} + \mathbf{B}^{-1})^{-1},$ $\mathbf{c} = \mathbf{C}(\mathbf{A}^{-1}\mathbf{a} + \mathbf{B}^{-1}\mathbf{b})$	precisions add; information form of the update
$d^2(\mathbf{x}) = (\mathbf{x} - \mu)^\top \Sigma^{-1}(\mathbf{x} - \mu) \sim \chi_n^2$	Theorem B.23; gating
$\mathbf{x} = \mu + \mathbf{L}\xi, \mathbf{L}\mathbf{L}^\top = \Sigma, \xi \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$	how to draw a sample

which turns a **prior** $p(\mathbf{x})$ and a **likelihood** $p(\mathbf{z} | \mathbf{x})$ into the **posterior** $p(\mathbf{x} | \mathbf{z})$; the denominator is a constant that makes the posterior integrate to one. For discrete events the integrals are sums. For example, a detector that flags an intruder with probability 0.9 when one is present and 0.2 when none is, on a day with intruders present 10 % of the time, raises the probability of an intruder on a flag only to $0.09/(0.09 + 0.18) = 1/3$: false alarms on the common case outnumber true detections on the rare one.

Every filter of Chapters 18 and 19 is these two rules applied over and over: with the Markov assumption $p(\mathbf{x}_k | \mathbf{x}_{k-1}, \mathbf{z}_{1:k-1}) = p(\mathbf{x}_k | \mathbf{x}_{k-1})$ and measurements that depend only on the current state, the belief about $\mathbf{x}_k$ given $\mathbf{z}_{1:k}$ is obtained in two steps,

$$p(\mathbf{x}_k | \mathbf{z}_{1:k-1}) = \int p(\mathbf{x}_k | \mathbf{x}_{k-1}) p(\mathbf{x}_{k-1} | \mathbf{z}_{1:k-1}) d\mathbf{x}_{k-1} \quad (\text{predict: total probability}), \quad (\text{B.22})$$

$$p(\mathbf{x}_k | \mathbf{z}_{1:k}) \propto p(\mathbf{z}_k | \mathbf{x}_k) p(\mathbf{x}_k | \mathbf{z}_{1:k-1}) \quad (\text{update: Bayes' rule}). \quad (\text{B.23})$$

When the motion and measurement models are linear with Gaussian noise, both steps map Gaussians to Gaussians (Theorem B.19 and Corollary B.21) and the recursion is the Kalman filter; when they are not, Chapter 19 approximates the two integrals by linearization, by sigma points or by samples [144, 168].

B.3.4 Monte Carlo estimation

Many quantities in this book are expectations without a closed form, such as the probability that a predicted trajectory hits an obstacle or the collision rate of a planner over random scenarios. The **Monte Carlo** estimate of $\mathbb{E}[g(\mathbf{x})]$ from N independent samples $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}$ drawn from p is

$$\hat{g}_N = \frac{1}{N} \sum_{i=1}^N g(\mathbf{x}^{(i)}), \quad \mathbb{E}[\hat{g}_N] = \mathbb{E}[g(\mathbf{x})], \quad \text{Var}(\hat{g}_N) = \frac{\text{Var}(g(\mathbf{x}))}{N}. \quad (\text{B.24})$$

The estimate is unbiased and its standard error shrinks like $1/\sqrt{N}$ whatever the dimension of $\mathbf{x}$. For a probability $\mathbb{P}(\mathbf{x} \in \mathcal{R})$ take g the indicator of $\mathcal{R}$; then the standard error is $\sqrt{\hat{p}(1-\hat{p})/N}$, the error bar to attach to a collision rate estimated from N random scenarios (Chapter 25). To draw from $\mathcal{N}(\mu, \Sigma)$, draw $\xi \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and set $\mathbf{x} = \mu + \mathbf{L}\xi$ with the Cholesky factor $\mathbf{L}\mathbf{L}^\top = \Sigma$ (Theorem B.19 with $\mathbf{A} = \mathbf{L}$ and $\mathbf{b} = \mu$). Drawing $N = 10\,000$ samples this way from the Gaussian of Example B.22 and counting those with $d^2 \leq 4$ gives $\hat{p} = 0.8695$ with standard error 0.0034, within 1.5 standard errors of the exact value $1 - e^{-2} = 0.8647$ of Theorem B.23 below. When samples can only be drawn from another density q , **importance sampling** weights them by $w^{(i)} \propto p(\mathbf{x}^{(i)})/q(\mathbf{x}^{(i)})$, normalized to sum to one; the particle filter of Chapter 19 is a sequential version of this idea [36].

B.3.5 Mahalanobis distance and chi-square gating

Theorem B.23 (Mahalanobis distance of a Gaussian vector). *If $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$ in $\mathbb{R}^n$ with $\Sigma \succ \mathbf{0}$, then $d^2(\mathbf{x}) = (\mathbf{x} - \mu)^\top \Sigma^{-1}(\mathbf{x} - \mu)$ has the **chi-square distribution** with n degrees of freedom, χ_n^2 , whose mean is n . For $n = 2$, $\mathbb{P}(d^2 \leq \gamma) = 1 - e^{-\gamma/2}$.*

Proof. Let $\mathbf{L}\mathbf{L}^\top = \Sigma$ and $\xi = \mathbf{L}^{-1}(\mathbf{x} - \mu)$. By Theorem B.19, $\xi \sim \mathcal{N}(\mathbf{0}, \mathbf{L}^{-1}\Sigma\mathbf{L}^{-\top}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$, so its entries are n independent standard normal variables, and $d^2 = \xi^\top \xi = \sum_i \xi_i^2$ is by definition χ_n^2 distributed; $\mathbb{E}[\xi_i^2] = 1$ gives the mean. For $n = 2$ the density of (ξ_1, ξ_2) is $\frac{1}{2\pi}e^{-\varrho^2/2}$ in polar coordinates (ϱ, ϕ) , and $\mathbb{P}(d^2 \leq \gamma) = \int_0^{\sqrt{\gamma}} e^{-\varrho^2/2} \varrho d\varrho = 1 - e^{-\gamma/2}$. $\square$

The theorem is the basis of **gating**: a filter that expects a measurement with innovation $\mathbf{y} = \mathbf{z} - \hat{\mathbf{z}}$ of covariance $\mathbf{S}$ accepts it only if $\mathbf{y}^\top \mathbf{S}^{-1} \mathbf{y} \leq \gamma$, with γ the quantile of χ_n^2 at the chosen confidence p : for $n = 2$ the 95 % and 99 % gates are $\gamma = -2\ln(1-p) = 5.991$ and 9.210, for $n = 1$ they are 3.841 and 6.635, and for $n = 3$ they are 7.815 and 11.345. A true measurement is then wrongly rejected with probability $1-p$, and a detection from another target or from clutter is rejected whenever it lands outside the ellipsoid [7]. This is the gate of Chapters 18 and 19; the same quantiles size the uncertainty-inflated constraints of Chapter 24, where an obstacle known only as a Gaussian is enlarged to the ellipse that holds a required probability mass.

B.4 Optimization

Throughout this section $\mathbf{z}$ is the vector of decision variables of a mathematical program, not the measurement vector of Appendix B.3, which is written with the same bold letter.

B.4.1 Convex sets and convex functions

Definition B.24 (Convex set, convex function). A set $\mathcal{C} \subseteq \mathbb{R}^n$ is **convex** if the segment between any two of its points stays inside it: $\mathbf{x}, \mathbf{y} \in \mathcal{C}$ and $\theta \in [0, 1]$ imply $\theta\mathbf{x} + (1-\theta)\mathbf{y} \in \mathcal{C}$. A function $f: \mathcal{C} \rightarrow \mathbb{R}$ on a convex set is **convex** if its graph lies below every chord,

$$f(\theta\mathbf{x} + (1-\theta)\mathbf{y}) \leq \theta f(\mathbf{x}) + (1-\theta)f(\mathbf{y}) \quad \text{for all } \mathbf{x}, \mathbf{y} \in \mathcal{C}, \theta \in [0, 1], \quad (\text{B.25})$$

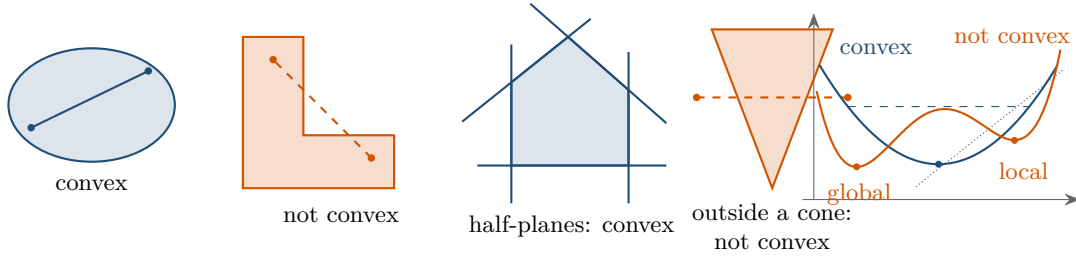


Figure B.2. Convexity. Left: in a convex set every chord stays inside; in the non-convex set the dashed chord leaves it. Middle: an intersection of half-planes, the situation of ORCA in Chapter 13, is convex, while the set of velocities outside a cone, the situation of a velocity obstacle in Chapter 12, is not. Right: a convex function lies below its chords and above its tangents, so its only stationary point is the global minimum; the non-convex function has a local minimum in which gradient descent can stop.

strictly convex if the inequality is strict for $\mathbf{x} \neq \mathbf{y}$ and $\theta \in (0, 1)$, and concave if $-f$ is convex.

Half-spaces, balls, ellipsoids and affine subspaces are convex, and any intersection of convex sets is convex: the velocities allowed by all ORCA half-planes in Chapter 13 form a convex polygon, whereas the collision-free velocities outside a velocity obstacle of Chapter 12 do not form a convex set (Figure B.2). Linear functions, norms, $\|\mathbf{x} - \mathbf{c}\|^2$ and $\mathbf{x}^\top \mathbf{A} \mathbf{x}$ with $\mathbf{A} \succeq \mathbf{0}$ are convex, as are non-negative sums and pointwise maxima of convex functions, and the sublevel sets $\{\mathbf{x} : f(\mathbf{x}) \leq \alpha\}$ of a convex function are convex. What makes convexity the dividing line of optimization is the following fact.

Proposition B.25 (Local minima of convex functions are global). *Let f be convex on a convex set $\mathcal{C}$ and let $\mathbf{x}^*$ be a local minimizer: $f(\mathbf{x}^*) \leq f(\mathbf{x})$ for all $\mathbf{x} \in \mathcal{C}$ within some distance $\varepsilon > 0$ of $\mathbf{x}^*$. Then $f(\mathbf{x}^*) \leq f(\mathbf{x})$ for all $\mathbf{x} \in \mathcal{C}$. If moreover $\mathcal{C} = \mathbb{R}^n$ and f is differentiable, then $\mathbf{x}^*$ is a global minimizer iff $\nabla f(\mathbf{x}^*) = \mathbf{0}$; on a general convex $\mathcal{C}$ the first-order condition is instead $\nabla f(\mathbf{x}^*)^\top (\mathbf{x} - \mathbf{x}^*) \geq 0$ for all $\mathbf{x} \in \mathcal{C}$.*

Proof. Suppose $f(\mathbf{y}) < f(\mathbf{x}^*)$ for some $\mathbf{y} \in \mathcal{C}$. The points $\mathbf{x}_\theta = \theta \mathbf{y} + (1 - \theta) \mathbf{x}^*$ lie in $\mathcal{C}$ and satisfy $f(\mathbf{x}_\theta) \leq \theta f(\mathbf{y}) + (1 - \theta) f(\mathbf{x}^*) < f(\mathbf{x}^*)$ for every $\theta \in (0, 1]$; for θ small enough $\mathbf{x}_\theta$ is within ε of $\mathbf{x}^*$, contradicting local minimality. For the second claim, a differentiable convex function satisfies $f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x})$ for all $\mathbf{x}, \mathbf{y}$, that is, its graph lies above its tangent planes [24, Section 3.1]. On $\mathcal{C} = \mathbb{R}^n$ a zero gradient therefore makes $\mathbf{x}$ a global minimizer, and conversely an unconstrained minimizer of a differentiable function has zero gradient. On a general convex $\mathcal{C}$ the same inequality gives $f(\mathbf{x}) \geq f(\mathbf{x}^*) + \nabla f(\mathbf{x}^*)^\top (\mathbf{x} - \mathbf{x}^*) \geq f(\mathbf{x}^*)$ whenever the stated condition holds; and if $\nabla f(\mathbf{x}^*)^\top (\mathbf{x} - \mathbf{x}^*) < 0$ for some $\mathbf{x} \in \mathcal{C}$, then f decreases along the segment from $\mathbf{x}^*$ toward $\mathbf{x}$, which lies in $\mathcal{C}$, so $\mathbf{x}^*$ is not optimal [24, Section 4.2]. The gradient need not vanish at a constrained optimum: $f(x) = x$ on $\mathcal{C} = [0, 1]$ is minimized at $x^* = 0$ with $\nabla f(x^*) = 1$. Figure B.3 draws the constrained condition for a linear program: at the optimal vertex $-\mathbf{c}$ lies in the cone of the normals of the active constraints, so no feasible direction decreases the objective. That example *maximizes*, so the arrow shown inside the cone of the active normals is $+\mathbf{c}$, which is the same statement with the sign of the objective flipped. $\square$

For twice differentiable f the test is the Hessian: f is convex on $\mathbb{R}^n$ iff $\nabla^2 f(\mathbf{x}) \succeq \mathbf{0}$ everywhere [24, Section 3.1]. This is why the quadratic costs of Chapter 21 are built from PSD weights, and why the potential fields of Chapter 15, whose repulsive terms are not convex, can trap a drone in a local minimum.

B.4.2 Optimality conditions: Lagrange multipliers and KKT

The general constrained problem is

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad \text{subject to} \quad g_i(\mathbf{x}) \leq 0 \quad (i = 1, \dots, m), \quad h_j(\mathbf{x}) = 0 \quad (j = 1, \dots, p), \quad (\text{B.26})$$

with differentiable f , g_i and h_j . Its **Lagrangian** attaches a multiplier to every constraint, $\mathcal{L}(\mathbf{x}, \lambda, \nu) = f(\mathbf{x}) + \sum_i \lambda_i g_i(\mathbf{x}) + \sum_j \nu_j h_j(\mathbf{x})$.

Theorem B.26 (Karush–Kuhn–Tucker conditions). *If $\mathbf{x}^*$ is a local minimizer of Equation (B.26) and the constraints satisfy a constraint qualification at $\mathbf{x}^*$ (for instance, the gradients of the active constraints are linearly independent, or all constraints are affine), then there exist multipliers $\lambda^* \in \mathbb{R}^m$ and $\nu^* \in \mathbb{R}^p$ with*

$$\nabla f(\mathbf{x}^*) + \sum_i \lambda_i^* \nabla g_i(\mathbf{x}^*) + \sum_j \nu_j^* \nabla h_j(\mathbf{x}^*) = \mathbf{0} \quad (\text{stationarity}), \quad (\text{B.27})$$

$$g_i(\mathbf{x}^*) \leq 0, \quad h_j(\mathbf{x}^*) = 0 \quad (\text{primal feasibility}), \quad (\text{B.28})$$

$$\lambda_i^* \geq 0 \quad (\text{dual feasibility}), \quad (\text{B.29})$$

$$\lambda_i^* g_i(\mathbf{x}^*) = 0 \quad (\text{complementary slackness}). \quad (\text{B.30})$$

If f and the g_i are convex and the h_j affine, these conditions are also sufficient: every point that satisfies them is a global minimizer.

The proof is in Nocedal and Wright [119, Chapter 12] and, for the convex case, in Boyd and Vandenberghe [24, Chapter 5]. At the optimum the negative gradient of the cost is balanced by the normals of the *active* constraints, those with $g_i(\mathbf{x}^*) = 0$, which by Equation (B.30) are the only ones that can carry a positive multiplier; λ_i^* measures how much the optimal cost would drop if constraint i were relaxed by one unit. This is the theory behind the soft constraints of Chapter 21: a slack variable with a linear penalty reproduces the hard constraint exactly, whenever it is feasible, as soon as the penalty weight exceeds the multiplier [119, Chapter 17]. With equality constraints only, the conditions reduce to the classical Lagrange multipliers.

B.4.3 Linear programs

Definition B.27 (Linear program). A **linear program** (LP) minimizes a linear function over a polyhedron:

$$\min_{\mathbf{z} \in \mathbb{R}^n} \mathbf{c}^\top \mathbf{z} \quad \text{subject to} \quad \mathbf{A}\mathbf{z} \leq \mathbf{b}, \quad P = \{\mathbf{z} : \mathbf{A}\mathbf{z} \leq \mathbf{b}\} \text{ the feasible polyhedron.} \quad (\text{B.31})$$

Equality constraints, sign constraints $\mathbf{z} \geq \mathbf{0}$ and maximization are all reduced to this form by writing $\mathbf{a}^\top \mathbf{z} = \beta$ as two inequalities, $z_j \geq 0$ as $-z_j \leq 0$ and $\max \mathbf{c}^\top \mathbf{z}$ as $\min(-\mathbf{c})^\top \mathbf{z}$. Simplex solvers and most textbooks use instead the **standard form** $\min \mathbf{c}^\top \mathbf{z}$ subject to $\mathbf{A}\mathbf{z} = \mathbf{b}$, $\mathbf{z} \geq \mathbf{0}$, reached from Equation (B.31) by adding a *slack* variable $s_i \geq 0$ to every inequality, $\mathbf{a}_i^\top \mathbf{z} + s_i = b_i$, and by splitting a free variable into $z_j = z_j^+ - z_j^-$ with $z_j^\pm \geq 0$; `scipy.optimize.linprog` accepts inequalities, equalities and bounds directly.

An LP is convex, so Theorem B.26 applies with $\nabla g_i = \mathbf{a}_i$: at an optimum, $-\mathbf{c}$ is a non-negative combination of the normals of the active constraints. In two dimensions this is a picture (Figure B.3): P is a polygon, the level lines $\mathbf{c}^\top \mathbf{z} = \text{const}$ are parallel, and sliding a level line in the direction $-\mathbf{c}$ as far as P allows ends at a vertex, or along a whole edge when the level lines are parallel to it (Figure B.3 maximizes, so there the level line slides in the direction $+\mathbf{c}$).

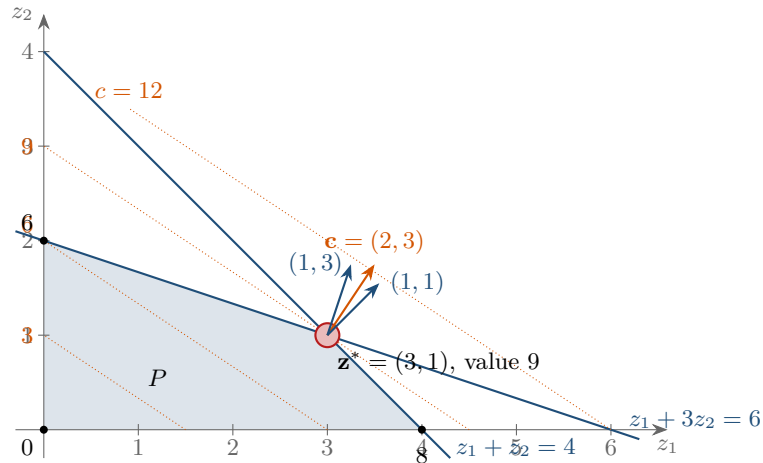


Figure B.3. The LP of Example B.29. The shaded polygon is the feasible region P , cut out by four half-planes; the dotted lines are level lines of the objective $2z_1 + 3z_2$. Sliding a level line in the direction of $\mathbf{c}$ until it is about to leave P stops at the vertex $(3, 1)$, where $\mathbf{c}$ lies between the normals of the two active constraints.

Theorem B.28 (Vertex optimality). *Every LP of the form Equation (B.31) either is infeasible ($P = \emptyset$), or is unbounded ($\mathbf{c}^\top \mathbf{z} \rightarrow -\infty$ on P), or attains its optimum. In the last case, if P has at least one vertex—which holds in particular for the standard form with $\mathbf{z} \geq \mathbf{0}$ —then some vertex of P is optimal.*

A proof is in Nocedal and Wright [119, Chapter 13]. The simplex method walks from vertex to adjacent vertex, never worsening the objective and improving it away from degenerate pivots, until no neighbor is better, which by convexity is a global optimum (an anti-cycling rule such as Bland’s guarantees termination); interior-point methods approach the optimum through the inside of P and are polynomial in the worst case. Chapter 22 calls such solvers through `scipy.optimize`, and branch and bound (Appendix B.4.5) solves a sequence of LPs. The velocity choice of ORCA in Chapter 13 is the same geometry: the velocity closest to the preferred one inside an intersection of half-planes, found by a small linear-programming routine that adds one half-plane at a time.

Example B.29 (A two-variable LP). Maximize $2z_1 + 3z_2$ subject to $z_1 + z_2 \leq 4$, $z_1 + 3z_2 \leq 6$ and $z_1, z_2 \geq 0$. The polygon P has the vertices $(0, 0)$, $(4, 0)$, $(3, 1)$ and $(0, 2)$, where the objective takes the values 0, 8, 9 and 6; the optimum is $\mathbf{z}^* = (3, 1)$ with value 9 (Figure B.3). At $\mathbf{z}^*$ both structural constraints are active, and the stationarity condition for the maximization reads $\mathbf{c} = \lambda_1(1, 1)^\top + \lambda_2(1, 3)^\top$, solved by $\lambda_1 = 1.5$ and $\lambda_2 = 0.5$: both non-negative, as required, so $\mathbf{c}$ lies in the cone spanned by the two active normals and no feasible direction improves the objective. Relaxing the first constraint to $z_1 + z_2 \leq 5$ raises the optimum by exactly $\lambda_1 = 1.5$, to 10.5 at $(4.5, 0.5)$.

B.4.4 Quadratic programs

A **quadratic program** (QP) has a quadratic objective and linear constraints,

$$\min_{\mathbf{z}} \frac{1}{2} \mathbf{z}^\top \mathbf{H} \mathbf{z} + \mathbf{f}^\top \mathbf{z} \quad \text{subject to} \quad \mathbf{l} \leq \mathbf{G} \mathbf{z} \leq \mathbf{u}, \quad (\text{B.32})$$

which is the form of the MPC problem of Chapter 21. It is convex iff $\mathbf{H} \succeq \mathbf{0}$, and then Theorem B.26 is necessary and sufficient. Without constraints the solution is the linear system $\mathbf{H} \mathbf{z} = -\mathbf{f}$, unique when $\mathbf{H} \succ \mathbf{0}$; the least-squares problem of Theorem B.11 is the case

$\mathbf{H} = 2\mathbf{A}^\top \mathbf{A}$, $\mathbf{f} = -2\mathbf{A}^\top \mathbf{b}$. With equality constraints $\mathbf{G}\mathbf{z} = \mathbf{g}$ only, the KKT conditions are the single linear system

$$\begin{bmatrix} \mathbf{H} & \mathbf{G}^\top \\ \mathbf{G} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{z} \\ \nu \end{bmatrix} = \begin{bmatrix} -\mathbf{f} \\ \mathbf{g} \end{bmatrix}, \quad (\text{B.33})$$

a block matrix whose Schur complement $-\mathbf{G}\mathbf{H}^{-1}\mathbf{G}^\top$ (Theorem B.9) is what a solver eliminates first. Inequality constraints are handled by active-set methods, which guess the active set and solve Equation (B.33) for it, by interior-point methods [119], or by operator-splitting methods such as ADMM, which alternate projections onto the constraints with linear solves.

B.4.5 Mixed-integer programs, branch and bound, big-M

A **mixed-integer linear program** (MILP) is an LP in which some variables are restricted to integers, most often to the binary values $b \in \{0, 1\}$. The integrality constraint destroys convexity: the feasible set is a union of polyhedral slices, one for every assignment of the integer variables—a scatter of isolated points only when *every* variable is integer—and convex in neither case. The problem is NP-hard in general. Dropping the integrality gives the **LP relaxation**, whose optimal value is a bound on the MILP optimum (a lower bound for a minimization, an upper bound for a maximization), because the relaxation optimizes over a larger set. **Branch and bound** turns this bound into an exact method [181]:

1. Solve the LP relaxation of the current subproblem. If it is infeasible, or if its bound cannot beat the best integer solution found so far (the *incumbent*), discard the subproblem (*prune*).
2. If the relaxed solution is integral, it is the best solution of this subproblem; make it the incumbent if it is better than the current one.
3. Otherwise pick a variable z_j with fractional value v and *branch*: create two subproblems with the extra constraint $z_j \leq \lfloor v \rfloor$ and $z_j \geq \lceil v \rceil$ respectively, and repeat.

The method terminates, for bounded integer variables, because every branch cuts away a slice of the relaxed region that contains no integer point, and it is exact because a subproblem is pruned only when it provably contains nothing better than the incumbent; its running time is exponential in the worst case but often small in practice, the trade-off that Chapter 22 explores.

Example B.30 (Branch and bound by hand). Maximize $5z_1 + 4z_2$ subject to $6z_1 + 4z_2 \leq 24$, $z_1 + 2z_2 \leq 6$, $z_1, z_2 \geq 0$ and integer. The relaxation (node 1) has the optimum $(3, 1.5)$ with value 21, so no integer solution can exceed 21. Branching on z_2 gives node 2 ($z_2 \leq 1$) with relaxed optimum $(3.33, 1)$ and bound 20.67, and node 3 ($z_2 \geq 2$) with $(2, 2)$ and bound 18. Branching node 2 on z_1 gives node 4 ($z_1 \leq 3$), whose relaxation $(3, 1)$ is integral with value 19 and becomes the incumbent, and node 5 ($z_1 \geq 4$), whose relaxation $(4, 0)$ is integral with value 20 and replaces it. Node 3 is pruned: its bound 18 cannot beat 20. The optimum is $(4, 0)$ with value 20, which enumerating all feasible integer points confirms; rounding the relaxed optimum to $(3, 2)$ would be infeasible and to $(3, 1)$ suboptimal.

Big-M constraints. Binaries make it possible to say “at least one of these constraints holds”, which no LP can express. Given a constant M larger than any value that $\mathbf{a}^\top \mathbf{z} - \beta$ can take on the feasible set, the pair

$$\mathbf{a}^\top \mathbf{z} \leq \beta + Mb, \quad b \in \{0, 1\}, \quad (\text{B.34})$$

enforces the constraint when $b = 0$ and switches it off when $b = 1$. To keep a drone out of an axis-aligned box $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$, Chapter 22 writes the four “outside” conditions $x \leq x_{\min}$, $x \geq x_{\max}$, $y \leq y_{\min}$ and $y \geq y_{\max}$ in the form Equation (B.34) with binaries

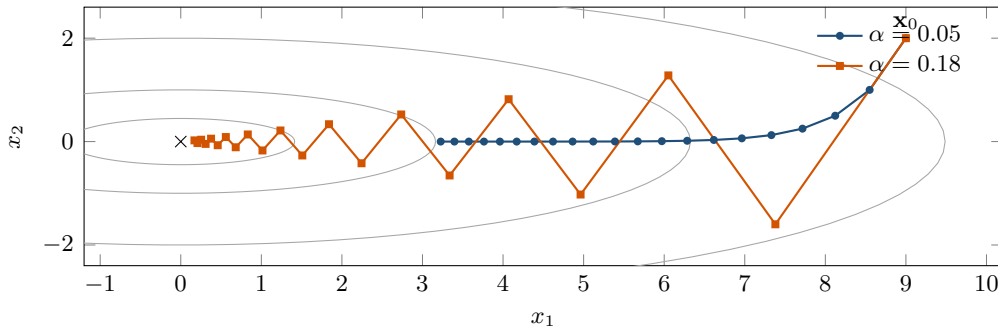


Figure B.4. Gradient descent on $f = \frac{1}{2}(x_1^2 + 10x_2^2)$ from $(9, 2)$ (Example B.32). The ellipses are level sets. The small step $\alpha = 0.05$ (blue) reaches the valley floor at once and then advances by only 5% per step; the larger step $\alpha = 0.18$ (orange) zigzags across the valley and gets much closer to the minimum in the same 20 steps. Any $\alpha > 0.2$ diverges.

$b_1, \dots, b_4$ and adds $b_1 + b_2 + b_3 + b_4 \leq 3$, so that at least one condition stays enforced. The value of M matters: too small cuts off feasible positions, too large makes the LP relaxation so loose that branch and bound has little to prune, besides the numerical trouble of mixing coefficients of very different sizes.

B.4.6 Gradient descent

When a problem has no constraints and is too large, too nonlinear or too irregular for the solvers above, the tool of last resort is to follow the negative gradient:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k), \quad \alpha > 0 \text{ the step size (learning rate)}. \quad (\text{B.35})$$

By Proposition B.14, $f(\mathbf{x}_{k+1}) \approx f(\mathbf{x}_k) - \alpha \|\nabla f(\mathbf{x}_k)\|^2$, so a small enough step always decreases f ; the question is how small.

Proposition B.31 (Convergence of gradient descent). *Let f be convex and differentiable with an L -Lipschitz gradient, $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$, and let $\mathbf{x}^*$ be a minimizer. With a constant step $\alpha \leq 1/L$ the iterates of Equation (B.35) satisfy $f(\mathbf{x}_k) - f(\mathbf{x}^*) \leq \|\mathbf{x}_0 - \mathbf{x}^*\|^2 / (2\alpha k)$. If in addition f is μ -strongly convex, $f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{\mu}{2} \|\mathbf{y} - \mathbf{x}\|^2$ for all $\mathbf{x}, \mathbf{y}$ (equivalently $\nabla^2 f \succeq \mu \mathbf{I}$ when f is twice differentiable), then $\|\mathbf{x}_k - \mathbf{x}^*\| \leq (1 - \alpha\mu)^k \|\mathbf{x}_0 - \mathbf{x}^*\|$ for $\alpha \leq 1/L$: the error shrinks geometrically, at a rate governed by the condition number L/μ .*

Proofs are in Boyd and Vandenberghe [24, Chapter 9] and Nocedal and Wright [119, Chapter 3]. The quadratic $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x}$ with $\mathbf{A} \succ \mathbf{0}$ shows the mechanism exactly: in the eigenbasis of $\mathbf{A}$ each coordinate is multiplied by $1 - \alpha\lambda_i$ per step, so the iteration converges iff $\alpha < 2/\lambda_{\max}$, and the slowest coordinate is the one with the smallest eigenvalue: a large eigenvalue ratio, a long narrow valley, forces a small step and a slow crawl along the valley floor.

Example B.32 (Step sizes on a narrow valley). Take $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 10x_2^2)$, so $\lambda_{\min} = 1$, $\lambda_{\max} = 10$ and $\nabla f = (x_1, 10x_2)^\top$, and start at $\mathbf{x}_0 = (9, 2)^\top$. With $\alpha = 0.05$ the factors are 0.95 and 0.5: the iterates drop to the valley floor and then crawl along it, reaching $\mathbf{x}_{20} = (3.23, 0.00)^\top$ with $f = 5.20$. With $\alpha = 0.18$ the factors are 0.82 and -0.80 : the iterates zigzag across the valley but make faster progress along it, $\mathbf{x}_{20} = (0.170, 0.023)^\top$ with $f = 0.017$. With $\alpha = 0.21$ the factor -1.1 on x_2 makes the iteration diverge: after 20 steps $x_2 = 13.5$ and $f = 905$. Figure B.4 shows the first two paths.

The same picture explains two phenomena in the book. The potential-field controller of Chapter 15 is gradient descent on its potential, with the drone's motion as the iteration: its

oscillations in narrow corridors are the zigzag of Figure B.4, its local minima the non-convexity of Figure B.2. Training the prediction networks of Chapter 20 is gradient descent on a loss averaged over data, with the gradient obtained by the chain rule Equation (B.9) on random mini-batches (*stochastic* gradient descent); the learning rate is the α above, and momentum and adaptive step sizes are remedies for the narrow-valley effect.

B.5 Graphs and the Laplacian

B.5.1 Adjacency, degree and Laplacian matrices

Throughout this section $\mathbf{A}$ is the adjacency matrix and $\mathbf{L}$ the Laplacian of a graph, not the generic matrix of Appendix B.1 or the Cholesky factor of Proposition B.6, and Laplacian eigenvalues are numbered in *increasing* order, the reverse of Theorem B.4. An undirected graph $G = (V, E)$ with n vertices numbered $1, \dots, n$ is the communication network of Chapter 23: the vertices are drones and an edge $\{i, j\}$ means that i and j can exchange messages. The neighbors of i are $N_i = \{j : \{i, j\} \in E\}$.

Definition B.33 (Adjacency, degree and Laplacian matrices). The **adjacency matrix** $\mathbf{A} \in \mathbb{R}^{n \times n}$ has $a_{ij} = a_{ji} = 1$ if $\{i, j\} \in E$ and 0 otherwise (for a weighted graph, $a_{ij} = w_{ij} \geq 0$). The **degree** of i is $d_i = \sum_j a_{ij}$, the degree matrix is $\mathbf{D} = \text{diag}(d_1, \dots, d_n)$, and the **graph Laplacian** is

$$\mathbf{L} = \mathbf{D} - \mathbf{A}, \quad (\mathbf{L}\mathbf{x})_i = \sum_{j \in N_i} a_{ij}(x_i - x_j). \quad (\text{B.36})$$

The second form of Equation (B.36) is the one to remember: applied to a vector of drone states, $\mathbf{L}$ returns for every drone the sum of its disagreements with its neighbors, which is exactly what a consensus protocol feeds back.

Proposition B.34 (The Laplacian quadratic form). For every $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{x}^\top \mathbf{L} \mathbf{x} = \sum_{\{i,j\} \in E} a_{ij} (x_i - x_j)^2$.

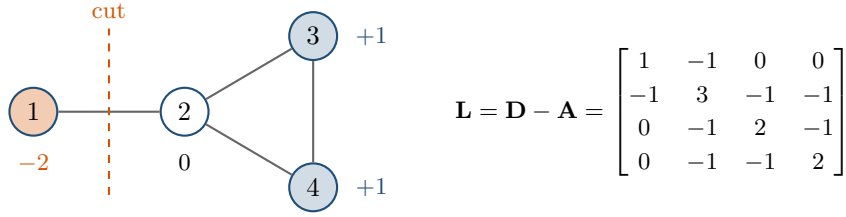
Proof. $\mathbf{x}^\top \mathbf{D} \mathbf{x} - \mathbf{x}^\top \mathbf{A} \mathbf{x} = \sum_i d_i x_i^2 - \sum_{i,j} a_{ij} x_i x_j = \frac{1}{2} \sum_{i,j} a_{ij} (x_i^2 + x_j^2 - 2x_i x_j) = \frac{1}{2} \sum_{i,j} a_{ij} (x_i - x_j)^2$, using $d_i = \sum_j a_{ij}$; each edge appears twice in the double sum, which removes the factor $\frac{1}{2}$. $\square$

Theorem B.35 (Spectrum of the Laplacian). Let $\mathbf{L}$ be the Laplacian of an undirected graph with eigenvalues $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ and maximum degree $d_{\max}$.

- (i) $\mathbf{L}$ is symmetric and PSD.
- (ii) $\mathbf{L}\mathbf{1} = \mathbf{0}$, so $\lambda_1 = 0$ with eigenvector $\mathbf{1}$.
- (iii) The multiplicity of the eigenvalue 0 equals the number of connected components of G . In particular $\lambda_2 > 0$ iff G is connected.
- (iv) $\lambda_n \leq 2d_{\max}$.

Proof. (i) Symmetry is inherited from $\mathbf{A}$ and $\mathbf{D}$; $\mathbf{x}^\top \mathbf{L} \mathbf{x} \geq 0$ by Proposition B.34. (ii) Every row of $\mathbf{L}$ sums to $d_i - \sum_j a_{ij} = 0$. (iii) By Proposition B.34, $\mathbf{x}^\top \mathbf{L} \mathbf{x} = 0$ iff $x_i = x_j$ on every edge, that is, iff $\mathbf{x}$ is constant on each connected component; for a PSD matrix, $\mathbf{x}^\top \mathbf{L} \mathbf{x} = 0$ iff $\mathbf{L} \mathbf{x} = \mathbf{0}$ (write $\mathbf{x}$ in the eigenbasis). The null space is therefore spanned by the indicator vectors of the components, which are linearly independent, so its dimension is the number of components, and this dimension is the multiplicity of 0 for a symmetric matrix. (iv) By Gershgorin's theorem every eigenvalue lies in one of the disks centered at $l_{ii} = d_i$ with radius $\sum_{j \neq i} |l_{ij}| = d_i$, hence in $[0, 2d_{\max}]$. $\square$

The eigenvalue λ_2 is the **algebraic connectivity** of the graph [44]: zero exactly when the



degrees (1, 3, 2, 2), Fiedler vector $(-2, 0, 1, 1)$, eigenvalues $\lambda = (0, 1, 3, 4)$; $\lambda_2 = 1 > 0$: connected

Figure B.5. The network of Example B.36 and its Laplacian. Each diagonal entry is a degree and each -1 an edge. The entries of the Fiedler vector are written next to the vertices; the sign change across the edge $\{1, 2\}$ marks the cut that disconnects the graph.

network is cut, and the larger, the faster information spreads. Chapter 23 shows that under the consensus dynamics $\dot{\mathbf{x}} = -\mathbf{L}\mathbf{x}$ the average $\frac{1}{n}\mathbf{1}^\top \mathbf{x}$ is conserved, because $\mathbf{1}^\top \mathbf{L} = \mathbf{0}^\top$ by (i) and (ii), and the disagreement decays like $e^{-\lambda_2 t}$ [123]. The discrete version $\mathbf{x}_{k+1} = (\mathbf{I} - \varepsilon \mathbf{L})\mathbf{x}_k$ multiplies eigencomponent i by $1 - \varepsilon \lambda_i$ per step, so it converges iff $0 < \varepsilon < 2/\lambda_n$, and by (iv) any $\varepsilon < 1/d_{\max}$ is safe without knowing the spectrum.

Example B.36 (A four-drone network). The graph of Figure B.5 has the edges $\{1, 2\}$, $\{2, 3\}$, $\{3, 4\}$ and $\{2, 4\}$, so the degrees are (1, 3, 2, 2) and

$$\mathbf{L} = \begin{bmatrix} 1 & -1 & 0 & 0 \\ -1 & 3 & -1 & -1 \\ 0 & -1 & 2 & -1 \\ 0 & -1 & -1 & 2 \end{bmatrix}, \quad \lambda = (0, 1, 3, 4).$$

The graph is connected, so $\lambda_2 = 1 > 0$, and $\lambda_n = 4 \leq 2d_{\max} = 6$. The eigenvector of λ_2 , the Fiedler vector $(-2, 0, 1, 1)^\top$, is negative on drone 1 and positive on drones 3 and 4: its sign pattern points at the bridge $\{1, 2\}$ as the weakest link. Deleting that edge isolates drone 1 and the spectrum becomes $(0, 0, 3, 3)$, with the double zero of Theorem B.35(iii). From $\mathbf{x}_0 = (4, 0, 1, -1)^\top$ the consensus dynamics converge to the average 1 on every drone.

B.5.2 Shortest-path notation

The search chapters use a directed graph $G = (V, E)$ with a cost $c(u, v) \geq 0$ on every edge, successors $\text{Succ}(u) = \{v : (u, v) \in E\}$ and predecessors $\text{Pred}(u)$. A path $\pi = (v_0, \dots, v_m)$ follows edges and costs $\text{cost}(\pi) = \sum_i c(v_i, v_{i+1})$; the **shortest-path distance** $\text{dist}(u, v)$ is the least cost of a path from u to v , ∞ if there is none, and it satisfies the triangle inequality $\text{dist}(u, w) \leq \text{dist}(u, v) + \text{dist}(v, w)$. Chapter 3 computes $\text{dist}(s, \cdot)$ from a start s and writes it $\delta(s, v)$; Chapter 4 keeps for every node the cost $g(n)$ of the best path found so far, a heuristic estimate $h(n)$ of $\text{dist}(n, \gamma)$ to the goal γ and the priority $f = g + h$, with the frontier in OPEN and the expanded nodes in CLOSED. A 4- or 8-connected grid is such a graph with $|E| \leq 4|V|$ or $8|V|$, and the space-time graph of Chapters 4 and 8 has the $(T + 1)|V|$ states (v, t) , $t = 0, \dots, T$, with a wait edge from (v, t) to $(v, t + 1)$ beside the move edges.

B.6 Complexity and data structures

Table B.3. Costs of the operations of a binary heap and of the Python containers used in the search code. Hash-table costs are expected costs; worst cases are $\mathcal{O}(n)$.

Structure	Operation	Cost	Python
binary heap	peek minimum	$\mathcal{O}(1)$	<code>heap[0]</code>
binary heap	insert	$\mathcal{O}(\log n)$	<code>heapq.heappush</code>
binary heap	extract minimum	$\mathcal{O}(\log n)$	<code>heapq.heappop</code>
binary heap	build from n items	$\mathcal{O}(n)$	<code>heapq.heapify</code>
binary heap	decrease-key	$\mathcal{O}(\log n)$	push again; skip stale entries
hash table	insert, lookup, delete	$\mathcal{O}(1)$ exp.	<code>dict</code> , <code>set</code>
dynamic array	append; index	$\mathcal{O}(1)$ amort.; $\mathcal{O}(1)$	<code>list.append</code> , <code>a[i]</code>
dynamic array	pop front; membership	$\mathcal{O}(n)$	<code>list.pop(0)</code> , <code>x in a</code>
sorted array	insert	$\mathcal{O}(n)$	<code>bisect.insort</code>
any	sort n items	$\mathcal{O}(n \log n)$	<code>sorted</code>

B.6.1 Asymptotic notation

Definition B.37 (Big-O, Omega and Theta). For functions $f, g: \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$, $f(n) = \mathcal{O}(g(n))$ if there are constants $c > 0$ and n_0 with $f(n) \leq c g(n)$ for all $n \geq n_0$; $f(n) = \Omega(g(n))$ if there are constants $c > 0$ and n_0 with $f(n) \geq c g(n)$ for all $n \geq n_0$; and $f(n) = \Theta(g(n))$ if there are constants $c_1, c_2 > 0$ and n_0 with $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$, that is, if both bounds hold.

Big-O is an upper bound up to a constant factor for large inputs; Θ says the bound is tight. Constants and lower-order terms are dropped ($3n^2 + 50n = \Theta(n^2)$), the base of a logarithm is irrelevant, and the usual ladder is $1 \prec \log n \prec n \prec n \log n \prec n^2 \prec 2^n \prec n!$. A bound describes the worst case unless it says *expected* (average over random choices) or *amortized* (average over a sequence of operations), and it must be read with the right variables: Dijkstra’s algorithm with a binary heap runs in $\mathcal{O}((|V| + |E|) \log |V|)$ (Chapter 3), which on a grid with $|E| = \Theta(|V|)$ is $\mathcal{O}(|V| \log |V|)$; a space-time search over T steps has $(T + 1)|V|$ states; and the exponential worst case of the multi-agent solvers of Chapters 9 and 10 is in the number of agents, not in the size of the map.

B.6.2 Binary heaps

A **binary heap** stores n keys in an array $a[0], \dots, a[n-1]$ read as a complete binary tree: the children of position i are $2i+1$ and $2i+2$, its parent is $\lfloor (i-1)/2 \rfloor$, and the *heap property* says that no key is larger than its children’s, so $a[0]$ is the minimum. Inserting appends the key and swaps it upward while it is smaller than its parent (*sift-up*); extracting the minimum moves the last key to the root and swaps it downward with its smaller child (*sift-down*); each walks one root-to-leaf path of length $\lfloor \log_2 n \rfloor$, hence $\mathcal{O}(\log n)$, and building a heap from an unsorted array by sifting down from the last internal node costs only $\mathcal{O}(n)$ [31, Chapter 6]. Table B.3 collects the costs of the operations a search needs, with those of the Python containers used in the book’s code.

B.6.3 Hashing, and the `heapq` and `dict` idiom

A **hash table** maps a key to an array position with a hash function and resolves the occasional collision by probing or chaining, so that insertion, lookup and deletion take constant expected time [31, Chapter 11]; `dict` and `set` are hash tables. Keys must be *hashable*, in Python immutable: tuples of integers such as a cell (x, y) or a space-time state (x, y, t) are ideal, lists and NumPy arrays are not accepted, and floating-point keys are legal but dangerous because

two values equal on paper may differ in the last bit, which is why the search code of the book indexes cells by integer tuples.

The `heapq` module implements a binary heap on a plain list, but it has no decrease-key. The standard repair is the lazy-deletion queue that [Chapter 2](#) states as [Algorithm 2.1](#) and every planner of the book calls: a `dict` holds the best known key of each node, an improved key is pushed as a fresh entry $(f, -g, counter, node)$ rather than edited in place, and a popped entry whose key is no longer the best one is discarded. The heap then holds at most one entry per relaxation, $\mathcal{O}(|E|)$ in total, which leaves the asymptotic bounds unchanged.

Heap entries must be comparable

`heapq` orders tuples lexicographically, so when two entries agree on f and g it compares the next field. If that field is a node without an ordering (a dictionary, an object) the search crashes at the first tie, and if it is a NumPy array it crashes at the first comparison. A running counter from `itertools.count()`, placed before the node, guarantees that the comparison never reaches the node and makes tie-breaking deterministic; the entry $-g$ before the counter prefers the larger g among equal f , as [Chapter 4](#) recommends.

B.7 Summary

Chapter summary

- A symmetric matrix is a scaling along orthogonal axes, $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$; it is PSD iff all $\lambda_i \geq 0$, and every covariance is PSD. Block matrices are handled with the Schur complement, which gives the block inverse, the block determinant and the Woodbury identity.
- Least squares is solved by the normal equations $\mathbf{A}^\top \mathbf{A} \mathbf{x} = \mathbf{A}^\top \mathbf{b}$; never form an inverse to do it.
- Linearization keeps the Jacobian and drops an error of second order in the displacement; the double-integrator step $\mathbf{p}' = \mathbf{p} + \mathbf{v}\Delta t + \frac{1}{2}\mathbf{a}\Delta t^2$, $\mathbf{v}' = \mathbf{v} + \mathbf{a}\Delta t$ is exact for a piecewise-constant input.
- Covariances propagate as $\mathbf{A}\mathbf{\Sigma}\mathbf{A}^\top$ under any linear map; Gaussians stay Gaussian under affine maps, marginalization and conditioning, and the conditioning formula [Equation \(B.18\)](#) is the Kalman update in disguise.
- A filter is total probability (predict) followed by Bayes' rule (update); Monte Carlo errors shrink like $1/\sqrt{N}$; the squared Mahalanobis distance of a Gaussian vector is χ_n^2 , which sets gates such as 5.991 for $n = 2$ at 95 %.
- Convexity makes local minima global and the KKT conditions sufficient. An LP attains its optimum at a vertex, a convex QP is an LP with a quadratic cost, a MILP is solved exactly by branch and bound with big- M binaries encoding disjunctions, and gradient descent converges for $\alpha < 2/L$ with L the Lipschitz constant of the gradient (that is $2/\lambda_{\max}$ for a quadratic) but crawls on ill-conditioned problems.
- The Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{A}$ is PSD with $\mathbf{L}\mathbf{1} = \mathbf{0}$; $\lambda_2 > 0$ iff the graph is connected and sets the speed of consensus, and $\lambda_n \leq 2d_{\max}$ bounds the safe step size.
- A binary heap gives $\mathcal{O}(\log n)$ insertion and extraction; `heapq` plus a `dict`, the lazy deletion of [Algorithm 2.1](#) and a tie-breaking counter is the priority queue of every search in this book.

Further reading. Linear algebra, convexity and optimality conditions are treated with great clarity in Boyd and Vandenberghe [24], whose appendices cover the matrix facts of [Appendix B.1](#); the algorithms of optimization are in Nocedal and Wright [119] and integer programming in Wolsey [181]. The probabilistic material follows Thrun, Burgard, and Fox [168] and Särkkä [144]; gating is treated in Bar-Shalom, Li, and Kirubarajan [7] and sequential Monte Carlo in Doucet, Freitas, and Gordon [36]. Spectral graph theory and consensus are surveyed in Olfati-Saber, Fax, and Murray [123], building on Fiedler [44]; heaps, hashing and asymptotic notation are in Cormen et al. [31].

Hints and Solutions to Selected Exercises

This appendix collects hints and solutions for selected exercises. Solutions are grouped by chapter; each entry names the exercise it answers.

Exercise 1.1. (a) Global planner: a planned conflict, known before take-off, resolved by CBS or ECBS. (b) Prediction layer plus local safety layer: the bird is tracked and its motion predicted; with a time to collision of 1.5 s inside the safety horizon, the local layer produces an avoidance velocity. (c) Replanning layer: the local layer can keep C clear of the hovering intruder but cannot make progress through the blocked corridor, so reconnection fails and D^* Lite replans from the current state; the new path must then be re-checked against the swarm. (d) Global planner: the zone is known before the flight, so it becomes part of the map. (e) Global planner, in its re-check role (step 5 of the decision logic): a new inter-agent conflict is a planned conflict again. (f) Prediction layer: the Kalman filter turns the noisy measurements into a state estimate and the predictor extrapolates it. (g) None of the four layers: the flight controller tracks the commanded velocity and corrects the disturbance; only if the deviation grew large enough to threaten a conflict would the local layer notice it. (h) Local safety layer with a formation constraint, and the replanner if the constraint cannot be met; how to trade the constraint against the collision risk is one of the research directions of [Section 1.5](#).

Exercise 1.2. Dijkstra: complete (strong form) and optimal. Weighted A^* with $w = 1.5$: complete and bounded suboptimal with factor 1.5, not optimal. Prioritized planning: none of the six—it is fast, but it is neither complete nor optimal and it guarantees no bound. CBS: complete on solvable instances and optimal for the sum of costs. ECBS: complete on solvable instances and bounded suboptimal. ARA^* : complete, bounded suboptimal at every iteration, and anytime. D^* Lite: complete, optimal and incremental. ORCA: real-time, and nothing else—it is the only real-time entry in the list, and it is neither complete (it can deadlock) nor optimal. RRT: probabilistically complete. RRT^* : probabilistically complete and asymptotically optimal. Optimal and real-time never appear together, and the conflict is structural: optimality is a statement about a whole route, so it needs a search whose size grows with the map and the number of drones, while a real-time method must bound the work per cycle and therefore chooses among a bounded set of velocities for the next fraction of a second. Bounding the work bounds what can be examined, and what is never examined cannot be proved best.

Exercise 1.4. (a) In a corridor one cell wide, two drones heading toward each other see the same situation mirrored. A reactive rule maps the current situation to an action and has no memory and no look-ahead, so both drones slow down, stop, or try to sidestep into the same wall; neither has a reason to back up to the last passing place, because backing up does not reduce the immediate collision risk and moves the drone away from its goal. They hover facing each other: a deadlock. (b) Four drones at the corners of a square, each heading for the

opposite corner, all meet in the middle at the same time. A symmetric rule gives symmetric actions, so the drones either all stop at the same distance from the center or all circle around it; nothing in the rule breaks the symmetry, and none of them reaches its goal. A global plan adds exactly what the reactive rule lacks: knowledge of the other drones' full routes and of time, which lets a planner decide that one drone waits or takes a detour *before* the encounter. A random component does break the symmetry and often resolves the deadlock in practice, but at the cost of losing every guarantee: there is no bound on how long the resolution takes, the outcome is not reproducible, and in a corridor a random choice can still oscillate for a long time. [Chapter 13](#) shows this behavior with ORCA and [Chapter 15](#) with potential fields.

Exercise 1.5. (a) At $t = 4$ drone A is at its fifth cell $(3, 3)$ and drone C at its fifth cell $(3, 3)$: a vertex conflict. An edge conflict would need A and C to exchange cells between two consecutive steps; at $t = 3 \rightarrow 4$ the moves are $(4, 3) \rightarrow (3, 3)$ and $(3, 4) \rightarrow (3, 3)$, and at $t = 4 \rightarrow 5$ they are $(3, 3) \rightarrow (2, 3)$ and $(3, 3) \rightarrow (3, 2)$, so no swap occurs. (b) The new path of C is $(6, 4), (5, 4), (4, 4), (3, 4), (3, 4), (3, 3), (3, 2), (3, 1)$ with 7 steps. At $t = 4$ drone C is at $(3, 4)$ and A at $(3, 3)$; at $t = 5$ drone C enters $(3, 3)$ while A has moved on to $(2, 3)$, so there is no conflict, and `find_conflicts` confirms it. The sum of costs is $10 + 9 + 7 = 26$ and the makespan stays 10. Letting A wait instead costs the same: sum of costs 26, makespan 11 because A is the longest path, so waiting C is better for the makespan. (c) The relative position is $\mathbf{p}_I(t) - \mathbf{p}_B(t) = (11 - 0.54t - 1.5 - t, 7.5 - 0.38t - 5.5) = (9.5 - 1.54t, 2 - 0.38t)$. Its squared norm is a parabola in t whose minimum lies where the derivative vanishes: $1.54(9.5 - 1.54t) + 0.38(2 - 0.38t) = 0$, that is, $t^* = (1.54 \cdot 9.5 + 0.38 \cdot 2) / (1.54^2 + 0.38^2) = 15.39 / 2.5160 \approx 6.12$. At that time the relative position is $(0.08, -0.32)$ and the distance is ≈ 0.33 cells, slightly less than the 0.38 that the table reports at the integer time $t = 6$; checking only integer steps can miss the true closest approach. (d) With the book's code, waiting one step gives a minimum distance of 0.72 cells (at $t = 7$), two steps 0.98, and three steps 1.06, so three wait steps are needed. Such a decision would be made by the local safety layer, during the flight, from the prediction; before take-off the intruder is not known, and even if it were, its velocity is a prediction that may be wrong by the time the wait would matter.

Exercise 1.6. With $\mathbf{p}_0 = (11.0, 7.5)$ and $\mathbf{v} = (-0.54, -0.38)$ the predicted positions are $\mathbf{p}(14) = (3.44, 2.18)$, $\mathbf{p}(15) = (2.90, 1.80)$ and $\mathbf{p}(19) = (0.74, 0.28)$. The goal cells are $(3, 1)$ for C and $(0, 0)$ for A , with centers $(3.5, 1.5)$ and $(0.5, 0.5)$, so the intruder passes 0.68 cells from parked C at $t = 14$, 0.67 cells from it at $t = 15$ and 0.33 cells from parked A at $t = 19$: three encounters closer than one cell, none of which the default horizon of makespan plus one ($t \leq 11$) ever looks at. The horizon must therefore cover the whole time the drones are exposed, not the time the plan is being flown. The rule that a drone stays at its goal is what creates the exposure: a parked drone is a stationary obstacle for the rest of the mission, and it is exactly the parked drones that a makespan-length check forgets. Protecting them is the job of the local safety layer, which keeps running on every drone for as long as it is airborne; if an intruder settles on a goal cell, the replanning layer must give that drone somewhere else to wait.

Exercise 1.7. (a) At 5 m/s the intruder moves $5 \times 0.02 = 0.1$ m in one control cycle, $5 \times 0.5 = 2.5$ m during a replan and $5 \times 3 = 15$ m during a global plan: one control cycle is harmless, one global plan is three city buses. (b) Ten control cycles are $\tau_h = 10 \times 20 \text{ ms} = 0.2$ s. The two close at $5 + 3 = 8$ m/s, so in that time the gap shrinks by $8 \times 0.2 = 1.6$ m and the risk must be declared at $2 + 8 \times 0.2 = 3.6$ m of separation. (c) A constant-velocity prediction is worthless 30 s ahead—the intruder will have turned long before—so a 30 s horizon would flag almost every distant object and the drones would spend the flight avoiding conflicts that never happen. The horizon must match the range over which the prediction is trustworthy, not the range over which the local layer would like warning.

Exercise 1.8. Hints rather than a full solution. (a) Draw the obstacles with the `sample` method of `random.Random(seed)`, then draw $2k$ distinct free cells and keep them only if every start-goal pair is at a minimum Manhattan distance and breadth-first search reaches every goal; redraw the obstacles if that fails too often. For the intruder, choose a border side, a random entry point on it and a random exit point on the opposite side, and normalize the difference to a random speed; walk along the line in small steps and reject it if it enters a blocked cell. (b) Clamp the time index to the last cell so that a drone stays at its goal; a vertex conflict is $v_t^i = v_t^j$, an edge conflict is $v_t^i = v_{t+1}^j$ and $v_{t+1}^i = v_t^j$ with $v_t^i \neq v_{t+1}^i$. (c) With the book's generator on a 12×8 grid with 15% blocked cells and seeds 0 to 199, the fraction of scenarios whose independent shortest paths conflict is about 0.215 for $k = 2$, 0.595 for $k = 4$ and 0.995 for $k = 8$ (199 scenarios out of 200); your numbers will differ with the tie-breaking of your search and the minimum start-goal distance, but the trend is the same: with a handful of drones on a small map, planning independently almost always produces a collision, which is why [Chapters 7 to 9](#) exist. (d) Store the intruders as a list of position-velocity pairs and keep the check that every line is free; the book's `to_json` method shows a suitable layout.

Exercise 2.2. Take the empty 8-connected grid with $s = (0, 0)$ and $\gamma = (4, 0)$. With a diagonal cost of 1 the four straight moves and the zigzag $(0, 0), (1, 1), (2, 0), (3, 1), (4, 0)$ both cost 4, so a cheapest grid path may be the zigzag, whose polyline through the cell centers has length $4\sqrt{2} \approx 5.657$ instead of 4. The cost model is now the Chebyshev distance $\max(|\Delta x|, |\Delta y|)$, which is smaller than the Euclidean distance: from $(0, 0)$ to $(4, 4)$ the cheapest path costs 4 while the straight-line estimate is $4\sqrt{2} \approx 5.657$. A heuristic that returns the straight-line distance would therefore overestimate, lose admissibility, and let A^* return a suboptimal path ([Chapter 4](#)); the admissible heuristic for this cost model is the Chebyshev distance itself.

Exercise 2.4. The time-expanded graph with horizon $T_{\max}$ has $|V| (T_{\max} + 1)$ vertices, one copy of every vertex per time layer. Between two consecutive layers there are $|V|$ wait edges and, for an undirected base graph, $2|E|$ move edges (each edge $\{u, v\}$ can be traversed in both directions), so there are $T_{\max} (|V| + 2|E|)$ edges in total. Every edge goes from layer t to layer $t + 1$, so no edge ever returns to an earlier layer and the graph is acyclic. A time-indexed path $(v_0, \dots, v_{T_{\max}})$ visits one vertex per layer, and consecutive vertices are either equal (a wait edge) or adjacent in G (a move edge); hence it is exactly a path from $(v_0, 0)$ to $(v_{T_{\max}}, T_{\max})$ in the time-expanded graph.

Exercise 2.5. For the plan of [Example 2.20](#) the arrival times are $\text{cost}(\pi_A) = 3$ and $\text{cost}(\pi_B) = 3$, so $\text{MS} = 3$ and $\text{SoC} = 6$. For the instance in which the two objectives disagree, take a corridor of cells $(0, 1), \dots, (5, 1)$ with agent A going from $(0, 1)$ to $(5, 1)$, agent B_1 from $(1, 2)$ to $(1, 0)$ and agent B_2 from $(2, 3)$ through $(2, 2)$ to $(2, 0)$; every other cell is blocked. Without waiting, B_1 would enter cell $(1, 1)$ at $t = 1$ and B_2 would enter $(2, 1)$ at $t = 2$, exactly when A is there. If A never waits ($T_A = 5$), both crossing agents must wait one step: $T_{B_1} = 3$, $T_{B_2} = 4$, giving $\text{MS} = 5$ and $\text{SoC} = 12$. If instead A waits one step at its start, neither crossing agent is delayed: $T_A = 6$, $T_{B_1} = 2$, $T_{B_2} = 3$, giving $\text{MS} = 6$ and $\text{SoC} = 11$. No plan does better on both counts. Every agent needs at least its unobstructed travel time (5, 2 and 3 steps), so $\text{SoC} \geq 10$ with equality only if nobody waits, which is impossible because of the two conflicts. Hence $\text{SoC} \geq 11$, and a plan with $\text{SoC} = 11$ contains exactly one unit of delay; a delay of B_1 alone leaves the conflict of A with B_2 , a delay of B_2 alone leaves the conflict with B_1 , so the delayed agent must be A and the makespan is 6. Conversely, makespan 5 forces A to move without waiting, so B_1 cannot enter $(1, 1)$ before $t = 2$ and B_2 cannot enter $(2, 1)$ before $t = 3$, which gives $\text{SoC} \geq 5 + 3 + 4 = 12$. The two objectives therefore disagree on this instance.

Exercise 2.7. With $\mathbf{p} = \mathbf{p}_B - \mathbf{p}_A$ and $\mathbf{v} = \mathbf{v}_A - \mathbf{v}_B$, the signs of [Chapters 12](#) and [13](#), the squared distance is $D(t)^2 = \|\mathbf{p} - t\mathbf{v}\|^2 = \mathbf{p} \cdot \mathbf{p} - 2t \mathbf{p} \cdot \mathbf{v} + t^2 \mathbf{v} \cdot \mathbf{v}$, a parabola in t .

Differentiating the square gives $\frac{d}{dt}D(t)^2 = -2\mathbf{p} \cdot \mathbf{v} + 2t\mathbf{v} \cdot \mathbf{v}$, which vanishes at t^* ; solving for t^* gives $t^* = (\mathbf{p} \cdot \mathbf{v})/(\mathbf{v} \cdot \mathbf{v})$. Writing $\mathbf{p} = \mathbf{p}_{\parallel} + \mathbf{p}_{\perp}$ with $\mathbf{p}_{\parallel}$ along $\mathbf{v}$, the term $\mathbf{p}_{\parallel} - t^*\mathbf{v}$ vanishes, so $d_{\min} = \|\mathbf{p}_{\perp}\| = |\mathbf{p} \times \mathbf{v}|/\|\mathbf{v}\|$ because $|\mathbf{p} \times \mathbf{v}| = \|\mathbf{p}\| \|\mathbf{v}\| |\sin \phi|$ is the area of the parallelogram spanned by the two vectors. For $\mathbf{p} = (4, 2)$, $\mathbf{v} = (1, 1)$: $t^* = 6/2 = 3$, $\mathbf{p} \times \mathbf{v} = 4 - 2 = 2$, $d_{\min} = 2/\sqrt{2} = \sqrt{2}$. Along the line of motion the disk of radius R is entered at $t_c = t^* - \sqrt{R^2 - d_{\min}^2}/\|\mathbf{v}\|$, which for $R = 1.5$ gives $t_c = 3 - \sqrt{0.25}/\sqrt{2} \approx 2.646$; for $R = 1 < \sqrt{2}$ the square root is undefined and the disks never touch, in agreement with `time_to_collision` in `ch02_toolbox.py`.

Exercise 2.9. The characteristic polynomial of $\Sigma = \begin{pmatrix} 2 & 1.2 \\ 1.2 & 1 \end{pmatrix}$ is $\lambda^2 - 3\lambda + 0.56 = 0$, so $\lambda_1 = 2.8$ and $\lambda_2 = 0.2$. The eigenvector of λ_1 solves $-0.8e_x + 1.2e_y = 0$, hence $\mathbf{e}_1 \propto (3, 2)$ and the major axis makes the angle $\arctan(2/3) \approx 33.7^\circ$ with the x -axis. The 1σ semi-axes are $\sqrt{2.8} \approx 1.673$ and $\sqrt{0.2} \approx 0.447$; the 2σ ellipse doubles both. In two dimensions the mass inside the $k\sigma$ ellipse is $1 - e^{-k^2/2}$: 39.3% for $k = 1$ and 86.5% for $k = 2$ (not 68% and 95%, which are the one-dimensional values); the 400 seeded samples of Figure 2.7 give 39.8% and 85.8%.

Exercise 2.1. With the map `["....", ".#..", "...."]` the blocked cell is (1, 1). Cell (0, 0) has the neighbors (1, 0) and (0, 1) at cost 1; the diagonal to (1, 1) is blocked. Cell (2, 1) is adjacent to the blocked cell (1, 1), so the straight moves lead to (3, 1), (2, 0) and (2, 2) at cost 1. Of the four diagonals, those to (1, 0) and (1, 2) pass between (1, 1) and a free cell and are forbidden by the corner-cutting rule of Definition 2.5; only (3, 0) and (3, 2) remain, at cost $\sqrt{2}$. Cell (2, 1) therefore has five neighbors. Cell (3, 0) has (2, 0) and (3, 1) at cost 1 and (2, 1) at cost $\sqrt{2}$. For the counts: a 4-connected $W \times H$ grid has $W(H - 1)$ vertical and $H(W - 1)$ horizontal edges, and each of the $(W - 1)(H - 1)$ unit squares contributes two diagonals. For $W = 4$, $H = 3$ this gives $4 \cdot 2 + 3 \cdot 3 = 17$ and $17 + 2 \cdot 3 \cdot 2 = 29$.

Exercise 2.3. A cell center at integer offset (i, j) from the blocked cell is at distance $\max(|i| - \frac{1}{2}, 0)$ and $\max(|j| - \frac{1}{2}, 0)$ from the obstacle square in the two axes, so the distance to the square is $\sqrt{\max(|i| - \frac{1}{2}, 0)^2 + \max(|j| - \frac{1}{2}, 0)^2}$. The thresholds are therefore 0.5 (the four side neighbors), $\sqrt{0.5} \approx 0.707$ (the four diagonal neighbors), 1.5, $\sqrt{2.5} \approx 1.581$ and $1.5\sqrt{2} \approx 2.121$. Hence $r = 0.6$ blocks 5 cells, $r = 1.0$ blocks 9, and $r = 1.6$ blocks 21: the 3×3 block, the four cells at distance 1.5 and the eight at distance $\sqrt{2.5}$. The blocked-cell count is a sampled area, and the exact Minkowski sum of the unit square with the disk of radius r has area $1 + 4r + \pi r^2$; the ratio of the two tends to 1 as r grows, because the sampling error lives on the boundary and grows only like r .

Exercise 2.6. Braking from $v = 2$ m/s with $a_{\max} = 1$ m/s² takes $v/a_{\max} = 2$ s, that is 20 steps of $\Delta t = 0.1$ s. The exact model Equation (2.9) covers $v^2/(2a_{\max}) = 2.0$ m. Forward Euler updates the position with the *old* velocity, so it adds $\Delta t \sum_{k=0}^{19} (2 - 0.1k) = 0.1(2 \cdot 20 - 0.1 \cdot 190) = 2.1$ m: it overestimates by $\frac{1}{2}\Delta t^2 a_{\max}$ per step, $20 \cdot 0.005 = 0.1$ m in total. The admissibility rule is $v \leq \sqrt{2a_{\max}d}$, equivalently $v^2/(2a_{\max}) \leq d$; Chapter 14 uses it to prune the dynamic window.

Exercise 2.8. Here $d = 4$, $r = 2$, so $\cos \alpha = r/d = \frac{1}{2}$, $\alpha = 60^\circ$ and $\sin \alpha = \sqrt{3}/2$. With $\mathbf{u} = (\mathbf{p} - \mathbf{c})/d = (-1, 0)$ and $\mathbf{u}^\perp = (0, -1)$, Equation (2.13) gives $\mathbf{t}_\pm = (4, 0) + 2(\frac{1}{2}(-1, 0) \pm \frac{\sqrt{3}}{2}(0, -1)) = (3, \mp\sqrt{3})$. The tangent length is $\ell = \sqrt{d^2 - r^2} = \sqrt{12} = 2\sqrt{3}$ and the half-angle is $\theta = \arcsin(r/d) = 30^\circ$. In general the right triangle $\mathbf{ptc}$ has the right angle at $\mathbf{t}$, hypotenuse d and opposite leg r , so $\sin \theta = r/d$. As $d \rightarrow r^+$ the cone opens to a half-plane ($\theta \rightarrow 90^\circ$) and as $d \rightarrow \infty$ it closes to a single direction. For a drone this says that an intruder that is already close forbids almost every velocity, which is why avoidance must start while d is still large.

Exercise 2.10. Only checkpoints are given here, since the code is yours. (a) On an empty lattice the cheapest path from (0, 0, 0) to (3, 2, 1) costs $3 + 2 + 1 = 6$ when 6-connected

and $1 + \sqrt{2} + \sqrt{3} \approx 4.146$ when 26-connected (one triple diagonal, one double diagonal, one straight move); `lattice_path_cost` in `ch02_toolbox.py` returns the second number, and a correct search must match both. Inflating a single blocked cell of a 3D lattice by $r = 1.0$ blocks the $3 \times 3 \times 3$ block of 27 cells, the analog of the 9 cells of [Exercise 2.3](#). (b) For $\mathbf{p}_A = (0, 0, 0)$ with $\mathbf{v}_A = (1, 0, 0)$ and $\mathbf{p}_B = (4, 2, 1)$ with $\mathbf{v}_B = (0, -1, 0)$ the relative vectors are $\mathbf{p} = \mathbf{p}_B - \mathbf{p}_A = (4, 2, 1)$ and $\mathbf{v} = \mathbf{v}_A - \mathbf{v}_B = (1, 1, 0)$, so $t^* = 6/2 = 3$ s and $d_{\min} = \|(1, -1, 1)\| = \sqrt{3} \approx 1.732$ m; with a collision distance $R = 2$ the first contact is at $t_c = 3 - \sqrt{4 - 3}/\sqrt{2} \approx 2.293$ s, and with $R = 1.5 < \sqrt{3}$ there is no contact. [Equation \(2.15\)](#) and [Equation \(2.16\)](#) are unchanged in three dimensions; only $|\mathbf{p} \times \mathbf{v}|$ has to be read as the norm of the 3D cross product. (c) Stale pops never exceed pushes because `LazyPQ.pop` discards one heap entry per stale pop and every heap entry was created by exactly one push, so stale pops + successful pops $\leq$ pushes; as soon as the search pops one state successfully the inequality is strict. On a 50×50 map with 20% random obstacles a uniform-cost search over space-time states typically pushes a few times more entries than it pops successfully, and the ratio of stale pops to pushes grows with the branching factor.

Exercise 3.1. With $c(B, E) = 4$ the second step pushes $(6, E)$ instead of $(9, E)$, and the rest of the trace changes accordingly. The pops are $(0, A)$, $(2, B)$, $(3, C)$, $(5, C)$ stale, $(5, D)$, $(6, D)$ stale, $(6, E)$, $(8, G)$, $(9, F)$, $(10, F)$ stale and $(13, F)$ stale: 11 pops and 4 stale entries. When D is expanded, E already has the tentative cost $6 < 5 + 3$, so D pushes only $(13, F)$; when E is expanded it pushes $(10, F)$ and $(8, G)$; G then improves F to 9. The final distances are $\text{dist}(A, B) = 2$, $\text{dist}(A, C) = 3$, $\text{dist}(A, D) = 5$, $\text{dist}(A, E) = 6$, $\text{dist}(A, G) = 8$ and $\text{dist}(A, F) = 9$, and the shortest path to G is A, B, E, G of cost 8. The self-test of `ch03_dijkstra.py` contains this variant.

Exercise 3.2. F is pushed for the first time in step 5 with cost 13 and parent D , so a search that stops at discovery reports $\text{dist}(A, F) = 13$ and the path A, B, C, D, F , whereas the true distance is 11 along A, B, C, D, E, G, F . For G the first push, in step 7, already carries the optimal cost 10, but only because no cheaper route to G exists; the algorithm has no way of knowing that at the time. The proof of [Theorem 3.7](#) compares the key of u with the keys still in the heap *because the heap returned u as its minimum*. A vertex that has merely been pushed has not been returned by the heap, so nothing relates its tentative cost to the keys of the other open vertices; the only guarantee it has is $g(v) \geq \text{dist}(s, v)$, which is the wrong direction.

Exercise 3.4. Take the graph of [Figure 3.4](#) and add $K = 2$ to every edge: $S \rightarrow A$ costs 3, $S \rightarrow B$ costs 4, $A \rightarrow T$ costs 3 and $B \rightarrow A$ costs 0. In the shifted graph S, A, T costs 6 and S, B, A, T costs 7, so the algorithm returns S, A, T ; in the original graph these paths cost 2 and 1, and S, B, A, T is optimal. A path with k edges gains exactly kK , so the shift penalizes paths with more edges. It is harmless only when every path under comparison has the same number of edges, which is the case, for example, between two layers of the time-expanded graph of [Chapter 2](#), where every path from time 0 to time T has exactly T edges.

Exercise 3.6. (a) The backward run from G settles, in this order, $h^*(G) = 0$, $h^*(F) = 1$, $h^*(E) = 2$, $h^*(D) = 5$, $h^*(C) = 7$, $h^*(B) = 8$, $h^*(A) = 10$. Consistency holds on every edge, with equality on the edges of the backward tree: for instance $h^*(D) = 5 \leq c(D, F) + h^*(F) = 8 + 1$, $h^*(A) = 10 \leq c(A, C) + h^*(C) = 5 + 7$ and $h^*(A) = 10 = c(A, B) + h^*(B) = 2 + 8$. Note that $h^*(A) = \text{dist}(A, G) = 10$ agrees with [Table 3.1](#). (b) A move into a hatched cell costs 3 but a move out of it costs 1, so the cost of a path depends on its direction, and $\text{dist}(T, v) \neq \text{dist}(v, T)$ exactly when the cheapest route between v and T enters a different number of hatched cells in the two directions. Running `dijkstra(grid, T)` and `backward_dijkstra_heuristic(grid,`

T) shows that the two tables differ on the following cells (row, column): (1, 1): 10 forward against 8 backward, (1, 2): 13 forward against 11 backward, (1, 3): 10 forward against 8 backward, (2, 1): 9 forward against 7 backward, (2, 3): 7 forward against 5 backward, (3, 1): 6 forward against 4 backward, (3, 2): 5 forward against 3 backward, (3, 3): 4 forward against 2 backward. (c) The constraint removes the single state $((4, 2), 6)$ from the space-time graph. Every path that respects it is still a space-time path, and dropping the time index turns it into a walk in the grid of no larger cost, so $h^*(v) = \text{dist}(v, T)$ remains a lower bound (item 3 of [Theorem 3.11](#)). It is no longer exact: with unit costs the optimal path of [Example 3.5](#) reaches (4, 2) at time step 6, precisely when the cell is forbidden, so the drone must wait one step or take a detour and, with a wait action of cost 1 as in the unit-cost setting of [Chapter 2](#), the true constrained cost from S is 9, while $h^*(S) = 8$.

Exercise 3.8. (a) Take the undirected graph S – a of cost 1, a – b of cost 28, b – T of cost 1, and S – u of cost 16, u – T of cost 16, and alternate the two sides. Forward settles S (relaxing a to 1 and u to 16), backward settles T (relaxing b to 1 and u to 16), forward settles a , backward settles b ; then $\text{OPEN}_f = \{u:16, b:29\}$ and $\text{OPEN}_b = \{u:16, a:29\}$, and the next vertex settled by both sides is u , with $g_f(u) + g_b(u) = 32$. But $\text{dist}(S, T) = 30$ along S, a, b, T , so the naive rule returns a path 2 units too expensive. The rule with μ does better: when the forward expansion of a scanned the edge (a, b) , $g_b(b) = 1$ was already finite, so $\mu = 1 + 28 + 1 = 30$ was recorded, and [Equation \(3.2\)](#) fires at $16 + 16 \geq 30$ — before u is settled twice — returning S, a, b, T . (b) Let π be an S – T path with $c(\pi) < \mu$ when the rule fires. If some vertex of π is still open on the forward side and a later one is still open on the backward side, let u be the first vertex of π not settled forward and w the last one not settled backward, so u comes no later than w . The predecessor of u on π is settled and expanded forward, so u carries the key $g_f(u) = \text{dist}(S, u)$ in OPEN_f (step (ii) of the proof of [Theorem 3.7](#)), and symmetrically $g_b(w) = \text{dist}(w, T)$ in OPEN_b . Since the prefix of π up to u costs at least $\text{dist}(S, u)$, the suffix from w at least $\text{dist}(w, T)$ and the piece between them at least 0, $c(\pi) \geq \min_{\text{OPEN}_f} g_f + \min_{\text{OPEN}_b} g_b \geq \mu$, a contradiction. Otherwise the forward-settled prefix and the backward-settled suffix of π touch, so π has an edge (x, y) with x settled forward and y settled backward. Whichever of the two was settled later scanned (x, y) during its own expansion, when $g_f(x) = \text{dist}(S, x)$ and $g_b(y) = \text{dist}(y, T)$ were both final, so $\mu \leq g_f(x) + c(x, y) + g_b(y) \leq c(\pi)$, again a contradiction. Hence no path is cheaper than μ . (c) Report the two settled-cell counts side by side for each grid. The disk argument predicts a factor of about one half on open maps; the measured factor is worse whenever obstacles funnel both waves through the same corridor, since the two disks then overlap long before they meet.

Exercise 3.7. Checkpoints, not a solution. In `ch03_dijkstra.py` the models are `GridGraph` (successors, with the corner-cutting rule), `dijkstra`, `reconstruct_path`, `backward_dijkstra_heuristic` and, as an oracle for (c), `brute_force_distances`. Four structures carry the search: a heap of triples $(g, \text{counter}, v)$, a g dictionary, a parent dictionary and a CLOSED set — no decrease-key, only lazy deletion. Steps: test the successor function alone on [Figure 3.3](#), where a diagonal is legal only when both of its axis neighbors are free; add the loop of [Algorithm 3.1](#), discarding a popped entry whose key exceeds the stored g ; put the early exit at pop time, on line 9; run the same code on the reversed graph for (c). Expected numbers ([Figure 3.5](#), 20% obstacles): on 30×30 the search settles 662.9 of 678.8 free cells at a path cost of 49.0; on 100×100 , 7437.7 of 7498.5 and 165.5. The early exit saves almost nothing corner to corner (662.5) and about 60% with a central goal (280.9): run both, that contrast is the plot. Classic mistakes: testing the goal at generation time; counting stale pops as expansions (the example graph pops twelve entries of which five are stale); cutting a corner between two blocked cells. Done, for the Week 1 milestone of [Appendix A](#), when the settled cells are always those with cost at most the last popped key ([Theorem 3.7](#)), your true-distance table matches a forward

search from every free cell, and A^* with $h \equiv 0$ (Chapter 4) expands the same cells up to tie-breaking.

Exercise 4.1. Step 11 expands $(3, 4)$ with $g = 7$, $h = 4$, $f = 11$, and OPEN becomes $(0, 4):11$, $(1, 5):13$, $(2, 5):13$, $(3, 3):11$, $(3, 5):13$. Step 12 expands $(3, 3)$ with $g = 8$, $h = 3$, $f = 11$, and OPEN becomes $(0, 4):11$, $(1, 5):13$, $(2, 5):13$, $(3, 2):11$, $(3, 5):13$. The cell $(0, 4)$ has $f = 11$ as well, so it ties with the cells of the column $x = 3$ at every one of these steps; the tie-breaking rule of Algorithm 4.1 prefers the larger g , and $g(0, 4) = 4$ is much smaller than $g(3, 4) = 7$, $g(3, 3) = 8$, $g(3, 2) = 9$. The search therefore finishes the dive into the dead end first and pops $(0, 4)$ at step 14, immediately after $(3, 2)$, whose only successor $(3, 1)$ has $f = 13$.

Exercise 4.2. At $(1, 4)$: $d_x = 4$, $d_y = 2$, so $h_M = 6$, $h_O = 4 + 2(\sqrt{2} - 1) \approx 4.83$, $h_E = \sqrt{20} \approx 4.47$, $h_C = 4$. At $(3, 1)$: $d_x = 2$, $d_y = 1$, so $h_M = 3$, $h_O = 2 + (\sqrt{2} - 1) \approx 2.41$, $h_E = \sqrt{5} \approx 2.24$, $h_C = 2$. (a) The order is the same everywhere, $h_M \geq h_O \geq h_E \geq h_C$, so the Manhattan distance is the most informed of the four; the first inequality holds because $d_x + d_y = \max + \min \geq \max + (\sqrt{2} - 1) \min$. (b) On the 4-connected instance all four are admissible (Table 4.1), and the Manhattan distance is exact on an empty grid. On an 8-connected instance the Manhattan distance is not admissible, the other three are. (c) The best of the four is the Manhattan distance, and it underestimates by $8 - 6 = 2$ at $(1, 4)$ and by $5 - 3 = 2$ at $(3, 1)$. Both errors come from the second wall, the cells $(4, 1)$ to $(4, 4)$: from $(1, 4)$ the route must climb to the top row before it can cross $x = 4$, and from $(3, 1)$ it must drop to $y = 0$, in each case two moves that the straight-line count does not see.

Exercise 4.3. Write $M = \max(d_x, d_y)$ and $m = \min(d_x, d_y)$, so $h_O = M + (\sqrt{2} - 1)m$. On an obstacle-free 8-connected grid a route from n to γ needs at least M moves, and each move changes $\min$ by at most one, so at most m of them can be diagonal; taking m diagonal and $M - m$ straight moves is feasible and costs $\sqrt{2}m + (M - m) = M + (\sqrt{2} - 1)m$, which is therefore exactly h^* on an empty grid. For consistency, take a move from n to n' and let M', m' be the values at n' . A straight move changes exactly one of d_x, d_y by ± 1 ; a diagonal move changes both by ± 1 . Enumerating: a straight move that decreases M gives $h_O(n) - h_O(n') = 1 \leq 1$; a straight move that decreases m gives $\sqrt{2} - 1 \leq 1$; a diagonal move that decreases both gives $1 + (\sqrt{2} - 1) = \sqrt{2} \leq \sqrt{2}$; a diagonal move that decreases one and increases the other gives $|1 - (\sqrt{2} - 1)| = 2 - \sqrt{2} \leq \sqrt{2}$. Moves that increase M or m only make the left-hand side smaller. In every case $h_O(n) \leq c(n, n') + h_O(n')$, and $h_O(\gamma) = 0$. Finally, consistency is a condition on the edges that exist: forbidding the corner-cutting diagonals of Definition 2.5 removes edges, and removing an edge cannot violate an inequality that held on all of them. (Admissibility with obstacles follows because obstacles only lengthen paths.) The same argument is Proposition 4.5 with d the octile metric.

Exercise 4.4. Let h be consistent and let no closed node ever be re-opened. Claim: every node n that is expanded satisfies $g(n) \leq w g^*(n)$. Induct on the order of expansion. For s it is clear. Let n be expanded and let $P = (s, n_1, \dots, n_k = n)$ be a cheapest path to n ; let n_i be the last node of P that was expanded before n with $g(n_i) \leq w g^*(n_i)$ (it exists: s qualifies). Then n_{i+1} is in OPEN with $g(n_{i+1}) \leq g(n_i) + c(n_i, n_{i+1}) \leq w g^*(n_{i+1})$. Since n was popped before n_{i+1} ,

$$g(n) + w h(n) \leq g(n_{i+1}) + w h(n_{i+1}) \leq w g^*(n_{i+1}) + w h(n_{i+1}) \leq w(g^*(n_{i+1}) + h^*(n_{i+1})),$$

using admissibility in the last step, and consistency along the rest of P gives $h(n_{i+1}) \leq g^*(n) - g^*(n_{i+1}) + h(n)$, whence $g(n) \leq w g^*(n)$. Applying the claim to the goal gives $g(\gamma) \leq w C^*$. Consistency is used exactly once, in the step that propagates the bound along P ; with a merely admissible h that step fails, a node may be expanded with a g worse than

$w g^*$, and, with re-opening disabled, that error is never corrected. This is the argument of Likhachev, Gordon and Thrun for A^* (Chapter 6).

Exercise 4.5. (a) $h(n) = \max(h_1(n), h_2(n)) \leq h^*(n)$ because each term is, so h is admissible. If both are consistent, then for an edge (n, n') and for j the index attaining the maximum at n , $h(n) = h_j(n) \leq c(n, n') + h_j(n') \leq c(n, n') + h(n')$; and $h(\gamma) = 0$. (b) $h \geq h_1$ and $h \geq h_2$ by definition, so Theorem 4.13 says that A^* with h expands, among nodes with $f < C^*$, a subset of what either of the two expands. (c) On the grid of Example 4.6, take h_1 the Manhattan distance and $h_2(n) = |x_n - 5| + 2$ on the cells with $x_n \leq 3$ and $y_n \geq 1$ (which must first reach the row $y = 0$ or the row $y = 5$ to cross the wall at $x = 4$, so two extra vertical moves are unavoidable), with $h_2 = h_1$ on all other cells: their maximum is strictly larger than h_1 at, for example, $(3, 1)$, where it gives 4 instead of 3. One does not simply maximize everything because each term must be computed at every generated node: a heuristic that is slightly more informed but ten times more expensive loses, and this trade-off between node quality and node rate is the reason the cheap closed-form distances of Table 4.1 remain the default.

Exercise 4.6. Sort the absolute differences as $d_{(1)} \geq d_{(2)} \geq d_{(3)}$. *6-connected* (only axis moves, cost 1): $h_6 = d_x + d_y + d_z$, the 3D Manhattan distance, exact on an empty lattice. *26-connected* (costs $1, \sqrt{2}, \sqrt{3}$): use $d_{(3)}$ triple moves, then $d_{(2)} - d_{(3)}$ double moves, then $d_{(1)} - d_{(2)}$ single moves, so $h_{26} = \sqrt{3}d_{(3)} + \sqrt{2}(d_{(2)} - d_{(3)}) + (d_{(1)} - d_{(2)})$; no route can do better, because each move reduces the sum $d_x + d_y + d_z$ by at most the number of coordinates it changes and $\sqrt{3} < \sqrt{2} + 1 < 3$. *18-connected* (axis and face-diagonal moves, costs $1, \sqrt{2}$): with $S = d_x + d_y + d_z$ the number of double moves is at most $m = \min(d_{(2)} + d_{(3)}, \lfloor S/2 \rfloor)$, and using m of them is feasible, so $h_{18} = \sqrt{2}m + (S - 2m)$. Since a richer move set can only make paths cheaper, the true costs satisfy $C_6 \geq C_{18} \geq C_{26}$ and the heuristics satisfy $h_6 \geq h_{18} \geq h_{26}$ with equality to the corresponding true cost on an empty lattice. Hence h_{26} is admissible on all three lattices, h_{18} on the 6- and 18-connected ones but not on the 26-connected one, and h_6 only on the 6-connected one — the three-dimensional version of the Manhattan trap of Table 4.1. The Euclidean distance is admissible on all three and is dominated by all of them.

Exercise 4.7. Take the 4×4 grid whose only obstacles are $(0, 1)$ and $(2, 1)$, with $s = (0, 0)$ and $\gamma = (3, 3)$, 8-connected with the corner-cutting rule of Definition 2.5. A^* with the Manhattan heuristic returns $(0, 0), (1, 0), (2, 0), (3, 0), (3, 1), (3, 2), (3, 3)$, six straight moves of cost 6, while the optimum is $4 + \sqrt{2} \approx 5.41$; `dijkstra_grid(grid, s, 8)` and A^* with the octile heuristic both return 5.41. The proof of Theorem 4.9 fails at the inequality $f(n_i) = g^*(n_i) + h(n_i) \leq g^*(n_i) + h^*(n_i) \leq C^*$: with $h > h^*$ the node n_i of the optimal path can have $f(n_i) > C^*$, so nothing forces the algorithm to pop it before the suboptimal goal entry, and the contradiction disappears.

Exercise 4.8. Checkpoints rather than a solution. (a) If your expansion count is 22 or 23 instead of 21, you are probably testing the goal at generation time or breaking ties toward the smaller g (24 expansions). If Python raises `TypeError: '<' not supported between instances of ...`, you forgot the counter in the priority tuple. (b) The frontier picture is easiest as a grid of colored squares redrawn after every pop; keep the closed cells gray, the open cells blue, and the current node highlighted, exactly as in Figure 4.3. (c) The three traps are the goal test (Definition 4.17), the edge constraint (it is indexed by the departure time t , not the arrival time $t + 1$) and the horizon: without it your search never terminates on an unsolvable instance. (d) The four assertions of the chapter's self-test are the ones to copy: equal cost to Dijkstra, zero re-openings with a consistent heuristic, a non-decreasing sequence of f values, and `space_time_astar` cost equal to `static_distances` when there are no constraints. Compare your numbers with `ch04_astar.py` only after your own version runs.

Exercise 4.9. (a) With both $\langle a, (1, 1), 1 \rangle$ and $\langle a, (1, 1), 2 \rangle$ the middle cell is unusable at $t = 1$ and $t = 2$, so the agent waits twice: the path is $(0, 1), (0, 1), (0, 1), (1, 1), (2, 1)$, cost 4, two waits, and `space_time_astar` expands 5 states. A detour through $(1, 0)$ or $(1, 2)$ also costs 4 but arrives no earlier, and the tie-breaking rule prefers the state with the larger g , hence the wait. (b) The single edge constraint $\langle a, (1, 1), (2, 1), 1 \rangle$ forbids the last move at the time the unconstrained plan makes it, so the agent moves to $(1, 1)$ at $t = 1$, waits there and steps to the goal at $t = 3$: the path is $(0, 1), (1, 1), (1, 1), (2, 1)$, cost 3, 4 expansions. With the time index 2 instead of 1 the constraint forbids a move the agent never makes at that time, and the unconstrained path $(0, 1), (1, 1), (2, 1)$ of cost 2 stays optimal — the off-by-one that hides in every hand-written constraint. (c) The expansion counts are 5, 4 and 3 against the 4 of Table 4.3. (d) With `max_time = 3` instance (a) is reported unsolvable, because its solution arrives at $t = 4$; instance (b) still returns its 3-step path. This does not contradict Proposition 4.18, which guarantees completeness only for $T_{\max} = H + 1 + D$: here $H = 2$ and $D = 3$, so the sufficient horizon is 6, and 3 is a hand-supplied truncation.

Exercise 4.10. (a) Example 4.20 itself: $H = 1$, so a horizon of $H + 1 = 2$ makes the search fail, while the optimal constrained path arrives at $t = 3$ (this is the assertion `max_time=2` → failure, `max_time=3` → cost 3 in the self-test). (b) The same instance in the other direction: $T_{\max} = 1 + 1 + 3 = 5$ while the arrival time is 3. Any instance whose constraints are far from the route makes the gap arbitrarily large, because D is the eccentricity of the goal and does not look at the start. (c) For agent 2 the table contains agent 1's five vertex constraints, its four reversed edges and the parked cell $(4, 0)$ from $t = 4$, so $H = 4$; treating $(4, 0)$ as a wall, the largest distance to $(0, 0)$ is $D = 3$ (from the bay $(2, 1)$), giving $T_{\max} = 8$, while the search arrives at $t = 5$. (d) Take the ring map: the 4×4 grid whose four interior cells are blocked, so the 12 border cells form a cycle. Let $\gamma = (0, 0)$, let the agent start at $(2, 0)$ and let another agent be parked on $(1, 0)$ from $t = 0$, so $H = 0$. In the raw grid the eccentricity of γ is 6, giving the naive horizon $0 + 1 + 6 = 7$; but the parked cell cuts the short side of the ring and the only route left is the long way round, ten moves, so the true arrival time is 10. With `max_time = 7` the search returns failure, and a prioritized planner (Chapter 8) would report an unsolvable instance that in fact has a solution. Computing D in the grid *with* the parked cells treated as walls, as `default_horizon()` does, gives $D = 10$ and $T_{\max} = 11$, and the path is found.

Exercise 5.1. (a) $rhs(s)$ rises to the new value (unless a second predecessor offers the old total, in which case nothing changes) while $g(s)$ keeps the old, smaller value: s is underconsistent with key $[g(s) + h(s); g(s)]$, built from the old value. (b) $rhs(s)$ drops below $g(s)$: overconsistent, key $[rhs(s) + h(s); rhs(s)]$, built from the new value. (c) $rhs(s)$ is a minimum over the edges *into* s , so it does not change and s stays consistent; it is the head of the changed edge that `UpdateVertex` is called on and that may become underconsistent. (d) The minimum in Equation (5.1) is attained by the second predecessor with the same total, so $rhs(s)$ and hence the state of s do not change; `UpdateVertex` still runs on s (it is a successor of the popped vertex) but removes and re-inserts nothing.

Exercise 5.2. Keys are only ever compared inside `ComputeShortestPath`, and that function is not called while nothing changes: lines 35–40 of `Main` run only when the scan reports a changed edge, so during the two moves the queue is never read. The stored keys do become wrong as the robot walks (each step lowers $h((x, 3), \cdot)$ for the cells ahead), but a wrong key that nobody looks at costs nothing, which is exactly what k_m exploits: instead of repairing the queue at every step, D^* Lite repairs it once, at the moment it is next used. That is also why the increment is a single lump. The variable s_{last} holds the position at which the stored keys were computed — here the original start $(0, 3)$, because k_m was last updated there — so line 35 adds $h(s_{\text{last}}, s_{\text{start}}) = h((0, 3), (2, 3)) = 2$ in one step. Adding $1 + 1$ would require

executing the line after each move, which needs s_{last} to be reset each time; by the triangle inequality the two agree here, but in general the lump sum $h(s_{\text{last}}, s_{\text{start}})$ is the smaller (and therefore safer, see [Exercise 5.5](#)) of the two, since a detour makes the sum of the per-step increments exceed the direct estimate.

Exercise 5.3. Blocking $(1, 0)$ changes the eight edges around it, so `UpdateVertex` runs on $(1, 0)$, $(0, 0)$, $(2, 0)$ and $(1, 1)$. Only $(1, 0)$ changes: its rhs was 2 through $(1, 1)$ and becomes ∞ , which equals its g , so it simply leaves the queue. $(0, 0)$ keeps $rhs = 1$ through S , $(2, 0)$ keeps $rhs = 3$ through $(2, 1)$, and $(1, 1)$ keeps $rhs = 1$; none of them ever relied on $(1, 0)$, whose g was still ∞ . The queue now holds nine cells with keys $[6; 1]$, $[6; 1]$, $[6; 2]$, $[6; 3]$, $[6; 3]$, $[6; 4]$, $[6; 4]$, $[6; 5]$, $[6; 5]$, the goal is consistent with $g(T) = 4$, and the smallest key $[6; 1]$ is not smaller than $k(T) = [4; 4]$: `ComputeShortestPath` returns after zero expansions and the path along the middle row stands. The difference to [Table 5.2](#) is that $(3, 1)$ carried a finite g -value on which the goal depended, whereas $(1, 0)$ was never expanded, so no stored value was stale. Unblocking $(3, 1)$ in the state of the right panel calls `UpdateVertex` on $(3, 1)$ and its four neighbors; $rhs(3, 1)$ falls from ∞ to 3 through $(2, 1)$, so $(3, 1)$ is *overconsistent* with key $[4; 3]$, while the neighbors are unaffected because $g(3, 1)$ is still ∞ . The repair pops $(3, 1)$ (key $[4; 3]$, $g \leftarrow 3$), which lowers $rhs(T)$ from 6 to 4 and queues T with key $[4; 4]$, then pops T ($g \leftarrow 4$); two expansions, and the path is the middle row again. A cost decrease can only lower rhs -values, so only overconsistent vertices appear, and the queued side cells with keys $[6; \cdot]$ are never touched.

Exercise 5.4. Take the vertices S, u, w, T with edges $S \rightarrow u$ (cost 1), $u \rightarrow w$ (1), $w \rightarrow T$ (1) and $u \rightarrow T$ (2), start S , goal T , and the exact heuristic $h(S) = 3$, $h(u) = 2$, $h(w) = 1$, $h(T) = 0$, which stays consistent when costs increase. Both routes to T cost 3, so every vertex has $k_1 = 3$ in the first search. With k_1 only and ties broken against us, T is expanded before w , and w stays in the queue with $g(w) = \infty$ and $rhs(w) = 2$ derived from $g(u) = 1$. Now raise $c(S, u)$ to 2 and $c(u, T)$ to 10 in one batch. `UpdateVertex` gives $rhs(u) = 2$ (u underconsistent, $k_1 = 1 + 2 = 3$) and $rhs(T) = \min(1 + 10, \infty) = 11$ (T underconsistent, $k_1 = 3$); w keeps $k_1 = 2 + 1 = 3$. Pop T (underconsistent, $g(T) \leftarrow \infty$, re-queued with $rhs = 11$), then w *first* among the ties: it is overconsistent and accepts the stale value $g(w) = 2$, which makes $rhs(T) = 3$ and $k_1(T) = 3$. Pop u : $g(u) \leftarrow \infty$, so $rhs(w) = \infty$ and w is underconsistent with $k_1 = 3$, while u re-enters with $rhs(u) = 2$, $k_1 = 4$. Pop w a second time ($g(w) \leftarrow \infty$; T and w both become consistent at ∞ and leave the queue), pop u ($g(u) \leftarrow 2$, $rhs(w) = 3$, $rhs(T) = 12$), pop w a *third* time ($g(w) \leftarrow 3$, $rhs(T) = 4$) and finally T with $g(T) = 4$: seven expansions and w three times. If instead the tie after the second step is broken toward T , the loop even stops with the wrong value $g(T) = 3$. With the two-component key the first search expands w ($[3; 2]$) before T ($[3; 3]$), and after the change the pops are u $[3; 1]$, w $[3; 2]$, T $[3; 3]$ (all underconsistent), then u $[4; 2]$, w $[4; 3]$, T $[4; 4]$: six expansions, each vertex twice, and the correct cost 4.

Exercise 5.5. Let s be in U with the key stored when the robot stood at s_{last} and the modifier had the value k_m : $k_1^{\text{old}} = m + h(s_{\text{last}}, s) + k_m$ with $m = \min(g(s), rhs(s))$, and $k_2^{\text{old}} = m$. Neither $g(s)$ nor $rhs(s)$ changed since, because any change goes through `UpdateVertex`, which re-inserts s with a fresh key. After line 35 the current key is $k_1 = m + h(s_{\text{start}}, s) + k_m + h(s_{\text{last}}, s_{\text{start}})$, and the triangle inequality $h(s_{\text{last}}, s) \leq h(s_{\text{last}}, s_{\text{start}}) + h(s_{\text{start}}, s)$ gives $k_1 \geq k_1^{\text{old}}$ with $k_2 = k_2^{\text{old}}$, so the stored key is a lower bound. If the batch is the second one since the key was stored, apply the argument twice: the sum of the two legs is at least the direct distance, again by the triangle inequality. For the counterexample take [Table 5.4](#): u was stored with $[10; 4]$; the robot moves two cells toward u and, without the modifier, the current key is $[8; 4]$, so the stored key is a strict upper bound. A vertex v inserted now with $[9; 6]$ is popped before u although its current key is larger, which breaks invariant (iii) of the proof sketch of [Theorem 5.8](#), that vertices are expanded in nondecreasing order of their current keys; v may then be expanded with

an rhs -value that still depends on u , and the loop may stop with u buried under an inflated key while $g(s_{\text{start}})$ is not the true distance. Finally, k_m must be accumulated: keys stored at different times were computed with different values of k_m and different robot positions, and each of them needs the offset of the legs walked *since its insertion*. The sum of the legs is at least $h(s_{\text{last}}, s_{\text{start}})$ for every one of them, whereas the direct distance $h(s_{\text{orig}}, s_{\text{start}})$ from the original start can shrink when the robot turns back (it is 0 if the robot returns to s_{orig}), and then keys computed later would be smaller than keys stored earlier, which is the upper-bound situation above.

Exercise 5.6. Hint and sketch. In the reversed graph G^R one has $\text{Pred}_R(s) = \text{Succ}(s)$ and $c^R(s', s) = c(s, s')$, so LPA^{*}'s lookahead Equation (5.1) on G^R with the roles of start and goal swapped becomes $rhs(s) = \min_{s' \in \text{Succ}(s)} (c(s, s') + g(s'))$, which is line 10 of Algorithm 5.2, and the seed $rhs(s_{\text{goal}}) = 0$ is line 8. The query vertex of the reversed search is s_{start} , so the heuristic must be measured from it, $h(s_{\text{start}}, \cdot)$, and the loop condition becomes $\text{TopKey}() < k(s_{\text{start}})$ or $rhs(s_{\text{start}}) \neq g(s_{\text{start}})$ (line 14); an expansion updates $\text{Succ}_R(u) = \text{Pred}(u)$ (lines 20–24), and the path is read forward from s_{start} over the successor minimizing $c + g$. That is exactly Algorithm 5.2 with $k_m \equiv 0$ and with lines 15–18 deleted. Only four things are there because the start moves: line 31 (the robot step, which reassigns s_{start}), line 35 together with $s_{\text{last}} \leftarrow s_{\text{start}}$ (the key modifier), and the k_{old} test of lines 15–18, which repairs stored keys that the moves have turned into strict lower bounds. Everything else is LPA^{*} read backward.

Exercise 5.7. Expected outcome. (a) The two path costs must agree at *every* tick of every run; a single disagreement means a bug, and the usual causes are calling `UpdateVertex` on the head instead of the tail of a changed edge, forgetting $\text{Pred}(u) \cup \{u\}$ in the underconsistent case, or updating k_m once per changed cell instead of once per batch. (b) Most events change a cell that is not on the current path and not in the expanded region, and D^{*} Lite should then cost 0 expansions, while A^{*} re-expands its whole search region; over a run of a few dozen events expect D^{*} Lite to expand an order of magnitude fewer vertices in total, as in Figure 5.5. The first search is the exception: it is several times more expensive than A^{*} (a factor of about 4.6 in Section 5.8.1), because the second key component expands the whole f -band, so plot event 0 separately or the axis will hide everything else. Wall-clock time need not follow the expansion count: the per-expansion cost of D^{*} Lite is several times that of A^{*}, so on small grids repeated A^{*} can stay ahead in time while losing badly in expansions.

Exercise 5.8. Expected outcome. For very small ϕ (a handful of changed cells) D^{*} Lite wins by orders of magnitude in *expansions*: most changes touch cells whose goal distance does not change at all, and the repair costs almost nothing, while A^{*} re-expands its whole search region every time. The crossover in *time* comes much earlier than the crossover in expansions, because one D^{*} Lite expansion costs several times an A^{*} expansion — about 28 μs against 4 μs in Section 5.8.1 — so D^{*} Lite can already be losing in seconds while still winning by a factor of ten in expansions. Around a ϕ of a few per cent on a 100×100 grid the changes are no longer local: the repair touches a constant fraction of the vertices, Theorem 5.9 then permits up to $2|V|$ expansions plus a `UpdateVertex` on every neighbor of each of them, and a fresh A^{*} — which expands every vertex at most once and relaxes each edge in $\mathcal{O}(1)$ — is faster. At $\phi = 0.5$ nothing of the old search survives and D^{*} Lite is strictly worse. The rule for the replanning layer of Chapter 24 is therefore: count the changed cells reported per replanning cycle, keep a running estimate of the two costs, and fall back to A^{*} from scratch above the measured threshold — and always after a full map replacement, a goal change, or a re-localization that shifts the whole grid, since none of the stored values is then worth repairing. Exact numbers are implementation- and machine-dependent (a compiled indexed heap moves the crossover to the right, an interpreted lazy queue to the left); report your own crossover, in expansions and in time, and say which grid and which language produced it.

Exercise 5.9. Hint and sketch. (a) For each tick $t \leq H$ rasterize the disk of radius $r_t = r_0 + \sigma t$ around $\mathbf{p}_t$, inflated by the drone's own radius, and take $B = \bigcup_t B_t$; raise the cost of every edge into a cell of B to ∞ and call `UpdateVertex` on each such cell and its neighbors, as in [Section 5.10](#). When the next prediction arrives, do *not* rebuild B : form the new set B' , and call `UpdateVertex` only on the symmetric difference $B \triangle B'$ (newly blocked cells, and cells released because the intruder has passed or the prediction shifted) and their neighbors. Cells in $B \cap B'$ cost nothing, which is the whole point of an incremental search; keep B between calls so the difference can be formed in $O(|B| + |B'|)$. (b) Blocking the whole tube for the whole horizon is conservative in space *and* in time: a cell that the intruder occupies for two ticks is forbidden for all H , so a corridor that would be free by the time the drone reaches it is closed, and the planner either takes a long detour or reports *no path* although a safe schedule exists. A space-time search ([Chapter 4](#)) over states (cell, t) blocks (cell, t) only for the ticks in which B_t covers the cell; edge changes then touch a single time layer, so the repair is even more local, at the price of a state space H times larger and of a plan that must be re-timed rather than just re-routed. (c) If the goal moves, the g -values, which are distances *to the goal*, change with every goal step, and the property that made the backward search cheap is gone; k_m cannot help, because the goal enters the g -values and not the heuristic. One must either re-initialize or use Moving Target D* Lite [\[163\]](#), which salvages the still-valid part of the search tree and discards the rest.

Exercise 6.1. (a) The five cells generated in step 2, with their keys for $\varepsilon = 3$ and, in brackets, for $\varepsilon = 1$: (2, 2), $g = 2.41$, $h = 5.00$, key 17.41 [7.41]; (2, 3), 2.00, 5.41, 18.24 [7.41]; (2, 4), 2.41, 5.83, 19.90 [8.24]; (1, 2), 2.00, 6.00, 20.00 [8.00]; (1, 4), 2.00, 6.83, 22.49 [8.83]. The diagonal step to (2, 2) costs 0.41 more in g than the straight step to (2, 3), but it also reduces h by 0.41 more; with $\varepsilon = 3$ that saving counts three times, $3 \cdot 0.41 = 1.24 > 0.41$, so the diagonal key is 0.83 smaller and the diagonal neighbors go first. With $\varepsilon = 1$ the two are tied at 7.41 and only the tie-break decides. (b) Keys $g + 2h$ at the start of iteration 2: (3, 3) (from INCONS) $3.00 + 8.83 = 11.83$; γ $12.24 + 0 = 12.24$; (1, 2) 14.00; (2, 4) 14.07; (2, 1) 14.24; (3, 0) 14.49; (4, 5) 15.31; (1, 4) 15.66; (1, 1) 16.66. The corrected state (3, 3) is smallest and the goal second, which is why the postponed re-expansion happens at once and the iteration is over after four expansions. (c) $g(\gamma) = 12.24 = 8 + 3\sqrt{2}$ is the value inherited from iteration 1 along s , (1, 3), (2, 2), (3, 2), (4, 3), (5, 4), (5, 5), (6, 5), (7, 5), (7, 4), (7, 3), γ ; nothing in iteration 2 relaxes γ , so it stays. The pointer path is read backward from γ and now enters (4, 3) through the corrected (3, 3): s , (1, 3), (2, 3), (3, 3), (4, 3), (5, 4), (5, 5), (6, 5), (7, 5), (7, 4), (7, 3), γ , ten straight moves and one diagonal, $10 + \sqrt{2} = 11.41$. The two differ by exactly the correction $4.83 - 4.00 = 0.83$ that (3, 3) passed to (4, 3). To push $g(\gamma)$ down to 11.41 inside iteration 2 the search would have to re-expand the whole tail (5, 5), (6, 5), (7, 5), (7, 4), (7, 3) and relax γ once more — five expansions whose keys, starting at $6.41 + 2 \cdot 3.83 = 14.07$, all exceed the goal's key 12.24, so the termination test fires first. ARA* publishes the pointer path instead and gets the same cost for nothing.

Exercise 6.2. (a) With the octile distance to $\gamma = (7, 2)$: (1, 1): $3.83 + 6.41 = 10.24$; (1, 2): $2.00 + 6.00 = 8.00$; (1, 4): $2.00 + 6.83 = 8.83$; (2, 1): $3.41 + 5.41 = 8.83$; (2, 4): $2.41 + 5.83 = 8.24$; (3, 0): $4.83 + 4.83 = 9.66$; (4, 5): $6.83 + 4.24 = 11.07$; (7, 2): $12.24 + 0 = 12.24$; and (3, 3) in INCONS: $3.00 + 4.41 = 7.41$. So $L = 7.41$ and $\varepsilon' = \min(3, 12.24/7.41) = 1.65$. (b) Over OPEN alone the minimum is 8.00 from (1, 2), giving $\varepsilon' = 1.53$. On this instance the number happens to be a valid bound, since the true ratio is 1.20, but nothing guarantees it: [Exercise 6.4](#) shows a graph on which the same mistake certifies a suboptimal path as optimal. (c) Let n have a finite g and let P be its pointer path from s , of cost at most $g(n)$ by [Proposition 6.7](#). Applying $h(u) \leq c(u, u') + h(u')$ along the edges of P gives $h(s) \leq \text{cost}(P) + h(n) \leq g(n) + h(n)$. Hence $h(s) \leq L$, and because $h(s) \leq C^*$ by admissibility, $g(\gamma) \leq (g(\gamma)/h(s)) C^*$ is a valid certificate;

but $g(\gamma)/L \leq g(\gamma)/h(s)$, so the certificate of Equation (6.3) is never worse. In iteration 1 the two coincide, $h(s) = 7.41 = L$, because the straight route $s \rightarrow (1, 3) \rightarrow (2, 3) \rightarrow (3, 3)$ followed by the octile estimate from $(3, 3)$ costs exactly the octile estimate from s : no detour has yet been forced on the cheapest inconsistent state. As soon as an inconsistent state lies behind a wall, L exceeds $h(s)$ and the list-based certificate wins.

Exercise 6.3. (a) Sketch. Show that each operation preserves the three claims: generation of a new state (pushes it into OPEN, $v = \infty > g$), relaxation of a non-closed state (stays in or enters OPEN, still inconsistent), relaxation of a closed state (enters INCONS; it is closed, so it was expanded in this iteration), expansion (removes the state from OPEN, sets $v = g$, so it becomes consistent and is in neither list), and the merge on line 8 (moves states between the lists without changing v or g). If CLOSED were not emptied, a state expanded in an earlier iteration and improved in this one would be sent to INCONS although it was not expanded in this iteration: the last sentence of (iii) fails, and with it Case 3 of the proof of Theorem 6.8, which needs that a state in INCONS was selected in the *current* call, where the induction hypothesis applies. Its improvement would also not be propagated until the next iteration, for no reason. (b) Let $u \in \text{INCONS}$ be the first inconsistent state on the optimal path P . All states before u on P are consistent, so Lemma 6.6(ii) with $v = g$ telescopes to $g(u) \leq g^*(u)$, and (i) gives equality. Membership of u in $\text{OPEN} \cup \text{INCONS}$ is (iii); that u is in INCONS rather than OPEN changes nothing, because Equation (6.2) ranges over both lists. Admissibility of h then gives $L \leq g(u) + h(u) \leq g^*(u) + h^*(u) = C^*$. (c) With $\varepsilon = 1$: expand s (key 0), which gives a the key $1 + 3 = 4$ and b the key $3 + 0 = 3$; expand b , which gives γ the key 5; expand a (key $4 < 5$), which lowers $g(b)$ from 3 to 2, but b is closed and goes to INCONS. Now OPEN holds only γ with key 5, the loop stops, and $g(\gamma) = 5 > C^* = 4$. The lists give $L = \min(5, 2 + 0) = 2$ and $\varepsilon' = \min(1, 5/2) = 1$: a false certificate (the pointer path $s \rightarrow a \rightarrow b \rightarrow \gamma$ happens to cost 4, but the value $g(\gamma)$ that the certificate is about is 5). In the proof, take $n = \gamma$ at the exit and the optimal path s, a, b, γ : the first violating state is γ itself, $m = b$ is in INCONS, and Case 3 says that b violated the claim when it was selected with $v(b) = 3 > g^*(b) = 2$. At that moment a was in OPEN, and Case 2 for that selection needs $h(a) \leq c(a, b) + h(b) = 1$; the actual value $h(a) = 3$ breaks the inequality, $f_1(a) = 4 > 3 = f_1(b)$, and no contradiction arises. Consistency is used exactly there.

Exercise 6.4. (a) Every edge satisfies $h(u) \leq c(u, u') + h(u')$: for instance $h(s) = 2 \leq 1 + h(b) = 2$, $h(a) = 2 \leq 1 + h(c) = 2$ and $h(e) = 2 \leq 2 + 0$. The optimal path is $s \rightarrow a \rightarrow c \rightarrow e \rightarrow \gamma$ with $C^* = 5$; the decoy $s \rightarrow d \rightarrow \gamma$ costs 5.5 and $s \rightarrow b \rightarrow c \rightarrow \gamma$ costs 8. (b) Iteration 1, $\varepsilon = 3$: expand s (key 6), generating a ($g = 1$, key 7), b (1, key 4) and d (2, key 8); expand b , generating c (3, key 6); expand c , generating γ (8, key 8) and e (4, key 10); expand a (key 7), which lowers $g(c)$ to 2 — c is closed and goes to INCONS. The goal's key 8 is now at most the smallest key in OPEN (d with 8), so the iteration stops after four expansions with $\text{INCONS} = \{c\}$. The pointer path is $\gamma \rightarrow c \rightarrow a \rightarrow s$, cost 7, while $g(\gamma) = 8$; $L = \min(c: 2 + 1, d: 2 + 2, e: 4 + 2, \gamma: 8) = 3$ and $\varepsilon' = \min(3, 8/3) = 2.67$. Iteration 2, $\varepsilon = 1$: OPEN holds c (key 3), d (4), e (6), γ (8). Expand c : γ drops to 7, e to 3 (key 5); expand d : γ drops to 5.5 with parent d ; expand e (key $5 < 5.5$): γ drops to 5 with parent e . The loop stops, the published path is $s \rightarrow a \rightarrow c \rightarrow e \rightarrow \gamma$ with cost 5, $L = 5$ and $\varepsilon' = 1$. (c) Without INCONS the second iteration starts from $\text{OPEN} = \{d, e, \gamma\}$ with keys 4, 6, 8: expand d , which lowers γ to 5.5, and the loop stops at once because $5.5 \leq 6$. The published path is $s \rightarrow d \rightarrow \gamma$ with cost $5.5 > C^*$, and the lists give $L = \min(e: 4 + 2, \gamma: 5.5) = 5.5$, hence $\varepsilon' = 1$: a suboptimal path certified as optimal. The cause is that e still carries the stale value $g(e) = 4$ computed from $v(c) = 3$; the correction $g(e) = 2$ was never propagated. (d) Re-opening c in iteration 1 would expand it again with $g = 2$, lower γ to 7 and e to 3 at once, and the bound of weighted A^* with re-opening (Chapter 4) still holds. The cost is that a state may be expanded many times in one iteration on inflated keys,

so the “at most $|V|$ expansions per iteration” guarantee of [Proposition 6.10\(i\)](#) is lost, and with it the predictability of the time to the next published path.

Exercise 6.5. (a) Decrease of the mean cost ratio per 100 expansions, iteration by iteration. ARA*: iterations 2 and 3 improve nothing at no cost; iteration 4 gains 0.0003 for a single expansion; then 0.0152 (iteration 5, 83 expansions), 0.0136 (435), 0.0021 (480) and 0.0001 (328). Restarts: 0.0056 (224 expansions), 0.0075 (244), 0.0069 (298), 0.0028 (346), 0.0022 (590), 0.0009 (875), 0.00003 (1 173). Improvement is cheapest in the middle for both, iterations 5 and 6 for ARA* and 3 and 4 for the restarts, and it becomes very expensive at the end, where the work buys the guarantee rather than a shorter path. (b) ARA*'s cumulative expansions are 215, 215, 215, 216, 298, 733, 1213, 1541: after 800 expansions iteration 6 ($\varepsilon = 1.25$) is the last complete one, so the published path costs $1.0105 C^*$ with the certificate 1.226, and iteration 7 is running. The restarts stand at 683 after the run with $\varepsilon = 2$ and are in the middle of the one with $\varepsilon = 1.75$, so they offer $1.0518 C^*$ with the guarantee 2. Plain A* needs 1 177 expansions and has nothing at all. (c) By [Proposition 6.10\(iv\)](#) an iteration expands nothing when the incumbent already satisfies the termination test for the new ε , which is the case here: $g(\gamma) \leq \varepsilon \cdot L$ holds for $\varepsilon = 2.5$ and 2 because it holds with the ratio 1.412. Neither $g(\gamma)$ nor L changes, so $\varepsilon' = \min(\varepsilon, g(\gamma)/L)$ keeps the value 1.412: it cannot rise to 2.5 or 2, since the minimum takes the ratio, and it cannot fall, since nothing was improved. (d) From $\varepsilon_1 = 3$ the first certificate is $\varepsilon'_1 = 1.65$, so $\varepsilon_2 = \max(1, 1.65 - 0.2) = 1.45$; that iteration costs 13 expansions, finds the optimal path (cost 10.24) and certifies it with $\varepsilon'_2 = 1.16$; then $\varepsilon_3 = \max(1, 1.16 - 0.2) = 1$, and two expansions raise L to 10.24 and give $\varepsilon' = 1$. The schedule is $3 \rightarrow 1.45 \rightarrow 1$ with $17 + 13 + 2 = 32$ expansions, the same total as 3, 2, 1, but the optimal path is already on the table after 30 of them, and one intermediate publication is saved.

Exercise 6.6. Expected outcome. (a) The three iterations must give 17, 4 and 11 expansions with published costs 12.24, 11.41 and 10.24 and certificates 1.65, 1.53 and 1.00; a different count in iteration 1 usually means a different tie-breaking rule (the reference breaks ties on the key toward the smaller h , then first in first out), a different count in iteration 2 means that INCONS or the re-keying is wrong, and a cost of 12.24 instead of 11.41 in iteration 2 means that the path is read from stale g -values instead of from the parent pointers. (b) The checks are those of the self-test of `ch06_arastar.py`; the one that fails most often in fresh implementations is $L \leq C^*$, because L was computed over OPEN only. (c) Against expansions the curves look like [Figure 6.4](#): identical first points for every method, flat stretches where ARA* expands nothing, and a total between 1.2 and 1.5 times the expansions of A* for ARA*. Against wall-clock time the picture is the same up to a constant, except that the re-keying between iterations becomes visible as small steps for long schedules; the first-solution time of ARA* is that of weighted A* with ε_1 , and plain A* has no point at all before its single one. (d) With a budget the generator must be interrupted inside `improve_path()`, for instance by checking the clock every 200 expansions and raising a flag that makes the while loop exit; the pointer path at that moment is valid ([Proposition 6.7](#)) and no worse than $g(\gamma)$, which has not increased since the last publication, so the last certificate still applies. Keep the best path published so far and return that if the interrupted iteration has not improved on it; the self-test reports no instance with a non-monotone published cost sequence, but the guard costs nothing.

Exercise 6.7. (a) $20\,000 \cdot 0.05 = 1\,000$ expansions per cycle. On the maps of [Table 6.4](#) ARA* has then completed iteration 6 ($\varepsilon = 1.25$, 733 expansions cumulative) and is inside iteration 7, so it delivers a path of $1.0105 C^*$ with the certificate 1.226; the restarts have completed the run with $\varepsilon = 1.75$ (980) and deliver $1.0312 C^*$ with the guarantee 1.75; plain A* needs 1 177 expansions and delivers nothing. With a 20 ms cycle, 400 expansions: ARA* has finished

iteration 5 and offers $1.0698C^*$ with the certificate 1.378, the restarts have only the $\varepsilon = 3$ run (215) and offer $1.0827C^*$, and A^* again has nothing. (b) At any moment inside an iteration the parent pointers from γ form a valid simple path of cost at most the current $g(\gamma)$ (Proposition 6.7), and $g(\gamma)$ never increases, so it is at most the value published at the end of the previous iteration. That value was certified by ε' , hence the interrupted path also costs at most $\varepsilon'C^*$. What the interrupt does not give is a *better* certificate: L computed in mid-iteration is still a valid lower bound by Theorem 6.8, but the new $g(\gamma)$ has not yet passed the termination test, so the honest answer is the old pair. (c) The iteration-2 path runs $s, (1, 3), (2, 3), (3, 3), (4, 3), (5, 4), (5, 5), (6, 5), (7, 5), (7, 4), (7, 3), \gamma$, so blocking $(4, 3)$ invalidates the pointer chain of every cell behind it: $(5, 3), (5, 4), (5, 5), (6, 5), (7, 5), (7, 4), (7, 3)$ and γ keep g -values that no longer correspond to any path. In the vocabulary of Chapter 5 they are underconsistent, $g < rhs$; ARA^* has no machinery for that case, because raising a g -value never happens in a search from a fixed start, so all it can do is start again from scratch (the g -values of the cells not behind $(4, 3)$ may be kept as an initialization, but the guarantee then has to be re-earned). Anytime D^* is the algorithm that handles it: it searches backward from the goal, so the drone's own motion only changes the key modifier, and it propagates the raised values first with uninflated keys before inflating again; if the increase is large it also raises ε , trading the guarantee for the speed needed to repair. (d) A workable rule: if the certificate is below the mission tolerance and the map has not changed, keep improving; if a change invalidates cells on the current path but the remaining budget is at least the cost of one iteration at the current ε , raise ε to the last certificate plus a margin and repair; restart only when the change is near the start or the pointer path is no longer valid at all. The quantities to read are the certificate ε' , the expansions of the last iteration as an estimate of the next one, and the number of invalidated cells on the path.

Exercise 7.1. (a) $\pi_1[1] = \pi_2[0] = B$: a following conflict only; allowed. (b) $\pi_1[1] = \pi_2[0]$ and $\pi_2[1] = \pi_1[0]$: a swapping conflict, that is, the degenerate case $m = 2$ of the rotation pattern (which is why cycle conflicts are defined for $m \geq 3$), and a pair of following conflicts; forbidden. (c) $\pi_1[1] = \pi_2[1] = B$: a vertex conflict at $t = 1$; forbidden. (d) Both agents traverse $A \rightarrow B$: an edge conflict, which by Proposition 7.5(a) contains the vertex conflicts at A ($t = 0$) and at B ($t = 1$); forbidden. (e) Agent 2 waits at B while agent 1 enters it: a following conflict that is also the vertex conflict $\langle a_1, a_2, B, 1 \rangle$ by Proposition 7.5(d); forbidden.

Exercise 7.2. π_1 arrives at its goal $(0, 2)$ at $t = 3$ and never leaves, so $\text{cost}(\pi_1) = 3$: the wait at $(0, 1)$ before the arrival is paid, the two waits at the goal are free. π_2 starts at its goal and $\text{cost}(\pi_2) = 0$. π_3 reaches $(1, 1)$ at $t = 1$, leaves it at $t = 2$ and returns at $t = 3$; the last arrival after which the agent never leaves is at $t = 3$, so $\text{cost}(\pi_3) = 3$ by Equation (7.1). Hence $\text{SoC} = 3 + 0 + 3 = 6$ and $\text{MS} = 3$. π_3 does not cost 1 because under stay-at-target the agent is only “done” once it stays at its goal forever, and an agent that leaves the goal was not done.

Exercise 7.4. Hints. (a) Two agents crossing an empty 3×3 grid through its center, one horizontally and one vertically, meet at the center at $t = 1$ and nowhere else. (b) Put the agents at the two ends of a corridor of *even* length and let them exchange ends: for four cells $(0, 0), \dots, (0, 3)$ the positions at $t = 1$ are $(0, 1)$ and $(0, 2)$, at $t = 2$ they are $(0, 2)$ and $(0, 1)$, so the agents swap along the middle edge without ever sharing a cell. In a corridor of odd length the two agents meet in the middle cell instead, which is a vertex conflict and not a swap. (c) A convoy: agent 1 from $(0, 0)$ to $(0, 3)$ and agent 2 from $(0, 1)$ to $(0, 4)$ on a single row; agent 2 always leaves the cell that agent 1 enters. (d) Agent 1 from $(0, 1)$ to $(0, 2)$ (cost 1) and agent 2 from $(0, 0)$ to $(0, 3)$: agent 2 reaches $(0, 2)$ at $t = 2$ while agent 1 is parked there. (e) Four agents on a 2×2 block, each with its goal at the next cell clockwise, all move at $t = 0$; `all_conflicts` returns the empty list because a rotation is allowed, and the plan is valid. All five cases appear in the self-test of `ch07_mapf.py`.

Exercise 7.7. At an integer time the two drones sit at distinct lattice vertices, and distinct vertices of a lattice with cell side ℓ are at least ℓ apart. During a step, each drone moves along a segment of length ℓ or waits, and the squared distance between two points moving at constant velocity is a convex quadratic in the fraction $\tau \in [0, 1]$ of the step, so the minimum is attained at an endpoint or at a single interior point. Enumerate the pairs of moves that a conflict-free plan allows: two waits, a wait and a move, two parallel moves, two moves on edges without a common vertex, and following moves along the same line or around a corner (moving toward the same vertex is a vertex conflict, and reversing the same edge is a swap). All cases except the last keep the distance at ℓ or more, because the segments are parallel or separated by a full cell. For the corner case take drone 1 moving from $(0, 0)$ to $(\ell, 0)$ and drone 2 leaving $(\ell, 0)$ for (ℓ, ℓ) at the same time; their positions are $(\tau\ell, 0)$ and $(\ell, \tau\ell)$, the squared distance is $\ell^2((1 - \tau)^2 + \tau^2)$, and its minimum at $\tau = \frac{1}{2}$ is $\ell^2/2$, so the distance is $\ell/\sqrt{2}$. To keep two drones of diameter d at least m apart at all times one therefore needs $\ell/\sqrt{2} \geq d + m$, that is $\ell \geq \sqrt{2}(d + m)$; forbidding following conflicts removes the corner case and $\ell \geq d + m$ suffices.

Exercise 7.3. (b) Write L for the common shortest-path length. By [Proposition 7.10](#) every solution has $\text{cost}(\pi_i) \geq \text{dist}(s_i, \gamma_i) = L$ for every i , hence $\text{SoC} \geq kL$ and $\text{MS} \geq L$. A solution Π^* with $\text{MS}(\Pi^*) = L$ has $\text{cost}(\pi_i) \leq L$ for every i and therefore $\text{cost}(\pi_i) = L$ for every i , so $\text{SoC}(\Pi^*) = kL$: both lower bounds are attained, and the optima are $\text{SoC}^* = kL$ and $\text{MS}^* = L$. Now for any solution Π , $\text{SoC}(\Pi) = kL$ forces $\text{cost}(\pi_i) = L$ for all i (each summand is at least L), which is exactly $\text{MS}(\Pi) = L$; and conversely $\text{MS}(\Pi) = L$ forces $\text{cost}(\pi_i) = L$ for all i and hence $\text{SoC}(\Pi) = kL$. So Π is optimal for one objective if and only if it is optimal for the other. The hypothesis matters: without a solution of makespan L the two optima need not be attained together, which is [Example 7.7](#).

Exercise 7.5. (b) Under disappear-at-target an agent occupies its goal at its arrival time and is gone from the next step on. In the naive plan of [Example 7.9](#), a_3 arrives at $(0, 1)$ at $t = 1$, so it is still there at $t = 1$ and absent from $t = 2$: the vertex conflict of a_1 and a_3 at $(0, 1)$ at $t = 1$ remains, the swap of a_1 and a_2 on the edge $(0, 1) - (0, 2)$ between $t = 1$ and $t = 2$ remains (both agents are still flying), and the vertex conflict of a_2 and a_3 at $(0, 1)$ at $t = 2$ disappears. Two of the three conflicts survive. In the implementation, `Position` must return “absent” instead of γ_i for $t > T_i$, and every conflict test must skip a pair in which one agent is absent; the horizon argument is unchanged, because after T nobody moves and nobody is present twice. [Equation \(7.1\)](#) becomes the *first* arrival time, $\text{cost}(\pi_i) = \min \{t' : \pi_i[t'] = \gamma_i\}$, since a path now ends the moment it reaches the goal and can never leave it again.

Exercise 7.6. (a) Number the cells $0, \dots, n - 1$ from left to right and let $p_1[t], p_2[t]$ be the positions of the two agents. Because vertex conflicts are forbidden, $p_1[t] \neq p_2[t]$ at every t , so the sign of $D[t] = p_2[t] - p_1[t]$ is defined at every t ; initially $D[0] = n - 1 > 0$. Each agent moves by at most one cell per step, so $|D[t + 1] - D[t]| \leq 2$. A change of order means $D[t] \geq 1$ and $D[t + 1] \leq -1$. If $D[t + 1] = 0$ the two agents share a cell, a vertex conflict. Otherwise $D[t] = 1$ and $D[t + 1] = -1$, which forces both agents to move toward each other by one cell, that is $p_1[t + 1] = p_2[t]$ and $p_2[t + 1] = p_1[t]$: a swapping conflict on the edge between the two adjacent cells. Both are forbidden, so $D[t] > 0$ for all t and the left-to-right order never changes. The exchange would need $D = -(n - 1) < 0$, so the instance has no solution. This is the smallest instance in which feasibility fails for a combinatorial rather than a connectivity reason, and it is why the pocket of part (b) is needed.

(b) Number the corridor $(0, 0), \dots, (0, 3)$ and put the pocket at $(1, 1)$. Let agent 1 step

aside into the pocket and come back out behind the other agent:

$$\begin{aligned}\pi_1 &= ((0, 0), (0, 1), (1, 1), (0, 1), (0, 2), (0, 3)), \\ \pi_2 &= ((0, 3), (0, 2), (0, 1), (0, 0)).\end{aligned}$$

The two agents never share a cell and never exchange an edge (between $t = 2$ and $t = 3$ agent 1 leaves the pocket for $(0, 1)$ while agent 2 leaves $(0, 1)$ for $(0, 0)$, which is a following conflict, not a swap), so the plan is a solution. Its costs are $\text{cost}(\pi_1) = 5$ and $\text{cost}(\pi_2) = 3$, hence $\text{SoC} = 8$ and $\text{MS} = 5$. Both are optimal: `optimal_makespan_bruteforce` and `optimal_soc_bruteforce` of `ch07_mapf.py` return 5 and 8 on this instance in the self-test.

(c) [Theorem 7.11](#) does not promise that an instance with two empty vertices is solvable; it promises that solvability can be *decided* in polynomial time. The corridor with $n = 4$ and $k = 2$ satisfies $k \leq |V| - 2$, the decision procedure applies, and its answer is “no solution”. Push-and-Rotate ([Chapter 11](#)) is complete for exactly this class, so it does not loop or time out: it reports that the instance is unsolvable. With the pocket of part (b) the graph is no longer a path, the order argument of part (a) breaks, and the same solver returns a plan.

Exercise 7.8. Checkpoints. Write your own first, then compare with `validate_plan`, `first_conflict` and `all_conflicts` ([Algorithm 7.1](#)), their hashed twins ([Algorithm 7.2](#)) and the parsers in `ch07_mapf.py`. A path is a list of cells indexed by time, and the one function `position(path, t)`, which returns the last cell beyond the arrival, encodes the parking rule of [Definition 7.2](#); the hashed detector needs, per time step, a dictionary from cell to agent and one from the ordered pair (u, v) to agent. Steps: write `position` and test it past the end of a path; scan $t = 0, \dots, H$ with the horizon of [Definition 7.3](#), emitting records that carry both agents, the cell or edge and the time; rewrite with the two dictionaries and assert that both detectors agree on thousands of random plans; add the parsers, keeping in mind that a `.scen` line stores x before y . Expected numbers: on [Example 7.9](#) the naive plan has exactly three conflicts and the conflict-free plan has $\text{SoC} = 12$, $\text{MS} = 5$. On random 32×32 maps with 20% obstacles the naive plan carries about 1 conflict at $k = 10$, 50 at $k = 40$ and 260 at $k = 100$ — quadratic, the shape of [Figure 7.1](#) — and the pairwise detector needs 0.8 and 120 ms against 0.2 and 3.4 ms for the hashed one. Classic mistakes: comparing positions only, so swaps vanish; indexing a move by its arrival instead of its departure; ending the scan at the shorter path. Done when the checker finds every planted conflict of [Exercise 7.4](#) and nothing else — half of the Week 3 milestone ([Appendix A](#)). Keep it: it is the CBS detector of Week 4 and the capstone’s re-check.

Exercise 7.9. Checkpoints. This is a throwaway [Chapter 8](#): one space-time A^* per agent ([Chapter 4](#)) against the paths already planned, no reservation table yet. Reuse your search and your detector from [Exercise 7.8](#); `prioritized_planning` in `ch08_prioritized.py` is the reference to read afterward. Steps: after planning agent i , give every later agent a vertex constraint $\langle v, t \rangle$ per occupied cell, a constraint on the *reversed* edge $\langle v, u, t \rangle$ per move $u \rightarrow v$ (this, and only this, forbids swaps) and a permanent block of its goal from its arrival on; give the low level a horizon, or it loops forever on an agent that cannot get through; validate every plan before you believe any cost. Expected numbers: on [Example 7.9](#) the index order returns $\text{SoC} = 17$ against the optimum 12, only the order (a_2, a_1, a_3) reaches 12, and four of the six orders fail outright — an early sight of [Proposition 8.6](#) and [Proposition 8.7](#). On the corridor of [Exercise 7.6](#) with $n = 4$ and the pocket at $(1, 1)$, planning the left agent first fails and planning the right agent first succeeds with costs 5 and 3, $\text{SoC} = 8$, which is optimal. Classic mistakes: forgetting the parked goal, so a later agent plans through an arrived one; forbidding the same edge instead of the reversed one; raising an exception on failure instead of returning a value, which hides how often the planner gives up. Done when you own one instance that one order solves and another does not, and validator and planner agree on every run.

Exercise 8.1. (a) Agent 2 takes the straight path $\pi_2[t] = (0, t)$: R_V receives $((0, t), t)$ for $t = 0, \dots, 6$, R_E receives $((0, t), (0, t+1))$ for $t = 0, \dots, 5$, and $(0, 6)$ is parked from $t = 6$. Agent 3 then finds nothing in its way and goes straight down, $(0, 3), (1, 3), (2, 3)$: R_V gains $((0, 3), 0), ((1, 3), 1), ((2, 3), 2)$, R_E gains $((0, 3), (1, 3)), 0$ and $((1, 3), (2, 3)), 1$, and $(2, 3)$ is parked from $t = 2$. (b) The goal $(2, 6)$ of agent 1 is never reserved, so $t_\gamma = -1$: any arrival time is acceptable. (c) From $(2, 0)$ agent 1 can reach $(2, 1)$ and $(2, 2)$, but $(2, 3)$ is parked from $t = 2$ and agent 1 cannot be there before $t = 3$, so the bottom corridor is cut. The other route climbs through $(1, 0)$ to $(0, 0)$ at $t = 2$ (free: agent 2 left it at $t = 0$) and follows agent 2 along row 0 two steps behind it, which following conflicts allow, but $(0, 6)$ is parked from $t = 6$ and agent 1 would arrive there at $t = 8$. Since $(1, 6)$ and the cells $(2, 4), (2, 5), (2, 6)$ are reachable only through $(0, 6)$ or $(2, 3)$, the goal is unreachable. The implementation collapses the time index beyond $t_{\text{static}} = H + 1 = 7$, the last timed entry being agent 2's arrival at $t = 6$; every reachable cell is therefore expanded at most eight times, the open list runs empty, and the function returns **None**.

Exercise 8.2. (a) Reserve every occupied cell for one step longer: for a path with arrival time T put both $(\pi[t], t)$ and $(\pi[t], t+1)$ into R_V for $0 \leq t \leq T$ (equivalently, test (v', t) as well as $(v', t+1)$ before entering v' ; the parked goal already covers all later times). A cell that a higher-priority agent leaves at t is then blocked at $t + 1$, which is exactly the one-cell gap. The swap test becomes redundant under this convention: to exchange cells with an agent that moves $u \rightarrow v$ during step t , the lower-priority agent would have to enter u at $t + 1$, and the trailing entry $(u, t+1)$ already forbids that. Keep the test all the same – it costs one lookup, and it keeps the two conventions independent, so that switching the gap off does not silently allow swaps. (b) Under disappear-at-target an agent vanishes when it arrives, so no goal is parked on and the goal test of line 4 loses its condition $t > t_\gamma$. Agent 3 then reserves only $((0, 3), 0), ((1, 3), 1), ((2, 3), 2)$ and the two moves between them, and it is gone from $t = 2$ on. Agent 1 crosses $(2, 3)$ at $t = 3$ and agent 2 crosses $(0, 3)$ at $t = 3$, one step after agent 3 has left, which is a following move and therefore allowed. No agent ever blocks another, so all six orders succeed with the independent shortest paths, costs 6, 6, 2 and $\text{SoC} = 14$: exactly the lower bound quoted under Table 8.3. The whole difficulty of Example 8.4 lives in the parked reservations, and this is why the convention must be stated before any claim about prioritized planning is made.

Exercise 8.4. (a) With $\pi_1[t] = (1, t)$ for $t \leq 4$ and $(1, 4)$ parked afterward, agent 2 starts at $(1, 4)$. At $t = 1$ it can be at $(1, 4)$ or $(1, 3)$. At $t = 2$ the cells $(1, 4)$ and $(1, 3)$ remain; $(1, 2)$ is agent 1's cell at that time. At $t = 3$ agent 1 is at $(1, 3)$, so agent 2 must be at $(1, 4)$, which it reaches by waiting or by moving back from $(1, 3)$ (not a swap, because agent 1 comes from $(1, 2)$). At $t = 4$ agent 1 arrives at $(1, 4)$: waiting is a vertex conflict and moving to $(1, 3)$ exchanges cells with agent 1, a swap. The set of legal states is empty and the search fails; the pocket $(0, 2)$ was never reachable, since it requires being at $(1, 2)$ at $t = 1$, two cells away from the start. The order $(2, 1)$ is the mirror image. (b) The second pocket $(0, 3)$ is adjacent to $(0, 2)$, so the two pockets form a two-cell bypass above $(1, 2)$ and $(1, 3)$. The order $(1, 2)$ now succeeds: agent 1 goes straight (cost 4) and agent 2 takes $(1, 4), (1, 3), (0, 3), (0, 2), (1, 2), (1, 1), (1, 0)$, climbing into the bypass at $t = 2$ while agent 1 is at $(1, 2)$, moving along it at $t = 3$ while agent 1 is at $(1, 3)$, and coming down at $t = 4$ as agent 1 parks at $(1, 4)$; cost 6, sum 10. The order $(2, 1)$ still fails: agent 2's straight path reaches $(1, 2)$ at $t = 2$ and $(1, 1)$ at $t = 3$, and agent 1, two cells from the nearest pocket, cannot get out of the corridor in time. (c) Yes. The lower bound is $4 + 4 = 8$, and two agents cannot pass each other inside a corridor, so one of them must leave it, which costs at least two extra steps (one up, one down); the plan of (b) spends exactly these two steps and nothing else, so 10 is optimal, as `optimal_soc` confirms.

Exercise 8.5. (a) First check in $\mathcal{O}(k)$ that the $2k$ endpoints are pairwise distinct; an instance

that fails this test is not well-formed, and [Theorem 8.9](#) says nothing about it. Then, for each agent i , remove the $2k - 2$ other endpoints from G and run one breadth-first search from s_i ; the instance is well-formed if and only if each search reaches γ_i . Each search costs $\mathcal{O}(|V| + |E|)$, hence $\mathcal{O}(k(|V| + |E|))$ in total, which dominates the distinctness test. This is the order in which `is_well_formed` of `code/ch08_prioritized.py` proceeds. (b) The step “waiting is unblocked”: it needs that no entry (s_i, t) exists in R_V , which holds only because every earlier agent had s_i in its static obstacle set. For the instance take a 3×5 open grid with agent 1 from $(0, 0)$ to $(0, 4)$ and agent 2 from $(0, 2)$ to $(1, 2)$. It is well-formed: agent 1 can avoid both endpoints of agent 2 through the bottom row, and agent 2 reaches its goal in one step. Without the rule, agent 1’s shortest path runs along the top row and reserves $(s_2, 2) = ((0, 2), 2)$; agent 2 must have left its start by then, which here it can do by moving to its goal at $t = 1$, but nothing in the algorithm guarantees such an escape in general. With the rule agent 1 treats $(0, 2)$ as an obstacle and takes a path of length 6 around it, through $(1, 1), (1, 2), (1, 3)$; agent 2 then has to step back to $(0, 2)$ while agent 1 passes its goal and arrives at $t = 4$, so the guaranteed plan costs 10 where the unguarded one cost 5. (c) On a 3×3 open grid let agent 1 go from $(1, 0)$ to $(1, 2)$ and agent 2 from $(0, 1)$ to $(1, 1)$. Both endpoint-free paths exist (agent 1 around the bottom row, agent 2 straight down), so the instance is well-formed, but agent 1’s shortest path passes through $\gamma_2 = (1, 1)$. In the order $(2, 1)$ agent 2 parks on $(1, 1)$ at $t = 1$ and [Algorithm 8.2](#) returns a detour of cost 4 for agent 1 through row 0 or row 2; the code takes $(1, 0), (0, 0), (0, 1), (0, 2), (1, 2)$, passing through s_2 after agent 2 has left it. In the order $(1, 2)$ agent 1 takes the straight path and visits $(1, 1)$ at $t = 1$, so $t_\gamma = 1$ for agent 2, which waits one step and arrives at $t = 2$; both orders succeed, as the theorem promises.

Exercise 8.6. (a) On this map the two heuristics agree: for every free cell the true distance to $\gamma_1 = (2, 6)$ equals the Manhattan distance. Row 2 gives 6, 5, 4, 3, 2, 1, 0, row 0 gives 8, 7, 6, 5, 4, 3, 2, and the three free cells of row 1 give $h^*((1, 0)) = 7$, $h^*((1, 3)) = 4$ and $h^*((1, 6)) = 1$. The walls of row 1 remove cells but never lengthen a route toward $(2, 6)$, and a wall that costs no detour is exactly what the Manhattan distance is allowed to miss. (b) The two searches are therefore identical: both expand 22 states for agent 1 and return the same path of cost 10. The true distance pays off only where walls do lengthen routes, as the 32 509 against 11 034 expansions on the cluttered 20×20 grids of [Section 8.7.1](#) show. Both begin in the bottom corridor because from $s_1 = (2, 0)$ the states $(2, 1)$ at $t = 1$ and $(2, 2)$ at $t = 2$ have $f = 6$, the smallest value anywhere in the search; only when $(2, 3)$ turns out to be parked on from $t = 2$ does the search fall back on $(1, 0)$ at $t = 1$, with $f = 8$, and climb to the top route. (c) The modified heuristic is not admissible. Take the open 3×3 grid with agent 1 from $(2, 1)$ to $(0, 1)$ and agent 2 from $(0, 0)$ to $(0, 2)$, planned in that order. Agent 1 parks on $(0, 1)$ from $t = 2$, but agent 2 crosses that cell at $t = 1$, one step before it is occupied, and arrives at cost 2. Delete $(0, 1)$ from the map and the modified heuristic reports 4 at the start: it overestimates by two, and A^* with it returns the detour through row 1. Every error is of this kind – a parked cell is an obstacle only from its parking time on, while the heuristic treats it as one from the beginning. The heuristic is safe when every parked cell is already parked on at the time of the state being evaluated, so that it blocks the whole remainder of the plan; it is then a distance in the graph to which the agent is really confined, and it is exact when in addition no timed vertex or move entry forces the agent to wait.

Exercise 8.7. (a) Let the agents be adjacent, agent 1 at $(1, 3)$ and agent 2 at $(1, 4)$, and let agent 1 be planned first in this round. The table starts empty, so agent 1 does not see agent 2 at all: it plans straight ahead, $(1, 3), (1, 4), (1, 5), \dots$, and reserves those cells. Agent 2 now finds every cell in front of it taken: stepping to $(1, 3)$ at $t = 1$ would swap with agent 1’s move $(1, 3) \rightarrow (1, 4)$, and waiting at $(1, 4)$ collides with agent 1 there at $t = 1$. Its cheapest unblocked

path is to retreat, $(1, 4), (1, 5), (1, 6), \dots$, and to turn around later – beyond the window, where agent 1 is invisible. Each agent executes one step: agent 1 advances to $(1, 4)$, agent 2 falls back to $(1, 5)$. Line 11 then makes agent 2 the first planner, the roles are exchanged, and agent 1 is pushed back to $(1, 3)$. The pair oscillates between the two configurations for ever.

(b) There are two different failures here, and the window decides which one you see. In every round the agent that plans first meets an empty table, so it never sees the other agent at all: it plans the straight path through it and reserves it. Only the second planner sees a conflict, and every cell in front of it is taken – each reserved vertex forbids the wait, each reserved move forbids the turn-around through the swap test – so its cheapest unblocked path is to retreat and to turn around later, beyond the window.

For $w \leq 5$ that retreat always exists, and the oscillation of part (a) never stops: `whca_star` reaches `max_rounds` and returns `None`.

For $w \geq 6$ the retreat runs out of corridor. Take $w = 6$ and follow the round in which agent 1 stands at $(1, 3)$, agent 2 at $(1, 4)$, and agent 1 plans first. Agent 1 reserves the straight path from $(1, 3)$ at $t = 0$ to $(1, 8)$ at $t = 5$ and parks $(1, 8)$ from $t = 5$. Agent 2 is now chased backward down the corridor – $(1, 5)$ at $t = 1$, $(1, 6)$ at $t = 2$, $(1, 7)$ at $t = 3$, $(1, 8)$ at $t = 4$ – and there it is cornered: waiting at $(1, 8)$ until $t = 5$ hits the parked goal, and stepping back to $(1, 7)$ exchanges cells with the reserved move $((1, 7), (1, 8))$ of step $t = 4$. `space_time_astar` returns `None`, agent 2 takes the stay-put branch of line 6, and agent 1 walks into it; `validate` reports `vertex conflict agents 0,1 at (1, 4) t=5`. For $w = 7, 8$ the same collision happens one cell further on, at $(1, 5)$ and $t = 5$, and for $w = 9, \dots, 12$ at $(1, 5)$ and $t = 7$. The boundary at $w = 6$ is the parked entry: `in_window` keeps an entry only if $t < w$, so for $w \leq 5$ the arrival at $t = 5$ is invisible, the cornered agent waits at the dead end and turns around beyond the window, and the rounds go on for ever.

Enlarging w helps in neither regime: a larger window lets the second planner see further and plan a longer retreat with a turn at its end, but only the first $\delta = 1$ step of that plan is ever executed, and that step is the retreat. The single pocket $(0, 1)$ sits at the far end of the corridor, behind agent 1's start, so no window that reaches it from the meeting point is ever used: the agents meet near $(1, 4)$ and never get back.

(c) Rotation is what causes the symmetry: it hands the right of way to the two agents alternately, so neither ever completes a passing maneuver. Freezing the order for several rounds is what helps. With the full horizon and the fixed order $(2, 1)$ the instance is solved: agent 2 goes straight and arrives at $t = 8$, and agent 1 first retreats into the pocket $(0, 1)$, lets agent 2 pass and then walks the whole corridor, arriving at $t = 15$ (`prioritized_planning` returns costs 15 and 8). Any of the following removes the deadlock: keep the priority order fixed until every agent has reached its goal; grow the window until each agent can see a pocket, and re-plan with the same order; or detect the repeated configuration and hand the two agents to a coupled solver (Chapter 9) for the corridor.

Exercise 8.8. Checkpoints. Four objects of `ch08_prioritized.py` carry the exercise: the class `ReservationTable`, the search `space_time_astar` (Cooperative A^* , Algorithm 8.2), the loop `prioritized_planning` (Algorithm 8.1) and `validate`. The table of Definition 8.1 is three dictionaries, not one: occupied cells keyed by (v, t) , occupied moves keyed by (u, v, t) , and a last-reserved time per cell that is ∞ for a parked agent. The low-level state is (v, t) with a wait action and the true-distance heuristic of Chapter 3. Steps: build the table and its three queries first and unit-test them on hand-written paths; add the goal-stay test of line 4, so the search may return only when the goal is free from t on; loop over the agents, plan, reserve, and report failure as a value; add well-formedness and restarts last. Expected numbers: on Example 8.4 four of six orders succeed, at SoC = 16 or 21; on the pocket corridor every order fails although the optimum is 11 (Proposition 8.6); on the bypass corridor every order returns 16 against 15 (Proposition 8.7). On the 20×20 grids of Figure 8.6 a random order succeeds on

about 97 % of instances at $k = 16$ and 87 % at $k = 32$, while longest-first and five restarts stay near 100 %. Classic mistakes: reserving vertices but not moves, so swaps slip through; letting an arrived agent free its goal; returning at the first pop of the goal; a Manhattan heuristic in a cluttered map, which costs about three times the expansions of the true-distance table. Done when your validator passes every plan, well-formed instances always solve, and you own one instance that one order fails and another solves — the Week 3 milestone of [Appendix A](#).

Exercise 9.2. Planned alone, both agents take the only shortest route through the gap: $\pi_1 = (0, 0), (1, 0), (1, 1), (1, 2), (2, 2)$ and $\pi_2 = (2, 0), (1, 0), (1, 1), (1, 2), (0, 2)$, each of cost 4, so the root has cost 8. The first conflict is the vertex conflict $\langle a_1, a_2, (1, 0), 1 \rangle$ (the two agents also meet at $(1, 1)$ at $t = 2$ and at $(1, 2)$ at $t = 3$, but CBS splits only the earliest one). Child N_1 adds $\langle a_1, (1, 0), 1 \rangle$: agent a_1 waits one step at $(0, 0)$ and then follows one cell behind a_2 , $\pi_1 = (0, 0), (0, 0), (1, 0), (1, 1), (1, 2), (2, 2)$, cost 5; the plan is conflict-free (following is not a conflict) with cost 9. Child N_2 adds $\langle a_2, (1, 0), 1 \rangle$ and is the mirror image, also of cost 9. The high level pops one of the two (both cost 9 and have no conflict), validates it and returns a plan of cost 9. The tree has three nodes and one split; the optimal sum of costs is 9, one more than the root, because the gap admits only one agent per time step. The joint-space search, in contrast, works with pairs of positions on the seven free cells of the grid: up to $7 \times 6 = 42$ pairs per time step, and it expands a few dozen of them before the two agents are through the gap. CBS settles the instance with three CT nodes and four low-level searches (two at the root, one in each child), which is the bottleneck case of [Section 9.6.1](#): a single shortest path per agent leaves the constraint tree nothing to branch on.

Exercise 9.6. Write $x_1(t), y_1(t)$ for the position of a_1 and $x_2(t), y_2(t)$ for a_2 . A shortest path from $(0, 2)$ to $(4, 3)$ has cost 5 and consists of four moves to the right and one move up in some order, so it never moves left or down; there are $\binom{5}{1} = 5$ such orders. Likewise a shortest path from $(1, 1)$ to $(3, 4)$ makes two moves right and three moves up, in $\binom{5}{2} = 10$ orders. Along both paths $x + y$ grows by exactly one per step, so at every time $t \leq 5$ both agents lie on the anti-diagonal $x + y = 2 + t$. On that line a cell is determined by its x -coordinate, so the two agents are in the same cell at time t if and only if $x_1(t) = x_2(t)$. At $t = 0$ we have $x_1 = 0 < 1 = x_2$; at $t = 5$ we have $x_1 = 4 > 3 = x_2$. Let t be the last time with $x_1(t) < x_2(t)$. Then $x_1(t+1) \geq x_2(t+1)$, and because x -coordinates grow by at most one per step, $x_1(t) + 1 \geq x_1(t+1) \geq x_2(t+1) \geq x_2(t) \geq x_1(t) + 1$; all these are equalities, so $x_1(t+1) = x_2(t+1)$ and the agents collide at time $t+1$. Every one of the 5×10 pairs of shortest paths therefore contains a vertex conflict, and the optimal sum of costs is at least $5 + 5 + 1 = 11$ (the self-test of `ch09_cbs.py` confirms that CBS finds 11).

Exercise 9.7. Keep the low level unchanged and let the high level order the constraint tree by $\max_i \text{cost}(N.\pi_i)$ instead of $\sum_i \text{cost}(N.\pi_i)$. The optimality argument of [Section 9.6](#) goes through word for word: the lemma that no split loses a solution does not mention the objective, and for any solution Π that respects the constraints of N we have $\text{cost}(\pi_i) \geq \text{cost}(N.\pi_i)$ for every agent, hence $\max_i \text{cost}(\pi_i) \geq \max_i \text{cost}(N.\pi_i)$, so the node's makespan still lower-bounds the makespan of every solution in its subtree. Best-first search on this bound is therefore makespan-optimal. Two practical differences: ties are far more common, because a constraint on an agent that is not the slowest one leaves the node's makespan unchanged, so the tie-breaking on the number of conflicts matters much more; and the high-level heuristics of [Section 9.7](#) (which count cost increases per agent) must be reformulated before they are admissible for the makespan.

Exercise 9.9. Checkpoints. The reference is `ch09_cbs.py: low_level` ([Algorithm 9.2](#)), `cbs` ([Algorithm 9.1](#)), `all_conflicts`, `validate`, and `print_trace` to compare your log against

Table 9.1. A constraint-tree node holds one immutable constraint set per agent, one path per agent and their sum of costs; the high level is a heap keyed on that sum with the conflict count as tie-break, and a child copies its parent’s set plus one entry. Steps: write the low level alone — state (v, t) , wait action, vertex and edge constraints, the goal-stay test of Equation (9.2), a horizon — and check it against your unconstrained space-time A^* of Chapter 4; reuse the Week-3 checker for detection with parked agents; add the high-level loop; certify tiny instances against a brute-force joint-space search. Expected numbers: on Example 9.4 the root costs 10, the tree generates seven nodes, expands four, calls the low level nine times and returns 12. On random 8×8 grids with 15 % obstacles (Figure 9.6) expect about 1.3 CT nodes and 0.4 ms at $k = 2$, 4 nodes and 1.4 ms at $k = 4$, and 55 nodes, 1 300 low-level expansions and 0.45 s at $k = 8$, where one instance in twelve already misses a five-second limit. Classic mistakes: an edge constraint stored without its direction or time; a low level that returns at the first pop of the goal; sorting the high level by makespan while reporting SoC; missing conflicts after the shorter path has ended. Done when your plans pass the Week-3 checker, your costs equal the reference everywhere, and a hand-drawn two-agent tree matches your log — the Week 4 milestone (Appendix A).

Exercise 9.10. Checkpoints. Build on Exercise 9.9; the reference is the cardinal branch of `_choose_conflict` and `_bypass` in `ch09_cbs.py`. The cheap classifier: scan a node’s conflicts in time order, generate *both* children of each, and rank the conflict by how many children cost more than the parent — two means cardinal (Definition 9.11), one semi-cardinal, zero non-cardinal; stop at the first cardinal one and keep the children you built, or you pay for them twice. The bypass: if a child has the parent’s cost and fewer conflicts, adopt its paths in the parent and do not split. The MDD classifier: two breadth-first searches in space-time per agent, forward from the start and backward from the goal, keep the states on some optimal path, and read cardinality off the two widths at the conflict’s time step, with no child generation at all. Expected numbers on the 8×8 benchmark of Figure 9.6: at $k = 8$ the mean falls from about 55 CT nodes and 0.45 s to 15 nodes and 0.045 s, with success rising to 1.0; at $k = 12$, from 622 to 92 nodes. Report the ratio: low-level expansions per CT node rise from roughly 23 to 170, which is what the node count was bought with. On Example 9.4 nothing changes — four expansions, seven nodes, no bypass — so do not tune on it. Classic mistakes: counting probe children as expansions; a bypass that changes a node’s cost; MDDs built to the unconstrained depth instead of the agent’s current optimal cost. Done when both classifiers agree on every conflict of a few hundred random nodes and your scaling plot shows the book’s crossover.

Exercise 10.1. (a) $f_{\min} = 10$ and the band is $f \leq 13$, so $\text{FOCAL} = \{(10, 3), (11, 1), (12, 0), (13, 4)\}$ and the node $(12, 0)$ is expanded. (b) The successor $(12, 2)$ enters the band, $(16, 0)$ does not; $\text{FOCAL} = \{(10, 3), (11, 1), (13, 4), (12, 2)\}$ and $(11, 1)$ is expanded. (c) $(15, 0)$ is in FOCAL exactly when $15 \leq 10w$, that is, $w \geq 1.5$. It is still not expanded before $(12, 0)$: both have $h_{\text{FOCAL}} = 0$ and the tie goes to the smaller f . (d) The band is $f \leq 1.3 \cdot 13 = 16.9$, so $\text{FOCAL} = \{(13, 4), (14, 2), (15, 0), (16, 0)\}$; the two nodes with $h_{\text{FOCAL}} = 0$ tie and $(15, 0)$, with the smaller f , is expanded. Note that $(16, 0)$ sat in OPEN with $h_{\text{FOCAL}} = 0$ the whole time and was ignored until the band reached it; this is the ruler doing its job.

Exercise 10.3. Low level: with $w = 1$ the band is $\{n \in \text{OPEN} : f(n) \leq f_{\min}\}$, that is, the states with $f = f_{\min}$. The goal state (γ_i, t) is chosen from the band, so $t = f(\gamma_i, t) = f_{\min}$, and Lemma 10.9 gives $f_{\min} \leq C_i^*(\mathcal{C}_i) \leq t = f_{\min}$: the path is optimal and the returned bound equals its cost. Hence every path in every CT node is optimal for its constraints and $\text{LB}_i(N) = \text{cost}(N.\pi_i)$, so $\text{LB}(N) = N.\text{cost}$ for every node (the maximum at line 18 of Algorithm 10.2 changes nothing, because a child’s constrained optimum is at least the parent’s). High level: the band is $\{N \in \text{OPEN} : N.\text{cost} \leq \text{LB}\}$ with $\text{LB} = \min_{N'} \text{LB}(N') = \min_{N'} N'.\text{cost}$; since

$N.\text{cost} \geq \text{LB}$ for every open node, the band consists of the nodes of minimum cost, and the one with the fewest conflicts is expanded. This is exactly CBS with OPEN ordered by cost and ties broken by h_c , which returns an optimal solution by the argument of [Chapter 9](#), or directly by [Theorem 10.11](#) with $w = 1$. The self-test of `ch10_ecbs.py` checks on random instances that `ecbs(inst, w=1.0)` returns the cost of `cbs(inst)`.

Exercise 10.4. (a) The waiting state has $f = 4$ and $f_{\min} = 3$, so it enters the band exactly when $4 \leq 3w$, that is, $w \geq 4/3$. For $w < 4/3$ the band contains only states with $f = 3$ and the conflicting straight path is returned; for $w \geq 4/3$ the waiting state is expanded first because its conflict count is 0, and the path of cost 4 is returned. (b) In Python, $3 * (4/3)$ evaluates to 4.0, so the test happens to succeed, but this is an accident of rounding: $4/3$ is stored slightly below $4/3$ and the product happens to round up. Store w as a fraction p/q and test $qf \leq pf_{\min}$ in integers, or compare against $wf_{\min} + 10^{-9}$. (c) Now B occupies $(1, 1)$ at $t = 1$ and $t = 2$, so both the straight path and the path with one wait have a conflict, and every conflict-free path costs 5: wait twice at $(1, 0)$, or go around through row 0 or row 2. With $w = 1.5$ the band is $f \leq 4.5$ and no conflict-free path fits: the search expands the wait first, finds that $(1, 1)$ at $t = 2$ conflicts too, and returns the straight path of cost 3 with one conflict and the bound 3 after four expansions. With $w = 2$ the band reaches $f \leq 6$ and the horizon $[2 \cdot 4] = 8$; the search returns the path that waits twice, $(1, 0), (1, 0), (1, 0), (1, 1), (1, 2), (1, 3)$, of cost 5 with no conflict, again with the bound 3. `focal_space_time_astar` confirms all four runs.

Exercise 10.5. Take N_7 of [Table 10.1](#) and [Figure 10.4\(a\)](#). It inherits $\langle a_3, (1, 1), 3 \rangle$ from N_2 and $\langle a_3, (2, 1), 2 \rangle$ from N_4 and adds $\langle a_1, (1, 1), 4 \rangle$; its paths cost $5 + 4 + 6 = 15$ and it is conflict-free. Under $w = 1$ every low-level path is optimal for its constraints, so $\text{LB}_i(N_7) = \text{cost}(N_7.\pi_i)$ and $\text{LB}(N_7) = 15 > 12 = C^*$: a node's own bound is a lower bound on the optimum of *that node's subtree*, not on C^* , because the node's constraints may exclude every optimal solution. Now suppose the high level tested a node against its own bound, $N.\text{cost} \leq w \text{LB}(N)$. For $w = 1.2$ the node N_7 passes, since $15 \leq 1.2 \cdot 15 = 18$, and it is conflict-free, so the rule may return it; its ratio is $15/12 = 1.25 > 1.2$ and the guarantee is broken. The global $\text{LB} = \min_{N \in \text{OPEN}} \text{LB}(N)$ is what makes [Theorem 10.11](#) work: by [Lemma 10.10](#) some open node is consistent with an optimal solution, and the proof of [Theorem 10.11](#) shows that its bound is at most C^* , so the minimum over OPEN never exceeds C^* while a single node's bound may.

Exercise 10.6. Fix an optimal solution Π^* and call a node consistent if Π^* respects all its constraints. The proof of [Lemma 10.10](#) goes through unchanged: the root is consistent, a consistent node has a child that is consistent, and that child's low-level call succeeds. Now let N be consistent. Each path $N.\pi_i$ was produced by a focal low level with factor w_L under a constraint set $\mathcal{C} \subseteq N.\mathcal{C}_i$ (the constraints of N itself or of the ancestor in which a_i was last replanned). By [Theorem 10.8](#) applied to that search, $\text{cost}(N.\pi_i) \leq w_L C_i^*(\mathcal{C})$, and since π_i^* respects $\mathcal{C}$, $C_i^*(\mathcal{C}) \leq \text{cost}(\pi_i^*)$. Summing, $N.\text{cost} \leq w_L \sum_i \text{cost}(\pi_i^*) = w_L C^*$. Hence at every moment $\min_{N' \in \text{OPEN}} N'.\text{cost} \leq w_L C^*$, and the solution node returned from the band satisfies $\text{cost} \leq w_H \min_{N'} N'.\text{cost} \leq w_H w_L C^*$. ECBS saves the factor w_L because it compares the band with LB, which bounds C^* directly ($\text{LB} \leq C^*$), instead of with the smallest cost, which only bounds $w_L C^*$; the price is that every low-level search must return its $f_{\min}$ and every node must carry the per-agent bounds.

Exercise 10.7. Checkpoints. Two functions of `ch10_ecbs.py` are the algorithm: `focal_space_time_astar` ([Algorithm 10.1](#)) and `ecbs` ([Algorithm 10.2](#)); `OtherPaths` answers “how many other agents does this state hit” in constant time, and `cbs` in the same file is the $w = 1$ reference. Each level needs *two* orderings of the same nodes: a heap on f (or on SoC) that

owns the lower bound of Equation (10.3), and a heap on the conflict count h_c restricted to the band $f \leq w f_{\min}$ of Equation (10.1). Steps: make the low level return the pair (path, $f_{\min}$), because the high-level bound is the sum of those minima and never the sum of the returned path costs; add h_c and the focal choice downstairs; add the same band upstairs; wrap the benchmark in a runner that writes one row per (instance, solver) and validates before it records. Expected numbers on the 8×8 grids of Figure 10.5: at $k = 10$ CBS needs 221 CT nodes and 0.22 s against 2.2 nodes and 3 ms for $ECBS(1.5)$; at $k = 16$ CBS solves a fifth of the instances in about 3 s while $ECBS(1.5)$ solves all of them in 23 ms at a mean cost/ C^* of 1.07. The knee sits near $w \approx 1.1$ – 1.2 , and the largest ratio you are likely to see is about 1.3 for $w = 1.5$: the guarantee of Theorem 10.11 is sound but loose, and saying so with your own numbers is the point of (c). Classic mistakes: using the popped node's f as the band reference instead of the minimum over OPEN; taking the returned path cost as the lower bound; never re-growing the focal list when $f_{\min}$ rises; averaging cost/ C^* over instances CBS failed on. Done when $w = 1$ reproduces the optimal costs exactly and one paragraph says, with numbers, what your speed-up cost — the Week 5 milestone (Appendix A).

Exercise 10.8. Checkpoints. All three parts are small edits to `ch10_ecbs.py`, so copy it once and keep the four variants side by side in one benchmark script. (a) $BCBS(w, 1)$ is the band upstairs only: the low level goes back to plain A^* on f and returns the cheapest path, ignoring h_c ; $BCBS(1, w)$ is the mirror image. Run all four at $w = 1.5$ on the benchmark of Exercise 10.7 and plot CT expansions and cost/ C^* together; expect the low-level band to do most of the work, because a path of equal cost with fewer conflicts removes constraint-tree nodes before they are ever generated, while the high-level band mostly reorders nodes that already exist. (b) The duplicate rule is one line (line 9): a state already expanded with an equal g is dropped today, and you want it re-inserted when the new path has a smaller h_c , so keep the closed entry as a pair (g, h_c) and dominate on both. On Example 10.6 the root has cost 11, per-agent bounds (2, 4, 5) and one conflict, and $ECBS(1.2)$ returns 13 after one split; the question is whether the new rule finds an equal-cost conflict-free path for one of the three agents, which would make the root itself the answer at cost 11. (c) Expect a small gain that grows with k , and report it as a paired comparison on the same seeds. Classic mistakes: admitting a state with a larger g , which breaks Lemma 10.9; treating h_c as static when it is measured against the other agents' current paths; comparing variants at different time limits. Done when the four variants are one flag apart and your table says which level the band pays off at.

Exercise 11.1. (a) With $|V| = 300$ there are $300^3 = 2.7 \cdot 10^7$ joint configurations for three agents, of which $300 \cdot 299 \cdot 298 \approx 2.67 \cdot 10^7$ are collision-free, and $5^3 = 125$ joint moves per node. For twelve agents there are $300^{12} \approx 5.3 \cdot 10^{29}$ configurations, about $300^{12} \approx 4.2 \cdot 10^{29}$ collision-free ones (the product $300 \cdot 299 \cdots 289$), and $5^{12} \approx 2.4 \cdot 10^8$ joint moves per node. (b) One full joint step of plain joint A^* generates all $5^{12} \approx 2.4 \cdot 10^8$ successors of a node. Operator decomposition generates 5 successors per intermediate state and needs 12 levels; if nothing were pruned, the twelve levels would again enumerate 5^{12} leaves, but every level is an A^* node with its own f , so the search expands only the intermediate states whose f is small enough, and in practice a handful of states per level: the enumeration is lazy and pruned by the heuristic, which the Cartesian product of plain joint A^* cannot be. (c) A heuristic decides which node is expanded next, not how many successors an expansion generates; every expansion of plain joint A^* enumerates $(b+1)^k$ joint moves and checks each for collisions, so even a search that expands only the nodes on an optimal plan pays $(b+1)^k$ per expansion.

Exercise 11.2. The policies are $\phi_1(a) = b$, $\phi_1(b) = c$, $\phi_2(c) = b$, $\phi_2(b) = a$ and $\phi_3(d) = d$ (agent 3 is at its goal and rests for free). Step 1 expands the start (a, c, d) with $C = \emptyset$ and generates the policy successor (b, b, d) , in which agents 1 and 2 share the vertex b : the successor

is discarded, $\{1, 2\}$ is stored on it and backpropagated to the start, whose collision set becomes $\{1, 2\}$; the start is re-opened. There is no node expanded with an empty collision set other than the start, so step 2 is the re-expansion of the start with $C = \{1, 2\}$: agent 1 chooses among $\{a, b\}$, agent 2 among $\{c, b\}$ and agent 3 follows its policy and stays at d , which gives the limited neighbors (a, c, d) (the joint wait, cost 2), (a, b, d) , (b, c, d) and (b, b, d) (the known collision). Two of them enter OPEN with $g = 2$ and $f = 5$: (a, b, d) and (b, c, d) (one agent moves, the other waits away from its goal, agent 3 rests at its goal for free). The joint wait leads back to the start itself, whose g is already 0, so it is generated but never queued—a useful reminder that a limited neighbor is not automatically a new node. Step 3 expands one of the two nodes with $f = 5$, whichever was pushed first; its collision set is empty, and its policy successor is a swap of agents 1 and 2 along the edge $a-b$ or $b-c$, so $\{1, 2\}$ is backpropagated to it and to the start, and the node is re-opened. Agent 3 enters a collision set at expansion 5, the second expansion of (a, b, d) . Once agent 2 is coupled it may leave its policy, and one of its limited neighbors there is the move $b \rightarrow d$ into the vertex on which agent 3 rests: a vertex collision of agents 2 and 3. That set is stored on the successor and backpropagated, so $C((a, b, d))$ grows to $\{1, 2, 3\}$ and the start is re-opened with $C = \{1, 2, 3\}$ at expansion 6. The start therefore ends with the collision set $\{1, 2, 3\}$ and the search degenerates into full joint A^* over the three agents. The two-agent plan of cost 7 needs d as a passing place, and with agent 3 parked there the instance is unsolvable: agents 1 and 2 must pass each other on the path $a-b-c$ and cannot. M^* therefore returns *failure*, after 12 expansions and 84 generated candidates with the chapter code, and [Theorem 11.9](#) guarantees that it returns at all, once every joint move of the three agents has been tried.

Exercise 11.3. (a) Every agent i must make at least $d_i(v^i)$ moves, each of cost one, so the cost to go is at least $\max_i d_i(v^i)$, and $h_{\max} \leq h^*$. Since all $d_i \geq 0$, $\max_i d_i(v^i) \leq \sum_i d_i(v^i)$, so the sum dominates the maximum; by the dominance theorem of [Chapter 4](#) it expands no more nodes with $f < C^*$. (b) If waiting were free everywhere, a wait of an agent away from its goal would cost 0 while d_i stays the same, so $h(v) \leq c(v, v') + h(v')$ still holds (both sides are equal) and the heuristic remains consistent; but the joint graph now has zero-cost cycles (everybody waits), so g can no longer distinguish waiting from not waiting, the cheapest plan may contain arbitrarily many waits, and the search stays finite only because A^* never re-expands a node with the same g ; the returned plan minimizes the number of moves, which is a different objective. (c) For the makespan, let every joint move cost 1 unless every agent rests at its goal, in which case it costs 0; the admissible heuristic is $h_{\max}(v) = \max_i d_i(v^i)$, because at least that many time steps remain. Along the policy path every step costs 1 and reduces $h_{\max}$ by exactly 1 until the slowest agent arrives, so f is constant and the policy path from v costs $h_{\max}(v)$, a lower bound; hence it is an optimal continuation whenever it is conflict-free.

Exercise 11.5. (a) Seven of the twelve are re-expansions after a collision among the node's *own* successors: the node was expanded, one of its limited neighbors turned out to be a vertex or a swap collision, and line 12 added the two agents to the node itself before backpropagating. The other five received their set from a child (line 13): the collision was found one or more steps deeper and traveled up the back sets. The start is one of the five, which is why its two expansions are steps 1 and 4: at step 1 its single policy successor is legal, and the head-on meeting appears only when that successor is expanded at step 2. (b) The collision set of v is defined by what the search finds *below* v , and M^* has no oracle for it: computing it in advance would mean knowing which agents interact on the way to the goal, which is the problem itself. So every node starts with $C(v) = \emptyset$, is expanded along the policies, and is re-opened when a descendant proves that this was too narrow; the start is expanded once with $C = \emptyset$ (step 1) and again with $C = \{1, 2\}$ (step 4), and no rule that keeps optimality can avoid the first of the two. The price is bounded: a node is re-expanded at most once per strict growth of its

collision set, hence at most k times, which is the argument of [Theorem 11.9](#). (c) The instance of [Exercise 11.2](#) does exactly this: on the path $a-b-c$ with the spur $b-d$, agents 1 and 2 must exchange places and agent 3 rests on d . The start is re-opened with $C = \{1, 2\}$ at expansion 2, and once agent 2 is free to leave its policy it tries the move $b \rightarrow d$ onto agent 3, so the start is re-opened again with $C = \{1, 2, 3\}$ at expansion 6; the run ends in failure after 12 expansions. A three-drone aisle with a single bay behaves the same way as soon as the third drone is parked in the bay.

Exercise 11.6. (a) The leader is d steps from the junction v and the follower one step behind it. The multipush moves the leader d times and, after each of its moves, the follower once into the vertex just vacated, which is $2d$ moves; at v the leader stands on the junction and the follower on the vertex u before it. The exchange is the fixed sequence of [Figure 11.6\(b\)](#): six moves. The reversal undoes the $2d$ preparatory moves one by one, whoever stands on the target of each move stepping back to its source, which is again $2d$ moves. In total $2d + 6 + 2d = 4d + 6$, and nothing else moved because no other agent was cleared. (b) In the swap example the code executes 14 moves: one push of b from c_1 to c_2 , one move of a to its goal c_1 , the swap, in which b leads from c_2 to the junction c_3 with $d = 1$ and which therefore costs $4 \cdot 1 + 6 = 10$ moves, one move of b to c_0 and one move of the displaced a back to c_1 . With the junction d steps from c_2 the count is $4d + 10$, the makespan of the one-move-per-step plan is $4d + 10$, and its sum of costs is the sum of the times of the last moves of the two agents, $(4d + 10) + (4d + 9) = 8d + 19$, which is 27 for $d = 1$. The optimal joint plan sends both agents toward the junction together, parks a in the spur for one step while b turns around behind it, and brings both back: with the junction three steps from a 's start, as in the example, that is makespan 7 and sum of costs 14, and in general each agent needs the walk to the junction and back plus one or two steps, so both quantities grow linearly in the distance as well, but with the slope of two walks instead of four and with both agents moving at the same time. (c) Number the six moves $a : v \rightarrow n_1$, $b : u \rightarrow v$, $b : v \rightarrow n_2$, $a : n_1 \rightarrow v$, $a : v \rightarrow u$, $b : n_2 \rightarrow v$. They pair into the three time steps $\{1, 2\}$, $\{3, 4\}$ and $\{5, 6\}$: each of the three pairs is a following move (one agent enters the vertex the other leaves in the same step, which [Section 11.3](#) allows), the two moves of a pair run on different edges, and neither is a swap along one edge, so no pair has a conflict. No fewer than three steps are possible because each of the two agents has to make three moves and an agent moves at most once per step; three steps are therefore exactly optimal. In the multipush the follower always enters the vertex the leader is leaving, which is a following move, so all d pairs of moves can run in d time steps.

Exercise 11.7. Checkpoints for an optional extension: the study plan asks only that you know what M^* and Push-and-Swap do. The reference is `ch11_mstar.py: joint_astar` with its operator-decomposition flag, `independence_detection`, `mstar` with `limited_neighbors`, and the `PushSwapState` primitives. A joint state is a tuple of k cells, so it hashes and keys a dictionary directly; the individual policies of [Definition 11.4](#) are one backward breadth-first search per agent, computed once and shared; a collision set is a set of agent indices stored *per joint state*. Steps: write the joint successor function and the cost of [Equation \(11.1\)](#) first, with a parked agent paying nothing, and test two agents against your CBS of [Chapter 9](#); add the heuristic of [Equation \(11.2\)](#); for (b) merge two groups only when re-planning one around the other at *equal* cost fails; for (c) reverse a completed swap by replaying the recorded moves *by position*, not by agent. Expected numbers: on the chapter's worked example joint A^* expands 24 states and generates 612, operator decomposition 100/292, independence detection 27/162 and M^* 31/127, all four at cost 15. On random 8×8 maps ([Figure 11.5](#)) the mean largest group stays near 2.8 at $k = 6$ while joint A^* generates over a hundred thousand states against about a thousand for independence detection at the median. Push-and-Swap solves the swap example in 14 moves at $\text{SoC} = 27$ where the optimum is a makespan of 7

and $\text{SoC} = 14$ (Theorem 11.12). Classic mistakes: a collision set stored per agent instead of per state; back-propagating collisions without re-opening the parent; a swap that leaves everyone but the two agents displaced. Done when every plan validates, your solvers agree with `joint_aster` on a few dozen random instances, and you can say why these methods lose on Example 11.8.

Exercise 12.1. With $R = r_A + r_B = 0.8\text{ m}$ and $d = 4\text{ m}$, Equation (12.5) gives $\theta = \arcsin(0.8/4) = \arcsin(0.2) = 11.54^\circ$, a wedge 23.07° wide. Forgetting r_A leaves $R = 0.5$ and $\theta = \arcsin(0.125) = 7.18^\circ$ (wedge 14.36°), so the cone is too narrow by more than a third of its width; forgetting both radii leaves $R = 0$ and $\theta = 0^\circ$, a single dangerous direction and no wedge at all. That is the mistake of the pitfall *Forgetting to add the radii* in Section 12.10: the test reports safety while the disks graze. The half-angle reaches 30° when $\sin \theta = R/d = 0.5$, i.e., at $d = R/\sin 30^\circ = 1.6\text{ m}$. As $d \downarrow R = 0.8\text{ m}$, $\arcsin(R/d) \rightarrow 90^\circ$ and the cone opens into the half-plane of all velocities with a positive component toward B ; at $d = R$ the disks touch, Algorithm 12.1 returns 0, and no choice of velocity can undo the contact — which is why the horizon must trigger the maneuver while θ is still small.

Exercise 12.2. (a) Let $\mathbf{t}$ be a point of contact of a tangent from the origin to the circle of radius R around $\mathbf{p}_{\text{rel}}$. The radius $\mathbf{p}_{\text{rel}} - \mathbf{t}$ is perpendicular to the tangent, so the triangle with vertices $\mathbf{0}$, $\mathbf{t}$, $\mathbf{p}_{\text{rel}}$ has a right angle at $\mathbf{t}$, hypotenuse d and the leg R opposite to the angle θ at the origin; hence $\sin \theta = R/d$ and, by Pythagoras, $\|\mathbf{t}\| = \sqrt{d^2 - R^2}$. (b) If $t\mathbf{v} \in D(\mathbf{p}_{\text{rel}}, R)$ for some $t > 0$, then $(t/\lambda)(\lambda\mathbf{v})$ is the same point, so $\lambda\mathbf{v} \in CC_{A|B}$ for every $\lambda > 0$. For the velocity obstacle, $\lambda(\mathbf{v}_B + \mathbf{v}) = \mathbf{v}_B + (\lambda\mathbf{v} + (\lambda - 1)\mathbf{v}_B)$ lies in $\mathbf{v}_B \oplus CC_{A|B}$ for all $\lambda > 0$ and all $\mathbf{v} \in CC_{A|B}$ only if $(\lambda - 1)\mathbf{v}_B$ is a difference of cone vectors for every λ , which fails for $\mathbf{v}_B \neq \mathbf{0}$. Concretely, take any $\mathbf{v} \in CC_{A|B}$ and let $\lambda \rightarrow 0^+$: then $\lambda(\mathbf{v}_B + \mathbf{v}) \rightarrow \mathbf{0}$, and $\mathbf{0} \in \mathbf{v}_B \oplus CC_{A|B}$ only if $-\mathbf{v}_B \in CC_{A|B}$, i.e., only if flying at $-\mathbf{v}_B$ would hit B . Whenever $-\mathbf{v}_B \notin CC_{A|B}$ the points $\lambda(\mathbf{v}_B + \mathbf{v})$ leave $VO_{A|B}(\mathbf{v}_B)$ for small λ , so the set is not a cone with apex $\mathbf{0}$. (c) By (b) membership in $CC_{A|B}$ depends only on the direction of $\mathbf{v}_A - \mathbf{v}_B$. After truncation the collision must occur before τ , and the time to collision scales like $1/\|\mathbf{v}_{\text{rel}}\|$, so a slow velocity in the cone may be allowed while a fast one in the same direction is forbidden.

Exercise 12.3. By Equation (12.3) the disks are in contact at time $t > 0$ exactly when $\|\mathbf{p}_{\text{rel}} - t\mathbf{v}\| \leq R$; dividing by t gives $\|\mathbf{p}_{\text{rel}}/t - \mathbf{v}\| \leq R/t$, i.e. $\mathbf{v} \in D(\mathbf{p}_{\text{rel}}/t, R/t)$. That disk is the image of $D(\mathbf{p}_{\text{rel}}, R)$ under the scaling $\mathbf{x} \mapsto \mathbf{x}/t$ about the origin, and a scaling about the origin maps every ray from the origin onto itself and preserves tangency, so every one of these disks is inscribed in the same cone. For the two distances use the scaling identity $t_c(s\mathbf{v}) = t_c(\mathbf{v})/s$ of Equation (12.12). Along the axis, the unit vector $\mathbf{p}_{\text{rel}}/d$ first touches the disk at $t_c = d - R$, so the truncated set begins on the axis at $s = (d - R)/\tau$. Along a leg, the unit direction touches the disk at its tangent point, at distance $\sqrt{d^2 - R^2}$ by Exercise 12.2, so the tangency point of the truncation disk sits at $s = \sqrt{d^2 - R^2}/\tau$. With $d = \sqrt{50}$, $R = 1$ and $\tau = 5$ these are $(7.0711 - 1)/5 = 1.2142$ and $7/5 = 1.4$. Check them in Figure 12.2(b), where the apex is $\mathbf{v}_B = (0, 0.9)$ and the truncation disk has center $(1, -0.1)$ and radius $R/\tau = 0.2$: the leg point $(0, 0.9) + 1.4(0.8, -0.6) = (1.12, 0.06)$ is at distance $\|(0.12, 0.16)\| = 0.2$ from that center, and the axis point $(0, 0.9) + 1.2142(0.7071, -0.7071) = (0.8586, 0.0414)$ is at distance 0.2 from it as well. Both lie on the truncation circle, the first at the tangency with the leg and the second at the point of the arc nearest the apex.

Exercise 12.4. With $\mathbf{p} = (6, 2)$, $\mathbf{v} = (3, 0)$ and $R = 2.5$: $\mathbf{v} \cdot \mathbf{v} = 9$, $\mathbf{p} \cdot \mathbf{v} = 18$, $\mathbf{p} \cdot \mathbf{p} - R^2 = 40 - 6.25 = 33.75$, so the quadratic is $9t^2 - 36t + 33.75 = 0$ with $\Delta/4 = 324 - 303.75 = 20.25$ and $\sqrt{\Delta}/4 = 4.5$. The roots are $t = (18 \mp 4.5)/9$, i.e., $t_c = 1.5$ and $t_2 = 2.5$: the disks overlap between 1.5 and 2.5. The closest approach is at $t^* = \mathbf{p} \cdot \mathbf{v}/(\mathbf{v} \cdot \mathbf{v}) = 2$,

where the relative position is $(0, 2)$ and the distance $2 < R$, consistent with the overlap interval being symmetric around t^* . For $\mathbf{v} = (-3, 0)$ the discriminant is the same but $\mathbf{p} \cdot \mathbf{v} = -18 < 0$: both roots are negative (-1.5 and -2.5), the “collision” lies in the past and [Proposition 12.6](#) reports none. `time_to_collision((6, 2), (3, 0), 2.5)` returns `1.5` and `time_to_collision((6, 2), (-3, 0), 2.5)` returns `inf`.

Exercise 12.5. The apex $\mathbf{v}_B = -s \mathbf{p}_{\text{rel}}/d$ lies on the axis of the wedge at distance s from the origin, on the side away from B . The reachable disk $D(\mathbf{0}, v_{\text{max}})$ is centered on the axis at distance $s > v_{\text{max}}$ from the apex, so seen from the apex it subtends the half-angle $\arcsin(v_{\text{max}}/s)$ (the same right triangle as in [Exercise 12.2](#)). It lies entirely inside the wedge of half-angle $\arcsin(R/d)$ if and only if $v_{\text{max}}/s \leq R/d$, i.e., $s \geq v_{\text{max}}d/R$. With $v_{\text{max}} = 1.5$, $d = 7.07$, $R = 1$ the threshold is $s = 10.6$ m/s. Just above it every candidate collides; the fallback minimizes $\|\mathbf{v} - \mathbf{v}^{\text{pref}}\| + \kappa/t_c$ and, for a large enough κ , picks the reachable velocity with the largest time to collision, which is the one closest to the boundary of the wedge: full speed perpendicular to the line of sight, slightly away from B . Truncation does not help because at $s \approx 10.6$ the collision happens after about $(d - R)/(s + v) \approx 0.5 \text{ s} < \tau$. For s well below the threshold but d small, the wedge is wide ($\theta \rightarrow 90^\circ$ as $d \rightarrow R$), the feasible velocities are those pointing away from B or nearly perpendicular to the line of sight, and the closer B is, the harder the maneuver; a horizon of a few times R/v makes the agent react while θ is still small.

Exercise 12.6. (a) With $\mathbf{v}_B = \mathbf{0}$, [Equation \(12.6\)](#) reads $VO_{A|B} = \mathbf{0} \oplus CC_{A|B} = CC_{A|B}$, so the velocity obstacle is the collision cone itself, and the truncation disk $D(\mathbf{v}_B + \mathbf{p}_{\text{rel}}/\tau, R/\tau)$ of [Proposition 12.8](#) becomes $D(\mathbf{p}_{\text{rel}}/\tau, R/\tau)$. (b) A velocity is dangerous exactly when its ray from the origin meets $\mathcal{O}$, so the forbidden set is the union of the rays through the points of $\mathcal{O}$, i.e., the set of directions $\{\mathbf{x}/\|\mathbf{x}\| : \mathbf{x} \in \mathcal{O}\}$ scaled by every positive factor. Since $\mathcal{O}$ is convex and does not contain the origin, it lies in an open half-plane through the origin, so those directions form an arc of angular width less than π ; and if two directions are attained, so is every direction between them, because the segment joining the two witnesses stays in $\mathcal{O}$ and its rays sweep the intermediate directions. The arc is therefore closed and bounded by the two extreme tangent rays, and the union is the wedge between them. (c) Inflating the wall by $r_A = 0.5$ moves its face to 1.5 m from the drone’s center. Let $\mathbf{n}$ be the unit normal pointing at the wall. Untruncated, the obstacle forbids every velocity with $\mathbf{v} \cdot \mathbf{n} > 0$: any component toward the wall reaches it eventually, and only velocities parallel to the wall or away from it survive — the frozen behavior of the pitfall *No truncation*. With $\tau = 4 \text{ s}$ the drone must cover 1.5 m within the horizon to be in danger, so the forbidden velocities are those with $\mathbf{v} \cdot \mathbf{n} > 1.5/4 = 0.375 \text{ m/s}$: the drone may close on the wall slowly and is constrained only when the horizon catches up.

Exercise 12.7. Hints. Use [Algorithm 12.3](#) with synchronous updates and record positions and velocities at every step; the metrics are the minimum over time and pairs of the center distance, the path length divided by the start–goal distance, the first time at which the goal is within a tolerance, and the count of steps at which the feasible set of [Algorithm 12.2](#) was empty. Count sign changes of the velocity component perpendicular to the start–goal line. With the book’s parameters ($\Delta t = 0.1$, $\tau = 5$, $R = 1$, speed 1, $v_{\text{max}} = 1.5$) the circle scenario gives the numbers of [Table 12.2](#); with $\tau = \infty$ agents facing a wall of static disks stop before reaching it, with $\tau = 1$ the agents of the circle scenario turn late and pairs collide from $n = 4$ on, and 36 directions produce visibly larger sign-change counts than 180. Keep the scenario generator, the metrics and the plotting code separate from the avoidance rule, so that RVO and ORCA can be plugged in as alternative implementations of `choose_velocity` in [Chapter 13](#).

Exercise 12.8. (a) The proof of [Theorem 12.3](#) never uses the dimension. For $\mathbf{v} \neq \mathbf{0}$ let φ be the angle between $\mathbf{v}$ and $\mathbf{p}_{\text{rel}}$; the distance from the center to the ray $\{t\mathbf{v} : t \geq 0\}$ is $d \sin \varphi$

when $\varphi \leq \pi/2$ and d otherwise. Since $d > R$ the second case never meets the ball, so the ray meets it exactly when $\varphi \leq \pi/2$ and $d \sin \varphi \leq R$, that is $\varphi \leq \arcsin(R/d)$. The set of vectors of $\mathbb{R}^3$ making an angle at most θ with a fixed axis is the right circular cone of half-angle θ , and cutting it with a plane through the axis leaves the planar wedge of [Theorem 12.3](#). (b) By the rotational symmetry about the axis, the tangency set is the circle traced by the planar tangent point of [Exercise 12.2](#), which lies at distance $\sqrt{d^2 - R^2}$ from the apex at the angle θ from the axis. Its distance along the axis is $\sqrt{d^2 - R^2} \cos \theta = (d^2 - R^2)/d$ and its radius is $\sqrt{d^2 - R^2} \sin \theta = R\sqrt{d^2 - R^2}/d$. For the self-test geometry $\mathbf{p}_{\text{rel}} = (3, 4, 12)$, $d = 13$, $R = 1$ this is a circle at $168/13 = 12.923$ along the axis with radius $\sqrt{168}/13 = 0.997$. (c) Sample a few speeds times a few hundred well-spread directions on the unit sphere (a Fibonacci lattice spreads them evenly) and feed the candidates to `time_to_collision`, which already accepts 3-vectors. In a strictly head-on encounter $\mathbf{v}_{\text{rel}}$ lies on the axis of the cone, so by that same rotational symmetry a climb and a lateral turn need exactly the same deviation from $\mathbf{v}^{\text{pref}}$: the geometry does not prefer either, and the choice is made by the weights in [Equation \(12.14\)](#), by the acceleration limits and by airspace rules. Offset the encounter slightly and the cheaper escape is the one on the side the offset points to.

Exercise 13.1. From B 's point of view the relative position is $\mathbf{p}_A - \mathbf{p}_B = -\mathbf{p}_{\text{rel}}$ and the relative velocity is $\mathbf{v}_B - \mathbf{v}_A = -\mathbf{v}_{\text{rel}}$, so $VO_{B|A}^\tau = \{\mathbf{v} : \exists t \in (0, \tau], t\mathbf{v} \in D(-\mathbf{p}_{\text{rel}}, R)\} = -VO_{A|B}^\tau$. Point reflection through the origin maps the boundary to the boundary and preserves distances, so the boundary point of $VO_{B|A}^\tau$ closest to $-\mathbf{v}_{\text{rel}}$ is $-\mathbf{q}$, the change is $-\mathbf{q} - (-\mathbf{v}_{\text{rel}}) = -\mathbf{u}$, and the outward normal at $-\mathbf{q}$ is $-\mathbf{n}$. [Definition 13.7](#) then gives the point $\mathbf{v}_B + \frac{1}{2}(-\mathbf{u})$ and the normal $-\mathbf{n}$. Adding $(\mathbf{v}_A^{\text{new}} - \mathbf{v}_A - \frac{1}{2}\mathbf{u}) \cdot \mathbf{n} \geq 0$ and $(\mathbf{v}_B^{\text{new}} - \mathbf{v}_B + \frac{1}{2}\mathbf{u}) \cdot (-\mathbf{n}) \geq 0$ gives $(\mathbf{v}_A^{\text{new}} - \mathbf{v}_B^{\text{new}} - \mathbf{v}_{\text{rel}} - \mathbf{u}) \cdot \mathbf{n} = (\mathbf{v}_{\text{rel}}^{\text{new}} - \mathbf{q}) \cdot \mathbf{n} \geq 0$, the closed half-plane on the outer side of the tangent line at $\mathbf{q}$; [Lemma 13.9](#) finishes the argument.

Exercise 13.2. Reflecting the instance in the x -axis maps $\mathbf{p}_{\text{rel}}$ to $(4, -0.5)$ and leaves $\mathbf{v}_{\text{rel}} = (2, 0)$ unchanged, so every number of [Table 13.1](#) is reflected too. Now $\varphi = -7.13^\circ$, $\mathbf{w} = (1, 0.125)$ and $\mathbf{p}_{\text{rel}} \times \mathbf{w} = +1 > 0$, so the closest boundary point is on the *left* leg, with direction $\mathbf{d}_L = (0.9920, 0.1260)$, foot $\mathbf{q} = (1.9682, 0.2500)$, $\mathbf{u} = (-0.0318, 0.2500)$ and outward normal $\mathbf{n} = (-0.1260, 0.9920)$ (the left-leg normal is $\mathbf{d}$ rotated by $+90^\circ$). A 's half-plane has point $\mathbf{v}_A + \frac{1}{2}\mathbf{u} = (0.9841, 0.1250)$ and normal $\mathbf{n}$, and the program returns $\mathbf{v}_A^{\text{new}} = (1.1809, 0.1500)$: A passes above B instead of below. The mirroring is exact because the construction uses only distances, dot products and the sign of one cross product, all of which commute with a reflection.

Exercise 13.3. (a) The ray $t \mapsto t\mathbf{v}_{\text{rel}}$ with $\mathbf{v}_{\text{rel}} = (0.4, 0.05)$ points at 7.13° , almost exactly along $\mathbf{p}_{\text{rel}}$, so it enters the disk $D(\mathbf{p}_{\text{rel}}, 1)$ when $t \|\mathbf{v}_{\text{rel}}\| \approx \|\mathbf{p}_{\text{rel}}\| - 1$, that is, at $t \approx 3.03/0.403 \approx 7.5$ s $> \tau$: outside $VO_{A|B}^\tau$ (`time_to_collision` gives 7.52 s). (b) $\mathbf{c} = (1, 0.125)$, $\rho = 0.25$, $\mathbf{w} = (-0.6, -0.075)$, $\|\mathbf{w}\| = 0.6047$; $\mathbf{w} \cdot \mathbf{p}_{\text{rel}} = -2.4375 < 0$ and $(\mathbf{w} \cdot \mathbf{p}_{\text{rel}})^2 = 5.941 > R^2 \|\mathbf{w}\|^2 = 0.366$, so the arc case applies: $\mathbf{n} = \mathbf{w} / \|\mathbf{w}\| = (-0.9923, -0.1240)$, $\mathbf{u} = (\rho - \|\mathbf{w}\|)\mathbf{n} = (0.3519, 0.0440)$ and $\mathbf{q} = \mathbf{v}_{\text{rel}} + \mathbf{u} = (0.7519, 0.0940)$, the point of the arc nearest to the apex. $\mathbf{u} \cdot \mathbf{n} = -0.3547 < 0$: the relative velocity is outside the truncated cone by 0.355 m/s, and A 's half-plane through $\mathbf{v}_A + \frac{1}{2}\mathbf{u}$ allows A to move up to 0.177 m/s toward the cone. (c) Let T be the tangent point of the leg with direction $\mathbf{d}$, so $T = \ell_\tau \mathbf{d}$ with $\ell_\tau = \sqrt{\|\mathbf{c}\|^2 - \rho^2}$, and let $\mathbf{m}$ be the unit vector from $\mathbf{c}$ to T , perpendicular to $\mathbf{d}$. The foot of the perpendicular from $\mathbf{v}_{\text{rel}} = \mathbf{c} + \mathbf{w}$ is $(\mathbf{v}_{\text{rel}} \cdot \mathbf{d})\mathbf{d}$, and it lies beyond T exactly when $\mathbf{v}_{\text{rel}} \cdot \mathbf{d} \geq \ell_\tau = \mathbf{c} \cdot \mathbf{d}$, that is, when $\mathbf{w} \cdot \mathbf{d} \geq 0$. The front sector is bounded by the two radii to the tangent points; $\mathbf{w}$ outside the sector and on the side of this leg means that the angle between $\mathbf{w}$ and $\mathbf{m}$ is at most 90° beyond the radius direction, and since $\mathbf{d}$ is $\mathbf{m}$ rotated by 90° toward the outside of the sector, $\mathbf{w} \cdot \mathbf{d} \geq 0$ follows. With $\mathbf{w}$ inside the sector the arc case applies instead, so the leg formula is never used with a foot short of T .

Exercise 13.4. (a) The two inequalities read $(\mathbf{v}_A^{\text{new}} - \mathbf{v}_A - \alpha_A \mathbf{u}) \cdot \mathbf{n} \geq 0$ and $(\mathbf{v}_B^{\text{new}} - \mathbf{v}_B + \alpha_B \mathbf{u}) \cdot (-\mathbf{n}) \geq 0$; their sum is $(\mathbf{v}_{\text{rel}}^{\text{new}} - \mathbf{v}_{\text{rel}} - (\alpha_A + \alpha_B) \mathbf{u}) \cdot \mathbf{n} \geq 0$, and the guarantee needs $(\mathbf{v}_{\text{rel}}^{\text{new}} - \mathbf{v}_{\text{rel}} - \mathbf{u}) \cdot \mathbf{n} \geq 0$. The difference is $(\alpha_A + \alpha_B - 1) \mathbf{u} \cdot \mathbf{n}$. If $\mathbf{v}_{\text{rel}}$ is inside the cone, $\mathbf{u} \cdot \mathbf{n} > 0$ and a sum below 1 is not enough; if it is outside, $\mathbf{u} \cdot \mathbf{n} < 0$ and a sum above 1 lets the pair drift too far toward the cone. Only $\alpha_A + \alpha_B = 1$ works in both cases; with $\alpha_B = 0$ the inequality of B must hold with $\mathbf{v}_B^{\text{new}} = \mathbf{v}_B$, that is, B keeps the velocity that A assumed. (b) A alone moves to $(1.1809, -0.1500)$, so the relative velocity is $(2.1809, -0.1500)$, at $\angle(\mathbf{v}_{\text{rel}}, \mathbf{p}_{\text{rel}}) = -11.06^\circ$ (-3.93° in the world frame), inside the cone of half-angle 14.36° . (c) Both take the full change: the relative velocity moves by $2\mathbf{u}$, twice what is needed, which is safe but is the velocity obstacle rule; at the next step each sees the other's over-reaction, the preferred velocities are permitted again, and the dance of [Proposition 13.2](#) returns.

Exercise 13.5. $2\mathbf{v} - \mathbf{v}_A \in \mathbf{v}_B + VO_{A|B}^\tau$ holds exactly when $\mathbf{v} \in \frac{1}{2}(\mathbf{v}_A + \mathbf{v}_B) + \frac{1}{2}VO_{A|B}^\tau$. By [Equation \(13.3\)](#),

$$\begin{aligned} \frac{1}{2}VO_{A|B}^\tau &= \bigcup_{t \in (0, \tau]} \frac{1}{2}D(\mathbf{p}_{\text{rel}}/t, R/t) = \bigcup_{t \in (0, \tau]} D(\mathbf{p}_{\text{rel}}/2t, R/2t) \\ &= \bigcup_{t' \in (0, 2\tau]} D(\mathbf{p}_{\text{rel}}/t', R/t') = VO_{A|B}^{2\tau}, \end{aligned}$$

with $t' = 2t$. The horizon doubles because the reciprocal test looks at $2\mathbf{v} - \mathbf{v}_A$, a velocity twice as far from the current one, so a collision that the average velocity would cause at time 2τ shows up at τ in the test velocity. An implementation that wants RVO agents to react only to collisions within τ must therefore build the velocity obstacle with horizon $\tau/2$ before applying the reciprocal transformation, or, equivalently, test $2\mathbf{v} - \mathbf{v}_A$ against $VO^{\tau/2}$; ORCA avoids the issue because it works with the relative velocity directly.

Exercise 13.6. (a) Both orders return the same velocity, because the program has one optimum; what differs is the work. With H_1, H_2, H_3 the optimum for H_1 alone is the projection of $\mathbf{v}^{\text{pref}} = (1.2, 0.3)$ on the boundary of H_1 (it violates H_1), then H_2 is already satisfied and costs constant time, and H_3 is checked last. With H_3, H_2, H_1 the calls to the one-dimensional program come in a different order and one of them clips the interval with two earlier half-planes instead of none. The lesson is the one behind [Proposition 13.12](#): the number of one-dimensional programs depends on the order, the result does not, and a random order makes the expected number small. (b) Suppose the optimum $\mathbf{v}_i^*$ for $H_1, \dots, H_i$ lies in the interior of H_i while $\mathbf{v}_{i-1}^*$ violates H_i . The feasible set is convex, so the segment from $\mathbf{v}_i^*$ to $\mathbf{v}_{i-1}^*$ stays inside $H_1, \dots, H_{i-1}$ and the disk, and the objective $\|\mathbf{v} - \mathbf{v}^{\text{pref}}\|$ decreases strictly along it (the optimum of a strictly convex objective over a convex set is unique). Moving a little way from $\mathbf{v}_i^*$ toward $\mathbf{v}_{i-1}^*$ stays in H_i and lowers the objective, contradicting optimality. Hence $\mathbf{v}_i^*$ lies on the boundary line of H_i . (c) With $\mathbf{v}^{\text{pref}} = (1.6, 0.4)$ the unconstrained optimum has norm $1.649 > v_{\text{max}}$, so the first step already clamps to the disk, $\mathbf{v} = 1.5 \mathbf{v}^{\text{pref}} / \|\mathbf{v}^{\text{pref}}\|$, and the chord test of [Algorithm 13.2](#) decides every later one-dimensional program.

Exercise 13.7. (a) At the optimum of [Equation \(13.12\)](#) the violations of the half-planes that are active are equal: if one were strictly smaller, one could move a little in the direction that decreases the largest ones without making that one the largest, and δ would drop. With three symmetric half-planes forming a triangle around the disk, the solution is therefore the center of the disk (or the point equidistant, in violation, from the three boundary lines) and δ is that common violation. (b) [Algorithm 13.3](#) walks the half-planes; when H_i is violated more deeply than the current record it minimizes viol_i subject to $\text{viol}_j \leq \text{viol}_i$ for every earlier j . Each such constraint is the half-plane whose boundary is the bisector of the boundary lines of H_i and H_j :

it passes through their intersection point, where both violations vanish, with normal $\mathbf{n}_i - \mathbf{n}_j$ up to scale. Three half-planes give at most three bisectors. (c) The objective never mentions $\mathbf{v}^{\text{pref}}$, so the fallback returns the same velocity for every preferred velocity: safety first. For a drone that is the right default for one or two steps but a poor steady state, because it discards the mission entirely; an implementation should keep the fallback rare (larger radii margins, a shorter τ , fewer neighbors) and hand control to the global layer of [Chapter 24](#) when it fires repeatedly.

Exercise 13.8. (a) Take the nearest wall disk above the agent: $\mathbf{p}_{\text{rel}} = (0, 1.25)$, $R = 0.5 + 0.5 = 1$, $\|\mathbf{p}_{\text{rel}}\| = 1.25$ and $\ell = \sqrt{1.25^2 - 1^2} = 0.75$. The wall does not move, so $\mathbf{v}_{\text{rel}} = (1, 0)$; the truncating disk for $\tau_{\text{obst}} = 2$ has center $(0, 0.625)$ and radius 0.5, and $\mathbf{w} = (1, -0.625)$ has $\mathbf{w} \cdot \mathbf{p}_{\text{rel}} = -0.7813 < 0$ but $(\mathbf{w} \cdot \mathbf{p}_{\text{rel}})^2 = 0.6104 < R^2 \|\mathbf{w}\|^2 = 1.3906$, so [Lemma 13.6](#) gives a leg, and $\mathbf{p}_{\text{rel}} \times \mathbf{w} = -1.25 < 0$ makes it the right leg: $\mathbf{d} = (0.8, 0.6)$, $\mathbf{q} = (0.64, 0.48)$, $\mathbf{u} = (-0.36, 0.48)$, $\mathbf{n} = (0.6, -0.8)$. A wall takes no responsibility, so the agent takes all of $\mathbf{u}$ and the half-plane is $(\mathbf{v} - (0.64, 0.48)) \cdot (0.6, -0.8) \geq 0$. The offset vanishes because the point lies on the leg through the origin, and the constraint reduces to $0.6v_x - 0.8v_y \geq 0$, that is $v_y \leq 0.75v_x$. The disk below gives the mirror image $v_y \geq -0.75v_x$, hence $|v_y| \leq 0.75v_x$: the corridor permits a lateral speed proportional to the forward speed, so an agent that has slowed to a stop can no longer step sideways. (b) Head-on, $\mathbf{v}_{\text{rel}}$ points along $-\hat{\mathbf{p}}$ with $\hat{\mathbf{p}} = \mathbf{p}_{\text{rel}}/\|\mathbf{p}_{\text{rel}}\|$ and the arc case applies: $\mathbf{u} = (d/\tau)\hat{\mathbf{p}} - \mathbf{v}_{\text{rel}}$ and $\mathbf{n} = -\hat{\mathbf{p}}$, so the half-plane $(\mathbf{v} - \mathbf{v}_A - \frac{1}{2}\mathbf{u}) \cdot \mathbf{n} \geq 0$ reads $(\mathbf{v} - \mathbf{v}_A) \cdot \hat{\mathbf{p}} \leq \frac{1}{2}(d/\tau - \mathbf{v}_{\text{rel}} \cdot \hat{\mathbf{p}})$, which caps A's approach speed along $\hat{\mathbf{p}}$ at $d/(2\tau)$; B is capped symmetrically, so the gap closes at no more than d/τ . One step of [Section 13.5.3](#) therefore leaves $d_{k+1} \geq d_k - \Delta t(d_k/\tau) = d_k(1 - \Delta t/\tau)$, with equality while both agents push at the cap. The factor $1 - \Delta t/\tau$ is a constant in $(0, 1)$, so $d_k \rightarrow 0$ geometrically and both speeds decay with it: the agents converge to the stalled state without ever reaching it, and neither ever reaches its goal. (c) Inside the local layer, break the symmetry with the responsibility shares of [Section 13.4.1](#): give one agent $\alpha = 1$ and the other $\alpha = 0$ (a priority rule, or a coin flip refreshed every few seconds), so the low-priority agent takes the whole of $\mathbf{u}$ and backs into the bay. This resolves the pair and keeps the pairwise guarantee of [Theorem 13.10](#) because the shares still sum to one, but it guarantees nothing globally: with three agents in the corridor the priority order can cycle. In the architecture of [Chapter 24](#), detect the stall (speed below a threshold for several consecutive steps) and ask the global planner for a new joint plan, which is complete on the grid and can route one drone into the bay explicitly; the price is a replanning latency of seconds, which is why the local layer must keep the drones safe in the meantime.

Exercise 13.9. Checkpoints. The reference is `ch13_orca.py`, where `simulate` runs the loop, `crossing_scenario` builds the instance, `orca_half_plane` implements [Algorithm 13.1](#), `orca_velocity` solves [Algorithm 13.2](#) with the dense fallback of [Algorithm 13.3](#), and `choose_sampled` gives the VO baseline. One **Agent** record per drone (position, velocity, radius, v_{max} , goal) and one half-plane per neighbor, stored as a point and an *inward* normal — get that orientation wrong once and the agents fly into each other. Steps: update all agents from the same snapshot, computing every new velocity before anybody moves; run the three modes on the same scenario and seed; record per mode the colliding pairs, the smallest center distance, the last arrival, the mean path length and the reversals of one agent, as in [Table 13.2](#); only then sweep τ and the spacing. Expected numbers on the default crossing (spacing 2 m, $r = 0.5$ m, $v_{\text{max}} = 1.5$ m/s, $\tau = 5$ s, $\Delta t = 0.1$ s): with no avoidance four pairs overlap and the separation reaches 0 near $t = 11$ s; sampled VO still leaves four colliding pairs at 0.05 m, arrives at 19.8 s and reverses 101 times with a velocity variation of 83.6 m/s (the dance of [Proposition 13.2](#)); ORCA is collision-free at exactly 1.000 m, arrives at 16.4 s against 16.0 s for the straight line and reverses twice. At $\tau = 10$ the program is infeasible on about a dozen agent-steps; at $\tau = 2$

the perfectly symmetric crossing deadlocks; below 1.5 m of spacing the dense fallback starts returning least-violating velocities. Classic mistakes: updating agents one at a time; building a half-plane from $\mathbf{v}_A - \mathbf{v}_B$ but applying it to $\mathbf{v}_A$; giving a non-cooperating intruder half the avoidance instead of all of $\mathbf{u}$. Done when you can draw the cone and the half-plane for one pair and your three modes reproduce collide / oscillate / smooth — the Week 6 milestone of Appendix A.

Exercise 13.10. Checkpoints. `orca_half_space_3d` already does the geometry: the truncated velocity obstacle is rotationally symmetric about $\mathbf{p}_B - \mathbf{p}_A$, so the closest boundary point lies in the plane spanned by that axis and $\mathbf{v}_{\text{rel}}$ and the planar case analysis of Lemma 13.6 applies inside it. It returns the normal and the point $\mathbf{v}_A + \frac{1}{2}\mathbf{u}$, so the half-space is $\{\mathbf{v} : (\mathbf{v} - \text{point}) \cdot \mathbf{n} \geq 0\}$, which is Equation (13.10). Steps: place six drones on the octahedron vertices of a sphere of radius 10 with antipodal goals, so all six routes cross the center; build $k - 1$ half-spaces per drone per step; solve by brute force first, drawing a few thousand velocities uniformly in the ball $\|\mathbf{v}\| \leq v_{\text{max}}$ with $r = v_{\text{max}}U^{1/3}$ and a uniform direction, keeping those that satisfy every half-space and taking the one closest to $\mathbf{v}_{\text{pref}}$, or the least-violating sample when none survives; replace it by the recursive incremental algorithm only once the sampled version is right, since in $\mathbb{R}^3$ the recursion projects onto a plane and calls Algorithm 13.2. Expected numbers with $r = 0.5$ m, $v_{\text{max}} = 1.5$ m/s, $\tau = 5$ s and $\Delta t = 0.1$ s: the run is collision-free with a smallest center distance near 1.02 m against the threshold $r_A + r_B = 1$ m, and all six arrive within 20–21 s against the 20 s of the straight line. Classic mistakes: sampling the ball with a uniform radius, which crowds the samples near the center; testing the half-spaces at exactly zero tolerance, so rounding fires the fallback every step; forgetting that six antipodal routes are perfectly symmetric and need a tie-break. Done when the minimum center distance never drops below $r_A + r_B$ and the sampled and incremental solvers agree to within the sample spacing.

Exercise 14.1. (a) $s_b = v^2/(2a_b) = 4/4 = 1$ m. (b) The command is held for Δt before braking starts: $v\Delta t + v^2/(2a_b) = 0.5 + 1 = 1.5$ m. (c) Continuous condition: $\sqrt{2 \cdot 0.608 \cdot 2} = 1.56$ m/s; discrete condition: $-0.5 + \sqrt{0.25 + 4 \cdot 0.608} = 1.14$ m/s. (d) The physics is the same, but in a digital loop the drone cannot begin to brake before the current command has run its course, so the reaction delay Δt costs $v\Delta t$ of the free distance.

Exercise 14.2. With $n = a_{\text{max}}\Delta t/\Delta v = 2$ the grid has $(2n + 1)^{\text{dim}}$ points: 25 in 2D and 125 in 3D; with $\Delta v = 0.1$ m/s, $n = 5$ and the counts are 121 and 1331. With $\Delta t = 0.1$ s the window spans only ± 0.2 m/s, less than one grid step of 0.25 m/s, so every grid point except $\mathbf{v}_a$ itself is clipped away by the bounds $[\mathbf{lo}, \mathbf{hi}]$. From $\mathbf{v}_a = (1, 0)$ the candidate set is $\{(1, 0), (0.8, 0)\}$, the current velocity and the braking candidate: the drone can only keep its speed or brake along its current direction, never turn. From rest the set is $\{\mathbf{0}\}$ and the drone never moves at all. `DwaParams(dt=0.1)` reproduces both cases.

Exercise 14.3. With $(0.8, 0.1, 0.1)$: $G(1.0, 0) = 0.800 + 0.020 + 0.050 = 0.870$, $G(1.0, 0.5) = 0.675 + 0.100 + 0.056 = 0.831$, $G(0.75, 0.5) = 0.644 + 0.100 + 0.045 = 0.789$. The straight command wins (it is also the winner among all 16 admissible candidates): the heading-dominant drone keeps flying at the obstacle and swerves only when the braking test forces it to. With $(0.2, 0.6, 0.2)$ the scores are 0.422, 0.881 and 0.851: the swerve wins because clearance now counts three times as much as heading. The straight-flying drone does not crash because $(1.0, 0)$ is admissible, its braking distance of 0.5 m fits into the 0.608 m of free distance, and by the safety theorem the braking candidate remains admissible in every later step.

Exercise 14.4. Integrating $\dot{\theta} = \omega$ gives $\theta(t) = \theta + \omega t$; substituting into $\dot{x} = v \cos \theta(t)$ and integrating gives $x(t) = x + (v/\omega)(\sin(\theta + \omega t) - \sin \theta)$, and the same step for $\dot{y}$ gives $y(t) =$

$y - (v/\omega)(\cos(\theta + \omega t) - \cos \theta)$, which is Equation (14.1). With $x_c = x - (v/\omega) \sin \theta$ and $y_c = y + (v/\omega) \cos \theta$ the two equations read $x(t) - x_c = (v/\omega) \sin(\theta + \omega t)$ and $y(t) - y_c = -(v/\omega) \cos(\theta + \omega t)$, so $(x(t) - x_c)^2 + (y(t) - y_c)^2 = (v/\omega)^2$: a circle of radius v/ω about (x_c, y_c) , and $t = 0$ puts the start pose on it. For $(v, \omega) = (1, 0.5)$ from $(0, 0, 0)$ after 2 s: $\theta = 1$ rad, $x = 2 \sin 1 = 1.683$ m, $y = 2(1 - \cos 1) = 0.919$ m, which is what `diffdrive_rollout()` returns. The predicted pose matters for the differential drive because it turns while it flies the arc: its orientation at the end differs from the one the angle was measured at by $\omega \Delta t$, and the next command is issued from there. A drone flies a straight segment and keeps its direction, so only the position shifts, which changes the angle to the target by a second-order amount.

Exercise 14.7. (a) The scene is mirror-symmetric about the goal line, so the free distance, the heading angle and the speed of (v_x, v_y) and $(v_x, -v_y)$ coincide and the tie is broken by the order of the grid. After one step the drone is off the axis by $v_y \Delta t$; the side it moved to now has a slightly worse heading and a slightly better clearance, and depending on the weights the mirror candidate can win the next step and move the drone back. (b) If $|G(\mathbf{v}_1) - G(\mathbf{v}_2)| < \eta$ and $\mathbf{v}_a = \mathbf{v}_1$ is admissible, then $G(\mathbf{v}_a) \geq G(\mathbf{v}_2) - \eta$ and the rule keeps $\mathbf{v}_1$; the alternation can only start once the scores differ by at least η . (c) Hysteresis only decides between candidates whose scores are within η of each other; in the U-trap the commands that would leave the U score far below the commands along the wall for every η , so the drone keeps shuttling or settles near an arm. With `DwaParams(hysteresis=0.05)` the run on `utrap_scene()` still ends inside the U after 300 steps, having flown about 36 m instead of 80 m: the oscillation is damped, the trap remains.

Exercise 14.5. (a) The braking candidate is $\mathbf{v}_b = \lambda \mathbf{v}_a$ with $\lambda = \max(0, 1 - a_b \Delta t / \|\mathbf{v}_a\|)$, so $\|\mathbf{v}_b - \mathbf{v}_a\| = (1 - \lambda) \|\mathbf{v}_a\| = \min(\|\mathbf{v}_a\|, a_b \Delta t) \leq a_b \Delta t \leq a_{\max} \Delta t$: it lies in the Euclidean ball, which is the smaller and physically honest window (the corner of the box is $\sqrt{\dim} a_{\max} \Delta t$ away from $\mathbf{v}_a$). (b) The proof uses static obstacles once, in “because the obstacles have not moved... $d' \geq d - \|\mathbf{v}\| \Delta t$ ”. Let $a_b = 2$ m/s², $\Delta t = 0.25$ s and let the drone fly at $\|\mathbf{v}\| = 1$ m/s toward an obstacle at exactly $d = \|\mathbf{v}\| \Delta t + \|\mathbf{v}\|^2 / (2a_b) = 0.5$ m, which is admissible with equality. Next step the braking candidate has $v' = 0.5$ m/s and needs $v' \Delta t + v'^2 / (2a_b) = 0.1875$ m of free distance. With a static obstacle $d' = 0.25$ m and it passes; if the obstacle closes at $u = 0.3$ m/s then $d' = 0.5 - (1 + 0.3) \cdot 0.25 = 0.175 < 0.1875$ m and the braking candidate is inadmissible. The repair is the relative-velocity ray cast of Section 14.9: cast along $\mathbf{v} - \mathbf{v}_{\text{obs}}$, which restores the hypothesis. (c) Use the guaranteed free distance $d - \epsilon$: require $\|\mathbf{v}\| \Delta t + \|\mathbf{v}\|^2 / (2a_b) \leq d - \epsilon$, that is $\|\mathbf{v}\| \leq -a_b \Delta t + \sqrt{a_b^2 \Delta t^2 + 2a_b(d - \epsilon)}$. Claim (iii) then holds verbatim, because the drone moves less than the true free distance. Claim (ii) needs in addition that the query is consistent along a ray, $d' \geq d - \|\mathbf{v}\| \Delta t$ for the reported values too; a march with step size Δs satisfies this with $\epsilon = \Delta s$.

Exercise 14.6. (a) Scoring the 16 admissible candidates of Example 14.10 with $G = 0.6$ heading + 0.2 dist + 0.2 $\|\mathbf{v}\|$ in meters, (1.00, 0.50) wins with $0.506 + 0.600 + 0.224 = 1.330$, ahead of (0.75, 0.50) with 1.263. It is also the winner of the normalized score, so on this scene the bug is invisible; that is why unit errors survive testing. (b) In centimeters the same candidate wins with $0.506 + 60.0 + 22.4 = 82.87$, but the heading term now contributes 0.6% of the score: the drone is steered by clearance and speed alone. Put the goal behind the drone and open space ahead of it – a corridor whose mouth is 3 m wide in the direction opposite to the goal – and every candidate that flies away from the goal beats every candidate toward it. (c) Replacing the length unit by one λ times smaller multiplies dist and $\|\mathbf{v}\|$ by λ and leaves the angle θ , hence the heading term, unchanged: $G_\lambda = \alpha \text{ heading} + \lambda \beta \text{ dist} + \lambda \gamma \|\mathbf{v}\|$. Dividing by $\lambda > 0$ does not change the arg max, so the ranking is the one of the weights $(\alpha/\lambda, \beta, \gamma)$: the effective β and γ have grown by λ relative to α . In Equation (14.8) each term is divided by a

quantity in its own unit ($\pi, d_{\max}, v_{\max}$), so all three are dimensionless, λ cancels in $\text{dist}/d_{\max}$ and $\|\mathbf{v}\|/v_{\max}$, and the ranking is invariant.

Exercise 14.8. Checkpoints. In `ch14_dwa.py` start from `window_candidates` (the grid over the window of Definition 14.5, with the braking candidate always appended), `free_distance` along a rollout, `evaluate` (the three normalized terms of Equation (14.8)) and `dwa_command`; `simulate` closes the loop and three of the four failure scenes are already built. Obstacles are disks and boxes with closed-form ray tests, the state is a position and a velocity vector (the drone of Definition 14.2 is holonomic, so there is no heading), and one `DwaParams` record holds $v_{\max}$, $a_{\max}$, Δt , the horizon, the grid step Δv and the three weights. Steps: write `free_distance` and test it alone — a wall 1 m ahead at 2 m/s must be a collision within three steps; add the admissibility filter of Equation (14.6) *before* scoring; normalize the three terms over the admissible set, then score; only then build the failures and their repairs. Expected numbers: in an open field the drone arrives in about 17 steps over 6.2 m; the U-trap is *not* solved — the drone stalls against the back wall near (4.7, 0.0) — and a global A^* path repairs it in 25 steps and 10.2 m; the crossing intruder collides under a static rollout and is cleared in 18 steps by the relative-velocity rollout. In 3D the candidate count is $(2a_{\max}\Delta t/\Delta v + 1)^3$: 125 candidates and about 1.7 ms per step at $\Delta v = 0.25$ m/s against 1331 and 17 ms at 0.1 m/s — the whole argument for a coarse grid plus a global path. Classic mistakes: summing the three terms before normalizing, so the units choose the trajectory; skipping the braking check; sampling the window without intersecting it with $\|\mathbf{v}\| \leq v_{\max}$. Done when each failure has a figure, a number that explains it and one remedy that fixes it, and you own a small tuning table of the weights — the Week 7 milestone of Appendix A.

Exercise 15.1. (a) Write $g(\mathbf{p}) = 1/\rho(\mathbf{p}) - 1/\rho_0$, so $U_{\text{rep}} = \frac{1}{2}k_{\text{rep}}g^2$ inside the influence region. By the chain rule $\nabla U_{\text{rep}} = k_{\text{rep}}g \nabla g$ and $\nabla g = -\rho^{-2}\nabla\rho$, hence $\mathbf{F}_{\text{rep}} = -\nabla U_{\text{rep}} = k_{\text{rep}}(1/\rho - 1/\rho_0)\rho^{-2}\nabla\rho$. Outside the region both U_{rep} and its gradient are zero, and at $\rho = \rho_0$ the inside expression is zero as well, so the force is continuous. (b) For a disk with center $\mathbf{c}$ and radius r , $\rho = \|\mathbf{p} - \mathbf{c}\| - r$ and $\nabla\rho = (\mathbf{p} - \mathbf{c})/\|\mathbf{p} - \mathbf{c}\|$. For a segment, ρ is the distance to the closest point $\mathbf{s}$ of the segment (Chapter 2) and $\nabla\rho = (\mathbf{p} - \mathbf{s})/\|\mathbf{p} - \mathbf{s}\|$; the gradient is continuous everywhere except on the set of points that have two closest points, which the drone crosses in an instant. (c) With $d = \|\mathbf{p} - \mathbf{p}_g\|$ and $\nabla d = (\mathbf{p} - \mathbf{p}_g)/d$, the product rule gives $\nabla(U_{\text{rep}}d^n) = d^n\nabla U_{\text{rep}} + n d^{n-1}U_{\text{rep}}\nabla d$, so $\mathbf{F}'_{\text{rep}} = d^n\mathbf{F}_{\text{rep}} - \frac{n}{2}k_{\text{rep}}(1/\rho - 1/\rho_0)^2 d^{n-1}(\mathbf{p} - \mathbf{p}_g)/d$. The second term points toward the goal. At the goal $d = 0$ and, for $n \geq 2$, both terms vanish; for $n = 1$ the second term has the magnitude $\frac{1}{2}k_{\text{rep}}(1/\rho - 1/\rho_0)^2$ in every direction of approach, so the force is discontinuous at the goal (a conic pull).

Exercise 15.3. On the axis $y = 4$ with $1 < x < 3$ the force is horizontal by symmetry, with $F_x(x) = (8 - x) - (\frac{1}{3-x} - \frac{1}{2})\frac{1}{(3-x)^2}$. It is 7 at $x = 1$ (the edge of the influence region) and tends to $-\infty$ as $x \rightarrow 3$, so it has a root; bisection gives $x^* = 2.4872$, which is where the simulation of Section 15.7.1 stops. The Hessian of U there has the eigenvalues -2.64 (in y) and 36.96 (in x), computed with central differences of the analytic force: the equilibrium is a saddle, because the repulsion pushes a slightly off-axis drone further off the axis faster than the attraction pulls it back. Only the measure-zero set of starts exactly on the axis is trapped. For the peanut-shaped obstacle of the basin experiment the stopping point (2.676, 4.1) has the eigenvalues 9.87 and 24.89: a genuine local minimum, whose basin contains 179 of the 441 starts.

Exercise 15.5. (a) Without obstacles and without a speed limit the update reads $\mathbf{p}_{k+1} = \mathbf{p}_k - \Delta t k_{\text{att}}(\mathbf{p}_k - \mathbf{p}_g)$, so the error $\mathbf{e}_k = \mathbf{p}_k - \mathbf{p}_g$ obeys $\mathbf{e}_{k+1} = (1 - k_{\text{att}}\Delta t)\mathbf{e}_k$. The factor has modulus below one, and the error shrinks, exactly when $0 < k_{\text{att}}\Delta t < 2$; for $k_{\text{att}}\Delta t < 1$ the

approach is monotone, for $1 < k_{\text{att}}\Delta t < 2$ the sign alternates (overshoot), and for $k_{\text{att}}\Delta t > 2$ the error grows. The self-test of `ch15_potential_fields.py` checks the cases 0.5 and 1.5. (b) Near any minimum $\mathbf{p}^*$ of a smooth U , $\mathbf{F}(\mathbf{p}) \approx -\mathbf{H}(\mathbf{p} - \mathbf{p}^*)$ with the Hessian $\mathbf{H}$, so the error is multiplied by $\mathbf{I} - \Delta t\mathbf{H}$; along an eigenvector with eigenvalue λ the factor is $1 - \Delta t\lambda$, and the condition becomes $\Delta t\lambda_{\text{max}} < 2$. (c) In the passage of width 1.0 the stiffness at the midpoint is 81.0, so $\Delta t = 0.01$ gives 0.81 (stable, monotone) and $\Delta t = 0.05$ gives 4.05 (unstable); the narrower passages have the stiffnesses 204 (width 0.8) and 668 (width 0.6), which grow roughly like the inverse fourth power of the width.

Exercise 15.4. Hint: with the factor d^n the total potential is $U' = \frac{1}{2}k_{\text{att}}d^2 + U_{\text{rep}}d^n \geq 0$ and $U'(\mathbf{p}_g) = 0$, so the goal is the global minimum; with $n \geq 2$ the force there is zero by the formula of [Exercise 15.1\(c\)](#). The self-test checks both facts for the goal (5.4, 4). For $n = 1$ the goal is still the global minimum, but the force does not vanish there: the drone chatters across the goal with an amplitude of one step, $\Delta t v_{\text{max}} = 0.01$. With the chapter's tolerance $\varepsilon_g = 0.05$ the chatter is inside the tolerance and the run reports reached after 508 steps; with `goal_tol=1e-3` the same run is declared stuck after 695 steps, its distance to the goal alternating between 0.0027 and 0.0074. With $n = 2$ the force does vanish at the goal, the approach is smooth, and the run reports reached – after 509 steps with `goal_tol=1e-3` and after 425 with $\varepsilon_g = 0.05$. To see that the correction does not remove local minima elsewhere, keep $n = 2$ and rerun the start (0, 4) of [Section 15.7.1](#): the drone still stops on the axis, at a slightly different x .

Exercise 15.7. (a) The repulsive force is $k_{\text{rep}}(1/\rho - 1/\rho_0)\rho^{-2}\nabla\rho$ inside the influence region and 0 outside; the bracket vanishes at $\rho = \rho_0$, so the force is continuous there, but its derivative in ρ tends to k_{rep}/ρ_0^4 from inside and is 0 outside ([Proposition 15.6](#)). The force is C^0 but not C^1 across the boundary. A finite-difference estimate of the stiffness [Equation \(15.10\)](#) whose two probes straddle $\rho = \rho_0$ therefore averages two different curvatures and reports a value that belongs to neither side; take the probes on one side of the boundary, or keep the step below the distance to it. (b) The detector sees only displacement: it fires whenever the drone moves less than δ in W steps, which happens both at a critical point (the axis start of [Section 15.7.1](#)) and on a long, nearly flat plateau (the same scene with $k_{\text{att}} = 10^{-3}$, where the force is small but never zero). The caller cannot tell them apart from the path alone, and it should not try: in both cases the local layer is not making progress and the global layer must supply a new waypoint.

Exercise 15.2. (a) At $d = d^*$ the quadratic branch has the value $\frac{1}{2}k_{\text{att}}(d^*)^2$ and the conic branch $k_{\text{att}}d^*d^* - \frac{1}{2}k_{\text{att}}(d^*)^2$, the same number; both forces point along $-(\mathbf{p} - \mathbf{p}_g)/d$ and have the magnitude $k_{\text{att}}d^*$ there, so value and gradient agree and U_{att} is C^1 . (b) The clipped command has the magnitude $\min(k_{\text{att}}d, v_{\text{max}})$ and the direction of $-(\mathbf{p} - \mathbf{p}_g)$; the hybrid force has the magnitude $\min(k_{\text{att}}d, k_{\text{att}}d^*)$ and the same direction. The two agree for every $\mathbf{p}$ exactly when $d^* = v_{\text{max}}/k_{\text{att}}$. (c) The solution of $\dot{\mathbf{p}} = -k_{\text{att}}(\mathbf{p} - \mathbf{p}_g)$ is $\mathbf{p}(t) = \mathbf{p}_g + e^{-k_{\text{att}}t}(\mathbf{p}(0) - \mathbf{p}_g)$, whose distance $De^{-k_{\text{att}}t}$ is positive at every finite time: the goal is reached only in the limit, which is why [Algorithm 15.2](#) needs ε_g . The tolerance is met after $t = k_{\text{att}}^{-1} \ln(D/\varepsilon_g)$. In the last row of [Table 15.2](#) this predicts $\ln(1.007/0.05) = 3.00$ time units, that is 300 steps, from $k = 800$; the run takes 299.

Exercise 15.6. (a) On the axis the two vertical components cancel and the horizontal ones add, which gives [Equation \(15.11\)](#); scanning x shows a first (stable) root at $x = 3.163$, where the simulation stops, and a second, unstable root near $x = 3.91$ inside the mouth of the passage. (b) Raising the attraction moves the first root out of the passage. With $k_{\text{rep}} = 1$ and $\rho_0 = 2$ the threshold lies between 2.0 and 2.5: at $k_{\text{att}} = 2$ the drone still stops, at $x = 3.464$, and at

$k_{\text{att}} = 2.5$ it crosses and reaches the goal in 871 steps. (c) The same $k_{\text{att}} = 2.5$ in the scene of [Example 15.5](#) gives a run of 929 steps with a minimum clearance of 0.396 instead of 0.519: the gain that opens the doorway shaves a fifth off the margin everywhere else, which is the tuning dilemma of [Section 15.7.5](#). (d) A waypoint inside the passage replaces the attraction toward the far goal by one toward a point between the disks, where the horizontal repulsions cancel by symmetry; the drone enters, and from (4, 4) the original goal is reached in 599 steps with a clearance of 0.300. The global layer, not the gains, has solved the problem.

Exercise 15.8. Checkpoints. `simulate_swarm` in `ch15_potential_fields.py` already carries separation (the pairwise term of [Equation \(15.13\)](#)) and an optional spring, and it builds the pairwise distance matrix once per step — keep that matrix and read $N_i = \{j \neq i : \|\mathbf{p}_i - \mathbf{p}_j\| \leq \rho_a\}$ off it as a boolean mask instead of looping. Steps: add cohesion as an attractive potential toward the neighbor centroid, $\frac{1}{2}k_c \|\mathbf{p}_i - \bar{\mathbf{p}}\|^2$, so that it enters the same force sum as everything else; apply the velocity update of [Equation \(15.14\)](#) *after* the force integration, with a gain α small enough that $\alpha|N_i| < 1$ for every drone, which is the only stability condition you need; log the standard deviation of the velocities and the radius of the position cloud at every step. Expected behavior: on a graph that starts connected the velocity spread decays roughly geometrically to zero while the position spread settles at a constant and the flock translates rigidly, at a rate set by the algebraic connectivity — [Chapter 23](#) in disguise. For the split, place two tight clusters more than ρ_a apart with velocities pointing away from each other: two components, two limits, a gap that only grows. Keep separation switched on: the minimum pairwise distance stays above 1.0 in the chapter’s swarm run and drops to 0 without it. Classic mistakes: an alignment gain not scaled by the neighbor count, so a drone with eight neighbors overshoots; recomputing neighborhoods from stale positions; reading convergence off the mean velocity, which [Equation \(15.14\)](#) conserves. Done when one figure shows the velocity spread going to zero on a connected graph and holding two levels on a disconnected one, and you can name the eigenvalue that sets the rate.

Exercise 15.9. Checkpoints; read `ch15_potential_fields.py` only when stuck. The functions to compare against afterward are `total_force` ([Algorithm 15.1](#)), `simulate_many` ([Algorithm 15.2](#)), `numerical_force`, `stiffness`, `basin_experiment` and `follow_waypoints`. One `ApfParams` record holds the gains, ρ_0 , Δt and v_{max} ; obstacles are objects with a single distance method, so a segment is a drop-in replacement for a disk. Steps: write the potentials first and the forces second, then assert that your force matches a central difference of your potential to about 10^{-6} at a hundred random points — this one test catches most sign and chain-rule errors; add the clipped Euler step, a step budget and a stuck detector; build the four failure scenes; sweep the basin; add the A^* layer last. Expected numbers: the local minimum traps the drone at $x = 2.487$ after 354 steps, exactly the root of F_x on the axis; GNRON stops 0.568 m short and the exponent $n = 2$ of [Equation \(15.9\)](#) arrives in 425 steps; in the passage the stiffness of [Equation \(15.10\)](#) is 667.7 for a 0.6 m gap and 81.0 for a 1.0 m gap, so $\Delta t = 0.05$ gives $\Delta t \lambda_{\text{max}} \approx 33$ and a crash while $\Delta t = 0.01$ merely oscillates — the explaining number is $\Delta t \lambda_{\text{max}}$, not the gap. On the 21×21 grid of starts of [Section 15.8.1](#), 59.4 % arrive plain and 87.5 % with random-walk escapes, at about 40 % more steps. Classic mistakes: an unclamped repulsion, so one step crosses the obstacle; using the center distance instead of the clearance of [Equation \(15.1\)](#); calling every slowdown a local minimum. Done when each failure has a figure and its number, and your arrival fractions land within a few points of the chapter’s — the Week 7 milestone ([Appendix A](#)).

Exercise 16.3. (a) Take the wall $[4.00, 4.04] \times [0, 10]$ and the horizontal segment from $\mathbf{a} = (3.995, 5)$ to $\mathbf{b} = (4.045, 5)$. Its length is $0.05 = \delta$, so the check uses only the two endpoints; both lie outside the wall ($3.995 < 4.00$ and $4.045 > 4.04$) and the segment crosses it. With the

inflation by $\delta/2 = 0.025$ the point $\mathbf{a}$, at distance 0.005 from the wall, is rejected. (b) Let $\mathbf{o}$ be an obstacle point and write the squared distance from $\mathbf{o}$ to the point at arc length s along a segment as $g(s) = (s - s^*)^2 + h_*^2$, where h_* is the distance from $\mathbf{o}$ to the line and s^* the arc length of its projection. If some point $\mathbf{q}$ of the segment, between the test points at $s = 0$ and $s = \delta$, has $\text{dist}(\mathbf{q}, \mathbf{o}) = h \leq r$, then $h_* \leq h$ and one of the two test points has $(s - s^*)^2 \leq \delta^2/4$ (the one on the far side of s^* , or either one if s^* is outside $[0, \delta]$), so its distance to $\mathbf{o}$ is at most $\sqrt{h^2 + \delta^2/4} \leq \sqrt{r^2 + \delta^2/4}$ and it fails a test with that margin. Hence the margin $\sqrt{r^2 + \delta^2/4}$, which is smaller than $r + \delta/2$ for $r > 0$, also guarantees a collision-free segment. It is tight: a point obstacle at perpendicular distance exactly r from the midpoint between two test points is at distance $\sqrt{r^2 + \delta^2/4}$ from both of them, so any smaller margin accepts the segment although the robot touches the obstacle at the midpoint. (c) The ball of radius d_0 around $\mathbf{a}$ contains no point within r of an obstacle, so every segment point closer than d_0 to $\mathbf{a}$ is free without a test. The adaptive check therefore jumps to the point at distance d_0 along the segment, computes its own free distance d_1 , jumps by d_1 , and so on, until it passes $\mathbf{b}$ (the segment is free) or until a free distance is ≤ 0 (collision). In open space a few jumps cover the whole segment; near an obstacle the jumps shrink, and an implementation stops and reports a collision once a jump falls below a minimum length, which keeps the check conservative and finite. This is the “conservative advancement” scheme used by many collision libraries.

Exercise 16.4. (a) Call iteration i a success if its sample lands in the ball $B_{K_{i-1}+1}$, the one just beyond the stage reached so far. The ball is determined by the past and the sample is independent of the past and uniform with probability $1 - p_{\text{goal}}$, so the conditional success probability is at least p whatever happened before; by a standard coupling the number of successes S_n in n iterations is stochastically at least $\text{Bin}(n, p)$. Every success raises the stage by at least one (the claim in the proof of [Theorem 16.5](#)), so $T > n$ implies $S_n < m$ and $\mathbb{P}(T > n) \leq \mathbb{P}(\text{Bin}(n, p) < m)$. For $n > 2m/p$ the mean np exceeds $2m$, and the Chernoff bound $\mathbb{P}(\text{Bin}(n, p) \leq (1 - \alpha)np) \leq \exp(-\alpha^2 np/2)$ with $\alpha = 1/2$ gives $\mathbb{P}(T > n) \leq \exp(-np/8)$, an exponential decay in n . (b) Take [Example 16.4](#) with $p_{\text{goal}} = 1$. Every sample is the goal, so every iteration extends the vertex nearest to $(9, 9)$ by a step of η along the straight line toward it; after four steps the segment enters the wall $[3, 5] \times [0, 6]$, is rejected, and the same rejected extension is repeated for ever, although a path exists. In the proof, the probability that a sample lands in a ball off the straight line is $p = (1 - p_{\text{goal}}) \text{vol}(B)/\text{vol}(\mathcal{X}) = 0$. (c) The proof uses $\eta > 0$ once, to choose $\nu \leq \eta/3$ so that a sample within 3ν of the tree is reached exactly by STEER. With $\eta_i = 1/i$ the balls must shrink with i , their volumes are proportional to i^{-d} , and $\sum_i i^{-d} < \infty$ for $d \geq 2$: the expected total number of samples that ever land in the current target ball is finite, so with positive probability the tree stops advancing after finitely many stages and the algorithm is not probabilistically complete. (The tree may still reach the goal by other routes, but the argument no longer proves that it does.)

Exercise 16.5. The bisector of $\mathbf{v}_0$ and $\mathbf{v}_1$ is the line $x = 0.5$ and that of $\mathbf{v}_1$ and $\mathbf{v}_2$ the line $x = 1.5$, so the Voronoi regions inside $\mathcal{X} = [-1, 3] \times [-2, 2]$ (area 16) are the strips $-1 \leq x \leq 0.5$ (area 6), $0.5 \leq x \leq 1.5$ (area 4) and $1.5 \leq x \leq 3$ (area 6). The next sample selects $\mathbf{v}_0$ with probability $6/16 = 0.375$, $\mathbf{v}_1$ with 0.25 and $\mathbf{v}_2$ with 0.375: the interior vertex is the least likely, and the root is a tip of the chain too. For a chain of $k + 1$ vertices spaced η apart and centered in a square of side $L \gg k\eta$, the bisectors of consecutive vertices are parallel lines η apart, so every interior vertex owns a strip of width η and area at most ηL , probability at most η/L ; the two tips share the rest, $(L^2 - (k - 1)\eta L)/2$ each, so each tip is chosen with probability $\frac{1}{2} - (k - 1)\eta/(2L)$, close to $\frac{1}{2}$. The chain therefore grows almost only at its two ends, and mostly at the end that faces the larger empty region. Centering matters: if the chain starts near a corner and points into the map, as in [Example 16.4](#), the region of the single tip is the part of the square beyond the last bisector, which is almost all of it, so that tip is selected

with probability close to 1 rather than $1/2$.

Exercise 16.6. Expect a U-shaped curve: with $p_{\text{goal}} = 0$ the tree spends its samples on the whole map, while with a large bias almost every sample is the goal, so the vertex nearest the goal is extended again and again into the wall and the tree stops growing elsewhere (Section 16.8.4); the median is smallest in between, near $p_{\text{goal}} = 0.05\text{--}0.1$, matching Table 16.3, where 0.5 is worse than no bias at all. Moving the goal to $(9, 1)$ removes one of the two walls from the straight line between start and goal, so greedy steps toward the goal are blocked over a shorter stretch and recover sooner: the penalty for a large bias shrinks and the best bias moves upward.

Exercise 16.7. (a) The replaced part of the path is a polyline from $\mathbf{a}$ to $\mathbf{b}$, and the triangle inequality (applied along it) gives its length as at least $\|\mathbf{a} - \mathbf{b}\|$, the length of the new segment; the rest of the path is unchanged. The new path is collision-free because the untouched segments were, and the new segment was accepted by COLLISIONFREE. (b) PRUNE outputs only input vertices, in strictly increasing index order, starting at $\mathbf{x}_0$ and ending at $\mathbf{x}_k$ (the jump to $j = i + 1$ is always free), so the output has at most $k + 1$ vertices. Example: $\mathbf{x}_0 = (0, 0)$, $\mathbf{x}_1 = (1, 1.2)$, $\mathbf{x}_2 = (2, 0)$, $\mathbf{x}_3 = (3, 3)$ with a disk obstacle of radius 0.05 at $(0.5, 0.5)$. The disk blocks $[\mathbf{x}_0, \mathbf{x}_3]$ (the diagonal passes through its center) and nothing else, since it is at distance 0.5 from $[\mathbf{x}_0, \mathbf{x}_2]$, about 0.064 from $[\mathbf{x}_0, \mathbf{x}_1]$ and further from the other segments. Greedy pruning from $\mathbf{x}_0$ tries $\mathbf{x}_3$ (blocked) and jumps to $\mathbf{x}_2$, then to $\mathbf{x}_3$: length $2 + \sqrt{10} \approx 5.16$. The path $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_3$ is also free and has length $\sqrt{2.44} + \sqrt{7.24} \approx 4.25$. (c) Every point of the initial path $(1, 5), (1, 9.75), (9, 9.75), (9, 5)$ has $y \geq 5$. A shortcut replaces a sub-path by a segment between two of its points, and every point of that segment is a convex combination of two points with $y \geq 5$, so the invariant $y \geq 5$ survives every shortcut. Passing below the wall $[4, 6] \times [2, 9.5]$ requires a point with $4 \leq x \leq 6$ and $y < 2$, which never appears; the paths obtained by shortcutting all cross the wall's column above it and are at least as long as $(1, 5), (4, 9.5), (6, 9.5), (9, 5)$, that is $2\sqrt{9 + 20.25} + 2 \approx 12.82$, whereas the route below the wall through $(4, 2)$ and $(6, 2)$ has length $2\sqrt{9 + 9} + 2 \approx 10.49$.

Exercise 16.1. The volume is $200 \times 200 \times 60$ m, so at 0.25 m the lattice has $800 \times 800 \times 240 = 1.536 \times 10^8$ cells, 153.6 MB at one byte per cell, and every interior cell has $3^3 - 1 = 26$ neighbors in the 26-connected lattice. Adding the velocity, ± 4 m/s per axis in steps of 0.25 m/s, gives 33 values per axis and $33^3 = 35\,937$ velocity cells per position cell: $1.536 \times 10^8 \times 35\,937 \approx 5.5 \times 10^{12}$ states, 5.5 TB at one byte each, and $3^6 - 1 = 728$ neighbors if all axes may change at once (in practice only the accelerations that respect the dynamics are edges, which is fewer but still a lattice one cannot store). An RRT with 20 000 vertices in three dimensions needs $20\,000 \times (3 \times 8 + 4) = 560\,000$ bytes, about 0.56 MB: seven orders of magnitude less than the velocity-augmented grid, because it stores only the states it actually visited.

Exercise 16.2. Iteration 11, $\mathbf{x}_{\text{rand}} = (9.70, 5.16)$: the distances to vertices 0 to 6 are smallest for vertex 6 at $(2.86, 2.92)$, $\|\mathbf{x}_{\text{rand}} - \mathbf{x}_6\| = 7.197$ (vertex 5 gives 7.424), so $\mathbf{x}_{\text{near}} = \mathbf{x}_6$. The unit vector toward the sample is $(6.84, 2.24)/7.197 = (0.9504, 0.3112)$, so $\mathbf{x}_{\text{new}} = \mathbf{x}_6 + 0.5(0.9504, 0.3112) = (3.335, 3.076)$, which is inside the wall $[3, 5] \times [0, 6]$: the extension is rejected. Iteration 12, $\mathbf{x}_{\text{rand}} = (6.23, 7.77)$: again vertex 6 is nearest (5.906 against 6.355 for vertex 5), the unit vector is $(0.5706, 0.8212)$ and $\mathbf{x}_{\text{new}} = (3.145, 3.331)$, again inside the wall and rejected. These two lines are computed from the two-decimal coordinates of Table 16.2; the trace printed by `ch16_rrt.py`, which carries the full precision of $\mathbf{x}_6$, gives $(3.332, 3.073)$ and $(3.143, 3.327)$ – the same conclusion, the difference being the rounding of the tabulated vertex. The Voronoi picture explains the pattern: the tree is a chain that stops at $x \approx 2.9$, so the bisectors between consecutive vertices are almost vertical lines and the region of the

tip, vertex 6, contains the whole upper right of the map, while vertex 5 owns the lower right. Every sample with $x > 3$, about 70 % of the map, therefore selects vertex 5 or 6, and every step from them toward such a sample crosses the wall.

Exercise 16.8. Checkpoints. The pieces exist: `scene_3d` builds the box field, `rrt` and `rrt_connect` plan in it, `shortcut` and `prune smooth` (Algorithm 16.4), `path_is_free` re-checks a path at ten times the resolution, and `grid_astar_reference` in `ch17_rrt_star.py` is a ready 26-connected grid A^* with string pulling. Keep the tree as three parallel arrays (positions, parents, count) rather than linked objects, and the world as arrays of box corners so that a whole segment sample is tested at once. Steps: fix the benchmark protocol *before* the first run, with the same 30 start–goal pairs, the same seeds and one row per (query, planner); report path length relative to the best path any method found on that query, never against the straight line; give every planner the same budget and smooth every path, grid paths included. Expected numbers on the chapter’s $10 \times 10 \times 10$ field (straight line 15.59 m): RRT solves near iteration 320 with about 160 vertices at length 24.5, which shortcutting brings to 20.4; RRT-Connect solves near iteration 120 with 59 vertices. Grid A^* gives 19.90 at $h = 1$ m (1331 cells), 17.44 at 0.5 m and 17.27 at 0.25 m (68921 cells, 1.6 s): on a small map the grid wins on quality and the sampler on memory. The last part makes the crossover visible — $200 \times 200 \times 60$ m at 0.25 m is 1.5×10^8 cells, which no Python grid search will hold, while the samplers do not notice. Classic mistakes: checking the new vertex but not the segment to it; a linear-scan nearest neighbor (0.03 ms at a thousand vertices, 2.9 ms at a hundred thousand, called every iteration); averaging over queries one planner failed. Done when your length-against-time plot carries all three methods and one paragraph says, with your numbers, when you would use graph search and when sampling — the Week 8 milestone of Appendix A.

Exercise 17.1. The nearest vertex is $\mathbf{v}_8 = (4.4, 3.0)$ at distance $0.721 < \eta$, so $\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{rand}} = (5.0, 2.6)$, and the segment $\mathbf{v}_8\mathbf{x}_{\text{new}}$ is free. Near with $r_n = 2$ returns $\{\mathbf{v}_4, \mathbf{v}_7, \mathbf{v}_8, \mathbf{v}_9\}$ at distances 1.393, 1.772, 0.721 and 1.217. CHOOSEPARENT sorts the candidates by $\text{Cost}(\mathbf{v}) + \|\mathbf{x}_{\text{new}} - \mathbf{v}\|$: $\mathbf{v}_7$ ($4.043 + 1.772 = 5.815$), $\mathbf{v}_4$ (6.141), $\mathbf{v}_8$ (7.173), $\mathbf{v}_9$ (8.799). The first candidate $\mathbf{v}_7$ has a free segment, so it is chosen after a single collision check and $\mathbf{x}_{\text{new}}$ becomes $\mathbf{v}_{10}$ with cost 5.815; the nearest vertex $\mathbf{v}_8$ would have given 7.173. REWIRE tests the others through $\mathbf{v}_{10}$: $\mathbf{v}_4$ would cost $5.815 + 1.393 = 7.208 > 4.748$ and $\mathbf{v}_8$ would cost $6.536 > 6.451$, both rejected by the improvement test without a collision check; $\mathbf{v}_9$ would cost $5.815 + 1.217 = 7.032 < 7.583$ and its segment is free, so its parent changes from $\mathbf{v}_8$ to $\mathbf{v}_{10}$ and its cost drops to 7.032. It has no children, so nothing else changes. `tiny_example(x_rand=(5.0, 2.6))` returns exactly these values.

Exercise 17.2. (a) The cap stops being active when $13.3\sqrt{\log n/n} = 1$, that is at $n = 1264$ vertices. At $n = 10^4$ the expected size of the NEAR set is $\zeta_2\gamma^2 \log n / \mu(\mathcal{X}_{\text{free}}) = \pi \cdot 13.3^2 \cdot \log(10^4)/76 = 67$ vertices. (b) With $d = 3$ and $\zeta_3 = 4\pi/3$, $\gamma_{\text{RRT}^*}^* = 2(4/3)^{1/3}(716/\zeta_3)^{1/3} = 12.22$ for the true free volume and 13.66 for the box volume $\mu(\mathcal{X}) = 1000$, so the $\gamma = 15$ of the code is admissible for either. The cap $\eta = 1.5$ stops being active when $15(\log n/n)^{1/3} = 1.5$, that is $\log n/n = 10^{-3}$ and $n \approx 9119$ vertices. (c) A ball of radius $r_n = \gamma \log n/n$ has area $\zeta_2\gamma^2(\log n/n)^2$, so the expected number of vertices in it is $n \cdot \zeta_2\gamma^2(\log n/n)^2 / \mu(\mathcal{X}_{\text{free}}) = \zeta_2\gamma^2(\log n)^2 / (n\mu(\mathcal{X}_{\text{free}})) \rightarrow 0$: the NEAR set is empty from some n on, RRT* degenerates to RRT and stops improving. The exponent $1/d$ is what keeps the count growing like $\log n$.

Exercise 17.3. (a) Put the foci at $\mathbf{f}_{\pm} = (\pm c_{\min}/2, 0, \dots, 0)$ and write $\mathbf{y} = (y_1, \mathbf{y}_{\perp})$. The condition $\|\mathbf{y} - \mathbf{f}_{-}\| + \|\mathbf{y} - \mathbf{f}_{+}\| \leq c_{\text{best}}$ depends on $\mathbf{y}$ only through y_1 and $\|\mathbf{y}_{\perp}\|$, so the set is a body of revolution about the first axis, and in the plane spanned by $\mathbf{e}_1$ and any perpendicular direction it is the classical ellipse with foci at distance $c_{\min}/2$ from the center and string

length c_{best} : squaring twice gives $y_1^2/a^2 + \|\mathbf{y}_\perp\|^2/b^2 \leq 1$ with $a = c_{\text{best}}/2$ and $b^2 = a^2 - c_{\text{min}}^2/4$, that is $b = \frac{1}{2}\sqrt{c_{\text{best}}^2 - c_{\text{min}}^2}$. This is the image of the unit ball under $\mathbf{L} = \text{diag}(a, b, \dots, b)$. The world frame has the focal axis along $\mathbf{a}_1$ and the center at $\mathbf{x}_{\text{center}}$, so $\mathbf{x} = \mathbf{C}\mathbf{y} + \mathbf{x}_{\text{center}}$ with any rotation $\mathbf{C}\mathbf{e}_1 = \mathbf{a}_1$; rotations preserve distances, so the foci land on $\mathbf{x}_{\text{init}}$ and $\mathbf{x}_{\text{goal}}$. (b) If $\mathbf{X}$ has density p on B and $\mathbf{Y} = \mathbf{A}\mathbf{X} + \mathbf{t}$, the change-of-variables formula gives $\mathbf{Y}$ the density $p(\mathbf{A}^{-1}(\mathbf{y} - \mathbf{t}))/|\det \mathbf{A}|$ on $\mathbf{A}B + \mathbf{t}$. For $p \equiv 1/\mu(B)$ this is the constant $1/(\mu(B)|\det \mathbf{A}|) = 1/\mu(\mathbf{A}B + \mathbf{t})$: uniform. With $\mathbf{A} = \mathbf{C}\mathbf{L}$, $|\det \mathbf{A}| = ab^{d-1}$. (c) For $\mathbf{X}$ uniform in the unit ball, $\mathbb{P}(\|\mathbf{X}\| \leq \rho) = \rho^d$ (volume ratio) and the direction is independent of the radius and uniform on the sphere. With $w \sim U(0, 1)$, $\mathbb{P}(w^{1/d} \leq \rho) = \mathbb{P}(w \leq \rho^d) = \rho^d$, and $\mathbf{u}/\|\mathbf{u}\|$ is uniform on the sphere because the standard Gaussian density $\propto e^{-\|\mathbf{u}\|^2/2}$ is invariant under rotations; the two factors are independent, so the product is uniform in the ball. (d) $\mathbf{C} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$ is a product of orthogonal matrices, hence orthogonal, and $\det \mathbf{C} = \det \mathbf{U} \cdot (\det \mathbf{U} \det \mathbf{V}) \cdot \det \mathbf{V} = (\det \mathbf{U})^2(\det \mathbf{V})^2 = 1$. The matrix $\mathbf{a}_1\mathbf{e}_1^\top$ has rank one, so its SVD has $\sigma_1 = 1$, $\mathbf{u}_1 = s\mathbf{a}_1$ and $\mathbf{v}_1 = s\mathbf{e}_1$ for a common sign s , and the remaining columns of $\mathbf{V}$ are orthogonal to $\mathbf{e}_1$; thus $\mathbf{V}^\top\mathbf{e}_1 = s\mathbf{e}_1$, $\mathbf{D}$ leaves $\mathbf{e}_1$ unchanged ($d \geq 2$), and $\mathbf{C}\mathbf{e}_1 = s\mathbf{U}\mathbf{e}_1 = s\mathbf{u}_1 = s^2\mathbf{a}_1 = \mathbf{a}_1$. In 2D the only rotations are $\begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}$, and the first column must be $\mathbf{a}_1 = (\cos \phi, \sin \phi)$; the code returns $\begin{bmatrix} 0.707 & -0.707 \\ 0.707 & 0.707 \end{bmatrix}$ for the diagonal start-goal pair of the worked example.

Exercise 17.4. *No cycles.* Suppose REWIRE considers making $\mathbf{x}_{\text{new}}$ the parent of $\mathbf{v}$ while $\mathbf{x}_{\text{new}}$ is a descendant of $\mathbf{v}$, that is, the tree path from $\mathbf{x}_{\text{init}}$ to $\mathbf{x}_{\text{new}}$ passes through $\mathbf{v}$. Costs add along tree paths, so $\text{Cost}(\mathbf{x}_{\text{new}}) = \text{Cost}(\mathbf{v}) + \ell$ where ℓ is the length of the tree path from $\mathbf{v}$ to $\mathbf{x}_{\text{new}}$, and $\ell \geq \|\mathbf{x}_{\text{new}} - \mathbf{v}\|$ by the triangle inequality. The candidate cost is therefore $\text{Cost}(\mathbf{x}_{\text{new}}) + \|\mathbf{v} - \mathbf{x}_{\text{new}}\| \geq \text{Cost}(\mathbf{v}) + 2\|\mathbf{v} - \mathbf{x}_{\text{new}}\| \geq \text{Cost}(\mathbf{v})$, and the strict test $c < \text{Cost}(\mathbf{v})$ fails. *Example.* Root $\mathbf{v}_0 = (0, 0)$ with child $\mathbf{a} = (1, 0)$ and grandchild $\mathbf{p} = (2, 0)$, so $\text{Cost}(\mathbf{a}) = 1$, $\text{Cost}(\mathbf{p}) = 2$; obstacle $[1.2, 1.4] \times [0.3, 0.5]$; sample $\mathbf{x}_{\text{new}} = (1.6, 0.8)$; $r_n = 1$, so $\text{Near} = \{\mathbf{a}, \mathbf{p}\}$ (distances 1.0 and 0.894). CHOOSEPARENT tests $\mathbf{a}$ first ($1 + 1.0 = 2.0$) but the segment crosses the obstacle; $\mathbf{p}$ ($2 + 0.894 = 2.894$) is free, so $\mathbf{x}_{\text{new}}$ hangs under $\mathbf{p}$ with cost 2.894. REWIRE then considers $\mathbf{a}$: the cost via $\mathbf{x}_{\text{new}}$ would be $2.894 + 1.0 = 3.894 > 1$, so the candidate that would have closed the cycle $\mathbf{a} \rightarrow \mathbf{p} \rightarrow \mathbf{x}_{\text{new}} \rightarrow \mathbf{a}$ is rejected before any collision check. *Order.* Let $\mathbf{v}$ be an ancestor of $\mathbf{u}$ and both be candidates with $c_{\mathbf{v}} = \text{Cost}(\mathbf{x}_{\text{new}}) + \|\mathbf{v} - \mathbf{x}_{\text{new}}\| < \text{Cost}(\mathbf{v})$ and $c_{\mathbf{u}} = \text{Cost}(\mathbf{x}_{\text{new}}) + \|\mathbf{u} - \mathbf{x}_{\text{new}}\| < \text{Cost}(\mathbf{u})$. If $\mathbf{u}$ is processed first, both are rewired and both end up as children of $\mathbf{x}_{\text{new}}$. If $\mathbf{v}$ is processed first, $\text{Cost}(\mathbf{u})$ drops to $c_{\mathbf{v}} + \ell(\mathbf{v}, \mathbf{u})$ where $\ell(\mathbf{v}, \mathbf{u}) \geq \|\mathbf{u} - \mathbf{v}\|$ is the tree distance; but $c_{\mathbf{u}} \leq \text{Cost}(\mathbf{x}_{\text{new}}) + \|\mathbf{v} - \mathbf{x}_{\text{new}}\| + \|\mathbf{u} - \mathbf{v}\| \leq c_{\mathbf{v}} + \ell(\mathbf{v}, \mathbf{u})$, with equality only when $\mathbf{x}_{\text{new}}, \mathbf{v}, \mathbf{u}$ are collinear in that order, so $\mathbf{u}$ is still rewired and the final tree is the same. Candidates that are not rewired are unaffected by the others, because their costs do not change. The work differs: processing $\mathbf{v}$ first propagates through $\mathbf{u}$'s subtree twice. Processing candidates in decreasing order of Cost (descendants before ancestors) touches every subtree once.

Exercise 17.6. (a) With $c_{\text{min}} = 11.31$ and Equation (17.5), the ellipse of the 30×30 field has area $\pi \cdot \frac{c_{\text{best}}}{2} \cdot \frac{\sqrt{c_{\text{best}}^2 - c_{\text{min}}^2}}{2}$: 410 at $c_{\text{best}} = 24.3$ (45.6 % of 900), 231 at 19.1 (25.7 %) and 91 at 14.0 (10.1 %). Rejection sampling would need $1/0.456 = 2.2$, 3.9 and 9.9 uniform draws per accepted sample. (b) In 3D the volume is $\frac{4\pi}{3}r_1r_2^2$ with $r_1 = c_{\text{best}}/2$ and $r_2 = \frac{1}{2}\sqrt{c_{\text{best}}^2 - 15.59^2}$: 3768 at 23.45 and 2217 at 21.07, larger than the box of volume 1000, so the informed set is (nearly) the whole box—Monte Carlo with 2×10^6 points in $[0, 10]^3$ puts 100 % and 98.4 % of it inside, that is 1.00 and 1.02 draws per accepted sample—and the code samples the box and rejects by $\hat{f}$; 410 at 17.0 and 109 at 16.0, that is 41 % and 10.9 % of the box volume. These are volumes of the ellipsoid; the ellipsoid of $c_{\text{best}} = 17.0$ still pokes out of the box, so only 38.7 % of the box lies inside the informed set and rejection sampling of the box needs 2.6

draws per accepted sample, while the ellipsoid of 16.0 fits inside and needs 9.2. The conjugate axes enter with the power $d - 1$, which is why the fraction falls faster in 3D. (c) The map is inside the ellipse as long as its farthest corners are, and the corners (10, 0) and (0, 10) have $\hat{f} = 2\sqrt{81 + 1} = 18.11$ (the corners (0, 0) and (10, 10) have $\hat{f} = 14.14$). Below $c_{\text{best}} = 18.11$ the corners drop out, but the excluded area is tiny; at $c_{\text{best}} = 17.32$ the code rejected 8 of 2657 draws.

Exercise 17.7. Checkpoints; read `ch17_rrt_star.py` only when stuck, but copy its shape. `StarTree` keeps five parallel arrays — positions, parents, edge lengths, costs and children lists — and that last one is the trick: without children you cannot propagate a rewire, and a tree whose costs silently drift is the classic bug of this chapter. `connection_radius` implements Equation (17.1) and `InformedSampler` the transform of Equation (17.6). Steps: add costs to your RRT of Chapter 16 and check them against a walk to the root; add NEAR and CHOOSEPARENT, sorting the candidates by $\text{Cost}(\mathbf{v}) + \|\mathbf{x}_{\text{new}} - \mathbf{v}\|$ and collision-checking in that order, stopping at the first free one; add REWIRE with subtree propagation; add informed sampling last; write the self-test of (d) before the experiments. Expected numbers: on the hand example the sorted candidate costs are 4.578 (blocked), 5.264, 5.440, 5.692, 7.436, so the parent is $\mathbf{v}_7$ at 5.264, and the rewire lowers $\mathbf{v}_8$ from 6.451 to 6.249 and $\mathbf{v}_9$ from 7.583 to 7.380. On the 2D map with seed 1 the first solution costs 21.00 at iteration 351 and improves to 18.21 and 17.32 at 1000 and 3000 iterations, against an optimum of 16.91 and a string-pulled grid path of 17.23 — inside the 5% the self-test asks — while plain RRT returns 25.72 and never improves. In the 3D field with $\eta = 1.5$ expect about 21.1 after 1500 iterations, still some 20% above the 17.27 that grid A^* at $h = 0.25$ m reaches in under two seconds; informed sampling helps only where the ellipsoid shrinks. Classic mistakes: rewiring without updating descendants; a radius that never shrinks because the cap is applied the wrong way round; comparing a raw RRT* path with a string-pulled grid path. Done when the self-test passes and your convergence plot and grid table support the one-paragraph rule of the Week 8 milestone (Appendix A).

Exercise 18.1. Predict:

$$\begin{aligned}\hat{\mathbf{x}}_2^- &= \mathbf{F}\hat{\mathbf{x}}_1 = (1.195 + 0.859, 0.276 + 0.143, 0.859, 0.143) \\ &= (2.054, 0.419, 0.859, 0.143), \\ a^- &= 0.2226 + 2 \cdot 0.1150 + 0.6172 + 0.0333 = 1.1031, \\ b^- &= 0.1150 + 0.6172 + 0.05 = 0.7822, \quad c^- = 0.6172 + 0.1 = 0.7172.\end{aligned}$$

Update: $S = 1.1031 + 0.25 = 1.3531$, $k_p = 1.1031/1.3531 = 0.8152$, $k_v = 0.7822/1.3531 = 0.5781$; $\mathbf{y}_2 = (1.71 - 2.054, 0.92 - 0.419) = (-0.344, 0.501)$; $\hat{\mathbf{x}}_2 = (2.054 - 0.8152 \cdot 0.344, 0.419 + 0.8152 \cdot 0.501, 0.859 - 0.5781 \cdot 0.344, 0.143 + 0.5781 \cdot 0.501) = (1.774, 0.827, 0.660, 0.433)$; $a = (1 - 0.8152) \cdot 1.1031 = 0.2038$, $b = 0.1848 \cdot 0.7822 = 0.1445$, $c = 0.7172 - 0.5781 \cdot 0.7822 = 0.2650$. All agree with Table 18.1 to the rounding of the hand computation. Step 3 prediction: $\hat{\mathbf{x}}_3^- = (1.774 + 0.660, 0.827 + 0.433, 0.660, 0.433) = (2.434, 1.260, 0.660, 0.433)$. Its x -component is below the true 3 because the measurement $\mathbf{z}_2$ was 0.29 m short of the true position and the filter, still trusting the sensor heavily ($k_p = 0.82$), followed it, and because the velocity estimate 0.66 m/s is still well below the true 1 m/s: the velocity is only seen through position differences and needs several steps to converge (it is 1.13 after step 5).

Exercise 18.3. (a) $\mathbf{P}(\mathbf{K}) = \mathbf{P}^- - \mathbf{KHP}^- - \mathbf{P}^- \mathbf{H}^\top \mathbf{K}^\top + \mathbf{KSK}^\top$ with $\mathbf{S} = \mathbf{HP}^- \mathbf{H}^\top + \mathbf{R}$. Taking the trace and differentiating with the two rules, $\partial \text{tr} \mathbf{P}(\mathbf{K}) / \partial \mathbf{K} = -2\mathbf{P}^- \mathbf{H}^\top + 2\mathbf{KS}$, which vanishes for $\mathbf{K}^* = \mathbf{P}^- \mathbf{H}^\top \mathbf{S}^{-1}$. (b) By Equation (18.19), $\mathbf{P}(\mathbf{K}) - \mathbf{P}(\mathbf{K}^*) = (\mathbf{K} - \mathbf{K}^*)\mathbf{S}(\mathbf{K} - \mathbf{K}^*)^\top$ is positive semidefinite for every $\mathbf{K}$, so every diagonal entry of $\mathbf{P}(\mathbf{K})$ is at least the corresponding

entry of $\mathbf{P}(\mathbf{K}^*)$, and the trace is minimized; a stationary point of a function bounded below by its value there is a global minimum. (c) At the optimum $\mathbf{K}^*\mathbf{S} = \mathbf{P}^{-\mathbf{H}^T}$, so $\mathbf{K}^*\mathbf{S}\mathbf{K}^{*\mathbf{T}} = \mathbf{P}^{-\mathbf{H}^T}\mathbf{K}^{*\mathbf{T}} = (\mathbf{K}^*\mathbf{H}\mathbf{P}^-)^{\mathbf{T}} = \mathbf{K}^*\mathbf{H}\mathbf{P}^-$, the last step because $\mathbf{K}^*\mathbf{H}\mathbf{P}^- = \mathbf{P}^{-\mathbf{H}^T}\mathbf{S}^{-1}\mathbf{H}\mathbf{P}^-$ is symmetric. Substituting into the expansion of (a), the two middle terms and the last combine to $-\mathbf{K}^*\mathbf{H}\mathbf{P}^-$, giving $(\mathbf{I} - \mathbf{K}^*\mathbf{H})\mathbf{P}^-$, and $\mathbf{P}^- - \mathbf{K}^*\mathbf{S}\mathbf{K}^{*\mathbf{T}}$ is the same matrix.

Exercise 18.4. (a) A unit impulse of jerk at time τ raises the acceleration by 1 from τ on, hence the velocity at Δt by $\Delta t - \tau$ and the position by $(\Delta t - \tau)^2/2$; superposing gives the stated integral. With $s = \Delta t - \tau$ and $\mathbb{E}[j(\tau)j(\tau')] = q\delta(\tau - \tau')$, $\mathbf{Q} = q \int_0^{\Delta t} (s^2/2, s, 1)^{\mathbf{T}} (s^2/2, s, 1) ds$, whose entries are $q \int_0^{\Delta t} s^4/4 = q\Delta t^5/20$, $q \int s^3/2 = q\Delta t^4/8$, $q \int s^2/2 = q\Delta t^3/6$, $q \int s^2 = q\Delta t^3/3$, $q \int s = q\Delta t^2/2$ and $q\Delta t$. (b) Direct multiplication: with $\mathbf{F} = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix}$ and $\mathbf{Q}(\Delta t) = q \begin{pmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{pmatrix}$, $\mathbf{F}\mathbf{Q}\mathbf{F}^{\mathbf{T}} = q \begin{pmatrix} \Delta t^3/3 + \Delta t^3 + \Delta t^3 & \Delta t^2/2 + \Delta t^2 \\ \Delta t^2/2 + \Delta t^2 & \Delta t \end{pmatrix}$, and adding $\mathbf{Q}(\Delta t)$ gives $q \begin{pmatrix} 8\Delta t^3/3 & 2\Delta t^2 \\ 2\Delta t^2 & 2\Delta t \end{pmatrix} = \mathbf{Q}(2\Delta t)$. For the induction, note that the integral form $\mathbf{Q}(T) = \int_0^T e^{\mathbf{A}s} \mathbf{G} q \mathbf{G}^{\mathbf{T}} e^{\mathbf{A}^{\mathbf{T}}s} ds$ splits at $s = T - \Delta t$ into $\mathbf{Q}(T - \Delta t)$ transported by $\mathbf{F}(\Delta t)$ plus $\mathbf{Q}(\Delta t)$; equivalently, the sum over $i < j$ is a Riemann sum of the same integral. This is why q is the invariant parameter and $\mathbf{Q}$ is not. (c) `process_noise_numeric(A, [0,0,1], q, dt)` with the 3×3 nilpotent $\mathbf{A}$ matches `ca_model` to 10^{-6} ; the self-test of `ch18_kalman.py` contains both checks.

Exercise 18.5. (a) $\hat{\mathbf{x}}_0$ is the linear map $\begin{pmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{I}/\Delta t & \mathbf{I}/\Delta t \end{pmatrix}$ of $(\mathbf{z}_a, \mathbf{z}_b)$, whose covariance is $\text{diag}(\mathbf{R}, \mathbf{R})$; the product $\mathbf{M} \text{diag}(\mathbf{R}, \mathbf{R}) \mathbf{M}^{\mathbf{T}}$ has blocks $\mathbf{R}$, $\mathbf{R}/\Delta t$, $\mathbf{R}/\Delta t$ and $(\mathbf{R} + \mathbf{R})/\Delta t^2$. (b) Per axis, with $q = 0$: predicting from $(a, b, c) = (r, 0, V)$ gives $a^- = r + \Delta t^2 V$, $b^- = \Delta t V$, $c^- = V$, hence $S = 2r + \Delta t^2 V$, $k_p = (r + \Delta t^2 V)/S \rightarrow 1$ and $k_v = \Delta t V/S \rightarrow 1/\Delta t$ as $V \rightarrow \infty$. The updated mean is $\hat{p} = z_a + k_p(z_b - z_a) \rightarrow z_b$ and $\hat{v} = k_v(z_b - z_a) \rightarrow (z_b - z_a)/\Delta t$. The updated covariance is $a = (1 - k_p)a^- = r(r + \Delta t^2 V)/(2r + \Delta t^2 V) \rightarrow r$, $b = (1 - k_p)b^- = r\Delta t V/(2r + \Delta t^2 V) \rightarrow r/\Delta t$ and $c = c^- - k_v b^- = V - \Delta t^2 V^2/(2r + \Delta t^2 V) = 2rV/(2r + \Delta t^2 V) \rightarrow 2r/\Delta t^2$, which is Equation (18.22). (c) Starting from $\mathbf{z}_b$ with a zero velocity of variance $v_{\max}^2$ discards the information in $\mathbf{z}_a$ and, unless $v_{\max}$ is large compared with the true speed, biases the first velocity estimates toward zero; the two-point start has a much larger initial velocity variance ($2\sigma_p^2/\Delta t^2$) but is unbiased and correctly correlated with the position, and in experiment A its velocity error falls below 1 m/s within about two seconds, while the zero-velocity start with $v_{\max} = 5$ m/s takes noticeably longer.

Exercise 18.2. (a) $\text{Var}(\lambda z_1 + (1 - \lambda)z_2) = \lambda^2 \sigma_1^2 + (1 - \lambda)^2 \sigma_2^2$; setting the derivative $2\lambda \sigma_1^2 - 2(1 - \lambda)\sigma_2^2$ to zero gives $\lambda^* = \sigma_2^2/(\sigma_1^2 + \sigma_2^2)$ and the minimum $\sigma^2 = \sigma_1^2 \sigma_2^2/(\sigma_1^2 + \sigma_2^2)$, that is $1/\sigma^2 = 1/\sigma_1^2 + 1/\sigma_2^2$: precisions add. (b) Write $K = 1 - \lambda^* = \sigma_1^2/(\sigma_1^2 + \sigma_2^2)$; with $z_1 = \hat{x}^-$, $\sigma_1^2 = P^-$, $z_2 = z$ and $\sigma_2^2 = R$ this is $K = P^-/(P^- + R)$ and the estimate is $(1 - K)\hat{x}^- + Kz = \hat{x}^- + K(z - \hat{x}^-)$, exactly Equation (18.15). The one-dimensional Kalman update is therefore nothing but the inverse-variance-weighted average of two readings. (c) $\lambda^* = 0.25/1.25 = 0.2$, $K = 0.8$, $\hat{x} = 0.2 \cdot 2 + 0.8 \cdot 3 = 2.8$ m and $\sigma^2 = 0.2$ m² ($\sigma = 0.447$ m), the numbers of Example 18.6.

Exercise 18.6. (a) $\mathbf{O} = \begin{pmatrix} \mathbf{H} \\ \mathbf{H}\mathbf{F} \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix}$ has rank 1: a velocity-only sensor never sees the position. The filter's position variance grows without bound — it is the integral of the velocity error — while the velocity variance settles, which is why dead reckoning drifts. (b) With $\mathbf{F} = \begin{pmatrix} 1 & \Delta t & \Delta t^2/2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{pmatrix}$ and $\mathbf{H} = (1 \ 0 \ 0)$ the rows of $\mathbf{O}$ are $(1, 0, 0)$, $(1, \Delta t, \Delta t^2/2)$ and $(1, 2\Delta t, 2\Delta t^2)$, with determinant $\Delta t^3 \neq 0$: rank 3, so position, velocity *and* acceleration are all recoverable from a position sensor, as the CV case (rank 2) already suggested. (c) $\mathbf{H} = (1 \ 0 \ 0 \ 0)$ on the state (p_x, p_y, v_x, v_y) gives rank 2; the observable subspace is spanned by p_x and v_x , and p_y, v_y are unobservable. `observability_rank` returns 2. In a simulation the x -block of $\mathbf{P}$ converges

to the steady state of [Proposition 18.10](#) while the y -block follows the pure prediction recursion: $\text{Var}(v_y)$ grows linearly in t (like qt) and $\text{Var}(p_y)$ like $qt^3/3$, so the reported uncertainty is honest — the filter says it does not know y .

Exercise 18.7. Hint. The swap rate is set by the separation of the two predicted positions at the crossing measured in units of the innovation standard deviation. With $\sigma_p = 1$ m and a 30° crossing the two tracks stay within a couple of σ_p of each other for several steps, and nearest-neighbor assignment is then close to a coin toss at each of those steps, so swaps are common; at 90° the paths separate within one or two steps and swaps become rare, and halving σ_p has the same effect as doubling the crossing angle's separation in σ units. Count a swap by comparing the identity of each track's estimate with the truth after the intruders have separated by more than $10\sigma_p$, not at the crossing itself. The velocity-change test of part (c) helps exactly in the ambiguous window and costs missed associations when the intruders really turn, so report both the swap rate and the number of dropped tracks.

Exercise 18.8. Hint. With $\mathbf{H} = \mathbf{I}_4$ the innovation has $m = 4$ components, so the mean NIS should sit near 4, not 2: the band of [Equation \(18.21\)](#) is $4 \pm 1.96\sqrt{8/N}$ over N accepted updates, about $[3.6, 4.4]$ for $N \approx 180$, and the gate at $P_G = 0.99$ is $\gamma = 13.28$ from [Table 18.3](#). Compute the statistics only over accepted updates and only after the transient of the two-point initialization, and remember that a dropped report is a predict-only step, not a zero innovation. The velocity channel makes the filter much better than in experiment A, but it does not make finite differences respectable: the position-only filter is the fair baseline for part (a), and the honest headline for part (c) is the step-to-step change of the 3 s extrapolation, which the raw reports make jump by meters and the filtered state by centimeters.

Exercise 18.9. Hint. On a maneuvering target the minimum moves to a larger q than the truth of the straight-flying scenario, and the curve becomes steep on the small- q side — the opposite asymmetry to [Figure 18.8](#), because too small a q now costs a lag in the turn rather than a little extra smoothing. The diagnostics of [Section 18.7.2](#) select the q that whitens the innovations over the whole run, which is a compromise between the legs and the turn and therefore sits near but not exactly at the RMSE optimum. For (c), the CA model wins in the turn, where a constant-velocity prediction is systematically wrong, and loses on the straight legs, where its extra state is driven by noise only; a single fixed model cannot win both, which is precisely the argument for the interacting-multiple-model estimator.

Exercise 19.1. (a) Neither: f is the constant-velocity matrix and h selects the position; use the Kalman filter. (b) h is nonlinear (square root and arctangent), f linear; an EKF or UKF, the UKF when the bearing noise is large or the range short. (c) f is nonlinear through $v \cos \psi$, $v \sin \psi$ and the division by ω , h linear; the EKF is adequate for mild maneuvers. (d) Both nonlinear; EKF or UKF at long range, UKF at short range, a particle filter if there are occlusions or a map. (e) With ω known and fixed, the Cartesian-velocity form $(x, \dot{x}, y, \dot{y})$ evolves by a constant matrix whose entries are $\sin(\omega\Delta t)/\omega$, $(1 - \cos \omega\Delta t)/\omega$, $\cos \omega\Delta t$ and $\sin \omega\Delta t$, and h selects the position: a linear-Gaussian model, so the plain Kalman filter is exact. The nonlinearity of the coordinated-turn model comes entirely from ω being unknown and multiplied with the velocity.

Exercise 19.2. Write $\psi' = \psi + \omega\Delta t$. The first component is $x' = x + \frac{v}{\omega}(\sin \psi' - \sin \psi)$; its derivatives with respect to x , y , v are 1, 0 and $(\sin \psi' - \sin \psi)/\omega$; with respect to ψ , since $\partial\psi'/\partial\psi = 1$, $\frac{v}{\omega}(\cos \psi' - \cos \psi)$; with respect to ω , by the product rule on $v\omega^{-1}$ and $\partial\psi'/\partial\omega = \Delta t$, $-\frac{v}{\omega^2}(\sin \psi' - \sin \psi) + \frac{v\Delta t}{\omega} \cos \psi'$. The second component $y' = y - \frac{v}{\omega}(\cos \psi' - \cos \psi)$ gives, in the same way, $-(\cos \psi' - \cos \psi)/\omega$, $\frac{v}{\omega}(\sin \psi' - \sin \psi)$ and $\frac{v}{\omega^2}(\cos \psi' - \cos \psi) + \frac{v\Delta t}{\omega} \sin \psi'$.

For the limits use $\sin \psi' = \sin \psi + \omega \Delta t \cos \psi - \frac{1}{2} \omega^2 \Delta t^2 \sin \psi - \frac{1}{6} \omega^3 \Delta t^3 \cos \psi + \dots$ and $\cos \psi' = \cos \psi - \omega \Delta t \sin \psi - \frac{1}{2} \omega^2 \Delta t^2 \cos \psi + \frac{1}{6} \omega^3 \Delta t^3 \sin \psi + \dots$: then $(\sin \psi' - \sin \psi)/\omega \rightarrow \Delta t \cos \psi$, $\frac{v}{\omega}(\cos \psi' - \cos \psi) \rightarrow -v \Delta t \sin \psi$, and in F_{15} the terms of order $1/\omega$ cancel, leaving $-\frac{1}{2} v \Delta t^2 \sin \psi + \mathcal{O}(\omega)$; the second row is analogous with $\Delta t \sin \psi$, $v \Delta t \cos \psi$ and $\frac{1}{2} v \Delta t^2 \cos \psi$. The chapter code evaluates exactly these series for $|\omega| < 10^{-4}$, and a central finite difference with step 10^{-5} agrees with `CoordinatedTurnModel.jacobian` to better than 10^{-5} at both requested turn rates.

Exercise 19.4. With $n = 5$ and $\lambda = \alpha^2(n + \kappa) - n$: for $(1, 2, 0)$, $\lambda = 0$, the points are $\sqrt{5} \approx 2.236$ standard deviations from the mean, $W_0^{(m)} = 0$, $W_0^{(c)} = 2$, $W_i^{(m)} = 0.1$. For $(1, 2, -2)$, $\lambda = -2$, $n + \lambda = 3$, the points are at $\sqrt{3} \approx 1.732$ standard deviations, $W_0^{(m)} = -2/3$, $W_0^{(c)} = 4/3$, $W_i^{(m)} = 1/6$. For $(10^{-3}, 2, 0)$, $\lambda = 5 \cdot 10^{-6} - 5$, $n + \lambda = 5 \cdot 10^{-6}$, the points are $\sqrt{5} \cdot 10^{-3}$ standard deviations from the mean, $W_0^{(m)} = \lambda/(n + \lambda) = 1 - n/(n + \lambda) = 1 - 10^6$, $W_0^{(c)} = W_0^{(m)} + 3 - 10^{-6} \approx 3 - 10^6$, and $W_i^{(m)} = 1/(2(n + \lambda)) = 10^5$. The second and third choices have a negative $W_0^{(m)}$. In every case $W_0^{(m)} + 10 W_i^{(m)} = 1$: $0 + 1$, $-2/3 + 10/6$ and $(1 - 10^6) + 10^6$. The danger of the third choice is cancellation: the mean is $(1 - 10^6)$ times the center image plus 10^5 times the sum of ten images that differ from the center by about 10^{-3} standard deviations, that is, the difference of two numbers of size 10^6 that must be accurate to a relative 10^{-6} ; six of the sixteen decimal digits of double precision are lost, and any rounding inside g is amplified a millionfold.

Exercise 19.6. (a) $N_{\text{eff}} = 1/(4 \cdot 0.0625) = 4$; $1/(0.49 + 3 \cdot 0.01) = 1/0.52 \approx 1.92$; $1/1 = 1$. (b) Cauchy–Schwarz with the vectors $(\omega^{(1)}, \dots, \omega^{(N)})$ and $(1, \dots, 1)$ gives $1 = (\sum_i \omega^{(i)})^2 \leq N \sum_i (\omega^{(i)})^2$, so $N_{\text{eff}} \leq N$, with equality exactly when the two vectors are proportional, that is, all weights equal; and $\sum_i (\omega^{(i)})^2 \leq \max_i \omega^{(i)} \leq 1$ with equality exactly when one weight is 1. (c) Any set of N particles with equal weights, wherever they are: take all particles 100 m from the truth before any measurement has been processed. N_{eff} measures how evenly the weight is spread over the particles, not whether the particles are anywhere near the truth; a filter that has lost the track and resampled has $N_{\text{eff}} = N$. (d) The cumulative sums are $(0.7, 0.8, 0.9, 1.0)$ and the grid points $0.1, 0.35, 0.6, 0.85$. The first three fall into $(0, 0.7]$ and select particle 1; 0.85 falls into $(0.8, 0.9]$ and selects particle 3. The counts $(3, 0, 1, 0)$ lie in $\{\lfloor 2.8 \rfloor, \lceil 2.8 \rceil\} = \{2, 3\}$, $\{0, 1\}$, $\{0, 1\}$, $\{0, 1\}$ as [Proposition 19.12](#) requires, and average over u to $(2.8, 0.4, 0.4, 0.4)$.

Exercise 19.3. The update is linear in the innovation, so the naive estimate differs from the correct one by $\mathbf{K}_2$ times the difference of the two innovations, and that difference is exactly $6.2117 - (-0.0715) = 2\pi$. The shifts are $3.21 \cdot 2\pi = +20.2$ m in x , $-47.05 \cdot 2\pi = -295.6$ m in y and $2.56 \cdot 2\pi = +16.1$ rad in ψ (that is $+3.5$ rad after wrapping): starting from the updated estimate $(-110.94, -2.13)$ of [Table 19.1](#), the position lands near $(-90.8, -297.8)$, about 296 m south of the intruder, with a covariance that claims a meter of accuracy. K_{22} is negative because the intruder is almost due west of the sensor, $\varphi \approx \pi$: there $\partial y / \partial \varphi = r \cos \varphi \approx -r$, so a bearing that reads larger than predicted means a target further *south*, and the gain inherits the sign.

Exercise 19.5. Write $\varphi = \pi/2 + \epsilon$ with $\epsilon \sim \mathcal{N}(0, \sigma_\varphi^2)$, so $\cos \varphi = -\sin \epsilon$ and $\sin \varphi = \cos \epsilon$. The characteristic function of a centered Gaussian gives $\mathbb{E}[\cos \epsilon] = e^{-\sigma_\varphi^2/2}$ and $\mathbb{E}[\sin \epsilon] = 0$; with r independent of φ , $\mathbb{E}[r \cos \varphi] = 0$ and $\mathbb{E}[r \sin \varphi] = \mu_r e^{-\sigma_\varphi^2/2}$. For $\mu_r = 100$ and $\sigma_\varphi = 0.3$ this is $100 e^{-0.045} = 95.60$ m, against the EKF's 100. For the unscented transform, $\lambda = \alpha^2(n + \kappa) - n = 1 \cdot 3 - 2 = 1$, so $n + \lambda = 3$, $W_0^{(m)} = 1/3$ and $W_i^{(m)} = 1/6$, and $\mathbf{L} = \sqrt{3\mathbf{P}} = \text{diag}(8.660, 0.5196)$. The five points $(100, \pi/2)$, $(108.660, \pi/2)$, $(100, \pi/2 \pm 0.5196)$ and $(91.340, \pi/2)$ map to $(0, 100)$, $(0, 108.660)$, $(\mp 49.66, 86.80)$ and $(0, 91.340)$. The x images cancel in pairs, so the mean is 0;

the y mean is $\frac{1}{3} \cdot 100 + \frac{1}{6}(108.660 + 86.80 + 91.340 + 86.80) = 95.60$ m, the exact value to three decimals. Five function evaluations, no derivative, and the 4.4 m bias of Figure 19.3 is gone.

Exercise 19.7. Hint and expected behavior. (a) Without the map likelihood nothing forbids a particle from flying through the building, so the cloud stays one connected blob straddling $x = 0$ instead of splitting; the two lobes of Figure 19.5 are produced by the constraint, not by the motion model. (b) Resampling at every step throws away diversity that the process noise has not yet restored: after the six blind seconds the distinct-state count collapses and the fractions west and east swing far from 68/28, run to run, because one lobe is often wiped out entirely (sample impoverishment, Section 19.6.2). (c) With $N = 200$ each lobe holds a few dozen particles, the fractions become very noisy, and N_{eff} at $k = 20$ often falls to single digits: the first measurement after the occlusion kills almost every particle, and the filter can lose the track altogether. Report the three numbers with at least ten different seeds; a single run tells you nothing about (b) and (c).

Exercise 19.8. Hint. Reuse `simulate_turning_target`, `RangeBearingSensor` and `run_tracking` from the chapter code and drive them exactly as `code/figures/gen_ch19_compare.py` does: one `numpy.random.Generator` per run for the truth and the measurements, a separate one for the particle filter, and the RMSE taken after the ten settling steps. (a) With 100 runs the baseline row reproduces to about ± 0.05 m; a difference of 0.02 m between the EKF and the UKF is inside the Monte Carlo error, which is the point of the row. (b) Expect the two curves to lie on top of each other up to about 4° and to separate beyond it, the UKF gaining roughly 10 % by 8° ; report the median as well as the mean, because a single lost run moves the mean. (c) The RMSE flattens once N is a few thousand while the time per step grows linearly, so plot both on the same axis and read off the knee. (d) During a five-second occlusion the Gaussian filters coast on the coordinated-turn model and recover if the intruder did not maneuver; the particle filter spreads and keeps the alternatives, which is what Section 19.6.4 shows.

Exercise 20.2. By Proposition 20.7 the error at step h is $e_h = \frac{1}{2} \|\mathbf{a}\| (h \Delta t)^2 = \frac{1}{2} \cdot 2 \cdot (0.2h)^2 = 0.04 h^2$ meters. Hence FDE = $0.04 \cdot 144 = 5.76$ m and ADE = $\frac{1}{2} \|\mathbf{a}\| \Delta t^2 (H+1)(2H+1)/6 = 0.04 \cdot 13 \cdot 25/6 = 2.17$ m. The error exceeds 0.5 m when $0.04 h^2 > 0.5$, i.e., $h^2 > 12.5$, so first at $h = 4$ (0.8 s, $e_4 = 0.64$ m). Constant velocity is a safe short-horizon predictor only while $\frac{1}{2} \|\mathbf{a}\| (h \Delta t)^2$ stays below the margin; a constant-acceleration predictor would be exact here, but on a turn it overshoots (Figure 20.2), so which of the two is safer depends on the maneuvers that actually occur.

Exercise 20.3. Write the observations as $\mathbf{z}_{T-j} = \mathbf{p}_T - j \Delta t \mathbf{v} + \varepsilon_{T-j}$ for $j = 0, \dots, k$. The least-squares line fit estimates the position at time T and the velocity by linear regression on the times $-j \Delta t$; with $\bar{j} = k/2$ and $S = \sum_j (j - \bar{j})^2 = k(k+1)(k+2)/12$, the fitted velocity is $\hat{\mathbf{v}} = -\sum_j (j - \bar{j}) \mathbf{z}_{T-j} / (\Delta t S)$ and the fitted position at T is $\hat{\mathbf{p}}_T = \bar{\mathbf{z}} + \bar{j} \Delta t \hat{\mathbf{v}}$. Both are unbiased, so the prediction $\hat{\mathbf{p}}_T + h \Delta t \hat{\mathbf{v}}$ has zero mean error, and its error is a linear combination $\sum_j w_j \varepsilon_{T-j}$ with weights $w_j = \frac{1}{k+1} - \frac{(j-\bar{j})(h+\bar{j})}{S}$, giving $\mathbb{E}[e_h^2] = 2\sigma^2 \sum_j w_j^2 = 2\sigma^2 [\frac{1}{k+1} + \frac{(h+\bar{j})^2}{S}]$. For $k = 7$, $h = 12$: $S = 42$, $\bar{j} = 3.5$, so $\mathbb{E}[e_h^2] = 2\sigma^2 [0.125 + 240.25/42] = 2\sigma^2 \cdot 5.85 = 11.7\sigma^2$, against $2\sigma^2[(19/7)^2 + (12/7)^2] = 20.6\sigma^2$ for the endpoint difference: the line fit reduces the RMS error by a factor 1.33 (0.17 against 0.23 m at $\sigma = 0.05$ m). For the endpoint difference $2\sigma^2[(1 + h/k)^2 + (h/k)^2]$ decreases monotonically in k , so $k = 7$ is best at every h on straight flight. On a turn the velocity rotates during the window, the average velocity over k intervals points $\omega k \Delta t/2$ behind the current heading, and the resulting bias $\approx v \omega k \Delta t h \Delta t/2$ grows with k ; the validation set of Section 20.8 balanced the two effects at $k = 2$.

Exercise 20.1. The predictions differ from the truth only in y , so $e_1 = 0.1$, $e_2 = 0.5$, $e_3 = 1.2$ m, giving ADE = $(0.1 + 0.5 + 1.2)/3 = 0.6$ m and FDE = $e_3 = 1.2$ m. The second sample has

$e_1 = 0.1$, $e_2 = 0.1$, $e_3 = 0.4$ m, hence an ADE of 0.2 m and a final error of 0.4 m. Therefore $\min \text{ADE}_2 = \min(0.6, 0.2) = 0.2$ m and $\min \text{FDE}_2 = \min(1.2, 0.4) = 0.4$ m. Both are minima of a set that contains the first sample's values, so neither can exceed them — which is why a $\min \text{ADE}_K$ may never be compared with the ADE of a deterministic predictor.

Exercise 20.6. Projections: $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ each cost nd^2 multiply-adds and nd memory. Scores: $\mathbf{QK}^\top$ costs n^2d and stores n^2 numbers; the softmax is $\Theta(n^2)$; the product with $\mathbf{V}$ is n^2d . Total $\Theta(nd^2 + n^2d)$ time and $\Theta(nd + n^2)$ memory, and the n^2 memory is the usual bottleneck for long sequences. With n_h heads each projection is $n \times d \times d/n_h$ and each score matrix n^2d/n_h ; summed over heads the time is unchanged, $\Theta(nd^2 + n^2d)$, while the score memory becomes $n_h n^2$. For a scene of N agents over T steps as one token set, $n = NT$ gives $\Theta(NTd^2 + N^2T^2d)$ time and $\Theta(N^2T^2)$ score memory per layer; per-agent attention over time only costs $N \cdot \Theta(Td^2 + T^2d)$ and misses the interactions. An LSTM per agent costs $\Theta(TD(M + D)) \approx \Theta(TD^2)$ per agent, $\Theta(NTD^2)$ in total, sequential in T but with no quadratic term. With $d = D = 64$ the joint attention layer costs $64NT(64 + NT)$ against $64^2NT \cdot 4$ for the four gate matrices of the LSTM (the factor 4 accounts for $4D(M + D)$ with $M \ll D$), so attention becomes more expensive when $64 + NT > 256$, i.e., when $NT > 192$: about 10 agents over 20 steps, or 24 agents over 8 steps. Beyond that, attention pays for itself only if it predicts better, and it can be made cheaper by attending over agents at one time step and over time within one agent separately.

Exercise 20.7. Let $\mathbf{y}^{(1)} = (-2, 4)$ with probability π and $\mathbf{y}^{(2)} = (2, 4)$ otherwise. By [Theorem 20.9](#) the squared-error-optimal end point is the conditional mean $\mathbf{g}^* = \pi \mathbf{y}^{(1)} + (1 - \pi) \mathbf{y}^{(2)} = (2 - 4\pi, 4)$. Its expected FDE is $\pi \|\mathbf{y}^{(1)} - \mathbf{g}^*\| + (1 - \pi) \|\mathbf{y}^{(2)} - \mathbf{g}^*\| = \pi \cdot 4(1 - \pi) + (1 - \pi) \cdot 4\pi = 8\pi(1 - \pi)$. For $\pi = \frac{1}{2}$ the prediction is $(0, 4)$, the center of the junction, with expected FDE 2 m and a guaranteed miss of 2 m in every single case; for $\pi = 0.9$ it is $(-1.6, 4)$, expected FDE 0.72 m, which is 0.4 m off even when the drone turns left. A Gaussian read-out reports $\mu_x = 2 - 4\pi$ and $\sigma_x^2 = \text{Var}(y_x) = 16\pi(1 - \pi)$, so $\sigma_x = 2$ m for $\pi = \frac{1}{2}$ and 1.2 m for $\pi = 0.9$: it does not resolve the two modes, but its large σ_x announces that something is wrong with a single line. The two-sample predictor that outputs both end points has $\min \text{FDE}_2 = 0$, which is why $\min \text{ADE}_K$ and $\min \text{FDE}_K$ are the natural scores for multimodal predictors and why they cannot be compared with the FDE of a deterministic one.

Exercise 20.5. Treat the pair $(\sin \theta_i t, \cos \theta_i t)$ with $\theta_i = 10000^{-2i/d}$ on its own. The addition theorems give $\sin \theta_i(t + \delta) = \cos(\theta_i \delta) \sin \theta_i t + \sin(\theta_i \delta) \cos \theta_i t$ and $\cos \theta_i(t + \delta) = -\sin(\theta_i \delta) \sin \theta_i t + \cos(\theta_i \delta) \cos \theta_i t$, so the pair is multiplied by the rotation $\mathbf{R}(\theta_i \delta)$, which depends on δ but not on t . Stacking the $d/2$ pairs, $\mathbf{M}_\delta = \text{blockdiag}(\mathbf{R}(\theta_0 \delta), \dots, \mathbf{R}(\theta_{d/2-1} \delta))$ is block-diagonal and orthogonal, and $\text{PE}(t + \delta) = \mathbf{M}_\delta \text{PE}(t)$ for every t . For the second part, let a head score the pair (t, t') by the bilinear form $\text{PE}(t)^\top \mathbf{W} \text{PE}(t')$, which is what queries and keys computed by linear maps from the encodings amount to. Choosing $\mathbf{W} = \mathbf{M}_2$ gives the score $\text{PE}(t)^\top \text{PE}(t' + 2) = \sum_i \cos(\theta_i(t - t' - 2))$, a function of the offset alone whose every term is maximal, and equal to 1, exactly when $t' = t - 2$. One fixed weight matrix therefore implements “attend to the step two before me” at every t , which is what makes an absolute encoding usable for relative reasoning; a learned per-position embedding gives no such guarantee.

Exercise 20.8. Six violations, and each one flatters the learned model. (i) *Overlapping windows split at random.* With a stride of 1, two windows 0.2 s apart share 19 of 20 steps; shuffling before the split puts near-copies of test windows into training, so the network can recall rather than predict. This is the leakage of [Section 20.3](#): split by scene or recording, never by window. (ii) *No validation split.* With only train and test, the architecture, the learning rate and the stopping epoch must have been chosen on the test set — rule 4 — which turns the reported number into a training score of the model-selection procedure. (iii) *Normalization*

with statistics of the whole data set. The mean and standard deviation carry test information into preprocessing, a second, milder leak; and normalizing *positions* rather than displacements predicts in absolute coordinates, which [Table 20.2](#) shows to be a bad idea for a different reason. (iv) *An untuned baseline.* “Velocity from the last two positions” is $\text{CV}(k=1)$, the noisiest member of the family; in this chapter’s data it is 88 % worse than the tuned window on straight flight, so most of the reported gap may be the baseline’s noise rather than the model’s skill. (v) *One ADE, no horizon.* A single number cannot say whether the model helps at 0.4 s, where the local layer acts, or only at 2.4 s; report ADE_h and FDE_h curves and per-maneuver subsets. (vi) *A minADE_{20} quoted next to deterministic ADEs.* minADE_K is a minimum over K samples and falls with K by construction ([Exercise 20.1](#)); comparing 0.12 m with a deterministic 0.58 m compares two different quantities. Report the deterministic ADE of the sample mean as well, or give the baseline the same K samples.

Exercise 20.9. Checkpoints. The pipeline is in `ch20_prediction.py`: `generate_dataset` and `prepare_sequences` (the agent frame of [Equation \(20.11\)](#)), `predict_cv`, `predict_ca` and `predict_kf`, `Seq2SeqPredictor` with `train_predictor`, and `evaluate` with its per-horizon variant. The data are one array of shape $(N, T_{\text{obs}} + T_{\text{pred}}, 2)$ plus a family label per trajectory, split *by trajectory* and never by time step. Steps: metrics first, implementing [Equation \(20.1\)](#) and checking ADE and FDE by hand on a two-step example; baselines next, tuning their one knob (the smoothing window k , the process noise q) on validation only; then the network, predicting displacements in the agent frame as a residual over constant velocity; finally repeat every learned run over at least three seeds. Expected numbers at the 2.4 s horizon with $\sigma = 0.05$ m: ADE 1.23 for raw two-point CV, 1.14 tuned, 1.13 for the tuned Kalman filter and 0.79–0.82 for the LSTM; per horizon the tuned CV runs 0.30, 0.68, 1.14 at 0.8, 1.6 and 2.4 s against 0.23, 0.48, 0.79. Look hardest at the *straight* subset, where the tuned CV wins (0.34 against 0.41): the network buys its average at the turns and the evasive maneuvers. Training is about fifteen seconds per model in NumPy, so three seeds are cheap; in (b) expect the gap to shrink as σ grows and to widen with the training set. Classic mistakes: leakage between splits; predicting absolute coordinates; a straw-man baseline built from two raw points; comparing at different horizons; tuning anything on the test set. Done when you have an ADE/FDE table per horizon and method over several seeds and one honest sentence about it — “the LSTM did not beat constant velocity at 0.8 s” is a legitimate result, and the Week 10 milestone ([Appendix A](#)) asks for exactly that.

Exercise 20.10. Checkpoints. Three functions do the bookkeeping: `calibration` returns the fraction of true positions inside the p -ellipse per horizon step, `chi2_radius` is the Mahalanobis radius for a given p , and `inflated_radius` turns a covariance into the single obstacle radius of [Equation \(20.21\)](#) that ORCA can take. Keep γ_h as a vector of length T_{pred} , one factor per horizon step, fitted on validation and then frozen. Steps: measure the coverage you actually have before fitting anything — on the chapter’s data the tuned Kalman filter contains 0.85, 0.81 and 0.79 of the true positions at 0.8, 1.6 and 2.4 s and the LSTM-NLL head only 0.76, 0.63 and 0.54, so both are overconfident and the network far more so; fit γ_h by bisection on the empirical coverage, expecting roughly 1.25 to 1.4 for the Kalman filter and 1.45 to 2.0 for the LSTM, growing with h ; verify on test, where a factor fitted on validation should land within a couple of points of 95 %; feed `inflated_radius` into the ORCA loop of [Chapter 13](#) as the intruder radius and compare three predictors on the same seeds on two numbers only, the minimum separation and how many steps the local safety layer was active. The expected shape of the result is a trade: a bigger radius buys separation and costs maneuvers, so report both or the comparison is empty. Classic mistakes: one global γ instead of one per step; fitting γ on test; inflating by the trace instead of the largest eigenvalue; comparing avoidance counts across different intruder trajectories. Done when the calibrated ellipses hold 95 % on held-out

data and your ORCA table shows what the extra radius bought and what it cost.

Exercise 21.2. By induction on k : $\mathbf{x}_1 = \mathbf{A}\mathbf{x}_0 + \mathbf{B}\mathbf{u}_0$, and if $\mathbf{x}_k = \mathbf{A}^k\mathbf{x}_0 + \sum_{j=0}^{k-1} \mathbf{A}^{k-1-j}\mathbf{B}\mathbf{u}_j$ then $\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k = \mathbf{A}^{k+1}\mathbf{x}_0 + \sum_{j=0}^k \mathbf{A}^{k-j}\mathbf{B}\mathbf{u}_j$. Stacking $k = 1, \dots, N$ gives $\mathbf{X} = \mathbf{S}_x\mathbf{x}_0 + \mathbf{S}_u\mathbf{U}$ with the k -th block row of $\mathbf{S}_x$ equal to $\mathbf{A}^k$ and the block (k, j) of $\mathbf{S}_u$ equal to $\mathbf{A}^{k-1-j}\mathbf{B}$ for $j < k$ and $\mathbf{0}$ otherwise. Substituting into the cost, $(\mathbf{X} - \mathbf{X}^{\text{ref}})^\top \mathbf{Q}(\mathbf{X} - \mathbf{X}^{\text{ref}}) + \mathbf{U}^\top \mathbf{R}\mathbf{U} = \mathbf{U}^\top (\mathbf{S}_u^\top \mathbf{Q} \mathbf{S}_u + \bar{\mathbf{R}}) \mathbf{U} + 2(\mathbf{S}_x\mathbf{x}_0 - \mathbf{X}^{\text{ref}})^\top \bar{\mathbf{Q}} \mathbf{S}_u \mathbf{U} + \text{const}$, so $\mathbf{H} = 2(\mathbf{S}_u^\top \bar{\mathbf{Q}} \mathbf{S}_u + \bar{\mathbf{R}})$ and $\mathbf{f} = 2\mathbf{S}_u^\top \bar{\mathbf{Q}} (\mathbf{S}_x\mathbf{x}_0 - \mathbf{X}^{\text{ref}})$. $\mathbf{H}$ is positive definite because $\bar{\mathbf{R}}$ is and $\mathbf{S}_u^\top \bar{\mathbf{Q}} \mathbf{S}_u$ is positive semidefinite; hence the QP is strictly convex and has a unique minimizer. The velocity bound $-v_{\max} \leq v_{k,i} \leq v_{\max}$ becomes $-v_{\max} - (\mathbf{S}_x\mathbf{x}_0)_{v_{k,i}} \leq (\mathbf{S}_u)_{v_{k,i}} \mathbf{U} \leq v_{\max} - (\mathbf{S}_x\mathbf{x}_0)_{v_{k,i}}$, one row of $\mathbf{S}_u$ per velocity component and step.

Exercise 21.3. (a) If $\mathbf{n}^\top(\mathbf{p} - \mathbf{o}) \geq r$ with $\|\mathbf{n}\| = 1$, the Cauchy–Schwarz inequality gives $\|\mathbf{p} - \mathbf{o}\| \geq \mathbf{n}^\top(\mathbf{p} - \mathbf{o}) \geq r$, so every point of the half-plane is outside the disk; the half-plane is the tangent half-plane at the point $\mathbf{o} + r\mathbf{n}$ and it also excludes the collision-free points “behind” the disk as seen from the linearization point. (b) The half-planes $\mathbf{n}_m^\top(\mathbf{p} - \mathbf{c}) \leq e \cos(\pi/M)$ for M equally spaced unit directions describe the regular M -gon inscribed in the disk of radius e : its apothem is $e \cos(\pi/M)$ and its vertices lie on the circle, so the polygon is inside the disk and the constraint is conservative. The largest loss is at the flat sides, $e(1 - \cos(\pi/M))$: 7.6% of e for $M = 8$ and 1.9% for $M = 16$. (c) In three dimensions replace the disk by a ball and the polygon by a polyhedron, e.g., the 26 directions $(\pm 1, \pm 1, \pm 1)$, $(\pm 1, \pm 1, 0)$ and permutations, normalized; the apothem is then the smallest distance from the center to a face and must be computed once.

Exercise 21.4. Write $\tilde{\mathbf{p}} = 100\mathbf{p}$ and $\tilde{\mathbf{v}} = 100\mathbf{v}$; the accelerations are unchanged, so $\mathbf{A}$ is the same and $\mathbf{B}$ is multiplied by 100. The cost must take the same value on the same physical trajectory, so q_p and q_v must both shrink by 10^4 (a position error of 1 m is now the number 100, and $100^2 q_p' = q_p$), while r is unchanged because accelerations still carry their old units. The slack s_k of an avoidance row is a length: w_1 shrinks by 10^2 and w_2 by 10^4 . The multipliers are derivatives of the cost with respect to a bound, so every multiplier of a row whose bound is a length (avoidance, keep-in) or a velocity is divided by 100, and the multipliers of the input box are unchanged — which is exactly why Proposition 21.7 must be re-read in the new units before w_1 is chosen (Section 21.7.1).

Exercise 21.5. A drone at speed v needs the time $v/a_{\max}$ and the distance $v^2/(2a_{\max})$ to stop (Chapter 2). If the horizon $N\Delta t$ is shorter than the stopping time, the QP is asked to guarantee separation at steps it cannot influence enough, and a hard constraint that first appears at the end of the horizon can already be impossible to satisfy. With $v_{\max} = 2$ m/s and $a_{\max} = 2$ m/s² the stopping time is 1 s, i.e., $N \geq 10$ at $\Delta t = 0.1$ s; at the cruising speed 1 m/s of the example it is 0.5 s, $N \geq 5$, which is why the generated experiment shows failures at $N = 3$ and none from $N = 4$ on. Add the reaction latency and the prediction horizon of the intruder to this bound in practice.

Exercise 21.7. (a) Take $\mathcal{X}_f = \{\mathbf{x} : \mathbf{x}^\top \mathbf{P}_f \mathbf{x} \leq c\}$ with the Riccati $\mathbf{P}_f$ and $\kappa_f(\mathbf{x}) = -\mathbf{K}\mathbf{x}$. Invariance is free: Equation (21.11) gives $(\mathbf{A} - \mathbf{BK})^\top \mathbf{P}_f (\mathbf{A} - \mathbf{BK}) - \mathbf{P}_f = -(\mathbf{Q} + \mathbf{K}^\top \mathbf{R} \mathbf{K}) \preceq \mathbf{0}$, so $\mathbf{x}^\top \mathbf{P}_f \mathbf{x}$ does not increase along the LQR loop and every sublevel set is invariant; the same identity is condition (ii) with equality. What c must satisfy is admissibility: $\mathcal{X}_f$ must lie inside the geofence, the velocity box and the input box under $-\mathbf{K}$. For one linear row $\mathbf{h}^\top \mathbf{x} \leq b$ the maximum over the ellipsoid is $\sqrt{c \mathbf{h}^\top \mathbf{P}_f^{-1} \mathbf{h}}$, so the condition is $c \leq b^2 / (\mathbf{h}^\top \mathbf{P}_f^{-1} \mathbf{h})$ for every such row, including the rows $\pm(\mathbf{K}^\top)_i \mathbf{x} \leq a_{\max}$ of the input box; take the smallest of these numbers. With c so chosen, the shifted candidate of Theorem 21.6 is feasible whenever the

problem was feasible one step earlier, and the geofence and the boxes are time invariant, so the argument repeats forever. (b) The half-plane is an inner approximation that depends on the linearization point, and the approximations at two consecutive times are not nested. Put the drone head-on toward the intruder at a separation just above r_{safe} , with a shifted plan that passes on the left, so that the constraint at time t is satisfied by a small left deviation; let the intruder move exactly as predicted but such that the new shifted plan lies on the right of it. The half-planes at $t + \Delta t$ then require a lateral displacement to the right that the input box cannot supply within the first step, and the hard problem is infeasible although nothing was mispredicted.

Exercise 21.8. The scalar $\mathbf{n}_k^T(\mathbf{p}_k - \mathbf{o}_k)$ is Gaussian with mean $\mu = \mathbf{n}_k^T(\mathbf{p}_k - \hat{\mathbf{o}}_k)$ and variance $\sigma^2 = \mathbf{n}_k^T \Sigma_k \mathbf{n}_k$, so $\mathbb{P}(\mu + \sigma Z \geq r) = 1 - \Phi((r - \mu)/\sigma) \geq 1 - \delta$ iff $(r - \mu)/\sigma \leq \Phi^{-1}(\delta) = -\kappa_\delta$, i.e., $\mu \geq r + \kappa_\delta \sigma$, which is Equation (21.13): the same row with an inflated right-hand side. `normal_quantile` gives $\kappa_{0.2} = 0.842$, $\kappa_{0.1} = 1.282$, $\kappa_{0.05} = 1.645$, $\kappa_{0.01} = 2.326$; the minimum separation grows with κ_δ (with $\Sigma_0 = (0.05 \text{ m})^2 \mathbf{I}$, $\Sigma_v = (0.2 \text{ m/s})^2 \mathbf{I}$ and $\delta = 0.05$ the code reports 1.143 m instead of 1.000 m) and so does the RMS tracking error: caution is paid for in tracking. Sampling the intruder's true trajectory from the same Gaussian gives an empirical violation frequency below δ , because the per-step guarantee is applied at every step of the horizon (Boole's inequality) and because Proposition 21.3 makes the disk constraint at least as safe as the half-plane.

Exercise 21.6. Write the soft problem as $\min_{\mathbf{U}, \mathbf{s}} J(\mathbf{U}) + w \sum_k s_k$ subject to $g_k(\mathbf{U}) \leq s_k$, $s_k \geq 0$. Its Lagrangian coincides with that of the hard problem plus the terms $(w - \lambda_k - \mu_k)s_k$, where $\lambda_k \geq 0$ is the multiplier of the avoidance row and $\mu_k \geq 0$ that of $s_k \geq 0$. At a solution of the hard problem with multipliers λ_k^* , choosing $s_k = 0$ and $\mu_k = w - \lambda_k^* \geq 0$ satisfies all optimality conditions of the soft problem as soon as $w \geq \max_k \lambda_k^*$; because the problem is convex these conditions are sufficient, so the soft solution equals the hard one. If $w < \lambda_k^*$ for some k , the penalty is cheaper than the tracking cost it saves and the solver buys a violation $s_k > 0$. In the worked example the largest multiplier is about 19 (the largest over the whole run is 18.7; the sampled trace of Table 21.2 shows 16.2), so $w = 50$ and $w = 100$ reproduce the hard trajectory exactly, $w = 10$ is short of it by 2 mm of separation, and $w = 1$ lets the drone cut 0.22 m into the safety disk.

Exercise 21.9. Sketch. Fly the leader with an ordinary tracking MPC and broadcast its plan; each follower then runs Algorithm 21.2 with two extra blocks of rows built from that plan: the keep-in polygon of Proposition 21.4 around $\hat{\mathbf{p}}_k^{\text{lead}} + \mathbf{d}_{ij}$ with apothem $e_{\text{max}} \cos(\pi/M)$, and the same polygon with radius R_{comm} around $\hat{\mathbf{p}}_k^{\text{lead}}$ (the class `Follower` builds both). Log, per step (`simulate(..., followers=...)` already records both under `rec["form_err"]` and `rec["dist"]`): the formation error $\|\mathbf{p}_t^i - \mathbf{p}_t^{\text{lead}} - \mathbf{d}_{ij}\|$ (report mean, maximum and the number of steps above e_{max}), the distance to the leader and the count of steps with $\text{dist} > R_{\text{comm}}$, and, in the soft version, $\sum_k s_k$ per step and its maximum. The reactive baseline is the same tracker with the formation rows removed and a proportional pull toward $\mathbf{p}_t^{\text{lead}} + \mathbf{d}_{ij}$ switched on only while the error exceeds e_{max} ; because it corrects after the fact, it overshoots the tolerance for a few steps after every leader turn and after the intruder pushes the leader off the path, whereas the constrained run keeps the maximum formation error below e_{max} and the range violations at zero. Expect the constrained follower to pay a slightly larger tracking error, which is the milestone's point: the constraint is enforced in advance rather than repaired afterward.

Exercise 22.1. The vertices are $(0, 0)$, $(4, 0)$, $(4, 1.5)$, $(2.5, 3.75)$ and $(0, 2.5)$ with objective values 0, 4, 7, 10 and 5; the LP optimum is $(2.5, 3.75)$ with value 10. Branching on x first: the child $x \leq 2$ has the LP optimum $(2, 3.5)$ with bound 9 (at $x = 2$ the row $-x + 2y \leq 5$

gives $y \leq 3.5$); the child $x \geq 3$ has the LP optimum $(3, 3)$ with value 9, which is integral and becomes the incumbent. The first child's bound 9 is not better than the incumbent, so it is pruned: three nodes. Branching on y first: the child $y \leq 3$ has the optimum $(3, 3)$, value 9, integral; the child $y \geq 4$ needs $x \geq 3$ from $-x + 2y \leq 5$ and $x \leq 7/3$ from $3x + 2y \leq 15$, so it is infeasible: again three nodes. Both trees prove that the integer optimum is 9, one below the LP bound 10.

Exercise 22.2. The block of Example 22.6 is $[4, 6] \times [1.5, 5.5]$ inside the workspace $W = [0, 10] \times [0, 8]$, so Equation (22.4) gives $M_1 = X_{\max} - x_{\min} = 6$ m, $M_2 = x_{\max} - X_{\min} = 6$ m, $M_3 = Y_{\max} - y_{\min} = 6.5$ m and $M_4 = y_{\max} - Y_{\min} = 5.5$ m. For the separation rows Equation (22.6) with $d_{\min} = 1$ m, $M_x = d_{\min} + (X_{\max} - X_{\min}) = 11$ m and $M_y = d_{\min} + (Y_{\max} - Y_{\min}) = 9$ m.

Too small an M cuts off legal positions. Take $M_1 = 3$ instead of 6 and the point $\mathbf{p}_k = (9, 6.5)$, which is inside W and above the block, hence legal. The first row reads $x_k \leq 4 + 3b_{k,1}$: with $b_{k,1} = 0$ it demands $x_k \leq 4$, so $b_{k,1} = 1$ is forced, and then it demands $x_k \leq 7 < 9$. No choice of binaries admits the point, so a feasible position has become infeasible. With the tight $M_1 = 6$ the switched-off row reads $x_k \leq 10$ and holds everywhere in W , which is all it should do.

With $M_1 = 0$ the first row is $x_k \leq x_{\min} = 4$ whatever $b_{k,1}$ is: the row can no longer be switched off, and the drone is pinned to the strip left of the block at every step. The model of Example 22.6 is then infeasible, because drone A must reach $x = 9$ and drone B starts there.

Exercise 22.3. Apply Proposition 22.3 to the square $S = [x_k^j - d_{\min}, x_k^j + d_{\min}] \times [y_k^j - d_{\min}, y_k^j + d_{\min}]$ of side $2d_{\min}$ centered on drone j : the condition $\|\mathbf{p}_k^i - \mathbf{p}_k^j\|_{\infty} \geq d_{\min}$ says exactly that $\mathbf{p}_k^i$ lies outside S , and Equation (22.6) is Equation (22.3) for that square, written with the differences $x_k^i - x_k^j$ and $y_k^i - y_k^j$ so that every row stays linear in both positions. If binaries satisfy Equation (22.6), the sum row forces some $c_{k,q}^{ij} = 0$ and the corresponding side condition of Equation (22.5) holds, so the drones are separated. Conversely, if one side condition holds, set its binary to 0 and the other three to 1; those rows then read $x_k^i - x_k^j \geq d_{\min} - M_x$ and $y_k^i - y_k^j \geq d_{\min} - M_y$, which hold for every pair of positions in W because M_x and M_y are the largest violations possible inside W . The square contains the disk of radius $d_{\min}$, so the encoding is conservative, never optimistic.

A convex polygon $P = \{\mathbf{p} : \mathbf{a}_l^\top \mathbf{p} \leq c_l, l = 1, \dots, r\}$ with r edges needs r binaries per step, $\mathbf{a}_l^\top \mathbf{p}_k \geq c_l - M_l b_{k,l}$ with $M_l = c_l - \min_{\mathbf{p} \in W} \mathbf{a}_l^\top \mathbf{p}$, and $\sum_{l=1}^r b_{k,l} \leq r - 1$: outside a convex set means on the far side of at least one of its supporting half-planes. A box in three dimensions is the same with $r = 6$ faces, six binaries and $\sum_j b_{k,j} \leq 5$.

Four drones, two hexagonal obstacles and $N = 20$ therefore need $6 \cdot 4 \cdot 2 \cdot 20 = 960$ obstacle binaries and $4 \binom{4}{2} \cdot 20 = 480$ separation binaries, 1 440 in all – about three times the 480 of four drones with one rectangle, and the reason a polygonal approximation of a round obstacle is expensive.

Exercise 22.4. With $v_{\max} = 3$ m/s and $\Delta t = 0.5$ s, $\delta = v_{\max} \Delta t / 2 = 0.75$ m and the sampled separation must be $d_{\min} + v_{\max} \Delta t = 2.5$ m. The inflation must be applied on all four sides because the proposition bounds the ∞ -distance to the nearest sample in every direction, and a drone skimming the top edge can dip below it between two samples just as one passing the left edge can drift right. The self-test of `ch22_milp.py` solves the safe instance: fuel 16.78 m/s instead of 12.02 (a price of 40% for a guarantee), about 2.6 s and 48 nodes, no penetration of the original block between samples and a continuous-time minimum separation of 4.08 m, far above the required 1 m because the margins assume top speed in the worst direction at every step. Inflating the block only, with $d_{\min} = 1$ m at the samples, gives a plan whose separation between samples drops to 0.06 m: both margins are needed.

Exercise 22.5. Let k_i be the step with $w_{k_i}^i = 1$; the choice row $\sum_k w_k^i = 1$ makes it unique. For $k \geq k_i$ the sum $\sum_{k' \leq k} w_{k'}^i$ equals 1, the right-hand side of the last row of Equation (22.9) is $M_g \cdot 0 = 0$, and $|p_{k,d}^i - p_{g,d}^i| \leq 0$ forces $\mathbf{p}_k^i = \mathbf{p}_g^i$ in both coordinates: the drone is at its goal at every sampled instant from its arrival step on. For $k < k_i$ the sum is 0, the right-hand side is M_g , and with M_g the workspace diameter the row holds for every position in W : it is vacuous.

Two plans can share the arrival step and differ in fuel. In Example 22.6 both drones can arrive at $k = 9$ by accelerating hard for one step, coasting and decelerating hard once, or by wandering left and right first and still arriving at $k = 9$; J_{time} is identical, J_{fuel} is not, and the solver may return either. The tie-breaker $10^{-3} J_{\text{fuel}}$ picks the cheaper one and makes the optimum well defined. The multiple must be small enough never to buy an arrival step: one step costs $\Delta t = 0.5$ s of J_{time} while $10^{-3} J_{\text{fuel}}$ is a few hundredths of a second's worth here.

A time window $[t_a^i, t_b^i]$ needs no new variable, only a narrower choice row,

$$\sum_{k: k\Delta t \in [t_a^i, t_b^i]} w_k^i = 1,$$

which sets every binary outside the window to zero. Forbidding two drones in one arrival step is the “at most one” gadget of Table 22.1, $\sum_i w_k^i \leq 1$ for every k , one row per step (feasible only when $N \geq m$).

Exercise 22.6. Let $z_{i,p,q} \in \{0, 1\}$ mean “drone i lands on pad p in slot q ”, with $p = 1, 2$ and $q = 1, \dots, Q$; slot q ends at $30q$ seconds, which is also the waiting time it charges. The program is

$$\begin{aligned} \min \quad & \sum_i \sum_p \sum_q 30q z_{i,p,q} && \text{total waiting time,} \\ & \sum_p \sum_q z_{i,p,q} = 1 && \text{for every drone } i, \\ & \sum_i z_{i,p,q} \leq 1 && \text{for every pad } p \text{ and slot } q, \\ & \sum_i (z_{i,p,q} + z_{i,p,q+1}) \leq 1 && \text{for every pad } p, q = 1, \dots, Q-1, \\ & z_{i,p,q} = 0 && \text{whenever } 30q > T_i. \end{aligned}$$

The first family is the choice gadget of Table 22.1 (each drone lands exactly once), the second and third are the at-most-one and assignment gadgets: the third forbids two landings on one pad in the same or in adjacent slots, which is the “at least two slots apart” rule, and it implies the second for every slot that has a neighbor. The deadline rows simply delete the variables that violate T_i (fixing the bound to 0 is equivalent). The pairwise form $z_{i,p,q} + z_{i',p,q'} \leq 1$ for $i \neq i'$ and $|q - q'| < 2$ is also correct but needs many more rows and gives a weaker relaxation.

For $m = 6$ drones, two pads and $Q = 8$ slots there are $6 \cdot 2 \cdot 8 = 96$ binaries, 6 assignment rows, $2 \cdot 8 = 16$ pad-slot rows and $2 \cdot 7 = 14$ consecutive-slot rows, minus the deadline variables that are fixed to zero (the pairwise form would need $2 \cdot 7 \cdot 6 \cdot 5 = 420$ rows in place of the 14). Two pads with the two-slot rule take at most $2 \lceil Q/2 \rceil = 8$ landings, so six drones fit if their deadlines allow. The relaxation of this program is tight – the constraint matrix is close to an assignment matrix – which is why scheduling is the one place in this chapter where a MILP of this size is solved in milliseconds.

Exercise 22.7. Agent 1 has one binary per edge of the time-expanded graph: from each state (v, t) with $t < 2$ there is a wait edge and one move edge per neighbor on the cycle, so $4 \cdot 3 \cdot 2 = 24$ binaries per agent. Flow conservation at $(a, 0)$: $f_{(a,0) \rightarrow (a,1)}^1 + f_{(a,0) \rightarrow (b,1)}^1 + f_{(a,0) \rightarrow (c,1)}^1 = 1$;

at $(b, 1)$: outgoing minus incoming equals 0; at $(d, 2)$: incoming equals 1. Capacity of $(b, 1)$: $f_{(a,0) \rightarrow (b,1)}^1 + f_{(b,0) \rightarrow (b,1)}^1 + f_{(d,0) \rightarrow (b,1)}^1 + (\text{the same for agent 2}) \leq 1$. Anti-swap on $\{a, b\}$ at $t = 0$: $f_{(a,0) \rightarrow (b,1)}^1 + f_{(b,0) \rightarrow (a,1)}^1 + f_{(a,0) \rightarrow (b,1)}^2 + f_{(b,0) \rightarrow (a,1)}^2 \leq 1$. The drawn flows use $(a, 0) \rightarrow (b, 1) \rightarrow (d, 2)$ and $(d, 0) \rightarrow (c, 1) \rightarrow (a, 2)$: every state is entered at most once and no edge is used in both directions. The dashed alternative $(d, 0) \rightarrow (b, 1)$ makes the capacity row of $(b, 1)$ equal to 2. On the 32×32 grid an interior state has five outgoing edges (four moves and a wait), but a state on an edge of the grid has four and a corner state three, so one layer of the time-expanded graph has $\sum_v (\deg v + 1) = 1024 + 2 \cdot (2 \cdot 32 \cdot 31) = 4992$ edges. That gives $20 \cdot 60 \cdot 4992 \approx 6.0 \cdot 10^6$ binaries, the “about six million” of [Section 22.8.1](#); $1024 \cdot 60 \approx 6.1 \cdot 10^4$ capacity rows; and $1984 \cdot 60 \approx 1.2 \cdot 10^5$ anti-swap rows, one per undirected grid edge and step, the grid having $2 \cdot 32 \cdot 31 = 1984$ edges.

Exercise 22.8.

Hints. (a) Build an **Instance** whose three **Vehicle** entries are $(1, 1) \rightarrow (9, 7)$, $(1, 4) \rightarrow (9, 4)$ and $(1, 7) \rightarrow (9, 1)$, with the obstacle $[4, 6] \times [2, 6]$, the workspace $[0, 10] \times [0, 8]$, `horizon=12`, `dt=0.5`, `a_max=2`, `v_max=3`, `d_min=1` and `objective="fuel"`; pass it to `solve_instance(inst, time_limit=120)` and repeat the run with `objective="time"`. The fuel model has 3 obstacle groups and 3 pair groups per step, hence $4 \cdot 12 \cdot 6 = 288$ binaries among 588 variables and 672 rows. The reference run finishes inside the limit with `sol.status = 0`: the fuel optimum is $J_{\text{fuel}} = 21.19$ m/s after 9755 branch-and-bound nodes and about 44 s, tens of times the two-drone instance of [Example 22.6](#). The minimum-time model (324 binaries) is much easier, 11 nodes and about 2 s, with arrival times summing to 13.5 s (objective 13.535 including the fuel tie-breaker). If your run hits the limit, report `sol.status = 1` and `sol.gap` instead of concluding that the model is wrong. `continuous_check` shows what the coarse Δt costs: the fuel plan cuts the block by 0.45 m and its separation drops to 0.83 m between samples, so the margins of [Proposition 22.9](#) are needed before such a plan is flown. (b) A separation constraint is active at step k when `check_solution`-style distances give $\|\mathbf{p}_k^i - \mathbf{p}_k^j\|_\infty = d_{\min}$ within 10^{-6} . (c) With the limit the status is 1 and `sol.gap` holds the relative gap; the scaling experiment of the chapter obtained gaps below 10 % after 30 s for this instance. (d) Fix the bound of the binary `model.cpair((0, 1), k, 0)` to 1 for all k before calling `model.solve`. By the convention of [Equation \(22.6\)](#) this *switches off* the row “drone 1 at least $d_{\min}$ to the right of drone 2”, so that pair must be separated in one of the three other directions at every step and the mirror-image subtree disappears; compare `sol.nodes` and `sol.solve_time`. The optimum is unchanged only when an optimal plan that never relies on that direction exists, so check the objective against the unconstrained run before trusting the faster one.

Exercise 23.1. With the edges $\{1, 2\}, \{2, 3\}, \{3, 4\}, \{4, 1\}, \{1, 3\}$ the degrees are $(3, 2, 3, 2)$ and

$$\mathbf{L} = \begin{pmatrix} 3 & -1 & -1 & -1 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & -1 \\ -1 & 0 & -1 & 2 \end{pmatrix}.$$

Every row sums to zero, so $\mathbf{L}\mathbf{1} = \mathbf{0}$. For $\mathbf{x} = (1, 0, 0, 1)^\top$ the quadratic form is $\mathbf{x}^\top \mathbf{L} \mathbf{x} = \sum_{\{i,j\} \in E} (x_i - x_j)^2 = 1 + 0 + 1 + 0 + 1 = 3$ (the edges $\{1, 2\}$, $\{3, 4\}$ and $\{1, 3\}$ each contribute 1). Guess eigenvectors orthogonal to $\mathbf{1}$: $\mathbf{L}(0, 1, 0, -1)^\top = 2(0, 1, 0, -1)^\top$, $\mathbf{L}(1, 0, -1, 0)^\top = 4(1, 0, -1, 0)^\top$ and $\mathbf{L}(1, -1, 1, -1)^\top = 4(1, -1, 1, -1)^\top$, so the spectrum is $\{0, 2, 4, 4\}$ (check: the trace is 10). Hence $\lambda_2 = 2$, the time constant is $1/\lambda_2 = 0.5$ s, the safe step size is $h < 1/d_{\max} = 1/3$ and the exact bound is $h < 2/\lambda_4 = 1/2$. A useful shortcut: the

complement of the graph in K_4 is the single edge $\{2, 4\}$ with Laplacian spectrum $\{0, 0, 0, 2\}$, and $\mathbf{L}(G) + \mathbf{L}(\bar{G}) = 4\mathbf{I} - \mathbf{1}\mathbf{1}^\top$ gives the non-zero eigenvalues of G as $4 - \{2, 0, 0\}$.

Exercise 23.2. (a) $\mathbf{1}^\top \mathbf{x}_{k+1} = \mathbf{1}^\top (\mathbf{I} - h\mathbf{L}) \mathbf{x}_k = \mathbf{1}^\top \mathbf{x}_k$ because $\mathbf{1}^\top \mathbf{L} = \mathbf{0}^\top$, so the sum and hence the average is the same at every step, for every h . Row i of the update reads $x_{i,k+1} = (1 - hd_i)x_{i,k} + h \sum_j w_{ij} x_{j,k}$: the weights are non-negative when $h \leq 1/d_{\max}$ and they sum to 1, so $x_{i,k+1}$ is a convex combination of states at step k and cannot exceed $\max_j x_{j,k}$ (nor fall below the minimum). (b) The two components are $\{1, 2\}$ and $\{3, 4\}$, and the gap that separates them is the 9 m between drone 2 at $x = 1$ m and drone 3 at $x = 10$ m: no smaller R_{comm} adds any edge, so the four agree globally exactly when $R_{\text{comm}} \geq 9$ m. At $R_{\text{comm}} = 9$ m the graph is the path $1 - 2 - 3 - 4$, whose Laplacian spectrum is $2 - 2\cos(k\pi/4) = \{0, 2 - \sqrt{2}, 2, 2 + \sqrt{2}\}$: $\lambda_2 = 2 - \sqrt{2} = 0.586$ and the time constant is $1/\lambda_2 = 1.71$ s, nearly twice the 1 s of [Example 23.7](#), because agreement now has to travel along a chain. At $R_{\text{comm}} = 10$ m the edges $\{1, 3\}$ and $\{2, 4\}$ appear as well (only the 11 m pair $\{1, 4\}$ is still out of range), the spectrum becomes $\{0, 2, 4, 4\}$ as in [Exercise 23.1](#), $\lambda_2 = 2$ and the time constant 0.5 s, twice as fast as the $\lambda_2 = 1$ of [Example 23.7](#). Range buys connectivity, and then convergence speed. (c) Write $\mathbf{u}_i = \mathbf{y}_i - \bar{\mathbf{y}}$, so $\sum_i \mathbf{u}_i = \mathbf{0}$. Then $\sum_{i < j} \|\mathbf{y}_i - \mathbf{y}_j\|^2 = \frac{1}{2} \sum_i \sum_j \|\mathbf{u}_i - \mathbf{u}_j\|^2 = \frac{1}{2} (n \sum_i \|\mathbf{u}_i\|^2 + n \sum_j \|\mathbf{u}_j\|^2 - 2 \|\sum_i \mathbf{u}_i\|^2) = n \sum_i \|\mathbf{u}_i\|^2$. For the complete formation graph $|E_F| = n(n-1)/2$, so $e_F^2 = \frac{2}{n(n-1)} \cdot n \sum_i \|\mathbf{u}_i\|^2 = \frac{2n}{n-1} e_c^2$ using $e_c^2 = \frac{1}{n} \sum_i \|\mathbf{u}_i\|^2$. For [Example 23.7](#), $n = 4$ gives $e_F = \sqrt{8/3} e_c = 1.633 \times 1.794 = 2.930$ m over all six pairs, larger than the 2.500 m of the four edges of the communication graph, which omits the two pairs $\{1, 4\}$ and $\{2, 4\}$.

Exercise 23.3. Write $\mathbf{P} = \mathbf{I} - h\mathbf{L}$. It is symmetric, and if $\mathbf{L}\mathbf{v}_k = \lambda_k \mathbf{v}_k$ then $\mathbf{P}\mathbf{v}_k = (1 - h\lambda_k)\mathbf{v}_k$, so $\mathbf{P}$ has the eigenvalues $\mu_k = 1 - h\lambda_k$ with the same orthonormal eigenvectors. Because $\mathbf{1}^\top \mathbf{L} = \mathbf{0}^\top$ we have $\mathbf{1}^\top \mathbf{x}_{k+1} = \mathbf{1}^\top \mathbf{x}_k$: the average $\bar{x}$ is preserved. Let $\delta_k = \mathbf{x}_k - \bar{x}\mathbf{1}$; then $\delta_{k+1} = \mathbf{P}\delta_k$ and $\delta_k \perp \mathbf{1}$ for all k . Expanding $\delta_0 = \sum_{k \geq 2} c_k \mathbf{v}_k$ gives $\delta_m = \sum_{k \geq 2} c_k \mu_k^m \mathbf{v}_k$ and, by orthonormality, $\|\delta_m\| \leq \rho^m \|\delta_0\|$ with $\rho = \max_{k \geq 2} |1 - h\lambda_k| = \max(|1 - h\lambda_2|, |1 - h\lambda_n|)$. It remains to show $\rho < 1$: since the graph is connected, $\lambda_2 > 0$, and $h > 0$ gives $1 - h\lambda_k < 1$ for $k \geq 2$; since $\lambda_n \leq 2d_{\max}$ (Gershgorin: every eigenvalue of $\mathbf{L}$ lies in a disk centered at some d_i with radius d_i) and $h < 1/d_{\max}$, we get $1 - h\lambda_k \geq 1 - h\lambda_n > 1 - 2 = -1$. So every μ_k , $k \geq 2$, lies strictly inside $(-1, 1)$, $\rho < 1$, and $\mathbf{x}_m \rightarrow \bar{x}\mathbf{1}$ geometrically. The exact condition is $h < 2/\lambda_n$; the bound $1/d_{\max}$ only uses information every drone has locally.

Exercise 23.4. (a) With $w_{ij} = 1$ when i hears j , $w_{21} = w_{32} = 1$ and $\mathbf{D} = \text{diag}(0, 1, 1)$, so $\dot{x}_1 = 0$, $\dot{x}_2 = x_1 - x_2$, $\dot{x}_3 = x_2 - x_3$. Drone 1 is a root that reaches 2 and 3, so the digraph has a directed spanning tree. Integrating, $x_1 \equiv x_1(0)$, $x_2(t) = x_1(0) + (x_2(0) - x_1(0))e^{-t}$ and $x_3(t) = x_1(0) + (x_3(0) - x_1(0))e^{-t} + (x_2(0) - x_1(0))te^{-t}$: every state converges to $x_1(0)$. Solving $\mathbf{w}^\top \mathbf{L} = \mathbf{0}^\top$ with $\mathbf{w}^\top \mathbf{1} = 1$ gives $\mathbf{w} = (1, 0, 0)^\top$, which is exactly “the value of the root”. (b) Adding “drone 1 hears drone 3” makes the digraph the directed cycle $1 \leftarrow 3 \leftarrow 2 \leftarrow 1$; every drone has in-degree and out-degree 1, so the digraph is balanced, $\mathbf{1}^\top \mathbf{L} = \mathbf{0}^\top$, the sum is conserved and $\mathbf{w} = \mathbf{1}/3$: the states converge to the average. (c) With only $w_{21} = w_{23} = 1$, drones 1 and 3 have no incoming edge, so neither can be reached from any root and there is no directed spanning tree. Here $\dot{x}_1 = \dot{x}_3 = 0$ and $\dot{x}_2 = x_1 + x_3 - 2x_2$, so the limits are $x_1(0)$, $\frac{1}{2}(x_1(0) + x_3(0))$ and $x_3(0)$: consensus only if $x_1(0) = x_3(0)$. No protocol can do better, because drones 1 and 3 receive nothing at all and can therefore never learn of each other.

Exercise 23.5. (a) Summing the protocol over i gives $\sum_i \dot{\mathbf{p}}_i = \sum_i \sum_j w_{ij} (\mathbf{p}_j - \mathbf{p}_i) - \sum_i \sum_j w_{ij} \mathbf{d}_{ij}$. The first double sum vanishes because the graph is undirected (each pair contributes $(\mathbf{p}_j - \mathbf{p}_i) + (\mathbf{p}_i - \mathbf{p}_j)$). In general, therefore, the centroid moves with the constant velocity $-\frac{1}{n} \sum_i \sum_{j \in N_i} \mathbf{d}_{ij}$ and the swarm drifts away for ever, even though the relative positions settle. That velocity vanishes whenever $\mathbf{d}_{ji} = -\mathbf{d}_{ij}$ on every edge, because the two terms of each edge then cancel

in pairs; edgewise antisymmetry is sufficient but not necessary, since only the total sum has to vanish. On the path $1-2-3$ with $\mathbf{d}_{12} = \mathbf{d}_{21} = (1, 0)^\top$ and $\mathbf{d}_{23} = \mathbf{d}_{32} = (-1, 0)^\top$ the four terms cancel across the two edges and the centroid stands still, although no edge is antisymmetric. (b) With antisymmetric $\mathbf{d}_{ij}$ the protocol is $\dot{\mathbf{p}}_i = -\nabla_{\mathbf{p}_i} J$ for $J(\mathbf{p}) = \frac{1}{2} \sum_{\{i,j\} \in E} \|\mathbf{p}_j - \mathbf{p}_i - \mathbf{d}_{ij}\|^2$ (differentiate one term with respect to $\mathbf{p}_i$ to check). J is a convex quadratic bounded below, so gradient descent converges to a minimizer and J decreases monotonically. If the vectors close around every cycle, $J_{\min} = 0$ and the minimizers are the translated formations; otherwise $J_{\min} > 0$ and the swarm settles into a least-squares compromise with residual formation error $e_F = \sqrt{2J_{\min}/|E|}$. For the triangle with $\mathbf{d}_{12} = \mathbf{d}_{23} = (1, 0)^\top$ and $\mathbf{d}_{31} = (-1, 0)^\top$ the mismatch $(1, 0)^\top$ is spread evenly, each edge ends with error $(1/3, 0)^\top$ and $e_F = 1/3$ m. (c) Storing offsets $\mathbf{o}_i$ and computing $\mathbf{d}_{ij} = \mathbf{o}_j - \mathbf{o}_i$ makes both antisymmetry and cycle closure automatic.

Exercise 23.6. (a) $(\mathbf{p}_i, \mathbf{p}_j) \mapsto \mathbf{p}_i - \mathbf{p}_j$ is linear and the ball $\|\mathbf{z}\| \leq R_{\text{comm}}$ is convex, and the preimage of a convex set under a linear map is convex. In the MPC of [Chapter 21](#) the constraint is $\|\mathbf{p}_i(k) - \mathbf{p}_j(k)\| \leq R_{\text{comm}}$ for every kept edge $\{i, j\} \in E_{\text{keep}}$ and every step $k = 1, \dots, N$ of the horizon: a second-order cone in the decision variables. (b) The Q inequalities describe the regular Q -gon whose facets are tangent to the circle of radius $R_{\text{comm}} \cos(\pi/Q)$; its circumradius is $R_{\text{comm}} \cos(\pi/Q) / \cos(\pi/Q) = R_{\text{comm}}$, so the polygon is inscribed in the disk and every plan that satisfies the inequalities keeps $\|\mathbf{p}_i - \mathbf{p}_j\| \leq R_{\text{comm}}$. The price is paid in the facet directions, where only $R_{\text{comm}} \cos(\pi/Q)$ of the range is usable: for $Q = 8$ that is $1 - \cos(\pi/8) = 7.6\%$, or 0.24 m of a 3.2 m radio. (c) For $\ell_{\text{act}} < \ell < R_{\text{comm}}$, $V'_{\text{range}}(\ell) = \frac{(\ell - \ell_{\text{act}})[2(R_{\text{comm}} - \ell) + (\ell - \ell_{\text{act}})]}{(R_{\text{comm}} - \ell)^2}$, which tends to 0 as $\ell \downarrow \ell_{\text{act}}$ and to $+\infty$ as $\ell \uparrow R_{\text{comm}}$. The velocity $-V'(\ell)$ therefore matches the value 0 of the inactive branch: the term switches on continuously, with no jump in the command. (d) λ_2 is one number for the whole graph: it falls when *some* cut is about to open, but it does not say which edge, and an edge of E_{keep} can be at $0.99R_{\text{comm}}$ while λ_2 is still comfortable because other edges hold the graph together. Log the individual lengths ℓ_{ij} of the kept edges, or the margin $\min_{\{i,j\} \in E_{\text{keep}}} (R_{\text{comm}} - \ell_{ij})$, alongside λ_2 .

Exercise 23.7. (a) In the eigenbasis of $\mathbf{L}$ the mode with eigenvalue λ obeys $\ddot{q} + (k_v \lambda + k_d)\dot{q} + k_p \lambda q = 0$, with characteristic polynomial $s^2 + (k_v \lambda + k_d)s + k_p \lambda$. A quadratic with positive coefficients has both roots in the open left half-plane, so every mode with $\lambda > 0$ decays if $k_p > 0$ and $k_v \lambda + k_d > 0$; the mode $\lambda = 0$ gives $s \in \{0, -k_d\}$. With $k_v = k_d = 0$ the roots are $s = \pm i\sqrt{k_p \lambda}$: undamped oscillation at the angular frequencies $\sqrt{k_p \lambda_k}$, which for the example graph and $k_p = 1$ are 1, $\sqrt{3}$ and 2 rad/s. For $k_d = 0$ the damping ratio of mode λ is $\zeta = \frac{1}{2}k_v \sqrt{\lambda/k_p}$; choosing $k_v = 2\sqrt{k_p/\lambda_2}$ makes the slowest mode critically damped and the faster ones overdamped. (b) Without feed-forward the error $\mathbf{y} = \mathbf{p} - \mathbf{o} - \mathbf{r}(t)$ of the first-order leader-follower controller obeys $\dot{\mathbf{y}} = -(k_c \mathbf{L} + k_l \mathbf{B})\mathbf{y} - \mathbf{1} \otimes \dot{\mathbf{r}}$, so with $\dot{\mathbf{r}} = v \mathbf{e}$ constant the steady state is $\mathbf{y}_{ss} = -v((k_c \mathbf{L} + k_l \mathbf{B})^{-1} \mathbf{1}) \otimes \mathbf{e}$. For the example graph, $k_c = k_l = 1$ and drone 1 pinned, solving $(\mathbf{L} + \mathbf{B})\mathbf{z} = \mathbf{1}$ gives $\mathbf{z} = (4, 16/3, 17/3, 20/3)^\top$: at $v = 1$ m/s the leader trails its reference by 4 m and the tail drone by 6.67 m, and the edge errors $z_j - z_i$ give $e_F = \sqrt{(16/9 + 25/9 + 1/9 + 1)/4} \approx 1.19$ m for ever. Adding $\dot{\mathbf{r}}$ to every drone's command removes the forcing term and the lag.

Exercise 23.8. Expected output, so that you can check your own implementation against `code/ch23_consensus.py` without reading it. (a) The radius graph of [Example 23.7](#) at $R_{\text{comm}} = 3$ m has the four edges $\{1, 2\}, \{1, 3\}, \{2, 3\}, \{3, 4\}$ and the spectrum $\{0, 1, 3, 4\}$, so $\lambda_2 = 1$ and $1/\lambda_2 = 1$ s. (b) From the same initial positions the square settles at $\mathbf{o}_i + (0.625, 0.125)$ m, the template translated by the initial centroid shift of [Theorem 23.13](#), with $e_F(0) = 2.500$ m falling below 1% of that after 4.1 s. Pinned to the waypoint path, the formation error after 20 s is 0.0025 m with feed-forward and 1.00 m without it: the lag of [Proposition 23.12](#) does

not decay. (c) With $R_{\text{comm}} = 3.2$ m the square is a complete graph ($\lambda_2 = 4$ throughout, sides 2 m and diagonals 2.83 m), the intruder is within 2 m of some drone from $t = 6.9$ s to $t = 11.7$ s, the formation error peaks at $e_F = 0.36$ m at $t = 10.5$ s and is back below 0.05 m at $t = 13.6$ s; the closest approach to the intruder is 0.91 m, the longest edge 3.04 m, and no edge ever exceeds R_{comm} . (d) At $R_{\text{comm}} = 2.5$ m the diagonals drop out, the nominal graph is the 4-cycle, whose spectrum $\{0, 2, 2, 4\}$ gives $\lambda_2 = 2$; λ_2 is no longer constant, since a diagonal that the maneuver shortens below 2.5 m adds an edge and one that it stretches removes it again. Recovery is slower because the slowest disagreement mode now decays at half the rate, and without the range barrier a drone pushed outward loses both of its edges and the swarm splits into two pieces that converge to different places, exactly the pitfall of a disconnected graph.

Exercise 24.1. (a) The stop form of Equation (24.5) gives $\tau_h \geq \Delta t + v_{\text{max}}/a_{\text{max}} = 0.05 + 1 = 1.05$ s, the reversal form $\tau_h \geq 0.05 + 2 = 2.05$ s, and the hand-over condition $\tau_h \geq \tau = 2$ s. If a stop is enough, $\tau_h = 2$ s satisfies everything; if the local layer may have to reverse the velocity, $\tau_h = 2.05$ s is the smallest choice, and rounding to a multiple of Δt gives 2.05 s exactly. (b) The intruder must be tracked before it enters the horizon: the range must cover the closing distance during the warm-up and the horizon plus the inflated separation, $(v_{\text{cruise}} + v_B)(\tau_h + n_{\text{warm}}\Delta t) + R_{\text{safe}} + \kappa_p\sigma(\tau_h) = (1.5 + 3)(2 + 20 \cdot 0.05) + 1 + 0.5 = 15$ m, the closing speed being the cruise speed because the drone is Nominal while it waits for the trigger; a drone that may already be flying at $v_{\text{max}} = 2$ m/s when the intruder appears needs $(2 + 3) \cdot 3 + 1.5 = 16.5$ m. (c) With $\tau_h < \tau$ the executive hands control to ORCA only when the current relative velocity already lies inside the truncated velocity obstacle VO^τ , so the first half-plane demands a large change of velocity at once, which a drone with an acceleration limit may not be able to execute and which makes the program more likely to be infeasible; with $\tau_h \geq \tau$ the first correction is small. With τ_h far beyond the look-ahead at which $\kappa_p\sigma(s)$ reaches the lane spacing, the inflated disk of an intruder in the *next* lane covers the drone's own lane, the trigger fires for intruders that would never come close, drones leave their plans needlessly, and every needless deviation costs reconnections and re-checks; the trigger becomes a false-alarm generator rather than a safety device.

Exercise 24.3. (a) Line 14 of Algorithm 24.1 replaces π_i by its tail from index j and sets $t_0^i \leftarrow t_0^i + j + \Delta$, so the waypoint $\pi_i[j]$ is now scheduled at grid time $t_0^i + j + \Delta$ and every later waypoint, including the goal, at exactly Δ steps later than before; the arrival time of a_i rises by Δ . No other path changes, so SoC rises by exactly Δ and MS by at most Δ (by less if another drone arrives later than a_i), provided the re-check finds no conflict, because a repair would change further paths. (b) With $\Delta = 0$ line 14 is skipped and (π_i, t_0^i) is unchanged, hence the reservation that the other drones see is unchanged; the plan in force was conflict-free before the deviation (the invariant maintained by Propositions 24.5 and 24.8), so it is conflict-free after. The only new motion is the straight segment, which line 14 of Algorithm 24.3 has verified against the teammates' intended positions and line 13 against the prediction; physical separation on the segment is therefore assured without any re-check of the plans. (c) Build the simulation, step it until $t = 4.9$ s, take C's nominal path from index 5 with $t_0 = 6$ and B's nominal path with $t_0 = 0$, and call `first_conflict` on the pair from $t = 4$: C occupies $(6, 2)$ at $t = 7$ and so does B, giving $\langle B, C, (6, 2), 7 \rangle$. The teammate test of the segment checks only the interval of the segment itself, $t \in [4.9, 6]$, during which B is still at $(6, 3)$; the conflict arises at $t = 7$, after C has rejoined its plan, and it is a property of the *shifted plan*, which no line of Algorithm 24.3 examines. That is precisely why every delayed reconnection is followed by the re-check.

Exercise 24.4. Padding extends a path backward to its first cell, so a detector started at grid time 0 places A at $(6, 5)$ for all $t \leq 9$; B's nominal path visits $(6, 5)$ at $t = 3$, and the detector

reports $\langle A, B, (6, 5), 3 \rangle$, a conflict between a drone that did not exist at $(6, 5)$ at that time and one that flew through it six seconds before the replan. Started at $\lfloor t \rfloor = 9$, the detector compares A at $(6, 5)$ for $t = 9, 10$, $(6, 6)$ for $t = 11$, and so on along row 6, with B at $(6, 1)$ at $t = 9$ and parked at $(6, 0)$ from $t = 10$ (its repaired path) and with C along row 1 and row 2: no cell is shared at any time, and no swap occurs, so nothing is reported. Start times are read on line 2 of Algorithm 24.3 (j_0 from $t/\Delta T - t_0^i$) and in the arrival time $t_j = (t_0^i + j + \Delta)\Delta T$; written on line 14 of Algorithm 24.1 ($t_0^i \leftarrow t_0^i + j + \Delta$); read on line 3 of Algorithm 24.4 (the reservation of every (π_j, t_0^j)) and on line 9 (the detector); and written on line 8 ($t_0^i \leftarrow t_{\text{start}} - 1$) and after the local CBS ($t_0^i \leftarrow t_0^j \leftarrow t_{\text{start}}$). Every one of them is an absolute grid time, and t_{start} itself is computed from the absolute clock t .

Exercise 24.2. The particle trigger fires at look-ahead s when $\sum \left\{ w^{(n)} : \|\tilde{\mathbf{p}}_i(s) - \mathbf{p}_h^{(n)}\| < R_{\text{safe}} \right\} > 1 - p$; no covariance is formed. On a bimodal prediction (half the weight on each side of a building) the Gaussian summary puts its mean in the gap between the modes and gets a covariance whose largest eigenvalue is roughly half the distance between them, so the inflated disk $R_{\text{safe}} + \kappa_p \sigma(h)$ covers the empty lane *between* the modes: the Gaussian trigger fires early, on a place the intruder will almost surely not be, and the replanner blocks that lane as well. The particle trigger fires only when one mode carries more than $1 - p$ of the weight within R_{safe} , that is when the drone's own lane is the threatened one; it costs $\mathcal{O}(N)$ per look-ahead instead of $\mathcal{O}(1)$, and its threshold is a weight, not a count, so unequal weights are handled without resampling.

Exercise 24.5. (a) With a finite cost c_B the search may cross the tube, so the statement “the replanned path is clear of the p -disc at every step” is lost; what survives is Proposition 24.8, which is about conflicts between *plans*, and completeness of the replanner, which blocking can destroy in a corridor. Make c_B depend on how deep the cell lies inside the inflated disk, for instance $c_B = 1 + c_0 (d_h^{\text{safe}} - \|\mathbf{c}(v) - \hat{\mathbf{p}}_{T+h}\|) / (\kappa_p \sigma(h))$, so that a sharp prediction produces a narrow, expensive tube and a vague one a wide, cheap one; a constant cost ignores the covariance and makes the same detour in both cases. Expect A to keep the row 6 detour while c_B exceeds the seven extra steps it costs and to cut through the tube once it is cheaper. (b) Emergency is entered from any state when the local program has been infeasible for m consecutive cycles, brakes to hover (or climbs, if the third dimension is available), and is tested before every other transition of Algorithm 24.1; it leaves to Avoiding when the program is feasible again for n_{clear} cycles, to Replanning when the drone is at rest and a path exists, and never directly to Nominal, because the plan in force has not been re-checked.

Exercise 24.6. (a) C is a follower with $\mathbf{d}_{AC} = (0, -2)$, so while C was Nominal Equation (24.6) added $k_F(\mathbf{p}_A + \mathbf{d}_{AC} - \mathbf{p}_C)$ to its preferred velocity: when A climbed away from the intruder, C's slot climbed with it, C's *intended* positions moved toward the inflated tube, and the trigger of Definition 24.2, which tests intended positions and not the current geometry, fired at $t = 4.3$ s although the intruder never came closer to C than 2.25 m. Had the term stayed active while C was Avoiding, it would have pulled C back toward a slot that lies in the maneuver A is making, the ORCA half-plane would have pushed it out again, and the two would have fought at 10 Hz; dropping the term while Avoiding is what Section 24.8 prescribes. (b) Expect the peak of e_F to fall and its recovery to shorten as k_F grows, at the price of a larger pull into the tube and a smaller minimum separation; ramping the gain over a second removes the step in $\mathbf{v}^{\text{pref}}$ at the entry to Reconnecting without changing the recovery time much. (c) When the *leader* avoids, freeze the formation reference at the leader's nominal reference position $\mathbf{p}_\ell^{\text{ref}}(t)$ instead of its actual position (or hand leadership to a follower that is not in conflict) so that the whole formation does not follow one drone into its maneuver; restore the actual position when the leader is Nominal again.

Exercise 24.7. The rollouts need, per rollout step h , the predicted mean $\hat{\mathbf{p}}_{T+h}$ and the inflated radius $R_{\text{safe}} + \kappa_p \sigma(h)$ (and the teammates' intended positions with R_{mate}), so that the clearance term of Chapter 14 is evaluated against a moving disk rather than a static obstacle. The heading term points at $\mathbf{p}_i^{\text{ref}}(t + t_{\text{la}})$ while Nominal, at the reconnection target while Reconnecting, and at the same pursuit target with the formation term dropped while Avoiding, which keeps DWA's objective aligned with Equation (24.2). ORCA's feasibility flag is replaced by “the admissible window is empty”: no sampled (v, ω) has positive clearance over the rollout, which is the same escalation to Replanning on line 8. With a double integrator the dynamic window enforces a_{max} by construction, so the reversal form of Equation (24.5) becomes the binding constraint on τ_h ; expect a slightly larger minimum separation (DWA brakes rather than sidesteps), a longer time in Avoiding and, because the drift bound is reached later, no more replans than with ORCA.

Exercise 24.8. Checkpoints for the capstone; `ch24_hybrid.py` is the reference build, not the one to copy. Its shape is the advice: one `HybridSimulation` owns the clock, the grid and the drones; one `Drone` record holds the nominal path, the mode and the reconnection bookkeeping; the four layers appear as four methods — `cbs` before take-off, the tracker and the local layer every cycle, `replan` on demand — behind exactly the interfaces of Table 24.1, with the executive in `decide`, `try_reconnect` and `recheck`. Steps: fix the world model of Section 24.3 first, with one conversion between grid cells at integer time and continuous positions at real time, written once; get the state machine of Figure 24.2 running with a perfect predictor before any noise; add the reconnection test of Definition 24.3, the re-check and the replanner; swap the fresh space-time A^* for D^* Lite (Chapter 5) and count expansions per repair against a fresh search, which is the answer to (b); make the three policies three objects with one interface. Expected numbers on the chapter's scenario: the nominal plan costs $\text{SoC} = 35$ with a makespan of 13 after two CT nodes; the intruder is first seen at $t = 1.3$ s; drone A leaves *Nominal* at 3.5 s and drone C at 4.3 s; one reconnection succeeds with a delay of one step and two fail on a blocked segment, each triggering a replan, and there is no collision; the minimum separation is 1.20 m to the intruder and 0.80 m between teammates and the formation error ends at 0.18 m. Classic mistakes: one file that mixes the layers; two different time steps for the grid plan and the continuous loop; a reconnection accepted without re-checking teammates; a re-check run before the replan instead of after. Done when the matrix of (e) runs unattended from one script, every run is reproducible from its seed, and your one-page conclusion says which policy wins where — the Week 12 milestone of Appendix A.

Exercise 25.1. Fix everything the question does not ask about and vary only what it does: $k = 4$ controlled drones, $m = 1$ intruder, the hybrid strategy, no formation requirement and no communication limit, one map, 20 seeds, and the same 20 scenarios for both predictors (a paired design). The one factor varied is prediction quality, at the two levels “constant-velocity” and “learned”; add the “perfect state” level as an upper bound and “noisy state” as a lower one if the budget allows. Baselines: no avoidance (does the scenario need avoidance at all?) and the hybrid with perfect prediction (how much of the gap is prediction and how much is the planner?). Primary metric and comparison, named before the runs: the paired difference in $d_{\text{min}}^{\text{di}}$ between the two predictors, with its 95 % interval, d_z and an exact signed-rank p ; the collision rate is too coarse to decide anything at $N = 20$ (Proposition 25.9). Secondary, reported but not tested: collision and near-miss rates, ρ_L , makespan, replans, computation time per decision, success rate. With the sixth factor added, the full factorial of Table 25.2 grows from $3 \times 4 \times 4 \times 3 \times 4 = 576$ cells to $576 \times 3 = 1728$; the one-factor-at-a-time design around the base cell costs the base cell plus the alternative levels of each factor, $1 + (2 + 3 + 3 + 2 + 3) = 14$ cells, and the crossing angle adds its two other levels for $14 + 3 - 1 = 16$.

Exercise 25.2. (a) Drone–drone: the sampled distances are 2, $\sqrt{5}$, $\sqrt{5}$, $\sqrt{8}$ and the segment

minima are no smaller, so $d_{\min}^{\text{dd}} = 2.0$ m. Drone 1 against the intruder: samples $\sqrt{20}, \sqrt{10}, 2, \sqrt{2}$; on the last interval the relative position goes from $(0, -2)$ to $(1, -1)$, $s^* = 1$, so the segment minimum is again $\sqrt{2} = 1.414$ m. Drone 2 against the intruder: samples $\sqrt{8}, \sqrt{5}, 1, \sqrt{2}$, and on the interval from $t = 2$ to $t = 3$ the relative position goes from $(-1, 0)$ to $(-1, 1)$ with $s^* = 0$: minimum 1.0 m. Hence $d_{\min}^{\text{di}} = 1.0$ m, attained at a sample; in this log no interval minimum is below its endpoint values, which is not true in general. (b) With $R = 0.3 + 0.8 = 1.1$ m the pair (drone 2, intruder) collides: $X_{\text{coll}} = 1$, $n_{\text{coll}} = 1$; with an intruder radius of 0.3 m there is no collision. (c) $L_1 = 3$ m, $L_2 = 1$ m; drone 1 is last away from its goal at $t = 2$, so $T_1 = 3$ s; drone 2 at $t = 1$, so $T_2 = 2$ s; makespan 3 s, sum of costs 5 s. (d) $\mathbf{p}_2 - \mathbf{p}_1 - \mathbf{d}_{12}$ is $(0, 0), (-1, 0), (-1, 0), (-2, 0)$, so $e_F = 0, 1, 1, 2$ m with mean 1.0 m and peak 2 m. (e) The distances are 2, 2.236, 2.236, 2.828: for $R_{\text{comm}} = 2.5$ m the graph is disconnected ($\lambda_2 = 0$) at $t = 3$ only, one violation; for 2.2 m at $t = 1, 2, 3$, three violations. `_selftest_metrics()` in `ch25_evaluation.py` asserts every one of these values.

Exercise 25.3. Hints. (1) Unpaired: “10 random scenarios per method” means different scenarios per row; run all methods on the same seeds. (2) The ORCA row is a number copied from a paper: different scenarios, different machine, different implementation; run ORCA yourself on the same seeds, or drop the row. (3) “Averaged over successful runs” without a success rate hides the failures; add the success rate and the count of unfinished runs, and compare on the commonly finished scenarios. (4) No spread, no N in the table: add intervals and IQRs. (5) “0 % of 10 runs” only bounds the rate: the rule of three gives $p \leq 0.30$ (exactly $1 - 0.05^{1/10} = 0.26$), the Wilson upper limit is 0.28; 0 % and 4 % are indistinguishable at this N . (6) No minimum separation: two methods with zero collisions may differ greatly in $d_{\min}$. (7) Absolute path lengths depend on the scenarios; report the ratio to the nominal plan or to the oracle. (8) Runtimes: state the machine, whether the times are per decision or per run, and add the 99th percentile. (9) “Ours” is presumably tuned; state the tuning seeds and whether the baselines were tuned equally. The rewritten table has one row per (cell, method) with N , success rate, collision rate [Wilson], $d_{\min}$ median (IQR), ρ_L [interval], makespan, replans, $\bar{c}$ and c_{99} , plus paired-difference rows against the primary baseline.

Exercise 25.4. (a) By Proposition 25.8 with $\sigma_A = \sigma_B = \sigma$, the paired variance is $2\sigma^2(1 - \rho)/N_p$ and the unpaired one $2\sigma^2/N_u$; equality needs $N_u = N_p/(1 - \rho) = 20/0.1 = 200$ runs per strategy. (b) Pairing hurts when $\rho < 0$: a scenario feature that helps A must hurt B , for instance a local avoider that profits from head-on geometry (large closing speed makes the velocity obstacle narrow and early) while a replanner suffers from it. In planner evaluations the dominant scenario features — density, closing speed, how well aimed the intruder is — make life harder for every strategy at once, so ρ is positive. (c) Under the null hypothesis each scenario is a fair coin; $\mathbb{P}(X \leq 4 \mid n = 20) = (1 + 20 + 190 + 1140 + 4845)/2^{20} = 6196/1\,048\,576 = 0.0059$, two-sided $p = 0.0118$, which is the 0.012 printed in the p_{sign} column of Table 25.5 for ρ_L of hybrid against replan-only with four drones (the row with 16/4/0). The Wilcoxon p -value of that row is smaller, 0.006, because the sign test discards the sizes of the differences and keeps only their signs: here the four scenarios in which the hybrid flew the longer path lost by less than the sixteen it won by (0.022 against 0.042 on average), and the signed-rank statistic uses exactly that. (d) In 13 of 20 scenarios the hybrid never left its local phase, so its decisions were identical to local-only and the two runs coincide; the seven scenarios in which it differs are exactly those with a replan, and there the hybrid’s $d_{\min}$ was larger every time (it never was smaller). The tie count therefore says how often the hybrid’s extra machinery was used at all.

Exercise 25.5. (a) $(1 - p)^N = 0.05$ with $p = 0.005$ gives $N = \ln 0.05 / \ln 0.995 = 597.6$, so 598 collision-free runs; the rule of three gives $3/0.005 = 600$. A single collision in those runs voids the claim and the count restarts from a larger N . (b) The half-width scales with $1/\sqrt{N}$:

halving it needs $4 \times 20 = 80$ seeds, and 0.005 needs $N = 20 (0.026/0.005)^2 \approx 541$ seeds. (c) The rate is the expensive goal: a mean gains information from every run, while a rate near zero gains it only from the rare events, so a claim about a small rate needs hundreds of clean runs; this is why [Section 25.3](#) insists on reporting the minimum separation, which is observed in every run.

Exercise 25.6. (a) Plot the sorted times against the run index: A rises to 10 at 70 ms, B rises steeply to 8 at 15 ms and then stops, and the gap between B’s plateau and $N = 10$ is its failure rate. (b) Success rates 10/10 and 8/10; means over the finished runs $308/10 = 30.8$ ms and $72/8 = 9.0$ ms; PAR-2 with the 100 ms cap counts an unfinished run as 200 ms, giving 30.8 ms for A and $(72 + 2 \times 200)/10 = 47.2$ ms for B. (c) Neither is “faster”. Write instead: “A solved 10/10 instances within the 100 ms cap, B 8/10; over the runs each solved, A took 30.8 ms on average and B 9.0 ms.” and “Counting an unsolved run as twice the cap (PAR-2), A scores 30.8 ms and B 47.2 ms, so B is the faster planner only where it works at all.” (d) A’s numbers do not move: its slowest run is 70 ms, below either cap. B’s success rate and mean are unchanged if the two runs still fail, but its PAR-2 becomes $(72 + 2 \times 400)/10 = 87.2$ ms — the penalty, not the planner, changed. A mean runtime quoted without its cap and success rate is therefore uninterpretable: it is an average over an unstated subset, and the subset moves with the cap.

Exercise 25.8. Sketch. A physics simulator does not execute a commanded velocity at once: with a first-order tracking lag of $\tau = 0.3$ s the drone still flies close to its old velocity for about τ after the avoidance maneuver is decided, so the maneuver starts late and the minimum separation falls by roughly the lag times the closing speed, here $0.3 \text{ s} \times 2\text{--}3 \text{ m/s} \approx 0.6\text{--}0.9 \text{ m}$ — enough to turn most of the toy’s near misses into collisions. The acceleration limit rounds the corners of the dodge and costs a little path length and makespan; the physics rate changes how finely $d_{\min}$ is sampled, not what the planner does; the computation time per decision barely moves, because the planner is the same code. The margin that restores the toy’s near-miss rate is therefore about one lag of closing distance on the safety radius, $r_{\text{safe}} + \tau v_{\text{close}} \approx 1 + 0.75 = 1.75 \text{ m}$, and the comparison should be reported paired by seed, toy against simulator, one row per metric.

Exercise 25.7. Checkpoints. `ch25_evaluation.py` is built as the chapter asks you to build yours: `make_scenario` turns a seed and a cell into a scenario, `run_episode` returns a log and nothing else, `compute_metrics` turns one log into the ten numbers of [Table 25.1](#), `run_study` loops over cells, strategies and seeds, and `summarize`, `wilson` and `holm` aggregate. Keep one flat table of raw rows, one per run, carrying cell, strategy, seed and every metric; write it to disk and never aggregate inside the simulator. Steps: fix the seed contract first, so that the same seed gives the same scenario for *every* strategy and [Proposition 25.8](#) applies; run the sanity baseline before anything else, since it must collide when an intruder is aimed at a drone and never when $m = 0$; budget the run, knowing that the four-cell mini-study is 320 runs in about 15 s, so the one-factor-at-a-time design is minutes rather than hours; aggregate by script, with Wilson intervals for rates and t intervals for means. Expected numbers: with $m = 0$ all four strategies produce identical rows (path ratio 0.999, makespan 13.2), which is a test and not a result; with $k = 4$, $m = 1$ the sanity baseline collides on 75 % of seeds, `local` and `hybrid` on 5 % and `replan` on none, at path ratios 1.03, 1.05 and 1.08 and means of 0, 1.5 and 7.5 replans. The paired comparison carries the conclusion: hybrid against replan-only differs by -5.95 replans $[-7.59, -4.31]$ and -0.71 m of minimum separation, and against local-only by $+0.021$ $[+0.006, +0.036]$ of path ratio, a real but tiny effect. Classic mistakes: reporting success rate only; unpaired comparisons over paired seeds; a mean path ratio taken over runs that ended in a collision; one p -value per cell with no correction. Done when the matrix runs

unattended, every row is reproducible from its seed, and the figures and the table come out of the raw rows with no manual step — the Week 12 milestone of [Appendix A](#). Part (d) tests the interface: if swapping the stand-ins for your real modules touches anything but `run_episode`, the seam was in the wrong place.

Glossary

This glossary collects the terms that the book sets in bold when it defines them. Each entry names the chapter that introduces the term; a term used in several chapters points to the first.

Activation length ℓ_{act}

The link length, below the communication range R_{comm} , at which the soft range term of the preferred velocity starts to pull a drone toward its nearest teammate. (Chapter 24)

Active constraint

An inequality constraint that holds with equality at the solution and has a positive Lagrange multiplier; it is the constraint that shapes the solution. (Chapter 21)

ADE / FDE / minADE_K

Average displacement error, the mean Euclidean error over the prediction horizon, and final displacement error, the error at the last step, with per-horizon versions ADE_h , FDE_h ; minADE_K and minFDE_K score the best of K sampled trajectories and are not comparable with the ADE/FDE of a deterministic predictor. (Chapter 20)

Adjacency list

A representation of a graph that stores, for every vertex, the list of its neighbors together with the edge weights. (Chapter 2)

Admissible heuristic

A heuristic that never overestimates, $0 \leq h(n) \leq h^*(n)$; it guarantees that A^* with re-opening returns an optimal path. (Chapter 4)

Admissible velocity

A velocity from which the robot can still stop before the nearest obstacle on its rollout: $v \leq \sqrt{2 \text{dist } a_b}$, or $v\Delta t + v^2/(2a_b) \leq \text{dist}$ when the command is held for one interval before braking starts. The rotational condition $|\omega| \leq \sqrt{2 \text{dist } \dot{\omega}_b}$ of Fox et al. is a heuristic cap rather than a braking bound: the braking argument for the rotation uses the heading change $\Delta\theta = |\omega| \text{dist} / v$ still available, not the free distance itself. (Chapter 14)

ADMM

The alternating direction method of multipliers, a first-order splitting method for convex problems; the OSQP iteration used by the book's QP solver. (Chapter 21)

Agent

One of the k entities of a MAPF instance, with its own start vertex and goal vertex. (Chapter 7)

Agent-centered frame

The input normalization in which the last observed position is the origin, the last heading is the x -axis and displacements are divided by a typical step length; it makes the predictor translation and rotation invariant. (Chapter 20)

Algebraic connectivity

The second-smallest eigenvalue λ_2 of the graph Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{A}$; it is positive iff the graph is connected and sets the convergence rate of consensus, while $\lambda_n \leq 2d_{\text{max}}$ bounds the safe step size. (Appendix B, Chapter 23)

Angle wrapping

Mapping an angle to $(-\pi, \pi]$ with $\text{wrap}(\theta) = \theta - 2\pi[(\theta - \pi)/(2\pi)]$; applied to every

difference of angles in an innovation, a residual or a likelihood, and to the heading after every update. (Chapter 19)

Anytime algorithm

An algorithm that returns a first solution quickly and improves it for as long as it is allowed to run. (Chapter 1, Chapter 6)

Anytime D*

The combination of ARA* with D* Lite: an anytime planner whose *g/rhs* bookkeeping also repairs the search when edge costs change, inflating the keys of overconsistent states only. (Chapter 6)

Apex (of a velocity obstacle)

The vertex of the wedge, at $\mathbf{v}_B$ for a velocity obstacle, at the origin for a static obstacle and at $(\mathbf{v}_A + \mathbf{v}_B)/2$ for a reciprocal velocity obstacle. (Chapter 12)

Artificial potential field

A scalar function $U(\mathbf{p})$ over the free space, low at the goal and high near obstacles, whose negative gradient $-\nabla U$ is used as the velocity (or force) command of a robot. (Chapter 15)

Asymptotic optimality

The property that the cost of the best path in the tree converges to the optimal cost c^* with probability one as the number of samples grows; RRT* has it, RRT does not. (Chapter 17)

Attention

The operation $\text{softmax}(\mathbf{Q}\mathbf{K}^T/\sqrt{d_k})\mathbf{V}$ that returns for every query a convex combination of the values weighted by the similarity of the query to the keys; over time steps it replaces recurrence, over agents it models interaction, at cost $\Theta(n^2d)$. (Chapter 20)

Attractive potential

The goal term $U_{\text{att}} = \frac{1}{2}k_{\text{att}}\|\mathbf{p} - \mathbf{p}_g\|^2$ (quadratic) or $k_{\text{att}}d^*\|\mathbf{p} - \mathbf{p}_g\| - \frac{1}{2}k_{\text{att}}(d^*)^2$ beyond the switch distance d^* (conic); the hybrid of the two keeps the force bounded far away and smooth at the goal. (Chapter 15)

Back set

The set of all nodes from which a node has been generated; it must contain every parent, not only the cheapest one, for backpropagation to be complete. (Chapter 11)

Backpropagation (of collision sets)

Passing a collision set from a node to every node in its back set, recursively, and re-opening every node whose set grew so that it is expanded again with more neighbors. (Chapter 11)

Backward Dijkstra

A run of Dijkstra's algorithm from the goal over the graph with every edge reversed, which yields the distance to the goal from every vertex. (Chapter 3)

Bayes' rule

$p(\mathbf{x} | \mathbf{z}) \propto p(\mathbf{z} | \mathbf{x})p(\mathbf{x})$: the posterior is the likelihood times the prior, normalized by $p(\mathbf{z})$; applied after the law of total probability it gives the predict–update recursion that every filter of the book runs. (Appendix B)

BCBS(w_H, w_L)

Bounded CBS: focal search with factor w_L at the low level and w_H at the high level, the latter measured against the smallest cost in OPEN instead of a lower bound; guarantee $w_H w_L C^*$ only. (Chapter 10)

Bidirectional search

Two simultaneous waves, one forward from the source and one backward from the target, stopped when the sum of the two smallest queue keys reaches the cost of the best connecting path found. (Chapter 3)

Big-M constraint

The pair $\mathbf{a}^\top \mathbf{z} \leq \beta + Mb$, $b \in \{0, 1\}$, with M a large constant, which enforces a linear constraint when $b = 0$ and switches it off when $b = 1$; with a sum constraint on the binaries it expresses “at least one of these constraints holds”. ([Appendix B](#), [Chapter 22](#))

Big-O notation

$f(n) = \mathcal{O}(g(n))$ when $f(n) \leq cg(n)$ for all n beyond some n_0 : an upper bound up to a constant factor, with Ω the matching lower bound and Θ both at once; a bound must always be read in the right input variables. ([Appendix B](#))

Binary heap

A complete binary tree stored in an array, in which no key is larger than its children’s, so the minimum is at the root; insertion and extraction of the minimum sift one key along a root-to-leaf path and cost $\mathcal{O}(\log n)$, and building a heap from n unsorted items costs $\mathcal{O}(n)$. ([Appendix B](#))

BIT*

Batch informed trees: samples are drawn in batches inside the informed set and the implicit random geometric graph is searched with an edge queue ordered by $\hat{f} = \hat{g} + \hat{h}$ with lazy collision checks, reusing the search across batches. ([Chapter 17](#))

Blocked layer $\mathcal{B}_t$

The set of grid cells that the predicted intruder may occupy at grid time t , the cells within the inflated radius plus half a cell of the predicted mean; the replanner treats them as blocked or expensive in time layer t of the space-time graph. ([Chapter 24](#))

Bootstrap filter

The particle filter that uses the motion model as its proposal distribution, so that each weight is multiplied by the likelihood $p(\mathbf{z}_k \mid \mathbf{x}_k^{(i)})$ of the new measurement; also called sequential importance resampling (SIR). ([Chapter 19](#))

Bounded suboptimality

The guarantee that a solution costs at most w times the optimal cost, for a factor $w \geq 1$ chosen by the user. ([Chapter 1](#))

Bounded-suboptimal algorithm

An algorithm with a parameter $w \geq 1$ that returns, on every solvable instance, a solution of cost at most wC^* without computing the optimum C^* ; w is the suboptimality factor. ([Chapter 10](#))

Braking candidate

The command that keeps the current direction and reduces the speed by $a_b \Delta t$; it always lies in the window and is admissible whenever the previous command was, so DWA never runs out of safe options with static obstacles. ([Chapter 14](#))

Braking distance

The distance $v^2/(2a_b)$ covered while decelerating from speed v at the constant rate a_b to a standstill. ([Chapter 14](#))

Branch-and-bound

The exact MILP search that solves LP relaxations, branches on a fractional integer variable and discards subproblems whose bound cannot beat the incumbent. ([Chapter 22](#))

Branch-and-bound pruning

Deleting tree vertices with $\text{Cost}(\mathbf{v}) + \|\mathbf{x}_{\text{goal}} - \mathbf{v}\| \geq c_{\text{best}}$, which can never lie on a better path; used in anytime RRT*. ([Chapter 17](#))

Branch-and-cut

Branch-and-bound strengthened by cutting planes, valid inequalities that cut off fractional relaxation optima, plus presolve and primal heuristics, as in HiGHS. ([Chapter 22](#))

Breadth-first search

The unit-cost special case of Dijkstra’s algorithm in which a first-in-first-out queue replaces the heap. (Chapter 3)

Bypass

An ICBS improvement: when a child has the same cost as its parent and fewer conflicts, the parent adopts the child’s paths instead of being split, so a non-cardinal conflict is removed without growing the tree. (Chapter 9)

Cactus plot

For each strategy, the number of runs finished with effort at most x plotted against x ; it shows the whole runtime distribution and the success rate at once. (Chapter 25)

Calibration

Agreement between a nominal probability p and the fraction of true positions that fall inside the predicted p -ellipse; learned predictors are often over-confident and need a per-horizon inflation factor fitted on validation data. (Chapter 20)

Cardinal conflict

A conflict whose resolution raises the cost of the node in both children; semi-cardinal if only one child’s cost rises, non-cardinal if neither does. Splitting cardinal conflicts first keeps the constraint tree small. (Chapter 9)

Chance constraint

A constraint required to hold with probability at least $1 - \delta$; for a Gaussian predicted intruder it becomes the half-plane constraint with the radius inflated by $\kappa_\delta \sqrt{\mathbf{n}_k^\top \Sigma_k \mathbf{n}_k}$. (Chapter 21)

ChooseParent

The RRT* step that connects the new vertex to the near vertex giving the smallest cost-to-come through a collision-free segment, with the nearest vertex as fallback. (Chapter 17)

Classical MAPF

The multi-agent path finding problem on an undirected graph with discrete time, synchronous unit-time move and wait actions, distinct starts and goals, stay-at-target, and vertex and swapping conflicts forbidden. (Chapter 7)

Clearance

The distance $\rho(\mathbf{p})$ from the robot position to the nearest point of an obstacle; $\nabla\rho$ is the unit vector pointing away from that point. (Chapter 15)

Clearance term

The free distance of the rollout divided by $d_{\max}$; it rewards open space ahead of the robot. (Chapter 14)

Collision checking by subdivision

Testing points spaced at most δ apart along a segment against the obstacles inflated by $r + \delta/2$; every accepted segment then has clearance at least the robot radius r . (Chapter 16)

Collision cone

$CC_{A|B}$, the set of relative velocities whose ray from the origin meets the disk of radius $r_A + r_B$ around $\mathbf{p}_{\text{rel}}$; a wedge of half-angle $\arcsin((r_A + r_B)/\|\mathbf{p}_{\text{rel}}\|)$ bounded by the tangent rays. (Chapter 12)

Collision rate

The fraction of runs in which some pair of agents comes closer than the sum of their radii; a proportion, reported with a Wilson interval and, when it is zero, bounded by the rule of three. (Chapter 25)

Collision set

The set $C(v)$ of agents that are allowed to branch over all their moves at the node v ; it

starts empty, receives the agents involved in collisions found below v , and only grows. (Chapter 11)

Communication graph

The graph $G = (V, E)$ whose vertices are the drones and whose edges join pairs that can exchange messages; for a radio of range R_{comm} it is the radius graph with $\{i, j\} \in E$ iff $\|\mathbf{p}_i - \mathbf{p}_j\| \leq R_{\text{comm}}$, and it changes as the drones move. (Chapter 23)

Completeness

The property of finding a solution whenever one exists; in the strong form the algorithm also reports failure when no solution exists, which needs a finite search space or a separate feasibility test. (Chapter 1)

Condensed QP

The form of the MPC problem in which the predicted states are eliminated with $\mathbf{X} = \mathbf{S}_x \mathbf{x}_0 + \mathbf{S}_u \mathbf{U}$, leaving a dense quadratic program in the stacked inputs $\mathbf{U}$ alone. (Chapter 21)

Conditional Gaussian

For jointly Gaussian $(\mathbf{x}_a, \mathbf{x}_b)$, the distribution $\mathbf{x}_a \mid \mathbf{x}_b \sim \mathcal{N}(\mu_a + \Sigma_{ab} \Sigma_{bb}^{-1}(\mathbf{x}_b - \mu_b), \Sigma_{aa} - \Sigma_{ab} \Sigma_{bb}^{-1} \Sigma_{ba})$; the Kalman update is this formula applied to a state and a linear measurement of it. (Appendix B)

Configuration space

The set of all positions (configurations) of a robot; it splits into the free part $\mathcal{C}_{\text{free}}$ and the obstacle part $\mathcal{C}_{\text{obs}}$. (Chapter 2)

Conflict heuristic h_c

The number of vertex and swapping conflicts in the paths of a CT node, or of a partial low-level path with the other agents' current paths; the secondary heuristic of ECBS at both levels. (Chapter 10)

Conflict re-check

The run of the conflict detector over the padded paths of all drones, from the current grid time on, after any change of a path in force (a replan, a repair or a delayed reconnection). (Chapter 24)

Conflict splitting

Resolving a conflict $\langle a_i, a_j, v, t \rangle$ by generating two children, one with $\langle a_i, v, t \rangle$ and one with $\langle a_j, v, t \rangle$, and replanning only the constrained agent in each; every valid solution satisfies at least one of the two constraints. (Chapter 9)

Conflict-based search (CBS)

A two-level optimal MAPF algorithm: the high level searches a binary constraint tree best-first on the sum of costs, the low level plans one agent at a time with space-time A^* under that agent's constraints. (Chapter 9)

Conflict-graph heuristic (CG)

An admissible high-level heuristic for CBS: the size of a minimum vertex cover of the graph whose vertices are agents and whose edges are cardinal conflicts, a lower bound on the cost increase needed to resolve them. (Chapter 9)

Connection radius

$r_n = \min\{\gamma_{RRT^*}(\log n/n)^{1/d}, \eta\}$ with n the number of tree vertices; asymptotic optimality needs $\gamma_{RRT^*} > 2(1 + 1/d)^{1/d}(\mu(\mathcal{X}_{\text{free}})/\zeta_d)^{1/d}$, and the schedule keeps the expected NEAR set at $\mathcal{O}(\log n)$ vertices. (Chapter 17)

Connectivity maintenance

Keeping the communication graph connected while the drones move, by constraining the edges of a spanning subgraph to stay shorter than R_{comm} or by a potential that becomes infinite at the range boundary; monitored through λ_2 . (Chapter 23)

Consensus protocol

$\dot{\mathbf{x}}_i = \sum_{j \in N_i} (\mathbf{x}_j - \mathbf{x}_i)$, i.e., $\dot{\mathbf{x}} = -\mathbf{L}\mathbf{x}$; on a connected undirected graph every state converges to the average of the initial states. (Chapter 23)

Consistent heuristic

A heuristic with $h(\gamma) = 0$ and $h(n) \leq c(n, n') + h(n')$ on every edge (also called monotone); it is admissible, and with it A^* expands every node at most once and pops nodes in non-decreasing order of f . (Chapter 4)

Constant-velocity baseline

Extrapolation of the last observed position with a velocity estimated from the last k intervals; exact on straight flight, with a noise error that grows like h/k , and the baseline every learned predictor must beat. (Chapter 20)

Constant-velocity model

The kinematic model with state $(\mathbf{p}, \mathbf{v})$, $\mathbf{p}_k = \mathbf{p}_{k-1} + \Delta t \mathbf{v}_{k-1}$, $\mathbf{v}_k = \mathbf{v}_{k-1}$, whose unknown accelerations are treated as process noise. (Chapter 18)

Constraint tree (CT)

The binary tree searched by the high level of CBS; each node holds a set of constraints, one shortest path per agent that respects them, and the sum of those path costs. (Chapter 9)

Convex function

A function whose graph lies below every chord, $f(\theta \mathbf{x} + (1-\theta)\mathbf{y}) \leq \theta f(\mathbf{x}) + (1-\theta)f(\mathbf{y})$; every local minimum of a convex function on a convex set is global, and a twice differentiable function is convex iff its Hessian is positive semidefinite everywhere. (Appendix B)

Cooperative A^*

Silver's name for space-time A^* against a reservation table, with the goal-stay test and a horizon; the low-level search of prioritized planning. (Chapter 8)

Coordinated-turn model

The motion model with state (x, y, v, ψ, ω) , constant speed v and constant turn rate ω , whose position update follows a circular arc of radius $v/|\omega|$; nonlinear through $v \cos \psi$, $v \sin \psi$ and the division by ω . (Chapter 19)

Corner cutting

A diagonal move on an 8-connected grid that passes the corner of an obstacle cell; the book forbids it because the swept disk of the robot would clip the obstacle. (Chapter 2, Chapter 22)

Corridor symmetry

Two agents traversing a narrow corridor in opposite directions; every possible waiting time becomes a separate branch unless a corridor constraint forces one agent to wait outside until the other has passed. (Chapter 9)

Cost-to-come

$\text{Cost}(\mathbf{v})$, the length of the tree path from the start to vertex $\mathbf{v}$; RRT^* keeps $\text{Cost}(\mathbf{v}) = \text{Cost}(\text{parent}(\mathbf{v})) + \|\mathbf{v} - \text{parent}(\mathbf{v})\|$ as an invariant. (Chapter 17)

Covariance ellipse

The set of points at a fixed Mahalanobis distance from the mean of a Gaussian; its axes are the eigenvectors of the covariance and its semi-axes are the square roots of the eigenvalues. (Chapter 2)

CT node

A triple $N = (N.\mathcal{C}, N.\pi, N.\text{cost})$; the root has no constraints, a goal node has a conflict-free solution, and a child adds exactly one constraint to its parent's set. (Chapter 9)

Curse of dimensionality

The growth of a uniform grid as N^d with the resolution N per axis and the dimension d , which makes grid search impossible in large 3D volumes and in state spaces that include

velocities. (Chapter 16)

Cycle conflict

A rotation of $m \geq 3$ agents that all move at the same time step, each into the distinct vertex that the next one is leaving, along a fully occupied cycle; allowed in this book. The degenerate case $m = 2$ of the same pattern is the swapping conflict, which is why the cycle conflict is defined for $m \geq 3$. (Chapter 7)

D* Lite

LPA* turned around to search backward from the goal, with a key modifier k_m so that the robot may move while it replans; the replanning algorithm of the hybrid architecture. (Chapter 5)

Deadline

The time by which the replanning layer must deliver a path; an anytime planner meets it with the best path found so far, a fixed-budget optimal search may miss it altogether. (Chapter 6)

Degeneracy

The state of a particle filter in which a few particles carry almost all the weight after repeated weighting without resampling. (Chapter 19)

Deliberative method

A method that searches or optimizes over possible futures, using a model of the world, before it acts. (Chapter 1)

Dense fallback

The remedy for an empty intersection of ORCA half-planes: choose the velocity that minimizes the largest violation over all agent half-planes while keeping the obstacle half-planes hard; van den Berg et al. call it the safest possible velocity, the one which penetrates the half-planes least. It is a heuristic: it minimizes a penetration, not the time to the first collision. (Chapter 13)

Differential drive

A wheeled robot commanded by a translational speed v and a rotational speed ω ; with both constant it moves on a circular arc of radius v/ω . (Chapter 14)

Dijkstra's algorithm

The algorithm that computes the cheapest cost from one source to every vertex of a graph with non-negative edge costs by repeatedly settling the open vertex with the smallest tentative cost. (Chapter 3)

Disagreement vector

$\delta = \mathbf{x} - \bar{\mathbf{x}}\mathbf{1}$, the distance of the states from agreement; it decays like $e^{-\lambda_2 t}$ under the consensus protocol. (Chapter 23)

Disappear-at-target

The convention in which an agent is removed from the graph the moment it first reaches its goal; not used in this book. (Chapter 7)

Discrete-time consensus

$\mathbf{x}_{k+1} = (\mathbf{I} - \varepsilon\mathbf{L})\mathbf{x}_k$, the sampled version of the protocol; it converges for $0 < \varepsilon < 2/\lambda_n$, and $\varepsilon < 1/d_{\max}$ is the sufficient bound every drone can check locally. (Chapter 23)

Displacement-based formation control

$\dot{\mathbf{p}}_i = \sum_{j \in N_i} (\mathbf{p}_j - \mathbf{p}_i - \mathbf{d}_{ij})$ with desired relative positions $\mathbf{d}_{ij} = \mathbf{o}_j - \mathbf{o}_i$; it is consensus on the shifted variables $\mathbf{p}_i - \mathbf{o}_i$ and converges to the formation up to a translation. (Chapter 23)

Distance-based formation control

Formation control that prescribes only inter-drone distances, needs no common frame, defines the shape up to rotation and reflection, and requires a rigid formation graph. (Chapter 23)

Dominance

h_2 dominates h_1 if $h_2 \geq h_1$ everywhere; the more informed admissible heuristic expands no more nodes with $f < C^*$. (Chapter 4)

Double integrator

The motion model in which the control input is the acceleration, so that the state consists of position and velocity. (Chapter 2)

Drift $d_i(t)$

The distance between a drone and its reference position; a bound $d_{\max}$ on the drift is one of the two exits from the Avoiding state into Replanning. (Chapter 24)

Dynamic window

The set V_d of velocities a robot can reach from its current velocity within one control interval under its acceleration limits; DWA searches only inside it. (Chapter 14)

Early exit

Stopping a search when the target is popped from the queue, never when it is first discovered. (Chapter 3)

ECBS

Enhanced conflict-based search: CBS with focal search at the low level (conflict-avoiding paths within w of the agent's optimum, returned with a lower bound) and at the high level (OPEN by LB(N), FOCAL by conflicts); its solutions cost at most $w C^*$. (Chapter 10)

Edge conflict

Two agents traversing the same edge in the same direction in the same time step; it implies a vertex conflict at the start of the step. (Chapter 7)

Edge constraint

A tuple $\langle a, u, v, t \rangle$ forbidding agent a to move from u to v between times t and $t + 1$; reserving the reverse of every move of a planned path prevents swaps. (Chapter 4)

EECBS

ECBS with explicit estimation search at the high level, estimates learned online during the search, and the CBS improvements (bypass, cardinal conflicts, symmetry reasoning, high-level heuristics); keeps the bound w and scales to hundreds of agents. (Chapter 10)

Effect size d_z

The mean of the paired differences divided by their standard deviation; a unit-free measure of how large a difference is relative to its scenario-to-scenario variability. (Chapter 25)

Effective sample size

$N_{\text{eff}} = 1 / \sum_i (w^{(i)})^2$, a number between 1 and N that measures how evenly the weight is spread; resampling is triggered when it falls below a threshold such as $N/2$. (Chapter 19)

Epsilon schedule

The sequence $\varepsilon_1 > \varepsilon_2 > \dots > \varepsilon_K = 1$ of inflation factors that ARA^* runs through; it fixes the time to the first path, the number of published improvements and the total work. (Chapter 6)

Exact penalty

A linear slack weight w larger than every Lagrange multiplier of the hard problem, for which the soft solution coincides with the hard one whenever the latter is feasible. (Chapter 21)

Execution log

The record of one run: positions of drones and intruders at every sampling instant, the state of each drone's state machine, the computation time of every decision and the events (replans), from which all metrics are computed. (Chapter 25)

Executive

The per-drone state machine that runs every control cycle and decides which layer is in

charge: it follows the plan, hands the velocity to the local layer when a predicted conflict is inside the safety horizon, reconnects to the plan when the conflict has cleared, replans when reconnection fails, and re-checks conflicts after every change of a path. (Chapter 24)

Expansion

One non-stale pop from the priority queue together with the generation of the successors of the popped vertex; the unit in which the work of a search is counted. (Chapter 3)

Experiment matrix

The factors of an evaluation (drones, intruders, prediction quality, avoidance strategy, constraints) with their levels; a cell is one level of every factor, and a full factorial runs every cell while a fractional design runs a chosen subset such as one factor at a time around a base cell. (Chapter 25)

Explicit estimation search (EES)

A bounded-suboptimal search that orders nodes by a lower bound, by an inadmissible estimate of the final solution cost, and by a distance-to-go estimate, and chooses among the three fronts while keeping every expanded solution within w of the lower bound. (Chapter 10)

Extended Kalman filter (EKF)

The Kalman filter applied to the tangents of f at the previous estimate and of h at the prediction, with the Jacobians $\mathbf{F}_k$ and $\mathbf{H}_k$ in place of the linear model matrices; exact for linear models, biased when the functions curve across the belief. (Chapter 19)

Feasibility solver

A method such as Push-and-Swap or Push-and-Rotate that returns some conflict-free plan whenever one exists (with at least two empty vertices) but gives no bound on its cost. (Chapter 11)

Feasible velocity set

The reachable velocities $\|\mathbf{v}\| \leq v_{\max}$ minus the union of the truncated velocity obstacles of all obstacles; the local rule picks the feasible velocity closest to the preferred one. (Chapter 12)

Flocking

Olfati-Saber's controller with a gradient term (separation and cohesion from a smooth pairwise potential), velocity alignment (consensus on velocities) and navigation feedback toward a target; the formal descendant of Reynolds' boids. (Chapter 23)

Flocking rules

Reynolds's separation, alignment and cohesion: repel close neighbors, match their average velocity, and move toward their centroid; alignment is a velocity consensus and cohesion plus separation a pairwise spacing potential. (Chapter 15)

FOCAL list

The band $\{n \in \text{OPEN} : f(n) \leq w f_{\min}\}$ of a focal search, kept as a second heap ordered by the secondary heuristic and refilled from OPEN whenever $f_{\min}$ rises. (Chapter 10)

Focal search (A^*_ϵ)

Best-first search that keeps OPEN ordered by an admissible $f = g + h$ and expands, among the open nodes with $f \leq w f_{\min}$, the one preferred by a secondary heuristic; its solution costs at most $w C^*$ whatever the secondary heuristic is. (Chapter 10)

Following conflict

One agent entering, at time $t + 1$, the vertex that another agent occupied at time t ; allowed in this book because synchronous motion keeps a one-edge gap at every integer time. (Chapter 7)

Formation and communication-range constraints

Requirements $\|\mathbf{p}_k^i - \mathbf{p}_k^j - \mathbf{d}_{ij}\| \leq e_{\max}$ and $\|\mathbf{p}_k^i - \mathbf{p}_k^j\| \leq R_{\text{comm}}$ on pairs of drones, convex

disks around a moving center that MPC enforces through inscribed polygons. (Chapter 21)

Formation error

$e_F = \sqrt{\frac{1}{|E_F|} \sum_{\{i,j\} \in E_F} \|\mathbf{p}_j - \mathbf{p}_i - \mathbf{d}_{ij}\|^2}$, the RMS violation of the desired displacements over the edges of the formation graph; translation invariant and zero exactly on the formation. (Chapter 23)

Free distance

The distance along a rollout to the first obstacle inflated by the robot radius, capped at the sensing range $d_{\max}$. (Chapter 14)

Gaussian negative log-likelihood

The loss $\sum_d [\log \sigma_d + (y_d - \mu_d)^2 / (2\sigma_d^2)]$ for a read-out that predicts a mean and a standard deviation per step, minimized by the conditional mean and the conditional variance. (Chapter 20)

Global lower bound LB

The smallest $\text{LB}(N) = \sum_i \text{LB}_i(N)$ over the open CT nodes; it never exceeds C^* , so a conflict-free node with cost $\leq w \text{LB}$ is w -suboptimal, and cost/LB is the bound proven by a run. (Chapter 10)

Global planner

A planner that computes a complete route from start to goal from a map of all known obstacles; in the hybrid architecture, the multi-agent planner that produces the nominal paths. (Chapter 1)

GNRON

Goals non-reachable with obstacles nearby: when the goal lies inside the influence region of an obstacle, the repulsion at the goal is non-zero and the minimum of U moves away from the goal; the standard fix multiplies the repulsive term by $\|\mathbf{p} - \mathbf{p}_g\|^n$. (Chapter 15)

Goal bias

The probability p_{goal} with which SAMPLE returns the goal instead of a uniform configuration; small values (0.05–0.1) speed the search up, large values make the tree greedy and trapped. (Chapter 16)

Goal region

The ball of radius r_{goal} around the goal point; the search ends when a vertex enters it, since continuous samples never hit a point exactly. (Chapter 16)

Goal-stay test (goal-occupied-later test)

The low-level rule that a path may end at the goal at time t only if no vertex constraint forbids the goal at any time $\geq t$, because the agent stays there for ever afterward. (Chapter 9)

Goal-stay time

The last time t_γ at which the goal is constrained; space-time A^* may finish at (γ, t) only if $t > t_\gamma$, so that the agent can stay there for ever. (Chapter 4)

Graph Laplacian

$\mathbf{L} = \mathbf{D} - \mathbf{A}$, the degree matrix minus the adjacency matrix; symmetric, positive semidefinite, with $\mathbf{L}\mathbf{1} = \mathbf{0}$ and $\mathbf{x}^\top \mathbf{L} \mathbf{x} = \sum_{\{i,j\} \in E} (x_i - x_j)^2$. (Chapter 23)

Harmonic potential

A potential satisfying Laplace's equation $\nabla^2 U = 0$ in the free space; by the maximum principle it has no interior local minima, but it must be computed globally on a grid. (Chapter 15)

Hash table

A container that maps a key to an array position with a hash function and resolves collisions by probing or chaining, so insertion, lookup and deletion take constant expected time; Python's `dict` and `set` are hash tables and require immutable keys. (Appendix B)

HCA*

Hierarchical Cooperative A*: Cooperative A* with the exact static distance to the goal, computed by a backward search on the map without agents, as heuristic. (Chapter 8)

Heading term

$1 - \theta/\pi$, where θ is the angle between the candidate velocity and the direction to the target seen from the predicted position; it rewards progress toward the goal or a lookahead point. (Chapter 14)

Heuristic

A function $h(n)$ that estimates the cost from n to the goal without searching; A* adds it to the cost so far and expands the node with the smallest sum $f = g + h$. (Chapter 4)

Horizon of a plan

The largest time index $T = \max_i T_i$ that any path of the plan mentions explicitly; it is at least the makespan and equals it only when no path ends with waits at its goal; no vertex or swapping conflict can first appear after it. (Chapter 7)

Hybrid architecture

The four-layer design of the book: a global planner, a prediction layer, a local safety layer and a replanning layer, tied together by a five-step decision logic. (Chapter 1)

Hybrid collision-avoidance architecture

The four-layer design of the drone system: a global planner (CBS/ECBS) for the nominal time-indexed paths, a prediction layer (Kalman filter/LSTM) for unknown objects, a local safety layer (ORCA/DWA) for short avoidance maneuvers, and a replanning layer (D* Lite/A*) from the current state, joined by a per-drone executive. (Chapter 24)

Hybrid world model

The shared representation of the layers: a grid with discrete time (paths with their start times, the reservation table, the blocked layers) and continuous positions, velocities and predictions, with one clock and one frame and explicit conversions between the two sides. (Chapter 24)

Hysteresis (DWA)

Keeping the current command unless the best candidate beats its score by a margin η ; it damps the oscillation between two nearly equal commands but does not remove a trap. (Chapter 14)

Importance weight

The factor that corrects a sample drawn from a proposal distribution so that the weighted set represents the target distribution; in the bootstrap filter it is proportional to the accumulated measurement likelihoods. (Chapter 19)

ImprovePath

The inner loop of ARA*: expand the state with the smallest f_ϵ until the goal's key is no larger than every key in OPEN; each state is expanded at most once per call and the result is an ϵ -suboptimal path. (Chapter 6)

INCONS list

The states that were expanded in the current iteration and whose g was lowered afterward; ARA* records the improvement but does not re-expand them until the next iteration, when they are merged into OPEN. (Chapter 6)

Inconsistent state

A state whose g -value has been lowered since it was last expanded (or that has never been expanded); the set of inconsistent states is exactly $\text{OPEN} \cup \text{INCONS}$, and only they can still improve the search. (Chapter 6)

Incremental linear program

The randomized algorithm that finds the point of an intersection of half-planes and a disk

closest to a given point by adding the half-planes one at a time and, whenever the current optimum violates the new one, solving a one-dimensional program on its boundary line; expected linear time in the number of half-planes. (Chapter 13)

Incremental search

A search that reuses the previous search when the graph changes slightly, instead of starting over; the basis of replanning with D^{*} Lite. (Chapter 1, Chapter 5)

Incumbent

The best solution with integral values found so far during branch-and-bound; every node whose bound is not better is pruned. (Chapter 22)

Independence detection

A wrapper that plans groups of agents separately and merges two groups into one joint search only when their plans conflict; optimal whenever the group solver is optimal. (Chapter 11)

Individual policy

A rule ϕ_i that moves agent i one step along a shortest path to its goal from any cell; following all policies from a configuration gives the policy path, whose cost equals the sum heuristic. (Chapter 11)

Inflated safety radius d_h^{safe}

The required separation $R_{\text{safe}} = r_A + r_B + \Delta r$ plus $\kappa_p \sqrt{\lambda_{\max}(\Sigma_h)}$, the radius of the disk that contains the p -confidence ellipse of the prediction at step h ; keeping this distance from the predicted mean is a per-step chance constraint with probability p . (Chapter 24)

Inflation factor

The weight $\varepsilon \geq 1$ that ARA^{*} puts on the heuristic in its key $f_\varepsilon = g + \varepsilon h$; a large ε makes the search greedy and fast, $\varepsilon = 1$ makes it plain A^{*}. (Chapter 6)

Influence distance

The radius ρ_0 beyond which an obstacle exerts no repulsion; the repulsive potential and its gradient both vanish at $\rho = \rho_0$, so U is continuously differentiable. (Chapter 15)

Informed RRT^{*}

RRT^{*} that, once a solution of cost c_{best} exists, samples the informed set uniformly by mapping the unit ball through $\mathbf{x} = \mathbf{C}\mathbf{L}\mathbf{x}_{\text{ball}} + \mathbf{x}_{\text{center}}$. (Chapter 17)

Informed set

$\{\mathbf{x} : \|\mathbf{x} - \mathbf{x}_{\text{init}}\| + \|\mathbf{x} - \mathbf{x}_{\text{goal}}\| \leq c_{\text{best}}\}$, the only region in which a path cheaper than the current best can pass; the analog of the region $f \leq C^*$ of A^{*}. (Chapter 17)

Innovation

$\mathbf{y}_k = \mathbf{z}_k - \mathbf{H}\hat{\mathbf{x}}_k^-$, the part of the measurement the prediction did not anticipate; its covariance is $\mathbf{S}_k = \mathbf{H}\mathbf{P}_k^- \mathbf{H}^T + \mathbf{R}$, and a consistent filter has zero-mean, white innovations with that covariance. (Chapter 18)

Inter-agent repulsion

A pairwise repulsive term between drones with clearance $\rho_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\| - 2r$, the potential-field form of Reynolds's separation rule. (Chapter 15)

Intruder

An unknown, non-cooperative moving object, usually another drone, that appears during the flight and whose future motion is uncertain. (Chapter 1)

Jacobian

The matrix $\partial g_i / \partial x_j$ of first derivatives of a vector function, one row per output; it is the linear map that best approximates the function near a point, with an error of second order in the displacement. (Appendix B)

Joint configuration

A tuple $v = (v^1, \dots, v^k)$ giving the cell of every agent; the vertices of the joint graph are the collision-free configurations and its edges the joint moves without vertex or swap conflicts. (Chapter 11)

Joint state space

The set of $|V|^k$ configurations of k agents on $|V|$ cells, searched by joint A^* with up to $(b+1)^k$ successors per node; its size is what makes coupled search hopeless beyond a few agents. (Chapter 11)

Joseph form

The covariance update $(\mathbf{I} - \mathbf{KH})\mathbf{P}^-(\mathbf{I} - \mathbf{KH})^\top + \mathbf{K}\mathbf{R}\mathbf{K}^\top$, valid for any gain and always symmetric positive semidefinite; it equals $(\mathbf{I} - \mathbf{KH})\mathbf{P}^-$ for the Kalman gain. (Chapter 18)

 k -nearest RRT*

The variant of RRT* whose NEAR set is the $k_n = \lceil k_{\text{RRG}} \log n \rceil$ nearest vertices with $k_{\text{RRG}} > e(1 + 1/d)$; it needs no estimate of $\mu(\mathcal{X}_{\text{free}})$. (Chapter 17)

Kalman extrapolation

Running the predict step of the constant-velocity Kalman filter h times without updates; the optimal linear predictor under its noise model, with a covariance that grows with the horizon. (Chapter 20)

Kalman filter

The recursive estimator for a linear-Gaussian state-space model: it keeps a Gaussian belief $\mathcal{N}(\hat{\mathbf{x}}_k, \mathbf{P}_k)$ and alternates a predict step through the motion model and an update step with the measurement; it is the exact posterior and the minimum-mean-square-error estimator, and the best linear estimator when the noise is not Gaussian. (Chapter 18)

Kalman gain

$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}^\top \mathbf{S}_k^{-1}$, the matrix by which the innovation corrects the predicted mean; it weights prediction and measurement by their precisions and corrects unmeasured state components through their covariance with the measured ones. (Chapter 18)

kd-tree

A binary tree that splits points along coordinate axes and answers nearest-neighbor queries in $\mathcal{O}(\log n)$ expected time in low dimension, against $\mathcal{O}(n)$ for a linear scan. (Chapter 16)

Key modifier

The running sum k_m of the heuristic distances the robot has moved between repairs; added to every new key of D^* Lite, it turns the keys already in the queue into lower bounds instead of forcing the heap to be rebuilt. (Chapter 5)

Kinodynamic RRT

RRT over states $(\mathbf{p}, \mathbf{v})$ in which STEER is replaced by forward simulation of sampled controls and NEAREST uses a weighted state-space metric; the result is a trajectory that obeys the dynamics. (Chapter 16)

KKT conditions

Stationarity of the Lagrangian, primal and dual feasibility and complementary slackness; necessary for a local minimizer under a constraint qualification and sufficient for convex problems, with the multipliers measuring the cost of tightening each active constraint. (Appendix B)

Lazy deletion

The priority-queue idiom of pushing a duplicate entry instead of decreasing a key, and skipping entries that are stale when they are popped. (Chapter 2)

Leader-follower (pinned) consensus

Consensus in which some drones add a term $b_i(\mathbf{r} - \mathbf{x}_i)$ pulling them to an external reference $\mathbf{r}$; with a connected graph and at least one pinned drone all states converge to $\mathbf{r}$. (Chapter 23)

Lifelong Planning A* (LPA*)

The incremental version of A* for a fixed start and goal: it keeps g - and rhs -values across edge-cost changes and re-expands only locally inconsistent vertices. (Chapter 5)

Limited neighbors

The successors of a node in M^* : the agents in the collision set take any move, all other agents take their policy step, so a node has $(b+1)^{|C(v)|}$ successors. (Chapter 11)

Linear program (LP)

Minimization of a linear objective subject to linear equalities, inequalities and bounds; its feasible set is a polyhedron and its optimum is attained at a vertex. (Chapter 22)

Linear-Gaussian state-space model

$\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{B}\mathbf{u}_k + \mathbf{w}_k$, $\mathbf{z}_k = \mathbf{H}\mathbf{x}_k + \mathbf{v}_k$ with independent Gaussian noises $\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$ and $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$; $\mathbf{F}$ is the state-transition matrix and $\mathbf{H}$ the measurement matrix. (Chapter 18)

Linearization error

The difference between the Gaussian produced by the tangent of a nonlinear function and the true (non-Gaussian, “banana”) distribution of its output; it grows with the curvature of the function times the width of the belief. (Chapter 19)

Linearized avoidance constraint

The half-plane $\mathbf{n}_k^T(\mathbf{p}_k - \hat{\mathbf{o}}_k) \geq r_k$ that replaces the non-convex disk constraint $\|\mathbf{p}_k - \hat{\mathbf{o}}_k\| \geq r_k$ at step k , with the normal $\mathbf{n}_k$ taken from the previous plan; it is convex and conservative. (Chapter 21)

Local consistency

The property $g(s) = rhs(s)$; only locally inconsistent vertices are in the priority queue and need work. (Chapter 5)

Local minimum (DWA)

A state from which the greedy, memoryless rule keeps the robot at rest or shuttling although a collision-free path to the goal exists; the reason why DWA is not complete. (Chapter 14)

Local minimum (of a potential field)

A point other than the goal where the attractive and repulsive forces cancel and the potential rises in every direction; a gradient-descent robot that enters its basin stops there. (Chapter 15)

Local planner

A planner that looks only a few seconds ahead, uses only what the sensors see, and outputs the next velocity or maneuver. (Chapter 1)

Longest path first

The ordering rule that plans agents in decreasing length of their independent shortest paths; rarely fails but produces expensive plans. (Chapter 8)

Lookahead point

The point at a fixed arc length ahead of the robot’s projection on a global path, used as the target of the heading term to keep a local planner out of local minima. (Chapter 14)

LP relaxation

The linear program obtained from a mixed-integer program by dropping the integrality constraints; its optimal value bounds the integer optimum, which branch and bound exploits by splitting on a fractional variable and pruning subproblems whose bound cannot beat the incumbent. (Appendix B, Chapter 22)

LSTM cell

A recurrent unit whose cell state is updated by gated addition, $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$, with forget, input and output gates in $(0, 1)$ produced by logistic functions; the direct gradient path through $\mathbf{c}$ is $\text{diag}(\mathbf{f}_t)$. (Chapter 20)

Mahalanobis distance

$d(\mathbf{x}) = \sqrt{(\mathbf{x} - \mu)^\top \Sigma^{-1}(\mathbf{x} - \mu)}$, the distance from the mean measured in units of the covariance; for a Gaussian vector d^2 is χ_n^2 distributed, which is the basis of gating. (Appendix B)

Makespan

The number of time steps until the last agent of a plan has reached its goal for good. (Chapter 2)

Meta-agent CBS (MA-CBS)

A variant that merges agents which conflict more than B times into a meta-agent planned jointly at the low level, interpolating between plain CBS ($B = \infty$) and Standley's independence detection ($B = 0$), in which every pair is merged at its first conflict, so each group of interacting agents is solved by a joint-space A^* while agents that never conflict are still planned alone. (Chapter 9)

Minimum separation $d_{\min}$

The smallest distance between any two agents over a run, evaluated with linear motion between samples; reported as a median with IQR and a 5th percentile, and turned into the near-miss rate by comparing it with the safety radius. (Chapter 25)

Minimum-fuel objective

The total velocity change $\Delta t \sum_k \|\mathbf{u}_k\|_1$ of a trajectory, linearized with one auxiliary variable per input component. (Chapter 22)

Minkowski sum

The set of all sums $\mathbf{a} + \mathbf{b}$ with $\mathbf{a}$ in one set and $\mathbf{b}$ in another; inflating an obstacle by the robot's radius is a Minkowski sum with a disk. (Chapter 2)

MIP gap

The relative difference between the incumbent's cost and the best remaining bound; zero certifies optimality, and a time-limited solve reports the gap it reached. (Chapter 22)

Mixed-integer linear program (MILP)

An LP in which some variables are restricted to integers; with binary variables it encodes choices, either-or conditions, switches and assignments. (Chapter 22)

Model predictive control (MPC)

A feedback controller that, at every step, solves a finite-horizon optimal control problem from the measured state, applies the first input and repeats; constraints are part of the optimization rather than an afterthought. (Chapter 21)

Multi-source Dijkstra

Dijkstra's algorithm started with several sources at cost zero, which yields the cost to the nearest source for every vertex. (Chapter 3)

Multi-valued decision diagram (MDD)

For an agent and a cost c , the layered graph whose level t contains every cell that lies on some path of cost c at time t ; a conflict is cardinal when both agents have a single cell at its level. (Chapter 9)

Multimodal prediction

A prediction that puts probability mass on several distinct futures (mixture density outputs, sampled rollouts of a GAN or CVAE, ensembles), needed because a squared-error predictor converges to the conditional mean, which averages the modes. (Chapter 20)

Multivariate Gaussian

The density $\mathcal{N}(\mu, \Sigma)$ whose exponent is $-\frac{1}{2}$ times the squared Mahalanobis distance from the mean; it stays Gaussian under affine maps, under the addition of independent Gaussian noise, and under marginalization and conditioning, which is what makes the Kalman filter a closed-form recursion. (Appendix B)

Naive plan

The plan made of independent shortest paths, one per agent, ignoring all other agents; its costs are lower bounds for every valid solution. (Chapter 7)

Narrow-passage problem

The growth of the expected number of iterations like $(1/w)^d$ for a passage of width w , because uniform samples rarely fall into it. (Chapter 16)

Navigation function

A potential on a bounded free space with a unique minimum at the goal, whose other critical points are non-degenerate saddles; Rimón and Koditschek construct one explicitly for sphere worlds. (Chapter 15)

Near

The set $\text{Near}(V, \mathbf{x}, r) = \{\mathbf{v} \in V : \|\mathbf{v} - \mathbf{x}\| \leq r\}$ of tree vertices within the connection radius of a point; the candidate set of CHOOSEPARENT and REWIRE in RRT*. (Chapter 17)

Non-autoregressive decoder

A decoder that emits all H predicted steps in one pass, from queries built out of the positional encodings of the future steps rather than out of its own previous outputs; it has no exposure bias, but no step can condition on the step before it. (Chapter 20)

Non-cooperative agent

An agent that does not share the avoidance rules of the swarm; a drone that meets it must take the full responsibility for avoiding it. (Chapter 1)

Non-cooperative obstacle

An object that does not react to the agent, such as an unknown drone or a bird; it is avoided with the plain velocity obstacle (full responsibility), not with a reciprocal one. (Chapter 12, Chapter 13)

Normal equations

The linear system $\mathbf{A}^\top \mathbf{A} \mathbf{x} = \mathbf{A}^\top \mathbf{b}$ whose solution minimizes $\|\mathbf{A} \mathbf{x} - \mathbf{b}\|^2$; unique when $\mathbf{A}$ has full column rank. (Appendix B)

Normalized innovation squared (NIS)

$d_k^2 = \mathbf{y}_k^\top \mathbf{S}_k^{-1} \mathbf{y}_k$, the squared Mahalanobis distance of a measurement from its prediction; chi-square with m degrees of freedom for a consistent filter, used to tune $\mathbf{Q}$ and to gate outliers. (Chapter 18)

Observability

The property that the observability matrix $(\mathbf{H}; \mathbf{H}\mathbf{F}; \dots; \mathbf{H}\mathbf{F}^{n-1})$ has full rank, so that the state can be recovered from a finite number of noise-free measurements; without it the covariance of the unobserved components grows without bound. (Chapter 18)

Octile distance

$\max(d_x, d_y) + (\sqrt{2} - 1) \min(d_x, d_y)$, the exact cost between two cells of an obstacle-free 8-connected grid with moves of cost 1 and $\sqrt{2}$; consistent, and the standard heuristic for such grids. (Chapter 4)

Operator decomposition

Standley's device of letting the agents choose their moves one at a time through intermediate states, which cuts the branching factor of joint $\mathbf{A}^*$ from $(b+1)^k$ to $b+1$. (Chapter 11)

Oracle baseline

An offline optimum computed with full knowledge of the future (a MILP that sees the intruder's whole trajectory) on tiny instances; it bounds what any online strategy can achieve. (Chapter 25)

ORCA half-plane

$ORCA_{A|B}^\tau = \left\{ \mathbf{v} : (\mathbf{v} - (\mathbf{v}_A + \frac{1}{2}\mathbf{u})) \cdot \mathbf{n} \geq 0 \right\}$, the velocities permitted to A with respect to

B for the horizon τ , where $\mathbf{u}$ is the smallest change of the relative velocity that reaches the boundary of $VO_{A|B}^\tau$ and $\mathbf{n}$ the outward normal there; in three dimensions a half-space. (Chapter 13)

Oscillation (velocity obstacles)

The step-by-step reversal of avoidance maneuvers that occurs when two agents apply the velocity-obstacle rule simultaneously, each assuming that the other keeps its velocity; the motivation for reciprocal velocity obstacles. (Chapter 12)

Overconsistent vertex

A vertex with $g(s) > rhs(s)$: a cheaper value is available and expanding it sets $g(s) \leftarrow rhs(s)$, as an A^* expansion would. (Chapter 5)

Paired design

An evaluation in which every strategy runs on the same seeded scenarios and comparisons are made on the per-scenario differences, which removes the scenario-to-scenario spread from the comparison. (Chapter 25)

Pareto plot

A plot with one point per strategy and cell in the plane of an efficiency loss against a safety loss; the points not dominated by any other form the front and state the trade-off. (Chapter 25)

Parked goal

A goal cell on which an agent has arrived at time T and stays for ever under stay-at-target; reserved for all $t \geq T$. (Chapter 8)

Partial replanning (local CBS)

The repair of a conflict found by the re-check by running CBS on the two agents in conflict only, from the next grid time, with the paths of all other drones reserved. (Chapter 24)

Particle filter

A filter that represents the belief by N weighted samples, propagates them through the motion model with sampled noise, weights them by the measurement likelihood and resamples when the weights degenerate; it can represent multimodal and non-Gaussian beliefs at a cost linear in N . (Chapter 19)

Path-length ratio ρ_L

The flown path length divided by the length of the nominal plan, averaged over the drones of a run; the efficiency cost of avoidance, independent of the arena size. (Chapter 25)

Per-agent lower bound $LB_i(N)$

A number stored in a CT node that is at most the cost of the cheapest path of agent a_i under the node's constraints; the focal low level returns it as the $f_{\min}$ of its OPEN when the goal is popped. (Chapter 10)

Plan (MAPF)

A tuple of one time-indexed path per agent; viewed as a table with agents as rows and time steps as columns. (Chapter 7)

Planned conflict

A collision between two controlled drones that follows from their planned time-indexed routes alone; detected and resolved before the flight. (Chapter 1)

Polyhedron

The intersection of finitely many half-spaces; convex, with flat faces and vertices, the feasible set of every LP. (Chapter 22)

Positional encoding

Sinusoids of geometrically spaced frequencies added to the token embeddings so that attention, which is otherwise a set operation, can tell the order of the time steps. (Chapter 20)

Positive semidefinite matrix

A symmetric matrix $\mathbf{A}$ with $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$ for all $\mathbf{x}$, equivalently with no negative eigenvalue, written $\mathbf{A} \succeq \mathbf{0}$; every covariance matrix is one, and $\mathbf{A} \succ \mathbf{0}$ (positive definite) additionally excludes zero eigenvalues. (Appendix B)

Predict step

$\hat{\mathbf{x}}_k^- = \mathbf{F}\hat{\mathbf{x}}_{k-1} + \mathbf{B}\mathbf{u}_k$, $\mathbf{P}_k^- = \mathbf{F}\mathbf{P}_{k-1}\mathbf{F}^\top + \mathbf{Q}$: the belief pushed through the motion model, with the covariance inflated by the process noise. (Chapter 18)

Predicted time to collision t_c

The first look-ahead at which the margin $\|\tilde{\mathbf{p}}_i(s) - \hat{\mathbf{p}}_{T+h}\| - \kappa_p\sigma(h) - R_{\text{safe}}$ is negative; the trigger fires when $t_c \leq \tau_h$. (Chapter 24)

Prediction

Extrapolating the estimated state of a moving object into the future, over a horizon, together with the growing uncertainty. (Chapter 1)

Prediction covariance

$\mathbf{P}_{k+j|k} = \mathbf{F}^j \mathbf{P}_k (\mathbf{F}^j)^\top + \sum_{i<j} \mathbf{F}^i \mathbf{Q} (\mathbf{F}^i)^\top$, the uncertainty of the state j steps ahead given the measurements up to k ; it grows without bound in j and is what the local safety layer inflates an intruder's radius with. (Chapter 18)

Prediction matrices

$\mathbf{S}_x$ and $\mathbf{S}_u$, built from powers of $\mathbf{A}$ and $\mathbf{B}$, that map the current state and the stacked inputs to the stacked predicted states. (Chapter 21)

Preferred velocity

$\mathbf{v}^{\text{pref}}$, the velocity an agent would fly if nothing were in the way, typically toward the goal or along the nominal route at the nominal speed. (Chapter 12, Chapter 13)

Prioritized planning

Planning the agents of a MAPF instance one after another in a priority order, each with a single-agent space-time search that treats the paths of the earlier agents as moving obstacles; fast, sound, but incomplete and suboptimal in general. (Chapter 8)

Priority order

A permutation σ of the agents; $\sigma(1)$ is planned first and never yields to anyone, $\sigma(k)$ yields to everyone. (Chapter 8)

Priority-based search

A two-level search over partial priority orders (Ma et al.): the high level branches on a conflicting pair of agents by adding $i \prec j$ or $j \prec i$ and searches depth-first, the low level replans the affected agents in topological order against the paths of their higher-priority agents. (Chapter 8)

Probabilistic completeness

The guarantee that, when a solution with positive clearance exists, the probability of having found one after n iterations tends to one as $n \rightarrow \infty$; it says nothing when no solution exists. (Chapter 16)

Probabilistic roadmap (PRM)

The multi-query counterpart of RRT: sample free configurations, connect each to its nearest neighbors by checked segments, and answer queries by graph search on the roadmap. (Chapter 16)

Prolate hyperspheroid

The shape of the informed set: an ellipse (2D) or ellipsoid of revolution (3D) with foci at the start and the goal, transverse diameter c_{best} and conjugate diameter $\sqrt{c_{\text{best}}^2 - c_{\text{min}}^2}$. (Chapter 17)

Push (primitive)

Moving an agent along its shortest path and shifting the chain of unlocked agents in its way one step toward the nearest empty vertex off the path. (Chapter 11)

Quadratic program (QP)

Minimization of a convex quadratic function $\frac{1}{2}\mathbf{z}^T\mathbf{H}\mathbf{z} + \mathbf{f}^T\mathbf{z}$ subject to linear inequality constraints; the problem MPC with a linear model, quadratic cost and linearized constraints solves at every step. (Chapter 21)

Random restarts

Running prioritized planning with several random orders and keeping the cheapest plan that succeeds. (Chapter 8)

Range-bearing measurement

The pair (r, φ) of distance and direction from a sensor to the intruder, $r = \|(x, y) - \mathbf{s}\|$ and $\varphi = \text{atan2}(y - s_y, x - s_x)$; a bearing error of σ_φ radians displaces the position by about $r\sigma_\varphi$ across the line of sight. (Chapter 19)

Rapidly-exploring random tree (RRT)

A tree rooted at the start configuration that grows one vertex per random sample by moving at most η from the nearest vertex toward the sample when that segment is collision-free; the first vertex in the goal region yields a path. (Chapter 16)

Re-opening

Moving a closed node back to OPEN when a cheaper path to it is found; necessary for optimality with an admissible but inconsistent heuristic, never triggered with a consistent one. (Chapter 4)

Reactive method

A method that maps the current situation directly to an action, with little or no search. (Chapter 1)

Receding horizon

The moving window of N future steps over which MPC optimizes; after each step the window is shifted by one step and the plan is recomputed. (Chapter 21)

Reciprocal dance

The oscillation of two agents that avoid each other with plain velocity obstacles (both swerve, both return, both swerve) or, for RVO, that disagree about the side on which to pass; ORCA removes it by fixing the share of the responsibility. (Chapter 13)

Reciprocal velocity obstacle (RVO)

The set $RVO_{A|B}(\mathbf{v}_B, \mathbf{v}_A)$ of velocities $\mathbf{v}$ for which $2\mathbf{v} - \mathbf{v}_A$ lies in the velocity obstacle $\mathbf{v}_B + VO_{A|B}$: the velocities that A must avoid when it takes half of the avoidance and trusts B to take the other half; it is the velocity obstacle cone with its apex moved to $\frac{1}{2}(\mathbf{v}_A + \mathbf{v}_B)$. (Chapter 13)

Reconnection

The return to the nominal path after an avoidance maneuver: a straight flight to a waypoint $\pi_i[j]$ ahead on the plan, arriving at grid time $t_0^i + j + \Delta$ with a delay $\Delta \leq \Delta_{\max}$, admissible when it is slow enough and clear of obstacles, of the inflated prediction and of the teammates' intended positions. (Chapter 24)

Rectangle symmetry

Two agents crossing an open rectangle in perpendicular directions whose shortest paths all pairwise conflict; the plain constraint tree grows exponentially, and barrier constraints resolve the whole rectangle in one split. (Chapter 9)

Recursive feasibility

The property that feasibility of the MPC problem at one step implies feasibility at the next; with a terminal set and a static world it follows from the shifted plan. (Chapter 21)

Reference position $\mathbf{p}_i^{\text{ref}}(t)$

The continuous position that a time-indexed path prescribes at clock time t , obtained by linear interpolation between the centers of consecutive cells; the preferred velocity of the local layer is a pursuit of the reference a short lookahead ahead. (Chapter 24)

Relative position

$\mathbf{p}_{\text{rel}} = \mathbf{p}_B - \mathbf{p}_A$, the position of agent B seen from agent A ; with the relative velocity it determines whether two agents on constant velocities collide. (Chapter 12)

Relative velocity

$\mathbf{v}_{\text{rel}} = \mathbf{v}_A - \mathbf{v}_B$, the velocity of A in the frame of B ; two disks collide at time t if and only if $\|\mathbf{p}_{\text{rel}} - t\mathbf{v}_{\text{rel}}\| \leq r_A + r_B$. (Chapter 12)

Relaxation

The update of a neighbor's tentative cost and parent pointer when a cheaper path to it through the vertex being expanded is found. (Chapter 3)

Replanning layer

The layer of the hybrid drone architecture that, when the local avoidance maneuver cannot rejoin the nominal route, recomputes a route from the drone's current cell with the predicted intruder region encoded as changed edge costs. (Chapter 5)

Repulsive potential

Khatib's obstacle term $U_{\text{rep}} = \frac{1}{2}k_{\text{rep}}(1/\rho - 1/\rho_0)^2$ for $\rho \leq \rho_0$ and 0 otherwise, where ρ is the clearance to the obstacle; its force $k_{\text{rep}}(1/\rho - 1/\rho_0)\rho^{-2}\nabla\rho$ grows without bound as $\rho \rightarrow 0$. (Chapter 15)

Reservation table

The sets R_V of (vertex, time) entries and R_E of (move, time) entries of the paths planned so far, together with the parked goal cells; a state or move is blocked if it hits an entry. (Chapter 8)

Responsibility share

The fraction α of the change $\mathbf{u}$ that an agent takes: $\frac{1}{2}$ between agents that both run ORCA, 1 (full responsibility) against an obstacle or a non-cooperative intruder; the pairwise guarantee holds exactly when the two shares of a pair sum to one. (Chapter 13)

Restarted weighted A^*

The naive anytime search that runs weighted A^* from scratch once per inflation factor; it repeats the work of every previous run, which ARA^* avoids by keeping g -values, OPEN and INCONS. (Chapter 6)

Revised rule

The variant of prioritized planning in which each agent also treats the start cells of all lower-priority agents as static obstacles; needed for the completeness guarantee on well-formed instances. (Chapter 8)

Rewire

The RRT^* step that makes the new vertex the parent of every near vertex whose cost-to-come it lowers through a collision-free segment, propagating the saving to all descendants of that vertex. (Chapter 17)

rhs-value

The one-step lookahead $rhs(s)$ of a vertex, the best value its neighbors currently imply (min over predecessors of g + edge cost in LPA^* , over successors in D^* Lite); it is always kept up to date. (Chapter 5)

Robustly optimal solution

An optimal path that is the limit in cost of collision-free paths with positive clearance; the assumption under which sampling-based planners can converge to it. (Chapter 17)

Rollout

The trajectory a candidate command would produce if held constant: a circular arc of radius v/ω for a differential drive, a straight segment for a drone; its free distance is the clearance of the candidate. (Chapter 14)

Rotate (primitive)

Advancing every agent on a fully occupied cycle by one position: one agent steps out into a cleared neighboring vertex, the others advance, and it steps back in. (Chapter 11)

RRT-Connect

A bidirectional RRT with one tree from the start and one from the goal, in which the other tree greedily repeats EXTEND toward every new vertex (CONNECT) and the trees swap roles every iteration. (Chapter 16)

Rule of three

With no event in N runs the one-sided 95 % upper bound on the event probability is about $3/N$. (Chapter 25)

Safety horizon

The look-ahead time within which a predicted collision counts as a near-term risk that triggers local avoidance. (Chapter 1)

Safety horizon τ_h

The look-ahead window of the trigger: a predicted conflict is inside the horizon when the predicted separation minus the uncertainty inflation drops below the safety radius within τ_h seconds; chosen at least as long as the ORCA horizon and the time the drone needs to execute a velocity change. (Chapter 24)

Sample impoverishment

The loss of diversity after resampling, when the surviving particles are copies of a few ancestors that the process noise has not yet separated; aggravated by resampling at every step and by small process noise. (Chapter 19)

Sampling-based planning

Planning in a continuous space by drawing random configurations, keeping the collision-free ones and connecting them by short collision-free segments, so that memory grows with the number of samples rather than with the size or dimension of the space. (Chapter 16)

Sanity baseline

A strategy that must fail or succeed in a known way (no avoidance collides with aimed intruders and never without them); its purpose is to validate the scenario generator and the plan, not to be beaten. (Chapter 25)

Scenario file

A `.scen` text file of the MAPF benchmark listing start–goal pairs as x (column) and y (row) coordinates on a named `.map` grid. (Chapter 7)

Schur complement

For a block matrix $\begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{C} & \mathbf{D} \end{bmatrix}$ with invertible $\mathbf{D}$, the matrix $\mathbf{A} - \mathbf{B}\mathbf{D}^{-1}\mathbf{C}$; it is the top-left block of the inverse's inverse, gives the block determinant, and is the covariance of a conditional Gaussian. (Appendix B)

Second-order consensus

Consensus for double integrators, $\ddot{\mathbf{p}}_i = \sum_j a_{ij}[k_p(\mathbf{p}_j - \mathbf{p}_i - \mathbf{d}_{ij}) + k_v(\mathbf{v}_j - \mathbf{v}_i)] - k_d\mathbf{v}_i$; the velocity terms provide the damping without which the swarm oscillates for ever. (Chapter 23)

Secondary heuristic h_{Focal}

The function that chooses the node inside the focal band; it need not be admissible or even estimate cost. In ECBS it is the number of conflicts. (Chapter 10)

Sequence-to-sequence predictor

An encoder LSTM that summarizes the observed displacements and a decoder LSTM that rolls the future out one step at a time from the handed-over state, with a read-out per step. (Chapter 20)

Settled vertex

A vertex that has been popped from the priority queue and added to the closed set; its tentative cost is then final. (Chapter 3)

Shortcutting

Path post-processing that replaces the part of a path between two points by the straight segment when that segment is collision-free; it never increases the length but cannot carry the path across an obstacle. (Chapter 16)

Shortest-path distance

The smallest cost $\delta(u, v)$ of any path from u to v , taken to be infinite when no path exists. (Chapter 3)

Shortest-path tree

The tree formed by the parent pointers of a single-source search; following them from any vertex back to the source traces a shortest path. (Chapter 3)

Sigma points

The $2n + 1$ points $\bar{\mathbf{x}}$ and $\bar{\mathbf{x}} \pm \mathbf{l}_i$, where $\mathbf{l}_i$ are the columns of a square root of $(n + \lambda)\mathbf{P}$ with $\lambda = \alpha^2(n + \kappa) - n$; their weighted mean and covariance equal $\bar{\mathbf{x}}$ and $\mathbf{P}$ exactly. (Chapter 19)

Social pooling

The Social-LSTM construction that sums the hidden states of the neighbors in each cell of a grid centered on the agent and feeds the embedded grid to the agent's LSTM. (Chapter 20)

Soft constraint

A constraint $g(\mathbf{z}) \leq s$ with a slack variable $s \geq 0$ that is penalized in the cost; it keeps the problem feasible and gives the least violation when the hard version cannot be satisfied. (Chapter 21)

Space-time $\mathbf{A}^*$

$\mathbf{A}^*$ over states (v, t) with a wait action, vertex and edge constraints, a goal-stay test and a time horizon; the low-level search of prioritized planning and of conflict-based search. (Chapter 4)

Space-time state

A pair (v, t) of a vertex and a discrete time index; the states of the time-expanded graph. (Chapter 2)

Stale entry

A priority-queue entry of a vertex that has already been settled with a smaller key; lazy deletion discards it when it is popped. (Chapter 3)

Stay-at-target

The convention in which an agent that has reached its goal remains there forever and keeps blocking the vertex; the cost of its path is the time of its last arrival. (Chapter 7)

Step size

The largest length η of a new edge: STEER moves from the nearest vertex toward the sample by at most η . (Chapter 16)

Stiffness

The largest eigenvalue $\lambda_{\max}$ of the Hessian of U at a point; the explicit update $\mathbf{p} \leftarrow \mathbf{p} + \Delta t \mathbf{F}$ is stable near a minimum only if $\Delta t \lambda_{\max} < 2$, and stiffness grows steeply in narrow passages. (Chapter 15)

Strong δ -clearance

A path all of whose points are at distance at least δ from the obstacles. (Chapter 17)

Subdimensional expansion

Wagner and Choset’s principle of searching the joint space along the individual policies and raising the dimension of the search only where a collision has been found. (Chapter 11)

Suboptimality certificate

The bound $\varepsilon' = \min(\varepsilon, g(\gamma)/\min_{n \in \text{OPEN} \cup \text{INCONS}}(g(n) + h(n)))$ published with every ARA* path; the path costs at most $\varepsilon' C^*$, and usually ε' is much smaller than ε . (Chapter 6)

Success rate (with a cap)

The fraction of runs that finish within a time or computation cap without collision; finished-run metrics are averaged over successful runs only and reported next to it, or replaced by a penalized score such as PAR-2. (Chapter 25)

Sum of costs

The total number of time steps, summed over all agents, until each agent has reached its goal for good. (Chapter 2)

Swap (primitive)

Exchanging two adjacent agents at a vertex of degree at least three with two empty neighbors: a multipush to the vertex, six exchange moves, and the reversal of the preparation with the agents exchanged; $4d + 6$ moves when the junction is d steps away. (Chapter 11)

Swapping conflict

Two agents traversing the same edge in opposite directions in the same time step, exchanging vertices; forbidden in this book. (Chapter 7)

Switching topology

A communication graph that changes with time (drones move, packets are lost); consensus still converges if the union of the graphs over bounded time windows is connected. (Chapter 23)

Symmetry breaking

An extra constraint that removes mirror-image solutions of equal cost from a MILP, such as fixing on which side one drone passes another, so that branch-and-bound explores half the tree. (Chapter 22)

Systematic resampling

Resampling with one uniform draw $u \in [0, 1/N)$ and the grid $u + (j - 1)/N$ over the cumulative weights, which copies each particle $\lfloor Nw^{(i)} \rfloor$ or $\lceil Nw^{(i)} \rceil$ times in $\mathcal{O}(N)$ time. (Chapter 19)

Tangent half-angle

$\theta = \arcsin(R/d)$ with $R = r_A + r_B$ and $d = \|\mathbf{p}_{\text{rel}}\|$, the angle between the axis of the collision cone and each of its boundary rays. (Chapter 12)

Teacher forcing

Feeding the decoder the true previous displacement during training instead of its own prediction; fast to train but subject to exposure bias, the mismatch between training inputs and free-running inputs. (Chapter 20)

Tentative cost

The cost $g(v)$ of the best path to v found so far; it never underestimates the true distance and only ever decreases. (Chapter 3)

Terminal cost

The weight $\mathbf{x}_N^\top \mathbf{P} \mathbf{x}_N$ on the last predicted state, usually the LQR cost-to-go from the Riccati equation, which stands in for the cost beyond the horizon and underlies the stability proof of MPC. (Chapter 21)

Tie-breaking

The rule that orders nodes with equal f ; preferring the larger g (key $(f, -g, counter)$) makes A^* dive along one optimal path instead of sweeping the whole $f = C^*$ band. (Chapter 4)

Time horizon

The largest time $T_{\max}$ for which space-time states are generated; $T_{\max} = H + 1 + D$, with H the last constrained time and D the largest static distance to the goal, keeps the search complete and finite. (Chapter 4)

Time horizon (ORCA)

τ , the time span for which the chosen velocities are guaranteed collision-free; recomputed every $\Delta t \ll \tau$, with a shorter horizon τ_{obst} for static obstacles. (Chapter 13)

Time horizon (velocity obstacle)

The time τ beyond which predicted collisions are ignored by the local avoidance rule; it must exceed the decision interval and allow the next maneuver. (Chapter 12)

Time of closest approach

The time at which two objects moving with constant velocities are closest to each other. (Chapter 2)

Time to collision

$t_c(\mathbf{v}_{\text{rel}})$, the first time at which the disks touch, the smaller root of $(\mathbf{v} \cdot \mathbf{v})t^2 - 2(\mathbf{p} \cdot \mathbf{v})t + (\mathbf{p} \cdot \mathbf{p} - R^2) = 0$; infinite if the discriminant is negative or $\mathbf{p} \cdot \mathbf{v} \leq 0$. (Chapter 12)

Time-expanded flow formulation

The integer program of Yu and LaValle for MAPF: one unit of flow per agent on the time-expanded graph, with capacity-one states and anti-swap rows. (Chapter 22)

Time-expanded graph

The graph whose vertices are the space-time states and whose edges are the move and wait actions between consecutive time layers. (Chapter 2)

Time-indexed occupancy region

The sequence of p -probability ellipses $\mathcal{E}_h(p)$ of a predicted distribution, one per horizon step, from which the planners derive an inflated radius, a cost region or a chance-constraint margin. (Chapter 20)

Time-indexed path

A sequence of vertices $\pi_i[0], \dots, \pi_i[T_i]$ from start to goal in which consecutive entries are equal (a wait) or adjacent in the graph (a move). (Chapter 7)

Tracking

Estimating the current state of a moving object, typically position and velocity, from a sequence of noisy measurements. (Chapter 1)

Trajectory

A time-parametrized curve in continuous time, represented in the book by waypoints with velocities. (Chapter 2)

Trajectory prediction

Mapping the last T_{obs} observed positions of an agent (and possibly its neighbors) to its next T_{pred} positions, ideally as a distribution per horizon step. (Chapter 20)

True-distance heuristic

The exact cost-to-go $h^*(v) = \delta(v, t)$ computed by backward Dijkstra; it is consistent, exact on the static map and still admissible in space-time and under constraints. (Chapter 3)

Truncated velocity obstacle

$VO_{A|B}^\tau$, the velocities that collide within the horizon τ ; the wedge with its tip cut off by the disk of center $\mathbf{v}_B + \mathbf{p}_{\text{rel}}/\tau$ and radius R/τ . (Chapter 12, Chapter 13)

Two-component key

The priority $[\min(g, rhs) + h; \min(g, rhs)]$ of a queued vertex, compared lexicographically; the first component is the f -value of A^* , the second breaks ties toward smaller values so that stale values are retracted before their dependents are expanded. (Chapter 5)

Two-point initialization

Starting a track from two measurements one step apart with $\hat{\mathbf{x}}_0 = (\mathbf{z}_b, (\mathbf{z}_b - \mathbf{z}_a)/\Delta t)$ and the matching covariance $\begin{pmatrix} \mathbf{R} & \mathbf{R}/\Delta t \\ \mathbf{R}/\Delta t & 2\mathbf{R}/\Delta t^2 \end{pmatrix}$. (Chapter 18)

Underconsistent vertex

A vertex with $g(s) < rhs(s)$: its stored value relies on a cost that no longer exists; expanding it retracts the value ($g(s) \leftarrow \infty$) before it can propagate further. (Chapter 5)

Unexpected obstacle

An object that was not part of the plan and is observed only during the flight; it must be handled in real time from a prediction of its motion. (Chapter 1)

Uniform-cost search

Dijkstra's algorithm with an early exit: the search stops as soon as the target is popped, and only vertices no farther than the target are settled. (Chapter 3)

Unscented Kalman filter (UKF)

The filter that replaces both linearizations of the EKF by unscented transforms and uses the cross-covariance between state and predicted measurement in place of $\mathbf{PH}^T$; it needs no Jacobians. (Chapter 19)

Unscented transform

The approximation of the mean, covariance and cross-covariance of $g(\mathbf{x})$ obtained by mapping $2n + 1$ sigma points through g and combining the images with the weights $W^{(m)}$ and $W^{(c)}$; exact for affine g and second-order accurate in the mean for any smooth g . (Chapter 19)

Valid solution

A plan whose paths are legal and which has no vertex or swapping conflict at any time under stay-at-target. (Chapter 7)

Validation gate

The ellipsoid $d^2 \leq \gamma$ around the predicted measurement, with γ a chi-square quantile; measurements outside it are treated as missing, and with several tracks the gates decide which measurement may be assigned to which track. (Chapter 18)

Velocity obstacle

$VO_{A|B}(\mathbf{v}_B) = \mathbf{v}_B \oplus CC_{A|B}$, the set of velocities of A that lead to a collision with B if B keeps its velocity; the collision cone with its apex moved to $\mathbf{v}_B$. (Chapter 12)

Velocity space

The space of velocity commands, (v, ω) for a differential drive and (v_x, v_y, v_z) for a velocity-controlled drone, in which DWA searches. (Chapter 14)

Velocity term

The candidate speed divided by $v_{\max}$; it rewards fast motion. (Chapter 14)

Vertex conflict

Two agents occupying the same vertex at the same time step; forbidden in this book. (Chapter 7)

Vertex constraint

A tuple $\langle a, v, t \rangle$ forbidding agent a to occupy cell v at time t . (Chapter 4)

Voronoi bias

The property that a vertex of the tree is selected for extension with probability equal to the relative volume of its Voronoi region, and extended when the step toward the sample is free,

so that vertices facing large unexplored regions grow the tree most often. ([Chapter 16](#))

Wait action

The action that keeps an agent at its current vertex for one time step. ([Chapter 2](#))

Wait-then-go path

The witness path of the completeness proof: wait at the start until all higher-priority agents are parked, then follow the endpoint-free path. ([Chapter 8](#))

Warm start

Initializing the QP solver with the previous plan shifted by one step, which typically halves the number of iterations. ([Chapter 21](#))

Weighted A*

A* with the key $g + wh$ for $w \geq 1$; it expands fewer nodes and returns a path of cost at most wC^* . ([Chapter 4](#))

Well-formed instance

An instance in which every agent has a path from its start to its goal that visits no start or goal of another agent; the revised algorithm succeeds on it for every order. ([Chapter 8](#))

WHCA*

Windowed Hierarchical Cooperative A*: the reservation table is respected only for the next w steps, a prefix of each plan is executed, and all agents replan with rotated priorities; used for lifelong and online planning, without guarantees. ([Chapter 8](#))

White-noise acceleration

The process-noise model in which the acceleration is continuous-time white noise of intensity q (m^2/s^3), giving per axis $\mathbf{Q} = q \begin{pmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{pmatrix}$; q is a rate, independent of the sampling interval. ([Chapter 18](#))

Wilson interval

A confidence interval for a proportion x/N that stays informative at $0/N$ and N/N ; for $0/20$ it is $[0, 0.16]$. ([Chapter 25](#))

Woodbury identity

The matrix inversion lemma $(\mathbf{A} + \mathbf{UCV})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{U}(\mathbf{C}^{-1} + \mathbf{VA}^{-1}\mathbf{U})^{-1}\mathbf{VA}^{-1}$, which trades a large inverse for a small one and links the covariance and information forms of the Kalman update. ([Appendix B](#))

Bibliography

- [1] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese. “Social LSTM: Human Trajectory Prediction in Crowded Spaces”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 961–971.
- [2] J. Alonso-Mora, A. Breitenmoser, M. Ruffi, P. Beardsley, and R. Siegwart. “Optimal Reciprocal Collision Avoidance for Multiple Non-Holonomic Robots”. In: *Distributed Autonomous Robotic Systems: The 10th International Symposium*. Vol. 83. Springer Tracts in Advanced Robotics. Springer, 2013, pp. 203–216.
- [3] B. D. O. Anderson, C. Yu, B. Fidan, and J. M. Hendrickx. “Rigid Graph Control Architectures for Autonomous Formations”. In: *IEEE Control Systems Magazine* 28.6 (2008), pp. 48–63.
- [4] A. Andreychuk, K. Yakovlev, D. Atzmon, and R. Stern. “Multi-Agent Pathfinding with Continuous Time”. In: *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 2019, pp. 39–45.
- [5] I. Arasaratnam and S. Haykin. “Cubature Kalman Filters”. In: *IEEE Transactions on Automatic Control* 54.6 (2009), pp. 1254–1269.
- [6] M. S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp. “A Tutorial on Particle Filters for Online Nonlinear/Non-Gaussian Bayesian Tracking”. In: *IEEE Transactions on Signal Processing* 50.2 (2002), pp. 174–188.
- [7] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. Wiley, 2001.
- [8] M. Barer, G. Sharon, R. Stern, and A. Felner. “Suboptimal Variants of the Conflict-Based Search Algorithm for the Multi-Agent Pathfinding Problem”. In: *Proceedings of the International Symposium on Combinatorial Search (SoCS)*. 2014, pp. 19–27.
- [9] R. S. Barr, B. L. Golden, J. P. Kelly, M. G. C. Resende, and W. R. Stewart. “Designing and Reporting on Computational Experiments with Heuristic Methods”. In: *Journal of Heuristics* 1.1 (1995), pp. 9–32.
- [10] J. Barraquand and J.-C. Latombe. “Robot Motion Planning: A Distributed Representation Approach”. In: *The International Journal of Robotics Research* 10.6 (1991), pp. 628–649.
- [11] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer. “Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 28. 2015, pp. 1171–1179.
- [12] M. Bennewitz, W. Burgard, and S. Thrun. “Finding and Optimizing Solvable Priority Schemes for Decoupled Path Planning Techniques for Teams of Mobile Robots”. In: *Robotics and Autonomous Systems* 41.2–3 (2002), pp. 89–99.
- [13] J. L. Bentley. “Multidimensional Binary Search Trees Used for Associative Searching”. In: *Communications of the ACM* 18.9 (1975), pp. 509–517.

- [14] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha. “Reciprocal n -Body Collision Avoidance”. In: *Robotics Research: The 14th International Symposium ISRR*. Ed. by C. Pradalier, R. Siegwart, and G. Hirzinger. Vol. 70. Springer Tracts in Advanced Robotics. Springer, 2011, pp. 3–19.
- [15] J. van den Berg, M. Lin, and D. Manocha. “Reciprocal Velocity Obstacles for Real-Time Multi-Agent Navigation”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2008, pp. 1928–1935.
- [16] J. P. van den Berg and M. H. Overmars. “Prioritized Motion Planning for Multiple Robots”. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2005, pp. 430–435.
- [17] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars. *Computational Geometry: Algorithms and Applications*. 3rd ed. Berlin, Heidelberg: Springer, 2008.
- [18] C. M. Bishop. *Mixture Density Networks*. Tech. rep. NCRG/94/004. Neural Computing Research Group, Aston University, 1994.
- [19] H. A. P. Blom and Y. Bar-Shalom. “The Interacting Multiple Model Algorithm for Systems with Markovian Switching Coefficients”. In: *IEEE Transactions on Automatic Control* 33.8 (1988), pp. 780–783.
- [20] J. Borenstein and Y. Koren. “The Vector Field Histogram—Fast Obstacle Avoidance for Mobile Robots”. In: *IEEE Transactions on Robotics and Automation* 7.3 (1991), pp. 278–288.
- [21] F. Borrelli, A. Bemporad, and M. Morari. *Predictive Control for Linear and Hybrid Systems*. Cambridge University Press, 2017.
- [22] E. Boyarski, A. Felner, R. Stern, G. Sharon, D. Tolpin, O. Betzalel, and S. E. Shimony. “ICBS: Improved Conflict-Based Search Algorithm for Multi-Agent Pathfinding”. In: *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 2015, pp. 740–746.
- [23] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein. “Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers”. In: *Foundations and Trends in Machine Learning* 3.1 (2011), pp. 1–122.
- [24] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- [25] O. Brock and O. Khatib. “High-Speed Navigation Using the Global Dynamic Window Approach”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 1999, pp. 341–346.
- [26] F. Bullo, J. Cortés, and S. Martínez. *Distributed Control of Robotic Networks: A Mathematical Approach to Motion Coordination Algorithms*. Princeton University Press, 2009.
- [27] M. Čáp, P. Novák, A. Kleiner, and M. Selecký. “Prioritized Planning Algorithms for Trajectory Coordination of Multiple Mobile Robots”. In: *IEEE Transactions on Automation Science and Engineering* 12.3 (2015), pp. 835–849.
- [28] H. Choset, K. M. Lynch, S. Hutchinson, G. Kantor, W. Burgard, L. E. Kavraki, and S. Thrun. *Principles of Robot Motion: Theory, Algorithms, and Implementations*. MIT Press, 2005.
- [29] J. Cohen. *Statistical Power Analysis for the Behavioral Sciences*. 2nd ed. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988.

- [30] C. I. Connolly, J. B. Burns, and R. Weiss. “Path Planning Using Laplace’s Equation”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 1990, pp. 2102–2106.
- [31] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. 3rd ed. MIT Press, 2009.
- [32] D. Crisan and A. Doucet. “A Survey of Convergence Results on Particle Filtering Methods for Practitioners”. In: *IEEE Transactions on Signal Processing* 50.3 (2002), pp. 736–746.
- [33] R. Dechter and J. Pearl. “Generalized Best-First Search Strategies and the Optimality of A*”. In: *Journal of the ACM* 32.3 (1985), pp. 505–536.
- [34] E. W. Dijkstra. “A Note on Two Problems in Connexion with Graphs”. In: *Numerische Mathematik* 1 (1959), pp. 269–271.
- [35] R. Douc, O. Cappé, and E. Moulines. “Comparison of Resampling Schemes for Particle Filtering”. In: *Proceedings of the 4th International Symposium on Image and Signal Processing and Analysis (ISPA)*. 2005, pp. 64–69.
- [36] A. Doucet, N. de Freitas, and N. Gordon, eds. *Sequential Monte Carlo Methods in Practice*. Springer, 2001.
- [37] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.
- [38] M. Erdmann and T. Lozano-Pérez. “On Multiple Moving Objects”. In: *Algorithmica* 2 (1987), pp. 477–521.
- [39] A. Felner. “Position Paper: Dijkstra’s Algorithm versus Uniform Cost Search or a Case Against Dijkstra’s Algorithm”. In: *Proceedings of the International Symposium on Combinatorial Search (SoCS)*. Vol. 2. 1. 2011, pp. 47–51.
- [40] A. Felner, J. Li, E. Boyarski, H. Ma, L. Cohen, T. K. S. Kumar, and S. Koenig. “Adding Heuristics to Conflict-Based Search for Multi-Agent Path Finding”. In: *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*. 2018, pp. 83–87.
- [41] A. Felner, R. Stern, S. E. Shimony, E. Boyarski, M. Goldenberg, G. Sharon, et al. “Search-Based Optimal Solvers for the Multi-Agent Pathfinding Problem: Summary and Challenges”. In: *Proceedings of the International Symposium on Combinatorial Search (SoCS)*. 2017, pp. 29–37.
- [42] D. Ferguson and A. Stentz. “Using Interpolation to Improve Path Planning: The Field D* Algorithm”. In: *Journal of Field Robotics* 23.2 (2006), pp. 79–101.
- [43] C. Ferner, G. Wagner, and H. Choset. “ODrM*: Optimal Multirobot Path Planning in Low Dimensional Search Spaces”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2013, pp. 3854–3859.
- [44] M. Fiedler. “Algebraic Connectivity of Graphs”. In: *Czechoslovak Mathematical Journal* 23.2 (1973), pp. 298–305.
- [45] P. Fiorini and Z. Shiller. “Motion Planning in Dynamic Environments Using Velocity Obstacles”. In: *The International Journal of Robotics Research* 17.7 (1998), pp. 760–772.
- [46] D. Fox, W. Burgard, and S. Thrun. “The Dynamic Window Approach to Collision Avoidance”. In: *IEEE Robotics & Automation Magazine* 4.1 (1997), pp. 23–33.
- [47] M. L. Fredman and R. E. Tarjan. “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms”. In: *Journal of the ACM* 34.3 (1987), pp. 596–615.

- [48] J. D. Gammell, T. D. Barfoot, and S. S. Srinivasa. “Informed Sampling for Asymptotically Optimal Path Planning”. In: *IEEE Transactions on Robotics* 34.4 (2018).
- [49] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot. “Informed RRT*: Optimal Sampling-Based Path Planning Focused via Direct Sampling of an Admissible Ellipsoidal Heuristic”. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2014, pp. 2997–3004.
- [50] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot. “Batch Informed Trees (BIT*): Sampling-Based Optimal Planning via the Heuristically Guided Search of Implicit Random Geometric Graphs”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2015, pp. 3067–3074.
- [51] C. E. García, D. M. Prett, and M. Morari. “Model Predictive Control: Theory and Practice—A Survey”. In: *Automatica* 25.3 (1989), pp. 335–348.
- [52] S. S. Ge and Y. J. Cui. “New Potential Functions for Mobile Robot Path Planning”. In: *IEEE Transactions on Robotics and Automation* 16.5 (2000), pp. 615–620.
- [53] R. Geraerts and M. H. Overmars. “Creating High-Quality Paths for Motion Planning”. In: *The International Journal of Robotics Research* 26.8 (2007), pp. 845–863.
- [54] F. A. Gers, J. Schmidhuber, and F. Cummins. “Learning to Forget: Continual Prediction with LSTM”. In: *Neural Computation* 12.10 (2000), pp. 2451–2471.
- [55] F. Giuliari, I. Hasan, M. Cristani, and F. Galasso. “Transformer Networks for Trajectory Forecasting”. In: *Proceedings of the International Conference on Pattern Recognition (ICPR)*. 2021, pp. 10335–10342.
- [56] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.
- [57] N. J. Gordon, D. J. Salmond, and A. F. M. Smith. “Novel Approach to Nonlinear/Non-Gaussian Bayesian State Estimation”. In: *IEE Proceedings F (Radar and Signal Processing)* 140.2 (1993), pp. 107–113.
- [58] A. Gupta, J. Johnson, L. Fei-Fei, S. Savarese, and A. Alahi. “Social GAN: Socially Acceptable Trajectories with Generative Adversarial Networks”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 2255–2264.
- [59] A. A. Hagberg, D. A. Schult, and P. J. Swart. “Exploring Network Structure, Dynamics, and Function Using NetworkX”. In: *Proceedings of the 7th Python in Science Conference (SciPy 2008)*. Pasadena, CA, 2008, pp. 11–15.
- [60] E. A. Hansen and R. Zhou. “Anytime Heuristic Search”. In: *Journal of Artificial Intelligence Research* 28 (2007), pp. 267–297.
- [61] D. Harabor and A. Grastien. “Online Graph Pruning for Pathfinding on Grid Maps”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2011, pp. 1114–1119.
- [62] P. E. Hart, N. J. Nilsson, and B. Raphael. “A Formal Basis for the Heuristic Determination of Minimum Cost Paths”. In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107.
- [63] P. E. Hart, N. J. Nilsson, and B. Raphael. “Correction to “A Formal Basis for the Heuristic Determination of Minimum Cost Paths””. In: *SIGART Newsletter* 37 (1972), pp. 28–29.
- [64] S. Hochreiter and J. Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

- [65] S. Holm. “A Simple Sequentially Rejective Multiple Test Procedure”. In: *Scandinavian Journal of Statistics* 6.2 (1979), pp. 65–70.
- [66] W. Hönig, T. K. S. Kumar, L. Cohen, H. Ma, H. Xu, N. Ayanian, and S. Koenig. “Multi-Agent Path Finding with Kinematic Constraints”. In: *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*. 2016, pp. 477–485.
- [67] W. Hönig, J. A. Preiss, T. K. S. Kumar, G. S. Sukhatme, and N. Ayanian. “Trajectory Planning for Quadrotor Swarms”. In: *IEEE Transactions on Robotics* 34.4 (2018), pp. 856–869.
- [68] J. N. Hooker. “Testing Heuristics: We Have It All Wrong”. In: *Journal of Heuristics* 1.1 (1995), pp. 33–42.
- [69] Q. Huangfu and J. A. J. Hall. “Parallelizing the Dual Revised Simplex Method”. In: *Mathematical Programming Computation* 10.1 (2018), pp. 119–142.
- [70] A. Jadbabaie, J. Lin, and A. S. Morse. “Coordination of Groups of Mobile Autonomous Agents Using Nearest Neighbor Rules”. In: *IEEE Transactions on Automatic Control* 48.6 (2003), pp. 988–1001.
- [71] L. Janson, E. Schmerling, A. Clark, and M. Pavone. “Fast Marching Tree: A Fast Marching Sampling-Based Method for Optimal Motion Planning in Many Dimensions”. In: *The International Journal of Robotics Research* 34.7 (2015).
- [72] R. G. Jeroslow. “Trivial Integer Programs Unsolvable by Branch-and-Bound”. In: *Mathematical Programming* 6 (1974), pp. 105–109.
- [73] M. Ji and M. Egerstedt. “Distributed Coordination Control of Multiagent Systems While Preserving Connectedness”. In: *IEEE Transactions on Robotics* 23.4 (2007), pp. 693–703.
- [74] S. J. Julier and J. K. Uhlmann. “New Extension of the Kalman Filter to Nonlinear Systems”. In: *Signal Processing, Sensor Fusion, and Target Recognition VI*. Vol. 3068. Proceedings of SPIE. 1997, pp. 182–193.
- [75] S. J. Julier and J. K. Uhlmann. “Unscented Filtering and Nonlinear Estimation”. In: *Proceedings of the IEEE* 92.3 (2004), pp. 401–422.
- [76] R. E. Kalman. “A New Approach to Linear Filtering and Prediction Problems”. In: *Journal of Basic Engineering* 82.1 (1960), pp. 35–45.
- [77] S. Karaman and E. Frazzoli. “Sampling-Based Algorithms for Optimal Motion Planning”. In: *The International Journal of Robotics Research* 30.7 (2011), pp. 846–894.
- [78] S. Karaman, M. R. Walter, A. Perez, E. Frazzoli, and S. Teller. “Anytime Motion Planning using the RRT*”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2011.
- [79] R. M. Karp. “Reducibility among Combinatorial Problems”. In: *Complexity of Computer Computations*. Ed. by R. E. Miller and J. W. Thatcher. Plenum Press, 1972, pp. 85–103.
- [80] L. E. Kavraki, P. Švestka, J.-C. Latombe, and M. H. Overmars. “Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces”. In: *IEEE Transactions on Robotics and Automation* 12.4 (1996), pp. 566–580.
- [81] E. C. Kerrigan and J. M. Maciejowski. “Soft Constraints and Exact Penalty Functions in Model Predictive Control”. In: *Proceedings of the UKACC International Conference on Control 2000*. Cambridge, UK, 2000.
- [82] H. K. Khalil. *Nonlinear Systems*. 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2002.

- [83] O. Khatib. “Real-Time Obstacle Avoidance for Manipulators and Mobile Robots”. In: *The International Journal of Robotics Research* 5.1 (1986), pp. 90–98.
- [84] D. P. Kingma and J. Ba. “Adam: A Method for Stochastic Optimization”. In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2015.
- [85] G. Kitagawa. “Monte Carlo Filter and Smoother for Non-Gaussian Nonlinear State Space Models”. In: *Journal of Computational and Graphical Statistics* 5.1 (1996), pp. 1–25.
- [86] M. Kleinbort, K. Solovey, Z. Littlefield, K. E. Bekris, and D. Halperin. “Probabilistic Completeness of RRT for Geometric and Kinodynamic Planning With Forward Propagation”. In: *IEEE Robotics and Automation Letters* 4.2 (2019).
- [87] D. E. Koditschek and E. Rimon. “Robot Navigation Functions on Manifolds with Boundary”. In: *Advances in Applied Mathematics* 11.4 (1990), pp. 412–442.
- [88] S. Koenig and M. Likhachev. “D* Lite”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2002, pp. 476–483.
- [89] S. Koenig and M. Likhachev. “Fast Replanning for Navigation in Unknown Terrain”. In: *IEEE Transactions on Robotics* 21.3 (2005), pp. 354–363.
- [90] S. Koenig, M. Likhachev, and D. Furcy. “Lifelong Planning A*”. In: *Artificial Intelligence* 155.1–2 (2004), pp. 93–146.
- [91] Y. Koren and J. Borenstein. “Potential Field Methods and Their Inherent Limitations for Mobile Robot Navigation”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 1991, pp. 1398–1404.
- [92] D. Kornhauser, G. Miller, and P. Spirakis. “Coordinating Pebble Motion on Graphs, the Diameter of Permutation Groups, and Applications”. In: *Proceedings of the 25th Annual Symposium on Foundations of Computer Science (FOCS)*. 1984, pp. 241–250.
- [93] L. Krick, M. E. Broucke, and B. A. Francis. “Stabilisation of Infinitesimally Rigid Formations of Multi-Robot Networks”. In: *International Journal of Control* 82.3 (2009), pp. 423–439.
- [94] J. J. Kuffner and S. M. LaValle. “RRT-Connect: An Efficient Approach to Single-Query Path Planning”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2000, pp. 995–1001.
- [95] B. Lakshminarayanan, A. Pritzel, and C. Blundell. “Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017.
- [96] A. H. Land and A. G. Doig. “An Automatic Method of Solving Discrete Programming Problems”. In: *Econometrica* 28.3 (1960), pp. 497–520.
- [97] F. Large, S. Sekhavat, Z. Shiller, and C. Laugier. “Using Non-Linear Velocity Obstacles to Plan Motions in a Dynamic Environment”. In: *Proceedings of the 7th International Conference on Control, Automation, Robotics and Vision (ICARCV)*. 2002.
- [98] S. M. LaValle. *Rapidly-Exploring Random Trees: A New Tool for Path Planning*. Tech. rep. TR 98-11. Computer Science Department, Iowa State University, 1998.
- [99] S. M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006.
- [100] S. M. LaValle and J. J. Kuffner. “Randomized Kinodynamic Planning”. In: *The International Journal of Robotics Research* 20.5 (2001), pp. 378–400.
- [101] A. Lerner, Y. Chrysanthou, and D. Lischinski. “Crowds by Example”. In: *Computer Graphics Forum* 26.3 (2007), pp. 655–664.

- [102] J. Li, A. Felner, E. Boyarski, H. Ma, and S. Koenig. “Improved Heuristics for Multi-Agent Path Finding with Conflict-Based Search”. In: *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 2019, pp. 442–449.
- [103] J. Li, G. Gange, D. Harabor, P. J. Stuckey, H. Ma, and S. Koenig. “New Techniques for Pairwise Symmetry Breaking in Multi-Agent Path Finding”. In: *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*. 2020, pp. 193–201.
- [104] J. Li, D. Harabor, P. J. Stuckey, H. Ma, and S. Koenig. “Symmetry-Breaking Constraints for Grid-Based Multi-Agent Path Finding”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2019, pp. 6087–6095.
- [105] J. Li, W. Ruml, and S. Koenig. “EECBS: A Bounded-Suboptimal Search for Multi-Agent Path Finding”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2021, pp. 12353–12362.
- [106] M. Likhachev, D. Ferguson, G. Gordon, A. Stentz, and S. Thrun. “Anytime Dynamic A*: An Anytime, Replanning Algorithm”. In: *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*. 2005, pp. 262–271.
- [107] M. Likhachev, D. Ferguson, G. Gordon, A. Stentz, and S. Thrun. “Anytime Search in Dynamic Graphs”. In: *Artificial Intelligence* 172.14 (2008), pp. 1613–1643.
- [108] M. Likhachev, G. J. Gordon, and S. Thrun. “ARA*: Anytime A* with Provable Bounds on Sub-Optimality”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 16. 2003, pp. 767–774.
- [109] T. Lozano-Pérez. “Spatial Planning: A Configuration Space Approach”. In: *IEEE Transactions on Computers* C-32.2 (1983), pp. 108–120.
- [110] R. Luna and K. E. Bekris. “Push and Swap: Fast Cooperative Path-Finding with Completeness Guarantees”. In: *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 2011, pp. 294–300.
- [111] H. Ma, D. Harabor, P. J. Stuckey, J. Li, and S. Koenig. “Searching with Consistent Prioritization for Multi-Agent Path Finding”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2019, pp. 7643–7650.
- [112] H. Ma, J. Li, T. K. S. Kumar, and S. Koenig. “Lifelong Multi-Agent Path Finding for Online Pickup and Delivery Tasks”. In: *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*. 2017, pp. 837–845.
- [113] A. Martelli. “On the Complexity of Admissible Search Algorithms”. In: *Artificial Intelligence* 8.1 (1977), pp. 1–13.
- [114] P. S. Maybeck. *Stochastic Models, Estimation, and Control*. Vol. 1. New York: Academic Press, 1979.
- [115] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert. “Constrained Model Predictive Control: Stability and Optimality”. In: *Automatica* 36.6 (2000), pp. 789–814.
- [116] C. C. McGeoch. *A Guide to Experimental Algorithmics*. Cambridge: Cambridge University Press, 2012.
- [117] D. Mellinger and V. Kumar. “Minimum Snap Trajectory Generation and Control for Quadrotors”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2011, pp. 2520–2525.
- [118] M. Mesbahi and M. Egerstedt. *Graph Theoretic Methods in Multiagent Networks*. Princeton University Press, 2010.

- [119] J. Nocedal and S. J. Wright. *Numerical Optimization*. 2nd ed. Springer, 2006.
- [120] P. Ögren and N. E. Leonard. “A Convergent Dynamic Window Approach to Obstacle Avoidance”. In: *IEEE Transactions on Robotics* 21.2 (2005), pp. 188–195.
- [121] K.-K. Oh, M.-C. Park, and H.-S. Ahn. “A Survey of Multi-Agent Formation Control”. In: *Automatica* 53 (2015), pp. 424–440.
- [122] R. Olfati-Saber. “Flocking for Multi-Agent Dynamic Systems: Algorithms and Theory”. In: *IEEE Transactions on Automatic Control* 51.3 (2006), pp. 401–420.
- [123] R. Olfati-Saber, J. A. Fax, and R. M. Murray. “Consensus and Cooperation in Networked Multi-Agent Systems”. In: *Proceedings of the IEEE* 95.1 (2007), pp. 215–233.
- [124] R. Olfati-Saber and R. M. Murray. “Consensus Problems in Networks of Agents with Switching Topology and Time-Delays”. In: *IEEE Transactions on Automatic Control* 49.9 (2004), pp. 1520–1533.
- [125] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok, and A. P. Schoellig. “Learning to Fly—A Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-Agent Quadcopter Control”. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2021, pp. 7512–7519.
- [126] J. Pearl. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.
- [127] J. Pearl and J. H. Kim. “Studies in Semi-Admissible Heuristics”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 4.4 (1982), pp. 392–399.
- [128] S. Pellegrini, A. Ess, K. Schindler, and L. Van Gool. “You’ll Never Walk Alone: Modeling Social Behavior for Multi-Target Tracking”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2009, pp. 261–268.
- [129] R. D. Peng. “Reproducible Research in Computational Science”. In: *Science* 334.6060 (2011), pp. 1226–1227.
- [130] M. Penrose. *Random Geometric Graphs*. Oxford University Press, 2003.
- [131] M. Phillips and M. Likhachev. “SIPP: Safe Interval Path Planning for Dynamic Environments”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2011, pp. 5628–5635.
- [132] I. Pohl. “Heuristic Search Viewed as Path Finding in a Graph”. In: *Artificial Intelligence* 1.3–4 (1970), pp. 193–204.
- [133] H. E. Rauch, F. Tung, and C. T. Striebel. “Maximum Likelihood Estimates of Linear Dynamic Systems”. In: *AIAA Journal* 3.8 (1965), pp. 1445–1450.
- [134] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. 2nd ed. Nob Hill Publishing, 2017.
- [135] W. Ren and R. W. Beard. “Consensus Seeking in Multiagent Systems Under Dynamically Changing Interaction Topologies”. In: *IEEE Transactions on Automatic Control* 50.5 (2005), pp. 655–661.
- [136] W. Ren and R. W. Beard. *Distributed Consensus in Multi-Vehicle Cooperative Control: Theory and Applications*. Springer, 2008.
- [137] W. Ren, R. W. Beard, and E. M. Atkins. “Information Consensus in Multivehicle Cooperative Control: Collective Group Behavior Through Local Interaction”. In: *IEEE Control Systems Magazine* 27.2 (2007), pp. 71–82.

- [138] C. W. Reynolds. “Flocks, Herds and Schools: A Distributed Behavioral Model”. In: *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*. 1987, pp. 25–34.
- [139] A. Richards and J. P. How. “Aircraft Trajectory Planning with Collision Avoidance Using Mixed Integer Linear Programming”. In: *Proceedings of the American Control Conference (ACC)*. 2002, pp. 1936–1941.
- [140] E. Rimon and D. E. Koditschek. “Exact Robot Navigation Using Artificial Potential Functions”. In: *IEEE Transactions on Robotics and Automation* 8.5 (1992), pp. 501–518.
- [141] A. Rudenko, L. Palmieri, M. Herman, K. M. Kitani, D. M. Gavrila, and K. O. Arras. “Human Motion Trajectory Prediction: A Survey”. In: *The International Journal of Robotics Research* 39.8 (2020), pp. 895–935.
- [142] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2020.
- [143] T. Salzmann, B. Ivanovic, P. Chakravarty, and M. Pavone. “Trajectron++: Dynamically-Feasible Trajectory Forecasting with Heterogeneous Data”. In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2020, pp. 683–700.
- [144] S. Särkkä. *Bayesian Filtering and Smoothing*. Cambridge University Press, 2013.
- [145] G. Sartoretti, J. Kerr, Y. Shi, G. Wagner, T. K. S. Kumar, S. Koenig, and H. Choset. “PRIMAL: Pathfinding via Reinforcement and Imitation Multi-Agent Learning”. In: *IEEE Robotics and Automation Letters* 4.3 (2019), pp. 2378–2385.
- [146] C. Schöller, V. Aravantinos, F. Lay, and A. Knoll. “What the Constant Velocity Model Can Teach Us About Pedestrian Motion Prediction”. In: *IEEE Robotics and Automation Letters* 5.2 (2020), pp. 1696–1703.
- [147] T. Schouwenaars, B. De Moor, E. Feron, and J. How. “Mixed Integer Programming for Multi-Vehicle Path Planning”. In: *Proceedings of the European Control Conference (ECC)*. 2001, pp. 2603–2608.
- [148] R. Seidel. “Small-Dimensional Linear Programming and Convex Hulls Made Easy”. In: *Discrete & Computational Geometry* 6.3 (1991), pp. 423–434.
- [149] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant. “Conflict-Based Search for Optimal Multi-Agent Path Finding”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2012, pp. 563–569.
- [150] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant. “Conflict-Based Search for Optimal Multi-Agent Pathfinding”. In: *Artificial Intelligence* 219 (2015), pp. 40–66.
- [151] G. Sharon, R. Stern, M. Goldenberg, and A. Felner. “The Increasing Cost Tree Search for Optimal Multi-Agent Pathfinding”. In: *Artificial Intelligence* 195 (2013), pp. 470–495.
- [152] B. Siciliano and O. Khatib, eds. *Springer Handbook of Robotics*. 2nd ed. Springer, 2016.
- [153] D. Silver. “Cooperative Pathfinding”. In: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*. 2005, pp. 117–122.
- [154] J. Snape, J. van den Berg, S. J. Guy, and D. Manocha. “The Hybrid Reciprocal Velocity Obstacle”. In: *IEEE Transactions on Robotics* 27.4 (2011), pp. 696–706.
- [155] K. Solovey, L. Janson, E. Schmerling, E. Frazzoli, and M. Pavone. “Revisiting the Asymptotic Optimality of RRT*”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2020.

- [156] T. Standley. “Finding Optimal Solutions to Cooperative Pathfinding Problems”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2010, pp. 173–178.
- [157] T. Standley and R. Korf. “Complete Algorithms for Cooperative Pathfinding Problems”. In: *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 2011, pp. 668–673.
- [158] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd. “OSQP: An Operator Splitting Solver for Quadratic Programs”. In: *Mathematical Programming Computation* 12.4 (2020), pp. 637–672.
- [159] A. Stentz. “Optimal and Efficient Path Planning for Partially-Known Environments”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 1994, pp. 3310–3317.
- [160] A. Stentz. “The Focussed D* Algorithm for Real-Time Replanning”. In: *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 1995, pp. 1652–1659.
- [161] R. Stern, N. R. Sturtevant, A. Felner, S. Koenig, H. Ma, T. T. Walker, et al. “Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks”. In: *Proceedings of the International Symposium on Combinatorial Search (SoCS)*. 2019, pp. 151–158.
- [162] N. R. Sturtevant. “Benchmarks for Grid-Based Pathfinding”. In: *IEEE Transactions on Computational Intelligence and AI in Games* 4.2 (2012), pp. 144–148.
- [163] X. Sun, W. Yeoh, and S. Koenig. “Moving Target D* Lite”. In: *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*. 2010.
- [164] P. Surynek. “A Novel Approach to Path Planning for Multiple Robots in Bi-connected Graphs”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2009, pp. 3613–3619.
- [165] P. Surynek. “An Optimization Variant of Multi-Robot Path Planning Is Intractable”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2010, pp. 1261–1263.
- [166] H. G. Tanner, A. Jadbabaie, and G. J. Pappas. “Flocking in Fixed and Switching Networks”. In: *IEEE Transactions on Automatic Control* 52.5 (2007), pp. 863–868.
- [167] J. T. Thayer and W. Ruml. “Bounded Suboptimal Search: A Direct Approach Using Inadmissible Estimates”. In: *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 2011.
- [168] S. Thrun, W. Burgard, and D. Fox. *Probabilistic Robotics*. MIT Press, 2005.
- [169] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017, pp. 5998–6008.
- [170] T. Vicsek, A. Czirók, E. Ben-Jacob, I. Cohen, and O. Shochet. “Novel Type of Phase Transition in a System of Self-Driven Particles”. In: *Physical Review Letters* 75.6 (1995), pp. 1226–1229.
- [171] J. P. Vielma. “Mixed Integer Linear Programming Formulation Techniques”. In: *SIAM Review* 57.1 (2015), pp. 3–57.
- [172] G. Wagner and H. Choset. “M*: A Complete Multirobot Path Planning Algorithm with Performance Bounds”. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2011, pp. 3260–3267.

- [173] G. Wagner and H. Choset. “Subdimensional Expansion for Multirobot Path Planning”. In: *Artificial Intelligence* 219 (2015), pp. 1–24.
- [174] E. A. Wan and R. van der Merwe. “The Unscented Kalman Filter for Nonlinear Estimation”. In: *Proceedings of the IEEE Adaptive Systems for Signal Processing, Communications, and Control Symposium (AS-SPCC)*. 2000, pp. 153–158.
- [175] G. Welch and G. Bishop. *An Introduction to the Kalman Filter*. Tech. rep. TR 95-041. Department of Computer Science, University of North Carolina at Chapel Hill, 1995.
- [176] P. J. Werbos. “Backpropagation Through Time: What It Does and How to Do It”. In: *Proceedings of the IEEE* 78.10 (1990), pp. 1550–1560.
- [177] F. Wilcoxon. “Individual Comparisons by Ranking Methods”. In: *Biometrics Bulletin* 1.6 (1945), pp. 80–83.
- [178] B. de Wilde, A. W. ter Mors, and C. Witteveen. “Push and Rotate: A Complete Multi-Agent Pathfinding Algorithm”. In: *Journal of Artificial Intelligence Research* 51 (2014), pp. 443–492.
- [179] E. B. Wilson. “Probable Inference, the Law of Succession, and Statistical Inference”. In: *Journal of the American Statistical Association* 22.158 (1927), pp. 209–212.
- [180] R. M. Wilson. “Graph Puzzles, Homotopy, and the Alternating Group”. In: *Journal of Combinatorial Theory, Series B* 16.1 (1974), pp. 86–96.
- [181] L. A. Wolsey. *Integer Programming*. Wiley, 1998.
- [182] L. Xiao and S. Boyd. “Fast Linear Iterations for Distributed Averaging”. In: *Systems & Control Letters* 53.1 (2004), pp. 65–78.
- [183] P. Yang, R. A. Freeman, G. J. Gordon, K. M. Lynch, S. S. Srinivasa, and R. Sukthankar. “Decentralized Estimation and Control of Graph Connectivity for Mobile Sensor Networks”. In: *Automatica* 46.2 (2010), pp. 390–396.
- [184] J. Yu and S. M. LaValle. “Structure and Intractability of Optimal Multi-Robot Path Planning on Graphs”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2013, pp. 1443–1449.
- [185] J. Yu and S. M. LaValle. “Optimal Multirobot Path Planning on Graphs: Complete Algorithms and Effective Heuristics”. In: *IEEE Transactions on Robotics* 32.5 (2016), pp. 1163–1177.
- [186] M. M. Zavlanos, M. B. Egerstedt, and G. J. Pappas. “Graph-Theoretic Connectivity Control of Mobile Robot Networks”. In: *Proceedings of the IEEE* 99.9 (2011), pp. 1525–1540.
- [187] H. Zhu and J. Alonso-Mora. “Chance-Constrained Collision Avoidance for MAVs in Dynamic Environments”. In: *IEEE Robotics and Automation Letters* 4.2 (2019), pp. 776–783.

Index

- C^* , optimal cost, 66
- h^* , true cost to go, 66
- 15-puzzle, 144
- A^*
 - as generalization of Dijkstra, 69
 - complexity, 74
 - consistency, 72
 - $f = g + h$, 65
 - optimality theorem, 71
 - space-time, 77
 - weighted, 75
- A^*_ϵ , 197
- active constraint, 455
- active set, 455
- Adam optimizer, 429
- ADE (average displacement error), 422
- adjacency list, 23
- adjacency matrix, 493, 602
- admissible velocity, 289, 291
- agent, 136, 219
 - disk model, 242
- agent-centered frame, 426
- algebraic connectivity, 602
- algebraic connectivity λ_2 , 493
 - as a monitor, 507
- alpha–beta filter, 393
- angle wrapping, 403
- anytime algorithm, 8, 113
- Anytime D*, 104, 125
- anytime planning
 - RRT*, 367
 - while flying, 369
- anytime weighted A^* , 126
- ARA*, 112
 - completeness, 122
 - complexity, 123
 - ϵ schedule, 115
 - expansions per iteration, 122
 - goal never expanded, 116
 - INCONS list, 115
 - inflated key, 115
 - pitfalls, 128
 - pointer path, 121
 - suboptimality certificate, 115, 122
 - suboptimality theorem, 121
 - zero-work iteration, 123
- arrival time, 28
- artificial potential field, 309
 - attractive potential, 311
 - influence distance, 311
 - local minimum, 312, 318
 - local-minimum detection, 313
 - repulsive potential, 311
- asymptotic optimality, 356
 - theorem, 362
- attention, 429
 - complexity, 439
 - multi-head, 430
 - scaled dot-product, 430
- avoidance constraint
 - linearized, 451
- back set, 221
- backpropagation through time, 425
- backward Dijkstra, 55
- banana distribution, 400
- baseline (evaluation), 546
- basin of a local minimum, 312
- Bayes filter, 401
- Bayes’ rule, 594
- BCBS, 208
- Bellman–Ford algorithm, 52
- benchmark
 - gym-pybullet-drones, 558
 - MAPF, 146, 557
- best linear unbiased estimator, 386
- best-first search, 65
- bidirectional search, 57
- big- M trick, 471
 - choice of M , 474
 - implementation, 487
- big-M constraint, 600
- big-O notation, 604
- binary heap, 604
- binary variable, 473

- BIT*, 367
- blocked layer, 518, 525
- boids, 505
- bootstrap, 550
- Borel–Cantelli lemma, 363
- bounded suboptimality, 8, 194
 - definition, 196
- bounding sphere, 26
- box plot, 551
- braking candidate, 294
- braking distance, 292
- branch-and-bound, 476, 600
 - bounding, 477
 - branching, 477
 - correctness, 480
 - worst case, 481
- branch-and-bound pruning, 367
- branch-and-cut, 478
- branching factor
 - joint, 220
- breadth-first search, 57
- bypass, 184
- cactus plot, 551
- calibration, 434
- capstone, 578
- Cauchy–Schwarz inequality, 587
- CBS, 173
 - as solver family, 144
 - bypass, 184
 - completeness, 181
 - complexity, 182
 - high level, 176
 - high-level heuristic, 185
 - improved, 183
 - key idea, 174
 - local (partial replanning), 525
 - low level, 177
 - meta-agent, 184
 - optimality, 181
- CBSH, 185
- centralized, 7
- chain rule, 591
- chance constraint, 462, 528
- chi-square distribution, 596
- Cholesky factorization, 407, 588
 - numerical issues, 416
- clearance, 311, 338
 - strong, 355
- clearance term, 293
- closed set, 68
- closest approach
 - time of, 35
- code files
 - self-test, 570
- collision checking
 - by subdivision, 335
 - resolution, 335
 - sphere versus box, 346
- collision cone, 241, 243
- collision distance, 30
- collision rate, 542
 - of predictions, 422
- collision set, 221
- combined radius, 242, 265
- communication graph, 493
- communication violation, 495
 - as a metric, 543
- completeness, 8
 - ORCA, 280
- completion checklist, 581
- complexity
 - Dijkstra’s algorithm, 53
 - of D* Lite, 103
- computation time, 543
- condensed form, 453
- conditional mean, 439
- confidence interval, 550
- configuration, 25
- configuration space, 6, 25
 - free space, 25
 - in sampling-based planning, 334
 - obstacle region, 25
- conflict
 - cardinal, 183
 - cycle, 137
 - detection, 139
 - edge, 137, 175
 - following, 137
 - non-cardinal, 183
 - semi-cardinal, 183
 - splitting, 176
 - swap, 219
 - swapping, 137, 155
 - vertex, 137, 155, 175, 219
- conflict avoidance table, 223
- conflict heuristic h_c , 197
- conflict-based search, 173
- Connect heuristic, 340
- connection radius, 356
 - shrinking, 358
- connectivity maintenance, 506

- consensus
 - average, 494
 - definition, 494
 - gain k_c , 495
 - leader–follower, 496
 - protocol, 495
 - second-order, 497
 - step-size condition, 495
 - time constant $1/\lambda_2$, 499
- constant-acceleration model, 380, 424
- constant-velocity model, 379
 - as prediction baseline, 424
- constraint
 - edge, 77, 175
 - vertex, 77, 175
- constraint tree, 174
 - node, 175
 - node consistent with an optimal solution, 205
- continuous-time MAPF, 150
- control, 7
- conventions of the book, 17
- convex corridor, 451
- convexity
 - function, 596
 - set, 596
- Cooperative A*, 155
- cooperative agent, 7
- coordinated-turn model, 399
 - definition, 401
- corner cutting, 24, 482
- cost
 - joint move, 220
- cost-to-come, 355
- covariance, 592
- covariance ellipse, 36
- critical point, 312
- cross product, 587
 - 2D, 33
- cross-covariance, 407
- crossing angle, 547
- cubature Kalman filter, 415
- curse of dimensionality, 332
- D*
 - original (Stentz), 104
- D* Lite, 96
 - backward search, 96
 - key modifier, 97
 - optimized version, 98
- data association, 390
 - nearest neighbor, 390
- data leakage, 423
- deadline, 128
- decentralized, 7
- decision logic
 - capstone, 579
- degeneracy, 410
- degree matrix, 493
- deliberative, 7
- differential drive, 290
- Dijkstra’s algorithm, 45
 - backward, 55
 - bidirectional, 57
 - complexity, 53
 - correctness, 51
 - dense graphs, 53
 - early exit, 48
 - in the drone system, 61
 - multi-source, 55
 - settling order, 50
 - unit costs, 57
- disagreement vector, 494
- disappear-at-target, 136
- divergence (EKF), 406
- dot product, 33, 587
- double integrator, 31, 592
 - consensus, 497
 - kinodynamic RRT, 342
- drift from the plan, 518
- drone
 - controlled, 2
 - velocity-controlled, 290
- DWA, 288
 - admissible velocities, 291
 - braking candidate, 294
 - clearance term, 293
 - complexity, 300
 - dynamic window, 292
 - heading term, 293
 - hysteresis, 302
 - local minimum, 299
 - objective function, 293
 - oscillation, 299
 - possible velocities, 291
 - search set, 293
 - tuning, 300
 - velocity term, 293
- dynamic window, 289, 292
- dynamic window approach, *see* DWA, 309
- early stopping, 429

- ECBS, 194
 - suboptimality theorem, 206
- EECBS, 208
- effect size, 550
- effective sample size, 410
- eigenvalue, 587
- endpoint, 156
- ensemble, 434
- execution log, 541
- executive, 516
- expansion, 48
- expectation, 592
- experiment matrix, 544, 580
- explicit estimation search (EES), 208
- exposure bias, 428
- extended Kalman filter, 399
 - equations, 403
- FDE (final displacement error), 422
- feasibility solver, 232
- feasible velocities, 246
- feed-forward, 496
- Fibonacci heap, 53
- Fiedler vector, 499
- Field D^* , 104
- filter selection, 414
- filtering problem, 378, 401
- floating point
 - cost comparison, 60
- flocking, 505
 - alignment, 325
 - cohesion, 325
 - separation, 325
- FMT*, 366
- FOCAL list, 195
 - implementation with two heaps, 210
- focal search, 195
 - bounded suboptimality theorem, 205
 - definition, 197
- force (potential field), 310, 311
- formation
 - desired displacement $\mathbf{d}_{ij}$, 494
 - graph, 494
 - template, 494
- formation control
 - displacement-based, 496
 - distance-based, 504
- formation error, 494
 - as a metric, 543
- fractional design, 545
- free distance, 290
- free space, 334
- full factorial design, 544
- gating, 389, 596
- Gaussian
 - conditional, 593
 - marginal, 593
 - multivariate, 593
- generative model
 - for prediction, 434
- global dynamic window approach, 302
- global planner, 9
- GNRON (goals non-reachable with obstacles nearby), 318
- goal
 - stay condition, 78
- goal bias, 334, 356
- goal region, 334
- goal-occupied-later test, 176
- goal-stay check, 157
- goal-stay test, 176
- gradient, 590
- gradient descent, 601
- graph, 2
 - directed, 23
 - implicit, 23
 - undirected, 23
 - weighted, 23, 46
- graph Laplacian, 493, 602
- grid, 2, 24
 - 4-connected, 24
 - 8-connected, 24
 - obstacle cell, 24
 - occupancy, 24
 - stretch factor, 24
- hard constraint, 452
- harmonic potential, 323
- hash table, 604
- HCA*, 163
- heading term, 293
- heapq, 58
- Hessian, 590
- heuristic, 65
 - admissible, 55, 66
 - backward consistent, 96
 - Chebyshev, 67
 - consistent, 55, 66
 - dominance, 74
 - Euclidean, 67
 - inadmissible, 73

- Manhattan, 67
- monotone, 66
- octile, 67
- secondary (focal), 197
- sum of individual costs, 220
- true-distance, 55
- holonomic, 31, 290
- horizon, 28
- hybrid architecture, 9, 515
 - capstone, 568
 - core idea, 4
 - reconnection, 523
 - safety horizon, 520
 - world model, 517
- hysteresis, 302
- ICBS, 183
- ICTS, 144
- importance sampling, 410, 596
- inconsistent state, 115
- incremental search, 8, 90
 - moving start, 90
- incumbent, 476
 - time limit, 487
- independence detection, 222
- individual policy, 221
- inflated radius, 528
- information filter, 382
- informed set, 356
 - geometry, 364
- innovation, 381
- integer program, 473
- intended position, 520
- inter-agent repulsion, 324
- inter-sample safety, 482
- interacting multiple model (IMM), 415
- interaction-aware prediction, 422
- interior-point method, 472
- intruder, 3
 - trajectory families, 547
- iterated EKF, 415
- Jacobian, 401, 590
 - coordinated turn, 404
 - range-bearing, 404
- joint A*, 222
- joint configuration, 219
- joint state space, 144, 219
 - size, 220
- Joseph form, 381
- Kalman filter, 376
 - consistency, 389
 - extrapolation, 424
 - Joseph form, 381
 - predict step, 376
 - prediction, 378
 - update step, 376
- Kalman gain, 381
- kd-tree, 347
- key
 - two-component, 91
- key modifier, 97
- kinematic constraints (MAPF), 149
- kinodynamic planning, 342
- KKT conditions, 598
- Koren–Borenstein critique, 322
- lab notebook, 569
- Lagrange multiplier, 454
- Lagrangian, 598
- Laplacian, *see* graph Laplacian
- lattice
 - 26-connected, 25
 - 3D, 25, 149
 - 6-connected, 25
- lazy deletion, 38, 47
- leader–follower, 496
- least squares, 590
- lifelong MAPF, 150
- Lifelong Planning A*, *see* LPA*
- limited neighbors, 221
- linear program, 471, 598
 - definition, 472
 - incremental, 273
 - maximum penetration, 274
 - standard form, 598
- linearization, 591
- linearization error, 400
- linearization point, 451
- local consistency, 91
- local minimum, 299
- local safety layer, 10
- log-weights, 411
- lookahead point, 295
- loss function
 - Gaussian negative log-likelihood, 428
 - mean squared error, 428
- lower bound
 - global LB, 197
 - of a CT node, 197
 - on the optimal cost, 122
 - per agent, 197

- returned by the low level, 205
- LP relaxation, 471, 600
- LPA*, 92
 - g-value, 90
 - key, 91
 - local consistency, 91
 - rhs-value, 90
- LSTM (long short-term memory), 425
 - cell state, 425
 - complexity, 439
 - gates, 426
- M*
 - completeness, 227
 - complexity, 228
 - in the drone system, 235
 - inflated, 232
 - optimality, 227
 - recursive (rM*), 232
- Mahalanobis distance, 593
 - gating, 389
- makespan, 29, 138
 - as a metric, 542
- map file (.map), 146
- map of the book, 12
- MAPF, 153, 196
 - as an integer program, 482
 - classical, 136
 - feasibility, 139
 - instance, 136
 - NP-hardness, 144
 - optimal solution, 138
 - solution, 136
- MAPF (multi-agent path finding), 76, 134
- margin function, 520
- mathematical refresher, 586–606
- matrix
 - inverse, 587
 - positive definite, 588
 - positive semidefinite, 588
 - rank, 587
 - transpose, 587
- matrix square root, 407
- MDD, 183
- measurement noise, 377
- meta-agent CBS, 184
- metrics
 - capstone, 581
- MILP, *see* mixed-integer linear program
 - scaling, 484
- minADE_K, 422
- minimum separation, 542
- minimum-fuel objective, 476
- minimum-time objective, 476
- Minkowski sum, 26, 265
 - in velocity obstacles, 242
- MIP gap, 481
- mixed-integer linear program, 470, 600
 - definition, 473
- mixture density network, 433
- MMSE estimator, 385
- model predictive control, 448
 - condensed form, 453
 - control law, 450
 - sparse formulation, 460
 - terminal ingredients, 458
- Monte Carlo estimation, 596
- motion model, 377
- move action, 27, 136
- Moving Target D* Lite, 105
- multi-agent path finding (MAPF), 7
- multi-commodity flow, 482
- multi-source Dijkstra, 55
- multimodal posterior, 413
- multimodal prediction, 422
- naive plan, 134
 - lower bound, 143
- narrow-passage problem, 339
- navigation function, 322
- near miss, 542
- nearest-neighbor search, 347
- NEES, 389
- negative edge cost, 52
- neighborhood N_i , 493
- NIS, 389
- non-autoregressive decoder, 432
- non-cooperative agent, 7
- non-cooperative obstacle, 272
- non-holonomic, 31
- norm
 - 1-norm and infinity-norm, 587
 - Euclidean, 587
- normal equations, 590
- normalization, 294
- NP-hardness
 - MAPF, 144
- objective
 - makespan, 138
 - sum of costs, 138
- observability, 387

- obstacle
 - as infinite edge cost, 90
- obstacle inflation, 26
- occupancy region
 - time-indexed, 434
- open set, 68, 115
- operator decomposition, 222
- optimality, 8
- oracle baseline, 546
- ORCA
 - complexity, 280
 - deadlock, 280
 - dense fallback, 274
 - full responsibility, **272**
 - guarantee, 279
 - half-plane, **270**
 - key idea, 264
 - normal orientation, 270
 - reciprocal maximality, 271
 - three-dimensional, 272
- oscillation, 299
 - velocity obstacles, 266
- overconsistent vertex, 91
- paired comparison, 550
- paired design, 547
- PAR-2 score, 551
- parent pointer, 48
- Pareto plot, 552
- partial replanning, 525
- particle filter, 399
 - algorithm, 411
 - bootstrap, 410
 - convergence, 414
 - cost, 414
 - degeneracy, 410
 - impoverishment, 411
 - particle set, 410
 - variants, 415
 - vectorization, 417
- path, 23
 - collision-free, 355
 - cost, 23
 - cost (MAPF), 136
 - length, 29
 - start time t_0 , 517
 - time-indexed, 136, 155
- path length (metric), 542
- path reconstruction, 48
- path smoothing, 343
- pebble motion, 143, 229
- pinning, 496
- pitfall
 - absolute velocity, 259
 - forgetting the radii, 259
 - no truncation, 259
 - static intruder, 6
 - time scales, 12
 - velocity sampling, 259
- plan, 7, 28
 - horizon, 136
 - MAPF, 136
- planned conflict, **3**
- planning, 7
 - continuous, 7
 - discrete, 7
 - global, 7
 - local, 7
- point-segment distance, 33
- policy path, 221
- polishing, 455
- polyhedron, 471
- positional encoding, 430
- potential, 310, 311
- prediction, 8
 - uncertainty inflation, 528
- prediction horizon, 422, 449
- prediction layer, 9
- prediction matrices, 453
- preferred velocity, 246, 265
 - from a path, 518
- prioritized planning, 145, 153
 - plan, 156
 - revised rule, 156
- priority order, 156
 - longest path first, 165
 - most constrained first, 165
 - random restarts, 165
- priority queue
 - lazy deletion, 38, 47, 68, 105, 210
- priority tags, 12
- priority-based search, 165
- PRM*, 366
- probabilistic completeness, 338
- probabilistic roadmap (PRM), 345
- process noise, 377
 - coordinated turn, 402
- prolate hyperspheroid, 364
- push (primitive), 229
- Push-and-Rotate, 145
 - completeness, 232
- Push-and-Swap

- completeness, 231
- Python stack, 18, 569
- quadratic form, 588
- quadratic program, 453, 599
 - active-set method, 454
 - ADMM, 454
 - interior-point method, 454
- radius graph, 493
- random vector, 592
- random walk (escape from local minima), 323
- range-bearing measurement, 402
- range-keeping barrier, 507
- re-opening, 72
- reachable velocities, 246
- reactive, 7
- reading order, 583
- reading paths, 16
- real-time, 8
- receding horizon, 449
- reciprocal dance, 266
 - of RVO, 268
- reciprocal velocity obstacle, 267
- reciprocity, 264
- reconnection, 523
- recurrent neural network, 424
- recursive feasibility, 459
- reference position, 518
- relative position, 35, 242
- relative velocity, 35, 242
- relaxation, 46
- replanning, 8
 - start time t_{start} , 518
- replanning count, 543
- replanning layer, 10
 - anytime, 128
- reproducibility, 555
- resampling, 411
- research directions, 15, 585
- reservation table, 154
 - blocked state, 155
 - definition, 155
 - parked agent, 155
 - with time offsets, 518
- residual
 - dual, 455
 - primal, 455
- reverse resumable A^* , 163
- rhs-value, 90
- Riccati equation, 387, 459
- rigidity, 504
- RMSE, 376
- robust optimality, 355
- rollout, 289
- rotate (primitive), 230
- rotation to world frame, 365
- roughening, 411
- RRG, 363, 366
- RRT
 - CollisionFree, 335
 - goal bias, 334
 - key idea, 333
 - kinodynamic, 342
 - Nearest, 335
 - not asymptotically optimal, 362
 - probabilistic completeness, 338
 - RRT-Connect, 340
 - Sample, 334
 - Steer, 335
 - step size, 334
 - Voronoi bias, 333
- RRT*
 - asymptotic optimality, 362
 - ChooseParent, 354
 - complexity, 363
 - k -nearest, 359
 - Near, 356
 - probabilistic completeness, 362
 - Rewire, 354
- RRT-Connect, 340
- run (evaluation), 541
- saddle point, 318
- safety horizon, 10, 520
- sample impoverishment, 411
- sampling-based planning, 332
- scenario, 546
 - signature, 547
- scenario file (`.scen`), 146
- scenario generator, 541
- Schur complement, 589
- `scipy.optimize.milp`, 485
- search problem
 - heuristic, 66
- secondary heuristic, 197
- self-check, 582
- separation, 30
 - minimum, 30
- separation constraint
 - MILP, 475

- sequence-to-sequence model, 426
- sequential importance resampling, 410
- settled-node invariant, 51
- shortcutting, 343
- shortest path, 23
 - distance, 46
 - incremental, 90
 - single-source, 44
- shortest-path distance, 603
- shortest-path tree, 46
- sigma points, 407
- simplex method, 472
- simulated annealing, 323
- single integrator, 31
- slack variable, 452
- smoother
 - Rauch–Tung–Striebel, 394
- smoothing, 294
- Social LSTM, 429
- social pooling, 429
- soft constraint, 452
- solver families (MAPF), 144
- space-time
 - MAPF view, 142
- space-time A^* , 77
- space-time state, 27, 77
- spanning tree, directed, 502
- sphere world, 322
- stage cost, 450
- stale entry, 48
- state-space model
 - linear-Gaussian, 377
 - nonlinear, 401
- static obstacle
 - as velocity obstacle, 247
- stay-at-goal convention, 28
- stay-at-target, 136, 154
- step size, 334
- stiffness of a potential, 320
- stopping distance, 32
- stratified resampling, 411
- study load, 569
- study plan, 16, 568–585
 - schedule, 570
 - timeline, 571
 - week 1, 572
 - week 2, 573
 - week 3, 574
 - week 4, 574
 - week 5, 574
 - week 6, 575
 - week 7, 575
 - week 8, 576
 - week 9, 576
 - week 10, 577
 - week 11, 577
 - week 12, 577
- subdimensional expansion, 218
- suboptimality factor, 197
- success rate, 146, 550
- sum of costs, 29, 138, 155, 175
 - as a metric, 542
- survival plot, 551
- swap (primitive), 229
- swarm, 2
- switching topology, 493
- symmetry
 - corridor, 185
 - rectangle, 185
- symmetry breaking, 487
- systematic resampling, 411
- teacher forcing, 428
- tentative cost, 47
- terminal cost, 450
- terminal set, 458
- tie-breaking, 58, 68
 - in D^* Lite, 103
 - in focal search, 212
 - in LPA^* , 95
 - larger g first, 74
- time discretization
 - MILP, 487
- time horizon
 - obstacles, 283
 - ORCA, 269
 - velocity obstacle, 245
- time scale
 - global, 12
 - local safety layer, 11
 - replanning, 12
- time to collision, 10, 35, 244, 266
 - predicted, 520
- time-expanded graph, 27, 77, 482
- time-indexed path, 3, 7, 28, 517
- total probability, 594
- traceability table, 582
- track initialization, 391
- tracking, 8
- trajectory, 28
- trajectory prediction, 422
- Transformer, 429

- non-autoregressive decoder, 432
- travel time, 29, 542
- true-distance heuristic, 55
- underconsistent vertex, 91
- unexpected obstacle, 4
- uniform-cost search, 46
 - correctness, 52
- unscented Kalman filter, 399
 - algorithm, 408
 - versus EKF, 410
- unscented transform, 407
 - parameters, 407
 - second-order accuracy, 414
- validator (MAPF), 148
- value function, 458
- vector field histogram, 322
- velocity limit, 313
- velocity obstacle, 241, **243**, 265
 - apex, 244
 - in three dimensions, 256
 - truncated, 245, 265
 - union of, 247
- velocity space, 290
- velocity term, 293
- Voronoi bias, 333
- Voronoi region, 333
- w -suboptimal solution, 196
- wait action, 27, 77, 136
- warm start, 456, 487
- waypoint, 28
- weighted A*, 75, 113
 - restarted, 114
- weights (DWA), 300
- well-formed instance, 156
- WHCA*, 164
- white-noise acceleration, 379
- Wilson interval, 550
- window (WHCA*), 164
- workspace, 6, 25
- world model
 - hybrid, 517